



SIEMENS EDA

# Tessent™ Shell User's Manual

Software Version 2024.2  
Document Revision 34

Unpublished work. © 2024 Siemens

This Documentation contains trade secrets or otherwise confidential information owned by Siemens Industry Software Inc. or its affiliates (collectively, "Siemens"), or its licensors. Access to and use of this Documentation is strictly limited as set forth in Customer's applicable agreement(s) with Siemens. This Documentation may not be copied, distributed, or otherwise disclosed by Customer without the express written permission of Siemens, and may not be used in any way not expressly authorized by Siemens.

This Documentation is for information and instruction purposes. Siemens reserves the right to make changes in specifications and other information contained in this Documentation without prior notice, and the reader should, in all cases, consult Siemens to determine whether any changes have been made.

No representation or other affirmation of fact contained in this Documentation shall be deemed to be a warranty or give rise to any liability of Siemens whatsoever.

If you have a signed license agreement with Siemens for the product with which this Documentation will be used, your use of this Documentation is subject to the scope of license and the software protection and security provisions of that agreement. If you do not have such a signed license agreement, your use is subject to the Siemens Universal Customer Agreement, which may be viewed at <https://www.sw.siemens.com/en-US/sw-terms/base/uca/>, as supplemented by the product specific terms which may be viewed at <https://www.sw.siemens.com/en-US/sw-terms/supplements/>.

SIEMENS MAKES NO WARRANTY OF ANY KIND WITH REGARD TO THIS DOCUMENTATION INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, AND NON-INFRINGEMENT OF INTELLECTUAL PROPERTY. SIEMENS SHALL NOT BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, CONSEQUENTIAL OR PUNITIVE DAMAGES, LOST DATA OR PROFITS, EVEN IF SUCH DAMAGES WERE FORESEEABLE, ARISING OUT OF OR RELATED TO THIS DOCUMENTATION OR THE INFORMATION CONTAINED IN IT, EVEN IF SIEMENS HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

TRADEMARKS: The trademarks, logos, and service marks (collectively, "Marks") used herein are the property of Siemens or other parties. No one is permitted to use these Marks without the prior written consent of Siemens or the owner of the Marks, as applicable. The use herein of third party Marks is not an attempt to indicate Siemens as a source of a product, but is intended to indicate a product from, or associated with, a particular third party. A list of Siemens' Marks may be viewed at: [www.plm.automation.siemens.com/global/en/legal/trademarks.html](http://www.plm.automation.siemens.com/global/en/legal/trademarks.html). The registered trademark Linux<sup>®</sup> is used pursuant to a sublicense from LMI, the exclusive licensee of Linus Torvalds, owner of the mark on a world-wide basis.

### **About Siemens Digital Industries Software**

Siemens Digital Industries Software is a global leader in the growing field of product lifecycle management (PLM), manufacturing operations management (MOM), and electronic design automation (EDA) software, hardware, and services. Siemens works with more than 100,000 customers, leading the digitalization of their planning and manufacturing processes. At Siemens Digital Industries Software, we blur the boundaries between industry domains by integrating the virtual and physical, hardware and software, design and manufacturing worlds. With the rapid pace of innovation, digitalization is no longer tomorrow's idea. We take what the future promises tomorrow and make it real for our customers today. Where today meets tomorrow. Our culture encourages creativity, welcomes fresh thinking and focuses on growth, so our people, our business, and our customers can achieve their full potential.

Support Center: [support.sw.siemens.com](http://support.sw.siemens.com)

Send Feedback on Documentation: [support.sw.siemens.com/doc\\_feedback\\_form](http://support.sw.siemens.com/doc_feedback_form)

# Revision History ISO-26262

---

| Revision | Changes                                                                                                                                                                                                                                                                  | Status/<br>Date      |
|----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------|
| 34       | Modifications to improve the readability and comprehension of the content. Approved by Lucille Woo.<br>All technical enhancements, changes, and fixes listed in the <i>Tessent Release Notes</i> for this product are reflected in this document. Approved by Ron Press. | Released<br>Jun 2024 |
| 33       | Modifications to improve the readability and comprehension of the content. Approved by Lucille Woo.<br>All technical enhancements, changes, and fixes listed in the <i>Tessent Release Notes</i> for this product are reflected in this document. Approved by Ron Press. | Released<br>Mar 2024 |
| 32       | Modifications to improve the readability and comprehension of the content. Approved by Lucille Woo.<br>All technical enhancements, changes, and fixes listed in the <i>Tessent Release Notes</i> for this product are reflected in this document. Approved by Ron Press. | Released<br>Dec 2023 |
| 31       | Modifications to improve the readability and comprehension of the content. Approved by Lucille Woo.<br>All technical enhancements, changes, and fixes listed in the <i>Tessent Release Notes</i> for this product are reflected in this document. Approved by Ron Press. | Released<br>Aug 2023 |

Author: In-house procedures and working practices require multiple authors for documents. All associated authors for each topic within this document are tracked within the Siemens documentation source. For specific topic authors, contact the Siemens Digital Industries Software documentation department.

Revision History: Released documents include a revision history of up to four revisions. For earlier revision history, refer to earlier releases of documentation on Support Center.





# Table of Contents

---

## Revision History ISO-26262

### Chapter 1

|                                                       |           |
|-------------------------------------------------------|-----------|
| <b>Tessent Shell Introduction .....</b>               | <b>27</b> |
| What Is Tessent Shell? .....                          | 27        |
| What Can You Do With Tessent Shell? .....             | 28        |
| Tessent Shell Tcl Interface .....                     | 30        |
| Command Conventions .....                             | 30        |
| Command Completion .....                              | 31        |
| Command History .....                                 | 33        |
| Escaped Hierarchical Names in Command Tcl Lists ..... | 33        |
| Dofile Transcription .....                            | 34        |
| Tcl Command Registration .....                        | 35        |

### Chapter 2

|                                                                |           |
|----------------------------------------------------------------|-----------|
| <b>Tool Invocation, Contexts, Modes, and Data Models .....</b> | <b>37</b> |
| Tool Invocation .....                                          | 37        |
| Contexts and System Modes .....                                | 39        |
| Contexts .....                                                 | 39        |
| System Modes .....                                             | 41        |
| Context and System Mode Combinations .....                     | 42        |
| Design Levels .....                                            | 43        |
| Design Data Models .....                                       | 44        |
| Flat Design Data Model .....                                   | 44        |
| Hierarchical Design Data Model .....                           | 44        |
| ICL Data Model .....                                           | 46        |
| Object Attributes .....                                        | 46        |

### Chapter 3

|                                                                              |           |
|------------------------------------------------------------------------------|-----------|
| <b>Design Introspection and Editing .....</b>                                | <b>49</b> |
| Design Introspection .....                                                   | 50        |
| Object Specification Format .....                                            | 50        |
| Collections .....                                                            | 50        |
| Design Introspection Examples .....                                          | 53        |
| Design Introspection Command Summary .....                                   | 57        |
| Bundle Object Introspection .....                                            | 59        |
| Bundle Object .....                                                          | 59        |
| Bundle Object Data Model .....                                               | 64        |
| Introspection .....                                                          | 67        |
| Bundle Object Introspection Examples .....                                   | 71        |
| Fan-in and Fanout Tracing of Complex Signals and Bundle Object Tracing ..... | 74        |
| Connectivity Through Complex Signals .....                                   | 78        |

|                                                                   |            |
|-------------------------------------------------------------------|------------|
| Design Editing .....                                              | 83         |
| Design Editing Command Summary .....                              | 84         |
| Editing Complex Signals .....                                     | 86         |
| Editing Complex Signals With the DftSpecification Wrapper .....   | 86         |
| Editing Complex Signals: Limitations and Compatibility .....      | 87         |
| Design Editing Examples .....                                     | 88         |
| Simulation Contexts .....                                         | 94         |
| Simulation Context Overview .....                                 | 94         |
| Introspection and Analysis Using Simulation Contexts .....        | 95         |
| Automatic Design Mapping .....                                    | 98         |
| ICL and TCD Post-Synthesis Update .....                           | 98         |
| Updating ICL Attributes From the Design .....                     | 99         |
| Matching Rules for Port Names in Post-Synthesis Update .....      | 100        |
| Controlling the Name Mapping .....                                | 100        |
| ICL Objects Versus Design Objects Introspection .....             | 102        |
| <br><b>Chapter 4</b>                                              |            |
| <b>DFT Architecture Guidelines for Hierarchical Designs .....</b> | <b>105</b> |
| Hierarchical DFT Overview .....                                   | 106        |
| Physical Layout Regions in Hierarchical Test .....                | 106        |
| Pattern Retargeting .....                                         | 107        |
| Wrapped Cores and Wrapper Cells .....                             | 107        |
| Internal Mode and External Mode .....                             | 108        |
| On-Chip Clock Controller .....                                    | 109        |
| Graybox Model .....                                               | 110        |
| Top-Down Planning Before Bottom-Up Implementation .....           | 112        |
| Clocking Architecture .....                                       | 112        |
| Resource Availability .....                                       | 113        |
| Test Scheduling .....                                             | 113        |
| Specific Tasks That May Require Planning .....                    | 116        |
| Sample DFT Planning Steps .....                                   | 117        |
| DFT Implementation Strategy .....                                 | 117        |
| <br><b>Chapter 5</b>                                              |            |
| <b>RTL DFT Analysis and Insertion .....</b>                       | <b>121</b> |
| RTL DFT Analysis in the Overall Flow .....                        | 122        |
| RTL Design Rule Checks .....                                      | 123        |
| RTL IP Insertion .....                                            | 124        |
| RTL Design Assessment .....                                       | 127        |
| RTL Code Complexity Score .....                                   | 127        |
| RTL Test Point Insertion Suitability Score .....                  | 128        |
| Detailed Metrics .....                                            | 130        |
| Gate Node Location Types .....                                    | 131        |
| Preparations for RTL DFT Analysis .....                           | 134        |
| Test Point Analysis at RTL .....                                  | 136        |
| Test Point Insertion at RTL .....                                 | 136        |
| Specifying RTL Test Points With Tessent Shell Commands .....      | 138        |
| Observation Scan Type of Test Point .....                         | 139        |

## Table of Contents

|                                                                    |     |
|--------------------------------------------------------------------|-----|
| Test Point Implementations .....                                   | 143 |
| Test Points as Function Calls .....                                | 148 |
| How to Preserve DFT Logic Inserted at RTL .....                    | 153 |
| How to Report Logic Excluded From Test Point Analysis .....        | 155 |
| X-Bounding Analysis and Insertion at RTL .....                     | 158 |
| Wrapper Analysis and Dedicated Wrapper Cell Insertion at RTL ..... | 161 |
| Smart Uniquification for RTL Pro. ....                             | 163 |
| Incremental Insertion .....                                        | 168 |
| Limitations of RTL DFT Analysis and Insertion .....                | 173 |

## Chapter 6

### Tessent Shell Workflows ..... 175

|                                                                                  |     |
|----------------------------------------------------------------------------------|-----|
| Tessent Shell Flow for Flat Designs .....                                        | 176 |
| Overview of the RTL and Scan DFT Insertion Flow .....                            | 177 |
| First DFT Insertion Pass: Performing MemoryBIST and Boundary Scan .....          | 180 |
| Second DFT Insertion Pass: EDT, Hybrid TK/LBIST, and OCC .....                   | 184 |
| Loading the Design. ....                                                         | 185 |
| Specifying and Verifying the DFT Requirements .....                              | 187 |
| Creating the DFT Specification .....                                             | 190 |
| Generating the EDT, Hybrid TK/LBIST, and OCC Hardware .....                      | 194 |
| Extracting the ICL Module Description .....                                      | 194 |
| Generating ICL Patterns and Running Simulation .....                             | 195 |
| Performing Synthesis .....                                                       | 196 |
| Performing Scan Chain Insertion (Flat Design) .....                              | 197 |
| Performing ATPG Pattern Generation .....                                         | 199 |
| Simulating LBIST Faults .....                                                    | 201 |
| Considerations for Using Gate-Level Verilog Netlists .....                       | 203 |
| Tessent Shell Flow for Hierarchical Designs .....                                | 205 |
| Hierarchical DFT Terminology .....                                               | 205 |
| How the DFT Insertion Flow Applies to Hierarchical Designs .....                 | 208 |
| RTL and Scan DFT Insertion Flow for Physical Blocks .....                        | 210 |
| First DFT Insertion Pass: Performing Block-Level MemoryBIST .....                | 211 |
| Second DFT Insertion Pass: Inserting Block-Level EDT and OCC .....               | 213 |
| Specifying and Verifying the DFT Requirements: DFT Signals for Wrapped Cores ... | 216 |
| Performing Scan Chain Insertion: Wrapped Core .....                              | 218 |
| Verifying the ICL Model .....                                                    | 221 |
| Performing ATPG Pattern Generation: Wrapped Core .....                           | 222 |
| Running Recommended Validation Step for Pre-Layout Design Sign Off .....         | 227 |
| RTL and Scan DFT Insertion Flow for the Top Chip .....                           | 228 |
| Top-Level DFT Insertion Example .....                                            | 229 |
| First DFT Insertion Pass: Performing Top-Level MemoryBIST and Boundary Scan ...  | 231 |
| Second DFT Insertion Pass: Inserting Top-Level EDT and OCC .....                 | 234 |
| Top-Level Scan Chain Insertion Example .....                                     | 238 |
| Top-Level ATPG Pattern Generation Example .....                                  | 239 |
| Performing Top-Level ATPG Pattern Retargeting .....                              | 240 |
| RTL and Scan DFT Insertion Flow for Sub-Blocks .....                             | 243 |
| DFT Insertion Flow for the Sub-Block .....                                       | 243 |
| DFT Insertion Flow for the Next Parent Level .....                               | 244 |

|                                                                               |     |
|-------------------------------------------------------------------------------|-----|
| RTL and Scan DFT Insertion Flow for Instrument Blocks .....                   | 247 |
| DFT Insertion Flow for the Instrument Block .....                             | 247 |
| Instrument Block DFT Insertion Flow for the Next Parent Level .....           | 250 |
| RTL and DFT Insertion Flow With Third-Party Scan .....                        | 252 |
| DFT Insertion Flow With Third-Party Scan Insertion .....                      | 252 |
| Wrapped Core DFT Insertion With Third-Party Scan .....                        | 254 |
| Top Chip DFT Insertion with Third-Party Scan .....                            | 269 |
| Tessent Shell Flow for Tiled Designs .....                                    | 278 |
| Tiling DFT Terminology .....                                                  | 279 |
| How the DFT Insertion Flow Applies to Tiled Designs .....                     | 279 |
| Clocking During Logic Test .....                                              | 284 |
| Pre-DFT Design Modification and Integration .....                             | 288 |
| Chip Level Port Requirements and Connections .....                            | 288 |
| Tile Level Port Connections and Requirements .....                            | 288 |
| Creation of IJTAG Graybox .....                                               | 292 |
| Example Flow for a Tiled Design .....                                         | 293 |
| Tiling Flow Used in Example .....                                             | 294 |
| DFT Flow for Tiles Without TAP and BISR Controllers .....                     | 296 |
| Embedded Boundary Scan Insertion in Tiles Without a TAP Controller .....      | 296 |
| IJTAG Network Insertion in Tiles Without a TAP Controller .....               | 300 |
| Memory Test Insertion in Tiles Without TAP and BISR Controllers .....         | 303 |
| SSN and EDT Insertion in Tiles Without TAP and BISR Controllers .....         | 305 |
| Scan Insertion and Scan Modes in Tiles Without TAP and BISR Controllers ..... | 309 |
| Patterns Generation in Tiles Without a TAP Controller .....                   | 310 |
| Verification and Simulations .....                                            | 313 |
| Embedded Boundary Scan Insertion in Tiles With a TAP Controller .....         | 314 |
| IJTAG Network Insertion in Tiles With a TAP Controller .....                  | 320 |
| Memory BISR Insertion for Tiles With a BISR Controller .....                  | 322 |
| SSN Insertion for Tiles With Primary SSN Ports .....                          | 324 |
| Scan Insertion and Scan Modes for Tiles with Promoted OCCs .....              | 326 |
| Chip-Level Integration for the Tiling Flow .....                              | 328 |
| BSDL File Extraction .....                                                    | 328 |
| IJTAG Based Verification Patterns .....                                       | 329 |
| Tile Level Patterns Retargeting .....                                         | 329 |
| Tile-to-Tile Test at the Package Level .....                                  | 337 |
| Tessent Shell Post-Layout Validation Flow .....                               | 339 |
| Overview of the Post-Layout Validation Flow .....                             | 339 |
| Soft Link TSDB and Post-Layout Netlist .....                                  | 340 |
| Verify MemoryBIST, Boundary Scan and IJTAG Patterns .....                     | 341 |
| Verifying the Scan-Inserted ATPG Patterns .....                               | 342 |
| Post-Layout Validation When You Have Ungrouped IJTAG/OCC/EDT Logic .....      | 344 |
| Testbench Generation and Simulation in RTL Mode .....                         | 349 |
| Simulation Wrapper Creation .....                                             | 349 |
| Testbench Examples .....                                                      | 352 |
| Hybrid TK/LBIST Flow for Flat Designs .....                                   | 357 |
| RTL and Scan DFT Insertion Flow With Hybrid TK/LBIST .....                    | 357 |
| Perform the First DFT Insertion Pass: Hybrid TK/LBIST .....                   | 358 |
| Perform the Second DFT Insertion Pass: Hybrid TK/LBIST .....                  | 361 |
| Perform Test Point Insertion .....                                            | 366 |

## Table of Contents

|                                                                                               |            |
|-----------------------------------------------------------------------------------------------|------------|
| Perform Scan Chain Insertion (Hybrid Flow) . . . . .                                          | 369        |
| Performing ATPG Pattern Generation: Hybrid TK/LBIST . . . . .                                 | 371        |
| Performing LogicBIST Fault Simulation . . . . .                                               | 374        |
| Perform LogicBIST Pattern Generation . . . . .                                                | 376        |
| Running Multiple Load ATPG on Wrapped Core Memories with Built-In Self Repair. . . . .        | 379        |
| Overview of Multiple Load ATPG on Memories for Wrapped Cores With Built-in Self Repair<br>379 |            |
| Performing Multiple Load ATPG Pattern Generation . . . . .                                    | 380        |
| Performing Multiple Load Top-Level ATPG Pattern Retargeting . . . . .                         | 383        |
| Built-In Self Repair in Hierarchical Tessent MemoryBIST Flow . . . . .                        | 386        |
| Performing Block Level BISR Insertion . . . . .                                               | 386        |
| Performing Chip Level BISR Insertion . . . . .                                                | 389        |
| Functional Repair Clock. . . . .                                                              | 389        |
| Connection of the BISR Controller to Existing BISR Chains . . . . .                           | 389        |
| Connection of the BISR Controller to an External Fuse Box . . . . .                           | 390        |
| Connection of the BISR Controller to System Logic . . . . .                                   | 391        |
| Verification of Block- and Chip-Level BISR . . . . .                                          | 391        |
| <br><b>Chapter 7</b>                                                                          |            |
| <b>Tessent Shell Automotive Workflow . . . . .</b>                                            | <b>393</b> |
| Introduction to Tessent Automotive . . . . .                                                  | 394        |
| Test Case Overview and Objective . . . . .                                                    | 395        |
| Core-Level DFT Insertion for Automotive . . . . .                                             | 400        |
| DFT Insertion Flow for the Processor Core Physical Block . . . . .                            | 401        |
| First DFT Insertion Pass: processor_core . . . . .                                            | 402        |
| Second DFT Insertion Pass: processor_core . . . . .                                           | 405        |
| Test Point, X-Bounding, and Scan Insertion: processor_core . . . . .                          | 409        |
| ATPG Pattern Generation: processor_core . . . . .                                             | 413        |
| LogicBIST Fault Simulation: processor_core . . . . .                                          | 415        |
| LogicBIST Pattern Generation: processor_core . . . . .                                        | 416        |
| Interconnect Bridge/Open UDFM Generation: processor_core . . . . .                            | 418        |
| Cell-Neighborhood UDFM Generation: processor_core . . . . .                                   | 420        |
| Automotive-Grade ATPG Pattern Generation: processor_core . . . . .                            | 424        |
| DFT Insertion Flow for the GPS Baseband Physical Block . . . . .                              | 431        |
| DFT Insertion Pass With In-System Test: gps_baseband . . . . .                                | 432        |
| LogicBIST Fault Simulation With One NCP: gps_baseband . . . . .                               | 436        |
| Interconnect Bridge/Open UDFM Generation: gps_baseband . . . . .                              | 436        |
| Cell-Neighborhood UDFM Generation: gps_baseband . . . . .                                     | 437        |
| Automotive-Grade ATPG Pattern Generation: gps_baseband . . . . .                              | 437        |
| Top-Level DFT Insertion for the Automotive Flow . . . . .                                     | 438        |
| First DFT Insertion Pass: Top with MemoryBIST, BISR, and Boundary Scan . . . . .              | 438        |
| Second DFT Insertion Pass: Top with EDT, OCC, and In-System Test . . . . .                    | 444        |
| Scan Insertion for the Top Design . . . . .                                                   | 449        |
| ATPG Pattern Generation for the Top Design . . . . .                                          | 451        |
| ATPG Pattern Retargeting for the Top Design . . . . .                                         | 452        |
| Interconnect Bridge/Open UDFM Generation for the Top Design . . . . .                         | 452        |
| Cell-Neighborhood UDFM Generation for the Top Design . . . . .                                | 453        |
| Automotive-Grade ATPG Pattern Generation for the Top Design . . . . .                         | 453        |

|                                                                          |            |
|--------------------------------------------------------------------------|------------|
| UDFM Generation for Cell-Aware ATPG .....                                | 453        |
| TCA Based Pattern Sorting. ....                                          | 456        |
| Functional Mode Fault Tolerance for Static IJTAG Signals .....           | 458        |
| <b>Chapter 8</b>                                                         |            |
| <b>TSDB Data Flow for the Tessent Shell Flow .....</b>                   | <b>463</b> |
| Core-Level or Flat TSDB Data Flow .....                                  | 463        |
| Top-Level TSDB Data Flow .....                                           | 469        |
| <b>Chapter 9</b>                                                         |            |
| <b>Streaming Scan Network (SSN).....</b>                                 | <b>477</b> |
| Tessent SSN Workflows .....                                              | 478        |
| Block-Level SSN Insertion and Verification .....                         | 481        |
| First DFT Insertion Pass: Performing Block-Level MemoryBIST .....        | 483        |
| Second DFT Insertion Pass: Inserting Block-Level EDT, OCC, and SSN ..... | 483        |
| Synthesis for Block-Level Insertion .....                                | 490        |
| Creating the Post-Synthesis TSDB View (Block-Level) .....                | 491        |
| Performing Scan Chain Insertion With Tessent Scan (Block-Level) .....    | 493        |
| Verifying the ICL-Based Patterns After Synthesis (Block-Level) .....     | 496        |
| Generating Block-Level ATPG Patterns .....                               | 499        |
| Top-Level SSN Insertion and Verification .....                           | 506        |
| First DFT Insertion Pass: Performing Top-Level MemoryBIST .....          | 508        |
| Second DFT Insertion Pass: Inserting Top-Level EDT, OCC, and SSN .....   | 511        |
| Synthesis for Top-Level Insertion .....                                  | 519        |
| Creating the Post-Synthesis TSDB View (Top-Level) .....                  | 520        |
| Performing Scan Chain Insertion With Tessent Scan (Top-Level) .....      | 520        |
| Verifying the ICL-Based Patterns After Synthesis (Top-Level) .....       | 521        |
| Generating Top-Level ATPG Patterns .....                                 | 521        |
| Retargeting ATPG Patterns .....                                          | 523        |
| Performing Reverse Failure Mapping for SSN Pattern Diagnosis .....       | 526        |
| Advanced Topics .....                                                    | 531        |
| Streaming-Through-IJTAG Scan Data .....                                  | 532        |
| On-Chip Compare With SSN .....                                           | 535        |
| Types of Clock Networks To Use With SSN .....                            | 545        |
| Broadcast to Identical SSN Datapaths .....                               | 549        |
| Determine Failing Cores on ATE With SSN .....                            | 549        |
| Yield Statistics on ATE With SSN .....                                   | 553        |
| Manufacturing Patterns With SSN .....                                    | 558        |
| Manufacturing Pattern Quick Reference .....                              | 559        |
| SSN Pattern Structure .....                                              | 560        |
| How to Write Complete SSN Patterns .....                                 | 561        |
| How to Write JTAG and Payload Procedures Separately .....                | 562        |
| How to Write SSN On-Chip Compare Patterns .....                          | 566        |
| How To Write Streaming-Through-IJTAG Patterns .....                      | 568        |
| SSN Debug on the Tester .....                                            | 570        |
| Signoff Patterns With SSN .....                                          | 577        |
| Block-Level Signoff Patterns .....                                       | 578        |
| Top-Level Signoff Patterns .....                                         | 582        |

## Table of Contents

|                                                           |     |
|-----------------------------------------------------------|-----|
| Signoff Pattern Quick Reference .....                     | 585 |
| SSN SDC Constraints in the Design Flow .....              | 587 |
| Overview of SSN-Related SDC Procs and Constraints .....   | 588 |
| SSN/SSH SDC Constraint Descriptions .....                 | 597 |
| Fast IO .....                                             | 625 |
| Design Requirements .....                                 | 625 |
| SSN Fast IO Functionality .....                           | 627 |
| Modeling a Double Data Rate Interface .....               | 630 |
| Adaptive Tuning on the ATE .....                          | 636 |
| SSN Fast IO Bandwidth Improvement .....                   | 639 |
| Fast IO Pattern Generation .....                          | 640 |
| SSN Fast IO Limitations .....                             | 640 |
| Burn-In Pattern Support .....                             | 642 |
| Burn-In Pattern Overview .....                            | 642 |
| Recommended Flows for Creating Burn-In Patterns .....     | 644 |
| Burn-In Functionality Limitations .....                   | 646 |
| LVX Support .....                                         | 647 |
| LVX Hardware Requirements .....                           | 647 |
| Creating EDT With LVX Hardware .....                      | 648 |
| LVX Operation Modes With LVX Hardware Configuration ..... | 653 |
| Configuring LVX Patterns .....                            | 654 |
| Writing LVX Loop Patterns .....                           | 666 |
| Writing LVX Verification Patterns .....                   | 667 |
| LVI_PATCHING_INFO .....                                   | 668 |
| Retention Testing With SSN .....                          | 671 |
| Size Resolution Technology With SSN .....                 | 673 |
| Size Resolution Limitations .....                         | 674 |
| Tessent SSN Examples and Solutions .....                  | 677 |
| Third-Party OCCs With SSN .....                           | 677 |
| SSN Frequently Asked Questions .....                      | 679 |
| SSN Limitations .....                                     | 681 |

## Chapter 10

### Tessent Examples and Solutions ..... 685

|                                                                                                               |     |
|---------------------------------------------------------------------------------------------------------------|-----|
| How to Handle Parameterized Blocks During DFT Insertion .....                                                 | 686 |
| Problem .....                                                                                                 | 687 |
| Solution .....                                                                                                | 689 |
| Creation of Parameterization Wrappers and Initial TSDBs for Typical Designs .....                             | 691 |
| Block Uniquification and Creation of Parameterization Wrappers and Initial TSDBs for<br>Complex Designs ..... | 693 |
| Modifications to the First DFT Insertion Pass for Blocks .....                                                | 695 |
| Modifications to the Second DFT Insertion Pass for Blocks .....                                               | 696 |
| Synthesis for Physical Blocks .....                                                                           | 697 |
| Creation of the Gate-Level TSDB for a Physical Block .....                                                    | 697 |
| Scan Insertion for Physical Blocks .....                                                                      | 698 |
| Modifications to the First DFT Insertion Pass for Chips .....                                                 | 699 |
| Modifications to the Second DFT Insertion Pass for Chips .....                                                | 699 |
| Synthesis for the Chip Level .....                                                                            | 700 |



|                                                                                                            |            |
|------------------------------------------------------------------------------------------------------------|------------|
| Discussion . . . . .                                                                                       | 701        |
| Typical Scenario Without Uniquification . . . . .                                                          | 702        |
| Complex Scenario With Uniquification . . . . .                                                             | 708        |
| How to Avoid Simulation Issues When Using the Two-Pin Serial Port Controller . . . . .                     | 710        |
| How to Handle Clocks Sourced by Embedded PLLs During Logic Test . . . . .                                  | 713        |
| How to Design Capture Windows for Hybrid TK/LBIST . . . . .                                                | 719        |
| How to Use Boundary Scan in a Wrapped Core . . . . .                                                       | 721        |
| How to Use an Older Core TSDB With Newly-Inserted DFT Cores . . . . .                                      | 723        |
| TAP Configuration . . . . .                                                                                | 725        |
| Insert a Stand-Alone TAP in a Design . . . . .                                                             | 725        |
| Insert a TAP with an IJTAG Host Scan Interface . . . . .                                                   | 727        |
| Insert a Compliance Enable TAP With an IJTAG Interface . . . . .                                           | 729        |
| Insert a Daisy-Chained TAP . . . . .                                                                       | 730        |
| Insert a Primary TAP . . . . .                                                                             | 731        |
| Insert a Secondary TAP . . . . .                                                                           | 733        |
| Connecting to a Third-Party TAP . . . . .                                                                  | 736        |
| How to Create a WSP Controller in Tessent Shell . . . . .                                                  | 736        |
| How to Set Up Third-Party Synthesis . . . . .                                                              | 744        |
| How to Set Up Support for Third-Party OCCs . . . . .                                                       | 747        |
| How to Configure Files for Third-Party OCCs . . . . .                                                      | 747        |
| Test Logic Insertion . . . . .                                                                             | 748        |
| Configuration for Scan Insertion . . . . .                                                                 | 750        |
| Pattern Generation and Simulation . . . . .                                                                | 750        |
| Post-Synthesis Update . . . . .                                                                            | 754        |
| Limitations Related to SystemVerilog Interface Arrays . . . . .                                            | 754        |
| Matching Requirements for Port Names in Post-Synthesis Update . . . . .                                    | 755        |
| Design Name Mapping Commands . . . . .                                                                     | 756        |
| Design Name Mapping . . . . .                                                                              | 756        |
| Default Matching Rules for the get_pins, get_ports, and get_instances -match_rtl_reg<br>Commands . . . . . | 756        |
| <b>Chapter 11</b>                                                                                          |            |
| <b>Test Procedure File . . . . .</b>                                                                       | <b>759</b> |
| Test Procedure File Creation . . . . .                                                                     | 760        |
| Test Procedure File Syntax . . . . .                                                                       | 760        |
| Test Procedure File Structure . . . . .                                                                    | 765        |
| #include Statement . . . . .                                                                               | 765        |
| Set Statement . . . . .                                                                                    | 766        |
| Alias Definition . . . . .                                                                                 | 769        |
| Timing Variables . . . . .                                                                                 | 771        |
| Timeplate Definition . . . . .                                                                             | 774        |
| Multiple-Pulse Clocks . . . . .                                                                            | 780        |
| The pulse_clock Statement . . . . .                                                                        | 780        |
| Inferred Timing . . . . .                                                                                  | 781        |
| Differences Between Default add_clock and 1x Multiplier Clock . . . . .                                    | 782        |
| Always Block . . . . .                                                                                     | 782        |
| Procedure Definition . . . . .                                                                             | 783        |
| Test Procedures . . . . .                                                                                  | 795        |



## Table of Contents

|                                                                      |     |
|----------------------------------------------------------------------|-----|
| Test_Setup (Optional) . . . . .                                      | 796 |
| Shift (Required) . . . . .                                           | 799 |
| Alternate Shift Procedure (Optional) . . . . .                       | 801 |
| Load_Unload (Required) . . . . .                                     | 803 |
| Shadow_Control (Optional) . . . . .                                  | 806 |
| Master_Observe (Sometimes Required) . . . . .                        | 808 |
| Shadow_Observe (Optional) . . . . .                                  | 808 |
| Skew_Load (Optional) . . . . .                                       | 809 |
| Clock_Control (Optional) . . . . .                                   | 811 |
| Clock_run (Optional) . . . . .                                       | 823 |
| Capture Procedures (Optional) . . . . .                              | 824 |
| Rules for Creating and Editing a Default Capture Procedure . . . . . | 825 |
| Rules for Creating and Editing Named Capture Procedures . . . . .    | 825 |
| Slow and Load Types in the Cycle Statement . . . . .                 | 828 |
| launch_capture_pair Statement . . . . .                              | 829 |
| Clock_sequential (Optional) . . . . .                                | 830 |
| Init_force (Optional) . . . . .                                      | 831 |
| Test_end (Optional, all ATPG tools) . . . . .                        | 832 |
| Sub_procedure . . . . .                                              | 833 |
| Additional Support for Test Procedure Files . . . . .                | 835 |
| Creating Test Procedure Files for End Measure Mode . . . . .         | 836 |
| Serial Register Load and Unload for LogicBIST and ATPG . . . . .     | 840 |
| Register Load and Unload Use Models . . . . .                        | 840 |
| Static Versus Dynamic Register Variables . . . . .                   | 840 |
| Test Procedure File Modifications . . . . .                          | 841 |
| Dofile Modifications . . . . .                                       | 844 |
| Serial Load and Unload DRC Rules . . . . .                           | 847 |
| P13 and P54 . . . . .                                                | 847 |
| P66 . . . . .                                                        | 847 |
| W5 . . . . .                                                         | 848 |
| Procedure Examples . . . . .                                         | 849 |
| Notes About Using the stil2tessent Tool . . . . .                    | 853 |
| Extraction of Strobe Timing Information From STIL (SPF) . . . . .    | 853 |
| The STIL ClockStructures Block . . . . .                             | 853 |
| Test Procedure File Commands and Output Formats . . . . .            | 854 |

## Chapter 12

|                                                         |            |
|---------------------------------------------------------|------------|
| <b>Tessent Visualizer . . . . .</b>                     | <b>857</b> |
| Invoking Tessent Visualizer . . . . .                   | 859        |
| Invoke Explicitly on the Same Machine . . . . .         | 859        |
| Invoke Explicitly on a Different Machine . . . . .      | 860        |
| Invoke Implicitly . . . . .                             | 861        |
| License-Protected Tabs . . . . .                        | 862        |
| Framework Overview . . . . .                            | 863        |
| Tessent Visualizer Components and Preferences . . . . . | 866        |
| Tables . . . . .                                        | 867        |
| Table Toolbar Features . . . . .                        | 868        |
| Columns and Filters Editor . . . . .                    | 870        |

|                                                                          |            |
|--------------------------------------------------------------------------|------------|
| Table Filters .....                                                      | 871        |
| Data Sorting .....                                                       | 874        |
| Row Highlighting .....                                                   | 875        |
| Schematics .....                                                         | 876        |
| Toolbar .....                                                            | 877        |
| Schematic Symbols .....                                                  | 880        |
| Context Tables .....                                                     | 892        |
| Signal Net Tracing Strategies .....                                      | 902        |
| Context Menu Tracing .....                                               | 905        |
| Displayed Property .....                                                 | 905        |
| Tessent Shell Attributes .....                                           | 906        |
| Mouse Gestures .....                                                     | 907        |
| Tessent Visualizer Preferences .....                                     | 909        |
| Macros .....                                                             | 910        |
| Tooltips .....                                                           | 911        |
| Saving and Restoring the Session State .....                             | 912        |
| Window Title Prefixes .....                                              | 913        |
| Tessent Visualizer GUI Reference .....                                   | 915        |
| Hierarchical Schematic .....                                             | 915        |
| Flat Schematic .....                                                     | 921        |
| ICL Schematic .....                                                      | 923        |
| Instance Browser .....                                                   | 929        |
| ICL Instance Browser .....                                               | 933        |
| Cell Library Browser .....                                               | 934        |
| DRC Browser .....                                                        | 935        |
| Config Data Browser .....                                                | 938        |
| Transcript .....                                                         | 942        |
| Wave Viewer .....                                                        | 944        |
| Wave Viewer Toolbar .....                                                | 946        |
| Context Menu Actions .....                                               | 947        |
| Text/HDL Viewer .....                                                    | 947        |
| JTAG Network Viewer .....                                                | 949        |
| iProc Viewer .....                                                       | 952        |
| Diagnosis Report Viewer .....                                            | 953        |
| Search Features .....                                                    | 955        |
| Using Tessent Visualizer to Debug Design Issues .....                    | 959        |
| Accessing Tutorials for Tessent Visualizer .....                         | 963        |
| Accessing Videos for Tessent Visualizer .....                            | 964        |
| Keyboard Shortcuts .....                                                 | 965        |
| <br><b>Chapter 13</b>                                                    |            |
| <b>Timing Constraints (SDC) .....</b>                                    | <b>969</b> |
| Generating and Using SDC for Tessent Shell Embedded Test IP .....        | 970        |
| SDC File Generation With Tessent Shell .....                             | 970        |
| SDC Design Synthesis With Tessent Shell .....                            | 972        |
| Preparation Step 1: Sourcing SDC File .....                              | 972        |
| Preparation Step 2: Setting and Redefining Tessent Tcl Variables .....   | 972        |
| Preparation Step 3: Verifying the Declaration of Functional Clocks ..... | 974        |

## Table of Contents

|                                                                  |      |
|------------------------------------------------------------------|------|
| Preparation Step 4: Redefining Other Tessent Tcl Variables ..... | 975  |
| Synthesis Step 1: Applying the SDC Constraints .....             | 976  |
| Synthesis Step 2: Preparing the DFT Logic for Synthesis .....    | 976  |
| Synthesis Step 3: Synthesizing Your Design .....                 | 977  |
| Synthesis Step 4: Writing Out Your Final SDC .....               | 977  |
| Synthesis Step 5: Writing Out Your Final Netlist .....           | 977  |
| Running Layout with Tessent Shell DFT .....                      | 977  |
| SDC for Modal Static Timing Analysis .....                       | 983  |
| Checking Your Functional Logic Alone .....                       | 983  |
| Checking Your Embedded Test Logic .....                          | 984  |
| VHDL and Mixed Language Designs .....                            | 986  |
| VHDL Generate Blocks .....                                       | 986  |
| SDC File Contents .....                                          | 988  |
| tessent_set_default_variables .....                              | 988  |
| tessent_create_functional_clocks .....                           | 989  |
| tessent_<design_name>_set_dft_signals .....                      | 989  |
| <ltest_prefix>_disable .....                                     | 990  |
| tessent_set_non_modal .....                                      | 991  |
| tessent_<design_name>_kill_functional_paths .....                | 991  |
| IJTAG Instrument .....                                           | 992  |
| LOGICTEST Instruments .....                                      | 993  |
| MemoryBIST Instrument .....                                      | 1007 |
| BoundaryScan Instrument .....                                    | 1010 |
| Hierarchical STA in Tessent .....                                | 1011 |
| Mapping Procs .....                                              | 1016 |
| Synthesis Helper Procs .....                                     | 1017 |
| Example Scripts Using Tessent Tool-Generated SDC .....           | 1019 |
| Example Design Compiler Synthesis Script .....                   | 1019 |
| Example Fusion Compiler Synthesis Script .....                   | 1020 |
| Example Genus Synthesis Script .....                             | 1022 |
| Example PrimeTime STA Script .....                               | 1025 |

## Appendix A

### The Tessent Tcl Interface ..... 1033

|                                                                  |      |
|------------------------------------------------------------------|------|
| General Tcl Guidelines in Tessent Shell .....                    | 1033 |
| Encoding Tcl Scripts in Tessent Shell .....                      | 1035 |
| Guidelines for Modifying Existing Dofiles for Use With Tcl ..... | 1036 |
| Special Tcl Characters .....                                     | 1037 |
| Custom Tcl Packages in Tessent Shell .....                       | 1039 |
| Tcl Resources .....                                              | 1040 |

## Appendix B

### Synthesis Guidelines for RTL Designs with Tessent Inserted DFT ..... 1041

|                                                                         |      |
|-------------------------------------------------------------------------|------|
| DFT Insertion With Tessent .....                                        | 1042 |
| Synthesis Guidelines .....                                              | 1043 |
| SystemVerilog and Port Name Matching .....                              | 1045 |
| Synthesis Guidelines for Parameterization Wrapper Flows .....           | 1046 |
| Tcl Procedures for Synthesis in the Parameterization Wrapper Flow ..... | 1047 |

---

|                                                          |             |
|----------------------------------------------------------|-------------|
| <b>Appendix C</b>                                        |             |
| <b>Clocking Architecture Examples.....</b>               | <b>1049</b> |
| Clocks Driven by Primary Inputs .....                    | 1049        |
| Clock Generators Outside the Cores .....                 | 1050        |
| Clock Generators Inside the Cores .....                  | 1050        |
| Clock Sourced From a Core With Embedded PLL .....        | 1051        |
| Clock Mesh Synthesis .....                               | 1052        |
| <b>Appendix D</b>                                        |             |
| <b>Formal Verification .....</b>                         | <b>1055</b> |
| Golden RTL Versus DFT-Inserted RTL .....                 | 1055        |
| Pre-Layout Versus Post-Layout. ....                      | 1056        |
| Embedded Boundary Scan at the Physical Block Level ..... | 1057        |
| <b>Appendix E</b>                                        |             |
| <b>Solutions for Genus Synthesis Issues .....</b>        | <b>1059</b> |
| <b>Appendix F</b>                                        |             |
| <b>Getting Help .....</b>                                | <b>1061</b> |
| The Tessent Documentation System .....                   | 1061        |
| Global Customer Support and Success .....                | 1061        |
| <b>Index</b>                                             |             |
| <b>Third-Party Information</b>                           |             |

## List of Figures

---

|                                                                                                 |     |
|-------------------------------------------------------------------------------------------------|-----|
| Figure 2-1. Hierarchical Design Example . . . . .                                               | 45  |
| Figure 3-1. Hierarchical Design Example With Colors. . . . .                                    | 54  |
| Figure 3-2. Complex Signals with Tessent Visualizer . . . . .                                   | 80  |
| Figure 3-3. Hierarchical Design Example . . . . .                                               | 89  |
| Figure 3-4. Inverter Inception . . . . .                                                        | 91  |
| Figure 3-5. Tessent Visualizer Flat Schematic (gate level). . . . .                             | 96  |
| Figure 4-1. Wrapped Cores . . . . .                                                             | 107 |
| Figure 4-2. Internal Mode . . . . .                                                             | 108 |
| Figure 4-3. External Mode. . . . .                                                              | 109 |
| Figure 4-4. On-Chip Clock Controller. . . . .                                                   | 110 |
| Figure 4-5. Graybox Model . . . . .                                                             | 111 |
| Figure 4-6. Compression Analysis Example . . . . .                                              | 115 |
| Figure 5-1. RTL DFT Analysis in the Overall Insertion Flow . . . . .                            | 123 |
| Figure 5-2. Example Hierarchical Design . . . . .                                               | 129 |
| Figure 5-3. Example Design . . . . .                                                            | 132 |
| Figure 5-4. Schematic of an OST Module (use_rtl_cells : off) . . . . .                          | 140 |
| Figure 5-5. Partial Schematic of an OST for an Expression . . . . .                             | 142 |
| Figure 5-6. Example Gate Logic with Two Control Points . . . . .                                | 152 |
| Figure 6-1. Two-Pass Insertion Flow for RTL, Flat Designs . . . . .                             | 178 |
| Figure 6-2. DFT-Ready Design . . . . .                                                          | 179 |
| Figure 6-3. After the First DFT Insertion Pass . . . . .                                        | 179 |
| Figure 6-4. After the Second DFT Insertion Pass . . . . .                                       | 180 |
| Figure 6-5. Flow for the Second DFT Insertion Pass . . . . .                                    | 184 |
| Figure 6-6. Flow for Loading the Design . . . . .                                               | 185 |
| Figure 6-7. Flow for Specifying and Verifying the DFT Requirements . . . . .                    | 187 |
| Figure 6-8. Flow for Creating the DFT Specification . . . . .                                   | 190 |
| Figure 6-9. Flow for Inserting the Second-Pass Hardware . . . . .                               | 194 |
| Figure 6-10. Flow for Extracting ICL . . . . .                                                  | 195 |
| Figure 6-11. Flow for Generating and Simulating ICL Patterns . . . . .                          | 196 |
| Figure 6-12. Two-Pass Insertion Flow for RTL, Gate-Level Designs . . . . .                      | 203 |
| Figure 6-13. Hierarchical Design Levels. . . . .                                                | 207 |
| Figure 6-14. Two-Pass Insertion Flow, Physical Blocks. . . . .                                  | 210 |
| Figure 6-15. Flow for Specifying and Verifying the DFT Requirements for Wrapped Cores . . . . . | 216 |
| Figure 6-16. ATPG Pattern Generation Flow for Wrapped Cores . . . . .                           | 222 |
| Figure 6-17. Two-Pass Insertion Flow for RTL, Top Level . . . . .                               | 228 |
| Figure 6-18. Top-Level Example, Before DFT Insertion . . . . .                                  | 230 |
| Figure 6-19. Top-Level Example, After DFT Insertion for Wrapped Cores. . . . .                  | 230 |
| Figure 6-20. Top-Level Example, After DFT Insertion at the Top Level. . . . .                   | 231 |
| Figure 6-21. Two-Pass Insertion Flow for RTL, Wrapped Cores, and Third-Party Scan . . . . .     | 254 |
| Figure 6-22. DFT Signals and Multiplexer Logic Support Hierarchical ATPG . . . . .              | 257 |

|                                                                                                                                    |     |
|------------------------------------------------------------------------------------------------------------------------------------|-----|
| Figure 6-23. At-Speed Flows in the Wrapper Chain .....                                                                             | 259 |
| Figure 6-24. Current Level Path to Child Core Wrapper Chains .....                                                                 | 263 |
| Figure 6-25. Two-Pass Insertion Flow for RTL, Top Level, and Third-Party Scan .....                                                | 269 |
| Figure 6-26. Top Level EDT Connection to Child Cores .....                                                                         | 273 |
| Figure 6-27. Schematic of Tiling Example .....                                                                                     | 279 |
| Figure 6-28. Schematic of Boundary Scan Logic .....                                                                                | 281 |
| Figure 6-29. Schematic of IJTAG Network. ....                                                                                      | 282 |
| Figure 6-30. Schematic of Memory BISR Network .....                                                                                | 283 |
| Figure 6-31. Schematic of SSN Network .....                                                                                        | 284 |
| Figure 6-32. Clocking in Internal Test Mode .....                                                                                  | 285 |
| Figure 6-33. Clocking in External Mode .....                                                                                       | 286 |
| Figure 6-34. Forward and Backward Propagation of Data Between Tiles .....                                                          | 286 |
| Figure 6-35. Simplified Waveform of Forward Propagation, Negedge to Posedge .....                                                  | 287 |
| Figure 6-36. Simplified Waveform of Return Posedge to Posedge Propagation .....                                                    | 287 |
| Figure 6-37. Schematic of IJTAG Network in C2 Tile and IJTAG Ports Created on C2_i1<br>Instance With Their Input Connections ..... | 292 |
| Figure 6-38. Schematic of a Sample Test Case .....                                                                                 | 293 |
| Figure 6-39. Tiling Flow Used in Example Test Case .....                                                                           | 295 |
| Figure 6-40. Schematic of Boundary Scan Logic in Example Test Case .....                                                           | 297 |
| Figure 6-41. Schematic of Boundary Scan Logic Inside C1 .....                                                                      | 298 |
| Figure 6-42. Schematic of Embedded Boundary Scan Logic in C2 Without Pads .....                                                    | 300 |
| Figure 6-43. Schematic of IJTAG Network After Memory BIST Insertion in C1 .....                                                    | 301 |
| Figure 6-44. Schematic of IJTAG Network After Memory BIST and Logic Test insertion in Tile<br>301                                  | 301 |
| Figure 6-45. Schematic of C2 Tile With Additional Host Scan Interfaces .....                                                       | 303 |
| Figure 6-46. Schematic of MemoryBISR Inside the C2 Tile .....                                                                      | 305 |
| Figure 6-47. Schematic of SSN Network Inserted in the C1 Tile .....                                                                | 307 |
| Figure 6-48. Schematic of C2 Tile With SSN Network Inserted .....                                                                  | 309 |
| Figure 6-49. Embedded Boundary Scan Segment With Logic Test Support Ready for Integration<br>315                                   | 315 |
| Figure 6-50. Schematic After Boundary Scan Insertion in the C3 Tile. ....                                                          | 319 |
| Figure 6-51. Schematic of IJTAG Network Inserted in C3 Tile .....                                                                  | 322 |
| Figure 6-52. Schematic of Memory BISR Network Inserted in C3 Tile .....                                                            | 324 |
| Figure 6-53. Schematic of SSN Network in the C3 Tile .....                                                                         | 326 |
| Figure 6-54. Schematic of the SSN Network Configured for Retargeting of Internal Stuck<br>Patterns for All Tiles .....             | 331 |
| Figure 6-55. Schematic of SSN network Configured for Retargeting of Internal Transition<br>Patterns for Single C1_i2 Tile .....    | 332 |
| Figure 6-56. Post-Layout Validation Flow .....                                                                                     | 339 |
| Figure 6-57. Post-Layout Validation Flow with Ungrouped IJTAG/OCC/EDT Logic ....                                                   | 344 |
| Figure 6-58. Two-Pass Insertion Flow With Hybrid TK/LBIST .....                                                                    | 358 |
| Figure 6-59. Two-Pass Insertion Flow for RTL, Wrapped Cores .....                                                                  | 380 |
| Figure 6-60. Two-Pass Insertion Flow for RTL, Top Level .....                                                                      | 383 |
| Figure 7-1. Integrated Tessent DFT Solution for Automotive .....                                                                   | 394 |
| Figure 7-2. Initial Design for Automotive Test Case .....                                                                          | 397 |

## List of Figures

|                                                                                                |     |
|------------------------------------------------------------------------------------------------|-----|
| Figure 7-3. DFT Integration at the Core Level for Automotive .....                             | 397 |
| Figure 7-4. DFT Integration at the Top Level for Automotive .....                              | 398 |
| Figure 7-5. DFT Insertion Flow for Automotive Test Case .....                                  | 399 |
| Figure 7-6. DFT Insertion Flow for processor_core .....                                        | 401 |
| Figure 7-7. Enhanced Clocking and DFT Controls After Second DFT Insertion Pass .....           | 406 |
| Figure 7-8. ATPG Pattern Generation Flow for processor_core .....                              | 413 |
| Figure 7-9. Interconnect Bridge/Open UDFM Generation Flow for processor_core .....             | 419 |
| Figure 7-10. Cell-Neighborhood UDFM Generation Flow for processor_core .....                   | 422 |
| Figure 7-11. ATPG Topoff Run Flow With a UDFM for processor_core .....                         | 426 |
| Figure 7-12. ATPG Run Flow With All UDFM for processor_core .....                              | 429 |
| Figure 7-13. DFT Insertion Flow for gps_baseband .....                                         | 431 |
| Figure 7-14. Dofile Example for Top-Level Scan Chain Insertion, Automotive Flow .....          | 450 |
| Figure 7-15. Cell-Aware UDFM Generation Flow .....                                             | 455 |
| Figure 7-16. TCA Based Pattern Sorting Flow .....                                              | 457 |
| Figure 7-17. Hardware Fault Tolerant Module Example .....                                      | 460 |
| Figure 7-18. HFT Module in the Single-Pass Flow .....                                          | 461 |
| Figure 7-19. HFT Modules in the Two-Pass Flow .....                                            | 462 |
| Figure 8-1. Tessent Shell Task Flow .....                                                      | 464 |
| Figure 8-2. TSDB File Structure, Core Level, First Insertion Pass .....                        | 466 |
| Figure 8-3. TSDB File Structure, Core Level, Second Insertion Pass .....                       | 467 |
| Figure 8-4. TSDB File Structure, Core Level, Synthesis and Scan Insertion .....                | 468 |
| Figure 8-5. TSDB File Structure, Core Level, Pattern Generation .....                          | 468 |
| Figure 8-6. TSDB Data Flow, Top Level, First Insertion Pass .....                              | 471 |
| Figure 8-7. TSDB Data Flow, Top Level, Second Insertion Pass .....                             | 472 |
| Figure 8-8. TSDB Data Flow, Top Level, Scan Insertion .....                                    | 473 |
| Figure 8-9. TSDB Data Flow, Top Level, ATPG Pattern Generation .....                           | 474 |
| Figure 8-10. TSDB Data Flow, Top Level, ATPG Pattern Generation With Pattern Retargeting ..... | 475 |
| Figure 9-1. Directory Structure of Reference Testcase .....                                    | 479 |
| Figure 9-2. SSN Block-Level Workflow .....                                                     | 482 |
| Figure 9-3. SSN Top-Level Workflow .....                                                       | 507 |
| Figure 9-4. Multiplexer Node for Bypassing the gps_baseband Blocks .....                       | 515 |
| Figure 9-5. Simplified Tessent Diagnosis Two-Step Flow .....                                   | 527 |
| Figure 9-6. Streaming-Through-IJTAG Mode .....                                                 | 533 |
| Figure 9-7. Faults in Comparator Logic That Could Mask Real Faults .....                       | 539 |
| Figure 9-8. SSN Datapath in Tiling Architecture .....                                          | 547 |
| Figure 9-9. Stepping Down the Clock Tree on the Last Pipeline Stage .....                      | 548 |
| Figure 9-10. Example sticky_status Bit Annotation .....                                        | 550 |
| Figure 9-11. Example Core Instance Annotations .....                                           | 550 |
| Figure 9-12. SSN Packet Rotation Cycles .....                                                  | 551 |
| Figure 9-13. Pin Tables With Modulus and Core Instance Relationships .....                     | 551 |
| Figure 9-14. Determine Failing Core Instance With Modulus and Pin Tables .....                 | 552 |
| Figure 9-15. Example Modulus and Pin Table Annotations .....                                   | 552 |
| Figure 9-16. Annotation Where the Cycle Rotation Begins .....                                  | 553 |
| Figure 9-17. Packet Rotation on the SSN Bus .....                                              | 554 |



|                                                                                                                    |     |
|--------------------------------------------------------------------------------------------------------------------|-----|
| Figure 9-18. SSN Pattern Structure .....                                                                           | 561 |
| Figure 9-19. SSN Patterns in a Single STIL File .....                                                              | 562 |
| Figure 9-20. SSN Pattern Files With Order for Application on Tester .....                                          | 565 |
| Figure 9-21. SSN Pattern Files With Combined Setup Showing Tester Order .....                                      | 566 |
| Figure 9-22. SSN Pattern Files With On-Chip Compare Self-Test .....                                                | 568 |
| Figure 9-23. Tester Application Order for Streaming-Through-IJTAG .....                                            | 570 |
| Figure 9-24. Order To Apply Patterns for SSN Pattern Failure Debugging .....                                       | 571 |
| Figure 9-25. Order To Apply ICLNetwork Verify Patterns to the Tester .....                                         | 572 |
| Figure 9-26. Order To Apply SSN Continuity Patterns to the Tester .....                                            | 573 |
| Figure 9-27. Order To Apply SSN On-Chip Compare Self-Test Patterns .....                                           | 574 |
| Figure 9-28. Order To Apply SSH Loopback Patterns .....                                                            | 575 |
| Figure 9-29. Order To Apply SSH Loopback, Chain, and Scan Patterns .....                                           | 576 |
| Figure 9-30. FIFO MCP Example .....                                                                                | 605 |
| Figure 9-31. SSH Clock Signal Generation .....                                                                     | 607 |
| Figure 9-32. SSH Clock Group Constraints .....                                                                     | 609 |
| Figure 9-33. SSH scan_en and edt_update Generation Circuit .....                                                   | 610 |
| Figure 9-34. scan_en Margin Definition Diagram .....                                                               | 610 |
| Figure 9-35. Gated Scan Clocks With All *extra_cycle* Variables Set to 1 (Default) .....                           | 612 |
| Figure 9-36. Gated Scan Clocks With All *extra_cycle* Variables Set to 0 .....                                     | 612 |
| Figure 9-37. Divided Shift Clocks With Ratio of 4 and *extra_cycle* Variables at 0 .....                           | 612 |
| Figure 9-38. Divided Shift Clocks With Ratio of 4 and *extra_cycle* Variables at 1 .....                           | 613 |
| Figure 9-39. SSH Scan Chain Interface Circuit .....                                                                | 613 |
| Figure 9-40. Recommended SSH CTS Stop Points .....                                                                 | 621 |
| Figure 9-41. Example SSN Fast IO Bus With SDR Clocking .....                                                       | 628 |
| Figure 9-42. Example SSN Fast IO Bus With SDR Clocking and Alternate External Interface<br>Access to the SSN ..... | 628 |
| Figure 9-43. Example SSN Fast IO Bus With DDR Clocking .....                                                       | 629 |
| Figure 9-44. Example SSN Fast IO Bus With DDR Clocking and Alternate External Interface<br>Access to the SSN ..... | 629 |
| Figure 9-45. Sample DDR Interface Schematic .....                                                                  | 631 |
| Figure 9-46. Proper Tuning of Input Data and the Output Strobe .....                                               | 636 |
| Figure 9-47. Tessent LVX TDRs .....                                                                                | 649 |
| Figure 9-48. LVX Hardware With enable_one_chain for Channel Input .....                                            | 650 |
| Figure 9-49. LVX Hardware Without enable_one_chain for Channel Input .....                                         | 650 |
| Figure 9-50. Propagation of LVX Scan Outputs With enable_one_chain Hardware for Channel<br>Output .....            | 651 |
| Figure 9-51. Propagation of the LVX Scan Output Without enable_one_chain Hardware for<br>Channel Output .....      | 652 |
| Figure 9-52. ScanHost With One chain_group .....                                                                   | 675 |
| Figure 9-53. ScanHost With Multiple chain_groups .....                                                             | 676 |
| Figure 9-54. Localized Scan Timing With Shift-Only Mode OCC .....                                                  | 678 |
| Figure 10-1. Hierarchical DFT Insertion Flow with Parameterization Wrappers .....                                  | 690 |
| Figure 10-2. Example Chip With Parameterized Physical Blocks .....                                                 | 702 |
| Figure 10-3. Example Chip - Post Synthesis .....                                                                   | 706 |
| Figure 10-4. Chip With Parameterized Physical Blocks .....                                                         | 708 |



## List of Figures

---

|                                                                                     |     |
|-------------------------------------------------------------------------------------|-----|
| Figure 10-5. Example Chip (Post-Uniquification) .....                               | 709 |
| Figure 10-6. TRST Pulse Generator inside TPSP .....                                 | 712 |
| Figure 10-7. Example Chip with PLL Embedded Inside Lower Core .....                 | 715 |
| Figure 10-8. Active OCCs During Internal Test Modes of corec and coreb .....        | 715 |
| Figure 10-9. Active OCCs During Internal Test Modes of corea and coreb .....        | 716 |
| Figure 10-10. Active OCCs During Test Modes of Top Level .....                      | 718 |
| Figure 10-11. Wrapped Core Boundary Scan Flow .....                                 | 722 |
| Figure 10-12. IEEE 1687 IJTAG Network Model of the IEEE 1500 WSP Interface .....    | 741 |
| Figure 11-1. 200ns Timing Waveform .....                                            | 781 |
| Figure 11-2. Shift Procedure .....                                                  | 799 |
| Figure 11-3. Timing Diagram for Shift Procedure .....                               | 801 |
| Figure 11-4. Load_Unload Procedure .....                                            | 804 |
| Figure 11-5. Timing Diagram for Load_Unload Procedure .....                         | 805 |
| Figure 11-6. Shadow_Control Procedure .....                                         | 806 |
| Figure 11-7. Master_Observe Procedure .....                                         | 808 |
| Figure 11-8. Shadow_Observe Procedure .....                                         | 809 |
| Figure 11-9. Skew_Load Procedure .....                                              | 810 |
| Figure 11-10. Skew_load applied within Pattern. ....                                | 811 |
| Figure 11-11. Full Clock Sequential Pattern .....                                   | 831 |
| Figure 11-12. Init_force Procedure Usage .....                                      | 831 |
| Figure 12-1. Tessent Visualizer Split View .....                                    | 864 |
| Figure 12-2. Moving a Tab to the Left or Right Pane .....                           | 865 |
| Figure 12-3. Common Table Toolbar Features .....                                    | 868 |
| Figure 12-4. Columns and Filters Editor .....                                       | 870 |
| Figure 12-5. Sticky Column. ....                                                    | 873 |
| Figure 12-6. Sticky Indicator in Column Header .....                                | 873 |
| Figure 12-7. Selection Colors .....                                                 | 875 |
| Figure 12-8. Schematic With Overview Pane and Attributes Table .....                | 877 |
| Figure 12-9. Schematic Elements: Toolbar, Address Bar, and Selection Buttons .....  | 878 |
| Figure 12-10. Hierarchical Schematic Showing Pins, Ports, Instances, and Nets ..... | 880 |
| Figure 12-11. Hierarchical Instance With Callout and Pin Data .....                 | 881 |
| Figure 12-12. Showing the Internal Connectivity of a Hierarchical Instance .....    | 882 |
| Figure 12-13. Hierarchical Instance With DRC Callout .....                          | 882 |
| Figure 12-14. Hierarchical Cell .....                                               | 883 |
| Figure 12-15. Hierarchical Instance Representing an Assign Statement .....          | 883 |
| Figure 12-16. Black-Boxed Instance Example .....                                    | 884 |
| Figure 12-17. Objects in the Flat Schematic Tab .....                               | 884 |
| Figure 12-18. Persistent Buffer Symbol .....                                        | 885 |
| Figure 12-19. Bus Index Displayed at Rip Point .....                                | 886 |
| Figure 12-20. Collapsed Buffers and Inverters With No Hidden Fanout .....           | 889 |
| Figure 12-21. Expanded Buffers and Inverters With No Hidden Fanout .....            | 889 |
| Figure 12-22. Collapsed Buffers/Inverters With Hidden Fanout .....                  | 890 |
| Figure 12-23. Expanded Buffers/Inverters With Hidden Fanout. ....                   | 890 |
| Figure 12-24. Expanded Buffers/Inverters With Fanout Revealed .....                 | 891 |
| Figure 12-25. Unfolding an Instance .....                                           | 892 |

|                                                                                       |     |
|---------------------------------------------------------------------------------------|-----|
| Figure 12-26. Folded Cell Group. . . . .                                              | 892 |
| Figure 12-27. Unfolded Cell Group. . . . .                                            | 892 |
| Figure 12-28. Schematic Elements: Context Table . . . . .                             | 893 |
| Figure 12-29. Tracer Context Table . . . . .                                          | 894 |
| Figure 12-30. Context Table for Instance Pins (Hierarchical Schematic). . . . .       | 895 |
| Figure 12-31. Context Table for Instances (Hierarchical Schematic). . . . .           | 896 |
| Figure 12-32. Context Table for Nets (Hierarchical Schematic). . . . .                | 897 |
| Figure 12-33. Fanout Replaced by User-Added Primary Input. . . . .                    | 898 |
| Figure 12-34. Flat Sink . . . . .                                                     | 898 |
| Figure 12-35. Fanout Replaced by Multiple User-Added Primary Inputs . . . . .         | 899 |
| Figure 12-36. Multiple Flat Sinks . . . . .                                           | 899 |
| Figure 12-37. Context Table for Multiple Flat Sinks . . . . .                         | 900 |
| Figure 12-38. User-Added Primary Inputs . . . . .                                     | 901 |
| Figure 12-39. Netlist Driver and Context Table for User-Added Primary Inputs. . . . . | 902 |
| Figure 12-40. User-Added Primary Inputs (Flat Schematic). . . . .                     | 902 |
| Figure 12-41. Decision Point Strategy on Hierarchy Boundary at q[6] Port. . . . .     | 903 |
| Figure 12-42. Choosing a Path From the Tracing Table . . . . .                        | 903 |
| Figure 12-43. Decision Point Strategy. . . . .                                        | 904 |
| Figure 12-44. Tessent Shell Attributes Table . . . . .                                | 906 |
| Figure 12-45. Mouse Gestures in Tessent Visualizer Schematics. . . . .                | 908 |
| Figure 12-46. Light, Dark, and High-Contrast Color Themes. . . . .                    | 909 |
| Figure 12-47. Preferences Dialog Box (Text viewers Tab). . . . .                      | 910 |
| Figure 12-48. Macro Editor . . . . .                                                  | 911 |
| Figure 12-49. Running a Macro. . . . .                                                | 911 |
| Figure 12-50. Tooltip for a Collapsed Buffer Indicator. . . . .                       | 912 |
| Figure 12-51. Tooltip for a Filter Box . . . . .                                      | 912 |
| Figure 12-52. Tooltip for an Action Button. . . . .                                   | 912 |
| Figure 12-53. Setting a Window Title Prefix . . . . .                                 | 913 |
| Figure 12-54. Visualizer Window Label . . . . .                                       | 914 |
| Figure 12-55. Hierarchical Instances With Pins Shown and Hidden. . . . .              | 916 |
| Figure 12-56. Hierarchical Schematic Symbols . . . . .                                | 917 |
| Figure 12-57. Bus Tracing in the Hierarchical Schematic. . . . .                      | 918 |
| Figure 12-58. Bus Sub-Ranges . . . . .                                                | 919 |
| Figure 12-59. Showing All Bus Pins in the Hierarchical Schematic. . . . .             | 920 |
| Figure 12-60. Loopback Connection . . . . .                                           | 920 |
| Figure 12-61. Example of a Tied-Value Marker With Associated Pin Table . . . . .      | 921 |
| Figure 12-62. Flat Schematic (Cell Grouping On) . . . . .                             | 922 |
| Figure 12-63. Alias in the ICL Schematic . . . . .                                    | 924 |
| Figure 12-64. Automatically Determined Values . . . . .                               | 925 |
| Figure 12-65. Mux With Select Values Displayed . . . . .                              | 925 |
| Figure 12-66. Pins Driven or Active As Needed. . . . .                                | 926 |
| Figure 12-67. Implicitly Connected Pins. . . . .                                      | 927 |
| Figure 12-68. Displaying the Definition of an ICL Instance. . . . .                   | 928 |
| Figure 12-69. Traced and Highlighted Scan Path . . . . .                              | 929 |
| Figure 12-70. Instance Browser Elements. . . . .                                      | 931 |

## List of Figures

---

|                                                                                    |      |
|------------------------------------------------------------------------------------|------|
| Figure 12-71. Cell Grouping in Hierarchical Instance Browser .....                 | 932  |
| Figure 12-72. Debugging in the Instance Browser .....                              | 933  |
| Figure 12-73. Cell Library Browser .....                                           | 935  |
| Figure 12-74. DRC Browser With Data Grouped By Rule .....                          | 936  |
| Figure 12-75. Flat Schematic Showing a DRC Violation .....                         | 937  |
| Figure 12-76. Config Data Browser .....                                            | 939  |
| Figure 12-77. Wrapper Editing in the Context Menu .....                            | 940  |
| Figure 12-78. List of Configuration Snapshots .....                                | 942  |
| Figure 12-79. Transcript Tab With an Interactive Command Line .....                | 943  |
| Figure 12-80. Wave Viewer. ....                                                    | 945  |
| Figure 12-81. Text/HDL Viewer .....                                                | 948  |
| Figure 12-82. IJTAG Network Viewer .....                                           | 949  |
| Figure 12-83. Mux Select Values and Active Input .....                             | 950  |
| Figure 12-84. Simulation Values in IJTAG Network Viewer .....                      | 951  |
| Figure 12-85. Highlighting SIBs in the IJTAG Network Viewer .....                  | 952  |
| Figure 12-86. iProc Viewer .....                                                   | 953  |
| Figure 12-87. iProc Viewer Tooltip. ....                                           | 953  |
| Figure 12-88. Diagnosis Report Viewer (Text View) .....                            | 954  |
| Figure 12-89. Diagnosis Report Viewer (Table View) .....                           | 955  |
| Figure 12-90. Search Tab: Instances .....                                          | 956  |
| Figure 12-91. Search Tab: Pins .....                                               | 956  |
| Figure 12-92. Search Tab: Gate Pins .....                                          | 957  |
| Figure 12-93. ICL Instances Search .....                                           | 957  |
| Figure 12-94. iProc Search .....                                                   | 958  |
| Figure 12-95. Config Data Search .....                                             | 959  |
| Figure 12-96. Searching by Name .....                                              | 962  |
| Figure 13-1. Locations for Exclude Points for OCC CTS .....                        | 979  |
| Figure 13-2. tessent_test_clock and tessent_shift_capture_clock Creation. ....     | 996  |
| Figure 13-3. Generated Clock Muxed Multi-Clock Memories .....                      | 1009 |
| Figure B-1. Problem Occurrence Creating the Gate Level Netlist .....               | 1041 |
| Figure C-1. OCCs for Clocks Driven by Primary Inputs .....                         | 1049 |
| Figure C-2. OCCs With Clock Generators at Chip Level, Asynchronous Clocks .....    | 1050 |
| Figure C-3. OCCs With Clock Generators Inside the Cores, Asynchronous Clocks ..... | 1051 |
| Figure C-4. Clock Sourced From a Core With Embedded PLL, With MUX .....            | 1051 |
| Figure C-5. Clock Sourced From a Core With Embedded PLL, Without MUX .....         | 1052 |
| Figure C-6. Clock Mesh Synthesis .....                                             | 1053 |



# List of Tables

|                                                                               |     |
|-------------------------------------------------------------------------------|-----|
| Table 1-1. Tessent Shell Command Conventions .....                            | 31  |
| Table 1-2. Commands for Tcl Command Registration .....                        | 35  |
| Table 2-1. Application-Specific Environment Variables .....                   | 38  |
| Table 2-2. Tessent Shell Contexts .....                                       | 39  |
| Table 2-3. Tessent Shell System Modes .....                                   | 41  |
| Table 2-4. Tessent Shell Context and System Mode Commands .....               | 42  |
| Table 3-1. Commands That Interact With Collections .....                      | 52  |
| Table 3-2. Design Introspection Commands .....                                | 57  |
| Table 3-3. Attribute Introspection Commands .....                             | 58  |
| Table 3-4. base_class and base_type SystemVerilog Examples .....              | 65  |
| Table 3-5. Complex Signals Design Introspection Commands .....                | 71  |
| Table 3-6. Design Editing Commands .....                                      | 83  |
| Table 3-7. Design Editing Command Summary .....                               | 84  |
| Table 4-1. Test Schedule Example .....                                        | 116 |
| Table 5-1. Tessent IP Configuration and Insertion Instructions .....          | 125 |
| Table 5-2. RTL Code Complexity Score .....                                    | 128 |
| Table 5-3. RTL Test Point Insertion Suitability Score .....                   | 129 |
| Table 5-4. Gate Node Location Mapping .....                                   | 132 |
| Table 6-1. Test Pattern Bit Name Assignments .....                            | 353 |
| Table 8-1. Core-Level TSDB Data Flow Inputs and Outputs .....                 | 464 |
| Table 8-2. Top-Level TSDB Data Flow Inputs and Outputs .....                  | 469 |
| Table 9-1. SSN Manufacturing Pattern Summary .....                            | 558 |
| Table 9-2. Manufacturing Pattern Quick Reference .....                        | 559 |
| Table 9-3. Block-Level Verification Pattern Summary .....                     | 581 |
| Table 9-4. Top-Level Verification Pattern Summary .....                       | 584 |
| Table 9-5. Signoff Pattern Quick Reference .....                              | 585 |
| Table 9-6. set_load_unload_timing_options Arguments and SDC Variables .....   | 607 |
| Table 9-7. SSN Bandwidth Comparison for I/O Port/Frequency Combinations ..... | 639 |
| Table 9-8. LVX Modes With enable_one_chain Hardware Configuration .....       | 654 |
| Table 9-9. Size Resolution Value Limitations Summary .....                    | 674 |
| Table 10-1. Comparison of IEEE 1687 and IEEE 1500 Ports .....                 | 739 |
| Table 10-2. Mux Select Signals and wsp_ir Value to Select Input 1 .....       | 742 |
| Table 11-1. Reserved Punctuation Characters .....                             | 761 |
| Table 11-2. Procedure File Tool Command Summary .....                         | 854 |
| Table 12-1. Filtering Summary .....                                           | 874 |
| Table 12-2. Schematic Toolbar Actions .....                                   | 878 |
| Table 12-3. Marker Symbols .....                                              | 886 |
| Table 12-4. Config Data Browser Buttons and Actions .....                     | 941 |
| Table 12-5. Wave Viewer Toolbar .....                                         | 946 |
| Table 12-6. Wave Viewer Context Menu .....                                    | 947 |

---

|                                                                             |      |
|-----------------------------------------------------------------------------|------|
| Table 12-7. Global Shortcuts .....                                          | 965  |
| Table 12-8. Schematic Shortcuts .....                                       | 965  |
| Table 12-9. Instance Browser Shortcuts .....                                | 966  |
| Table 12-10. Filter Editing Shortcuts .....                                 | 966  |
| Table 12-11. Text Search Shortcuts .....                                    | 966  |
| Table 12-12. Wave Viewer Mouse Shortcuts .....                              | 966  |
| Table 13-1. set_ijtag_retargeting_options Arguments and SDC Variables ..... | 992  |
| Table A-1. Common Dofile Issues and Solutions .....                         | 1036 |
| Table A-2. Common Tcl Characters .....                                      | 1038 |

# Chapter 1

## Tessent Shell Introduction

---

Tessent™ Shell is a platform from which you can run all the Tessent tools. The platform includes shared design data, a common database, and powerful scripting utilities that provide a fully automated design-for-test (DFT) flow as well as the ability to customize the flow to fit specific requirements.

|                                                       |           |
|-------------------------------------------------------|-----------|
| <b>What Is Tessent Shell?</b> .....                   | <b>27</b> |
| <b>What Can You Do With Tessent Shell?</b> .....      | <b>28</b> |
| <b>Tessent Shell Tcl Interface</b> .....              | <b>30</b> |
| Command Conventions. ....                             | 30        |
| Command Completion .....                              | 31        |
| Command History .....                                 | 33        |
| Escaped Hierarchical Names in Command Tcl Lists ..... | 33        |
| Dofile Transcription .....                            | 34        |
| Tcl Command Registration .....                        | 35        |

## What Is Tessent Shell?

Tessent™ Shell is a DFT environment within which you can perform all the tasks required to insert DFT hardware, generate manufacturing test patterns, and perform post-silicon tasks such as diagnosis and yield analysis.

Tessent Shell provides a flexible design flow that supports a high level of automation or customization. Key aspects of the environment that support automation as well as enhance flexibility:

- **Shared Data Models** — Tessent Shell uses data models to store your design data. The Tessent environment shares these models across all tools and functions, and you can access the data stored within them to customize your design flow.

For standard automated flows, you do not need to be aware of this infrastructure. For those flows that need to extend the native tool commands, they have the same access to the design data model as the Tessent Shell tools do.

For more information about the data models, refer to “[Design Data Models](#)” on page 44.

- **Attributes** — Attributes are characteristics associated with design objects, such as library cells, pins, and modules, that are stored within data models. Some are predefined, and you can also create your own attributes. Using attributes increases visibility into your design, enabling you to manipulate and query the features of the design objects.

- **Design Introspection** — Introspection is the act of examining the design objects and attributes stored within the data models. Introspection enables you to retrieve the design data you need so that you can use Tcl scripting techniques to automate your own custom designs flows.

For more information about design introspection, refer to “[Design Introspection](#)” on page 50.

- **Tcl** — Use this scripting language across all Tessent tools and functions. Through scripting, you can customize and extend the Tessent Shell standard flows as needed for your design requirements.

For more information about Tcl usage, refer to “[Tessent Shell Tcl Interface](#)” on page 30.

- **Tessent Shell Database (TSDB)** — The TSDB is a database that stores all the directories and files that Tessent Shell generates as you move through a design flow. The TSDB enhances flow automation by acting as the central location where Tessent tools can access the data it requires for the current task, whether that task be reading in a design, performing DRC, inserting logic test hardware, or performing ATPG pattern generation.

For more information about the TSDB, refer to “[TSDB Data Flow for the Tessent Shell Flow](#)” on page 463.

- **IJTAG Automation** — By default, the DFT instruments that you create with Tessent Shell are controlled through an IJTAG network and the IEEE 1687 protocol. Tessent Shell automatically inserts the IJTAG infrastructure. This simplifies test setup and control of all the instruments at all levels of hierarchy, and enables reuse in hierarchical designs and testbenches at any stage of chip development.

For more information about IJTAG, refer to the [Tessent IJTAG User’s Manual](#).

## What Can You Do With Tessent Shell?

The Tessent Shell platform is a jumping-off point for accomplishing the full array of DFT tasks available within the Tessent tool suite. Specific DFT tasks may require different licenses, but all tools may be launched from and managed via Tessent Shell.

DFT tasks include:

- **Instrument Insertion** — Generate and insert logic test hardware such as EDT (embedded deterministic testing) and OCC (on-chip clock controller), in addition to LogicBIST, MemoryBIST, in-system test, and boundary scan.
- **Scan Analysis and Insertion** — Perform scan analysis and scan chain insertion.
- **ATPG** — Generate ATPG patterns and perform fault simulation, including compression technology.



- **Defect Diagnosis and Yield Analysis** — Perform test failure diagnosis to determine a defect’s most probable failure mechanism, as well as statistical analysis of diagnostic failures to find systemic defects.

Additionally, Tessent Shell provides general-purpose design editing support so that you can modify your design netlists as needed. You can work on the command line as described in “[Design Editing](#),” or use the [Tessent Visualizer](#) graphical interface.

If you are getting started with Tessent Shell, refer to “[Tessent Shell Workflows](#)” for information about the Tessent Shell workflow for RTL and scan DFT insertion.

# Tessent Shell Tcl Interface

The Tessent Shell environment uses a Tcl-based interface that you can use from the command line or by writing dofile scripts.

The Tcl interface supports Tcl constructs such as variables, command substitution, flow control, and procedures. You can embed Tcl constructs in tool commands and embed tool commands within Tcl constructs the same as with any Tcl command.

For guidelines about using Tcl in Tessent Shell, and information about converting existing dofiles and using special characters, refer to “[The Tessent Tcl Interface](#)” appendix.

|                                                              |           |
|--------------------------------------------------------------|-----------|
| <b>Command Conventions</b> .....                             | <b>30</b> |
| <b>Command Completion</b> .....                              | <b>31</b> |
| <b>Command History</b> .....                                 | <b>33</b> |
| <b>Escaped Hierarchical Names in Command Tcl Lists</b> ..... | <b>33</b> |
| <b>Dofile Transcription</b> .....                            | <b>34</b> |
| <b>Tcl Command Registration</b> .....                        | <b>35</b> |

## Command Conventions

Tessent Shell provides a unified Tcl-style command set and naming convention. Commands that begin with a certain first word (for example, “get” in get\_attribute\_value\_list) perform operations on the current data model.

[Table 1-1](#) provides a summary listing by command first word of the Tessent Shell commands. Refer to the [Tessent Shell Reference Manual](#) for a complete list of commands and options. You can also use the “help” command with a wildcard to see a complete list of commands that start with the same word:

```
> help get_*
```

Table 1-1. Tessent Shell Command Conventions

| Command First Word                                                                           | Types of Operations Performed                                                                                                                                                                  |
|----------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| append_<br>compare_<br>copy_<br>filter_<br>foreach_<br>index_<br>remove_<br>sizeof_<br>sort_ | Operates on collections. Refer to “ <a href="#">Collections</a> ” for more information.                                                                                                        |
| get_                                                                                         | Returns data strings or collections of lists, objects, or object attributes from the design object model for introspection.                                                                    |
| read_                                                                                        | Reads files into memory, such as libraries and netlists.                                                                                                                                       |
| report_                                                                                      | Displays information about a specific item, such as the current context or licenses currently checked out.                                                                                     |
| set_                                                                                         | Specifies options, contexts, modes, attributes, and the current design. Refer to “ <a href="#">Contexts and System Modes</a> ” and “ <a href="#">Object Attributes</a> ” for more information. |
| write_                                                                                       | Writes design data to a file or set of files.                                                                                                                                                  |

## Command Completion

In the Tessent Shell interface, you can use the Tab key to complete command names, command option names, and command option values. If the command you type is ambiguous, pressing the Tab key lists all matching commands. Additionally, within a command, pressing the Tab key can match variable names with \$.

Many Tessent Shell commands are constructed as follows:

*command\_name command\_options command\_option\_values*

For example:

**set\_config\_value -in\_wrapper /TopBuilder(CHIP)**

Using Tab key completion, you can complete each of the command parts (name, options, and option values).

For information about Tcl shell handling in Tessent Shell, refer to the [set\\_tcl\\_shell\\_options](#) command.

## Command Name Tab Completion

As an example of command name completion, you can abbreviate the `get_attribute_list` command in the Tessent Shell interface by typing the following:

```
SETUP> g_a_l <tab>
```

Pressing the Tab key after the string completes and expands the command so that it looks like this:

```
SETUP> get_attribute_list
```

## Command Option Name Tab Completion

Tab completion for command option names is available for most commands. For example:

```
SETUP> get_config_value -m<tab>
-meta_id      -meta_name  -meta_object

SETUP> get_config_value -meta_
```

## Command Option Value Tab Completion

Tab completion for command option values is available for the following value types:

- Configuration data path
- Enum (Enumerated) values

### Configuration Data Path Example

In this example, pressing the Tab key on the partial configuration data path completes the path:

```
SETUP> set_config_value -in_wrapper /TopB<tab>
SETUP> set_config_value -in_wrapper /TopBuilder(CHIP)/
```

### Enum Value Example

In this example, using the Tab key lists the supported time values `-time_unit` option to the `get_config_value` command:

```
SETUP> get_config_value -time_unit <tab>
as_is fs    ms    ns    ps    s    us

SETUP> get_config_value -time_unit
```

## Dofile Entries

You can use abbreviated commands (for example, **g\_a\_l** for **get\_attribute\_list**) in Tessent Shell dofiles, but it is best to always use the full command name. There is no fixed minimum typing for any command or option, and current minimum typing could change in future releases because of the addition of new commands or options. Using the full command names also adds readability to your dofiles.

## Command History

Tessent Shell provides many interactive command line conveniences found in other Linux shells.

On the command line, Tessent Shell binds the up arrow key to scroll back in history to show previous commands, which you can easily rerun by pressing the enter key. Tessent Shell also binds the down arrow key to scroll forward in the command history.

Tessent Shell also supports history expansion with an exclamation mark “!” as follows:

- **!!** — reruns the last command.
- **!*n*** — reruns the *<n>*th command. *<n>* must be an unsigned integer.
- **!*-n*** — reruns the *<n>*th last command. “!*-1*” is equivalent to “!!”.
- **!*?str*** — reruns the last command that contains the *<str>* string.
- **!*str*** — reruns the last command that starts with the *<str>* string.

Tessent Shell does not support any other command selection or replacement options.

## Escaped Hierarchical Names in Command Tcl Lists

Options, introspection, and editing commands that accept multiple object specifications as an argument expect a list of names (for example, `get_pins`, `create_connections`, `set_ijtag_instance_options`). Verilog escaped identifiers contain spaces that, if not correctly passed to these commands, could prevent Tessent from finding the object.

You must pass these names in a list to the command. You can do this in Tcl by adding a layer of braces or by framing the string or variable within a “list” command.

For example:

The `get_pins` command interprets the string of the pin path as a list and breaks it into two name patterns: “`escaped@mem_inst`” and “`/CLKW`,” which do not match anything in the design.

```
SETUP> get_pins {\escaped@mem_inst /CLKW}  
// Error: No result.
```

Passing that string in a list passes the correct full pattern “\escaped@mem\_inst /CLKW,” and Tessent Shell can find the requested object. Here are four different methods of accomplishing this.

```
SETUP> get_pins [list {\escaped@mem_inst /CLKW}]
```

```
{ {\escaped@mem_inst /CLKW} }
```

```
SETUP> get_pins {{\escaped@mem_inst /CLKW}}
```

```
{ {\escaped@mem_inst /CLKW} }
```

```
SETUP> set pin_name {\escaped@mem_inst /CLKW}
```

```
SETUP> get_pins [list $pin_name]
```

```
{ {\escaped@mem_inst /CLKW} }
```

```
SETUP> lappend pin_list {\escaped@mem_inst /CLKW}
```

```
SETUP> get_pins $pin_list
```

```
{ {\escaped@mem_inst /CLKW} }
```

## Dofile Transcription

By default, the transcript style is Full, which means the tool writes out all commands read from a dofile before any Tcl evaluation occurs. The command appears in the transcript as entered but is commented out.

During Tcl evaluation, the Tcl commands are written to the transcript as follows:

- The tool writes all commands from the dofile with a “*// command:*” prefix. This includes Tessent Shell commands, Tcl commands, and complete loop and if/else constructs.
- The tool writes all commands embedded in Tcl constructs to the transcript as run with a “*// sub\_command:*” prefix. Tessent Shell completes the variable and command substitutions and writes the resolved values in the loop to the transcript.

---

### Note



This does not apply to introspection commands. Introspection commands are not written to the transcript as subcommands.

---

- The tool indents nested dofiles when they are written to the logfile.
- The tool can read a Tcl routine in much the same manner as a dofile. If you issue the source command (>source *my\_report\_env*), then the Tcl commands in the routine are not transcribed.

Control the Tessent Shell transcription behavior using the [set\\_transcript\\_style](#) command.

For information about modifying existing dofiles for use with the Tessent Shell Tcl interface, refer to “[Guidelines for Modifying Existing Dofiles for Use With Tcl](#).”

## Tcl Command Registration

You can convert any Tcl procedure into a Tessent Shell application command. This enables you to implement functionality in Tcl and register that functionality as a command so that Tessent Shell handles it like any other built-in command. Like any built-in command, newly registered Tcl commands support Tab completion, and you can control their availability relative to the context and system mode. The help system also includes the new command.

The following commands enable you to register new commands and to define arguments and options for those commands.

**Table 1-2. Commands for Tcl Command Registration**

| Command                                   | Description                                 |
|-------------------------------------------|---------------------------------------------|
| <a href="#">display_message</a>           | Controls messages produced by Tcl commands. |
| <a href="#">lock_current_registration</a> | Locks all current Tcl commands.             |
| <a href="#">register_tcl_command</a>      | Registers a Tcl command.                    |
| <a href="#">unregister_tcl_command</a>    | Unregisters a Tcl command.                  |

When you first run the new command, the tool performs syntactic and semantic checks, and provides the parsed results to your Tcl implementation of the command. The tool also provides a mechanism to automatically define and register user-defined commands at tool invocation.

For more information about creating your own application commands, refer to the examples included with the [register\\_tcl\\_command](#) description in the *Tessent Shell Reference Manual*.





# Chapter 2

## Tool Invocation, Contexts, Modes, and Data Models

---

In the Tessent Shell environment, setting the context and system mode orients the tool as to which task you want to perform. The tool uses design data models to store the relevant data about your design.

|                                        |           |
|----------------------------------------|-----------|
| <b>Tool Invocation</b> .....           | <b>37</b> |
| <b>Contexts and System Modes</b> ..... | <b>39</b> |
| <b>Design Data Models</b> .....        | <b>44</b> |

## Tool Invocation

When you invoke Tessent Shell, the tool starts up in setup mode.

You can invoke Tessent Shell from a Linux shell using the following syntax:

```
% tessent -shell
```

To use most Tessent Shell functionality, you must load a cell library after invocation using the `read_cell_library` command.

Refer to the **tessent** shell command description in the *Tessent Shell Reference Manual* for additional invocation options.

### Tessent Startup File

During invocation, Tessent Shell reads the `.tessent_startup` startup file. You can use this startup file for both general and tool-specific settings. The default location for the startup file is the home directory: `$HOME/.tessent_startup`.

This startup file is common to the contexts listed in [Table 2-2](#) on page 39.

### Tessent Shell Environment Variables

Tessent Shell provides environment variables for Tessent Shell environment operation in addition to application-specific environment variables. During invocation, the tool reads all environment variables that you have set.

The following table lists the environment variables that you can set for Tessent Shell environment operation.

**Table 2-1. Application-Specific Environment Variables**

| Variable                        | Description                                                                                                                                                                                                                                                                                                       |
|---------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| TESENT_LICENSE_EXCLUDE          | Specifies licenses that Tessent tools cannot check out. The list of licenses uses the same terminology accepted by the “ <a href="#">set_context -license</a> ” command.                                                                                                                                          |
| TESENT_LICENSE_INCLUDE          | Specifies the only licenses that Tessent tools can check out. The list of licenses uses the same terminology accepted by the “ <a href="#">set_context -license</a> ” command.                                                                                                                                    |
| TESENT_LICENSE_ORDER            | Specifies the order in which Tessent tools check out licenses. The list of licenses uses the same terminology accepted by the “ <a href="#">set_context -license</a> ” command.                                                                                                                                   |
| TESENT_STARTUP                  | Specifies the pathname of a directory that contains a Tessent tool startup file.<br>Default: No default                                                                                                                                                                                                           |
| TESENT_TMP_LOCATION             | Specifies the location at which the tool creates the <i>.tessent.tmp.hostname.process_id</i> scratch directory. For more information, see the <code>command</code> description in the <i>Tessent Shell Reference Manual</i> .                                                                                     |
| TESENT_UNDERSCORE_COMMANDS_ONLY | Directs the tool to only enable commands that use the underscore style. When this environment variable is set to any value, the legacy commands that used spaces are disabled. For example, the tool would accept <code>analyze_drc_violation</code> but would not accept “ <code>analyze drc violation</code> ”. |

## Related Topics

[read\\_cell\\_library](#) [Tessent Shell Reference Manual]

[tessent](#) [Tessent Shell Reference Manual]

Managing Mentor Tessent Software

[set\\_context](#) [Tessent Shell Reference Manual]

[get\\_scratch\\_directory](#) [Tessent Shell Reference Manual]

# Contexts and System Modes

One of the first tasks you perform after invoking Tessent Shell is setting the context and system mode for the current session.

The term “context” refers to a broad category of functionality that often corresponds to a specific point tool, product, or license feature, such as Tessent FastScan. Each context includes several system modes, which specify the tool’s current state of operation. By setting the context and then the system mode, you indicate the type of tasks you want Tessent Shell to perform.

In addition, you must specify the design level where you want Tessent Shell to run tasks.

|                                                   |           |
|---------------------------------------------------|-----------|
| <b>Contexts</b> .....                             | <b>39</b> |
| <b>System Modes</b> .....                         | <b>41</b> |
| <b>Context and System Mode Combinations</b> ..... | <b>42</b> |
| <b>Design Levels</b> .....                        | <b>43</b> |

## Contexts

The context specifies the functional category of tasks you want to perform with Tessent Shell.

[Table 2-2](#) lists the contexts that are currently available.

**Table 2-2. Tessent Shell Contexts**

| Context              | Description                                                                                                                                                                                                                                                               |
|----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| dft                  | Editing and introspection of the following types of designs: gate-level Verilog, RTL Verilog, RTL SystemVerilog, and RTL VHDL.                                                                                                                                            |
| dft -edt             | EDT IP generation and optional insertion. This corresponds to the IP creation phase of Tessent TestKompress®.                                                                                                                                                             |
| dft -logic_bist -edt | Configuration, generation, and insertion of the EDT/LBIST hybrid controller IP. This corresponds to Tessent LogicBIST.                                                                                                                                                    |
| dft -no_sub_context  | Specifies that a command is only available in the dft context when no sub-context, such as -scan and -edt, was specified. See the <a href="#">register_tcl_command</a> in the <i>Tessent Shell Reference Manual</i> for more information.                                 |
| dft -scan            | Scan analysis and scan chain insertion. This corresponds to Tessent Scan. Additionally in this context you can prepare a BIST-ready design and include tasks such as X-bounding, MCP/FP handling, and test point insertion. These features correspond to Tessent ScanPro. |

**Table 2-2. Tessent Shell Contexts (cont.)**

| Context                   | Description                                                                                                                                                                                                                                                                                         |
|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| dft -test_points          | Test point identification and insertion. The test point identification algorithm focuses on either pattern count reduction or improving random pattern testability of the design.                                                                                                                   |
| patterns -failure_mapping | Reverse map top-level failures to the core so that you can perform diagnosis with Tessent Diagnosis. Used after performing scan pattern retargeting within the patterns -scan_retarting context.                                                                                                    |
| patterns -ijtag           | PDL command retargeting for IJTAG (IEEE 1687-2014) plus extraction of the ICL network from the design.                                                                                                                                                                                              |
| patterns -scan            | Test pattern generation and good and fault simulation. This corresponds to Tessent FastScan and the test pattern generation phase of Tessent TestKompress. The patterns -scan context supports uncompressed scan ATPG/fault simulation, EDT ATPG/fault simulation, and Logic BIST fault simulation. |
| patterns -scan_diagnosis  | Test failure diagnosis to determine a defect's failure mechanism and location. This corresponds to Tessent Diagnosis.                                                                                                                                                                               |
| patterns -scan_retarting  | Scan pattern retargeting for retargeting core-level test patterns at the top level.                                                                                                                                                                                                                 |
| patterns -silicon_insight | Control, simulate, debug, and characterize BIST-related memories, logic, PLLs, and SerDes. This corresponds to Tessent SiliconInsight.                                                                                                                                                              |

## Context Specification

Set the Tessent Shell context using the [set\\_context](#) command. For example:

```
SETUP> set_context dft -scan
```

You must set the context after you invoke Tessent Shell and before you can enter most commands. The `set_context` command is available only in setup mode. You can use the [get\\_context](#) command to see the current context.

Prior to setting the context, you can only run a small set of setup commands. These commands are those that you would typically place inside the startup file.

## Context and Licensing

When you set the context, the tool automatically acquires the appropriate license, if available. Alternatively, you can directly specify the license Tessent Shell acquires by using the [set\\_context](#) command with the optional `-license` switch.

You can control the order in which Tessent Shell checks out licenses by setting the `TESSENT_LICENSE_ORDER` environment variable. You provide, as the value of the environment variable, a space-separated list of licenses using the terminology described for “`set_context -license`.” For example:

```
setenv TESSENT_LICENSE_ORDER "Scan TestKompress IJTAG"
```

Any available licenses not explicitly listed in the value of the `TESSENT_LICENSE_ORDER` environment variable are appended to the list in their original order. The tool issues a warning if the environment variable contains a license that does not exist.

You can exclude licenses such that Tessent Shell cannot check them out by setting the `TESSENT_LICENSE_EXCLUDE` environment variable. You provide, as the value of the environment variable, a space-separated list of licenses using the terminology described for “`set_context -license`.” The following example excludes two licenses:

```
setenv TESSENT_LICENSE_EXCLUDE "testkompress mtslogicbistf"
```

You can limit the licenses that Tessent Shell can check out by setting the `TESSENT_LICENSE_INCLUDE` environment variable. You provide, as the value of the environment variable, a space-separated list of licenses using the terminology described for “`set_context -license`.” The tool excludes all other licenses if you set this environment variable. The following example includes only two licenses for Tessent Shell to check out:

```
setenv TESSENT_LICENSE_INCLUDE "mtfastscanf fastscan"
```

For more information about specifying a license feature, refer to the [set\\_context](#) command description in the *Tessent Shell Reference Manual*. For a complete list of license feature names, refer to [Managing Tessent Software](#).

## System Modes

A system mode in Tessent Shell defines the operational state of the tool. The default system mode is `setup`. The available system mode depends on the current context of the tool.

[Table 2-3](#) lists the available system modes.

**Table 2-3. Tessent Shell System Modes**

| System Mode | Description                                                                                                   |
|-------------|---------------------------------------------------------------------------------------------------------------|
| setup       | Used as the entry point into the tool. Used to define the current context and specify the design information. |
| analysis    | Used to perform design analysis, test pattern generation, PDL retargeting, and simulation.                    |
| insertion   | Used to perform design editing and introspection.                                                             |

Change the Tessent Shell system mode using the [set\\_system\\_mode](#) command. For example:

```
SETUP> set_system_mode insertion
```

## Context and System Mode Combinations

Together, the context and system modes imply a Tessent Shell task flow from setup to analysis to insertion when you are working with DFT designs, and from setup to analysis when you are working with patterns. For example, if you are working on a DFT design while in setup mode, you cannot perform scan analysis.

The following matrix lists the actions you can perform in the available contexts and system modes. The tool filters out the functionality that does not apply to the context/mode you are currently in and issues an error when you attempt to perform a task not within the scope of the current context/system mode.

| Context  | Setup Mode                                                                                                                                                                       | Analysis Mode                                                                                                                                                                                                                                                           | Insertion Mode                                                                                               |
|----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| dft      | <ul style="list-style-type: none"><li>• Read and configure design</li><li>• Prepare for design editing, IP creation, scan insertion, test point analysis and insertion</li></ul> | <ul style="list-style-type: none"><li>• Scan analysis</li><li>• Test point analysis</li><li>• EDT IP creation</li><li>• Hybrid TK/LBIST IP creation</li><li>• MemoryBIST IP creation</li><li>• Boundary scan IP creation</li><li>• In-System Test IP creation</li></ul> | <ul style="list-style-type: none"><li>• RTL or gate-level design editing with design introspection</li></ul> |
| patterns | <ul style="list-style-type: none"><li>• Read and configure design</li><li>• Define test pattern generation type to perform</li></ul>                                             | <ul style="list-style-type: none"><li>• ATPG/fault simulation</li><li>• PDL retargeting</li><li>• Scan pattern retargeting</li><li>• Run an IJTAG test pattern</li><li>• Perform SimDUT simulation</li></ul>                                                            | (not applicable)                                                                                             |

The tool provides a set of commands that enable you to interact with contexts and system modes.

**Table 2-4. Tessent Shell Context and System Mode Commands**

| Command                     | Description                                                                                                                                                                                         |
|-----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">get_context</a> | Returns the current context as specified by the <a href="#">set_context</a> command. Also returns inferred sub-contexts, such as whether patterns -scan is configured to perform LogicBIST or ATPG. |
| <a href="#">set_context</a> | Sets the current context and its options.                                                                                                                                                           |

**Table 2-4. Tessent Shell Context and System Mode Commands (cont.)**

| Command                         | Description           |
|---------------------------------|-----------------------|
| <a href="#">set_system_mode</a> | Sets the system mode. |

## Design Levels

When working with DFT designs—that is, you are working in context dft—you must set the design level at which you are performing tasks. There is no default setting for the design level.

The available design levels are chip, physical block, and sub-block. For flat designs, you always work at the chip level. For hierarchical designs, it is crucial to differentiate whether you are work at the top-level (chip) or at lower levels within physical blocks or sub-blocks. Refer to “[Hierarchical DFT Terminology](#)” for definitions of these terms.

For complete information, refer to the [set\\_design\\_level](#) command description in the *Tessent Shell Reference Manual*.

# Design Data Models

Tessent Shell has the following data models: the hierarchical design data model, the flat design data model, and the ICL data model. Each of these data models contains one or more design objects, such as pins and modules, and each design object is associated with a set of attributes, such as an ID or bit-width of a bus.

For a complete description of the various data models, object types, and attributes, refer to the “Data Models” chapter in the *Tessent Shell Reference Manual*.

|                                             |           |
|---------------------------------------------|-----------|
| <b>Flat Design Data Model</b> .....         | <b>44</b> |
| <b>Hierarchical Design Data Model</b> ..... | <b>44</b> |
| <b>ICL Data Model</b> .....                 | <b>46</b> |
| <b>Object Attributes</b> .....              | <b>46</b> |

## Flat Design Data Model

The flat design data model is an internal, flattened representation of a hierarchical design that the tool creates when entering analysis mode. You can also explicitly create the flat model using the `create_flat_model` command.

The flat design data model consists of gates that are connected together. A gate is an instance of a primitive module, and a `gate_pin` object represents a pin on a gate instance. `Gate_pin` objects have no unique instance name. Instead they have a unique ID that differentiates one from another. The format of the ID is two integers separated by a period character. The first integer represents the gate ID, and the second integer represents the pin index (where 0 is the output pin, 1 is first input pin, and so on). However, this ID does not remain in place from one netlist version to the next or from one Tessent Shell invocation to the next. For this reason, you should avoid hard-coding this ID into a script.

Note

 You can preserve hierarchical pins in the flat model by using the [set\\_attribute\\_options -preserve\\_boundary\\_in\\_flat\\_model](#) option.

For more information about the flat design data model and the `gate_pin` object type, refer to “Flat Design Data Model” in the *Tessent Shell Reference Manual*.

## Hierarchical Design Data Model

Tessent Shell translates your design netlist into a hierarchical design data model in memory when you load the design into the tool using a command such as [read\\_verilog](#).  
The design data remains in memory during the Tessent Shell session until you delete the model or exit the tool.



The hierarchical design data model contains the following object types:

- **Module** — A basic building block of your design. A module can be a Verilog module, a Tessent library model, or a built-in primitive.
- **Instance** — A single instantiation of a module.
- **Port** — An input, output, or inout interface of a module.
- **Pin** — An input or output interface of an instance.
- **Net** — A wire that connects the pins of instances.
- **Pseudo\_port** — Represents a user-added primary input or primary output.

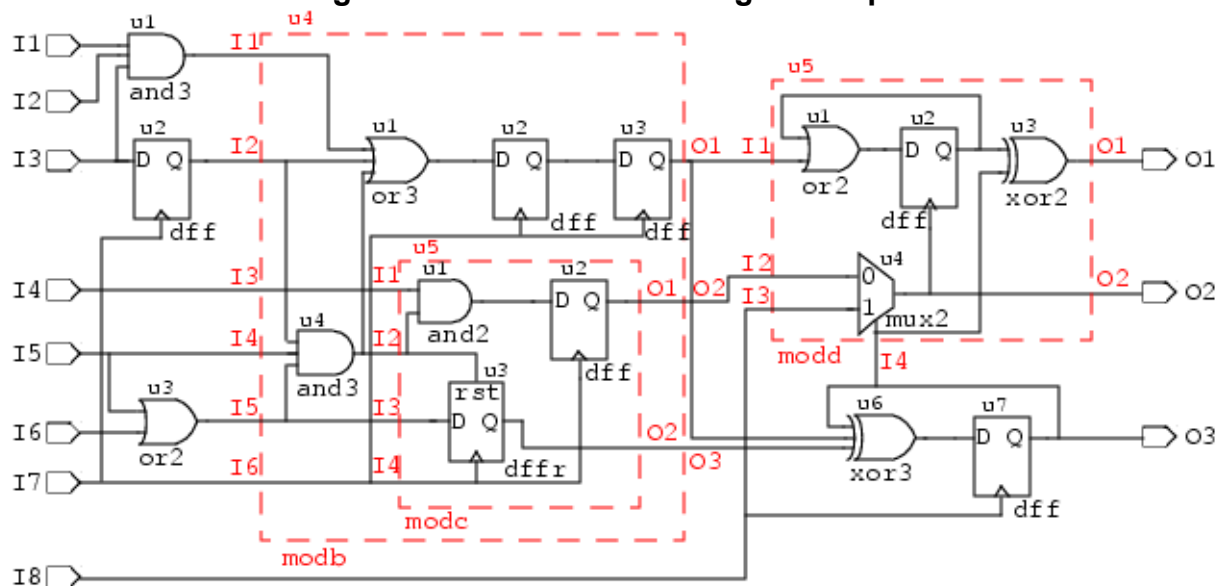
These object types are described in detail in the “[Data Models](#)” chapter of the *Tessent Shell Reference Manual*.

Use the [set\\_current\\_design](#) command to designate one module within your design as the current design. Instances, pins, and nets are hierarchical objects defined relative to the current design.

[Figure 2-1](#) shows a hierarchical design upon which many of the examples in this chapter are based. The label above the elements is the instance name while the label below the elements is the module name. For example, the module name of the AND gate in the upper-left corner is `and3`, and its instance name is `u1`.

The dashed boxes are modules instantiated in the parent module. For example, module `modc` is instantiated inside of module `modb`. Its complete instance path is `u4/u5`.

**Figure 2-1. Hierarchical Design Example**



## ICL Data Model

The Instrument Connectivity Language (ICL) describes the elements that comprise the IJTAG network as well as their logical (though not necessarily physical) connections to each other and to the instruments at the endpoints of the network.

ICL bears a loose resemblance to a hierarchical netlist such as Verilog; it is organized by modules that may contain instances of other modules, and it describes the connections between the pins of the instances. ICL is not a complete netlist. The connections are port-to-port rather than through nets, and ICL uses abstraction rather than include the detailed physical construction of the circuitry. ICL represents only the behavioral operation of the network.

The ICL data model defines objects as one of various object types, such as `icl_module`, `icl_instance`, `icl_port`, and `icl_pin`.

Each `icl_module` object has its list of objects including the following:

- `icl_port`
- `icl_instance_in_module`
- `icl_pin_in_module`
- `icl_scan_register_in_module`
- `icl_scan_mux_in_module`
- `icl_one_hot_scan_group_in_module`
- `icl_alias_in_module`
- `icl_scan_interface_of_module`

Once the current design is set using the [set\\_current\\_design](#) command, the ICL data model is elaborated downward from the current design and `icl_instance` objects are created for each instance of `icl_module`, and `icl_pin` objects are created for each port of the modules associated to the `icl_instances`. The `icl_instance` and `icl_pin` objects are hierarchical objects and therefore only exist after the current design is set. The same holds for objects of type `icl_scan_register`, `icl_scan_mux`, `icl_one_hot_scan_group`, `icl_alias`, and `icl_scan_interface`.

For more information about the ICL data model and the `icl_module`, `icl_instance`, `icl_port`, and `icl_pin` object types, refer to “[ICL Data Model](#)” in the *Tessent Shell Reference Manual*.

## Object Attributes

Each design object in a design data model has a list of characteristics, called attributes, attached to that object. For example, each pin has an attribute that specifies its hierarchical name and its parent instance. There are both predefined and user-defined attributes.

Predefined attributes provide access to information known to the tool such as the name of a module or the direction of a pin. All design objects have some predefined attributes, and every design object can have user-defined attributes as well. For a complete list of attributes for each type of data model, refer to “[Data Models](#)” chapter in the *Tessent Shell Reference Manual*.

The process for creating a new user-defined attribute is called *registration*. The predefined attributes, unlike the user-defined attributes, do not need to be registered.

You must register each new user-defined attribute and specify a default value before you can use the attribute. If you want to later change the default value, you must unregister and then re-register the attribute. For more information about the registration process, refer to the description of the [register\\_attribute](#) command in the *Tessent Shell Reference Manual*.

You can query any attribute and change any user-defined attribute. Most predefined attributes are read-only, although some are read-write, such as the `is_hard_module` attribute.

You can filter and sort objects based on the attributes and their values. The `filter_collection` and `sort_collection` commands perform these operations. For more information about filtering attributes, refer to “[Attribute Filtering Equation Syntax](#)” in the *Tessent Shell Reference Manual*.

Intuitively named “`get_`” commands return collections of objects from the data model and the model’s attributes that you can use for design introspection of various design objects.



# Chapter 3

## Design Introspection and Editing

---

Tessent Shell provides a full range of commands for examining (introspecting) and editing designs.

Along with the command-line interface, Tessent Shell provides a graphical user interface, Tessent Visualizer, that enables you to edit and introspect designs, and adjust object attributes. For details, refer to “[Tessent Visualizer](#)” on page 857.

|                                                             |            |
|-------------------------------------------------------------|------------|
| <b>Design Introspection .....</b>                           | <b>50</b>  |
| <b>Design Editing .....</b>                                 | <b>83</b>  |
| <b>Simulation Contexts.....</b>                             | <b>94</b>  |
| <b>Automatic Design Mapping .....</b>                       | <b>98</b>  |
| <b>ICL Objects Versus Design Objects Introspection.....</b> | <b>102</b> |

## Design Introspection

Tessent Shell introspection commands enable you to examine the design objects within a design data model. They provide access to the tool's internal data structures, giving you the flexibility of Tcl scripting while keeping all compute-intensive processing in the tool's backend. The commands operate on object specifications that you include as arguments to the commands, and they return collections.

|                                                   |           |
|---------------------------------------------------|-----------|
| <b>Object Specification Format</b> .....          | <b>50</b> |
| <b>Collections</b> .....                          | <b>50</b> |
| <b>Design Introspection Examples</b> .....        | <b>53</b> |
| <b>Design Introspection Command Summary</b> ..... | <b>57</b> |
| <b>Bundle Object Introspection</b> .....          | <b>59</b> |

## Object Specification Format

An object specification lists the design objects (such as instances, pins, or nets) that the specified command acts upon. The object specification can be the name of an object, a Tcl list of names of objects, or a collection of objects. For design objects that have unique IDs, like instances, you can use the IDs as the object names.

The following examples show the `add_schematic_objects` command with valid object specifications listed within square brackets (`[]`) or braces (`{}`).

```
add_schematic_objects [get_instances u*] -display hierarchical_schematic
```

```
add_schematic_objects [list u1 u2 u3] -display hierarchical_schematic
```

```
add_schematic_objects {u1 u2 u3} -display hierarchical_schematic
```

These commands display the results in Tessent Visualizer.

All Tessent Shell commands operating on user-specified objects accept object specifications as arguments.

## Collections

Collections are an extension of Tcl that are specific to Tessent Shell applications. Native Tcl commands such as “foreach” and “puts” do not recognize collections. A collection represents a group of zero (an empty set) or more design objects.

Design introspection commands return collections of objects. The objects are stored in the internal data structures, and the tool returns a string handle to this collection. The string handle is a “@” character followed by a numeric ID (in this case, “@1”). The entire data volume remains in the tool's internal data structures, so the Tcl interface is not overloaded with large amounts of data.

Introspection commands run directly from the shell display the “name” attribute of the first 50 elements of the collection because the name is more useful than displaying the collection’s ID. However, the tool does not display names in non-interactive mode, such as when running a dofile.

The following example demonstrates the use of the [get\\_instances](#) and [get\\_name\\_list](#) commands to return collections of objects:

```
SETUP>set var1 [get_instances u* -hierarchical -of_modules MOD1]
```

```
{u1 u2 u3 u4 u5 u6 u7}
```

```
SETUP>puts $var1@1
```

```
SETUP>puts [get_name_list $var1]
```

```
u1 u2 u3 u4 u5 u6 u7
```

In this example, the first command returns a collection of names. The objects are stored in internal data structures, and the tool returns a string handle to this collection that is “@” followed by a number ID (“@1”).

In the following example, the collection created by the [get\\_instances](#) command is stored in the variable `instCollection`, which means that the collection is referenced.

```
set instCollection [get_instances -of_type cell]
```

Tessent Shell deletes the collection when you unset the variable `instCollection` or set the variable to a new value, or when the Tcl variable(s) referring to the collection go out of scope.

In the following example, the collection created by the command [get\\_pins](#) is passed to the [get\\_attribute\\_value\\_list](#) command. This means that the collection is referenced.

```
get_attribute_value_list [get_pins u1/a*] -name direction
```

Tessent Shell deletes the collection when the command [get\\_attribute\\_value\\_list](#) returns a value.

Collections can refer to objects that no longer exist because they have been deleted using editing commands, such as [delete\\_pins](#). The built-in attribute “is\_valid” is set to false when an object has been deleted. All commands that accept collection pointers as input automatically ignore objects with the “is\_valid” attribute set to false.

## Persistence of Collections

The tool references a collection when the collection is stored in a Tcl variable or when it is passed to a command or procedure. The tool automatically deletes a collection when it is no longer referenced.

You should not use Tcl built-in commands that expect a Tcl list, such as the `foreach` command, with collections. When you pass a collection to this type of command, the command dereferences the collection and, as a result, deletes the collection.

## How to Transcript the Contents of Collections

Use the [get\\_name\\_list](#) command to return a list of the names of all the elements in the collection. So, to transcript the names in the log file, you can use the “puts” command with [get\\_name\\_list](#) (or any other Tessent [get\\_\\*](#) command):

## Commands That Work With Collections or Tcl Lists

Tessent Shell provides commands that create, manipulate, and query collections. [Table 3-1](#) presents some Tessent Shell commands based on how they interact with collections.<sup>1</sup>

**Table 3-1. Commands That Interact With Collections**

| Commands that...                | Command Name                             |
|---------------------------------|------------------------------------------|
| Create collections or Tcl lists | <a href="#">get_attribute_list</a>       |
|                                 | <a href="#">get_attribute_option</a>     |
|                                 | <a href="#">get_attribute_value_list</a> |
|                                 | <a href="#">get_current_design</a>       |
|                                 | <a href="#">get_fanins</a>               |
|                                 | <a href="#">get_fanouts</a>              |
|                                 | <a href="#">get_gate_pins</a>            |
|                                 | <a href="#">get_instances</a>            |
|                                 | <a href="#">get_modules</a>              |
|                                 | <a href="#">get_name_list</a>            |
|                                 | <a href="#">get_nets</a>                 |
|                                 | <a href="#">get_pins</a>                 |
|                                 | <a href="#">get_ports</a>                |
|                                 |                                          |
| Operate on collections          | <a href="#">add_to_collection</a>        |
|                                 | <a href="#">append_to_collection</a>     |
|                                 | <a href="#">compare_collections</a>      |
|                                 | <a href="#">copy_collection</a>          |
|                                 | <a href="#">filter_collection</a>        |
|                                 | <a href="#">foreach_in_collection</a>    |
|                                 | <a href="#">index_collection</a>         |
|                                 | <a href="#">remove_from_collection</a>   |
|                                 | <a href="#">sort_collection</a>          |

1. The table does not list all the applicable commands. Refer to the *Tessent Shell Reference Manual*.



**Table 3-1. Commands That Interact With Collections (cont.)**

| Commands that...                                                           | Command Name                             |
|----------------------------------------------------------------------------|------------------------------------------|
| Read and write attributes of objects found within collections or Tcl lists | <a href="#">get_attribute_list</a>       |
|                                                                            | <a href="#">get_attribute_value_list</a> |
|                                                                            | <a href="#">report_attributes</a>        |
|                                                                            | <a href="#">set_attribute_value</a>      |

## Design Introspection Examples

Design introspection enables you to create procedures to show design data of interest, create and set attributes, create and manipulate collections, create custom reports, and other tasks.

### Example of Creating a Procedure to Show Flip-Flop Fanouts

The following example is based on [Figure 2-1](#) on page 45 and creates a procedure called `show_fanouts` that returns the fanouts of a specified flip-flop.

```
proc show_fanouts {flop} {
    set ff_fanout [get_fanouts $flop/Q]
    puts "$flop/Q is connected to: [get_name_list $ff_fanout]"
}
```

The following two examples show how to use the `show_fanouts` procedure you just created.

**> show\_fanouts u2**

```
u2/Q is connected to : u4/u4/A0 u4/u1/A1
```

**> foreach\_in\_collection ff [get\_instances \* -of\_module dff\* -hierarchical] \**  
**{show\_fanouts [get\_name\_list \$ff]}**

```
u2/Q is connected to: u4/u1/A1 u4/u4/A0
u7/Q is connected to: O3 u6/A0 u5/u3/A1 u5/u4/S0
u4/u2/Q is connected to: u4/u3/D
u4/u3/Q is connected to: u6/A1 u5/u1/A1
u4/u5/u2/Q is connected to: u5/u4/A0
u5/u2/Q is connected to: u5/u1/A0 u5/u3/A0
u4/u5/u3/Q is connected to: u6/A2
```

### Example of Displaying the Number of Input Ports

The following example displays the number of input ports on modules (objects with names that begin with “mod” in [Figure 2-1](#)).

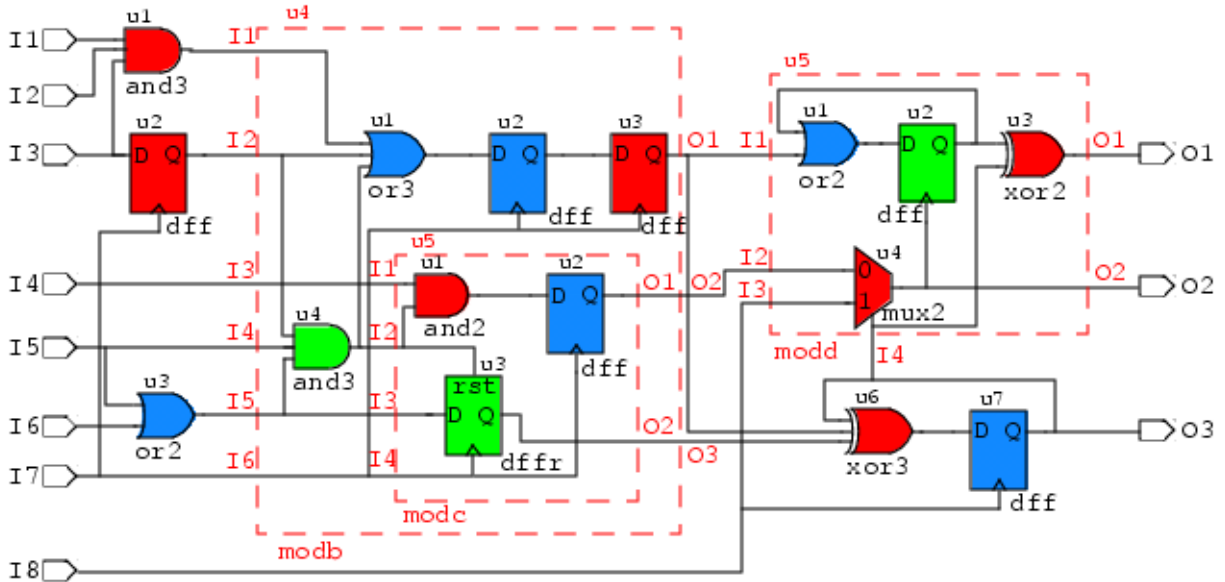
**> set mods [get\_modules mod\* -of\_type design]**

**> foreach\_in\_collection itr \$mods \{puts "The number of input ports of \**  
**[get\_attribute\_value\_list \$itr -name name] \**  
**is [sizeof\_collection [get\_ports -of\_module \$itr -direction input]]"**

The number of input ports of modb is 6  
The number of input ports of modc is 4  
The number of input ports of modd is 4

The design in Figure 3-1 is identical to Figure 2-1, with the addition of colors to help you understand the examples that follow.

**Figure 3-1. Hierarchical Design Example With Colors**



## Example of Creating Attributes and Manipulating Collections

The following example is based on Figure 3-1 and shows the complete process of creating and setting user-defined attributes, and then creating and manipulating collections.

```
register_attribute -name color -value_type string -default "blue" -enum "red green blue" \
  -object_types instance
```

Set the “color” attribute of each instance to match Figure 3-1.

```
set_attribute_value {/u4/u4 /u4/u5/u3 /u5/u2} -name color -value "green"
```

```
{u4/u4 u4/u5/u3 u5/u2}
```

```
set_attribute_value {u1 u2 u6 /u4/u5/u1 /u4/u3 /u5/u4 /u5/u3} -name color -value "red"
```

```
{u1 u2 u6 u4/u5/u1 u4/u3 u5/u4 u5/u3}
```

```
set_attribute_value {u3 u7 /u4/u1 /u4/u2 /u4/u5/u2 /u5/u1} -name color -value "blue"
```

```
{u3 u7 u4/u1 u4/u2 u4/u5/u2 u5/u1}
```

Create collections of instances with the same color values.

```
set_all_inst [get_instances -of_type cell]
```

```
{u1 u2 u3 u6 u7 u4/u1 u4/u2 u4/u3 u4/u4 u4/u5/u1 u4/u5/u2 u4/u5/u3 u5/u1
u5/u2 u5/u3 u5/u4}
```

```
set red_inst [filter_collection $all_inst {color == "red"}]
```

```
{u1 u2 u6 u4/u3 u4/u5/u1 u5/u3 u5/u4}
```

```
set blue_inst [filter_collection $all_inst {color == "blue"}]
```

```
{u3 u7 u4/u1 u4/u2 u4/u5/u2 u5/u1}
```

```
set green_inst [filter_collection $all_inst {color == "green"}]
```

```
{u4/u4 u4/u5/u3 u5/u2}
```

Manipulate the collections and create additional collections.

```
puts "Number of red cells in the design: [sizeof_collection $red_inst]"
```

```
Number of red cells in the design: 7
```

```
puts [compare_collections $red_inst $blue_inst]
```

```
1
```

```
set red_green [add_to_collection $red_inst $green_inst]
```

```
{u1 u2 u6 u4/u3 u4/u5/u1 u5/u3 u5/u4 u4/u4 u4/u5/u3 u5/u2}
```

```
set sort_red [sort_collection $red_inst name -descending]
```

```
{u6 u5/u4 u5/u3 u4/u5/u1 u4/u3 u2 u1}
```

## Example of Displaying Attribute Values of a Collection

After Tessent Shell has run the previous commands, the following commands display the colors assigned to gates in the collection named ANDs.

```
set ANDs ""
```

```
append_to_collection ANDs [get_instances -of_type cell -filter {module_name == "AND"}]
```

```
foreach_in_collection itr $ANDs {puts "The color value for \  
[get_attribute_value_list $itr -name name] is [get_attribute_value_list $itr -name color] "}
```

```
The color value for u1 is red
```

```
The color value for u4/u4 is green
```

```
The color value for u4/u5/u1 is red
```

As an alternate way to register the attribute in the previous example, you can use the `register_attribute` command as shown here:

```
register_attribute -name is_red -value_type bool -object_types "instance" \  
-description "True if color is red."
```

So, instead of using the color attribute with enumerated values of red, green, and blue as shown previously, you could instead register three Boolean type attributes of `is_red`, `is_green`, and `is_blue`. This enables you to use Boolean values for filtering and tracing. For example:

```
> set red_inst [filter_collection $all_inst {is_red}]
```

## Example of Changing an Attribute of a Collection

The following command example is based on [Figure 3-1](#) and shows how to change the color attribute of a collection of DFFs to green.

```
set DFFs [get_instances -of_modules dff]

# display all of the instances that have a color attribute of blue

set all_blue [filter_collection $all_inst "color == blue"]
{u7 u4/u2 u4/u5/u2}

# change all instances in the collection DFFs to have a "color" attribute value of green

set_attribute_value $DFFs -name color -value green

# now check the results
set all_green [ filter_collection [$all_inst {color == "green"}] ]
{u2 u7 u5/u2 u4/u4 u4/u2 u4/u3 u4/u5/u2 u4/u5/u3}
```

## Example of Removing Duplicate Objects from a Collection

The following example shows how to create a collection containing three duplicate objects and then remove the duplicate objects from the collection.

```
set t [get_instances {u3 u3 u3}]
{u3 u3 u3}

sizeof_collection $t
3

set tt [add_to_collection "" $t -unique]
{u3}

sizeof_collection $tt
1
```

## Example of Creating a Custom Report

The following dofile example creates a collection of all instance objects with a leaf name starting with “u” and then displays ten instance names per row.

```
set cnt 0
set line ""
foreach_in_collection element [get_instances u* -hierarchical] {
    if {$cnt < 10} {
        append line "[get_single_name $element] "
        incr cnt
    } else {
        puts $line
        set line ""
        append line "[get_single_name $element] "
        set cnt 1
    }
}
if {$cnt > 0} {puts $line}
```

## Design Introspection Command Summary

The design introspection commands include the `get_` commands as well as various commands for manipulating attributes.

### Design Introspection Command Summary

Table 3-2 lists the common design introspection commands. Refer to the *Tessent Shell Reference Manual* for more information.

**Table 3-2. Design Introspection Commands**

| Command Name                               | Description                                                                                                  |
|--------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| <a href="#">get_common_parent_instance</a> | Returns the common ancestor of all instances, pins, or nets found in <code>object_spec</code> .              |
| <a href="#">get_current_design</a>         | Returns a collection of one element that specifies the top-level design when previously set.                 |
| <a href="#">get_design_level</a>           | Returns the design level previously defined with the <code>set_design_level</code> command.                  |
| <a href="#">get_fanins</a>                 | Returns a collection of all objects found in the fan-in of the specified pin, net, or port objects.          |
| <a href="#">get_fanouts</a>                | Returns a collection of all requested objects found in the fanout of the specified pin, net, or port object. |
| <a href="#">get_gate_pins</a>              | Returns a collection of all <code>gate_pin</code> objects found below the current design in the flat model.  |
| <a href="#">get_instances</a>              | Returns a collection of instances relative to the current design.                                            |
| <a href="#">get_modules</a>                | Returns a collection of modules.                                                                             |
| <a href="#">get_nets</a>                   | Returns a collection of nets relative to the current design.                                                 |
| <a href="#">get_pins</a>                   | Returns a collection of pins relative to the current design.                                                 |

**Table 3-2. Design Introspection Commands (cont.)**

| Command Name              | Description                                      |
|---------------------------|--------------------------------------------------|
| <a href="#">get_ports</a> | Returns a collection of ports on a given module. |

## Attribute Introspection Command Summary

[Table 3-3](#) lists all of the commands that introspect attributes.

**Table 3-3. Attribute Introspection Commands**

| Command Name                             | Description                                                                          |
|------------------------------------------|--------------------------------------------------------------------------------------|
| <a href="#">get_attribute_list</a>       | Lists registered attributes.                                                         |
| <a href="#">get_attribute_option</a>     | Returns the current setting of an attribute's option.                                |
| <a href="#">get_attribute_value_list</a> | Returns the attribute's value.                                                       |
| <a href="#">get_name_list</a>            | Returns the name attribute of the specified objects.                                 |
| <a href="#">get_single_name</a>          | Returns a string with the name of the element specified by the object_spec argument. |
| <a href="#">register_attribute</a>       | Registers a new attribute by adding it to the attribute manager.                     |
| <a href="#">report_attributes</a>        | Prints a report for the registered attributes.                                       |
| <a href="#">reset_attribute_value</a>    | Resets the attribute to its default value.                                           |
| <a href="#">set_attribute_options</a>    | Configures attribute options.                                                        |
| <a href="#">set_attribute_value</a>      | Defines the attribute's value.                                                       |
| <a href="#">unregister_attribute</a>     | Makes an attribute unusable by removing it from the attribute manager.               |

# Bundle Object Introspection

The Bundle Object Hierarchy is an extension of the module instance hierarchy, and the signals (pins, ports, and nets) are considered as a tree of sub signals. The root bundle is the signal itself, the leaf objects are the bit-blasted elements of the given signal, and the nodes are the middle bundle objects.

To support this bundle object concept new object types are added to [Tessent Shell Hierarchical Design Data Model](#). The leaf objects are represented by net, pin, port object types; the sub and root bundle objects are represented by net\_bundle, pin\_bundle, and port\_bundle objects types.

The bundle object is set of bits that represents a sub signal of a given net or port. This concept is applied to complex and simple signals for RTL and no RTL flow. The difference between simple and complex signals is in the number of levels that can be explored through the introspection commands.

Each complex signal has a select part. For example, for the following complex signal:

```
B [5] .c [2] .d
```

The portion in red ([5].c[2].d) is the select part, and the “B” is the root bundle. Refer to the “Bundle Object” topic for complete details. The select part of the signals helps to explore the Bundle Object Hierarchy level-by-level.

Tessent Shell supports introspection of RTL complex signals through a bundle object, which is a set of bits that represent a sub signal of a given RTL net or port. Using Tessent Shell introspection commands enable exploration inside the bundle object and selection of different bundle parts of the signals using the hierarchical bitselect.

|                                                                                    |           |
|------------------------------------------------------------------------------------|-----------|
| <b>Bundle Object.....</b>                                                          | <b>59</b> |
| <b>Bundle Object Data Model .....</b>                                              | <b>64</b> |
| <b>Introspection.....</b>                                                          | <b>67</b> |
| <b>Bundle Object Introspection Examples.....</b>                                   | <b>71</b> |
| <b>Fan-in and Fanout Tracing of Complex Signals and Bundle Object Tracing.....</b> | <b>74</b> |
| <b>Connectivity Through Complex Signals .....</b>                                  | <b>78</b> |

## Bundle Object

A bundle object is a set of bits that represents a whole signal or its sub signals of a given RTL port or net.

The bundle is made up of Tessent Shell [Hierarchical Design Data Model](#) object types that you introspect using Tessent Shell introspection commands relative to the current design. The following illustrates this concept:

```
Root bundle (Signal itself)
|--> Node (Sub Signal)
|   |--> Leaf (Bit-blasted elements)

Example:net_bundle (Bundle Object Type)
|--> net_bundle (Bundle Object Type)
|   |--> net (Bit-blasted Object Type)

pin_bundle (Bundle Object Type)
|--> pin_bundle (Bundle Object Type)
|   |--> pin (Bit-blasted Object Type)

port_bundle (Bundle Object Type)
|--> port_bundle (Bundle Object Type)
|   |--> port (Bit-blasted Object Type)
```

The following object types contain several common attributes that are also associated with the bit-blasted object types (net, pin, and port):

- [net\\_bundle](#)
- [pin\\_bundle](#)
- [port\\_bundle](#)

All attributes added to port, pin, and net are also added to port\_bundle, pin\_bundle, and net\_bundle.

For example, the naming is common between a bit-blasted object type and a bundle object type.

See “[Bundle Object Data Model](#)” on page 64 for complete information.

## Naming Attributes

The following table shows the list of the principal name attributes associated with net, port, and pin object types.



| Attribute           | Description                                                                                                                                                                                                                                           | RTL View                                         | Synthesized View                                                                                     |
|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------------------------------------------|
| name                | The active name. The name depends on the selected view. This includes the select part in the case of signals (port, pin, and net).<br><br>The -rtl mode has the RTL view and the synthesized view.<br><br>The -no_rtl mode only has the netlist view. | The full RTL name including all the select part. | The full synthesis name including all the select part (flatten position in case of complex signals). |
| parent_bundle_name  | The bundle parent of the port, pin, or net.                                                                                                                                                                                                           | The parent name of the selected RTL object.      | The parent name of the selected synthesized object.                                                  |
| post_synthesis_name | The synthesis name, created by the quick synthesis.                                                                                                                                                                                                   | The computed equivalent post synthesis name.     | Equal to “name” attribute.                                                                           |
| pre_synthesis_name  | RTL name, the original name before synthesis.                                                                                                                                                                                                         | Equal to “name” attribute.                       | Full RTL name, the original name before synthesis.                                                   |
| root_bundle_name    | The active name without the select part.                                                                                                                                                                                                              | The base RTL name without the select part.       | The base synthesis name without the select part.                                                     |

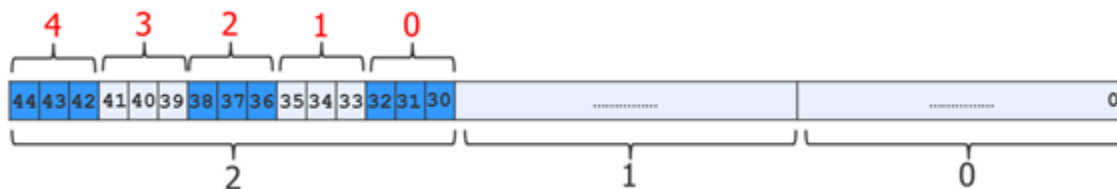
## Bundle Naming Examples

### Example 1

The `post_synthesis_name` attribute for complex signals is usually the base name with the flatten position as the index. SystemVerilog interface signals are the exception. For example, in case of the following input port “data\_in” declared as array of structure in SystemVerilog:

```
typedef struct{
    logic a;
    logic [1:0] b;
} data_t;
input data_t [2:0][4:0] data_in;
```

The following figure illustrates the representation in the flatten dimension:



| Name Attribute (RTL View) | post_synthesis_name Attribute |
|---------------------------|-------------------------------|
| data_in[2][1].a           | data_in[35]                   |
| data_in[2][1].b           | data_in[34:33]                |
| data_in[2][1].b[1]        | data_in[34]                   |

The following table shows the different naming of the signal “data\_in[2][1].b[1]”

| Attribute           | RTL View           | Synthesis View     |
|---------------------|--------------------|--------------------|
| name                | data_in[2][1].b[1] | data_in[34]        |
| pre_synthesis_name  | data_in[2][1].b[1] | data_in[2][1].b[1] |
| post_synthesis_name | data_in[34]        | data_in[34]        |
| root_bundle_name    | data_in            | data_in            |

## Example 2

In this example, the “net1” from Example 1 is declared inside nested generate blocks:

```
typedef struct{
    logic a;
    logic [1:0] b;
} data_t;

genvar i, j;
generate
    for (i=0; i < 2; i=i+1) begin
        for (j=0; j < 2; j=j+1) begin : MEM_j
            data_t [2:0] [4:0] net1;
            .....
        end
    end
endgenerate
```

The following table shows the different naming of “genblk1[0].MEM\_j[0].net1[2][1].b[1]”

| Attribute | RTL View                            | Synthesis View                 |
|-----------|-------------------------------------|--------------------------------|
| name      | genblk1[0].MEM_j[0].net1[2][1].b[1] | \genblk1[0].MEM_j[0].net1 [34] |

| Attribute           | RTL View                            | Synthesis View                      |
|---------------------|-------------------------------------|-------------------------------------|
| pre_synthesis_name  | genblk1[0].MEM_j[0].net1[2][1].b[1] | genblk1[0].MEM_j[0].net1[2][1].b[1] |
| post_synthesis_name | \genblk1[0].MEM_j[0].net1 [34]      | \genblk1[0].MEM_j[0].net1 [34]      |
| root_bundle_name    | genblk1[0].MEM_j[0].net1            | \genblk1[0].MEM_j[0].net1           |

**Example 3**

This example illustrates an interface declared as a net to connect two sub instances.

```

interface intf;
  logic a;
  logic b;
  modport in (input a, output b);
  modport out (input b, output a);
endinterface

module top;
  intf i ();
  u_a m1 (.i1(i));
  u_b m2 (.i2(i));
endmodule

module u_a(intf.in i1);
endmodule
module u_b(intf.out i2);
endmodule

```

The following tables shows the different naming of the signals “i.a” and “m1/i1.a”.

For net: “i.a”

| Attribute           | RTL View | Synthesized View |
|---------------------|----------|------------------|
| name                | i.a      | \i.a             |
| object_type         | net      | net              |
| pre_synthesis_name  | i.a      | i.a              |
| post_synthesis_name | \i.a     | \i.a             |
| root_bundle_name    | i        | i                |

For pin: “m1/i1.a”

| Attribute           | RTL View | Synthesized View |
|---------------------|----------|------------------|
| name                | m1/i1.a  | m1/i1.a          |
| object_type         | pin      | pin              |
| pre_synthesis_name  | m1/i1.a  | m1/i1.a          |
| post_synthesis_name | m1/i1.a  | m1/i1.a          |

| Attribute        | RTL View | Synthesized View |
|------------------|----------|------------------|
| root_bundle_name | m1/i1    | m1/i1.a          |

#### Example 4

The values of the name attributes are language independent and have the same values for both VHDL and Verilog RTL objects. The following example in VHDL uses the record datatype:

```
library ieee;
package record_pkg is

    -- Outputs from the FIFO.
    type t_FROM_FIFO is record
        wr_full  : std_logic;           -- FIFO Full Flag
        rd_empty : std_logic;           -- FIFO Empty Flag
        rd_dv    : std_logic;
        rd_data   : std_logic_vector(7 downto 0);
    end record t_FROM_FIFO;
    .....
end package record_pkg;
use work.record_pkg.all; -- USING PACKAGE HERE!
entity top is
    port (
        i_clk  : in  std_logic;
        i_fifo : in  t_FROM_FIFO;
        ..... );
end top;

architecture behave of top is

begin
    .....
end behave;
```

The following table shows the different naming of the port “i\_fifo.rd\_data(5)” in VHDL. VHDL uses “()” to represent the index, but Tessent Shell uses “[]” instead.

| Attribute           | RTL View          | Synthesis View    |
|---------------------|-------------------|-------------------|
| name                | i_fifo.rd_data[5] | i_fifo[5]         |
| pre_synthesis_name  | i_fifo.rd_data[5] | i_fifo.rd_data[5] |
| post_synthesis_name | i_fifo[5]         | i_fifo[5]         |
| root_bundle_name    | i_fifo            | i_fifo            |

## Bundle Object Data Model

Bundle objects utilize the Tessent Shell Hierarchical Design Data Model object types.

For each object type, there are attributes associated with the design objects found on, in, and below the current design (set with the [set\\_current\\_design](#) Tessent Shell command). A bundle object has a select part and type information. Otherwise a bit-blasted object has a bit select and also type information. An object type can be a complex type in case of bundle object, like struct, enum, typedef, multi-dimensional array. In the case of a bit-blasted object it is always a 1 bit datatype. It can be register, wire, logic, bit, tri, for example. Several attributes are associated with the following Hierarchical Design Data Model object types:

- [Net](#)
- [Net Bundle](#)
- [Pin](#)
- [Pin Bundle](#)
- [Port](#)
- [Port Bundle](#)

Each of these object types is covered in detail in the [Tessent Shell Reference Manual](#).

## base\_class and base\_type Attributes

Each bundle object type contains a base\_class and base\_type attribute as follows:

- **base\_class** — String character that represents the class category of the base type. It helps to complete the base\_type definition. Possible values are: = 4\_state\_signed | 4\_state\_unsigned | 2\_state\_signed | 2\_state\_unsigned | enum | struct\_packed | struct\_unpacked | union\_backed | union\_unpacked | interface | interface\_modport | other.
- **base\_type** — String character that represents the base type of the selected object. The base type is the type of the object without any dimensions. It represents the base type of the sub bundle type if the selected object is a sub bundle object. It can be a base type of the root type if the selected object is a root bundle. The base\_type is a key or user name that help to understand the base type of this bundle. Possible values are: logic | register | wire | bit | supply0 | supply1 | tri | triand | trior | tri0 | tri1 | wand | wor | byte | shortint | integer | longint | *user\_name* | other

The following examples show the values for base\_class and base\_type bundle object type attributes in SystemVerilog:

**Table 3-4. base\_class and base\_type SystemVerilog Examples**

| Examples                                     | base_type and base_class Values                  |
|----------------------------------------------|--------------------------------------------------|
| Packed struct having typedef name “data_t”   | base_type = data_t, base_class = struct_packed   |
| Unpacked struct having typedef name “data_t” | base_type = data_t, base_class = struct_unpacked |

**Table 3-4. base\_class and base\_type SystemVerilog Examples (cont.)**

| Examples                                     | base_type and base_class Values                                                                 |
|----------------------------------------------|-------------------------------------------------------------------------------------------------|
| shortint                                     | base_type = shortint, base_class = 2_state_signed                                               |
| unsigned shortint                            | base_type = shortint, base_class = 2_state_unsigned                                             |
| integer                                      | base_type = integer, base_class = 4_state_signed                                                |
| int                                          | base_type = integer, base_class = 2_state_signed                                                |
| unsigned integer                             | base_type = integer, base_class = 4_state_unsigned                                              |
| unsigned int                                 | base_type = integer, base_class = 2_state_unsigned                                              |
| reg                                          | base_type = register, base_class = 4_state_unsigned                                             |
| logic                                        | base_type = logic, base_class = 4_state_unsigned                                                |
| bit                                          | base_type = logic, base_class = 2_state_unsigned                                                |
| supply0, supply1                             | base_type= supply0 or supply1,<br>base_class = 2_state_unsigned                                 |
| tri, wand, triand, wor, trior, tri0,<br>tri1 | base_type = tri, wand, triand....<br>For all those datatypes, the base_class = 4-state_unsigned |
| module top (INTF1 Bus)                       | bundle “Bus” has as base_type=INTF1,<br>base_class = interface                                  |
| module top (INTF1.secondary<br>Bus)          | bundle “Bus” has as base_type=INTF1.secondary,<br>base_class = interface_modport                |

## Net Bundle

The Net Bundle object type has net\_bundle datatype and attributes. For complete details, refer to [net\\_bundle](#) in the *Tessent Shell Reference Manual*.

## Pin Bundle

The Pin Bundle object type has pin\_bundle datatype and attributes. For complete details, refer to [pin\\_bundle](#) in the *Tessent Shell Reference Manual*.

## Port Bundle

The Port Bundle object type has port\_bundle datatype and attributes. For complete details, refer to [port\\_bundle](#) in the *Tessent Shell Reference Manual*.

## ICL Data Model

The ICL data model is a separate data model that reflects the design logic. The ICL data model defines objects as one of the following four data types: icl\_module, icl\_instance, icl\_port, and icl\_pin.

When using bundle objects, the naming style may be different between the ICL data model and the design logic data model. For this reason, bundle object-specific attributes are required to bind the ICL objects (`icl_module` and `icl_port`) to design logic (module and port).

### `icl_module`

The following `icl_module` built-in attribute applies to design objects:

- **`tessent_design_module`** — The attribute reflects the name of the design module that corresponds to the ICL module at the time of ICL extraction.

### `icl_port`

The following `icl_port` built-in attribute applies to design objects:

- **`tessent_design_gate_ports`** — The attribute reflects the `post_synthesis_name` of the design port corresponding to the ICL port at the time of ICL extraction.
- **`tessent_design_rtl_ports`** — An attribute that specifies the `pre_synthesis_name` of the design port corresponding to the ICL port at the time of ICL extraction.

## Introspection

Using bundle object types, Tessent Shell enables you to introspect and explore any kind of signal within the design data model.

Both simple datatypes and complex datatypes can be explored using the introspection commands listed in “[Complex Signal Introspection Commands](#)” on page 71.

Any signal objects (ports, pins, and nets) are considered as a tree of sub signals. The given tree is referred to as a Bundle Object Hierarchy. The root bundle is the signal; the leaf objects are the bit-blasted elements of the given signal; the root, leaf, and middle bundle are all nodes of the tree. The middle bundles and the leaf objects are the sub bundle objects. To support this concept there are object types in the Hierarchical Design Data Model. The leaf objects are represented by `net`, `pin`, and `port` object types, the root and middle bundles are represented by `net_bundle`, `pin_bundle`, and `port_bundle` objects types. See “[Bundle Object Data Model](#)” on page 64.

The concept of bundle object type is applied to complex and simple signals for RTL and no RTL flow. The difference between simple and complex signals is in the number of levels that can be explored through the introspection commands. The select part of the signals helps to explore the Bundle Object Hierarchy level by level.

---

### Tip



When performing introspection of complex signals, see also “[Object Specification Format](#)” on page 50 and “[Collections](#)” on page 50.

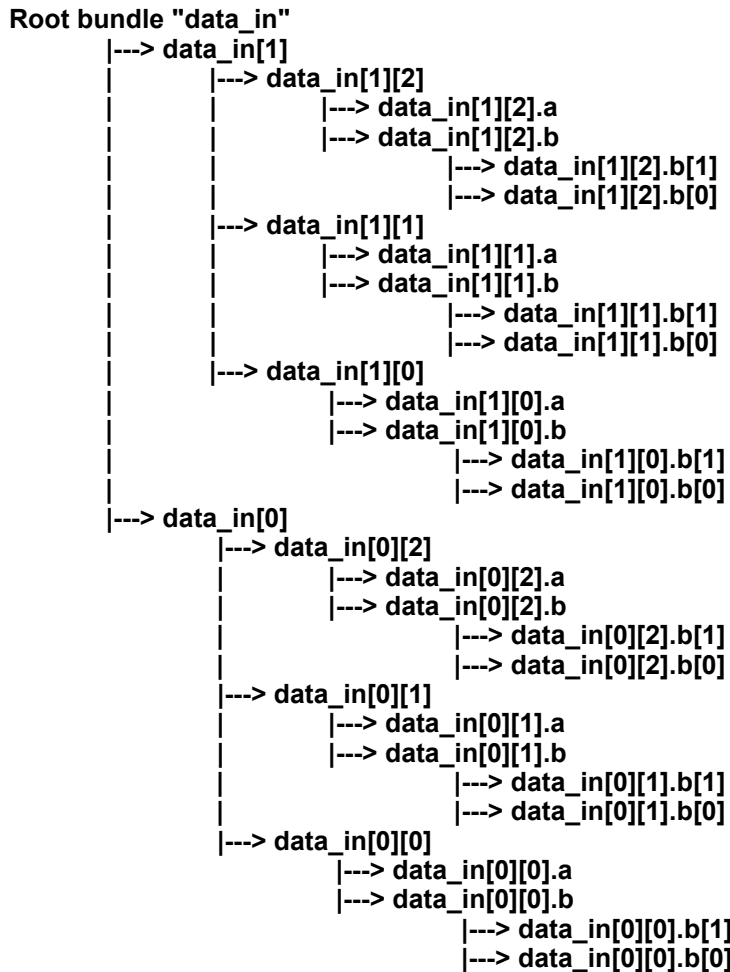
---

## How Complex Signal Introspection Works

Consider the bundle hierarchy of the “data\_in” port as follows:

```
typedef struct{  
    logic a;  
    logic [1:0] b;  
} data_t;  
input data_t [1:0] [2:0] data_in;
```

The following shows the hierarchy of the “data\_in” port in the Tessent Shell design data model.



Referring to the figure, using the *name\_patterns* argument of the introspection commands, the tool splits each pattern into the different levels so that a local sub-pattern is applied to each local level.

For the bundle part, the characters “.” and “[<index>]” enable access to sub bundle hierarchy, “.” to access field sub bundle objects of struct or SVinterface and “[<index>]” to access array element sub bundle objects. For example, the pattern “data\_in[0][1].b[\*]” is really five sub-patterns; “data\_in”, “[0]”, “[1]”, “.b”, and “[\*]”, representing five levels of bundle hierarchy.



For simple wildcard patterns, the '\*' is used the exact same way as it is used in instance path matching. It does not cross hierarchy. When specified within a field name it does not cross the "." that precedes or follows the field. When applied within [ ], it only matches elements within the [ ]. In other words delimiters must be explicit.

For regular expression patterns, an individual regular expression only matches elements within the sub-pattern in which they are found. Again, it does not cross the adjacent delimiters.

Without the -regexp switch in the introspection command, the tool treats the bundle hierarchy delimiters ("[]" and ".") as normal delimiters and serves to break the pattern into multiple sub-patterns. When you include the -regexp switch in the introspection command, the tool treats the bundle hierarchy delimiters as regular expression meta-characters (unless you have escaped those delimiters). That is, in regexp mode the delimiters must be explicit and, unlike simple wildcard patterns, you must escape them.

For example, to match "data\_in[1]", you need to use "data\_in\[.\*\]". The sequence '.\*' matches any character zero or more times. For more details, see the [Example With regexp](#).

See "[Glob and Regular Expression Pattern Matching Syntax](#)" in the *Tessent Shell Reference Manual* for complete information on pattern matching.

---

**Note**

The examples use the "get\_ports ... -bundle" command. For "get\_nets ... -bundle" and "get\_pins ... -bundle", the behavior is the same.

---

## Examples

These examples use the "data\_in" hierarchy described in the preceding.

```
SETUP> get_ports data_* -bundle
```

```
data_in
```

```
SETUP> get_ports data_in* -bundle
```

```
data_in
```

```
SETUP> get_ports data_in[*] -bundle
```

```
{data_in[1]} {data_in[0]}
```

```
SETUP> get_ports data_in[0][*] -bundle
```

```
{data_in[0][2]} {data_in[0][1]} {data_in[0][0]}
```

```
SETUP> get_ports data_in[0] -bundle
```

```
{data_in[0]}
```

```
SETUP> get_ports data_in[0][*].b -bundle
```

```
{data_in[0][2].b} {data_in[0][1].b} {data_in[0][0].b}
```

```
SETUP> get_ports data_in[1][2].* -bundle
```

```
{data_in[1][2].a} {data_in[1][2].b}
```

```
SETUP> get_ports data_in[1][*] -bundle
```

```
{data_in[1][2]} {data_in[1][1]} {data_in[1][0]}
```

---

### Note



Integer, enum, and others are bit-blasted in unbundled objects in introspection commands when the `-bundle` switch is omitted.

---

### Example With regexp

In the following example the second command call is equivalent to the first, but uses `-regexp`. You should enclose the regexp in double braces (`{{ }}`) to avoid having to use the `“\\”` everywhere, that is, `“{{data_in\[.*\]\.da.*\[.*\]\..*}}”`.

With wildcard syntax:

```
SETUP> puts [join [get_name_list [get_ports data_in[*].da*[*].* ]] "\n"]
```

```
data_in[1].data[1].data1[1][1]
data_in[1].data[1].data1[1][0]
data_in[1].data[1].data1[0][1]
data_in[1].data[1].data1[0][0]
data_in[1].data[1].data2[1][2]
...
data_in[0].data[0].data2[0][2]
data_in[0].data[0].data2[0][1]
data_in[0].data[0].data2[0][0]
```

With regular expression syntax:

```
SETUP> puts [join [get_name_list [get_ports {{data_in\[.*\]\.da.*\[.*\]\..*}} -regexp]] "\n"]
```

```
data_in[1].data[1].data1[1][1]
data_in[1].data[1].data1[1][0]
data_in[1].data[1].data1[0][1]
data_in[1].data[1].data1[0][0]
data_in[1].data[1].data2[1][2]
...
data_in[0].data[0].data2[0][2]
data_in[0].data[0].data2[0][1]
data_in[0].data[0].data2[0][0]
```

## Complex Signal Introspection Commands

The following commands support introspection of complex signals:

**Table 3-5. Complex Signals Design Introspection Commands**

| Command Name                          | Description/Usage                                                                                                                                                                                                                                                                                                            |
|---------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">get_bundle_objects</a>    | Returns a collection of bundle/leaf objects in the current design specified by the name_patterns list in conjunction with the specified options. Those bundle objects (leaf in case of descendants) are ancestors or descendants of the objects that match the specified name_patterns depending upon the specified options. |
| <a href="#">get_nets ... -bundle</a>  | When the -bundle switch is specified, constrains the tool to return net bundle/leaf objects. When the pattern matches a bundle, the command returns the bundle. When it matches leaf objects, it returns those leaf objects.                                                                                                 |
| <a href="#">get_pins ... -bundle</a>  | When the -bundle switch is specified, constrains the tool to return pin bundle/leaf objects. When the pattern matches a bundle, the command returns the bundle. When it matches leaf objects, it returns those leaf objects.                                                                                                 |
| <a href="#">get_ports ... -bundle</a> | When the -bundle switch is specified, constrains the tool to return port bundle/leaf objects. When the pattern matches a bundle, the command returns the bundle. When it matches leaf objects, it returns those leaf objects.                                                                                                |

## Bundle Object Introspection Examples

The following SystemVerilog code shows several kinds of datatypes: struct, array of interfaces, enumeration.

```
typedef enum logic [1:0] {
    STATE_1, STATE_2, STATE_3, STATE_4, STATE_5, STATE_6
} State_t;

typedef struct packed {
    logic [2:0] eventbus;
} event_bus_packet_t;

typedef struct packed {
    State_t state_encoded;
    event_bus_packet_t dt_lcp_pulse;
    logic [3:0] eventbus_encoded;
} event_bus_split_packet_t;

interface MSBus;
    event_bus_split_packet_t Addr;
    logic [1:0] Data;
    logic RWn;
    logic Clk;
    modport Secondary (input Addr, RWn, Clk, output Data);
endinterface

module top(MSBus.Secondary Bus[2], input wire i_clk);
    RAM TheRAM (.MemBus(Bus));
endmodule
```

## Example 1

The following command returns all bit-blasted ports of “top” module (default behavior):

**SETUP> get\_ports -of\_modules top**

```
{Bus[0].Addr.state_encoded[2]} {Bus[0].Addr.state_encoded[1]}
{Bus[0].Addr.state_encoded[0]} {Bus[0].Addr.dt_lcp_pulse.eventbus[2]}
{Bus[0].Addr.dt_lcp_pulse.eventbus[1]}
{Bus[0].Addr.dt_lcp_pulse.eventbus[0]} {Bus[0].Addr.eventbus_encoded[3]}
{Bus[0].Addr.eventbus_encoded[2]} {Bus[0].Addr.eventbus_encoded[1]}
{Bus[0].Addr.eventbus_encoded[0]} {Bus[0].RWn} {Bus[0].Clk}
{Bus[0].Data[1]} {Bus[0].Data[0]}
{Bus[1].Addr.state_encoded[2]} {Bus[1].Addr.state_encoded[1]}
{Bus[1].Addr.state_encoded[0]} {Bus[1].Addr.dt_lcp_pulse.eventbus[2]}
{Bus[1].Addr.dt_lcp_pulse.eventbus[1]}
{Bus[1].Addr.dt_lcp_pulse.eventbus[0]} {Bus[1].Addr.eventbus_encoded[3]}
{Bus[1].Addr.eventbus_encoded[2]} {Bus[1].Addr.eventbus_encoded[1]}
{Bus[1].Addr.eventbus_encoded[0]} {Bus[1].RWn} {Bus[1].Clk}
{Bus[1].Data[1]} {Bus[1].Data[0]}
```

## Example 2

The following commands return all root bundle ports of “top” module or sub bundle objects according to name patterns used:

**SETUP> set b [get\_ports -of\_modules top -bundle]**

Bus i\_clk

**SETUP> get\_ports Bus -bundle**

Bus

**SETUP> get\_ports B\* -bundle**

Bus

**SETUP> get\_ports Bus\* -bundle**

Bus

**SETUP> get\_ports Bus[0].\* -bundle**

{Bus [0] .Addr} {Bus [0] .RWr} {Bus [0] .Clk} {Bus [0] .Data}

**SETUP> get\_ports Bus[0].Addr\* -bundle**

{Bus [0] .Addr}

**SETUP> get\_ports Bus[0].Addr.\* -bundle**

{Bus [0] .Addr .state\_encoded} {Bus [0] .Addr .dt\_lcp\_pulse .eventbus}  
{Bus [0] .Addr .eventbus\_encoded}

**SETUP> get\_ports Bus[0].Addr.\***

{Bus [0] .Addr .state\_encoded[2]} {Bus [0] .Addr .state\_encoded[1]}  
{Bus [0] .Addr .state\_encoded[0]} {Bus [0] .Addr .dt\_lcp\_pulse .eventbus [2]}  
{Bus [0] .Addr .dt\_lcp\_pulse .eventbus [1]}  
{Bus [0] .Addr .dt\_lcp\_pulse .eventbus [0]} {Bus [0] .Addr .eventbus\_encoded [3]}  
{Bus [0] .Addr .eventbus\_encoded [2]} {Bus [0] .Addr .eventbus\_encoded [1]}  
{Bus [0] .Addr .eventbus\_encoded [0]}

**SETUP> get\_ports Bus[\*] -bundle**

{Bus [0]} {Bus [1]}

**SETUP> get\_ports Bus[\*].\* -bundle**

{Bus [0] .Addr} {Bus [0] .RWr} {Bus [0] .Clk} {Bus [0] .Data} {Bus [1] .Addr}  
{Bus [1] .RWr} {Bus [1] .Clk} {Bus [1] .Data}

**SETUP> get\_ports Bus[0]**

{Bus [0] .Addr .state\_encoded[2]} {Bus [0] .Addr .state\_encoded[1]}  
{Bus [0] .Addr .state\_encoded[0]} {Bus [0] .Addr .dt\_lcp\_pulse .eventbus [2]}  
{Bus [0] .Addr .dt\_lcp\_pulse .eventbus [1]}  
{Bus [0] .Addr .dt\_lcp\_pulse .eventbus [0]} {Bus [0] .Addr .eventbus\_encoded [3]}  
{Bus [0] .Addr .eventbus\_encoded [2]} {Bus [0] .Addr .eventbus\_encoded [1]}  
{Bus [0] .Addr .eventbus\_encoded [0]} {Bus [0] .RWr} {Bus [0] .Clk}  
{Bus [0] .Data [1]} {Bus [0] .Data [0]}

**SETUP> set b1 [get\_bundle\_objects \$b -children]**

{Bus [0]} {Bus [1]}

**SETUP> set b2 [get\_bundle\_objects [index\_collection \$b1 0] -children]**

{Bus [0] .Addr} {Bus [0] .RWr} {Bus [0] .Clk} {Bus [0] .Data}

```
SETUP> get_bundle_objects -of_objects {Bus[0].Addr} -root
```

```
{Bus}
```

```
SETUP> get_bundle_objects -of_objects {Bus[0].Addr} -leaf
```

```
Bus[0].Addr.state_encoded[2] } {Bus[0].Addr.state_encoded[1] }  
{Bus[0].Addr.state_encoded[0] } {Bus[0].Addr.dt_lcp_pulse.eventbus[2] }  
{Bus[0].Addr.dt_lcp_pulse.eventbus[1] }  
{Bus[0].Addr.dt_lcp_pulse.eventbus[0] } {Bus[0].Addr.eventbus_encoded[3] }  
{Bus[0].Addr.eventbus_encoded[2] } {Bus[0].Addr.eventbus_encoded[1] }  
{Bus[0].Addr.eventbus_encoded[0] }
```

```
SETUP> set b3 [get_bundle_objects [index_collection $b2 0] -children]
```

```
{Bus[0].Addr.state_encoded} {Bus[0].Addr.dt_lcp_pulse}  
{Bus[0].Addr.eventbus_encoded}
```

```
SETUP> get_attribute_value_list [$b3] -name base_class
```

```
enum struct_packed 4-state_unsigned
```

```
SETUP> get_attribute_value_list [$b3] -name select_part_value_list
```

```
{0 Addr state_encoded} {0 Addr dt_lcp_pulse} {0 Addr eventbus_encoded}
```

```
SETUP> get_ports {Bus[0].Clk} -bundle
```

```
{Bus[0].Clk}
```

```
SETUP> get_ports {Bus[0].Addr.state_encoded}
```

```
{Bus[0].Addr.state_encoded[2] } {Bus[0].Addr.state_encoded[1] }  
{Bus[0].Addr.state_encoded[0] }
```

```
SETUP> get_bundle_objects -of_objects {Bus[0].Addr.state_encoded} -parent
```

```
{Bus[0].Addr}
```

```
SETUP> get_bundle_objects -of_objects {Bus[0].Addr} -parent
```

```
{Bus[0]}
```

```
SETUP> get_bundle_objects -of_objects {Bus[0]} -parent
```

```
{Bus}
```

```
SETUP> get_bundle_objects -of_objects {Bus} -parent
```

```
{}
```

## Fan-in and Fanout Tracing of Complex Signals and Bundle Object Tracing

Using Tessent Shell, you can introspect and trace any kind of signals (simple and complex) through their fan-ins and fanouts. It is also possible to trace through bundle objects.

The `get_fanins`/`get_fanouts` commands accept any kind of objects as input, bundle or not, and with simple types or complex types. The ending and intermediate nodes can also be an object or complex type. For example, they can trace from and to, and through any kind of complex signals in SystemVerilog (SV) and VHDL.

- [get\\_fanins](#)
- [get\\_fanouts](#)

The following shows a top-level RTL module (*top1.sv*) with SV Interface objects. The tool models SV Interfaces as a named bundle of wires. Within the tool, those are considered as normal bundle ports, pins, and nets. You access the bundle object using the [get\\_nets](#), [get\\_pins](#), and [get\\_ports](#) commands.

```
module top (clk, reset, ce, we, addr, datai, datao, cnto, datao2, en);

    localparam WID = 8;
    localparam AWID = 5;
    localparam LEN = 2**AWID;
    localparam GEN = 3;

    input en;
    input clk, reset, ce, we;
    input [AWID-1:0] addr;
    input [WID-1:0] datai;
    output [GEN*12-1:0] datao, datao2;
    output [GEN*WID-1:0] cnto;

    and enAnd2 (resetEn, reset, en);
    and enAnd3 (weEn, we, en);
    and enAnd4 (ceEn, ce, en);

    param_mem_if miff(
        .clk(clk),
        .reset(resetEn),
        .we(weEn),
        .ce(ceEn),
        .dati(datai),
        .addr(addr),
        .dato(datao)
    );

    genvar i;

    generate
        for (i=0; i<GEN; i++) begin: gen_mem
            SYNC_1RW_16x8 mem1 (.CLK(clk), .D(datai), .Q(datao2[(WID*(i+1)
                -1):i*WID]), .WE(we), .OE(ce), .RE(ce), .A(addr[3:0]));
        end
    endgenerate

    generate
        for (i=0; i<GEN; i++) begin: gen_mem_wrapper

            param_mem mem_inst (.mif(miff));
        end
    endgenerate

    generate
        for (i=0; i<GEN; i++) begin: gen_cnt
            param_counter #(WID(WID)) counter_inst (
                .clk(clk),
                .reset(reset),
                .enable(ce&we),
                .count(cnto[(WID*(i+1)-1):i*WID]));
        end
    endgenerate
endmodule
```



The following shows an example command sequence to introspect the fan-ins and fanouts in this design:

```
SETUP> set_context dft -rtl
SETUP> read_cell_library my_cell_library.lib
SETUP> set_design_sources -format verilog -y {verilog_source_directory} -ext {v}
SETUP> set_design_sources -format tcd_memory -y {verilog_source_directory} \
-ext {lvlib lib}
SETUP> read_verilog param_mem.sv -format sv2009
SETUP> read_verilog param_counter.sv -format sv2009
SETUP> read_verilog top1.sv -format sv2009
SETUP> set_current_design top
SETUP> set_design_level physical_block
SETUP> add_input_constraints en -C1
SETUP> puts [join [get_attribute_value_list \
[get_fanins gen_mem_wrapper[0].mem_inst/mem1/CLK ] -name name] "\n"]

clk
```

Here, from SV Interface sub bundle pins, the tool reaches the output pin of primitive. (Continuing from the previous example.)

```
SETUP> puts [join [get_name_list \
[get_fanouts gen_mem_wrapper[2].mem_inst/mif.ce ]] "\n"]

enAnd4/OUT

SETUP> puts [join [get_name_list \
[get_fanouts gen_mem_wrapper[2].mem_inst/mif.ce ] -name name] "\n"]

gen_mem_wrapper[2].mem_inst/mem1/OE
gen_mem_wrapper[2].mem_inst/mem1/RE
gen_mem_wrapper[2].mem_inst/mem2/OE
gen_mem_wrapper[2].mem_inst/mem2/RE
```

Here, from SV Interface sub bundle pins, the tool reaches the SV interface sub bundle net “miff.ce”. (Continuing with the previous example.)

```
SETUP> puts [join [get_name_list \
[get_fanins gen_mem_wrapper[2].mem_inst/mif.ce -stop_on net] \
-name name] "\n"]

miff.ce

SETUP> puts [join [get_name_list \
[get_fanouts gen_mem_wrapper[2].mem_inst/mif.ce -stop_on net] -name name] "\n"]

gen_mem_wrapper[2].mem_inst/mif.ce
```

## Additional Examples

The following command sequence shows introspection of bundle objects through both fan-ins and fanouts, as well as ports and pins:

```
SETUP> set bitblasted_port [get_ports jack]
SETUP> set bitblasted_conn [get_fanouts $bitblasted_port]
SETUP> puts [join [get_name_list $bitblasted_conn] "\n"]
```

```
i1/jim[2]
i2/jim[1]
i1/jim[1]
i1/joe[2]
i1/jim[0]
i2/joe[2]
```

```
SETUP> set bundle_port [get_ports jack -bundle]
SETUP> set bundle_conn [get_fanouts $bundle_port]
SETUP> puts [join [get_name_list $bundle_conn] "\n"]
```

```
i1/jim
```

```
SETUP> set bitblasted_pin [get_pins i1/jim]
SETUP> set bitblasted_conn [get_fanins $bitblasted_pin]
SETUP> puts [join [get_name_list $bitblasted_conn] "\n"]
```

```
jack[2]
jack[1]
jack[0]
```

```
SETUP> set bundle_pin [get_pins i1/jim -bundle]
SETUP> set conn [get_fanins $bundle_pin]
SETUP> puts [join [get_name_list $bundle_conn] "\n"]
```

```
jack
```

## Connectivity Through Complex Signals

Tessent Shell traces and reports on the connectivity of complex signals defined at RTL without synthesizing every module.

The [get\\_fanins/get\\_fanouts](#) commands can report the connectivity through complex signals such as RTL pins and nets on the hierarchical design view. The [trace\\_flat\\_model](#) command can report on a flat model.

The tool performs quick synthesis on the module only if it finds RTL logic that represents logic gates while performing path pre-tracing. The tool maintains the RTL view and the connectivity to and from the surrounding modules, even if a module with a SystemVerilog interface requires quick synthesis. This limits the number of RTL modules synthesized to perform a pre-DFT DRC for MemoryBIST and to perform ICL extraction.

When Tessent Shell flattens the design, only the leaf objects of complex signals are translated to gate pins using the post synthesis names (see the `post_synthesis_name` attribute on the [Port](#), [port\\_bundle](#), [Pin](#), [pin\\_bundle](#), [Net](#), and [net\\_bundle](#) object types for more information).

For objects found inside unsynthesized modules, the hierarchical design objects uses the pre-synthesis names (see the `pre_synthesis_name` attribute on the [Port](#), [port\\_bundle](#), [Pin](#), [pin\\_bundle](#), [Net](#), and [net\\_bundle](#) object types for more information).

However, the tool always uses the post-synthesis name of the leaf objects of complex signals for gate pin objects in the flat model, whether it synthesizes the modules or not.

The tool provides several switches on the introspection commands to simplify the introspection between the hierarchical design objects and the flat model gate\_pin objects. With complex signals, you cannot assume the two objects have the same name.

- [get\\_gate\\_pins](#) *-of\_pins pin\_objects | -of\_ports port\_objects | -of\_nets net\_objects*
- [get\\_nets](#) *[name\_patterns] -of\_gate\_pins gate\_pin\_objects*
- [get\\_pins](#) *[name\_patterns] -of\_gate\_pins gate\_pin\_objects*
- [get\\_ports](#) *[name\_patterns] -of\_gate\_pins gate\_pin\_objects*

The following example demonstrates loading an RTL design and performing tracing on the hierarchical data module using the [get\\_fanins](#) command. It also shows creating and tracing the flat model using the [trace\\_flat\\_model](#) command:

```
SETUP> set_context dft -rtl
SETUP> read_cell_library my_cell_library.lib
SETUP> open_tsdb InternalCore_tsdb
SETUP> read_design InternalCore -design_id rtl
SETUP> read_verilog RTL_Directory/Interfaces.sv -format sv2009
SETUP> read_verilog RTL_Directory/Core.sv -format sv2009
SETUP> set_current_design core
SETUP> set_design_level physical_block
SETUP> puts [get_name_list [get_fanins genloop[2].icw0/ic/OutBus[1]]]

{ InBus [1] }

SETUP> create_flat_model
// Flattening process completed, gates=79, Pls=12, POs=13, CPU time=0.00 sec.
// -----
// Begin circuit learning analyses.
// -----
// Learning completed, CPU time=0.00 sec.

SETUP> puts [get_name_list [trace_flat_model -from [get_gate_pins -of_pins \
    [get_pins genloop[2].icw0/ic/OutBus[1]]] -direction backward \
    -controllability connected]]

{ InBus [1] }
```

This example shows the use of the “[get\\_gate\\_pins](#) -of\_pin” and “[get\\_pins](#) -of\_gate\_pins” to illustrate that the matching objects do not have the same name:

```
SETUP> get_gate_pins -of_pins [get_pins genloop[2].icw0/ic/OutBus[1]]
```

```
{ {\genloop[2].icw0 /ic/OutBus[1] } }
```

```
SETUP> get_gate_pins -of_pins genloop[2].icw0/ic/OutBus[1]
```

```
{ {\genloop[2].icw0 /ic/OutBus[1] } }
```

```
SETUP> get_pins -of_gate_pins { {\genloop[2].icw0 /ic/OutBus[1]} }
```

```
{ genloop[2].icw0/ic/OutBus[1] }
```

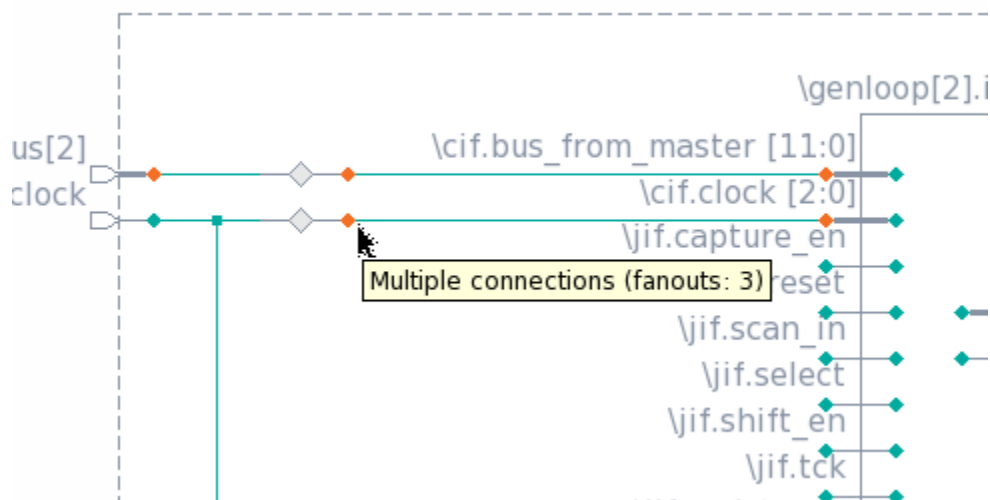
```
SETUP> get_pins -of_gate_pins [get_gate_pins { {\genloop[2].icw0 /ic/OutBus[1]} }]
```

```
{ genloop[2].icw0/ic/OutBus[1] }
```

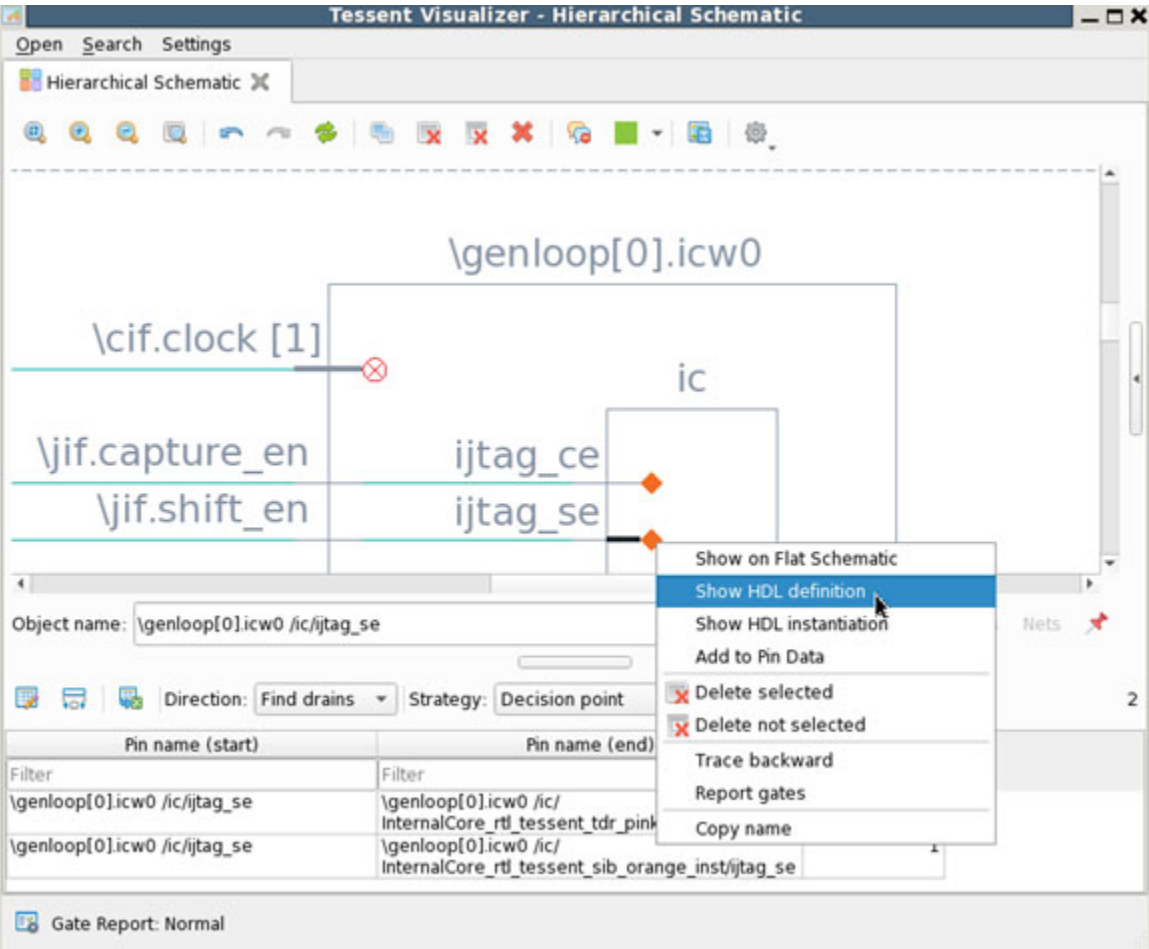
## Tessent Visualizer Tracing and Visualization

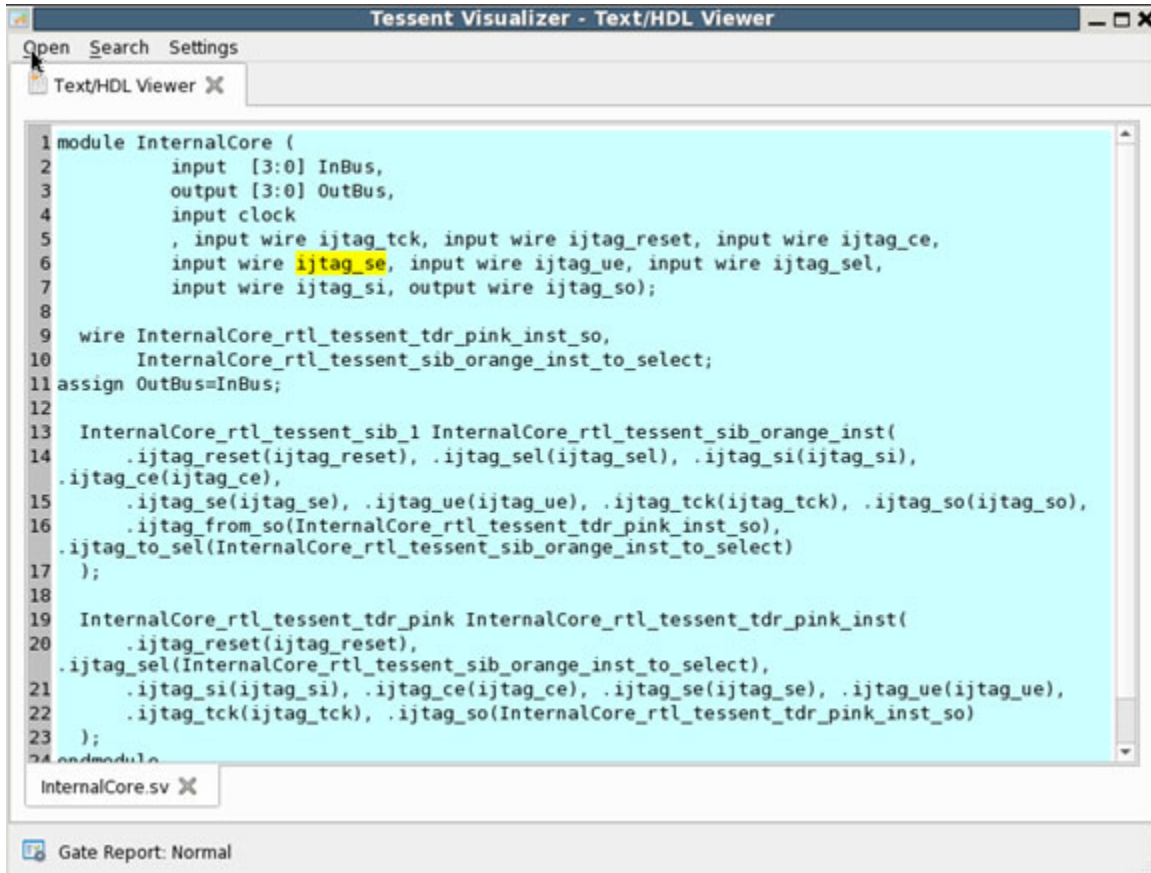
Tessent Visualizer provides a platform to trace and visualize bundle objects in the flat model. The Tessent Visualizer GUI shows the `post_synthesis_name` of the ports and pins even when the RTL view of the module is active. When you switch to the synthesized view of a module, its footprint remains unchanged in Tessent Visualizer. The RTL logic that is not shown in the RTL view becomes logic cell instances in the synthesized view.

**Figure 3-2. Complex Signals with Tessent Visualizer**



Tessent Visualizer facilitates cross-probing pins in the schematic to their associated RTL netlist.





For complete information on using Tessent Visualizer, see “[Tessent Visualizer](#)” on page 857.

# Design Editing

Tessent Shell design editing commands enable you to modify your design after reading in the RTL or gate-level netlist. Tessent Shell supports gate-level netlist editing or RTL design editing with full language support, including multiple logical libraries, VHDL, Verilog, and System Verilog. Parameterized modules are also fully supported.

The Design editing commands enable you to manipulate a design's modules, instances, nets, ports, and pins, either interactively or through Tcl scripting.

---

**Note**

 There are language restrictions. See “[HDL Limitations in the Tessent Shell Flow](#)” in the *Tessent Shell Reference Manual* for details.

---

Design editing commands work with collections and introspection commands, as well as native Tcl commands, to automate many tasks. [Table 3-6](#) presents the common design editing commands based on the function they perform—that is, whether they create, modify, or remove elements and so on. For more information on collections and introspection commands, see “[Design Introspection](#).”

---

**Caution**

 Take special care when dealing with non-unique design scopes, such as multiple instances of the same module or the inner part of a generate loop. Refer to “Example of Handling Non-Unique Design Scopes” within the “Design Editing Examples” topic listed below for details.

---

---

**Note**

 The design editing commands are not available when using some product licenses, such as Tessent FastScan and Tessent Scan.

---

**Table 3-6. Design Editing Commands**

| Commands that...                            | Command Name                                 |
|---------------------------------------------|----------------------------------------------|
| Read netlists and specify logical libraries | <a href="#">read_verilog</a>                 |
|                                             | <a href="#">read_vhdl</a>                    |
|                                             | <a href="#">set_logical_design_libraries</a> |

**Table 3-6. Design Editing Commands (cont.)**

| Commands that...           | Command Name                          |
|----------------------------|---------------------------------------|
| Create design elements     | <a href="#">create_connections</a>    |
|                            | <a href="#">create_instance</a>       |
|                            | <a href="#">create_module</a>         |
|                            | <a href="#">create_net</a>            |
|                            | <a href="#">create_pin</a>            |
|                            | <a href="#">create_port</a>           |
| Remove design elements     | <a href="#">delete_connections</a>    |
|                            | <a href="#">delete_instances</a>      |
|                            | <a href="#">delete_nets</a>           |
|                            | <a href="#">delete_pins</a>           |
|                            | <a href="#">delete_ports</a>          |
| Modify the design          | <a href="#">copy_module</a>           |
|                            | <a href="#">intercept_connection</a>  |
|                            | <a href="#">replace_instances</a>     |
|                            | <a href="#">uniquify_instances</a>    |
| Set or get editing options | <a href="#">get_insertion_option</a>  |
|                            | <a href="#">set_insertion_options</a> |

|                                             |           |
|---------------------------------------------|-----------|
| <b>Design Editing Command Summary .....</b> | <b>84</b> |
| <b>Editing Complex Signals .....</b>        | <b>86</b> |
| <b>Design Editing Examples.....</b>         | <b>88</b> |

## Design Editing Command Summary

The design editing commands enable you to work with modules, connections, instances, nets, pins, and ports.

The commands listed in the following table are commonly used design editing commands. Refer to the *Tessent Shell Reference Manual* for more information.

**Table 3-7. Design Editing Command Summary**

| Command Name                | Description                                                                                                                                                                           |
|-----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">copy_module</a> | Creates an exact copy of a design module and gives it a new name, which the tool can use as part of <a href="#">create_instance</a> and <a href="#">replace_instances</a> operations. |



**Table 3-7. Design Editing Command Summary (cont.)**

| Command Name                          | Description                                                                                                                                                                     |
|---------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">create_connections</a>    | Creates a connection between pin, net, or port objects.                                                                                                                         |
| <a href="#">create_instance</a>       | Instantiates a module (design or cell type) inside of a design module that is part of the current design.                                                                       |
| <a href="#">create_module</a>         | Creates a new design module.                                                                                                                                                    |
| <a href="#">create_net</a>            | Creates a net inside an instance of a design module.                                                                                                                            |
| <a href="#">create_pin</a>            | Creates a pin on an instance of a design module.                                                                                                                                |
| <a href="#">create_port</a>           | Creates a port on a design module.                                                                                                                                              |
| <a href="#">delete_connections</a>    | Disconnects the specified pin objects.                                                                                                                                          |
| <a href="#">delete_instances</a>      | Removes an instance of a module.                                                                                                                                                |
| <a href="#">delete_nets</a>           | Removes net objects inside an instance of a design module.                                                                                                                      |
| <a href="#">delete_pins</a>           | Removes pin objects on an instance of a design module.                                                                                                                          |
| <a href="#">delete_ports</a>          | Removes port objects on a design module.                                                                                                                                        |
| <a href="#">get_insertion_option</a>  | Introspects the default values of options affecting many design editing commands.                                                                                               |
| <a href="#">intercept_connection</a>  | Using the <code>get_dft_cell</code> command, obtains a cell with the specified function name and uses it to intercept a connection to a pin, port, or net.                      |
| <a href="#">move_connections</a>      | Moves a net connected on one pin or port to another pin or port. The first pin is left open after the move.                                                                     |
| <a href="#">rename_instance</a>       | Renames the leaf name of an instance object.                                                                                                                                    |
| <a href="#">replace_instances</a>     | Replaces the module object used in an instantiation.                                                                                                                            |
| <a href="#">set_insertion_options</a> | Specifies default values of options affecting many design editing commands.                                                                                                     |
| <a href="#">uniquify_instances</a>    | If the module of the specified instance has other instantiations in the design, the tool copies the module and the specified instance becomes an instance of the copied module. |

## Editing Complex Signals

Tessent Shell enables you to edit complex ports and pins with the DftSpecification wrapper.

|                                                                        |           |
|------------------------------------------------------------------------|-----------|
| <b>Editing Complex Signals With the DftSpecification Wrapper .....</b> | <b>86</b> |
| <b>Editing Complex Signals: Limitations and Compatibility .....</b>    | <b>87</b> |

## Editing Complex Signals With the DftSpecification Wrapper

Specify the complete RTL names of complex ports and pins using the DftSpecification wrapper.

### Overview

Create the DftSpecification wrapper object with the create\_dft\_specification command and customize it with configuration data editing commands. After you have finished modifying the DftSpecification, perform DFT insertion with the process\_dft\_specification command. You can make connections between pins and ports with complex types to other pins and ports with complex types, move these connections, and delete these connections.

Load the DftSpecification either before or after quick synthesis. You can use the full names of complex ports and pins regardless of whether the current views are quick-synthesized.

### Examples

When processing a DftSpecification on the RTL view of a design, you can use bundle ports and pins to define the DFT signals:

```
typedef struct packed {
    logic [3:0] in;
    logic [1:0] out;
} EDT_t;

input EDT_t edt_channel;

EdtChannelsIn(8:5) {
    port_int_name : edt_channel.in;
}
```

When processing a DftSpecification on the quick-synthesized view of a design, you can specify port and pin RTL names as bit-blasted:

```
EdtChannelsIn(8:5) {
    port_pin_name : edt_channel.in[3], edt_channel.in[2], \
    edt_channel.in[1], edt_channel.in[0];
}
```

You can also specify port and pin RTL names as slices:

```
EdtChannelsIn(8:5) {  
    port_pin_name : edt_channel.in[3:0];  
}
```

## Related Topics

[Configuration-Based Specification](#)

[DftSpecification Configuration Syntax](#)

[create\\_dft\\_specification](#)

[process\\_dft\\_specification](#)

## Editing Complex Signals: Limitations and Compatibility

There are certain tool limitations you must be aware of when editing complex signal ports.

- Sub-bundle objects inferred from SystemVerilog interfaces are not editable. All SystemVerilog interface declarations encapsulating sequential or concurrent statements, or other nested SystemVerilog interfaces, are considered with logic.
- Enumerated types declared within modules or SystemVerilog interface design units, and also at the global scope, are not editable.
- Named port declarations in SystemVerilog interfaces used in modports or in interface ports lists are not supported.
- The “create\_connections -constant” command does not support the named constant values defined by enumerated types.
- Multidimensional SystemVerilog interfaces are not supported for introspection or editing.
- The tool does not support editing or introspection of named port connections in a SystemVerilog modport.
- The tool does not support editing of SystemVerilog Streaming Operators on instance pins.

- The tool does not support the creation of temporary root bundles during complex signal editing. For example:

```
module parent();
    typedef logic logic_t;
    intf #(logic_t) obj();
    leaf (... , obj);
endmodule

module leaf(intf port1);
    ...
endmodule

// If you disconnect the sub-bundle of port1, the tool creates
// a temporary root bundle and hence incorrect RTL.

module leaf(intf port1);
    // glue logic insertion starts.
    intf #(logic_t) temp_bundle();
    temp_bundle.sig1 = port1.sig1;
    Etc.
endmodule
```

## Related Topics

[get\\_bundle\\_objects](#)

[get\\_nets](#)

[get\\_pins](#)

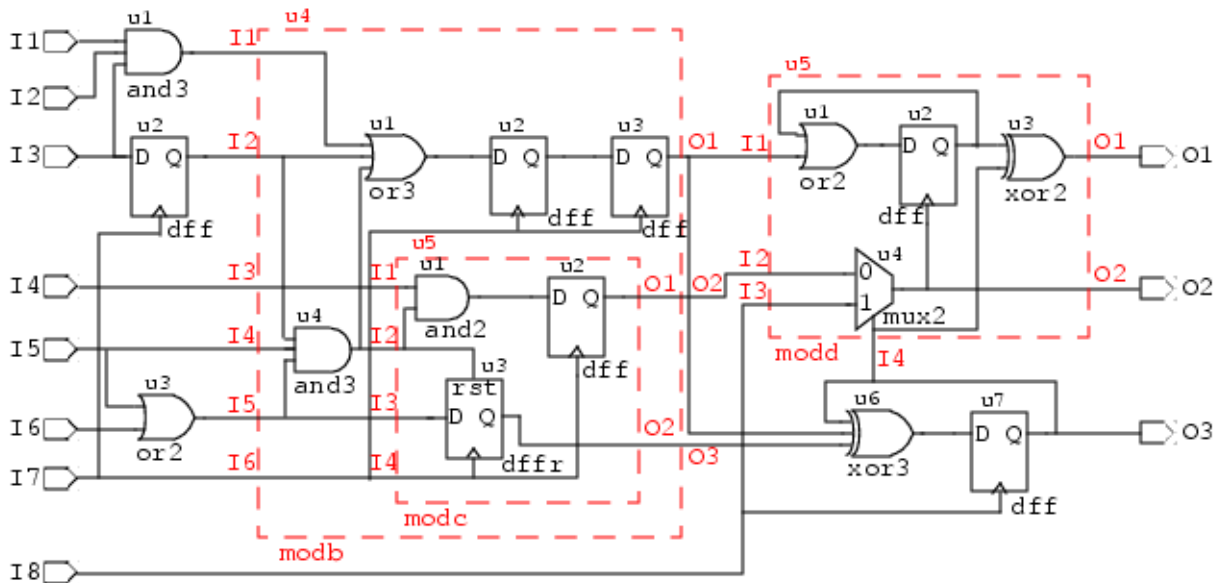
[get\\_ports](#)

## Design Editing Examples

There are many ways to create, modify, and remove elements from your design in a Tcl scripting environment.

[Figure 3-3](#) is an example design, which is followed by related examples that show some of the ways you can edit designs within Tessent Shell.

Figure 3-3. Hierarchical Design Example



### Example of Creating a Module From Scratch

The following example shows how to create module C (modc) in [Figure 3-3](#) as a standalone module.

```

set_context dft -no_rtl
set_system_mode insertion
create_module modc
set_current_design modc

create_port I1 -on_module modc -direction input
create_port I2 -on_module modc -direction input
create_port I3 -on_module modc -direction input
create_port I4 -on_module modc -direction input
create_port O1 -on_module modc -direction output
create_port O2 -on_module modc -direction output

create_instance u1 -of_module and2
create_instance u2 -of_module dff
create_instance u3 -of_module dffr

create_connection I1 u1/A0
create_connection I2 u1/A1
create_connection I2 u3/R
create_connection I3 u3/D
create_connection I4 u3/CLK
create_connection I4 u2/CLK
create_connection u1/Y u2/D
create_connection u2/Q O1
create_connection u3/Q O2

write_design -output_file MODC.v -replace

```

## Example of Replacing a Module With Another Module

The following example replaces the module definition for instance u5 (modd) to modc. The original module and the new module have the same pinout, and the connecting nets remain the same after issuing the `replace_instances` command. After replacing instance u5 with modc, that instance u5/u1 changes from an or2 gate to an and02 gate.

```
INSERTION> get_attribute_value_list u5 -name module_name
modd

INSERTION> get_fanin u5/I1 -stop_on net
{u4_O1}

INSERTION> replace_instances u5 -with_module modc
{u5}

INSERTION> get_fanin u5/I1 -stop_on net
{u4_O1}

INSERTION> set_system_mode setup
SETUP> set_design_level sub_block
SETUP> set_sys_mode analysis

// Flattening process completed, cell instances=15, gates=45, PIs=8
// POs=3, CPU time=0.00 sec.
// -----
// Begin circuit learning analyses.
// -----
// Learning completed, CPU time=0.00 sec.

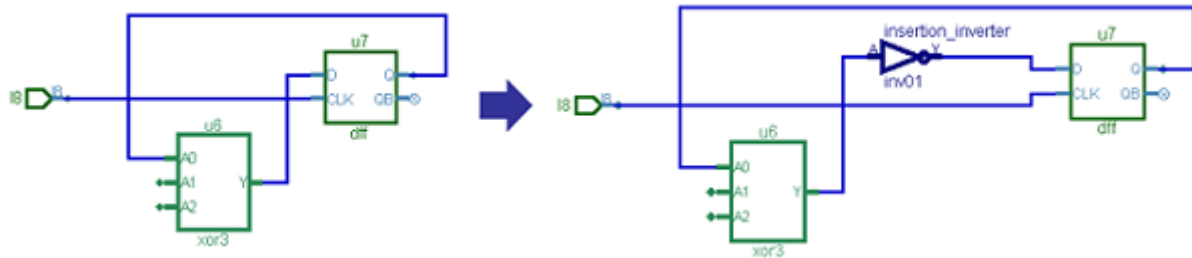
ANALYSIS> report_gates u5/u1

// /u5/u1 and02
// A0 I /u4/u3/Q
// A1 I /u4/u5/u2/Q
// Y O /u5/u2/D
```

## Example of Intercepting a Connection

Figure 3-4 shows an example of inserting an inverter by intercepting the existing connection between the u7/D pin and the u6/Y pin. After the interception, the inverter A pin is connected to the u6/Y pin and the inverter Y pin is connected to the u7/D pin.

Figure 3-4. Inverter Inception



```
INSERTION> intercept_connection u7/D -cell_function_name inverter  
{insertion_inverter}
```

### Example of Adding Input and Output Pads to a Design

The following dofile example adds input and output pads to a design by creating a collection of port names from the design, creating a collection of pad names that match the ports, then connecting the ports and pads together in the design. The example also adds the TAP's I/O ports, including the respective pads.

```
# Setting the context to netlist editing
set_context dft -no_rtl

# Reading the library and all necessary Verilog files
read_cell_library ../libs/libs/adk.atpg
read_verilog      ./counter_block2_edt_top_gate.v
read_verilog      ../libs/pads/*.v

# Telling the tool, which module 'top' is
set_top_module "counter_block2"
set_current_design $top_module

# Now for the editing
set_system_mode insertion

# First, insert the pad cells for all inputs of the netlist
set inputPortList [ get_ports -of_module $top_module -direction input ]
foreach_in_collection inputPort $inputPortList {

    # Creating the name of the pad cell:
    #   The following lines take care of '[' and ']' in the portname,
    #   substituting these by '_'. This makes the Tcl-life easier
    set portName [lindex [get_name_list $inputPort] 0]
    set padName   [ string map { \[ _ \] _ } ${portName}_PAD ]

    # Adding the pad cell
    create_instance $padName -of_module INPAD

# Connecting it up in the netlist
# The first line moves the existing net from the input port to the output
# of the pad cell. The second line creates a new net and connects the
# input port to the input of the pad cell
# Note: We are not connecting anything else here.
#       We leave this to boundary scan insertion
    move_connection -from $inputPort -to $padName/FP
    create_connection $inputPort $padName/IO
}

# Second, insert the pad cells for all outputs of the netlist
set outputPortList [ get_ports -of_module $top_module -direction output ]
foreach_in_collection outputPort $outputPortList {

    # Creating the name of the pad cell:
    #   The following lines take care of '[' and ']' in the portname,
    #   substituting these by '_'.
    set portName [lindex [ get_name_list $outputPort ] 0]
    set padName   [ string map { \[ _ \] _ } ${portName}_PAD ]

    # Adding the pad cell
    create_instance $padName -of_module OUTPAD_2S

# Connecting it up in the netlist
# The first line moves the existing net from the IO port to the input
# of the pad cell. The second line creates a new net and connects the
# IO port to the output of the pad cell.
# Note: We are not connecting anything else here.
#       We leave this to boundary scan insertion
    move_connection -from $outputPort -to $padName/TP
```



```
        create_connection $outputPort $padName/IO
    }

    # Next, add the JTAG IO pins and their respective PAD cell. Connecting #
    # them up.

    set inputPortList { TDI TMS TRST TCK }
    set outputPortList { TDO }

    foreach portName $inputPortList {

        # Adding the pad cell
        set padName      ${portName}_PAD
        create_instance $padName -of_module INPAD

        # Creating an Input IO pin and connecting the pad
        create_port $portName -direction input
        create_connection $portName $padName/IO
    }

    foreach portName $outputPortList {

        # Adding the pad cell
        set padName      ${portName}_PAD
        create_instance $padName -of_module OUTPAD_EN1

        # Creating an output pin and connecting the pad
        create_port $portName -direction output
        create_connection $portName $padName/IO
    }

    # Write the new netlist
    write_design -output_file "${top_module}.v" -replace
    exit
```

## Example of Handling Non-Unique Design Scopes

When you want to make edits to non-unique design scopes, such as multiple instances of the same module or the inner part of a generate loop, you must exercise caution.

When using the [delete\\_connections](#) or [move\\_connections](#) commands inside a non-uniquified design, only perform the move or disconnection once per module or once per generate loop count.

For an example, see how the `parent_instance` and the `leaf_name_hash` attributes are used in Example 5 of the [get\\_dft\\_cell](#) command description. (For more information on the `leaf_name_hash` attribute, see “[Instance](#)” in the *Tessent Shell Reference Manual*.)

## Simulation Contexts

Tessent Shell provides simulation contexts to aid your design analysis and introspection efforts. You can use these contexts to create “simulation scratch pads” for rapid investigation of good-machine behavior of specific portions of your design.

|                                                                  |           |
|------------------------------------------------------------------|-----------|
| <b>Simulation Context Overview.....</b>                          | <b>94</b> |
| <b>Introspection and Analysis Using Simulation Contexts.....</b> | <b>95</b> |

## Simulation Context Overview

Tessent Shell provides commands for creating and managing simulation contexts. From a particular user-defined simulation context, you can use these commands to apply stimulus forces to specified gate\_pins, run simulation for a specific number of cycles, and then introspect gate\_pins for simulation values.

### Commands for Managing Simulation Contexts

Use the following commands to create and manage user-defined simulation contexts, as well as use predefined simulation contexts:

- [add\\_simulation\\_context](#) — Creates a new user-defined simulation context.
- [copy\\_simulation\\_context](#) — Copies the simulation values and forces from one simulation context to another.
- [delete\\_simulation\\_contexts](#) — Deletes one or more user-defined simulation contexts.
- [get\\_current\\_simulation\\_context](#) — Returns the name of the current simulation context.
- [get\\_simulation\\_context\\_list](#) — Returns the available simulation contexts in a Tcl list.
- [report\\_simulation\\_contexts](#) — Lists the available simulation contexts and indicates the current simulation context.
- [set\\_current\\_simulation\\_context](#) — Sets the current simulation context.

### Commands for Managing Stimulus and Simulating within Simulation Contexts

The following commands are for managing stimulus (simulation forces and clock pulses) and running limited simulations within a simulation context:

- [add\\_simulation\\_forces](#) — Forces one or more gate\_pin objects to the specified value.
- [delete\\_simulation\\_forces](#) — Removes forces from one or more gate\_pin objects.
- [report\\_simulation\\_forces](#) — Lists the active forces on the specified gate\_pin objects.

- [simulate\\_clock\\_pulses](#) — Pulses one or more clocks within the current simulation context.
- [simulate\\_forces](#) — Simulates the queued forces in the current simulation context.

## Commands for Introspection and Analysis within Simulation Contexts

The following commands are for introspection and analysis within simulation contexts, which includes examining simulation results:

- [get\\_current\\_simulation\\_context](#) — Returns the name of the current simulation context.
- [get\\_simulation\\_context\\_list](#) — Returns the available simulation contexts in a Tcl list.
- [get\\_simulation\\_value\\_list](#) — Returns the simulation values on the specified gate\_pin objects.
- [report\\_simulation\\_contexts](#) — Lists the available simulation contexts and indicates the current simulation context.
- [report\\_simulation\\_forces](#) — Lists the active forces on the specified gate\_pin objects.
- [set\\_gate\\_report](#) — Specifies the information displayed by the report\_gates command. This command enables reporting of simulated values in the current simulation context.
- [trace\\_flat\\_model](#) — Traces the flat model within the current simulation context. This command always operates within the current simulation context.

## Attributes for gate\_pin Objects

For a complete list of attributes, refer to the “[Data Models](#)” chapter in the *Tessent Shell Reference Manual*.

# Introspection and Analysis Using Simulation Contexts

Simulation contexts enable you to perform various types of design analysis and introspection. The four predefined simulation contexts are: `stable_after_setup`, `stable_load_unload`, `stable_shift`, and `stable_capture`. After setting a simulation context, you can introspect gate\_pins for simulation values based on that specific simulation context. For example, the `trace_flat_model` command can do design tracing based on values specified for the current simulation context.

To use any of the following techniques in this chapter, you must be doing the following:

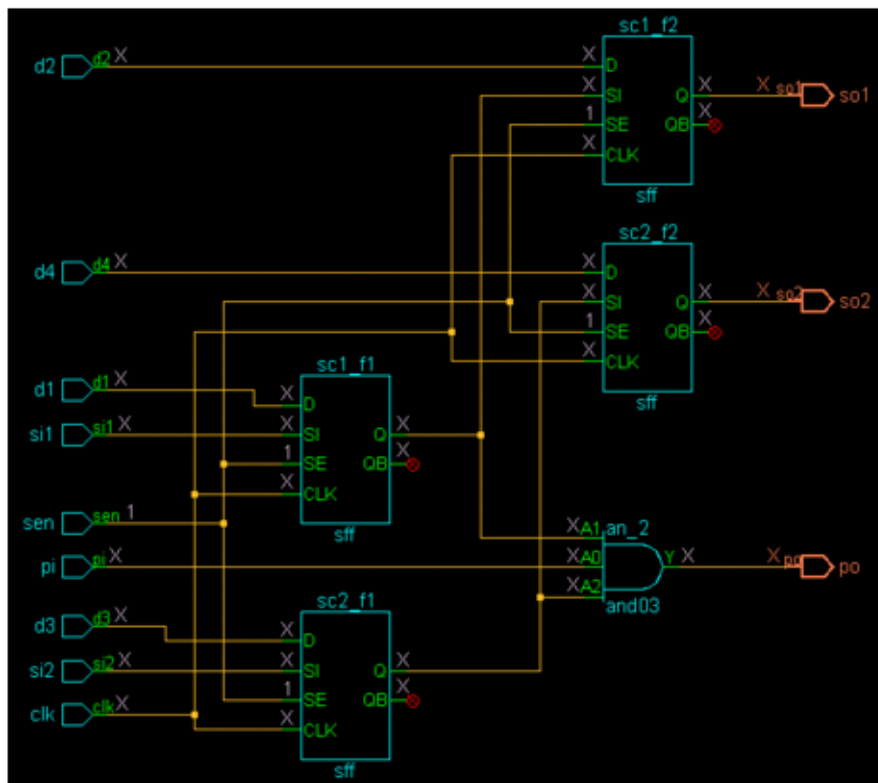
- Operating in the analysis system mode of Tessent Shell.

- Operating on a flattened design, and have read in the test procedure file (if available). At this point, the values specified in the setup, shift, capture, and load\_unload procedures are in place in the design when you set the corresponding simulation context as current.

## Example

For example, if you want to trace the design based on the values specified in the shift procedure, use the “set\_current\_simulation\_context stable\_shift” command. When you use this command with a small design containing two 2-cell scan chains, the values on the gate\_pins display in the Tessent Visualizer Flat Schematic as shown in [Figure 3-5](#).

**Figure 3-5. Tessent Visualizer Flat Schematic (gate level)**



In this case, no values are forced except for sen (scan enable), which is set to 1. The four possible simulation values are 0, 1, X, and Z.

In addition to viewing values in the Flat Schematic, you can get the current simulation values of gate\_pins using any of the following techniques:

- Use the get\_simulation\_value\_list command and specify the gate\_pin object(s):

```
ANALYSIS> set_current_simulation_context stable_shift
ANALYSIS> get_simulation_value_list sc2_f1/D
```

X

- Check the value of the simulation\_value attribute for the gate\_pins:

```
ANALYSIS> get_attribute_value_list [get_gate_pin sc2_f1/D]
-name simulation_value
```

```
X
```

- Issue the command “set\_gate\_report simulation\_context,” after which you can use the report\_gates command and the Flat Schematic to display the simulated values from the current simulation context:

```
ANALYSIS> set_gate_report simulation_context
ANALYSIS> report_gates sc2_f1
```

```
// /sc2_f1 sff
//      D      I  (X)  /d3
//      SI      I  (X)  /si2
//      SE      I  (1)  /sen
//      CLK     I  (X)  /clk
//      Q       O  (X)  /an_2/A2  /sc2_f2/SI
//      QB      O  (X)
```

You can use simulation context functionality for different types of analysis. For example, by default the trace\_flat\_model command uses the stable\_after\_setup values as a simulation background. You can now change it to stable\_capture, for example, if you want to trace based on sensitized paths during capture. Another example is to copy an existing simulation context to a new one, force some simulation value changes, then evaluate the results in the circuit after simulating the forces or clock pulses.

## Automatic Design Mapping

Logic synthesis may cause name and footprint changes in the design for both complex and regular ports.

|                                                                     |            |
|---------------------------------------------------------------------|------------|
| <b>ICL and TCD Post-Synthesis Update .....</b>                      | <b>98</b>  |
| <b>Updating ICL Attributes From the Design .....</b>                | <b>99</b>  |
| <b>Matching Rules for Port Names in Post-Synthesis Update .....</b> | <b>100</b> |
| <b>Controlling the Name Mapping .....</b>                           | <b>100</b> |

## ICL and TCD Post-Synthesis Update

Logic synthesis flattens (bit-blasts) ports. This changes the design footprint. Additionally, post-synthesis names differ from those in the initial RTL.

Tessent in `-no_rtl` mode uses the post-synthesis netlist when you run `set_current_design`. The ICL and TCD of the current design updates automatically to match the new design objects that are loaded from the post-synthesis netlist. The following ICL model attributes are updated to enable binding of the ICL objects and design objects later in the flow:

- `tessent_design_instance` (for the instance path)
- `tessent_design_gate_ports` (for ports)
- `tessent_design_rtl_ports` (for ports)

The `tessent_design_instance` attribute of the ICL instance object is automatically updated when you use any of the following commands:

- `set_current_design`
- `write_design`
- `update_icl_attributes_from_design`

The `tessent_design_gate_ports` attribute of the ICL port object is automatically updated when you use any of the following commands:

- `set_current_design`
- `get_ports -of_icl_ports`
- `get_pins -of_icl_pins`
- `get_pins -of_icl_ports`
- `update_icl_attributes_from_design`
- `write_design`

For ICL ports and instances, the ICL matching performed by `set_current_design` is insufficient if the current ICL model does not include the complete ICL of child instances under the physical blocks. In these cases, the ICL matching is completed later in the flow using the following commands:

- `get_ports -of_icl_ports`
- `get_pins -of_icl_pins`
- `get_pins -of_icl_ports`
- `update_icl_attributes_from_design`
- `write_design`

Use the `-use_path_matching_options` option with `get_pins`, `get_ports`, or `get_instances` to match a name pattern to the ports and pins in the current design when the TCD of instruments dumped during RTL mode is loaded in no RTL flow.

## Updating ICL Attributes From the Design

The `update_icl_attributes_from_design` command updates ICL port, instance, and module attributes for the gate views of a design. The update includes all ICL ports, instances, and modules in the current design.

The command updates the following attributes:

- **tessent\_design\_gate\_ports** — This attribute refers to the design port corresponding to the ICL port. The attribute value is the name of the design port as it appears in the netlist.
- **tessent\_design\_instance** — This attribute refers to the design instance corresponding to the ICL instance. The value is the hierarchical name relative to the parent ICL instance.
- **tessent\_design\_rtl\_gate\_port\_mapping** — This attribute maps the RTL name to the netlist name for those ports that appear in the following attribute name lists:
  - `forced_high_input_port_list`
  - `forced_low_input_port_list`
  - `forced_high_output_port_list`
  - `forced_low_output_port_list`
  - `forced_high_internal_input_port_list`
  - `forced_low_internal_input_port_list`
  - `tessent_ground_port_list`
  - `tessent_power_port_list`

- o `tessent_clock_domain_labels`

The attribute value is a list of names. The names at the even index positions in the list are the RTL names of the ports in the attribute name lists, and the names at the odd index positions are the netlist names of those ports.

The `update_icl_attributes_from_design` command has one Boolean option: `-verbose`. This option enables warnings when a port name cannot be found or matched in the netlist. The command is invoked automatically by the `"write_design -tsdb"` command. It also runs automatically as part of the `insert_test_logic` command just after insertion.

## Related Topics

[update\\_icl\\_attributes\\_from\\_design](#)

# Matching Rules for Port Names in Post-Synthesis Update

The following matching rules are supported when looking up a port name in a netlist:

1. Apply the transformation performed by the Synopsys Design Compiler® tool (`dc_shell`) with `"change_names -rules verilog"` to remove the escaping and replace special characters with an underscore (`"_"`). A single underscore matches multiple underscores.
2. Apply the transformation performed by Cadence® Genus™ Synthesis Solution to get the gate name from the RTL name. All strings and numerical indices in the RTL name are preserved. Only the delimiters are changed.
3. Apply the Genus transformation described in rule #2 but with escaping removed and special characters replaced with an underscore (`"_"`) as in rule #1.
4. Apply bit-blasted escaped names generated by a layout tool. The bit select is incorporated into the escaped name.
5. Apply end-user-specified matching rules.

## Controlling the Name Mapping

Use the `add_rtl_to_gates_mapping` and `delete_rtl_to_gates_mapping` commands to control the name mapping. Use the `report_rtl_to_gates_mapping` command to obtain information about the current mapping rules.

See the [Tessent Shell Reference Manual](#) for information on these commands.

## Default Matching Rules for the `get_pins` and `get_ports` Commands

- **Component Names** — All strings between non-escaped delimiters (component names) in the name to be looked up (lookup name) must match exactly to corresponding strings



in a netlist name unless they contain special characters (see below). The recognized delimiters are '.', '/', '[]', and '\_'.

An example lookup name:

```
abc.def
```

The 'abc' string in the name to be looked up must exactly match the string before the first delimiter in a netlist name. The 'def' string in the name to be looked up must exactly match the string after the last delimiter in the same netlist name.

- **Escaping** — Escape characters are not matched. They are only used to direct the matching procedure.
- **Delimiters** — Delimiters in the lookup name are used only to extract the component names.

All component names after the first are assumed to have a leading delimiter and optionally a trailing delimiter in a netlist name. There are two cases to consider when matching the delimiters for a component name in the netlist:

- a. Port name in netlist is escaped.

Possible matches for component name delimiters are as follows:

- Leading and trailing delimiters of [].
- Leading delimiter of '\_' and trailing delimiter of '\_' or null.
- Leading delimiter of '.' or '/' and trailing delimiter of null.

- b. Port name in netlist is not escaped.

Possible matches for component name delimiters are as follows:

- Leading delimiter of '\_' and trailing delimiter of '\_' or null.

- **Special Characters in the Lookup Name** — When matching to a non-escaped name in the netlist, any special characters (not allowed by the language without escaping) in the lookup name are replaced with the '\_' character. Matching allows truncation in the netlist name down to two trailing '\_' characters.

---

**Note**

This truncation rule applies only to '\_' characters derived from special characters, not delimiters.

---

- **Bit Select Lookup Name** — A bit select lookup name (last component name is an index) can match a one-dimensional vector with the final bit select applied to the match or match directly to a bit select.

## Related Topics

[add\\_rtl\\_to\\_gates\\_mapping](#)  
[delete\\_rtl\\_to\\_gates\\_mapping](#)  
[report\\_rtl\\_to\\_gates\\_mapping](#)

# ICL Objects Versus Design Objects Introspection

Several Tessent Shell commands enable you to introspect ICL objects and design objects in an ICL context.

For example, you can retrieve design modules and instances from ICL modules and instances:

- `get_module -of_icl_module icl_module_spec`
- `get_instance -of_icl_instance icl_instance_spec`

You can retrieve ICL modules and instances from design modules and instances:

- `get_icl_module -of_module module_spec`
- `get_icl_instance -of_instance instance_spec`

Certain additional command-line switches for these commands are necessary when your design includes complex ports, or simple ports with escaped names requiring name changes during logic synthesis.

And you can also retrieve design pins and ports from ICL pins and ports:

- `get_pins -of_icl_pins icl_pin_objects`
- `get_ports -of_icl_ports icl_port_objects`

You can also retrieve icl pins and ports from design pins and ports:

- `get_icl_pins -of_pins icl_pin_objects`
- `get_icl_ports -of_ports icl_port_objects`

## Related Topics

[get\\_modules](#)  
[get\\_instances](#)  
[get\\_icl\\_modules](#)  
[get\\_icl\\_instances](#)  
[get\\_pins](#)

[get\\_ports](#)  
[get\\_icl\\_pins](#)  
[get\\_icl\\_ports](#)



# Chapter 4

## DFT Architecture Guidelines for Hierarchical Designs

---

Most chips follow hierarchical place-and-route because of the increase in design sizes. For DFT, you can also use hierarchical design methodologies to perform test insertion, test generation, and diagnosis. This use of the hierarchical design methodologies is called hierarchical DFT or hierarchical test.

If the design is implemented as one chip with flat place-and-route, then perform DFT implementation once for the entire chip. Hierarchical DFT implementation is only applicable for block-based designs in which the tool run time consumption is not practical for implementing DFT only from the chip level.

Hierarchical DFT has several advantages:

- Divide and conquer technique that helps with parallelization of tasks.
- Reduced memory footprint and CPU requirements to perform the given task.
- Faster pattern generation and simulation run times.
- DFT performed earlier at block level and out of design critical path.

For information about the Tessent Shell two-pass DFT insertion flow for hierarchical designs, refer to “[Tessent Shell Flow for Hierarchical Designs](#)” on page 205.

|                                                                |            |
|----------------------------------------------------------------|------------|
| <b>Hierarchical DFT Overview .....</b>                         | <b>106</b> |
| <b>Top-Down Planning Before Bottom-Up Implementation .....</b> | <b>112</b> |
| <b>DFT Implementation Strategy .....</b>                       | <b>117</b> |

# Hierarchical DFT Overview

Hierarchical DFT is the process of using hierarchical design methodologies to perform test insertion, test generation, and diagnosis. Within the Tessent Shell environment, multiple components, such as physical layout regions and wrapped cores, are used to implement DFT in hierarchical designs.

|                                                           |            |
|-----------------------------------------------------------|------------|
| <b>Physical Layout Regions in Hierarchical Test .....</b> | <b>106</b> |
| <b>Pattern Retargeting .....</b>                          | <b>107</b> |
| <b>Wrapped Cores and Wrapper Cells.....</b>               | <b>107</b> |
| <b>Internal Mode and External Mode.....</b>               | <b>108</b> |
| <b>On-Chip Clock Controller .....</b>                     | <b>109</b> |
| <b>Graybox Model .....</b>                                | <b>110</b> |

## Physical Layout Regions in Hierarchical Test

Hierarchical designs typically have several physical layout regions, and these layout regions are instantiated into other physical layout regions. Usually, the instantiations occur at the chip level, so you have two levels of hierarchy (or sometimes more for some designs). From the DFT perspective, each layout region is where you perform test insertion, with the test insertion process performed independently for each region. You can implement DFT within the layout regions in parallel, thus improving throughput.

In some designs there may be more than two levels of physical hierarchy. From the DFT perspective, each layout region is where you perform test insertion, with the test insertion process performed independently for each region.

You must insert DFT into the lower-level designs before inserting it at the next higher level (at the chip level).

The following terms describe the layout regions of a hierarchical design:

- **Physical block, block, core, child core** — These terms are interchangeable and refer to layout regions below the chip level that you can instantiate within a chip, or across multiple chips. Blocks are logical entities that remain intact through tapeout. You perform synthesis on these blocks independent of the rest of the chip design.

The term “wrapped core” is a specialized type of block. Refer to “[Wrapped Cores and Wrapper Cells](#)” on page 107 for details.

- **Chip** — The chip is the top-level physical block — that is, the entire design — in which you typically find the pad I/O macros and clock controllers.

The chip and upper-level blocks in designs with more than two hierarchical levels are often referred to as the parent blocks and the tasks related to them as occurring at the parent level.

## Pattern Retargeting

Pattern retargeting is the process by which Tessent Shell preserves the ATPG patterns associated with cores for purposes of reuse when testing the logic at the chip (or parent) level. You do not have to regenerate the patterns when you process the chip. Instead, you retarget the core patterns to the chip level. Every instantiation of a core includes its associated ATPG patterns.

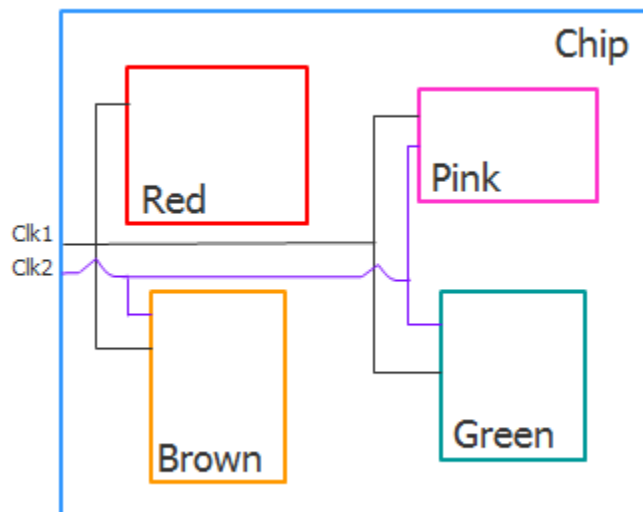
The pattern retargeting process consists of generating patterns for all cores, retargeting the core-level test patterns to the chip level, and generating patterns for the top-level chip logic only. This is also referred to as ATPG pattern retargeting or scan pattern retargeting. For details, refer to “[Scan Pattern Retargeting](#)” in the *Tessent Scan and ATPG User’s Manual*.

## Wrapped Cores and Wrapper Cells

In hierarchical DFT, you must wrap physical blocks to make them reusable through pattern retargeting. The term *wrapped cores* applies to these physical blocks. The purpose of wrapping a core is to isolate its internal logic.

In the following figure, the physical layout regions from a DFT perspective are physical blocks. Tessent performs test insertion on the Red, Pink, Brown, and Green physical blocks first. Clk1 and Clk2 are asynchronous clocks.

Figure 4-1. Wrapped Cores



The wrapping mechanism that you use for these blocks makes use of the functional flops that are already present at the boundary of these blocks. These flops are called shared wrapper cells (or sometimes wrapper cells) because they share the responsibility of their original functional use as I/O flops with isolating the core.

Optionally, you can add dedicated wrapper cells. These wrapper cells are added on primary inputs or primary outputs of a physical block that fan out or fan into large logic cones. By adding the dedicated wrapper cell, you can test the logic cone during internal testing of the core.

Shared and dedicated wrapper cells form the wrapper chains. Logic located within the wrapper chains is tested during internal testing of the core, called *intest*. Logic located outside the wrapper chains is tested during the external testing of the core, called *extest*.

## Internal Mode and External Mode

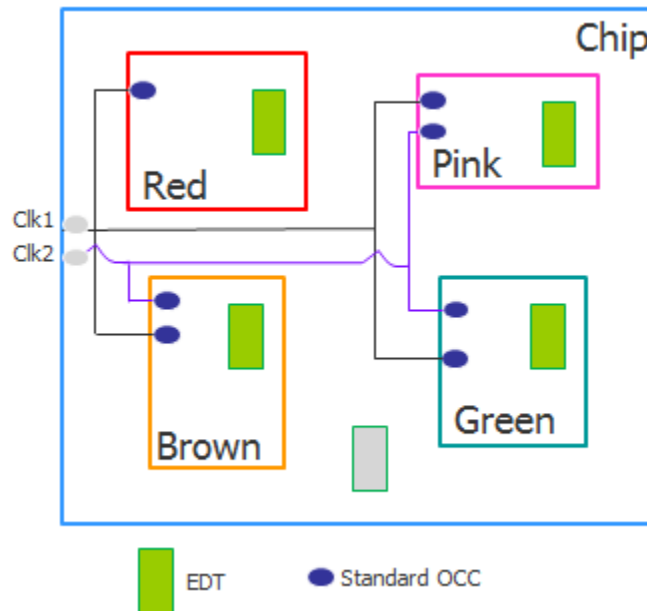
For purposes of pattern retargeting, Tessent Shell differentiates between a wrapped core's internal circuitry and its external circuitry.

Internal mode is the view into the wrapped core from the wrapper cells. That is, the logic completely internal to the core. Tessent Shell retargets internal mode ATPG patterns during ATPG pattern generation for the chip-level design.

External mode indicates the view out of the wrapped core from the wrapper cells. That is, the logic that connects the wrapped core to external logic. Tessent Shell uses the external mode to build graybox models, which are used by the internal modes of their parent physical blocks.

As shown in the following figure, based on chip pin availability, you can test the Red, Pink, Brown, and Green cores in internal mode at the same time when the on-chip clock controllers (OCCs) and EDT logic blocks inside the wrapped cores are active.

**Figure 4-2. Internal Mode**

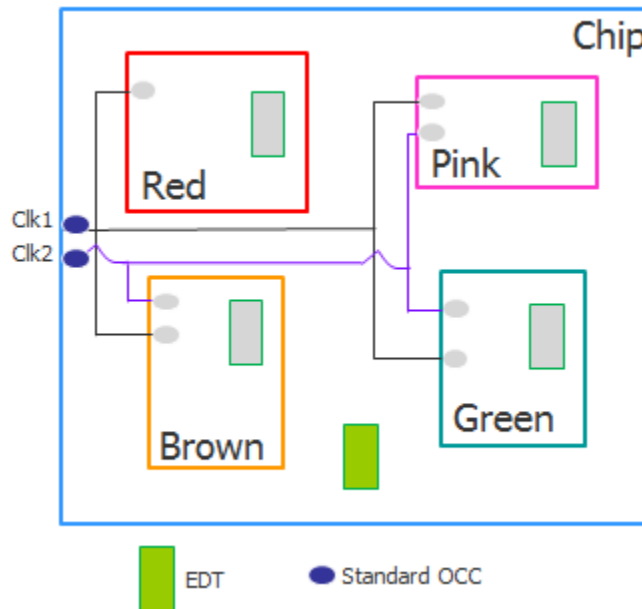




You can test one or more wrapped cores in internal mode based on several factors, such as availability of chip-level pins, tester channels, tester memory, and power dissipation. The OCCs and EDT compression logic at the chip level are inactive during internal mode.

In external mode, the wrapped cores at the given physical hierarchy participate. The OCCs and EDT logic blocks inside the cores are inactive, and the OCCs and EDT logic blocks outside the wrapped cores are active. In a typical external mode configuration, one EDT logic block is used to test the logic outside the wrapper chains, as shown in the following figure.

**Figure 4-3. External Mode**



External mode also includes the wrapper chains within the wrapped cores. Tessent tests the logic outside of the wrapper chains during external mode.

## On-Chip Clock Controller

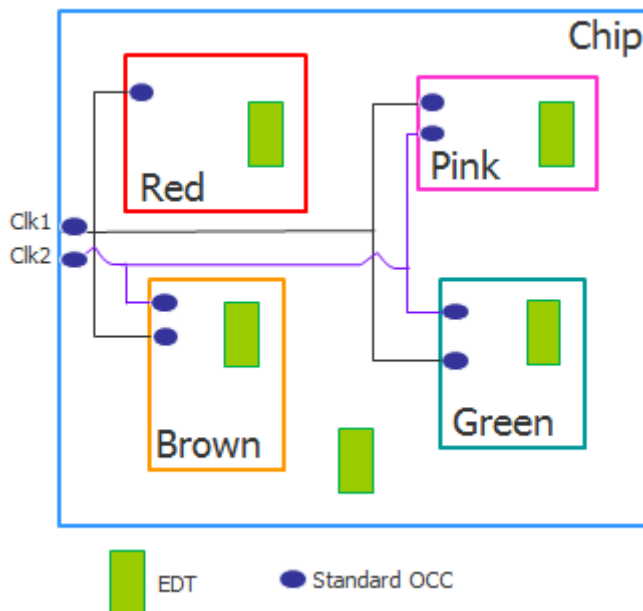
During hierarchical test, the existence of wrapped cores, whose patterns you want to retarget to the chip level, requires a mechanism to enable local, self-contained clock control. This mechanism is the on-chip clock controller (OCC), which you insert into each wrapped core. OCCs inside the wrapped cores make the ATPG patterns self-contained, which enables them to be retargeted.

The OCC is initialized for a clock waveform, and then the applicable patterns are retargeted to the parent- or chip-level design, as applicable. Block-level patterns with block-level OCCs can be retargeted and merged with other block patterns at the parent- or chip-level design independent of the clock waveforms.

Each wrapped core can contain one or more EDT logic blocks that contain the compression logic. Similarly there may be several options for testing the external mode of the cores. In the

following figure, OCC insertion occurs for each clock domain entering the cores. At the chip level, OCCs are also required for the clock domains at this level. Insert one EDT logic block for each internal mode of the core and one EDT logic block for the core's external mode.

**Figure 4-4. On-Chip Clock Controller**



To help facilitate test setup and automation, use a 1149.1 TAP controller along with an IJTAG network. You may also need to insert a Test Access Mechanism (TAM) to guide how these wrapped cores may be tested.

Standard OCCs include built-in clock selection, clock-chopping, and clock-gating functionality. For more information about standard OCCs, as well as parent and child OCCs, refer “[Tessent On-Chip Clock Controller](#)” in the *Tessent Scan and ATPG User’s Manual*.

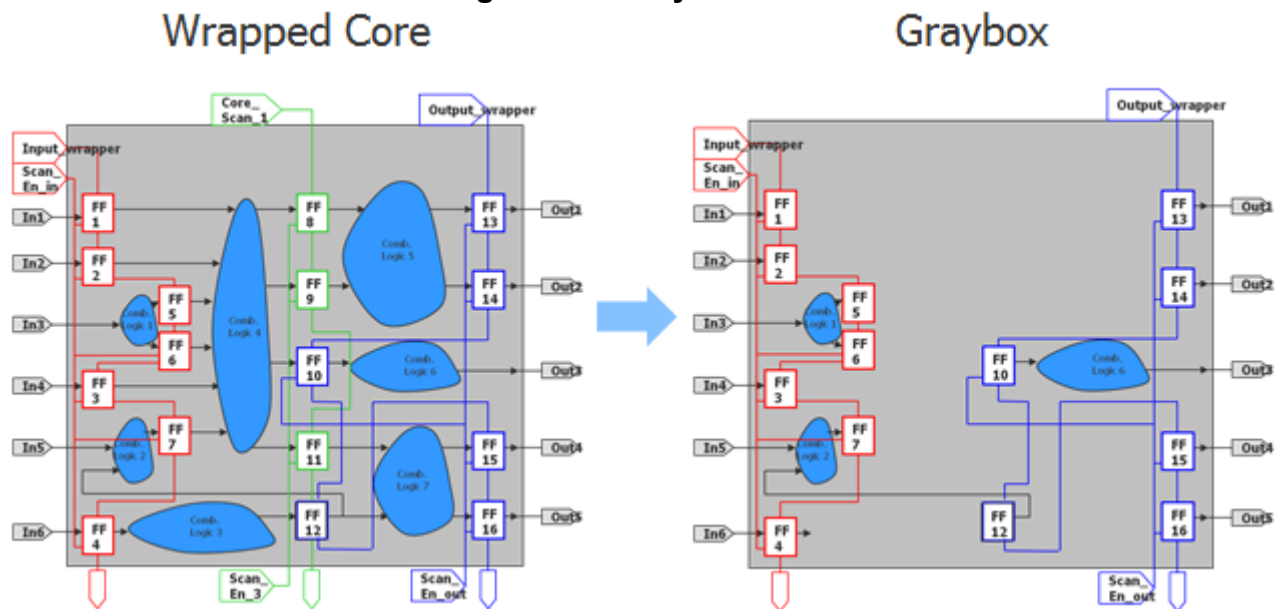
## Graybox Model

Graybox models are wrapped core models that preserve the core’s external mode logic, which includes the wrapper chains, along with the portion of the IJTAG network that needs to be tested along with the logic at the next physical level. The purpose of graybox models during hierarchical test is to retain the minimum logic required to generate ATPG patterns for the internal modes of the parent physical blocks. When testing the next physical level, using graybox models enables Tessent Shell to load the design faster than loading the full-chip design.

For details, refer to “[Graybox Overview](#)” in the *Tessent Scan and ATPG User’s Manual*.

The following figure shows a graybox model in which the internal logic of the wrapped core is removed. This graybox model implementation has separate input and output wrapper chains with separate scan enable signals for each wrapper chain type.

**Figure 4-5. Graybox Model**



The input and output wrapper chains do not have to be separate, and the scan enable signals also do not need to be separate. You can use one scan enable signal with other mechanisms to enable how the wrapper chains operate.

# Top-Down Planning Before Bottom-Up Implementation

To optimize your DFT implementation for hierarchical designs, top-down planning is necessary. Top-down planning enables you to properly budget and allocate the resources required for hierarchical test.

As described in “[Tessent Shell Flow for Hierarchical Designs](#)” on page 205, the process for inserting DFT into hierarchical designs is performed in a bottom-up process starting from the lowest level block. However, prior to insertion, it is best to plan the implementation in a top-down manner, starting at the chip level.

To address the design's challenges, consider the following factors:

- Optimizing pattern count to reduce test time
- Optimizing scan volume to fit within allocated tester memory
- Reducing chip pin requirements to fit within tester pin allocation

The planning process requires you to think about priorities such as test time, test quality, flow compatibility, and design schedule. During the planning stage consider the priorities and also the necessary tradeoffs to meet those priorities. In addition, be aware that how you implement DFT at the chip level impacts DFT implementation at the core level.

|                                                       |            |
|-------------------------------------------------------|------------|
| <b>Clocking Architecture</b> .....                    | <b>112</b> |
| <b>Resource Availability</b> .....                    | <b>113</b> |
| <b>Test Scheduling</b> .....                          | <b>113</b> |
| <b>Specific Tasks That May Require Planning</b> ..... | <b>116</b> |
| <b>Sample DFT Planning Steps</b> .....                | <b>117</b> |

## Clocking Architecture

In functional mode, the clock architecture can include clocks that are divided or multiplied. The original, divided, and multiplied clocks can be synchronous with each other within a core, and sometimes they can be synchronous across hierarchical cores. Divided clocks can also be asynchronous across hierarchical cores.

When generating ATPG patterns at the core level that you want to retarget, the core must include an OCC to control clocking as described in “[On-Chip Clock Controller](#)” on page 109. How you implement OCCs in the cores depends on various factors:

- **How the clocks are balanced** — The most common methods for clock balancing are clock tree synthesis and clock mesh synthesis. The method you use can influence what type of OCCs you insert (standard, parent, child).

- **OCC behavior during intest and extest** — How the OCCs need to behave during intest and extest of the cores also dictates how you should implement the OCCs.

Some OCCs are only required to perform clock chopping or clock-gating functionality. Sometimes OCCs may be necessary to mux clocks in for test purposes. You can use chip-level clocking to determine the architecture and OCC types (standard, parent, child) that you should use within the wrapped cores.

Refer to “[Clocking Architecture Examples](#)” on page 1049 for examples of clocking architectures with OCC.

## Resource Availability

Resource availability plays a significant role in how you implement DFT. For example, at the chip level, the number of pins available for test limits the number of EDT channel pins that you can use. The EDT channel pins may not be associated with only one EDT controller; they could be connected to multiple EDT controllers.

Likewise, the tester may have limited channel pins available for connecting to EDT channel pins on the chip, which means that you may need to consider the tester’s memory allocation for storing the test patterns. Available memory on the tester dictates whether you need to split the pattern set or test multiple cores in parallel.

You want to test the wrapped cores dictates how you create the test-access mechanism (TAM). TAMs carry scan data in and out of the chip for each group of wrapped cores you intend to run in parallel. You need a TAM if you have limited chip pins, and you are reusing those chip pins to connect to multiple EDT controllers inside the wrapped cores.

The TAM logic is dependent on how and when the cores get tested. The TAM schedules the tests for the wrapped cores at the top level, and it enables access to the chip level so that you can run these tests. The TAM also needs input from floor planning and placement of these cores to avoid routing congestion; you need to account for the proximity of the cores that you want to test in parallel and for the location of the chip pins that connect to them.

## Test Scheduling

Deciding which wrapped cores need to be tested at the same time on the tester—that is, in parallel—depends on a number of factors.

[Streaming Scan Network \(SSN\)](#) enables you to test the maximum number of cores within your power budget in a single pass. Because all cores accessible are through the SSN bus, you can delay the decision of which cores to run concurrently on the tester until ATPG or pattern retargeting. The SSN [ScanHost](#) drives the TestKompress EDT channels, decoupling them from top-level input and output pins.

SSN handles cores with different pattern counts by manipulating the network's bandwidth during retargeting and ATPG. The tool transfers network bandwidth from the cores with fewer patterns to the cores with more patterns. Through this bandwidth management, cores with different pattern counts finish on the tester closer together and have less padding than non-SSN patterns. Dynamic power is inherently reduced with SSN as each core independently transitions between shift and capture, significantly reducing the chance of a coincident clock edge during shift or capture.

SSN efficiently tests identical cores when you add the [On-Chip Compare](#) feature to the ScanHost. On-chip compare mode enables you to test any number of identical cores in constant time because it shares the same input, mask, and expected data across all the active cores.

For non-SSN designs, you may have to test the chip in pieces. The test scheduling for non-SSN designs depends on the following factors:

- Number of patterns
- Availability of chip pins
- Tester pin storage limitations
- Number of identical cores
- Cores with similar pattern counts
- Power dissipation budget and allocation
- Optimal compression at the core level
- Channel sharing of cores within modular EDT or sub-chip architectures; cores need to be tested where channels are shared

To achieve the optimal test schedule for a chip with a given number of hierarchical cores, generate an optimized compression for the internal mode of the cores. Within each core, compression could be based on asymmetric channel configurations. Designs with no X-sources require fewer output channels.

You can use the [analyze\\_compression](#) command to optimize compression. It requires gate-level scan stitched netlists. BIST logic used for memories needs to be scan stitched and included for the best compression estimation.

The following example shows a schedule based on compression analysis. Suppose 64 pins are available at the chip level as channel pins. There are twelve instances of blk\_a, eight instances of blk\_b, four instances of blk\_c, and one instance each of blk\_d, blk\_e, and blk\_f. The highlighted instances provide the best utilization of resources.

**Figure 4-6. Compression Analysis Example**

| <b>12 instances</b> |                |                 |              |            |                 |
|---------------------|----------------|-----------------|--------------|------------|-----------------|
| Block Name          | Input Channels | Output Channels | Shift Length | # Patterns | # Tester Cycles |
| blk_a               | 40             | 2               | 255          | 2584       | 658920          |
| blk_a               | 28             | 3               | 256          | 3044       | 779264          |
| blk_a               | 52             | 1               | 255          | 2692       | 686460          |
| blk_a               | 40             | 2               | 205          | 2505       | 513525          |
| blk_a               | 40             | 2               | 156          | 2409       | 375804          |
| <b>8 instances</b>  |                |                 |              |            |                 |
| Block Name          | Input Channels | Output Channels | Shift Length | # Patterns | # Tester Cycles |
| blk_b               | 56             | 1               | 256          | 1934       | 495104          |
| blk_b               | 48             | 2               | 256          | 1939       | 496384          |
| blk_b               | 40             | 3               | 256          | 1915       | 490240          |
| blk_b               | 32             | 4               | 256          | 1905       | 487680          |
| blk_b               | 32             | 4               | 202          | 1828       | 369256          |
| blk_b               | 32             | 4               | 154          | 1411       | 217294          |
| <b>4 instances</b>  |                |                 |              |            |                 |
| Block Name          | Input Channels | Output Channels | Shift Length | # Patterns | # Tester Cycles |
| blk_c               | 24             | 10              | 258          | 4475       | 1154550         |
| blk_c               | 16             | 12              | 261          | 7360       | 1920960         |
| blk_c               | 8              | 14              | 270          | 3895       | 1051650         |
| blk_c               | 12             | 13              | 264          | 4233       | 1117512         |
| <b>4 instances</b>  |                |                 |              |            |                 |
| Block Name          | Input Channels | Output Channels | Shift Length | # Patterns | # Tester Cycles |
| blk_d               | 28             | 28              | 263          | 2285       | 600955          |
| blk_d               | 36             | 20              | 260          | 1996       | 518960          |
| blk_d               | 40             | 16              | 258          | 2005       | 517290          |
| blk_d               | 38             | 18              | 259          | 1579       | 408961          |
| <b>4 instances</b>  |                |                 |              |            |                 |
| Block Name          | Input Channels | Output Channels | Shift Length | # Patterns | # Tester Cycles |
| blk_e               | 16             | 16              | 264          | 2233       | 589512          |
| blk_e               | 26             | 6               | 257          | 2818       | 724226          |
| blk_e               | 24             | 8               | 258          | 2112       | 544896          |
| blk_e               | 20             | 12              | 261          | 2112       | 551232          |
| blk_e               | 22             | 10              | 259          | 2033       | 526547          |
| <b>4 instances</b>  |                |                 |              |            |                 |
| Block Name          | Input Channels | Output Channels | Shift Length | # Patterns | # Tester Cycles |
| blk_f               | 16             | 16              | 264          | 2631       | 694584          |
| blk_f               | 22             | 10              | 259          | 2462       | 637658          |
| blk_f               | 26             | 6               | 257          | 2415       | 620655          |
| blk_f               | 28             | 4               | 257          | 2416       | 620912          |
| blk_f               | 31             | 1               | 255          | 2408       | 614040          |
| blk_f               | 30             | 2               | 256          | 1918       | 491008          |

Suppose you plan to group identical core instances for testing in parallel. The 40 input channel pins can broadcast to the blk\_a instances, providing the same input data to all the cores. Two output channels from each instance require a total of 24 additional pins. When testing the 12 blk\_a instances together, use the 64 available chip pins.

**Table 4-1. Test Schedule Example**

| Scan Config  | % Total Faults/Group | Input Channels | Output Channels | Pins Used/Config | Shift Length        | No. of Patterns | No. of Tester Cycles |
|--------------|----------------------|----------------|-----------------|------------------|---------------------|-----------------|----------------------|
| 12(blk_a)    | 68.59                | 40             | 24              | 64               | 156                 | 2409            | 375804               |
| 8(blk_b)     | 8.54                 | 32             | 32              | 64               | 154                 | 1411            | 217294               |
| 4(blk_c)     | 9.53                 | 8              | 56              | 64               | 270                 | 3895            | 1051650              |
| blk_f, blk_e | 7.33                 | 52             | 12              | 64               | 259                 | 2033            | 526547               |
| blk_d        | 6.01                 | 38             | 18              | 56               | 259                 | 1579            | 408961               |
| .            | .                    | .              | .               | .                | Total tester cycles |                 | 2646118              |

Next, test the eight instances of blk\_b together by applying the retargeted patterns at the chip level. The 32 input channel pins broadcast to the eight blk\_b instances. In addition, each instance uses four output channels for a total of 64 pins. After that, test the four blk\_c instances with eight input channels broadcast and 14 output channel pins per instance of blk\_c. Test the blk\_f and blk\_e instances together because they have similar complexity and channel requirements. Test the blk\_d instances that require the most channels by themselves.

Use the following guidelines when developing your test schedule:

1. Group identical core instances and run them at the same time.
2. Test cores of similar complexity and channel requirements together.
3. Test the cores that require the most channels by themselves.

## Specific Tasks That May Require Planning

Every design has its unique set of tasks that you need to perform during testing. Some tasks may involve planning while others may only require turning on or off features at the command line.

The following list provides examples of tasks that you may need to plan for ahead of time. This list is not exhaustive.

- Dual-mode configuration for Tessent TestKompress
- Low-power mode and threshold setup
- Multiple load ATPG patterns through the memories



- Test point provisioning for extra scan chains
- Support for multiple power islands and voltage islands
- Functional timing exceptions to be read in with SDC
- Low pin count test (LPCT) controllers
- Test setup sequence to enter test mode

## Sample DFT Planning Steps

There are some considerations when planning your DFT implementation.

### Note



This example may not be applicable to all designs.

1. Tally how many chip pins are available for scan channels and for ATE channels that are required for test. Use the smaller value when planning for the number of chip pins available for test.
2. Use a dedicated test clock apart from existing functional clocks. This clock can use the TCK signal as the clock source.
3. Assess the functional clock architecture for the chip and plan your OCC configuration. Consider how many OCCs you need, their OCC types (child, parent, standard), and where they need to be inserted for wrapped cores and the chip.
4. Based on the design size and pattern count, determine the number of channels required for each core.
5. Determine test scheduling for cores that you can run in parallel as described in “[Test Scheduling](#)” on page 113.
6. Determine the external mode configuration as described in “[DFT Implementation Strategy](#)” on page 117.
7. Confirm the top-level TAM that is required.

## DFT Implementation Strategy

For most designs, a few decisions can make a big difference to the overall DFT implementation architecture.

### Multiple Cores Testing in Internal and External Modes

You can test multiple cores in both internal and external modes. You must consider several factors when testing in these modes.

### Internal Mode

When you have multiple wrapped cores, fix the scan chain length at a constant value so that at the next hierarchical level when you combine cores to be tested in parallel, the shift length is identical.

If the OCC bits are roughly 25% of the longest chain, stitch OCC bits into a few dedicated compressed chains rather than distributing them across all of the chains. If the OCC bits add up to greater than 75% of the longest chain, then stitch the OCC bits as an uncompressed chain. If there are any remaining OCC bits, connect them into a compressed chain.

If there are embedded boundary scan cells present, ensure they are stitched with the rest of the boundary scan cells at the chip level.

### External Mode

The number of external mode chains from across multiple wrapped cores dictates how the chains should be handled, for example:

- Connect the external mode chains to one EDT controller.
- Connect the external mode chains to multiple EDT controllers.
- Connect the external mode chains directly to primary pins without any EDT compression logic. However, this can cause routing congestion.
- Connect the external mode chains when there is EDT compression logic. There may be EDT compression logic built inside the wrapped cores for the purpose of external mode, which leads to having at least two pins (one input and one output) for the compression logic to be used for external mode. This configuration eliminates routing congestion, but each EDT logic block is required to have two pins in the external mode of each core that is connected to the chip level.

If external mode chains from various cores are to be connected to one or more EDT controllers at chip level, then you must ensure that you balance the external mode chains so they are the same length. You can do this by using the chain length across multiple cores.

If there are multiple levels of hierarchy and the OCC needs to be active, you may need to include the OCC bits in the external mode. That is, if you want the OCC to be used in both internal and external modes, then you need to plan for the OCC bits to be included in the external chain also. Typically, OCCs are only included in internal mode.

During ATPG, you can include the boundary scan chain at chip level when running the external mode of the wrapped cores.

## Dedicated Test Clock

For hierarchical test, configure the test clock so that it does not share the functional clock port. The test clock needs an extra pulse to initialize the EDT hardware, which disturbs the scan cells and can result in D1 violations that cannot be fixed.

From the test clock, you can use DFT signals to generate the shift capture clock required for OCCs and the EDT clock required for EDT hardware.

Optionally, you can use TCK as your test clock. In this case, the TAP needs to run at the TCK rate, and the shift for ATPG is limited to the TCK frequency. Thus, you need to ensure that you are using an optimal TCK rate.

Using a dedicated test clock aids with timing closure and is beneficial for both internal test and external test because during shift when the scan enable signal is 1, the same clock path is chosen irrespective of whether the core is placed in internal or external mode of operation. For more information, refer to “Shift-Only Mode” under [Operating Modes](#) in the *Tessent Scan and ATPG User’s Manual*.

## Automated Features Within the Tessent Shell Flow

Tessent Shell provides automated features to help you implement your DFT strategy.

- **TSDB** — The Tessent Shell Data Base is a common database used by all the Tessent products, which means that test information generated by one tool is recognized and usable by downstream tools. For details, refer to “[TSDB Data Flow for the Tessent Shell Flow](#).”
- **IJTAG automation** — As described in “[What Is Tessent Shell?](#)” on page 27, IJTAG provides many benefits, including DFT setup reuse from blocks and sub-blocks to higher levels in the hierarchy. This is essential for using the bottom-up DFT insertion flow as described in “[Tessent Shell Flow for Hierarchical Designs](#)” on page 205.
- **Retargetable ATPG patterns** — Using OCCs inside hierarchical cores makes the ATPG patterns self-contained, which enables them to be retargeted as described in “[On-Chip Clock Controller](#)” on page 109.

---

### Note



Using the following Tessent automated features depends on your design implementation and how you are performing the two-pass RTL and scan DFT insertion flow for hierarchical designs as described in “[Tessent Shell Flow for Hierarchical Designs](#)” on page 205.

---

- **Reset control** — Tessent automatically fixes asynchronous resets with pre-DFT DRCs that are enabled when you set the `-logic_test` option of `set_dft_specification_requirements` to “on”. Tessent deactivates the reset during shift and tests the reset during capture. Optionally, provision is provided to override the reset

during capture by deactivating it. This auto-fix is beneficial when the functional reset is generated internally.

- **Boundary scan chain included for ATPG** — At the chip level, you can segment the boundary scan chain into smaller chains to be connected as compressed scan chains and controlled during ATPG. Optionally, you can configure these boundary scan chains to participate during capture or use them during shift.
- **Use memory to target shadow logic faults during logic test** — During logic test insertion, inserted memories get bypassed. The bypass can be built within the memories themselves or built within the MemoryBIST infrastructure. You can use IJTAG-controlled internal Test Data Registers (TDRs) to enable bypass or to use the memories during logic test. By using the memories during logic test, the shadow logic around the memories can be tested. This requires an ATPG model for the memories. For details, refer to the [Comprehensive RAM Primitive](#) information in the *Tessent Cell Library Manual*.
- **Unused scan chains** — During scan insertion and stitching if there are any over-specified scan chains, Tessent Scan automatically adds the unused scan chains with pipeline registers. This is beneficial when not all the scan chains of the inserted EDT compression logic are utilized.

# Chapter 5

## RTL DFT Analysis and Insertion

---

Performing DFT logic insertion at the register transfer level (RTL) provides better circuit optimization to help meet design constraints by considering both functional and test logic simultaneously. For example, adding observe points at RTL instead of gate level enables the tool to optimally size cells for better timing.

You can analyze the complexity of your RTL design to understand which areas could benefit from reorganization and which are suitable for test point insertion. Pre-DFT DRC rules help you catch mistakes early.

Provide detailed specifications of the DFT logic to meet your unique needs using commands such as [set\\_rtl\\_dft\\_analysis\\_options](#) and [set\\_test\\_point\\_analysis\\_options](#). The existing [process\\_dft\\_specification](#) command inserts test points, dedicated wrapper cells, and X-bounding logic in your RTL design at the same time as other Tessent IP, including EDT controllers, LBIST, OCC, and SSN. The incremental analysis and insertion capabilities at the gate level enable you to fine-tune the testability logic before generating test patterns.

|                                                                           |            |
|---------------------------------------------------------------------------|------------|
| <b>RTL DFT Analysis in the Overall Flow</b> .....                         | <b>122</b> |
| <b>RTL Design Rule Checks</b> .....                                       | <b>123</b> |
| <b>RTL IP Insertion</b> .....                                             | <b>124</b> |
| <b>RTL Design Assessment</b> .....                                        | <b>127</b> |
| RTL Code Complexity Score .....                                           | 127        |
| RTL Test Point Insertion Suitability Score .....                          | 128        |
| Detailed Metrics .....                                                    | 130        |
| Gate Node Location Types .....                                            | 131        |
| <b>Preparations for RTL DFT Analysis</b> .....                            | <b>134</b> |
| <b>Test Point Analysis at RTL</b> .....                                   | <b>136</b> |
| <b>Test Point Insertion at RTL</b> .....                                  | <b>136</b> |
| <b>Specifying RTL Test Points With Tessent Shell Commands</b> .....       | <b>138</b> |
| <b>Observation Scan Type of Test Point</b> .....                          | <b>139</b> |
| <b>Test Point Implementations</b> .....                                   | <b>143</b> |
| <b>Test Points as Function Calls</b> .....                                | <b>148</b> |
| <b>How to Preserve DFT Logic Inserted at RTL</b> .....                    | <b>153</b> |
| <b>How to Report Logic Excluded From Test Point Analysis</b> .....        | <b>155</b> |
| <b>X-Bounding Analysis and Insertion at RTL</b> .....                     | <b>158</b> |
| <b>Wrapper Analysis and Dedicated Wrapper Cell Insertion at RTL</b> ..... | <b>161</b> |

|                                                     |     |
|-----------------------------------------------------|-----|
| Smart Uniquification for RTL Pro. ....              | 163 |
| Incremental Insertion .....                         | 168 |
| Limitations of RTL DFT Analysis and Insertion ..... | 173 |

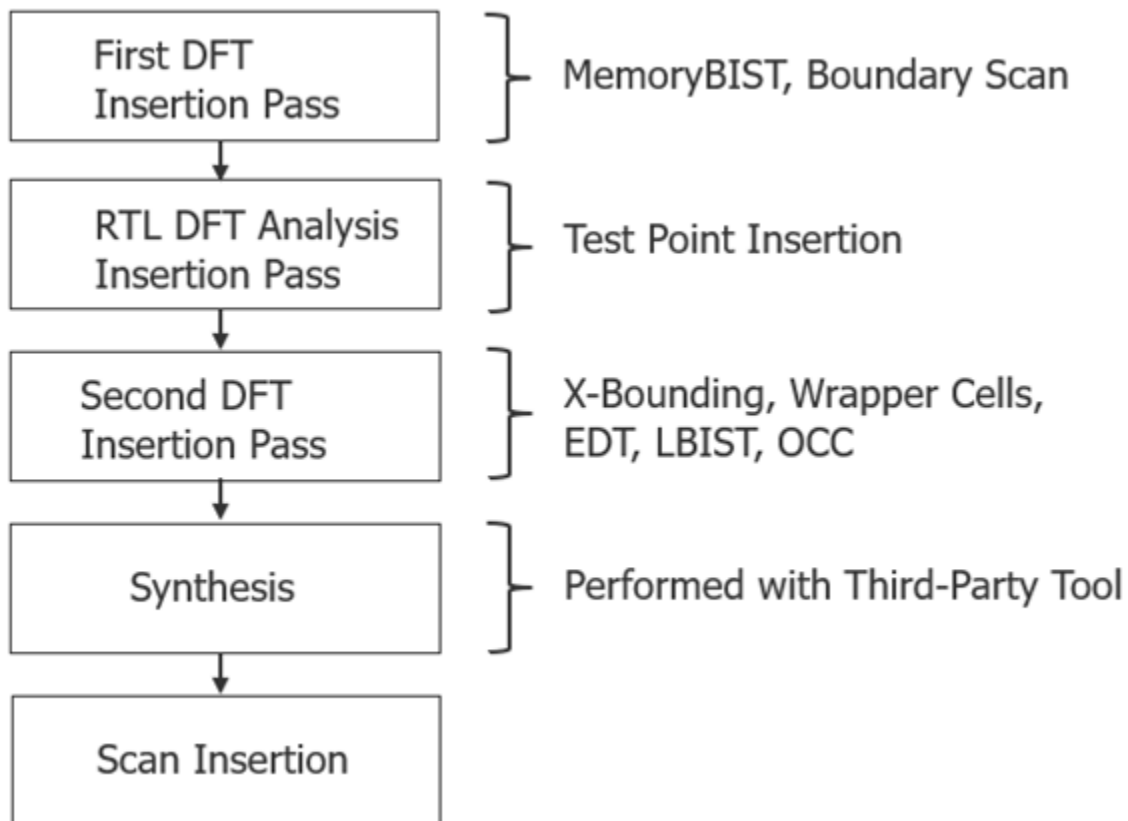
## RTL DFT Analysis in the Overall Flow

Performing DFT analysis early in the Tessent Shell workflow provides insight for later steps. For example, performing RTL DFT analysis early in the design flow enables you to more accurately estimate the number of sequential elements in the design, which you can use to compute the size of the Tessent TestKompress™ embedded deterministic testing (EDT) logic. [Figure 5-1](#) shows the overall test logic insertion flow. In the diagram, MemoryBIST refers to memory built-in self test (MBIST) logic, LBIST refers to hybrid TK/LBIST IP, and OCC refers to on-chip clock controllers.

**Note**

 We recommend running RTL DFT analysis between the first insertion pass of MemoryBIST and boundary scan and the second insertion pass of TK/LBIST and OCC, as shown in [Figure 5-1](#). You can also choose to perform RTL DFT analysis and insertion either before or after you insert the other test instruments in the RTL. If you insert TK/LBIST or EDT before test points, you must account for additional scan cells when sizing the EDT IP.

---

**Figure 5-1. RTL DFT Analysis in the Overall Insertion Flow**

You can perform RTL DFT analysis as part of a flow for flat, hierarchical, or tiled designs. Refer to “[Tessent Shell Flow for Flat Designs](#)”, “[Tessent Shell Flow for Hierarchical Designs](#)”, and “[How the DFT Insertion Flow Applies to Hierarchical Designs](#)” for more information about the full flow. X-bounding and test point insertion can be performed at the chip, physical block, or sub-block level.

**Note**

 We recommend that you run wrapper analysis at the physical block level.

## RTL Design Rule Checks

Before inserting DFT logic in an RTL design, perform design rule checks (DRCs) to prevent errors later in the flow.

**Note**

 You must run the “[set\\_dft\\_specification\\_requirements -logic\\_test on](#)” command before running the [check\\_design\\_rules](#) command to apply pre-DFT rules to your RTL design.

The following rules are particularly important for RTL designs:

- **DFT\_C6** — Runs with the `check_design_rules` command and verifies that each scannable flip-flop clock port has a defined clock in its controlling fan-in.
- **DFT\_C9** — Runs with the `check_design_rules` command and verifies that it is possible to disable the asynchronous set or reset pin of each scannable flip-flop.
- **SW7** — Runs automatically with the `analyze_wrapper_cells` command and checks for valid locations to insert dedicated wrapper cells that comply with all specifications and tool requirements.
- **SW8** — Runs automatically with the `analyze_wrapper_cells` command and checks that locations to insert dedicated wrapper cells are editable RTL constructs.
- **XB3** — Runs automatically with the `analyze_xbounding` command and checks that locations to insert X-bounding logic are editable RTL constructs.

As an additional check, you can use the `-validate_only` option on the `process_dft_specification` command to check the `DftSpecification` for the Tessent IP without inserting it.

## RTL IP Insertion

Use one general process for RTL insertion of all Tessent intellectual property (IP), such as MBIST, EDT, LBIST, test points, and SSN. Repeat the steps outlined in this section in multiple passes corresponding to different combinations of IP and levels of hierarchy.

Use the following steps insert the IP at RTL:

1. Set up your work area by setting the context to “dft -rtl”, setting the design level, and loading any required libraries. Refer to “[Tessent Shell Workflows](#)” on page 175.
2. Load your RTL design, by either reading the source code for the first time or reading the design in progress from the TSDB files. Refer to “[Loading the Design](#)” on page 185.
3. Check design rules with each subsequent step. Refer to “[RTL Design Rule Checks](#)” on page 123.
4. Specify the DFT logic and options you want to insert in this pass.
  - a. Define important signals using the `add_dft_signals` command.
  - b. Specify options to configure the IP, the optimization algorithms, and the generation process. The exact commands depend on the IP you are inserting. Refer to “[Tessent IP Configuration and Insertion Instructions](#)” on page 125 for a list of references.
  - c. Specify the large IP modules such as MBIST, EDT, LBIST controllers, IJTAG, and SSN networks. Refer to “[Configuration-Based Specification](#)” in the *Tessent Shell Reference Manual*.



- d. (Optional) Create post-insertion procs that perform design editing. Refer to “[First DFT Insertion Pass: Top with MemoryBIST, BISR, and Boundary Scan](#)” on page 438.
5. Run analysis algorithms for optimal insertion, if required.

**Note**

 This step is required for Tessent DFT logic such as test points ([analyze\\_test\\_points](#)), X-bounding ([analyze\\_xbounding](#)), and wrapper cells ([analyze\\_wrapper\\_cells](#)). It does not apply for IP that you specify completely, such as in-system test and LBIST.

6. Insert the IP in your RTL design by running the [process\\_dft\\_specification](#) command.
7. Prepare for synthesis by following the steps in “[How to Set Up Third-Party Synthesis](#)” on page 744 and “[How to Preserve DFT Logic Inserted at RTL](#)” on page 153.

Refer to “[Tessent Shell Workflows](#)” on page 175 and “[Tessent Shell Automotive Workflow](#)” on page 393 for examples of these steps applied to various design styles and situations.

Your unique design needs may require a combination of command options and [DftSpecification](#) properties. See the individual IP documentation for details on how to configure the IP for insertion in your RTL design.

**Table 5-1. Tessent IP Configuration and Insertion Instructions**

| Tessent DFT IP               | RTL Configuration and Insertion Instructions                                                                           |
|------------------------------|------------------------------------------------------------------------------------------------------------------------|
| Boundary Scan                | “ <a href="#">Getting Started With Tessent BoundaryScan</a> ” in the <i>Tessent BoundaryScan User’s Manual</i>         |
| Compression (EDT)            | “ <a href="#">Integrating Compression at the RTL Stage</a> ” in the <i>Tessent TestKompress User’s Manual</i>          |
| Dedicated Wrapper Cells      | “ <a href="#">Wrapper Analysis and Dedicated Wrapper Cell Insertion at RTL</a> ” on page 161                           |
| IJTAG                        | “ <a href="#">IJTAG Network Insertion</a> ” in the <i>Tessent IJTAG User’s Manual</i>                                  |
| In-System Test               | “ <a href="#">In-System Test IP Insertion and Pattern Generation</a> ” in the <i>Tessent MissionMode User’s Manual</i> |
| LBIST                        | “ <a href="#">EDT and LogicBIST IP Generation</a> ” in the <i>Hybrid TK/LBIST Flow User’s Manual</i>                   |
| MBIST                        | “ <a href="#">Planning and Inserting MemoryBIST</a> ” in the <i>Tessent MemoryBIST User’s Manual</i>                   |
| On-Chip Clock Control (OCC)  | “ <a href="#">Tessent On-Chip Clock Controller</a> ” in the <i>Tessent Scan and ATPG User’s Manual</i>                 |
| Streaming Scan Network (SSN) | “ <a href="#">Tessent SSN Workflows</a> ” on page 478                                                                  |

**Table 5-1. Tessent IP Configuration and Insertion Instructions (cont.)**

| Tessent DFT IP | RTL Configuration and Insertion Instructions                           |
|----------------|------------------------------------------------------------------------|
| Test Points    | <a href="#">“Test Point Analysis at RTL”</a> on page 136               |
| X-Bounding     | <a href="#">“X-Bounding Analysis and Insertion at RTL”</a> on page 158 |

# RTL Design Assessment

Tessent can assess your RTL design to give you in-depth information about the design and guide the DFT logic insertion process.

The tool reports two global scores: RTL code complexity and RTL test point suitability. For each hierarchy level, the RTL code complexity score is computed to guide any RTL refactoring efforts because it points to the most important modules of the design. Each score reflects distinct aspects of the design, as described in the following sections.

In addition to the scores, the report lists several key design metrics, both as global statistics and on a per-instance basis. When reading the reports, you need to understand how your RTL code corresponds to the tool's internal design representation down to gate-level nodes.

|                                                         |            |
|---------------------------------------------------------|------------|
| <b>RTL Code Complexity Score</b> .....                  | <b>127</b> |
| <b>RTL Test Point Insertion Suitability Score</b> ..... | <b>128</b> |
| <b>Detailed Metrics</b> .....                           | <b>130</b> |
| <b>Gate Node Location Types</b> .....                   | <b>131</b> |

## RTL Code Complexity Score

The RTL code complexity score, based on McCabe's cyclomatic complexity, represents the complexity of all the modules in the RTL design.

The Tessent solution applies the well-established McCabe technique<sup>1</sup> for calculating software complexity to hardware abstraction for test point insertion. The tool derives the metrics required to compute the score directly from the parse tree of elaborated RTL code. The chosen metrics provide an accurate cross-section of complexity. These metrics include a measure of the cyclomatic complexity of a module and the sub-modules.

A significant factor affecting the RTL code complexity score is the number of complex and nested control statements within each concurrent statement (such as "always" blocks or similar RTL constructs). Structuring the code with relatively few complex constructs within each concurrent statement results in a lower complexity score and a design that may be more testable.

Designs that concentrate more complex statements (meaning a larger amount of logic) inside relatively fewer concurrent statements receive a higher complexity score. An extreme example is a design with concurrent statements that contain so much complexity that the gate-level representation does not map back to the RTL. A very high complexity makes it difficult for the tool to identify locations for test point insertion

In addition, complex code is typically more difficult to maintain.

1. McCabe (December 1976). "A Complexity Measure". IEEE Transactions on Software Engineering. SE-2 (4): 308–320. doi:10.1109/tse.1976.233837. S2CID 9116234

To simplify the RTL quality evaluation, the tool divides the RTL code complexity score into four levels as shown in [Table 5-2](#):

**Table 5-2. RTL Code Complexity Score**

| Score     | Likely Maintenance Effort | Editable Nodes (visible RTL nets and expression boundaries) |
|-----------|---------------------------|-------------------------------------------------------------|
| Low       | Low                       | High percentage                                             |
| Medium    | Medium                    | Medium percentage                                           |
| High      | High                      | Low percentage                                              |
| Very High | Very high                 | Very low percentage                                         |

## Limiting Complexity During Development

Monitor the complexity of the modules and concurrent statements during development. RTL designers should consider splitting modules into smaller sub-modules or constructs whenever the cyclomatic complexity exceeds a threshold, in accordance with the NIST Structured Testing methodology<sup>2</sup> in software engineering.

## RTL Test Point Insertion Suitability Score

The RTL test point insertion suitability score is a metric derived after performing code complexity analysis.

To be effective, the areas requiring RTL test point insertion must be in editable locations. Even if a design has a “Low” complexity score, if you need most of the new test points in noneditable functional blocks or control logic, the test point suitability may be low. The inverse also holds: a highly complex block may require test points only in the editable nodes of the design, making the test point suitability score “High”.

The tool computes the suitability score for RTL test point insertion globally. This is done top-down for the whole design. The score is primarily based on RTL code complexity scores for every instance across the logic hierarchy. For every instance level, the tool adjusts the complexity score metric by considering pre-test point analysis metrics. Example metrics include the total number of testable gate nodes and the total editable testable gate nodes. Refer to “[Gate Node Location Types](#)” for more information.

---

2. Wallace, D. , Watson, A. and McCabe, T. (1996), Structured Testing: A Testing Methodology Using the Cyclomatic Complexity Metric, Special Publication (NIST SP), National Institute of Standards and Technology, Gaithersburg, MD (Accessed August 31, 2022)

To simplify the RTL suitability evaluation, the tool divides the RTL test point suitability score into three levels:

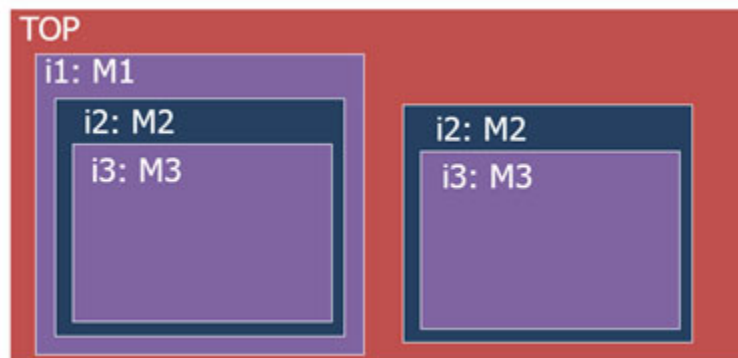
**Table 5-3. RTL Test Point Insertion Suitability Score**

| Score  | Editable Nodes<br>(visible RTL nets<br>and expression<br>boundaries) | Recommended Insertion Flow to Improve ATPG and<br>BIST Metrics                                                      |
|--------|----------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| High   | High percentage                                                      | RTL level                                                                                                           |
| Medium | Medium percentage                                                    | RTL insertion likely enhances the testability of the block.<br>Gate-level insertion may provide higher testability. |
| Low    | Low percentage                                                       | Gate level                                                                                                          |

If the sub-blocks have many logic elements tied constant or left floating, the tool can increase the suitability score by optimizing the sub-blocks' content. A design with many sub-blocks with "High" RTL code complexity does not necessarily have a "Low" suitability score.

For example, consider the hierarchical design in [Figure 5-2](#)

**Figure 5-2. Example Hierarchical Design**



with the RTL code complexity:

- M1, M3: Medium
- M2: High
- Top: Low

The code complexity score of module M2 might be considered "Low" if the number of testable editable nodes become a high percentage after excluding the constant and floating nodes. This transformation impacts the computation of the global RTL test point suitability score of the design. The result may be "High" RTL insertion suitability scores.

## Detailed Metrics

RTL code complexity analysis provides detailed metrics on your RTL design.

The code complexity analysis provides a summary code complexity score, a summary test point suitability score, and design metrics. The available design metrics change depending on when you perform complexity analysis:

- Prior to quick synthesis, the tool reports only high-level information about your design:
  - Overall RTL complexity score
  - Design profile
    - Identification of RTL and structural modules within the RTL
    - Number of black boxes
    - Total design instances
  - Library profile
    - Number of Synopsys DesignWare® modules and instances
    - Number of Tessent library modules and instances
- After quick synthesis, the tool reports additional information:
  - Overall test point suitability score
  - Library profile
    - Number of directly instantiated and synthesized sequential cells
    - Number of constant and floating cells
  - Detailed design pin information:
    - Number of gate nodes
    - Number of gate nodes at expression boundaries
    - Number of gate nodes inside functional blocks
    - Number of gate nodes at control logic
    - Number of floating gate nodes
    - Number of tied constant gate nodes
    - Number of gate nodes within DesignWare
- After test point analysis, the tool also reports the number of test points targeted for visible nodes and expression boundaries.

- Then, after test point insertion, the tool also reports the number of test points collapsed to inside control logic and functional blocks.
- You can run complexity analysis in a high-effort mode to provide greater detail as part of the complexity report after quick synthesis. See [report\\_rtl\\_complexity](#) for more information.

When you run complexity analysis with the “-effort high” option, the tool performs test point analysis but not insertion. It considers the entire design for test points, and compares the analysis to the editable RTL nodes. Metrics included in the high-effort report:

- Information about test points that target editable nodes of the RTL
  - Number of test points targeted to directly visible nodes
  - Number of test points targeted to expression boundaries
- Information about test points that target non-editable nodes of the RTL
  - Number of test points targeted to functional blocks
  - Number of test points targeted to control logic

## Gate Node Location Types

In RTL mode, the tool performs test point analysis after Tessent quick synthesis has transformed the input RTL design to a gate level netlist. The netlist view becomes the basis of all test point locations. This section describes how the gate-level nodes correspond to the RTL.

While searching for RTL locations to insert control points (CP) and observe points (OP), the tool may encounter several distinct types of gate nodes. The [get\\_rtl\\_location\\_type](#) command returns the type of specific objects including gate nodes in the design. The following are types of gate nodes:

- **Gate nodes directly mapped to RTL nets** — Examples include nets, and port pins declared and visible at RTL. These gate nodes are fully editable, and the tool can insert test points at the gate node locations of the closet visible RTL nets.

The tool inserts the CP/OP at stems instead of branches. When dealing with directly visible nodes, the tool targets individual stems instead of branches. With RTL, the same statement or equation may be used multiple times. In that case, if multiple test points target the same stem in different locations in the RTL, Tessent merges the multiple stem test points into a single branch test point.

- **Gate nodes directly mapped to RTL expression boundaries** — Examples include RTL sub-expressions. These gate nodes are fully editable, and Tessent can insert CPs and OPs at the boundaries of RTL expressions and sub-expressions. Because these

expressions typically use the same variables, Tessent considers branch locations during analysis.

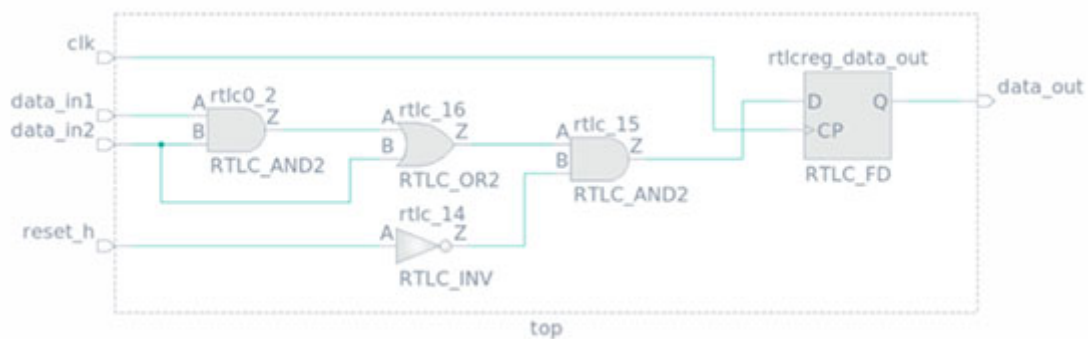
- **Gate nodes mapped inside functional blocks** — Examples include adders. These gate nodes are not editable, and the tool cannot insert test points.
- **Gate nodes mapped inside control logic** — Examples of sequential control statements include if-then-else and case. These gate nodes are not editable, and the tool cannot insert test points.

## Example

This example illustrates the terminology:

```
module top (input wire clk,
            input wire reset_h,
            input logic data_in1,
            input logic data_in2,
            output logic data_out);
    always_ff @(posedge clk) begin
        if (reset_h == 1'b1)
            data_out <= '0;
        else
            data_out <= (data_in1 & data_in2) | data_in2;
        end
    end
endmodule
```

**Figure 5-3. Example Design**



**Table 5-4. Gate Node Location Mapping**

| Terminology    | Description                                | Example Names                                        |
|----------------|--------------------------------------------|------------------------------------------------------|
| Gate instances | Cell instances inferred by quick synthesis | rtlc0_2, rtlc_16, rtlc_15, rtlc_14, rtlcreg_data_out |



**Table 5-4. Gate Node Location Mapping (cont.)**

| Terminology                         | Description                                                                                                                   | Example Names                                                                                                                                                                                                                                        |
|-------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Gate nodes (pins)                   | Pins of cells instances inferred by quick synthesis to represent the input/output of inferred logic<br>Primary inputs/outputs | rtlc0_2/A, rtlc0_2/B, rtlc0_2/Z,<br>rtlc_16/A, rtlc_16/B, rtlc_16/Z,<br>rtlc_15/A, rtlc_15/B, rtlc_15/Z,<br>rtlc_14/A, rtlc_14/Z,<br>rtlcreg_data_out/D,<br>rtlcreg_data_out/CP,<br>rtlcreg_data_out/Q,clk, data_in1,<br>data_in2, reset_h, data_out |
| Branch                              | All gate nodes (pins) can be considered as branches                                                                           | rtlc0_2/A, rtlc0_2/B                                                                                                                                                                                                                                 |
| Stem                                | Root source signal of one or more branches                                                                                    | data_in2 is the stem of rtlc0_2/B and rtlc_16/B.<br>clk is stem of rtlcreg_data_out/CP.                                                                                                                                                              |
| Visible RTL nets                    | All nets or portNets represented by RTL signals; both internal nets of ports and pure nets.                                   | clk, data_in1, data_in2, data_out, reset_h                                                                                                                                                                                                           |
| RTL expressions (sub-expression)    | All RTL expressions such as arithmetic, logical, and function calls                                                           | Expression 1: (reset_h == 1'b1)<br>Expression 2: (data_in1 & data_in2)<br>Expression 3: (data_in1 & data_in2)   data_in2<br>Corresponding gate instances are:<br>Expression 1: rtlc_14<br>Expression 2: rtlc0_2<br>Expression 3: rtlc0_2, rtlc_16    |
| Control logic vs Control statements | All logic inferred to represent the control statements in RTL such as if-then-else, case, and read/write expressions          | if (...) begin<br>...<br>end else begin<br>...<br>end;<br><br>This control statement is represented by the gate instances: rtlc_15. This logic is controlled by the reset signal "reset_h".                                                          |

**Table 5-4. Gate Node Location Mapping (cont.)**

| Terminology       | Description                                                                                                                  | Example Names |
|-------------------|------------------------------------------------------------------------------------------------------------------------------|---------------|
| Functional Blocks | All logic inferred to represent complex operators are named functional blocks, for example: ==, !=, +, -, *, /, >=, <=, <, > |               |

From the example, the tool may identify the following editable testable gate nodes (pins) as potential CPs and OPs:

- rtlc0\_2/A
- rtlc0\_2/B
- rtlc0\_2/Z
- rtlc\_16/A
- rtlc\_16/B
- rtlc\_16/Z
- rtlreg\_data\_out/Q
- data\_in1
- data\_in2
- data\_out

The tool identifies the gate nodes (pins) on the clock and reset paths as editable but not testable and not suitable as potential test points. Example untestable gate nodes:

- clk
- rtlreg\_data\_out/CP
- reset\_h
- rtlc\_14/A
- rtlc\_14/Z

## Preparations for RTL DFT Analysis

RTL DFT analysis shares many common steps with gate-level DFT insertion such as setting options, running analysis, and reporting results before insertion. However, performing analysis and insertion of wrapper cells, X-bounding logic, and test points in RTL requires unique preparations.

## Pre-DFT DRCs

Your design must pass the pre-DFT design rule checks (DRCs). Refer to “[RTL Design Rule Checks](#)” on page 123 for more information.

## DFT Analysis Options

You can exclude parts of the RTL such as floating logic, tied logic, and non-editable nodes. For more information, refer to the “[set\\_rtl\\_dft\\_analysis\\_options](#)” command in the *Tessent Shell Reference Manual*.

## RTL Complexity Analysis

Identify RTL Complexity and TPI Suitability using the “[report\\_rtl\\_complexity](#)” command described in the *Tessent Shell Reference Manual*.

## Handling Known Non-Scan Elements

Inserting DFT logic at the register transfer level requires all sequential elements to pass scannability checking, including clock/set/reset controllability checks. Therefore, non-scan elements must be defined to pass DRC for any cells which would not be scanned later in the flow.

---

### Tip



Use a consistent naming convention such as a prefix for non-scan elements and use the introspection commands to find them.

---

To refer to non-scan elements in the RTL code, use the “[add\\_nonscan\\_instances](#)” command with the `-rtl_reg` switch. For example:

```
add_non_scan_instances -rtl_reg [get_nets ${<nonscan_prefix>}* -hierarchical]
```

This example identifies instantiated non-scan elements:

```
add_non_scan_instances -instances [get_instances ${<nonscan_prefix>}* -hierarchical]
```

## Quick Synthesis

Control quick synthesis options with the “[set\\_quick\\_synthesis\\_options](#)” command. Options include adding parallel synthesis and skipping synthesis of large two-dimensional arrays.

## DFT-Inserted RTL Written to TSDB

The tool saves the modified RTL to the Tessent Shell database (TSDB).

## Test Point Analysis at RTL

Tessent RTL Pro enables you to analyze your RTL design for potential test point locations and optimize your strategy before insertion.

You can choose the goals of test point analysis by running the [set\\_test\\_point\\_types](#) command in setup mode. The tool can optimize for LBIST test coverage or deterministic test pattern count, or both. Refer to “[Test Point Usage Scenarios](#)” in the *Scan and ATPG User’s Manual*.

Specify your test point requirements using the [set\\_test\\_point\\_analysis\\_options](#) command. Optionally, you can add your own test points using the [add\\_control\\_points](#) and [add\\_observe\\_points](#) commands or restrict areas using the [add\\_notest\\_points](#) commands. These commands run in the “dft -rtl -test\_points” context for RTL insertion.

When you transition from setup to analysis mode, the tool automatically runs quick synthesis to generate a gate-level view of the RTL design. The tool builds the netlist view and marks all gate pins to be excluded from test point insertion based on the options you specify with the [set\\_rtl\\_dft\\_analysis\\_options](#) command. When you run the [analyze\\_test\\_points](#) command in analysis mode in the “dft -rtl -test\_points” context, the tool chooses the test point locations, which it inserts later in the process. It maps those test point locations based on the gate-level view back to RTL.

You can evaluate the results of test point analysis before insertion by running the [report\\_test\\_points](#) command. You can iterate the analysis process by returning to setup mode and setting different options. The tool automatically cleans up test point locations if you switch back to setup mode and keeps them while you stay in analysis mode.

After identifying the optimal settings for your RTL design, Tessent RTL Pro can insert those test points at RTL. Refer to “[Test Point Insertion at RTL](#)” on page 136 for more information.

For more in-depth information about test points, refer to the following topics in the *Tessent Scan and ATPG User’s Manual*:

- [What are Test Points?](#)
- [Test Points Special Topics](#)

## Test Point Insertion at RTL

The tool inserts the test points it identified with the [analyze\\_test\\_points](#) command and those you specified with the [add\\_control\\_points](#) and [add\\_observe\\_points](#) commands when you run the [process\\_dft\\_specification](#) command.

You must run the [set\\_current\\_design](#) command before starting test point insertion.

Control the test point insertion using the following commands in the “dft -rtl -test\_points” context:

- [set\\_test\\_point\\_analysis\\_options](#)
- [set\\_test\\_point\\_insertion\\_options](#)
- [set\\_test\\_point\\_sharing\\_restrictions](#)

The following example uses the `set_current_design` command to select the portion of the design for test point insertion, the [analyze\\_test\\_points](#) command to automatically identify where to insert control points and observe points, and the [process\\_dft\\_specification](#) command to insert the test points:

```
set_context dft -rtl -test_points -design_id rtl1

read_cell_library <library_path>/NangateOpenCellLibrary.atpglib \
  ../data/picdram.atpglib
read_verilog ../data/piccpul.v
read_verilog ../data/picdram.v -blackbox -exclude
read_cell_library ../data/cell_selection
set_cell_library_options -default_dft_cell_selection tc_selection

set_current_design piccpul
set_design_level physical_block

add_clocks 0 clk
add_clocks 0 ramclk
add_black_box -instance bbox

check_design_rules

analyze_test_points

report_test_points

create_dft_specification
process_dft_specification
```

## Test Point Sharing

To enable test point sharing, use the `set_test_point_analysis_options` command to specify the `-shared_control_points_per_flop` and `-shared_observe_points_per_flop` options. For more information, refer to the “[Test Point Sharing](#)” section of the *Tessent Scan and ATPG User’s Manual*.

A limitation at RTL involves an observe point (OP) within the clocked always statement at the boundary of an expression. Within the always statement, the tool makes an assignment to the test data output that observes the expression. Synthesis automatically infers a flip-flop that cannot be shared with other observe points.

For example, consider an OP at the expression “in1 & in2” in the following RTL code:

```
always_ff @(posedge clk , negedge reset )
  if ( !reset )
    done_sm_ps <= NOT_DONE ;
  else
    done_sm_ps <= in1 & in2;
```

The tool creates the following RTL after test point insertion:

```
reg [0:0] tessent_persistent_cell_op_test_out_1;
always_ff @(posedge clk , negedge reset )
  if ( !reset )
    done_sm_ps <= NOT_DONE ;
  else
    done_sm_ps <= InsertObservePointOutput(in1 & in2, op_en,
      tessent_persistent_cell_op_test_out_1);

top_rtl_tp_op_holder tessent_persistent_cell_op_holder_instance_1
  (.funcin(tessent_persistent_cell_op_test_out_1), .ff_out());

function logic InsertObservePointOutput(input logic sig_in,
                                         input logic TM,
                                         output logic test_out);

  test_out = sig_in & TM;
  return sig_in;
endfunction
```

Synthesis infers the flip-flop of this observe point by the test data output signal “tessent\_persistent\_cell\_op\_test\_out\_1”. The output of the flip-flop “tessent\_persistent\_cell\_op\_test\_out\_1” connects to an empty instance “tessent\_persistent\_cell\_op\_holder\_instance\_1” that does not include a flip-flop. When Tessent detects this kind of observe point, it automatically transforms it to a regular OP, and does not share the same flip-flop with other OPs in the same group.

## Specifying RTL Test Points With Tessent Shell Commands

You can use the `add_observe_points` and `add_control_points` commands in the “`dft -rtl -test_points`” context to add test points on RTL ports, pins, and nets.

Use these commands to specify observe points and control points to be inserted in RTL when you run the “[process\\_dft\\_specification](#)” command.

## Example

```
set_context dft -rtl -test_points

read_verilog ../prerequisites/insert_test_point_logic.tshell/rtl/tp.v
read_verilog -format sv2009 ../data/pkg1.sv
read_verilog -format sv2009 ../data/top1.sv

set_current_design top
add_clocks 0 clk
add_clocks 1 rstn
check_design_rules

add_control_points -location IF1.mode[0] -type AND \
  -enable ts_control_enable -clock clk
add_observe_points -location IF1.mode[2] \
  -enable ts_observe_enable -clock clk

analyze_test_points

create_dft_specification
process_dft_specification
```

## Observation Scan Type of Test Point

The observation scan type (OST) is an observe test point type for use in the RTL context that stitching tools can connect into scan chains.

Tessent scan technology inserts OST observe points in RTL by either instantiating a scan cell or describing the behavior of the scan cell. The tool exports information to enable the stitching of these cells into separate scan chains.

The OST uses a positive-edge scan flip-flop RTL cell, the PosedgeSff.

## Process

Use the following process to add OST observe points in RTL.

- Use the “[set\\_context dft -test\\_points -rtl](#)” command.
- Add the capture\_per\_cycle\_static\_en DFT signal in SETUP mode.
- Use the “[set\\_test\\_point\\_analysis\\_options -capture\\_per\\_cycle on](#)” command.
- Use the “[set\\_test\\_point\\_analysis\\_options -minimum\\_shift\\_length](#)” command.
- Run test point analysis with the “[analyze\\_test\\_points](#)” command.

### Note



The tool requests an LBIST-OST license.

- Insert OSTs by running the “[process\\_dft\\_specification](#)” command.

- Use the “set\_context dft -rtl” command.
- Synthesize the RTL design.
- Use the “set\_context dft -scan -no\_rtl” command.
- Insert scan chains.

---

**Note**

 After inserting test logic, you can synthesize the design. The behavioral scan cell flip-flops must have the set\_dont\_scan mark to prevent the synthesis tool from replacing them with scan flip-flops.

The OST observe points must have the set\_preserve\_boundary mark to prevent their removal by synthesis tool optimizations. The other observe and control test point types also use this mark during synthesis.

---

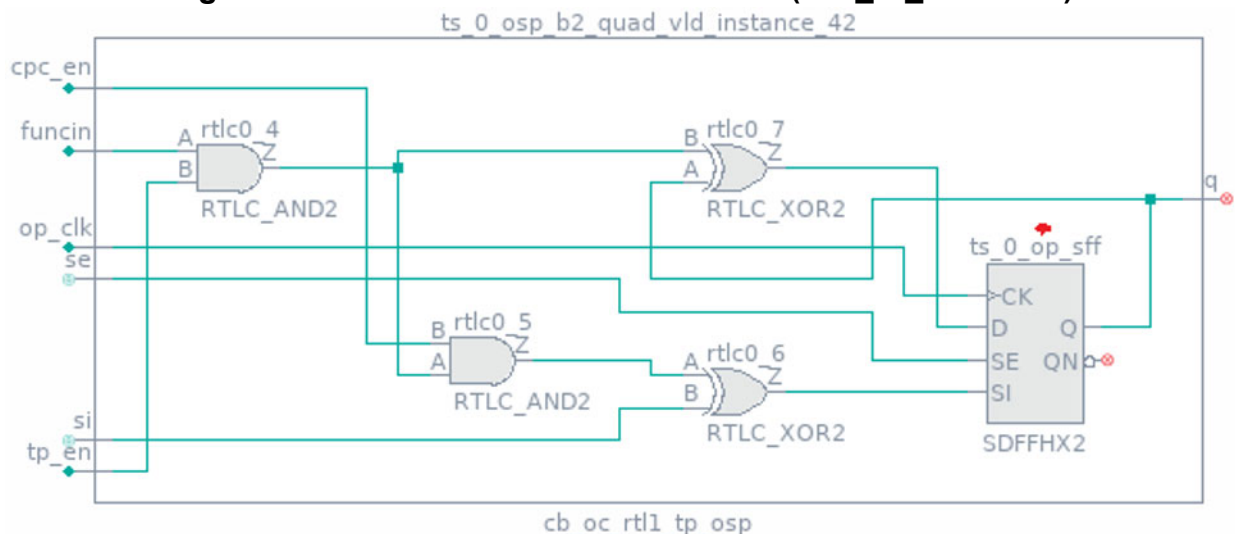
This flow is analogous to the post-synthesis insertion flow in the “dft -scan -test\_points” context.

## Inserting OST Modules

After test point analysis is complete, insert OST test point logic using the process\_dft\_specification command,. The use\_rtl\_cells property of the DftSpecification wrapper affects the RTL implementation.

Figure 5-4 shows an example of the logic that process\_dft\_specification inserts when use\_rtl\_cells is off. The logic module code is below the figure.

**Figure 5-4. Schematic of an OST Module (use\_rtl\_cells : off)**





```

module cb_oc_rtl1_tp_osp(funcin, tp_en, op_clk, cpc_en, si, se, q);
  input  funcin, tp_en, op_clk, cpc_en, si, se;
  output q;

  assign gatedFuncIn = funcin & tp_en;
  assign cpcGatedFuncIn = gatedFuncIn & cpc_en;
  assign siXored = cpcGatedFuncIn ^ si;
  assign qXored = q ^ gatedFuncIn;
  SDFFH2 ts_0_osp_sff (
    .D                                ( qXored ),
    .CK                              ( op_clk ),
    .SI                              ( siXored ),
    .SE                              ( se ),
    .Q                                ( q )
  );
endmodule

```

The `process_dft_specification` command inserts behavioral code for the `ts_0_osp_sff` scan flip-flop in [Figure 5-4](#) if `use_rtl_cells` is on. The code is a pos-edge RTL scan cell, `PosedgeSff`.

```

module cb_oc_rtl1_tp_tessent_posedge_scan_cell (
  input wire d,
  input wire si,
  input wire se,
  input wire clk,
  output reg so
);
  wire mux_net;
  assign mux_net = (se) ? si : d;
  always @ (posedge clk) begin
    so <= mux_net;
  end
endmodule

```

You can substitute the `PosedgeSff` behavioral cells with actual scan cells later in the flow.

## Inserting OSTs Inside Expressions

After test point analysis is complete, the tool inserts OST observe points inside expressions using a function call.

For example, if you want to insert an OST on `!data_conflict` in the following expression:

```

wire ready_no_conflict = (!data_conflict || resolve_enable) &&
(stored_cycle ? match : !match);

```

The insertion process creates the InsertObservePointV95 function call and modifies the expression to use it:

```
function InsertObservePointV95;
    input sig_in;
    input TM;
begin
    InsertObservePointV95 = sig_in & TM;
end
endfunction

assign ts_temp_0 = (!data_conflict);

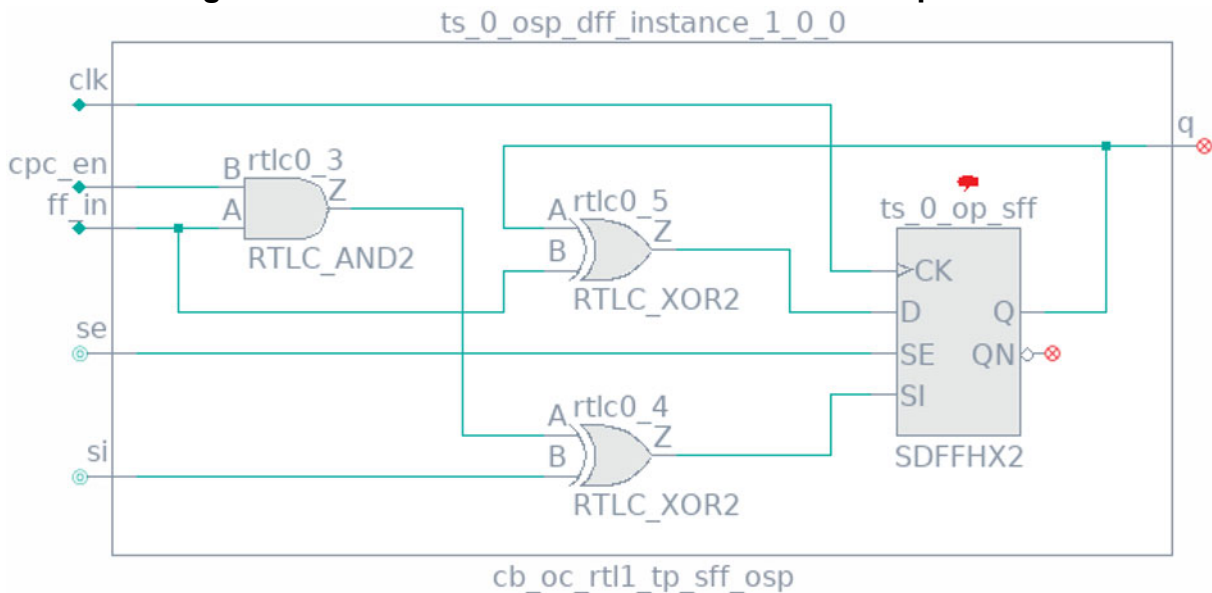
wire ready_no_conflict = (ts_temp_0 || resolve_enable) &&
(stored_cycle ? cycle_match : !cycle_match);

assign ts_0_osp_test_out_1_0 = InsertObservePointV95(ts_temp_0,
observe_test_point_en);

cb_oc_rtl1_tp_sff_osp ts_0_osp_dff_instance_1_0_0 (
    .clk(clk_ts1),
    .ff_in(ts_0_osp_test_out_1_0),
    .cpc_en(capture_per_cycle_en_out),
    .se(1'b0),
    .si(1'b0),
    .q()
);
```

Figure 5-5 shows the schematic of the cb\_oc\_rtl1\_tp\_sff\_osp cell and how it includes the rest of the OST scan cell.

**Figure 5-5. Partial Schematic of an OST for an Expression**



## Stitching OSTs Into Scan Chains

The scan cell within the OST already has logic in the scan-in path, which prevents the replacement of a D flip-flop with a scan flip-flop. Instead, the tool generates a TCD scan file for all OSTs during RTL generation in the `dft -rtl` context. The tool automatically imports this file in the `dft -scan` context.

The flow must stitch all OSTs into separate scan chains from the normal scan cells. The tool can stitch the observation scan cells correctly when the `module_type` property value is `observation_scan_cell`.

The following is an example TCD scan file, `cb_oc_rtl1_tp_tessent_osp.tcd_scan`, for an observation scan cell. An example path to the file is `tsdb_outdir/instruments/cb_oc_rtl1_tp_rtl_scan_dft_logic.instrument/`.

```
Core(cb_oc_rtl1_tp_tessent_osp) {
  Scan {
    module_type : observation_scan_cell;
    is_hard_module : 1;
    internal_scan_only : 0;
    traceable : 0;
    Clock(op_clk) {
      off_state : 1'b0;
    }
    ScanEn(se) {
      active_polarity : all_ones;
    }
    ScanChain {
      length : 1;
      scan_in_port: si;
      scan_out_port: q;
      scan_in_clock: op_clk;
      scan_out_clock: op_clk;
    }
  }
}
```

The tool can generate a CTL file for third-party scan insertion.

## Limitations

Insertion of scan cells inside always-clocked logic blocks is not yet possible. If you try to insert an OST observe point inside an always-clocked logic block, the tool replaces it with a regular test point.

# Test Point Implementations

The tool implements test points by instantiating control point and observe point flip-flops using the cell versions you specified with the `read_cell_library` command. You can use the

DftSpecification/use\_rtl\_cells property to force the tool to generate behavioral flops using RTL always statements in the RTLCells instrument directory.

```
DftSpecification(module_name,id) {  
    ...  
    use_rtl_cells                : on | off | auto ;  
    ...  
}
```

## Examples

### Example 1

This example specifies “use\_rtl\_cell on”:

```
set_context dft -rtl -test_points -design_id rtl1  
  
read_cell_library ../45nm/Tessent/NangateOpenCellLibrary.atpglib \  
    ../data/picdram.atpglib  
read_verilog ../data/piccpul.v  
read_verilog ../data/picdram.v -blackbox -exclude  
read_cell_library ../data/cell_selection  
set_cell_library_options -default_dft_cell_selection tc_selection  
  
set_current_design piccpul  
set_design_level physical_block  
  
add_clocks 0 clk  
add_clocks 0 ramclk  
add_black_box -instance bbox  
  
check_design_rules  
  
analyze_test_points  
  
report_test_points  
set_defaults_value DftSpecification/use_rtl_cells on  
set_spec [create_dft_specification -replace]  
process_dft_specification
```

The control point and observe point instruments instantiate the RTL behavior version of the DFF defined in the RTLCells directory as follows:

```
module dff_cp(clk, ff_out);
  input clk;
  wire clk;
  output ff_out;
  wire ff_out;
  piccpul_rtl2_tessent_posedge_dff ts_0_cp_dff (
    .d          ( ff_out          ),
    .clk        ( clk            ),
    .q          ( ff_out          )
  );
endmodule

module dff_op(clk, ff_in);
  input  clk;
  wire  clk;
  input ff_in;
  wire ff_in;
  piccpul_rtl2_tessent_posedge_dff ts_0_op_dff (
    .d          ( ff_in          ),
    .clk        ( clk            ),
    .q          (                )
  );
endmodule

module cpor(funcin, tp_en, cp_clk, cp_out);
  input funcin;
  input tp_en;
  input cp_clk;
  wire funcin;
  wire tp_en;
  wire cp_clk;
  output cp_out;
  wire ff_out;
  piccpul_rtl2_tessent_posedge_dff ts_0_cp_dff (
    .d          ( ff_out          ),
    .clk        ( cp_clk          ),
    .q          ( ff_out          )
  );
  assign cp_out = (tp_en & ff_out) | funcin;
endmodule

module cpand(funcin, tp_en, cp_clk, cp_out);
  input funcin;
  input tp_en;
  input cp_clk;
  wire funcin;
  wire tp_en;
  wire cp_clk;
  output cp_out;
  wire cp_out;
  wire ff_out;
  piccpul_rtl2_tessent_posedge_dff ts_0_cp_dff (
    .d          ( ff_out          ),
    .clk        ( cp_clk          ),
    .q          ( ff_out          )
  );
  assign cp_out = (~tp_en | ff_out) & funcin;
```

```
endmodule module op(funcin, tp_en, op_clk);
  input  funcin, tp_en, op_clk;
  wire  funcin, tp_en, op_clk;
  piccpul_rtl2_tessent_posedge_dff ts_0_op_dff (
    .d          ( funcin & tp_en          ),
    .clk        ( op_clk                  ),
    .q          (                          )
  );
endmodule
```

### Example 2

This example specifies “use\_rtl\_cell off”:

```
set_context dft -rtl -test_points -design_id rtl1

read_cell_library ../45nm/Tessent/NangateOpenCellLibrary.atpglib \
  ../data/picdram.atpglib
read_verilog ../data/piccpul.v
read_verilog ../data/picdram.v -blackbox -exclude
read_cell_library ../data/cell_selection
set_cell_library_options -default_dft_cell_selection tc_selection

set_current_design piccpul
set_design_level physical_block

add_clocks 0 clk
add_clocks 0 ramclk
add_black_box -instance bbox

check_design_rules

analyze_test_points

report_test_points
set_defaults_value DftSpecification/use_rtl_cells off
set_spec [create_dft_specification -replace]
process_dft_specification
```

The control point and observe point instruments instantiate the available cells as follows:

```
module dff_cp(clk, ff_out);
  input clk;
  wire clk;
  output ff_out;
  wire ff_out;
  DFF_X2 ts_0_cp_dff (
    .D          ( ff_out          ),
    .CK         ( clk            ),
    .Q          ( ff_out          )
  );
endmodule

module dff_op(clk, ff_in);
  input  clk;
  wire  clk;
  input ff_in;
  wire ff_in;
  DFF_X2 ts_0_op_dff (
    .D          ( ff_in          ),
    .CK         ( clk            ),
    .Q          (                )
  );
endmodule

module cpor(funcin, tp_en, cp_clk, cp_out);
  input funcin;
  input tp_en;
  input cp_clk;
  wire funcin;
  wire tp_en;
  wire cp_clk;
  output cp_out;
  wire ff_out;
  DFF_X2 ts_0_cp_dff (
    .D          ( ff_out          ),
    .CK         ( cp_clk          ),
    .Q          ( ff_out          )
  );
  assign cp_out = (tp_en & ff_out) | funcin;
endmodule

module cpand(funcin, tp_en, cp_clk, cp_out);
  input funcin;
  input tp_en;
  input cp_clk;
  wire funcin;
  wire tp_en;
  wire cp_clk;
  output cp_out;
  wire cp_out;
  wire ff_out;
  DFF_X2 ts_0_cp_dff (
    .D          ( ff_out          ),
    .CK         ( cp_clk          ),
    .Q          ( ff_out          )
  );
  assign cp_out = (~tp_en | ff_out) & funcin;
endmodule
```

```
module op(funcin, tp_en, op_clk);
  input  funcin, tp_en, op_clk;
  wire  funcin, tp_en, op_clk;
  DFF_X2 ts_0_op_dff (
    .D                ( funcin & tp_en                ),
    .CK                ( op_clk                        ),
    .Q                 (                               )
  );
endmodule
```

### Example 3

This example specifies “use\_rtl\_cell auto”:

```
set_context dft -rtl -test_points -design_id rtl1

read_cell_library ../45nm/Tessent/NangateOpenCellLibrary.atpglib \
  ../data/picdrum.atpglib
read_verilog ../data/piccpul.v
read_verilog ../data/picdrum.v -blackbox -exclude
read_cell_library ../data/cell_selection
set_cell_library_options -default_dft_cell_selection tc_selection

set_current_design piccpul
set_design_level physical_block

add_clocks 0 clk
add_clocks 0 ramclk
add_black_box -instance bbox

check_design_rules

analyze_test_points

report_test_points
set_defaults_value DftSpecification/use_rtl_cells auto
set_spec [create_dft_specification -replace]
process_dft_specification
```

The control point and observe point instruments instantiate the available cells as shown in Example 2.

## Test Points as Function Calls

RTL functions represent the combinatorial logic portions of control and observe test points identified at the boundaries of RTL expressions.



## Examples

### Example 1

This is an example of a 1-bit control point:

```
Always
@(posedge clk or negedge rst_n)
begin : TXID_SLOT_CNT_FF
    if ((~rst_n))
        txid_slot_cnt[7] <= 3'd0;
    else if (block_en_n)
        txid_slot_cnt[7] <= 3'd0;
    else
        begin
            if (((conf_ofdma_mode & active_txids[7]) &
                o_fifo_wr[txid_active_slot[7]])
                & fifo_llr_last[txid_active_slot[7]]))
                begin
                    if (InsertControlPointORTypeV95(
                        txid_slot_cnt[7]
                        == (conf_txid_slots_array[7] - 4'd1)),
                        control_test_point_en,
                        ts_0_cp_test_in_360_0)
                        txid_slot_cnt[7] <= 3'd0;
                    else
                        txid_slot_cnt[7] <= (txid_slot_cnt[7] + 3'd1);
                end
            end
        end
    end
end

function InsertControlPointORTypeV95;
input sig_in;
input TM;
input test_in;
reg sig_in_tp;
begin
    sig_in_tp = (TM & test_in) | sig_in;
    InsertControlPointORTypeV95 = sig_in_tp;
end
endfunction
```

## Example 2

This example has two control points on the same expression:

```
assign length_burst = (((in_length_burst[3:0] >
4'd0)&&(in_length_burst[3:0] <=4'd4))
? 4'hc :
    InsertControlPoint4Bits2ORTypeV95_1309(
        (((in_length[3:0] > 4'd4)&&(in_length[3:0]
<=4'd8))
? 4'h8 :
        (((in_length[3:0] > 4'd8)&&
(in_length[3:0] <=4'd12))
? 4'h4 :
        ((in_length[3:0] ==4'd13)
? 4'h3 :
        ((in_length[3:0] ==4'd14)
? 4'h2 :
        ((in_length[3:0] ==4'd15)
? 4'h1 :
        4'h0))))),
        control_test_point_en,
        ts_0_cp_test_in_782_0)
    );

function [4-1:0] InsertControlPoint4Bits2ORTypeV95_1309;
    input [4-1:0] sig_in;
    input TM;
    input [2-1:0] test_in;
    reg [4-1:0] sig_in_tp;
begin
    sig_in_tp[0] = (TM & test_in[0]) | sig_in[0];
    sig_in_tp[1] = (TM & test_in[1]) | sig_in[1];
    sig_in_tp[2] = sig_in[2];
    sig_in_tp[3] = sig_in[3];
    InsertControlPoint3Bits2ORTypeV95_1309 = sig_in_tp;
end
endfunction
```

## Example 3

This example inserts a control point and connects the associated flip-flop to a negative edge-triggered clock:

The original RTL specifies a negative edge-triggered clock.

```
module top(input clk,in1,in2,
    output reg out);
    sample i1(clk,in1,in2,out);
endmodule
module sample (input clk, in1,in2,
    output reg out);
    always @(negedge clk) begin
        out = in1&in2;
    end
endmodule
```

Assume that the tool inserts a control point on “in1&in2” on “path i1”. The following code is the modified RTL with a negative edge-triggered test point inserted.

```
module top(input clk,in1,in2, output reg out, input wire cp_en);
  sample i1(clk,in1,in2,out, cp_en);
endmodule
module sample (input clk, in1,in2, output reg out, input wire cp_en);
  wire [0:0] tessent_persistent_cell_cp_test_in_1_0;
  always @(negedge clk) begin
    out = InsertControlPointANDTypeSV09((in1 & in2), cp_en,
      tessent_persistent_cell_cp_test_in_1_0);
  end
  top_tessent_dff_cp_neg tessent_persistent_cell_cp_dff_instance_1_0_0(
    .clk(clk), .ff_out(tessent_persistent_cell_cp_test_in_1_0)
  );
  function logic InsertControlPointANDTypeSV09(input logic sig_in,
    input logic TM,
    input logic test_in);

    return (~TM | test_in) & sig_in;
  endfunction
endmodule
```

The following Verilog code defines the negative edge-triggered flip-flop associated with the inserted control point:

```
module top_tessent_dff_cp_neg(clk, ff_out);
  input clk;
  wire clk;
  output ff_out;
  wire ff_out;
  top_tessent_negedge_dff tessent_persistent_cell_cp_dff (
    .d ( ff_out ),
    .clk ( clk ),
    .q ( ff_out )
  );
endmodule

module top_tessent_negedge_dff (
  input wire d,
  input wire clk,
  output reg q
);
  always @ (negedge clk) begin
    q <= d;
  end
endmodule
```

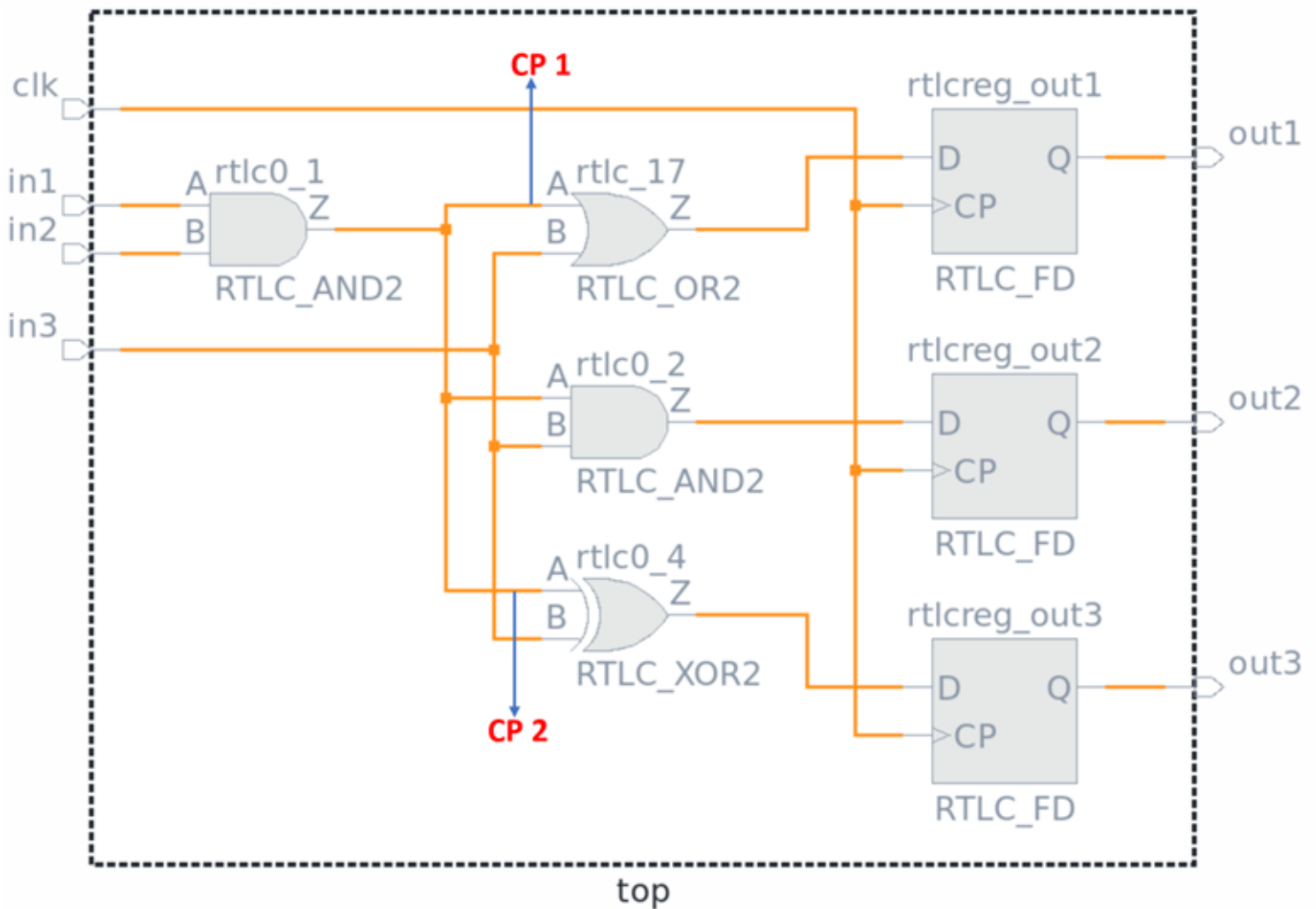
#### Example 4

This example shows control point insertion on input pins driven by multiple branches of a sub-expression. In the original RTL, the “in1 & in2” expression has three branches. For the purpose of this example, control points are inserted on only two of the branches, which are highlighted in red in the following code. The other branch does not get a control point and is highlighted in green.

```
module top(input clk, input in1,in2,in3,
           output reg out1, out2, out3);
  always @(posedge clk) begin
    out1 <= (in1 & in2) | in3;
    out2 <= (in1 & in2) & in3;
    out3 <= (in1 & in2) ^ in3;
  end
endmodule
```

The following figure shows the corresponding gate logic with the two control point locations marked in red.

**Figure 5-6. Example Gate Logic with Two Control Points**



The tool modifies the RTL to insert the two control points as function calls, which are highlighted in red. The unmodified branch of the original expression is highlighted in green.

```

module top(
  input clk, input in1,in2,in3,
  output reg out1, out2, out3, input wire cp_en);
wire [0:0] tessent_persistent_cell_cp_test_in_1_0,
          tessent_persistent_cell_cp_test_in_1_00;
always @(posedge clk) begin
  out1 <= (InsertControlPointORTTypeSV09((in1 & in2), cp_en,
    tessent_persistent_cell_cp_test_in_1_0) | in3);
  out2 <= ((in1 & in2) & in3);
  out3 <= (InsertControlPointORTTypeSV09((in1 & in2), cp_en,
    tessent_persistent_cell_cp_test_in_1_00) ^ in3);
end
top_tessent_dff_cp tessent_persistent_cell_cp_dff_instance_1_0_0(
  .clk(clk), .ff_out(tessent_persistent_cell_cp_test_in_1_0)
);
top_tessent_dff_cp tessent_persistent_cell_cp_dff_instance_1_0_0_1(
  .clk(clk), .ff_out(tessent_persistent_cell_cp_test_in_1_00)
);
function logic InsertControlPointORTTypeSV09(input logic sig_in,
  input logic TM,
  input logic test_in);
  return (TM & test_in) | sig_in;
endfunction
endmodule

```

## How to Preserve DFT Logic Inserted at RTL

RTL insertion adds new DFT logic, such as test points, X-bounding logic, and dedicated wrapper cells, directly into the design. In a gate-level insertion flow, the tool stitches the DFT logic into scan chains in the same run in which it is inserted. In an RTL insertion flow, the tool inserts the DFT logic before the scan chains are connected, and you must explicitly direct the synthesis tool to preserve the DFT logic.

For example, control point registers have a holding path from output to input, while observe point registers are simply connected to, and act as, a sink to signals fanning out from functional logic. Synthesis tools are likely to view these cells as static and unused (respectively) and optimize them away if they are not explicitly maintained through the synthesis process.

To avoid this situation, the synthesis script must be modified to preserve all boundaries and content of the DFT logic inserted at RTL. You can control the naming of the inserted logic to aid in identification.

### Prerequisites

- Predefined DFT logic generated into the following sub-directory of <TSDB output directory>/instruments:
  - <current\_design\_name>\_<design\_id\_name>\_rtl\_scan\_dft\_logic.instrument

## Optional Method to Customize Naming

By default, the tool uses “tessent\_persistent\_cell\_” as a prefix in the names of the logic it inserts. You can customize this prefix by using the following method:

- **Configuration-based specification** — Steps:
  - a. Create an empty DftSpecification configuration wrapper and copy the newly created wrapper object to the variable “spec” to customize the specification later.
  - b. Set a new value for DftSpecification/persistent\_cell\_prefix. For example:

```
set spec [create_dft_specification]
set_defaults_value DftSpecification/persistent_cell_prefix \
    testEx_
process_dft_specification
```

## Methods for Preserving Instances

You must decide whether to manually add commands to the Synopsys Design Compiler® synthesis script (if you have not run the [extract\\_icl](#) command to generate an SDC file) or to use the Tessent SDC created as part of the Tessent Shell flow (recommended).

- **Modify the synthesis script** — Steps:
  - a. Add the following commands to the synthesis script:

```
set_test_point_insertion_options -persistent_cell_prefix testEx_
set rtl_test_points [get_cells testEx_* -hier]
set_preserve_instances $rtl_test_points
set_boundary_optimization $rtl_test_points false
```
  - b. Run Design Compiler.
- **Use the Tessent SDC and the tessent\_get\_preserve\_instances proc** — Steps:
  - a. Insert DFT logic into the RTL design using the [process\\_dft\\_specification](#) command.
  - b. Re-extract the Tessent SDC using the [extract\\_sdc](#) command.
  - c. Follow the instructions in the “[SDC Design Synthesis With Tessent Shell](#)” section, and refer to the “[Synthesis Helper Procs](#)” topic for more information.

### Example

This example adds the following commands to the synthesis script:

```
source ../tsdb_outdir/dft_inserted_designs \
  cb_oc_rtl1.dft_inserted_design/cb_oc.sdc
source <TSDB output directory>/dft_inserted_designs/ \
  <current_design_name>_<design_id_name>.dft_inserted_design/ \
  <current_design_name>_<design_id_name>.sdc
tessent_set_default_variables
set_preserve_instances [tessent_get_preserve_instances scan_insertion]
set_boundary_optimization$preserve_instances true
set_ungroup $preserve_instances false
set_boundary_optimization [tessent_get_optimize_instances] true
set_size_only -all_instances [tessent_get_size_only_instances]
set_app_var compile_enable_constant_propagation_with_no_boundary_opt \
  false
set_app_var compile_seqmap_propagate_high_effort false
set_app_var compile_delete_unloaded_sequential_cells false
```

## How to Report Logic Excluded From Test Point Analysis

Use the “`report_notest_points -tool_identified`” command to report areas of logic excluded from the test point analysis.

The “`report_notest_points`” command by default reports the circuit points you mark with the “`add_notest_points`” command. However, with the optional `-tool_identified` switch, it prints the logic areas that the tool excluded according to your “`set_rtl_dft_analysis_options`” command settings.

The report shows the reason and type for each pin excluded at the gate level after quick synthesis. The following are the reasons and their meanings:

- **constant\_cone** — logic tied to a constant 0 or 1.
- **pins\_in\_floating\_logic** — all unused logic that is not observed by any primary output ports.
- **design\_ware\_internal** — logic excluded because it is part of a macro that requires synthesis by Synopsys DesignWare®.
- **not\_editable\_in\_rtl** — logic excluded because it is not editable RTL. For example, control logic or logic inside functional blocks.

The tool sets the `no_control_reason` and `no_observe_reason` predefined object attributes to the reason literal when the predefined attributes `no_control_point` and `no_observe_point` are set to true.

The list of reasons in this section applies only to the RTL flow. Other `no_control_reason` and `no_observe_reason` values apply to the gate-level flow. For more information, refer to “[Object Attributes](#)” on page 46.

## Examples

### Example 1

This example reports the logic excluded from test point analysis:

```
set_tsdb_output_directory tsdb_outdir
set_context dft -test_points -rtl

read_verilog -f ../data/filelist.f
read_verilog ../prerequisites/insert_test_point_logic.tshell/rtl/tp.v

set_current_design lifo_top

set_design_level physical
add_black_box -auto

add_clocks 1 rbu_reset_n
add_clocks 1 reset_n
add_clocks 0 clk

set_test_point_type {lbist_test_coverage edt_pattern_count}
set_rtl_dft_analysis_options -exclude_floating_logic on \
    -exclude_tied_logic on -nodes_available_for_analysis editable

check_design_rules

analyze_test_points
report_notest_points -tool_identified
```



| Pin name                                        | Type             | Reason                  |
|-------------------------------------------------|------------------|-------------------------|
| -----                                           | -----            | -----                   |
| /i_scan_clk_216_pad                             | control, observe | clock_cone              |
| /i _ s c a n _ c _ 108 _ pad                    | control, observe | clock_cone              |
| /i_pinmux_clock_108_enable                      | control, observe | pins_in_floating_logic  |
| /i_mbist_control[14]                            | control, observe | pins_in_floating_logic  |
| /i_mbist_control[13]                            | control, observe | pins_in_floating_logic  |
| /i_mbist_control[12]                            | control, observe | pins_in_floating_logic  |
| /ijtag_ce                                       | control, observe | tessent_ip_instance     |
| /ijtag_se                                       | control, observe | tessent_ip_instance     |
| /edt_update                                     | control, observe | pin_with_constant_value |
| /mjc_edge/mjc_fcb_inst/REV_MOD_07/CONST/o       | control, observe | constant_cone           |
| /mjc_edge/mjc_fcb_inst/REV_MOD_07/CONST/ob      | control, observe | constant_cone           |
| /mjc_edge/mjc_fcb_inst/REV_MOD_06/CONST/o       | control, observe | constant_cone           |
| /mjc_edge/mjc_fcb_inst/REV_MOD_06/CONST/ob      | control, observe | constant_cone           |
| /mjc_edge/mjc_bic_inst/.../u_tree_chroma/U919/Z | control, observe | pin_with_constant_value |
| /mjc_edge/mjc_bic_inst/.../u_tree_chroma/U920/Z | control, observe | pin_with_constant_value |
| /mjc_edge/mjc_bic_inst/.../pixel_0_mult/U272/Z  | control, observe | pin_with_constant_value |
| /mjc_edge/mjc_bic_inst/.../pixel_0_mult/U273/Z  | control, observe | pin_with_constant_value |
| /mjc_edge/mjc_bic_inst/.../pixel_1_mult/U272/Z  | control, observe | pin_with_constant_value |
| /mjc_edge/mjc_bic_inst/.../u_tree_chroma/U517/Z | control, observe | design_ware_internal    |
| /mjc_edge/mjc_bic_inst/.../u_tree_chroma/U980/Z | control, observe | design_ware_internal    |
| /mjc_edge/mjc_bic_inst/.../u_tree_chroma/U912/Z | control, observe | design_ware_internal    |
| /mjc_edge/mjc_bic_inst/.../u_tree_chroma/U871/Z | control, observe | design_ware_internal    |
| /mjc_edge/mjc_bic_inst/.../u_tree_chroma/U625/Z | control, observe | design_ware_internal    |

## Example 2

The following example uses attribute introspection such as the “[report\\_attributes](#)” command to show the value on any gate-pin/port/net/pin objects:

```
ANALYSIS> report_attributes axi_lfm_awprot[0]
```

```

Attribute Definition Report
Attributes defined for object 'axi_lfm_awprot[0]':
Name                                     Value                                     Inheritance
-----
base_class                             4_state_unsigned                        -
base_type                               wire                                    -
bus_left_index                          2                                       -
bus_name                                axi_lfm_awprot                          -
bus_range_direction                     down                                    -
bus_right_index                         0                                       -
bus_width                               3                                       -
design_hierarchy_level                   1                                       net
direction                              output                                  -
gate_pin_id                             66087.1                                -
has_functional_source                   true                                    -
has_rtl                                 false                                   net
index                                   0                                       -
is_bus                                  true                                    -
is_created                              false                                   -
is_non_editable                         false                                   -
is_non_editable_reason                  -                                       -
is_valid                                true                                    -
leaf_name                               axi_lfm_awprot[0]                       net
library_name                            work                                    -
master_module_name                      lifo_top                                -
module_name                             lifo_top                                -
name                                     axi_lfm_awprot[0]                       -
no_control_point                        true                                    -
no_control_reason                       pin_with_constant_value                 -
no_observe_point                        true                                    -
no_observe_reason                       pin_with_constant_value                 -
object_id                               port:6                                  -
object_type                             port                                    -
parent_bundle_name                      axi_lfm_awprot                          -
parent_instance                         net                                     net
parent_is_hard_module                   false                                   net
post_synthesis_name                     axi_lfm_awprot[0]                       -
pre_synthesis_leaf_name                 axi_lfm_awprot[0]                       net
pre_synthesis_name                      axi_lfm_awprot[0]                       -
root_bundle_name                       axi_lfm_awprot                          -
select_part_value_list                  {0}                                     -
tie_value                               -

```

## X-Bounding Analysis and Insertion at RTL

Starting from RTL, the tool performs quick synthesis to generate a flat model for X-bounding analysis. It stores the results of the analysis in the TSDB. The tool performs the X-bounding insertion in RTL when you run the `process_dft_specification` command.

The “[analyze\\_xbounding](#)” command is enabled in the `dft` (with no sub-context) context with both `-rtl` and `-no_rtl` options and the analysis system mode. You can control the process with the “[set\\_xbounding\\_options](#)” command and see the results with the “[report\\_xbounding](#)” command.

---

**Note**

 These commands are permitted only if “[set\\_dft\\_specification\\_requirements -logic\\_test](#)” is set to on. This ensures definition and controllability requirements are met for clocks, sets, and resets during pre-DFT DRC.

---

To insert other instruments but not the X-bounding logic after the X-bounding analysis is done, you must clear the analysis results using the “[analyze\\_xbounding -reset](#)” command before running the [process\\_dft\\_specification](#) command.

The tool considers all flip-flops as scannable, unless they are explicitly declared as non-scan. Flip-flops in X-bounding analysis have the following capabilities:

1. They can be used to drive the inserted multiplexer gates.
2. They can observe Xs.

## X-Bounding Logic

The tool inserts the following X-bounding logic:

1. A flip-flop with a multiplexer that intercepts an X-source when the corresponding enable signal is enabled (when you specify the “[set\\_xbounding\\_options -connect\\_to new\\_scan\\_cell](#)” option).
2. A multiplexer that intercepts an X-source when the corresponding enable signal is enabled, with the other input connected to an existing potential scan cell (when the “[set\\_xbounding\\_options -connect\\_to](#) is set to [existing\\_scan\\_cell](#)”, which is the default).
3. An AND/OR gate intercepting the existing connection, with the relevant enable signal controlling the gate (if the tool finds a flip-flop to drive the multiplexer or when you specify “[set\\_xbounding\\_options -xbounding\\_enable -connect\\_to constant\\_value](#)”).
4. A clock gater fix. If an X propagates to a functional enable pin of a clock gater, then the tool connects the clock gater’s `test_enable` to the X-bounding enable signal (if you specify “[set\\_xbounding\\_options -bound\\_clock\\_gater\\_enables](#)”).
5. A recirculating feedback path with a multiplexer with an inverter on the D input of the destination scan cells of false and multicycle paths. The inversion ensures that the destination scan cell output has a transition during a broadside test to achieve higher coverage. Refer to “[False and Multicycle Paths Handling](#)” in the *Hybrid TK/LBIST Flow User’s Manual*.

## X-Sources

The tool considers the following as potential sources of unknown values:

1. Unconstrained primary inputs, except for test-related ports (such as DFT signals, ICL ports, and so on).

2. Bus-related X-sources (based on E9-E11 DRC handling setting).
3. Non-scan flops specified with `add_nonscan_instances`.
4. Black boxes.
5. False and multicycle paths.
6. Memories are considered sources of unknown values. However, the tool forces the `memory_bypass_en` DFT signal to “on”, which prevents those Xs from propagating to scan cells if the memory has a bypass.

Instruments with the `keep_active_during_scan` attribute set to true are not considered X-sources.

If boundary scan is present, you must put it in the shift mode using input constraints to prevent redundant bounding of the wrapped primary inputs. Alternatively, you can exclude the inputs with the “`set_xbounding_options -exclude`” command. The boundary scan cells are not considered X-sources because all flops are considered scannable (unless defined non-scan).

Ports and pins with DFT control points of `async_set_reset` type that are added with `add_dft_control_point` or by `DFT_C9` block the X propagation.

The tool can bound the unknown states introduced by false and multicycle paths when you read an SDC file before starting the analysis. See “[Limitations of RTL DFT Analysis and Insertion](#)” on page 173 for details about SDC file parsing.

Synthesis typically optimizes unused (constrained or floating) logic. Adding X-bounding multiplexers in such areas can prevent synthesis optimizations and lead to elevated area overhead. Consequently, the tool tries to mimic synthesis by forcing undriven logic to 0 and excluding these pins from X-bounding. Such locations have the `no_control_reason` attribute set and are excluded from DFT analysis. Floating logic is also analyzed and marked with the `no_control_reason` attribute. The tool makes sure not to use the flip-flops that are only driving floating logic, or that are driven by undriven logic, as a source of non-X input of X-bounding multiplexers.

## X-Bounding Enable Signal

The tool selects the first available DFT signal as the enable signal for X-bounding logic, in the following order:

1. `mcp_bounding_en` (only for multicycle path bounding logic)
2. `x_bounding_en`
3. `lbist_en`

## Non-editable Nodes

Some regions of the RTL design cannot be modified. The tool marks them with `no_control_point/no_observe_point` attributes, and the corresponding `no_control_point_reason` and `no_observe_point_reason` attributes.

# Wrapper Analysis and Dedicated Wrapper Cell Insertion at RTL

When starting from RTL, the tool performs quick synthesis to generate a flat model for wrapper analysis. The tool performs the dedicated wrapper cell insertion in RTL when you run the `process_dft_specification` command.

The “[analyze\\_wrapper\\_cells](#)” command runs in analysis mode in the dft (with no sub-context) context for both `-rtl` and `-no_rtl` options. You can control the wrapper analysis with the “[set\\_wrapper\\_analysis\\_options](#)” and “[set\\_dedicated\\_wrapper\\_cell\\_options](#)” commands.

---

### Note



These commands are permitted only if “[set\\_dft\\_specification\\_requirements -logic\\_test](#)” is set to on. This ensures definition and controllability requirements are met for clocks, sets, and resets during pre-DFT DRC.

---

You can inspect the analysis results using the “[report\\_wrapper\\_cells](#)” command or by introspection. The tool does not insert dedicated wrapper cells if you run the “`analyze_wrapper_cells -reset`” command before the [process\\_dft\\_specification](#) command.

When you run the `process_dft_specification` command, the tool writes the locations of the dedicated wrapper cells to the `dft_info_dictionary` in the corresponding TSDB directory. If you use a third-party scan insertion tool, you need to identify dedicated wrapper cells for scan stitching. The following example shows how to get information for dedicated wrapper cells after the “[extract\\_icl](#)” command is complete:

```
format_dictionary [dict get [get_dft_info_dictionary] dedicated_wrapper_cells]
```

```

dft_signals {
  async_set_reset_dynamic_disable {
    connection_node_name tp_cell_async_set_reset_dynamic_disable/y
    connection_node_type pin
  }
  ...
}
dedicated_wrapper_cells {
  cpu_en {
    ts_0_dihw_2568smodpl_i {
      intercept cpu_en
      capture_window_behavior invert_hold
    }
  }
  dco_enable {
    ts_0_dohw_2350smodpl_i {
      intercept dco_enable
      capture_window_behavior invert_hold
    }
  }
  ...
}

```

## Features and Capabilities of RTL Wrapper Analysis and Insertion

The tool supports the following:

- Most switches of the `set_dedicated_wrapper_cell_options` and `set_wrapper_analysis_options` commands, including those that define thresholds, handle blackboxes, and exclude ports. Setting the dedicated wrapper cell location with respect to power isolation cells and level shifters does not apply in RTL. The tool does not support the `-input_module_name` and `-output_module_name` switches of the `set_dedicated_wrapper_cell_options` command in RTL.
- Insertion of all types of dedicated wrapper cells selected by the “`set_wrapper_analysis_options -capture_window_behavior`” command and switch: shifting, holding with inversion, and holding with programmable inversion.
- Scan segment handling.
- Clock source analysis to determine the clock source for dedicated wrapper cells.
- Tessent wrapper analysis identifies the constant, undriven, and floating logic that synthesis tools remove from the RTL during optimization. If wrapper analysis determines that a port connects only to this kind of logic, it ignores the port and reports it as “Optimized post-synthesis”. You can still force a dedicated wrapper cell for the port by using the “`set_dedicated_wrapper_cell_options on-ports`” command and switch.

## Limitations

The quick synthesis does not use multi-bit cells or flip-flop trays, unlike regular synthesis. For this reason, wrapper analysis does not account for multi-bit cells the same way it does in the “dft

-scan” context. In the “dft -scan” context, it accounts for their cost towards flip-flop threshold by dividing their size by the number of ports that reach it. In dft (with no sub\_context), you can alleviate the impact of this limitation by reducing input/output fanout/fan-in flip-flop thresholds using the `set_wrapper_analysis_options` command.

## Smart Uniquification for RTL Pro

RTL Pro can perform smart uniquification to help you manage the size and complexity of your design.

The “[set\\_insertion\\_options](#) -auto\_uniquify\_edited\_modules” command controls smart [uniquification](#) for RTL DFT insertion.

When the `-auto_uniquify_edited_modules` switch is set to “on”, the tool creates a new unique module for every instance that has test point, X-bounding, or dedicated wrapper cell DFT logic inserted.

When the `-auto_uniquify_edited_modules` switch is set to “when\_needed” (the default), a smart algorithm identifies where a new unique view is needed and whether the tool can share the same unique view by several instances. The tool creates a copy of a module and uniquifies it for each parameterized view rather than for each instance. For example, if all instances have DFT logic insertion locations and they are instantiated over two parameterized views, the number of unique copies created equals two, in addition to the original.

The tool distributes all DFT locations (such as test points, X-bounding logic, and dedicated wrapper cells) per elaborated views. Elaborated views are new modules created by design elaboration to represent the hierarchy of logic instances after propagating parameters and the System Verilog interface from the top module down.

Before starting insertion of the test logic into the RTL design, RTL Pro identifies all sets of instances of the same elaborated view that require the same test logic. If a set of instances has the same required test logic but some descendants do not, the tool creates different unique views.

## Example 1

This example shows the behavior of the “set\_insertion\_options -auto\_uniquify\_edited\_modules” command. The example design has 10 instances of the same module “DECODE” under the top module “TOP” as follows:

```
module TOP (.....);  
    DECODE #(.WITDH(8)) u1 (.....);  
    DECODE #(.WITDH(16)) u2 (.....);  
    DECODE #(.WITDH(8)) u3 (.....);  
    DECODE #(.WITDH(8)) u4 (.....);  
    DECODE #(.WITDH(8)) u5 (.....);  
    DECODE #(.WITDH(16)) u6 (.....);  
    DECODE #(.WITDH(8)) u7 (.....);  
    DECODE #(.WITDH(8)) u8 (.....);  
    DECODE #(.WITDH(8)) u9 (.....);  
    DECODE #(.WITDH(16)) u10 (.....);  
endmodule
```

After design elaboration, the tool creates two elaborated views for the “DECODE” module according to the WIDTH parameter.

- DECODE@PV1 for instances: u1, u3, u4, u5, u7, u8, u9
- DECODE@PV2 for instances: u2, u6, u10

The following scenarios show how the tool uniquifies the design depending on the test logic inserted and the setting you specified for the -auto\_uniquify\_edited\_modules switch:

### Scenario 1

The tool inserts ten control points implemented with OR gates. All control points are on the same output of the same expression (same logic).

In this case, the tool performs the following uniquification:

- **-auto\_uniquify\_edited\_modules on —**
  - 10 new unique views (DECODE\_1, DECODE\_2, ..., DECODE\_10)
- **-auto\_uniquify\_edited\_modules when\_needed —**
  - DECODE\_1 for instances {u1, u3, u4, u5, u7, u8, u9}
  - DECODE\_2 for instances {u2, u6, u10}

### Scenario 2

The tool inserts ten control points implemented with OR gates. All control points are on the same output of the same expression (same logic). The tool also inserts six observe points, with three observe points on the same location for u1, u2, and u3, and three other observe points on the same location for u4, u5, and u6.



In this case, the tool performs the following uniquification:

- **-auto\_uniquify\_edited\_modules on —**
  - 10 new unique views (DECODE\_1, DECODE\_2, ..., DECODE\_10)
- **-auto\_uniquify\_edited\_modules when\_needed —**
  - DECODE\_1 for instances {u1, u3}
  - DECODE\_2 for instances {u2, u6, u10}
  - DECODE\_3 for instances {u4, u5, u7, u8, u9}

If you enable test point sharing, the uniquification changes because the tool always inserts the common flip-flop at the common ancestor instance, as shown in Scenario 3.

### Scenario 3

As in Scenario 2, the tool inserts ten control points implemented with OR gates. All control points are on the same output of the same expression (same logic). The tool also inserts six observe points, with three observe points on the same location for u1, u2, and u3, and three other observe points on the same location for u4, u5, and u6.

This scenario enables test point sharing with the following command:

**set\_test\_point\_analysis\_options -shared\_control\_points\_per\_flop 5**

In this case, the tool creates two clusters of control points (CP) that share a single flip-flop for each cluster.

- Cluster 1 = {CP of u1, u2, u3, u4, u5}
- Cluster 2 = {CP of u6, u7, u8, u9, u10}

The tool performs the following uniquification:

- **-auto\_uniquify\_edited\_modules on —**
  - 10 new unique views (DECODE\_1, DECODE\_2, ..., DECODE\_10)
- **-auto\_uniquify\_edited\_modules when\_needed —**
  - DECODE\_1 for instances {u1, u3}
  - DECODE\_2 for instances {u2}
  - DECODE\_3 for instances {u6, u10}
  - DECODE\_4 for instances {u4}
  - DECODE\_5 for instances {u5, u7, u8, u9}

The tool identified the same test points for Scenario 2 and 3 but a different number of unique views because of test point sharing.

To have better control on uniquification when the test point sharing is enabled, use the [set\\_test\\_point\\_sharing\\_restrictions](#) command. For example, use the -module switch to restrict sharing to specific elaborated views only:

```
set_test_point_sharing_restrictions on -modules {DECODE@PV1 DECODE@PV2}
```

## Example 2

This example shows the behavior of the “set\_insertion\_options -auto\_uniquify\_edited\_modules” command on the following design, *top.sv*:

```
module M1 (input clk, input [3:0] in, output reg
out1,out2,out3,out4);
    always @(posedge clk) begin
        out1 <= in[0] & in[2];
    end
    M2 U3(clk, in, out2);
    M2 U4(clk, in, out3);
    M2 U5(clk, in, out4);
endmodule
module M2 (input clk, input [3:0] in, output reg out1);
    always @(posedge clk) begin
        out1 <= in[1] & in[3];
    end
endmodule
module top(input clk,input [3:0]in,output out1,
out2,out3,out4,out5,out6,out7,out8);
    M1 U1(clk, in, out1, out2,out3,out4);
    M1 U2(clk, in, out5, out6,out7,out8);
endmodule
```

The dofile for this example includes the following commands, where the gate pin “rtl0\_0/Z” represents the output pin of expression “in[1] & in[3]”:

```
add_control_points -clock U1/U3/clk -location U1/U3/rtl0_0/Z \
-enable cp_en -type and
add_control_points -clock U1/U4/clk -location U1/U4/rtl0_0/Z \
-enable cp_en -type and
add_control_points -clock U2/U3/clk -location U2/U3/rtl0_0/Z \
-enable cp_en -type and
add_control_points -clock U2/U4/clk -location U2/U4/rtl0_0/Z \
-enable cp_en -type and
```

The tool inserts four control points on the same RTL location. With “-auto\_uniquify\_edited\_modules when\_needed”, the tool creates two new unique views, M1\_1 and M2\_1. The following shows the RTL for *top.sv* after test point insertion:

```
module M1 (input clk, input [3:0] in,
          output reg out1,out2,out3,out4,
          input wire cp_en);
  always @(posedge clk) begin
    out1 <= in[0] & in[2];
  end
  M2_1 U3(clk, in, out2, cp_en);
  M2_1 U4(clk, in, out3, cp_en);
  M2 U5(clk, in, out4);
endmodule

module M1_1 (input clk, input [3:0] in,
            output reg out1,out2,out3,out4,
            input wire cp_en);
  always @(posedge clk) begin
    out1<= in[0] & in[2];
  end
  M2_1 U3(clk, in, out2, cp_en);
  M2_1 U4(clk, in, out3, cp_en);
  M2 U5(clk, in, out4);
endmodule

module M2 (input clk, input [3:0] in, output reg out1);
  always @(posedge clk) begin
    out1<= in[1] & in[3];
  end
endmodule

module M2_1 (input clk, input [3:0] in,
            output reg out1, input wire cp_en);
  wire [0:0] tessent_persistent_cell_cp_test_in_1_0;
  always @(posedge clk) begin
    out1<= InsertControlPointANDTypeV95((in[1] & in[3]), cp_en,
    tessent_persistent_cell_cp_test_in_1_0);
  end
  top1_rtl_tessent_dff_cp
    tessent_persistent_cell_cp_dff_instance_1_0_0(.clk(clk),
    .ff_out(tessent_persistent_cell_cp_test_in_1_0));
  function InsertControlPointANDTypeV95;
    input sig_in;
    input TM;
    input test_in;
    begin
      InsertControlPointANDTypeV95 = (~TM | test_in) & sig_in;
    end
  endfunction
endmodule
```

```
module top(input clk,input [3:0]in,
           output out1,out2,out3,out4,out5,out6,out7,out8,
           input wire cp_en);
  M1_1 U1(clk, in, out1, out2,out3,out4, cp_en);
  M1 U2(clk, in, out5, out6,out7,out8, cp_en);
endmodule
```

## Incremental Insertion

After synthesizing the DFT logic inserted at RTL, you can perform incremental insertion at the gate level for fine-tuning before test pattern generation.

You can perform incremental test point analysis at the gate level. The [Example](#) in this section shows how to use it to increase LBIST test coverage.

You can perform incremental X-bounding at the gate level for these reasons:

- Bound any Xs introduced by synthesis.
- Bound false and multicycle paths after reading in a gate-level SDC file. For more information, refer to “[False and Multicycle Paths Handling](#)” in the *Tessent Hybrid TK/LBIST Flow User’s Manual*.

You can perform incremental wrapper cell analysis at the gate level. Refer to “[Automatic Identification of Pre-Existing Dedicated Wrapper Cells](#)” in the *Tessent Scan and ATPG User’s Manual* for detailed information about the following topics:

- How the tool handles dedicated wrapper cells inserted at RTL.
- The behavior of the [analyze\\_wrapper\\_cells](#) command if you run it again in the “dft -scan” context on a gate-level design.

### Example

This example first inserts test points in an RTL design and then incrementally inserts test points at the gate level. It focuses on the commands related to test points only. For more information on the overall flow, refer to “[Tessent Shell Workflows](#)” on page 175.

Define DFT-specific signals in the first insertion pass, including the following signals for test points:

```
add_dft_signals observe_test_point_en -create_with_tdr
add_dft_signals control_test_point_en -create_with_tdr
```

The following code shows the test point insertion at RTL with the goal of increasing LBIST test coverage:

```
set_context dft -test_points -rtl -design_id rtl1_tp
read_design lifo_top -design_id rtl1 -verbose
set_current_design lifo_top
add_black_boxes -auto
set_test_point_types lbist_test_coverage
set_test_point_analysis_options -test_coverage_target 90
set_test_point_insertion_options -control_point_enable
lifo_top_rtl1_tessent_tdr_sri_ctrl_inst/control_test_point_en \
  -observe_point_enable \
  lifo_top_rtl1_tessent_tdr_sri_ctrl_inst/observe_test_point_en
check_design_rules
set_test_point_analysis_options -capture_per_cycle_observe_points on
set_test_point_analysis_options -minimum_shift_length 4
analyze_test_points
create_dft_specification
process_dft_specification
```

The following tool transcript shows the results of the [analyze\\_test\\_points](#) command for RTL insertion:

```
// command: analyze_test_points
// Identifying locations of x-bounding muxes prior to test point
// analysis.
// Note: Test points cannot be inserted at 72057 (63.0%) gate-pins.
// 10978 ( 9.6%) gate-pins are located on clock lines or on the
// scan path.
// 23033 (20.1%) gate-pins are constant due to inputs that are
// constrained to 0 or 1 or as a result of the test setup procedure.
// 32061 (28.0%) gate-pins are in excluded uneditable regions.
// Warning: This design may not be suitable for test point analysis
// unless you can take the following action(s) -
// a) Reduce the number of uneditable regions by improving the RTL.
// For guidance, users can analyze the RTL complexity of the whole
// design using the command "report_rtl_complexity".
//
// Test Coverage Report before Test Point Analysis
// -----
// Target number of random patterns          10000
//
// Total Number of Faults                    225688
// Testable Faults                          195086 ( 86.44%)
// Logic Bist Testable                      169632 ( 75.16%)
// Blocked by xbounding                     9200 ( 4.08%)
// Uncontrollable/Unobservable              5586 ( 2.48%)
//
// Estimated Maximum Test Coverage           86.95%
// Estimated Test Coverage (pre test points) 54.86%
// Estimated Relevant Test Coverage (pre test points) 58.04%
//
```

```
//
// Incremental Test Point Analysis
// -----
// TPs 100 = 61 (CP) + 39 (OP), Est_RTC 68.68
// TPs 200 = 112 (CP) + 88 (OP), Est_RTC 70.13
...
// TPs 1700 = 178 (CP) + 1522 (OP), Est_RTC 81.37
// TPs 1800 = 178 (CP) + 1622 (OP), Est_RTC 81.44
//
// Incremental optimization to find more effective test points is in
// progress. The final distribution of control and observe points
// may change.
//
// Test Coverage Report after Test Point Analysis
// -----
// Target number of random patterns          10000
//
// Total Number of Faults                    225688
//   Testable Faults                        195086 ( 86.44%)
//   Logic Bist Testable                    169632 ( 75.16%)
//   Blocked by xbounding                     9200 (  4.08%)
//   Uncontrollable/Unobservable             5586 (  2.48%)
//
// Estimated Test Coverage (post test points) 79.51%
// Estimated Relevant Test Coverage (post test points) 84.11%
//
//
// Test point analysis completed: no more useful test points could be
// identified.
// Total inserted test points                1473
//   Control Points                          274
//   Observation scan Test Points            1199
//   Maximum control point per path          4
//   CPU_time (secs)                         15.7
```

After insertion, the RTL design is ready for synthesis. At the gate level, perform fault simulation of the LBIST patterns to evaluate the test coverage. The following tool transcript shows the results of fault simulation:

```
// command: simulate_patterns -source bist -store_patterns all
// Warning: Current sequential depth is not large enough to apply named
// capture procedures.
//       Automatically increase the sequential depth to be 2.
// -----
// Simulation performed for #gates = 93375 #faults = 234443
// Run fault simulation and store all patterns.
//   pattern source = 1000 LBIST-OST patterns
// -----
// #patterns test #faults #faults # eff. # test process
// simulated coverage in list detected patterns patterns CPU time
// Note: Pattern classification is automatically turned off.
// 64 73.28% 54435 180008 64 64 5.19 sec
// 128 74.66% 49786 4649 64 128 8.28 sec
...
// 960 79.07% 34718 1791 29 960 47.49 sec
// 1000 79.13% 34508 210 15 1000 50.09 sec
// 21 faults were identified as detected by implication.
```

The result is that test point insertion at RTL added 1473 test points, which increased the estimated test coverage from 54.86% to 79.51%. The fault simulation confirmed the LBIST test coverage at 79.13% for 1000 patterns simulated.

To further increase LBIST test coverage, this example script performs incremental test point insertion in the design at the gate level:

```
set_context dft -scan -test_points -no_rtl -design_id gate_tpi_1
read_cell_library adk.tcelllib
read_verilog lifo_top.vg
read_design lifo_top -design_id rtl1_tp -no_hdl -verbose
set_current_design lifo_top
add_black_boxes -auto
set_test_point_types lbist_test_coverage
set_test_point_analysis_options -test_coverage_target 90
set_test_point_insertion_options -control_point_enable \
    lifo_top_rtl1_tessent_tdr_sri_ctrl_inst/control_test_point_en \
    -observe_point_enable \
    lifo_top_rtl1_tessent_tdr_sri_ctrl_inst/observe_test_point_en
set_test_point_analysis_options -capture_per_cycle_observe_points on
set_test_point_analysis_options -minimum_shift_length 4
check_design_rules
analyze_test_points
insert_test_logic
```

The following tool transcript shows the test points added during gate-level incremental insertion:

```
// command: analyze_test_points
// Identifying locations of x-bounding muxes prior to test point
// analysis.
// Note: Test points cannot be inserted at 37804 (29.4%) gate-pins.
// 12855 (10.0%) gate-pins are located on clock lines or on the
// scan path.
// 17191 (13.4%) gate-pins are constant due to inputs that are
// constrained to 0 or 1 or as a result of the test setup procedure.
// 5746 ( 4.5%) gate-pins are restricted by add_notest_points
// command(s) and/or no_control_point/no_observe_point attribute
// settings.
// Warning: Consider taking the following action(s) -
// a) Reduce the number of user-specified no_control_point
// and no_observe_point locations.
//
```

```
// Test Coverage Report before Test Point Analysis
// -----
// Target number of random patterns          10000
//
// Total Number of Faults                    310642
//   Testable Faults                        232586 ( 74.87%)
//   Logic Bist Testable                    204504 ( 65.83%)
//   Blocked by xbounding                    9691 ( 3.12%)
//   Uncontrollable/Unobservable            6788 ( 2.19%)
//
// Estimated Maximum Test Coverage            87.93%
// Estimated Test Coverage (pre test points)  70.00%
// Estimated Relevant Test Coverage (pre test points) 73.68%
//
// Incremental Test Point Analysis
// -----
// TPs 100 = 33 (CP) + 67 (OP), Est_RTC 87.50
//
// Incremental optimization to find more effective test points is in
// progress. The final distribution of control and observe points may
// change.
//
// Test Coverage Report after Test Point Analysis
// -----
// Target number of random patterns          10000
//
// Total Number of Faults                    311012
//   Testable Faults                        232103 ( 74.63%)
//   Logic Bist Testable                    204243 ( 65.67%)
//   Blocked by xbounding                    9691 ( 3.12%)
//   Uncontrollable/Unobservable            6574 ( 2.11%)
//
// Estimated Test Coverage (post test points) 86.07%
// Estimated Relevant Test Coverage (post test points) 90.60%
//
// Test point analysis completed: target estimated test coverage has been
// achieved.
// Total inserted test points                191
//   Control Points                          78
//   Observation scan Test Points            113
//   Maximum control point per path         3
//   CPU_time (secs)                        5.6
//
// command: insert_test_logic
// =====
Test Logic Insertion Summary:
// =====
// Structural Data:
// -----
// Added top-level port count:              0
// Added instance count:                    950
//
// Logical Data:
// -----
// Added control testpoint logic count:      272
// Added observation scan testpoint logic count: 113
```



After incrementally inserting test points, you can fault simulate the LBIST patterns. Here is an excerpt from the fault simulation tool transcript:

```
// command: simulate_patterns -source bist -store_patterns all
// Warning: Current sequential depth is not large enough to apply named
// capture procedures.
//           Automatically increase the sequential depth to be 2.
// -----
// Simulation performed for #gates = 94868 #faults = 237762
// Run fault simulation and store all patterns.
// pattern source = 1000 LBIST-OST patterns
// -----
// #patterns test #faults #faults # eff. # test process
// simulated coverage in list detected patterns patterns CPU time
// Note: Pattern classification is automatically turned off.
// 64 80.63% 30232 206893 64 64 5.25 sec
// 128 82.34% 24377 5855 64 128 7.50 sec
// ...
// 960 86.32% 10742 318 36 960 32.62 sec
// 1000 86.35% 10643 99 18 1000 34.29 sec
// 35 faults were identified as detected by implication.
```

The result is that incremental insertion at the gate level put in an additional 191 test points, further increasing the estimated test coverage from 79.51% to 86.07%. The fault simulation confirmed the LBIST test coverage as 86.35% for 1000 patterns simulated.

## Limitations of RTL DFT Analysis and Insertion

Tessent Shell RTL DFT analysis and insertion capabilities have the following limitations:

- At RTL, you can perform test point insertion in incremental mode, but we recommend that you insert dedicated wrapper cells and X-bounding logic with a single call to the [process\\_dft\\_specification](#) command. At the gate-level, you can incrementally insert dedicated wrapper cells, X-bounding logic, and test points after RTL insertion.
- The tool identifies dedicated wrapper cell structures that are inserted by Tessent in RTL designs. These are only a subset of dedicated wrapper cells that are part of IEEE Std 1500.
- The tool does not support insertion of dedicated wrapper cells and X-bounding logic in VHDL.
- Parts of the design marked as non-editable RTL for test points are treated as non-editable for dedicated wrapper cells and X-bounding logic, too.
- You must use the `no_control_point` attribute to mark the parts of the design where you do not want the tool to insert X-bounding logic because the [add\\_notest\\_points](#) command is not available.
- If X-bounding is necessary in the logic analyzed but not marked with the `no_control_point` attribute, and the logic is in non-editable RTL, the tool reports a

violation of the [XB3](#) DRC rule. You can remedy this by marking the logic with the `no_control_point` attribute.

- The “-use\_scan\_cells {off | on}” option to the [set\\_test\\_point\\_insertion\\_options](#) and [set\\_xbounding\\_options](#) commands is not supported in the dft (with no sub-context) context.
- The -input\_module\_name and -output\_module\_name switches of the [set\\_dedicated\\_wrapper\\_cell\\_options](#) command are not supported in RTL.
- The -dedicated\_wrapper\_cells\_location and -identify\_existing\_dedicated\_wrappers switches of the [set\\_wrapper\\_analysis\\_options](#) command are not supported in RTL.
- The -prefer\_scan\_cells switch of the [set\\_test\\_point\\_insertion\\_options](#) command is not supported in RTL.
- The tool only supports a subset of the SDC file syntax. See the [read\\_sdc](#) command in the *Tessent Shell Reference Manual* for more information. The tool may not support the attributes and commands specific to third-party tools. A possible workaround is to read the SDC file in the synthesis tool and write out a preprocessed version after elaboration and before synthesis. However, this can result in tool-specific, post-synthesis design object names that may not match the design objects in Tessent, requiring some additional manual post-processing. Another alternative is to prepare a tool-independent SDC file by extracting the tool-specific queries and implementing them separately for Tessent and third-party tools.
- You must first create an ICL representation of your RTL design in the TSDB by running the [extract\\_icl](#) command before running the [extract\\_sdc](#) command to generate the `tessent_get_preserve_instances` procs in the SDC file. These procs can be used to preserve the inserted DFT logic during synthesis.

# Chapter 6

## Tessent Shell Workflows

---

Tessent Shell workflows are grouped into two main subflows: prelayout and postlayout. The prelayout DFT insertion workflow describes the process for using Tessent Shell for flat and hierarchical designs. The post-insertion validation workflow describes the flow for netlists that have completed placement and routing.

|                                                                                      |            |
|--------------------------------------------------------------------------------------|------------|
| <b>Tessent Shell Flow for Flat Designs .....</b>                                     | <b>176</b> |
| <b>Tessent Shell Flow for Hierarchical Designs.....</b>                              | <b>205</b> |
| <b>Tessent Shell Flow for Tiled Designs .....</b>                                    | <b>278</b> |
| <b>Tessent Shell Post-Layout Validation Flow.....</b>                                | <b>339</b> |
| <b>Testbench Generation and Simulation in RTL Mode .....</b>                         | <b>349</b> |
| <b>Hybrid TK/LBIST Flow for Flat Designs .....</b>                                   | <b>357</b> |
| <b>Running Multiple Load ATPG on Wrapped Core Memories with Built-In Self Repair</b> | <b>379</b> |
| <b>Built-In Self Repair in Hierarchical Tessent MemoryBIST Flow.....</b>             | <b>386</b> |

## Tessent Shell Flow for Flat Designs

In the RTL and scan DFT insertion flow for flat designs, you perform DFT insertion for the entire chip-level design.

The flat DFT implementation aligns with the physical implementation of the design. The flow consists of the following steps:

1. Performing DFT hardware insertion
2. Synthesis
3. Scan insertion (optional)
4. Gate-level ATPG

Understanding the RTL and scan DFT prelayout insertion flow for flat designs helps you manipulate hierarchical designs or implement a variation of the basic flow.

Refer to the following test case for a detailed usage example of the flow described in this section:

```
tessent_example_flat_flow_<software_version>.tgz
```

You can access this test case by navigating to the following directory:

```
<software_release_tree>/share/UsageExamples/
```

|                                                                                |            |
|--------------------------------------------------------------------------------|------------|
| <b>Overview of the RTL and Scan DFT Insertion Flow .....</b>                   | <b>177</b> |
| <b>First DFT Insertion Pass: Performing MemoryBIST and Boundary Scan .....</b> | <b>180</b> |
| <b>Second DFT Insertion Pass: EDT, Hybrid TK/LBIST, and OCC .....</b>          | <b>184</b> |
| Loading the Design .....                                                       | 185        |
| Specifying and Verifying the DFT Requirements .....                            | 187        |
| Creating the DFT Specification .....                                           | 190        |
| Generating the EDT, Hybrid TK/LBIST, and OCC Hardware .....                    | 194        |
| Extracting the ICL Module Description .....                                    | 194        |
| Generating ICL Patterns and Running Simulation .....                           | 195        |
| <b>Performing Synthesis.....</b>                                               | <b>196</b> |
| <b>Performing Scan Chain Insertion (Flat Design) .....</b>                     | <b>197</b> |
| <b>Performing ATPG Pattern Generation .....</b>                                | <b>199</b> |
| <b>Simulating LBIST Faults .....</b>                                           | <b>201</b> |
| <b>Considerations for Using Gate-Level Verilog Netlists .....</b>              | <b>203</b> |

## Overview of the RTL and Scan DFT Insertion Flow

Whether you are using a flat design or a hierarchical design, the RTL and scan DFT insertion flow requires you to use a two-pass insertion process to insert the DFT hardware.

As shown in the following figure, insert MemoryBIST and boundary scan prior to embedded deterministic test (EDT) or hybrid TK/LBIST (LBIST), and the on-chip clock controller (OCC). In this section, EDT refers to embedded deterministic test IP only; and LBIST refers to hybrid TK/LBIST IP, which also includes EDT.

---

### Note

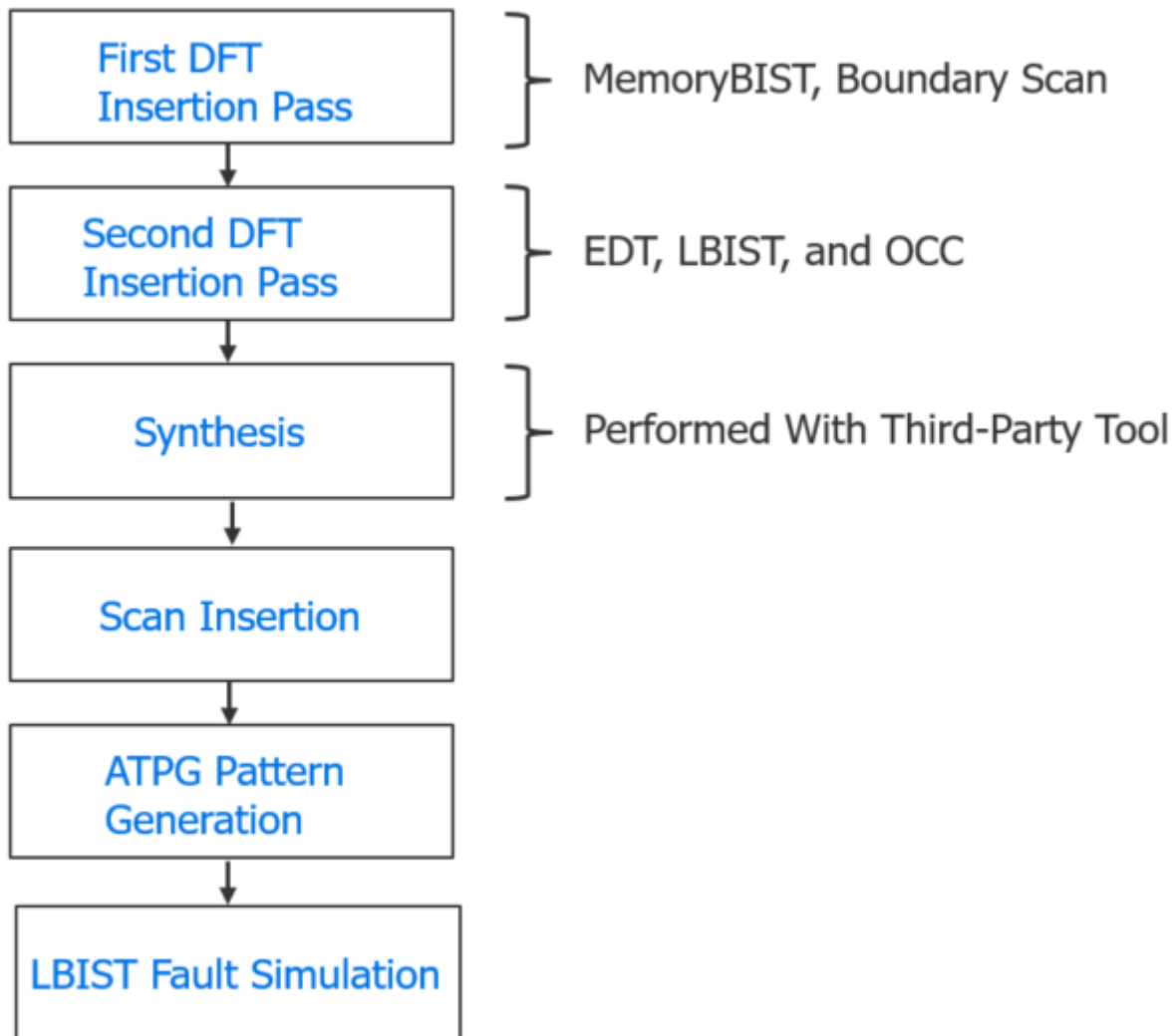


EDT is a subset of LBIST. You can choose EDT and no LBIST, but you cannot choose LBIST without it also including EDT.

---

The DFT logic you insert during the first DFT insertion pass gets tested by EDT or LBIST. When inserting DFT into an RTL design, you only need to run synthesis once after you have performed the two DFT insertion passes.

**Figure 6-1. Two-Pass Insertion Flow for RTL, Flat Designs**



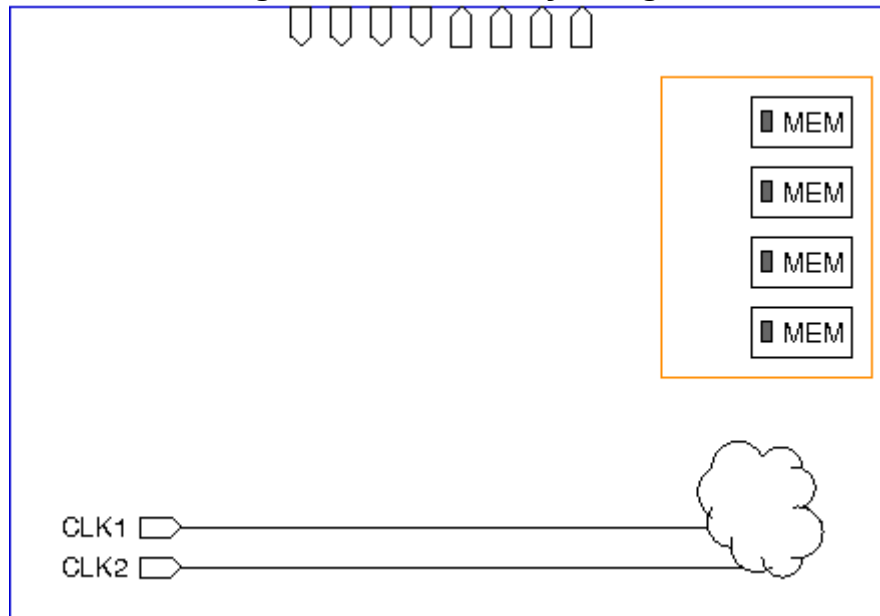
During the first pass, Tessent inserts the IJTAG network and any IJTAG instruments that have ICL descriptions. Refer to the command for details about this process.

During the second pass, the tool checks MemoryBIST's logic for rule compliance with the rest of the functional logic to prevent implications for the DFT signals and IJTAG network connections that could result in coverage loss and pattern count increase.

Optionally, you can perform scan insertion at the same time as synthesis. This does not affect how you perform DFT insertion, but you do lose some of the automation that Tessent Shell provides during ATPG pattern generation.

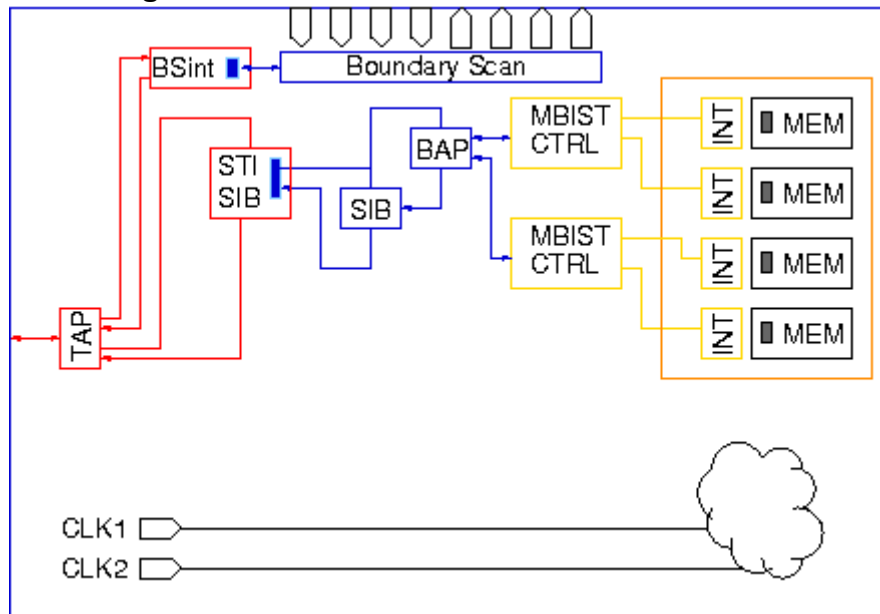
The following figures show an example of the progression of DFT hardware inserted into a DFT-ready design.

**Figure 6-2. DFT-Ready Design**



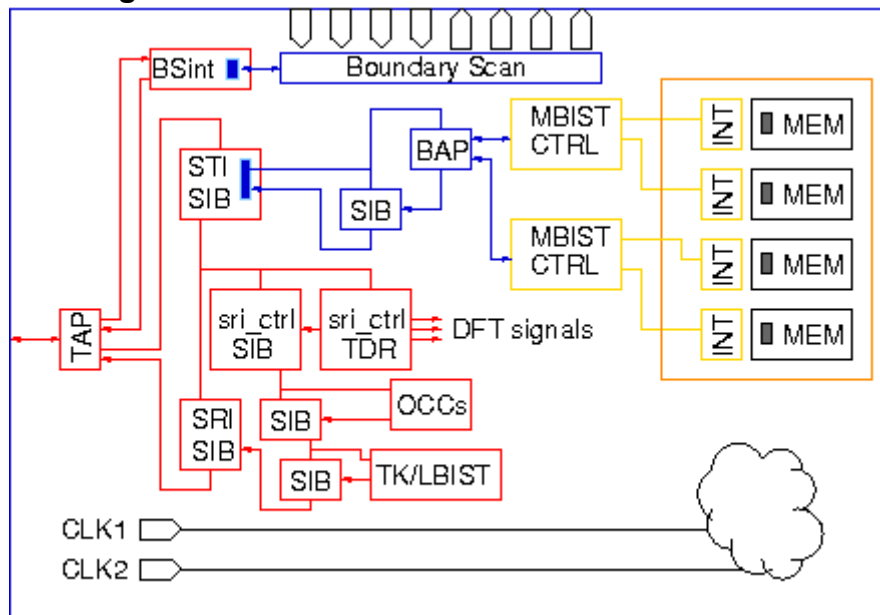
In the first DFT insertion pass, Tessent inserts the MemoryBIST and boundary scan hardware.

**Figure 6-3. After the First DFT Insertion Pass**



In the second DFT insertion pass, Tessent inserts the EDT or LBIST, and OCC hardware. You clock the MemoryBIST logic (shown in yellow in [Figure 6-4](#)) using the same functional clock that feeds the memories. The IJTAG network (blue) is scan tested using the IJTAG clock, which is the TCK clock. The TAP network (red) is not scan tested or is made non-scan during ATPG.

Figure 6-4. After the Second DFT Insertion Pass



## First DFT Insertion Pass: Performing MemoryBIST and Boundary Scan

Insert MemoryBIST and boundary scan prior to EDT and OCC so that you can accurately estimate the number of scannable sequential elements (often referred to as “flops”) to be tested. The number of flops determines the size of the EDT controller that Tessent generates in the second DFT insertion pass.

The flow for inserting MemoryBIST and boundary scan together is the same as inserting them separately. For details about this insertion pass flow, refer to:

- “[Getting Started](#)” in the *Tessent MemoryBIST User’s Manual*
- “[Getting Started With Tessent BoundaryScan](#)” in the *Tessent BoundaryScan User’s Manual*

### Notes About This Procedure

- The “[set\\_dft\\_specification\\_requirements -memory\\_test](#)” switch in the reference testcase is set to “On” even though this testcase has no memories in the top level. This requests that the memory clocks at the boundary of the child blocks receive design rule checks (DRCs) up through the top level. Although the [extract\\_icl](#) process would detect any issues, you would need to redo the DFT insertion to fix them. By performing the DRC before DFT, you can detect and correct those issues sooner.
- The line numbers used in this procedure refer to the command flow dofile in [Example 6-1](#) on page 182.



## Prerequisites

- To insert boundary scan, you must have an RTL design with instantiated I/O pads if you are using a chip-level design.
- For RTL netlists, you must have a Tessent cell library or the pad library for the I/O pads. For more information, refer to the [Tessent Cell Library Manual](#).

## Procedure

1. Load the RTL design data (see lines 1-7).
2. For the first insertion pass, set the [set\\_context](#) -design\_id switch. By convention, the identifier used for this is typically “rtl1”.

The -design\_id switch stores all the data associated with a particular DFT insertion pass in the TSDB. For the first pass, rtl1 contains the data for the MemoryBIST and boundary scan hardware, and for the JTAG network.

---

### Note

 rtl1 is the recommended naming convention for the design ID for the first insertion pass, but you can specify any name. Refer to “[Loading the Design](#)” for more information about setting the design ID.

---

3. Set the [set\\_dft\\_specification\\_requirements](#) command to “auto” for -memory\_test and “on” for -boundary\_scan. (See line 11.)

This tells Tessent Shell that you are generating the MemoryBIST and boundary scan hardware at the same time.

4. Set the design level (set\_design\_level chip, see line 12).

Refer to “[Design Levels](#)” for more information.

5. Identify test pins and apply options to special pins. (See lines 14-26.)
6. Apply the check\_design\_rules command to instruct the tool to leave setup mode and enter analysis mode.

If there are issues with the design, the tool remains in setup mode. (See line 31.)

7. Create the DFT specification. (See lines 33-43.)
  - a. To configure the functional pins so that they are shared as EDT channel input and output pins, add the required logic using the AuxiliaryInput ports and AuxiliaryOutput ports wrappers, respectively.

Functional pins can be shared as EDT channel pins. You must insert auxiliary input and output logic at the same time as you insert boundary scan to avoid cascading two multiplexers along the functional output paths. For additional information, refer to the [AuxiliaryInputOutputPorts](#) wrapper in the *Tessent Shell Reference Manual*.

**Note**

 You can also share test\_clock, scan\_en, and edt\_update pins with functional pins. The functional coverage is maintained when you use boundary scan during scan test.

8. Create the DFT hardware, IJTAG network connectivity, and input test patterns. (See lines 46-53.)
9. Run simulations to verify the design. (See lines 55-62.)

**Results**

For MemoryBIST, Tessent inserts the MemoryBIST controllers, interfaces, [BIST Access Port \(BAP\)](#), and [segment insertion bits \(SIBs\)](#). This hardware is later scan-tested using EDT, which you insert during the second insertion pass. In addition, Tessent automatically connects pre-existing scan testable instruments and scan resource instruments to the [Scan Tested Instrument \(STI\)](#) and [Scan Resource Instrument \(SRI\)](#) sides of the IJTAG network, respectively.

For boundary scan, you can segment the boundary scan chain into smaller chains that are used during logic testing with Tessent TestKompress. To segment the boundary scan chain into smaller chains to be connected to the EDT, specify [max\\_segment\\_length\\_for\\_logictest](#) within the [BoundaryScan](#) wrapper or alternatively specify the following command prior to running create\_dft\_specification:

```
set_defaults_value DftSpecification/BoundaryScan/max_segment_length_for_logictest
```

**Examples**

The following dofile example shows a typical command flow.

The highlighted command lines illustrate additional considerations for inserting the Tessent Shell MemoryBIST and Tessent Shell Boundary Scan instruments in the first insertion pass of a two-pass DFT insertion process. The functional pins are equipped with logic so that they can be shared as EDT channel input and output pins.

**Example 6-1. Dofile Example for MemoryBIST and BoundaryScan**

```
1 # Load the design
2 set_context dft -rtl -design_id rtl1
3 set_tsdb_output_directory ../tsdb_rtl
4 read_verilog ../RTL/cpu_top.v
5 read_cell_library ../library/tessent/adk.tcelllib
6 set_design_sources -format tcd_memory -Y ../library/memory \
7   -extension memlib
8 set_current_design cpu_top
9
10 # Specify and verify the DFT requirements
11 set_dft_specification_requirements -memory_test auto -boundary_scan on
12 set_design_level chip
13
14 # Identify TAP pins
15 set_attribute_value tck_p -name function -value tck
```

```

16 set_attribute_value tdi_p -name function -value tdi
17 set_attribute_value tms_p -name function -value tms
18 set_attribute_value trst_p -name function -value trst
19 set_attribute_value tdo_p -name function -value tdo
20 set_boundary_scan_port_options ramclk_p -cell_options clock
21 set_boundary_scan_port_options reset_p -cell_options sample
22
23 # Some pins cannot add any boundary scan cells
24 set_boundary_scan_port_options test_clock_p -cell_options dont_touch
25 set_boundary_scan_port_options edt_update -cell_options dont_touch
26 set_boundary_scan_port_options scan_en_p -cell_options dont_touch
27
28 # Specify the clocks feeding the memories
29 add_clocks 0 ramclk_p -Period 5ns
30
31 check_design_rules
32
33 # Create DFT specification
34 set_spec [create_dft_specification]
35 report_config_data $spec
36 set_config_value $spec/BoundaryScan/max_segment_length_for_logictest 150
37 read_config_data -in ${spec}/BoundaryScan -from_string {
38     AuxiliaryInputOutputPorts {
39         auxiliary_input_ports : portain_p[0], portain_p[1], portain_p[2] ;
40         auxiliary_output_ports : portbout_p[0], portbout_p[1],
41         portbout_p[2];
42     }
43 }
44 report_config_data $spec
45
46 # Create the hardware for the DFT inserted with the DFT specification
47 process_dft_specification
48
49 # Extract the IJTAG network connectivity and create ICL for the design
50 extract_icl
51
52 # Create patterns specification for validating the inserted hardware
53 create_patterns_specification
54
55 # Process the patterns specification and create simulation testbenches
56 process_patterns_specification
57
58 # Run and check testbench simulations
59 set_simulation_library_sources -v ../library/verilog/adk.v \
60     -v ../library/pad_cells.v -v ../library/memory/ram.v
61 run_testbench_simulations
62 check_testbench_simulations -report_status

```

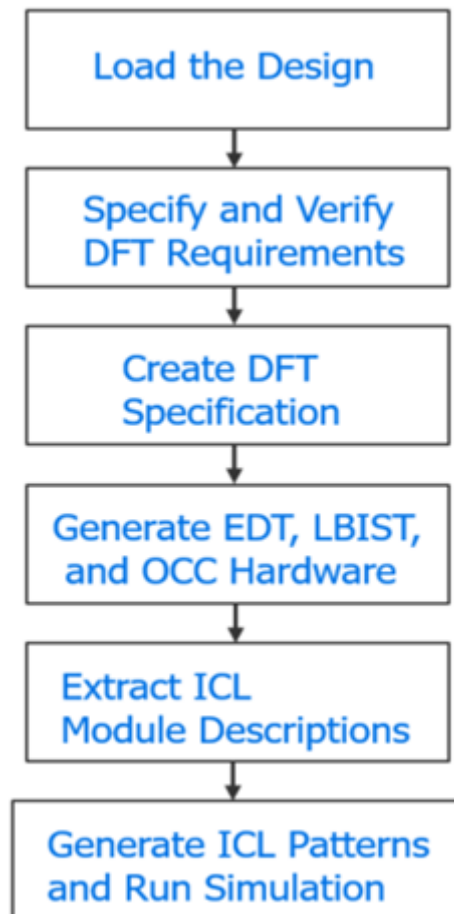
## Second DFT Insertion Pass: EDT, Hybrid TK/LBIST, and OCC

In the second DFT insertion pass, insert either the EDT or hybrid TK/LBIST logic test hardware, and OCC. This insertion pass includes requirements and considerations that differ from the first DFT insertion pass, including inserting the DFT signals.

For information about OCC, refer to “[Tessent On-Chip Clock Controller](#)” in the *Tessent Scan and ATPG User’s Manual*.

Before running this flow for chip-level designs, source nodes must be present in the RTL design so that you can define dynamic DFT signals, as described in “Specifying and Verifying the DFT Requirements”. Dynamic DFT signals (for example, scan enable, edt\_clock, and edt\_update) need to change during specific tests.

**Figure 6-5. Flow for the Second DFT Insertion Pass**



### Note

 For the second DFT insertion pass, use the process described in “[First DFT Insertion Pass: Performing MemoryBIST and Boundary Scan](#)” on page 180 to generate ATPG patterns and perform simulation.

---

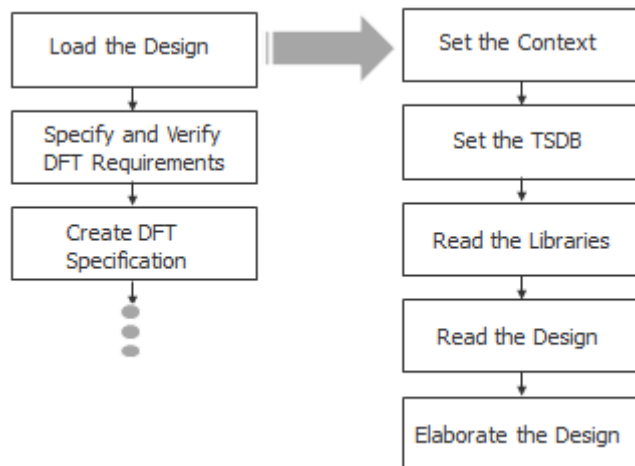
|                                                                    |            |
|--------------------------------------------------------------------|------------|
| <b>Loading the Design .....</b>                                    | <b>185</b> |
| <b>Specifying and Verifying the DFT Requirements.....</b>          | <b>187</b> |
| <b>Creating the DFT Specification .....</b>                        | <b>190</b> |
| <b>Generating the EDT, Hybrid TK/LBIST, and OCC Hardware .....</b> | <b>194</b> |
| <b>Extracting the ICL Module Description .....</b>                 | <b>194</b> |
| <b>Generating ICL Patterns and Running Simulation .....</b>        | <b>195</b> |

## Loading the Design

When loading the design for the second DFT insertion pass, you must ensure that you specify a new design ID name and, optimally, use the same TSDB directory that you used in the first insertion pass. In addition, you must read in the design from the first insertion pass and elaborate the design.

The design loading process for the second pass is similar to the process you used in the first insertion pass, with a few exceptions.

**Figure 6-6. Flow for Loading the Design**



### Prerequisites

- For chip-level designs, source nodes must be present in the RTL design so that you can define dynamic DFT signals as described in “[Specifying and Verifying the DFT Requirements](#).” Dynamic DFT signals are signals such as scan enable, edt\_clock, edt\_update, and so on, that need to change during specific tests.

## Procedure

1. Apply the `set_context` command with the ID “rtl2” for the second pass. For example:

```
set_context dft -rtl design_id rtl2
```

In the first DFT insertion pass, you set the design ID to “rtl1” with the `command`.

---

### Note



This manual uses recommended naming conventions for the design IDs for flat designs, which are “rtl1” for the first DFT insertion pass, “rtl2” for the second DFT insertion pass, and “gate” for the scan chain insertion pass. Refer to “[Considerations for Using Gate-Level Verilog Netlists](#)” for the naming conventions when using gate-level Verilog netlists.

---

For a specified design, the design ID stores all the data associated with a DFT insertion pass into the TSDB. For the first pass, “rtl1” contains the data for the MemoryBIST and boundary scan hardware. Setting the `design_id` to `rtl2` at the beginning of the second DFT insertion pass identifies that “rtl2” stores the EDT or LBIST, and OCC hardware data generated during the second pass.

The `rtl2` design data is cumulative. That is, it contains the necessary `rtl1` data in addition to the new data generated for EDT or LBIST, and OCC. The “rtl2” designation indicates that the second insertion pass is performed on the resulting edits of the first pass RTL data. In subsequent insertion passes, you can use either design ID to load the design and its supporting files.

Tessent generates JTAG nodes during both insertion passes and their module names are differentiated using the design ID you specified in each pass.

2. Specify the `set_tsdb_output_directory` command with the same directory location for both the first and second DFT insertion passes:

```
set_tsdb_output_directory ../tsdb_rtl
```

If you forget to specify the command for the second DFT insertion pass, Tessent creates a default `tsdb_outdir` directory in the current working directory. If you use a different output TSDB directory for the two insertion passes, ensure that you open the TSDB used by the first insertion pass by specifying the `open_tsdb` command.

For more information about the TSDB, refer to “[Tessent Shell Data Base \(TSDB\)](#)” in the *Tessent Shell Reference Manual*. For information about the TSDB data flow, refer to “[TSDB Data Flow for the Tessent Shell Flow](#).”

3. Specify the `read_cell_library` command to read in the library file for the standard cells and the pad I/O macros.

The following command reads the Tessent cell library file for the standard cells and pad I/O macros:

```
read_cell_library ../library/adk_complete.tcelllib
```

If the pad I/O library and standard cell library are separate, use the following commands to read in the atpg.lib files and the library for the pad I/O description:

```
read_cell_library ../library/atpg.lib
read_cell_library ../library/pad.library
```

4. Specify the [read\\_design](#) command to load the design:

```
read_design cpu_top -design_id rtl1
```

The design was created in the first DFT insertion pass when you used the `read_verilog` or `read_vhdl` commands. The `read_design` command also loads supporting files such as the TCD, ICL, and PDL, if present in the TSDB.

The design is read in using the design ID from the first DFT insertion. In this case, the design ID is “rtl1”

To load the design correctly for the second insertion pass, the `read_design` command refers to the design source dictionary that was created during the first DFT insertion pass and stored in the [dft\\_inserted\\_designs](#) directory.

5. Specify the [set\\_current\\_design](#) command to elaborate the design:

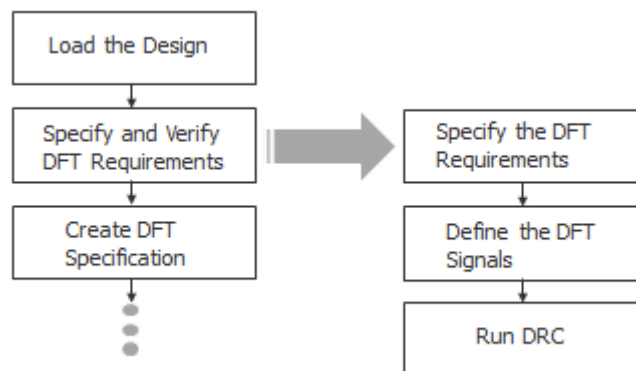
```
set_current_design cpu_top
```

If any module descriptions are missing, design elaboration identifies them. You can fix elaboration errors by adding the missing modules or by specifying the “[add\\_black\\_boxes](#) -module” command.

## Specifying and Verifying the DFT Requirements

After loading your design data, define the DFT requirements for the EDT or LBIST, and OCC hardware you want to insert. The requirements definition includes adding DFT signals followed by performing design rule checking.

**Figure 6-7. Flow for Specifying and Verifying the DFT Requirements**



## Procedure

1. Specify the [set\\_dft\\_specification\\_requirements](#) command to run pre-DFT design rule checking as follows:

```
set_dft_specification_requirements -logic_test on
```

Because you already specified that you were working at the chip level in the first DFT insertion pass, you do not need to specify this information for the second insertion pass.

2. Specify the [add\\_dft\\_signals](#) command to define the DFT signals. For example:

```
add_dft_signals ltest_en  
...
```

See “[DFT Signals](#)” on page 188 for a more complete example.

DFT signals are used to determine the various modes of operation, control values, and signal behaviors that are necessary for each test mode. The DFT signals capability in Tessent Shell automates the following tasks:

- Adding DFT signals
- Configuring DFT signals for various modes of operation
- Transferring information about DFT signals from one step to the next

Based on the specified mode of operation, Tessent creates the necessary setup procedures to control DFT signals through an IJTAG network.

3. Specify the [check\\_design\\_rules](#) command to run pre-DFT design rule checking.

Tessent Shell generates DFT\_C violations when error conditions are detected. For details, refer to “[Pre-DFT Clock Rules \(DFT\\_C Rules\)](#)” in the *Tessent Shell Reference Manual*.

## Results

When DRC passes, Tessent Shell shifts from setup mode to analysis mode. Pre-DFT DRC verifies that clocks are defined for all of the scannable sequential elements to be scan-tested and identifies the async sets and resets so that they can be turned off during shift operations. In addition, if you have specified the [add\\_dft\\_clock\\_enable](#) command, Tessent checks clock-gating logic and module-type clocks.

## Examples

### DFT Signals

You can add static DFT signals and dynamic DFT signals. Static DFT signals include global DFT control, logic test control, and scan mode signals. As described in the [add\\_dft\\_signals](#) command description, these DFT signals are typically controlled by a Test Data Register that is part of the IJTAG network.



Most dynamic DFT signals originate from primary input ports. For chip-level designs, these primary input ports must already be present in the RTL and be pre-connected to a pad buffer cell. The three dynamic DFT signals that originate from primary inputs are `test_clock`, `scan_en`, and `edt_update`. To share their input ports with the functional mode, ensure that you added auxiliary input logic for them during boundary scan insertion. Tessent cannot create the nodes as ports.

The following example shows the required DFT signals for the second insertion pass when you are inserting EDT or LBIST, and OCC.

```
// Create the static DFT signals
add_dft_signals ltest_en

// Generate dynamic DFT signals from source nodes
add_dft_signals scan_en edt_update test_clock \
    -source_node { porta_in1 portb_in1 porta_in2 }

// Create shift_capture_clock and edt_clock as gated versions of
// test_clock
add_dft_signals shift_capture_clock edt_clock -create_from_other_signals

// Used by top-level EDT
add_dft_signals edt_mode

// Required for boundary scan to be used with logic test without
// contacting inputs
add_dft_signals int_ltest_en output_pad_disable

// Required for scan-tested instruments (STI network) such as MemoryBIST
// and boundary scan
add_dft_signals tck_occ_en

// To bypass memories or to run multiple load ATPG for memories
add_dft_signals memory_bypass_en
```

---

### Note



Tessent Shell automatically recognizes scan-tested instruments and stitches them into scan chains.

---

Refer to the *Tessent Shell Reference Manual* for information about the following commands:

- **add\_dft\_signals** — Insert DFT signals.
- **register\_static\_dft\_signal\_names** — Register your DFT signal, if, for example, you need to augment the default DFT signals for specific usage requirements.
- **report\_dft\_signals** — View a list of the DFT signals added with the `add_dft_signals` command.
- **delete\_dft\_signals** — Delete any previously specified DFT signals.

- **add\_dft\_modal\_connections** — Though not normally required for flat designs, if you have multiple EDT configurations that you want to multiplex into a common set of top-level I/Os, use this command to implement the multiplexing. In addition, if the EDT channel pins are shared with functional pins, they need to include auxiliary input/output muxing logic.

## Related Topics

[add\\_dft\\_signals](#)

[register\\_static\\_dft\\_signal\\_names](#)

[report\\_dft\\_signal\\_names](#)

[report\\_dft\\_signals](#)

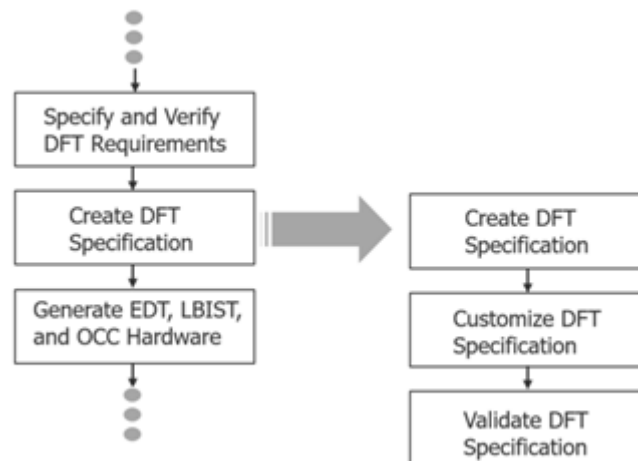
[delete\\_dft\\_signals](#)

[add\\_dft\\_modal\\_connections](#)

## Creating the DFT Specification

After defining the DFT requirements for the DFT signals in setup mode, you are ready to create the DFT specification for EDT or LBIST, and OCC in analysis mode. The DFT specification defines how the tool inserts hardware into the design.

**Figure 6-8. Flow for Creating the DFT Specification**



## Prerequisites

- For EDT or LBIST, and OCC, you must first generate a skeleton DFT specification that contains three empty SRI SIBs that specify the EDT, LBIST, and OCC sections of the IJTAG network. Creating the DFT specification for EDT or LBIST, and OCC differs from the process you use for MemoryBIST and boundary scan in the first DFT insertion pass.

## Procedure

1. Specify the [create\\_dft\\_specification](#) command as follows:

```
create_dft_specification -sri_sib_list {occ edt lbist}
```

2. Apply commands to customize the DFT specification using one of the following interfaces:

To customize the DFT specification on the command line, type the EDT or LBIST, and OCC data as an argument to the [read\\_config\\_data](#) -from\_string command. Modify the DFT specification with introspected data using the [add\\_config\\_element](#) and [set\\_config\\_value](#) commands. Tessent automatically saves modifications to the dofile/scripts directory for use in future sessions. See “[DFT Customization Example](#)” on page 191 for an example.

---

### Note

 To input the EDT or LBIST, and OCC wrapper details for the DFT specification, you can either use the Tessent GUI, known as Tessent Visualizer (see “Customizing the DFT Specification for EDT”), or the Tessent Shell command line.

---



---

### Note

 Tessent automatically connects the divided boundary scan segment (if present from the first DFT insertion pass) to the EDT hardware that Tessent inserts in the second insertion pass. Refer to [connect\\_bscan\\_segments\\_to\\_lsb\\_chains](#) for details.

---

3. Specify the following command to ensure that no errors exist in the DFT specification:

```
process_dft_specification -validate_only
```

Complete this step before generating the EDT, LBIST, and OCC hardware.

## Examples

### DFT Customization Example

The following example modifies a DFT specification to add LBIST (including EDT) and OCC and saves the changes.

---

### Note

 If you are using Tessent OCC, Tessent Scan automatically identifies and stitches the sub-chains in the OCC into scan chains.

---

```

read_config_data -in $spec -from_string {
  OCC {
    ijtag_host_interface : Sib(occ);
  }
}
# The following illustrates a generic way to populate the OCC. The clock
# list is design specific and needs to be updated for the design to the
# OCC, scan_enable and shift_capture_clock are connected automatically
# Modify the following for your specific design requirements
set id_clk_list [list \
  clk1 clk1_p \
  clk2 clk2_p \
  clk3 clk3_p \
  clk4 clk4_p \
  ramclk_p ramclk_p \
]

foreach {id clk} $id_clk_list {
  set occ [add_config_element OCC/Controller($id) -in $spec]
  set_config_value clock_intercept_node -in $occ $clk
}
report_config_data $spec

# To the EDT controller, the edt_clock and edt_update are connected
# automatically. The EDT controller is built-in with bypass
# Modify the following for your specific design requirements
read_config_data -in $spec -from_string {
  EDT {
    ijtag_host_interface : Sib(edt);
    Controller (c1) {
      longest_chain_range : 65, 100;
      scan_chain_count : 85;
      input_channel_count : 3;
      output_channel_count : 3;
      Connections +{
        EdtChannelsIn(1) {
        }
        EdtChannelsIn(2) {
        }
        EdtChannelsIn(3) {
        }
        EdtChannelsOut(1) {
        }
        EdtChannelsOut(2) {
        }
        EdtChannelsOut(3) {
        }
      }
    }
  }
}

```

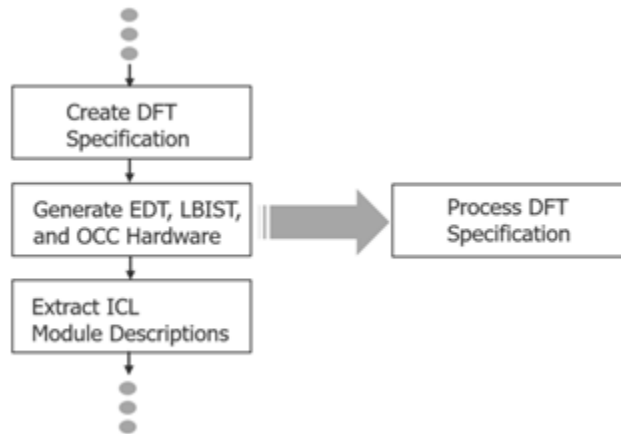
```
# Connecting the EDT channel in to the primary input and connecting
# EDT channel out to the primary output
# You had inserted the auxiliary logic for these ports during the first
DFT insertion pass
set_config_value port_pin_name \
  -in $spec/EDT/Controller(c1)/Connections/EdtChannelsIn(1) \
[get_single_name [get_auxiliary_pins portain_p[0] -direction input]]
set_config_value port_pin_name \
  -in $spec/EDT/Controller(c1)/Connections/EdtChannelsIn(2) \
[get_single_name [get_auxiliary_pins portain_p[1] -direction input]]
set_config_value port_pin_name \
  -in $spec/EDT/Controller(c1)/Connections/EdtChannelsIn(3) \
[get_single_name [get_auxiliary_pins portain_p[2] -direction input]]
set_config_value port_pin_name \
  -in $spec/EDT/Controller(c1)/Connections/EdtChannelsOut(1) \
[get_single_name [get_auxiliary_pins portbout_p[0] -direction output] ]
set_config_value port_pin_name \
  -in $spec/EDT/Controller(c1)/Connections/EdtChannelsOut(2) \
[get_single_name [get_auxiliary_pins portbout_p[1] -direction output] ]
set_config_value port_pin_name \
  -in $spec/EDT/Controller(c1)/Connections/EdtChannelsOut(3) \
[get_single_name [get_auxiliary_pins portbout_p[2] -direction output] ]
report_config_data $spec
```

```
LogicBist {
  ijttag_host_interface : Sib(lbist) ;
  Controller(C0) {
    ShiftCycles {
      max : 100 ;
    }
    CaptureCycles {
      max : 10 ;
    }
    PatternCount {
      max : 16384;
    }
    Connections {
      shift_clock_src : clk1;
    }
  }
  NcpIndexDecoder {
    Ncp(no_pulse) {
    }
    Ncp(pulse_clk1) {
      cycle(0) : $occ;
    }
    Ncp(pulse_clk2) {
      cycle(0) : $occ;
      cycle(1) : $occ;
    }
  }
}
```

## Generating the EDT, Hybrid TK/LBIST, and OCC Hardware

After creating the DFT specification, generate the EDT or hybrid TK/LBIST, and OCC hardware. The `process_dft_specification` command generates and inserts the DFT hardware, as defined by the DFT specification, into your design.

**Figure 6-9. Flow for Inserting the Second-Pass Hardware**



### Procedure

1. Specify the `process_dft_specification` command to insert EDT or LBIST, and OCC in the second pass:

```
process_dft_specification
```

2. (Optional) If you want to generate the hardware, but not insert it into the design, specify the following command:

```
process_dft_specification -no_insertion
```

You can then insert the hardware into the design manually.

## Extracting the ICL Module Description

After inserting the EDT or LBIST, and OCC hardware, verify the connectivity of the ICL modules that were inserted with the `process_dft_specification` command. This is a preparatory step for ICL pattern generation.

**Figure 6-10. Flow for Extracting ICL**



## Procedure

1. Specify the [extract\\_icl](#) command to find all modules (both Tessent instruments and non-Tessent instruments) and their associated ICL descriptions, and to run DRC to verify their connectivity.

The top-level ICL description corresponds to the design name you specified with the `set_current_design` command during the first insertion pass (which is also the same design name you specified when you elaborated the design at the beginning of the second insertion pass).

2. Specify the [analyze\\_drc\\_violation](#) command to debug the DRC violations.

Tessent generates I-rule errors when it detects ICL extraction errors and opens Tessent Visualizer to a schematic view of the error. For more information about the DRC I-rule errors, refer to “[ICL Extraction Rules \(I Rules\)](#)” in the *Tessent Shell Reference Manual*. For details about using Tessent Visualizer, refer to “[Tessent Visualizer](#)” on page 857.

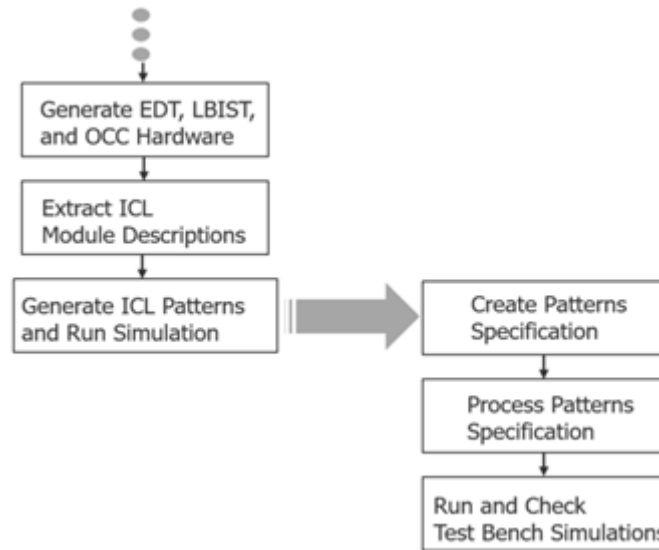
The `extract_icl` command also creates a Synopsys Design Compiler file that you can use for synthesis. Refer to “[Timing Constraints \(SDC\)](#)” for more information.

## Generating ICL Patterns and Running Simulation

Generate and process the ICL verification patterns for EDT or LBIST, and OCC, then verify the testbench simulations.

The following figure shows the tasks required to complete the second insertion pass and move on to synthesis.

**Figure 6-11. Flow for Generating and Simulating ICL Patterns**



## Procedure

1. Generate the test patterns for the design:

```
create_patterns_specification
```

2. Validate and process the content of the test patterns:

```
process_patterns_specification
```

3. Run and check testbench simulations. For example:

```
# Run and check testbench simulations
set_simulation_library_sources -v ../library/verilog/adk.v \
    -v ../library/pad_cells.v -v ../library/memory/ram.v

run_testbench_simulations
check_testbench_simulations -report_status
```

## Performing Synthesis

Synthesize the original RTL and the DFT-inserted RTL for MemoryBIST, boundary scan, EDT or LBIST, and OCC. For RTL designs, perform synthesis once after performing the first and second DFT insertion passes.

### Prerequisites

- For information regarding synthesis with third-party tools and Tessent DFT methodologies, refer to [“Synthesis Guidelines for RTL Designs with Tessent Inserted DFT”](#) on page 1041.
- Tessent Shell supports several third-party synthesis tools by generating Synopsys Design Constraints (SDC) to convey timing constraint information. See [“Timing](#)



[Constraints \(SDC\)](#)” for details. Find example scripts for the supported synthesis tools in [“Example Scripts Using Tessent Tool-Generated SDC”](#) on page 1019.

## Procedure

1. Specify the [write\\_design\\_import\\_script](#) command to create a design load script that loads the RTL design as it exists after the two DFT insertion passes. The following is an example command:

```
write_design_import_script for_synthesis.tcl -replace
```

---

### Note

 If you are not using a supported third-party synthesis tool, you can still use the `write_design_import_script` command to create a script and adjust it to match the command set of your synthesis tool.

---

2. Synthesize the hardware using the synthesis manager of the [run\\_synthesis](#) command to run the third-party synthesis tool. The following is an example command:

```
run_synthesis -startup_file for_synthesis.tcl
```

Alternatively, you can generate the synthesis script without running synthesis by specifying your local startup file. The following example targets `dc_shell`:

```
run_synthesis -startup_file ./synopsys_dc.setup
```

Use the following command to generate a script for other third-party synthesis tools:

```
run_synthesis -startup_file for_synthesis.tcl -generate_script_only
```

## Performing Scan Chain Insertion (Flat Design)

During scan chain insertion, Tessent inserts and stitches scan chains on the gate-level netlist that was synthesized after DFT insertion. After you run scan chain insertion, proceed to ATPG pattern generation.

During scan insertion using Tessent Scan, the non-scan instances such as EDT are automatically understood. In addition, the built-in OCC sub-chains are understood and stitched into scan chains. Tessent uses DFT signals such as the scan enable that you specified in the previous insertion passes, therefore you do not need to specify the DFT signals again.

For information about scan chain insertion using Tessent Shell, refer to the [“Internal Scan and Test Circuitry Insertion”](#) in the *Tessent Scan and ATPG User’s Manual*.

## Procedure

1. Specify the following command to set the DFT context:

```
set_context dft -scan -design_id gate
```

When setting the context, ensure that you specify the design ID with a unique name. The recommended name is gate for a flat design. For a gate-level netlist the recommended name is gate3.

2. Load the synthesized netlist. For example:

```
../Synthesis/cpu_top_synthesized.vg
```

This netlist contains the gates for the original RTL design and the DFT-inserted hardware.

3. Specify the same output directory you used in the first and second DFT passes:

```
../tsdb_rtl
```

4. Load the rtl2 design data for the DFT hardware that you inserted. For example:

```
cpu_top -design_id rtl2 -no_hdl
```

The `-no_hdl` switch specifies to read in all of the DFT data files—such as ICL, PDL, and TCD—except for the design files. (You are using the synthesized design from this point forward.)

After design elaboration and design rule checking, Tessent Shell transitions from Setup mode to Analysis mode.

5. Specify the `add_scan_mode` `edt_mode` command to connect the scan chains to the EDT signals and EDT hardware that you inserted during the second insertion pass.

The use of preregistered DFT signal `edt_mode` as the scan mode using the `add_scan_mode` command infers the `-enable_dft_signal` also as the `edt_mode` DFT signal.

## Results

The scan stitched and inserted netlist is located in the TSDB under the design ID “gate” for RTL designs or “gate3” for gate-level netlists. Refer to “[Considerations for Using Gate-Level Verilog Netlists](#)” for details.

## Examples

The following dofile shows a command flow for scan insertion and stitching:

```
set_context dft -scan -design_id gate
read_cell_library ../library/tessent/adk.tcelllib
read_verilog ../Synthesis/cpu_top_synthesized.vg
set_tsdb_output_directory ../tsdb_rtl
read_design cpu_top -design_id rtl2 -no_hdl
set_current_design cpu_top

check_design_rules
```

```
set edt_instance [get_name_list [get_instance -of_module \  
                                [get_name [get_icl_module -filter \  
                                tessent_instrument_type==mentor::edt]]]]  
add_scan_mode edt_mode -edt_instance $edt_instance  
analyze_scan_chains  
report_scan_chains  
insert_test_logic  
exit
```

## Related Topics

[read\\_verilog](#)

[set\\_tsdb\\_output\\_directory](#)

[read\\_design](#)

# Performing ATPG Pattern Generation

After inserting scan chains, proceed to ATPG pattern generation. You can create patterns for various fault model types, including stuck-at, transition, path delay, and IDDQ.

## Procedure

1. Do one of the following:
  - If you used Tessent Scan to insert the scan chains, run the “[import\\_scan\\_mode edt\\_mode](#)” command for ATPG pattern generation.
  - If you did not use Tessent Scan for scan insertion, use the TCD IP mapping flow as described in “[Running ATPG Patterns without Tessent Scan](#)” in the *Tessent Scan and ATPG User’s Manual*.

When you run the `import_scan_mode` command, Tessent Shell passes through the scan-insert design data for the EDT or LBIST, and OCC logic. This data includes the scan structures (scan chains and scan channels) that are stored in the TSDB under the “gate” design ID. The gate design ID was created during scan insertion when you specified the [insert\\_test\\_logic](#) command.

In addition, Tessent automatically creates and simulates the `test_setup` procedure cycles that are required to initialize the EDT or LBIST, and OCC static signals. You only need to specify non-default parameter values, if, for example, you run EDT with `bypass on` or set `int_ltest_en` to 1 to use the boundary scan as the source of the core values and isolate the ATPG test from the top-level IOs.

You can create ATPG patterns for any mode that you need, such as stuck-at and transition. These patterns are stored in the TSDB database under the [logic\\_test\\_cores](#) directory. The `import_scan_mode` command uses the same scan configuration—that is, the DFT signals—that were used for the `add_scan_mode` command during scan insertion.

2. Specify the `set_current_mode` command to indicate the type of pattern you are generating (see “[Stuck-at ATPG patterns](#)” on page 200 and “[Transition at-speed ATPG patterns](#)” on page 200).

In addition, the name you give to the generated ATPG pattern sets must differ from the mode name you specify for `import_scan_mode` (that is, “`edt_mode`”).

3. Specify the `write_patterns` command to write out the Verilog testbenches and STIL patterns.
4. Specify the `write_tsdb_data` command to save the flat model, fault list, PatDB, and TCD files into the TSDB.

For details about ATPG pattern generation, refer to “[Running ATPG Patterns](#)” in the *Tessent Scan and ATPG User’s Manual*.

## Examples

### Stuck-at ATPG patterns

The following example generates stuck-at ATPG patterns as indicated by the `set_current_mode edt_stuck` command. It shows that you have a choice between using the boundary scan chains for capture during ATPG or using the primary pins of the chips (using the pads).

```
set_context patterns -scan
read_cell_library ../library/tessent/adk.tcelllib
read_cell_library ../library/memory/memory.lib
open_tsdb ../tsdb_rtl
read_design cpu_top -design_id gate
set_current_design cpu_top
set_current_mode edt_stuck
import_scan_mode edt_mode

# To apply the patterns through the boundary scan chains and not through
# the pads use:
# set_static_dft_signal_values int_ltest_en 1
# set_static_dft_signal_value output_pad_disable 1
# To allow the shift_capture_clock during capture phase on Scan Tested
# Instruments:
set_system_mode analysis
create_patterns
write_tsdb_data -replace
write_patterns patterns/cpu_top_stuck_parallel.v -verilog -parallel \
-replace -scan
write_patterns patterns/cpu_top_stuck_serial.v -verilog -serial -replace
exit
```

### Transition at-speed ATPG patterns

The following example generates transition at-speed patterns. The `import_scan_mode` command uses the `-fast_capture_mode` switch to indicate that the OCC supplies the fast clock during capture. The `set_current_mode` command indicates `edt_transition`, and the `set_fault_type` command indicates transition faults.

```
set_context patterns -scan
read_cell_library ../library/tessent/adk.tcelllib
read_cell_library ../library/memory/memory.lib
open_tsdb ../tsdb_rtl
read_design cpu_top -design_id gate
set_current_design cpu_top
set_current_mode edt_transition
import_scan_mode edt_mode -fast_capture_mode on -verbose

set_static_dft_signal_values int_ltest_en 1
set_static_dft_signal_value output_pad_disable 1
set_system_mode analysis
set_fault_type transition
set_external_capture_options -pll_cycles 5 [lindex [get_timeplate_list] 0]
create_patterns
write_tsdb_data -replace
write_patterns patterns/cpu_top_transition_parallel.v -verilog \
               -parallel -replace -scan
write_patterns patterns/cpu_top_transition_serial.v -verilog -serial \
               -replace
exit
```

## Simulating LBIST Faults

If your design contains logic BIST, you must run LBIST fault simulation after scan insertion.

### Prerequisites

- Scan insertion has been completed.

### Procedure

1. Set the context:

```
set_context pattern -scan -design_id rtl2
```

2. Load the cell and memory libraries:

```
set_tsdb_output_directory tsdb_outdir
read_cell_library ../prereq/techlib_adk.tnt/tessent/adk.tcelllib
read_cell_library design/mem/mems.atpglib
```

3. Load the scan-inserted gate-level design:

```
read_design top -design_id rtl2
```

4. Elaborate the design:

```
set_current_design top
```

5. Import the scan insertion data:

```
import_scan_mode int_mode
```

6. Define the LBIST core instance:

```
add_core_instances -module top_dft2_tessent_lbist
```

7. Define DFT signals that differ from their default values:

```
set_static_dft_signal_values      tck_occ_en 1
set_static_dft_signal_values      int_ltest_en 1
set_static_dft_signal_values      ltest_en 1
set_static_dft_signal_values      control_test_point_en 1
set_static_dft_signal_values      observe_test_point_en 1

set_core_instance_parameters -instrument_type occ -parameter_values
{static_clock_control external}
```

8. Point to the dofile containing the proc definitions:

```
dofile tsdb_outdir/instruments/
top_dft2_lbist_ncp_index_decoder.instrument/
top_dft2_tessent_lbist_ncp_index_decoder.dofile

set_static_dft_signal_values x_bounding_en 1
```

9. Add required pin constraints and output masks:

```
add_input_constraints [get_ports {[ABC].*} -regexp] -CX
add_output_masks [get_ports Y*]

report_static_dft_signal_settings
```

10. Run rule checks:

```
set_system_mode analysis
report_scan_cells> scan_cells_fsm.rpt
report_clocks
report_pin_constraints
```

11. Read the test proc file containing NCP definitions:

```
read_procfile tsdb_outdir/instruments/
top_dft2_lbist_ncp_index_decoder.instrument/
top_dft2_tessent_lbist_ncp_index_decoder.testproc
```

12. Run pattern fault simulation:

```
add_faults -all
set_random_patterns 4096
simulate_patterns -source bist -store all
```

13. Report the test coverage and other test data:

```
report_statistics
```

14. Write the patterns using the `write_patterns` or `write_tsdb_data` command:

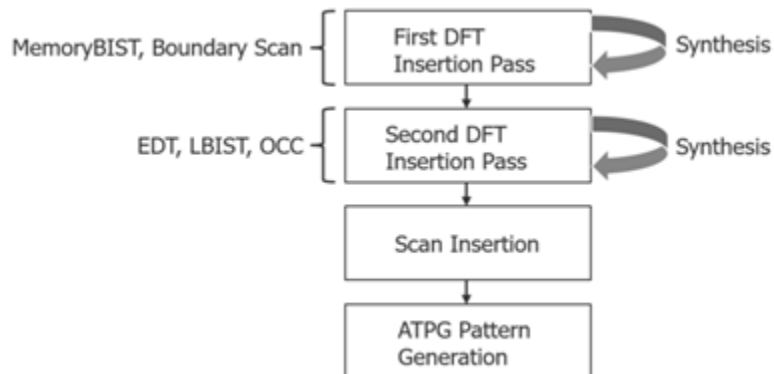
```
#write_patterns eltl_lbist_patt_parallel_ncp.v -verilog -parallel  
# -replace -mode_internal -begin 0 -end 260  
write_tsdb_data -rep  
  
report_lbist_configuration -hardware_default_compatibility  
  
exit
```

## Considerations for Using Gate-Level Verilog Netlists

In some cases, you may only have a gate-level Verilog netlist, not the RTL. You can use Tessent Shell to perform the DFT insertion flow when you have a gate-level netlist. The flow is similar to the RTL and scan DFT insertion flow. The main difference is that you must perform synthesis after each DFT insertion pass.

Figure 6-12 shows the gate-level DFT insertion sequence in which you insert MemoryBIST and boundary scan in the first insertion pass and then immediately run synthesis on these instruments before inserting the logic test instruments for the second DFT insertion pass (EDT, LBIST, OCC, and so on). After inserting the logic test instruments, run synthesis a second time before proceeding to scan chain insertion.

**Figure 6-12. Two-Pass Insertion Flow for RTL, Gate-Level Designs**



The following differences apply when using a gate-level Verilog netlist rather than RTL. Other than these differences, plus performing synthesis after each DFT insertion pass, you can follow the flow as described starting with “[First DFT Insertion Pass: Performing MemoryBIST and Boundary Scan](#).”

- **Prerequisites** — You must have the Tessent cell library or the ATPG library for the standard cells, in addition to the Tessent cell library for the I/O pad cells.

- **Design Loading** — During the first and second DFT insertion passes, ensure the following:
  - Specify the [set\\_context](#) command with the `-no_rtl` option rather than the `-rtl` option.
  - When setting the context, use the recommended naming conventions for the design IDs, which are “gate1” for the boundary scan/MemoryBIST insertion pass and “gate2” for the EDT/OCC insertion pass. If you are following this convention, you would then use design ID “gate3” for scan insertion.
  - Use the [read\\_cell\\_library](#) command to read in the library files for both the standard cells and pad I/O macros that are instantiated in the design.



# Tessent Shell Flow for Hierarchical Designs

Tessent Shell uses the same “divide and conquer” methodology for hierarchical DFT by enabling you to perform RTL and scan DFT insertion at the sub-physical block level rather than only at the chip level. Implementing DFT hierarchically in a bottom-up process, starting with the lowest level blocks, helps divide the task and make it manageable.

Hierarchical design methodologies provide efficiencies for large designs. Rather than design at the chip level, chip designers break designs down into RTL blocks that enable design teams to work on different functional blocks in parallel. This method streamlines the process and enables RTL modifications, engineering change orders (ECOs), and other changes to be localized to particular blocks.

Tip

 Before performing DFT on a hierarchical design, familiarize yourself with “[DFT Architecture Guidelines for Hierarchical Designs](#)” on page 105.

This section builds on what you learned in “[Tessent Shell Flow for Flat Designs](#)” to describe the pre-layout RTL and scan DFT insertion process for hierarchical designs.

Refer to the following test case for a detailed usage example of the flow described in this section:

```
tessent_example_hierarchical_flow_<software_version>.tgz
```

You can access this test case by navigating to the following directory:

```
<software_release_tree>/share/UsageExamples/
```

|                                                                         |            |
|-------------------------------------------------------------------------|------------|
| <b>Hierarchical DFT Terminology .....</b>                               | <b>205</b> |
| <b>How the DFT Insertion Flow Applies to Hierarchical Designs .....</b> | <b>208</b> |
| <b>RTL and Scan DFT Insertion Flow for Physical Blocks .....</b>        | <b>210</b> |
| <b>RTL and Scan DFT Insertion Flow for the Top Chip .....</b>           | <b>228</b> |
| <b>RTL and Scan DFT Insertion Flow for Sub-Blocks .....</b>             | <b>243</b> |
| <b>RTL and Scan DFT Insertion Flow for Instrument Blocks.....</b>       | <b>247</b> |
| <b>RTL and DFT Insertion Flow With Third-Party Scan .....</b>           | <b>252</b> |

## Hierarchical DFT Terminology

Tessent Shell uses a particular set of terms related to working with blocks in a hierarchical design. You must understand these terms to perform the RTL and scan DFT insertion process on hierarchical designs.

Refer to “[set\\_design\\_level](#)” in the *Tessent Shell Reference Manual* for more information.

- **Physical Block** — Physical blocks are logical entities that remain intact through tapeout. They are synthesis and layout regions. Below the top level of a chip, these are blocks that you can reuse, or instantiate, within a chip or across multiple chips. You perform synthesis on these blocks independent of the rest of the chip design.

When performing DFT insertion on physical blocks, Tessent preserves the ports at the root of the physical block. Instances of the physical block that exist below the current physical block may not be preserved in the final layout when ungrouping is used.

In Tessent Shell, the hierarchical DFT insertion flow distinguishes between three types of physical blocks: wrapped cores, unwrapped cores, and chip.

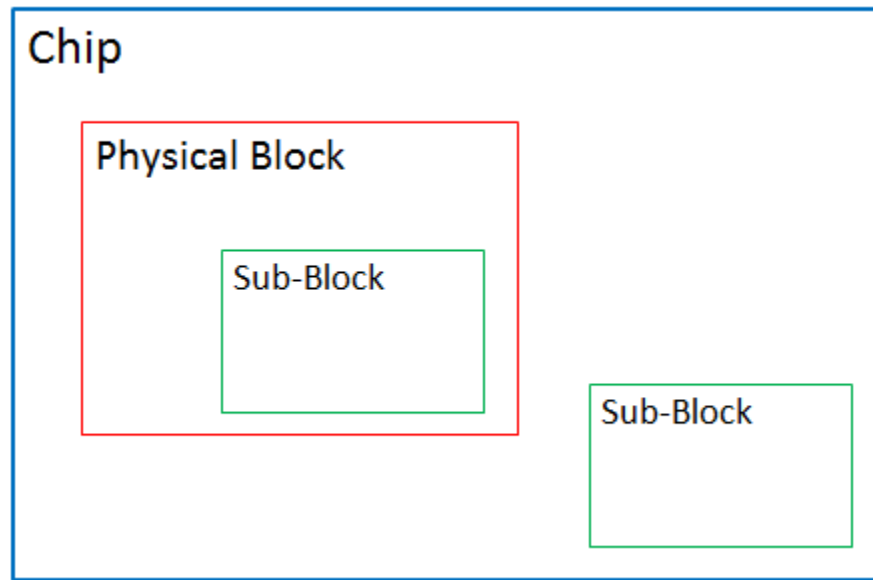
- **Wrapped core.** Wrapped cores contain wrapper cells that are used to isolate the internal logic of the core. Wrapper cells are inserted when you perform scan chain insertion. Wrapped cores are required to make the cores reusable through ATPG pattern retargeting. Wrapped cores can contain sub-blocks. (See the following description.)
- **Unwrapped core.** Unwrapped cores do not contain wrapper cells but can contain sub-blocks.

For additional information, refer to “[Unwrapped Cores Versus Wrapped Cores](#)” in the *Tessent Scan and ATPG User’s Manual*.

- **Chip.** The chip is the top-level physical block—that is, the entire design—in which you typically find the pad I/O macros and clock controllers. A chip may include another physical block or sub-block. Physical blocks can be wrapped cores or unwrapped cores. Unlike the other types of physical blocks, chips are layout regions.
- **Sub-Block** — Sub-blocks are designs that exist within parent blocks and are synthesized with their parent blocks, which could be wrapped cores, unwrapped cores, or the top level of the chip. Sub-blocks merge into their parent physical blocks during synthesis of the parent block. Refer to [set\\_design\\_level](#) for details.

Sub-blocks are not layout physical regions. After layout is performed on the post-layout netlist, the sub-block module boundary may or may not be preserved. Sometimes the same sub-block is instantiated at both the physical block level and the chip level, as shown in the following figure.

Figure 6-13. Hierarchical Design Levels



You can insert DFT hardware such as MemoryBIST, EDT, and OCC into sub-blocks, but you perform synthesis and scan insertion at the sub-block's parent physical block level (where the sub-block is instantiated). To learn about how to use sub-blocks within the Tessent Shell flow, refer to [“RTL and Scan DFT Insertion Flow for Sub-Blocks.”](#)

- **Instrument Block** — The design is a special empty module into which the DFT elements are inserted. The special module is then manually instantiated into its parent block and its pins are manually connected inside the parent block.

The synthesis, scan chain insertion, and pattern generation steps are done the usual way, as described in [“Instrument Block DFT Insertion Flow for the Next Parent Level”](#) on page 250.

- **ATPG (or scan) pattern retargeting** — The process by which Tessent Shell preserves the ATPG patterns associated with wrapped cores for purposes of reuse when testing the logic at the parent instantiation level. This means you do not have to regenerate the patterns when you process the top level of the chip. Instead, you retarget the wrapped core ATPG patterns to the top level. Every instantiation of a wrapped core includes its associated ATPG patterns.

For details, refer to [“Scan Pattern Retargeting”](#) in the *Tessent Scan and ATPG User's Manual*.

For purposes of ATPG pattern retargeting and graybox modeling (see the following description), Tessent Shell differentiates between a wrapped core's internal circuitry and its external circuitry.

- **Internal mode.** Internal mode is the view into the wrapped core from the wrapper cells. That is, the logic completely internal to the core. Tessent Shell retargets

internal mode ATPG patterns during ATPG pattern generation for the chip-level design.

- External mode. External mode indicates the view out of the wrapped core from the wrapper cells. That is, the logic that connects the wrapped core to external logic. Tessent Shell uses the external mode to build graybox models, which are used by the internal modes of their parent physical blocks.
- **Graybox** — Graybox models are wrapped core models that preserve only the core's external mode logic along with the portion of the IJTAG network needed for test setup of the logic test modes. The purpose of graybox models in the bottom-up hierarchical DFT process is to retain the minimum logic required to generate ATPG patterns for the internal modes of the parent physical blocks. For details, refer to “[Graybox Overview](#)” in the *Tessent Scan and ATPG User's Manual* and “[Graybox Model](#)” on page 110.

## How the DFT Insertion Flow Applies to Hierarchical Designs

Performing RTL and scan DFT insertion on hierarchical designs assumes basic knowledge of the RTL and scan insertion flow for flat designs. The process for hierarchical designs builds on the process you would use for a flat design.

You perform a two-pass pre-scan insertion process for flat designs. For hierarchical designs, the same flow applies except that you perform the insertion process in a bottom-up manner for each core and sub-block in the design. After you have inserted DFT into all the lower-level physical blocks, you perform the same insertion into the parent blocks until you have reached the top level of the chip.

The following considerations apply when performing DFT insertion on a hierarchical design as opposed to a flat design:

- When performing hierarchical DFT, you must specify the hierarchical design level at which you are performing the DFT insertion process. For flat designs, the [set\\_design\\_level](#) command is always set to chip. For hierarchical designs, you can also specify `physical_block` or `sub_block`.
- Inserting boundary scan during the first DFT insertion pass typically applies to the chip design level. For hierarchical designs, this means that for cores and sub-blocks you insert only MemoryBIST during the first DFT insertion pass unless you have cores with embedded pad I/O macros. If you have cores with embedded pad I/O macros, then you need to insert boundary scan into the pad IOs using the [embedded boundary scan](#) flow as described in the *Tessent Boundary Scan User's Manual*.
- Within the hierarchical flow, each physical block and sub-block has a unique design name and should have its own TSDB.

**Tip**

 To facilitate data management, save each design (whether core, sub-block, or chip) in its own TSDB directory. This is the recommended practice when using Tessent Shell for DFT insertion. Using different directories ensures that you can run all sibling physical and sub-blocks in parallel without causing read-write errors into the TSDB directories between the parallel runs. Only when all the child physical and sub-blocks of a given block are completed can you then implement the DFT into the given block. Refer to “[TSDB Data Flow for the Tessent Shell Flow](#)” for more information.

---

- You can perform ATPG pattern retargeting of core-level patterns when you process the parent physical block of the wrapper cores, as shown in section “[Top-Level ATPG Pattern Generation Example](#).”

## RTL and Scan DFT Insertion Flow for Physical Blocks

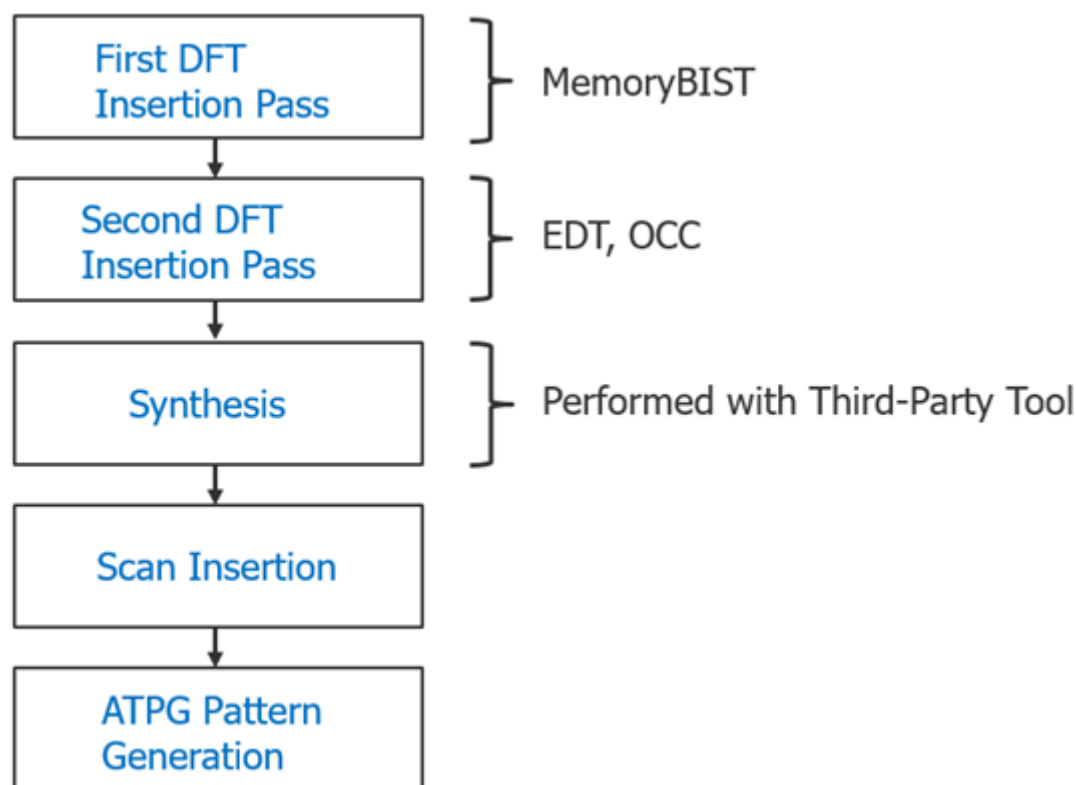
For each wrapped core, perform a two-pass pre-scan DFT insertion process as you would for a flat design except that in the first DFT insertion pass, do not insert boundary scan unless you have embedded pad IO macros present inside the core. The flow details for working with wrapped cores differ from those when working with flat designs.

Refer to “[Tessent Shell Flow for Flat Designs](#)” for overall flow details.

### Note

 This discussion assumes that your design consists of wrapped cores as your lower level physical blocks, and that the wrapped cores do not contain embedded pad IOs, so boundary scan is not required.

### Figure 6-14. Two-Pass Insertion Flow, Physical Blocks



|                                                                                       |            |
|---------------------------------------------------------------------------------------|------------|
| <b>First DFT Insertion Pass: Performing Block-Level MemoryBIST.....</b>               | <b>211</b> |
| <b>Second DFT Insertion Pass: Inserting Block-Level EDT and OCC .....</b>             | <b>213</b> |
| <b>Specifying and Verifying the DFT Requirements: DFT Signals for Wrapped Cores..</b> | <b>216</b> |
| <b>Performing Scan Chain Insertion: Wrapped Core .....</b>                            | <b>218</b> |
| <b>Verifying the ICL Model.....</b>                                                   | <b>221</b> |

|                                                                          |     |
|--------------------------------------------------------------------------|-----|
| Performing ATPG Pattern Generation: Wrapped Core .....                   | 222 |
| Running Recommended Validation Step for Pre-Layout Design Sign Off ..... | 227 |

## First DFT Insertion Pass: Performing Block-Level MemoryBIST

Because you are not inserting boundary scan at the same time as MemoryBIST (as in the flat flow), you can follow the DFT insertion process for Tessent MemoryBIST.

Refer to “[Getting Started](#)” in the *Tessent MemoryBIST User’s Manual*.

### Note



The line numbers used in this procedure refer to the command flow dofile in [Example 6-2](#) on page 212.

## Procedure

1. Load the RTL design data. (See lines 1-22.) The following steps are important for the first DFT insertion pass for wrapped cores:

- a. Set the [set\\_context](#) -design\_id switch to “rtl1.”

All the data associated with MemoryBIST insertion for the wrapped core is stored under the ID “rtl1” in the wrapped core’s TSDB.

### Note



“rtl1” is the recommended naming convention for the design ID for the first insertion pass, but you can specify any name. Refer to “[Loading the Design](#)” for more information about setting the design ID.

- b. Specify the [read\\_verilog](#) command with a list of the RTL filenames and locations for Tessent to read and compile for the design. For example:

```
../rtl/omsp_timerA_defines.v
../rtl/omsp_timerA_undefines.v
../rtl/omsp_timerA.v
../rtl/omsp_wakeup_cell.v
../rtl/omsp_watchdog.v
../rtl/openMSP430_defines.v
../rtl/openMSP430_undefines.v
../rtl/openMSP430.v
../rtl/processor_core.v
```

- c. Set the design level to “physical\_block” so that the layout of this core is maintained as an independent logical entity through tapeout.
2. Create the DFT specification. (See lines 24-30.)
  3. Generate the MemoryBIST hardware and extract the ICL. (See lines 32-36.)

4. Create the input test patterns and simulation testbenches. (See lines 38-41.)
5. Run simulations to verify the design. (See lines 43-47.)

## Examples

The following dofile example shows a typical command flow as detailed in the procedure listed in the preceding.

### Example 6-2. Dofile Example for MemoryBIST in Physical Blocks

```
1 # Set context to dft and indicate DFT insertion into an rtl-level design
2
3 set_context dft -rtl -design_id rtl1
4
5 # Set the TSDB directory location to be used
6
7 set_tsdb_output_directory ../tsdb_core
8
9 # Read in the memory library model
10 set_design_sources -format tcd_memory -y ../../../../library/memory \
11     -extension tcd_memory
12
13 # Read in memory Verilog model
14 set_design_sources -format verilog -v {../../../../library/memory/*.v }
15
16 # Read in the design
17 read_verilog -f rtl_file_list
18
19 set_current_design processor_core
20
21 # Set the design level as a physical_block
22 set_design_level physical_block
23
24 # Specify to insert memory test
25 set_dft_specification_requirements -memory_test on
26 add_clocks 0 dco_clk -period 3
27 check_design_rules
28 report_memory_instances
29 set_spec [create_dft_specification]
30 report_config_data $spec
31
32 # Generate the memoryBIST hardware
33 process_dft_specification
34
35 # Create ICL for this design
36 extract_icl
37
38 # Generate testbenches
39 create_patterns_specification
40 process_patterns_specification
41 set_simulation_library_sources -v \
42     ../../../../library/standard_cells/verilog/NangateOpenCellLibrary.v
43
44 # Run simulations
45 run_testbench_simulations
46
```



```
47 # If simulation fails use the following command
48 //check_testbench_simulations
49 exit
```

## Second DFT Insertion Pass: Inserting Block-Level EDT and OCC

In the second DFT insertion pass for wrapped cores, insert the EDT and OCC hardware. Unlike flat designs, which use standard OCCs, for wrapped cores you have a choice between inserting OCCs of type child or type standard.

See “[Tessent OCC Types and Usage](#)” in the *Tessent Scan and ATPG User’s Manual* for details.

If your physical design implementation does not support using a clock MUX in the clock path, then you can use an OCC of type child. With the child type OCC, only clock chopping and clock gating functions occur inside the wrapped core, and these functions are sufficient for ATPG pattern retargeting (later in the flow) as long as at the chip-level you have included a parent OCC or other hardware that can perform clock selection.

Clock selection is used to select between shift and capture clocks when generating ATPG patterns for the internal mode of the core. Typically, scan chain shifting occurs at a significantly slower speed than the capture clock, hence the need for the clock.

Most of the steps for the second DFT insertion pass for wrapped cores are the same as those you would perform for a flat design. Refer to the applicable sections.

---

### Note



The line numbers used in this procedure refer to the command flow dofile in [Example 6-3](#).

---

## Procedure

1. Load the design (lines 1-10). See “[Loading the Design](#)” on page 185.
2. Specify and verify the DFT requirements (lines 12-31). See “[Specifying and Verifying the DFT Requirements](#)” on page 187.

---

### Note



Wrapped cores have their own DFT requirements for the clock signals.

---

3. Create the DFT specification (lines 33-37). See “[Creating the DFT Specification](#)” on page 190.
4. Generate the EDT and OCC hardware (lines 39-77). See “[Generating the EDT, Hybrid TK/LBIST, and OCC Hardware](#)” on page 194.
5. Extract the ICL module description (lines 79-83). See “[Extracting the ICL Module Description](#)” on page 194.

6. Generate ICL patterns and run the simulation (lines 85-94). See “[Generating ICL Patterns and Running Simulation](#)” on page 195.

## Examples

The following dofile example shows that you set the design ID to “rtl2” for the second DFT insertion pass, set the internal mode and external mode for the wrapped core, and have chosen to specify an OCC of type child.

### Example 6-3. Dofile for Second DFT Pass

```
1 # Set context to dft and specify design_id as rtl2
2 set_context dft -rtl -design_id rtl2
3
4 # Specify where the tsdb_outdir is to be located, default is at the
5 # current working directory
6 set_tsdb_output_directory ../tsdb_core
7
8 # Use read_design with the design ID from the first DFT insertion pass
9 read_design processor_core -design_id rtl1
10 set_current_design processor_core
11
12 # No need to reset the design level, which remains "physical_block" from
13 # the first pass
14 # Add DFT signals for the wrapped core
15
16 # The following commands are required for logic test
17 add_dft_signals ltest_en -create_with_tdr
18 add_dft_signals scan_en edt_update test_clock -source_node {
19     scan_enable edt_update test_clock_u }
20 add_dft_signals edt_clock shift_capture_clock -create_from_other_signals
21
22 # Required for memories to be tested with multiple load ATPG patterns
23 add_dft_signals memory_bypass_en -create_with_tdr
24
25 # Required for STI network to be tested during logic test
26 add_dft_signals tck_occ_en -create_with_tdr
27
28 # Required for hierarchical DFT and used during scan insertion
29 # Specifying both the internal mode and the external mode
30 add_dft_signals int_ltest_en ext_ltest_en int_mode ext_mode
31
32 report_dft_signals
33
34 # Run pre-DFT DRC
35 set_dft_specification_requirements -logic_test on
36
37 check_design_rules
38 set_spec [create_dft_specification -sri_sib_list {edt occ} ]
39
40 # Use report_config_syntax DftSpecification/edt|occ to see full syntax
41 report_config_data $spec
42 read_config_data -in $spec -from_string {
43     occ {
```

```

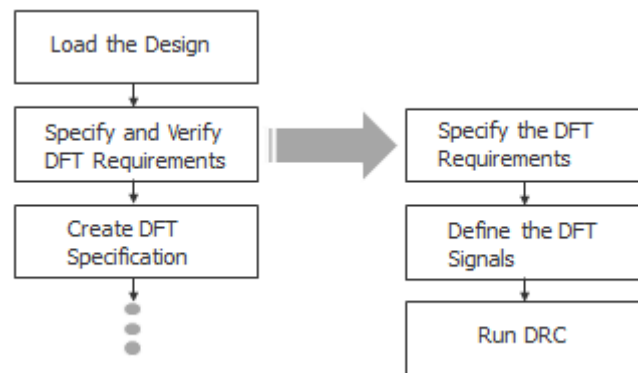
44     ijtag_host_interface : Sib(occ);
45 }
46 }
47 # The following is a generic way to populate the OCC
48 # The clock list is design specific and needs to be updated for the design
49 # To the OCC - scan_enable and shift_capture_clock gets connected
50 # automatically
51 # Modify the following to your design requirements
52 set id_clk_list [list \
53     dco_clk dco_clk \
54 ]
55
56 foreach {id clk} $id_clk_list {
57     set occ [add_config_element OCC/Controller($id) -in $spec]
58     set_config_value clock_intercept_node -in $occ $clk
59 }
60 report_config_data $spec
61
62 ## To EDT controller, the edt_clock and edt_update get auto connected
63 ## The EDT controller is built-in with bypass
64 ## Modify the following to your design requirements
65 read_config_data -in $spec -from_string {
66     EDT {
67         ijtag_host_interface : Sib(edt);
68         Controller (c1) {
69             longest_chain_range : 200, 300;
70             scan_chain_count : 60;
71             input_channel_count : 3;
72             output_channel_count : 2;
73         }
74     }
75 }
76 report_config_data $spec
77 //display_spec
78 # Generate the hardware
79 process_dft_specification
80
81 # Extract the IJTAG network and create ICL file for core level
82 extract_icl
83
84 # Write updated RTL into this new file to elaborate and synthesize later
85 write_design_import_script for_dc_synthesis.tcl -replace
86
87 # Generate testbench
88 create_patterns_specification
89 process_patterns_specification
90
91 set_simulation_library_sources -v \
92     ../../../../library/standard_cells/verilog/NangateOpenCellLibrary.v
93
94 # Run Verilog simulation
95 run_testbench_simulations
96 # If simulation fails, use the command below to see which pattern failed
97 //check_testbench_simulations
98
99 exit

```

## Specifying and Verifying the DFT Requirements: DFT Signals for Wrapped Cores

After loading the design data from the TSDB, define the DFT requirements for the EDT and OCC hardware. This includes adding DFT signals and performing DRC.

**Figure 6-15. Flow for Specifying and Verifying the DFT Requirements for Wrapped Cores**



### Procedure

1. Specify the following command to set DFT requirements:

```
set_dft_specification_requirements -logic_test on
```

#### Note

 The design level as specified by `set_design_level` remains “physical\_block” from the first DFT insertion pass, so you do not need to specify this command again.

2. Use the following procedure to define the DFT signals for wrapped cores in the second DFT insertion pass:

- a. Specify a global DFT signal to enter logic test mode. For example:

```
add_dft_signals ltest_en -create_with_tdr
```

- b. Define the DFT signals. For example:

```
add_dft_signals scan_en edt_update test_clock -source_node \  
{ scan_enable edt_update test_clock_u }
```

```
add_dft_signals edt_clock shift_capture_clock \  
-create_from_other_signals
```

If these ports do not exist, the tool creates new ports.

- c. Specify the following command to test with multiple load ATPG patterns in MemoryBIST:

```
add_dft_signals memory_bypass_en -create_with_tdr
```

- d. Specify the following command to test an STI network during logic test.

```
add_dft_signals tck_occ_en -create_with_tdr
```

- e. Specify both the internal mode and the external mode for hierarchical DFT. This is required for scan insertion.

```
add_dft_signals int_ltest_en ext_ltest_en int_mode ext_mode
```

This command specifies that wrapped cores have both internal modes and external modes, and that you must specify both. Differentiating between internal mode and external mode enables Tessent to stitch the scan chains into internal chains and external chains as described in “[Performing Scan Chain Insertion: Wrapped Core.](#)”

The internal and external modes are also required for proper ATPG pattern retargeting and graybox modeling later in the insertion flow. Refer to “[Hierarchical DFT Terminology](#)” for more information.

3. After defining the DFT signals, run DRC as in the flat design flow.

If the design includes clock gating that is implemented in RTL and not with an integrated clock gating cell, you must specify their func\_en and test\_en ports using the [add\\_dft\\_clock\\_enables](#) command. Tessent checks for proper clock and asynchronous set and reset controllability.

## Results

Tessent Shell generates DFT\_C errors for DRCs that are run. For details, refer to “Pre-DFT Clock Rules (DFT\_C Rules)” in the *Tessent Shell Reference Manual*.

## Examples

To define the DFT signals, the following example shows the required DFT signals for wrapped cores in the second DFT insertion pass:

```
# Required global DFT signal to enter logic test mode
add_dft_signals ltest_en -create_with_tdr

# If these ports do not exist, the tool creates new ports
add_dft_signals scan_en edt_update test_clock -source_node \
{ scan_enable edt_update test_clock_u }
add_dft_signals edt_clock shift_capture_clock -create_from_other_signals

# Required for MemoryBIST to be tested with multiple load ATPG patterns
add_dft_signals memory_bypass_en -create_with_tdr

# Required for STI network to be tested during logic test
add_dft_signals tck_occ_en -create_with_tdr

# Required for hierarchical DFT and used during scan insertion
# Specifying both the internal mode and the external mode
add_dft_signals int_ltest_en ext_ltest_en int_mode ext_mode
```

## Performing Scan Chain Insertion: Wrapped Core

Scan chain insertion for wrapped cores includes a task for performing wrapper analysis, which prepares the functional scannable sequential elements (or “flops”) for reuse as wrapper cells. Specifically, these cells are called shared wrapper cells. Using shared wrapper cells reduces the number of flops required for isolating the internal test modes from the primary input ports of the wrapper core. It also reduces the number of flops required for isolating the external test modes from the internal logic. Tessent automatically generates shared wrapper cells.

The [analyze\\_wrapper\\_cells](#) command performs wrapper analysis. In addition to preparing the shared wrapper cells, the command infers dedicated wrapper cells for ports having a large fanout or fan-in. You can configure the size of the fan-in and fanout using the `set_wrapper_analysis_options -input_fanout_flop_threshold` and `-output_fanin_flop_threshold` options. By default, Tessent infers dedicated wrapper cells on input ports fanning out to 32 or more scannable flip-flops, and on output ports in the fanout of 32 or more scannable flip-flops.

Both shared wrapper cells and dedicated wrapper cells can coexist in the same wrapper chains, which helps Tessent maintain wrapper chains of similar length. Scan insertion uses the DFT signals `int_ltest_en` and `ext_ltest_en` along with the scan enable signal to control the wrapper cell.

For information about scan chain insertion using Tessent Shell, refer to the “[Internal Scan and Test Circuitry Insertion](#)” in the *Tessent Scan and ATPG User’s Manual*.

---

### Note



The line numbers used in this procedure refer to the command flow dofile in [Example 6-4](#) on page 219.

---

## Prerequisites

- Prior to scan chain insertion, perform synthesis as described in section “[Performing Synthesis](#).”

## Procedure

1. Specify the following command to set the DFT context:

```
set_context dft -scan -design_id gate
```

2. Load the synthesized netlist. For example:

```
read_verilog ../3.synthesis/processor_core_synthesized.vg
```

3. Specify the same output directory you used in the first and second DFT passes:

```
set_tsdb_output_directory ../tsdb_core
```

4. Load the rtl2 design data for the DFT hardware you previously inserted. For example:

```
read_design processor_core -design_id rtl2 -no_hdl
```

5. Run design rule checking (DRC) using the following command:

```
check_design_rules
```

6. Set up the wrapper cells as follows (see lines 25-35):
  - a. Specify the options for analyzing the wrapper cells.
  - b. Add dedicated wrappers, as necessary.
  - c. Analyze the wrapper cells with the [analyze\\_wrapper\\_cells](#) command.
  - d. Ensure that you exclude the EDT channel IO ports from wrapper analysis.

For details, refer to “[Scan Insertion for Wrapped Core](#)” in the *Tessent Scan and ATPG User’s Manual*.

7. Specify the [add\\_scan\\_mode](#) command to connect the scan chains to the EDT signals and EDT hardware that you inserted during the second insertion pass.

Scan insertion for wrapper cells requires using [multi-mode scan insertion](#) as described in the *Tessent Scan and ATPG User’s Manual*. Do the following (see lines 41-46):

- a. Create one scan mode for the entire population of scan cells.

The entire population of scan cells are stitched into the first scan mode using the `int_mode` command to generate a scan mode consisting of all the scan cells stitched together.

- b. Create a second scan mode only for the shared and dedicated wrapper cells.

In the second DFT insertion pass, you had generated a DFT signal named “`int_mode`” with the `add_dft_signal` command. This signal enables this scan mode. You do not need to specify the `add_scan_mode -enable_dft_signal` switch when the mode name matches the name of a DFT signal of type scan mode.

The `add_scan_mode ext_mode` command stitches the shared and dedicated wrapper cells together, similar to the DFT signal you generated named `ext_mode`. `ext_mode` enables the scan mode for shared and dedicated wrapper cells.

## Examples

The following dofile shows a command flow for scan insertion. The highlighted statements illustrate additional considerations for performing scan insertion for wrapper cores. For a general overview, refer to “[Performing Scan Chain Insertion \(Flat Design\)](#)” for a flat design.

### Example 6-4. Scan Chain Insertion in Hierarchical Flow

```
1 set_context dft -scan -design_id gate
2
3 # You must respecify the TSDB
4 set_tsdb_output_directory ../tsdb_core
5
6 # Read Tessent library
```

```
7 read_cell_library \  
8   ../../../../library/standard_cells/tessent/NangateOpenCellLibrary.tcelllib  
9  
10 # Read synthesized netlist  
11 read_verilog ../3.synthesis/processor_core_synthesized.vg  
12  
13 # Use read_design to read in the DFT signals and other data from  
14 # the second DFT insertion pass  
15 read_design processor_core -design_id rtl2 -no_hdl  
16 set_current_design processor_core  
17  
18 # If there are no ATPG models available for memories, use blackboxes  
19 add_black_boxes -modules { \  
20   SYNC_1RW_8Kx16 \  
21 }  
22  
23 check_design_rules  
24  
25 report_clocks  
26 # Exclude the EDT channel in and out ports from wrapper chain analysis  
27 # The ijtag_* edt_update ports are automatically excluded  
28 set_wrapper_analysis_options \  
29   -exclude_ports [ get_ports {*_edt_channels_*} ]  
30  
31 # To force insertion of dedicated wrapper cell use the following command  
32 # set_dedicated_wrapper_cell_options on -ports {.... }  
33  
34 # Perform wrapper cell analysis  
35 analyze_wrapper_cells  
36 report_wrapper_cells -Verbose  
37  
38 # Find edt_instance  
39 set edt_instance [get_instances -of icl_instances \  
40   [get_icl_instances -filter tessent_instrument_type==mentor::edt]]  
41  
42 # Specify different modes (internal and external) of the chains that need  
43 # to be stitched  
44 # The type internal/external and enable_dft_signal are inferred from the  
45 # registered DFT signals(int_mode and ext_mode)  
46 add_scan_mode int_mode -edt_instances $edt_instance  
47 add_scan_mode ext_mode -chain_count 2  
48  
49 # Before scan insertion you can analyze the different scan modes and scan  
50 # chains  
51 analyze_scan_chains  
52 report_scan_chains  
53 //delete_scan_modes -all  
54 # Insert scan chains and write the scan inserted design into the TSDB  
55 insert_test_logic  
56 report_scan_chains  
57 report_scan_cells > scan_cells.list  
58  
59 exit
```



## Verifying the ICL Model

The ICL-based verification step verifies the ICL network before the pattern generation step.

During this verification, the tool checks the ICL network against semantic rules to verify the connectivity of the network. The tool verifies that it can access each IJTAG test data register through iWrite and iRead. The checks includes MemoryBIST logic if it is present in the design.

During scan chain stitching with Tessent Scan, the tool automatically updates the ICL model when complex ports generate loops, which causes ports and instances names to change through synthesis. When using a third-party scan insertion tool, additional ICL module matching rules may be required to complete the ICL-based verification step.

The following procedure and example dofile show how to verify the ICL model for SSN.

### Procedure

1. Set the context to patterns to create ICL-based patterns.
2. Set the location of the *tsdb\_outdir* directory and load the cell libraries.
3. Read the third-party scan-inserted netlist.
4. Load the collateral from the last DFT insertion step before scan insertion without reading the netlist.
5. Create and write the ICL-based pattern sets. This includes ICLNetwork verify patterns and MemoryBIST patterns, if memories are present.
6. Define the path to the Verilog simulation libraries, simulate the patterns, and check the simulation results.

### Examples

This example dofile shows how to verify the ICL model for SSN.

```
# Set context to patterns
set_context patterns

# Open the previous TSDB directories if not in the current
# working directory
set_tsdb_output_directory ../tsdb_outdir

# Read Tessent Library
read_cell_library \
  ../../../../library/standard_cells/tessent/NangateOpenCellLibrary.tcelllib
read_verilog \
  ../../../../library/memories/SYNC_1RW_8Kx16.v -interface_only

# Read the design files from the scan insertion pass
read_design processor_core -design_identifier gate -verbose
set_current_design processor_core
```

```
# Generate patterns to verify the inserted DFT logic
set_spec [create_patterns_specification]
report_config_data $spec
process_patterns_specification

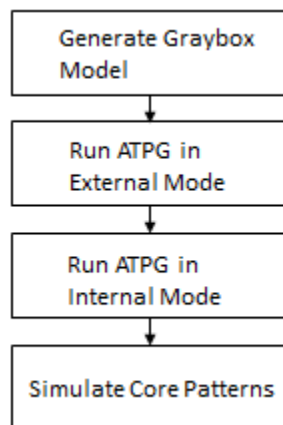
# Point to the libraries and run the simulation
set_simulation_library_sources \
  -v ../../../../library/standard_cells/verilog/NangateOpenCellLibrary.v \
  -v ../../../../library/memories/*.v
run_testbench_simulations
check_testbench_simulations -report_status
exit
```

## Performing ATPG Pattern Generation: Wrapped Core

Generating ATPG patterns for a wrapped core includes a step for generating a graybox model. In addition, you must perform ATPG twice, once for the core's external mode and once for the core's internal mode.

Refer to “[Performing ATPG Pattern Generation](#)” for flat designs for a general description.

**Figure 6-16. ATPG Pattern Generation Flow for Wrapped Cores**



As described in “[Hierarchical DFT Terminology](#),” the graybox model excludes the internal mode logic of the wrapped core, preserving only the external mode logic that needs to be tested at the parent level. The JTAG infrastructure is preserved in the graybox model also. Specifically, Tessent preserves the external logic that is present between the primary input and the input wrapper cells, plus any logic between the output wrapper cells and primary output. The parent could be another wrapper core or the top level of the chip.

You can use external mode patterns, if generated, for calculating fault coverage for the entire core (both internal and external mode). Use the internal mode ATPG patterns for ATPG pattern retargeting when performing the RTL and scan DFT insertion process on the top level of the chip.

## Procedure

1. Generate graybox models (see [Example 6-5](#) on page 224 for a command flow example):
  - a. Load in the design using the same design ID as you used for scan insertion to write the graybox to the TSDB.  
  
(Recommended) Use “gate” as the design ID if you inserted the MemoryBIST and EDT logic at the RTL level and “gate3” if the logic was also inserted into the gate level.
  - b. Specify the [analyze\\_graybox](#) command to generate graybox models.  
  
When working at the parent level, using the same design ID for the graybox model as you used for scan insertion enables Tessent to access the full design view or the graybox model with the same design ID.
2. Run ATPG on the *external* mode of the wrapped core. This ATPG pattern is only used to calculate the entire core's fault coverage and cannot be reused from the chip-level. To generate ATPG patterns for external mode, do the following:
  - a. Read in the graybox model of the design with the [read\\_design](#) command.  
  
Use the [set\\_current\\_mode](#) command to specify a unique ATPG mode name that represents the purpose of the pattern. The mode type is external.
  - b. Use the [import\\_scan\\_mode](#) command to retrieve the core's external mode data. Tessent uses the graybox model of the core. Using the [import\\_scan\\_mode](#) command assumes that you used Tessent Scan to perform scan chain stitching.
  - c. Run design rule checking (DRC) using the following command:  
  

```
check_design_rules
```
  - d. Generate the ATPG patterns using the following command:  
  

```
create_patterns
```
  - e. Use the [write\\_tsdb\\_data](#) command to store the TCD, flat model, fault list and PatDB files in the TSDB.
  - f. Use the [write\\_patterns](#) command to write out the testbench required to simulate the generated ATPG patterns.See [Example 6-6](#) on page 225.
3. Run ATPG on the *internal* mode of the wrapped core. This results in the ATPG pattern that you retarget at the top level of the chip. To generate ATPG patterns for internal mode, do the following:
  - a. Load in the graybox views for the wrapper cores that contain child wrapper cores.

- b. If you used Tessent Scan for scan insertion, specify `import_scan_mode` to import the internal mode.
- c. Specify a unique ATPG mode name with the `set_current_mode` command.

The current mode type is internal. Typical mode names are `scan_mode_name_sa` or `scan_mode_name_tdf`.

- d. Run design rule checking (DRC) using the following command:

```
check_design_rules
```

- e. Generate the ATPG patterns using the following command:

```
create_patterns
```

- f. Store the TCD, flat model, fault list and PatDB files in the TSDB using the `write_tsdb_data` command.
- g. Use the [read\\_faults](#) command to merge the fault list from running external mode to find the total overall fault coverage of the wrapped core.

See [Example 6-7](#) on page 226.

- 4. Run Verilog simulation of the core-level ATPG patterns.

Performing this task ensures that the patterns function as needed when they are retargeted at the parent level. For parallel load patterns as specified by the `-parallel` command, simulate all the patterns. For serial load patterns, a handful of patterns are sufficient; the run time for simulating gate-level serial load patterns is significant.

## Examples

### Generate the Graybox Model

The following example creates a graybox model for a scan-inserted `processor_core` design and saves the data under the “gate” design ID.

#### Example 6-5. Graybox Example

```
set_context patterns -scan -design_id gate
set_tsdb_output_directory ../tsdb_core
read_cell_library \
  ../../../../library/standard_cells/tessent/NangateOpenCellLibrary.tcelllib

## Reads in the scan inserted netlist/design
read_design processor_core -design_id gate -verbose
set_current_design processor_core
add_black_boxes -modules { \
  SYNC_1RW_8Kx16 \
}
```

```
report_dft_signals

import_scan_mode ext_mode
check_design_rules
analyze_graybox
write_design -tsdb -graybox -verbose

exit
```

### Run ATPG on the Core's External Mode

The following example shows the command flow for running ATPG on the external mode of a core.

#### Example 6-6. ATPG External Mode

```
set_context patterns -scan
set_tsdb_output_directory ../tsdb_core
read_cell_library \
    ../../../../library/standard_cells/tessent/NangateOpenCellLibrary.tcelllib

# Read in the graybox model using the -view switch
read_design_processor_core -design_id gate -view graybox -verbose
set_current_design processor_core

# Specify a different name that what was used during scan insertion with
# the add_scan_mode command
set_current_mode edt_multi_SAF -type external
report_dft_signals

# Extract the external mode specified during scan insertion
import_scan_mode ext_mode
report_core_instances
report_static_dft_signal_settings

# Run DRC, Tessent Shell prompts changes to Analysis
check_design_rules
report_clocks
set_xclock_handling x

# Generate ATPG patterns
create_patterns

# Store TCD, flat_model, fault list, and PatDB files in the TSDB
write_tsdb_data -replace
write_patterns patterns/processor_core_ext_stuck_parallel.v -verilog \
    -parallel -replace
set_pattern_filtering -sample_per_type 2
write_patterns patterns/processor_core_ext_stuck_serial.v \
    -verilog -serial -replace
exit
```

### Run ATPG on the Core's Internal Mode

The following example shows the command flow for running ATPG on the internal mode of a core.

### Example 6-7. ATPG Internal Mode

```
set_context patterns -scan
set_tsdb_output_directory ../tsdb_core
read_cell_library \
  ../../../../library/standard_cells/tessent/NangateOpenCellLibrary.tcelllib
# Read in the full scan-inserted netlist
read_design_processor_core -design_id gate
set_current_design_processor_core

# Extract the internal mode specified during scan insertion
import_scan_mode int_mode

# Use add_scan_mode to specify a different name than what was used during
# scan insertion
# Specify import_scan_mode before set_current_mode because
# import_scan_mode overrides the test mode type specified by
# set_current_mode
set_current_mode int_mode_sa -type internal

report_dft_signals
report_core_instances
report_static_dft_signal_settings

# Run DRC
check_design_rules
report_clocks
report_core_instances
add_fault -all
report_statistics -detail

# Generate ATPG patterns
create_patterns
report_statistics -detail

# Store TCD, flat_model, fault list, and PatDB files in the TSDB
write_tsdb_data -replace
write_patterns patterns/processor_core_stuck_parallel.v -verilog \
  -parallel -replace
set_pattern_filtering -sample_per_type 2
write_patterns patterns/processor_core_stuck_serial.v \
  -verilog -serial -replace
exit

# To view the coverage of the faults testable by internal mode, you must
# eliminate the undetected faults, which would be detected in
# external mode. Do this by merging in the fault list generated from
# performing ATPG on the graybox in external mode
read_faults -mode ext_multi_SAF -fault_type stuck -merge

# Final coverage of the core that includes internal and external modes
report_statistics -detail
exit
```

## Running Recommended Validation Step for Pre-Layout Design Sign Off

During the first pass of the hierarchical DFT insertion flow for a wrapped core, you insert and simulate the MemoryBIST hardware, mostly at the RTL level. To ensure that the MemoryBIST simulation passes before the core is signed off as the pre-layout netlist, rerun MemoryBIST simulation at gate level on the core's scan-inserted netlist.

### Procedure

1. Read in the scan-inserted netlist from the TSDB.

Using the recommended naming conventions for the RTL and scan DFT insertion flow, the design ID would be "gate" if you inserted the MemoryBIST and EDT logic at the RTL level and "gate3" if the logic was also inserted into the gate level.

2. Elaborate the design and run DRC.
3. Create the input test patterns and simulation files.
4. Simulate the testbench.

Tessent Shell stores the generated testbench in the TSDB under the Patterns directory using the design ID "gate".

### Examples

The following example shows how to validate the MemoryBIST patterns for a scan-inserted netlist.

```
set_context patterns -ijtag -design_id gate
set_tsdb_output_directory ../tsdb_core

##Reading the tessent cell library
read_cell_library \
  ../../../../library/standard_cells/tessent/NangateOpenCellLibrary.tcelllib

## Reading the scan inserted netlist
read_design_processor_core -design_id gate -verbose -view interface
set_current_design processor_core
add_black_boxes -modules { \
  SYNC_1RW_8Kx16 \
}
check_design_rules

create_patterns_specification
process_patterns_specification
set_simulation_library_sources \
  -v ../../../../library/standard_cells/verilog/NangateOpenCellLibrary.v \
  -v ../../../../library/memories/SYNC_1RW_8Kx16.v.v
run_testbench_simulation
exit
```

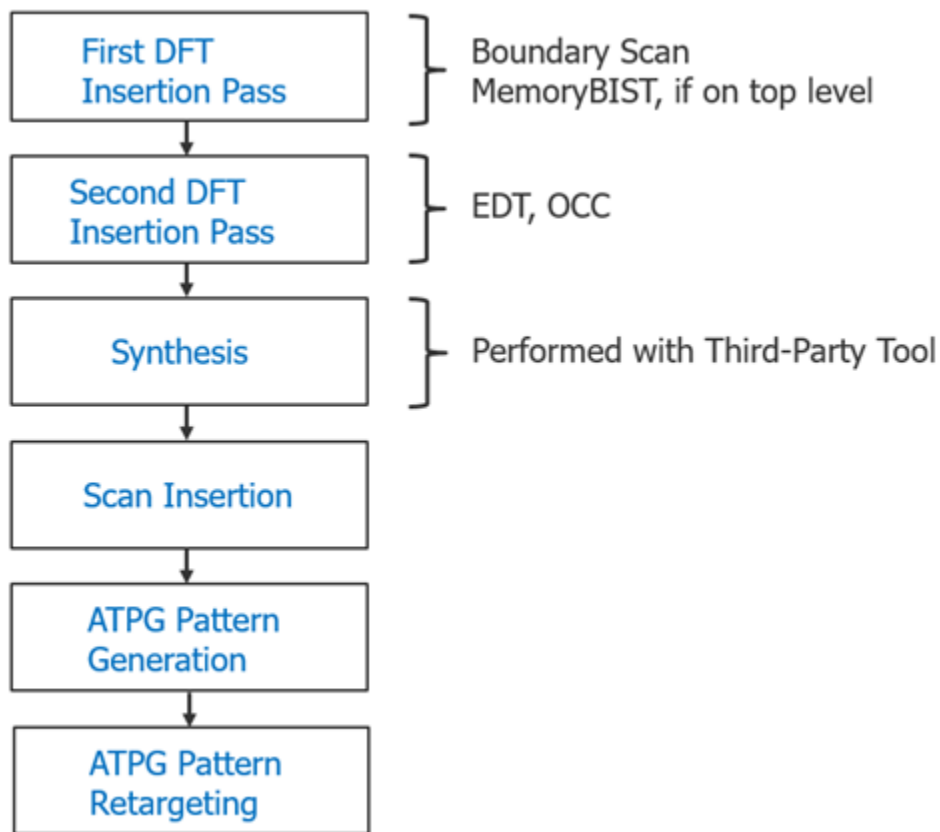
## RTL and Scan DFT Insertion Flow for the Top Chip

After performing the RTL and scan DFT insertion flow for each wrapped core in your design, you can perform the DFT insertion process for the top-level chip design.

Before implementing DFT at the top level of the chip, plan how you should test the wrapped cores at the chip level. The planning for logic testing the wrapped cores is not automated, so you must decide how you want to allocate the resources and organize the test schedule (especially for ATPG pattern retargeting) and specify your intent by using the [add\\_dft\\_modal\\_connections](#) command.

The RTL and scan DFT insertion flow for the top level of a chip follows the same basic process you used for the cores, with the addition of a step for retargeting the ATPG patterns you generated for the wrapped cores.

**Figure 6-17. Two-Pass Insertion Flow for RTL, Top Level**



In addition, processing at the chip level differs from wrapped cores in that you must insert a Test Access Mechanism (TAM). A TAM is a mechanism that you use to carry the scan data in and out of the chip for each group of wrapped cores you intend to run in parallel. The TAM schedules the tests for the wrapped cores at the top level, and enables access to the chip level so that you can run these tests.



During insertion and pattern generation, you open the TSDBs that store the wrapped core design data and read in the designs for their graybox models. The DFT insertion flow for the top level of the chip requires differentiating between three design views of the wrapped cores.

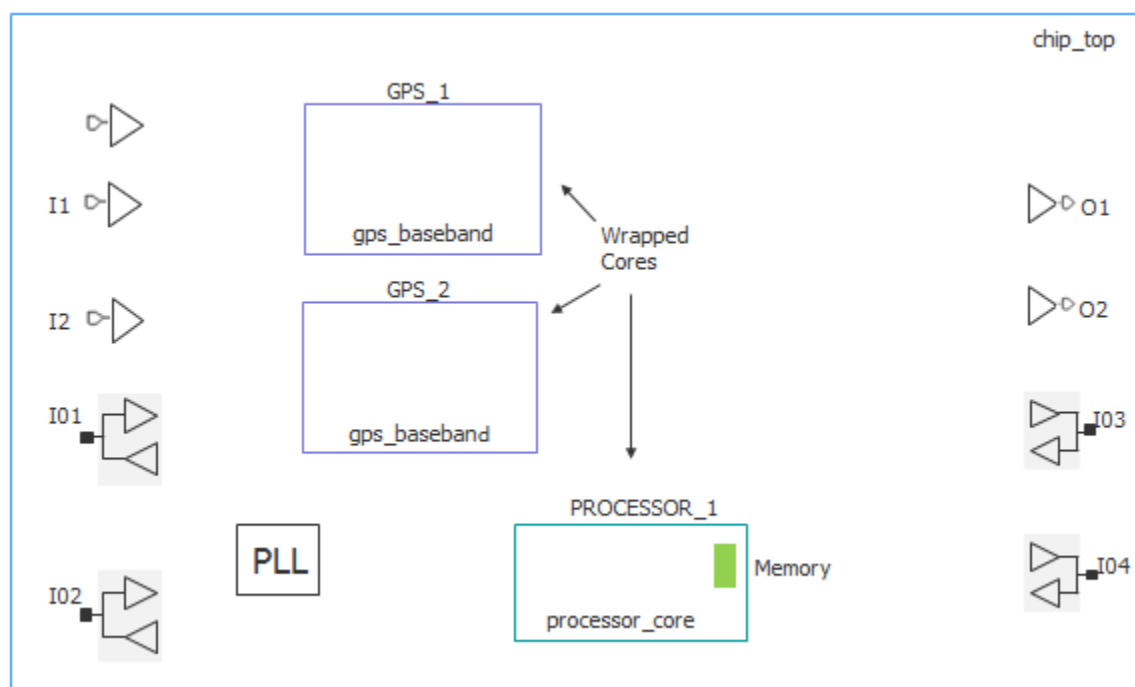
- **Full netlist view** — All the logic for the core. This is the default view when you do not explicitly specify a design view with the [read\\_design](#) command.
- **Graybox model** — External mode logic for the core as described in “[Hierarchical DFT Terminology](#),” including its IJTAG interface. You use this view so that Tessent Shell can connect the wrapped cores at the top level for logic testing of the chip.
- **Interface view** — The core’s ports only. Tessent auto-loads the interface view of any sub-physical block for which you have not used `read_design` to load its view.

|                                                                                          |            |
|------------------------------------------------------------------------------------------|------------|
| <b>Top-Level DFT Insertion Example .....</b>                                             | <b>229</b> |
| <b>First DFT Insertion Pass: Performing Top-Level MemoryBIST and Boundary Scan. ....</b> | <b>231</b> |
| <b>Second DFT Insertion Pass: Inserting Top-Level EDT and OCC .....</b>                  | <b>234</b> |
| <b>Top-Level Scan Chain Insertion Example. ....</b>                                      | <b>238</b> |
| <b>Top-Level ATPG Pattern Generation Example .....</b>                                   | <b>239</b> |
| <b>Performing Top-Level ATPG Pattern Retargeting .....</b>                               | <b>240</b> |

## Top-Level DFT Insertion Example

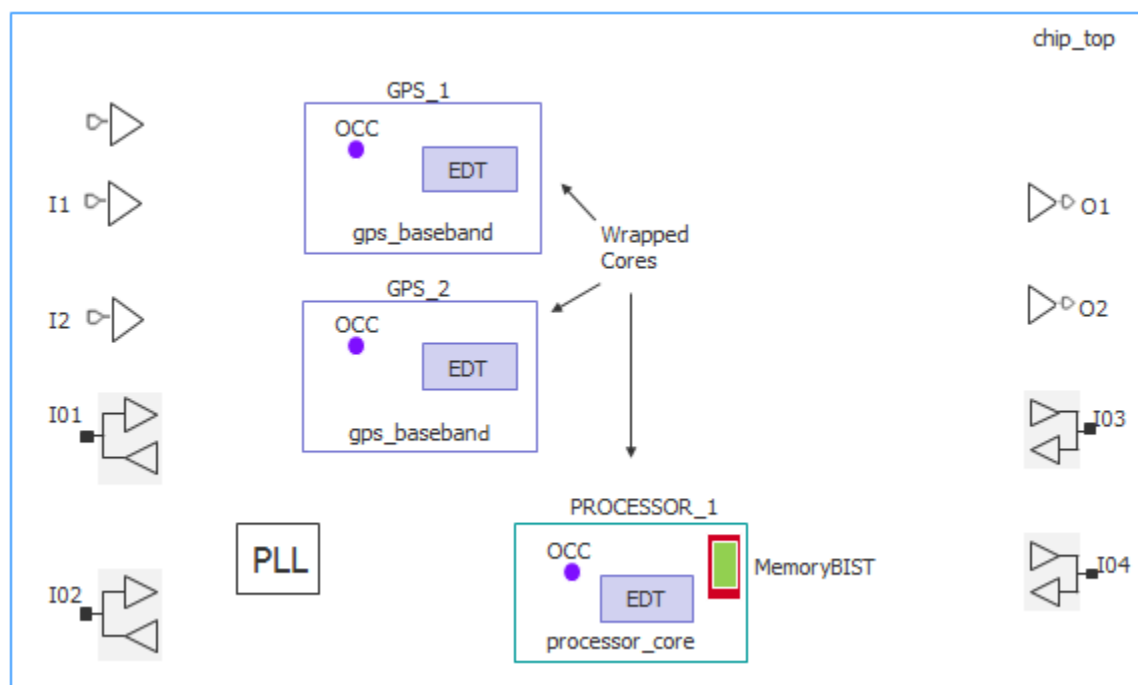
The RTL and scan DFT insertion flow for the top level of a chip can be illustrated using two wrapped cores, `processor_core` and `gps_baseband`. The processor core has memory instantiated within it, and the GPS baseband is a logic-only core that is instantiated twice.

**Figure 6-18. Top-Level Example, Before DFT Insertion**



After performing the RTL and scan DFT insertion process for the wrapped cores, each of the cores has an EDT controller and a child OCC. The processor core has the memories with MemoryBIST already inserted.

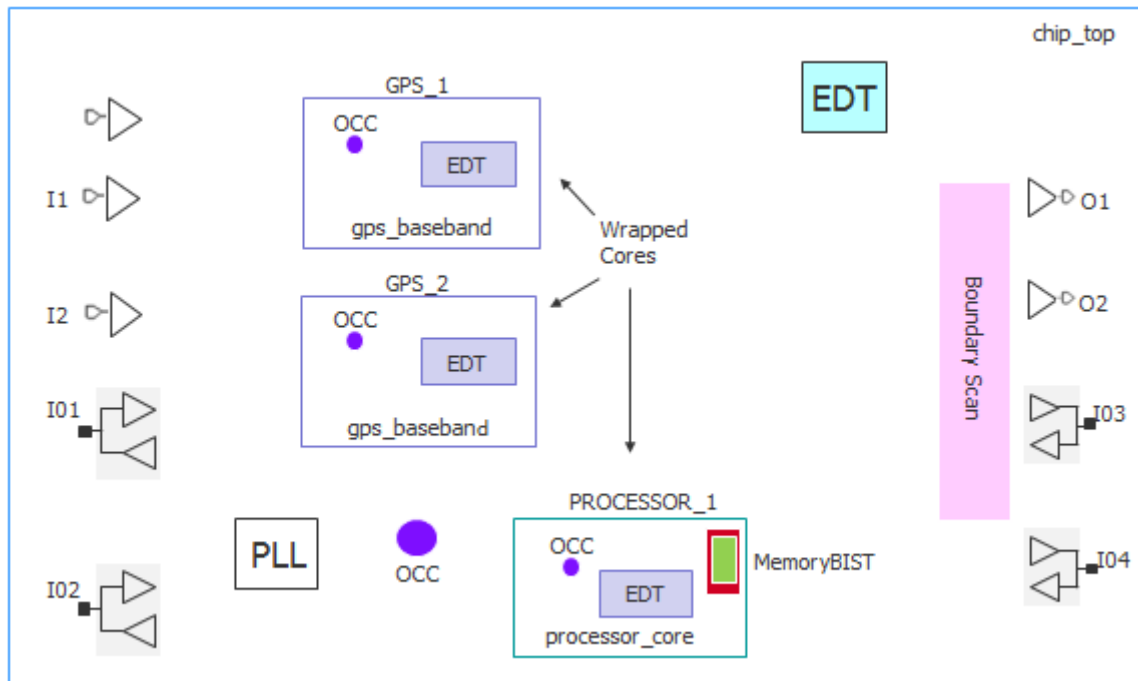
**Figure 6-19. Top-Level Example, After DFT Insertion for Wrapped Cores**



After inserting DFT at the top level, the design includes a TAP controller along with boundary scan, a top-level EDT controller, a parent OCC, and a TAM for purposes of retargeting the wrapped cores (not shown in [Figure 6-20](#) on page 231).

In this top-level example test case, the hierarchical core starts with RTL insertion, and in addition, at the top level you want to perform DFT at RTL as well. However, there is no RTL logic at the top level that needs to be synthesized. The PLL and pad IOs are Verilog macros. The only RTL that needs to be synthesized is the Tessent-generated RTL. Hence, this test case uses design IDs “gate1” and “gate2” for the first two DFT insertion passes, respectively.

**Figure 6-20. Top-Level Example, After DFT Insertion at the Top Level**



## First DFT Insertion Pass: Performing Top-Level MemoryBIST and Boundary Scan

For the top level of a chip, insert boundary scan plus MemoryBIST for any memories that are present at the top level. In addition, build the IJTAG network for the wrapped cores at the top level and insert a TAP controller or reuse an existing TAP controller.

As with flat designs, the flow for inserting MemoryBIST and boundary scan together is the same as inserting them separately. For details about this insertion pass flow, refer to:

- “[Getting Started](#)” in the *Tessent MemoryBIST User’s Manual*, or
- “[Getting Started With Tessent BoundaryScan](#)” in the *Tessent BoundaryScan User’s Manual*

For information about TAP controller reuse, refer to “[create\\_dft\\_specification](#)” in the *Tessent Shell Reference Manual*.

## Prerequisites

- Refer to “[First DFT Insertion Pass: Performing MemoryBIST and Boundary Scan](#)” (for flat designs) for general information and prerequisites. For example, as with flat designs, you can segment the boundary scan chain into smaller chains that are used during logic testing with Tessent TestKompress by using the [max\\_segment\\_length\\_for\\_logictest](#) command.

## Procedure

1. Load the design.

---

### Note



Use the [open\\_tsdb](#) command to open the TSDBs for all the lower-level cores. Opening their TSDBs makes their design data available.

There is no need to specify the `read_design` command because, by default, Tessent reads in the interface views of the wrapped cores, which is all that is required for DRC for the first DFT insertion pass.

---

2. Set the design level to “chip” for the top level of the chip.  
When working with the wrapped cores, you had set the design level to “physical\_block.”
3. Create the DFT specification.
4. Specify enough auxiliary input and output ports for the largest retargeting wrapper core group.
5. Generate the hardware and extract the ICL.
6. Create the input test patterns and simulation testbenches.
7. Run simulations to verify the design.

## Examples

The following dofile example shows a typical command flow for inserting MemoryBIST and boundary scan. The flow is the same as you would use for a flat design with the exception of the commands highlighted in bold. The chip-level design data exists in its own TSDB. This is recommended for the data flow as described in “[TSDB Data Flow for the Tessent Shell Flow](#).”

```

set_context dft -rtl -design_id rtl1
set_tsdb_output_directory ../tsdb_outdir

# Open the TSDB of all the wrapped cores
open_tsdb ../../wrapped_cores/processor_core/tsdb_outdir
open_tsdb ../../wrapped_cores/gps_baseband/tsdb_outdir

# Read the Tessent cell library
read_cell_library \
    ../../library/standard_cells/tessent/NangateOpenCellLibrary.tcelllib
read_cell_library ../../library/pad_cells/tessent/Nangate_pads.tcelllib

# Read hard macros
read_verilog ../../library/plls/pll.v -blackbox \
    -exclude_from_file_dictionary

# Read the design
set_design_sources -format verilog \
    -y ../../library/pad_cells/verilog -extension v
read_verilog ../rtl/chip_top.v
read_verilog ../rtl/rds_process.v
set_current_design chip_top
set_design_level chip

# Specify and verify the DFT requirements
set_dft_specification_requirements -boundary_scan on -memory_bist on

# Identify the TAP pins
set_attribute_value TCK -name function -value tck
set_attribute_value TDI -name function -value tdi
set_attribute_value TMS -name function -value tms
set_attribute_value TRST -name function -value trst
set_attribute_value TDO -name function -value tdo

# Some pins cannot add any boundary scan cells
set_boundary_scan_port_options TEST_CLOCK_top -cell_options dont_touch
set_boundary_scan_port_options EDT_UPDATE_top -cell_options dont_touch
set_boundary_scan_port_options SCAN_ENABLE -cell_options dont_touch
set_boundary_scan_port_options RESET_N -cell_options dont_touch
set_boundary_scan_port_options INCLK -cell_options dont_touch

# Add DFT signals
add_dft_signals scan_en -source_nodes SCAN_ENABLE
report_dft_signals
add_clocks PLL_1/pll_clock_0 -reference REF_CLK -FREQ_Multiplier 16
add_clock REF_CLK -period 48
check_design_rules

# Create a dft specification
set_spec [create_dft_specification]
report_config_data $spec
# Segment the boundary scan to be used during logic test
set_config_value $spec/BoundaryScan/max_segment_length_for_logic_test 80

# Equip these ports with auxiliary input/output muxes to be used by EDT
# channel pins
read_config_data -in ${spec}/BoundaryScan -from_string {
    AuxiliaryInputOutputPorts {

```

```
        auxiliary_input_ports  : GPIO1_0, GPIO1_1, GPIO1_2, GPIO1_3;
        auxiliary_output_ports : GPIO2_0, GPIO2_1, GPIO2_2, GPIO2_3, GPIO1_0,
        GPIO1_1 ;
    }
}

report_config_data $spec
process_dft_specification
extract_icl
create_patterns_specification
process_patterns_specification
set_simulation_library_sources \
    -v ../../library/standard_cells/verilog/*.v \
    -v ../../library/pad_cells/verilog/*.v \
    -y ../../library/plls \
    -y ../../library/memories \
    -extension v
run_testbench_simulations
exit
```

## Second DFT Insertion Pass: Inserting Top-Level EDT and OCC

For the top level of a chip, additional considerations for the second DFT insertion pass for EDT and OCC include grayboxes, DFT signals for purposes of ATPG retargeting, and TAMs. EDT, or Embedded Deterministic Test, reduces scan data volume and test time.

---

### Note



This procedure follows a similar flow as flat designs as documented in “[First DFT Insertion Pass: Performing MemoryBIST and Boundary Scan](#)” on page 180.

---

## Procedure

1. Specify the [open\\_tsdb](#) command to open the TSDBs for the wrapped cores, as in the first DFT insertion pass for the chip.
2. Specify the [read\\_design](#) command to read in the graybox models of the wrapped cores.

This enables Tessent to perform DRC on the wrapper chains in addition to the rest of the top-level logic to be tested.

3. Define a retargeting mode for each group of wrapped cores whose ATPG patterns you want to retarget to run in parallel.

For ATPG pattern retargeting purposes, Tessent requires you to include retargeting mode DFT signals in addition to the DFT signals defined in section “[DFT Signals](#)”. The `retargeting<X>_mode` signals along with the TAM specified with the `add_dft_modal_connections` command ensure that ATPG pattern retargeting occurs correctly. Optionally, you can register your own DFT signals to be used for retargeting purposes.

[Example 6-8](#) on page 235 demonstrates retargeting of two wrapped cores: processor\_core and gps\_baseband.

4. Apply the `add_dft_modal_connections` command to specify the TAM.

During the first insertion pass, you identified the functional pins to be shared with EDT channel pins and added the required auxiliary input and auxiliary output logic. Now you use the TAM to sensitize paths to and from the top-level EDT using the `set_config_value` command.

In [Example 6-8](#) on page 235, the functional input pin GPI01\_0 is shared as an EDT channel input pin. Similarly, the functional output pin GPI02\_0 is shared as an EDT channel output pin.

5. Combine with the top-level EDT mode signal you defined by connecting the EDT channel IOs of the wrapped cores to top-level pins via the TAM. Use the `add_dft_modal_connections` command.
6. Apply the `set_defaults_value` command to specify that Tessent Shell simulate the instruments within the wrapped cores in addition to the top-level instruments.

## Examples

The following dofile example shows that the DFT insertion flow for inserting EDT and OCC into the top level of a chip follows the same basic process as for flat designs as described in “[Second DFT Insertion Pass: EDT, Hybrid TK/LBIST, and OCC](#),” with the exception of the highlighted commands.

### Example 6-8. Top-Level Second DFT Pass

```
set_context dft -rtl -design_id rtl2
set_tsdb_output_directory ../tsdb_outdir
open_tsdb ../../wrapped_cores/processor_core/tsdb_outdir
open_tsdb ../../wrapped_cores/gps_baseband/tsdb_outdir

# Read the tessent cell library
read_cell_library \
  ../../library/standard_cells/tessent/NangateOpenCellLibrary.tcelllib
read_cell_library ../../library/pad_cells/tessent/Nangate_pads.tcelllib

# Read the hard macros
read_verilog ../../library/plls/pll.v -blackbox \
  -exclude_from_file_dictionary

# Read the verilogs
set_design_sources -format verilog -y ../../library/pad_cells/verilog \
  -extension v
read_design chip_top -design_id rtl1 -verbose
read_design processor_core -design_id gate -view graybox -verbose
read_design gps_baseband -design_id gate -view graybox -verbose
set_current_design chip_top
```

```
# Add DFT Signals
# When using boundary scan in logic test without contacting inputs
add_dft_signals int_ltest_en output_pad_disable -create_with_tdr

# Needed for Scan Tested Instruments such as MemoryBIST and boundary scan
add_dft_signals tck_occ_en -create_with_tdr

# When you are using logic test
add_dft_signals ltest_en -create_with_tdr

# Used by top-level EDT
add_dft_signals edt_mode -create_with_tdr

# These are used for the top-level EDT
add_dft_signals test_clock edt_update \
    -source_nodes {TEST_CLOCK_top EDT_UPDATE_top}
add_dft_signals shift_capture_clock edt_clock -create_from_other_signals

# Retargeting mode signals used for ATPG pattern retargeting
add_dft_signals retargeting1_mode retargeting2_mode retargeting3_mode \
retargeting4_mode

# Add TAM connections
# For chip-level EDT mode, the asterisk(*) means do not make connection to
the data inputs
add_dft_modal_connections -ports GPIO1_0 -input_data_destination_nodes * \
    -enable_dft_signal edt_mode
add_dft_modal_connections -ports GPIO2_0 -output_data_source_nodes * \
    -enable_dft_signal edt_mode

# Connect wrapped core EDT channel I/Os to top level for retargeting1_mode
signal
add_dft_modal_connections -ports GPIO1_2 -input_data_destination_nodes
PROCESSOR_1/processor_core_rtl2_controller_c1_edt_channels_in[0] \
    -enable_dft_signal retargeting1_mode
add_dft_modal_connections -ports GPIO2_2 -output_data_source_nodes
PROCESSOR_1/processor_core_rtl2_controller_c1_edt_channels_out[0] \
    -enable_dft_signal retargeting1_mode

# Connect wrapped core EDT channel I/Os to top level for retargeting2_mode
signal
add_dft_modal_connections -ports GPIO1_2 -input_data_destination_nodes
GPS_1/gps_baseband_rtl1_controller_c1_edt_channels_in[0] \
    -enable_dft_signal retargeting2_mode
add_dft_modal_connections -ports GPIO1_2 -input_data_destination_nodes
GPS_2/gps_baseband_rtl1_controller_c1_edt_channels_in[0] \
    -enable_dft_signal retargeting2_mode
...
add_dft_modal_connections -ports GPIO1_0 -output_data_source_nodes GPS_1/
gps_baseband_rtl1_controller_c1_edt_channels_out[0] \
    -enable_dft_signal retargeting2_mode -pipeline_stages 1
add_dft_modal_connections -ports GPIO1_1 -output_data_source_nodes GPS_1/
gps_baseband_rtl1_controller_c1_edt_channels_out[1] \
    -enable_dft_signal retargeting2_mode -pipeline_stages 1
...
```



```

report_dft_modal_connections
set_dft_specification_requirements -logic_test On
add_clocks INCLK -period 10ns
check_design_rules
report_dft_control_points

# Create DFT specification
set_spec [create_dft_specification -sri_sib_list {occ edt} ]
report_config_data $spec
read_config_data -in $spec -from_string {
  occ {
    ijtag_host_interface : Sib(occ);
  }
}
set_id_clk_list [list \
  pll_clock_0 PLL_1/pll_clock_0 \
  INCLK INCLK \
]
foreach {id clk} $id_clk_list {
  set occ [add_config_element OCC/Controller($id) -in $spec]
  set_config_value clock_intercept_node -in $occ $clk
}
report_config_data $spec
read_config_data -in $spec -from_string {
  EDT {
    ijtag_host_interface : Sib(edt);
    ...
  }
}

set_config_value port_pin_name \
  -in $spec/EDT/Controller(c1)/Connections/EdtChannelsIn(1) \
  [get_single_name [get_auxiliary_pins GPIO1_0 -direction input]]
set_config_value port_pin_name \
  -in $spec/EDT/Controller(c1)/Connections/EdtChannelsOut(1)
  [get_single_name [get_auxiliary_pins GPIO2_0 -direction output] ]
report_config_data $spec
process_dft_specification
extract_icl

# By setting this value, all the lower level instruments in the wrapped
# cores are simulated
set_defaults_value /PatternsSpecification/SignoffOptions/
simulate_instruments_in_lower_physical_instances on
create_patterns_specification
process_patterns_specification
set_simulation_library_sources \
  -v ../../library/standard_cells/verilog/*.v \
  -v ../../library/pad_cells/verilog/*.v \
  -y ../../library/plls \
  -y ../../library/memories \
  -extension v
  ...
run_testbench_simulations
exit

```

## Top-Level Scan Chain Insertion Example

For the top level of the chip, additional considerations for scan chain insertion include opening the TSDBs for the wrapped cores and reading in the wrapped core graybox models as you did for the second DFT insertion pass.

---

### Note



Prior to scan chain insertion, perform synthesis as described in section “[Performing Synthesis](#).”

---

Logic that is present at the top-level needs to be scan stitched along with the wrapper chains (external mode) of the cores.

Refer to “[Performing Scan Chain Insertion \(Flat Design\)](#)” (for flat designs) for more information about scan chain insertion. The following dofile example shows that Tessent Shell needs to access the graybox models for each wrapped core.

### Example 6-9. Top-Level Scan Chain Insertion

```
set_context dft -scan -design_id gate
set_tsdb_output_directory ../tsdb_outdir
open_tsdb ../../wrapped_cores/processor_core/tsdb_core
open_tsdb ../../wrapped_cores/gps_baseband/tsdb_core

# Read the tessent cell library
read_cell_library \
  ../../library/standard_cells/tessent/NangateOpenCellLibrary.tcelllib
read_cell_library ../../library/memories/SYNC_1RW_8Kx16.atpglib
read_cell_library ../../library/pad_cells/tessent/Nangate_pads.tcelllib

# Read the hard macros
read_verilog ../../library/plls/pll.v -blackbox
# Read the verilogs
read_verilog ../3.synthesize_rtl/chip_top_synthesized.vg
read_design chip_top -design_id rtl2 -no_hdl -verbose
read_design processor_core -design_id gate -view graybox -verbose
read_design gps_baseband -design_id gate -view graybox -verbose
set_current_design chip_top

check_design_rules
report_clocks

set edt_instance [get_name_list [get_instance -of_module \
                                [get_name [get_icl_module -of_instances chip_top* \
                                -filter tessent_instrument_type==mentor::edt]]] ]
add_scan_mode edt_mode -edt_instance $edt_instance
analyze_scan_chains
report_scan_chains
insert_test_logic
exit
```

## Top-Level ATPG Pattern Generation Example

At the top level, Tessent Shell uses the scan-inserted design data for the chip. In the recommended flow this equates to design ID “gate.” In addition, Tessent uses the graybox models for the wrapped cores so that you can use the external mode of the wrapper chains to run ATPG.

Refer to “[Performing ATPG Pattern Generation](#)” (for flat designs) for information about generating the ATPG patterns. The flow for the top level of a chip builds onto the process by adding in the grayboxes and blackboxes. As with flat designs, you have a choice between using the boundary scan chains for capture or using the primary pins of the chips (using the pads).

The following dofile example generates ATPG patterns at the top level using the stuck-at fault model. The dofile shows that you can read in the stuck-at fault models for the wrapped cores to calculate the total fault coverage for the chip.

### Example 6-10. Top-Level ATPG Pattern Generation

```
set_context pattern -scan
set_tsdb_output_directory ../tsdb_top
open_tsdb ../../wrapped_cores/processor_core/tsdb_core
open_tsdb ../../wrapped_cores/gps_baseband/tsdb_core
read_cell_library \
  ../../library/standard_cells/tessent/NangateOpenCellLibrary.tcelllib
read_cell_library ../../library/memories/ \
  SYNC_1RW_8Kx16.atpglib
read_cell_library ../../library/pad_cells/tessent/Nangate_pads.tcelllib

# Read the hard macros
read_verilog ../../library/plls/pll.v -blackbox \
  -exclude_from_file_dictionary

read_design chip_top -design_id gate
read_design processor_core -design_id gate -view graybox -verbose
read_design gps_baseband -design_id gate -view graybox -verbose
set_current_design chip_top
set_current_mode edt_top_stuck

import_scan_mode edt_mode
report_core_instances

set_static_dft_signal_values tck_occ_en 1
report_static_dft_signal_settings

set_system_mode analysis
add_fault -all
report_statistics -detail
create_patterns
report_statistics -detail
write_tsdb_data -replace
```

```
# Create manufacturing patterns
write_patterns patterns/chip_top_edt_stuck.stil -stil -replace
# Create parallel simulation patterns
write_patterns patterns/chip_top_edt_parallel.v -verilog -parallel \
-replace -scan
# Create a sampled set of patterns for simulation
set_pattern_filtering -sample_per_type 2
write_patterns patterns/chip_top_edt_serial.v -verilog -serial -replace

# Total fault coverage for stuck-at patterns at the chip level including
# the cores
read_faults -module processor_core -mode edt_int_stuck \
-fault_type stuck -merge -graybox
read_faults -module gps_baseband -module edt_int_stuck \
-fault_type stuck -merge -graybox
report_statistics -detail
exit
```

## Performing Top-Level ATPG Pattern Retargeting

For each wrapped core, retarget the internal ATPG patterns for all the modes you have specified and all the fault types.

---

### Note



This procedure is similar to “[Performing ATPG Pattern Generation: Wrapped Core](#)” on page 222.

---

## Procedure

1. Specify the [set\\_context](#) patterns -retargeting command to retarget the ATPG patterns generated for the wrapped cores.
2. Use the same TSDB for ATPG retargeting as you used for ATPG pattern generation.
3. Set the current mode to a unique mode name that, ideally, indicates the core name, the fault type, and the retargeting mode DFT signal you had previously defined.
4. Specify which wrapped core ATPG patterns you are retargeting by enabling the correct retargeting mode DFT signal. Set the [set\\_static\\_dft\\_signal\\_values](#) command to the retargeting mode for this wrapped core.
5. Apply the [add\\_core\\_instances](#) command to specify the wrapped core whose internal ATPG patterns you want to retarget.

**Note:** If you are using this procedure for the automotive flow, also apply the “[add\\_clocks](#) -period” command to add the top-level free-running repair clock.

6. Apply the [read\\_patterns](#) command to read in the stuck-at ATPG patterns that you want to retarget.

## Examples

The following dofile example shows how you would retarget the stuck-at ATPG patterns for the wrapped core, processor\_core.

### Example 6-11. Retarget Stuck-At ATPG Patterns for the Top-Level

```
set_context pattern -scan_retargeting
set_tsdb_output_directory ../tsdb_top
open_tsdb ../../wrapped_cores/processor_core/tsdb_core
open_tsdb ../../wrapped_cores/gps_baseband/tsdb_core

# Read the tessent cell library
read_cell_library \
  ../../library/standard_cells/tessent/NangateOpenCellLibrary.tcelllib
read_cell_library ../../library/memories/SYNC_1RW_8Kx16.atpglib
read_cell_library ../../library/pad_cells/tessent/Nangate_pads.tcelllib
# Read the hard macros
read_verilog ../../library/plls/pll.v -blackbox \
  -exclude_from_file_dictionary
read_design chip_top -design_id gate
read_design processor_core -design_id gate -view graybox -verbose
read_design gps_baseband -design_id gate -view graybox -verbose

set_current_design chip_top

# Retarget processor_core stuck-at patterns
set_current_mode retarget1_processor_stuck
set_static_dft_signal_values retargeting1_mode 1
add_core_instances -instances {PROCESSOR_1} -core processor_core \
  -mode edt_int_stuck

report_core_descriptions
report_clocks

set_system_mode analysis
write_tsdb_data -replace

# Read the patterns to be retargeted
read_patterns -module processor_core -fault_type stuck -mode edt_int_stuck
set_external_capture_options -pll_cycles 5 [lindex [get_timeplate_list] 0]

write_patterns patterns/processor_core_edt_stuck_retargeted.v -verilog \
  -parallel -replace -begin 0 -end 7 -scan
write_patterns patterns/processor_core_edt_stuck_retargeted_serial.v \
  -verilog -serial -replace -Begin 0 -End 2

# Write out the STIL file to use on the tester
write_patterns patterns/processor_core_edt_stuck_retargeted.stil \
  -stil -replace
exit
```

The following dofile snippet shows how you would retarget at-speed transition ATPG patterns for the wrapped core, gps\_baseband. Commands that are not shown are the same as the commands for processor core in [Example 6-11](#) on page 241.

### Example 6-12. Retarget At-Speed Transition ATPG Patterns for the Top-Level

```
# Set the context and open TSDBs ...

# Read in cell libraries, PLL, macro IO pads, designs ...

# Set the current design ...

# Retarget gps_baseband transition patterns
set_current_mode retarget2_gps_transition
set_static_dft_signal_values retargeting2_mode 1
add_core_instances -instances {GPS_1 GPS_2} -core gps_baseband \
    -mode edt_int_transition
report_core_descriptions
import_clocks -verbose
report_clocks

set_system_mode analysis
write_tsdb_data -replace
# Read the patterns to be retargeted
read_patterns -module gps_baseband -mode edt_int_transition \
    -fault_type transition
set_external_capture_options -pll_cycles 5 [lindex[get_timeplate_list] 0]

write_patterns patterns/gps_edt_transition_retargated.v -verilog \
    -parallel -replace -begin 0 -end 7 -scan
write_patterns patterns/gps_edt_transition_retargated_serial.v \
    -verilog -serial -replace -Begin 0 -End 2

# Write out the STIL file to use on the tester
write_patterns patterns/gps_edt_transition_retargated.stil -stil -replace

exit
```

## RTL and Scan DFT Insertion Flow for Sub-Blocks

When performing the DFT insertion flow with sub-blocks, you insert MemoryBIST and pre-DFT DRCs at the sub-block level and then move up to the sub-block's next parent physical block level (where the sub-block is instantiated) to perform synthesis and scan insertion.

Refer to “[Hierarchical DFT Terminology](#)” for more information about sub-blocks.

You may want to use the sub-block flow for any of the following reasons.

- **Multiple instantiations** — You only need to perform the DFT insertion flow once for a sub-block. Thereafter, every instantiation of the sub-block includes the inserted DFT hardware.
- **Small size** — Most sub-blocks are not big enough to be considered their own physical regions.
- **Readiness** — Sometimes the sub-block RTL is complete before the RTL for the physical layout region, thus you can begin DFT insertion on the sub-block as soon as RTL is ready.

**DFT Insertion Flow for the Sub-Block** ..... 243

**DFT Insertion Flow for the Next Parent Level**..... 244

## DFT Insertion Flow for the Sub-Block

The DFT insertion flow for sub-blocks is similar to the insertion flow for physical blocks with a few exceptions.

- You do not perform synthesis or scan insertion at the sub-block level because the sub-block netlist may not exist after synthesis.
- During the second DFT insertion pass, you only insert pre-DFT DRCs.

Typically, you do not insert EDT controllers at the sub-block level because the logic inside of the sub-block is too small, and the sub-block module may not exist after synthesis. You can insert an EDT controller at the next parent level that you dedicate to testing the logic inside of a sub-block. In most cases, this EDT controller is active at the same time as other EDT controllers that are present at the next parent level.

### Note

 The sub-block flow for inserting MemoryBIST and pre-DFT DRCs follows the same steps as described in “[RTL and Scan DFT Insertion Flow for Physical Blocks](#),” with the exception of generating the EDT and OCC hardware. There are slight variations, which are described in the following procedure.

## Procedure

1. [First DFT Insertion Pass: Performing MemoryBIST and Boundary Scan](#). Ensure that you set the design level to “sub\_block” rather than “physical\_block”.

```
set_design_level sub_block
```

2. Second DFT insertion pass: insert pre-DFT DRCs. Follow the steps for the “[Second DFT Insertion Pass: Inserting Block-Level EDT and OCC](#)” procedure, excluding generating the EDT and OCC hardware.

Because you specified that you were working at the sub\_block level in the first DFT insertion pass, you do not need to re-specify this information for the second insertion pass.

After specifying and verifying the DFT requirements, Tessent Shell performs the following tasks automatically:

- At the sub-block level, any static DFT signals you have added are implemented as IJTAG ports rather than inserted via TDRs. Tessent Shell automatically connects the IJTAG ports to the TDR at the next parent level.
- At the next parent level, adds DFT signals such as ltest\_en and async\_set\_reset\_static\_disable. Tessent Shell infers the add\_dft\_control\_point command on the sub-block pins if you have DFT DRCs that need to be fixed.
- At the next parent level, adds the tck\_occ\_en DFT signal if there is a STI-SIB network inside the sub-block.

The following command performs pre-DFT DRC:

```
set_dft_specification_requirements -logic_test on
```

During ICL extraction, Tessent Shell generates the Synopsys Design Constraints (SDC) for the sub-block.

## Results

Proceed to performing the DFT insertion flow on the next parent level where this sub-block is instantiated.

## DFT Insertion Flow for the Next Parent Level

The next parent level in the bottom-up flow where a sub-block is instantiated may either be a physical layout block that is a wrapped core, or the physical layout region that is the chip (the top level). By moving the focus of DFT insertion from child block to immediate parent block, you maintain the correctness and consistency of the test hardware. The following procedure describes the flow when you have a sub-block instantiated at the chip level.



## Prerequisites

- You have performed the DFT insertion flow as described in “[DFT Insertion Flow for the Sub-Block](#)” for the sub-blocks instantiated at this parent level so that you have the design after a clean pre-DFT DRC run.

### Note

 This flow occurs after the DFT insertion process described in “[RTL and Scan DFT Insertion Flow for the Top Chip](#)”. When you have a sub-block inserted inside a wrapped core, you complete the procedures described in “[RTL and Scan DFT Insertion Flow for Physical Blocks](#)” on page 210 after you load the sub-block design.

---

## Procedure

1. [First DFT Insertion Pass: Performing Top-Level MemoryBIST and Boundary Scan](#). When you load the design, open the sub-block’s TSDB and load the design of the sub-block that passed pre-DFT DRCs.
2. [Second DFT Insertion Pass: Inserting Top-Level EDT and OCC](#). When you load the design, open the sub-block’s TSDB, load the full design for the sub-block, and load the design for the top-level after the first DFT insertion pass.
3. [Synthesis](#). Synthesis of the chip-level RTL and the sub-block’s post-DFT inserted RTL occur at the same time. Tessent Shell merges the sub-block into the parent-level logic.  
  
Refer to “[Timing Constraints \(SDC\)](#)” to learn how the synthesis constraints from the sub-block are merged at the next parent level.
4. [Scan Chain Insertion](#). Tessent Shell performs scan chain insertion on the sub-block and parent-level logic at the same time.
5. [ATPG Pattern Generation](#).

## Examples

### Design Loading for the First DFT Insertion Pass

The following example shows opening the TSDB for the sub-block and using read\_design to read in the sub-block (processor\_core) design.

```
# Set the context to insert DFT into RTL-level design
set_context dft -rtl -design_id rtl1
# Set the location of the TSDB. Default is the current working directory.
set_tsdb_output_directory ../tsdb_top
```

```
# Open the TSDBs for all the instantiated sub-blocks
open_tsdb ../../sub_block/tsdb_outdir
# Read the tessent cell library
read_cell_library \
  ../../library/standard_cells/tessent/NangateOpenCellLibrary.tcelllib
read_cell_library ../../library/pad_cells/tessent/Nangate_pads.tcelllib

# Read the hard macros
read_verilog ../../library/plls/pll.v -blackbox \
  -exclude_from_file_dictionary
# Read the design
read_verilog ../../rtl/chip_top.v

# Explicitly load the sub-block netlist
read_design_processor_core -design_id rtl2 -verbose
set_current_design chip_top
set_design_level chip

# Follow the rest of the flow for the first DFT insertion pass for a chip.
```

### Design Loading for the Second DFT Insertion Pass

```
# Set the context to insert DFT. Define a new design ID
set_context dft -rtl -design_id rtl2

# Set the location of the TSDB. Default is the current working directory.
set_tsdb_output_directory ../tsdb_top

# Open the TSDBs for all the instantiated sub-blocks
open_tsdb ../../sub_block/tsdb_outdir

# Read the tessent cell library
read_cell_library \
  ../../library/standard_cells/tessent/NangateOpenCellLibrary.tcelllib
read_cell_library ../../library/pad_cells/tessent/Nangate_pads.tcelllib
# Read the hard macros
read_verilog ../../library/plls/pll.v -blackbox
  -exclude_from_file_dictionary

set_design_sources -format verilog -y ../../library/pad_cells/verilog \
  -extension v
read_design chip_top -design_id rtl1 -verbose
# Explicitly load the sub-block netlist
read_design_processor_core -design_id rtl2 -verbose

set_current_design chip_top

# Follow the rest of the flow for the second DFT insertion pass for a chip
```

## RTL and Scan DFT Insertion Flow for Instrument Blocks

---

When performing the DFT insertion flow with instrument blocks, create a special empty module to insert DFT elements into, then move up to the instrument block's next parent physical block level to perform synthesis and scan insertion.

Use the instrument block flow to avoid having Tessent Shell modify your golden RTL.

|                                                                            |            |
|----------------------------------------------------------------------------|------------|
| <b>DFT Insertion Flow for the Instrument Block .....</b>                   | <b>247</b> |
| <b>Instrument Block DFT Insertion Flow for the Next Parent Level .....</b> | <b>250</b> |

### DFT Insertion Flow for the Instrument Block

Create a special empty module into which the DFT elements are inserted.

#### Procedure

1. Start with an empty DFT module that contains the port definitions for IJTAG and an input and output port for each functional clock. Optionally, create output ports for the `all_test` and `async_set_reset_dynamic_disable` DFT signals, which you can use to control clock multiplexers and disable your asynchronous set and reset signals during scan shifting.

The following RTL shows an example module definition required for the instrument block flow:

```
module elt1_dft_box (  
    input clk,  
    output clk_occ,  
    input ijtag_tck,  
    input ijtag_reset,  
    input ijtag_ce,  
    input ijtag_se,  
    input ijtag_ue,  
    input ijtag_sel,  
    input ijtag_si,  
    output ijtag_so,  
    input [1:0] edt_channels_in,  
    output edt_channels_out,  
    input scan_en,  
    input test_clock,  
    input edt_update,  
    output async_set_reset_dynamic_disable  
);  
  
// Drive the DFT signal to its off value  
  
assign async_set_reset_dynamic_disable = 1'b0;  
  
// Add feedthroughs for ijtag chain  
assign ijtag_so = ijtag_si;  
// Add feedthrough for func clock that will be equipped with OCC  
assign clk_occ = clk;  
endmodule
```

2. Perform the DFT insertion flow within the DFT module as normal, but you must add DFT control points to output ports corresponding to DFT signals.

```

read_verilog design/rtl/elt1_dft_box.v
set_current_design elt1_dft_box
set_design_level instrument_block

set_dft_specification_requirements -logic_test on
add_dft_signals ltest_en int_mode int_ltest_en ext_mode ext_ltest_en
add_dft_signals scan_en test_clock edt_update \
    -source_node {scan_en test_clock edt_update}
add_dft_signals edt_clock shift_capture_clock \
    -create_from_other_signals
add_dft_signals x_bounding_en mcp_bounding_en \
    observe_test_point_en control_test_point_en -create_with_tdr
add_dft_signals async_set_reset_static_disable
add_dft_signals async_set_reset_dynamic_disable \
    -create_from_other_signals
add_dft_signals capture_per_cycle_static_en -create_with_tdr

#Add DFT control points
add_dft_control_points async_set_reset_dynamic_disable \
    -type dynamic_dft_control \
    -dft_signal_source_name async_set_reset_dynamic_disable

add_clocks clk -period [expr {1000.0/300.0}]

check_design_rules

set_spec [create_dft_specification -sri_sib_list {edt lbist occ}]
set_config_value use_rtl_cells off -in_wrapper $spec

read_config_data -in_wrapper $spec -from_string {
    Edt { // {{{
        ijttag_host_interface : Sib(edt);
        Controller(c1) {
            ...
            Connections {
                EdtChannelsIn(1:2) {
                    port_pin_name : edt_channels_in;
                }
                EdtChannelsOut(1) {
                    port_pin_name : edt_channels_out;
                }
            }
        }
    } // }}}
    LogicBist { // {{{
        ijttag_host_interface : Sib(lbist);
        Controller(lbist_1) {
            ...
            Connections {
                shift_clock_src : clk;
            }
        }
        NcpIndexDecoder {
            ...
        }
    } // }}}
    OCC { // {{{
        ijttag_host_interface : Sib(occ);

```

```
        static_clock_control : external;  
        Controller(clk_controller) {  
            clock_intercept_node : clk;  
        }  
    } // }}}  
}  
  
process_dft_specification  
set_system_mode setup  
extract_icl  
write_design_import_script -replace -use_relative_path_to [pwd]
```

## Results

Proceed to performing the DFT insertion flow on the next parent level where this instrument block is instantiated.

## Instrument Block DFT Insertion Flow for the Next Parent Level

The next parent level in the bottom-up flow where an instrument block is instantiated may either be a physical layout block that is a wrapped core, or the physical layout region that is the chip (the top level). By moving the focus of DFT insertion from child block to immediate parent block, you maintain the correctness and consistency of the test hardware. The following procedure describes the flow when you have an instrument block instantiated at the physical\_block level.

## Prerequisites

- You have performed the DFT insertion flow as described in [DFT Insertion Flow for the Instrument Block](#) for the instrument blocks instantiated at this parent level so that you have the design after a clean pre-DFT DRC run.

## Procedure

1. Instantiate and connect the special empty DFT module in your parent module. Modify your RTL to use `clk_occ` wherever it used `clk` so that the clock of all your scannable elements is controlled by the OCC inserted into the block.

---

### Tip



Remember to manually connect in the parent module all the connections for the IJTAG network and all dynamic DFT signals.

---

2. At the next level, read your design normally, and then load the DFT module using the [read\\_design](#) command. Specify your clocks and run the [check\\_design\\_rules](#) command to validate and store them. Then, call the [extract\\_icl](#) command.

At that point, it is as if you had inserted the DFT elements from this level using the normal flow. After ICL is extracted, you can go to the DFT context and verify that the

RTL has a controllable clock and reset signals. DRC violation errors indicate if the RTL is not clean, which you must fix manually.

```
read_design elt1_dft_box -design_id dft
read_verilog design/rtl/elt1.v
set_current_design elt1
set_design_level physical_block
add_clocks CLK1 -period [expr {1000.0/300.0}]
check_design_rules
extract_icl
report_dft_signals
write_design_import_script -replace -use_relative_path_to [pwd]

#Check that it passes the pre-DFT DRC rules
set_context dft
set_dft_specification_requirements -logic_test on
set_drc_handling dft_c9 -auto_fix off
check_design_rules

# Generate patterns
set_spec [create_pattern_specification]
process_pattern_specification

# Run Simulation
set_simulation_library_sources -v \
    ../../../../library/standard_cells/verilog/NangateOpenCellLibrary.v
run_testbench_simulations
```

3. [Synthesis](#). Synthesis of the chip-level RTL and the `instrument_block`'s post-DFT inserted RTL occur at the same time. Tessent Shell merges the `instrument_block` into the parent-level logic.

Refer to "[Timing Constraints \(SDC\)](#)" to learn how the synthesis constraints from the `instrument_block` are merged at the next parent level.

4. [Scan Chain Insertion](#). Tessent Shell performs scan chain insertion on the `instrument_block` and parent-level logic at the same time.
5. [ATPG Pattern Generation](#).

# RTL and DFT Insertion Flow With Third-Party Scan

The RTL and DFT insertion flow enables you to use a third-party scan insertion tool instead of Tessent Scan. Regardless of which scan insertion tool you use, you must follow a number of sequences in the DFT insertion flow that include creating the logic for internal and external mode, and the multiplexing mechanism that Tessent Scan normally inserts in order to support hierarchical ATPG.

The workflow uses the standard hierarchical DFT insertion flow, both for each wrapped core and, once you have inserted DFT in each wrapped core, for top chip.

See “[Hierarchical DFT Terminology](#)” on page 205.

**Note**  
 If your design consists of wrapped cores as your lower level physical blocks, and the wrapped cores do not contain embedded pad IOs, refer to “[How to Use Boundary Scan in a Wrapped Core](#)” on page 721.

**Caution**  
 Third-party tools may have different insertion capabilities compared to Tessent Scan. This may affect the quality of the results you achieve.

|                                                                 |            |
|-----------------------------------------------------------------|------------|
| <b>DFT Insertion Flow With Third-Party Scan Insertion .....</b> | <b>252</b> |
| <b>Wrapped Core DFT Insertion With Third-Party Scan .....</b>   | <b>254</b> |
| <b>Top Chip DFT Insertion with Third-Party Scan .....</b>       | <b>269</b> |

## DFT Insertion Flow With Third-Party Scan Insertion

The DFT insertion flow with third-party scan insertion requires you to manually collect and provide certain data during hierarchical ATPG. Normally when using Tessent Scan for scan insertion and stitching, the information that is needed for performing hierarchical ATPG is transferred.

### Prerequisites

Before starting this flow, you should identify the DFT functions you need to insert in each core and identify how many scan chains and wrapper cells are needed for each core.

**Note**  
 Tessent Scan automatically concatenates scan chain and wrapper chains into mixed chains to achieve the number of scan channels on the EDT logic.



## CTL (IEEE 1450.6) Generation for Tessent IP With Embedded Scan Segments

Certain Tessent Shell IP blocks include embedded scan segments that must be a part of scan chains. To simplify integration with third-party scan insertion tools, Tessent Shell generates the CTL (IEEE 1450.6) file whenever generating the `tcd_scan` file for test IP.

The tool generates a CTL file that includes information about embedded scan segments for the following applications:

- A CTL file to describe the embedded chain to help in connecting the programmable registers inside the following IP:
  - OCC
  - STI SIB or Sib(STI)
- A CTL file to describe the controller chain mode (CCM) scan segment, when the tool generates test IP with enabled `segment_per_instrument` property. This applies to the following IP:
  - LogicBIST controller
  - EDT controller (available only when used as part of hybrid TK/LBIST IP)
  - Single Chain Mode Logic controller (part of hybrid TK/LBIST IP)
  - InSystemTest (MissionMode) controller

For information about CCM, see “[Controller Chain Mode](#)” in the *Hybrid TK/LBIST User’s Manual*. For information about CTL files, see “[Default Generation of CTL Files](#)” in the *Tessent Shell Reference Manual*.

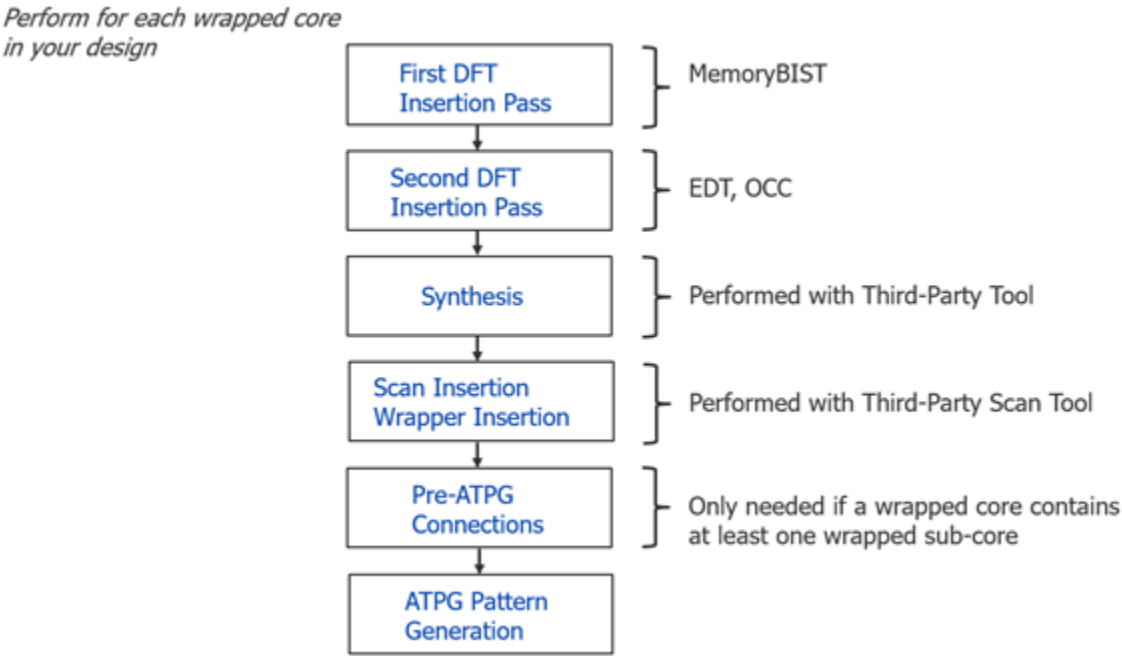
## Wrapped Core DFT Insertion With Third-Party Scan

For each wrapped core, perform a two-pass pre-scan DFT insertion process as you would for a flat design except that in the first DFT insertion pass, do not insert boundary scan unless you have embedded pad IO macros present inside the core. The flow details are unique to working with wrapped cores and using a third-party scan insertion tool to insert the scan.

Refer to “[RTL and Scan DFT Insertion Flow for Physical Blocks](#)” on page 210 for overall details.

**Note**  
This discussion assumes your design consists of wrapped cores as your lower level physical blocks, and the wrapped cores do not contain embedded pad IOs.

**Figure 6-21. Two-Pass Insertion Flow for RTL, Wrapped Cores, and Third-Party Scan**



|                                                                            |     |
|----------------------------------------------------------------------------|-----|
| Perform Two-Pass DFT Insertion and Synthesis for Wrapped Cores .....       | 255 |
| Third-Party Scan Insertion for Wrapped Cores .....                         | 256 |
| Writing the Design Back to the TSDB .....                                  | 260 |
| Make Pre-ATPG Connections with Third-Party Scan for Wrapped Cores .....    | 263 |
| Performing Wrapped Core Graybox Generation and ATPG Pattern Generation.... | 265 |

## Perform Two-Pass DFT Insertion and Synthesis for Wrapped Cores

This flow for wrapped cores is the same as the DFT insertion flow for physical blocks except you insert the scan with a third-party scan tool instead of Tessent Scan.

### Procedure

1. [First DFT Insertion Pass: Performing MemoryBIST and Boundary Scan](#). Ensure you set the design level to “physical\_block”.

```
set_design_level physical_block
```

2. [Second DFT Insertion Pass: Inserting Block-Level EDT and OCC](#). Ensure you set the design level to “physical\_block”.

- a. During the second insertion pass and when specifying the DFT signals, follow the procedure defined in “[Specifying and Verifying the DFT Requirements: DFT Signals for Wrapped Cores](#)” on page 216.

The following are the internal and external modes that are required for mode switching:

```
add_dft_signals int_ltest_en ext_ltest_en int_mode ext_mode \
    -create_with_tdr
```

```
add_dft_signals input_wrapper_scan_en output_wrapper_scan_en \
    -create_from_other_signals
```

- b. [Creating the DFT Specification](#).

The test case implements compressed ATPG test using Tessent TestKompess. You define the EDT logic for Tessent TestKompess and the Tessent OCC in an EDT DftSpecification wrapper as follows:

```
read_config_data -in $spec -from_string "  
  OCC {  
    ijtag_host_interface : Sib(occ);  
    type : standard;  
  }  
  EDT {  
    ijtag_host_interface : Sib(edt);  
    Controller (c1) {  
      longest_chain_range : 50, 65;  
      scan_chain_count : 80;  
      input_channel_count : 2;  
      output_channel_count : 1;  
      Connections {  
        EdtChannelsIn(2:1) {  
          port_pin_name : edt_channel_in[1:0];  
        }  
        EdtChannelsOut(1:1) {  
          port_pin_name : edt_channel_out[0:0];  
        }  
      }  
    }  
  }  
}"
```

You define the scan\_chain\_count as follows:

scan\_chain\_count = number of internal only scan chains + number of wrapper chains

This number covers all chains that the EDT logic at the current level needs to connect to. In other words the EDT hardware need to be connected to both the internal only chains and wrapper chains.

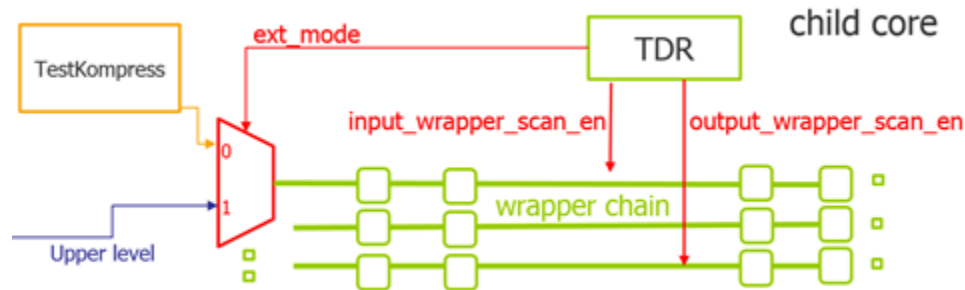
3. [Generating the EDT, Hybrid TK/LBIST, and OCC Hardware.](#)
4. [Extracting the ICL Module Description.](#)
5. [Generating ICL Patterns and Running Simulation.](#)
6. [Performing Synthesis.](#)

## Third-Party Scan Insertion for Wrapped Cores

You can use a third-party scan insertion tool to insert scan and perform scan stitching for wrapped cores in your design. The actual process depends on the third-party scan insertion tool you use. You must wrap all the inputs and outputs of hierarchical physical blocks and perform scan insertion and stitching in your design. Your choice of wrapper cells depends on the capabilities of the third-party scan tool you are using.

For hierarchical ATPG, you must insert multiplexers before each wrapper chain to multiplex the primary input from the top level and the current level of the EDT scan input. Figure 6-22 shows an abstract level depiction of the multiplexing logic. The inclusion of these multiplexers is mandatory for hierarchical ATPG support using Tessent tools.

**Figure 6-22. DFT Signals and Multiplexer Logic Support Hierarchical ATPG**



To the child cores, the preregistered static DFT signals `ext_mode` and `int_mode` control these multiplexers as follows:

- **External Mode** — The `ext_mode` DFT signal is active, and the `int_mode` DFT signal is inactive. The top level drives input into the wrapper chains through the wrapper's scan in on the core boundary.
- **Internal Mode** — The `ext_mode` DFT signal is inactive and the `int_mode` DFT signal is active. The EDT logic drives all the scan chains inside the hierarchical physical block (both internal only and wrapper chains).

When using this configuration, you must control the corresponding TDR bits for mode switching. See “[Performing Wrapped Core Graybox Generation and ATPG Pattern Generation](#)” on page 265 for a detailed explanation.

## Usage Guidelines

Use the following guidelines and criterion for configuring your third-party scan insertion tool for wrapper and scan stitching:

- For the hierarchical physical block on which you intend to perform scan insertion, you have completed the “[Perform Two-Pass DFT Insertion and Synthesis for Wrapped Cores](#)” on page 255 flow.
- You have identified the scan chains of the current core and how many of the scan chains should be wrapper chains. The total number of scan chains is the same as the EDT `scan_chain_count`, as defined in the previous DFT insertion step. The rest of the scan chains can be specified and used as core chains.

- During wrapper analysis, you must exclude the following pins from any wrapper insertion:
  - IJTAG-related pins
  - edt\_channel\_input pins
  - edt\_channel\_output pins
  - scan\_enable pins
  - Any clock pins
- The DFT signals for input and output wrapper scan enable that you have defined during [the second DFT insertion pass](#) should be used as scan enable signals for input and output wrapper chains, respectively, during wrapper analysis and insertion.
- To run ATPG after scan insertion, you must create and supply a test procedure file for the [Graybox generation step](#) to trace and control the wrapper chains. A test procedure file tells the tool how to clock the scan chains, and the tool automatically passes this to ATPG in the Tessent Shell flow. A graybox does not normally include an OCC and EDT logic for tracing and controlling wrapper chains; consequently, you must create the test procedure file manually when using the third-party scan insertion flow.

See “[Test Procedure File](#)” on page 759 for complete details on how you create and use a test procedure file. See also “[Example Test Procedure File](#)” on page 261.

- The Tessent Shell environment [get\\_dft\\_info\\_dictionary](#) command offers you a method to access the Tessent Database (TSDB) to use this information with your third-party scan insertion tool.

The command reads the scan information from the TSDB’s [dft\\_inserted\\_designs](#) directory, specifically in a Tcl dictionary file named:

`<design_name>.dft_info_dictionary`

The Tcl dictionary contains the information about the DFT inserted in the design that must be considered during scan insertion when using a third-party scan insertion tool. See “[Example dft\\_info\\_dictionary File](#)” on page 262.

All instances under "non\_scannable\_instance\_list" should be regarded as non-scan. All modules under "modules\_with\_chains" should be considered as sub-chains and should not be modified during scan insertion.

- The Tessent Shell environment provides a mechanism for generating an example Tcl script you can customize for your third-party scan insertion tool. Perform the following steps:
  - a. In Tessent Shell, run the following commands in the dft or pattern context:

```
set_tsdb_output_directory ../tsdb_outdir
read_design <design_name>
puts [ get_dft_info_dictionary -example_usage_script ]> ./usage_template.tcl
```

The tool creates an example usage script as shown in the following excerpt:

```
...
puts "Declare the Non-Scannable instances"

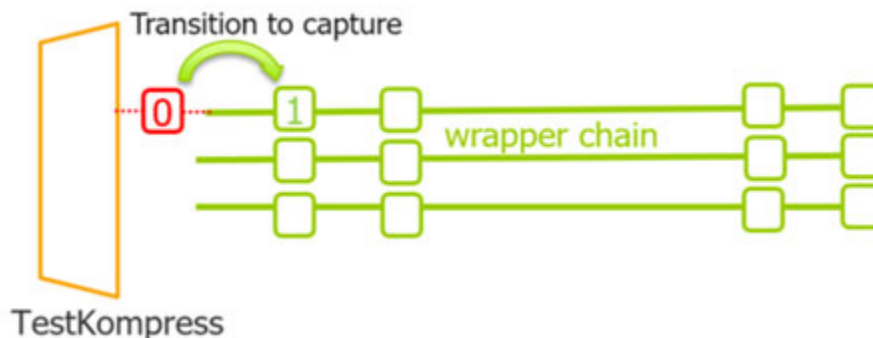
set non_scannable_instance_list [dict get $tessent_dft_info_dict
non_scannable_instance_list]
if {[llength $non_scannable_instance_list] > 0} {
    ### Command to declare the Non-Scannable instances
    add_nonscan_instances [get_instances
$non_scannable_instance_list]
}
...
```

- b. Use the script as a starting point to convert the dictionary into specific commands for your third-party scan insertion tool. In the script, the tool inserts pound signs (#) with comments that specify the actions your third-party tool must perform. You must replace those comments with the corresponding third-party tool-specific commands to the file.
- c. In the third-party scan insertion tool, source the Tessent dictionary from
 

```
../tsdb_outdir/dft_inserted_designs/
design_name_last_DFT_insertion_design_id.dft_inserted_design/
design_name.dft_info_dictionary.
```

An example path and filename is `../tsdb_outdir/dft_inserted_designs/gps_baseband_rtl_edt.dft_inserted_design/gps_baseband.dft_info_dictionary`.
- d. In the third-party scan insertion tool, source the example usage script you modified in previous steps.
- For the hierarchical transition fault model, you may need to insert one additional at-speed cell at the beginning of each wrapper chain to make the transition on the first cell of each wrapper chain. This way, you achieve the best coverage. The method you use depends on the third-party insertion tool. [Figure 6-23](#) shows the logic.

**Figure 6-23. At-Speed Flows in the Wrapper Chain**



- If required and after you have inserted the scan using your third-party tool, you can follow the procedure described in [“Make Pre-ATPG Connections with Third-Party Scan for Wrapped Cores”](#) on page 263 if required.
- If no pre-ATPG connections are required, then you can load the scan-inserted netlist into Tessent Shell and write the modified netlist and scan information back to the Tessent Shell Database (TSDB).

## Writing the Design Back to the TSDB

After completing scan insertion by your third-party scan insertion tool, you must write the scan-inserted netlist back to the TSDB to associate the netlist with Tessent DFT instruments. You create this association by creating a softlink to the scan inserted netlist in TSDB, which also creates the Tessent Core Description files and the ICL information.

### Prerequisites

- You have completed scan insertion with a third-party tool.

Use the following procedure to save a scan-inserted netlist to the TSDB. The line numbers are represented in the Example dofile under Examples:

### Procedure

1. Set the context (see lines 1-3).
2. Set the TSDB location if not already set (see line 6).
3. Load the cell library (see line 9).
4. Load the scan-inserted netlist (see line 12). This netlist is modified by your third-party scan insertion tool and contains the scan cells.
5. Load the supporting DFT files (except the netlist) from the last DFT insertion pass and set the current design (see lines 15-17).
6. Set the design level. Make sure you specify “physical\_block” (see line 21).
7. Write the design information to the TSDB and create the softlink (see line 22).

### Examples

#### Example dofile

```
1 # Use the design_id as "scan." Use this in all the ATPG
2 # runs that this design is read in.
3 set_context patterns -ijtag -design_id <scan>
4
5 # Set the location of the TSDB. Default is the current working directory
6 set_tsdb_output_directory ../tsdb_outdir
7
8 # Read the Tessent Cell Library
9 read_cell_library \
10 ../../library/standard_cells/tessent/NangateOpenCellLibrary.tcelllib
```



```

11
12 # Read the scan inserted netlist and elaborate the design
13 read_verilog ../3.synthesize_rtl/<current_core>_scan.vg
14
15 # Read the -no_hdl from the last DFT insertion pass
16 read_design <current_core> -design_id <rtl2> -no_hdl -verbose
17 read_verilog ../../../../library/memories/SYNC_1RW_8Kx16.v
18 set_current_design <current_core>
19
20 # Specify the design level before writing out a softlink of the design in
21 # TSDB
22 set_design_level physical_block
23 write_design -tsdb -softlink_netlist -verbose
24 exit

```

### Example Test Procedure File

```

set time scale 1.000000 ns ;
timeplate gen_tp1 =
    force_pi 0 ;
    measure_po 10 ;
    pulse_clock 20 10 ;
    pulse_clock2 20 10;
    period 40 ;
end;
procedure shift =
    scan_group grp1 ;
    timeplate gen_tp1 ;
    // cycle 0 starts at time 0
    cycle =
        force_sci ;
        measure_sco ;
        pulse_clock1;
        pulse_clock2;
    end;
end;

procedure load_unload =
    scan_group grp1 ;
    timeplate gen_tp1 ;
    // cycle 0 starts at time 0
    cycle =
        force_clock1 0;
        force_clock2 0;
        force_scan_enable 1 ;
    end ;
    apply_shift 76;
end;

```

### Example dft\_info\_dictionary File

```
set tessent_dft_info_dict {
  version 1
  dft_signals {
    dft_signal_name {
      connection_node_name node_name
      connection_node_type port|pin
      forced_value_in_pre_scan_drc 0|1
    }
  }
  modules_with_chains {
    module_name {
      pre_scan_drc_on_boundary_only 0|1
      module_type normal|memory|occ
      is_hard_module 0|1
      internal_scan_only 0|1
      allow_scan_out_retiming 0|1
      instance_list {inst_name ...}
      scan_en_ports|ltest_en_ports|set_reset_ports {
        port_name { active_polarity 0|1
        }
      }
      clock_ports {
        port_name {
          off_state 0|1
        }
      }
      clock_out_ports_or_pins {
        port_or_pin_name {}
      }
      scan_chains {
        scan_chain_name {
          length auto|int
          scan_in_port port_name
          scan_in_clock_name port_name
          scan_in_clock_inversion 0|1
          scan_out_port port_name
          scan_out_clock_name port_name
          scan_out_clock_inversion 0|1
        }
      }
    }
  }
  non_scannable_instance_list {inst_name ...}
  edt_instances {
    instance_name {
      edt_module_name module_name
      scan_chains {
        chain_name {
          scan_in_port port_name
          scan_out_port port_name
        }
      }
    }
  }
}
```

## Make Pre-ATPG Connections with Third-Party Scan for Wrapped Cores

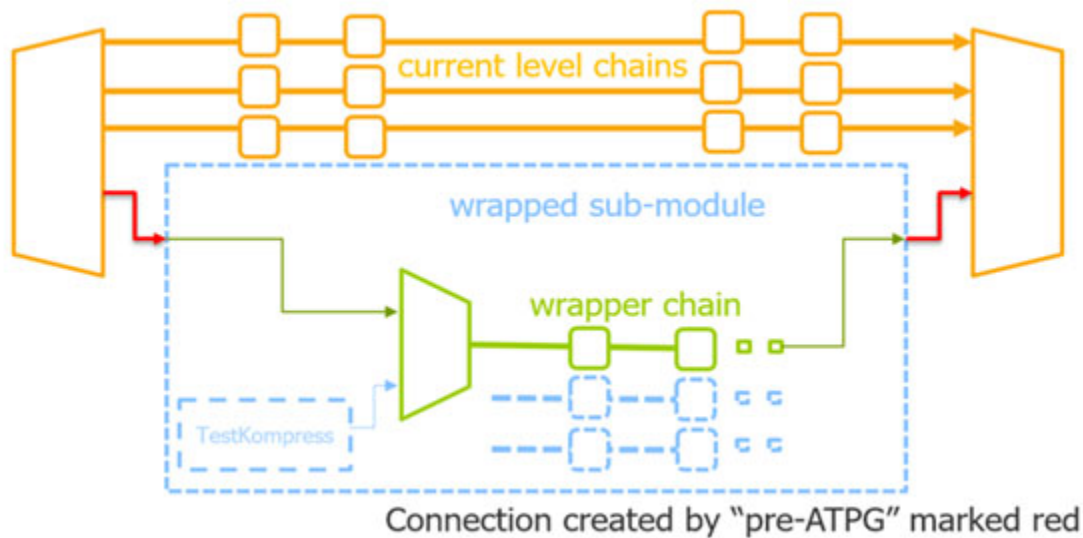
You perform this step in flow only if the wrapped core has at least one child wrapped core or there is one upper layer beyond the current module.

Otherwise, skip this step in the flow and proceed directly to [“Performing Wrapped Core Graybox Generation and ATPG Pattern Generation”](#) on page 265.

This step includes two parts:

1. It creates connections from the current level EDT logic to the wrapped child cores' wrapper chains for the paths depicted in red in [Figure 6-24](#).

**Figure 6-24. Current Level Path to Child Core Wrapper Chains**



The external mode logic and chains of child cores become part of the current test coverage.

2. It also creates logic from the current level wrapper chain to the upper-level logic, marked in green in [Figure 6-24](#). Logic added at this stage includes the muxes controlled by dft\_signals, ports at the module boundary, and connections between those ports and the EDT logic. This enables the external mode testing of the wrapped child core from the upper level.

### Note



The line numbers used in this procedure refer to the command flow dofile in [“Example Dofile for Pre-ATPG Connections for Wrapped Cores”](#) on page 264.

## Prerequisites

- You must have performed the two-pass DFT insertion as described in “[Perform Two-Pass DFT Insertion and Synthesis for Wrapped Cores](#)” on page 255.
- You must have inserted scan using your third-party scan insertion tool per the guidelines provided in “[Third-Party Scan Insertion for Wrapped Cores](#)” on page 256.

## Procedure

1. Load the design. (See lines 1-8.)
2. Change to insertion mode. (See line 10.)
3. Make the red connections shown in [Figure 6-24](#) on page 263. (See lines 13-27.)
4. Make the ports, logic, and connections shown in green in [Figure 6-24](#). (See lines 29-44.)
5. Save the design. (See line 46.)
6. Write the design back to the TSDB. (See lines 47-72.)

## Examples

### Example Dofile for Pre-ATPG Connections for Wrapped Cores

```
1  set_context dft -no_rtl
2
3  read_verilog ../3.synthesis/<current_design_name>_scan.vg
4
5  # Read the tessent cell library
6  read_cell_library \
7    ../../library/standard_cells/tessent/NangateOpenCellLibrary.tcelllib
8
9  set_current_design <current_design_name>
10
11 set_system_mode insertion
12
13
14 # Make the connections marked in red in the preceding figure (see Step 1)
15
16 # empty pins of edt logic were grounded, need to delete before connection
17 delete_connection <current_edt_instance>/edt_scan_out[]
18
19
20 delete_connection <current_edt_instance>/edt_scan_in[]
21
22
23 # connect edt scan-in/scan-out to ext_wsi/ext_wso of every chain
24 create_connection /<child_core_instance>/ext_wsi[] \
25     <current_edt_instance>/edt_scan_in[]
26
27 create_connection /<child_core_instance>/ext_wso[] \
28     <current_edt_instance>/edt_scan_out[]
29
30 # Make the ports, logic, and connection marked in green in the preceding
    figure (see Step 2)
31 delete_connection <current_edt_instance>/edt_scan_in[]
```

```
32 # create ports for upper level to connect to
33 create_port ext_wsi[]
34 create_port ext_wso[] -direction output
35 # create mux logic and control logic
36 create_instance wrapper_mux0 -of_module mux21
37 create_instance ext_mode_inv -of_module inv01
38 create_instance mode_and -of_module and02
39 create_connection <current_tdr_instance>/ext_mode ext_mode_inv/A
40 create_connection <current_tdr_instance>/int_mode mode_and/A0
41 # connect mux between ports and wrapper chain beginning/ends
42 create_connection wrapper_mux0/A1 <edt_instance>/edt_scan_in[]
43 create_connection wrapper_mux0/A0 ext_wsi[]
44 create_connection wrapper_mux0/Y <wrapper chain beginning cell or buffer
  before it>/A
45 create_connection ext_wso[] <wrapper chain ending cell or buffer after
  it>/Q
46
47 write_design -output_file ../3.synthesis/<current_design_name>.vg
48
49 # Use the design_id as "scan." Use this in all the ATPG
50 # runs that this design is read in.
51
52 set_context patterns -ijtag -design_id <scan>
53
54 # Set the location of the TSDB. Default is the current working directory
55 set_tsdb_output_directory ../tsdb_outdir
56
57 # Read the Tessent Cell Library
58 read_cell_library \
59   ../../library/standard_cells/tessent/NangateOpenCellLibrary.tcelllib
60
61 # Read the scan inserted netlist and elaborate the design
62 read_verilog ../3.synthesize_rtl/<current_core>_scan.vg
63
64 # Read the -no_hdl from the last DFT insertion pass
65
66 read_design <current_core> -design_id <rtl2> -no_hdl -verbose
67
68 read_verilog ../../../../library/memories/SYNC_1RW_8Kx16.v
69
70 set_current_design <current_core>
71
72 # Specify the design level before writing out a softlink of the design in
73 # TSDB
74 set_design_level physical_block
75 write_design -tsdb -softlink_netlist -verbose
76 exit
```

## Performing Wrapped Core Graybox Generation and ATPG Pattern Generation

This step in the flow creates the graybox model of the current wrapped core and generates ATPG patterns. You must perform this step for each wrapped core in your design.

## Prerequisites

- You must have completed the “[Performing Wrapped Core Graybox Generation and ATPG Pattern Generation](#)” on page 265 step for each wrapped core.
- You must have inserted scan with your third-party scan insertion tool per the guidelines cited in “[Third-Party Scan Insertion for Wrapped Cores](#)” on page 256 for each wrapped core, and written the scan-inserted netlist back into the Tessent Shell Database as described in “[Writing the Design Back to the TSDB](#)” on page 260.
- If required, make ATPG connections using the process outlined in “[Make Pre-ATPG Connections with Third-Party Scan for Wrapped Cores](#)” on page 263 for each wrapped core.

## Procedure

1. Generate the graybox model for the wrapped core and add a scan group of wrapper chains by importing the [test procedure file](#) you created during [Third-Party Scan Insertion for Wrapped Cores](#)—see “[Example Graybox Generation](#)” on page 267 and “[Performing ATPG Pattern Generation: Wrapped Core](#)” on page 222.

To generate external patterns to check fault coverage for the wrapped core, refer to “[Example External ATPG Pattern Generation](#)” on page 268. You do not retarget these patterns.

As long as the scan mode (scan chains and test logic) is stitched conforming to DRC, Tessent Shell generates internal ATPG patterns formed in the correct way.

2. Save the design and write the patterns to the TSDB using the [write\\_tsdb\\_data](#) command.

```
write_tsdb_data -replace
```

3. Repeat Steps 1 and 2 for each wrapped core to generate transition patterns.

All of the information is passed by the Tessent Shell through the TSDB.

## Results

When you complete graybox and ATPG for each wrapped core, perform scan insertion for the top chip. See “[Top Chip DFT Insertion with Third-Party Scan](#)” on page 269 for complete flow details.

## Examples

### Example Graybox Generation

```
set_context pattern -scan -design_id <scan>
set_tsdb_output_directory ../tsdb_outdir
# Read the tessent cell library
read_cell_library \
  ../../../../library/standard_cells/tessent/NangateOpenCellLibrary.tcelllib
read_design <current_design_name> -design_id <scan>
set_current_design <current_design_name>
# set memory modules black box
add_black_box -module SYNC_1RW_32x16_RC_BISR
add_black_box -module SYNC_1RW_32x4
add_clock 0 clock1
# Graybox generation requires core configured to external mode by
# setting DFT signal values:
set_static_dft_signal_values int_mode 0
set_static_dft_signal_values ext_mode 1
set_static_dft_signal_values int_ltest_en 0
set_static_dft_signal_values ext_ltest_en 1
# exclude edt_channels for graybox
set_attribute_value [get_ports *edt_channel*] \
  -name ignore_for_graybox -value true
# import wrapper chain information by importing testproc file of
# wrapper chains
add_scan_group grp1 ../4.scan_insertion/wrapper.testproc
# add every wrapper chain from input pin to output pin
add_scan_chains chain1 grp1 ext_wsi[0] ext_wso[0]
add_scan_chains chain2 grp1 ext_wsi[1] ext_wso[1]
add_scan_chains chain3 grp1 ext_wsi[2] ext_wso[2]
set_system_mode analysis
# external mode information saved in tsdb
write_design -graybox -tsdb -verbose
exit
```

### Example External ATPG Pattern Generation

```
set_context pattern -scan -design_id <scan>
set_tsdb_output_directory ../tsdb_outdir
# Read the tessent cell library
read_cell_library \
    ../../../../library/standard_cells/tessent/NangateOpenCellLibrary.tcelllib
read_design <current_design_name>-design_id <scan> -view graybox
set_current_design <current_design_name>
add_clock 0 clock
# create mode information for external ATPG
set_current_mode ext_multi_stuck -type external
# configure core to external mode
set_static_dft_signal_values int_mode 0
set_static_dft_signal_values ext_mode 1
set_static_dft_signal_values int_ltest_en 0
set_static_dft_signal_values ext_ltest_en 1
# for transition fault ATPG, replace stuck with transition
set_fault_type stuck
# import wrapper chain information
add_scan_group grp1 ../4.scan_insertion/wrapper.testproc
add_scan_chains chain1 grp1 ext_wsi[0] ext_wso[0]
add_scan_chains chain2 grp1 ext_wsi[1] ext_wso[1]
add_scan_chains chain3 grp1 ext_wsi[2] ext_wso[2]
set_system_mode analysis
create_pattern
write_tsdb_data -replace
exit
```

### Example Internal ATPG Pattern Generation

```
set_context patterns -scan -design_id <scan>
set_tsdb_output_directory ../tsdb_outdir
# Read the tessent cell library
read_cell_library \
    ../../../../library/standard_cells/tessent/NangateOpenCellLibrary.tcelllib
read_design <current_design_name> -design_id <scan>
add_black_box -module SYNC_1RW_32x16_RC_BISR
add_black_box -module SYNC_1RW_32x4
set_current_design <current_design_name>
# current_mode is used when add core instance at top level
set_current_mode edt_int_stuck -type internal
# import core instances of OCC, EDT and sib_sti if exist
add_core_instances -module <occ_instance_name> -parameter_values {}
add_core_instances -module <edt_instance_name>
add_core_instances -module <sib_sti_instance_name>
# for transition fault ATPG, replace stuck with transition
set_fault_type stuck
add_clock 0 clock -pulse_always
# configure core to internal mode
set_static_dft_signal_values int_mode 1
set_static_dft_signal_values ext_mode 0
set_static_dft_signal_values int_ltest_en 1
set_static_dft_signal_values ext_ltest_en 0
report_static_dft_signal_settings
set_system_mode analysis
create_pattern
write_tsdb_data -replace
exit
```



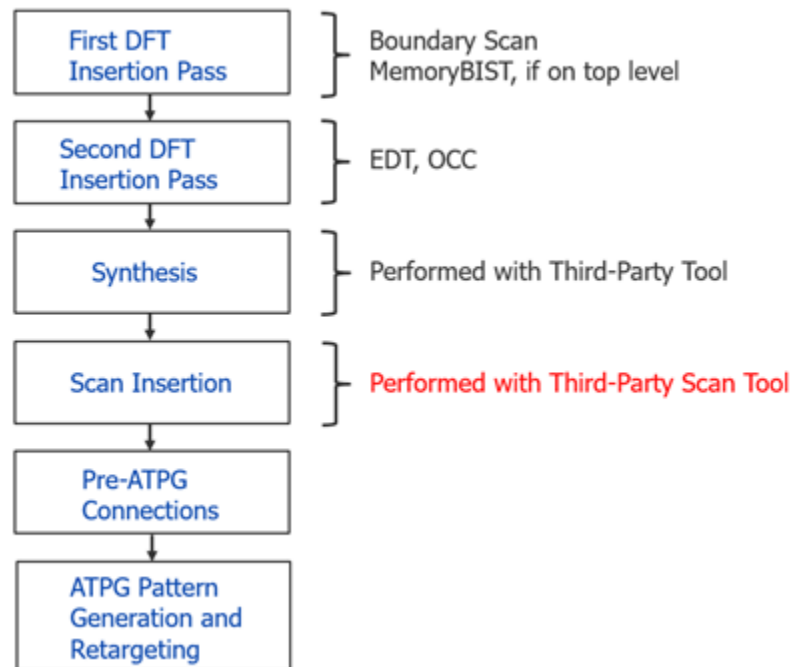
## Top Chip DFT Insertion with Third-Party Scan

After performing the RTL and scan DFT insertion flow for each wrapped core in your design, you can perform the DFT insertion process for the top-level chip design using a third-party scan insertion tool to insert the scan.

See “[RTL and Scan DFT Insertion Flow for the Top Chip](#)” on page 228 for overall flow details, and a detailed description of the Test Access Mechanism (TAM).

**Figure 6-25. Two-Pass Insertion Flow for RTL, Top Level, and Third-Party Scan**

*Perform after wrapped core  
DFT insertion*



|                                                                               |     |
|-------------------------------------------------------------------------------|-----|
| Perform Two-Pass Insertion and Synthesis for Top Chip .....                   | 269 |
| Third-Party Scan Insertion for Top Chip .....                                 | 270 |
| Make Pre-ATPG Connections with Third-Party Scan for Top Chip .....            | 272 |
| Perform Top-Chip ATPG Pattern Generation and Wrapped-Core Pattern Retargeting | 274 |

## Perform Two-Pass Insertion and Synthesis for Top Chip

After performing the RTL and scan DFT insertion flow for each wrapped core in your design, you can perform the DFT insertion process for the top-level chip design.

## Prerequisites

- Before you perform the top level steps, all the child cores must be ready with their retargetable patterns. A spec form for the top level logic is also recommended to define the number of scan chains and what DFT instruments need to be inserted.
- You must have completed the [Perform Two-Pass DFT Insertion and Synthesis for Wrapped Cores](#) step for each wrapped core.
- You must have completed the [Third-Party Scan Insertion for Wrapped Cores](#) step for each wrapped core.
- If required, you must have completed the [Make Pre-ATPG Connections with Third-Party Scan for Wrapped Cores](#) step for each wrapped core.
- You must have completed the [Performing Wrapped Core Graybox Generation and ATPG Pattern Generation](#) for each wrapped core.

## Procedure

1. Set the design level to “chip” for the top level of the chip and insert DFT for MemoryBIST and Boundary Scan. Refer to “[First DFT Insertion Pass: Performing MemoryBIST and Boundary Scan](#)” on page 180.
2. Insert top-level EDT and OCC. Refer to “[Second DFT Insertion Pass: EDT, Hybrid TK/LBIST, and OCC](#)” on page 184.

---

### Note



You must correctly define the scan\_chain\_count of the EDT, to include all scan chain counts of child cores in addition to the top-level scan chain count.

---

3. Perform synthesis. Refer to “[Performing Synthesis](#)” on page 196.

## Results

You now have a design ready for top level scan insertion with your third-party scan insertion tool. Proceed to “[Third-Party Scan Insertion for Top Chip](#)” on page 270.

## Third-Party Scan Insertion for Top Chip

You can use a third-party scan insertion tool to insert scan into the top chip of your design. Your process depends on the third-party scan insertion tool you use.

## Usage Guidelines

- Top level logic does not require wrapper chains as all primary inputs and primary outputs are controllable.

- The Tessent Shell environment `get_dft_info_dictionary` command offers you a method to access the Tessent Database (TSDB) to use this information with your third-party scan insertion tool.

The command reads the scan information from the TSDB's `dft_inserted_designs` directory, specifically in a Tcl dictionary file named `<design_name>.dft_info_dictionary`.

This file can be sourced to any Tcl script engine. The file contains the information about the DFT inserted in the design that must be considered during scan insertion when using a third-party scan insertion tool and contains the following sections:

- `dft_signals` — Contains all of the DFT signals.
  - `modules_with_chains` — Contains all modules that already scanned, which means that they should be stitched into scan chains as sub-chains.
  - `non_scannable_instance_list` — Contains those instances set to non-scan during scan insertion.
  - `edt_instances` — Contains and describes the EDT modules that the scan changes connect to.
- The Tessent Shell environment provides a mechanism for generating an example usage script you can customize to work with your third-party scan insertion tool. In the Tessent Shell tool, you invoke the following commands:

```
read_verilog design_netlist
source \
../tsdb_outdir/dft_inserted_designs/
design_name.last_DFT_insertion_design_id/
design_name.dft_info_dictionary
get_dft_info_dictionary -example_usage_script
```

After issuing these commands, the tool creates an example usage script. Use this script as a starting example to convert the dictionary into the specific commands used by your

third-party scan insertion tool. In the file, the tool inserts pound signs (#) with comments that specify which actions your third-party tool must perform. For example:

```
puts "Processing DFT signals"
set dft_signals_dict [dict get $tessent_dft_info_dict dft_signals]
puts "Setting up Static Dft Signals"
foreach dft_signal [dict keys $dft_signals_dict] {
  if {[dict exists $dft_signals_dict \
    $dft_signal forced_value_in_pre_scan_drc]} {
    set connection_node_name \
      [dict get $dft_signals_dict $dft_signal connection_node_name]
    set connection_node_type \
      [dict get $dft_signals_dict $dft_signal connection_node_type]
    set forced_value_in_pre_scan_drc \
      [dict get $dft_signals_dict $dft_signal \
        forced_value_in_pre_scan_drc]
    puts "--- Static Dft Signal $dft_signal ---"
    if {$connection_node_type eq "pin"} {
      ### Command to cut and force created pseudo port
      add_primary_input [get_pins [list $connection_node_name]] \
        -internal
      add_input_constraints \
        [get_pins [list $connection_node_name]] \
        -c${forced_value_in_pre_scan_drc}
    } else {
      ### Command tp force port
      add_input_constraints \
        [get_ports [list $connection_node_name]] \
        -c${forced_value_in_pre_scan_drc}
    }
  }
}
...
}
```

- If required and after you have inserted the scan using your third-party tool, you can proceed to [“Make Pre-ATPG Connections with Third-Party Scan for Top Chip”](#) on page 272.
- When you have completed scan insertion, the next step is to [“Perform Top-Chip ATPG Pattern Generation and Wrapped-Core Pattern Retargeting”](#) on page 274.

## Make Pre-ATPG Connections with Third-Party Scan for Top Chip

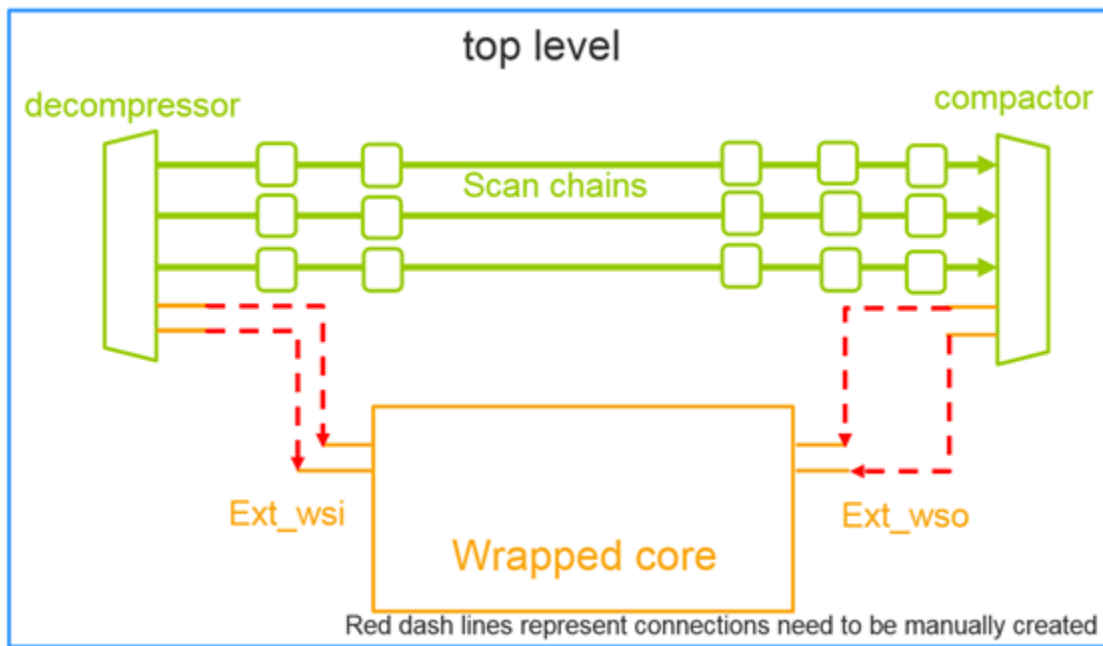
You perform this step in the flow only if the wrapped core has at least one sub-wrapped core.

If your design contains no sub-wrapped cores, skip this step in the flow and proceed directly to [“Perform Top-Chip ATPG Pattern Generation and Wrapped-Core Pattern Retargeting”](#) on page 274.

The most important logic to support hierarchical ATPG is the connection from the top level EDT scan in ports to the primary inputs that you created for each wrapper chain to cover external faults to all child cores.

Figure 6-26 illustrates this connection at an abstract level, marked in red.

**Figure 6-26. Top Level EDT Connection to Child Cores**



#### Note



The line numbers used in this procedure refer to the command flow dofile in “[Example Dofile for Pre-ATPG Connections for Top Chip](#)” on page 274.

## Prerequisites

- You must have performed the two-pass DFT insertion as described in “[Perform Two-Pass Insertion and Synthesis for Top Chip](#)” on page 269.
- You must have inserted scan using your third-party scan insertion tool per the guidelines provided in “[Third-Party Scan Insertion for Top Chip](#)” on page 270.

## Procedure

1. Load the design. (See lines 1-10.)
2. Change to insertion mode. (See line 13.)
3. Make the connections. (See lines 15-34.)
4. Save the design. (See line 37.)
5. Write the design back to the TSDB. (See lines 39-41.)

## Examples

### Example Dofile for Pre-ATPG Connections for Top Chip

```
1 # load the design
2 set_context dft -no_rtl
3 set_tsdb_output_directory ../tsdb_outdir
4 open_tsdb \ ../../cores_will_be_wrapped/<child_core_name>/tsdb_outdir
5 # Read the tessent cell library
6 read_cell_library \
7   ../../library/standard_cells/tessent/NangateOpenCellLibrary.tcelllib
8 read_design <top_level_name> -design_id <rtl2> -no_hdl
9 read_verilog ../<synthesis_path>/<top_level_name>.vg
10 read_design <child_core_name> -design_id <scan> -view graybox
11 set_current_design <top_level_name>
12
13 # make the ATPG connections:
14 set_system_mode insertion
15
16 # disconnect empty scan out from ground
17 delete_connection /chip_top_rtl2_tessent_edt_c1_inst/edt_scan_out[20]
18 delete_connection /chip_top_rtl2_tessent_edt_c1_inst/edt_scan_out[21]
19 delete_connection /chip_top_rtl2_tessent_edt_c1_inst/edt_scan_out[22]
20 delete_connection /chip_top_rtl2_tessent_edt_c1_inst/edt_scan_out[23]
21 ...
22 # connect top level EDT scan in to child core primary input of wrapper
23 # chain
24 create_connection /chip_top_rtl2_tessent_edt_c1_inst/edt_scan_in[20]
25 /<child_core_instance_1>/ext_wsi[0]
26 create_connection /chip_top_rtl2_tessent_edt_c1_inst/edt_scan_in[21]
27 /<child_core_instance_1>/ext_wsi[1]
28 ...
29 # connect top level EDT scan out to child core primary output of wrapper
30 # chain
31 create_connection /chip_top_rtl2_tessent_edt_c1_inst/edt_scan_out[20]
32 /<child_core_instance_1>/ext_wso[0]
33 create_connection /chip_top_rtl2_tessent_edt_c1_inst/edt_scan_out[21]
34 /<child_core_instance_1>/ext_wso[1]
35 # repeat for all ext_wsi/wso of all child core instances
36
37 # save design after insertion
38 write_design -output_file <top_level_name>_scan.vg -replace
39
40 # Specify the design level before writing out a softlink of the design
41 set_design_level physical_block
42 write_design -tsdb -softlink_netlist -verbose
43 exit
```

## Perform Top-Chip ATPG Pattern Generation and Wrapped-Core Pattern Retargeting

This step in the flow reads the graybox model view of the child cores, generates ATPG patterns, and retargets those patterns for each child core in the design.

## Prerequisites

- You must have completed the “[Perform Two-Pass Insertion and Synthesis for Top Chip](#)” on page 269 step for the top chip.
- You must have inserted scan with your third-party scan insertion tool per the guidelines cited in “[Third-Party Scan Insertion for Top Chip](#)” on page 270 for the top chip and written the scan-inserted netlist back into the Tessent Shell Database.
- If required, make ATPG connections using the process outlined in “[Make Pre-ATPG Connections with Third-Party Scan for Top Chip](#)” on page 272 for the top chip.

## Procedure

1. Retarget the internal ATPG patterns for each wrapped core. Refer to “[Performing Top-Level ATPG Pattern Retargeting](#)” on page 240.

See “[Top Level ATPG Pattern Generation Example](#)” on page 276 for details on top-level ATPG. Verification of these patterns is a must to ensure that the circuit functions.

See “[Pattern Retargeting of Each Child Core Example](#)” on page 277 for details on the retargeting of patterns for each child core.

Depending on tester channel availability on how many cores can be run in parallel, the internal mode ATPG patterns from each of the lower-level cores can be retargeted to the chip-level top.

2. Save the design and write the patterns to the TSDB using the [write\\_tsdb\\_data](#) command.

```
write_tsdb_data -replace
```

## Examples

### Top Level ATPG Pattern Generation Example

```
# Import design:
set_context_pattern -scan -design_id <scan>
set_tsdb_output_directory ../tsdb_outdir

# this open_tsdb should import all tsdb of child cores
open_tsdb ../../wrapped_cores/<child_core_name>/tsdb_outdir
# Read the tessent cell library
read_cell_library \
  ../../library/standard_cells/tessent/NangateOpenCellLibrary.tcelllib
# make sure to import all child cores in graybox view
read_design <child_core_name> -design_id <scan> -view graybox
read_design <top_level_name> -design_id <scan>
set_current_design <design_name>
set_design_level top

# Import core instances we want to active:
add_core_instances -module chip_top_gate2_tessent_occ_INCLK
add_core_instances -module chip_top_gate2_tessent_occ_pll_clock_0
add_core_instances -module chip_top_gate2_tessent_edt_c1

# Configure top level DFT signal values to edt_mode and all wrapped child
# core instances to external mode:
set_static_dft_signal_values edt_mode 1
set_static_dft_signal_values ltest_en 1
set_static_dft_signal_values tck_occ_en 1
set_static_dft_signal_values int_ltest_en 1

# configure child core by set_test_setup_icall
set_static_dft_signal_values ext_mode 1 -instance <child_core_name1>
set_static_dft_signal_values ext_ltest_en 1 -instance <child_core_name1>

# repeat for all child core instances

# generate and save pattern:
set_fault_type stuck
set_system_mode analysis
create_patterns
write_tsdb_data -replace

# write parallel testbench and serial testbench for simulation
write_patterns <top_level_name>_parallel.v -verilog -parallel
write_patterns <top_level_name>_serial.v -verilog -serial
# Repeat to generate transition patterns. Verification of these patterns
# are a must to ensure the circuit functions.
```



## Pattern Retargeting of Each Child Core Example

```
# Import design:
set_context pattern -scan_retargeting
set_tsdb_output_directory ../tsdb_outdir

# this open_tsdb should import all tsdb of child cores
open_tsdb ../../wraped_cores/<child_core_name>/tsdb_outdir
# Read the tessent cell library
read_cell_library \
    ../../library/standard_cells/tessent/NangateOpenCellLibrary.tcelllib
# make sure to import all child cores except the one we are doing
# retargeting in graybox view
read_design <child_core_name> -design_id <scan> -view graybox
read_design <current_core_name> -design_id <scan> -view graybox
read_design <top_level_name> -design_id <scan>
set_current_design <top_level_name>
set_design_level top

# Read in the internal mode core description file of current child core
# by add_core_instance:
# -mode should match the mode while generating internal mode patterns
add_core_instances -instance <current_core_instance_name> \
    -core <current_core_name> -mode edt_int_stuck

# Configure top level DFT signal values to retargeting_mode for this exact
# child core:
set_current_mode retarget1_<current_child_core_name>_stuck
# configure top level to retargeting mode for current child core
set_static_dft_signal_values retargeting1_mode 1

# generate and save pattern:
# for transition fault ATPG, replace stuck with transition
set_fault_type stuck
import_clocks
set_system_mode analysis
create_patterns
write_tsdb_data -replace

# write parallel testbench and serial testbench for simulation
write_patterns <current_child_core_name>_parallel.v -verilog \
    -parallel
write_pattenrs <current_child_core_name>_serial.v -verilog \
    -serial
```

# Tessent Shell Flow for Tiled Designs

A tile is a physical block that abuts to other tiles without any logic between them. The port placement is fixed early on so that the tile lines up with its neighbors. The DFT flow is adapted to the tiling flow because communication between non-neighboring tiles happens through the tile between them.

The Tessent Shell flow enables independent testing of tiles, retargeting internal modes from tiles, and creating consolidated patterns to detect faults on tile-to-tile connections.

Note

Because test planning uses a top-down approach, knowledge of the layout placement of tiles, including the DFT ports, is required. Use “[set\\_dft\\_specification\\_requirements](#)-design\_type tile” for all DFT insertions. This prevents any modification to tile boundaries, as the tool does not edit the layout.

The functional clock tree is localized within each tile and controlled by dedicated On-Chip Clock Controllers (OCCs).

A tile-based design affects different aspects of DFT insertion, and the Tessent Shell flow provides the following capabilities:

- Handling of boundary scan insertion using embedded boundary scan
- Infrastructure to support IJTAG/Memory BIST and BISR chains
- Clocking architecture for internal and external mode testing of tiles
- Logic testing of tiles with handling of identical cores

|                                                                             |            |
|-----------------------------------------------------------------------------|------------|
| <b>Tiling DFT Terminology .....</b>                                         | <b>279</b> |
| <b>How the DFT Insertion Flow Applies to Tiled Designs .....</b>            | <b>279</b> |
| <b>Clocking During Logic Test .....</b>                                     | <b>284</b> |
| <b>Pre-DFT Design Modification and Integration.....</b>                     | <b>288</b> |
| <b>Creation of IJTAG Graybox.....</b>                                       | <b>292</b> |
| <b>Example Flow for a Tiled Design .....</b>                                | <b>293</b> |
| <b>Tiling Flow Used in Example .....</b>                                    | <b>294</b> |
| <b>DFT Flow for Tiles Without TAP and BISR Controllers .....</b>            | <b>296</b> |
| <b>Embedded Boundary Scan Insertion in Tiles With a TAP Controller.....</b> | <b>314</b> |
| <b>IJTAG Network Insertion in Tiles With a TAP Controller.....</b>          | <b>320</b> |
| <b>Memory BISR Insertion for Tiles With a BISR Controller.....</b>          | <b>322</b> |
| <b>SSN Insertion for Tiles With Primary SSN Ports.....</b>                  | <b>324</b> |
| <b>Scan Insertion and Scan Modes for Tiles with Promoted OCCs .....</b>     | <b>326</b> |

## Tiling DFT Terminology

The Tessent Shell flow uses a set of terms when dealing with a tiled design. You must understand these terms to perform the RTL and scan DFT insertion process on tiled designs.

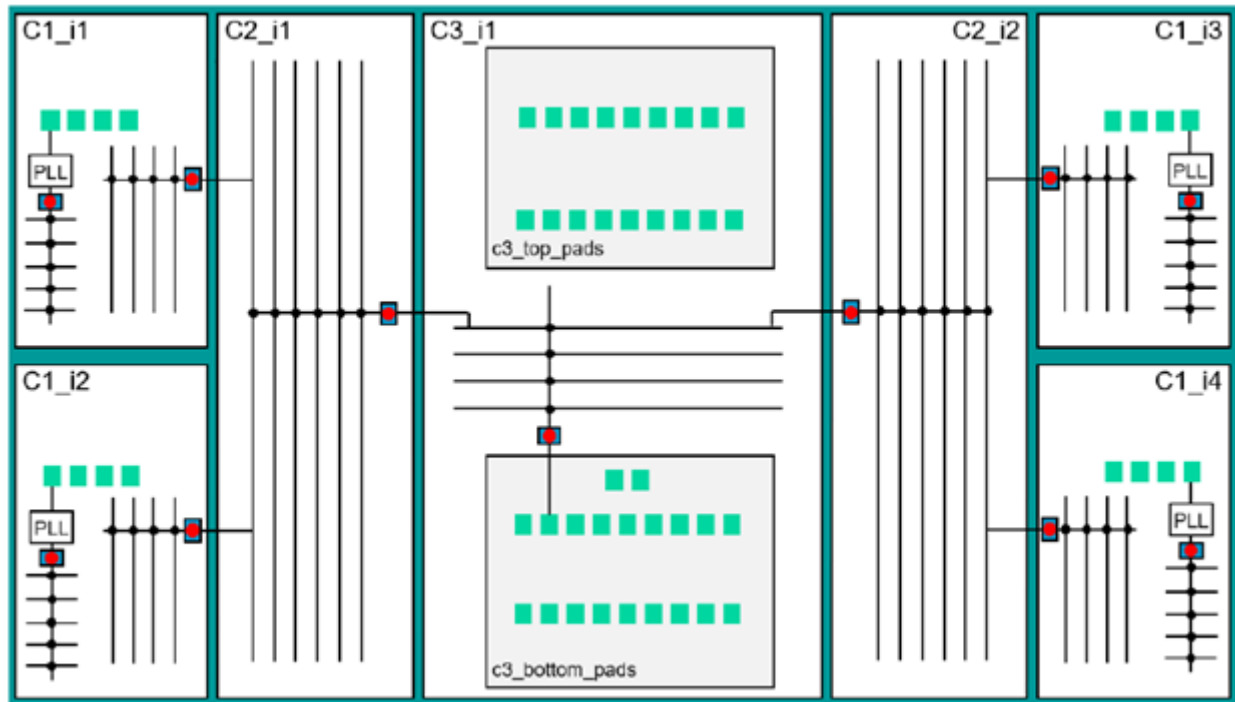
- **Tile** — A tile is a physical block designed to abut to other tiles at the top level directly.
- **Tile-based design** — A tile-based design has tiles abutted at the chip level. Therefore, there are just wire connections and no logic between the tiles, at the chip level. The chip acts as only a container for the tiles. Typically, a tiled design uses mirroring, symmetry, and repeated blocks.

## How the DFT Insertion Flow Applies to Tiled Designs

To perform DFT insertion on tiled designs, basic knowledge of the RTL and scan insertion flows for flat designs is required. The process for tiled designs builds on the process you use for a flat design.

During design, create a top-down floor plan of the chip. You must be aware of the placement of ports to build the tiled design bottom-up with the precise pin placements so that they are abutted correctly.

Figure 6-27. Schematic of Tiling Example



The example schematic, shown in [Figure 6-27](#), used to demonstrate this flow contains three unique tiles — C1, C2, and C3. C1 and C2 are mirrored on both sides of C3. Most of the I/O pads are located in the central C3 tile. For each tile, you perform the insertion process in a bottom-up manner, but there is no insertion of a standard boundary scan at the chip level. Because the tiling topology affects the insertion process of instruments, it differs from the hierarchical approach. The tiles must include DFT ports before the DFT insertion process. Refer to “[Chip Level Port Requirements and Connections](#)” on page 288 for more information.

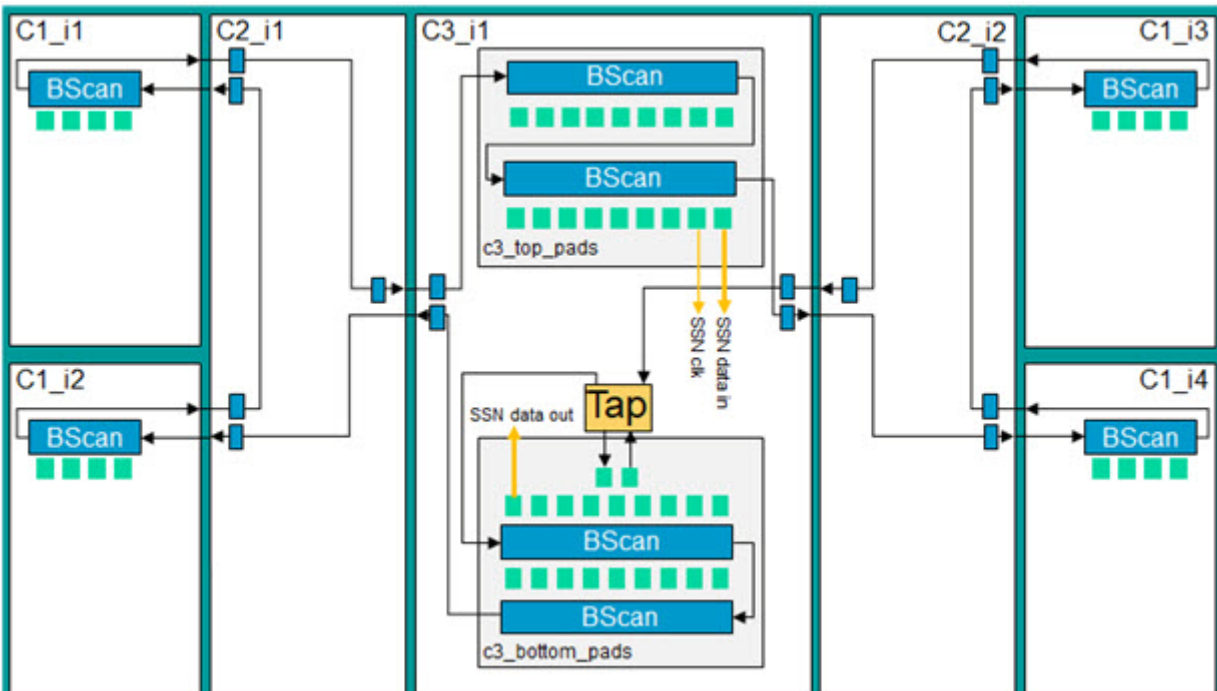
## Boundary Scan Insertion Using Embedded Boundary Scan

The insertion of a standard boundary scan at the chip design level is not present in this flow. Instead, you create the boundary scan chain from the embedded boundary scan segments in the tiles. [Figure 6-28](#) on page 281 presents a schematic of boundary scan topology.

When pads exist in a tile, like the C1 tiles in [Figure 6-28](#) on page 281, the tiles need segments of embedded boundary scan to be inserted. However, tiles without pads may also require internal boundary scan cells. These cells act as pipeline stages and provide access to the neighboring tiles equipped with scan segments, like the C2 modules in [Figure 6-28](#) on page 281.

Pads are wrapped with dedicated modules, in the C3 tile where TAP is located. Along with the shared and dedicated functional scan wrapper cells in each tile, the boundary scan cells are also used during logic test to provide scan isolation for the tile. The wrapper cell analysis does not include BIDI (inout) ports. Therefore, the boundary scan logic acts as dedicated wrapper cells for these ports and provides scan isolation. For more information on how tiles with TAP and without TAP are handled, refer to “[Embedded Boundary Scan Insertion in Tiles With a TAP Controller](#)” on page 314 and “[Embedded Boundary Scan Insertion in Tiles Without a TAP Controller](#)” on page 296, respectively.

**Figure 6-28. Schematic of Boundary Scan Logic**

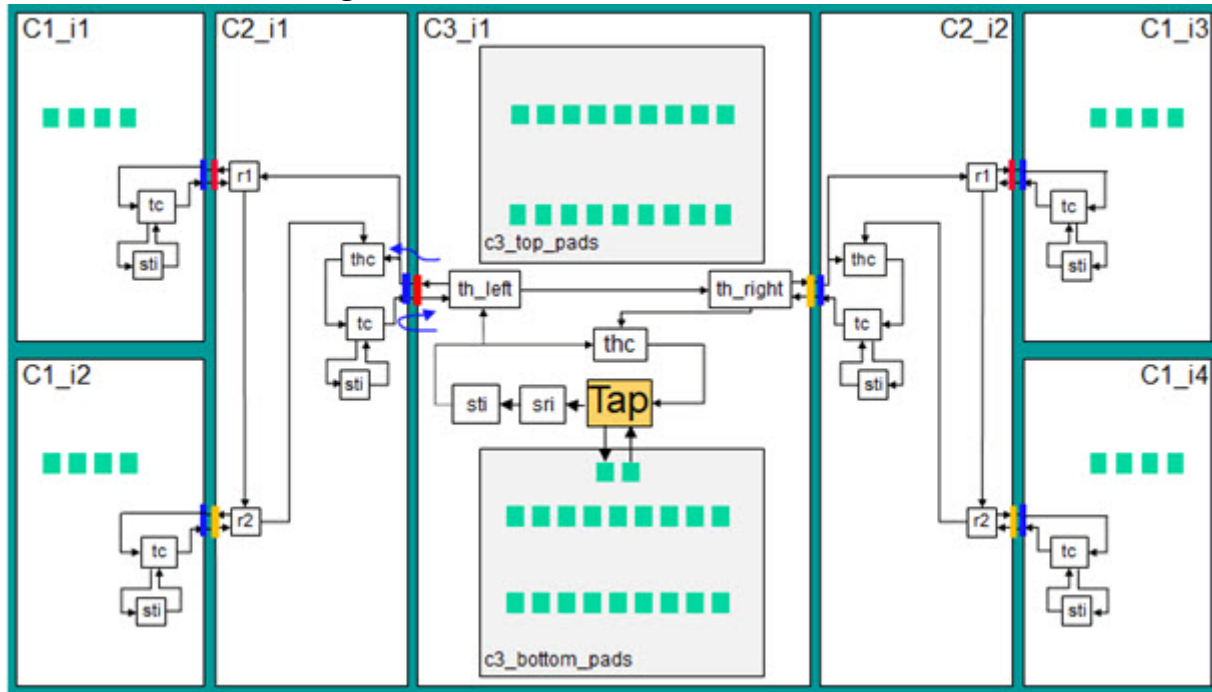


## Infrastructure to Support JTAG and Memory BIST

The JTAG network is used to set up the memory BIST and memory repair, and to configure the design in the required scan mode. [Figure 6-29](#) on page 282 shows the schematic of the JTAG network inserted in the described example.

Access to the inner tile JTAG network in tiles without a TAP controller is through their Tile Client (TC) Segment Insertion Bit (SIB), the Scan Tested Instrument (STI), and Scan Resource Instrument (SRI) SIBs located below the TC SIB. The TC SIB is not needed in the C3 tile because the TAP is present. If Secondary Host Scan Interfaces to connect neighboring tiles are present, a Tile Host Collector (THC) SIB is also added. The THC SIB provides access to the dedicated Tile Host (TH) SIBs for Secondary Host Scan Interfaces — th\_left and th\_right SIBs in the C3 module, or r1 and r2 SIBs in C2 tiles.

**Figure 6-29. Schematic of IJTAG Network**



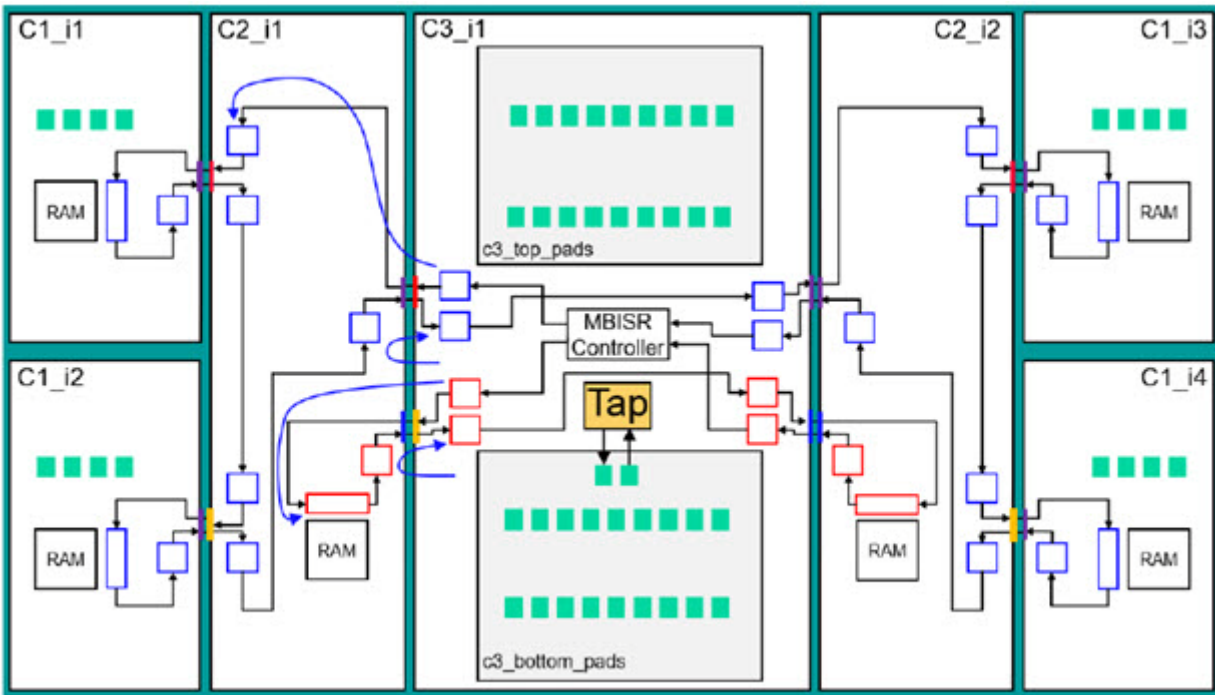
## Infrastructure to Support Memory BISR Chains

The insertion of memory BISR logic depends on the number of power domains in the tiling design. Each power domain must have a dedicated BISR chain connected to the MBISR controller. There are two power domains in the example; BISR registers from the first power domain (pdgA) in red, and registers from the second domain (pdgB) in blue. Figure 6-30 on page 283 shows the schematic of the BISR network.

Similar to the case of embedded boundary scan insertion, the BISR chain may be required in tiles without repairable memories because it acts as a pipeline stage to provide access to neighboring tiles. The C2 tiles in Figure 6-30 on page 283 have BISR chain logic from the blue domain to provide access to neighboring C1 tiles with repairable memories in the same power domain.

The Tessent tool cannot deduce this requirement when performing bottom-up insertion. Therefore, you must specify it using the “[set\\_dft\\_specification\\_requirements](#) -memory\_test on -memory\_bisr\_host\_list {<hosts\_list>}” command. You must be aware of the physical layout of the BISR network before the DFT process begins.

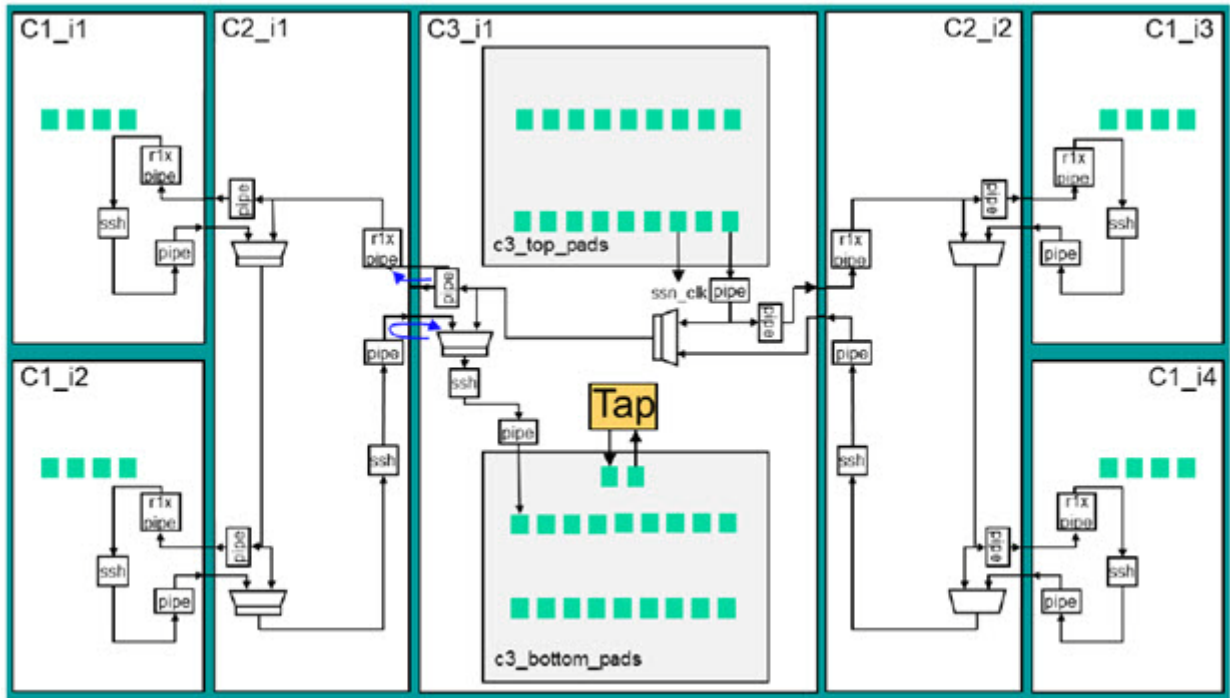
**Figure 6-30. Schematic of Memory BISR Network**



## Logic Testing of Tiles Using SSN

The SSN network must go through every ScanHost node (SSH) in the design to enable scan tests of all tiles in internal and external modes. You can also design the network to enable single testing of the tile for diagnosis purposes. [Figure 6-31](#) on page 284 shows the SSN network inserted in the example. The SSN bus has its primary inputs and outputs in the C3 tile. It starts and terminates on auxiliary data logic implemented during Embedded Boundary Scan insertion. The SSN network in each tile without SSN primary ports starts from the [Receiver1xPipeline](#) (r1x pipe) and ends in a standard [Pipeline](#). The pipeline stages are necessary to achieve correct timing in tile-to-tile connections. The SSN network is implemented in a plug-and-play manner. Therefore, there is no need to determine EDT requirements for each tile beforehand. For more information on how tiles with and without primary SSN ports are handled, refer to “[SSN Insertion for Tiles With Primary SSN Ports](#)” on page 324 and “[SSN and EDT Insertion in Tiles Without TAP and BISR Controllers](#)” on page 305, respectively.

Figure 6-31. Schematic of SSN Network



## Clocking During Logic Test

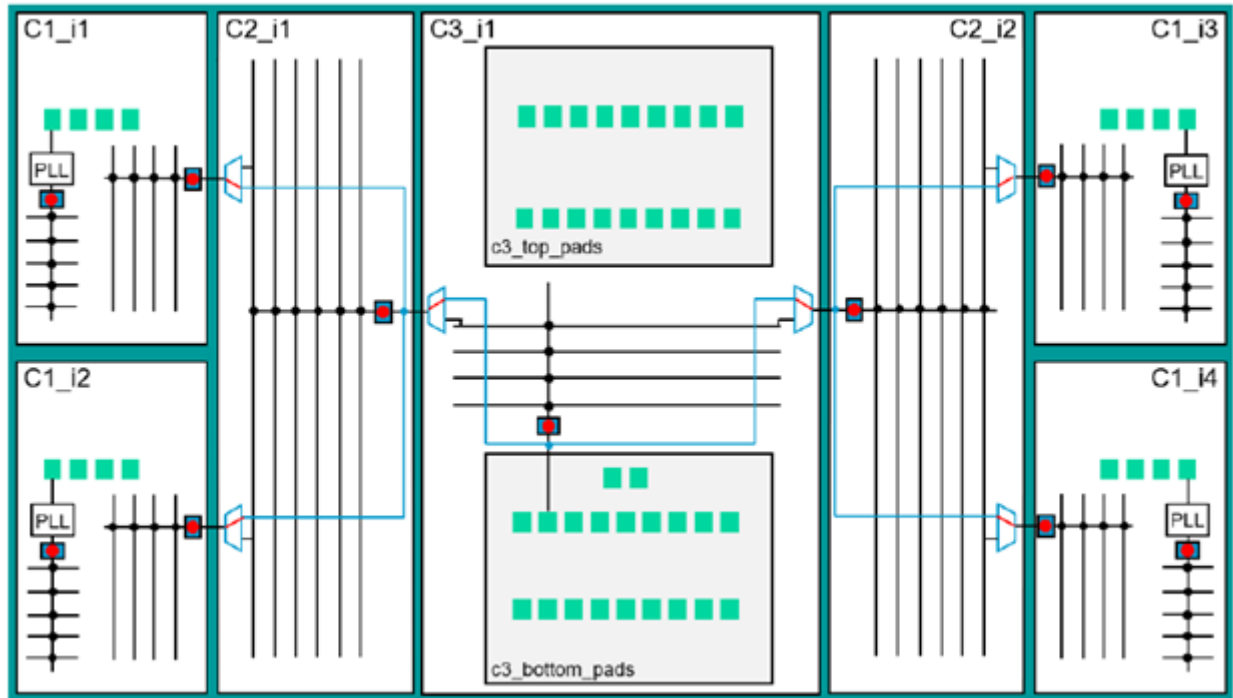
The tiling approach supports at-speed tile-to-tile test in external mode. Internal and external scan modes require different clock controls. You must add a clock multiplexer at every clockout port to support running several tiles in parallel while they are in their respective internal modes. This is to avoid cascaded active OCCs.

### Clock Control in Internal mode

During internal test, each tile is tested independently and must have its own clock control, using OCCs. Additional bypass clock multiplexers are required to provide free-running clocks directly to the dedicated OCCs and to avoid potentially cascading active OCCs. In the following figure, active OCCs are marked with a red dot, and the blue clock path indicates the free-running clock to the OCCs. The insertion of additional clock multiplexers is described in “[SSN and EDT Insertion in Tiles Without TAP and BISR Controllers](#)” on page 305.



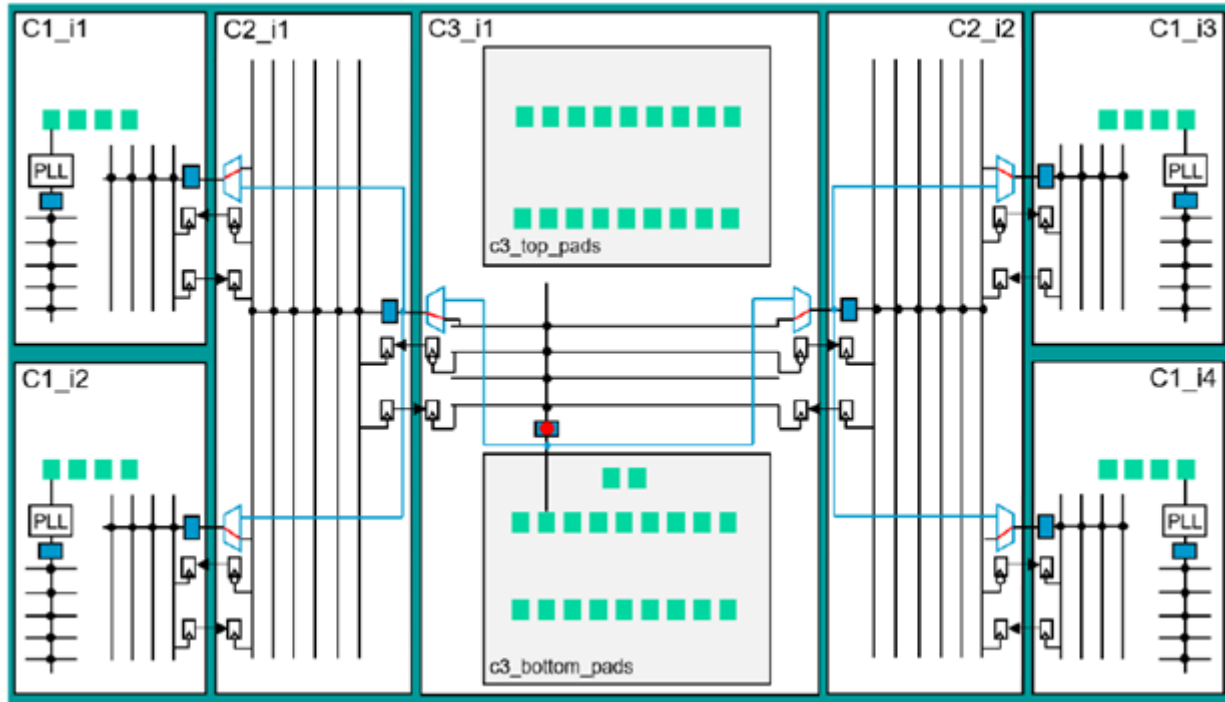
**Figure 6-32. Clocking in Internal Test Mode**



### Clock Control in External Mode

You must synchronize the scan chains in the tiles to test the tile-to-tile timing paths. Only the OCCs at the bases of the functional clock trees are active in the external mode. The native functional clock paths deliver the clock to all tiles (the additional clock multiplexers select the native clock path in external mode). This makes it possible to restore functional synchronicity between the tiles. The following figure shows this configuration. The OCC in the C3 tile is marked active, and the rest of the OCCs are transparent and operate in the shift-only mode.

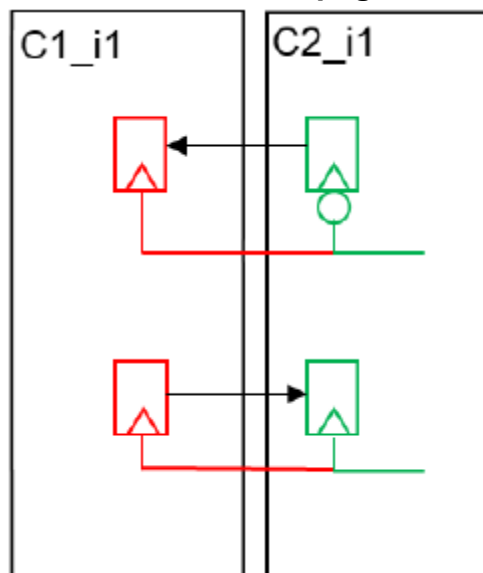
**Figure 6-33. Clocking in External Mode**



### Timing of Tile-To-Tile Connections

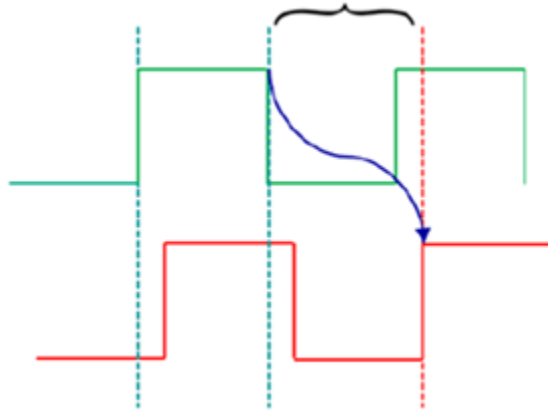
The IEEE 1687 IJTAG protocol is a robust two-edge protocol in which data is strobed from one node at negedge and then captured at the next network node at posedge. There is always one half of a TCK cycle to propagate data from one node to another. In some cases, this time window may be too short for correct propagation. [Figure 6-34](#) on page 286 shows the schematic of the forward and the return paths between tiles:

**Figure 6-34. Forward and Backward Propagation of Data Between Tiles**



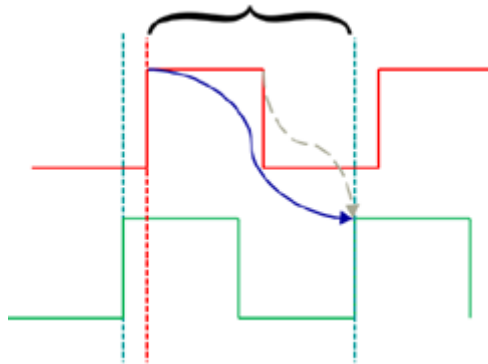
The clock in the C1\_i1 tile is always a delayed version of the clock in C2\_i1 because it is sourced from a leaf of the C2 clock tree. With this assumption, you extend the propagation window in the forward path by the value of the TCK clock delay. [Figure 6-35](#) on page 287 shows a simplified waveform of the forward propagation. The TCK clock delay reduces the time window for the returning path. The retiming element of the IJTAG node connected to the scan-out port is removed, making it a posedge-to-posedge propagation and extending the time window in this case.

**Figure 6-35. Simplified Waveform of Forward Propagation, Negedge to Posedge**



[Figure 6-36](#) on page 287 shows a simplified waveform for this propagation. The gray arrow depicts the propagation if the retiming stage is present and the path is not relaxed.

**Figure 6-36. Simplified Waveform of Return Posedge to Posedge Propagation**



The relaxation of return path data propagation is applied to all the Tessent instruments that need communication between the tiles (IJTAG network, memory BISR chains, and boundary scan chains).

## Pre-DFT Design Modification and Integration

The DFT ports and their connections between the tiles and to the package boundary must preexist.

If they do not, use the [TopModuleConnections](#) wrapper or insertion commands to create them.

To prevent any modification of the current module boundary during insertion, use the “[set\\_dft\\_specification\\_requirements](#) -design\_type tile” command. The “[allow\\_port\\_creation\\_on\\_current\\_design](#)” property is set to off by default in every processed DftSpecification if the design type is set to “tile”.

This means that the tool assumes all DFT ports required by Tessent instruments to already exist in each tile before inserting their DFT in a bottom-up fashion. If this is not the case, set the “[allow\\_port\\_creation\\_on\\_current\\_design](#)” property to on in the DftSpecification before specification processing.

|                                                           |            |
|-----------------------------------------------------------|------------|
| <b>Chip Level Port Requirements and Connections .....</b> | <b>288</b> |
| <b>Tile Level Port Connections and Requirements .....</b> | <b>288</b> |

## Chip Level Port Requirements and Connections

There is no logic at the chip level for a tiled design but the design must have certain preexisting ports for you to run the DFT flow.

The required chip-level DFT ports are:

- BISR ports: bisr\_prog\_voltage, bisr\_clock, bisr\_trigger
  - bisr\_clock and bisr\_trigger can be generated internally and are, therefore, optional ports.
- JTAG TAP client ports: tck, trst, tms, tdi, tdo

The SSN bus ports may not always be required because functional ports can be reused using boundary scan auxiliary logic.

You must add the dedicated SSN ports, that is, SSN input and output bus ports, and the SSN bus clock port.

If the required ports and connections are not present, use the [TopModuleConnections](#) specification to insert DFT ports in tiles and to connect them at the chip level.

## Tile Level Port Connections and Requirements

The manual integration of many tiles to a single chip in Verilog may be hard to achieve.

### Tip

 We recommend that you use the Tessent insertion mode and the [TopModuleConnections](#) specification to integrate DFT logic in tiles if the automation you used to create and connect the functional ports did not include the DFT ports.

---

The following example shows a Tessent script using the TopModuleConnections specification to integrate a tiling design and the syntax of the TopModuleConnections wrapper.

```
set_context dft -rtl -design_id rtl0
read_verilog ../../design/chip/chip.v
set_current_design chip

read_config_data \
top.top_level_ports_and_connections

process_top_module_connections
```

Start the TopModuleConnections specification from a definition of modules that are used in the integration. Use the ChildBlock wrapper to describe the module and the ports to be used, or to create them if not available. The Instance wrapper below ChildBlock is used to define the input sources for newly created input ports on each instance of the ChildBlock module.

```
TopModuleConnections(<design_name>) {
  ChildBlock(<design_name>) {
    file_name : <full_path_name> ;
    input_ports : <port_name>, ... ;
    output_ports : <port_name>, ... ;
    Instance(<instance_name>) {
      input_port_sources :
        <port_pin_name>, ... ;
    }
  }
}
```

### Note

 The order of sources in the Instance wrapper must correspond to the order of input ports in the ChildBlock wrapper.

---

## IJTAG DFT Ports

The ports required for IJTAG integration are listed here. Declare the additional IJTAG secondary interfaces in the ChildBlock wrapper.

### Note

 You also require preexisting ports for embedded boundary scan and memory BISR. Contact your Siemens representative to refer to the example test case.

---

IJTAG client input ports:

- ijtag\_tck
- ijtag\_reset
- ijtag\_ce
- ijtag\_se
- ijtag\_ue
- ijtag\_sel
- ijtag\_si

IJTAG client output ports:

- ijtag\_so

IJTAG secondary interface input ports:

- <interface\_name>\_ijtag\_from\_so

IJTAG secondary interface output ports:

- <interface\_name>\_ijtag\_to\_tck
- <interface\_name>\_ijtag\_to\_reset
- <interface\_name>\_ijtag\_to\_ce
- <interface\_name>\_ijtag\_to\_se
- <interface\_name>\_ijtag\_to\_ue
- <interface\_name>\_ijtag\_to\_sel
- <interface\_name>\_ijtag\_to\_si

The following code example shows the part of the ChildBlock wrapper that describes IJTAG ports and their connections on instance C2\_i1:

```
ChildBlock(c2) {
    file_name : ../../design/c2/c2.v;
    input_ports : ...
    #Blue in figure
        ijtag_tck,
        ijtag_reset,
        ijtag_ce,
        ijtag_se,
        ijtag_ue,
        ijtag_sel,
        ijtag_si,

    #Red in figure
        r1_ijtag_from_so,
    #Orange in figure
        r2_ijtag_from_so,
        ...

    output_ports : ...
    #Blue in figure
        ijtag_so,

    #Red in figure
        r1_ijtag_to_tck,
        r1_ijtag_to_reset,
        r1_ijtag_to_ce,
        r1_ijtag_to_se,
        r1_ijtag_to_ue,
        r1_ijtag_to_sel,
        r1_ijtag_to_si,

    #Orange in figure
        r2_ijtag_to_tck,
        r2_ijtag_to_reset,
        r2_ijtag_to_ce,
        r2_ijtag_to_se,
        r2_ijtag_to_ue,
        r2_ijtag_to_sel,
        r2_ijtag_to_si,
        ...

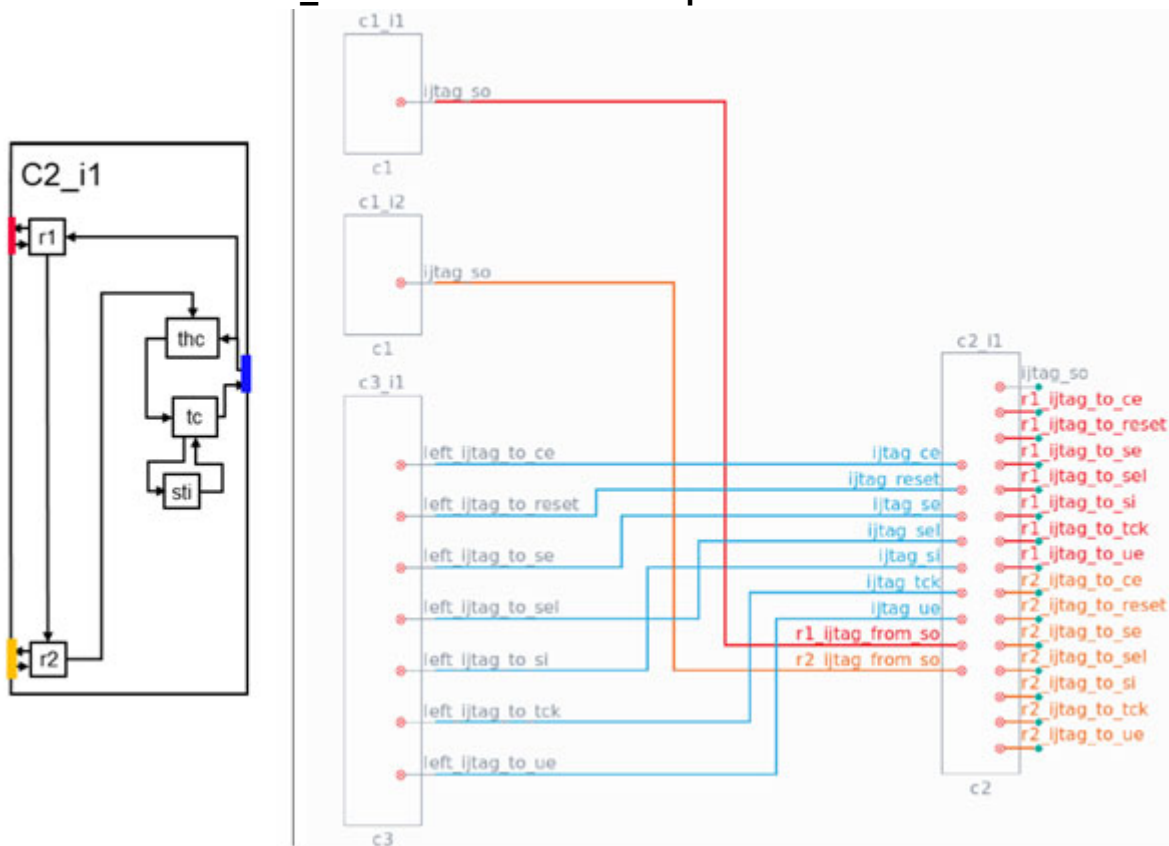
    Instance(c2_i1) {
        input_port_sources : ...
    #Blue in figure
        c3_i1/left_ijtag_to_tck,
        c3_i1/left_ijtag_to_reset,
        c3_i1/left_ijtag_to_ce,
        c3_i1/left_ijtag_to_se,
        c3_i1/left_ijtag_to_ue,
        c3_i1/left_ijtag_to_sel,
        c3_i1/left_ijtag_to_si,

    #Red in figure
        c1_i1/ijtag_so,
```

```
#Orange in figure
c1_i2/ijtag_so,
...
}
Instance(c2_i2) {
...
}
}
```

The following figure shows the schematic of the IJTAG network in the C2 tile and input connections of C2\_i1:

**Figure 6-37. Schematic of IJTAG Network in C2 Tile and IJTAG Ports Created on C2\_i1 Instance With Their Input Connections**



## Creation of IJTAG Graybox

After every DFT insertion phase (after the `process_dft_specification` command), extract the ICL description and generate the IJTAG graybox.

The IJTAG graybox is a design view that preserves only IJTAG instruments. Using an IJTAG graybox view instead of the full view of a tile reduces runtime and memory consumption in simulation tools.



In integrated package simulations, sometimes you cannot directly access a tile under test and may need to access it through other tiles. In such cases, you must read in the full view of the targeted tile and the IJTAG graybox views of all the other tiles.

**Note**

We recommend you use IJTAG graybox views whenever possible.

Running the `extract_icl` command generates the IJTAG graybox by default. The IJTAG graybox generation can also be turned off, as demonstrated in this code example:

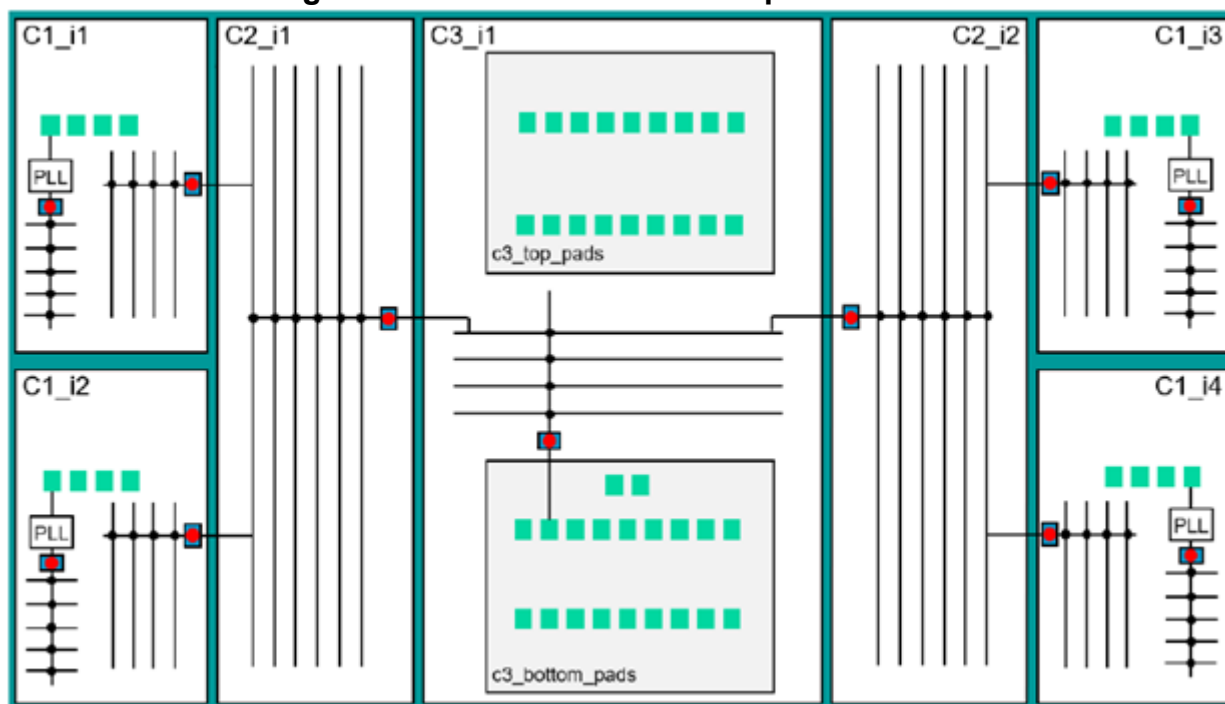
```
extract_icl -create_ijtag_graybox off
```

## Example Flow for a Tiled Design

The following figure shows a sample schematic of a tiled design. If you would like to run this design, contact your Siemens representative for the tiling test case matching the example used here.

The tiles are connected only to the neighboring tiles or to chip-level ports when pads for primary ports are present in the tile.

**Figure 6-38. Schematic of a Sample Test Case**



This design consists of seven tiles:

- Four instances of the C1 module

- Two instances of the C2 module
- One instance of the C3 module

The C1\_i3 and C1\_i4 instances are mirrored versions of C1\_i1 and C1\_i2 respectively, and, similarly, C2\_i2 is a mirrored version of C2\_i1. The OCCs are the blue rectangles with red dots, and the pads are the green squares.

This design has the TAP, the Memory BISR controller, and the SSN bus access pads in the C3\_i1 tile. The presence of these instruments affects the DFT flow for the C3 tile. This causes the DFT flow for the C3 tile to differ from the other tiles. All DFT ports are pre-created on each tile and pre-connected in the chip module (“[Pre-DFT Design Modification and Integration](#)” on page 288). [Example 1](#) in [process\\_top\\_module\\_connections](#) shows how you can automate this within the tool.

You do not need to insert TAP in the same tile as the Memory BISR controller or SSN bus access pads as shown in the example ([Figure 6-38](#) on page 293). If the controllers and pads are in different tiles, modify the DFT flow for those tiles accordingly.

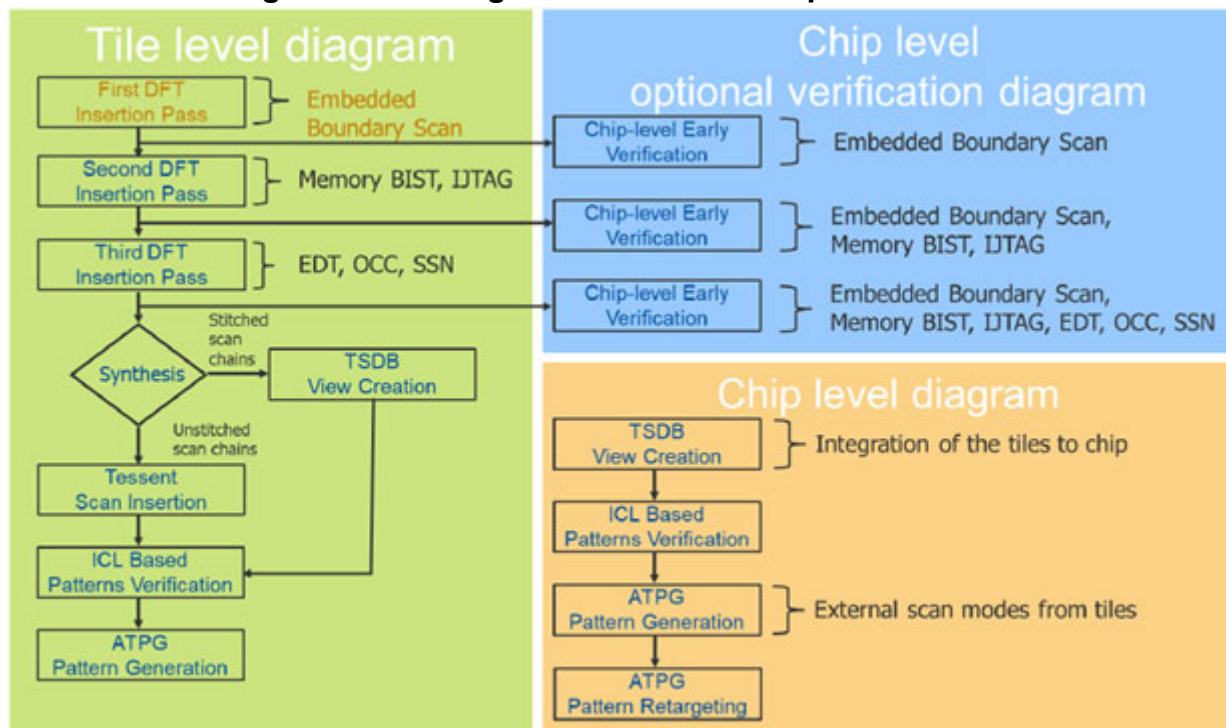
## Tiling Flow Used in Example

The flow used for the example schematic is depicted in the following flowchart.

The color coding used in the flow diagram is:

- **Green** — Steps related to tiles
- **Blue** — Steps for optional early verification
- **Orange** — Steps for chip-level integration

Figure 6-39. Tiling Flow Used in Example Test Case



## DFT Flow for Tiles Without TAP and BISR Controllers

---

For tiles without TAP or BISR controllers, the tiling DFT flow is similar to the Tessent Shell hierarchical DFT flow.

In the example test case (“[Example Flow for a Tiled Design](#)” on page 293), the tiles do not have nested physical blocks, but the presence of nested physical blocks does not affect the tiling flow. The design type of a tile affects the default specifications generated for embedded boundary scan, IJTAG, and MemoryBISR.

---

### Note

 For embedded boundary scan, IJTAG, MemoryBIST and MemoryBISR insertion, set the `design_type` property to “tile” using the “[set\\_dft\\_specification\\_requirements](#) -design\_type tile” command.

---

|                                                                                          |            |
|------------------------------------------------------------------------------------------|------------|
| <b>Embedded Boundary Scan Insertion in Tiles Without a TAP Controller . . . . .</b>      | <b>296</b> |
| <b>IJTAG Network Insertion in Tiles Without a TAP Controller . . . . .</b>               | <b>300</b> |
| <b>Memory Test Insertion in Tiles Without TAP and BISR Controllers . . . . .</b>         | <b>303</b> |
| <b>SSN and EDT Insertion in Tiles Without TAP and BISR Controllers . . . . .</b>         | <b>305</b> |
| <b>Scan Insertion and Scan Modes in Tiles Without TAP and BISR Controllers . . . . .</b> | <b>309</b> |
| <b>Patterns Generation in Tiles Without a TAP Controller . . . . .</b>                   | <b>310</b> |
| <b>Verification and Simulations . . . . .</b>                                            | <b>313</b> |

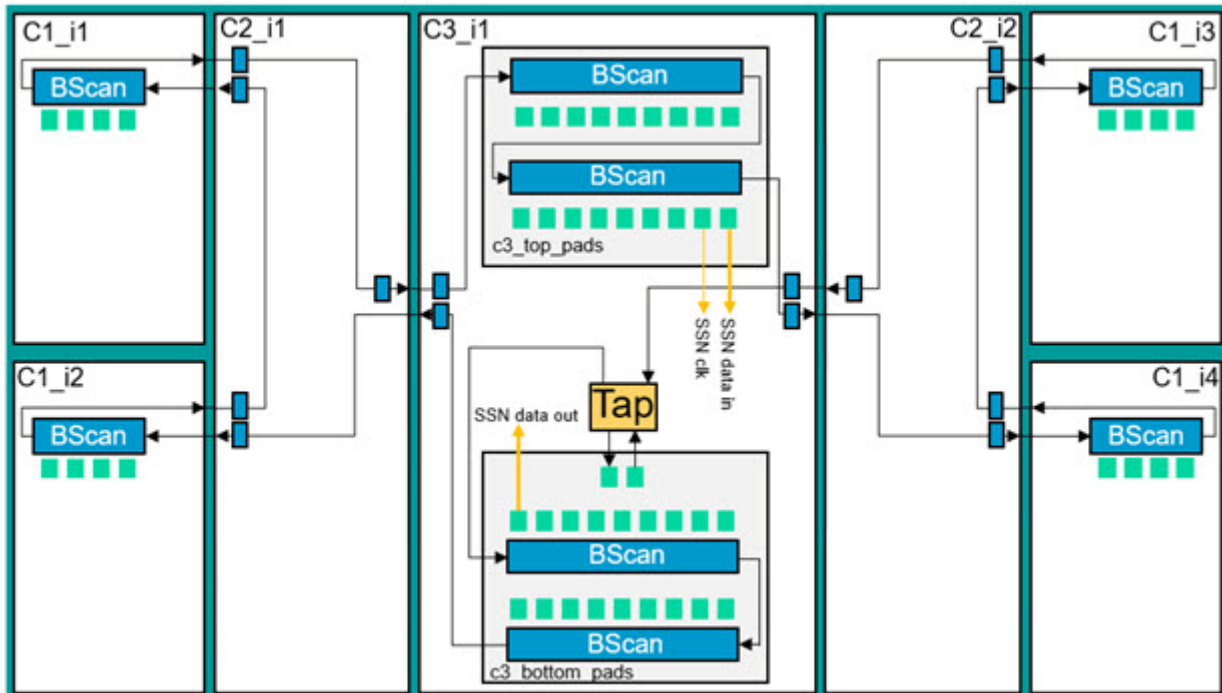
## Embedded Boundary Scan Insertion in Tiles Without a TAP Controller

Use the Embedded Boundary Scan flow to add boundary scan in tiled designs because the pads are embedded inside the child tile blocks. The insertion of internal boundary scan cells may also be required in tiles without pads.

These internal boundary scan cells, in the C2 tile, act as pipeline registers, and enable the C3 tile (with TAP interface) to access all the tiles with embedded boundary scan segments.

In the following figure, the C2 modules require boundary scan logic to connect segments in instances of the C1 module. You can insert the auxiliary pad logic for SSN during Embedded Boundary Scan insertion.

**Figure 6-40. Schematic of Boundary Scan Logic in Example Test Case**



## Embedded Boundary Scan Insertion in Tiles With Pads

Embedded boundary scan segments are reused during logic test to provide scan isolation from the chip-level I/Os for tiles during chip-level ATPG modes.

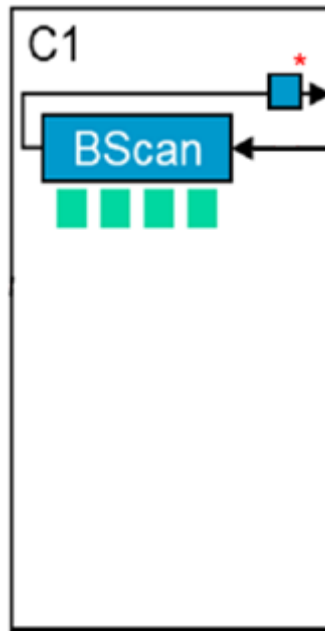
To enable test logic support for embedded boundary scan segments, you must set the [EmbeddedBoundaryScan/max\\_segment\\_length\\_for\\_logictest](#) property to “unlimited”, or to an integer value that corresponds to the required length of the embedded boundary scan segments used during scan chain stitching. You need the pipeline stage on the output of the embedded boundary scan segment to relax the timing loop on the return path. This is described in “[Timing of Tile-To-Tile Connections](#)” on page 286.

The following code presents the DftSpecification for Embedded Boundary Scan for the C1 tile with pads:

```
DftSpecification(c1,rtl1) {
  allow_port_creation_on_current_design : off;
  EmbeddedBoundaryScan {
    pad_io_ports : clk_ref, rx_data_in, ...;
    max_segment_length_for_logictest : 200;
    Interface {
      scan_out_pipeline : on;
    }
    BoundaryScanCellOptions {
      clk_ref : clock;
    }
  }
}
```

The following schematic displays the logic inside the C1 tile with pads:

**Figure 6-41. Schematic of Boundary Scan Logic Inside C1**



For more details on how the HostBscanInterface wrapper assembles the boundary scan chain in C1, refer to [Example 1](#) in [HostBScanInterface](#) in the *Tessent Shell Reference Manual*.

### Embedded Boundary Scan Insertion in Tiles Without Pads

In tiles without pads, embedded boundary scan logic is used to provide access to adjacent tiles equipped with other embedded boundary scan segments.

The following example shows the DftSpecification for a tile with one client interface and two host interfaces to connect neighboring tiles. For more details on how the HostBscanInterface wrapper assembles the boundary scan chain in C2, refer to [Example 1](#) in [HostBScanInterface](#) in the *Tessent Shell Reference Manual*.

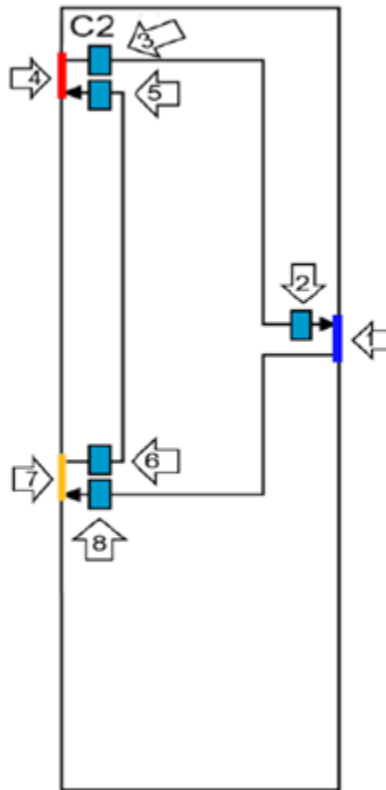
```

DftSpecification (c2,rtl1) {
  EmbeddedBoundaryScan {
    # 1 in figure
    HostBscanInterface(client){
      # 2 in figure
      EBScanPipeline(client_out) {
        so_retiming : off;
      }
      # 3 in figure is input pipeline
      EBScanPipelining(from_r1) {
      }
      # 4 in figure is a host interface
      SecondaryEBScanInterface(r1) {
      }
      # 5 in figure is input pipeline
      EBScanPipeline(to_r1) {
      }
      # 6 in figure is output pipeline
      EBScanPipeline(from_r2) {
      }
      # 7 in figure is a host interface
      SecondaryEBScanInterface(r2) {
      }
      # 8 in figure is output pipeline
      EBScanPipeline(to_r2) {
      }
    }
  }
}

```

The client interface in the resulting schematic ([Figure 6-42](#) on page 300), requires a pipeline on the scan output port (client interface marked “1”, output pipeline marked “2”). The host interfaces require input and output pipelining.

**Figure 6-42. Schematic of Embedded Boundary Scan Logic in C2 Without Pads**



## IJTAG Network Insertion in Tiles Without a TAP Controller

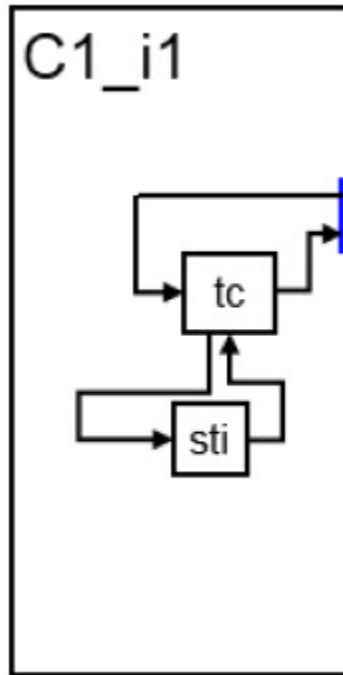
The IJTAG network used in the tiling flow is almost identical to the network used in the hierarchical flow. The tool modifies the network topology for the tiling flow when you set the design type to “tile”.

Access to the local Scan Resource Instrument (SRI) and Scan Test Instrument (STI) Segment Insertion Bits (SIBs) is guarded by an additional Tile Client (TC) SIB. This TC SIB, with internal scan-in pipelining and retiming stage removed, gets added to the default DftSpecification when you set the `-design_type` to “tile”. The output of the TC SIB is connected directly to the scan-out port. To relax the propagation of the loop timing interface, the retiming stage is removed as detailed in “[Timing of Tile-To-Tile Connections](#)” on page 286.

In this example (“[Example Flow for a Tiled Design](#)” on page 293), after Embedded Boundary Scan you insert the memory BIST that requires an IJTAG connection. [Figure 6-43](#) on page 301 presents the IJTAG network after insertion of Memory BIST. To build the IJTAG network, refer to [Example 3 in IJTAG Network](#) in the *Tessent Shell Reference Manual*.

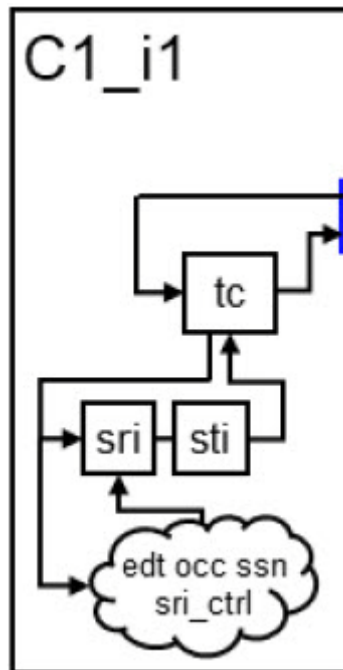


**Figure 6-43. Schematic of IJTAG Network After Memory BIST Insertion in C1**



The logic test support (EDT, OCC, and SSN) is inserted in the next insertion pass. In the tiling flow, new elements of the IJTAG network are connected below TC SIB, by default. The following figure depicts this:

**Figure 6-44. Schematic of IJTAG Network After Memory BIST and Logic Test insertion in Tile**



Sometimes, the IJTAG network requires additional scan interfaces to host neighboring tiles, such as the C2 tile. In such cases, a dedicated SIB hosts each secondary host scan interface. All SIBs that host secondary host scan interfaces are collected and hosted by a dedicated Tile Host Collector (THC) SIB. The THC SIB is placed in series with the TC SIB as shown in [Figure 6-45](#) on page 303. Use the [create\\_dft\\_specification](#) command to provide names of additional scan interfaces.

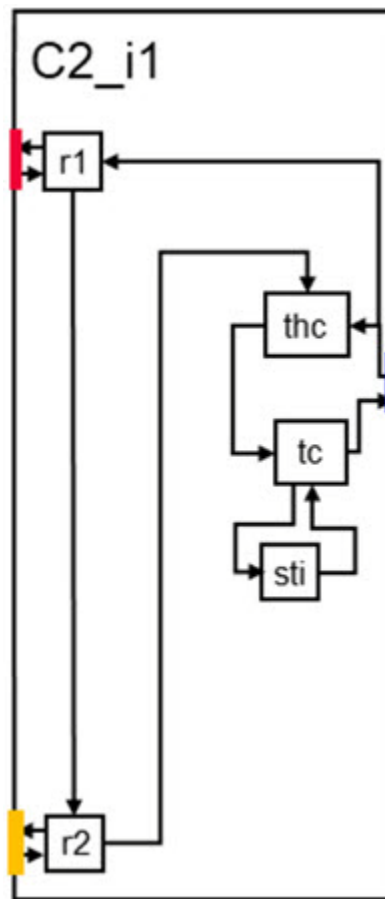
In the following example, r1 and r2 are two additional SecondaryHostScanInterfaces:

**create\_dft\_specification -tile\_ijtag\_host\_list {r1 r2}**

The following code sample presents an example DftSpecification with two additional IJTAG host scan interfaces:

```
IjtagNetwork {
#Blue in figure
  HostScanInterface(ijtag) {
    Sib(tc) {
      Attributes {
        tessent_dft_function : tile_client_sib;
      }
      to_scan_in_feedthrough : pipeline;
      so_retiming : off;
      Sib(sti) {
        ...
      }
    }
    Sib(thc){
      Attributes {
        tessent_dft_function : tile_host_collector;
      }
#Red in figure
      Sib(th_r2) {
        to_scan_in_feedthrough : pipeline;
        SecondaryHostScanInterface(r2) {
        }
      }
#Orange in figure
      Sib(th_r1) {
        to_scan_in_feedthrough : pipeline;
        SecondaryHostScanInterface(r1) {
        }
      }
    }
  }
}
```

**Figure 6-45. Schematic of C2 Tile With Additional Host Scan Interfaces**



## Memory Test Insertion in Tiles Without TAP and BISR Controllers

The MemoryBIST logic does not require any specific changes related to the tiling flow and is placed below the Scan Test Instrument (STI) Segment Insertion Bit (SIB). You can insert the memory test logic and the JTAG network in the same insertion pass.

To ensure access to the BISR chains in tiles, you must slightly modify the MemoryBISR logic. You can insert additional BISR chains for the different power domains. To specify these power domains and memories, we recommend that you use the Unified Power Format (UPF) file.

See “[Schematic of MemoryBISR Inside the C2 Tile](#)” on page 305 for an example BISR network in the C2 tile. There is one repairable RAM in the C2 tile. In this example, you require access to the neighboring tiles that are connected to the left of the C2 tile. The neighboring tiles operate in different power domains, so you need two BISR chains and two client interfaces:

- The first client interface, marked “1”, accesses the BISR segment of RAM in the tile.

- The second client interface, marked “2”, accesses BISR segments in two neighboring tiles that work in a different power domain, pdgB.
- The third and fourth interfaces, marked “3” and “4”, are host interfaces for neighboring tiles in the power domain pdgB. You can create these interfaces using the [set\\_dft\\_specification\\_requirements](#) command with the “-memory\_bisr\_host\_list” switch.

Place a pipeline element, without the retiming stage, at the end of each of the two BISR chains (the return path) to provide a full clock period in the loop timing interface (see “[Timing of Tile-To-Tile Connections](#)” on page 286).

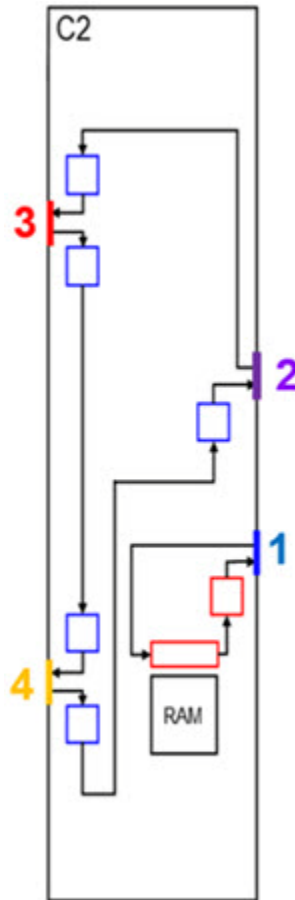
For more details on how memory BISR chains are inserted into a tile-based design, refer to [Example 3](#) in [MemoryBisr/Secondary Host Chain Interface](#) in the *Tessent Shell Reference Manual*. The following code generates and displays the DftSpecification used to insert the MemoryBisr logic:

```
set_dft_specification_requirements -memory_test on \  
  -design_type tile \  
  -memory_bisr_host_list {pdgB {pdgB_r1 pdgB_r2}}  
create_dft_specification -tile_ijtag_host_list {r1 r2}  
report_config_data DftSpecification(c2,rtl2)/MemoryBisr  
  
MemoryBisr {  
  bisr_segment_order_file : c2.bisr_segment_order;  
  BisrElement("SegmentEnd(*)") {  
    Pipeline(after) {  
      so_retiming : off;  
    }  
  }  
}
```

The following code sample shows the *bisr\_segment\_order* file. It lists the order of the elements in the BISR chain from scan-out to scan-in:

```
BisrSegmentOrderSpecification {  
  #1 in figure  
  PowerDomainGroup(pdgA) {  
    OrderedElements {  
      RAM; // RepairGroup:None BISRLength:24  
    }  
  }  
  #2 in figure  
  PowerDomainGroup(pdgB) {  
    OrderedElements {  
      #3 in figure  
      "SecondaryHostChainInterface(pdgB_r1)";  
      #4 in figure  
      "SecondaryHostChainInterface(pdgB_r2)";  
    }  
  }  
}
```

**Figure 6-46. Schematic of MemoryBISR Inside the C2 Tile**



## SSN and EDT Insertion in Tiles Without TAP and BISR Controllers

The next step in the tiling flow is insertion of logic test instruments such as EDT and SSN.

Insert two EDT controllers in each tile: one for internal mode and another for external mode. Usually, these two modes use different numbers of scan channels and lengths and numbers of scan chains. So, each mode requires a dedicated controller to achieve adequate compression.

Any DFT signal required for logic test that has not been included in the previous steps must be added at this step. Such signals include `int_ltest_en` and `ext_ltest_en`. For more information on how to insert SSN network in a tile-based design, refer to [Example 3](#) in *DftSpecification/SSN* in the *Tessent Shell Reference Manual*.

The clock multiplexers for output clocks are also inserted in this step. Refer to “[Timing of Tile-To-Tile Connections](#)” on page 286 for more information.

The following example adds clock multiplexers to bypass local OCC in internal test mode:

```
add_dft_clock_mux clkc_to_1 -dft_signal int_ltest_en \  
-test_clock_source clkc  
add_dft_clock_mux clkc_to_2 -dft_signal int_ltest_en \  
-test_clock_source clkc
```

This example shows the commands to add DFT signals in logic test:

```
add_dft_signals int_edt_mode ext_edt_mode  
add_dft_signals ltest_en int_ltest_en ext_ltest_en  
# ssn_en is required when the tile has auxiliary ports for SSN  
add_dft_signals ssn_en  
# Use the following only if memory BIST was inserted in a previous pass  
add_dft_signals memory_bypass_en  
tck_occ_en
```

The following code is the DftSpecification to insert EDTs for internal and external modes:

```
EDT {  
  ijttag_host_interface : Sib(edt);  
  Controller(c1_int) {  
    longest_chain_range : 10, 50;  
    scan_chain_count : 15;  
    input_channel_count : 4;  
    output_channel_count : 3;  
    Connections {  
      mode_enables : DftSignal(int_edt_mode);  
    }  
  }  
  Controller(c1_ext) {  
    longest_chain_range : 10, 50;  
    scan_chain_count : 3;  
    input_channel_count : 1;  
    output_channel_count : 1;  
    Connections {  
      mode_enables : DftSignal(ext_edt_mode);  
    }  
  }  
}
```

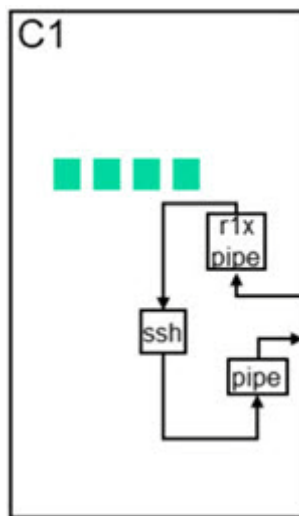
The SSN network inserted in the C1 tile does not require any SSN bus multiplexing or SSN extra output paths. The network begins with the receiver input pipeline stage, followed by SSH,

and ends at the output pipeline stage. You can use the DftSpecification from the following code sample to insert the SSN network:

```
read_config_data -in_wrapper $spec -from_string {  
  SSN {  
    ijttag_host_interface : Sib(ssn);  
    DataPath(1) {  
      output_bus_width : 4;  
      Pipeline(1) {  
      }  
      ScanHost(1) {  
        OnChipCompareMode {  
          present: on;  
        }  
      }  
      Receiver1xPipeline(in) {  
      }  
    }  
  }  
}
```

The following figure shows the inserted SSN network:

**Figure 6-47. Schematic of SSN Network Inserted in the C1 Tile**



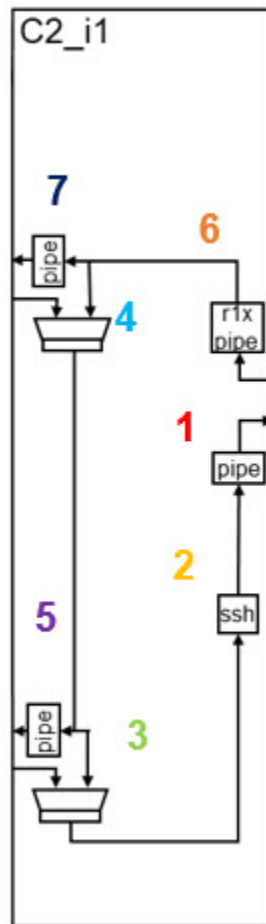
The SSN network inserted in the C2 tile has multiplexers to include or exclude neighboring tiles from the SSN datapath. The following example DftSpecification inserts an SSN network in the C2 tile:

```
SSN {
  ijttag_host_interface : Sib(ssn);
  DataPath(1) {
    output_bus_width : 4;
    // 1 in Figure
    Pipeline(1) {
    }
    // 2 in Figure
    ScanHost(1) {
    }
    // 3 in Figure
    Multiplexer (from_r2) {
      Connections {
        secondary_bus_data_in : r2_ssn_from_bus _out [3:0];
      }
    }
    // 4 in Figure
    Multiplexer (from_r1) {
      Connections {
        secondary_bus_data_in : r1_ssn_from_bus_out [3:0];
      }
    }
    // 5 in Figure
    ExtraOutputPath {
      Connections {
        bus_clock_out : r2_ssn_to_bus_clock;
        bus_data_out : r2_ssn_to_bus_data_in[3:0];
      }
      pipeline (r2) {
      }
    }
  }
  // 6 in Figure
  Receiver1xPipeline (in) {
  //7 in Figure
    ExtraOutputPath {
      Connections {
        bus_clock_out : r1_ssn_to_bus_clock;
        bus_data_out : r1_ssn_to_bus_data_in[3:0];
      }
      pipeline (r1) {
      }
    }
  }
}
```



The following figure shows the schematic of the SSN network in the C2 tile:

**Figure 6-48. Schematic of C2 Tile With SSN Network Inserted**



## Scan Insertion and Scan Modes in Tiles Without TAP and BISR Controllers

For tiles without a TAP or BISR controller, you should create at least two scan modes in each tile: internal and external.

The internal patterns from tiles are retargeted later in the tiling flow. You can combine the internal patterns into a single pattern that simultaneously tests internal modes in all tiles. Scan isolation between tiles enables simultaneous testing. After package integration, create patterns for external modes to test tile-to-tile connections. Perform wrapper cell analysis to provide scan isolation for the tile. During scan mode creation, specify the dedicated EDT instance name using the [add\\_scan\\_mode](#) command.

Run scan insertion with the [insert\\_test\\_logic](#) command. After scan insertion, change the context to “patterns -scan”, then import the internal scan mode, and run the [check\\_design\\_rules](#) command to start DRC before ATPG.

The following code example shows scan insertion in the C1 tiles and verification before ATPG. In the basic DRC check, the severity of the E5 rule that checks for the possible capture of “X” into scan chains, is modified to an error. Successful scan isolation removes E5 violations and any sources of “X” within your logic.

```
# Scan insertion step
analyze_wrapper_cells
add_scan_mode int_edt_mode -edt_instances c1_rtl3_tessent_edt_c1_int_inst
add_scan_mode ext_edt_mode -edt_instances c1_rtl3_tessent_edt_c1_ext_inst
analyze_scan_chains
insert_test_logic

# Scan mode verification and scan graybox generation step
set_context patterns -scan

# To ensure that there are no sources of X left over in both internal and
external modes
set_drc_handling E5 error
import_scan_mode int_edt_mode
check_design_rules
set_system_mode setup
import_scan_mode ext_edt_mode
#Enable boundary scan isolation
#Only if ebscan segments are present in the tile
set_static_dft_signal_values bscan_input_isolation_enable 1
set_static_dft_signal_values output_pad_disable 1
check_design_rules
analyze_graybox
write_design -tsdb -graybox
```

After scan insertion, you must generate scan graybox and IJTAG graybox views. Generate these views after importing external test mode. To generate the IJTAG graybox, you can refer to the following example:

```
# IJTAG graybox generation step
set_context patterns -ijtag
set_system_mode analysis
analyze_graybox -ijtag
write_design -tsdb -graybox -verbose
```

## Patterns Generation in Tiles Without a TAP Controller

In the patterns generation step, you generate patterns for internal modes. You must generate the external mode patterns after chip integration. External mode patterns are not retargeted to chip level.

However, you can generate and simulate some external patterns for stand-alone tile verification. We recommend that you create and verify some external mode patterns. At a minimum, you

must create a dedicated external ATPG mode. The results of internal pattern generation must be saved in a TSDB to enable reuse after design integration.

To create a dedicated ATPG mode:

1. Import the internal scan mode.
2. Change the setup of the DFT instruments, such as OCCs or DftSignals, if necessary, to meet the requirements.
3. Use the “[set\\_current\\_mode](#) <name> -type internal” command to set a new name for the current ATPG mode.

---

**Note**

---



All scan modes must be of “internal” type. This means that logic test responses are captured to scan chains only, and there is no external capture of ports from the tester side, except for SSN bus outputs. The external scan mode used to detect faults on tile-to-tile connections must be declared as “internal”.

---

4. Save the ATPG mode and pattern data for future retargeting using the [write\\_tsdb\\_data](#) command.

The following code imports the internal scan mode, generates patterns for the internal mode, and then saves them. In this example, the Tcl script (“[../utilities/write\\_testbenches.tcl](#)”) to automate writing patterns is used. It invokes the write commands multiple times to save commonly used pattern sets. You can also save the patterns using the [write\\_patterns](#) command.

```
import_scan_mode int_edt_mode -fast_capture_mode off -verbose
set_core_instance_parameters -instrument_type occ \
  -parameter_values {capture_window_size 3}
set_current_mode int_edt_mode_stuck -type internal

check_design_rules
set_fault_type stuck
create_patterns
write_tsdb_data -replace

set param_list {SIM_TOP_NAME TB SIM_COMPARE_SUMMARY 1}

# source ../utilities/write_testbenches.tcl
# Either use the Tcl script above or use write_patterns command calls as
# shown below

write_patterns patterns/parallel.v \
  -serial -verilog -replace \
  -param_list $param_list \
  -generate_info_file_dictionary on \
  -pattern_sets scan
write_patterns patterns/loopback.v \
  -serial -verilog -replace \
  -param_list $param_list \
  -generate_info_file_dictionary on \
  -pattern_sets ssn_loopback
```

The generation of external mode patterns is almost identical to the generation of internal mode patterns. The following code example generates an external scan mode and patterns for it. After importing scan mode “ext\_edt\_mode”, two signals to provide scan isolation are set to “1”, and then specify the ATPG mode using the [set\\_current\\_mode](#) command. Run the [write\\_tsdb\\_data](#) command after [check\\_design\\_rules](#) to save the ATPG mode in the Tessent Shell Database (TSDB).

```
import_scan_mode ext_edt_mode -fast_capture_mode off -verbose
# Isolate the PIs equipped with bscan in external mode
# IOs are tested in the BoundaryScan test
# Allow keeping SSN in internal capture mode when testing the paths
between the tiles.

set_static_dft_signal_values bscan_input_isolation_enable 1
set_static_dft_signal_values output_pad_disable 1

set_current_mode ext_edt_mode_stuck -type external

check_design_rules

set_fault_type stuck
set_atpg_limit -p 128
create_patterns

write_tsdb_data -replace

write_patterns patterns/parallel.v \
  -serial -verilog -replace \
  -param_list $param_list \
  -generate_info_file_dictionary on \
  -pattern_sets scan

write_patterns patterns/loopback.v \
  -serial -verilog -replace \
  -param_list $param_list \
  -generate_info_file_dictionary on \
  -pattern_sets ssn_loopback
```

During pattern generation, you can import the saved ATPG modes from the TSDB using the [add\\_core\\_instances](#) command. The values of any required signals and other settings related to the imported ATPG modes are also automatically restored. Refer to the code example in [Tile-to-Tile Test at the Package Level](#) where the external ATPG modes of the tiles are used to configure the internal ATPG modes of the chip.

For transition faults, you must generate dedicated ATPG scan modes and patterns again. The Tessent Shell scripts to generate these patterns are similar. However, you must create the ATPG mode using an additional switch “-fast\_capture\_mode on” and use the [set\\_fault\\_type](#) command to set the fault type to “transition”.

## Verification and Simulations

Run the verification simulations when a set of tile patterns is ready.

---

### Note



The [run\\_testbench\\_simulations](#) command is used to simulate the created ATPG patterns, as explained in “[ATPG-Based Testbench Simulation](#)” on page 314.

---

## ICL-Based Testbench Simulation

Generate the ICL-based patterns in the “patterns -ijtag” context. Read the core in the interface view. The `run_testbench_simulations` command generates the `PatternsSpecification` with a default set of patterns including ICL verification, MemoryBIST, MemoryBISR, and SSN continuity patterns.

The following code sample generates the ICL-based patterns and runs the related simulations:

```
set_context patterns -design_id gate -no_rtl
read_design c2 -view interface
set_current_design c2

set spec [create_patterns_specification]
process_patterns_specification

set_simulation_library_sources \
  -Y ../../library/macros/ -extensions {v} \
  -V ../../library/cells/adk.v

run_testbench_simulations
```

## ATPG-Based Testbench Simulation

The `run_testbench_simulations` command can simulate the ATPG patterns. To use this command, set the context to “patterns -scan” (`set_context patterns -scan`). You must also provide the file paths to the testbenches generated during the ATPG step. Use the `create_patterns_specification` command, with the `-testbench_files` switch to pass the file paths.

The following code example is a Tessent Shell script to run verification testbenches:

```
# Create a list of testbenches
# Tcl glob function can search for files matching the pattern
set tb_file_list [lsort -dictionary [glob ${tb_dir}/${td_prefix}*.v]]
set spec [create_patterns_specification -testbench_files $tb_file_list]
process_patterns_specification
set_simulation_library_sources \
  -v ../../library/cells/adk.v \
  -y ../../library/macros -extension v
run_testbench_simulations
```

## Embedded Boundary Scan Insertion in Tiles With a TAP Controller

The presence of a TAP controller affects the boundary scan insertion in a tile. Inserting Tessent Boundary Scan auxiliary logic enables the reuse of functional ports as SSN bus ports.

For performing embedded boundary scan, there must be a design hierarchy around the pad IO cells where you insert embedded boundary scan. If a dedicated container for pads does not exist in the tile where TAP is inserted, then you must create it. In this example, pads in the C3 tile are

split into two groups. So create two modules, C3\_top\_pads and C3\_bottom\_pads, and insert embedded boundary scan in each module. The DFT insertion in these modules makes them an embedded boundary scan segment with logic test support that can integrate into a bigger boundary scan chain.

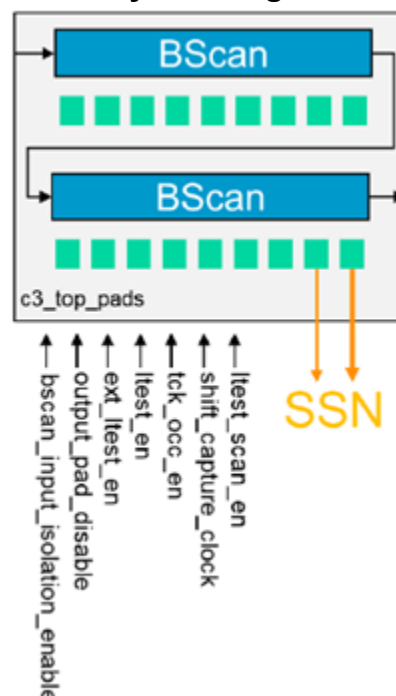
Insert embedded boundary scan logic using the “[set\\_design\\_level sub\\_block](#)” command. This connects the inserted DFT signals to the newly created ports. Refer to [Example 1](#) in [HostBScanInterface](#) in the *Tessent Shell Reference Manual* for more details on how the HostBScanInterface wrapper assembles the boundary scan chain in C3. The bscan\_inout\_isolation\_enable DFT signal is required to reuse the embedded boundary scan segments as scan isolation during logic test in external mode. The IJTAG graybox views for these modules are created automatically during ICL extraction as described in “[Creation of IJTAG Graybox](#)” on page 292.

The following code example shows the commands to create the required DFT signals:

```
add_dft_signal bscan_input_isolation_enable output_pad_disable
add_dft_signal tck_occ_en ext_ltest_en ltest_en

add_dft_signal scan_en -source_node ltest_scan_en
add_dft_signal shift_capture_clock -source_node shift_capture_clock
```

**Figure 6-49. Embedded Boundary Scan Segment With Logic Test Support Ready for Integration**



When the blocks with pads are complete, then perform the insertion of TAP and boundary scan logic to access other tiles. The IJTAG graybox views of the dedicated containers with pads (bottom pads and top pads modules) must be read in the next steps. Because the design\_level is

set to `physical_block`, you must force the insertion of the TAP interface using the [set\\_dft\\_specification\\_requirements](#) `-host_scan_interface_type tap` command (“tap” is the default type when `design_level` is “chip”).

The following code shows the setup used to integrate embedded boundary scan in the C3 tile:

```
set_context dft -rtl -design_id rtl1

# Read extra Verilog files
read_verilog ...

# Read pad containers using the ijtag_graybox view
read_design c3_top_pads -view ijtag_graybox
read_design c3_bottom_pads -view ijtag_graybox
set_current_design c3
set_design_level physical_block

# Force insertion of the TAP
set_dft_specification_requirement -host_scan_interface_type tap
check_design_rules
```



Use DftSpecification to perform the integration of embedded boundary scan segments with TAP and insertion of boundary scan logic. The following code sample shows the modifications to the default specification:

```
set spec [create_dft_specification]
set tap_wrapper ${spec}/IjtagNetwork/HostScanInterface(tap)/Tap(main)

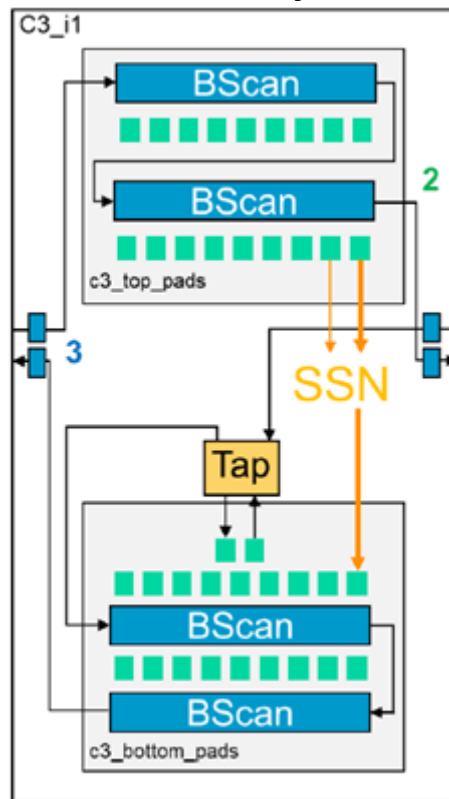
# Modify the TAP client if required
read_config_data -in $tap_wrapper -from_string {
  DeviceIDRegister {
    version_code : 4'h0;
    manufacturer_id_code : 11'h0;
    part_number_code : 16'h0;
  }
}

# Integration of boundary scan logic
read_config_data -in $spec -from_string {
  EmbeddedBoundaryScan {
    HostBscanInterface(tap) {
      Interface {
        ijtag_host_interface : Tap(main)/HostBscan;
      }
      EBScanPipeline(from_right) {
      }
      SecondaryEBScanInterface(right) {
      }
      EBScanPipeline(to_right) {
      }
      EBScanInstance(top_pads) {
      }
      EBScanPipeline(from_left) {
      }
      SecondaryEBScanInterface(left) {
      }
      EBScanPipeline(to_left) {
      }
      EBScanInstance(bottom_pads) {
      }
    }
  }
}
```

This code sample shows the detailed description of the specification used to integrate boundary scan in the C3 tile:

```
EmbeddedBoundaryScan {
  HostBscanInterface(tap) {
    Interface {
      ijtag_host_interface :
        Tap(main)/HostBscan;
    }
    EBScanPipeline(from_right) {
    }
    SecondaryEBScanInterface(right) {
    }
    EBScanPipeline(to_right) {
    }
    // 2 in Figure
    EBScanInstance(top_pads) {
    }
    // 3 in Figure
    EBScanPipeline(from_left) {
    }
    SecondaryEBScanInterface(left) {
    }
    EBScanPipeline(to_left) {
    }
    EBScanInstance(bottom_pads) {
    }
  }
}
```

**Figure 6-50. Schematic After Boundary Scan Insertion in the C3 Tile**



## Early Boundary Scan Verification

A BSDL file is required to generate the patterns, but the tool cannot infer the entire boundary scan network from the C3 tile scope.

Verify the inserted logic by adding loopbacks to unconnected SecondaryEBScanInterfaces and extracting a dummy BSDL file to generate the patterns.

### Note

This is not a mandatory step but helps in early verification of the boundary scan network at the tile level. You can also verify the boundary scan network after the integration of tiles at the chip level.

The following Tessent Shell script is used to add loopbacks, extract a fake BSDL file, generate patterns, and simulate the generated patterns:

```
# Close the host bscan scan interfaces and extract the fake BSDL file for
C3 for early verification
foreach bscan_host {left right} {
  lappend inputs [get_ports ${bscan_host}_bscan_from_scan_out]
  lappend outputs [get_ports ${bscan_host}_bscan_to_scan_in]
}
add_loadboard_loopback_pairs -inputs $inputs -outputs $outputs
set_flat_model_options -emulate_loadboard_loopback_pairs on

# Extract BSDL file from current design scope
set_dft_specification_requirements -bsdl_extraction on
set_boundary_scan_port_options -pad_io_ports [get_ports -filter \
equipped_with_bscan]
check_design_rules
# Generate the verification patterns
set_context patterns -IJTAG -rtl
set_spec [create_patterns_specification -bscan_sim_views ijtag_graybox]
process_patterns_specification
set_simulation_library_sources \
  -Y ../../library/macros/ -extensions {v} \
  -V ../../library/cells/adk.v
run_testbench_simulations
```

## IJTAG Network Insertion in Tiles With a TAP Controller

The presence of a TAP affects the IJTAG insertion in a tile.

Similar to the C2 tile that is a host for two C1 tiles, the C3 tile requires additional host scan interfaces to connect neighboring tiles. The Tile Client (TC) Segment Insertion Bit (SIB) is not required in the C3 tile because there is a TAP controller instead. Insert the Tile Host Collector (THC) SIB in series with Scan Resource Instrument (SRI) and Scan Test Instrument (STI) SIBs.

The following is used to generate the [DftSpecification](#):

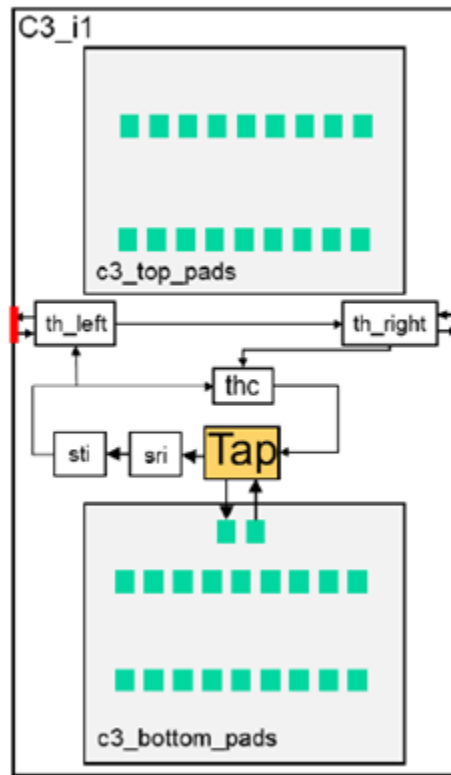
```
set_spec [create_dft_specification /
  -tile_ijtag_host_list {left right}]
report_config_data [get_config_value IjtagNetwork -in $spec]
```

Example 3 in [IJTAG Network](#) in the *Tessent Shell Reference Manual* to build the IJTAG network in the tiling flow. The following example shows the DftSpecification to insert the IJTAG network in the C3 tile:

```
IjtagNetwork {
  HostScanInterface(sti) {
    Interface {
      design_instance: c3_rtl1_tessent_sib_sri_inst;
      scan_interface : client;
    }
    Sib(thc) {
      Attributes {
        tessent_dft_function : tile_host_collector;
      }
    }
    Sib(th_right) {
      to_scan_in_feedthrough : pipeline;
      SecondaryHostScanInterface(right) {
      }
    }
    Sib(th_left) {
      to_scan_in_feedthrough : pipeline;
      SecondaryHostScanInterface(left) {
      }
    }
  }
  Sib(sti) { ... }
}
```

The following figure shows the schematic of the inserted IJTAG network. You can insert the IJTAG network with the MBIST and MBISR instruments simultaneously.

Figure 6-51. Schematic of IJTAG Network Inserted in C3 Tile



## Memory BISR Insertion for Tiles With a BISR Controller

The presence of a BISR controller in a tile affects the tiling flow. The insertion of a MemoryBIST and a MemoryBISR logic must include an MBISR controller and a fusebox.

If you set the `design_level` to “`physical_block`”, the insertion of the MBISR controller is not on by default. Enable the MBISR controller insertion using the “`set_dft_specification_requirements -memory_bisr_controller on`” command. This inserted BISR controller has to access two power domains. Therefore, you require additional host interfaces to host BISR chains in neighboring tiles. These interfaces are specified using `set_dft_specification_requirements -memory_bisr_host_list <host_list>`.

The following code sample shows how the `set_dft_specification_requirements` command and its switches are used during memory test insertion:

```
set_dft_specification_requirements \
    -memory_test on \
    -memory_bisr_controller on \
    -design_type tile \
    -memory_bisr_host_list {pdgA {pdgA_left pdgA_right} \
                           pdgB {pdgB_left pdgB_right}}
```

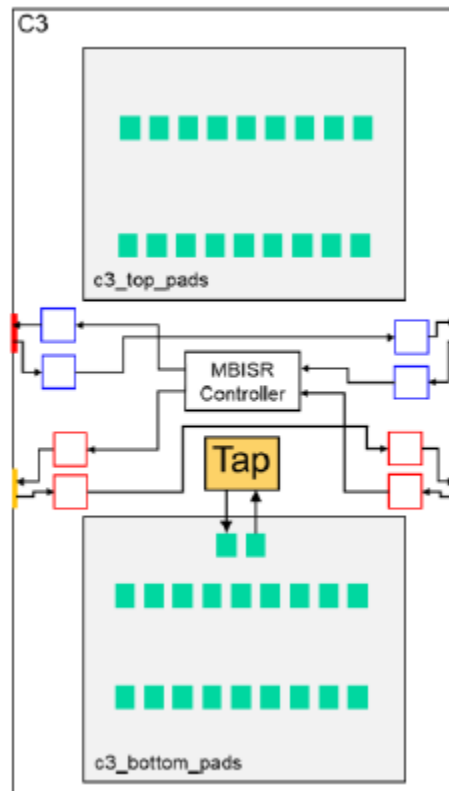
The part of the default DftSpecification that describes the memory test does not require any modifications. Read in the DEF file with coordinates for memories and ports using the [read\\_def](#) command to generate and stitch the BISR segments in layout-aware order. If the BISR segments are not in the correct order, edit the generated [bISR\\_segment\\_order\\_file](#) before processing DftSpecification. You can print the generated bISR\_segment\_order\_file using the cat command.

For more information, refer to Example 3 in MemoryBisr/[Secondary Host Chain Interface](#). The following code example shows the contents of the bISR\_segment\_order\_file generated for the C3 tile:

```
BisrSegmentOrderSpecification {
  PowerDomainGroup (pdgA) {
    OrderedElements {
      "SecondaryHostChainInterface (pdgA_left) ";
      "SecondaryHostChainInterface (pdgA_right) ";
    }
  }
  PowerDomainGroup (pdgB) {
    OrderedElements {
      "SecondaryHostChainInterface (pdgB_left) ";
      "SecondaryHostChainInterface (pdgB_right) ";
    }
  }
}
```

The following figure shows the inserted MBISR network:

**Figure 6-52. Schematic of Memory BISR Network Inserted in C3 Tile**



You cannot calculate the length of the BISR chains from the scope of the C3 tile with the `DftSpecification`. So you must set the “`repair_word_size`” and “`zero_counter_bist`” properties in the `MemoryBisr/Controller/AdvancedOptions` wrapper using the `set_config_value` command.

## SSN Insertion for Tiles With Primary SSN Ports

Inserting Tessent Boundary Scan auxiliary logic enables the reuse of functional ports as SSN bus ports. The SSN network insertion in the C3 tile is similar to that of a tile without SSN primary bus ports.

For more information on auxiliary logic insertion, refer to “[Embedded Boundary Scan Insertion in Tiles With a TAP Controller](#)” on page 314.

The SSN insertion step requires one additional DFT signal `ssn_en` to enable auxiliary logic in embedded boundary scan segments connected to the SSN bus. The SSN network in the C3 tile needs access to the SSH nodes on its left and right sides. Therefore, you need two multiplexers on the data path. The default data path configuration (after reset) does not include any neighboring tiles. For detailed information on how to insert the SSN network in a tile-based design, refer to [Example 3](#) in `DftSpecification/SSN` in the *Tessent Shell Reference Manual*.



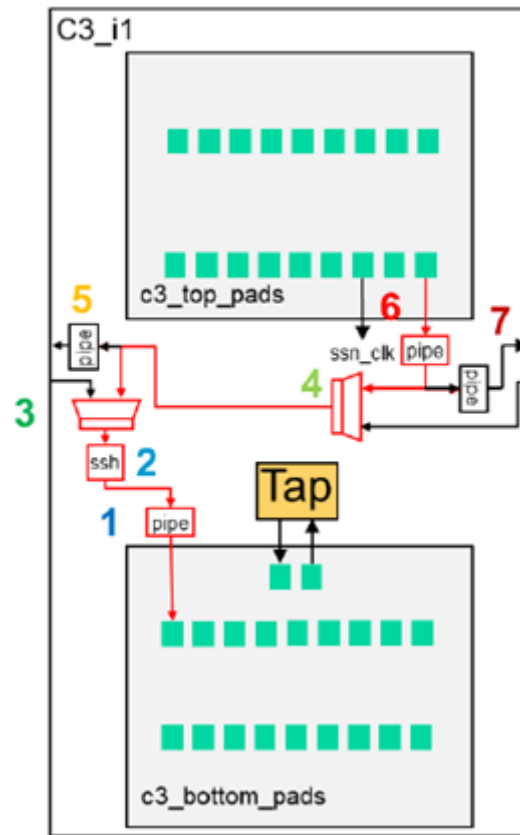
```

SSN {
  ijttag_host_interface : Sib(ssn);
  DataPath(1) {
    output_bus_width : 4;
    Connections {
      bus_clock_in : core_data_in[4];
      bus_data_in : core_data_in[3:0];
      bus_data_out : core_data_out[3:0];
    }
  }
  # 1 in figure
  Pipeline(1) {
  }
  # 2 in figure
  ScanHost(1) {
  }
  # 3 in figure
  Multiplexer (from_left) {
    Connections {
      secondary_bus_data_in : left_ssn_from_bus_data_out[3:0];
    }
  }
  # 4 in figure
  Multiplexer (from_right) {
    Connections {
      secondary_bus_data_in : right_ssn_from_bus_data_out[3:0];
    }
    ExtraOutputPath {
      Connections {
        bus_clock_out : left_ssn_to_bus_clock;
        bus_data_out : left_ssn_to_bus_data_in[3:0];
      }
    }
  }
  # 5 in figure
  pipeline (left) {
  }
}
# 6 in figure
pipeline (in) {
  ExtraOutputPath {
    Connections {
      bus_clock_out : right_ssn_to_bus_clock;
      bus_data_out : right_ssn_to_bus_data_in[3:0];
    }
  }
}
# 7 in figure
pipeline (right) {
}
}
}
}

```

The following figure is the schematic of the SSN network in the C3 tile. The path in red is a default SSN data path after reset:

**Figure 6-53. Schematic of SSN Network in the C3 Tile**



## Scan Insertion and Scan Modes for Tiles with Promoted OCCs

The scan insertion process in the C3 tile is almost identical to scan insertion in the C1 and C2 tiles. The difference between the two processes is that you must include OCC chains in external scan mode.

These chains are required for proper clocking. For more information, refer to “[Clock Control in External Mode](#)” on page 285.

The following code shows how the [add\\_scan\\_mode](#) command creates external scan mode in the C3 tile. To control the clocks, OCC chains must be a part of this scan mode:

```
# Promote clkcc OCC to external mode as there is
# no OCC outside the tile.
add_scan_mode ext_edt_mode \
  -edt_instances c3_rtl3_tessent_edt_c1_ext_inst \
  -include_elements [add_to_collection \
    [get_scan_element -class wrapper] \
    [get_scan_element *tessent_occ_clkcc*]]
```

ATPG pattern generation is not dependent on the presence of test instruments and is similar for all the tiles.

Refer to [Patterns Generation in Tiles Without a TAP Controller](#) to perform pattern generation.

## Chip-Level Integration for the Tiling Flow

When all the tiles are ready and verified standalone, you can integrate them onto the chip.

The following sections provide information on how you must integrate tiles into a single chip, generate tile-to-tile patterns, and retarget internal mode tile patterns.

|                                                     |            |
|-----------------------------------------------------|------------|
| <b>BSDL File Extraction .....</b>                   | <b>328</b> |
| <b>IJTAG Based Verification Patterns.....</b>       | <b>329</b> |
| <b>Tile Level Patterns Retargeting .....</b>        | <b>329</b> |
| <b>Tile-to-Tile Test at the Package Level .....</b> | <b>337</b> |

### BSDL File Extraction

You require a a BSDL file to generate correct patterns for the boundary scan network. Typically, you can generate the BSDL file during the insertion of the boundary scan with TAP. However, in the tiling flow the boundary scan segments may be placed in different tiles and the entire boundary scan chain is traceable only after chip-level integration. So, you must use the BSDL extraction feature to generate a chip-level BSDL file.

Read in the `ijtag_graybox` views of all tiles and then extract the ICL file using “[extract\\_icl](#) -write\_in\_tsdb on”. This creates a TSDB directory for the integrated chip that can be read in the next steps.

Perform the BSDL extraction using the “[set\\_dft\\_specification\\_requirements](#) -bsdl\_extraction on” command. The [check\\_design\\_rules](#) command triggers the process of BSDL extraction, and your result file is saved in the TSDB directory.

The following code example shows a Tessent script to extract integrated ICL and BSDL:

```
set_context dft -rtl -design_id rtl1
# Read libraries, macros
read_verilog ../../design/chip/chip.v_with_dft_port
...
# Read designs in ijtag_graybox view
read_design c1 -view ijtag_graybox
read_design c2 -view ijtag_graybox
read_design c3 -view ijtag_graybox
read_design c3_bottom_pads -view ijtag_graybox
read_design c3_top_pads -view ijtag_graybox
set_current_design chip
set_design_level chip
extract_icl -write_in_tsdb on
### Extract the chip level BSDL file
set_dft_specification_requirements -bsdl_extraction on
set_attribute_value [get_port bisr_prog_voltage] -name function \
    -value power
check_design_rules
```

## IJTAG Based Verification Patterns

The verification of IJTAG based patterns is performed after the ICL and the BSDL extraction. The BSDL file is used to generate the boundary scan patterns.

The PatternsSpecification creation must take lower physical blocks (tiles) into account. The set of generated patterns includes ICL Verification, MemoryBist, MemoryBISR, and SSN continuity patterns.

The following code sample shows a Tessent script to generate IJTAG based patterns:

```
set_defaults_value -in_wrapper /PatternsSpecification/SignoffOptions/ \
simulate_instruments_in_lower_physical_instances on
set spec [create_patterns_specification -bscan_sim_views ijtag_graybox]
report_config_data $spec
process_patterns_specification
set_simulation_library_sources -y ../../library/macros -extension v \
                             -v ../../library/cells/adk.v
run_testbench_simulations
```

## Tile Level Patterns Retargeting

To perform pattern retargeting, you must do a set of operations on each tile.

- Read in the Tessent core instance ([add\\_core\\_instances](#)).
- Define the setup for SSN multiplexing and concatenate it with the test set up procedure ([set\\_test\\_setup\\_icall](#)).
- Define the set of functional clocks in case of transition patterns (for stuck patterns use test\_clock).
- Read in the pattern file generated during internal mode ATPG.

You must set the MemoryBISR controller to not disturb the BISR logic in tiles; MBISR logic is under test during internal modes.

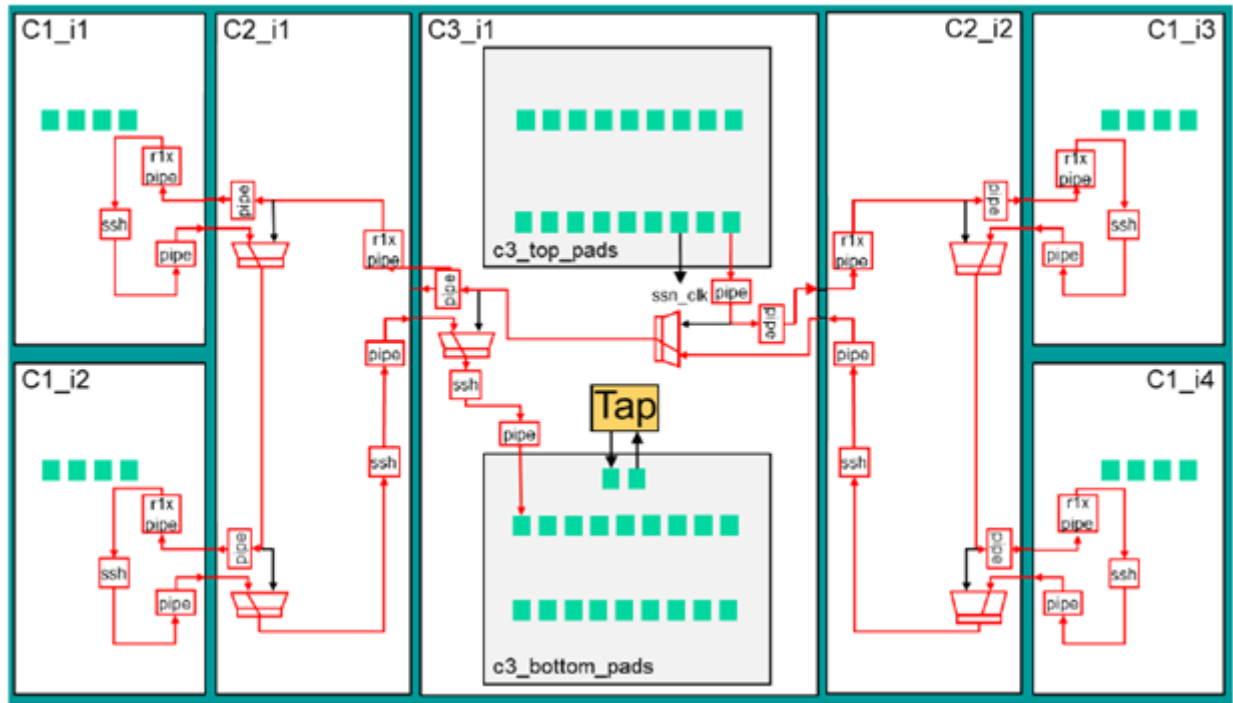
We recommend that you use a “retargeting dictionary” that helps manage the pattern retargeting. A retargeting dictionary is a Tcl dictionary with basic information about retargeting modes.

The following code shows a part of a retargeting dictionary that defines the “all\_stuck” test pattern that is retargeting stuck patterns from all tiles:

```
all_stuck {
  instances {
    c1_i1 {
      mode_name int_edt_mode_stuck
      fault_type stuck
    }
    c1_i2 {
      mode_name int_edt_mode_stuck
      fault_type stuck
    }
    c1_i3 {
      mode_name int_edt_mode_stuck
      fault_type stuck
    }
    c1_i4 {
      mode_name int_edt_mode_stuck
      fault_type stuck
    }
    c2_i1 {
      mode_name int_edt_mode_stuck
      fault_type stuck
    }
    c2_i2 {
      mode_name int_edt_mode_stuck
      fault_type stuck
    }
    c3_i1 {
      mode_name int_edt_mode_stuck
      fault_type stuck
    }
  }
  ssn_muxes {
    c2_i1.c2_rtl3_tessent_ssn_mux_from_r1_inst.setup
    c2_i1.c2_rtl3_tessent_ssn_mux_from_r2_inst.setup
    c2_i2.c2_rtl3_tessent_ssn_mux_from_r1_inst.setup
    c2_i2.c2_rtl3_tessent_ssn_mux_from_r2_inst.setup
    c3_i1.c3_rtl3_tessent_ssn_mux_from_left_inst.setup
    c3_i1.c3_rtl3_tessent_ssn_mux_from_right_inst.setup
  }
}
```

Figure 6-54 shows the SSN network configuration for the “all\_stuck” mode. The active data path is marked in red. The red multiplexers require reconfiguration; ICL instances of these multiplexers are listed under the “ssn\_muxes” key in the retargeting dictionary.

**Figure 6-54. Schematic of the SSN Network Configured for Retargeting of Internal Stuck Patterns for All Tiles**

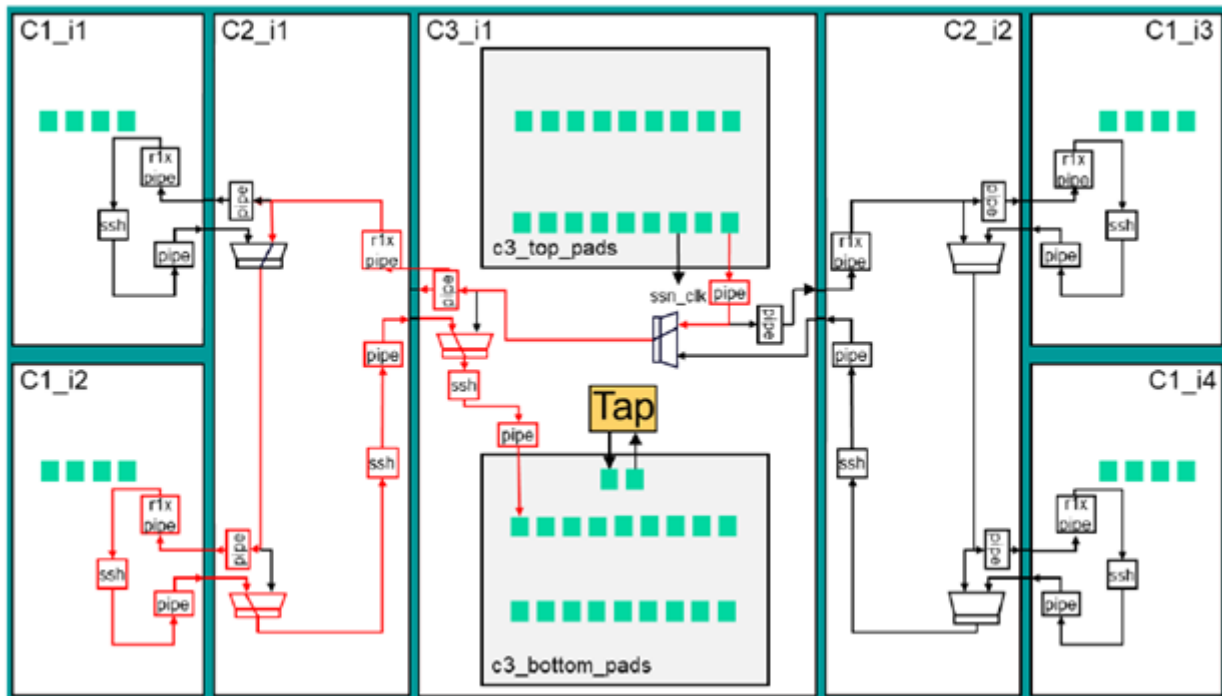


The following code shows part of the dictionary for single-tile retargeting of transition patterns:

```
c1_i2_tdf {
  instances {
    c1_i2 {
      mode_name int_edt_mode_tdf
      fault_type transition
    }
  }
  ssn_muxes {
    c2_i1.c2_rt13_tessent_ssn_mux_from_r2_inst.setup
    c3_i1.c3_rt13_tessent_ssn_mux_from_left_inst.setup
  }
}
```

Figure 6-55 shows the SSN Network configuration for this mode. You do not need to reconfigure the multiplexers marked in black but those marked in red need reconfiguring. The setup procedures related to the ICL instances of these multiplexers are listed under the “ssn\_muxes” dictionary key.

**Figure 6-55. Schematic of SSN network Configured for Retargeting of Internal Transition Patterns for Single C1\_i2 Tile**



The retargeting dictionary must be processed during the Tessent setup phase.

#### Tip

**i** We recommend using the automated script attached to the demonstration test case. The script reads the dictionary, performs the Tessent setup, and saves the retargeted pattern. Contact your Siemens representative for more information.



The following code is from a sample dofile that operates on the retargeting dictionary:

```
foreach retargeting_mode_name [dict keys $retarget_dict $mode_filter] {
# Set the name for new retargeting mode, use name from retargeting
dictionary
  set_current_mode $retargeting_mode_name
# Apply the required scan mode to the core instances
  set instances [dict keys [dict get $retarget_dict \
    $retargeting_mode_name instances]]
  foreach instance $instances {
    add_core_instances -instances $instance \
      -mode [dict get $retarget_dict $retargeting_mode_name \
        instances $instance mode_name]
  }

# Apply setting of ssn datapath muxes
  foreach ssn_mux [dict get $retarget_dict $retargeting_mode_name \
    ssn_muxes] {
    set_test_setup_icall [list $ssn_mux select_secondary_bus 1] \
      -append -merge
  }

# Add functional clocks in case of Transition Delay Faults
  if {[dict get $retarget_dict $retargeting_mode_name instances \
    $instance fault_type] eq "transition"} {
    if {$retargeting_mode_name in {c3_i1_tdf all_tdf}} {
      add_clocks bisr_clock -period 20ns
    }
    add_clocks clkc -period 10ns
    if {$retargeting_mode_name eq "all_tdf" || [regexp {^c1.*_tdf$} \
      $retargeting_mode_name]} {
      add_clocks clk_ref_1 -period 10ns
      add_clocks clk_ref_2 -period 10ns
      add_clocks clk_ref_3 -period 10ns
      add_clocks clk_ref_4 -period 10ns
    }
  }

# These settings force the BISR controller not to disturb the BISR chains
  set_static_dft_signal_values -icl_instance c3_i1 ltest_en 1
  if {$retargeting_mode_name ni {c3_i1_tdf all_tdf}} {
    add_input_constraints bisr_clock -c0
  }

# Change mode to ANALYSIS
  check_design_rules

# Read core faults from instances for retargeting
  foreach instance $instances {
    read_patterns -instance $instance \
      -mode [dict get $retarget_dict $retargeting_mode_name \
        instances $instance mode_name] \
      -fault_type [dict get $retarget_dict $retargeting_mode_name \
        instances $instance fault_type]
  }

# Write consolidated faults
```

```
write_tsdb_data -replace

# Write testbenches (patterns)
set testbench_type retargeting
source ../../utilities/write_testbenches.dofile

# Restore tool to setup for next retargeting dict entry
set_system_mode setup
reset_design
}
```

On completion, the design state is reset to start the next retargeting.

If you do not use the retargeting dictionary flow, you can use the following set of commands to retarget the TDF patterns on the C1\_i2 instance:

```
set_context patterns -scan_retargeting
...
read_design c1 -view ijtag_graybox
read_design c2 -view ijtag_graybox
read_design c3 -view ijtag_graybox
set_current_design chip
...
set_current_mode c1_i2_tdf
add_core_instances -instances c1_i2 -mode int_edt_mode_tdf
set_test_setup_icall \
  "c2_i1.c2_rtl3_tessent_ssn_mux_from_r2_inst.setup \
  select_secondary_bus 1" -append -merge set_test_setup_icall \
  "c3_i1.c3_rtl3_tessent_ssn_mux_from_left_inst.setup \
  select_secondary_bus 1" -append -merge
add_clocks bisr_clock -period 20ns
add_clocks clk_c -period 10ns
add_clocks clk_ref_1 -period 10ns
add_clocks clk_ref_2 -period 10ns
add_clocks clk_ref_3 -period 10ns
add_clocks clk_ref_4 -period 10ns
set_static_dft_signal_values -icl_instance c3_i1 ltest_en 1
check_design_rules
read_patterns -instance c1_i2 -mode int_edt_mode_tdf -fault_type
transition
write_tsdb_data -replace
write_patterns chip_retargeting.testbenches/chip_c1_i2_tdf_parallel.v \
  -verilog -replace -pattern_sets scan
write_patterns chip_retargeting.testbenches/chip_c1_i2_tdf_loopback.v \
  -serial -verilog -replace -pattern_sets ssn_loopback
write_patterns chip_retargeting.testbenches/chip_c1_i2_tdf_serial_chain.v \
  -verilog -serial -replace -pattern_sets chain
write_patterns chip_retargeting.testbenches/chip_c1_i2_tdf_serial_scan.v \
  -verilog -serial -replace -pattern_sets scan
set_system_mode setup
reset_design
```

Create the PatternSpecification for the ATPG testbenches using the [create\\_patterns\\_specification](#) command with the -testbench\_files switch and provide a list of written patterns.

```
create_patterns_specification -replace -testbench_files { \  
  chip_retargeting.testbenches/chip_c1_i2_tdf_loopback.v \  
  chip_retargeting.testbenches/chip_c1_i2_tdf_parallel.v \  
  chip_retargeting.testbenches/chip_c1_i2_tdf_serial_chain.v \  
  chip_retargeting.testbenches/chip_c1_i2_tdf_serial_scan.v \  
}
```

Simulating the retargeted patterns is automated with the PatternsSpecification flow. The `-testbench_files` switch ensures that the simulator compiles and uses the correct design view for each child block of the design.

The following code shows the created PatternsSpecification:

```
PatternsSpecification(chip,gate,c1_i2_tdf_signoff) {
  Patterns(chip_c1_i2_tdf_loopback) {
    SimulationOptions {
      LowerPhysicalBlockInstances {
        c1_i2 : full;
        c2_i1 : ijtag_graybox;
        c3_i1 : ijtag_graybox;
      }
    }
    TestbenchVerify(1) {
      testbench_file_name : \
        chip_retargeting.testbenches/chip_c1_i2_tdf_loopback.v;
    }
  }
  Patterns(chip_c1_i2_tdf_parallel) {
    SimulationOptions {
      LowerPhysicalBlockInstances {
        c1_i2 : full;
        c2_i1 : ijtag_graybox;
        c3_i1 : ijtag_graybox;
      }
    }
    TestbenchVerify(1) {
      testbench_file_name : \
        chip_retargeting.testbenches/chip_c1_i2_tdf_parallel.v;
    }
  }
  Patterns(chip_c1_i2_tdf_serial_chain) {
    SimulationOptions {
      LowerPhysicalBlockInstances {
        c1_i2 : full;
        c2_i1 : ijtag_graybox;
        c3_i1 : ijtag_graybox;
      }
    }
    TestbenchVerify(1) {
      testbench_file_name : \
        chip_retargeting.testbenches/chip_c1_i2_tdf_serial_chain.v;
    }
  }
  Patterns(chip_c1_i2_tdf_serial_scan) {
    SimulationOptions {
      LowerPhysicalBlockInstances {
        c1_i2 : full;
        c2_i1 : ijtag_graybox;
        c3_i1 : ijtag_graybox;
      }
    }
    TestbenchVerify(1) {
      testbench_file_name : \
        chip_retargeting.testbenches/chip_c1_i2_tdf_serial_scan.v;
    }
  }
}
```

---

**Note**

---

 The core C2\_i1 and C3\_i1 design views are set as IJTAG grayboxes because this view is sufficient for simulating the internal mode scan patterns of the C1\_i2 core.

---

Process the PatternsSpecification and run testbench simulations.

```
process_patterns_specification
set_simulation_library_sources -v ../data/adk.lib/verilog/adk.v -y ../data/mems -extension v
run_testbench_simulations
```

## Tile-to-Tile Test at the Package Level

The generation of tile-to-tile connection pattern is similar to the retargeting of internal modes for all tiles.

To test all the tiles, you must read in the accurate core descriptions and reconfigure the SSN network to consider all the tiles. The SSN network configuration is similar to “all\_stuck” or “all\_tdf” patterns, as shown in [Figure 6-54](#) on page 331.

Capture the responses from the scan pattern to scan chains only (internal capture) because the primary inputs and outputs get checked during boundary scan verification patterns. To do this, set the scan mode type to internal using the “[set\\_current\\_mode](#) -type internal” command.

The following code example shows how a new name for mode is set here:

```
set_context patterns -scan
# Read design prerequisites like libraries and extra Verilog files
...
read_design chip
read_design c1 -view scan_graybox
read_design c2 -view scan_graybox
read_design c3 -view scan_graybox
set_current_design chip
add_core_instances -instances [get_instances * -of_module c1] \
    -mode ext_edt_mode_stuck
add_core_instances -instances [get_instances * -of_module c2] \
    -mode ext_edt_mode
add_core_instances -instances [get_instances * -of_module c3] \
    -mode ext_edt_mode_stuck
# Using boundary Scan to isolate the chip from the outside
# Only covers tile-to-tile logic
# PIs/POs tested with boundary scan test
set_current_mode ext_edt_mode_stuck -type internal

foreach ssn_mux {
    c2_i1.c2_rtl3_tessent_ssn_mux_from_r1_inst \
    c2_i1.c2_rtl3_tessent_ssn_mux_from_r2_inst \
    c2_i2.c2_rtl3_tessent_ssn_mux_from_r1_inst \
    c2_i2.c2_rtl3_tessent_ssn_mux_from_r2_inst \
    c3_i1.c3_rtl3_tessent_ssn_mux_from_left_inst \
    c3_i1.c3_rtl3_tessent_ssn_mux_from_right_inst} {
    set_test_setup_icall \
    [list ${ssn_mux}.setup select_secondary_bus 1] -append -merge
}
check_design_rules
set_fault_type stuck
create_patterns
write_tsdb_data -replace

write_patterns chip_atpg.testbenches/chip_ext_edt_mode_stuck_parallel.v \
    -verilog -replace -pattern_set scan
```

You can simulate saved patterns using [run\\_testbench\\_simulations](#). Refer to the code sample in [Verification and Simulations](#).

# Tessent Shell Post-Layout Validation Flow

Performing physical place and route on pre-layout netlists results in post-layout netlists you must validate before you can proceed to tape-out.

This flow assumes that you are familiar with the pre-layout flows as described in “[Tessent Shell Flow for Flat Designs](#)” and “[Tessent Shell Flow for Hierarchical Designs](#),” especially as related to ATPG pattern generation.

Overview of the Post-Layout Validation Flow .....

Soft Link TSDB and Post-Layout Netlist .....

Verify MemoryBIST, Boundary Scan and IJTAG Patterns .....

Verifying the Scan-Inserted ATPG Patterns.....

Post-Layout Validation When You Have Ungrouped IJTAG/OCC/EDT Logic.....

339

340

341

342

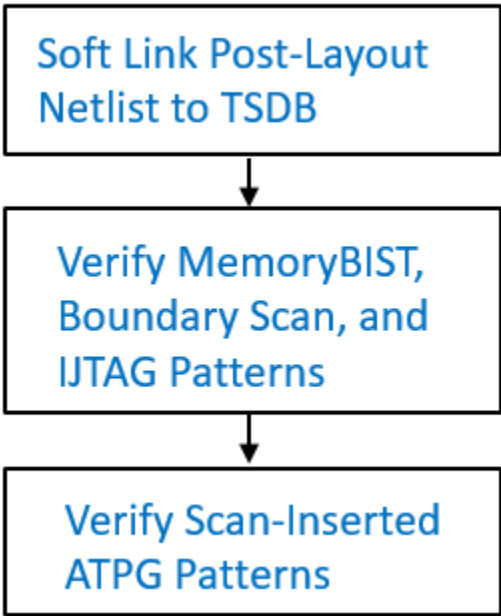
344

## Overview of the Post-Layout Validation Flow

You can validate the post-layout netlist for hierarchical core, hierarchical top, and flat chip-level designs.

As shown in the following figure, you must first generate a soft link in the TSDB that points to the post-layout netlist. Then you verify the patterns for the MemoryBIST, boundary scan (if any) and IJTAG network, followed by verifying the post-scan-inserted ATPG patterns.

Figure 6-56. Post-Layout Validation Flow



This flow assumes that within the post-layout netlist, modules exist for the IJTAG network and inserted EDT and OCC instruments. That is, they remain distinct logical entities. Refer to “[Post-Layout Validation When You Have Ungrouped IJTAG/OCC/EDT Logic](#)” if you have ungrouped (unpreserved) logic.

## Soft Link TSDB and Post-Layout Netlist

The soft link enables Tessent Shell to access the post-layout netlist for validation purposes. You are associating and linking the post-layout netlist (place-and-routed design) with all the pre-layout data files such as the ICL, TCD, and PDL.

### Prerequisites

- You have previously performed the pre-layout flow so that you have the post-scan inserted DFT data files (ICL, PDL, TCD, and so on).
- You have a post-layout netlist as a result of place and route.

### Procedure

1. Load the design, including the post-layout netlist (lines 1-16). Ensure that you specify a unique design ID, such as “post\_layout”.
2. Add any black boxes, as applicable, and set the design level, which can be chip, physical\_block, or sub\_block (lines 18-23).
3. Save the updated netlist (line 24). Ensure that you use the -softlink\_netlist option with the [write\\_design](#) command.

Using this switch references the post-layout netlist rather than copies it into the TSDB. This prevents duplication and enables the post-layout netlist to be updated without the need to repeat this step.

### Examples

The following example generates an updated netlist for a wrapped core named processor\_core that includes a soft link to the core’s post-layout netlist.

```
read_cell_library ../../library/memories/SYNC_1RW_8Kx16.atpglib

1  set_context patterns -ijtag -design_id post_layout
2
3  # Set the location of the TSDB
4  set_tsdb_output_directory ../tsdb_core
5
6  # Read the Tessent Cell Library
7  read_cell_library \
8  ../../library/standard_cells/tessent/NangateOpenCellLibrary.tcelllib
9  read_cell_library ../../library/memories/SYNC_1RW_8Kx16.atpglib
10 # Read the post-layout netlist and elaborate the design
11 read_verilog ../7.place_and_route/verilog/processor_core.post_layout.vg
12
```



```

13 # Read in the scan-inserted design data files generated during pre-layout
14 # The -no_hdl switch loads all relevant data except the original netlist
15 read_design processor_core -design_id gate -no_hdl -verbose
16
17 set_current_design processor_core
18
19 add_black_boxes -modules { \
20                     SYNC_1RW_8Kx16 \
21                     }
22
23 # Specify the design level before writing out the softlink
24 set_design_level physical_block
25 write_design -tsdb -softlink_netlist -verbose
26 exit

```

## Verify MemoryBIST, Boundary Scan and IJTAG Patterns

Create and validate the MemoryBIST, boundary scan (if present), and IJTAG network patterns by creating and processing a patterns specification with the `create_patterns_specification` and `process_patterns_specification` commands.

### Prerequisites

- You have generated a soft link in the TSDB that points to the post-layout netlist as described in “[Soft Link TSDB and Post-Layout Netlist](#).”

### Procedure

- Load the design by using `read_design` and pointing to the soft-linked post-layout netlist (lines 1-9).
- If needed, add black boxes, and then check the design rules (lines 11-14).
- Create, simulate, and check the testbenches (see lines 16-24). Refer to “`create_patterns_specification`” and “`process_patterns_specification`” in the *Tessent Shell Reference Manual* for details.

### Examples

#### Verify the Patterns With a New Patterns Specification

The following example verifies the patterns for a hierarchical core, `processor_core`.

```

1  set_context_patterns -ijtag -design_id post_layout
2  set_tsdb_output_directory ../tsdb_core
3
4  # Read the tessent cell library
5  read_cell_library \
6    ../../../../library/standard_cells/tessent/NangateOpenCellLibrary.tcelllib
7  # Reading the soft-linked post-layout netlist from the tsdb_core
8  read_design processor_core -design_id post_layout -verbose
9  set_current_design processor_core
10

```

```

11 add_black_boxes -modules { \
12                     SYNC_1RW_8Kx16 \
13                     }
14 check_design_rules
15
16 create_patterns_specification
17 process_patterns_specification
18
19 set_simulation_library_sources \
20   -v ../../../../library/standard_cells/verilog/NangateOpenCellLibrary.v \
21   -v ../../../../library/memories/SYNC_1RW_8Kx16.v
22
23 # Disable clock monitoring when running simulation on post-layout netlist
24 run_testbench_simulations -simulation_macro_definitions \
25   TESSENT_DISABLE_CLOCK_MONITOR
26 check_testbench_simulations -report_status
27 exit

```

### Verify the Patterns with a Customized Patterns Specification from Pre-Layout Signoff

You may have customized a patterns specification during pre-layout signoff. You can read in this patterns specification that was stored in the TSDB during pre-layout signoff. Use the `read_config_data` command instead of the `create_patterns_specification` command.

The following example shows how to read in a customized patterns specification from pre-layout netlist. The pre-layout patterns specification “processor\_core\_gate.patterns\_spec\_signoff” was used to create the patterns on the gate-level scan-inserted netlist.

```

set_context patterns -ijtag -design_id after_layout
set_tsdb_output_directory ../tsdb_core
# Reading the tessent cell library
# Read the soft-linked post-layout netlist and elaborate the design...
check_design_rules
read_config_data ../tsdb_core/patterns/ \
  processor_core_gate.patterns_spec_signoff
process_patterns_specification
# Read in the required libraries and simulate
...

```

## Verifying the Scan-Inserted ATPG Patterns

Perform ATPG on the post-layout netlist and save the patterns.

### Prerequisites

- In the SDC that was created during [ICL extraction](#) for the pre-layout DFT insertion flow, set the `tessent_get_preserve_instances` proc to `add_core_instances` when you do not need grayboxes. For example, when you are working with flat designs.

For wrapped cores, set the `tessent_get_preserve_instances` proc to `icl_extraction` to automatically include ICL instances in the grayboxes.

The SDC file is located in the TSDB directory under `dft_inserted_designs`. Refer to [“Timing Constraints \(SDC\)”](#) for more information.

## Procedure

1. Load the design (lines 1-9). Ensure that you set the context to patterns -scan and read in the soft-linked post-layout netlist.
2. Set the current mode (lines 11-12). Specify a different name than that used during scan insertion and used during pre-layout pattern generation.
3. Perform the remainder of the ATPG pattern generation flow (see lines 14-35).

## Examples

The following example shows how to verify scan-inserted ATPG patterns by using the patterns -scan context and a post-layout netlist. This example shows the flow for a flat design. For hierarchical designs, refer to [“Performing ATPG Pattern Generation: Wrapped Core”](#) for specifics related to wrapped cores and [“Top-Level ATPG Pattern Generation Example”](#) for a top-level example.

```
1  set_context patterns -scan
2  read_cell_library \
3    ../library/standard_cells/tessent/NangateOpenCellLibrary.tcelllib
4  read_cell_library ../library/mem_ver/memory.lib
5  # Point to the TSDB directory
6  set_tsdb_output_directory ../tsdb_rtl
7
8  # Reading the post-layout netlist
9  read_design cpu_top -design_id post_layout
10 set_current_design cpu_top
11
12 # Use a unique mode name
13 set_current_mode edt_stuck_final
14
15 report_dft_signals
16 # If Tessent Scan was used for scan insertion, can use the scan
   #configuration mode
17 import_scan_mode edt_mode
18 # Set the following DFT Signal values to use the boundary scan chain to
   #apply/capture values that would normally use the I/O pads
19 set_static_dft_signal_values int_ltest_en 1
20 set_static_dft_signal_values output_pad_disable 1
21 # Set the following DFT signal value to apply the shift_capture_clock to
   #the scan-tested network during capture phase of ATPG
22 set_static_dft_signal_value tck_occ_en 1
23 report_static_dft_signal_settings
24
25 set_system_mode analysis
26
27 # Generation of ATPG patterns
28 create_patterns
29 report_statistics -detailed_analysis
30 write_tsdb_data -replace
31
```

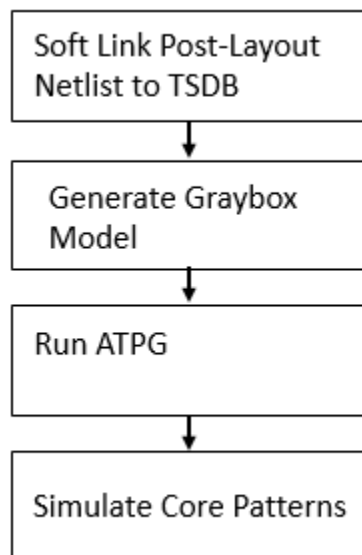
```
32 # Patterns written out for simulation
33 write_patterns patterns/cpu_top_stuck_parallel.v -verilog -parallel \
    -replace -scan
34 write_patterns patterns/cpu_top_stuck_serial.v -verilog -serial -replace \
35 # Writing the STIL pattern for tester
36 write_patterns patterns/cpu_top_stuck.stil -stil -replace
37 exit
```

## Post-Layout Validation When You Have Ungrouped IJTAG/OCC/EDT Logic

During layout, IJTAG, OCC, and EDT logic can become ungrouped, which means the hierarchy around this logic is dissolved so that the gates that were inside of the ungrouped instances become part of the next higher instance. Ungrouped test logic during layout can cause some of the automated setup for pattern generation to no longer operate seamlessly.

The automated flow relies on preservation of Tessent test logic's hierarchical names. Use the following post-layout validation flow to update the TSDB with the post-layout netlist and set up the design for ATPG. The flow assumes knowledge of the ATPG pattern generation flow for physical blocks as described in [“Performing ATPG Pattern Generation: Wrapped Core”](#).

**Figure 6-57. Post-Layout Validation Flow with Ungrouped IJTAG/OCC/EDT Logic**



The best way to avoid complications related to ungrouping in layout is to use the `tessent_get_preserve_instances` procedures in the generated SDC file to identify which instances must be preserved based on their intended uses. See the [“Synthesis Helper Procs”](#) section in the *Tessent Shell User's Manual*.

If you expect the layout process to ungroup the test instruments, you can preserve the hierarchical names by adding persistent buffers. You must generate the IP using the [DftSpecification](#). The `add_persistent_buffers_in_scan_resource_instruments` property is on by default. With persistent buffers present, you can run the `add_core_instances` command directly on the IP instance, even if it is ungrouped. Otherwise, you must follow the example using “`add_core_instances -current_design`” if the IP is ungrouped without persistent buffers present.

## Prerequisites

- If you do not preserve the instances, you must write out a TCD file for every mode of operation at the core level before you perform layout. In some cases, you may not need to generate patterns until after layout, but even then, you must run ATPG before layout to generate the core-level TCD files. Failure to perform this task could result in R14 or R15 rule check errors.

## Procedure

1. Soft link the post-layout netlist to the TSDB. Refer to “[Soft Link TSDB and Post-Layout Netlist](#)” for more information.
2. Generate the graybox model.

Graybox models are only required for hierarchical cores, not for hierarchical top or flat designs. (The line numbers in this step refer to the example “[Generate the Graybox Model](#)” on page 346.)

- a. When loading the design (line 4), specify the same design that you used to write the post-layout netlist to the TSDB, for example “`post_layout`”.

Using the same design ID for the graybox model that you used for the post-layout netlist enables Tessent to access the full design view or the graybox model with the same design ID.

- b. Read the core description for external mode using the `add_core_instances` command (line 9).

Because you already ran the pre-layout ATPG step and saved the TCD file to the TSDB, the `add_core_instances` command can read the existing TCD file and add the core instances that are active in external mode.

- c. Use `analyze_graybox` to generate the graybox (line 13). If the IJTAG network was ungrouped in layout, the `analyze_graybox` command can automatically find and preserve the IJTAG SRI network as long as the default names of the logic have not changed.

3. If you are running in hierarchical mode, run ATPG on the core’s internal mode of the wrapped core to generate the ATPG patterns that you retarget at the top level of the chip. If you are running in hierarchical top or in flat mode, run ATPG.

The line numbers in this example refer to example “[Run ATPG on the Core’s Internal Mode](#)” on page 347.

- a. Specify a unique ATPG mode name with the `set_current_mode` command (line 10). Append the name with “\_post\_layout” or “\_final” to clarify which mode to use for silicon diagnosis, and then set the current mode type to “internal”.

- b. Add the core instances using the design ID and mode from the pre-layout step (line 15).

This loads the TCD file that contains the information for the design’s core instances. It is unnecessary to match these instances to the test logic instances that may have been ungrouped during layout.

- c. (Optional) Use the [read\\_faults](#) command to merge the fault list from running external mode to find the total overall fault coverage of the wrapped core (line 37).

When you run ATPG in internal mode for transition patterns, you must do the following:

- Specify a unique name for the ATPG run with the `set_current_mode` command. For example, `edt_int_tdf_final`.
- Use the `add_core_instances` command to read the transition mode TCD. For example:

```
add_core_instance -current_design -design_id gate -mode
edt_int_transition
```

- Ensure that you set the correct fault type:

```
set_fault_type transition
```

- You can optimize the number of capture cycles used by the OCCs by specifying the optional `capture_window_size` parameter. The following command specifies a `capture_window_size` of 2:

```
set_core_instance_parameters -instrument_type occ
-parameter_values [list capture_window_size 2]
```

4. Run Verilog simulation of the core-level ATPG patterns.

Performing this task ensures that the patterns function as needed when they are retargeted at the parent level. For parallel load patterns, as specified by the `write_patterns -parallel` command, simulate all the patterns. For serial load patterns, a handful of patterns are sufficient; the run time for simulating gate-level serial load patterns is significant. The `set_pattern_filtering` command (line 28) is used to reduce the number of serial patterns saved for the simulation.

## Examples

### Generate the Graybox Model

The following example creates a graybox model for a post-layout processor\_core design and saves the data under the same design ID.

```
1 set_context patterns -scan -design_id post_layout
```

```

2 set_tsdb_output_directory ../tsdb_outdir
3 read_cell_library \
4   ../../library/standard_cells/tessent/NangateOpenCellLibrary.tcelllib
5 read_design_processor_core -design_id post_layout -verbose
6 read_verilog ../../library/memories/SYNC_1RW_8Kx16.v -interface_only
7 set_current_design_processor_core
8 # Use the add_core_instances command to read the TCD file.
9 #import_scan_mode ext_mode
10 add_core_instances -current_design -design_id gate
11 check_design_rules
12 report_scan_cells
13 # Create and write the updated graybox model to the TSDB.
14 analyze_graybox
15 write_design -tsdb -graybox -verbose
16 exit

```

### Run ATPG on the Core's Internal Mode

```

1 set_context patterns -scan -design_id post_layout
2 set_tsdb_output_directory ../tsdb_outdir
3 read_cell_library \
4   ../../library/standard_cells/tessent/NangateOpenCellLibrary.tcelllib
5 read_design_processor_core -design_id post_layout -verbose
6 read_verilog ../../library/memories/SYNC_1RW_8Kx16.v -interface_only
7 set_current_design_processor_core
8 # Specify the current mode using a different name than what was used
9 # during scan insertion or pre-layout
10
11 set_current_mode edt_int_stuck_final -type internal
12
13 # Use the -design_id and -mode from pre-layout to read in the TCD of that
14 # mode
15
16 add_core_instance -current_design -design_id gate -mode edt_int_stuck
17 set_system_mode analysis
18 add_fault -all
19 report_statistics -detail
20 create_patterns
21 report_statistics -detail
22 # Store TCD, flat_model, fault list and patDB format files in the TSDB
23 # directory
24
25 write_tsdb_data -replace
26
27 # Write Verilog patterns for simulation
28 write_patterns patterns/processor_core_stuck_parallel.v -verilog
   -parallel -replace
29 set_pattern_filtering -sample_per_type 2
30 write_patterns patterns/processor_core_stuck_serial.v -verilog -serial
   -replace
31 # Optional Step - Can run in external mode and calculate the fault
32 # coverage of the core(both Internal and External) as described below.
33 # In order to understand the coverage of the faults testable by Internal
34 # mode it is necessary to eliminate the undetected faults that would
35 # otherwise be detected in External mode. This is done by merging the
36 # fault list from running the graybox in External mode with read_faults:
37
38 #read_faults -mode ext_multi_stuck -fault_type stuck -merge -verbose

```

```
39
40 # Final coverage of the core that includes both Internal and External
41 # modes
42 # report_statistics -detail
43 exit
```



# Testbench Generation and Simulation in RTL Mode

Testbench module generation and the Standard Test Interface (STI) infrastructure support complex port data types. The tool generates a simulation wrapper to connect a design under test (DUT) with specific complex port types to the testbench environment.

The simulation wrapper port list includes ports of scalar and one-dimensional bus data types. The port names are the post-synthesis names of sub-bundle objects as inferred from the original RTL complex ports of the DUT. This approach isolates the DUT from the testbench environment and hides the complexity of its ports and other SystemVerilog dependencies.

|                                          |            |
|------------------------------------------|------------|
| <b>Simulation Wrapper Creation .....</b> | <b>349</b> |
| <b>Testbench Examples .....</b>          | <b>352</b> |

## Simulation Wrapper Creation

Simulation wrapper creation occurs while the tool is processing DFT specifications and extracting ICL. Use the simulation wrapper for designs with specific complex port types.

The tool creates simulation wrappers during [process\\_dft\\_specification](#) and [extract\\_icl](#). You need the simulation wrapper in the testbench for designs that contain the following complex port types if they drive (or are driven by) a testbench port:

- enumerations
- unpacked arrays
- unpacked structs
- SystemVerilog interfaces

You can also create the simulation wrapper manually with the [get\\_current\\_design](#), [report\\_current\\_design](#), and [write\\_design](#) commands.

The tool reports a warning if [read\\_design](#) encounters ports that can cause issues in simulation or in the TSDB flow:

```
// Warning: The design 'sv_mod1' just read has ports with 'unpacked struct'/'enumerate'  
// datatypes declared in Global ($unit) or local scope ('sv_mod1' scope).  
// If you try to elaborate this design within its parent, you will get an error.  
// It will only work if this design is your current design.
```

These issues typically arise when port declarations use data types that are not visible to the tool. The warning indicates that a call to [set\\_current\\_design](#) of the parent block raises additional warnings. Use [report\\_current\\_design](#) after running [set\\_current\\_design](#) to obtain more information about these ports.

The following example shows how the simulation wrapper is created during `extract_icl`:

```
// command: extract_icl
// Writing simulation wrapper : \
./tsdb_outdir/dft_inserted_designs/sv_mod1_RTL1.dft_inserted_design/
sv_mod1.sv09_simulation_wrapper
// Note: Updating the hierarchical data model to reflect RTL design
changes.
// Writing design source dictionary : \
./tsdb_outdir/dft_inserted_designs/
sv_mod1_RTL1.dft_inserted_design/sv_mod1.design_source_dictionary
// Flattening process completed, cell instances=111, gates=1310, PIs=470,
POs=166, CPU time=0.04 sec.
// -----
// Begin circuit learning analyses.
// -----
// Learning completed, CPU time=0.02 sec.
// -----
// Begin ICL extraction.
// -----
// ICL extraction completed, ICL instances=10, CPU time=0.16 sec.
// -----
// -----
// Begin ICL elaboration and checking.
// -----
// ICL elaboration completed, CPU time=0.09 sec.
// -----
// Writing ICL file : ./tsdb_outdir/dft_inserted_designs/
sv_mod1_RTL1.dft_inserted_design/sv_mod1.icl
// Writing consolidated PDL file: ./tsdb_outdir/dft_inserted_designs/
sv_mod1_RTL1.dft_inserted_design/sv_mod1.pdl
// Writing SDC file: ./tsdb_outdir/dft_inserted_designs/
sv_mod1_RTL1.dft_inserted_design/sv_mod1.sdc
// Writing DFT info dictionary: ./tsdb_outdir/dft_inserted_designs/
sv_mod1_RTL1.dft_inserted_design/
sv_mod1.dft_info_dictionary
```

The following example shows how the simulation wrapper is created during process\_dft\_specification:

```
// command: process_dft_specification
//
// Begin processing of /DftSpecification(sv_mod1,RTL2)
// --- IP generation phase ---
// Validation of IjtagNetwork
// Validation of OCC
// Validation of EDT
// Processing of IjtagNetwork
// Generating design files for IJTAG SIB module
sv_mod1_RTL2_tessent_sib_1
// Verilog RTL : ./tsdb_outdir/instruments/
sv_mod1_RTL2_ijtag.instrument/sv_mod1_RTL2_tessent_sib_1.v
// IJTAG ICL : ./tsdb_outdir/instruments/
sv_mod1_RTL2_ijtag.instrument/sv_mod1_RTL2_tessent_sib_1.icl
// Generating design files for IJTAG SIB module
sv_mod1_RTL2_tessent_sib_2
// Verilog RTL : ./tsdb_outdir/instruments/
sv_mod1_RTL2_ijtag.instrument/sv_mod1_RTL2_tessent_sib_2.v
// IJTAG ICL : ./tsdb_outdir/instruments/
sv_mod1_RTL2_ijtag.instrument/sv_mod1_RTL2_tessent_sib_2.icl
// Generating design files for IJTAG Tdr module
sv_mod1_RTL2_tessent_tdr_sri_ctrl
// Verilog RTL : ./tsdb_outdir/instruments/
sv_mod1_RTL2_ijtag.instrument/sv_mod1_RTL2_tessent_tdr_sri_ctrl.v
// IJTAG ICL : ./tsdb_outdir/instruments/
sv_mod1_RTL2_ijtag.instrument/sv_mod1_RTL2_tessent_tdr_sri_ctrl.icl
//
// Loading the generated RTL verilog files (2) to enable instantiating
the contained modules
// into the design.
// Processing of EDT
// Generating design files for EDT module
sv_mod1_RTL2_tessent_edt_edt1
// Verilog RTL : ./tsdb_outdir/instruments/
sv_mod1_RTL2_edt.instrument/sv_mod1_RTL2_tessent_edt_edt1.v
// IJTAG ICL : ./tsdb_outdir/instruments/
sv_mod1_RTL2_edt.instrument/sv_mod1_RTL2_tessent_edt_edt1.icl
// IJTAG PDL : ./tsdb_outdir/instruments/
sv_mod1_RTL2_edt.instrument/sv_mod1_RTL2_tessent_edt_edt1.pdl
// TCD : ./tsdb_outdir/instruments/
sv_mod1_RTL2_edt.instrument/sv_mod1_RTL2_tessent_edt_edt1.tcd
//
// Loading the generated RTL verilog files (1) to enable instantiating
the contained modules
// into the design.
// --- Instrument insertion phase ---
// Inserting instruments of type 'ijtag'
// Inserting instruments of type 'edt'
//
// Writing out modified source design in ./tsdb_outdir/
dft_inserted_designs/sv_mod1_RTL2.dft_inserted_design
// Writing simulation wrapper : ./tsdb_outdir/dft_inserted_designs/
sv_mod1_RTL2.dft_inserted_design/sv_mod1.sv09_simulation_wrapper
// Writing out specification in ./tsdb_outdir/dft_inserted_designs/
```

```
sv_mod1_RTL2.dft_spec
//      Done  processing of
DftSpecification(sv_mod1,RTL2
```

## Related Topics

[get\\_current\\_design](#)

[read\\_design](#)

[report\\_current\\_design](#)

[write\\_design](#)

[set\\_current\\_design](#)

## Testbench Examples

SystemVerilog design definition with the top level named “top” and one SystemVerilog interface port named “Bus”:

```
typedef enum logic [2:0] {
    RED, GREEN, BLUE, CYAN, MAGENTA, YELLOW
} color_t;

typedef struct packed{
    logic [2:0] eventbus;
} event_bus_packet_t;

typedef struct packed{
    color_t state_encoded;
    event_bus_packet_t dt_lcp_pulse;
    logic [3:0] eventbus_encoded;
} event_bus_split_packet_t;

interface MSBus;
    event_bus_split_packet_t Addr;
    logic [1:0] Data;
    logic RWn;
    logic Clk;
    modport Secondary (input Addr, RWn, Clk, output Data);
endinterface

module top(MSBus.Secondary Bus);
endmodule
```

Each bit-blasted bit of the DUT is connected to its associated test pattern wire using the post-synthesis pin name. The name of the test pattern bit is the same as that of the DUT pin bit, escaped with a backslash (\). The DUT instance (DUT\_inst) is the generated simulation

wrapper, with the DUT itself instantiated in it. This table cross-references the DUT pin names and the test pattern bit names:

**Table 6-1. Test Pattern Bit Name Assignments**

| DUT Pin of Port “Bus”  | Test Pattern Bits |
|------------------------|-------------------|
| DUT_inst\Bus.Addr [10] | \Bus.Addr [10]    |
| DUT_inst\Bus.Addr [9]  | \Bus.Addr [9]     |
| DUT_inst\Bus.Addr [8]  | \Bus.Addr [8]     |
| DUT_inst\Bus.Addr [7]  | \Bus.Addr [7]     |
| DUT_inst\Bus.Addr [6]  | \Bus.Addr [6]     |
| DUT_inst\Bus.Addr [5]  | \Bus.Addr [5]     |
| DUT_inst\Bus.Addr [4]  | \Bus.Addr [4]     |
| DUT_inst\Bus.Addr [3]  | \Bus.Addr [3]     |
| DUT_inst\Bus.Addr [2]  | \Bus.Addr [2]     |
| DUT_inst\Bus.Addr [1]  | \Bus.Addr [1]     |
| DUT_inst\Bus.Addr [0]  | \Bus.Addr [0]     |
| DUT_inst\Bus.RWn       | \Bus.RWn          |
| DUT_inst\Bus.Clk       | \Bus.Clk          |
| DUT_inst\Bus.Data [1]  | \Bus.Data[1]      |
| DUT_inst\Bus.Data [0]  | \Bus.Data[0]      |

The testbench module generated is as follows, in part:

```
`timescale 1ns / 1ns
module TB;
  integer      _write_DIAG_file;
  integer      _DIAG_file_header;
  integer      _diag_file;
  integer      _diag_chain_header;
  integer      _diag_scan_header;
  integer      _last_fail_pattern;
  . . .
  wire \Bus.Addr[10], \Bus.Addr[9], \Bus.Addr[8], \Bus.Addr[7],
      \Bus.Addr[6], \Bus.Addr[5], \Bus.Addr[4], \Bus.Addr[3],
      \Bus.Addr[2], \Bus.Addr[1], \Bus.Addr[0], \Bus.RWn ,
      \Bus.Clk , \Bus.Data[1], \Bus.Data[0], ijtag_tck,
      ijtag_reset, ijtag_ce, ijtag_se, ijtag_ue, ijtag_sel,
      ijtag_si, ijtag_so;
  assign \Bus.Addr[10] = _ibus[19];
  assign \Bus.Addr[9] = _ibus[18];
  assign \Bus.Addr[8] = _ibus[17];
  assign \Bus.Addr[7] = _ibus[16];
  assign \Bus.Addr[6] = _ibus[15];
  assign \Bus.Addr[5] = _ibus[14];
  assign \Bus.Addr[4] = _ibus[13];
  assign \Bus.Addr[3] = _ibus[12];
  assign \Bus.Addr[2] = _ibus[11];
  assign \Bus.Addr[1] = _ibus[10];
  assign \Bus.Addr[0] = _ibus[9];
  assign \Bus.RWn = _ibus[8];
  assign \Bus.Clk = _ibus[7];
  assign ijtag_tck = _ibus[6];
  assign ijtag_reset = _ibus[5];
  assign ijtag_ce = _ibus[4];
  assign ijtag_se = _ibus[3];
  assign ijtag_ue = _ibus[2];
  assign ijtag_sel = _ibus[1];
  assign ijtag_si = _ibus[0];
  assign _sim_obus[2] = \Bus.Data[1] ;
  assign _sim_obus[1] = \Bus.Data[0] ;
  assign _sim_obus[0] = ijtag_so;
  . . .
  reg[71:0] mem [0:1864134];
  top_wrapper DUT_inst (. \Bus.Addr ({\Bus.Addr[10], \Bus.Addr[9],
      \Bus.Addr[8], \Bus.Addr[7],
      \Bus.Addr[6], \Bus.Addr[5],
      \Bus.Addr[4], \Bus.Addr[3],
      \Bus.Addr[2], \Bus.Addr[1],
      \Bus.Addr[0]}),
      . \Bus.RWn (. \Bus.RWn ),
      . \Bus.Clk (. \Bus.Clk ),
      . \Bus.Data ({\Bus.Data[1], \Bus.Data[0]},
          .ijtag_tck(ijtag_tck),
          .ijtag_reset(ijtag_reset),
          .ijtag_ce(ijtag_ce),
          .ijtag_se(ijtag_se),
          .ijtag_ue(ijtag_ue),
          .ijtag_sel(ijtag_sel),
```

```
        .ijtag_si(ijtag_si),  
        .ijtag_so(ijtag_so));  
    . . .  
endmodule
```

Simulation wrapper:

```
module top_wrapper(input wire [10:0] \Bus.Addr ,  
                  input wire \Bus.RWn ,  
                  input wire \Bus.Clk ,  
                  output wire [1:0] \Bus.Data  
                  input wire ijtag_tck,  
                  input wire ijtag_reset,  
                  input wire ijtag_ce,  
                  input wire jtag_se,  
                  input wire ijtag_ue,  
                  input wire ijtag_sel,  
                  input wire ijtag_si,  
                  output wire ijtag_so);  
  
    MSBus tessent_intf_inst1();  
  
    assign tessent_intf_inst1.Addr = \Bus.Addr ;  
    assign tessent_intf_inst1.RWn = \Bus.RWn ;  
    assign tessent_intf_inst1.Clk = \Bus.Clk ;  
    assign \Bus.Data = tessent_intf_inst1.Data;  
  
    top_current_design(.Bus(tessent_intf_inst1),  
                      .ijtag_tck(ijtag_tck),  
                      .ijtag_reset(ijtag_reset),  
                      .ijtag_ce(ijtag_ce),  
                      .ijtag_se(ijtag_se),  
                      .ijtag_ue(ijtag_ue),  
                      .ijtag_sel(ijtag_sel),  
                      .ijtag_si(ijtag_si),  
                      .ijtag_so(ijtag_so));  
  
endmodule
```

The testbench dofile:

```
set_context dft -rtl -design_id RTL1
read_cell_library techlib_adk.tnt/current/tessent/adk.tcelllib
read_core_description design/mem/*.tcd_memory
read_verilog design/packages/types.sv -format sv2009
read_verilog -f design/sv.list -format sv2009
set_current_design top
set_design_level phys
set_dft_specification_requirements -memory_test on
add_clocks [get_ports Bus.Clk] -period 10ns
check_design_rules
set_spec [create_dft_spec]
// (the spec is populated here)
process_dft_spec
extract_icl
create_pattern_spec
process_pattern_spec
set_simulation_library_sources -v techlib_adk.tnt/current/verilog/adk.v
                                -logical_library_map_list memlib
                                ./memlib -v design/sv_v_files/pll.v
run_testbench_simulation -simulator_options {-t 10ps -L
memlib}
```



# Hybrid TK/LBIST Flow for Flat Designs

Tessent Shell supports inserting LogicBIST controllers that share IP resources with EDT controllers to reduce hardware overhead. The process for performing this task is known as the hybrid TK/LBIST flow. In this flow, Tessent Shell automatically adds hybrid logic to the EDT controllers for IP resource sharing. The shared LogicBIST and EDT IP is often referred to as hybrid TK/LBIST IPs, or simply hybrid IPs.

This discussion builds on the Tessent Shell RTL and scan DFT insertion flow as described in “[Tessent Shell Flow for Flat Designs](#)” on page 176. Refer to [Hybrid TK/LBIST Flow User’s Manual](#) for details about the hybrid TK/LBIST flow and inserted architecture.

## Tip

 This flow increments the basic Tessent Shell RTL and scan DFT insertion flow. To aid comprehension, ensure that you have reviewed the test case for flat designs.

Refer to the following test case for a detailed usage example of the flow described in this section. This test case illustrates hybrid IP insertion into a flat design with MemoryBIST in the first DFT insertion pass and EDT, OCC, and LogicBIST in the second DFT insertion pass.

```
tessent_example_hybrid_tk_lbist_flow_<software_version>.lbist
```

You can access this test case by navigating to the following directory:

```
<software_release_tree>/share/UsageExamples/
```

|                                                                    |            |
|--------------------------------------------------------------------|------------|
| <b>RTL and Scan DFT Insertion Flow With Hybrid TK/LBIST.....</b>   | <b>357</b> |
| <b>Perform the First DFT Insertion Pass: Hybrid TK/LBIST.....</b>  | <b>358</b> |
| <b>Perform the Second DFT Insertion Pass: Hybrid TK/LBIST.....</b> | <b>361</b> |
| <b>Perform Test Point Insertion .....</b>                          | <b>366</b> |
| <b>Perform Scan Chain Insertion (Hybrid Flow) .....</b>            | <b>369</b> |
| <b>Performing ATPG Pattern Generation: Hybrid TK/LBIST.....</b>    | <b>371</b> |
| <b>Performing LogicBIST Fault Simulation .....</b>                 | <b>374</b> |
| <b>Perform LogicBIST Pattern Generation.....</b>                   | <b>376</b> |

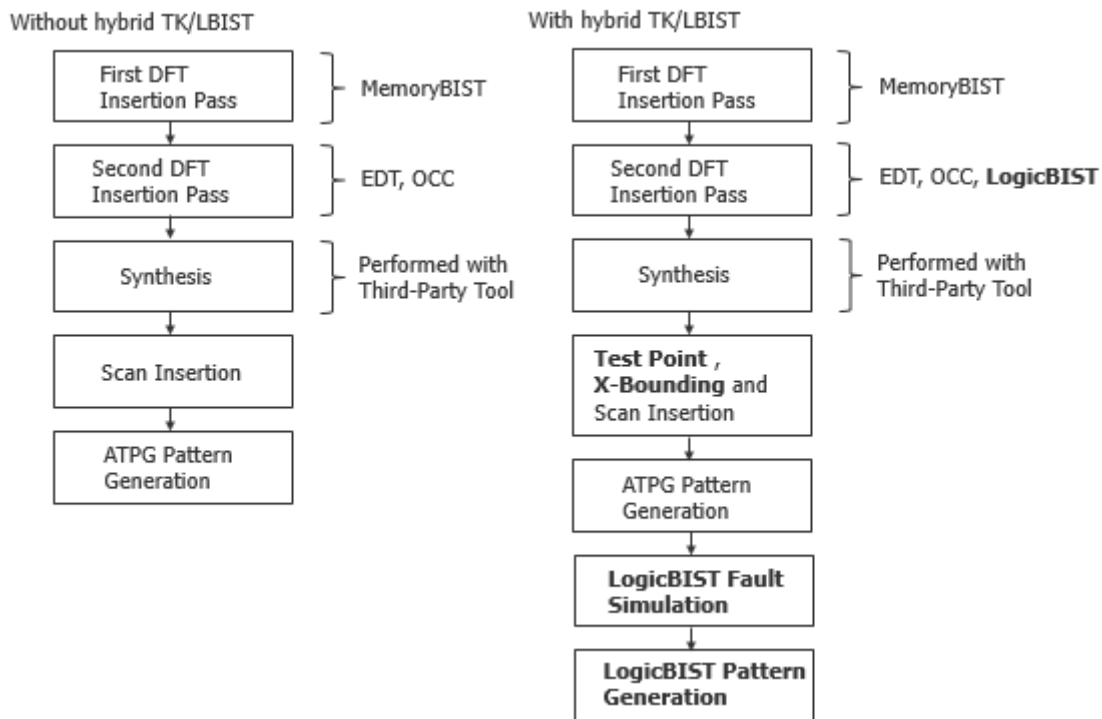
## RTL and Scan DFT Insertion Flow With Hybrid TK/LBIST

Within the two-pass DFT insertion process for a flat design, insert the hybrid TestKompress/LogicBIST IP during the second DFT insertion pass. Adding LogicBIST to your Tessent Shell flow introduces several new steps: X-bounding, test point insertion, and LogicBIST fault simulation and pattern generation. This hybrid solution adds self-test capabilities and is key to satisfying the reliability requirements of the ISO 26262 automotive safety standard.

### Note

The test case that illustrates the hybrid TK/LBIST flow does not include boundary scan insertion. Refer to “[First DFT Insertion Pass: Performing MemoryBIST and Boundary Scan](#)” on page 180 for information about inserting boundary scan in the first DFT insertion pass.

**Figure 6-58. Two-Pass Insertion Flow With Hybrid TK/LBIST**



## Perform the First DFT Insertion Pass: Hybrid TK/LBIST

When performing the hybrid TK/LBIST flow to insert the hybrid IP, perform the first pass of the two-pass DFT and scan insertion process as usual—insert MemoryBIST and boundary scan. In addition, for the hybrid TK/LBIST flow, enable controller chain mode (CCM) test by specifying a dedicated control signal.

CCM enables you to generate ATPG patterns that target the hybrid-IP logic so that you can test the test logic. Refer to “[Controller Chain Mode](#)” in the *Hybrid TK/LBIST User’s Manual* for details.

Refer to “[First DFT Insertion Pass: Performing MemoryBIST and Boundary Scan](#)” on page 180 for additional information.

### Note

 The line numbers used in this procedure refer to the command flow dofile in [Example 6-13](#) on page 359.

---

## Procedure

1. Load the RTL design data and set the design parameters. (See lines 1-11.)

### Note

 “rtl1” is the recommended naming convention for the design ID for the first insertion pass, but you can specify any name. Refer to “[Loading the Design](#)” for more information about setting the design ID. The default design ID for the current test case is “rtl.”

---

2. Add a DFT signal to use for controller chain mode. (See lines 13-16.) You must register the DFT signal name and then add the signal.

By default, Tessent Shell adds a pin at the current design level to control CCM. You can save this pin by using a TDR register to control CCM; do this by specifying the `add_dft_signal -create_with_tdr` switch.

Whether or not you are inserting MemoryBIST or boundary scan, you must add the CCM DFT signal in the first DFT insertion pass because its generated hardware is used in the second pass when you insert the hybrid IP.

3. Set the `set_dft_specification_requirements` command to “on” for `-memory_bist`. (See lines 18-19.)
4. Define the design clocks. (See lines 21-23.)
5. Check the design rules and set the system mode to analysis. (See lines 25-26.)
6. Create the DFT specification. (See lines 28-31.)
7. Generate the DFT hardware, JTAG network connectivity, and test patterns. (See lines 33-43.)
8. Run simulations to verify the design. (See lines 45-50.)

## Examples

### Example 1

The following dofile example shows a typical command flow as detailed in the preceding procedure. The highlighted command lines are unique to the process for inserting hybrid IP in a two-pass DFT insertion process.

### Example 6-13. Dofile Example for First Pass, Hybrid TK/LBIST With MemoryBIST

```
1 # Load the design
```

```
2  set_context dft -rtl
3  set_tsdb_output_directory ../tsdb_outdir
4  read_verilog ../rtl/piccpu_rtl.v
5  read_cell_library ../lib/tessent/adk.tcelllib \
6    ../lib/tessent/picdram.atpglib
7  set_design_source -format tcd_memory -y ../rtl -extension memlib
8
9  set_current_design piccpu
10
11 set_design_level physical_block
12
13 # Add DFT signal for controller chain mode test
14 add_dft_signal_controller_chain_mode -create_with_tdr
15
16 # Specify the DFT requirements
17 set_dft_specification_requirements -memory_test on
18
19 # Define memory clock
20 add_clocks 0 ramclk -period 100ns
21 add_clocks 0 clk -period 100ns
22
23 check_design_rules
24 set_system_mode analysis
25
26 # Create and report the DFT specification
27 # This test case reads in the DFT specification from a separate file
28 read_config_data mbist_ip.dftspec
29 report_config_data
30
31 # Generate and insert the hardware
32 process_dft_specification
33
34 extract_icl
35
36 # Generate MemoryBIST and IJTAG network verification patterns
37 create_patterns_specification
38
39 report_config_data
40
41 process_patterns_specification
42
43 # Run and check testbench simulations
44 set_simulation_library_sources -v {../lib/verilog/adk.v} \
45   -y ../lib/verilog -extension v
46
47 run_testbench_simulations
48 check_testbench_simulations -report_status
49
50 exit
```

### Example 2

Typically, you insert MemoryBIST or boundary scan in the first DFT insertion pass before inserting the hybrid IP (and OCC) in the second DFT insertion pass. The following sample dofile shows a first DFT insertion pass without MemoryBIST or boundary scan, in which you are adding the DFT signal for controller chain mode test.

### Example 6-14. Dofile Example for First DFT Pass, Hybrid TK/LBIST Only

```
# Load the design
set_context dft -rtl -design_id rtl1
set_tsdb_output_directory ../tsdb_outdir
set_design_sources -format tcd_memory -y ../rtl -extension memlib
read_verilog ../rtl/piccpu_rtl.v -verbose
read_cell_library ../library/tessent/adk.tcelllib \
    ../rtl/picdram.atpglib
set_current_design piccpu
set_design_level physical_block

# Specify the clocks
add_clocks 0 clk -period 100ns
add_clocks 0 ramclk -period 100ns

# Add DFT signal for controller chain mode test
add_dft_signal controller_chain_mode -create_with_tdr

check_design_rules

# Create DFT specification
set_spec [create_dft_specification]
set_config_value -in $spec user_rtl_cells on

process_dft_specification

extract_icl

# Generate patterns
create_patterns_specification

# Create patterns specification and create simulation testbenches
process_patterns_specification

# Run and check testbench simulations
set_simulation_library_sources -v ../library/verilog/adk.v \
    -v ../library/memory/ram.v -y ../library/verilog -extensionv
run_testbench_simulations -simulator_option +nowarnTSCALEexit
```

## Perform the Second DFT Insertion Pass: Hybrid TK/LBIST

In the second DFT insertion pass with the hybrid TK/LBIST flow, insert the EDT, OCC, and LogicBIST instruments.

#### Note

 The line numbers used in this procedure refer to the command flow dofile in “[Example 1: Dofile Flow for the Second DFT Insertion Pass: Hybrid TK/LBIST](#)” on page 363 unless otherwise noted.

## Procedure

1. Load the design and set the environment (see lines 1-6). Refer to “[Loading the Design](#)” on page 185 for more information.
2. Define the DFT signals (see lines 8-28). Refer to “[Specifying and Verifying the DFT Requirements](#)” on page 187 for more information.

### Note

 Hybrid TK/LBIST has additional DFT requirements for the clock signals.

---

3. Create the LogicBIST test point and X-bounding signals with a TDR (see lines 23-24).
4. Add a core instance for the MemoryBIST mini-OCC for LogicBIST test. (See lines 30-31.)

```
add_core_instances -instances [get_instances  
*_tessent_sib_sti_inst]
```

This is required when you have inserted MemoryBIST in the first DFT insertion pass.

5. [Creating the DFT Specification](#). (See lines 38-50.) Include a SIB for LogicBIST by specifying lbist in the create\_dft\_specification command.

```
create_dft_specification -sri_sib_list {edt occ lbist}
```

Customize the generated DFT specification as follows. See “[Example 2: DFT Specification Example for Second DFT Insertion Pass with Hybrid TK/LBIST](#)” on page 364.

- a. For OCC, set the static clock control to external and the capture trigger to capture\_en.

When static\_clock\_control is set to external, the OCC has an N-bit input (where N equals the number of shift register bits), which is driven by the NcpIndexDecoder. This is what gives the OCC programmability for LogicBIST.

When capture\_trigger is set to capture\_en, the OCC is capable of handling a free-running slow clock. In LogicBIST applications, the slow clock is a free-running clock (whereas for ATPG this comes from a top-level tester-controlled clock).

For details about OCC for the hybrid TK/LBIST flow, refer to “[Tessent OCC TK/LBIST Flow](#)” in the *Hybrid TK/LBIST Flow User’s Manual*.

- b. Specify a [LogicBist](#) wrapper that contains both a Controller wrapper and an NcpIndexDecoder wrapper.

This wrapper causes Tessent Shell to automatically convert the EDT controller into a hybrid controller for the hybrid TK/LBIST flow. In the Controller/Connections wrapper, you must specify the controller\_chain\_enable property if you are using a TDR to control CCM. Specify the full path to the pin or port name.

The NcpIndexDecoder wrapper specifies the named capture procedures (NCPs) for LogicBIST test. For details, refer to “[NCP Index Decoder](#)” in the *Hybrid TK/LBIST Flow User’s Manual*.

- c. In the EDT/Controller wrapper, define the [LogicBistOptions](#) if you are not using the default values.

When you specify the LogicBIST wrapper, the tool adds a LogicBistOptions wrapper to the EDT controller automatically populated with default values.

The misr\_input\_ratio property provides a way to specify the MISR size. The automatic (default) ratio results in the lowest hardware with a small MISR.

You can control how many chains are masked by a single mask register bit using the chain\_mask\_register\_ratio property. By default, the ratio is 1:1; there are as many bits in the chain mask register as there are number of scan chains.

Specify the low-power shift options specifically for hybrid IP usage separately from the low-power shift options for the EDT controller.

6. Process the DFT specification to generate the test hardware, including LogicBIST (see line 52). Refer to “[Generating the EDT, Hybrid TK/LBIST, and OCC Hardware](#)” on page 194 for more information.
7. Extract the ICL information (see line 54). Refer to “[Extracting the ICL Module Description](#)” on page 194 for more information.
8. Generate the patterns and simulate the testbench (see lines 56-63). Refer to “[Generating ICL Patterns and Running Simulation](#)” on page 195 for more information.

## Examples

### Example 1: Dofile Flow for the Second DFT Insertion Pass: Hybrid TK/LBIST

The following dofile example shows a typical command flow. The highlighted command lines are unique to the process for inserting Tessent Shell LogicBIST and hybrid IP instruments in a two-pass DFT insertion process. In this example, the dofile reads in the DFT specification that defines the instruments to be inserted.

```

1  set_context dft -rtl -design_id rtl2
2  set_tsdb_output_directory ../tsdb_outdir
3  read_design piccpu -design_id rtl1
4  read_cell_library ../lib/tessent/adk.tcelllib \
5  ../lib/tessent/picdram.atpglib
6  set_current_design piccpu
7
8  # Add the DFT signals
9  # For scan test MemoryBIST-related logic
10 # Define tck_occ_en to access mini-OCC during LogicBIST
11 add_dft_signal ltest_en memory_bypass_en tck_occ_en
12
13 # Scan test shift/capture and edt clocks are driven from the
14 # test clock (saves a port)
15 add_dft_signal scan_en edt_update -source_node {scan_en edt_update }
```

```

16 add_dft_signal test_clock -source_node test_clock
17 add_dft_signal edt_clock shift_capture_clock -create_from_other_signals
18 # Alternatively, they can be directly driven from a primary port as
19 # follows
20 # add_dft_signal edt_clock shift_capture_clock -source_node {edt_clock
21 # shift_capture_clock}
22
23 # Required DFT signals for hybrid TK/LBIST
24 add_dft_signals control_test_point_en observe_test_point_en X_bounding_en
25
26 add_dft_signals int_ltest_en ext_ltest_en
27
28 set_dft_specification_requirements -logic_test on
29
30 # Add the MemoryBIST mini-OCC for LogicBIST test
31 add_core_instances -instances [get_instances *_tessent_sib_sti_inst]
32
33 # add_clock clk -period 100ns
34 # Need to define it because of ICL clock port tracing
35
36 check_design_rules
37
38 # Create DFT specification
39 # Populated later in this dofile
40 set_spec [create_dft_specification -sri_sib_list {occ edt lbist } ]
41
42 # You can set this property if there are no library cells
43 # set_config_value -in $spec use_rtl_cells on
44
45 report_config_data $spec
46
47 # Read in the DFT specification data
48 # This test case reads in the DFT specification from a separate file
49 # This file is illustrated in the next example
50 read_config_data logic_instruments.dftspec -in_wrapper $spec -replace
51
52 process_dft_specification
53
54 extract_icl
55
56 create_patterns_specification
57 process_patterns_specification
58
59 set_simulation_library_sources -v {../lib/verilog/adk.v} \
60   -y ../lib/verilog -extension v
61
62 run_testbench_simulations
63 check_testbench_simulations -report_status
64
65 exit

```

**Example 2: DFT Specification Example for Second DFT Insertion Pass with Hybrid TK/LBIST**

The following example illustrates a DFT specification customized to include the wrappers for the LogicBIST controller and additional LogicBIST options for the EDT controller.

```

1 Occ {

```



```

2  ijtag_host_interface: Sib(occ);
3      capture_trigger: capture_en;
4      static_clock_control: both;
5      Controller(clk) {
6          clock_intercept_node: clk;
7      }
8      Controller(ramclk) {
9          clock_intercept_node: ramclk;
10     }
11 }
12
13 # Include the LogicBIST controller
14 # Example LogicBIST only; modify for your design requirements
15 LogicBist {
16     ijtag_host_interface : Sib(lbist);
17     Controller(c0) {
18         burn_in : on ;
19         pre_post_shift_dead_cycles : 8 ;
20         SingleChainForDiagnosis { Present : on ; }
21         ShiftCycles {max: 40; hardware_default : 1024;}
22         CaptureCycles {max: 4;}
23 # Hardware default is max
24     PatternCount {max: 10000; hardware_default : 1024;}
25 # Hardware default is 0
26     WarmupPatternCount { max : 512;}
27     ControllerChain {
28         present : on;
29         clock : tck;
30     }
31     Connections {
32         shift_clock_src: clk;
33         controller_chain_enable : piccpu_rtl_tessent_tdr_sri_ctrl_inst/ \
34             controller_chain_mode;
35     }
36 }
37 NcpIndexDecoder{
38     Ncp(clk_occ_ncp) {
39         cycle(0): OCC(clk);
40         cycle(1): OCC(clk);
41     }
42     Ncp(ramclk_occ_ncp) {
43         cycle(0): OCC(ramclk);
44         cycle(1): OCC(ramclk);
45     }
46 # References to piccpu_rtl_tessent_sib_1 only applicable when
47 # inserting MemoryBIST
48     Ncp(ALL_occ_ncp) {
49         cycle(0): OCC(clk) , OCC(ramclk) , piccpu_rtl_tessent_sib_1;
50     }
51     Ncp(sti_occ_ncp) {
52         cycle(0): piccpu_rtl_tessent_sib_1 ;
53         cycle(1): piccpu_rtl_tessent_sib_1 ;
54     }
55 }
56 }
57
58 # Example EDT only; modify for your design requirements
59 # Note the additional LogicBIST options for the hybrid TK/LBIST flow

```

```
60  EDT {
61    ijttag_host_interface : Sib(edt);
62    Controller (c0) {
63      longest_chain_range : 20, 60;
64      scan_chain_count : 10;
65      input_channel_count : 1;
66      output_channel_count : 1;
67      ShiftPowerOptions {
68        present : on ;
69        min_switching_threshold_percentage : 15 ;
70      }
71      LogicBistOptions {
72        misr_input_ratio : 1 ;
73        chain_mask_register_ratio : 1 ;
74        ShiftPowerOptions {
75          present : on ;
76          default_operation : disabled ;
77          SwitchingThresholdPercentage { min : 25 ; }
78        }
79      }
80    }
81 }
```

### Example 3: DFT Signals Required for the Hybrid TK/LBIST Flow

The following dofile snippet shows the DFT signals required if you are performing the hybrid TK/LBIST flow only (without MemoryBIST or boundary scan in the first DFT insertion pass).

```
# For logic test
add_dft_signals ltest_en -create_with_tdr
add_dft_signals scan_en edt_update -source_node \
{ scan_en edt_update }
# You can create edt_clock and shift_clock from a test clock
add_dft_signals test_clock -source_node { scan_test_clock }
add_dft_signals edt_clock shift_capture_clock -create_from_other_signals

add_dft_signals int_ltest_en ext_ltest_en

# Required for TK/LBIST IP
add_dft_signals observe_test_point_en -create_with_tdr
add_dft_signals control_test_point_en -create_with_tdr
add_dft_signals x_bounding_en -create_with_tdr
```

## Related Topics

[Sib](#)

## Perform Test Point Insertion

The two-pass DFT insertion flow with hybrid TK/LBIST includes a step for test point insertion prior to performing scan chain insertion. Inserting test points increases the testability of a design by improving controllability and observability during scan testing.

The tool inserts observation points that enable the tool to observe test responses during scan cycles and control points that activate and receive pseudo-random values during built-in self test. For details, refer to “[Test Points for LBIST Test Coverage Improvement](#)” in the *Tessent Scan and ATPG User’s Manual*.

Before inserting the test points, specify a DFT signal that enables X-bounding signals. X-bounding is the process of preventing signals originating from X-generating nets (from non-scan cells, black boxes, and primary inputs) from reaching the scan cells. The tool inserts a MUX with an X-bounding enable signal that is controlled by a top-level pin. X-bounding ensures that only valid binary values propagate through the scan cells during test. For details, refer to “[X-Bounding](#)” in the *Hybrid TK/LBIST Flow User’s Manual*.

## Prerequisites

- Prior to test point insertion, perform synthesis as described in section “[Performing Synthesis](#)” on page 196.”

## Procedure

1. Specify the following command to set the DFT context for test point insertion:

```
set_context dft -test_point -no_rtl -design_id gate1
```

For test point insertion, you must use a gate-level netlist. Ensure that you specify a design ID with a unique name from the design ID for scan chain insertion that you perform after test point insertion.

2. Load the cell libraries:

```
read_cell_library ../lib/tessent/adk.tcelllib ../rtl/  
picdram.atpglib
```

3. Specify the same output directory that you used in the first and second DFT passes:

```
set_tsdb_output_directory ../tsdb_outdir
```

4. Load the synthesized netlist. For example:

```
read_verilog ../03.synthesis/piccpu.vg
```

5. Load the design data for the DFT hardware you previously inserted. For example:

```
read_design piccpu -design_id rtl2 -no_hdl -verbose
```

6. Set the maximum number of test points you want to add. For example:

```
set_test_point_analysis_options -total_number 50
```

Typically, the maximum number of test points should be 1%-2% of the number of flops in the design.

7. Set the test point-related insertion options:

```
set_test_point_insertion_options -observe_point_share 5
```

The control\_test\_point\_en and observe\_test\_point\_en enable signals are required for test point insertion, and they are automatically inferred from the control\_test\_point\_en and observe\_test\_point\_en DFT signals you previously added.

8. Specify the type of test points you want to insert into the design:

```
set_test_point_types { lbist_test_coverage edt_pattern_count }
```

Specify both LogicBIST test coverage and EDT pattern count test point types so that the tool generates test points to reduce pattern count and improve random pattern testability.

9. Specify to insert test logic on the set and reset signals to make them controllable when you insert scan chains:

```
set_test_logic -set on -reset on
```

This command increases test coverage.

10. Prohibit test point insertion for Tessent-inserted scan-resource instruments (SRIs)—EDT, OCC, and LogicBIST:

```
add_notest_point [ get_instance *tessent* ]
```

You can only insert test points into scan-tested instruments (STIs).

11. Analyze and optimize the test points:

```
analyze_test_points
```

Tessent Shell returns a log file that lists the test points that are inserted into the tool. You can write out a test point dofile that lists the test point locations. You can use this dofile to edit the test points you want to insert.

```
write_test_point_dofile tpi.dofile -all -replace
```

12. Insert the test point and scan logic:

```
insert_test_logic
```

To generate a script to use with third-party test point insertion tools, use the following command to target the insertion script for Design Compiler.

```
insert_test_logic -write_in_tsdb off -write_insertion_script \  
testpoint_dc.tcl -insertion dc -replace
```

## Examples

### Example 6-15. Dofile Example for Test Point Insertion

```
set_context dft -test_point -no_rtl -design_id gate
read_cell_library ../lib/tessent/adk.tcelllib ../rtl/picdram.atpglib
set_tsdb_output_directory ../tsdb_outdir
read_verilog ../03.synthesis/piccpu.vg
read_design piccpu -design_id rtl2 -no_hdl -verbose

set_current_design piccpu
add_input_constraint reset -C0

set_test_point_analysis_options -total_number 50

# Following command inferred when X-bounding enable DFT signal was added
# -xbounding_enable [ get_dft_signal x_bounding_en ]
set_test_point_insertion_options -observe_point_share 5
set_test_point_types { lbist_test_coverage edt_pattern_count }
set_test_logic -set on -reset on
# no test points in Tessent-inserted IPs
add_notest_point [ get_instance *tessent* ]

set_system_mode analysis

analyze_test_points

insert_test_logic
report_test_logic

exit
```

## Perform Scan Chain Insertion (Hybrid Flow)

Scan chain insertion for the hybrid TK/LBIST flow includes additional commands for connecting the scan enable signal, performing DRC, specifying non-scannable design objects, and X-bounding.

### Note



The line numbers used in this procedure refer to the command flow dofile in [Example 6-16](#) on page 370.

## Procedure

1. Load the design data and set the design parameters. (See lines 1-6.) Ensure that you specify a unique design ID name from the name you used for test point insertion.
2. Set DRC handling to issue warnings for E9 and E11 DRC checks. (See lines 8-10.)

```
set_drc_handling E09 W
set_drc_handling E11 W
```

These DRCs check for possible bus contention issues. By default, Tessent Shell ignores these rules. Set these checks to issue warnings so that the X-bounding algorithm checks them and fixes any X-source contention issues.

3. Specify the pins/ports to exclude from X-bounding. (See lines 12-13.) For example:

```
set_xbounding_options -exclude { reset }
```

Exclude the pins/ports that are guaranteed to have a known value during fault simulation and never propagate an X value to the MISR. The tool does not insert X-bounding muxes at the specified pins/ports, or at the logic that is driven by these pins.

4. Run X-source analysis with the `analyze_xbounding` command to identify memory elements that might capture an unknown during LogicBIST. (See lines 17-19.)
5. Perform wrapper analysis and insertion. (See lines 21-34.)
6. Connect the scan chains to the EDT block, analyze the scan chains, and insert the scan chains. (See lines 36-50.)

## Examples

The following dofile example shows a typical command flow. The highlighted command lines are unique to the process for inserting Tessent Shell LogicBIST and hybrid IP instruments in a two-pass DFT insertion process.

### Example 6-16. Dofile Example for Scan Chain Insertion: Hybrid TK/LBIST

```
1 set_context dft -scan -design_id gate2
2 read_cell_library ../lib/tessent/adk.tcelllib \
3   ../lib/tessent/picdram.atpglib
4 set_tsdb_output_directory ../tsdb_outdir
5 # Reading in test point-inserted design
6 read_design piccpu -design_identifier gate1 -verbose
7
8 # Enable DRCs for X-bounding
9 set_drc_handling E09 w
10 set_drc_handling E11 w
11
12 # Set x-bounding options
13 set_xbounding_options -exclude {reset}
14
15 set_system_mode analysis
16
17 # Run X-source analysis
18 analyze_xbounding
19 report_xbounding -verbose -ignored_x_sources on
20
21 ## Perform wrapper analysis and insertion
22 # Exclude the edt_channel in and out ports from wrapper analysis
23 # The ijttag * edt_update ports are automatically excluded
24 set_wrapper_analysis_options -exclude_ports \
25   [ get_ports {*_edt_channels_*} ]
26
27 # Add a new wrapper dedicated cell on reset
```

```
28 set_dedicated_wrapper_cell_options on -ports { reset }
29 set_wrapper_analysis_options -input_fanout_flop_threshold 100 \
30   -output_fanin_flop_threshold 40
31
32 # Perform wrapper cell analysis
33 analyze_wrapper_cells
34 report_wrapper_cells -Verbose
35
36 # Find the edt_instance
37 # Instance of module *edt_lbist_c0
38 set edt_instance [get_instances -of_icl_instances [get_icl_instances \
39   -filter tessent_instrument_type==mentor::edt]]
40
41 # Connect scan chains to the EDT signals
42 add_scan_mode int_mode -type internal -edt_instances $edt_instance
43 add_scan_mode ext_mode -type external -chain_count 2
44 analyze_scan_chains
45
46 analyze_scan_chains
47 report_scan_chains
48
49 # Insert scan chains and write the scan-inserted design into the TSDB
50 insert_test_logic
51
52 report_scan_chains
53 report_scan_cells
54
55 exit
```

## Related Topics

[Performing Scan Chain Insertion: Wrapped Core](#)

# Performing ATPG Pattern Generation: Hybrid TK/LBIST

ATPG pattern generation for the hybrid TK/LBIST flow includes a process for generating controller chain mode patterns to test the TK/LBIST logic itself.

For details, refer to “[Performing Pattern Generation for CCM in the TSDB Flow](#)” in the *Hybrid TK/LBIST Flow User’s Manual*.

---

### Note



The line numbers used in this procedure refer to the command flow dofile in [Example 6-17](#) on page 372.

---

## Prerequisites

- Perform EDT pattern generation as described in “[Performing ATPG Pattern Generation](#)” on page 199.

## Procedure

1. Load the design and set the environment (see lines 1-12). Refer to “[Loading the Design](#)” on page 185 for more information.
2. Enable CCM (see lines 16-17).

```
set_static_dft_signal_values controller_chain_mode 1
```

You had added a DFT signal for `ccm_en` during IP creation, which is now being configured. If this is a port (default), then this command is not required.

3. Define the scan chain for CCM (see lines 19-22).
4. Specify `tck` or `edt_clock` for CCM, depending on which you are using in your implementation (see lines 24-25).
5. Define the pin constraints (see lines 33-37).

You must disable core clock and reset activity. If `ccm_en` was not added as a DFT signal, then also include a pin constraint for it (enabled).

6. For retargeting, specify to pulse the CCM clock during shift. (See line 39.) The following command is only required when you use `edt_clock` for CCM and `edt_clock` is a top-level port, or when you use `tck` for CCM.

```
set_procedure_retargeting_options -pulse_during_shift edt_clock
```

The tool automatically generates a test procedure file that configures the `scan_en` and `shift_capture_clock` DFT signals. If the `edt_clock` is derived from `test_clock`, do not specify the `set_procedure_retargeting_options` command.

7. Specify the `set_current_mode` command with a unique name to indicate that you are generating CCM patterns (see line 41). For example:

```
set_current_mode ccm_stuck
```

8. Create and write the patterns (see lines 43-60).

## Examples

The following example generates CCM ATPG patterns. The highlighted statements illustrate additional considerations for CCM.

### Example 6-17. Dofile Example for ATPG with Controller Chain Mode: Hybrid TK/LBIST

```
1 ### DESIGN VARIABLES
2 set DesignName    piccpu
3 set DesignLevel   physical_block
4
5 set_context patterns -scan
6 set_tsdb_output_directory ../tsdb_outdir
7 read_cell_library ../lib/tessent/adk.tcelllib \
```



```

8    ../lib/tessent/picdram.atpglib
9    set_design_source -format tcd_memory -y ../lib/tessent -extension memlib
10
11   # Read in the scan-inserted design
12   read_design $DesignName -design_id gate2
13
14   report_dft_signals
15
16   # Enable CCM
17   set_static_dft_signal_values controller_chain_mode 1
18
19   add_scan_groups grp1
20   # These are the required scan chains for CCM
21   add_scan_chains ccm_chain grp1 control_chain_scan_in \
22     control_chain_scan_out
23
24   # Add edt_clock or tck as the primary clock source for CCM
25   add_clock 0 tck
26
27   # Specify clocks driven by primary-input ports; this is optional
28   add_clocks 0 clk
29   add_clocks 0 ramclk
30   add_clocks 0 reset
31   add_clocks 0 test_clock
32
33   # Define the pin constraints
34   add_input_constraints clk -c0
35   add_input_constraints ramclk -c0
36   add_input_constraints reset -c0
37   add_input_constraints test_clock -c0
38
39   set_procedure_retargeting_options -pulse_during_shift edt_clock
40
41   set_current_mode ccm_stuck
42
43   set_system_mode analysis
44   add_fault -module [ get_module *tessent* ]
45   report_clocks
46
47   create_patterns
48   report_statistics -detailed_analysis
49
50   report_statistics -instance piccpu_rtl2_tessent_lbist
51   report_statistics -instance \
52     piccpu_rtl2_tessent_lbist_ncp_index_decoder_inst
53   report_statistics -instance piccpu_rtl2_tessent_single_chain_mode_logic
54   report_statistics -instance piccpu_rtl2_tessent_edt_lbist_c0_inst
55
56   write_tsdb_data -replace
57   write_patterns ${DesignName}_ccm_stuck_parallel.v -verilog -parallel \
58     -replace
59   write_patterns ${DesignName}_ccm_stuck_serial.v -verilog -serial \
60     -replace
61
62   exit

```

## Performing LogicBIST Fault Simulation

Perform LogicBIST fault simulation on the scan-inserted, gate-level design. LogicBIST fault simulation generates pseudo-random, parallel pattern sets.

Refer to “[Parallel Versus Serial Patterns](#)” in the *Tessent Scan and ATPG User’s Manual* for more information.

---

### Note



The line numbers used in this procedure refer to the command flow dofile in [Example 6-18](#) on page 375.

---

### Procedure

1. Load the design and set the environment (see lines 1-12). Refer to “[Loading the Design](#)” on page 185 for more information.
2. Set the current test mode to a unique name for the new pattern set you are creating to test the hybrid IP. (See line 7.)
3. Import the core’s internal mode data. (See line 9.)

Tessent Shell imports the scan-inserted design data for the EDT and OCC logic. For LogicBIST fault simulation, the tool requires EDT for PRPG/compactor/MISR configuration and chain tracing, and OCC for the clocks. Using the [import\\_scan\\_mode](#) command assumes that you used Tessent Scan to perform scan chain stitching.

For LogicBIST fault simulation, only the internal mode data is valid.

4. Add the hybrid TK/LBIST core instances. (See lines 11-12.) For example:

```
add_core_instances -instance piccpu_rtl2_tessent_lbist
```

For the hybrid TK/LBIST flow, you must explicitly add the LogicBIST controller core.

5. Specify the capture procedure names with the [set\\_lbist\\_controller\\_options](#) command. (See lines 16-18.)

Include the names of all Named Capture Procedures (NCPs) that are defined in the DftSpecification, along with their active percentages, regardless of whether you intend to use them in LogicBIST simulation.

The tool defines the NCPs in the Tessent Core Description (TCD) file while processing the DftSpecification. You can override the automatically generated NCPs with the [read\\_procfile](#) command.

6. Define the DFT signals (see lines 23-37). In addition to enabling the `ltest_en` and `int_ltest_en` logic test control signals, you must specify the following signals for the hybrid TK/LBIST flow:

```
set_static_dft_signal_values control_test_point_en 1
set_static_dft_signal_values observe_test_point_en 1
set_static_dft_signal_values xbounding_en 1
set_static_dft_signal_values tck_occ_en 1
set_static_dft_signal_values controller_chain_mode 0
```

7. Set the PLL external capture clocks if your design has a PLL that is a driving clock, but the PLL itself is driven by external clocks. (See lines 43-46.)

This indicates the number of cycles the NCPs take with respect to the LogicBIST controller clock, as specified with the `-fixed_cycles` switch.

8. Specify the maximum number of pseudo-random patterns you want the tool to simulate. (See line 48.)
9. Set the pattern source to LogicBIST and run fault simulation. (See line 51.)
10. Write out the parallel testbench and save all the data into the TSDB. (See lines 53-57.)

## Examples

The following dofile example shows a typical command flow as detailed in the preceding procedure. The highlighted text illustrates additional considerations for the hybrid TK/LBIST flow.

### Example 6-18. Dofile Example for LogicBIST Fault Simulation

```
1 set_context pattern -scan -design_id gate2
2 set_tsdb_output_directory ../tsdb_outdir
3 read_cell_library ../lib/tessent/adk.tcelllib ../rtl/picdram.atpglib
4 read_design piccpu
5 set_current_design piccpu
6
7 set_current_mode lbist
8
9 import_scan_mode int_mode
10
11 # Explicitly add the LogicBIST controller core
12 add_core_instances -instance piccpu rtl2_tessent_lbist
13 # EDT and OCC are loaded with the int_mode imported scan mode
14
15
16 # Define the number of patterns per NCP capture window
17 set_lbist_controller_options -capture_procedure {clk_occ_ncp 70 \
18     ramclk_occ_ncp 10 ALL_occ_ncp 10 sti_occ_ncp 10}
19
20 add_input_constraint test_clock -c0
21 add_nofault [get_instance *tessent*]
22
23 # Set the DFT signals
24 set_static_dft_signal_values ltest_en 1
```

```
25 # The following DFT signal is inferred by ltest_en 1 if you have memory
26 # set_static_dft_signal_value memory_bypass_en 1
27
28 set_static_dft_signal_values control_test_point_en 1
29 set_static_dft_signal_values observe_test_point_en 1
30 set_static_dft_signal_values x_bounding_en 1
31
32 set_static_dft_signal_values int_mode 1
33 # The following DFT signal is inferred by int_mode 1
34 # set_static_dft_signal_values int_ltest_en 1
35
36 set_static_dft_signal_values tck_occ_en 1
37 set_static_dft_signal_values controller_chain_mode 0
38
39 report_static_dft_signal_settings
40
41 set_system_mode analysis
42
43 # Specify the number of cycles the NCPs take with respect to the LogicBIST
44 # controller clock
45 # This ensures that the tool runs the testproc correctly
46 set_external_capture_options -fixed_cycles 4
47
48 set_random_patterns 1000
49 add_faults -all
50
51 simulate_patterns -source bist -store_patterns all
52
53 # Write the parallel pattern Verilog testbench
54 write_patterns piccpu_lbist_parallel.v -verilog -parallel -replace \
55     -parameter_list {SIM_POST_SHIFT 3}
56 # Save the TCD, PatternDB, flat model, and fault list to the TSDB
57 write_tsdb_data -replace
58
59 exit
```

## Perform LogicBIST Pattern Generation

Generate LogicBIST patterns using the scan-inserted and LogicBIST fault simulated design. The process includes IJTAG pattern retargeting from the extracted ICL module description for the design. Tessent Shell translates (or retargets) the PDL commands so that you can control the LogicBIST controller through the TAP controller/IJTAG infrastructure.

---

### Note



Do not confuse IJTAG retargeting with ATPG (or scan) pattern retargeting as described in “[Hierarchical DFT Terminology](#)” on page 205.

---

## Procedure

1. Load the design and set the environment (see lines 1-12). Refer to “[Loading the Design](#)” on page 185 for more information.

2. Set the context to patterns -ijtag and set the design ID to the name of the scan-inserted, gate-level netlist generated during scan chain insertion.

When you specify the -ijtag switch, Tessent Shell automatically accesses the ICL module description for the current design, which enables IJTAG retargeting mode.

3. Define clocks and constraints (see lines 7-10).
4. Generate and validate the IJTAG patterns for the design (see lines 14-16).
5. Run and check the testbench simulations (see lines 18-23). The following step is important for the hybrid TK/LBIST flow:
  - a. Specify the simulator option to keep all the logic gates for simulation.

The following example applies to Questa® SIM:

```
run_testbench_simulations -simulator_options \  
    { -voptargs="+acc" }
```

For correct pattern simulation, use the applicable simulator option to ensure that the necessary design logic is not optimized away during elaboration.

Tessent Shell simulates the logic test operations only, which means the testbench does not connect all the pins in the design. The tool issues warnings for the unconnected pins. To filter these warnings out, you can use the `run_testbench_simulations -simulation_option +nowarnTSCALE` option.

## Examples

The following dofile shows a command flow to generate IJTAG patterns for LogicBIST. Highlighted text illustrates additional considerations for the hybrid TK/LBIST flow.

```
1  set_context patterns -ijtag -design_id gate2
2  read_cell_library ../lib/tessent/adk.tcelllib ../rtl/picdrum.atpglib
3  set_tsdb_output_directory ../tsdb_outdir
4  read_design piccpu
5  set_current_design piccpu
6
7  # This is the clock driving the internal PLL
8  add_clocks 0 clk -period 100ns
9  # Scan enable has to be tied to low
10 add_input_constraint scan_en -c0
11
12 set_system_mode analysis
13
14 read_config_data ../lbist_mbist_pattern.patspec
15
16 process_patterns_specification
17
18 set_simulation_library_sources -v { ../lib/verilog/adk.v \  
19     ../lib/verilog/picdrum.v ../lib/verilog/SYNC_1R1W_256x16.v }
20
21 run_testbench_simulation -simulator_options { -voptargs="+acc" \  
22     -quiet } -wait
```

```
23 check_testbench_simulations -report_status
24
25 exit
```

# Running Multiple Load ATPG on Wrapped Core Memories with Built-In Self Repair

If your design has wrapped cores that include repairable memories, and you want to test the logic around the memories at speed, then you use multiple load ATPG patterns.

|                                                                                                     |            |
|-----------------------------------------------------------------------------------------------------|------------|
| <b>Overview of Multiple Load ATPG on Memories for Wrapped Cores With Built-in Self Repair .....</b> | <b>379</b> |
| <b>Performing Multiple Load ATPG Pattern Generation .....</b>                                       | <b>380</b> |
| <b>Performing Multiple Load Top-Level ATPG Pattern Retargeting .....</b>                            | <b>383</b> |

## Overview of Multiple Load ATPG on Memories for Wrapped Cores With Built-in Self Repair

When generating ATPG multiple load patterns for memories with Built-In Self Repair (BISR), the memories inside the wrapped cores are tested first. If any memories need to be repaired because of defects that are present inside the memory, then the BISR registers are programmed such that the contents of these registers vary based on the repair solution that is stored in the fuses at the parent level.

To generate multiple load ATPG patterns, the BISR registers need to be excluded from the retargetable ATPG patterns that you created for the wrapped core. At the wrapped core level, a test\_setup\_iCall resets the BISR registers, resulting in all spare elements being unallocated before ATPG is run. Only the memory control ports and data ports participate in the ATPG patterns; the memory repair ports are excluded. If memory defects are detected and repaired via the BISR registers, the retargeted ATPG patterns also pass.

You must provide an ATPG model of the eFuse during pattern generation to meet the E14 design rule requirements. If such a model is not available, you can instead provide an input constraint on the fuseValue output pin of the eFuse interface module. To do this, create a pseudo-port associated with the fuseValue output pin of the fuse box interface instance. Then, add a constant 0 input constraint on the “fuseValue” pseudo-port, as shown in the following example:

```
add_primary_inputs chip_rtl_tessent_mbisr_controller_inst/ \
    chip_rtl_tessent_mbisr_generic_fusebox_interface_instance/fuseValue \
    -pseudo_port_name fuseValue
add_input_constraints -c0 fuseValue
```

For more details, refer to the [add\\_primary\\_inputs](#) and [add\\_input\\_constraints](#) commands.

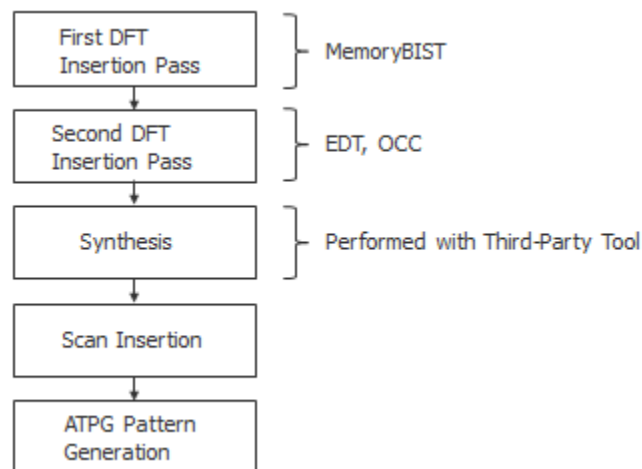
## Performing Multiple Load ATPG Pattern Generation

When you need to generate multiple load ATPG patterns when the memory is not bypassed but used during ATPG there is no difference in the flow steps used for inserting DFT. You use this procedure to bypass the memories.

The DFT insertion at the wrapped core level for memories with BISRs is similar to the two-pass pre-scan DFT insertion process for wrapped cores with the primary difference being the ATPG pattern generation. There are no changes to creating the graybox model, and running the wrapped core in external mode for stuck and transition patterns by bypassing the memories. There are no changes for running the wrapped core in internal mode for stuck and transition patterns by bypassing the memories.

You should be familiar with the two-pass pre-scan DFT insertion flow as described in “[RTL and Scan DFT Insertion Flow for Physical Blocks](#)” on page 210, especially as related to performing ATPG pattern generation. [Figure 6-59](#) shows this flow.

**Figure 6-59. Two-Pass Insertion Flow for RTL, Wrapped Cores**



The `init_bisr_chains` iProc contains Tcl code you can use to initialize the BISR registers at any level (core or chip-level) as follows:

- At the wrapped core level, the iProc performs an asynchronous clear of the BISR registers.
- At the chip-level, the iProc initiates a BISR controller power-up emulation.

The power-up emulation clears the BISR registers if the fuse box has not been programmed, or initializes the BISR registers with the repair data if memory repair information was programmed in the fuse box. See “[Creating and Inserting the BISR Controller](#)” in *Tessent MemoryBIST User's Manual*.



### Note

 The line numbers used in this procedure refer to the command flow dofile in [Example 6-19](#) on page 382.

---

## Prerequisites

- You have performed the “[RTL and Scan DFT Insertion Flow for Physical Blocks](#)” on page 210 up to the ATPG pattern generation step.

## Procedure

1. Load the Tessent Cell Library for the memory. (See lines 1-9.)
2. Load the design. (See lines 11-13.)
3. Set the DFT Signal memory\_bypass to 0, so memory is not bypassed. (See lines 24-26.)
4. Load the PDL for the Memory BISR chains, which has an iProc (init\_bisr\_chains) that is called with [set\\_test\\_setup\\_icall](#) -non\_retargetable. (See lines 28-35.)

The init\_bisr\_chains iProc contains Tcl code you can use to initialize the BISR registers at any level (core or chip-level) as follows:

- At the wrapped core level, the iProc performs an asynchronous clear of the BISR registers.
  - At the chip level, the iProc initiates a BISR controller power-up emulation.
5. Generate the multiple load ATPG patterns. (See lines 39-40.)
  6. Write the design data and patterns to the TSDB. (See lines 42-44.)
  7. Write out the Verilog testbenches for simulation. (See lines 47-50.)

## Results

At the wrapped core level, if you inject a fault in the repairable memory, and then run the multiple load ATPG Pattern, the run should fail. This check shows that if a repairable memory has a defect that is not repaired, then when you retarget the multiple load ATPG patterns from the top level, the multiple load ATPG pattern run fails.

You must run the memoryBIST inside the wrapped core to determine if the memory is repairable. If it is, then the repairable information from Built-In Redundancy Analysis (BIRA) to BISR must be stored and the repair performed by storing the repairable contents using the fusebox repair solution that you use at the top-level.

## Examples

The following example for the repairable memory “MGC\_SYNC\_1RW\_1024x8\_C” generates ATPG patterns that is run by the memory.

### Example 6-19. Multiple Load ATPG Pattern Generation for Memories with BISR

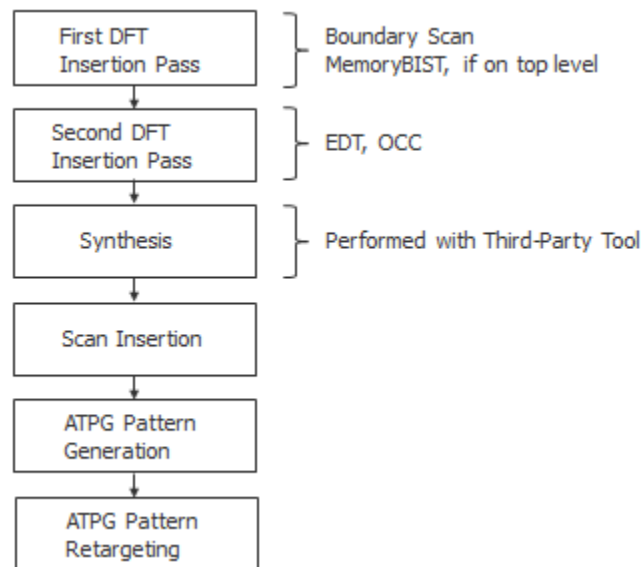
```
1  set_context patterns -scan
2
3  # Set the location of the TSDB. Default is the current working directory.
4  set_tsdb_output_directory ../tsdb_outdir
5
6  # Read Tessent Library
7  read_cell_library ../../../../library/standard_cells/tessent/adk.tcelllib
8  # Read the Tessent Cell Library for the memory
9  read_cell_library ../../../../library/memories/MGC_SYNC_1RW_1024x8_C.atpglib
10
11 # Read in the scan inserted netlist/design
12 read_design processor -design_id gate -verbose
13 set_current_design processor
14
15 # Specify the current mode using a different name than what was used
16 # with the add_scan_mode command
17 set_current_mode edt_int_multiple_load_atpg -type internal
18 report_dft_signals
19
20 # Extract the scan chains using the internal mode specified during
21 # scan insertion with the add_scan_mode command
22 import_scan_mode int_mode -fast_capture_mode on
23
24 # Memory is used during ATPG, by setting the DFTSignal memory_bypass
25 # to "0"
26 set_static_dft_signal_values memory_bypass_en 0
27
28 # Read in the PDL for the memory BISR
29 source \
30 ../tsdb_outdir/instruments/processor_rtl1_mbisr.instrument/\
31 processor_rtl1_tessent_mbisr.pdl
32
33 # Specify the iProc init_bisr_chains for the memory BISR as
34 # non_retargetable
35 # If the block contains a dedicated BISR controller, add -front to the
36 # following command.
37 set_test_setup_icall init_bisr_chains -non_retargetable
38
39 report_statistics -detail
40 # Generate patterns
41 create_patterns
42 report_statistics -detail
43
44 # Store TCD, flat_model, fault list and patDB format files in the TSDB
45 # directory
46 write_tsdb_data -replace
47
48 # Write testbenches for Verilog simulation
49 write_patterns patterns/processor_multiple_load_parallel.v \
50   -verilog -parallel -replace
51 write_patterns patterns/processor_multiple_load_serial.v \
52   -verilog -serial -replace
53 exit
```

## Performing Multiple Load Top-Level ATPG Pattern Retargeting

When you have wrapped cores with repairable memories, then at the chip-top level you need to insert a memory BISR controller. This can be done along with the TAP, boundary scan, and MemoryBIST insertion if there are any memories present at the top-level.

You should be familiar with the RTL and scan DFT insertion flow for the top as described in “[RTL and Scan DFT Insertion Flow for the Top Chip](#)” on page 228, especially as related to performing ATPG pattern retargeting. [Figure 6-60](#) shows this flow.

**Figure 6-60. Two-Pass Insertion Flow for RTL, Top Level**



For retargeting multiple load ATPG patterns where the memories are not bypassed from a lower level wrapped core that has repairable memories, specify the correct retargeting mode signal. Subsequently, use the [add\\_core\\_instances](#) command to load the specific core with the correct ATPG mode that you want to retarget.

Finally, you need to source the PDL of the BISR chain with the [iProc](#) `init_bisr_chains`. This PDL was created when the BISR controller RTL was generated at the chip-level. The `iProc` detects the presence of the BISR controller and initiates a `PowerUpEmulation`. When the `iProc` is called and no BISR controller is found, the `iProc` performs an asynchronous clear of the BISR chains using the primary inputs (`bisr_reset` port).

### Note

 The line numbers used in this procedure refer to the command flow dofile in [Example 6-20](#) on page 384.

## Prerequisites

- You have performed the “[RTL and Scan DFT Insertion Flow for the Top Chip](#)” on page 228 up to the ATPG retargeting step.

## Procedure

1. Set the context to pattern retargeting. (See lines 1-2.)
2. Specify the TSDB directory and open the TSDB. (See lines 4-8.)
3. Read the Tessent Cell Library. (See lines 10-11.)
4. Load the Verilog design. (See lines 13-15.)
5. Set the retargeting from the lower-level patterns. (See lines 20-24.)
6. Read the PDL of the memoryBISR chains that has the iProc “init\_bisr\_chains”. This PDL is different than the one created at the wrapped core level. (See lines 30-34.)
7. Change mode to analysis. (See line 36.)

## Examples

The following example retargets the ATPG pattern from the lower level to the chip-level.

### Example 6-20. ATPG Pattern Retargeting for Memories With BISR

```
1  # Set the context to retarget ATPG Patterns from lower level child cores
2  set_context pattern -scan_retargeting
3
4  # Point to the TSDB directory
5  set_tsdb_output_directory ../tsdb_outdir
6
7  # Open all the TSDB of the child cores
8  open_tsdb ../../wrapped_cores/processor/tsdb_outdir
9
10 # Read the tessent cell library
11 read_cell_library ../../library/standard_cells/tessent/adk.tcelllib
12
13 # Read the verilog
14 read_design chip_top -design_id gate
15 read_design processor -design_id gate -view graybox -verbose
16
17 set_current_design chip_top
18
19
20 # Retarget Transition patterns from processor
21 set_current_mode retarget1_processor_multiple_load
22 set_static_dft_signal_values retargeting1_mode 1
23 add_core_instances -instances {processor_inst1 processor_inst2} \
24   -core processor -mode edt_int_multiple_load_atpg
25
26 report_core_descriptions
27 import_clocks -verbose
28 report_clocks
29
```

```
30 # Read the PDL of the MBISR chains that has the iProc "init_bisr_chains"
31 # that needs to be called before running the ATPG pattern.
32 source ../tsdb_outdir/instruments/chip_top_rtl1_mbisr.instrument/\
33 chip_top_rtl1_tessent_mbisr.pdl
34 set_test_setup_icall init_bisr_chains -front
35
36 set_system_mode analysis
37 report_clocks
38
39 # write the TCD file for chip-level in the TSDB outdir
40 write_tsdb_data -replace
41 # Read the patterns to be retargeted
42 read_patterns -module processor -fault_type transition
43 set_external_capture_options -pll_cycles 5 [lindex [get_timeplate_list] 0]
44
45
46 # Write Verilog patterns for simulation
47 write_patterns patterns/processor_edt_multiple_load_retargeted.v \
48   -verilog -parallel -replace -begin 0 -end 7 -scan \
49 write_patterns patterns/processor_edt_multiple_load_retargeted_serial.v \
50   -verilog -serial -replace -Begin 0 -End 2 \
51 exit
```

# Built-In Self Repair in Hierarchical Tessent MemoryBIST Flow

You use the BISR registers to control the repair ports of repairable memory. The BISR logic insertion tasks include inserting BISR chains in a block and connecting a BISR controller to existing BISR chains, to an external fuse box, and to system logic. This omits the additional BISR capabilities of Repair Sharing and Fast BISR Loading.

You perform the first two tasks in a bottom-up, hierarchical design methodology where BISR chains are inserted in lower-level blocks before being connected at the chip top level to the BISR controller. This is the most frequently used method for inserting the repair logic.

You complete the BISR insertion task with the memoryBIST insertion in the same pass at the block or chip level. If the BISR insertion task is carried out separately from the memory BIST insertion, you must perform BISR insertion after you have completed all memoryBIST insertion tasks at the block or chip level in a single pass.

If you use Tessent Scan, do not scan test the BISR hardware if you plan to generate multiple load patterns for logic test. Tessent Scan does this automatically.

In contrast, if you use a third-party scan insertion tool, then the `non_scannable_instance_list` in the `dft_info_dictionary` should be honored and is located in the TSDB in the `dft_inserted_designs` directory—see “[RTL and DFT Insertion Flow With Third-Party Scan](#)” on page 252.

For additional information, see “[First DFT Insertion Pass: Performing Top-Level MemoryBIST and Boundary Scan](#)” on page 231.

You can also find related information in the *Tessent MemoryBIST User’s Manual* in the topics [Repair Sharing Overview](#) and [Fast BISR Loading Overview](#).

The flow consists of the following steps:

|                                                         |            |
|---------------------------------------------------------|------------|
| <b>Performing Block Level BISR Insertion .....</b>      | <b>386</b> |
| <b>Performing Chip Level BISR Insertion .....</b>       | <b>389</b> |
| <b>Verification of Block- and Chip-Level BISR .....</b> | <b>391</b> |

## Performing Block Level BISR Insertion

The insertion of BISR chains is automatic if memories instantiated in the design have spare resources described in their memory library file. BISR registers associated to repairable memories are connected together to form scan chains.

## Assigning Memories to Power Domains

By default, all repairable memories are part of the same power domain, and the tool creates a single BISR chain. If, however, more than one power domain exists, multiple BISR chains are required.

The tool determines the power domain of a memory instance by its `bisr_power_domain_name` attribute, which you specify in the following two ways:

- Reading a CPF or UPF file corresponding to the design using the [read\\_cpf](#) or [read\\_upf](#) command. This is the recommended method.
- Manually setting the `bisr_power_domain_name` attribute using the [set\\_attribute\\_value](#) command.

The following example defines power domains with UPF:

UPF:

```
create_power_domain pd_A
create_power_domain pd_AA -element {mem3 mem4}
create_power_domain pd_AB -element {mem5 mem6}
```

Generate BISR segment order file:

```
BisrSegmentOrderSpecification {
  PowerDomainGroup(pd_AA) {
    OrderedElements {
      mem3;
      mem4;
    }
  }
  PowerDomainGroup(pd_AB) {
    OrderedElements {
      mem5;
      mem6;
    }
  }
}
```

For additional implementation details, see “[Inserting BISR Chains in a Block](#)” in the *Tessent MemoryBIST User's Manual*.

## Controlling the BISR Chain Order

BISR chains are connected according to the content of the `BisrSegmentOrderSpecification` wrapper containing a list of memory instances defining the BISR chain order.

This file is generated automatically when [check\\_design\\_rules](#) successfully completes. When a DEF file is provided, the BISR segments are ordered using an algorithm that optimizes the routing based on the memory coordinates. If a DEF file is not provided, the memories are sorted alphabetically within each power domain group.

If you want to change the generated BISR change order, you can manually modify the `<design_name>.bistr_segment_order` file and change the order of memory instance names before running the [process\\_dft\\_specification](#) command.

## Turning Off Insertion of GISR Registers

It is possible to disable the generation of BISR registers for specific memory instances.

You do this by issuing the [set\\_memory\\_instance\\_options](#) command as follows:

```
set_memory_instance_options memory_inst -use_in_memory_bistr_dft_specification off
```

## Excluding Child Block BISR Chains

When you do not implement memory repair in a parent design but integrate sub-blocks or physical blocks that already contain BISR chains, you must not connect or use these chains, and you must properly tie the chains off.

---

### Note



This is not a typical use model and is rarely implemented.

---

## Related Topics

[Inserting BISR Chains in a Block](#)

[Excluding Child Block BISR Chains](#)



## Performing Chip Level BISR Insertion

As with the block level, the insertion of BISR chains is automatic if memories instantiated in the design at the chip level have spare resources. When you insert chip level BISR, you must also choose a functional repair clock and connect the BISR controller to an existing BISR chain, an external fuse box, and to system logic.

|                                                                        |            |
|------------------------------------------------------------------------|------------|
| <b>Functional Repair Clock .....</b>                                   | <b>389</b> |
| <b>Connection of the BISR Controller to Existing BISR Chains .....</b> | <b>389</b> |
| <b>Connection of the BISR Controller to an External Fuse Box .....</b> | <b>390</b> |
| <b>Connection of the BISR Controller to System Logic .....</b>         | <b>391</b> |

### Functional Repair Clock

The distributed architecture and conservative clocking methodology used for self-repair require some care in the selection of the functional repair clock used to apply repairs during chip power up. You should use a functional clock of 10 MHz or less in that functional mode.

For implementation details, refer to “[Inserting BISR Chains in a Block](#)” in the *Tessent MemoryBIST User's Manual*.

### Connection of the BISR Controller to Existing BISR Chains

As with block level BISR insertion, BISR chains are connected according to the content of the `BisrSegmentOrderSpecification` wrapper containing a list of memory instances defining the BISR chain order as well as lower level blocks containing pre-inserted BISR chains.

The tool automatically generates this file when the [check\\_design\\_rules](#) command successfully completes.

The following example shows two instances of the same block (blockA, instances blockA\_clka\_i1 and blockA\_clkb\_i2), which has two BISR chains partitioned by two power domains (pd\_AA and pd\_AB):

```
BisrSegmentOrderSpecification {
  PowerDomainGroup(pd_AA) {
    OrderedElements {
      blockA_clka_i1/pd_AA_bisr_si;
      blockA_clkb_i2/pd_AA_bisr_si;
    }
  }
  PowerDomainGroup(pd_AB) {
    OrderedElements {
      blockA_clka_i1/pd_AB_bisr_si;
      blockA_clkb_i2/pd_AB_bisr_si;
    }
  }
}
```

As with the block-level BISR insertion, if you change the generated BISR chain order (`<design_name>.bisr_segment_order`), you can manually modify this file and change the order of memory instance names before running the [process\\_dft\\_specification](#) command.

## Connection of the BISR Controller to an External Fuse Box

The BISR controller hardware is created at the top level of the chip automatically. The BISR controller is accessed using the TAP. The BISR chain control ports are automatically connected to the BISR controller. The BISR controller is also connected to the fuse box. A typical implementation of BISR is with an external fuse box.

The hardware implements several functions and can be used to perform the following operations:

- Compress the repair information and write the result into the fuse box
- Decompress the fuse box contents and shift the contents into the chip BISR chain
- Initialize or observe BISR chain content via the TAP
- Read and program fuses via the TAP

---

### Note



A BISR controller with an *internal* fuse box is supported. For details, refer to “[Implementing and Verifying Memory Repair](#)” in the *Tessent MemoryBIST User's Manual*.

---

The design instance for the external fuse box must already be instantiated in the design. Typically, the fuse box is instantiated within a module that also contains interface logic. All input ports of the module should be tied off, and the output ports should be left open. When

running the [process\\_dft\\_specification](#) command, the tool disconnects all input ports and then reconnects the ports to the BISR controller module.

When using an external fuse box, you must set the `fuse_box_location` property in the MemoryBisr/[Controller](#) wrapper to `external`. The `fuse_box_interface_module` property located in the same wrapper can be used to specify the library module for the external fuse box. If this is not specified, the library module is inferred from the design instance specified in the `design_instance` property of [ExternalFuseBoxOptions](#) wrapper. If neither of these properties are specified and only a single `tcd_fusebox` file exists in the design, the fuse box module is inferred from this `tcd_fusebox` description.

The core description for the external fuse box can automatically be read in during module matching using the “[set\\_design\\_sources -format tcd\\_fusebox](#)” command or directly using the [read\\_core\\_descriptions](#) command.

If the instantiated module has a core description with a `FuseBoxInterface` wrapper, then connections between the fuse box controller and the fuse box interface are completed automatically. If a core description is not available, or not complete, explicit connections can be made in the MemoryBisr/[Controller](#)/[ExternalFuseBoxOptions](#)/[ConnectionOverrides](#) wrapper.

## Related Topics

[Fuse Box Interface](#)

[Connecting the BISR Controller to an External Fuse Box](#)

## Connection of the BISR Controller to System Logic

Typically, system logic is connected to the BISR controller for initiating memory repair and monitoring the progress of the operation. All connections are specified in the `DftSpecification` configuration file in the MemoryBisr:Controller section.

- The BISR controller input `clk` must be driven by an appropriate functional clock. The connection is made by specifying the `repair_clock_connection` property in the `DftSpecification/MemoryBisr/Controller` wrapper.
- The BISR controller input `resetN` is the signal used to reset the BISR chains and initiate memory repair. The connection is made by specifying the `repair_trigger_connection` property in the `DftSpecification/MemoryBisr/` wrapper.

## Verification of Block- and Chip-Level BISR

You must verify the BISR at both the block and chip level to guarantee functional hardware.

## Block-Level BISR Verification

The default signoff is the PatternsSpecification you generate with Tessent Shell to test the connectivity of the BISR chain using a pattern named MemoryBisr\_BisrChainAccess.

You can create future verification patterns as follows:

- Running fault-inserted MemoryBIST
- Performing BIRA-to-BISR capture
  - Scan external BISR chain into the internal BISR chain
- Running post-repair memory BIST

## Chip-Level BISR Verification

The default signoff is the PatternsSpecification that you generate with Tessent Shell using the create\_patterns\_specification command for the BISR hardware. PatternsSpecification verifies the FuseBoxAccess, BisrChainAccess, and Autonomous modes of operation of the BISR controller and BISR chains via the TAP.

You do not need to run memoryBIST with fault-inserted memories at the top level of the chip. This type of verification is better performed at the block level where memoryBIST can be run on the full address space of all memories.

## Related Topics

[PatternsSpecification](#)

[create\\_patterns\\_specification](#)

[Verifying BISR at the Block Level](#)

[Top-Level Verification and Pattern Generation](#)

# Chapter 7

## Tessent Shell Automotive Workflow

---

This chapter describes a pre-layout DFT insertion workflow that helps you meet the safety, quality, and reliability requirements of the ISO 26262 “Road vehicles - Functional safety” standard. The usage model provides capability for both manufacturing and in-system chip testing and diagnosis. It also provides examples of how to run, and how to generate UDFM files for, Automotive-Grade ATPG.

This chapter highlights the important steps required for you to achieve the optimal automotive DFT strategy early in your design process. At a high level, this is a comprehensive hybrid TK/LBIST two-pass RTL and scan DFT insertion flow that includes MemoryBIST, MBISR, advanced BAP, boundary scan, and in-system test.

---

### Tip

**i** This is an advanced-level workflow, which assumes knowledge about the Tessent Shell two-pass DFT insertion process. Familiarize yourself with the basic hybrid TK/LBIST flow for flat models as described in “[Hybrid TK/LBIST Flow for Flat Designs](#)” on page 357. In addition, because this is a hierarchical design, the workflow described in this chapter follows the bottom-up DFT insertion process as described in “[Tessent Shell Flow for Hierarchical Designs](#)” on page 205.

---

Refer to the following test case for a detailed usage example of the flow described in this section:

```
tessent_automotive_reference_flow_<software_version>.tgz
```

You can access this test case by navigating to the following directory:

```
<software_release_tree>/share/UsageExamples/
```

|                                                                             |            |
|-----------------------------------------------------------------------------|------------|
| <b>Introduction to Tessent Automotive .....</b>                             | <b>394</b> |
| <b>Test Case Overview and Objective.....</b>                                | <b>395</b> |
| <b>Core-Level DFT Insertion for Automotive .....</b>                        | <b>400</b> |
| DFT Insertion Flow for the Processor Core Physical Block .....              | 401        |
| DFT Insertion Flow for the GPS Baseband Physical Block.....                 | 431        |
| <b>Top-Level DFT Insertion for the Automotive Flow .....</b>                | <b>438</b> |
| First DFT Insertion Pass: Top with MemoryBIST, BISR, and Boundary Scan..... | 438        |
| Second DFT Insertion Pass: Top with EDT, OCC, and In-System Test .....      | 444        |
| Scan Insertion for the Top Design .....                                     | 449        |
| ATPG Pattern Generation for the Top Design .....                            | 451        |
| ATPG Pattern Retargeting for the Top Design.....                            | 452        |

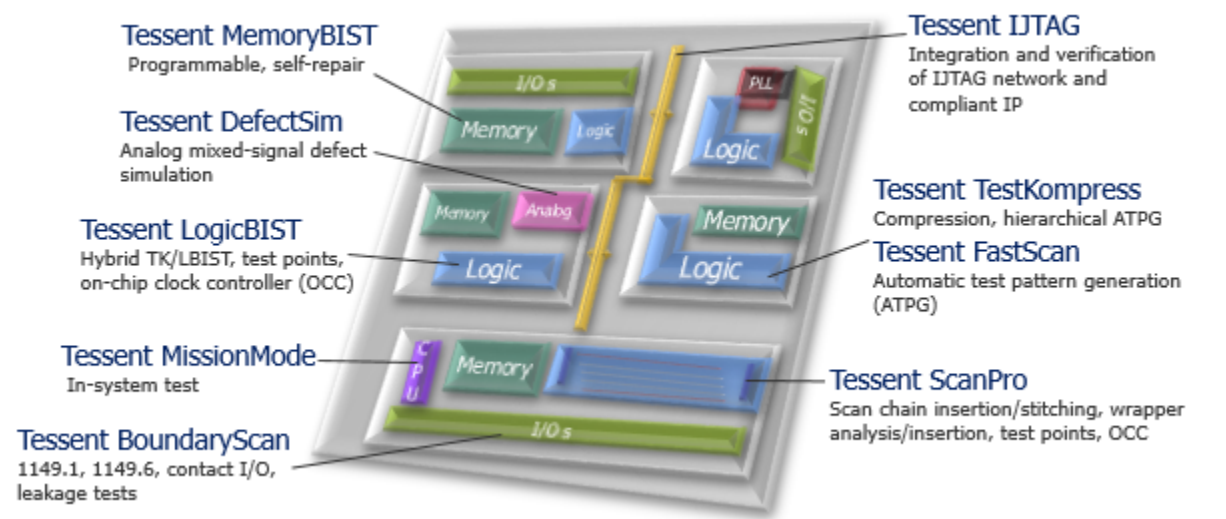
|                                                                      |            |
|----------------------------------------------------------------------|------------|
| Interconnect Bridge/Open UDFM Generation for the Top Design .....    | 452        |
| Cell-Neighborhood UDFM Generation for the Top Design.....            | 453        |
| Automotive-Grade ATPG Pattern Generation for the Top Design .....    | 453        |
| UDFM Generation for Cell-Aware ATPG .....                            | 453        |
| TCA Based Pattern Sorting.....                                       | 456        |
| <b>Functional Mode Fault Tolerance for Static IJTAG Signals.....</b> | <b>458</b> |

## Introduction to Tessent Automotive

Designs with aggressive quality and reliability requirements, such as those required to comply with the ISO 26262 standard, must exercise efficient test strategy to ensure high quality with very low defective parts per billion (DPPB). One of the key ISO 26262 requirements is the ability to test safety-critical portions of a system during power-on and periodically during in-system operation.

The Tessent product suite provides a flexible design flow that supports a high-level of automation and integration, creating an RTL-based hierarchical automotive flow that you can use for implementation of a DFT solution. It provides design rule checking, test planning, integration, and verification at the RTL- and gate-level, and it achieves high coverage for functional logic, memories, and test IPs.

Figure 7-1. Integrated Tessent DFT Solution for Automotive



Tessent automotive includes the following features:

- Low-impact in-system test.
  - Low resource requirements to design system-side logic for in-system test.
  - Interface to access IJTAG network at the chip or block level.
- Unified environment.

- Design introspection capabilities.
- Full access to design and pattern data.
- All information stored in one place, the Tessent Shell Database (TSDB).
- Test scheduling flexibility using an IJTAG infrastructure.
  - The MissionMode controller used for in-system test can access and operate any instrument in the IJTAG network.
  - Any LogicBIST or MemoryBIST (IJTAG-compliant IP) test can be converted to in-system test.
  - Run the same or different tests multiple times during in-system operation.
- Verification setup.
  - Test time is automatically calculated.
  - ROM data contents are also generated for the MissionMode controller.
  - Designer can easily verify in-system test with Verilog simulation.
- Area optimization
  - The hybrid TK/LBIST IP shares the compression logic for scan and LogicBIST.
  - Several optimization schemes for MemoryBIST.
- Fully hierarchical flow.
  - RTL-level generation and insertion.
  - Enable flow automation by using DFT signals.
  - Pattern re-targeting to generate and validate patterns at any level of the design hierarchy.
- Direct access to MemoryBIST.
  - Advanced BIST access port (BAP) to access MemoryBIST and consider limited test time of in-system test.
  - Direct control of algorithm, operation set, step, and memories.
  - Incremental repair to improve reliability of automotive application.

## Test Case Overview and Objective

In the `tessent_automotive_reference_flow` test case, you perform an RTL and scan DFT pre-layout insertion process in a bottom-up manner for each core and sub-block in the design. After inserting DFT into all the lower-level physical blocks, perform the same insertion process into

the parent blocks until you reach the top level of the chip. You also create ATPG core-level and top-level patterns and perform ATPG pattern re-targeting of the wrapped cores at the parent physical block and top level.

---

**Note**



The test case and this chapter assume that you understand the Tessent concepts and terms for hierarchical DFT as described in “[Hierarchical DFT Overview](#)” on page 106.

---

Your objective is to integrate the DFT IP and provide a mechanism to test your logic and memories during in-system operation. Considering your in-system test time constraints and coverage requirements, you must:

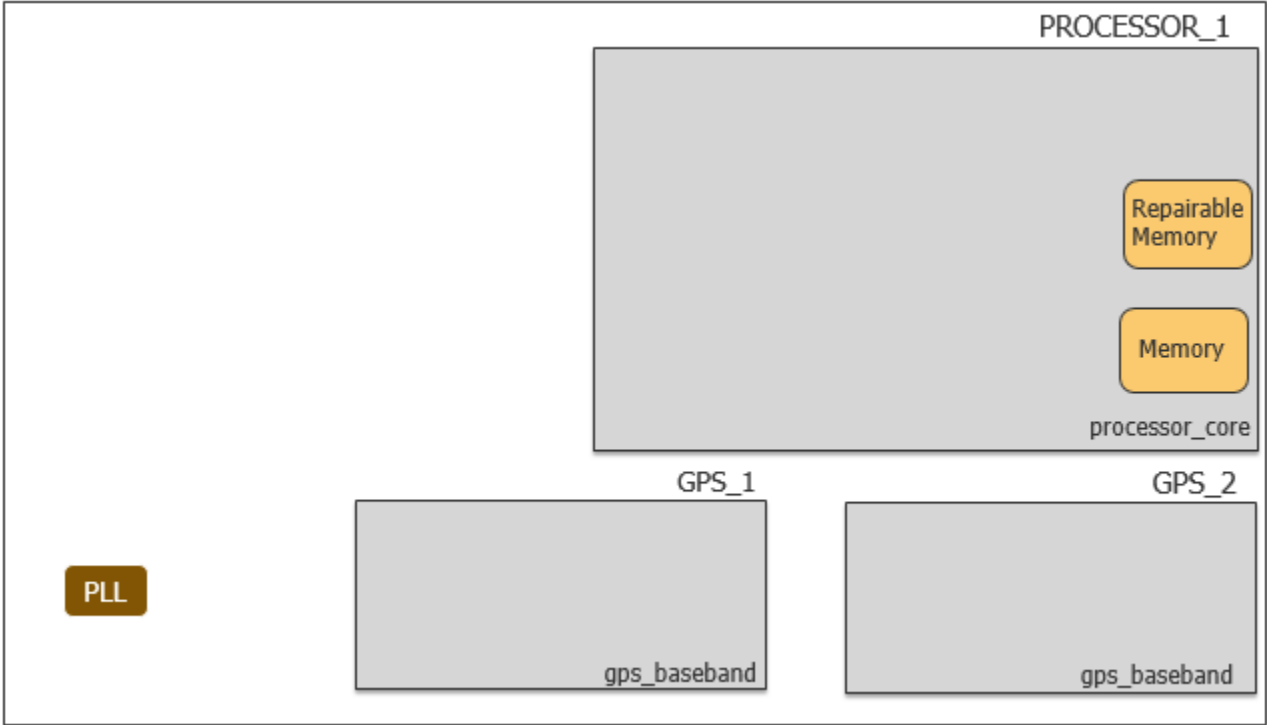
- Analyze and decide on the algorithms you want to run on the memories in-system for power-on reset (PoR) and periodic test.
- Analyze and decide the number of pseudo-random patterns during PoR and periodic test. (This depends on the scan chain length and shift clock frequency.)
- Insert test point and X-bounding cells for LogicBIST.
- Use the advanced BAP capability to provide a low-latency protocol for configuring the MemoryBIST controller, running Go/NoGo tests, and monitoring the pass/fail status for in-system and manufacturing test.
- Insert the MissionMode (also known as “in-system test”) IP to access IJTAG instruments in system, and, depending on the length of your IJTAG network, insert the in-system test controller within individual cores as well as at the top level. As a reference, the tool runs both CPU-based and DMA-based in-system test options.
- Test the inserted test logic. That is, the LogicBIST and EDT IP blocks in addition to testing the core design logic. Do this by using [controller chain mode](#), which enables you to generate ATPG patterns that target the hybrid EDT/LBIST blocks and LogicBIST controller in addition to the single chain mode logic and in-system test controller. While MemoryBIST IPs can be scan tested, under EDT test mode you can test them in system through LogicBIST.

## Design Overview

The test case uses an UltraSPARC (open-source) design with a processor core and two GPS basebands. The processor core design includes non-repairable and repairable memory, and the GPS baseband is a logic-only core that is instantiated twice.

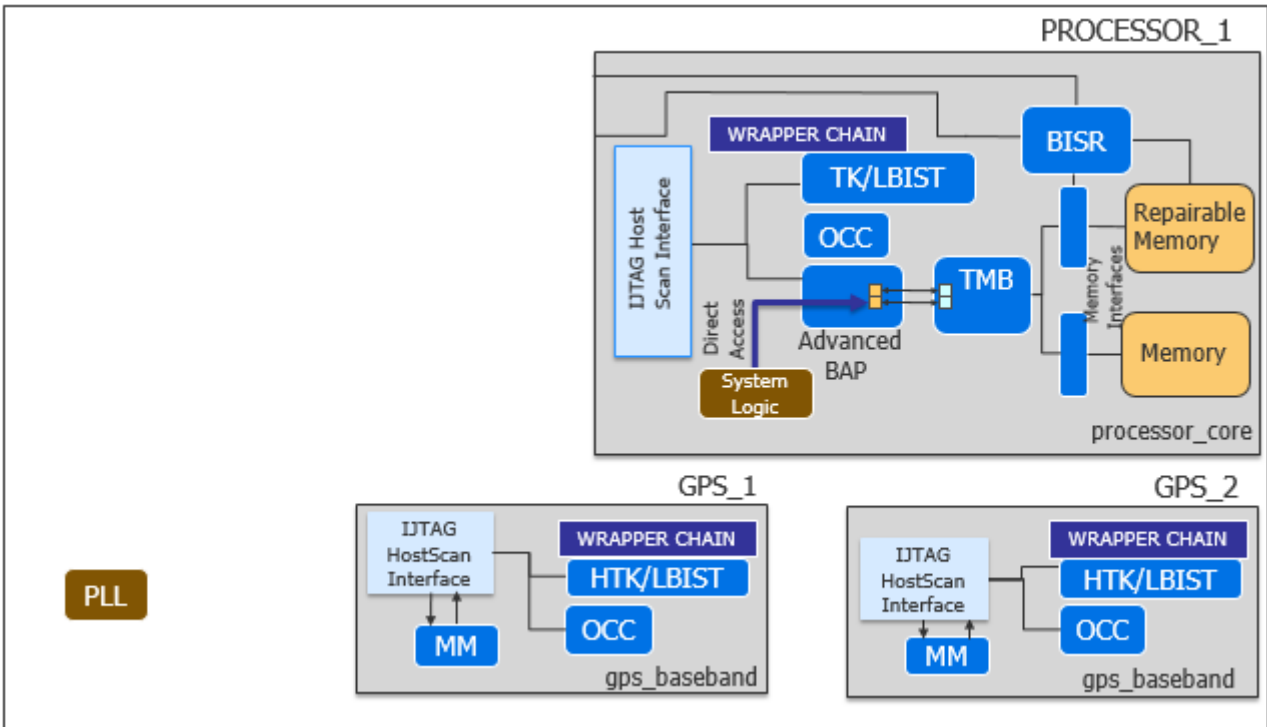


Figure 7-2. Initial Design for Automotive Test Case



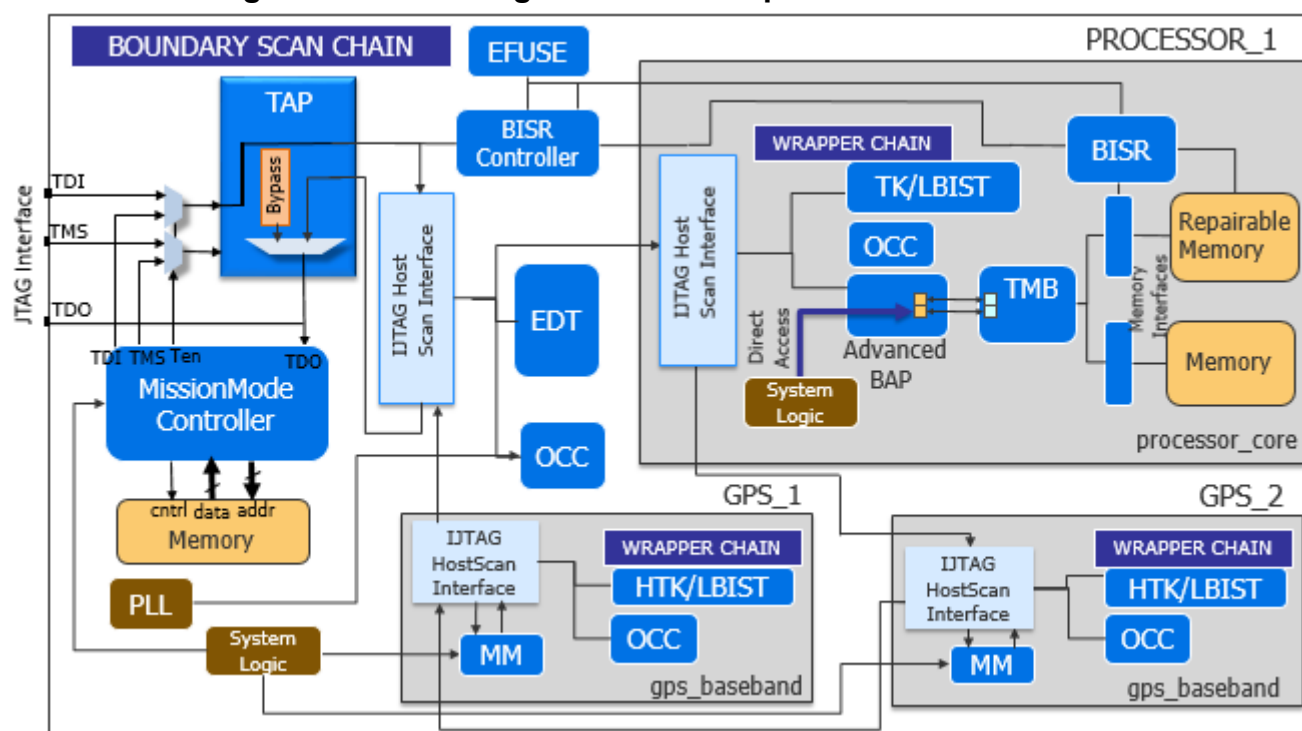
Following the bottom-up flow, you insert the following DFT logic at the core level.

Figure 7-3. DFT Integration at the Core Level for Automotive



After you have inserted the core-level IP, you perform top-level DFT logic insertion and integration, followed by ATPG and pattern retargeting.

### Figure 7-4. DFT Integration at the Top Level for Automotive

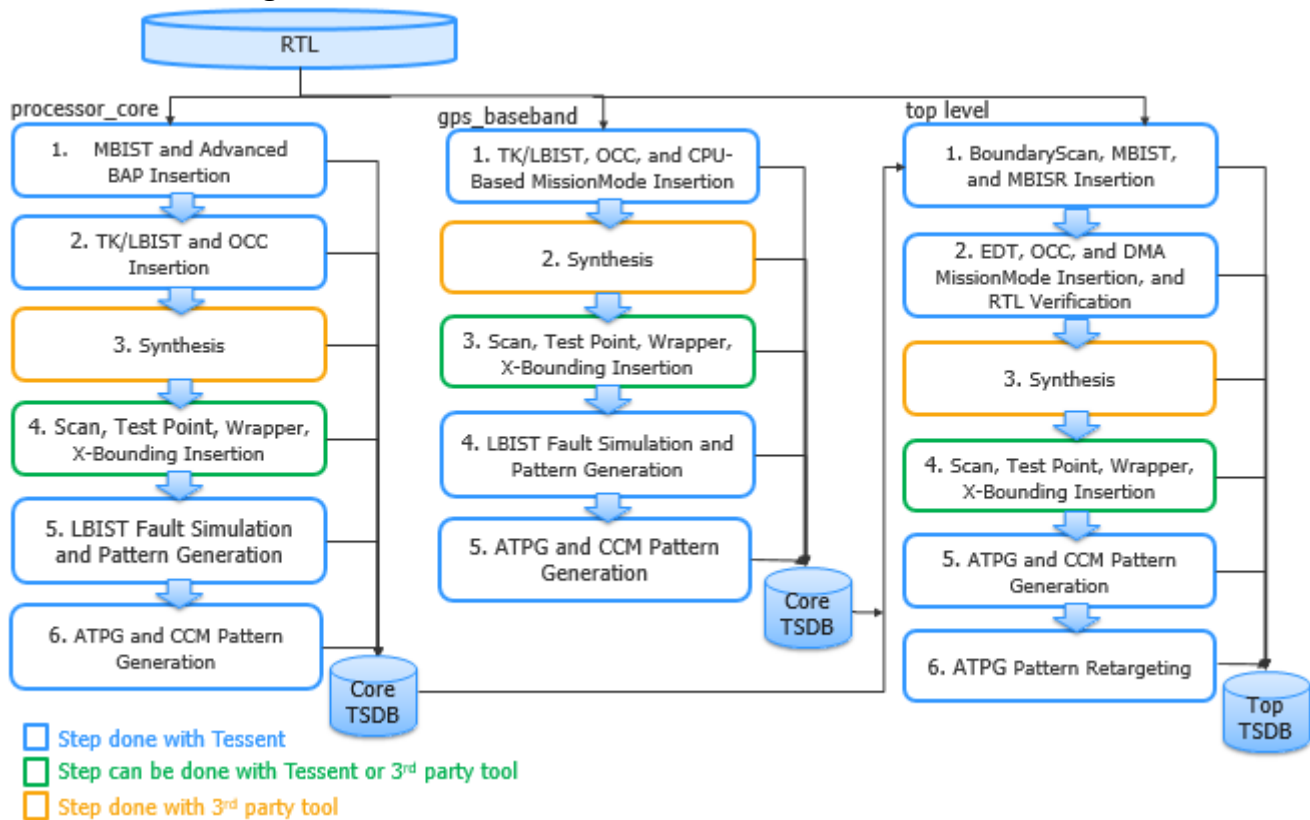


To test your IJTAG instrument in system and achieve high test coverage for your automotive IC, you insert a MissionMode controller (also called the In-System Test, or IST, controller) that intercepts the TAP controller and run MemoryBIST tests. This test case inserts a direct memory access (DMA) MissionMode controller at the top level and a CPU-based MissionMode (MM) controller within the GPS baseband cores. Depending on your IJTAG network length, you might or might not need an in-system test controller at the core. A single DMA MissionMode controller could be sufficient to run MemoryBIST and other IJTAG tests.

## DFT Insertion Flow

The following figure shows that Tessent Shell generates a TSDB for each core, and then at the top level, it uses the information stored in the core-level TSDBs for DFT integration.

**Figure 7-5. DFT Insertion Flow for Automotive Test Case**



For details about the TSDB data flow, refer to “TSDB Data Flow for the Tessent Shell Flow” on page 463.

# Core-Level DFT Insertion for Automotive

The process for inserting DFT into hierarchical designs is performed in a bottom-up process starting from the lowest level blocks. This section describes how to perform the DFT insertion flow for the processor core and GPS baseband cores.

For general information about the RTL and scan DFT insertion flow for physical blocks, refer to “[RTL and Scan DFT Insertion Flow for Physical Blocks](#)” on page 210. (The section relates to a hierarchical test case; however, the basic flow remains the same.)

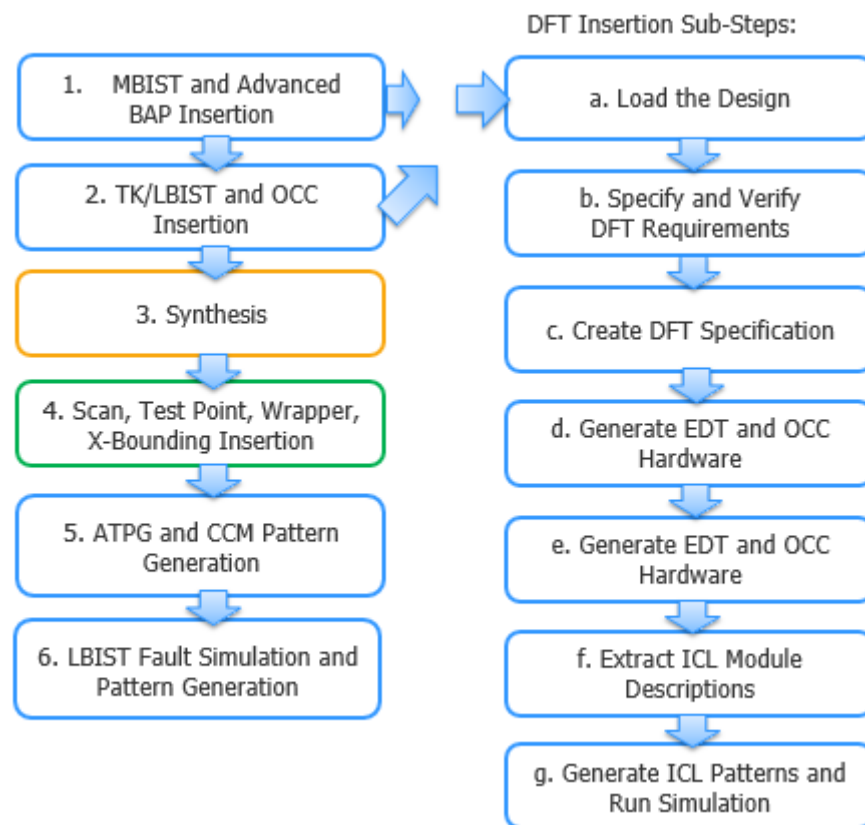
|                                                                           |            |
|---------------------------------------------------------------------------|------------|
| <b>DFT Insertion Flow for the Processor Core Physical Block . . . . .</b> | <b>401</b> |
| First DFT Insertion Pass: processor_core . . . . .                        | 402        |
| Second DFT Insertion Pass: processor_core . . . . .                       | 405        |
| Test Point, X-Bounding, and Scan Insertion: processor_core . . . . .      | 409        |
| ATPG Pattern Generation: processor_core . . . . .                         | 413        |
| LogicBIST Fault Simulation: processor_core . . . . .                      | 415        |
| LogicBIST Pattern Generation: processor_core . . . . .                    | 416        |
| Interconnect Bridge/Open UDFM Generation: processor_core . . . . .        | 418        |
| Cell-Neighborhood UDFM Generation: processor_core . . . . .               | 420        |
| Automotive-Grade ATPG Pattern Generation: processor_core . . . . .        | 424        |
| <b>DFT Insertion Flow for the GPS Baseband Physical Block. . . . .</b>    | <b>431</b> |
| DFT Insertion Pass With In-System Test: gps_baseband . . . . .            | 432        |
| LogicBIST Fault Simulation With One NCP: gps_baseband . . . . .           | 436        |
| Interconnect Bridge/Open UDFM Generation: gps_baseband . . . . .          | 436        |
| Cell-Neighborhood UDFM Generation: gps_baseband . . . . .                 | 437        |
| Automotive-Grade ATPG Pattern Generation: gps_baseband . . . . .          | 437        |

## DFT Insertion Flow for the Processor Core Physical Block

In addition to the MemoryBIST logic, during the first DFT insertion pass, insert the advanced BAP and the built-in self-repair (BISR) IP, which includes built-in redundancy analysis (BIRA). You can control whether to test the memory in the same controller step or not, depending on memory power domains, test algorithms, and other design factors.

The advanced BAP enables certain feature overrides in the hardware default (hw\_default) operating mode of the MemoryBIST controller and reduces test time by eliminating shift cycles to serially configure the controllers in the JTAG environment.

**Figure 7-6. DFT Insertion Flow for processor\_core**



|                                                                   |            |
|-------------------------------------------------------------------|------------|
| <b>First DFT Insertion Pass: processor_core</b>                   | <b>402</b> |
| <b>Second DFT Insertion Pass: processor_core</b>                  | <b>405</b> |
| <b>Test Point, X-Bounding, and Scan Insertion: processor_core</b> | <b>409</b> |
| <b>ATPG Pattern Generation: processor_core</b>                    | <b>413</b> |
| <b>LogicBIST Fault Simulation: processor_core</b>                 | <b>415</b> |
| <b>LogicBIST Pattern Generation: processor_core</b>               | <b>416</b> |
| <b>Interconnect Bridge/Open UDFM Generation: processor_core</b>   | <b>418</b> |

|                                                               |     |
|---------------------------------------------------------------|-----|
| Cell-Neighborhood UDFM Generation: processor_core .....       | 420 |
| Automotive-Grade ATPG Pattern Generation: processor_core..... | 424 |

## First DFT Insertion Pass: processor\_core

During the first DFT insertion pass, generate a default DFT specification for MemoryBIST that includes the BISR/BIRA, and then edit the DFT specification to include the additional connections for the advanced BAP.

---

### Note



The line numbers used in this procedure refer to the command flow dofile in [Example 7-1](#) on page 403.

---

For in-system tests, you can apply MemoryBIST during the power-on/off phase of the device operation or periodically while the device is operating. For details, refer to the [Tessent MissionMode User's Manual](#).

## Procedure

1. Load the RTL design data and set the design parameters. (See lines 1-15.)

---

### Note



“rtl1” is the recommended naming convention for the design ID for the first insertion pass, but you can specify any name. Refer to “[Loading the Design](#)” for more information about setting the design ID.

---

2. Set the [set\\_dft\\_specification\\_requirements](#) command to “on” for -memory\_bist. (See lines 17-18.)

When memory\_test is on, memory\_bist, memory\_bisr\_chains, and memory\_bisr\_controller default to auto; otherwise, they default to off. memory\_bisr\_chain auto changes to on for physical blocks having repairable memory, so the tool inserts a BISR chain with a BIRA engine created within the MemoryBIST controller.

3. Define the design clocks. (See lines 20-22.)
4. Check the design rules. (See lines 24.)
5. Create the DFT specification. (See lines 26-28.)
6. Edit the DFT specification to add the advanced BAP. (See lines 30-76.)

You can configure the connections to control the BAP directly through your system logic, thus bypassing the IJTAG network. Or, you can provide a way to control the advanced BAP as part of the IJTAG network. For this test case, implement the first strategy. When utilizing the advanced BAP feature, specify the options you plan to control directly using the ExecutionSelections wrapper in the DFT specification, and

then specify the BAP connections to the system logic. You can specify the connections within the DftSpecification wrapper or through a post-insertion procedure.

7. Generate the DFT hardware, IJTAG network connectivity, and test patterns. (See lines 79-86.)
8. Run simulations to verify the design. (See lines 88-92.)

## Examples

The following dofile example shows a typical automotive command flow for a core-level first DFT insertion. Modify the DFT specification for you design requirements.

### Example 7-1. Dofile Example for First DFT Insertion Pass, processor\_core

```
1 # Load the design
2 set_context dft -rtl -design_id rtl1
3 set_tsdb_output_directory ../tsdb_outdir
4 read_cell_library ../../lib/standard_cells/tessent/adk.tcelllib \
5 set_design_source -format tcd_memory -y ../../library/memories
6     -extension tcd_memory
7 read_verilog ../../library/memories/SYNC_1RW_32x16_RC.v
8     -interface_only -exclude_from_file_dictionary
9 read_verilog ../../library/memories/SYNC_1RW_8Kx16_RC.v
10    -interface_only -exclude_from_file_dictionary
11 read_verilog -f rtl_file_list
12
13 set_current_design processor_core
14
15 set_design_level physical_block
16
17 # Specify the DFT requirements
18 set_dft_specification_requirements -memory_test on
19
20 # Define memory clock
21 add_clocks 0 dco_clk -period 3
22 add_clocks 0 directclk -period 12
23
24 check_design_rules
25
26 # Create and report the DFT specification
27 set_spec [create_dft_specification]
28 report_config_data $spec
29
30 # Edit DFT specification to include additional advanced BAP connections
31 add_config_element DftSpecification(processor_core,rtl1)/MemoryBist/
32     BistAccessPort/DirectAccessOptions
33 set_config_value
34 DftSpecification(processor_core,rtl1)/MemoryBist/
35     BistAccessPort/DirectAccessOptions/direct_access_on
36 set_config_value
37 DftSpecification(processor_core,rtl1)/MemoryBist/
38     Controller(c1)/DirectAccessOptions/ExecutionSelections
39 set_config_value
40 DftSpecification(processor_core,rtl1)/MemoryBist/
41     Controller(c1)/DirectAccessOptions/algorithm on
```

```

42 set_config_value
43 DftSpecification(processor_core,rtl1)/MemoryBist/
44     Controller(c1)/DirectAccessOptions/operation_set on
45 set_config_value
46 DftSpecification(processor_core,rtl1)/MemoryBist/
47     Controller(c1)/DirectAccessOptions/memory on
48 set_config_value
49 DftSpecification(processor_core,rtl1)/MemoryBist/
50     Controller(c1)/DirectAccessOptions/step on
51 set_config_value
52 DftSpecification(processor_core,rtl1)/MemoryBist/Controller(c1)/
53     AdvancedOptions/extra_algorithms {SMARCHCHKB SMARCHCHKBCI SMARCH
54     SMARCHCHKBCIL}
55
56 report_config_data $spec
57
58 # Specify BAP connection to system logic
59 read_config_data -in $spec/MemoryBist/BistAccessPort/Connections/
60     -from_string {
61     DirectAccess {
62         controller_select      : DBAP_control/ctrl_select ;
63         step_select            : DBAP_control/step_select ;
64         step_select_enable     : DBAP_control/step_select_en ;
65         algorithm_select       : DBAP_control/algo_select ;
66         algorithm_select_enable : DBAP_control/algo_select_en ;
67         operation_set_select   : DBAP_control/opset_select ;
68         operation_set_select_enable : DBAP_control/opset_select_en ;
69         ClockDomain (-) {
70             clock      : DBAP_control/clkout ;
71             reset      : reset_n ;
72             test_start : DBAP_control/start;
73             test_done  : DBAP_control/done ;
74             test_pass  : DBAP_control/pass ;
75         }
76     }
77 }
78
79 # Generate and insert the hardware
80 process_dft_specification
81
82 extract_icl
83
84 # Generate testbench verification patterns
85 create_patterns_specification
86 process_patterns_specification
87
88 # Run and check testbench simulations
89 set_simulation_library_sources -y ../../../../library/memories/
90     -extension v -v ../../../../library/standard_cells/verilog/adk.v
91 run_testbench_simulations
92 check_testbench_simulations
93
94 exit

```



## Second DFT Insertion Pass: `processor_core`

In the second DFT insertion pass, insert the EDT, OCC, and LogicBIST instruments. Use the hybrid TK/LBIST process to share the EDT and LogicBIST IP. Control the clocking scheme for the LogicBIST pattern using a named capture procedure (NCP) index decoder, which decodes the NCP index output of the hybrid controller into clock sequences that are generated by Tessent OCCs across all capture procedures.

For details about OCC for the hybrid TK/LBIST flow, refer to “[Tessent OCC TK/LBIST Flow](#)” in the *Hybrid TK/LBIST Flow User’s Manual*.

---

### Note



The line numbers used in this procedure refer to the command flow dofile in [Example 7-2](#) on page 406 unless otherwise noted.

---

### Procedure

1. [Loading the Design](#).
2. Follow the steps described in “[Specifying and Verifying the DFT Requirements: DFT Signals for Wrapped Cores](#)” on page 216, ensuring that you also include the required DFT signals for the hybrid TK/LBIST flow and for controller chain mode. (See lines 3-31.)
3. [Creating the DFT Specification](#) with SIBs for EDT, OCC, and LogicBIST, and edit the default specification to include the OCC, hybrid IP, and LogicBIST. (See lines 33-125.)

Use the `read_config_data` command to edit the DFT specification as follows:

- Specify the [OCC](#) wrapper and specify your clock intercept node for the OCC.

For details about OCC for the hybrid TK/LBIST flow, refer to “[Tessent OCC TK/LBIST Flow](#)” in the *Hybrid TK/LBIST Flow User’s Manual*.

- Specify the EDT wrapper to include a [LogicBistOptions](#) wrapper, and specify a MISR ratio of one. This reduces the chances of aliasing at the MISR (for better debugging capability) while trading off the area.

If you want to use the `edt_clock` as a shift clock source for LogicBIST, specify a dash (-) value in the interface wrapper, which caused the tool to not create a `shift_clock_src` port.

- Specify a [LogicBist](#) wrapper that contains both a Controller wrapper and an `NcpIndexDecoder` wrapper. Tessent Shell automatically converts the EDT controller into a hybrid TK/LBIST controller.

You must specify the clocking combinations to be used during TK/LBIST test. The tool synthesizes the NCP index decoder and generates named capture procedures based on this description. For details, refer to “[NCP Index Decoder](#)” in the *Hybrid TK/LBIST Flow User’s Manual*.

### Note

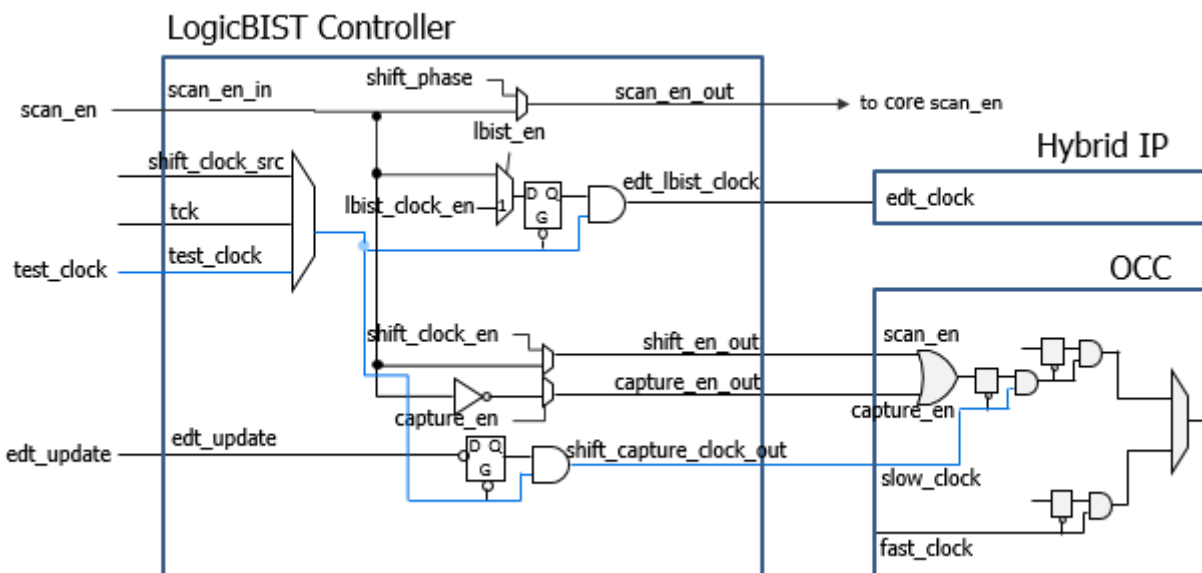
When you have only one NCP, the tool does not generate an NCP index decoder. You can use this configuration to implement a static clock sequence loaded through the ICL network. (You notice this for the GPS baseband block.)

4. [Generating the EDT, Hybrid TK/LBIST, and OCC Hardware](#), plus the LogicBIST hardware. (See line 125.)
5. [Extracting the ICL Module Description](#). (See line 127.)
6. [Generating ICL Patterns and Running Simulation](#). (See lines 129-140.)

## Results

The following figure shows the clocking and DFT controls after the second DFT insertion pass.

**Figure 7-7. Enhanced Clocking and DFT Controls After Second DFT Insertion Pass**



## Examples

The following dofile example shows a typical automotive command flow for a core-level second DFT insertion. Modify the DFT specifications for your design requirements.

### Example 7-2. Dofile Example for Second DFT Insertion Pass, processor\_core

```
1 // Load the design commands ...
2
3 set_dft_specification_requirements -logic_test on
4
5 # Add required DFT signals for logic test
6 add_dft_signals ltest_en -create_with_tdr
7 add_dft_signals scan_en edt_update test_clock
8     -source_node {scan_enable edt_update test_clock u }
```

```

9  add_dft_signals edt_clock shift_capture_clock -create_from_other_signals
10
11 # Add required DFT signals specific to hybrid TK/LBIST flow
12 add_dft_signals observe_test_point_en control_test_point_en x_bounding_en
13
14 # Add required DFT signal for MemoryBIST
15 # DFT signal used for scan-tested instruments such as MemoryBIST
16 add_dft_signals tck_occ_en
17
18 # Add DFT signals to bypass memories or run multiple load ATPG for them
19 add_dft_signals memory_bypass_en
20 add_dft_signals mcp_bounding_en
21
22 # Add DFT signals required for hierarchical DFT (external, internal modes)
23 add_dft_signals int_ltest_en ext_ltest_en int_mode ext_mode
24
25 # Add DFT signal for controller chain mode
26 add_dft_signals controller_chain_mode
27
28 # Enable Observation Scan Technology (OST) for capturing per shift and
  capture
29 add_dft_signals capture_per_cycle_static_en
30
31 check_design_rules
32
33 # Create DFT specification
34 set spec [create_dft_specification -sri_sib_list {occ edt lbist } ]
35
36 Use report_config_syntax DftSpecification/edt|occ to see full syntax
37 report_config_data $spec
38
39 # Edit DFT specification to specify the SIB of the OCC and to insert OCC
40 read_config_data -in $spec -from_string {
41     OCC {
42         ijtag_host_interface : Sib(occ);
43         static_clock_control : external;
44     }
45 }
46
47 # Specify OCC insertion and intercept node
48 # This is a generic method for populating the OCC; modify for your design
49 # Scan enable and shift capture clock signals are automatically connected
50 # to the OCC instances
51 set id_clk_list [list \
52 dco_clk dco_clk \
53 directclk directclk
54 ]
55 foreach {id clk} $id_clk_list {
56 set occ [add_config_element OCC/Controller($id) -in $spec]
57 set_config_value clock_intercept_node -in $occ $clk
58 }
59
60 # Specify the hybrid EDT configuration
61 read_config_data -in $spec -from_string {
62     EDT {
63         ijtag_host_interface : Sib(edt);
64         Controller (c1) {
65             longest_chain_range : 100, 200 ;

```

```

66         scan_chain_count : 10;
67         input_channel_count : 1;
68         output_channel_count : 1;
69         LogicBistOptions {
70             misr_input_ratio : 1 ;
71             ShiftPowerOptions {
72                 present : on ;
73                 default_operation : disabled ;
74                 SwitchingThresholdPercentage {
75                     min : 25 ;
76                 }
77             }
78         }
79     }
80 }
81 }
82
83 # Specify the LogicBIST controller with NCP index decoder
84 read_config_data -in $spec -from_string {
85     LogicBist {
86         ihtag_host_interface : Sib(lbist);
87         Controller(1%ctrl_lbist) {
88             burn_in : on ;
89             pre_post_shift_dead_cycles : 8 ;
90             SingleChainForDiagnosis { Present : on ; }
91             ControllerChain {
92                 present : on;
93                 clock : tck;
94             }
95             Connections {
96                 scan_en_in : scan_enable ;
97                 controller_chain_scan_in : ccm_scan_in ;
98                 controller_chain_scan_out : ccm_scan_out ;
99             }
100             Interface {
101                 shift_clock_src : - ;
102             }
103             ShiftCycles { max : 200 ; }
104             CaptureCycles { max : 7 ; }
105             PatternCount { max : 1024 ; }
106             WarmupPatternCount { max : 512 ; }
107         }
108     }
109     NcpIndexDecoder{
110         Ncp(dco_clk) {
111             cycle(0): OCC(dco_clk);
112             cycle(1): OCC(dco_clk);
113         }
114         Ncp(ALL) {
115             cycle(0): OCC(dcoo_clk);
116             cycle(1): OCC(directclk);
117         }
118         Ncp(sti_occ) {
119             cycle(0): processor_core_rtl1_tessent_sib_sti_inst ;
120         }
121     }
122 } // End of LogicBIST controller wrapper
123// End of report_config_data

```

```

124
125process_dft_specification
126
127extract_icl
128
129# Write script for design compilation
130write_design_import_script for_dc_synthesis.tcl -replace
131
132create_patterns_specification
133process_patterns_specification
134
135set_simulation_library_sources -v
136    ../../../../library/standard_cells/verilog/adk.v -y
137    ../../../../library/memories ../../../../library/standard_cells/verilog
138    -extension v
139
140run_testbench_simulations -simulator_option +nowarnTSCALE
141
142exit

```

## Test Point, X-Bounding, and Scan Insertion: processor\_core

After synthesizing the design using a third-party tool, perform test point analysis and insertion, X-bounding, wrapper analysis, and scan chain analysis and insertion.

- Test points — Inserting test points increases the testability of a design by improving controllability and observability during scan testing. This step is part of the hybrid TK/LBIST workflow; see “[Perform Test Point Insertion](#)” on page 366 for details.

In addition, a new type of observe point — the observation scan observe point (OP) — is inserted during test point insertion. The tool monitors observation scan OPs during every shift cycle and capture cycle compared with traditional OPs, which are monitored only during capture. Observation scan observe points are a part of observation scan technology (OST). For details, refer to “[Observation Scan Technology](#)” in the *Hybrid TK/LBIST Flow User’s Manual*.

---

### Note

 For OST, you must add the capture\_per\_cycle\_static\_en DFT signal during the DFT insertion pass.

---

- X-bounding — X-bounding ensures that only valid binary values propagate through the scan cells during test. This prevents X sources from reaching the memory elements and corrupting the signature during LogicBIST. For details, refer to “[X-Bounding](#)” in the *Hybrid TK/LBIST Flow User’s Manual*.
- Wrapper analysis — In hierarchical DFT, wrapper analysis prepares the functional scannable sequential elements (or “flops”) for reuse as wrapper cells. Use the [analyze\\_wrapper\\_cells](#) command.

- Scan chains — The tool inserts wrapper scan chains around the physical blocks that connects to each input and output. The input and output wrapper chains are exercised using the internal and external scan mode DFT control signals. For more information, see [“Performing Scan Chain Insertion: Wrapped Core”](#) on page 218.

In addition, you must configure the controller scan chains and add a scan mode for controller chain mode (CCM).

---

**Note**

 Tessent Shell works with any third-party scan insertion or other tools. However, because all Tessent products reside on the same code and database, you lose some automation with third-party tools.

---

---

**Note**

 The line numbers used in this procedure refer to the command flow dofile in [Example 7-3](#) on page 411 unless otherwise noted.

---

## Procedure

1. After loading the design and setting the design parameters, perform test point, X-bounding, wrapper, and scan insertion in the merged DFT context. (See lines 7.)
2. Read the design files and specify the requirements for test points and OST observe points. (See line 9-51.)
3. Post-test point insertion, specify the requirements for X-bounding. (See lines 60-70.)
4. Specify wrapper cell requirements and perform wrapper cell analysis. (See lines 72-77.)

For details, refer to [“Scan Insertion for Wrapped Core”](#) in the *Tessent Scan and ATPG User’s Manual*.

5. Activate controller chain segments. (See lines 79-82.)

By default, CCM scan segments in Tessent IP cores are not active to prevent them from being confused with standard scan elements. You must activate a CCM scan mode on Tessent IP instances before adding them to the scan mode population.

6. Specify the scan modes: internal, external, and controller chain. (See lines 84-94.)

For internal and external modes, connect the scan chains to the EDT block. For CCM mode, ensure that you only include CCM scan segments as valid scan elements.

7. Analyze scan chains and insert X-bounding, wrapper, and scan chain logic. (See lines 96-103.)

## Examples

The following dofile shows a command flow for test point and scan insertion.

### Example 7-3. Dofile Example for Test Point and Scan Insertion, processor\_core

```

1  ##### Test Point, X-bounding, Wrapper, and Scan Insertion #####
2
3  # Open the previous TSDB directory if it is not in the current working
   directory
4  set_tsdb_output_directory ../tsdb_outdir
5
6  # Setting context to dft since inserting DFT into gate-level design
7  set_context dft -test_point -scan -no_rtl -design_id gate1
8
9  ##Read in the design (you may use -exclude_from_file_dictionary to
   exclude the netlist from the tessent database)
10 read_cell_library ../../../../library/standard_cells/tessent/adk.tcelllib
11 # Read the synthesized netlist
12 read_verilog ../3.synthesize_rtl/processor_core_synthesized.vg
13
14 ##Read in the memory library model
15 set_design_sources -format tcd_memory -y ../../../../library/memories/
   -extension tcd_memory
16
17 ##Read in memory verilog model
18 read_verilog ../../../../library/memories/SYNC_1RW_32x16_RC.v
   -interface_only -exclude_from_file_dictionary
19 read_verilog ../../../../library/memories/SYNC_1RW_8Kx16.v -interface_only
   -exclude_from_file_dictionary
20
21 ##Read the design files from the RTL insertion pass
22 # Use the -no_hdl switch to skip reading the netlist as the synthesized
   netlist has already been read
23 read_design processor_core -design_id rtl2 -no_hdl
24
25 set_current_design processor_core
26
27 ##Setting the design level as physical_block
28 set_design_level physical_block
29
30 set_static_dft_signal_value ltest_en 1
31
32 report_static_dft_signal_settings
33
34 # Specify the number of testpoint (integer or integer %)
35 set_test_point_analysis_options -total_number 250
36
37 # Specify how many observe points to be observed by one scan cell
38 set_test_point_insertion_options -observe_point_share 5
39
40 # Specify both types of test points to reduce both pattern count
41 #   and improve random pattern testability
42 set_test_point_type { lbist_test_coverage edt_pattern_count }
43
44 # Specify capture per cycle observation for OST to observe per shift cycle
45 set_test_point_analysis_options -capture_per_cycle_observe_points on
46
47 # Setting the shift length to the anticipated scan chain length of the
   design for OST
48 set_test_point_analysis_options -minimum_shift_length 80

```

```
49
50 # Set mcp_bounding_en to 1 to gate BIST.CLK and prevents additional
    hardware bounding logic
51 set_static_dft_signal_values mcp_bounding_en 1
52
53 # Check design rule
54 set_system_mode analysis
55
56 # Report clocks and dft signals
57 report_clocks
58 report_dft_signals
59
60 # Analyze test points
61 analyze_test_points
62 write_test_point_dofile processor_core_tpi.dofile -all -replace
63
64 ## Exclude reset from X-bounding to avoid XB DRC. Handled by wrapper
    insertion. Automation in 19.2
65 set_xbounding_options -exclude {reset_n}
66
67 ## Perform X-bounding for LBIST
68 analyze_xbounding
69 report_xbounding -verbose > xbound_list
70 report_xbounding -verbose -ignored_x_sources on >
    xbound_list_with_ignored_x_source
71
72 # Exclude the edt_channel in and out ports from wrapper chain analysis.
    The ijtag_* edt_update ports are automatically excluded
73 set_wrapper_analysis_options -exclude_ports [ get_ports
    {*_edt_channels_*} ]
74
75 # Perform wrapper cell analysis
76 analyze_wrapper_cells
77 report_wrapper_cells -Verbose > wrapper_cell.rpt
78
79 # Set attribute value for in-built chains for controller_chain_mode
80 set_attribute_value [get_instances *tessent_single_chain_mode_logic_*]
    -name active_child_scan_mode -value controller_chain_mode
81 set_attribute_value [get_instances *tessent_lbist_inst] -name
    active_child_scan_mode -value controller_chain_mode
82 set_attribute_value [get_instance -of_module *_edt_lbist_c1] -name
    active_child_scan_mode -value controller_chain_mode
83
84 # Specify different modes (internal and external) how the chains need to
    be stitched
85 # The type internal/external and enable_dft_signal are inferred from
    registered DFT Signals(int_mode and ext_mode)
86 # Create a scan mode and specify EDT instance to connect the scan chains
    to
87 set ccm [get_scan_elements -of_child_scan_mode controller_chain_mode]
88 set core [remove_from_collection [get_scan_elements] $ccm]
89 #set core [remove_from_collection [get_scan_elements -class core] $ccm]
90 set edt_instance [get_instances -of_icl_instances [get_icl_instances
    -filter tessent_instrument_type==mentor::edt]]
91
92 add_scan_mode int_mode -edt $edt_instance -include_elements $core
    -enable_dft_signal int_mode
```



```

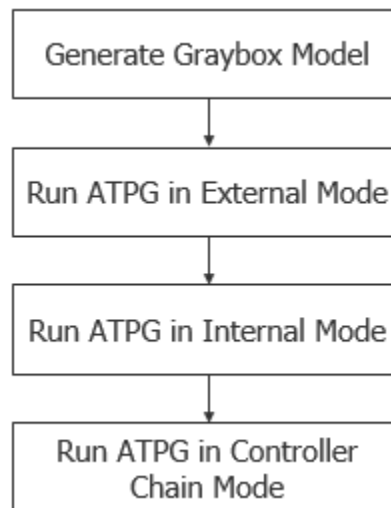
93 add_scan_mode controller_chain_mode -include_elements $ccm -chain_count 1
   -enable_dft_signal controller_chain_mode -si_port_format ccm_scan_in%d
   -so_port_format ccm_scan_out%d
94 add_scan_mode ext_mode -chain_count 2
95
96 # Analyze the scan chains and review the different scan modes and chains
   before stitching the chains
97 analyze_scan_chains
98
99 # Insert scan chains and writes the scan inserted design into tsdb_outdir
   directory
100 insert_test_logic
101
102 report_scan_chains
103 report_scan_cells > scan_cells.list

```

## ATPG Pattern Generation: processor\_core

Generating ATPG patterns for a wrapped core includes a step for generating a graybox model. In addition, you must perform ATPG twice, once for the core's external mode and once for the core's internal mode, which is typical for any hierarchical design. Controller chain mode, when used, also requires its own ATPG pass.

**Figure 7-8. ATPG Pattern Generation Flow for processor\_core**



For information about graybox models, refer to “[Graybox Model](#)” on page 110.

## Procedure

Perform ATPG as described in “[Performing ATPG Pattern Generation: Wrapped Core](#)” on page 222 with the following extra considerations:

- a. After running ATPG on the internal mode and saving the collateral data to the TSDB with the `write_tsdb_data` command (step 3-d), do the following:
  - i. Return to setup mode and disable the `memory_bypass_en` signal.
  - ii. Read the PDL of the BISR chains that have the `init_bisr_chains` iProc. The procedure is located at:

```
/tsdb_outdir/instruments/  
processor_core_rtl1_mbisr.instrument/  
processor_core_rtl1_tessent_mbisr.pdl
```

Invoke the procedure as follows:

```
set_test_setup_icall init_bisr_chains -front
```

- iii. Run ATPG on the internal mode again and save the data with the `write_tsdb_data` command.

ATPG generates a multiple load pattern that can scan through the memory. You can use multiple load scan patterns to generate scan patterns through ROM and RAM memories. Before generating multiple load patterns on repairable memories, any repair information that was previously generated by the MemoryBIST run must be applied to the memory repair ports.

For details, refer to “[Running Multiple Load ATPG on Wrapped Core Memories with Built-In Self Repair](#)” on page 379.

- iv. Use the `read_faults` command to merge the fault list from running external mode to find the total overall fault coverage of the wrapped core.
- b. Run ATPG on the controller chain mode to create ATPG patterns for the following test logic IPs: hybrid controller, LogicBIST controller, single chain mode logic, and in-system test controller. Do the following:
    - i. Load the wrapped cores that contain the child wrapped cores.
    - ii. If you used Tessent Scan for scan insertion, specify “`import_scan_mode controller_chain_mode`” to import the controller chain mode.
    - iii. Specify a unique ATPG mode name for controller chain mode, such as `ccm`. For example:

```
set_current_mode ccm
```

- iv. After running DRC, generate the ATPG patterns, and store the TCD, flat model, fault list and PatDB files in the TSDB using the `write_tsdb_data` command.

The generated ATPG patterns are core-level patterns. You generate these patterns again at the top level.

- v. Use the `read_faults` command to merge the fault list from running internal mode to find the total overall fault coverage of the wrapped core.

## LogicBIST Fault Simulation: `processor_core`

Perform LogicBIST fault simulation to generate pseudo-random, parallel pattern sets. Tessent Shell chooses the PRPG seed value based on the specified warm-up pattern and creates the signature for the MISR. The signature is stored in the TSDB. Specify the number of random patterns based on your in-system requirements for test time and LogicBIST coverage.

For details about seeding, refer to “[Warm-Up Patterns](#)” in the *Hybrid TK/LBIST Flow User’s Manual*.

Refer to “[Parallel Versus Serial Patterns](#)” in the *Tessent Scan and ATPG User’s Manual* for more information about pattern types.

---

### Note



The line numbers used in this procedure refer to the command flow in [Example 7-4](#) on page 416. Refer to “[Performing LogicBIST Fault Simulation](#)” on page 374 for additional details about some of the following steps.

---

## Procedure

1. [Loading the Design](#). (See lines 1-10.)
2. Set the current test mode to a unique name for the new pattern set you create to test the hybrid IP. (See line 12.)
3. Import the core’s internal mode data—that is, the scan-inserted design data for the EDT and OCC logic. (See line 14.)

Using the `import_scan_mode` command assumes that you used Tessent Scan to perform scan chain stitching.

4. Add the hybrid TK/LBIST core instances. (See line 16.)  
You must explicitly add the LogicBIST controller core.
5. Specify the capture procedure names and the count percentage to repeat the NCP once every 256 patterns. (See lines 18-19.)
6. Specify the order of the NCPs. (See lines 21-25.)
7. Read in the NCP testproc file. (See lines 29-32.)
8. Add the faults and specify the maximum number of pseudo-random patterns you want the tool to simulate. (See lines 34-35.)

9. Set the pattern source to LogicBIST and run fault simulation. (See line 36.)
10. Write out the parallel testbench and save the data into the TSDB. (See lines 38-40.)

## Examples

The following dofile example shows a typical command flow for LogicBIST fault simulation.

### Example 7-4. Dofile Example for LogicBIST Fault Simulation, processor\_core

```
1  set_context patterns -scan -design_id gate2
2  set_tsdb_output_directory ../tsdb_outdir
3  open_tsdb ../tsdb_outdir
4  read_cell_library ../../library/standard_cells/tessent/adk.tcelllib
5  read_cell_library ../../library/memories/SYNC_8X16.atpglib
6  read_cell_library ../../library/memories/SYNC_1RW_32x16_RC.atpg
7
8  read_design processor_core -design_id gate1
9
10 set_current_design processor_core
11
12 set_current_mode lbist_sa -type internal
13
14 import_scan_mode int_mode
15
16 add_core_instances -module *tessent_lbist
17
18 set_lbist_controller_options -capture_procedures {ALL 90 directclk 8
19     sti_occ 2 }
20
21 # Specify the full pathname to the dofile located in the
22 # *_lbist_ncp_index_decoder.instrument directory in the TSDB
23 dofile ../tsdb_outdir/instruments/processor_core_rtl2
24     _lbist_ncp_index_decoder.instrument/processor_core_rtl2_tessent
25     _lbist_ncp_index_decoder.dofile
26
27 set_system_mode analysis
28
29 # Specify the full pathname to the testproc file located in the
30 # *_lbist_ncp_index_decoder.instrument directory in the TSDB
31 read_procfile ../tsdb_outdir/instruments/processor_core_rtl2
32     _lbist_ncp_index_decoder.instrument/processor_core_rtl2
33
34 add_faults -all
35 set_random_pattern 1024
36 simulate_patterns -source bist -store_patterns all
37
38 write_patterns patterns/processor_core_lbist_parallel.v -verilog
39     -parallel -replace
40 write_tsdb -replace
```

## LogicBIST Pattern Generation: processor\_core

To program the LogicBIST controller, use the pattern specification in hardware default mode to generate testbench and pattern range-specific vectors.

### Note

 The line numbers used in this procedure refer to the command flow in [Example 7-5](#) on page 417. Refer to “[Perform LogicBIST Pattern Generation](#)” on page 376 for additional details about some of the steps in the following.

---

## Procedure

1. [Loading the Design](#), ensuring that you set the context to “patterns -ijtag”. (See lines 1-7.)

The design ID should be the same design you used for LogicBIST fault simulation.

2. Create the patterns specification. (See lines 15-16.)
3. Edit the patterns specification to specify the LogicBIST pattern configuration. (See lines 18-52.)

If you want to debug Xs in your MISR during simulation, you can enable debugging with the `logic_bist_debug` property.

4. Generate the IJTAG patterns for LogicBIST and run simulation. (See lines 55-63.)

## Examples

The following dofile shows a command flow for generating IJTAG patterns for LogicBIST in the automotive Tessent Shell flow.

### Example 7-5. Dofile Example for LogicBIST Pattern Generation, processor\_core

```

1  set_context patterns -ijtag
2  set_tsdb_output_directory ../tsdb_outdir
3  read_cell_library ../../../../library/standard_cells/tessent/adk.tcelllib
4  # read memory library models too
5  read_design processor_core -design_id gate2
6
7  set_current_design piccpu
8
9  # Report clock and DFT signal settings
10 report_static_dft_signal_settings
11 report_clocks
12
13 set_system_mode analysis
14
15 # Create patterns specification
16 set patt_spec [ create_patterns_specification ]
17
18 # Edit the patterns specification for LogicBIST pattern
19 # Example only; modify for your design requirements
20 read_config_data -replace -in $patt_spec -from_string {
21     AdvancedOptions {
22         ConstantPortSettings {
23             scan_enable : 0;
24         }
25     }

```

```

26     Patterns(ICLNetwork) {
27         ICLNetworkVerify(processor_core) {
28         }
29     }
30     Patterns(LogicBist_processor_core) {
31         ClockPeriods {
32             dco_clk : 3.00ns;
33             directclk: 12.00ns;
34             test_clock_u: 100.00ns;
35         }
36         SimulationOptions {
37 # You can opt to enable debugging Xs in the MISR during simulation
38             logic_bist_debug : off;
39         }
40         TestStep(serial_load) {
41             LogicBist {
42                 CoreInstance(.) {
43                     run_mode : run_time_prog;
44                     mode_name : lbist_sa ;//same as set_current_mode in
45                     lbist fault simulation
46                     begin_pattern : 0;
47                     end_pattern : 255 ;
48                     misr_compares : 1 ;
49                 }
50             }
51         }
52     }
53 }
54
55 process_patterns_specification
56
57 set_simulation_library_sources -y ../../../../library/memories/ -extension v
58     -v ../../../../library/standard_cells/verilog/adk.v
59
60 run_testbench_simulation -simulator_options { -voptargs="+acc" }
61
62 # Use the following command if simulation fails
63 check_testbench_simulations -report_status
64
65 exit

```

## Interconnect Bridge/Open UDFM Generation: processor\_core

If you want to perform Automotive-Grade ATPG targeting interconnect bridge and open defects, generate a UDFM for the interconnect bridge/open defects in your design. Then, read in the UDFM during Automotive-Grade ATPG.

### Note

 The line numbers used in this procedure refer to the command flow in “[Dofile Example for Interconnect Bridge/Open UDFM Generation, processor\\_core](#)” on page 419. Refer to the following directory, which has a usage example described in this section:

```
tessent_automotive_reference_flow_<software_version>/wrapped_cores/processor_core/  
11.extract_fault_sites
```

## Procedure

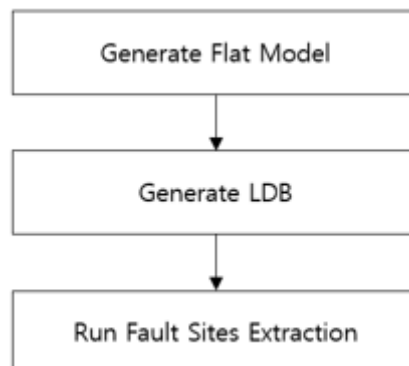
1. Load the post-layout design and generate a flat model. (See lines 1-19.)
2. Load the flat model. (See lines 21–23.)
3. Read in the layout data and generate an LDB. (See lines 25-34.)
4. Load the LDB by applying the `open_layout` command, and then apply the `extract_fault_sites` command with an output file name to generate a UDFM. (See lines 36-40.)

### Note

 Use “all” as the `defect_types` option for the `extract_fault_sites` command to write out both bridges and opens to the UDFM file.

## Examples

**Figure 7-9. Interconnect Bridge/Open UDFM Generation Flow for processor\_core**



The following dofile shows a command flow for generating a bridge/open UDFM for use in the Automotive-Grade ATPG flow.

### Example 7-6. Dofile Example for Interconnect Bridge/Open UDFM Generation, processor\_core

```
1 set_context patterns -scan
2 set design_name "processor_core"
3
```

```
4 # Read Tessent Library
5 read_cell_library \
6   ../.../library/NangateOpenCellLibrary_PDKv1_3_v2010_12/Front_End/DFT/
   NangateOpenCellLibrary.tcelllib
7
8 # Read in the post layout netlist
9 read_verilog ../9.pnr/pnr_db/golden/${design_name}.post.vg
10
11 #Read in the memory library model
12 read_cell_library ../.../library/memories/SYNC_1RW_8Kx16.atpglib
13 read_cell_library ../.../library/memories/SYNC_1RW_32x16_RC.atpg
14 set_design_sources -format tcd_memory -y ../.../library/memories/
   -extension tcd_memory
15
16 set_current_design $design_name
17
18 set_system_mode analysis
19
20 write_flat_model flat/${design_name}_post.flat -rep
21
22 set_context patterns -scan_diagnosis
23
24 read_flat_model flat/${design_name}_post.flat
25
26 set deffile [glob -directory ../9.pnr/pnr_db/golden ${design_name}.def]
27
28 set leffile [concat [glob -directory ../.../library/
   NangateOpenCellLibrary_PDKv1_3_v2010_12/Back_End/lef
   NangateOpenCellLibrary.macro.lef] \
29   [glob -directory ../.../library/
   NangateOpenCellLibrary_PDKv1_3_v2010_12/Back_End/lef
   NangateOpenCellLibrary.tech.lef] \
30   [glob -directory ../.../library/memories
   SYNC_1RW_8Kx16.lef] \
31   [glob -directory ../.../library/memories
   SYNC_1RW_32x16_RC.lef] \
32   ]
33
34 analyze_layout_hierarchy hier_db/${design_name}.hierdb -leflist $leffile
   -defflist $deffile -rep
35 create_layout ldb/${design_name}.ldb -rep -defflist $deffile -leflist
   $leffile
36
37 open_layout ldb/${design_name}.ldb
38
39 extract_fault_sites -output_file udfm/${design_name}_brdg_opn.udfm
   -defect_types all -rep
40
41 exit
```

## Cell-Neighborhood UDFM Generation: processor\_core

If you want to perform Automotive-Grade ATPG targeting cell-neighborhood defects, which may be located between cells, generate a UDFM for your design's cell-neighborhood defects. Use that UDFM during Automotive-Grade ATPG.



To generate a cell-neighborhood UDFM, extract cell-neighborhood pairs from the design LDB, then feed the cell pairs information into Tessent CellModelGen. Combine the UDFM files for all individual cell pairs into a single UDFM file. The required input data is the same as the data required for generating a cell-aware UDFM on standard cells. For information on Tessent CellModelGen or input requirements, refer to the *Tessent CellModelGen Tool Reference*.

### Note

 The line numbers used in this procedure refer to the command flow in “[Dofile Example for Extracting Cell-Neighborhood Pairs, processor\\_core](#)” on page 422. Refer to the following directory, which has a usage example described in this section:

```
tessent_automotive_reference_flow_<software_version>/wrapped_cores/processor_core/
11.extract_fault_sites
```

---

## Procedure

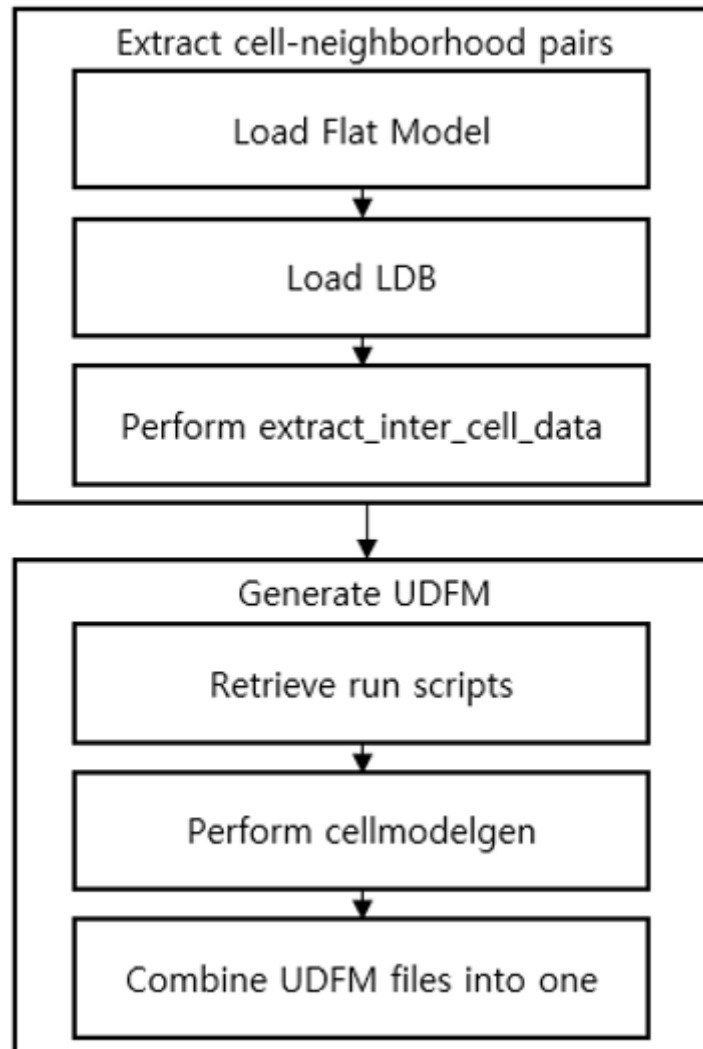
1. Extract cell-neighborhood pairs.
  - a. Load the flat model generated for interconnect bridge/open UDFM generation. (See lines 1-5.)
  - b. Load the LDB using the [open\\_layout](#) command, which is generated for interconnect bridge/open UDFM generation. (See line 7.)
  - c. Run the [extract\\_inter\\_cell\\_data](#) command, specifying an output file name to write the extracted cell-neighborhood pairs. (See line 9.)
2. Generate the UDFM using Tessent CellModelGen with the extracted cell-neighborhood pairs.
  - a. Before running Tessent CellModelGen, ensure that all the required input data is available and all the required tools are accessible.
  - b. Create an empty directory, and retrieve run scripts.
 

**cellmodelgen -get\_script all**
  - c. Copy the *run\_flow.merge* script into your working directory from the *lib/scripts* directory, which you get from step [b](#).
  - d. Modify the *run\_flow.merge* script for your design. Refer to the comments in the script for details. The output file from step [1](#), the extracted inter-cell pairs, is the input to the *run\_flow.merge* script.
  - e. Run the *run\_flow.merge* script to run Tessent CellModelGen with the cell-neighborhood pairs. Specify the filename of the extracted cell-neighborhood pairs and a working directory name.
  - f. Copy the *run\_export.merge* script into your working directory from the *lib/scripts* directory, which you get from step [b](#).

- g. Modify the *run\_export.merge* script for your working environment.
- h. Run the *run\_export.merge* script to combine all single UDFM files on each of the cell pairs into a single UDFM file.

## Examples

**Figure 7-10. Cell-Neighborhood UDFM Generation Flow for processor\_core**



The following dofile shows a command flow for generating a bridge/open UDFM for use in the Automotive-Grade ATPG flow.

### **Example 7-7. Dofile Example for Extracting Cell-Neighborhood Pairs, processor\_core**

```
1 set_context patterns -scan_diagnosis
2
3 set design_name "processor_core"
4
```

```
5 read_flat_model flat/${design_name}_post.flat
6
7 open_layout ldb/${design_name}.ldb
8
9 extract_inter_cell_data -output_file inter_cell_data/
  ${design_name}_cellpairs.data -rep
10
11 exit
```

# Automotive-Grade ATPG Pattern Generation: processor\_core

---

Generating automotive-grade ATPG patterns is similar to regular ATPG pattern generation. It needs several UDFM files for cell-internal defects, interconnect bridge or open defects, and cell-neighborhood defects.

Refer to [Interconnect Bridge/Open UDFM Generation: processor\\_core](#) for interconnect bridge/open UDFM generation on your design, and [Cell-Neighborhood UDFM Generation: processor\\_core](#) for cell-neighborhood UDFM generation for your design.

Refer to “[UDFM Generation for Cell-Aware ATPG](#)” on page 453 for details on how to generate the cell-aware UDFM for your technology library.

As you do for regular ATPG, you must generate a graybox model; then, you must perform ATPG twice: once for the core’s external mode and once for its internal mode. When you run ATPG in external mode or internal mode, you can run ATPG with toff runs by loading only the UDFM files you want to target during an ATPG run. Also, you can run ATPG all together by reading in all UDFM files at once.

|                                                                        |            |
|------------------------------------------------------------------------|------------|
| <b>Automotive-Grade Topoff ATPG Pattern Generation .....</b>           | <b>424</b> |
| <b>Automotive-Grade ATPG Pattern Generation Using Only UDFMs .....</b> | <b>428</b> |

## Automotive-Grade Topoff ATPG Pattern Generation

This procedure describes how to run cell-aware ATPG toff when you already have ATPG patterns from previous runs.

**Note**

 The line numbers used in this procedure refer to the command flow in “[Dofile Example for Interconnect Bridge/Open UDFM Generation, processor\\_core](#)” on page 419. Refer to the following directory, which has a usage example described in this section:

```
tessent_automotive_reference_flow_<software_version>/wrapped_cores/processor_core/  
12.generate_AGA_patterns
```

---

### Prerequisites

- Existing patterns from previous ATPG runs.
- Existing UDFM file(s) for your technology library and design.

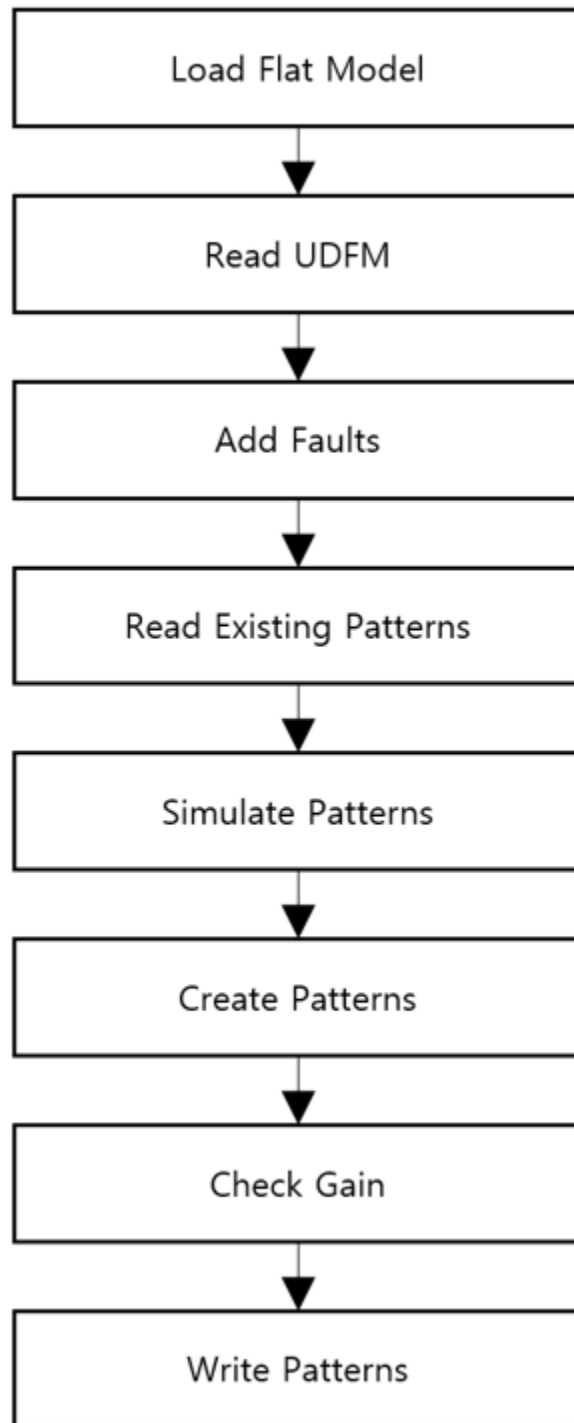
### Procedure

1. Load the flat model generated from a post-layout design. (See lines 1–11.)
2. Set a fault type, read in the UDFM file for your standard cells, and add faults. (See lines 14–18.)

3. Read patterns from previous ATPG runs. (See line 20.)
4. Simulate the patterns to see the baseline test coverage. (See line 21.)
5. Create patterns. (See line 28.)
6. Check how much benefit there is with the newly-created patterns (see line 31). Set a baseline to see the benefit after step 4. (See line 24.)
7. Write the newly-created patterns. (See lines 34–40.)
8. Check final ATPG test coverage. (See lines 47-51.)

## Examples

**Figure 7-11. ATPG Topoff Run Flow With a UDFM for processor\_core**



The following dofile shows a command flow for generating Automotive-Grade ATPG flow topoff patterns.

### Example 7-8. Dofile Example for Running Topoff ATPG With a UDFM in Internal Mode for processor\_core

```

1  set_context patterns -scan -design_id pnr
2  set design_name "processor_core"
3
4  # Specify the location of TSDB. Default is the current working directory
5  set_tsdb_output_directory ../tsdb_outdir
6
7  # Add more processors to run ATPG
8  add_processors localhost:4
9
10 # Read flat model
11 read_flat_model ../tsdb_outdir/logic_test_cores/
    ${design_name}_pnr.logic_test_core/${design_name}.atpg_mode_ATPG_int/
    ${design_name}_ATPG_int.flat.gz
12
13 # Cell-Aware Test Pattern Generation
14 set_fault_type udfm -static_fault
15 read_fault_sites \
16     ../../../../udfm_gen/stdlib/udfm/NangateOpenCellLibrary.udfm
17 report_fault_sites -undefined_cells
18 add_fault_sites -all
19 add_faults -all
20
21 read_patterns ../tsdb_outdir/logic_test_cores/
    ${design_name}_pnr.logic_test_core/${design_name}.atpg_mode_ATPG_int/
    ${design_name}_ATPG_int_stuck.patdb
22 simulate_patterns -source external
23
24 # Set baseline
25 report_udfm_statistics -set_baseline -group *OAI* *AOI* *OR* *AND* *INV*
    *BUF* *HA* *DFF* *MUX* *DLL*
26 report_statistic
27
28 # Generate patterns
29 create_patterns
30
31 # Check gain
32 report_udfm_statistics -group *OAI* *AOI* *OR* *AND* *INV* *BUF* *HA*
    *DFF* *MUX* *DLL*
33 report_statistic
34
35 write_patterns patterns/${design_name}_int_CAT_static_topoff.stil.gz
    -stil -replace
36 write_faults faults/${design_name}_int_CAT_static_topoff.faults.gz
    -replace
37
38 # Write Verilog patterns for simulation
39 write_patterns patterns/${design_name}_int_CAT_static_topoff_parallel.v
    -verilog -parallel -replace
40 set_pattern_filtering -sample_per_type 2
41 write_patterns patterns/${design_name}_int_CAT_static_topoff_serial.v
    -verilog -serial -replace
42
43 # To understand the coverage of the faults testable by Internal mode it is
44 # necessary to eliminate the undetected faults that would otherwise be

```

```
45 # detected in External mode. This is done by merging the fault list
46 # from running the graybox in External mode
47 read_faults faults/${design_name}_ext_CAT_static_topoff.faults.gz -merge
48
49 # Final coverage of the core that includes both Internal and External
50 # modes
51 report_stat -detail
52
53 exit
```

## Automotive-Grade ATPG Pattern Generation Using Only UDFMs

This procedure describes how to run cell-aware ATPG using only existing UDFMs.

### Note

 The line numbers used in this procedure refer to the command flow in “[Dofile Example for Running ATPG With Only UDFMs in Internal Mode for processor\\_core](#)” on page 429. Refer to the following directory, which has a usage example described in this section:

```
tessent_automotive_reference_flow_<software_version>/wrapped_cores/processor_core/
12.generate_AGA_patterns
```

---

## Prerequisites

- Existing UDFM file(s) for your technology library and design.

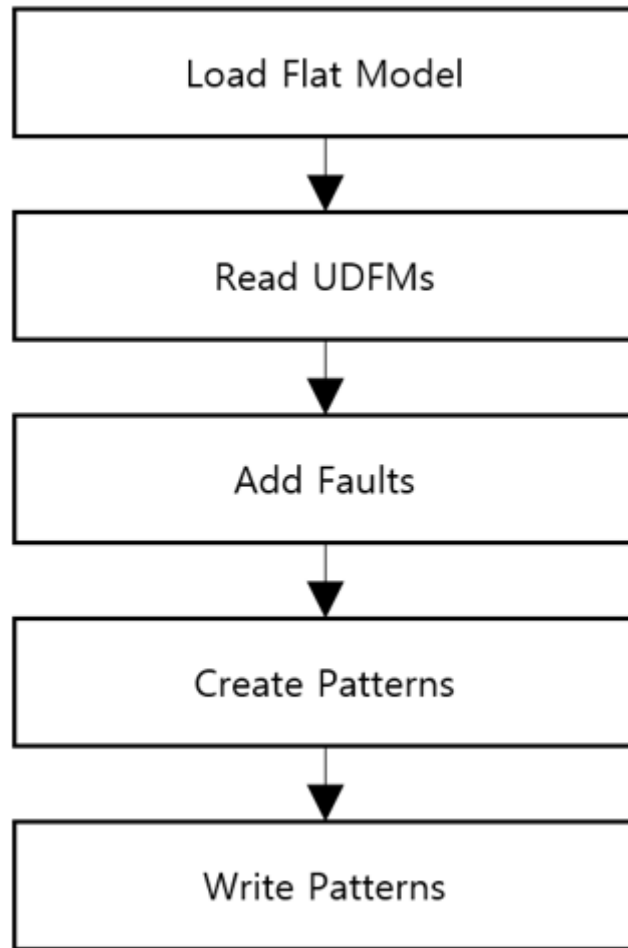
## Procedure

1. Load the flat model generated from a post-layout design. (See lines 1–11.)
2. Set a fault type and read in the UDFM files for your standard cells, interconnect bridge/open, and cell-neighborhood defects. Then add faults. (See lines 14-20.)
3. Create patterns. (See line 24.)
4. Write the patterns. (See line 27.)



## Examples

**Figure 7-12. ATPG Run Flow With All UDFM for processor\_core**



The following dofile shows a command flow for generating Automotive-Grade ATPG flow patterns using only UDFMs.

**Example 7-9. Dofile Example for Running ATPG With Only UDFMs in Internal Mode for processor\_core**

```

1  set_context patterns -scan -design_id pnr
2  set design_name "processor_core"
3
4  # Specify the location of TSDB. Default is the current working directory
5  set_tsdb_output_directory ../tsdb_outdir
6
7  # Add more processors to run ATPG
8  add_processors localhost:4
9
10 # Read flat model
11 read_flat_model ../tsdb_outdir/logic_test_cores/
    ${design_name}_pnr.logic_test_core/${design_name}.atpg_mode_ATPG_int/
    ${design_name}_ATPG_int.flat.gz
  
```

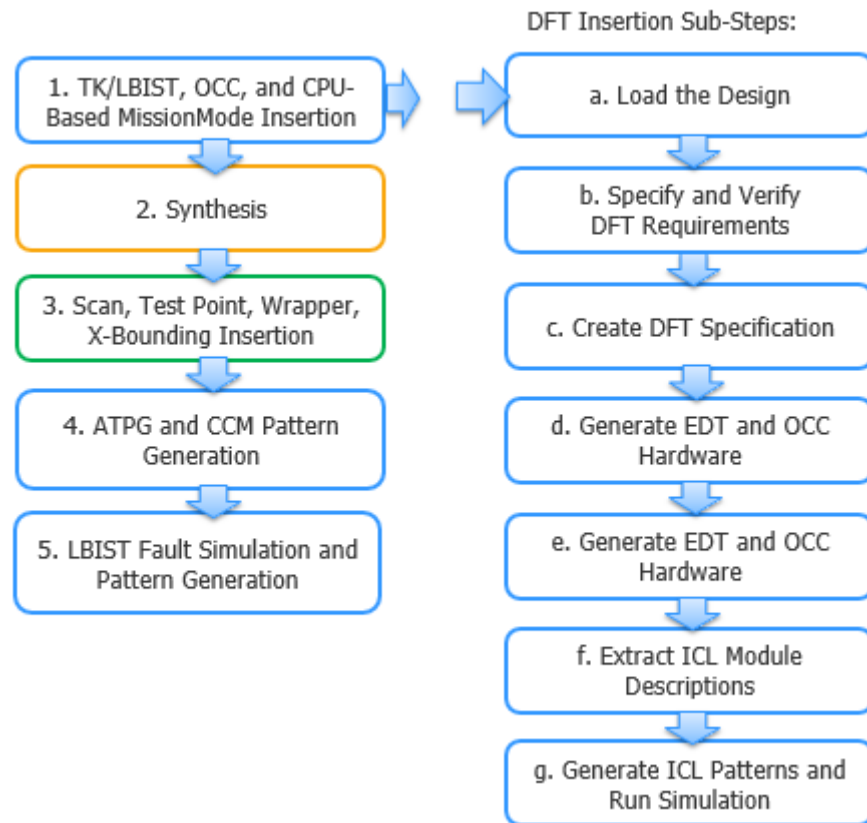
```
12
13 # Cell-Aware Test Pattern Generation
14 set_fault_type udfm -static_fault
15 read_fault_sites \
16   ../.../udfm_gen/stdlib/udfm/NangateOpenCellLibrary.udfm
17 read_fault_sites ../11.extract_fault_sites/udfm/
18   ${design_name}_brdg_opn.udfm
19 read_fault_sites ../11.extract_fault_sites/udfm/CELLS_neighbor.udfm
20 report_fault_sites -undefined_cells
21 add_fault_sites -all
22 add_faults -all
23 report_statistics -detail
24
25 # Generate patterns
26 create_patterns
27 report_statistics -detail
28
29 write_patterns patterns/${design_name}_int_AGA_static.stil.gz -stil -rep
30 write_faults faults/${design_name}_int_AGA_static.faults.gz -rep
31
32 # Write Verilog patterns for simulation
33 write_patterns patterns/${design_name}_int_AGA_static_parallel.v -verilog
34   -parallel -replace
35 set_pattern_filtering -sample_per_type 2
36 write_patterns patterns/${design_name}_int_AGA_static_serial.v -verilog
37   -serial -replace
38
39 # To understand the coverage of the faults testable by Internal mode it is
40   necessary to
41 # eliminate the undetected faults that would otherwise be detected in
42   External mode. This is done
43 # by merging the fault list from running the graybox in External mode
44 #read_faults -mode ATPG_ext -fault_type udfm -merge
45 read_faults faults/${design_name}_ext_AGA_static.faults.gz -merge
46
47 # Final coverage of the core that includes both Internal and External
48   modes
49 report_stat -detail
50
51 exit
```

## DFT Insertion Flow for the GPS Baseband Physical Block

The GPS baseband design does not include memory. Because MemoryBIST is not required, you can skip the first DFT insertion pass as described for the processor core. Instead, perform one DFT pass to insert the hybrid IP, OCC, and in-system test.

In addition to illustrating how to insert in-system test at the block level, the flow for this block shows you how to perform LogicBIST fault simulation when a design only has one clock, and, thus, no NCP decoder logic.

**Figure 7-13. DFT Insertion Flow for gps\_baseband**



This section highlights aspects of the flow for `gps_baseband` that differ from `processor_core`: DFT insertion and LogicBIST fault simulation. Refer to the test case for dofile details.

|                                                                                  |            |
|----------------------------------------------------------------------------------|------------|
| <b>DFT Insertion Pass With In-System Test: <code>gps_baseband</code> .....</b>   | <b>432</b> |
| <b>LogicBIST Fault Simulation With One NCP: <code>gps_baseband</code> .....</b>  | <b>436</b> |
| <b>Interconnect Bridge/Open UDFM Generation: <code>gps_baseband</code> .....</b> | <b>436</b> |
| <b>Cell-Neighborhood UDFM Generation: <code>gps_baseband</code> .....</b>        | <b>437</b> |
| <b>Automotive-Grade ATPG Pattern Generation: <code>gps_baseband</code> .....</b> | <b>437</b> |

## DFT Insertion Pass With In-System Test: gps\_baseband

This DFT insertion pass illustrates how to insert the in-system test controller at the core level. This could be a requirement for your design depending on the length of your IJTAG network. The tool generates a CPU-based controller, which is enabled from the system logic at the parent level.

---

### Note



The line numbers used in this procedure refer to the command flow dofile in [Example 7-10](#) on page 433 unless otherwise noted.

---

## Procedure

1. [Loading the Design](#). (See lines 1-8.)
2. Follow the steps described in “[Specifying and Verifying the DFT Requirements: DFT Signals for Wrapped Cores](#)” on page 216, ensuring that you also include the required DFT signals for the hybrid TK/LBIST flow and for controller chain mode. (See lines 10-35.)

Constrain the enable signal for the in-system test controller to 0 for manufacturing test. The tool emulates the control of this port during in-system test. (See lines 25-26.)

For the DFT signals, create a new port if a source node does not exist. The ports are created only if set\_design\_level is not chip.

3. [Creating the DFT Specification](#) with SIBs for EDT, OCC, and LogicBIST, and edit the default specification to insert the OCC, hybrid IP, and in-system test controllers. (See lines 37-138.)

As you did with the processor core, use the read\_config\_data command to edit the DFT specification.

- Specify the [OCC](#) wrapper and specify your clock intercept node for the OCC.

For details about OCC for the hybrid TK/LBIST flow, refer to “[Tessent OCC TK/LBIST Flow](#)” in the *Hybrid TK/LBIST Flow User’s Manual*.

- Specify the EDT wrapper to include a [LogicBistOptions](#) wrapper, and specify a MISR ratio of one.

The edt\_clock and edt\_update signals are automatically connected to EDT instances, and the EDT controller is built with bypass.

- Specify a [LogicBist](#) wrapper that contains both a Controller wrapper and an NcpIndexDecoder wrapper.

Because this core has a single clock, you only need one NCP; the tool does not generate the NCP index decoder.

- Specify an [InSystemTest](#) wrapper, ensuring that you specify that you are using the CPU interface.

Create a TCD segment for this instrument, and specify the in-system test controller connections using the Connections wrapper.

4. [Generating the EDT, Hybrid TK/LBIST, and OCC Hardware](#), plus the LogicBIST and in-system test hardware. (See line 139.)
5. [Extracting the ICL Module Description](#). (See line 141.)
6. [Generating ICL Patterns and Running Simulation](#). (See lines 143-151.)

## Examples

The following dofile example shows a typical automotive command flow for a core-level first DFT insertion. Modify the DFT specifications for your design requirements.

### Example 7-10. Dofile Example for DFT Insertion, gps\_baseband

```

1  # Load the design
2  set_context dft -rtl -design_id rtl1
3  set_tsdb_output_directory ../tsdb_outdir
4  read_verilog ../rtl/gps_baseband.v
5
6  set_current_design gps_baseband
7
8  set_design_level physical_block
9
10 # Add required DFT signals for logic test
11 add_dft_signals ltest_en
12 add_dft_signals scan_en edt_update test_clock
13     -source_node {SCAN_EN_w edt_update test_clock_w }
14 add_dft_signals edt_clock shift_capture_clock -create_from_other_signals
15
16 # Add DFT signals required for hierarchical DFT (external, internal modes)
17 add_dft_signals int_ltest_en ext_ltest_en int_mode ext_mode
18
19 # Add DFT signal for controller chain mode
20 add_dft_signals controller_chain_mode
21
22 # Add required DFT signals specific to hybrid TK/LBIST flow
23 add_dft_signals observe_test_point_en control_test_point_en x_bounding_en
24
25 # Constrain the in-system test controller to 0 for manufacturing test
26 add_input_constraints mission_test_enable -C0
27
28 set_dft_specification_requirements -logic_test on
29
30 add_clocks clk -period 3ns
31
32 # Enable Observation Scan Technology (OST) for capturing per shift and
    capture
33 add_dft_signals capture_per_cycle_static_en
34
35 check_design_rules

```

```

36
37 # Create and report the DFT specification
38 set spec [create_dft_specification -sri_sib_list {edt occ lbist} ]
39 report_config_data $spec
40
41 # Edit DFT specification to specify the SIB of the OCC and to insert OCC
42 # Modify for your design requirements
43 read_config_data -in $spec -from_string {
44     OCC {
45         ijtag_host_interface : Sib(occ);
46         static_clock_control : external;
47     }
48 }
49
50 # Specify OCC insertion and intercept node
51 # This is a generic method for populating the OCC; modify for your design
52 # Scan enable and shift capture clock signals are automatically connected
53 # to the OCC instances
54 set id_clk_list [list \
55 clk clk\
56 ]
57 foreach {id clk} $id_clk_list {
58 set occ [add_config_element OCC/Controller($id) -in $spec]
59 set_config_value clock_intercept_node -in $occ $clk
60 }
61
62 # Specify the hybrid EDT configuration
63 read_config_data -in $spec -from_string {
64     EDT {
65         ijtag_host_interface : Sib(edt);
66         Controller (c1) {
67             longest_chain_range : 50, 65 ;
68             scan_chain_count : 60;
69             input_channel_count : 2;
70             output_channel_count : 2;
71             LogicBistOptions {
72                 misr_input_ratio : 1 ;
73                 ShiftPowerOptions {
74                     present : on ;
75                     default_operation : disabled ;
76                     SwitchingThresholdPercentage {
77                         min : 25 ;
78                     }
79                 }
80             }
81         }
82     }
83 }
84
85 # Specify the LogicBIST controller with NCP index decoder
86 read_config_data -in $spec -from_string {
87     LogicBist {
88         ijtag_host_interface : Sib(lbist);
89         Controller(1%ctrl_lbist) {
90             burn_in : on ;
91             pre_post_shift_dead_cycles : 8 ;
92             SingleChainForDiagnosis { Present : on ; }
93             ControllerChain {

```

```

94         present : on;
95         clock : tck;
96     }
97     Connections {
98         scan_en_in : SCAN_EN_w ;
99         controller_chain_scan_in : ccm_scan_in ;
100        controller_chain_scan_out : ccm_scan_out ;
101    }
102    Interface {
103        shift_clock_src : - ;
104    }
105    NcpOptions {
106        count : 1 ;
107    }
108    ShiftCycles { max : 200 ; }
109    CaptureCycles { max : 7 ; }
110    PatternCount { max : 1024 ; }
111    WarmupPatternCount { max : 512 ; }
112 }
113
114# Specify the in-system test controller
115read_config_data -in $spec -from_string {
116    InSystemTest {
117        Controller(c0) {
118            protocol : cpu_interface ;
119            host_interface : HostScanInterface(ijtag) ;
120            data_width : 8 ;
121            ControllerChain {
122                present : on ;
123                clock : tck ;
124                segment_per_instrument : on ;
125            }
126            Connections {
127                reset : hw_rstn; //reset of gps_baseband core
128                CpuInterface {
129                    clock : cpu_interface_clock ;
130                    enable : mission_test_enable ;
131                    write_en : from_cpu_write_en ;
132                    data_in : data_in;
133                    data_out : data_out;
134                }
135            }
136        }
137    }
138}
139process_dft_specification
140
141extract_icl
142
143# Write script for design compilation
144write_design_import_script for_dc_synthesis.tcl -replace
145
146set pat_spec [ create_patterns_specification ]
147process_patterns_specification
148
149run_testbench_simulations
150# If simulation fails use the command below to see which pattern failed
151check_testbench_simulations

```

```
152
153exit
```

## LogicBIST Fault Simulation With One NCP: gps\_baseband

The GPS baseband core contains one clock, and, therefore, one NCP. There is no need for NCP decoder logic so Tessent Shell does not generate the NCP index decoder during IP insertion. This means that you must explicitly specify the clock sequence during LogicBIST fault simulation.

The following dofile extract invokes the clock sequence during LogicBIST fault simulation. The sequence is generated by a TDR, and the static control clock is set to “both” during IP insertion.

### Example 7-11. Dofile Example for LogicBIST Fault Simulation With One NCP

```
...

set_core_instance_parameters -instrument_type occ -parameter_values \
{static_clock_control internal clock_sequence 3}

add_bist_capture_range SINGLE_OCC_NCP 0 255

set_lbist_controller_options -programmable_ncp_list { SINGLE_OCC_NCP }

set_system_mode analysis

create_capture_procedures -name SINGLE_OCC_NCP \
-clock_sequence ([ get_name_list [ get_clock \
gps_baseband_rtl1_tessent_occ_clk_inst/ \
tessent_persistent_cell_clock_out_mux/y ]]) \
([ get_name_list [ get_clock \
gps_baseband_rtl1_tessent_occ_clk_inst/ \
tessent_persistent_cell_clock_out_mux/y ]])

...
```

## Interconnect Bridge/Open UDFM Generation: gps\_baseband

Generate an interconnect bridge and open UDFM on the gps\_baseband module in the same way as you did for the processor\_core module.

Refer to “[Interconnect Bridge/Open UDFM Generation: processor\\_core](#)” on page 418 to see how to generate an interconnect bridge/open UDFM on a wrapped core.



Refer to the following directory, which has an example on the `gps_baseband` module:

```
tessent_automotive_reference_flow_<software_version>/wrapped_cores/gps_baseband/  
10.extract_fault_sites
```

## Cell-Neighborhood UDFM Generation: `gps_baseband`

Generate a cell-neighborhood UDFM on the `gps_baseband` module in the same way as the `processor_core` module.

Refer to “[Cell-Neighborhood UDFM Generation: `processor\_core`](#)” on page 420 to see how to generate a cell-neighborhood UDFM on a wrapped core.

Refer to the following directory, which has an example on the `gps_baseband` module:

```
tessent_automotive_reference_flow_<software_version>/wrapped_cores/gps_baseband/  
10.extract_fault_sites
```

## Automotive-Grade ATPG Pattern Generation: `gps_baseband`

Generate Automotive-Grade ATPG tophoff and UDFM patterns on the `gps_baseband` module in the same way as you did for the `processor_core` module.

Refer to “[Automotive-Grade ATPG Pattern Generation: `processor\_core`](#)” on page 424 to see how to generate ATPG patterns on a wrapped core.

Refer to the following directory, which has an example on the `gps_baseband` module:

```
tessent_automotive_reference_flow_<software_version>/wrapped_cores/gps_baseband/  
11.generate_AGA_patterns
```

# Top-Level DFT Insertion for the Automotive Flow

---

You previously inserted CPU-based in-system test controllers inside the GPS baseband cores. At the top level, you now insert a DMA in-system test controller and associated memory block. In addition to a Verilog testbench, this controller requires memory or lookup table (LUT)-based logic to save both the controller instructions and the pattern data.

Typically, you use ROM for the DMA memory block, but you can use any storage mechanism as long as you configure it to behave like a clocked synchronous memory. You can perform MemoryBIST testing on this memory during manufacturing. For testing the memory in system, the process for applying power-on reset tests during initialization differs if you are using ROM, RAM, or LUT-based logic.

For general information about the RTL and scan DFT insertion flow for the top chip, refer to “[RTL and Scan DFT Insertion Flow for the Top Chip](#)” on page 228. (The documented process relates to a hierarchical test case; however, the basic flow remains the same.)

|                                                                                         |            |
|-----------------------------------------------------------------------------------------|------------|
| <b>First DFT Insertion Pass: Top with MemoryBIST, BISR, and Boundary Scan . . . . .</b> | <b>438</b> |
| <b>Second DFT Insertion Pass: Top with EDT, OCC, and In-System Test . . . . .</b>       | <b>444</b> |
| <b>Scan Insertion for the Top Design . . . . .</b>                                      | <b>449</b> |
| <b>ATPG Pattern Generation for the Top Design . . . . .</b>                             | <b>451</b> |
| <b>ATPG Pattern Retargeting for the Top Design . . . . .</b>                            | <b>452</b> |
| <b>Interconnect Bridge/Open UDFM Generation for the Top Design . . . . .</b>            | <b>452</b> |
| <b>Cell-Neighborhood UDFM Generation for the Top Design. . . . .</b>                    | <b>453</b> |
| <b>Automotive-Grade ATPG Pattern Generation for the Top Design . . . . .</b>            | <b>453</b> |
| <b>UDFM Generation for Cell-Aware ATPG . . . . .</b>                                    | <b>453</b> |
| <b>TCA Based Pattern Sorting . . . . .</b>                                              | <b>456</b> |

## First DFT Insertion Pass: Top with MemoryBIST, BISR, and Boundary Scan

The top design includes I/O pads. Insert boundary scan plus MemoryBIST for any memories that are present at the top level; this includes the memory block for the DMA IST controller. Build the IJTAG network for the wrapped cores at the top level. In addition, the design includes repairable memories in the processor core, so you must insert an efuse with an efuse interface and a BISR controller.

If you do not plan to implement hard repair, use the Tessent soft-repair only option.

---

**Note**

 You must perform the first DFT insertion pass, even if you do not use MemoryBIST or boundary scan to insert IJTAG to configure your child blocks. You cannot run logic test DRC in this first pass. Logic test DRC requires that the child blocks are connected to the network in order to be configured. Because of this, you can only run logic test DRC on the second DFT insertion pass, after the network is configured.

---

---

**Note**

 For general instructions about inserting MemoryBIST and boundary scan at the top level, refer to [“First DFT Insertion Pass: Performing Top-Level MemoryBIST and Boundary Scan”](#) on page 231. This test case adds in the BISR and DMA memory block to the baseline use case.

---

---

**Note**

 Because you cannot use the in-system test patterns from the DMA memory block to test that memory block, do not create in-system test patterns for testing the DMA memory.

---

If you plan to insert a LogicBIST controller at the same level as the DMA IST controller to generate in-system test logic test patterns (not illustrated by this test case), ensure that the DMA IST controller and its memory block have the same async clock source. In addition:

- Isolate the DMA memory block, its MemoryBIST logic, and the IST controller by pushing them into their own module.
- If isolation is not possible because of design considerations, exclude the observation logic inside the memory interface by disabling the `observation_xor_size` property. (The MemoryBIST controller and MemoryBIST interface are scannable logic.) This prevents a MISR signature difference between manufacturing and in-system LogicBIST tests. For example:

```
set_config_value DftSpecification(processor_core,rtl1)/MemoryBist/  
Controller(c2)/Step/MemoryInterface(m1)/observation_xor_size off
```

In addition, disable the memory library `DisableDuringScan` property, which removes both the gating logic at the memory inputs and the memory bypass mux that disables the memory control during scan and LogicBIST test. As needed, re-enable the memory bypass mux by setting the `scan_bypass_logic` property to `sync_mux`, as follows:

```
set_config_value DftSpecification(processor_core,rtl1)/MemoryBist/  
Controller(c2)/Step/MemoryInterface(m1)/scan_bypass_logic sync_mux
```

The memory bypass mux is supported in this scenario as long as you are directly connecting the memory data output to the IST controller data input.

**Note**

The line numbers used in this procedure refer to the command flow dofile in [Example 7-12](#) on page 441 unless otherwise noted.

---

**Prerequisites**

- To insert boundary scan, you must have an RTL design with instantiated I/O pads if you are using a chip-level design.
- For RTL netlists, you must have a Tessent cell library or the pad library.

**Procedure**

1. Load the design, including opening the TSDBs for the child cores: processor\_core and gsp\_baseband. (See lines 1-25.)

In addition, load the design for the DMA memory block, fusebox, and fusebox interface. The dofile example shows an example fusebox. You must include an efuse and efuse interface in your design unless you are opting for soft repair only.

2. Specify the DFT specification requirements. (See line 27.)

When you set the design level to chip, “-memory\_test on” automatically initiates BISR insertion if BISR chains exist on blocks instantiated in the current design or repairable memories exist in the current design. This assumes that you have not changed the [set\\_dft\\_specification\\_requirements](#) -memory\_bist, -memory\_bisr\_chains, and -memory\_bisr\_controller options from their default auto settings.

3. Identify TAP pins and pins that cannot add boundary scan cells. (See lines 29-41.)
4. Add DFT signals and clocks. (See lines 43-50.)
5. Create the connections for the CPU-based and DMA IST controllers. (See lines 52-68.)

You must connect the CPU-based IST controllers that you inserted at the block level so that they can be controlled by the system logic. In addition, create the connection between the DMA memory block and the DMA IST controller clock.

You can use a post-insertion script to connect the system logic with the IST controller.

6. Check the design rules and create the DFT specification. (See lines 70-76.)
7. Segment the boundary scan to be used during logic test. (See lines 78-79.)
8. Create the BISR controller connections for the clock, VDD, and fusebox. (See lines 81-91.)

To connect the BISR controller to the system logic, you must specify the connection for the BISR controller to use for repair clock, BISR reset, and VDD. Typically, system logic is connected to the BISR controller for initiating memory repair and monitoring the progress of the operation.

The BISR controller input clock must be driven by an appropriate functional clock. Specify the connection with the `repair_clock_connection` property in the `DftSpecification/MemoryBisr/Controller` wrapper. The BISR controller input signal resets the BISR chains and initiates memory repair. Specify the reset signal with the `repair_trigger_connection` property in the `DftSpecification/MemoryBisr/Controller` wrapper. Also, specify your fusebox module and its location.

9. Identify the functional pins to be shared with EDT channel pins and add the required auxiliary I/O logic. (See lines 93-100.)
10. Generate the hardware and extract ICL. (See lines 104-106.)
11. Generate ICL patterns and simulation testbenches for the controllers and instruments in the current and lower physical instances. (See lines 108-115.)

To re-run the lower physical instance simulations at higher levels, enable the `set_default_values simulate_instruments_in_lower_physical_instances` property.

12. Run and check testbench simulations. (See lines 117-130.)

## Examples

The following dofile example shows a command dofile for the top-level first DFT insertion pass for the automotive flow.

### Example 7-12. Dofile Example for Top-Level First DFT Insertion Pass, Automotive Flow

```

1  set_context dft -rtl -design_id rtl1
2  set_tsdb_output_directory ../tsdb_outdir
3  open_tsdb ../../wrapped_cores/processor_core/tsdb_outdir
4  open_tsdb ../../wrapped_cores/gps_baseband/tsdb_outdir
5  read_cell_library ../../library/standard_cells/tessent/adk.tcelllib
6  read_verilog ../rtl/noncore_blocks/pll.v -blackbox
7
8  set_design_sources -format verilog \
9      -v ../rtl/noncore_blocks/pad8_io_macro.v \
10     -v ../rtl/noncore_blocks/iopad_sel.v \
11     ../rtl/noncore_blocks/iopad.v
12
13 # Read the memory block for the DMA IST controller too
14 read_verilog ../../library/memories/SYNC_1R1W_4096x8.v \
15     -interface_only -exclude_from_file_dictionary
16
17 read_verilog ../rtl/chip_top.v
18
19 # Read fusebox, example only
20 read_verilog fusebox/LVFuseBox.vb -exclude
21 read_verilog fusebox/genericFuseBox.vb
22 read_core_description fusebox/genericFuseBox.tcd_fbox
23
24 set_current_design chip_top
25 set_design_level chip
26

```

```
27 set_dft_specification_requirements -boundary_scan on -memory_test on
28
29 # Specify the TAP pins
30 set_attribute_value TCK -name function -value tck
31 set_attribute_value TDI -name function -value tdi
32 set_attribute_value TMS -name function -value tms
33 set_attribute_value TRST -name function -value trst
34 set_attribute_value TDO -name function -value tdo
35
36 # Specify pins that cannot add any boundary scan cells
37 set_boundary_scan_port_options TEST_CLOCK_top -cell_options dont_touch
38 set_boundary_scan_port_options EDT_UPDATE_top -cell_options dont_touch
39 set_boundary_scan_port_options SCAN_ENABLE -cell_options dont_touch
40 set_boundary_scan_port_options RESET_N -cell_options dont_touch
41 set_boundary_scan_port_options INCLK -cell_options dont_touch
42
43 # Add DFT signals
44 add_dft_signals scan_en -source_nodes SCAN_ENABLE
45
46 # Specify the clocks
47 add_clocks PLL_1/pll_clock_0 -reference REF_CLK -freq_multiplier 16
48 add_clocks REF_CLK -period 48
49 add_clocks TEST_CLOCK_top -period 10
50 add_clocks INCLK -period 10ns
51
52 # Create connections for CPU-based IST controller inserted in gps_baseband
53 proc process_dft_specification.post_insertion {topWrapper args} {
54 create_connection RDS_1/mission_test_enable_gps GPS_1/mission_test_enable
55 create_connection RDS_2/mission_test_enable_gps GPS_2/mission_test_enable
56 create_connection istb/write_en GPS_1/from_cpu_write_en
57 create_connection istb/write_en GPS_2/from_cpu_write_en
58 create_connection istb/clock GPS_1/cpu_interface_clock
59 create_connection istb/clock GPS_2/cpu_interface_clock
60 create_connection istb/data_out GPS_1/data_in
61 create_connection istb/data_out GPS_2/data_in
62 create_connection GPS_1/data_out istb/data_in1
63 create_connection GPS_2/data_out istb/data_in2
64
65 # Create connection for DMA IST controller
66 create_connection IST_memory/CLK
67 chip_top_rtl1_tessent_in_system_test_post_inst/dma_clock
68 }
69
70 check_design_rules
71 set_system_mode analysis
72
73 report_clocks
74
75 set_spec [create_dft_specification]
76 report_config_data $spec
77
78 # Segment the boundary scan to be used during logic test
79 set_config_value $spec/BoundaryScan/max_segment_length_for_logictest 80
80
81 # Create BISR controller connection for clock, VDD and fusebox
82 set_config_value -in $spec MemoryBisr/Controller/
83   repair_clock_connection PLL_1/pll_clock_0
84 set_config_value -in $spec
```

```

85 MemoryBisr/Controller/programming_voltage_source VDD/C
86 set_config_value -in $spec MemoryBisr/Controller/
87   repair_trigger_connection BISR_RESET/C
88 set_config_value -in $spec MemoryBisr/Controller/fuse_box_location
89   external
90 set_config_value -in $spec MemoryBisr/Controller/
91   fuse_box_interface_module genericFuseBox
92
93 # Add auxiliary MUX on the inputs and outputs used for EDT channel pins
94 read_config_data -in ${spec}/BoundaryScan -from_string {
95   AuxiliaryInputOutputPorts {
96     auxiliary_input_ports   : GPIO1_0, GPIO1_1, GPIO1_2, GPIO1_3;
97     auxiliary_output_ports  : GPIO2_0, GPIO2_1, GPIO2_2, GPIO2_3, GPIO1_0,
98     GPIO1_1 ;
99   }
100}
101
102report_config_data $spec
103
104process_dft_specification
105
106extract_icl
107
108# Include the lower-level physical instances for IJTAG retargeting
109set_defaults_value PatternsSpecification/SignOffOptions/
110   simulate_instruments_in_lower_physical_instances on
111
112# Create pattern testbenches to verify the inserted DFT logic
113set pat_spec [create_patterns_specification]
114
115process_pattern_specification
116
117set_simulation_library_sources -v ../../library/standard_cells/verilog/ \
118*.v
119-v ../rtl/noncore_blocks/p11.v \
120-v ../rtl/noncore_blocks/pad8_io_macro.v \
121-v ../rtl/noncore_blocks/iopad_sel.v \
122-v ../rtl/noncore_blocks/iopad.v \
123-v ../../library/memories/SYNC_1RW_8Kx16.v \
124-v ../../library/memories/SYNC_1RW_32x16_RC.v \
125-v ../../library/memories/SYNC_1R1W_4096x8.v \
126-v ../../library/standard_cells/verilog/adk.v \
127-v fusebox/LVFFuseBox.vb
128
129run_testbench_simulations -exclude {InSystemTest_0_MemoryBist_P1}
130check_testbench_simulations -report_status
131
132exit

```

## Second DFT Insertion Pass: Top with EDT, OCC, and In-System Test

For the top level, the second DFT insertion pass for EDT and OCC include gray boxes, DFT signals for purposes of ATPG retargeting, and TAMs. In addition, for the automotive flow, you insert the DMA IST controller.

---

### Note



For top-level EDT and OCC, this procedure follows a typical second DFT insertion flow for a top design as detailed in “[Second DFT Insertion Pass: Inserting Top-Level EDT and OCC](#)” on page 234. Refer to this section for associated details.

---

---

### Note



The line numbers used in this procedure refer to the command flow dofile in [Example 7-13](#) on page 445 unless otherwise noted.

---

## Procedure

1. Load the design. (See lines 1-14.)

Ensure that you load in the TCD for the DMA memory block that is used for in-system test. (See line 10.)

2. Add DFT signals. (See lines 16-30.)

Ensure that you include a DFT signal for controller chain mode and define retargeting modes for each group of wrapped cores whose ATPG patterns you want to retarget.

3. Apply the [add\\_dft\\_modal\\_connections](#) command to specify the TAM and to connect the EDT channel IOs of the wrapped cores to top-level pins via the TAM. (See lines 32-76.)

Include connections for controller chain mode. For the processor core, use `retargeting_model`, and for the GPS baseband, use `retargeting_mode2`.

With Tessent Shell you can share functional pins as EDT channel pins. The dofile shows that the input pin `GPI01_0` is shared as an EDT channel input pin, and the output pin `GPI02_0` is shared as an EDT channel output pin. Different modes can also share input and output pins. For example, `GPI01_2` functions as both the processor core EDT mode channel input and the controller chain mode uncompressed input for the entire design.

4. Set the DFT specification requirements. (See line 78.)
5. Create and process the DFT specification, using the `read_config_data` command to add the EDT controller with bypass and the DMA IST controller. (See lines 83-150.)

The top-level EDT controller includes bypass, and the `edt_clock` and `edt_update` signals are automatically connected to EDT instances. The `connect_bscan_segments_to_lsb_chains` property defaults to auto and connects the



divided boundary scan segments from the previous insertion pass to the top-level EDT controller.

Using the `set_config_value` command, connect the top-level EDT channels that you left unconnected during step 3 to the GPIO ports. (See lines 108-114.) The controller chain connections are completed during scan insertion.

Finally, insert the DMA IST controller. (See lines 116-150.) Set the protocol property to `direct_memory_access`. To account for ATPG coverage of the IST controller under controller chain mode, you can specify a TCD scan segment. Specify the data width, address width, and `max_test_program_count` that determine the bus width of the IST controller's program index, and thus the number of patterns that are programmed for the controller. In addition, specify the connections for the `DirectMemoryAccess` interface. The DMA IST controller inserts a comparator circuit and outputs a fail flag and done bit.

6. Generate ICL patterns and simulation testbenches for the instruments in the current and lower physical instances. (See lines 152-172.)

Create the in-system test patterns for the JTAG instruments you want to test. The dofile example illustrates applying the `MemoryBist_P1` pattern to test the memory inside the processor core. The resulting Verilog testbench file is named *FilePrefix\_TestProgramIndex\_PatternSetName.v*, and the memory file (for use with the DMA IST controller) is named *testbench\_file\_name.mem*.

7. Run and check testbench simulations. (See line 174-194.)

For in-system test, ensure that you point to the memory file that contains the control instructions for the testbench and the data necessary for interaction with the DMA IST controller. (See line 191.) This file has the same name as the testbench file with the addition of a `“.mem”` suffix. The testbench loads the memory file during simulation.

In addition, add three levels to the IST controller's actual location because the run occurs inside three levels in the *simulation\_outdir* directory.

After running the in-system test patterns, run the remaining testbench simulations for the top and the physical blocks.

## Examples

The following dofile example shows a command dofile for the top-level second DFT insertion pass for the automotive flow.

### Example 7-13. Dofile Example for Top-Level Second DFT Insertion Pass, Automotive Flow

```
1 set_context dft -rtl -design_id rtl2
2 set_tsdb_output_directory ../tsdb_outdir
3 open_tsdb ../../wrapped_cores/processor_core/tsdb_outdir
4 open_tsdb ../../wrapped_cores/gps_baseband/tsdb_outdir
5
6 read_design chip_top -design_id rtl1 -verbose
```

```
7 read_design processor_core -design_id gate1 -view graybox -verbose
8 read_design gps_baseband -design_id gate1 -view graybox -verbose
9
10 read_core_description ../../library/memories/SYNC_1R1W_4096x8.lvlib
11 read_verilog ../../library/memories/SYNC_1R1W_4096x8.v \
12     -interface_only exclude_from_file_dictionary
13
14 set_current_design chip_top
15
16 # Add DFT signals
17 add_dft_signals int_ltest_en output_pad_disable
18 add_dft_signals tck_occ_en
19 add_dft_signals ltest_en
20 add_dft_signals edt_mode
21 add_dft_signals controller_chain_mode
22
23 # These are used for top-level EDT
24 add_dft_signals test_clock edt_update \
25     -source_nodes {TEST_CLOCK_top EDT_UPDATE_top}
26 add_dft_signals shift_capture_clock edt_clock -create_from_other_signals
27
28 # Define retargeting modes
29 add_dft_signals retargeting1_mode retargeting2_mode retargeting3_mode \
30     retargeting4_mode
31
32 # TAM connections
33 add_dft_modal_connections -ports GPIO1_0 \
34     -input_data_destination_nodes * -enable_dft_signal edt_mode
35 add_dft_modal_connections -ports GPIO2_0 \
36     -output_data_source_nodes * -enable_dft_signal edt_mode
37 add_dft_modal_connections -ports GPIO1_0 -input_data_destination_nodes \
38     * -enable_dft_signal controller_chain_mode
39 add_dft_modal_connections -ports GPIO2_0 -output_data_source_nodes \
40     * -enable_dft_signal controller_chain_mode
41
42 # For processor_core
43 add_dft_modal_connections -ports GPIO1_2 -input_data_destination_nodes \
44     PROCESSOR_1/processor_core_rtl2_controller_c1_edt_channels_in[0] \
45     -enable_dft_signal retargeting1_mode
46 add_dft_modal_connections -ports GPIO2_2 -output_data_source_nodes \
47     PROCESSOR_1/processor_core_rtl2_controller_c1_edt_channels_out[0] \
48     -enable_dft_signal retargeting1_mode
49
50 # For gps_baseband
51 add_dft_modal_connections -ports GPIO1_2 -input_data_destination_nodes \
52     GPS_1/gps_baseband_rtl1_controller_c1_edt_channels_in[0] \
53     -enable_dft_signal retargeting2_mode
54 add_dft_modal_connections -ports GPIO1_2 -input_data_destination_nodes \
55     GPS_2/gps_baseband_rtl1_controller_c1_edt_channels_in[0] \
56     -enable_dft_signal retargeting2_mode
57 add_dft_modal_connections -ports GPIO1_3 -input_data_destination_nodes \
58     GPS_1/gps_baseband_rtl1_controller_c1_edt_channels_in[1] \
59     -enable_dft_signal retargeting2_mode
60 add_dft_modal_connections -ports GPIO1_3 -input_data_destination_nodes \
61     GPS_2/gps_baseband_rtl1_controller_c1_edt_channels_in[1] \
62     -enable_dft_signal retargeting2_mode
63 add_dft_modal_connections -ports GPIO1_0 -output_data_source_nodes \
64     GPS_1/gps_baseband_rtl1_controller_c1_edt_channels_out[0] \
```

```

65     -enable_dft_signal    retargeting2_mode -pipeline_stages 1
66 add_dft_modal_connections -ports GPIO1_1 -output_data_source_nodes \
67     GPS_1/gps_baseband_rt11_controller_c1_edt_channels_out[1] \
68     -enable_dft_signal    retargeting2_mode -pipeline_stages 1
69 add_dft_modal_connections -ports GPIO2_0 -output_data_source_nodes \
70     GPS_2/gps_baseband_rt11_controller_c1_edt_channels_out[0] \
71     -enable_dft_signal    retargeting2_mode -pipeline_stages 1
72 add_dft_modal_connections -ports GPIO2_1 -output_data_source_nodes \
73     GPS_2/gps_baseband_rt11_controller_c1_edt_channels_out[1]
74     -enable_dft_signal    retargeting2_mode -pipeline_stages 1
75
76 report_dft_modal_connections
77
78 set_dft_specification_requirements -logic_test on -memory_test on
79
80 set_system_mode analysis
81 report_dft_control_points
82
83 # Create DFT specification
84 set_spec [create_dft_specification -sri_sib_list {occ edt lbist} ]
85 report_config_data $spec
86 read_config_data -in $spec -from_string {
87     OCC {
88         ijtag_host_interface : Sib(occ);
89     }
90 }
91 set id_clk_list [list \
92     pll_clock_0 PLL_1/pll_clock_0 \
93     INCLK INCLK \
94 ]
95 foreach {id clk} $id_clk_list {
96 set occ [add_config_element OCC/Controller($id) -in $spec]
97 set_config_value clock_intercept_node -in $occ $clk
98 }
99 report_config_data $spec
100
101 read_config_data -in $spec -from_string {
102     EDT {
103         ijtag_host_interface : Sib(edt);
104         Controller (c1) {
105             ...
106 }
107
108 # Connect the EDT channels to the GPIO ports
109 set_config_value port_pin_name \
110     -in $spec/EDT/Controller(c1)/Connections/EdtChannelsIn(1) \
111     [get_auxiliary_pins GPIO1_0 -direction input]]
112 set_config_value port_pin_name \
113     -in $spec/EDT/Controller(c1)/Connections/EdtChannelsOut(1) \
114     [get_single_name [get_auxiliary_pins GPIO2_0 -direction output] ]
115
116 # Insert DMA IST controller
117 read_config_data -in $spec -from_string {
118     InSystemTest {
119         Controller(post) {
120             DesignInstance (.) {}
121             // host_interface: HostScanInterface(tap);
122             data_width : 8 ;

```

```

123     protocol : direct_memory_access ;
124     ControllerChain {
125         present : on ;
126         clock: tck;
127         segment_per_instrument: on;
128     }
129     DirectMemoryAccessOptions {
130         max_wait_cycles: 6400;
131         address_width: 12;
132         max_test_program_count: 2;
133     }
134     Connections {
135         reset: RESET_N;
136         DirectMemoryAccess {
137             clock: PLL_1/pll_clock_0_100;    // free-running clock
138             enable: istb/enable;
139             test_program_index: istb/program_index;
140             mem_data: IST_memory/Q[%d];
141             mem_address: DMA_A/Y[%d];
142             fail_flag: istb/fail_flag ;
143             test_program_done: istb/test_done;
144         }
145     }
146 }
147 }
148}
149
150process_dft_specification
151
152extract_icl
153
154write_design_import_script  for_dc_synthesis.tcl -replace
155
156# Include the lower-level physical instances for IJTAG retargeting
157set_defaults_value PatternsSpecification/SignOffOptions/
158     simulate_instruments_in_lower_physical_instances on
159
160set pat_spec [create_patterns_specification]
161
162# Create in-system test pattern for core-level memory
163add_core_instance -module [ get_name [get_icl_module *post] ]
164read_config_data -replace -in $pat_spec -from_string {
165     InSystemTest {
166         Controller(chip_top_rtl1_tessent_in_system_test_post_inst) {
167             TestProgram(0) { pattern : MemoryBist_P1 ; }
168         }
169     }
170}
171
172process_pattern_specification
173
174set_simulation_library_sources
175     -v ../../library/standard_cells/verilog/*.v
176-v ../rtl/noncore_blocks/pll.v \
177-v ../rtl/noncore_blocks/pad8_io_macro.v \
178-v ../rtl/noncore_blocks/iopad_sel.v \
179-v ../rtl/noncore_blocks/iopad.v \
180-v ../../library/memories/SYNC_1RW_8Kx16.v \

```

```
181-v ../../library/memories/SYNC_1RW_32x16_RC.v \  
182-v ../../library/memories/SYNC_1R1W_4096x8.v \  
183-v ../../library/standard_cells/verilog/adk.v \  
184-v fusebox/LVFuseBox.vb  
185  
186# Add three levels to the actual location because the run occurs inside  
187# three levels in the simulation_outdir/  
188run_testbench_simulations -simulator_option  
189    { +nowarnTSCALE -voptargs="+acc"  
190      +IST_INIT_MEM=../../../../../tsdb_outdir/patterns/  
191        chip_top_rtl2.patterns_signoff/InSystemTest.mem }  
192    -select InSystemTest_0_MemoryBist_P1  
193  
194run_testbench_simulations -exclude {InSystemTest_0_MemoryBist_P1 }  
195  
196exit
```

## Scan Insertion for the Top Design

After synthesizing your design, stitch the scan logic at the top-level along with the wrapper chains (external mode) of the cores. During scan insertion, the non-scan instances such as EDT are automatically understood. In addition, the built-in OCC sub-chains are understood and stitched into scan chains. The TCD segments that were created for the hybrid IP and IST controller test logic must be stitched together within the controller chain mode so that you can test them during ATPG.

Tessent uses the DFT signals that you specified during the previous insertion passes, therefore you do not need to specify them again.

---

### Note

 Tessent Shell works with any third-party scan insertion or other tools. However, because all Tessent products reside on the same code and database, you lose some automation with third-party tools.

---

---

### Note

 The line numbers used in this procedure refer to the command flow dofile in [Example 7-14](#) on page 450 unless otherwise noted.

---

## Procedure

1. Load the design. (See lines 1-12.)

Ensure that you specify a unique design ID, and that you open the TSDB for all the child cores.

2. Exclude some design objects from the scan insertion process. (See lines 17-19.)

Exclude the pipeline stages that you added using `add_dft_modal_connections` and the fusebox instance.

3. Constrain the BISR reset to constant 1 (c1). (See line 21.)
4. Create EDT and controller chain mode scan modes. (See lines 25-55.)

Create an EDT scan mode to connect the scan chains to the EDT signals and EDT hardware that you inserted during the second DFT insertion pass.

Create a top-level controller chain mode. To do this, you must first set the `active_child_scan_mode` attribute value, which enables you to select the active child scan mode of a multi-mode core and build a single chain for all controller chain mode IPs in all cores and also the top-level IST controller. The IST controller must be tested in controller chain mode.

When you specify `add_chain_mode` for the controller chain mode, include controller chain mode segments from the processor core, GPS baseband cores and the top elements. To reuse the I/O pins you created with `add_dft_modal_connections` during the second DFT insertion pass, specify the auxiliary pin SI/SO connections. For controller chain mode, you must specify the SI/SO-associated ports so the tool recognizes that the chains are uncompressed.

5. Perform scan insertion. (See lines 57-61.)

## Examples

The following dofile example shows a command dofile for top-level scan insertion for the automotive flow.

**Figure 7-14. Dofile Example for Top-Level Scan Chain Insertion, Automotive Flow**

```
1 # Load the design
2 set_context dft -scan -design_id gate3
3 open_tsdb ../../wrapped_cores/processor_core/tsdb_outdir
4 open_tsdb ../../wrapped_cores/gps_baseband/tsdb_outdir
5 read_cell_library ../../library/standard_cells/tessent/adk.tcelllib
6 read_verilog ../3.synthesize_rtl/chip_top_synthesized.vg
7
8 read_design chip_top -design_id rt12 -no_hdl -verbose
9 read_design processor_core -design_id gate1 -verbose
10 read_design gps_baseband -design_id gate1 -view graybox -verbose
11
12 set_current_design chip_top
13
14 report_clocks
15 report_static_dft_signal_settings
16
17 # Specify the non-scan instances
18 add_nonscan_instances *tessent_dft_modal_*
19 add_nonscan_instances -instances fbi_instance
20
21 add_input_constraints BISR_RSTN_PAD -c1
```

```
22
23 set_system_mode analysis
24
25 # Create a scan mode to connect scan chains to EDT
26 set_edt_instance [get_name_list [get_instance -of_module [get_name
27     [get_icl_module -of_instances chip_top* -filter
28     tessent_instrument_type==mentor::edt]]] ]
29
30 add_scan_mode edt_mode -edt_instance $edt_instance
31
32 # Start creating a scan mode for controller_chain_mode
33 set_attribute_value [get_instance -of_module *in_system_test*] -name
34     active_child_scan_mode -value controller_chain_mode
35 set_attribute_value [get_instance PROCESSOR_1] -name
36     active_child_scan_mode -value controller_chain_mode
37 set_attribute_value [get_instance GPS_1] -name active_child_scan_mode
38     -value controller_chain_mode
39 set_attribute_value [get_instance GPS_2] -name active_child_scan_mode
40     -value controller_chain_mode
41
42 # Create a scan chain family for elements at the top
43 create_scan_chain_family ccm_mm_top -include_elements
44     [get_scan_elements -of_child_scan_mode controller_chain_mode]
45
46 # Create the ccm scan mode and include elements at core and top
47 add_scan_mode controller_chain_mode -include_chain_families
48     {ccm_mm_top}
49     -include_elements{controller_chain_mode/PROCESSOR_1/ccm_scan_out
50     controller_chain_mode/GPS_1/ccm_scan_out
51     controller_chain_mode/GPS_2/ccm_scan_out} -si_connections
52     [get_single_name [get_auxiliary_pins GPIO1_2 -direction input]]
53     -so_connection [get_single_name [get_auxiliary_pins GPIO2_2
54     -direction output] ] si_associated_ports GPIO1_2
55     -so_associated_ports GPIO2_2 -enable_dft_signal controller_chain_mode
56
57 analyze_scan_chains
58 report_scan_chains
59
60 insert_test_logic
61 report_scan_cells > scan_cells.list
62
63 exit
```

## ATPG Pattern Generation for the Top Design

For top-level ATPG pattern generation, use the scan-inserted design data for the chip. Tessent uses the graybox models for the wrapped cores so that you can use the external mode of the wrapper chains to run ATPG.

### Note



Top-level ATPG pattern generation for the automotive flow follows the typical hierarchical flow as described in “[Top-Level ATPG Pattern Generation Example](#)” on page 239.

If you used Tessent Scan to insert the scan chains, specify the `import_scan_mode` command for ATPG pattern generation. Tessent Shell passes the scan-inserted design data through for the EDT and OCC logic. This data includes the scan structures that are stored in the TSDB under the “gate” design ID. You can create ATPG patterns for any mode that you need, such as stuck-at, transition, and controller chain mode.

In addition, Tessent automatically creates and simulates the `test_setup` procedure cycles that are required to initialize the EDT and OCC static signals. You only need to specify non-default parameter values, if, for example, you run EDT with `bypass on` or set `int_ltest_en` to 1 to use the boundary scan as the source of the core values and isolate the ATPG test from the top-level IOs.

You can read in the stuck-at fault models for the wrapped cores to calculate the total fault coverage for the chip.

## ATPG Pattern Retargeting for the Top Design

For each wrapped core, retarget the internal ATPG patterns for all the modes you specified and all the fault types.

Perform pattern retargeting as described in “[Performing Top-Level ATPG Pattern Retargeting](#)” on page 240.

---

### Note



For the automotive flow, you must add the top-level free-running repair clock, as described in the procedure steps.

---

## Interconnect Bridge/Open UDFM Generation for the Top Design

Generate an interconnect bridge and open UDFM for the top design module in the same way as you did for the `processor_core` module.

Refer to “[Interconnect Bridge/Open UDFM Generation: processor\\_core](#)” on page 418 to see how to generate an interconnect bridge/open UDFM on a wrapped core.

Refer to the following directory, which has an example on the top design module:

`tessent_automotive_reference_flow_<software_version>/top/9.extract_fault_sites`



## Cell-Neighborhood UDFM Generation for the Top Design

Generate a cell-neighborhood UDFM for the top design module in the same way as the `processor_core` module.

Refer to “[Cell-Neighborhood UDFM Generation: processor\\_core](#)” on page 420 to see how to generate a cell-neighborhood UDFM on the top design.

Refer to the following directory, which has an example on the top design module:

`tessent_automotive_reference_flow_<software_version>/top/9.extract_fault_sites`

## Automotive-Grade ATPG Pattern Generation for the Top Design

Generating automotive-grade ATPG patterns on the top design is similar to regular ATPG pattern generation. It needs several UDFM files for cell-internal defects, interconnect bridge or open defects, and cell-neighborhood defects.

Refer to the [Interconnect Bridge/Open UDFM Generation: processor\\_core](#) for interconnect bridge/open UDFM generation on your design, and [Cell-Neighborhood UDFM Generation: processor\\_core](#) for cell-neighborhood UDFM generation for your design.

Refer to “[UDFM Generation for Cell-Aware ATPG](#)” on page 453 for details on how to generate the cell-aware UDFM for your technology library.

As you do for regular ATPG, you must use graybox models for wrapped cores. The only difference with the regular top-level ATPG, in terms of the flow, is that you must read UDFM files for Automotive-Grade ATPG on the top design. You can run ATPG with topoff runs by loading only the UDFM files you want to target during an ATPG run. Also, you can run ATPG all together by reading in all UDFM files at once.

Refer to “[Automotive-Grade ATPG Pattern Generation: processor\\_core](#)” on page 424 to see how to generate ATPG patterns on a wrapped core.

Refer to the following directory, which has an example on the top design module:

`tessent_automotive_reference_flow_<software_version>/top/10.generate_AGA_patterns`

## UDFM Generation for Cell-Aware ATPG

To run cell-aware ATPG, you should generate cell-aware UDFMs for standard cells using Tessent CellModelGen. Ensure that all input requirements and required tools are available before running Tessent CellModelGen.

For information on Tessent CellModelGen or input requirements, refer to the *Tessent CellModelGen User's Manual*.

---

**Note**

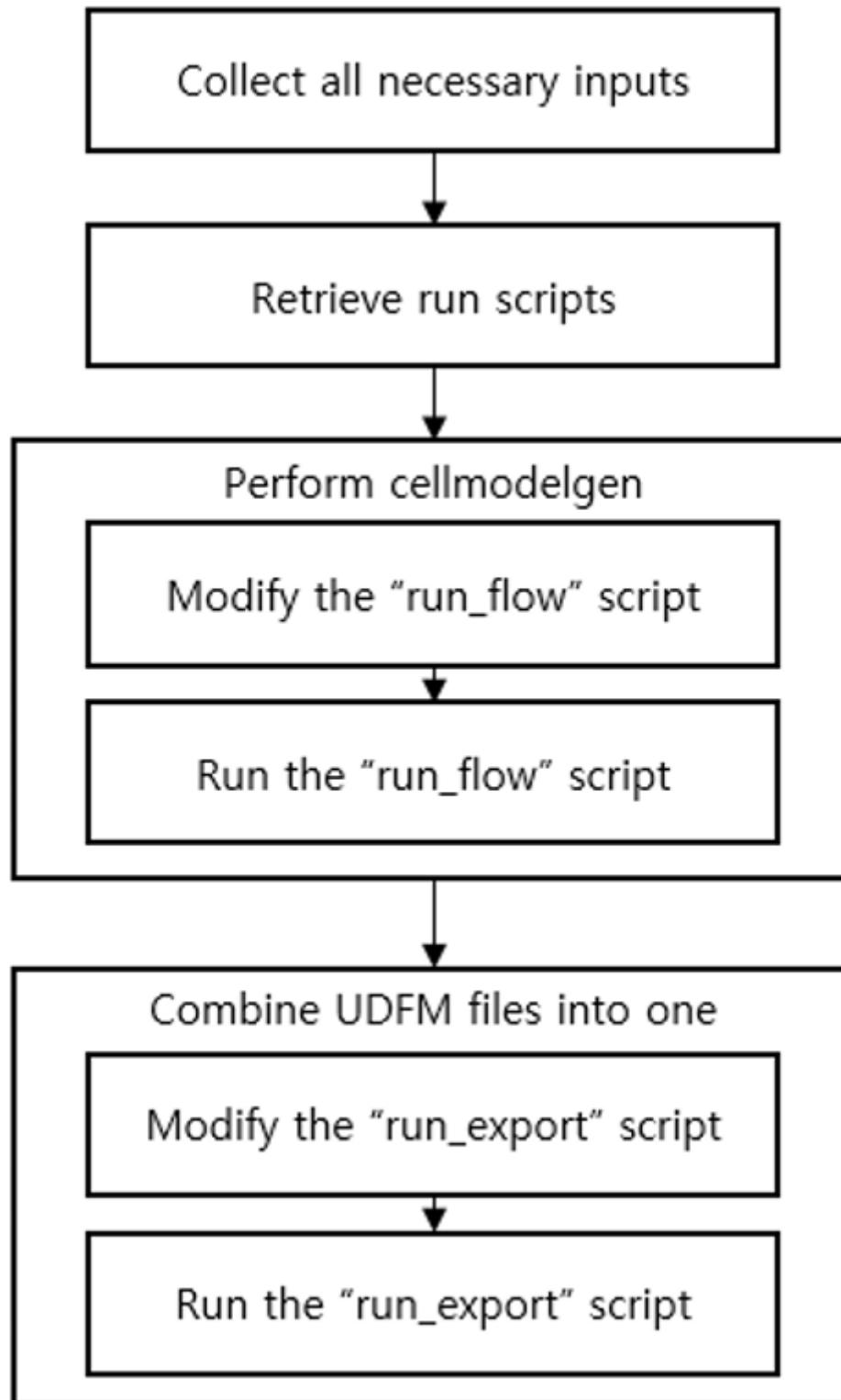


Refer to the following directory, which has a usage example described in this section:

`tessent_automotive_reference_flow_<software_version>/udfm_gen/stdlib`

---

Figure 7-15. Cell-Aware UDFM Generation Flow



## Procedure

1. Check all inputs and tools requirements are met.

For details, refer to the “[Required Licenses and Input Data](#)” chapter in the *Tessent CellModelGen Tool Reference*.

2. Create an empty directory, and retrieve run scripts:

```
cellmodelgen -get_script all
```

3. Look at the existing *run\_flow* script in the directory. Modify the *run\_flow* script for your design. Refer to the inline comments within the script for details.
4. Run the *run\_flow* script to run Tessent CellModelGen on your standard cells.
5. Look at the existing *run\_export* script in the directory. Modify the *run\_export* script for your design. Refer to the inline comments within the script for details.
6. Run the *run\_export* script to combine the individual standard cell UDFM files into a single UDFM file.

## TCA Based Pattern Sorting

Tessent Shell provides a way to sort ATPG patterns based on the total critical area (TCA) of defects. The TCA defect data is saved in cell-aware, interconnect bridge/open, and cell-neighborhood UDFM files.

---

### Note



The line numbers used in this procedure refer to the command flow in “[Dofile Example for Running ATPG With All Types of UDFM in Internal Mode for gps\\_baseband](#)” on page 457. Refer to the following directory, which has a usage example described in this section:

```
tessent_automotive_reference_flow_<software_version>/wrapped_cores/gps_baseband/  
12.pattern_sorting
```

---

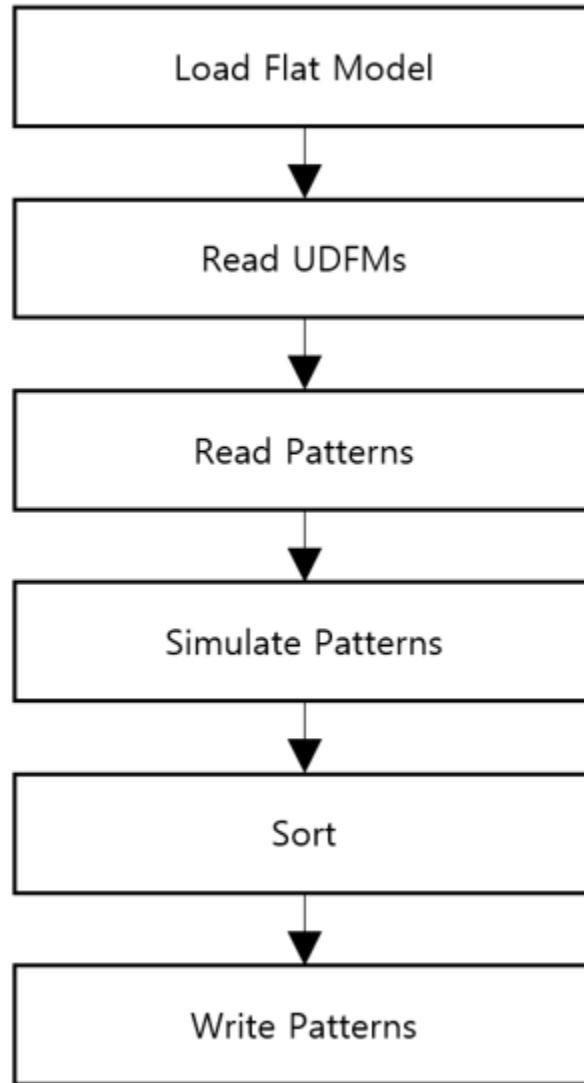
## Procedure

1. Load the flat model generated from a post-layout design. (See lines 1–11.)
2. Set a fault type, read in the UDFM file for your standard cells, and add faults. (See lines 13–18.)
3. Read the patterns you want to sort. (See lines 20–26.)
4. Run the “[set\\_critical\\_area\\_options](#) -reporting on” command to enable the TCA coverage reporting feature. (See line 28.)
5. Simulate the patterns. (See line 30.)
6. Run the “[order\\_patterns](#) -critical\_area” to sort the patterns based on TCA. (See line 31.)

7. Write out the sorted patterns (See line 33.)

## Examples

**Figure 7-16. TCA Based Pattern Sorting Flow**



The following dofile shows a command flow for generating a bridge/open UDFM for use in the Automotive-Grade ATPG flow.

**Example 7-14. Dofile Example for Running ATPG With All Types of UDFM in Internal Mode for gps\_baseband**

```
1 set_context patterns -scan -design_id pnr
2 set design_name "gps_baseband"
3
4 # Specify where the tsdb_outdir is to be located, default is the current
  working directory
5 set_tsdb_output_directory ../tsdb_outdir
```

```
6
7 # Add more processors to run ATPG
8 add_processors localhost:4
9
10 # Read flat model
11 read_flat_model ../tsdb_outdir/logic_test_cores/
    ${design_name}_pnr.logic_test_core/${design_name}.atpg_mode_ATPG_int/
    ${design_name}_ATPG_int.flat.gz
12
13 # Read UDFMs
14 set_fault_type udfm -static_fault
15 read_fault_sites \
16     ../.../udfm_gen/stdlib/udfm/NangateOpenCellLibrary.udfm
17 read_fault_sites ../10.extract_fault_sites/udfm/
    ${design_name}_brdg_opn.udfm
18 read_fault_sites ../10.extract_fault_sites/udfm/CELLS_neighbor.udfm
19 add_faults -all
20
21 # Read patterns
22 #read_patterns ../tsdb_outdir/logic_test_cores/
    ${design_name}_pnr.logic_test_core/${design_name}.atpg_mode_ATPG_int/
    ${design_name}_ATPG_int_stuck.patdb
23 read_patterns ../11.generate_AGA_patterns/patterns/
    ${design_name}_int_stuck.stil.gz
24 read_patterns ../11.generate_AGA_patterns/patterns/
    ${design_name}_int_CAT_static_topoff.stil.gz -append
25 read_patterns ../11.generate_AGA_patterns/patterns/
    ${design_name}_int_BRDG_static_topoff.stil.gz -append
26 read_patterns ../11.generate_AGA_patterns/patterns/
    ${design_name}_int_OPN_static_topoff.stil.gz -append
27 read_patterns ../11.generate_AGA_patterns/patterns/
    ${design_name}_int_intercell_static_topoff.stil.gz -append
28
29 set_critical_area_options -reporting on
30
31 simulate_patterns -source external -store_patterns all
32 order_patterns -critical_area
33
34 write_patterns patterns/${design_name}_int_static_topoff_sorting.stil.gz
    -stil -rep
35 exit
```

## Functional Mode Fault Tolerance for Static IJTAG Signals

Tessent Shell provides a way to implement gating logic to ensure that a single-event upset (SEU) does not affect the mission mode operation of a user design.

TDR outputs are gated in a hardware fault tolerant (HFT) module to create an alarm signal that you monitor during functional operation mode. The alarm indicates that a test signal register value has been altered.

This gating and monitoring logic is available for static DFT signals implemented with the `-create_with_tdr` option. You can control the implementation of this logic for individual signals.

## Procedure

1. Add the `dft_signal_disable` DFT signal to gate the TDR logic:

```
add_dft_signals dft_signal_disable
```

2. Enable the creation of the HFT module for all static DFT signals interacting with functional logic:

```
set_dft_signals_options -disable_for_functional_safety static
```

3. Add the individual DFT signals that you want to monitor:

```
add_dft_signals dft_signal -create_with_tdr \  
-disable_for_functional_safety on
```

4. Add any DFT signals you want to exclude from monitoring:

```
add_dft_signals dft_signal -create_with_tdr \  
-disable_for_functional_safety off
```

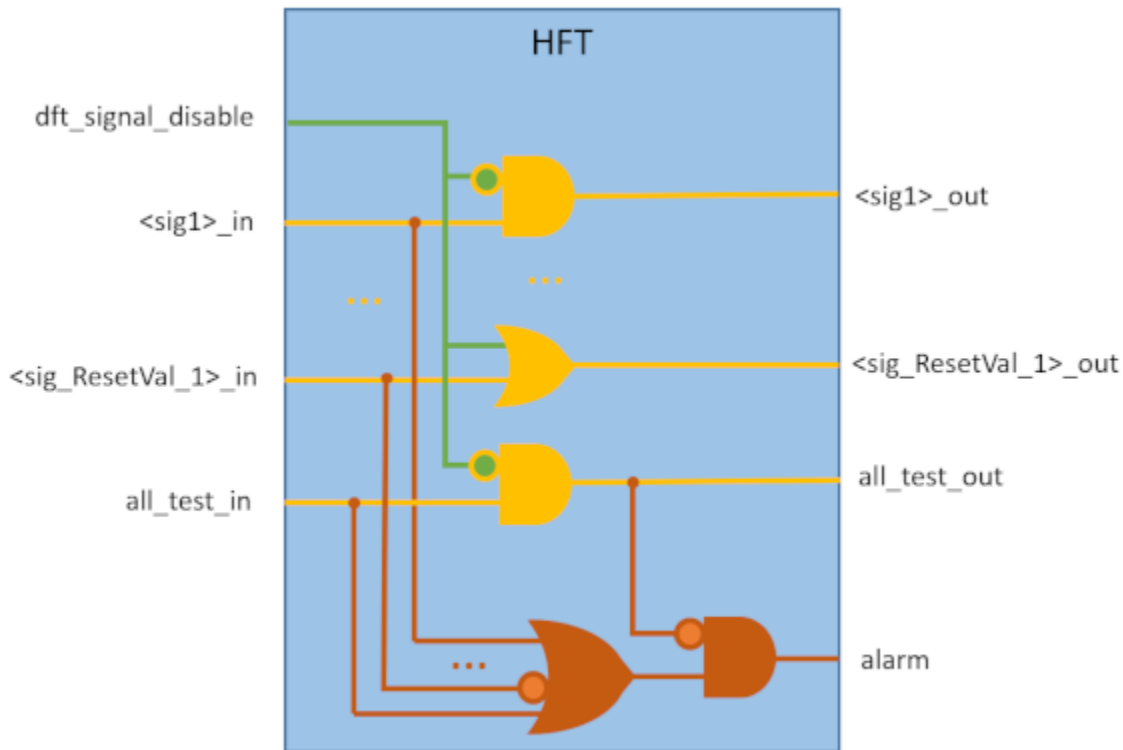
5. Create and process the `DftSpecification`:

```
check_design_rules  
create_dft_specification  
process_dft_specification
```

## Results

An instance of an HFT module is created in a TDR hosting the static DFT signals. This module provides the gating of the specified static DFT signals in addition to SEU monitoring logic. See [Figure 7-17](#) for an example.

**Figure 7-17. Hardware Fault Tolerant Module Example**



The alarm signal is raised only in functional mode, and is gated with the all\_test\_out signal (the all\_test signal gated with fault tolerant logic). It is accessible on a persistent buffer regardless of the DftSpecification add\_persistent\_buffers\_in\_scan\_resource\_instruments setting. The buffer instance name is <tessent\_persistent\_cell\_prefix>\_hft\_alarm\_buf.

Use the following command to introspect the alarm signal:

```
get_icl_pins -filter \
  {tessent_dft_function=="functional_mode_alarm"} \
  -of_instance [get_icl_instance -filter \
    {tessent_instrument_type=="mentor::ijtag_node"}]
```

---

**Note**

 If you do not add the DFT signals explicitly, the tool creates all\_test and dft\_signal\_disable automatically.

---



---

**Note**

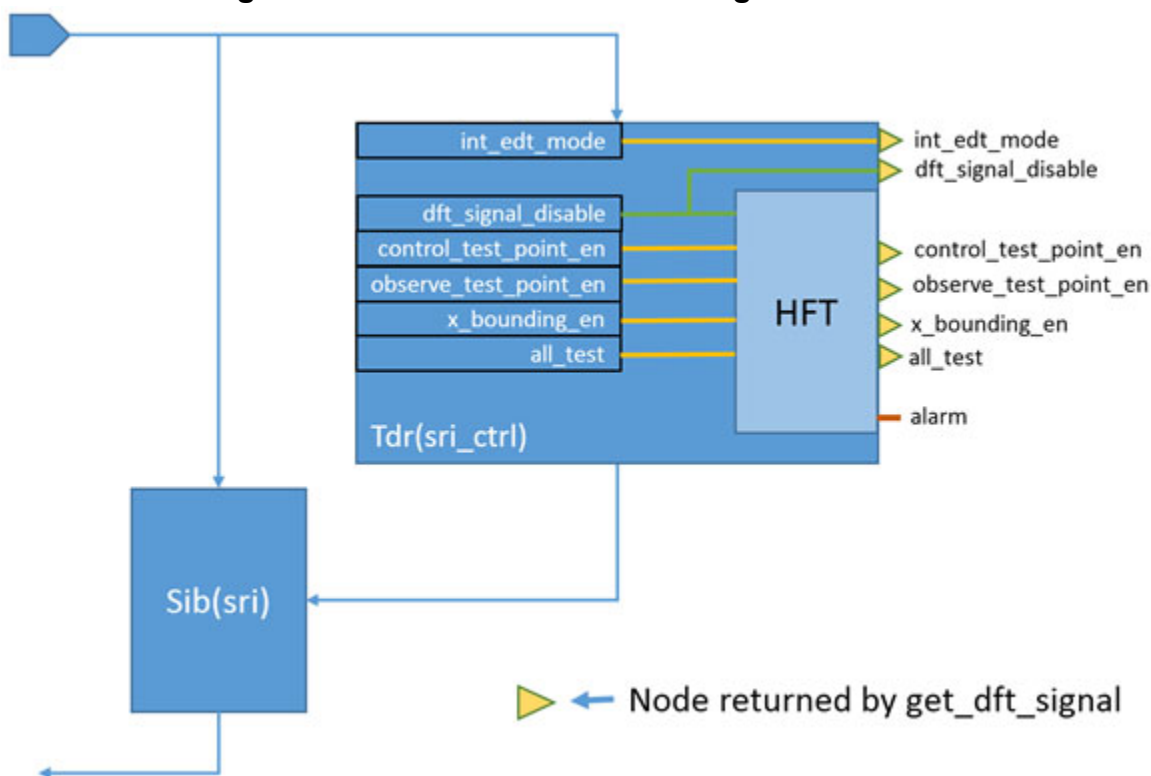
 Typically, the reset value for DFT signals is 0. When the reset value is 1, the gating logic is inverted, as shown with <sig\_ResetVal\_1> in [Figure 7-17](#). Each DFT signal holds its reset value in the functional mode of circuit operation.

---

In the single-pass DFT insertion flow, the dft\_signal\_disable signal is created in a Scan Resource Instrument (SRI) TDR. See [Figure 7-18](#) for an example.

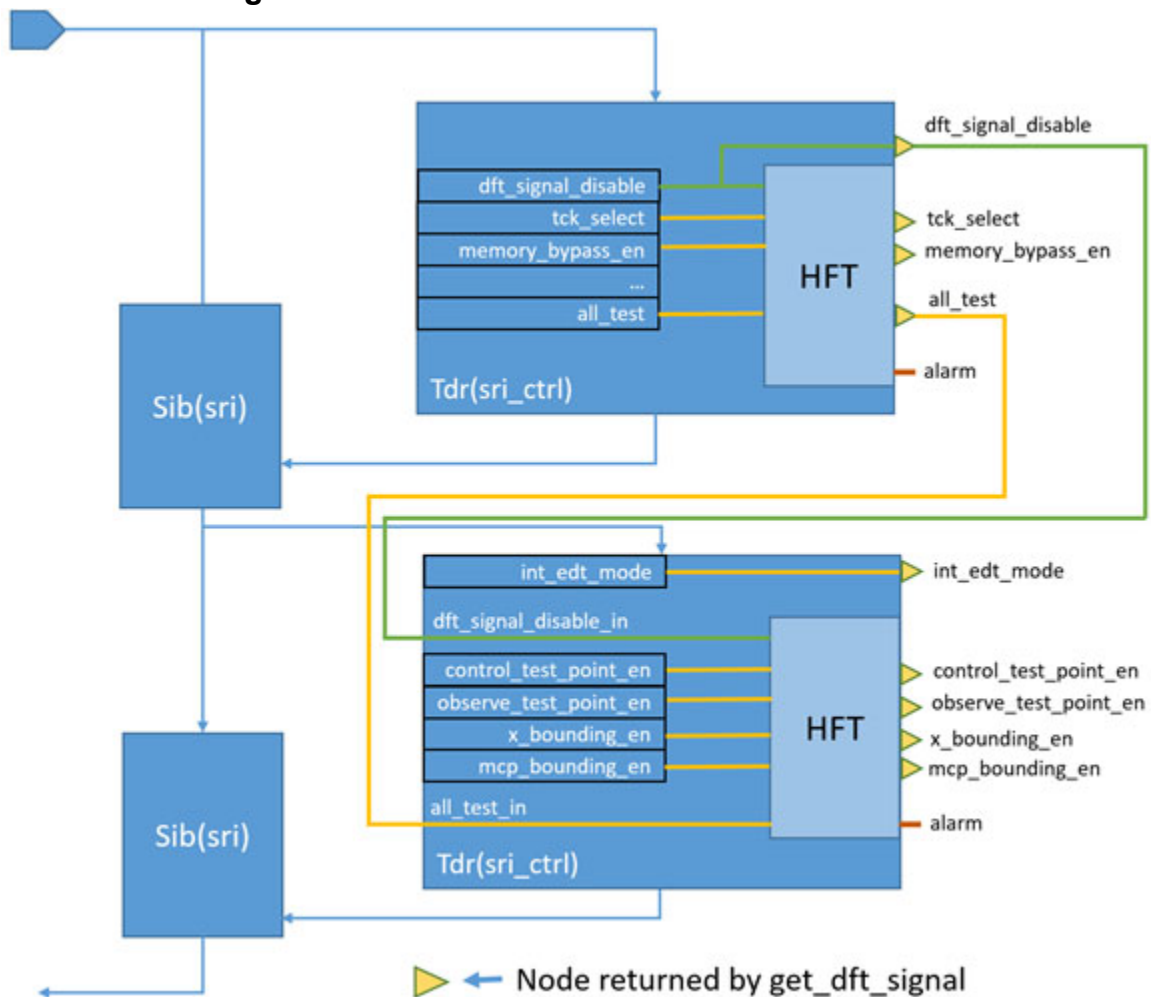


Figure 7-18. HFT Module in the Single-Pass Flow



In the two-pass DFT insertion flow, the **dft\_signal\_disable** and **all\_test** signals are defined in a **Tdr(sri\_ctrl)** module created during the first insertion pass. The TDR with logic test signals is created in the second DFT insertion pass, and takes the **all\_test** and **dft\_signal\_disable** signals as primary inputs. Each TDR instance contains its own HFT logic, and two alarm signals are available on the TDR output ports. See [Figure 7-19](#) for an example.

Figure 7-19. HFT Modules in the Two-Pass Flow



If you implement signal monitoring with the global `set_dft_signals_options` command, only those signals that affect the functional mode of operation can be monitored. If you implement signal monitoring and gating on a per-signal basis, any signal can be monitored.

By default, the `capture_per_cycle_static_en` and `se_pipeline_en` logic test control signals are not monitored. Also, no scan mode signals are monitored by default. Monitoring and gating is possible only for those static DFT signals you create with the `-create_with_tdr` option.

If you create the `dft_signal_disable` signal using the `-source_node` option, the tool implements the source node as part of an ICL network regardless of design level.

## Related Topics

[add\\_dft\\_signals](#)

[set\\_dft\\_signals\\_options](#)

[get\\_dft\\_signals\\_option](#)

# Chapter 8

## TSDB Data Flow for the Tessent Shell Flow

---

The Tessent Shell Database (TSDB) is a repository for all the files and directories that Tessent Shell generates. The TSDB enhances flow automation by acting as the central location where Tessent can access the data it requires for the current task, whether that task be reading in a design, performing DRC, inserting logic test hardware, or performing ATPG pattern generation.

The TSDB structure aids data management between steps in a process even if you are not performing these steps within the Tessent Shell platform. If the steps are performed within Tessent Shell, then specifying the correct design ID automatically ensures that Tessent uses the correct file inputs for the current task.

Refer to “[Tessent Shell Database \(TSDB\)](#)” in the *Tessent Shell Reference Manual* for details about the TSDB directory structure and contents.

The data flow content builds on the material described in “[Tessent Shell Flow for Flat Designs](#)” and “[Tessent Shell Flow for Hierarchical Designs](#)” regarding the RTL and scan DFT insertion flow. During the RTL and scan DFT insertion process, Tessent Shell generates many output files and directories that it accesses later in the flow as data inputs. This chapter illustrates the data flow through each step of the RTL and scan DFT insertion flow.

|                                                |            |
|------------------------------------------------|------------|
| <b>Core-Level or Flat TSDB Data Flow .....</b> | <b>463</b> |
| <b>Top-Level TSDB Data Flow .....</b>          | <b>469</b> |

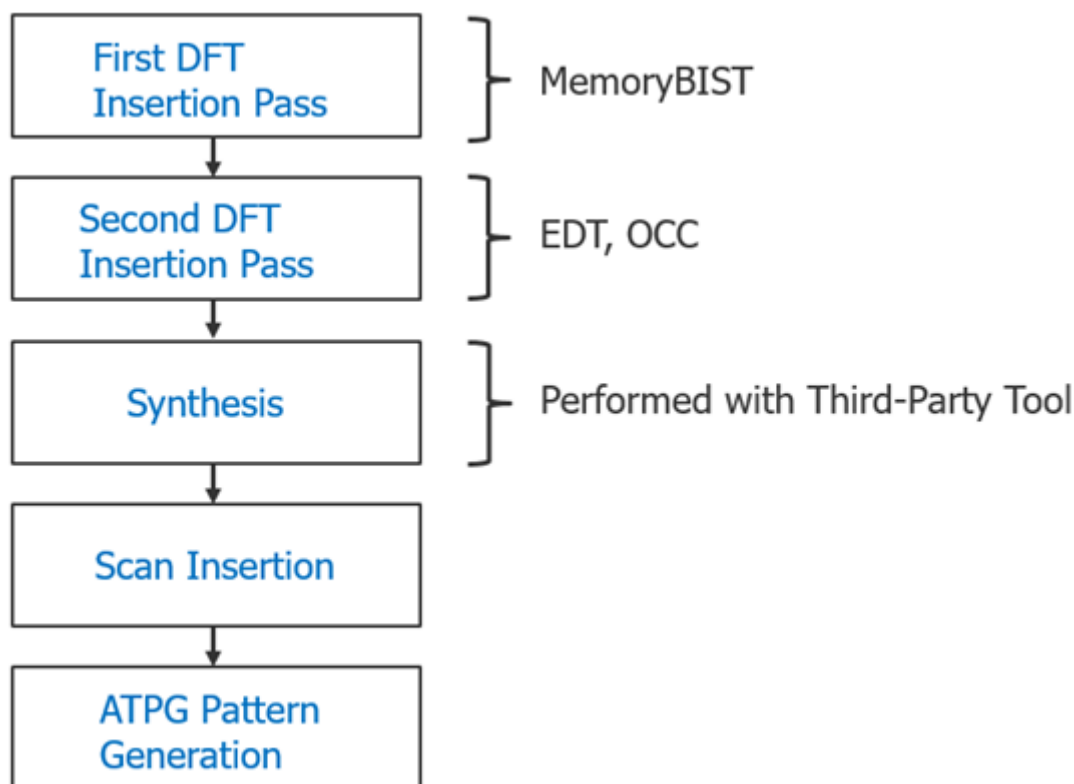
### Core-Level or Flat TSDB Data Flow

The core-level TSDB data flow applies to wrapped cores in a hierarchical DFT insertion flow. In the bottom-up insertion process, you process the wrapped cores before the top-level chip.

You can also apply this data flow to flat and DFT-inserted designs by adding boundary scan in the first DFT insertion pass (unless the core includes embedded pad I/O macros that need boundary scan). Refer to “[Tessent Shell Flow for Flat Designs](#)” for details.

The Tessent Shell task flow is shown in [Figure 8-1](#). The details of the inputs and outputs are shown in [Table 8-1](#).

**Figure 8-1. Tessent Shell Task Flow**



Refer to “[Tessent Shell Flow for Hierarchical Designs](#)” for details.

**Table 8-1. Core-Level TSDB Data Flow Inputs and Outputs**

| Task                                                           | Input                                                                                                                                                                                                        | Output                                                                                                                                                                                                                  |
|----------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| DFT insertion:<br>first pass<br>Design ID for<br>output: rtl1  | <ul style="list-style-type: none"> <li>• RTL netlist</li> <li>• Libraries</li> <li>• Required DFT signals</li> <li>• MemoryBIST requirements</li> <li>• For flat: boundary scan requirements also</li> </ul> | <ul style="list-style-type: none"> <li>• Modified RTL + new RTL</li> <li>• ICL for IJTAG network</li> <li>• TCD: clocks, DFT signals</li> <li>• ICL for Memory + PDL</li> <li>• For flat: boundary scan also</li> </ul> |
| DFT insertion:<br>second pass<br>Design ID for<br>output: rtl2 | <ul style="list-style-type: none"> <li>• rtl1 design data</li> <li>• Libraries</li> <li>• Required DFT signals</li> <li>• EDT and OCC requirements</li> </ul>                                                | <ul style="list-style-type: none"> <li>• Modified RTL + new RTL</li> <li>• ICL for IJTAG network</li> <li>• TCD: clocks, DFT signals</li> <li>• ICL for OCC and EDT + PDL</li> </ul>                                    |
| Synthesis<br>Third-party tool                                  | <ul style="list-style-type: none"> <li>• Output from write_design_import_script command (use rtl2 design data)</li> </ul>                                                                                    | <ul style="list-style-type: none"> <li>• Synthesized gate-level netlist</li> </ul>                                                                                                                                      |

**Table 8-1. Core-Level TSDB Data Flow Inputs and Outputs (cont.)**

| Task                                               | Input                                                                                                                                       | Output                                                                                                                                                                      |
|----------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Scan chain insertion<br>Design ID for output: gate | <ul style="list-style-type: none"> <li>• Synthesized gate-level netlist</li> <li>• Scan modes</li> <li>• TCD, ICL, PDL from rtl2</li> </ul> | <ul style="list-style-type: none"> <li>• Scan-stitched gate-level netlist</li> <li>• ICL, PDL</li> <li>• TCD with scan modes</li> <li>• Graybox model (not flat)</li> </ul> |
| ATPG pattern generation                            | <ul style="list-style-type: none"> <li>• Gate scan-inserted design data</li> <li>• ATPG run name</li> <li>• Scan mode</li> </ul>            | <ul style="list-style-type: none"> <li>• Flat model</li> <li>• Fault list</li> <li>• Patterns database</li> <li>• TCD</li> </ul>                                            |

During the first DFT insertion pass, you provide the required files for your DFT implementation. These files can include the RTL netlist, libraries for MemoryBIST insertion and boundary scan, and gate-level cells that require a Tessent cell library.

Tessent Shell generates the `dft_inserted_designs`, `instruments`, and `patterns` directories within the TSDB you specified. By default, Tessent Shell generates the TSDB in the current working directory if you do not specify a location. For details about these directories and the TSDB, refer to “[Tessent Shell Database \(TSDB\)](#)” in the *Tessent Shell Reference Manual*.

Tessent modifies the RTL netlist for the design into which it inserts the first-pass instrument hardware. This hardware may include a MemoryBIST controller, BAP interface, and IJTAG network. In the flat DFT implementation, it may also include boundary scan and a TAP controller. Tessent Shell generates new RTL for the newly inserted DFT instruments.

In addition, Tessent Shell produces the TCD, ICL, and PDL for the design and the inserted instruments. As shown in the red boxes in [Figure 8-2](#), the design-level files and modified RTL are stored within the `dft_inserted_designs` subdirectory for this insertion pass (design ID “rtl1”). However, the new RTL, TCD, ICL, and PDL files for each inserted instrument are stored in subdirectories within the `instruments` directory.

The `patterns` directory stores the patterns associated with the rtl1 design ID in an associated subdirectory.

#### Tip

 To facilitate data management, you can save each design (whether flat, core, sub-block, or chip) in its own TSDB. This is the recommended practice when using Tessent Shell for DFT insertion.

**Figure 8-2. TSDB File Structure, Core Level, First Insertion Pass**

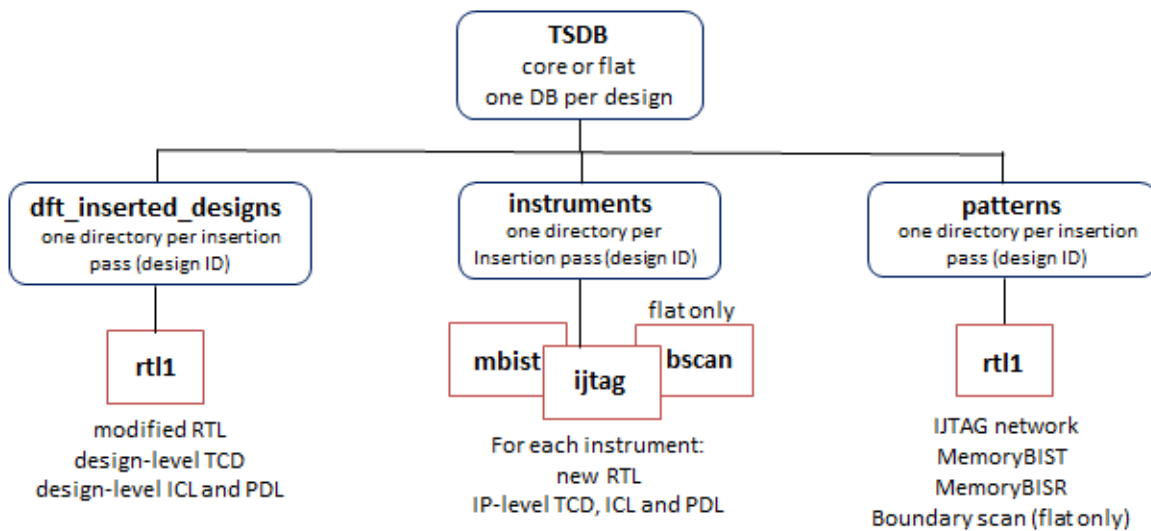


Figure 8-3 shows the file structure for the second DFT insertion pass. The yellow box shows the design data that the tool saved as **rtl1** in the first pass, which it uses as input for the second pass. The red boxes show the output files.

The relevant design RTL, TCD, ICL, and PDL files from the first DFT insertion pass are automatically read in after you specify the `read_design` command as described in “[Loading the Design](#)”. You only need to supply a library, if required, and any DFT input requirement for the DFT instruments you are inserting during this pass.

The hardware you insert in this pass, such as EDT and OCC, is stored in the **instruments** directory. Separate directories are created for each type of hardware inserted in each insertion pass.

The **patterns** directory stores the patterns associated with the **rtl2** design ID in an associated subdirectory.

**Figure 8-3. TSDB File Structure, Core Level, Second Insertion Pass**

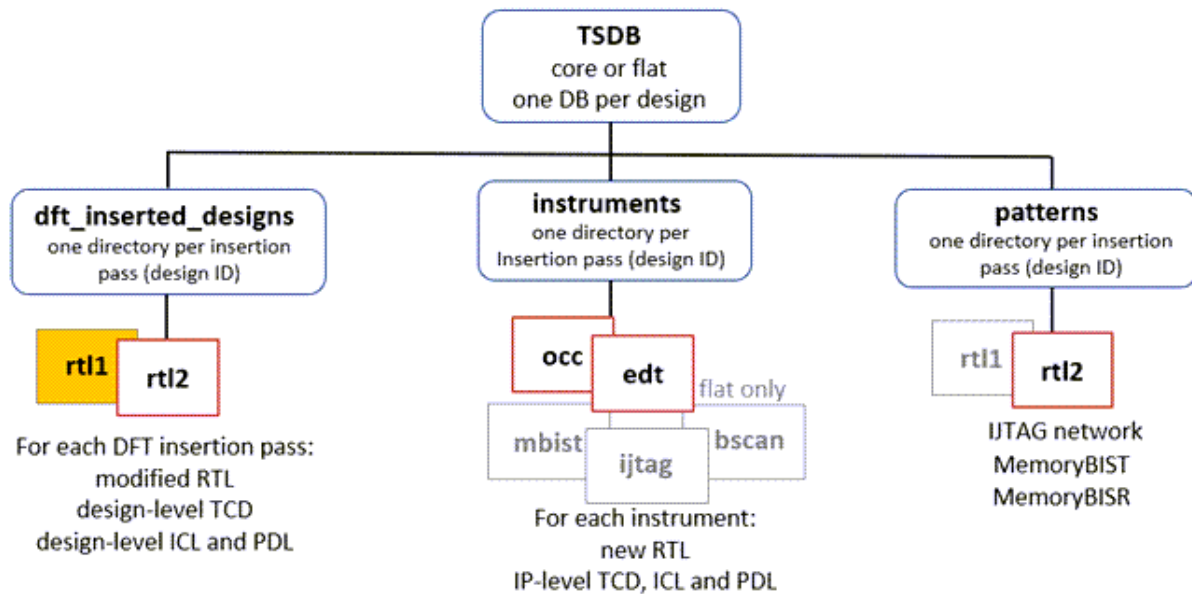
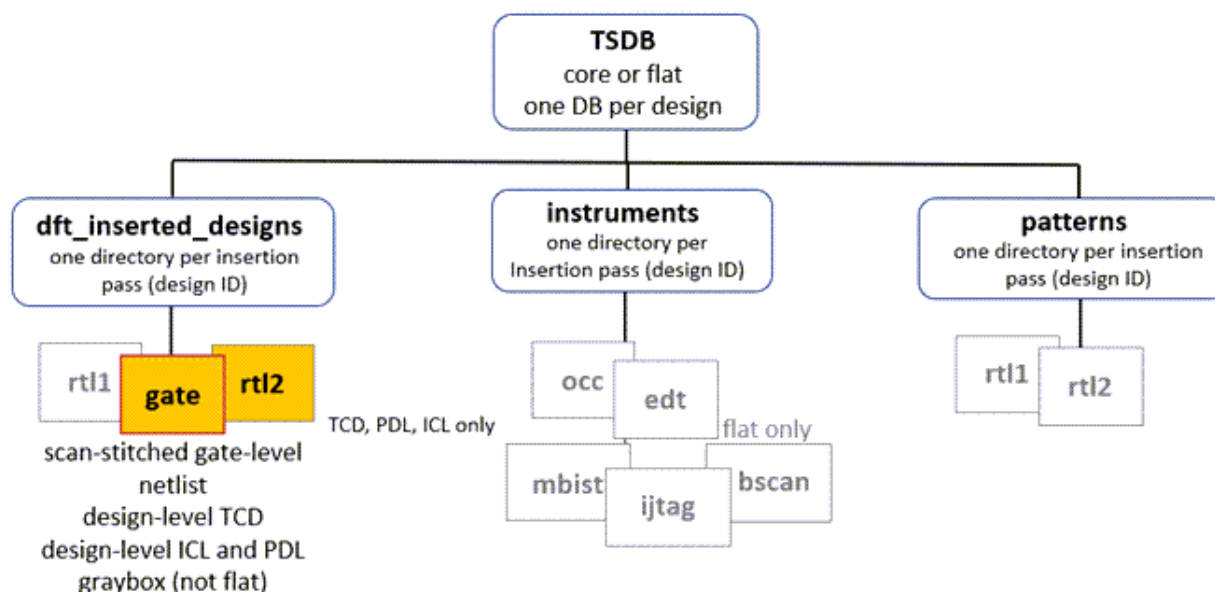


Figure 8-4 shows the TSDB file structure after synthesis and scan chain insertion. The third-party synthesis tool provides a synthesized gate-level netlist. This netlist, shown in Figure 8-4 in the yellow box with a red outline, is the input for scan chain insertion along with the design-level TCD, ICL, and PDL from the rtl2 design generated during the second DFT insertion pass.

For wrapped cores, Tessent performs wrapper analysis along with scan chain insertion, whereas for flat designs, Tessent performs scan chain replacement and stitching. For information about using Tessent Scan for scan insertion, refer to “[Internal Scan and Test Circuitry Insertion](#)” in the *Tessent Scan and ATPG User’s Manual*.

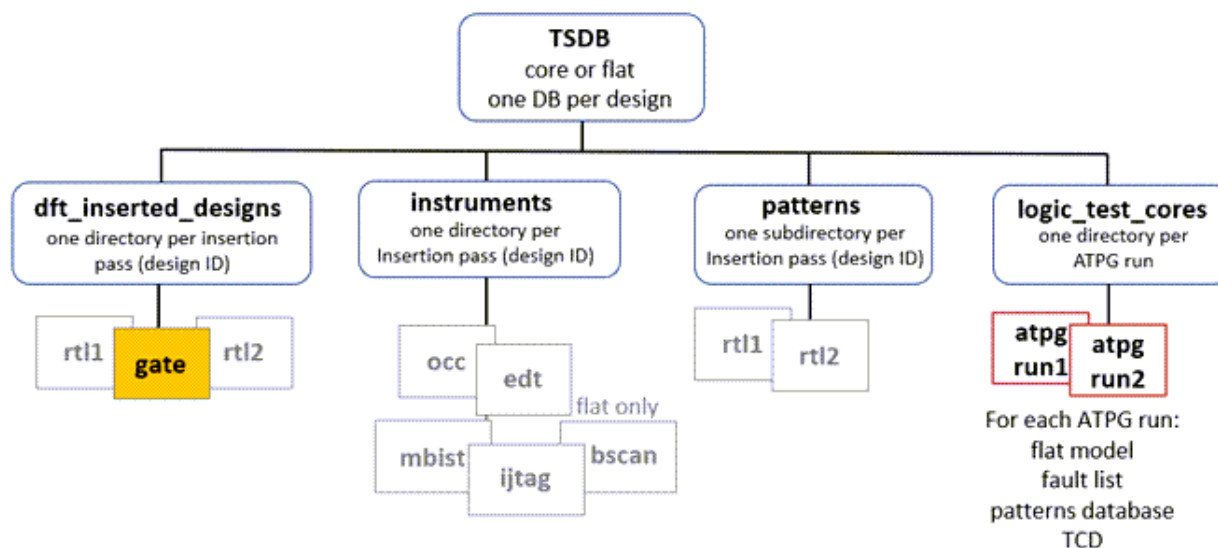
Scan insertion does not insert instruments, so the instruments and patterns directories are not utilized in this step.

Figure 8-4. TSDB File Structure, Core Level, Synthesis and Scan Insertion



The inputs to ATPG are the scan-inserted netlist and all the supporting files such as TCD, ICL, and PDL files that were stored in the TSDB during the scan insertion pass (design\_id “gate”), as shown in the yellow box in Figure 8-5. Tessent creates the *logic\_test\_cores* directory to store the output pattern data for each ATPG run, which can include runs for various test types and associated fault models as described in “[Fault Modeling Overview](#)” in the *Tessent Scan and ATPG User’s Manual*. This is shown in the red boxes in Figure 8-5. See [Generating Test Patterns for Different Fault Models and Fault Grading](#) in the *Tessent Scan and ATPG User’s Manual* for an example.

Figure 8-5. TSDB File Structure, Core Level, Pattern Generation





Before generating patterns for wrapped cores, Tessent creates a graybox model of the core. This model is stored using the same design ID as the one created during scan insertion (design ID “gate”), so that at the top-level you can either use the full design view or graybox view of the wrapped core. Refer to [“Performing ATPG Pattern Generation: Wrapped Core”](#) for more information.

## Top-Level TSDB Data Flow

The top-level TSDB data flow applies to the hierarchical RTL and scan DFT insertion flow. In the hierarchical DFT insertion flow, you insert boundary scan and MemoryBIST, if present, at the top level of a chip during the first DFT insertion pass.

At the chip level, the core-level design data that was stored in the core-level TSDBs is used for ATPG pattern retargeting. Refer to [“RTL and Scan DFT Insertion Flow for the Top Chip”](#) for details.

**Table 8-2. Top-Level TSDB Data Flow Inputs and Outputs**

| Task                                                           | Input                                                                                                                                                                                               | Output                                                                                                                                                                                 |
|----------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| DFT insertion:<br>first pass<br>design ID for<br>output: rtl1  | <ul style="list-style-type: none"> <li>• RTL netlist</li> <li>• Libraries</li> <li>• Required DFT signals</li> <li>• Boundary scan requirements</li> <li>• TSDBs, lower core design data</li> </ul> | <ul style="list-style-type: none"> <li>• Modified RTL + new RTL</li> <li>• ICL for IJTAG network</li> <li>• TCD: clocks, DFT signals</li> <li>• ICL for boundary scan + PDL</li> </ul> |
| DFT Insertion:<br>second pass<br>design ID for<br>output: rtl2 | <ul style="list-style-type: none"> <li>• rtl1 design data</li> <li>• Libraries</li> <li>• Required DFT signals</li> <li>• EDT and OCC requirements</li> </ul>                                       | <ul style="list-style-type: none"> <li>• Modified RTL + new RTL</li> <li>• ICL for IJTAG network</li> <li>• TCD: clocks, DFT signals</li> <li>• ICL for OCC and EDT + PDL</li> </ul>   |
| Synthesis<br>third-party tool                                  | <ul style="list-style-type: none"> <li>• Output from<br/>write_design_import_script<br/>command (use rtl2 design data)</li> </ul>                                                                   | <ul style="list-style-type: none"> <li>• Synthesized gate-level netlist</li> </ul>                                                                                                     |
| Scan chain<br>insertion<br>design ID for<br>output: gate       | <ul style="list-style-type: none"> <li>• Synthesized gate-level netlist</li> <li>• Scan modes</li> <li>• TCD, ICL, PDL from rtl2</li> </ul>                                                         | <ul style="list-style-type: none"> <li>• Scan-stitched gate-level netlist</li> <li>• ICL, PDL</li> <li>• TCD with scan modes</li> </ul>                                                |
| ATPG pattern<br>generation (flat)                              | <ul style="list-style-type: none"> <li>• Gate scan-inserted design data</li> <li>• ATPG run name</li> <li>• Scan mode</li> </ul>                                                                    | <ul style="list-style-type: none"> <li>• Flat model</li> <li>• Fault list</li> <li>• Patterns database</li> <li>• TCD</li> </ul>                                                       |

**Table 8-2. Top-Level TSDB Data Flow Inputs and Outputs (cont.)**

| Task                                 | Input                                                                                                                                                                             | Output                                                                                                                      |
|--------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| ATPG pattern generation (core level) | <ul style="list-style-type: none"><li>• Gate scan-inserted design data</li><li>• ATPG run name</li><li>• Scan mode</li><li>• Wrapped core ATPG pattern data, retargeted</li></ul> | <ul style="list-style-type: none"><li>• Flat model</li><li>• Fault list</li><li>• Patterns database</li><li>• TCD</li></ul> |

Figure 8-6 shows the data flow for the first DFT insertion pass. Tessent modifies the RTL netlist for the design and generates new RTL for the boundary scan and MemoryBIST (if inserted) hardware. In addition, it produces the TCD, ICL, and PDL for the design and the inserted instruments.

For integration at the top level, the scan-inserted design data and the interface views from the wrapped cores are used. This is done by opening the core TSDB directories and using the `read_design` command to read in the graybox model and the TCD, ICL, and PDL files.

The patterns directory stores the patterns associated with the rtl1 design ID in an associated subdirectory.

---

**Tip**

 To facilitate data management, you can save each design (whether flat, core, sub-block, or chip) in its own TSDB. This is the recommended practice when using Tessent Shell for DFT insertion. You should have one TSDB per design.

---

**Figure 8-6. TSDb Data Flow, Top Level, First Insertion Pass**

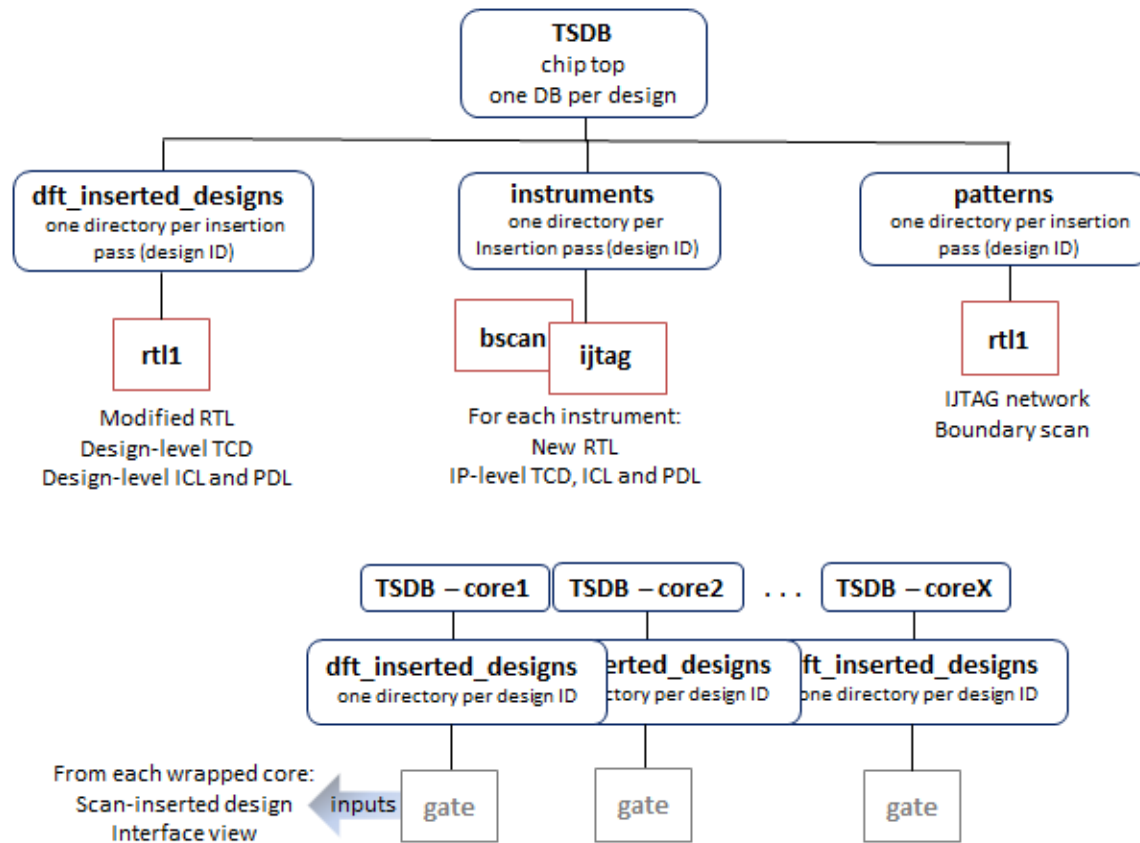


Figure 8-7 shows the data flow for the second DFT insertion pass. In addition to the other input requirements that you provide as shown in Table 8-2, Tessent uses the design data that was saved as rtl1 in the first pass, the gate scan-inserted design data, and graybox models from the wrapped cores.

Tessent saves the output design data for the EDT and OCC hardware in their applicable instruments subdirectories. The design-level TCD, ICL, PDL, and modified RTL that includes the EDT, OCC, and IJTAG network is placed in the dft\_inserted\_designs subdirectory for this insertion pass (rtl2).

The patterns directory stores the patterns associated with the rtl2 design ID in an associated subdirectory.

**Figure 8-7. TSDB Data Flow, Top Level, Second Insertion Pass**

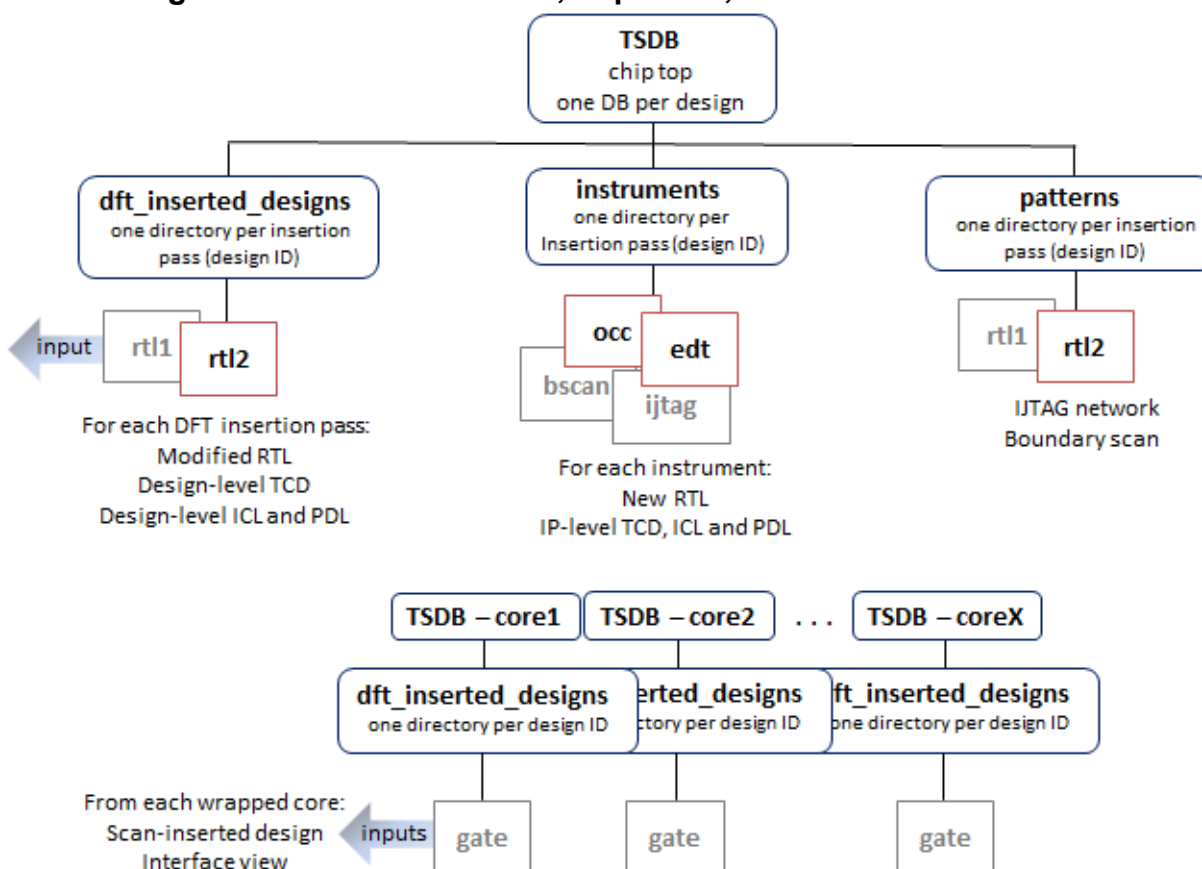


Figure 8-8 shows the data flow for scan chain insertion. Scan insertion does not insert instruments, so the instruments and patterns directories are not utilized in this step.

Figure 8-8. TSDB Data Flow, Top Level, Scan Insertion

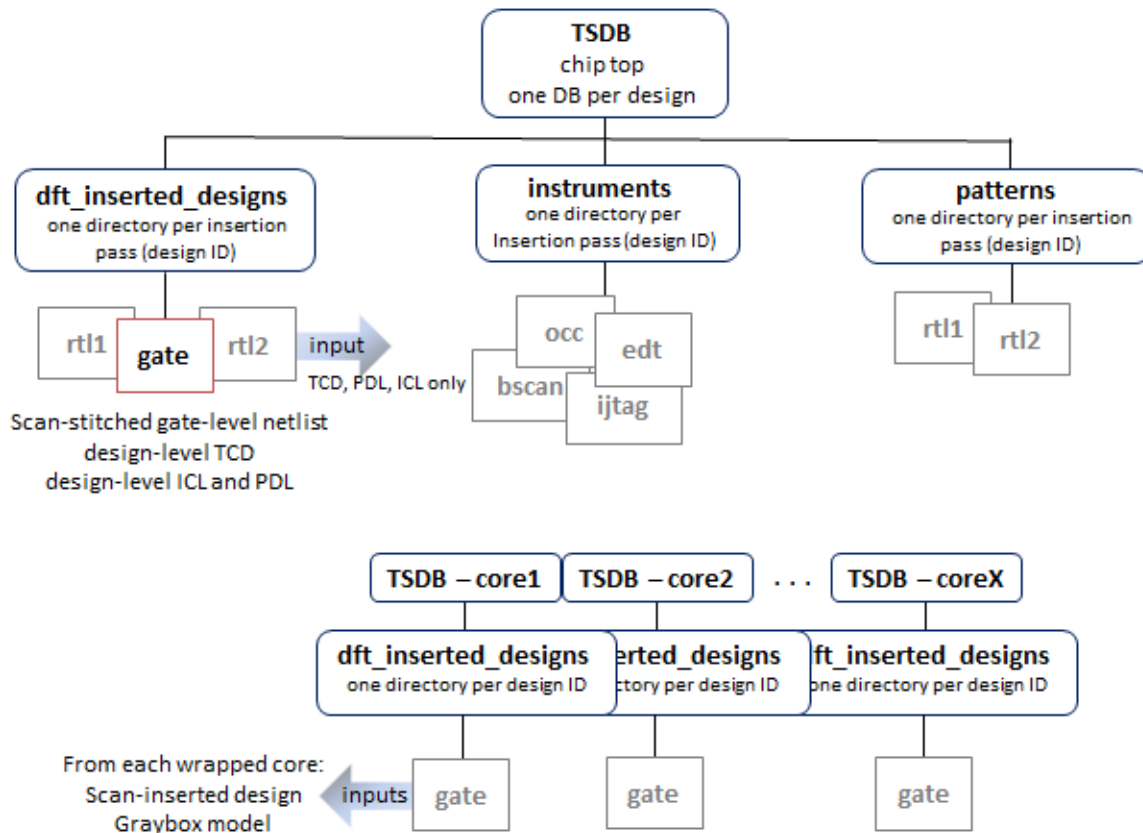
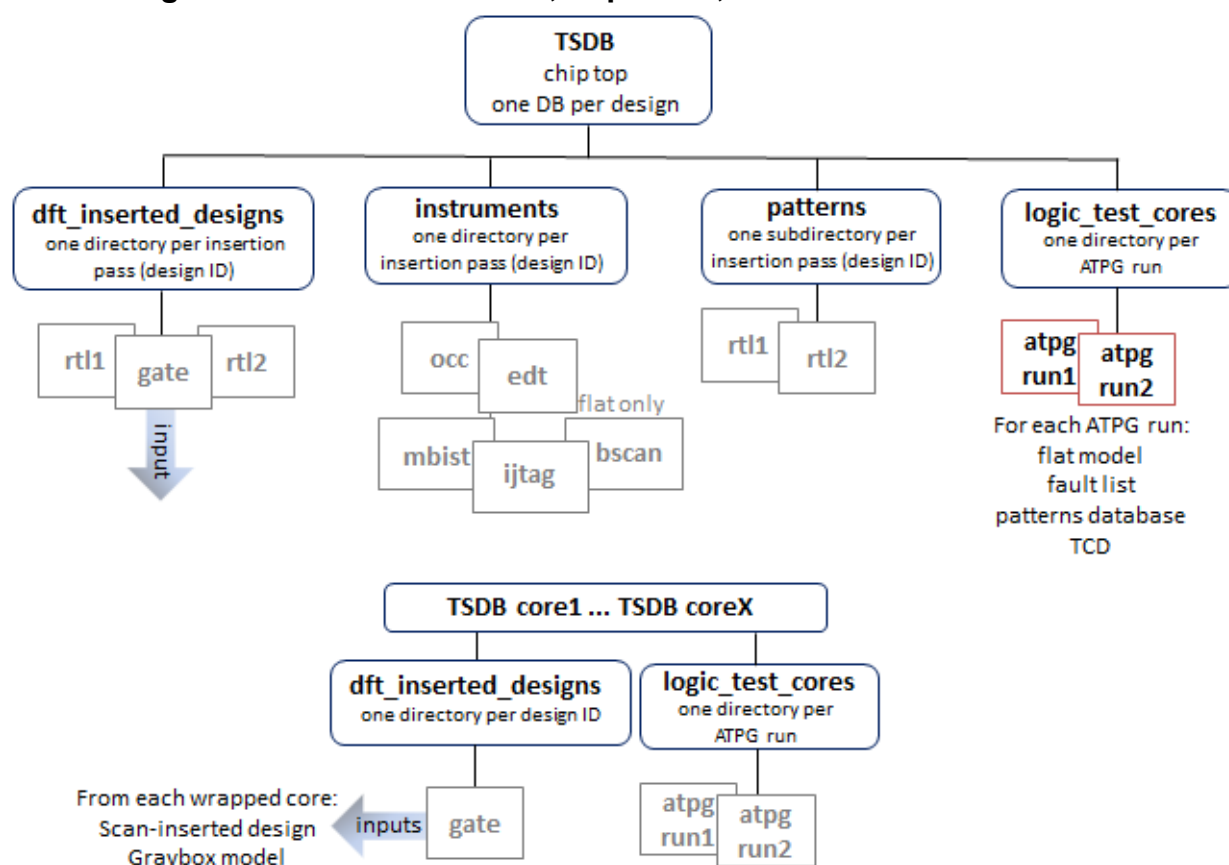


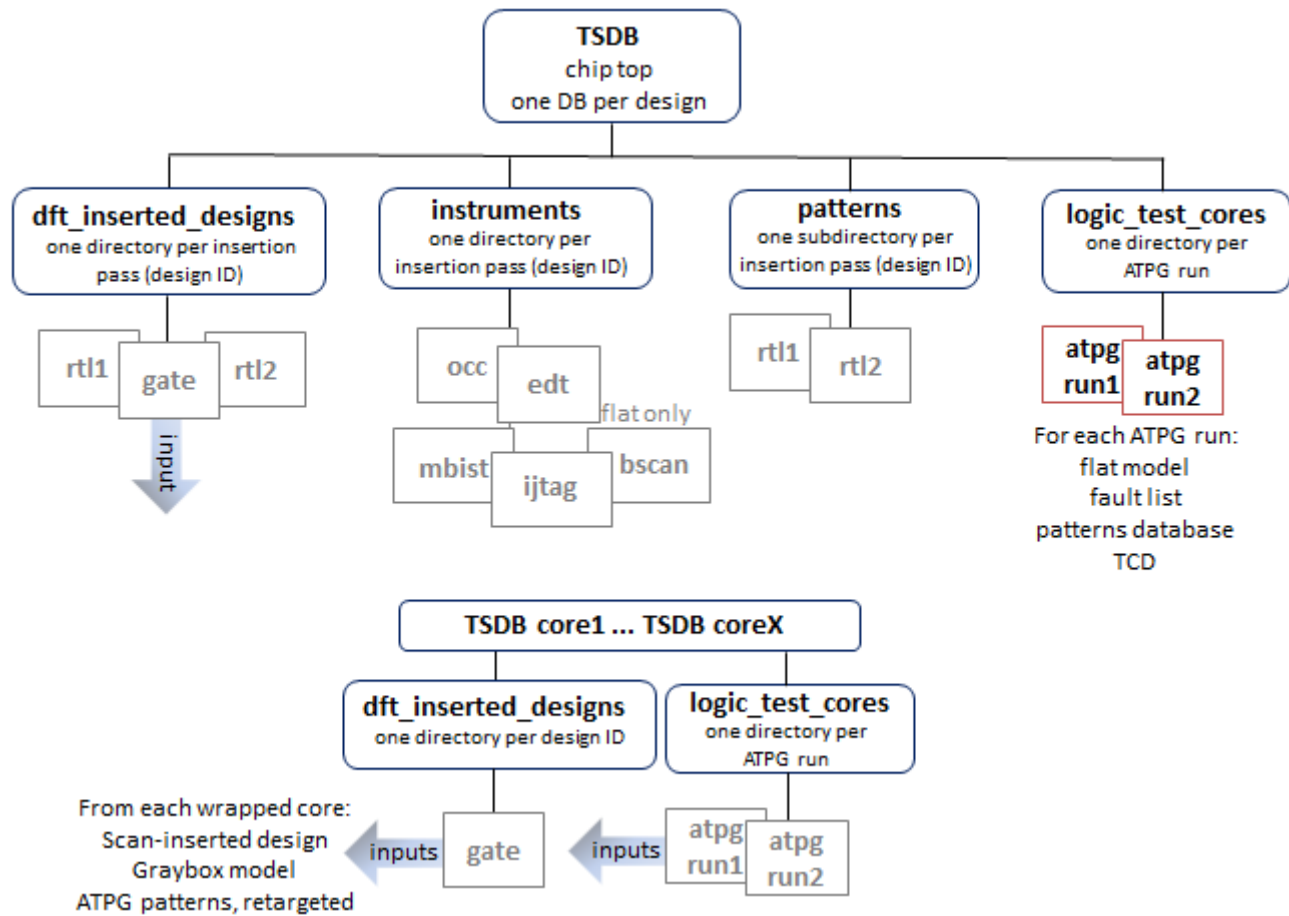
Figure 8-9 shows that output from ATPG pattern generation gets stored in the logic\_test\_cores directory. As inputs, Tessent uses the scan-inserted design data for the chip and for the cores.

**Figure 8-9. TSDB Data Flow, Top Level, ATPG Pattern Generation**



As the final step for the top level in a hierarchical design, you perform ATPG pattern retargeting of the core ATPG patterns as shown in “[Top-Level ATPG Pattern Generation Example](#)”. [Figure 8-10](#) shows that you read in the ATPG patterns from the logic\_test\_cores directory from each of the core TSDB directories and the scan-inserted design data for the chip and for the cores.

**Figure 8-10. TSDB Data Flow, Top Level, ATPG Pattern Generation With Pattern Retargeting**







# Chapter 9

## Streaming Scan Network (SSN)

---

Tessent Streaming Scan Network (SSN) is a bus-based scan data distribution architecture for use with Tessent TestKompress. The design of SSN addresses common DFT challenges in testing system-on-chip (SoC) designs, such as planning effort, tiled designs with abutment, test cost, and routing and timing closure.

SSN decouples test delivery and core-level DFT requirements. This means that instance core-level compression can be defined completely independently of chip I/O limitations. It enables programmatic decisions, such as selecting which cores are tested concurrently. In a traditional pin-muxed approach, these options would be hard-wired.

This section describes the recommended SSN work flows, including how to use SSN to effectively test designs with identical cores and how to use different clock networks with SSN.

|                                                  |            |
|--------------------------------------------------|------------|
| <b>Tessent SSN Workflows</b> .....               | <b>478</b> |
| Block-Level SSN Insertion and Verification ..... | 481        |
| Top-Level SSN Insertion and Verification .....   | 506        |
| <b>Advanced Topics</b> .....                     | <b>531</b> |
| Streaming-Through-IJTAG Scan Data .....          | 532        |
| On-Chip Compare With SSN .....                   | 535        |
| Types of Clock Networks To Use With SSN .....    | 545        |
| Broadcast to Identical SSN Datapaths .....       | 549        |
| Determine Failing Cores on ATE With SSN .....    | 549        |
| Yield Statistics on ATE With SSN .....           | 553        |
| Manufacturing Patterns With SSN .....            | 558        |
| Signoff Patterns With SSN .....                  | 577        |
| SSN SDC Constraints in the Design Flow .....     | 587        |
| Fast IO .....                                    | 625        |
| Burn-In Pattern Support .....                    | 642        |
| LVX Support .....                                | 647        |
| Retention Testing With SSN .....                 | 671        |
| Size Resolution Technology With SSN .....        | 673        |
| <b>Tessent SSN Examples and Solutions</b> .....  | <b>677</b> |
| Third-Party OCCs With SSN .....                  | 677        |
| <b>SSN Frequently Asked Questions</b> .....      | <b>679</b> |
| <b>SSN Limitations</b> .....                     | <b>681</b> |

## Tessent SSN Workflows

There are several SSN Workflows, which are based on the Tessent Shell Flow for Hierarchical Designs.

In the general SSN Workflow, you perform a bottom-up DFT insertion for each physical block followed by DFT insertion at the parent level. You continue this bottom-up flow until you have reached the top level of the design.

The SSN Workflow description focuses on the differences from the base flows described in the [Tessent Shell Workflows](#) section of this manual. The descriptions of the Workflows assume you understand the information contained in the following sections. It is recommended that you read them first if you are new to the Tessent Workflows.

- [“Tessent Shell Flow for Hierarchical Designs”](#)
- [“How the DFT Insertion Flow Applies to Hierarchical Designs”](#)
- [“Hierarchical DFT Terminology”](#)

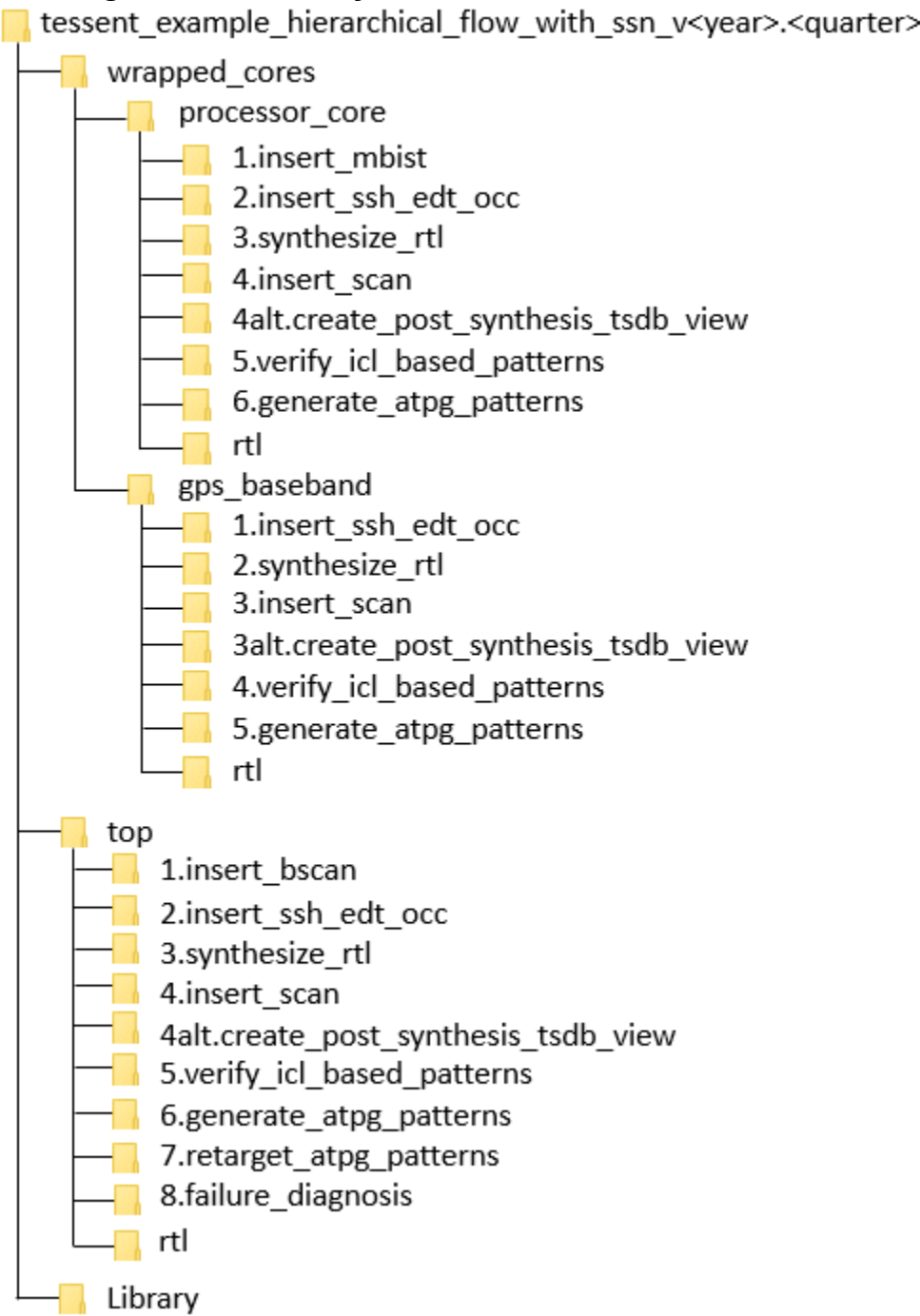
### Reference Testcase

The Tessent Shell release tree includes a testcase to accompany the block- and top-level workflows described in the following sections. From within Tessent Shell, use the **getenv TESSENT\_HOME** command to find the installation tree. The testcase is in *\$TESSENT\_HOME/share/UsageExamples* in the following file:

*tessent\_example\_hierarchical\_flow\_with\_ssn\_v<year>.<quarter>.tgz*

[Figure 9-1](#) shows the structure of the directories in the testcase. You can run each step of the workflow in one of these directories.

Figure 9-1. Directory Structure of Reference Testcase



|                                                                    |            |
|--------------------------------------------------------------------|------------|
| <b>Block-Level SSN Insertion and Verification</b>                  | <b>481</b> |
| First DFT Insertion Pass: Performing Block-Level MemoryBIST        | 483        |
| Second DFT Insertion Pass: Inserting Block-Level EDT, OCC, and SSN | 483        |
| Synthesis for Block-Level Insertion                                | 490        |
| Creating the Post-Synthesis TSDB View (Block-Level)                | 491        |
| Performing Scan Chain Insertion With Tessent Scan (Block-Level)    | 493        |

|                                                                            |            |
|----------------------------------------------------------------------------|------------|
| Verifying the ICL-Based Patterns After Synthesis (Block-Level) . . . . .   | 496        |
| Generating Block-Level ATPG Patterns . . . . .                             | 499        |
| <b>Top-Level SSN Insertion and Verification . . . . .</b>                  | <b>506</b> |
| First DFT Insertion Pass: Performing Top-Level MemoryBIST . . . . .        | 508        |
| Second DFT Insertion Pass: Inserting Top-Level EDT, OCC, and SSN . . . . . | 511        |
| Synthesis for Top-Level Insertion . . . . .                                | 519        |
| Creating the Post-Synthesis TSDB View (Top-Level). . . . .                 | 520        |
| Performing Scan Chain Insertion With Tessent Scan (Top-Level) . . . . .    | 520        |
| Verifying the ICL-Based Patterns After Synthesis (Top-Level) . . . . .     | 521        |
| Generating Top-Level ATPG Patterns . . . . .                               | 521        |
| Retargeting ATPG Patterns . . . . .                                        | 523        |
| Performing Reverse Failure Mapping for SSN Pattern Diagnosis . . . . .     | 526        |

## Block-Level SSN Insertion and Verification

This section presents the block-level Tessent Workflow for SSN by highlighting the minor differences from the normal workflow where SSN is not present.

The SSN Workflow for block-level insertion contains minor differences in the dofiles relative to the flow in the “[RTL and Scan DFT Insertion Flow for Physical Blocks](#)” section of this manual. These differences include minor additions and deletions required in the dofiles specific to each step. In [Figure 9-1](#) on page 479 in the section “[Tessent SSN Workflows](#)”, the *wrapped\_cores* subdirectories contain the block-level SSN flow. The text preceding the figure explains how to find the testcase within the release directory.

---

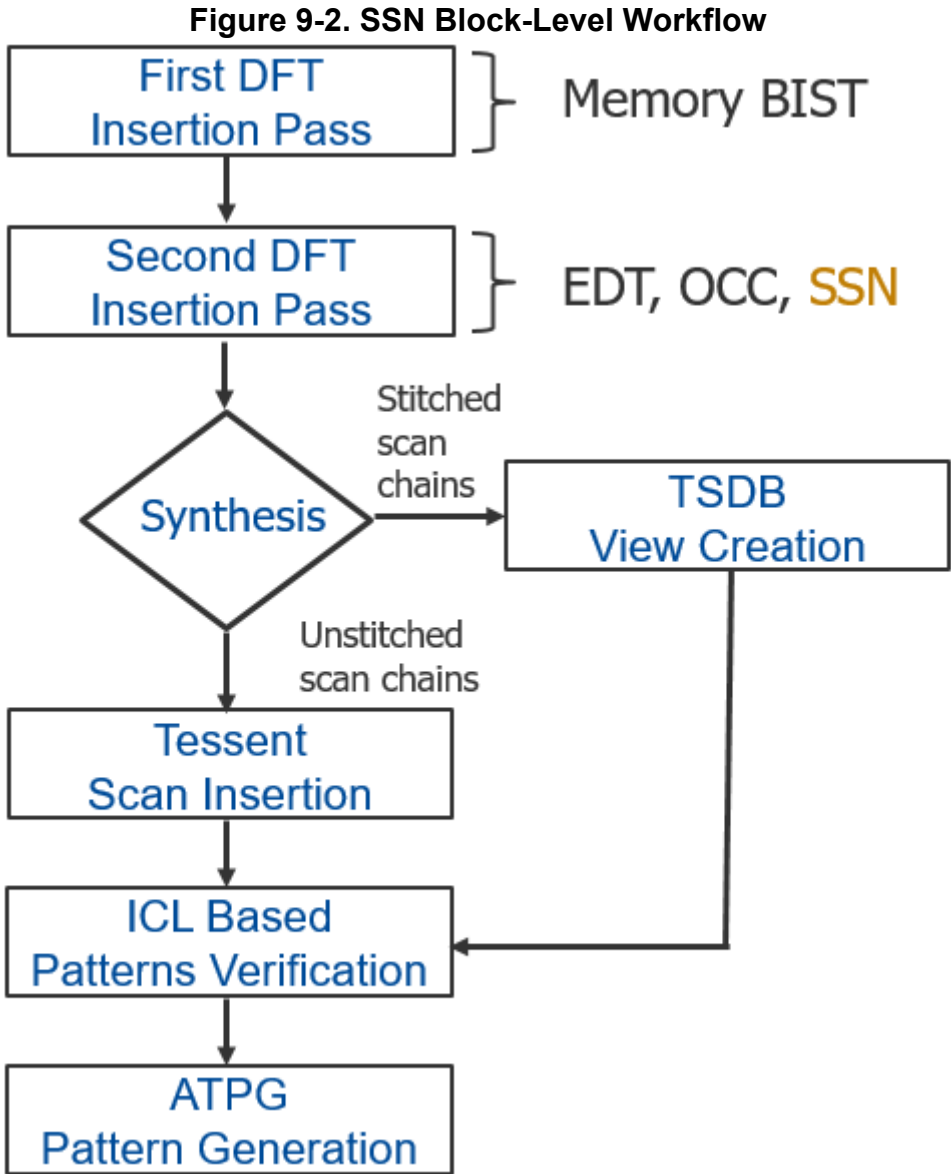
### Note



This procedure assumes your design meets the prerequisites of the hierarchical flow, as described in “[DFT Architecture Guidelines for Hierarchical Designs](#)”. Additionally, each physical block must have a full OCC with a shift clock injection mux and [shift\\_only\\_mode](#).

---

The following figure shows an identical block-level workflow used to process a child physical block to when you are not using SSN. As highlighted in the figure, you insert SSN into the design during the second DFT insertion pass. In this same step, you add a second smaller EDT to the design for the wrapper chains during external test. Click the boxes in the flow diagram to see the section describing each step.

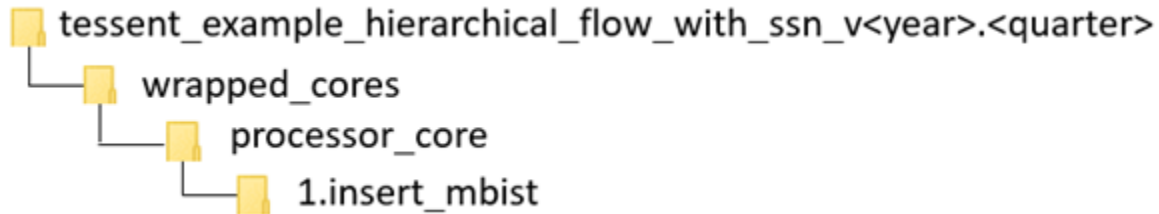


|                                                                         |     |
|-------------------------------------------------------------------------|-----|
| First DFT Insertion Pass: Performing Block-Level MemoryBIST.....        | 483 |
| Second DFT Insertion Pass: Inserting Block-Level EDT, OCC, and SSN..... | 483 |
| Synthesis for Block-Level Insertion .....                               | 490 |
| Creating the Post-Synthesis TSDB View (Block-Level).....                | 491 |
| Performing Scan Chain Insertion With Tessent Scan (Block-Level) .....   | 493 |
| Verifying the ICL-Based Patterns After Synthesis (Block-Level).....     | 496 |
| Generating Block-Level ATPG Patterns.....                               | 499 |

## First DFT Insertion Pass: Performing Block-Level MemoryBIST

The first DFT insertion pass is identical to the Tessent Shell Flow for Hierarchical designs.

You can perform this step in the following directory of the [Reference Testcase](#):



### Procedure

The first DFT insertion pass inserts the non-scan DFT elements, such as memory BIST and IJTAG. It happens before the second insertion pass inserts the logic test elements. Using SSN in the second DFT insertion pass does not affect the first DFT insertion pass. At the top level, you must equip the boundary scan cells with auxiliary input and output pins to connect the SSN bus. This change to the first DFT insertion pass for the top level is described in “[First DFT Insertion Pass: Performing MemoryBIST and Boundary Scan](#)” on page 180. For complete information about the current step at the block level, see “[First DFT Insertion Pass: Performing Block-Level MemoryBIST](#)” on page 211.

### Examples

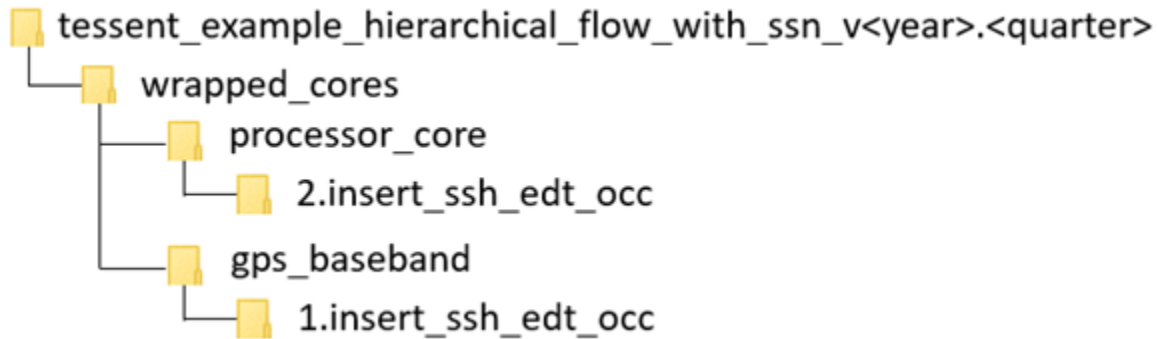
Refer to the example dofile *run\_mbist\_insertion* in the directory shown in the “Referenced Testcase Step” section earlier in this topic.

## Second DFT Insertion Pass: Inserting Block-Level EDT, OCC, and SSN

In the second DFT insertion pass, you insert the block-level EDT, OCC, and SSN elements.

This section explains the slight modifications that you do to the standard flow when you insert the SSN, EDT, and OCC into the physical block level. For a complete description of this step in the standard flow, refer to “[Second DFT Insertion Pass: Inserting Block-Level EDT and OCC](#)” on page 213.

You can perform this step in the following directories of the [Reference Testcase](#):



### Notes About This Procedure

- Similar to [EDT](#) and [OCC](#), the tool creates the SSN hardware based on the SSN wrapper of the DftSpecification. For a description of the SSN wrapper, see the “[SSN](#)” wrapper description in the *Tessent Shell Reference Manual*.
- When the tool creates the SSH, EDT, and OCC elements at the same time with a single invocation of the [process\\_dft\\_specification](#) command, the tool automates the connections between the SSH, EDT, and OCC instances.
- As with the normal flow, after the insertion of the SSN and the other logic test elements, you must simulate the ICLNetwork pattern to verify the correct IJTAG access to all inserted logic test elements, including the SSN elements. Following a successful simulation of the ICLNetwork pattern, you must simulate the SSN Continuity pattern to verify that the SSN datapath is correctly integrated into the design. The tool automates the ICL verification patterns and the SSN continuity patterns as part of the [create\\_patterns\\_specification](#) and [process\\_patterns\\_specification](#) commands. During validation of the SSN datapath, you are responsible for using the iProc command to write to the [Multiplexer](#) node to configure the datapath for which you want to verify continuity, as explained in Step 7 in the following procedure. You must complete simulation of both the ICLNetwork pattern and SSN continuity pattern before moving to the next step of the DFT flow.
- To use third-party OCC with SSN, refer to the topic “[Third-Party OCCs With SSN](#)” on page 677 in the “[Tessent SSN Examples and Solutions](#)” section.

To insert the logic test elements with SSN, use the following procedure to modify the dofile described in “[Second DFT Insertion Pass: Inserting Block-Level EDT and OCC](#)” on page 213.

### Lint Checks for the SSN Hardware



RTL output for SSN includes waivers for SpyGlass® to avoid lint errors. These waivers include comments detailing why the rules are waived and use the following format:

```
// <explanatory comment>
// spyglass disable_block <rule list>
// <waived code>
// spyglass enable_block <rule list>
```

The *<rule list>* uses SpyGlass naming for the rules that are being waived (for example, “W116 W164a”).

## Prerequisites

- SSN requires that each physical block have a full [OCC](#) with a clock injection multiplexer and support of the [shift\\_only\\_mode](#) feature.

## Procedure

1. Remove the [add\\_dft\\_signals](#) commands for the scan signals `edt_clock`, `edt_update`, `scan_en`, `shift_capture_clock`, and `test_clock`.

These signals are not needed because they are sourced by the [ScanHost](#) node during scan test. If you want to have your legacy channel access mechanism coexist with SSN for your first few designs, see [Example 2 in the ScanHost](#) reference page.

- a. Delete this line:

```
add_dft_signals scan_en edt_update test_clock -source_node \
{ scan_enable edt_update test_clock_u }
```

- b. Delete this line:

```
add_dft_signals edt_clock shift_capture_clock \
-create_from_other_signals
```

2. Update the “`create_dft_specification -sri_sib_list`” option to include SSN:

```
set spec [create_dft_specification -sri_sib_list {edt occ ssn} ]
```

This creates a dedicated IJTAG [Sib](#) node to serve as the host for the IJTAG interfaces of the [ScanHost](#) and [Multiplexer](#) SSN nodes.

3. Update the comment for the [report\\_config\\_syntax](#) command to include SSN as an option to view the syntax of the SSN DftSpecification:

```
// Use report_config_syntax DftSpecification/edt|occ|ssn to see full syntax.
```

4. Update the DftSpecification to include the [SSN](#) wrapper to create one or more SSN [Datapaths](#) with the nodes you want to insert in them. The following example creates a

32-bit wide Datapath with a [ScanHost](#) node preceded and followed by a [Pipeline](#) node. The SSN reference page contains several example of different SSN DftSpecifications:

```
SSN {
  ijtag_host_interface : Sib(ssn);
  DataPath(main) {
    output_bus_width      : 32;
    Pipeline(out) {
    }
    ScanHost(1) {
    }
    Pipeline(in) {
    }
  }
}
```

5. Update the EDT wrapper of the DftSpecification to include mode enables for the EDT, and add a second EDT for the wrapper chains to be used during external test mode.
  - a. Change the name of the EDT Controller from “Controller (c1)” to “Controller (c1\_int)” to indicate that this EDT controller is used for internal test mode and is different from the second EDT that is used in external test mode.
  - b. Add a Connections wrapper within the EDT “Controller (c1\_int)” wrapper with the [mode\\_enables](#) property to define the DFT Signal “int\_edt\_mode” as the mode enable for this EDT. This DFT Signal was added previously in the dofile and is used in the muxing logic that selects this EDT’s channel outputs to the SSH during the internal test mode.

```
EDT {
  ijtag_host_interface : Sib(edt);
  Controller (c1_int) {
    longest_chain_range      : 70, 80;
    scan_chain_count         : 20;
    input_channel_count      : 3;
    output_channel_count     : 3;
    Connections {
      mode_enables           : DftSignal(int_edt_mode);
    }
  }
}
```

- c. Add a second EDT controller for the wrapper chains to be used during external test mode. The “ext\_edt\_mode” DFT signal is used as the mode enable for this EDT. This DFT signal was added previously in the dofile and is used in the muxing logic that selects this EDT’s channel outputs to the SSH during the external test mode.

```

Controller (cl_ext) {
    longest_chain_range      : 70, 80;
    scan_chain_count        : 2;
    input_channel_count     : 1;
    output_channel_count    : 1;
    Connections {
        mode_enables        : DftSignal(ext_edt_mode);
    }
}

```

6. The SSN network that the specification just inserted is checked and extracted within the ICL model using the same [extract\\_icl](#) command that already checks and extracts the IJTAG network description. These design rule checks run automatically, and there is generally no need for additional input, except that you must specify the “-create\_ijtag\_graybox on” switch to create the IJTAG graybox in the TSDB.
7. The `create_patterns_specification` command creates a pattern specification for the ICL network that includes the SSN logic that was just inserted and all other element types, such as the Memory BIST logic (if present). The SSN continuity pattern is also created as part of the `create_patterns_specification` process and verifies the SSN datapath is properly connected. This is a mandatory verification step. The following example contains no [Multiplexer](#) node to configure at the block level. If you have such nodes at the block level, see how they are handled at the bottom of the dofile example in the section “[Second DFT Insertion Pass: Inserting Top-Level EDT, OCC, and SSN](#)” on page 511. Because you created an IJTAG graybox, the tool creates the ICL verification patterns wrapper and SSN continuity patterns wrapper using the IJTAG graybox view for both simulations.

---

### Note



You can also create the SSN continuity pattern with the low-level [create\\_ssn\\_continuity\\_patterns](#) command, as shown at the bottom of the example dofile found at the end of this section.

---

## Examples

The following is a dofile for the second DFT pass. You can also find it in the `run_ssh_edt_occ_insertion` scripts in the testcase directories shown in the “Referenced Testcase Step” section earlier in this topic. The testcase also includes the `run_continuity_sims` script that shows how to simulate the SSN continuity patterns.

```

# Set the context to insert DFT into RTL-level design
# Define a new design ID
set_context dft -rtl -design_id rtl2

# Set the location of the TSDB. Default is the current working directory.
set_tsdb_output_directory ../tsdb_outdir

# Read in the Tessent cell library
read_cell_library ../../../../library/standard_cells/tessent/adk.tcelllib
# Read in memory verilog model
read_verilog ../../../../library/memories/*.v -exclude_from_file_dictionary

```

```
# Use read_design with the design ID from the first DFT insertion pass
read_design processor_core -design_id rtl1 -verbose
set_current_design processor_core

# DFT Signal used for logic test
add_dft_signals ltest_en
# DFT Signal used to test memories with multiple load ATPG patterns
add_dft_signals memory_bypass_en
# DFT Signal required to test STI network during logic test
add_dft_signals tck_occ_en
# DFT Signals required for hierarchical DFT and used during scan insertion
add_dft_signals int_ltest_en ext_ltest_en int_mode, ext_mode, \
    int_edt_mode ext_edt_mode

report_dft_signals

# Run pre-DFT DRC rules
set_dft_specification_requirements -logic_test on
check_design_rules

# Create and report a DFT Specification
set_spec [create_dft_specification -sri_sib_list {edt occ ssn} ]
# Use report_config_syntax DftSpecification/edt|occ|ssn to see full syntax
report_config_data $spec

# Specify the logic to be created with the EDT, OCC, and SSN wrappers.
# The connection between the EDT, OCC, and SSH created by the too.
# Modify the below specification to your specific design requirements.
```

```

read_config_data -in_wrapper $spec -from_string {
  Occ {
    ijtag_host_interface : Sib(occ);
    include_clocks_in_icl_model : on;
    Controller(dco_clk) {
      clock_intercept_node      : dco_clk;
    }
  }
  SSN {
    ijtag_host_interface : Sib(ssn);
    DataPath(1) {
      output_bus_width      : 32;
      Pipeline(2) {
      }
      ScanHost(1) {
      }
      Pipeline(1) {
      }
    }
  }
  EDT {
    ijtag_host_interface : Sib(edt);
    Controller (c1_int) {
      longest_chain_range    : 70, 80;
      scan_chain_count       : 20;
      input_channel_count    : 3;
      output_channel_count   : 3;
      Connections {
        mode_enables         : DftSignal(int_edt_mode);
      }
    }
    Controller (c1_ext) {
      longest_chain_range    : 70, 80;
      scan_chain_count       : 2;
      input_channel_count    : 1;
      output_channel_count   : 1;
      Connections {
        mode_enables         : DftSignal(ext_edt_mode);
      }
    }
  }
}

# Display the content of the DftSpecification just defined above
report_config_data $spec

# Generate the hardware
process_dft_specification

# Extract IJTAG network and create ICL file for core level
extract_icl -create_ijtag_graybox on

# Write updated RTL into this new file to elaborate and synthesize later
write_design_import_script -use_relative_path_to . for_dc_synthesis.tcl -
replace

```

```

# Generate testbench for ICLNetwork and Continuity patterns
# Generate ICL Verify patterns. Use the variable to update it if needed.
set spec [create_patterns_specification]
process_patterns_specification

# Point to simulation libraries and run the Verilog simulation
set_simulation_library_sources \
    -v ../../../../library/standard_cells/verilog/adk.v \
    -v ../../../../library/memories/*.v
run_testbench_simulations

exit

```

The following is an alternate way to create the SSN continuity pattern. Refer to Step 7 in the Procedure section of this topic for an explanation of how to create the SSN continuity pattern.

```

#
# Create the continuity pattern
#
# Define SSN bus clock. If you are using time multiplexing
# you must specify the -freq_multiplier switch
add_clocks ssn_bus_clock

# check design rules and transition to analysis mode
check_design_rules

# set ijtag period
set_ijtag_retargeting_options -tck_period 10

# set ssn bus clock period
set_load_unload_timing_options -usage ssn -ssn_bus_clock_period 5

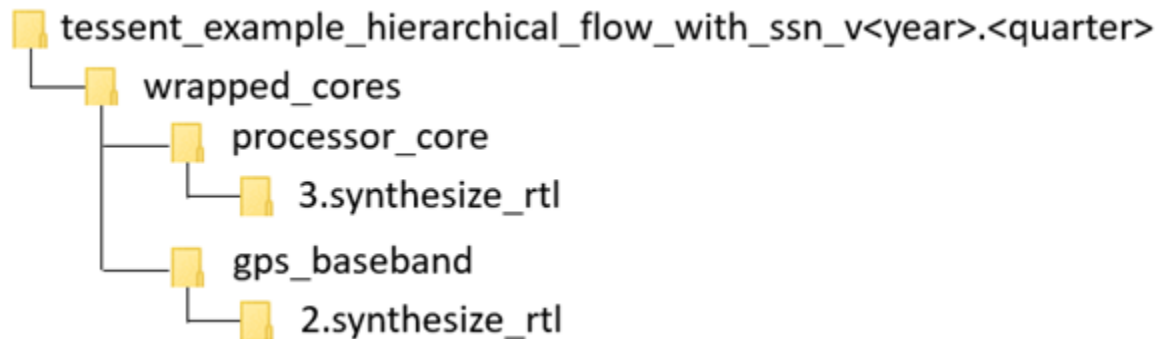
# Create the SSN continuity pattern
open_pattern_set ssn_continuity -usage ssn
    create_ssn_continuity_patterns
close_pattern_set
# Write simulation testbench
write_patterns patterns/ssn_continuity.v -verilog -replace \
    -parameter_list {SIM_COMPARE_SUMMARY 1}

```

## Synthesis for Block-Level Insertion

Use the following dofile changes to synthesize the DFT-inserted RTL block.

You can perform this step in the following directories of the [Reference Testcase](#):



In the dofile of the section “[Second DFT Insertion Pass: Inserting Block-Level EDT, OCC, and SSN](#)” on page 483, you used the [write\\_design\\_import\\_script](#) command to export a design load script for use in your synthesis tool.

Refer to the section “[Synthesis Guidelines for RTL Designs with Tessent Inserted DFT](#)” on page 1041 for guidelines on how to perform the synthesis step. Also, see the section “[Example Scripts Using Tessent Tool-Generated SDC](#)” on page 1019 for example scripts specific to various synthesis tools.

When synthesis completes, it produces a file containing the concatenated netlist of the physical block you just synthesized.

- If you performed scan insertion within the synthesis tool, continue with the steps described in the section “[Creating the Post-Synthesis TSDB View \(Block-Level\)](#)” on page 491.
- If you are inserting your scan chains within Tessent, continue with the steps described in the section “[Performing Scan Chain Insertion With Tessent Scan \(Block-Level\)](#)” on page 493.

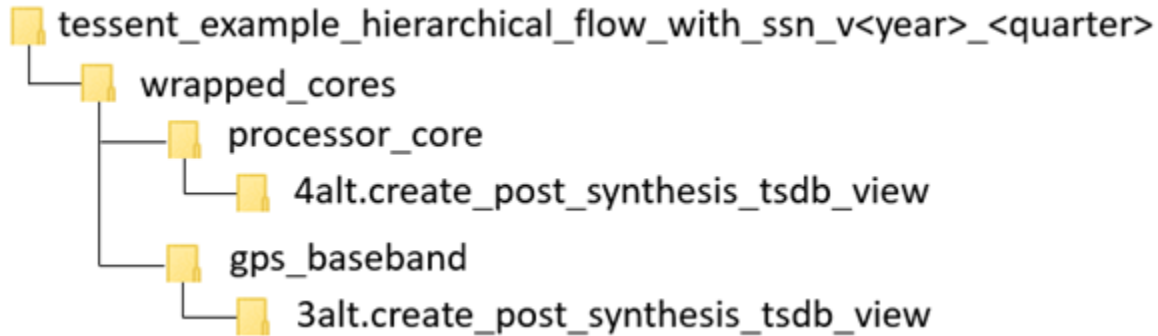
## Example

Example dofiles are available as the `<module_name>.dc_synth_script` files found in the testcase directories shown in the “Referenced Testcase Step” section earlier in this topic. These files specify the TCK period and the [set\\_load\\_unload\\_timing\\_options](#) values using a separate file called `<design_name>.timing_options`. When you source this file at this point, it configures the SDC during synthesis. A step later in the flow also sources this file from ATPG dofiles to configure the patterns with the exact timing options that the tool used during timing closure.

## Creating the Post-Synthesis TSDB View (Block-Level)

When you are using third-party scan insertion, use the following procedure to create the post-synthesis TSDB, separate from scan insertion. Creation of the post-synthesis TSDB occurs automatically as part of scan insertion when you use Tessent Scan.

You can perform this step in the following directories of the [Reference Testcase](#):



### Notes About This Procedure

This step combines a third-party scan-inserted netlist and the collateral from the RTL insertion step into a new *dft\_inserted\_design* directory. By consolidating the netlist and the collateral into the *dft\_inserted\_design* directory, you can use the tool automation in future steps to load the design using the [read\\_design](#) command.

The TSDB consolidation flow works by using the `read_verilog` command to read the third-party scan-inserted netlist into the tool and then using the “`read_design -no_hdl -design_id rtl2`” command to read the collateral from the last RTL insertion step. Write the design into the *tsdb\_outdir* directory using the “`write_design -tsdb -softlink_netlist -create_ijtag_graybox on`” command.

The following procedure and example dofile show how to create a *dft\_inserted\_design* directory with the post-synthesis TSDB flow.

### Procedure

1. Set the context to `-no_rtl` and define the `design_id` as “gate” using the [set\\_context](#) command.
2. Use the [read\\_verilog](#) command to read the scan-inserted design. This is the output of the third-party scan-insertion step.
3. Use the “`read_design -no_hdl -design_id rtl2`” command to load the collateral from the last DFT RTL insertion step.

The design collateral includes the block level ICL, PDL, SDC, TCD, and other files.

4. Use the “`write_design -tsdb -softlink_netlist -create_ijtag_graybox_on`” command to populate the *dft\_inserted\_design* directory with the collateral and a symbolic link to your scan-inserted netlist. This also creates a gate-level IJTAG graybox view for the design that you can use during the ICL-based pattern verification step, and also later during SSN scan retargeting at the top level.

The tool also updates design instance names in the ICL file. This happens when the ICL objects were below generated blocks in the RTL.



## Examples

The following is an example dofile for the TSDB consolidation step:

```
# Set context for this step
set_context dft -no_rtl -design_id gate

# Specify the TSDB output directory to write into.
set_tsdb_output_directory ../tsdb_outdir

# Read the scan-inserted design
read_verilog ../3.synthesize_rtl/processor_core_synthesized.vg

# Read only the collateral from the last DFT insertion step
read_design processor_core -design_identifier rtl2 -no_hdl -verbose
set_current_design processor_core

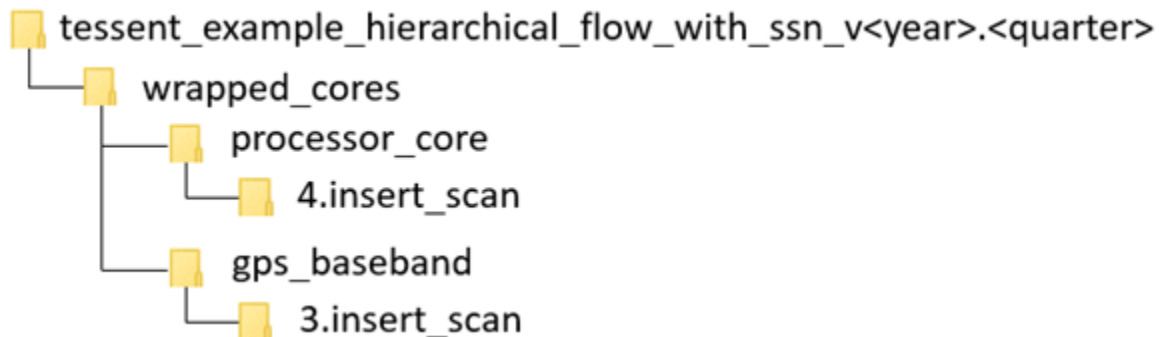
# Write the netlist and collateral to the tsdb_outdir
write_design -tsdb -softlink_netlist -create_ijtag_graybox on -verbose

exit
```

## Performing Scan Chain Insertion With Tessent Scan (Block-Level)

The Tessent Scan insertion step in the SSN flow is similar to the scan chain insertion step in the Tessent Shell Flow for hierarchical designs.

You can perform this step in the following directories of the [Reference Testcase](#):



The complete details for this step appear in the topic “[Performing Scan Chain Insertion: Wrapped Core](#)” on page 218.

When SSN is present, the tool connects wrapper chains to the second, smaller EDT controller added during the step “[Second DFT Insertion Pass: Inserting Block-Level EDT, OCC, and SSN](#)” on page 483.

The following procedure shows how to modify the dofile from the topic “[Performing Scan Chain Insertion: Wrapped Core](#)” when you are using SSN.

## Procedure

1. Remove the commands that exclude the `*_edt_channels_*` from wrapper analysis. For example, remove:

```
set_wrapper_analysis_options \  
-exclude_ports [ get_ports {*_edt_channels_*}]
```

In the SSN flow, the EDT channels connect directly to the [ScanHost](#) node instead of being brought to the boundary of the block.

2. Add the following code to find each EDT and assign it to its intended scan mode:

```
foreach_in_collection edt $edt_instance {  
  if {[regexp {.*int.*} [get_single_name $edt] int_edt_instance]}{  
    puts "Found instance $int_edt_instance. \  
        Will use this for internal scan_mode."  
  } elseif {[regexp {.*ext.*} [get_single_name $edt] \  
    ext_edt_instance]} {  
    puts "Found instance $ext_edt_instance. \  
        Will use this for external scan_mode."  
  }  
}
```

During the second DFT insertion pass, the tool gave each added EDT controller a name according to how it was intended to be used (internal mode EDT is “c1\_int” and external mode EDT is “c1\_ext”).

3. Add the following commands to assign each EDT to the intended scan mode. In this example, the scan mode is named using the name of the DFT signal that activates that mode:

```
add_scan_mode int_edt_mode -edt_instances $int_edt_instance  
add_scan_mode ext_edt_mode -edt_instances $ext_edt_instance
```

If you wanted to use different scan mode names, you would need to use the `-dft_signal_name` switch of the [add\\_scan\\_mode](#) command to associate the correct DFT signal to that mode.

4. After you implement all the scan modes using the [insert\\_test\\_logic](#) command, run the code block highlighted in gold at the end of the following example dofile to perform DRC on each scan mode and extract the graybox model for the external mode.
5. Create the gate-level IJTAG graybox view by changing the context to patterns-ijtag and then running the “analyze\_graybox -ijtag” command followed by the “write\_design -tsdb -graybox” command.

## Examples

The following is an example dofile for the scan insertion step:

```
# Set context to perform scan insertion and stitching  
set_context dft -scan
```

```
# Open the previous TSDB directories if not in the current working
# directory
set_tsdb_output_directory ../tsdb_outdir

# Read Tessent Library
read_cell_library ../../../../library/standard_cells/tessent/adk.tcelllib

# Read the synthesized netlist
read_verilog ../3.synthesize_rtl/processor_core_synthesized.vg

# Read the design data for the second RTL insertion pass
read_design processor_core -design_identifier rtl2 -no_hdl -verbose
set_current_design processor_core
add_black_boxes -modules { \
    SYNC_1RW_8Kx16 \
}
check_design_rules
report_clocks

# To force insertion of dedicated wrapper cell use the
# set_dedicated_wrapper_cell_options command
# set_dedicated_wrapper_cell_options on -ports {.... }
# Perform and Report wrapper cell analysis
analyze_wrapper_cells
report_wrapper_cells -Verbose

# Specify different modes (int/ext) how the chains need to be stitched
# The type internal/external and enable_dft_signal are inferred from
# registered DFT Signals(int_edt_mode and ext_edt_mode)
set edt_instance [get_instances -of_icl_instances [get_icl_instances -
filter tessent_instrument_type==mentor::edt]]
foreach_in_collection edt $edt_instance {
    if {[regexp {.*int.*} [get_single_name $edt] int_edt_instance]} {
        puts "Found EDT instance $int_edt_instance. Will use for int scan_mode."
    } elseif {[regexp {.*ext.*} [get_single_name $edt] ext_edt_instance]} {
        puts "Found EDT instance $ext_edt_instance. Will use for ext scan_mode"
    }
}
add_scan_mode int_edt_mode -edt_instances $int_edt_instance
add_scan_mode ext_edt_mode -edt_instances $ext_edt_instance

# Analyze the scan chains and review the different scan modes and chains
# before stitching the chains
analyze_scan_chains
report_scan_chains
```

```
# Insert scan chains and write the scan-inserted design into tsdb_outdir
insert_test_logic
report_scan_chains
report_scan_cells > scan_cells.list

# DRC check the scan chains in each mode and create graybox for
# external mode

set_context pattern -scan

foreach_in_collection mode_wrapper [get_config_elements \
    Core(processor_core)/Scan/Mode -part tcd -silent] {
    set mode_name [get_config_value $mode_wrapper -id <0>]
    set mode_type [get_config_value type -in $mode_wrapper]
    import_scan_mode $mode_name
    #In setup mode
    report_clocks
    check_design_rules
    #In Analysis mode
    if {$mode_type eq "external"} {
        report_scan_cells
        analyze_graybox -scan
        write_design -tsdb -graybox -verbose
    }
    set_system_mode setup
}

# Create the IJTAG graybox

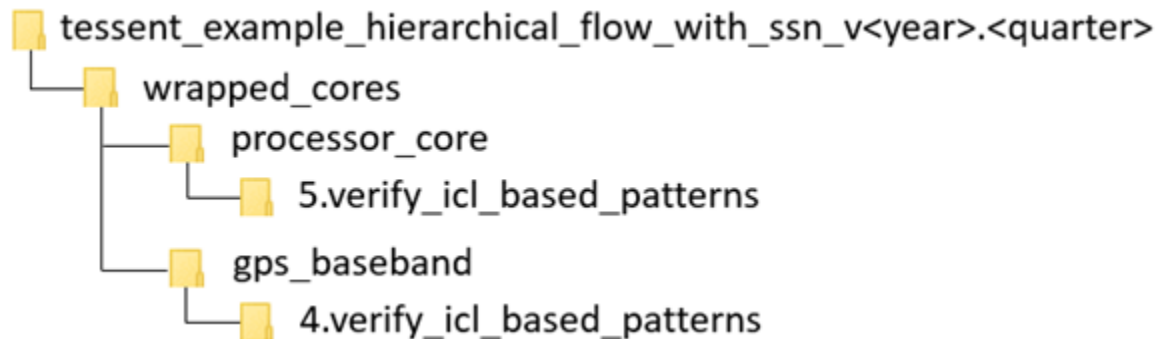
set_context patterns -ijtag
set_system_mode analysis
analyze_graybox -ijtag
write_design -tsdb -graybox -verbose

exit
```

## Verifying the ICL-Based Patterns After Synthesis (Block-Level)

Resimulating the MemoryBIST and ICL verification patterns after synthesis ensures the ICL network is fully functional and able to be reliably used during ATPG and Scan retargeting setup.

You can perform this step in the following directories of the [Reference Testcase](#):



### Notes About This Procedure

Verifying the ICL-based patterns after synthesis is especially critical when using SSN because the bulk of the test\_setup relies on IJTAG. Your MemoryBIST simulations are also redone in this step. For the complete details of this step, see “[Verifying the ICL Model](#)” on page 221.

When you use SSN, it is important to also reverify the SSN continuity after synthesis before you try to use it for ATPG and scan retargeting.

The following procedure and example dofile show how to verify an ICL model including SSN.

### Procedure

1. The top part of the dofile is identical to the normal flow. You can examine the PatternsSpecification generated by the [create\\_patterns\\_specification](#) command to see how it uses the IJTAG graybox view that you created in the previous step to simulate the ICLVerify patterns and the SSN continuity patterns.
2. As discussed earlier in this flow, you are responsible for configuring the SSN datapath you want to verify the continuity for by using iProcs containing iCall commands to the Tessent-generated pdl to write to the [Multiplexer](#) nodes when present. You must complete simulating both the ICLNetwork pattern and SSN continuity pattern before moving to the next step of the DFT flow.

#### Note

 You can also create the SSN continuity pattern with the low-level [create\\_ssn\\_continuity\\_patterns](#) command, as shown at the bottom of the example dofile found at the end of this section.

### Examples

This example dofile shows how to verify the ICL model for SSN.

The `create_patterns_specification` command creates a pattern specification for the SSN continuity pattern and the ICL network that includes the IJTAG logic, SSN logic, and the MemoryBIST logic, if present.

This example contains no [Multiplexer](#) node to configure. If you have such nodes at the block level, see how they are handled at the bottom of the dofile example in the section “[Second DFT Insertion Pass: Inserting Top-Level EDT, OCC, and SSN](#)” on page 511.

```
# Set context to patterns
set_context patterns

# Open the previous TSDB directories if it is not in the current

# working directory
set_tsdb_output_directory ../tsdb_outdir

# Read Tessent Library
read_cell_library ../../../../library/standard_cells/tessent/adk.tcelllib
read_verilog      ../../../../library/memories/SYNC_1RW_8Kx16.v -interface_only

# Read the design files from the scan insertion pass
read_design processor_core -design_identifier gate -verbose
set_current_design processor_core

# Generate patterns to verify the inserted DFT logic
set_spec [create_patterns_specification]
report_config_data $spec
process_patterns_specification

# Point to the libraries and run the simulation
set_simulation_library_sources -v ../../../../library/standard_cells/ \
    verilog/adk.v -v ../../../../library/memories/*.v
run_testbench_simulations
check_testbench_simulations -report_status

exit
```

The following is an alternate way to create the SSN continuity pattern. Refer to [Step 2](#) in the Procedure section of this topic for an explanation of how to create the SSN continuity pattern.

```
#
# Create the continuity pattern
#
# Define SSN bus clock. If you are using time multiplexing
# you must specify the -freq_multiplier switch
add_clocks ssn_bus_clock

# check design rules and transition to analysis mode
check_design_rules

# set ijtag period
set_ijtag_retargeting_options -tck_period 10

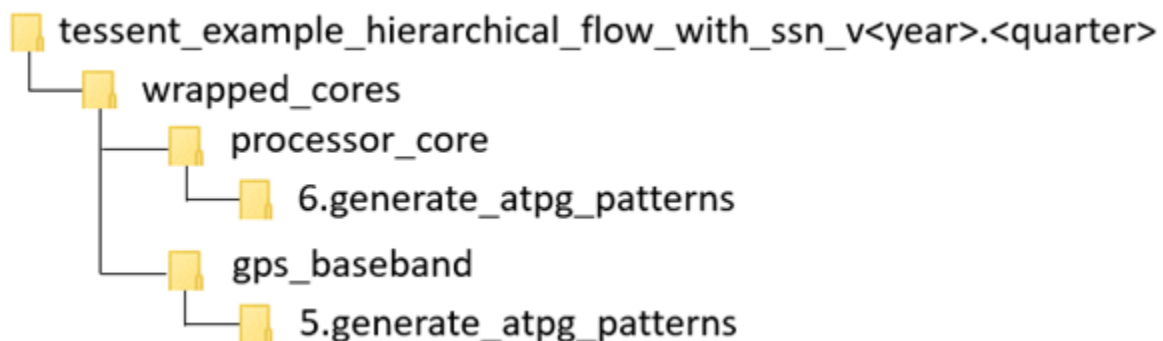
# set ssn bus clock period
set_load_unload_timing_options -usage ssn -ssn_bus_clock_period 5

# Create the SSN continuity pattern
open_pattern_set ssn_continuity -usage ssn
  create_ssn_continuity_patterns
close_pattern_set
# Write simulation testbench
write_patterns patterns/ssn_continuity.v -verilog -replace \
  -parameter_list {SIM_COMPARE_SUMMARY 1}
```

## Generating Block-Level ATPG Patterns

The process for generating ATPG patterns in the SSN flow is similar to the process used without SSN.

You can perform this step in the following directories of the [Reference Testcase](#):



### Notes About This Procedure

The complete details for this step appear in the “[Performing ATPG Pattern Generation: Wrapped Core](#)” on page 222. The SSN flow generates the scan graybox model as part of the “[Performing Scan Chain Insertion With Tessent Scan \(Block-Level\)](#)” step. Therefore, this step follows most of the standard flow, skipping the graybox generation because the insertion step has already completed that part. If you are performing scan insertion as part of synthesis, you create the graybox as part of creating ATPG patterns for the external mode, as shown in the

[Sample Dofile](#) in this section and illustrated in the `0opt.run_scan_graybox_generation` script in the reference testcase directories mentioned previously.

When SSN is present, the tool automatically generates all load and unload procedures and their timing based on the timing specified with the [set\\_load\\_unload\\_timing\\_options](#), [set\\_ijtag\\_retargeting\\_options](#), and [set\\_capture\\_timing\\_options](#) commands. The default bus\_clock frequency and shift\_clock frequency are 400 MHz and 100 MHz, respectively. Furthermore, you can improve the setup and hold timing around the edges of the scan\_en and edt\_updated signals. Use the `-scan_en_*_extra_cycles` and `-edt_update_*_extra_cycles` arguments to the [set\\_load\\_unload\\_timing\\_options](#) command to add extra cycles around the edges of scan\_en and edt\_update, respectively. If at any time during scan pattern retargeting or top-level ATPG you need to change the timing of a physical block, use the [set\\_current\\_physical\\_block](#) command to set the scope of all subsequent load/unload and capture timing settings. The reference page for the [set\\_load\\_unload\\_timing\\_options](#) command contains an explanation on how to use a single file that you source in both Tessent and the synthesis tool to configure the pattern generation and SDC consistently. The `1.run_internal_stuck` and `2.run_internal_transition` scripts in the reference testcase directories mentioned previously provide clear illustrations of this process. The `<module_name>.timing_options` file configures the SDC during synthesis, and you reuse it later to configure the patterns so that they are generated consistently.

If you did not use Tessent Scan, you cannot use the [import\\_scan\\_mode](#) command, and you use the [add\\_core\\_instances](#) command to select your active OCC and EDT controllers. You never need to use the `add_core_instances` command on the ScanHost instance. The tool automatically adds ScanHost core instances during the transition to analysis mode. When the tool performs the [check\\_design\\_rules](#) command, it automatically adds the ScanHost instance to which a scan chain or an active EDT traced. The `0opt.run_scan_graybox_generation` script in the reference testcase mentioned previously provides a clear illustration of this process.

After you run [create\\_patterns](#), the tool maps the SSN data stream to the SSN bus. It calculates SSH parameters when you write patterns with the [write\\_patterns](#) command. To see the final calculated parameters of the SSH, use the [report\\_core\\_instance\\_parameters](#) command after having called the `write_patterns` command.

Use the SSH loopback pattern to verify that the SSN network can successfully deliver packetized data to each active SSH. During the SSH loopback pattern, the ScanHost node does not send scan data out to the scan chains and EDT but instead loops it back internally.

- SSN patterns written in the serial testbench format deliver packetized scan data over the SSN data bus to each active SSH. Each active SSH transfers scan data to the scan chains and EDT and locally generates the scan signals. The scan out compares are mapped to the SSN bus\_out.
- SSN patterns written in the parallel testbench format apply scan data similarly to a non-SSN parallel testbench. The tool adds cut points to the boundary of the SSH to drive the scan signals.



During Streaming-Through-IJTAG, the tool replaces the parallel SSN bus by the serial JTAG interface. Select the serial JTAG interface as the streaming interface with the [set\\_ssn\\_options](#) command. The tool delivers packetized scan data synchronously through the TDI port using TCK as the clock. You can stream any SSN pattern set through the serial JTAG interface without modification. For more information about Streaming-Through-IJTAG, refer to “[Streaming-Through-IJTAG Scan Data](#)” on page 532.

You can create SSN patterns using a scaled-down SSN bus width. Scale the SSN bus width with the [set\\_ssn\\_datapath\\_options](#) command. You can reapply any SSN pattern using a scaled-down SSN bus.

When simulating patterns at the core level, the SSN bus clock typically runs slower because it is limited by shift clock frequency. To run the bus clock at maximum frequency, set the “[SIM\\_SSN\\_MAXIMUM\\_BUS\\_SPEED 1](#)” parameter value pair with the “[write\\_patterns -parameter\\_list](#)” command and switch to add padding to the packet.

SSN supports only the *.patdb* file format for scan pattern retargeting.

## Prerequisites

- Running ATPG with SSN requires a validated ICL model of the current design.

## Procedure

1. Run ATPG in internal mode on the wrapped core.

This step in the SSN flow is similar to the step “Run ATPG on the internal mode of the wrapped core” in the topic “[Performing ATPG Pattern Generation: Wrapped Core](#)” on page 222, in the hierarchical flow section of this manual. Refer to this section for a detailed description of this step.

Use the following steps to update the dofile from the hierarchical flow procedure referenced previously:

- a. Set the `ijtag_tck` period for this core to the required frequency using the [set\\_ijtag\\_retargeting\\_options](#) command. This should be the `ijtag_tck` clock frequency you closed timing at.
- b. Define the SSN bus\_clock and shift\_clock periods for the core using the [set\\_load\\_unload\\_timing\\_options](#) command. The bus\_clock period should be the maximum bus\_clock frequency you plan to use for the chip. If you specify a slower frequency, it limits the bus clock frequency of the entire datapath when this ATPG pattern is retargeted. The default bus\_clock period is 2.5 ns, and the default shift\_clock is 10 ns. The *1.run\_internal\_stuck* and *2.run\_internal\_transition* scripts in the reference testcase directories mentioned previously show you how to do this with the synthesis process used.
- c. Define the clock frequency used during the capture process. The SSH is capable of having a different frequency for the `shift_capture_clock` during the shift and the

capture cycles. The default slow capture clock frequency is 40 MHz to enable reliable capture staggering.

- d. Report core instances after you run [check\\_design\\_rules](#). As part of `check_design_rules`, the tool automatically adds the SSH core instance that was traced to from the active scan chains and EDT controllers.
- e. Write the SSH loopback pattern. Simulate this pattern to confirm that the SSN network can successfully deliver packetized data to the SSH. Start by simulating the parallel load scan patterns. Once those are clean, simulate the serial SSH loopback pattern, followed by one Serial Chain pattern and one Serial Scan pattern.

The sample dofile in the following Examples section also creates the on-chip comparator self-test pattern. This pattern set is only relevant for blocks having a ScanHost that supports the on-chip compare mode. This pattern set is not required for sign-off simulation but is critical during manufacturing test to ensure that no fault within the comparator logic can be present, which could mask faults within the scanned logic to be detected. Refer to the section “[On-Chip Compare With SSN](#)” on page 535 for more information about this pattern set.

- f. Run the “`set_chain_test -type nomask`” command. This enables writing the chain test patterns without masking. Simulating all chain test patterns is not practical for most designs. When you write the chain test patterns with the “`-end 0`” option, the tool writes one chain test pattern that simulates all the chains more efficiently.
- g. Write a single serial scan pattern. One serial scan pattern combined with a full set of parallel patterns provides sufficient coverage of the SSN and the scan chains.

Refer to the Examples section for a sample dofile updated using these steps. Also, see the *1.run\_internal\_stuck* and *2.run\_internal\_transition* scripts in the reference testcase directories mentioned previously.

2. Run ATPG in external mode on the wrapped core.

This step in the SSN flow is similar to the step “Run ATPG on the external mode of the wrapped core” in the topic “[Performing ATPG Pattern Generation: Wrapped Core](#)” on page 222, in the hierarchical flow section of this manual. During the external mode run, the core-level SSH sources the scan signals to the small wrapper chain EDT.

---

#### **Note**



This ATPG pattern is used only to verify the proper behavior of the wrapper chains. It cannot be reused from the chip level.

---

The procedure for modifying the dofile presented in the hierarchical flow section is the same as the procedure for modifying the internal mode dofile. Refer to Substeps “a” through “g” in Step 1 for the instructions on updating your dofile. See the Examples section for a sample dofile updated using these steps.

## Examples

### Sample Dofile

The following dofile shows the result of following the instructions for internal and external mode ATPG. Because the dofile for the two modes are very similar, the following dofile shows the full internal mode dofile. The differences for an external mode dofile appear as comments in green text marked “EXTERNAL MODE:”.

All changes from the base hierarchical flow for SSN appear in gold text.

```
set_context patterns -scan

# Set the location of the TSDB. Default is the current working directory.
set_tsdb_output_directory ../tsdb_outdir

# Read Tessent Library
read_cell_library ../../../../library/standard_cells/tessent/adk.tcelllib

# Read in the scan inserted netlist/design
read_design processor_core -design_id gate -verbose

set_current_design processor_core

# Specify the current mode using a unique name (different from
# add_scan_mode).
# Then bring in required scan mode configuration
set_current_mode int_edt_mode_stuck -type internal
import_scan_mode int_edt_mode -fast_capture off
# EXTERNAL MODE:
# set_current_mode ext_edt_mode_stuck -type external
# import_scan_mode ext_edt_mode -fast_capture off

# Define the tck period and ijtag retargeting options
set_ijtag_retargeting_options -tck_period 10

# Define the shift and bus clock timing
set_load_unload_timing_options -usage ssn \
                                -shift_clock_period 10ns \
                                -ssn_bus_clock_period 5ns

# Define the slow clock capture timing
set_capture_timing_options -usage internal \
                            -capture_clock_period 40ns

# Report core instances that were added as part of importing scan mode
report_core_instances

set_core_instance_parameters -instrument_type occ -parameter_values \
    {capture_window_size 3}

report_dft_signals
report_core_instances
report_static_dft_signal_settings

check_design_rules
```

```
# EXTERNAL MODE:
# If you inserted your scan chains during synthesis, extract the graybox
# model here.
# analyze_graybox
# write_design -graybox -tsdb -verbose

report_clocks
report_core_instances

# Confirm the SSN core instance was added
report_core_instance_parameters
set_fault_type stuck

add_fault -all
report_statistics -detail

# Generate patterns
create_patterns
report_statistics -detail

# Store TCD, flat_model, fault list and patDB format files in the TSDB
# directory
write_tsdb_data -replace
```

```
# Write Verilog patterns for simulation
# Write parallel load testbench
write_patterns patterns/processor_core_int_edt_mode_stuck_parallel.v \
-verilog -replace -parameter_list {SIM_COMPARE_SUMMARY 1} \
-pattern_set scan

# Write SSH loopback pattern
write_patterns patterns/processor_core_int_edt_mode_stuck_loopback.v \
-verilog -serial -replace -parameter_list {SIM_COMPARE_SUMMARY 1} \
-pattern_set ssh_loopback

# Use this option for signoff simulations to verify all chains with a
# single chain pattern
set_chain_test -type nomask
write_patterns patterns/processor_core_int_edt_mode_stuck_serial_chain.v \
-verilog -serial -replace -end 0 -parameter_list \
{SIM_COMPARE_SUMMARY 1} -pattern_set chain

write_patterns patterns/processor_core_int_edt_mode_stuck_serial_scan.v \
-verilog -serial -replace -end 0 -parameter_list \
{SIM_COMPARE_SUMMARY 1 ALL_EXCLUDE_UNUSED 0} \
-pattern_set scan

# EXTERNAL MODE:
# write_patterns patterns/processor_core_ext_edt_mode_stuck_parallel.v \
# -verilog -replace -parameter_list {SIM_COMPARE_SUMMARY 1} \
# -pattern_set scan

# write_patterns patterns/processor_core_ext_edt_mode_stuck_loopback.v \
# -verilog -serial -replace -parameter_list {SIM_COMPARE_SUMMARY 1} \
# -pattern_set ssn_loopback

# Use this option for signoff simulations to verify all chains with a
# single chain pattern
# set_chain_test -type nomask
# write_patterns patterns/processor_core_ext_edt_mode_stuck_serial_chain.v \
# -verilog -serial -replace -end 0 -parameter_list \
# {SIM_COMPARE_SUMMARY 1} -pattern_set chain
# write_patterns patterns/processor_core_ext_edt_mode_stuck_serial_scan.v \
# -verilog -serial -replace -end 0 -parameter_list \
# {SIM_COMPARE_SUMMARY 1 ALL_EXCLUDE_UNUSED 0} -pattern_set scan

# Write SSH on-chip comparator self test patterns
write_patterns patterns/processor_core_int_ssh_occomp_self_test.v \
-verilog -serial -replace -end 8 -parameter_list \
{SIM_COMPARE_SUMMARY 1} -pattern_set ssh_on_chip_compare
# report fault coverage
report_statistics -detail

exit
```

## Top-Level SSN Insertion and Verification

During top-level SSN insertion, the tool inserts other logic test elements into the design. This DFT insertion flow is similar to the one where SSN is not present.

This section explains the differences in the dofiles relative to the standard hierarchical flow (described in the section “[RTL and Scan DFT Insertion Flow for the Top Chip](#)” on page 228). These differences include modifications that specific dofiles require.

---

### Note

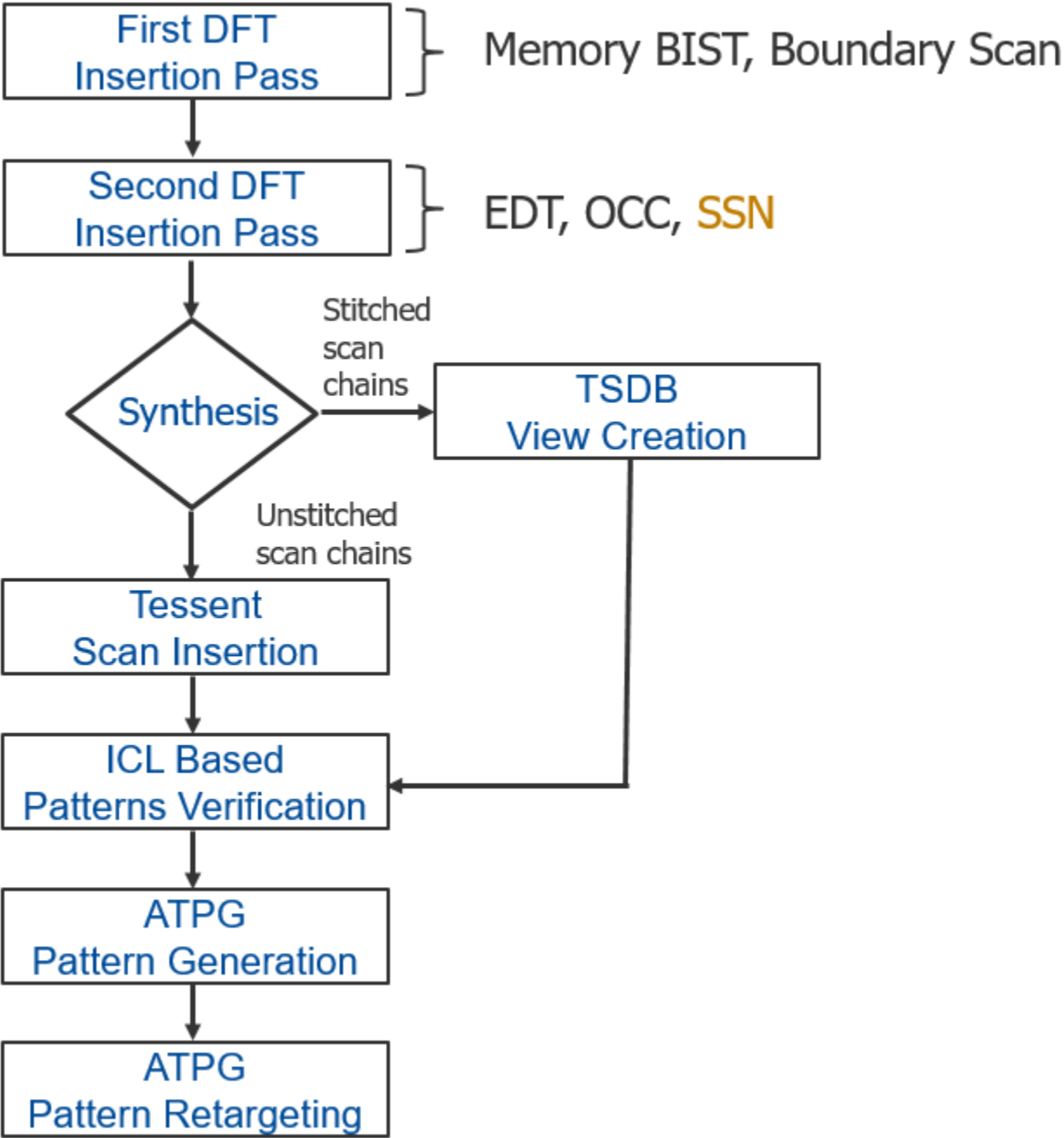


This procedure assumes your design meets the prerequisites of the hierarchical flow, as described in “[DFT Architecture Guidelines for Hierarchical Designs](#)”. Additionally, each physical block must have a full OCC with a shift clock injection mux and [shift\\_only\\_mode](#).

---

[Figure 9-3](#) shows the same top-level workflow you follow when not using SSN. It has been updated to indicate that you insert SSN into the design during the second DFT insertion pass. This step adds a single EDT if there is top-level logic. You can also add the boundary scan chains to this top-level EDT using the [BoundaryScan/max\\_segment\\_length\\_for\\_logictest](#) property. Unlike at the core-level SSN insertion flow, this process does not add a second, smaller EDT, because the top-level boundary scan chains can provide isolation for the top level during top-level ATPG. Click the boxes in the flow diagram to see the section describing each step. The steps described in this section are implemented in the “top” directory of the [Reference Testcase](#).

Figure 9-3. SSN Top-Level Workflow



First DFT Insertion Pass: Performing Top-Level MemoryBIST ..... 508

Second DFT Insertion Pass: Inserting Top-Level EDT, OCC, and SSN ..... 511

Synthesis for Top-Level Insertion ..... 519

Creating the Post-Synthesis TSDB View (Top-Level) ..... 520

Performing Scan Chain Insertion With Tessent Scan (Top-Level) ..... 520

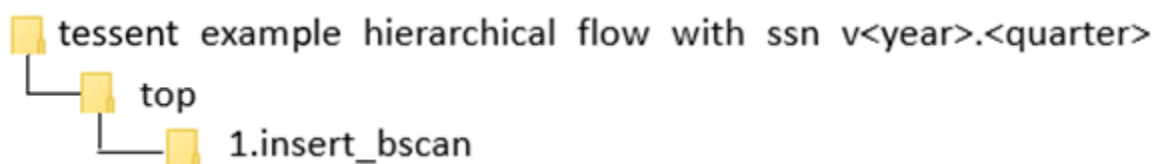
Verifying the ICL-Based Patterns After Synthesis (Top-Level) ..... 521

|                                                                    |     |
|--------------------------------------------------------------------|-----|
| Generating Top-Level ATPG Patterns .....                           | 521 |
| Retargeting ATPG Patterns .....                                    | 523 |
| Performing Reverse Failure Mapping for SSN Pattern Diagnosis ..... | 526 |

## First DFT Insertion Pass: Performing Top-Level MemoryBIST

The first DFT insertion pass at the top level adds non-scan DFT elements to the design. This step is similar to the first DFT insertion pass of the standard hierarchical flow. However, it uses the auxiliary input and output pins to connect the SSN bus and the bus\_clock instead of using them for the EDT channels.

You can perform this step in the following directory of the [Reference Testcase](#):



### Notes About This Procedure

- For a complete description of this step in the standard hierarchical flow, refer to the section “[First DFT Insertion Pass: Performing Top-Level MemoryBIST and Boundary Scan](#)” on page 231. To define the auxiliary logic pins for connecting the SSN bus and bus\_clock, modify the dofile described in this referenced procedure.
- Identify the top-level input and output ports in your design that you plan to use for the SSN bus and bus\_clock. The bus should have the same number of input and output ports (for example, 16 inputs and 16 outputs).
- The “[set\\_dft\\_specification\\_requirements -memory\\_test](#)” switch in the reference testcase is set to “On” even though this testcase has no memories in the top level. This requests that the memory clocks at the boundary of the child blocks receive design rule checks (DRCs) all the way to the top. Although the [extract\\_icl](#) process would detect any issues, you would need to redo the DFT insertion to fix them. By performing the DRC before DFT, you can detect and correct those issues sooner.

### Prerequisites

- SSN requires that the top-level design has an IEEE 1149.1 interface available during logic test.
- The lower-level physical blocks should have SSN inserted prior to inserting SSN into the top-level design. If the SSN is incomplete for any lower-level physical blocks, use the `ijtag_greybox` view to model the SSN datapath through the physical block during top-level SSN insertion. Alternatively, use an SSN multiplexer to force the SSN datapath around the incomplete physical block.



## Procedure

Modify the `auxiliary_input_ports` and `auxiliary_output_ports` wrappers with the top-level ports you plan to use for the SSN bus and `bus_clock`:

```
# Add auxiliary mux on the inputs and outputs used for SSN bus and
# bus_clock
# bus_in {GPIO3_0 GPIO3_1} bus_out {GPIO4_0 GPIO4_1} bus_clock
{GPIO3_2}
read_config_data -in ${spec}/BoundaryScan -from_string {
  AuxiliaryInputOutputPorts {
    auxiliary_input_ports    : GPIO3_0, GPIO3_1, GPIO3_2;
    auxiliary_output_ports   : GPIO4_0, GPIO4_1;
  }
}
```

## Examples

The following dofile is for the first DFT insertion pass. It contains changes to support SSN. These changes appear in gold text.

```
# Set the context to insert DFT into top-level design
set_context dft -rtl -design_id rtl1

# Set the location of the TSDB. Default is the current working directory.
set_tsdb_output_directory ../tsdb_outdir

# Open the TSDB of all the child cores
open_tsdb ../../wrapped_cores/processor_core/tsdb_outdir
open_tsdb ../../wrapped_cores/gps_baseband/tsdb_outdir

# Read the tessent cell library
read_cell_library ../../library/standard_cells/tessent/adk.tcelllib

# Read the design
read_verilog ../rtl/noncore_blocks/p11.v -blackbox

set_design_sources -format verilog -v ../rtl/noncore_blocks/iopad_sel.v
read_verilog ../rtl/chip_top.v
read_verilog ../rtl/rds_process.v
set_current_design chip_top

set_design_level chip

# Define set_dft_specification_requirements to insert boundary scan at
# chip level
set_dft_specification_requirements -boundary_scan on -memory_Ttest on

# Specify the TAP pins using set_attribute_value
set_attribute_value TCK -name function -value tck
set_attribute_value TDI -name function -value tdi
set_attribute_value TMS -name function -value tms
set_attribute_value TRST -name function -value trst
set_attribute_value TDO -name function -value tdo
```

```
# Specify all clocks so that the proper BSCAN cells gets inserted
automatically for them

add_clocks PLL_1/pll_clock_0 -reference REF_CLK -freq_multiplier 16
add_clock REF_CLK -period 48ns
add_clock INCLK -period 10ns

check_design_rules

# Create and report a DFT Specification
set_spec [create_dft_specification]

report_config_data $spec

# Segment the boundary scan to be used during logic test
set_config_value $spec/BoundaryScan/max_segment_length_for_logictest 80

# Add auxiliary mux on the inputs and outputs used for SSN bus and
# bus_clock
# bus_in {GPIO3_0 GPIO3_1} bus_out {GPIO4_0 GPIO4_1} bus_clock {GPIO3_2}
read_config_data -in ${spec}/BoundaryScan -from_string {
  AuxiliaryInputOutputPorts {
    auxiliary_input_ports : GPIO3_0, GPIO3_1, GPIO3_2;
    auxiliary_output_ports : GPIO4_0, GPIO4_1 ;
  }
}

report_config_data $spec

# Generate and insert the hardware
process_dft_specification -transcript_insertion_commands

# Extract IJTAG network and create ICL file for the design
extract_icl

# Create patterns(testbenches) to verify the inserted DFT logic
set_spec [create_patterns_specification]
process_patterns_specification

# Point to the libraries and run simulation
set_simulation_library_sources -v ../../library/standard_cells/verilog/*.v \
                              -v ../rtl/noncore_blocks/pll.v \
                              -v ../rtl/noncore_blocks/iopad_sel.v \
                              -v ../../library/memories/SYNC_1RW_8Kx16.v

run_testbench_simulations

exit
```

## Second DFT Insertion Pass: Inserting Top-Level EDT, OCC, and SSN

In the second DFT insertion pass at the top level, you insert any EDT, OCC, and SSN elements that exist at the top level. In addition, you use the `DftSpecification` to specify the physical order of lower-level cores from the SSN bus\_in to SSN bus\_out.

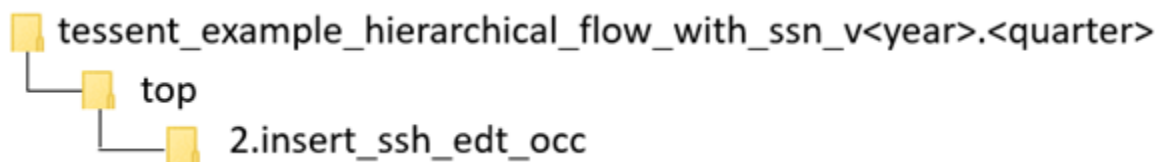
Specify this physical order using the following elements. SSN nodes that appear at the top of the `DftSpecification` are closest to SSN data\_out.

- [DesignInstance](#) wrapper
- Any of the following SSN nodes along the datapath:
  - [Pipeline](#)
  - [Receiver1xPipeline](#)
  - [OutputPipeline](#)
  - [Multiplexer](#)
  - [BusFrequencyDivider](#)
  - [BusFrequencyMultiplier](#)

This section explains the slight modifications that you do to the standard flow when you insert the SSN, EDT, and OCC at the top level. For a complete description of this step in the standard flow, refer to “[Second DFT Insertion Pass: Inserting Block-Level EDT and OCC](#)” on page 213.

### Reference Testcase Step

You can perform this step in the following directory of the [Reference Testcase](#):



### Notes About This Procedure

- Similar to the logic test insertion passes at the block level, the tool creates the SSN hardware based on the SSN wrapper of the `DftSpecification`. For a description of the SSN wrapper, see the “[SSN](#)” wrapper description in the *Tessent Shell Reference Manual*.
- When the tool creates the SSH, EDT, and OCC elements at the same time with a single `process_dft_specification` command, the tool automates the connections between the SSH, EDT, and OCC instances.

- You must complete an ICL-based verification step at the end of the second DFT insertion pass in order to verify the DFT elements you just added to the IJTAG network. The ICL-based patterns are created as part of running the [create\\_patterns\\_specification](#) and [process\\_patterns\\_specification](#) commands, and you simulate them using the [run\\_testbench\\_simulations](#) command. The following are the ICL-based pattern verifications:
  - **ICLNetwork Pattern** — Verifies IJTAG access to all new DFT elements added to the IJTAG network and to the child blocks.
  - **SSN Continuity Pattern** — Verifies you have correctly integrated the SSN datapath into your design and detects potential problems along the datapath. If your datapath includes SSN multiplexers, you must configure the multiplexer select to test each leg of the multiplexer, as demonstrated in step 5 and at the bottom of the *1.run\_ssh\_edt\_occ\_insertion* script in the directory of the reference testcase mentioned previously.

---

#### Note



You can also create the SSN continuity pattern with the low-level [create\\_ssn\\_continuity\\_patterns](#) command, as shown at the bottom of the example dofile found at the end of this section.

---

- To use a third-party OCC with SSN, refer to the topic “[Third-Party OCCs With SSN](#)” on page 677 in the “[Tessent SSN Examples and Solutions](#)” section.

To insert the logic test elements with SSN, use the following procedure to modify the dofile described in “[Second DFT Insertion Pass: Inserting Block-Level EDT and OCC](#)” on page 213.

## Prerequisites

- SSN requires that each physical block have a full [OCC](#) with a clock injection multiplexer and support of the [shift\\_only\\_mode](#) feature.
- It is recommended that you have terminated the wrapper chains of the child blocks on a local small EDT and that EDT is driven by the ScanHost node local to the block, as it was done in the flow description found in the section “[Second DFT Insertion Pass: Inserting Block-Level EDT, OCC, and SSN](#)” on page 483.

## Procedure

1. Remove the [add\\_dft\\_signals](#) commands for the scan and retargeting signals `edt_clock`, `edt_update`, `shift_capture_clock`, `test_clock`, `retargeting1_mode`, `retargeting2_mode`, `retargeting3_mode`, and `retargeting4_mode`.

The scan signals (`edt_clock`, `edt_update`, `shift_capture_clock`, and `test_clock`) are not needed because they are sourced by the SSH located in the top level. The scan signals to the lower-level EDT and wrapper chain are sourced by the SSH inside the lower-level child core. When the lower-level child cores are in external test mode, the internal mode EDT is inactive.

The retargeting signals (retargeting<n>\_mode) are not needed because each core-level EDT is accessible through the [ScanHost](#) node, effectively decoupling the dependency between core-level EDT channels and top-level I/O. With SSN, you can retarget each core to the top level individually or concurrently with any combination of the other cores.

To remove these commands, delete these lines:

```
add_dft_signals test_clock edt_update -source_nodes \  
    {TEST_CLOCK_top EDT_UPDATE_top}  
  
add_dft_signals shift_capture_clock edt_clock \  
    -create_from_other_signals  
  
add_dft_signals retargeting1_mode retargeting2_mode \  
    retargeting3_mode retargeting4_mode
```

2. Add the ssn\_en DFT signal:

```
# DFT Signal used for logic test  
add_dft_signals ltest_en ssn_en
```

3. Remove all of the [add\\_dft\\_modal\\_connections](#) commands that add multiplexer logic to support scan pattern retargeting.
4. Add the SSN wrapper to the DftSpecification, as shown later in this step. The Datapath connections point to the top-level ports that the previous step equipped with auxiliary input and output logic during Boundary Scan. The tool automatically maps the specified port connections to the corresponding auxiliary data pins and automatically connects the auxiliary en pins to the ssn\_en DFT signal.

A Multiplexer node wraps the two DesignInstance wrappers associated with the GPS instances. You may need to create a similar setup in your design if the datapath through the those two cores can be powered down. One way to avoid those multiplexers is to make sure the SSN datapath remains in the always on domain. In this example, it is used to illustrate the way you create an iProc to setup the datapath so that you can use it during SSN continuity and Scan retargeting.

```

read_config_data -in_wrapper $spec -from_string {
  SSN {
    ijttag_host_interface : Sib(ssn);
    Datapath(1) {
      output_bus_width : 2;
      Connections {
        bus_clock_in : GPIO3_2;
        bus_data_in   : GPIO3_%d;
        bus_data_out  : GPIO4_%d;
      }
      OutputPipeline(1) {
      }
      ScanHost(1) {
      }
      DesignInstance(PROCESSOR_1) {
      }
      Multiplexer(gps_byp) {
        DesignInstance(GPS_2) {
        }
        DesignInstance(GPS_1) {
        }
      }
      Receiver1xPipeline(1) {
      }
    }
  }
}

```

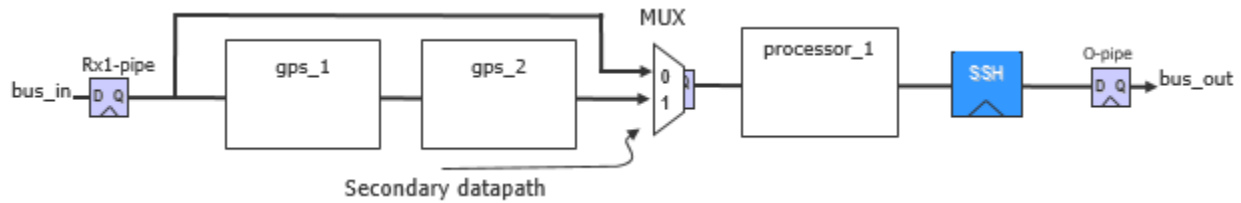
5. Create the SSN continuity pattern to test the primary and secondary paths through the SSN multiplexer.
  - a. Identify the different paths from SSN bus\_in to bus\_out through each leg of the SSN multiplexers. In the example in substep b, a single multiplexer is in the datapath. Some complex datapaths may contain multiple SSN multiplexers.
  - b. Create an **iProc** to program the secondary datapath through the multiplexer. Reuse the iProc during ATPG pattern generation to ensure that the datapath during ATPG has been checked. Some complex datapaths may have multiple iProc procedures to program the different multiplexer combinations.

```

#
# PDL for configuring top-level mux.
# When mux select=1'b1, gps_baseband physical blocks are included
#
iProcsForModule chip_top
iProc include_gps_blocks_in_ssn_datapath {} {
  iCall chip_top_rtl2_tessent_ssn_mux_gps_byp_inst.setup \
    select_secondary_bus 0b1
}

```

**Figure 9-4. Multiplexer Node for Bypassing the gps\_baseband Blocks**



- c. For each configuration of the multiplexers, use the ProcedureStep wrapper to create the continuity pattern.

```

# Add path through gps_baseband physical blocks to continuity pattern.
read_config_data -last -in wrapper $spec/Patterns(SSN) -from_string {
  ProcedureStep(cfg_datapath) {
    iCall(include_gps_blocks_in_ssn_datapath) {
    }
  }
  SSNContinuityVerify(include_gps_baseband) {
  }
}

```

6. Add the “-create\_ijtag\_graybox on” switch to the extract\_icl command. The ICL-based patterns verification step also makes use of the IJTAG graybox views of the top and child levels to optimize the simulations.

## Examples

The following is a dofile for the second DFT pass.

```

# Set the context to insert DFT
set_context dft -rtl -design_id rtl2

# Use dft_cell_selection that is part of the library
read_cell_library ../../library/standard_cells/tessent/adk.tcelllib

# Set the location of the TSDB. Default is the current working directory.
set_tsdb_output_directory ../tsdb_outdir

# Open the TSDB of all the child cores
open_tsdb ../../wrapped_cores/processor_core/tsdb_outdir
open_tsdb ../../wrapped_cores/gps_baseband/tsdb_outdir

# Read the tessent cell library
read_cell_library ../../library/standard_cells/tessent/adk.tcelllib

# Read the verilog
read_verilog ../rtl/noncore_blocks/pll.v -blackbox
set_design_sources -format verilog -v ../rtl/noncore_blocks/iopad_sel.v

# Read the synthesized netlist
read_design chip_top -design_id rtl1 -verbose
read_design processor_core -design_id gate -view graybox -verbose
read_design gps_baseband -design_id gate -view graybox -verbose

set_current_design chip_top

```

```
# The design level is already specified in the first pass; no need to specify it
# again

# Add DFT Signals
# DFT Signal used for BoundaryScan to be used with logic test without
contacting inputs
add_dft_signals int_ltest_en output_pad_disable
# DFT Signals used for Scan Tested Network with Instruments like
# memorybist/boundary scan
add_dft_signals tck_occ_en
# DFT Signal used for logic test
add_dft_signals ltest_en
# DFT Signal used by top-level EDT
add_dft_signals edt_mode_ssn_en

# Specify pre-DFT DRC rules
set_dft_specification_requirements -logic_test On

check_design_rules
report_dft_control_points

# Create and report a DFT Specification
set_spec [create_dft_specification -sri_sib_list {occ edt_ssn} ]
report_config_data $spec
# Use report_config_syntax DftSpecification/edt|occ to see full syntax

# The edt_clock and edt_update signals are automatically connected to EDT
instances
# The EDT controller is built with Bypass
# The shift_capture_clock is automatically connected to OCC instances
read_config_data -in_wrapper $spec -from_string {
  SSN {
    ihtag_host_interface : Sib(ssn);
    DataPath(1) {
      output_bus_width :2;
      Connections {
        bus_clock_in : GPIO3_2;
        bus_data_in  : GPIO3_%d;
        bus_data_out : GPIO4_%d;
      }
      OutputPipeline(1) {
      }
      ScanHost(1) {
      }
      DesignInstance(PROCESSOR_1) {
      }
      Multiplexer(gps_byp) {
        DesignInstance(GPS_2) {
        }
        DesignInstance(GPS_1) {
        }
      }
      Receiver1xPipeline(1) {
      }
    }
  }
}
```



```

Occ {
    ijtag_host_interface : Sib(occ);
    include_clocks_in_icl_model : yes;
    Controller(pll_clock_0) {
        clock_intercept_node      : PLL_1/pll_clock_0;
    }
    Controller(INCLK) {
        clock_intercept_node      : INCLK;
    }
}

EDT {
    ijtag_host_interface : Sib(edt);
    Controller (c1) {
        longest_chain_range : 60, 100;
        scan_chain_count : 17;
        input_channel_count : 2;
        output_channel_count : 2;
        Connections {
            EdtChannelsIn(1) {
            }
            EdtChannelsOut(1) {
            }
        }
    }
}

}

# Generate and insert the hardware
process_dft_specification

# Note the effective data and clock rate comments above each SSN node in the
# following command
report_config_data $spec

# Extract IJTAG network and create ICL file for the design
extract_icl -create_ijtag_graybox on

# Creates a synthesis script of ALL the RTL (original and newly-created)
# to be used in your synthesis tool in next Step
write_design_import_script -use_relative_path_to . for_dc_synthesis.tcl -replace

# Generate patterns to verify the inserted DFT logic
set_defaults_value /PatternsSpecification/SignoffOptions/
simulate_instruments_in_lower_physical_instances on

```

```
# Generate patterns. Use the variable to update it if needed.
set spec [create_patterns_specification]
# chip level iProc to select secondary datapath through mux
source ../ssn_datapath_configuration.pdl
# Add path through gps_baseband physical blocks to continuity pattern.
read_config_data -last -in_wrapper $spec/Patterns(SSN) -from_string {
    ProcedureStep(cfg_datapath) {
        iCall(include_gps_blocks_in_ssn_datapath) {
        }
    }
    SSNContinuityVerify(include_gps_baseband) {
    }
}

# Validate pattern specification
process_patterns_specification

# Point to the libraries and run the simulation
set_simulation_library_sources -v ../../library/standard_cells/verilog/*.v \
                               -v ../rtl/noncore_blocks/pll.v \
                               -v ../rtl/noncore_blocks/iopad_sel.v \
                               -v ../../library/memories/SYNC_1RW_8Kx16.v

run_testbench_simulations -simulation_macro_definitions
TESSENT_DISABLE_CLOCK_MONITOR
check_testbench_simulations

exit
```

The following is an alternate way to create the SSN continuity pattern. Refer to Step 5 in the Procedure section of this topic for an explanation of how to create the SSN continuity pattern.

```
#
# Create the continuity pattern
#

# Define SSN bus clock. If you are using time multiplexing
# you must specify the -freq_multiplier switch.
add_clocks GPIO3_2

# Check design rules and transition to analysis mode
check_design_rules

# set ijtag period
set_ijtag_retargeting_options -tck_period 10

# set ssn bus clock period.
set_load_unload_timing_options -usage ssn -ssn_bus_clock_period 5

# chip level iProc to select secondary datapath through mux
source ../ssn_datapath_configuration.pdl
```

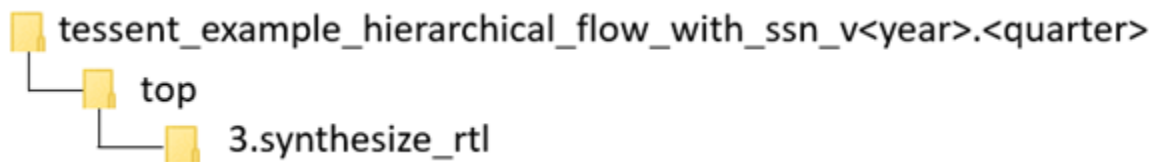
```
# Create the SSN continuity pattern
open_pattern_set ssn_continuity -usage ssn
# This is the default datapath configuration
create_ssn_continuity_patterns
# Select secondary datapath side of MUX
iCall include_gps_blocks_in_ssn_datapath
create_ssn_continuity_patterns
close_pattern_set

# Write simulation and STIL patterns
write_patterns patterns/ssn_continuity.v -verilog -replace -parameter_list \
{SIM_COMPARE_SUMMARY 1}
write_patterns patterns/ssn_continuity.stil -stil -replace
```

## Synthesis for Top-Level Insertion

Use the following dofile changes to synthesize the DFT-inserted RTL at the top level.

You can perform this step in the following directory of the [Reference Testcase](#):



In the dofile of the section “[Second DFT Insertion Pass: Inserting Top-Level EDT, OCC, and SSN](#)” on page 511, you used the `write_design_import_script` command to export a design load script for use in your synthesis tool.

Refer to the section “[Synthesis Guidelines for RTL Designs with Tessent Inserted DFT](#)” on page 1041 for guidelines on how to perform the synthesis step. Also, see the section “[Example Scripts Using Tessent Tool-Generated SDC](#)” on page 1019 for example scripts specific to various synthesis tools.

When synthesis completes, it produces a file containing the concatenated netlist of the physical block you just synthesized.

- If you performed scan insertion within the synthesis tool, continue with the steps described in the section “[Creating the Post-Synthesis TSDB View \(Block-Level\)](#)” on page 491.
- If you are inserting your scan chains within Tessent, continue with the steps described in the section “[Performing Scan Chain Insertion With Tessent Scan \(Block-Level\)](#)” on page 493.

## Example

Example dofiles are available as the `<module_name>.dc_synth_script` files found in the testcase directories shown in the “Referenced Testcase Step” section earlier in this topic. These

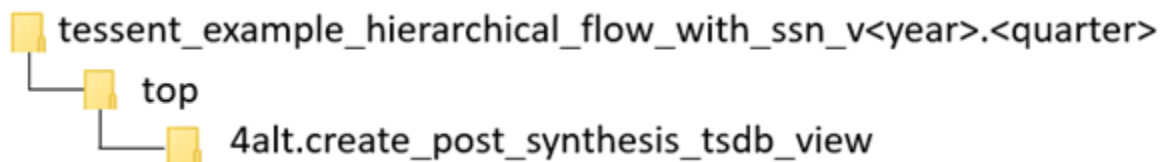
files specify the TCK period and the [set\\_load\\_unload\\_timing\\_options](#) values using a separate file called `<design_name>.timing_options`. When you source this file at this point, it configures the SDC during synthesis. A step later in the flow also sources this file from ATPG dofiles to configure the patterns with the exact timing options that the tool used during timing closure.

## Creating the Post-Synthesis TSDB View (Top-Level)

When you are using third-party scan insertion, use the following procedure to create the post-synthesis TSDB, separate from scan insertion. Creation of the post-synthesis TSDB occurs automatically as part of scan insertion when you use Tessent Scan.

### Reference Testcase Step

You can perform this step in the following directory of the [Reference Testcase](#):



### Procedure

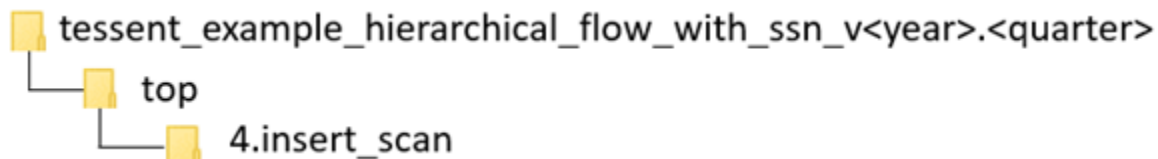
Creating the post-synthesis TSDB view at the top level is identical to the equivalent step at the block level. The only difference is that you also need to open the TSDB to the child blocks. Refer to the section “[Creating the Post-Synthesis TSDB View \(Block-Level\)](#)” on page 491 for the details. The `create_post_synthesis_tsdb_view` script in the directory shown previously in the reference testcase also illustrates this process.

## Performing Scan Chain Insertion With Tessent Scan (Top-Level)

The Tessent Scan insertion step in the SSN flow is similar to the scan chain insertion step in the Tessent Shell Flow for hierarchical designs.

### Reference Testcase Step

You can perform this step in the following directories of the [Reference Testcase](#):



## Procedure

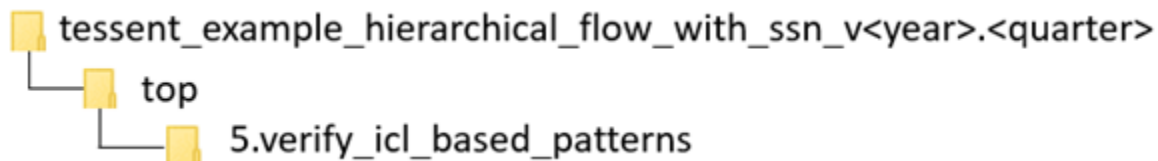
When you are using SSN, the scan insertion step at the top level is very similar to the flow you use at the block level. The only difference is that you do not need wrapper chains and, typically, have a single scan mode. The boundary scan chains implemented in the section “[First DFT Insertion Pass: Performing Top-Level MemoryBIST](#)” on page 508 were built so that the EDT controller can control them during scan test, and they can serve as isolation from the outside. Those chains were preconnected to the EDT ports and are left untouched during scan insertion. Refer to the section “[Performing Scan Chain Insertion With Tessent Scan \(Block-Level\)](#)” on page 493 for more details. It may also be helpful to look at the `run_scan_insertion` script in the directory shown previously in the reference testcase.

## Verifying the ICL-Based Patterns After Synthesis (Top-Level)

Resimulating the MemoryBIST and ICL verification patterns after synthesis ensures the ICL network is fully functional and able to be reliably used during ATPG and Scan retargeting setup.

### Reference Testcase Step

You can perform this step in the following directory of the [Reference Testcase](#):



## Procedure

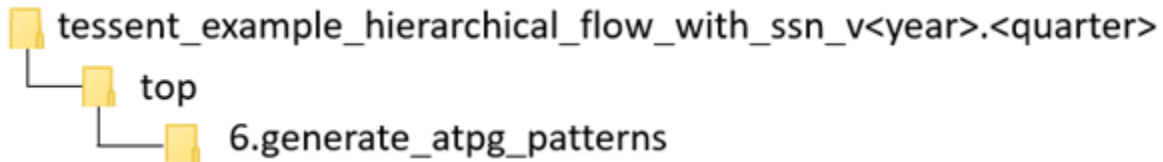
The verification of the ICL-based patterns at the top level is identical to the procedure at the block level. The only difference in the script is that you must open the TSDB for the child blocks. See the section “[Verifying the ICL-Based Patterns After Synthesis \(Block-Level\)](#)” on page 496 for the full details. It may also be helpful to review the `1.create_post_synthesis_icl_verification_patterns` script in the directory shown previously in the reference testcase.

## Generating Top-Level ATPG Patterns

The process for generating ATPG patterns in the SSN flow is similar to the process used without SSN.

### Reference Testcase Step

You can perform this step in the following directories of the [Reference Testcase](#):



## Procedure

Generating the ATPG patterns at the top level is identical to generating the patterns at the block level. The block levels are in external mode when you create top-level ATPG patterns, so you only need to read in their scan graybox views using the “`read_design -view scan_graybox`” command.

You also need to import the `.timing_options` file used for all blocks so that you create the ATPG patterns to satisfy the maximum frequencies within all the blocks. The `1.run_tk_chip_stuck` and `2.run_tk_chip_transition` scripts in the directory shown previously for the reference testcase illustrate the loading of the timing options. Use the following Tcl code used to perform this import procedure:

```
# Import timing options used during synthesis
set tck_period 0
foreach {physical_block path} {
    processor_core ../../wrapped_cores/processor_core/3.synthesize_rtl \
    gps_baseband   ../../wrapped_cores/gps_baseband/2.synthesize_rtl \
    chip_top       ../3.synthesize_rtl/} {

    set_current_physical_block -module $physical_block
    source ${path}/${physical_block}.timing_options
    if {[set ${physical_block}_tck_period] > $tck_period} {
        puts "Imported tck_period from $physical_block"
        set tck_period [set ${physical_block}_tck_period]
    }
}

# Define the tck period and ijttag retargeting options
set_ijtag_retargeting_options -tck_period $tck_period
```

Refer to the section “[Generating Block-Level ATPG Patterns](#)” on page 499 for more details about the content of the dofile, or examine the `1.run_tk_chip_stuck` and `2.run_tk_chip_transition` scripts in the directory shown previously for the reference testcase. The file `3.run_sims` in same directory shows how to perform the simulation. It compiles the full view of the top level and the scan graybox view of the child blocks.

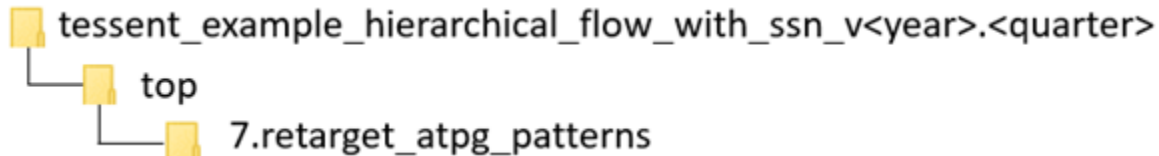
Similar to the block-level process, the parallel load scan patterns are simulated first, followed by the serial SSH loopback patterns and then the one Serial Chain and one Serial Scan patterns.

## Retargeting ATPG Patterns

The steps you use to retarget the ATPG scan patterns from the block to the top level are very similar to the steps used when not using SSN. When SSN is present, the process is easier and more flexible.

### Reference Testcase Step

You can perform this step in the following directories of the [Reference Testcase](#):



### Notes About This Procedure

When you use SSN, the design does not have a `scan_mode` to set at the top level that configures the channel access to the required blocks. Instead, you use the `add_core_instances` command to specify which mode you want to retarget on a given set of block modules or instances. During the DFT signoff process, you typically retarget one block at a time. You can speed up the simulation by using the IJTAG graybox views of all blocks other than the one you are retargeting. This is the usage model that the reference testcase demonstrates. The testcase retargets the stuck and transition patterns for `process_core` into a set of testbenches independently from the stuck and transition patterns for the `gps_baseband` block. The `3.run_retargt_processor_core_sims` and `6.run_retargt_gps_baseband_sims` scripts in the reference testcase directory show how to compile the correct view for each simulation.

When you want to create the manufacturing patterns, you can retarget any number of blocks in parallel. The flow is identical to the one in the dofile in the Examples section, except that you use the `add_core_instances` command to specify all the blocks you want to include in the given retargeting pattern set. The SSN procedure has the advantage that you do not hard code the grouping of blocks at design time, so you can optimize them later, between power and test time.

The example dofile shows how you source the `.timing_options` files from each block again at this time to ensure you create the SSN patterns consistently with how timing was closed within each block.

## Procedure

1. Load the design in Tessent Shell.

Regardless of which blocks you are retargeting, you always load the IJTAG graybox view of all blocks when doing scan retargeting with SSN. The three “`read_design -view ijtag_graybox`” commands used in the example dofile illustrate how to do this.

2. Import the timing options files for all blocks.

As shown in the dofile in the Examples section and in the description of the previous step, the tool automatically configures the SSN patterns to satisfy the requirements specified with the [set\\_load\\_unload\\_timing\\_options](#) command. You use the same command to configure the Tessent SSN SDC during timing closure. Specifying the command in a distinct file for each block enables you to share the exact same specification during timing closure and pattern generation to ensure they are always consistent.

3. Configure the SSN Multiplexer node to provide access to all active SSH blocks.

The third section of highlighted code in the example dofile is commented out because the iCall it contains configures the Multiplexer node when you want to include the `gps_baseband`. However, this dofile only retargets the patterns for the `processor_core` block. As illustrated in [Figure 9-4](#) on page 515 in the section “[Second DFT Insertion Pass: Inserting Top-Level EDT, OCC, and SSN](#)”, an inserted [Multiplexer](#) node enables you to bypass the `gps_baseband` blocks. When you want to retarget the scan patterns of the `gps_baseband` block, you must configure the Multiplexer to include them in the active SSN datapath.

## Examples

The following dofile example illustrates how to perform scan retargeting with SSN.

```
# Set the Context to retarget ATPG Patterns from lower level child cores
set_context pattern -scan_retargeting

# Point to the TSDB directory
set_tsdb_output_directory ../tsdb_outdir

# Open all the TSDB of the child cores
open_tsdb ../../wrapped_cores/processor_core/tsdb_outdir
open_tsdb ../../wrapped_cores/gps_baseband/tsdb_outdir

# Read the tessent cell library
read_cell_library ../../library/standard_cells/tessent/adk.tcelllib

# Read the hard macros
read_verilog ../../library/plls/pll.v -interface_only
read_verilog ../../library/memories/SYNC_1RW_8Kx16.v -interface_only
```



```
#For scan retargeting with SSN, only the ijtag_graybox view is needed.
read_design chip_top -design_id gate -view ijtag_graybox
read_design processor_core -design_id gate -view ijtag_graybox
read_design gps_baseband -design_id gate -view ijtag_graybox

set_current_design chip_top

# Import timing options used during synthesis
set tck_period 0
foreach {physical_block path} {processor_core ../../wrapped_cores/
processor_core/3.synthesize_rtl \
                                gps_baseband    ../../wrapped_cores/
gps_baseband/2.synthesize_rtl \
                                chip_top         ../3.synthesize_rtl/} {
    set_current_physical_block -module $physical_block
    source ${path}/${physical_block}.timing_options
    if {[set ${physical_block}_tck_period] > $tck_period} {
        puts "Imported tck_period from $physical_block"
        set tck_period [set ${physical_block}_tck_period]
    }
}

# Define the tck period and ijtag retargeting options
set_ijtag_retargeting_options -tck_period $tck_period

# Define the slow clock capture timing
set_capture_timing_options -usage internal \
                            -capture_clock_period 40ns

# Retarget Stuck-at patterns from processor_core
set_current_mode retarget1_processor_stuck

add_core_instances -instances {PROCESSOR_1} -core processor_core -mode
int_edt_mode_stuck
import_clocks

# Configure SSN datapath to include all SSH
# This is not needed for the processor_core as it is always part of
# the active SSN datapath

# source ../ssn_datapath_configuration.pdl
# set_test_setup_icall include_gps_blocks_in_ssn_datapath -append

check_design_rules

# Write the TCD for the chip-level in the TSDB directory
write_tsdb_data -replace

# Read the patterns to be retargeted. The mode is specified with the
add_core_instances command.
read_patterns -module processor_core -fault_type stuck
```

```
# Write Verilog patterns for simulation
# Write parallel load testbench
write_patterns patterns/\
    top_retarget_processor_core_stuck_parallel_scan.v \
    -parallel -v -replace -parameter_list {SIM_COMPARE_SUMMARY 1} \
    -pattern_set scan

# ssh loopback pattern
write_patterns patterns/\
    top_retarget_processor_core_stuck_ssh_loopback.v \
    -serial -v -replace -parameter_list {SIM_COMPARE_SUMMARY 1} \
    -pattern_set ssh_loopback

# ssh on-chip comparator self test when OComp mode present
# Not needed for Sign-off sims but critical as a manufacturing pattern
write_patterns patterns/top_retarget_gps_baseband_occomp_self_test.v \
    -serial -v -replace -parameter_list {SIM_COMPARE_SUMMARY 1} \
    -pattern_set ssh_on_chip_compare

# Write a single chain and single scan test pattern
write_patterns patterns/top_retarget_processor_core_stuck_serial_chain.v \
    -serial -v -replace -end 0 -parameter_list {SIM_COMPARE_SUMMARY 1} \
    -pattern_set chain
write_patterns patterns/top_retarget_processor_core_stuck_serial_scan.v \
    -serial -v -replace -end 0 -parameter_list {SIM_COMPARE_SUMMARY 1} \
    -pattern_set scan

exit
```

## Performing Reverse Failure Mapping for SSN Pattern Diagnosis

SSN patterns include core-level patterns retargeted to the top level to create chip-level tester patterns. In this way, they are similar to retargeted patterns for a hierarchical DFT design. Tessent Diagnosis requires a core-level design flat model as an input. This means that, to diagnose failures, you must reverse-map the tester-reported failures at the top level to their corresponding core level, which enables you to map the reported transactions at the SSN bus level into transactions at the active SSH and EDT scan chain level. This failure mapping process must include top-level ATPG. Tessent Diagnosis then uses the mapped failure file, design flat model, and test pattern database (patdb) to perform the diagnosis.

The following figure shows a simplified Tessent Diagnosis flow as a two-step flow. In the first step, you map tester-generated failures to core-level failures. Refer to the section “[Reverse Mapping Top-Level Failures to the Core](#)” in the *Tessent Diagnosis User’s Manual* for more details. In the second step, you run Tessent Diagnosis using the mapped failure files.

---

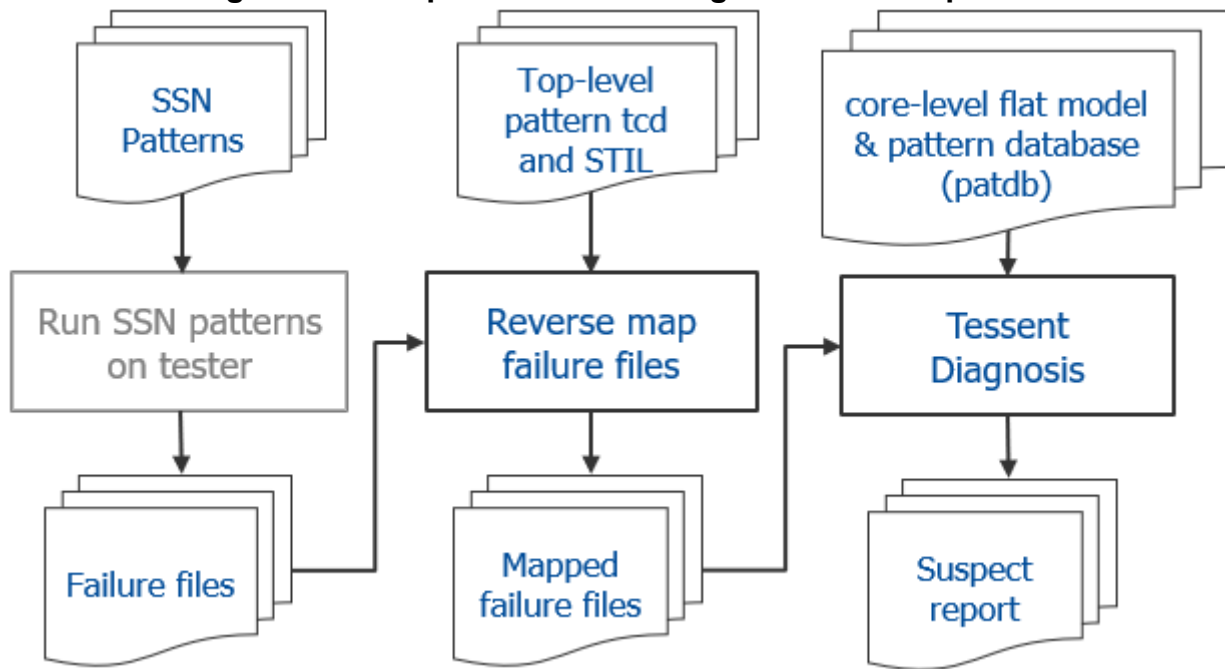
### Note



This procedure validates that reverse mapping failures back to the core level is working properly. It does not validate diagnosis.

---

**Figure 9-5. Simplified Tessent Diagnosis Two-Step Flow**



#### Note

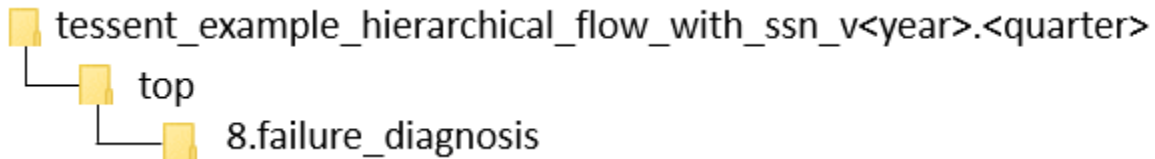
When you write SSN patterns with the `-max_loads` option and exclude one of the pattern files from being applied to the DUT on the tester, you should also exclude that file from the reverse failure mapping process from reading that pattern file.

Tessent tools support a flow for validating the reverse failure mapping process that leads into the diagnosis flow in the preceding figure by enabling the Verilog testbench to log simulation failure information in a format similar to a tester fail file. To generate those failures, you must inject a fault on a design primitive instance (sequential or combinational logic). The simplest way to do this is to force a design node to a fixed value during the entire simulation, as illustrated in the testcase demonstration.

This demonstration injects a fault at one of the core scan cells during the event-driving simulation using Questa Advanced Simulator or a third-party simulator. You use the fail file generated during pattern testbench simulation to generate a core-level failure file; this does not change the diagnosis flow. Then you use the failure file, design flat model, and pattern database to perform the diagnosis and generate the list of suspects in an effort to find the root cause.

#### Reference Testcase Step

You can perform this step in the following directory of the [Reference Testcase](#):



### Notes About This Procedure

- While the diagnosis flow requires post-layout design flat models and patterns, the testcase uses the post-synthesis flat model and patterns for the sole purpose of demonstrating the validation flow.
- This validation flow uses the Verilog testbench for serial patterns. You cannot use the parallel Verilog testbench.
- The Verilog testbench and STIL patterns should come from the same session, which means that the number of patterns written for Verilog simulation must match the STIL patterns.
- The number of Verilog testbench patterns affects the simulation results during fault injection. Choose a number that you can simulate in a reasonable amount of time while still being able to detect the injected fault. This is typically ten patterns if you inject faults on the D input of a scan flop.
- To demonstrate the flow, the testcase uses a Tessent-generated testbench of the serial scan patterns to simulate and inject a fault at a given scan cell.
- Tessent uses the provided name as a prefix. This results in the following generated filename:

```
<command-provided_filename>__<core_module_name>__<scan_test_mode>
```

### Prerequisites

- When you create the serial scan testbench, you must set the parameter `SIM_DIAG_FILE` with a value of two to create the fail file during the fault-injected simulation:  

```
write_patterns <testbench_filename> -serial -parameter_list {SIM_DIAG_FILE 2}
```
- You have injected a fault at any design cell. Do this by directly forcing any cell instances pin in the design to a constant value to emulate a fault. The testcase does this as part of the simulation invocation, as shown at the end of this prerequisite. The testcase uses the following cells as a fault location:
  - **gps\_baseband core** — `sw_rst_reg/D`
  - **processor\_core core** — `watchdog_0.wdt_reset_reg/D`

The following example uses the Questa Advanced Simulator force command to inject a fault on the data input of a design sequential cell:

```
vsim -c -voptargs="+acc"
chip_top_top_retargt_processor_core_stuck_serial_scan_v_ctl -l
logfile/verilog.log_top_retargt_processor_core_stuck_serial_scan \
    -do " force sim:/
chip_top_top_retargt_processor_core_stuck_serial_scan_v_ctl/
chip_top_inst/PROCESSOR_1/PROCESSOR_1/watchdog_0/wdt_reset_reg/D 1 -f ; \
    if {0} {log -r /*};run -all"
```

## Procedure

### 1. Perform reverse failure mapping:

#### a. Set the context for failure mapping.

**set\_context patterns -failure\_mapping**

#### b. Read in the retargeted pattern TCD file.

**read\_core\_descriptions <pattern\_tcd\_file>.tcd.gz**

#### c. Read in the retargeted STIL pattern.

**read\_patterns <pattern\_stil\_file>**

#### d. Read in the fail file generated during pattern fault injection and simulation.

**read\_failures <fail\_file>**

#### e. Write out the failure file.

**write\_failures <fail\_file> -replace**

### 2. Perform failure diagnosis:

#### a. Set the context for scan diagnosis.

**set\_context patterns -scan\_diagnosis**

#### b. Read in the core-level design flattened model.

**read\_flat\_model <flat\_model\_path>**

The ATPG process for the targeted pattern creates this model. Typically, this is a compressed file with a .gz extension.

#### c. Read in the core-level pattern database.

**read\_patterns <pattern\_database\_path>.patdb**

#### d. Run diagnosis on the failure file to produce a list of suspects.

**diagnose\_failures <failure\_mapped\_file>**

## Examples

### Example 1

The following dofile is for the gps\_baseband failure mapping. Find it in the run\_script in the testcase directories for this step.

```
# Set the Context to pattern
set_context pattern -failure_mapping

# read retargeted ATPG tcd file
read_core_descriptions ../tsdb_outdir/logic_test_cores/ \
    chip_top_gate.logic_test_core/ \
    chip_top.atpg_retargeting_mode_retarget1_gps_baseband_stuck/ \
    chip_top_retarget1_gps_baseband_stuck.tcd.gz

# set the current design to the chip_top
set_current_design chip_top
set_system_mode analysis

# read the STIL pattern
read_patterns ../7.retarget_atpg_patterns/patterns/ \
    top_retarget_gps_baseband_stuck_serial_scan.stil

# read the fail file generated by the testbench
read_failures ../7.retarget_atpg_patterns/patterns/ \
    top_retarget_gps_baseband_stuck_serial_scan.v.fail

# write failure file for the diagnosis tool
write_failures
top_retarget_gps_baseband_stuck_serial_scan.failure_diagnosis -replace
```

### Example 2

The following dofile is for the gps\_baseband failure diagnosis. Find it in the run\_script in the testcase directories for this step.

```
# Set the Context to pattern
set_context pattern -scan_diagnosis

# read flat model file
read_flat_model ../../wrapped_cores/gps_baseband/tsdb_outdir/ \
    logic_test_cores/gps_baseband_gate.logic_test_core/ \
    gps_baseband.atpg_mode_int_edt_mode_stuck/ \
    gps_baseband_int_edt_mode_stuck.flat.gz
read_patterns ../../wrapped_cores/gps_baseband/tsdb_outdir/ \
    logic_test_cores/gps_baseband_gate.logic_test_core/ \
    gps_baseband.atpg_mode_int_edt_mode_stuck/ \
    gps_baseband_int_edt_mode_stuck_stuck.patdb

# Diagnose failures
diagnose_failures top_retarget_gps_baseband_stuck_serial_scan. \
    failure_diagnosis___GPS_2__gps_baseband__int_edt_mode_stuck
```

## Advanced Topics

The following topics provide additional information about working with SSN. This information can help you use other features and modes with SSN, configure your design for optimal use with SSN, and understand the underlying structures of SSN.

|                                                           |            |
|-----------------------------------------------------------|------------|
| <b>Streaming-Through-IJTAG Scan Data</b> .....            | <b>532</b> |
| <b>On-Chip Compare With SSN</b> .....                     | <b>535</b> |
| <b>Types of Clock Networks To Use With SSN</b> .....      | <b>545</b> |
| <b>Broadcast to Identical SSN Datapaths</b> .....         | <b>549</b> |
| <b>Determine Failing Cores on ATE With SSN</b> .....      | <b>549</b> |
| <b>Yield Statistics on ATE With SSN</b> .....             | <b>553</b> |
| <b>Manufacturing Patterns With SSN</b> .....              | <b>558</b> |
| Manufacturing Pattern Quick Reference .....               | 559        |
| SSN Pattern Structure .....                               | 560        |
| How to Write Complete SSN Patterns .....                  | 561        |
| How to Write JTAG and Payload Procedures Separately ..... | 562        |
| How to Write SSN On-Chip Compare Patterns .....           | 566        |
| How To Write Streaming-Through-IJTAG Patterns .....       | 568        |
| SSN Debug on the Tester .....                             | 570        |
| <b>Signoff Patterns With SSN</b> .....                    | <b>577</b> |
| Block-Level Signoff Patterns .....                        | 578        |
| Top-Level Signoff Patterns .....                          | 582        |
| Signoff Pattern Quick Reference .....                     | 585        |
| <b>SSN SDC Constraints in the Design Flow</b> .....       | <b>587</b> |
| Overview of SSN-Related SDC Procs and Constraints .....   | 588        |
| SSN/SSH SDC Constraint Descriptions .....                 | 597        |
| <b>Fast IO</b> .....                                      | <b>625</b> |
| Design Requirements .....                                 | 625        |
| SSN Fast IO Functionality .....                           | 627        |
| Modeling a Double Data Rate Interface .....               | 630        |
| Adaptive Tuning on the ATE .....                          | 636        |
| SSN Fast IO Bandwidth Improvement .....                   | 639        |
| Fast IO Pattern Generation .....                          | 640        |
| SSN Fast IO Limitations .....                             | 640        |
| <b>Burn-In Pattern Support</b> .....                      | <b>642</b> |
| Burn-In Pattern Overview .....                            | 642        |
| Recommended Flows for Creating Burn-In Patterns .....     | 644        |
| Burn-In Functionality Limitations .....                   | 646        |
| <b>LVX Support</b> .....                                  | <b>647</b> |
| LVX Hardware Requirements .....                           | 647        |
| Creating EDT With LVX Hardware .....                      | 648        |

|                                                               |            |
|---------------------------------------------------------------|------------|
| LVX Operation Modes With LVX Hardware Configuration . . . . . | 653        |
| Configuring LVX Patterns . . . . .                            | 654        |
| Writing LVX Loop Patterns . . . . .                           | 666        |
| Writing LVX Verification Patterns. . . . .                    | 667        |
| LVI_PATCHING_INFO . . . . .                                   | 668        |
| <b>Retention Testing With SSN . . . . .</b>                   | <b>671</b> |
| <b>Size Resolution Technology With SSN. . . . .</b>           | <b>673</b> |
| Size Resolution Limitations . . . . .                         | 674        |

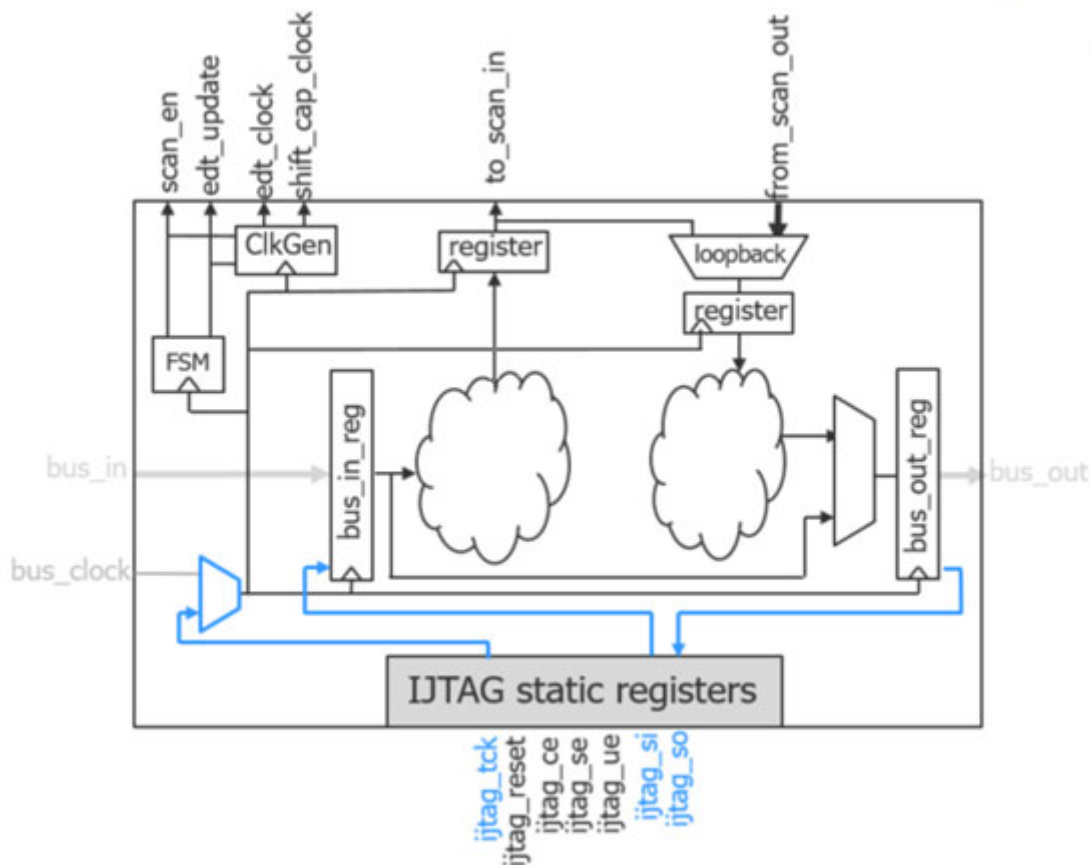
## Streaming-Through-IJTAG Scan Data

The ScanHost node is designed to also enable streaming the SSN data through the IJTAG network instead of through the SSN data bus. This feature is always available with no additional overhead because the IJTAG network is already connected to each ScanHost node, and the ScanHost nodes can be programmed to use a single-bit bus.

For details on enabling this mode during ATPG or scan pattern retargeting, see the [set\\_ssn\\_options](#) command in the *Tessent Shell Reference Manual*.

When activated, the `ijtag_si` data reaching the ScanHost node is injected on bit 0 of the input data register, and bit 0 of the output data register is fed back to the `ijtag_so` pin, as the blue lines in [Figure 9-6](#) show. TCK is also injected as the SSN bus clock within the ScanHost node.



**Figure 9-6. Streaming-Through-IJTAG Mode**

Scan data delivery into the chip is significantly slower when using Streaming-Through-IJTAG than with the SSN data bus. It is only one bit wide and limited to the maximum operating frequency of TCK. However, the IJTAG network is very robust due to its two-edge timing and is always accessible in the system through the IEEE 1149.1 JTAG TAP.

In silicon, if you find you have a timing problem in a section of the SSN datapath, the Streaming-Through-IJTAG mode is available as an alternative method to deliver patterns to bring up your initial parts. For this reason, the Streaming-Through-IJTAG mode is also referred to as a fail-safe mechanism.

The Streaming-Through-IJTAG feature is also a more flexible replacement for [LPCT-2](#). You are no longer limited to being able to drive only a single EDT controller having a single channel pair when using the IEEE 1149.1 protocol for delivering scan data.

If your IJTAG network architecture is an IJTAG streaming-compatible setup, in that it provides concurrent access to all ScanHost nodes you want to access through IJTAG (as is possible when using SIBs), you can test any number of blocks in parallel with an unlimited number of channels. This can be useful in a 3D or 2.5D package if IEEE 1838 compliance is a requirement.

The IEEE 1838 standard mandates that you can perform the die interconnect test through the IEEE 1149.1 JTAG TAP, using only TCK as the scan and capture clock. To do this, create a scan mode within each die that collects the wrapper cells interacting with the pins of the die, and create ATPG patterns using this scan mode to target the interconnect faults. Apply these patterns through the normal SSN bus, which is effectively the FFP mode of the IEEE 1838 standard. Alternatively, retarget those ATPG patterns to apply through the IEEE 1149.1 TAP by using the Streaming-Through-IJTAG mode that comes with SSN, and satisfy the IEEE 1838 requirements.

Consider the following while using Streaming-Through-IJTAG:

- All ScanHost nodes you want to include must be in the active IJTAG scan path. Streaming-Through-IJTAG mode does not support IJTAG broadcast. Star network configurations restrict the number of ScanHost nodes that can be active in parallel. The IJTAG solver within the Tessent tool automatically opens up the IJTAG path if the network allows it.
- The Streaming-Through-IJTAG mode supports both internal and external capture, and all fault models are supported.

---

**Tip**

 At the top level, make the boundary scan chains available to the top-level EDT controller for best results. (See the [max\\_segment\\_length\\_for\\_logictest](#) property description in the [DftSpecification/BoundaryScan](#) wrapper reference page for more details.) Add the `int_ltest_en` DFT signal at the top level and set it to 1 using the [set\\_static\\_dft\\_signal\\_values](#) command during ATPG.

This enables full coverage of the chip logic without the need to drive the primary inputs and observe the primary outputs during ATPG.

---

- You can run any number of ScanHost nodes concurrently in this mode as long as your IJTAG network allows placing all the ScanHost nodes along the active IJTAG scan path.
- When a Streaming-Through-IJTAG session is complete, the binary states of all SIBs along the active scan path are restored to their states that were present before streaming began. This provides a predictable IJTAG network configuration between Streaming-Through-IJTAG sessions.

All scan testable instruments on the IJTAG network must be isolated from the following IJTAG interface signals:

- `ijtag_select`
- `ijtag_se`

---

**Caution**

 If the state of these IJTAG interface signals is observable into scan cells, the enabling of the IJTAG streaming feature may invalidate the patterns.

---

You can accomplish this isolation by gating the signals with the `ltest_en` DFT signal. All scan tested instruments under the SIB STI are already isolated and do not require any hardware changes. Refer to [the figure “Sib\(sti\) With Scan Isolation Chain”](#) in the [Sib](#) reference page for mode details about the way scan tested instruments are isolated from the IJTAG network during scan test when you use the recommended Sib structure that is automatically inferred within the [create\\_dft\\_specification](#) command.

## On-Chip Compare With SSN

The on-chip compare feature enables the ScanHost node to compare expected data against the response of the scan chains within the node itself.

The following subsections describe in more detail how to get the most effective use of the on-chip compare feature and the self-test patterns used to test faults with it:

- [Common Usage Scenarios](#) — Scenarios that the on-chip compare feature is designed to address.
- [On-Chip Compare Mode Operation](#) — The methods used by the on-chip compare feature and the packet format when the feature is enabled.
- [Costs, Benefits, and Recommendations](#) — Trade-offs between DFT area and test time efficiency, and recommendations for when the on-chip compare feature is most effective.
- [On-Chip Compare Mode Setup](#) — Instructions for building your ScanHost with the on-chip compare feature and enabling the feature during ATPG and scan pattern retargeting.
- [Diagnosis Using On-Chip Compare](#) — Instructions for enabling detailed diagnostics when using the on-chip compare feature.

### Common Usage Scenarios

Comparing expected data to scan chain response in hardware within the ScanHost can be helpful in the following scenarios:

- You have one or more physical blocks that are instantiated multiple times within the design. With on-chip compare, the packet includes the chain input values and also the expected and mask values. The packet values are not altered by the [ScanHost](#) node because the chain input values are not replaced by the chain output values and thus can be reused by any number of instances of that physical block. The section “[Costs, Benefits, and Recommendations](#)” on page 536 describes the number of instances of the

physical block required for the on-chip compare feature to become beneficial for test time and test data volume considerations.

- You want to quickly identify the failing cores on the ATE during manufacturing because you have a Partial Good Die methodology that tolerates a subset of identical cores failing. In such a case, the failing cores must be identified on the tester, and the test flow must take actions to decommission them while testing the rest of the chip.

The on-chip compare mode provides a sticky status bit per ScanHost node that IJTAG scans out at the end of the pattern set to directly identify the failing blocks. You can also scan out a sticky status per channel output if you require. Refer to the [OnChipCompareMode/sticky\\_status\\_resolution](#) property description in the [ScanHost](#) reference page in the *Tessent Shell Reference Manual* for information about how to request usage of this feature. Refer to the [OnChipCompareMode/present](#) property description in the same topic for information about the special annotations added to the STIL file to quickly identify the comparisons that match the sticky status bits.

## On-Chip Compare Mode Operation

When you add the on-chip compare mode (OCComp mode) to the ScanHost node, it can still operate in the normal mode, in which the input time slots of the packet corresponding to the channel input values are replaced by the channel output values. Then the ATE compares them at the end of the SSN data bus when they come off the chip. The packet format in this normal mode of operation is illustrated in [the figure “SSN Packet Formats When Not Using On-Chip Compare”](#) in the ScanHost reference page in the *Tessent Shell Reference Manual*.

When you enable the on-chip compare mode of operation, the packet is modified to include the channel input values followed by the expected values, the mask values, and the optional status values. The packet format in on-chip compare mode is illustrated in [the figure “SSN Packet Formats When Using On-Chip Compare”](#) in the ScanHost reference page in the *Tessent Shell Reference Manual*. When building a ScanHost node with support for on-chip compare, you should use as few output channels as possible to minimize the time slot count used to carry the expected and mask data.

For more details about the packet format in both modes, refer to the section [“SSN Packet Formats”](#) in the ScanHost reference page.

## Costs, Benefits, and Recommendations

The [Common Usage Scenarios](#) section explains why it is beneficial to include the on-chip-compare mode within your [ScanHost](#) node when you reuse the physical block that contains it multiple times in the chip. Because the OCComp mode uses packet data time slots for the channel inputs, for the expected, mask, and status values, and for the status time slot, it becomes more efficient than the normal mode of operation when you have at least three identical instances of the physical block. With fewer instances, the test time and data volume reduction are negligible and do not justify the area overhead needed to implement the OCComp mode.

The typical area of a ScanHost node that does not support the OCComp mode is significantly smaller than a typical EDT controller. When the ScanHost node supports the OCComp mode, its area becomes comparable to a typical EDT controller. To minimize the area impact of OCComp on the ScanHost node, you must minimize the number of output channels you use with the EDT. In this case, you typically use a single output channel.

You can build the ScanHost node with the support of many status groups for diagnosis. Building support for many status groups increases the area of the ScanHost node, and using many status groups increases test time and data volumes. Refer to the section “[Diagnosis Using On-Chip Compare](#)” on page 537 for more information when and how to use multiple status groups.

## On-Chip Compare Mode Setup

To set up OCComp mode, you need to equip the ScanHost hardware node with the logic for it and then enable the mode once the node is set up for it. The following subsections describe how to do this.

### Equipping the ScanHost Hardware Node With On-Chip Compare Mode

The ScanHost node is not built with the on-chip compare mode by default. Building with OCComp mode doubles the area of the controller. Enable the creation of the OCComp mode by inserting the [OnChipCompareMode](#) wrapper within the [ScanHost](#) wrapper, typically when you have one of the conditions described in the section “[Common Usage Scenarios](#)” on page 535. Use the [OnChipCompareMode/sticky\\_status\\_resolution](#) property within the wrapper to specify whether you want one sticky status per output channel or only a single one for the entire ScanHost node.

### Enabling On-Chip Compare Mode During ATPG and Scan Pattern Retargeting

By default, using the [write\\_patterns](#) command in the `patterns -scan` or `patterns -scan_retargeting` context does not enable OCComp mode. Manually enable OCComp mode by using the “[set\\_core\\_instance\\_parameters](#) -parameter\_values {on\_chip\_compare\_enable on}” command. You can either specify a group of ScanHost instances by using the `-instances` switch or use the “`-instrument_type ssh`” switch to enable on-chip compare on all ScanHost nodes that support it.

## Diagnosis Using On-Chip Compare

As described in the figure “[SSN Packet Formats When Not Using On-Chip Compare](#)” and its preceding text in the [ScanHost](#) reference page in the *Tessent Shell Reference Manual*, you can program the ScanHost node to report per-cycle pass/fail information into special Status time slots when using the OCComp mode. You can choose to have no status time slots and rely purely on the Go/NoGo sticky status bit scanned out with IJTAG at the end of the session. This sticky status provides failure resolution down to the failing EDT output channel when you have specified [OnChipCompareMode/sticky\\_status\\_resolution](#) as “`output_chain`”, but it does not provide diagnostic resolution down to the failing gate because there are no per-cycle compare statuses.

When several ScanHost nodes are programmed to use the same status time slots, which is the default behavior, diagnosis may require reapplication of the pattern set to collect all the data needed to perform detailed diagnostics of every failing core. Consider a case where all identical core instances are placed into a single status group, such that their per-cycle pass/fail information aggregates into the same status time slots. If any of those bits indicate failures, you have the cumulative per-pin, per-cycle fail data but may not be able to determine which core or cores the failures came from. The sticky status bits unloaded at the end of the pattern set through JTAG indicate which cores failed at least once. If only one core in this group failed, then you know the per-cycle pass/fail data came from this core alone; therefore, you have all the information needed for diagnosis. However, if multiple cores fail, you must then separately test and observe each failing core to get its individual fail data. If two cores fail, for example, then you reapply the same test set twice, with patching applied to the `disable_on_chip_compare_contribution` bit for all but one ScanHost node at a time. There is no need to store separate patterns for diagnosis on the tester. Refer to the description of the `disable_on_chip_compare_contribution` bit in [the table “JTAG Registers in ScanHost Nodes”](#) in the [ScanHost](#) reference page in the *Tessent Shell Reference Manual* for more information about the special `iWriteVar` annotation that identifies the location of such patchable bits.

---

#### Note



If you disable contribution with the `disable_on_chip_compare_contribution` bit, the sticky status bits for that SSH are not set.

---

If identical core instances are split into multiple groups, this slightly increases the test time, but decreases the probability of resorting to multiple test applications for collecting diagnosis data. In the example shown in [the figure “SSN Packet Formats When Using On-Chip Compare”](#) in the [ScanHost](#) reference page in the *Tessent Shell Reference Manual*, the six cores are split into two groups. If you find cores A1 and A4 to have failed, there is no need for test reapplication, because cores A1 through A3 accumulate their status bits separately from cores A4 through A6. However, if cores A1 and A3 fail, you must reapply the tests with patching to acquire the individual fail data. In an extreme case, you may choose to assign each core instance to its own group to observe each core individually. This mode of operation may be better suited for silicon debug than high-volume manufacturing.

To learn about the ATE failure log format, refer to the section [“ATE Failure File Format Requirements”](#) in the *Tessent Diagnosis User’s Manual*. Performing on-chip compare requires special handling for scan diagnosis in both test application and failure logging. For details, refer to the section [“Logging Failures for SSN On-Chip Compare”](#) in the *Tessent Diagnosis User’s Manual*.

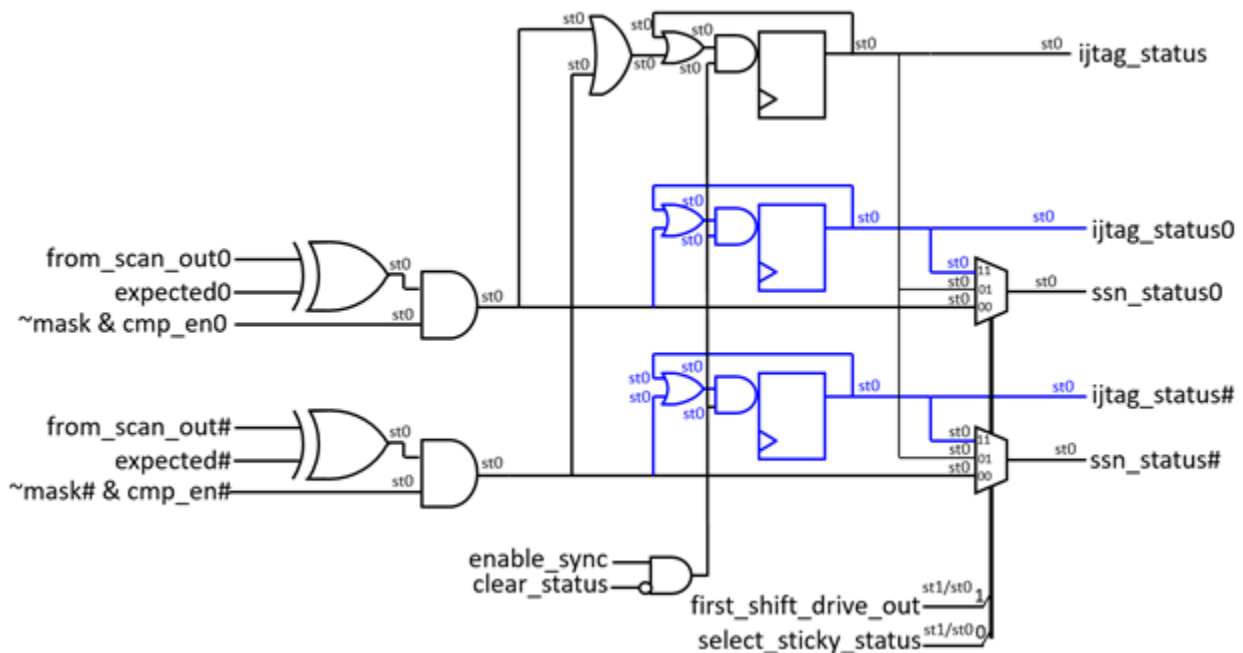
To learn about how to map the failures at the chip boundary to the core level and to enable performing detailed diagnostics, refer to the section [“Reverse Mapping Top-Level Failures to the Core”](#) in the *Tessent Diagnosis User’s Manual*. The failure mapping steps described in this section are required when using SSN, even for top-level ATPG patterns. This is unlike when you are not using SSN, where they are only required for retargeted scan patterns.



## On-Chip Comparator Self-Test Patterns

As described in this section, you can equip the SSN ScanHost node with On-Chip compare logic. This logic is described in the “[On-Chip Compare Logic](#)” figure in the [ScanHost](#) reference page. There are several possible faults in the comparator logic that could prevent the detection of mismatches caused by defects in the scanned circuit. [Figure 9-7](#) identifies those faults. The opposite fault polarity would result in systematic mismatches, and you can detect them with the normal On-Chip compare patterns. The faults listed in this figure would result in faults within the scan circuitry that can go undetected. You can generate a special pattern using the ScanHost loopback mode with expected values set opposite to the scan-in values, to cause mismatches and enable you to observe the effects on the SSN bus outputs. The logic shown in blue in the following figure exists only when you have specified the ScanHost/OnChipCompareMode/sticky\_status\_resolution property as “output\_chain”.

**Figure 9-7. Faults in Comparator Logic That Could Mask Real Faults**



### Structure of the On-Chip Comparator Patterns

Generate the on-chip comparator self-test pattern set using the “[write\\_patterns](#) -pattern\_set ssh\_on\_chip\_compare” command in the patterns -scan or -scan\_retargeting context. The tool performs the on-chip comparator self-test pattern on all active SSHs in the current atpg or scan retargeting session. Use the [report\\_core\\_instances](#) command to see the active SSHs

The self-test pattern programs the ScanHost node with the following register values, so that each scan load comprises exactly five packets.

```

edt_update_falling_launch_word      0
edt_update_falling_transition_words  0
extra_shift_packets                 0
force_suppress_capture               1
loop_back_en                        1
on_chip_compare_enable               1
on_chip_compare_group                1
on_chip_compare_group_count          1
scan_en_launch_packet                0
scan_en_transition_packets            0
total_shift_cnt_minus_one            1

```

With these settings, each scan load is exactly five packets long, and the packets have the following labels: `edt_update`, `shift0`, `shift1`, `post_shift0`, and `post_shift1`. The values of the `from_scan_out_bits` registers equal the number of output chains usable during on-chip compare for each chain group; refer to the [ScanHost](#) reference page for more details in the description of the `output_chain_count_in_on_chip_compare_mode` property. The values of the `to_scan_in_bits` registers match the exact value of the `from_scan_out_bits` register for each chain group. This ensures that there are exactly the same amount of scan-in values as there are comparators to test. The format of the five packets is shown in the following output for an example ScanHost having 3 comparators. The green values in the following output carry the scan payload in and out. The tool compares the `i0`, `i1`, and `i2` values provided in the `shift0` and `shift1` packets against the `e0`, `e1`, and `e2` values and masks them with the `m0`, `m1`, and `m2` values scanned in during the `post_shift0`, and `post_shift1` packets. The tool scans out the comparator statuses per output chain during the `post_shift0` and `post_shift1` packets.

| packet_id   | scanin                        | exp                           | mask                          | status                        |
|-------------|-------------------------------|-------------------------------|-------------------------------|-------------------------------|
| edt_update  | <b>i0</b> i1 i2               | e0 e1 e2                      | m0 m1 m2                      | s0 s1 s2                      |
| shift0      | <b>i0</b> <b>i1</b> <b>i2</b> | e0 e1 e2                      | m0 m1 m2                      | <b>s0</b> <b>s1</b> <b>s2</b> |
| shift1      | <b>i0</b> <b>i1</b> <b>i2</b> | e0 e1 e2                      | m0 m1 m2                      | s0 s1 s2                      |
| post_shift0 | <b>i0</b> i1 i2               | <b>e0</b> <b>e1</b> <b>e2</b> | <b>m0</b> <b>m1</b> <b>m2</b> | <b>s0</b> <b>s1</b> <b>s2</b> |
| post_shift1 | <b>i0</b> i1 i2               | <b>e0</b> <b>e1</b> <b>e2</b> | <b>m0</b> <b>m1</b> <b>m2</b> | <b>s0</b> <b>s1</b> <b>s2</b> |

The following list describes three control bits whose time slots are identified in gold in the preceding scan load description.

- **select\_sticky\_status** — This control bit is scanned in during the `i0` time slot of the `edt_update` packet. When the value is “1”, the `select_sticky_status` signal, shown in [Figure 9-7](#) on page 539, goes high at the end of the `shift1` packet within the same scan load.

As that figure illustrates, the logic injects the sticky status results into the status time slots of the `post_shift0` and `post_shift1` within the same scan load. It also injects the global sticky status into the status time slots of the `shift0` packet (shown in magenta in the preceding scan load description) of the next scan load. As previously described, the `select_sticky_status` signal goes up or down at the end of the `shift1` packet, which explains why the global sticky status observed in the `shift0` packet happens in the next scan load.



- **last\_scan\_load** — This control bit scans in during the i0 time slot of the post\_shift0 packet. It indicates to the ScanHost that the current scan load is the last one and to stop performing any comparisons in the next scan loads that may continue to flow through the datapath, which are to complete the testing of other ScanHosts with more comparators to test.
- **clear\_sticky\_status** — This control bit is scanned in during the i0 time slot of the post\_shift1 packet. When the value is “1”, the clear\_status signal, shown in [Figure 9-7](#) on page 539, goes high during the shift1 packet within the next scan load. As mentioned in the preceding “select\_sticky\_status” section, the tool observes the global sticky status during the shift0 packet, which is before the clear\_sticky\_status takes effect. Therefore, it observes the effect of the clear\_sticky\_status in the next scan load, assuming no new miscompares were injected during the post\_shift0 and post\_shift1 packet to cause the global sticky status to go up again after it was cleared at the end of the shift1 packet.

The on-chip comparator self-test pattern set comprises six phases, described in the following sections. Every compare in the pattern uses a unique label for identification, constructed as in the following:

```
<ssh_icl_instance>.<phase_id>.<packet_id>.scanin# for the scan in values
<ssh_icl_instance>.<phase_id>.<packet_id>.exp# for the expected values
<ssh_icl_instance>.<phase_id>.<packet_id>.mask# for the mask values
<ssh_icl_instance>.<phase_id>.<packet_id>.status# for the status values
```

The output patterns file such as STIL has one such bit\_annotation for each output time slot to help quickly identify the meaning of any miscompares. Refer to the section “[Retargeted Symbolic Variables](#)” in the *Tessent IJTAG User’s Manual* for information about the bit annotations.

The following describes the six phases of the on-chip compare self-test, with the faults from [Figure 9-7](#) on page 539 that they each cover.

### Phase 1

This phase consists of a single scan load formed with one set of the five packets shown previously. This scan load uses consistent scanin and expected values for both shift cycles so that it generates no miscompares and the sticky statuses get initialized to 0. The phase\_id is “phase1”.

|             | i0 | i1 | i2 | e0 | e1 | e2 | m0 | m1 | m2 | s0 | s1 | s2 |
|-------------|----|----|----|----|----|----|----|----|----|----|----|----|
| edt_update: | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | L  | L  | L  |
| shift0:     | 1  | 1  | 1  | 0  | 0  | 0  | 0  | 0  | 0  | L  | L  | L  |
| shift1:     | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | L  | L  | L  |
| post_shift0 | 0  | 0  | 0  | 1  | 1  | 1  | 0  | 0  | 0  | L  | L  | L  |
| post_shift1 | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | L  | L  | L  |

### Phase 2

This phase consists of  $N$  scan loads used to test that the  $N$  XOR gates in [Figure 9-7](#) on page 539 are not stuck at 0 when the scanin value is 0 and the expected value is 1. The phase\_id is

“phase2\_*X*” when testing the *X*th comparator. The following is the scan load used to test comparator 0.

|             | i0 | i1 | i2 | e0 | e1 | e2 | m0 | m1 | m2 | s0 | s1 | s2 |
|-------------|----|----|----|----|----|----|----|----|----|----|----|----|
| edt_update: | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | L  | L  | L  |
| shift0:     | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | L  | L  | L  |
| shift1:     | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | L  | L  | L  |
| post_shift0 | 0  | 0  | 0  | 1  | 0  | 0  | 0  | 0  | 0  | H  | L  | L  |
| post_shift1 | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | L  | L  | L  |

### Phase 3

This phase consists of *N* scan loads used to test that the *N* XOR gates in [Figure 9-7](#) on page 539 are not stuck at 0 when the scanin value is 1 and the expected value is 0. The phase\_id is “phase3\_*X*” when testing the *X*th comparator. The following is the scan load used to test comparator 0.

|             | i0 | i1 | i2 | e0 | e1 | e2 | m0 | m1 | m2 | s0 | s1 | s2 |
|-------------|----|----|----|----|----|----|----|----|----|----|----|----|
| edt_update: | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | L  | L  | L  |
| shift0:     | 1  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | L  | L  | L  |
| shift1:     | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | L  | L  | L  |
| post_shift0 | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | H  | L  | L  |
| post_shift1 | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | L  | L  | L  |

### Phase 4

This phase consists of two scan loads. It tests that the sticky statuses are able to hold. As shown in [Figure 9-7](#) on page 539, there is a global sticky status and optional sticky statuses per output chain. The [ScanHost/OnChipCompareMode/sticky\\_status\\_resolution](#) property controls the presence of the sticky statuses per output chains. The *phase\_id* for those two scan loads are “phase4\_0” and “phase4\_1”.

The effect of the select\_sticky\_status control bit takes effect at the end of the shift1 packet within the same scan load. The effect of the clear\_sticky\_status happens at the end of the shift1 packet within the next scan load.

When only the global sticky status is present, the pattern looks like the following diagram. The select\_sticky\_status takes effect at the end of the shift1 packet. The red H in the second scan load corresponds to the global sticky status being observed during the last status time slot of the shift0 packet. The global sticky status remembers the faults injected during Phase2 and Phase3 and clears at the end of the shift1 packet of the second scan load.

|             | i0 | i1 | i2 | e0 | e1 | e2 | m0 | m1 | m2 | s0 | s1 | s2 |                         |
|-------------|----|----|----|----|----|----|----|----|----|----|----|----|-------------------------|
| edt_update: | 1  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | L  | L  | L  | // select_sticky_status |
| shift0:     | 1  | 1  | 1  | 0  | 0  | 0  | 0  | 0  | 0  | L  | L  | L  |                         |
| shift1:     | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | L  | L  | L  |                         |
| post_shift0 | 0  | 0  | 0  | 1  | 1  | 1  | 0  | 0  | 0  | L  | L  | L  |                         |
| post_shift1 | 1  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | L  | L  | L  | // clear_sticky_status  |

```

      i0 i1 i2 e0 e1 e2 m0 m1 m2 s0 s1 s2
edt_update: 1 0 0 0 0 0 0 0 0 L L L // select_sticky_status
shift0:      1 1 1 0 0 0 0 0 0 X X H
shift1:      0 0 0 0 0 0 0 0 0 L L L // status cleared here
post_shift0 0 0 0 1 1 1 0 0 0 L L L
post_shift1 0 0 0 0 0 0 0 0 0 L L L

```

When the sticky statuses per output chain are present, the pattern looks like the following diagram. The global sticky status observation is identical to what was described previously. However, because the sticky statuses per output chain are present, the tool observes them during the first scan load. The clear\_sticky\_status control requested during the first scan load takes effect after the shift1 packet of the next scan load, which explains why the status values return to being L during the second scan load.

```

      i0 i1 i2 e0 e1 e2 m0 m1 m2 s0 s1 s2
edt_update: 1 0 0 0 0 0 0 0 0 L L L // select_sticky_status
shift0:      1 1 1 0 0 0 0 0 0 L L L
shift1:      0 0 0 0 0 0 0 0 0 L L L
post_shift0 0 0 0 1 1 1 0 0 0 H H H
post_shift1 1 0 0 0 0 0 0 0 0 H H H // clear_sticky_status

      i0 i1 i2 e0 e1 e2 m0 m1 m2 s0 s1 s2
edt_update: 1 0 0 0 0 0 0 0 0 L L L
shift0:      1 1 1 0 0 0 0 0 0 X X H // global sticky status
shift1:      0 0 0 0 0 0 0 0 0 L L L // status cleared here
post_shift0 0 0 0 1 1 1 0 0 0 L L L
post_shift1 0 0 0 0 0 0 0 0 0 L L L

```

## Phase 5

This phase consists of  $N$  scan loads. It tests that all inputs of the wide OR gate feeding the global sticky status register are not stuck at 0. The tool generates faults for each comparator in its respective scan load by setting the shift0 value to 0 and expecting a H. It observes the global sticky status in the last status bit of the Shift0 packet of the next scan load. The phase\_id for those two scan loads is “phase5\_x”.

The following diagram shows the first two scan loads with the sticky status per chain output feature enabled. The failure appears during the post\_shift1 packet, even though the tool detected

the fault in the post\_shift0 packet because of the presence of the blue flip-flops in Figure 9-7 on page 539:

```

      i0 i1 i2 e0 e1 e2 m0 m1 m2 s0 s1 s2
edt_update: 1 0 0 0 0 0 0 0 0 L L L // select_sticky_status
shift0      0 1 1 0 0 0 0 0 0 L L L // global sticky status
shift1      0 0 0 0 0 0 0 0 0 L L L
post_shift0 0 0 0 1 1 1 0 0 0 L L L
post_shift1 1 0 0 0 0 0 0 0 0 H L L // clear_sticky_status

edt_update: 1 0 0 0 0 0 0 0 0 L L L // select_sticky_status
shift0      1 0 1 0 0 0 0 0 0 X X H // global sticky status
shift1      0 0 0 0 0 0 0 0 0 L L L
post_shift0 0 0 0 1 1 1 0 0 0 L L L
post_shift1 1 0 0 0 0 0 0 0 0 L H L // clear_sticky_status

```

The following diagram shows the first two scan loads with the sticky status per chain output feature turned off. The failure appears during the post\_shift0 packet because of the absence of the blue flip-flops in Figure 9-7:

```

      i0 i1 i2 e0 e1 e2 m0 m1 m2 s0 s1 s2
edt_update: 1 0 0 0 0 0 0 0 0 L L L // select_sticky_status
shift0      0 1 1 0 0 0 0 0 0 L L L // global sticky status
shift1      0 0 0 0 0 0 0 0 0 L L L
post_shift0 0 0 0 1 1 1 0 0 0 H L L
post_shift1 1 0 0 0 0 0 0 0 0 L L L // clear_sticky_status

edt_update: 1 0 0 0 0 0 0 0 0 L L L // select_sticky_status
shift0      1 0 1 0 0 0 0 0 0 X X H // global sticky status
shift1      0 0 0 0 0 0 0 0 0 L L L
post_shift0 0 0 0 1 1 1 0 0 0 L H L
post_shift1 1 0 0 0 0 0 0 0 0 L L L // clear_sticky_status

```

## Phase 6

This phase consists of one scan load. It observes the last global sticky status result associated to the last comparator that was triggered during the last scan load of Phase 5. During this phase, the tool also fault-injects all comparators to set all sticky statuses per comparator and observes them with an IJTAG scan load as a last step when the feature is present. The phase\_id for this scan load is “phase6”.

```

      i0 i1 i2 e0 e1 e2 m0 m1 m2 s0 s1 s2
edt_update: 0 0 0 0 0 0 0 0 0 L L L
shift0      0 0 0 0 0 0 0 0 0 X X H
shift1      0 0 0 0 0 0 0 0 0 L L L
post_shift0 0 0 0 1 1 1 0 0 0 H H H // last scan load
post_shift1 0 0 0 0 0 0 0 0 0 L L L

```

## Types of Clock Networks To Use With SSN

The configuration of the clock network you use with SSN depends on the logic in your design. Frequency multipliers and dividers, along with other nodes, can help you optimize your clock network and avoid large a cumulative delay value.

The streaming scan network is a synchronous bus going from a set of inputs through several stages of pipelining across many physical regions until it comes back out on a set of outputs. The pipeline stages occur within the various node types along the datapath.

The [SSN DftSpecification](#) offers several node types to help with crossing clock domain boundaries. The following sections explain various ways in which you can implement clock trees to best meet your requirements.

---

### Note



For more information about clock tree synthesis (CTS) and implementing stop points for CTS, refer to the section “[CTS Exceptions for the SSH](#)” on page 621.

---

## Hierarchical Clock Tree

The most common and straightforward clock scheme is to build a hierarchical clock tree for the entire SSN datapath or for the SSN datapath within groups of physical blocks. Implementation of one or more hierarchical clock trees requires space in the upper physical regions to place the clock buffers around the child physical blocks. You may not want to or be able to build a hierarchical clock tree that is common for all child physical blocks. Instead, you can create more than one smaller Clock Tree Synthesis (CTS) regions. If you do this, you must use a pair of [BusFrequencyDivider](#) and [BusFrequencyMultiplier](#) nodes to reliably transfer the data from one clock domain to the next.

- To see how to insert the clock domain transfer node within the top physical regions, refer to [Example 1 of the BusFrequencyDivider](#) topic.
- To see how to insert the clock domain transfer node such that the source and destination clock domains remains localized to each physical block, refer to [Example 2 of the BusFrequencyDivider](#) topic.
- To see how to insert the clock domain transfer node inside the receiving physical block to keep the number of pins crossing the physical block boundary to a minimum, refer to [Example 3 of the BusFrequencyDivider](#) topic. When you do this, you need an extra miniature clock tree within the receiving physical block for the [BusFrequencyDivider](#) node and must use a source synchronous timing interface between the sourcing and receiving physical block. Refer to the next section for an explanation of such timing interfaces.

Refer to the [Using the BusFrequencyDivider to Cross CTS Regions](#) section of the [BusFrequencyDivider](#) topic in the *Tessent Shell Reference Manual* for an explanation on how the [BusFrequencyDivider](#) and [BusFrequencyMultiplier](#) nodes work together to provide an

arbitrarily large tolerance of skew between the transmitting and receiving clock domains by increasing the `frequency_ratio` value.

## Source Synchronous Timing Interface

A source synchronous timing interface, also called a forward timing interface, relies on the fact that the clock follows the data. The physical block sourcing the data to the receiving node also sources the clock input at the same time. The delay of the datapath closely matches the delay of the clock path. The data is launched on one edge of the clock at the source and is captured on the opposite clock edge at the destination. As long as the delay on the datapath matches the delay of the clock path within half a clock period, the transfer of data is synchronous and reliable.

[Example 3 of the BusFrequencyDivider](#) topic uses this technique. The transmitting physical block ends with a [Pipeline](#) node, which launches the data on the positive edge of the clock. The receiving [BusFrequencyDivider](#) is built with the `input_retimed` property set to “on” so that it is equipped with a negative edge retiming flip-flop stage on its input data bus. The retiming stage ensures that the receiving node strobes the data on the opposite edge of the clock relative to the launch edge of the transmitter.

The forward timing technique is used in [Example 3 of the BusFrequencyDivider](#) topic to hand off data from one physical block to the next without requiring the balance clock tree of one block to enter the other. It also uses the narrow data bus when crossing the physical block boundaries. It is important to use this technique combined with a [BusFrequencyDivider](#) and [BusFrequencyMultiplier](#) node pair so that you can return to using a hierarchical clock tree for the other section of the datapath. Although it is possible to use a source synchronous interface between each physical block, due to its easy implementation, this is not a recommended practice in a large design, because this would accumulate a CTS insertion delay in each physical block. The data output of the last physical block would become so delayed relative to the clock input coming from the tester that it would be impossible to reliably sample the output data on the ATE and find a strobe point that would work for best- and worst-case conditions.

The following sections explain how to use forward design timing between each node when the datapath goes and comes back between each pair of physical blocks.

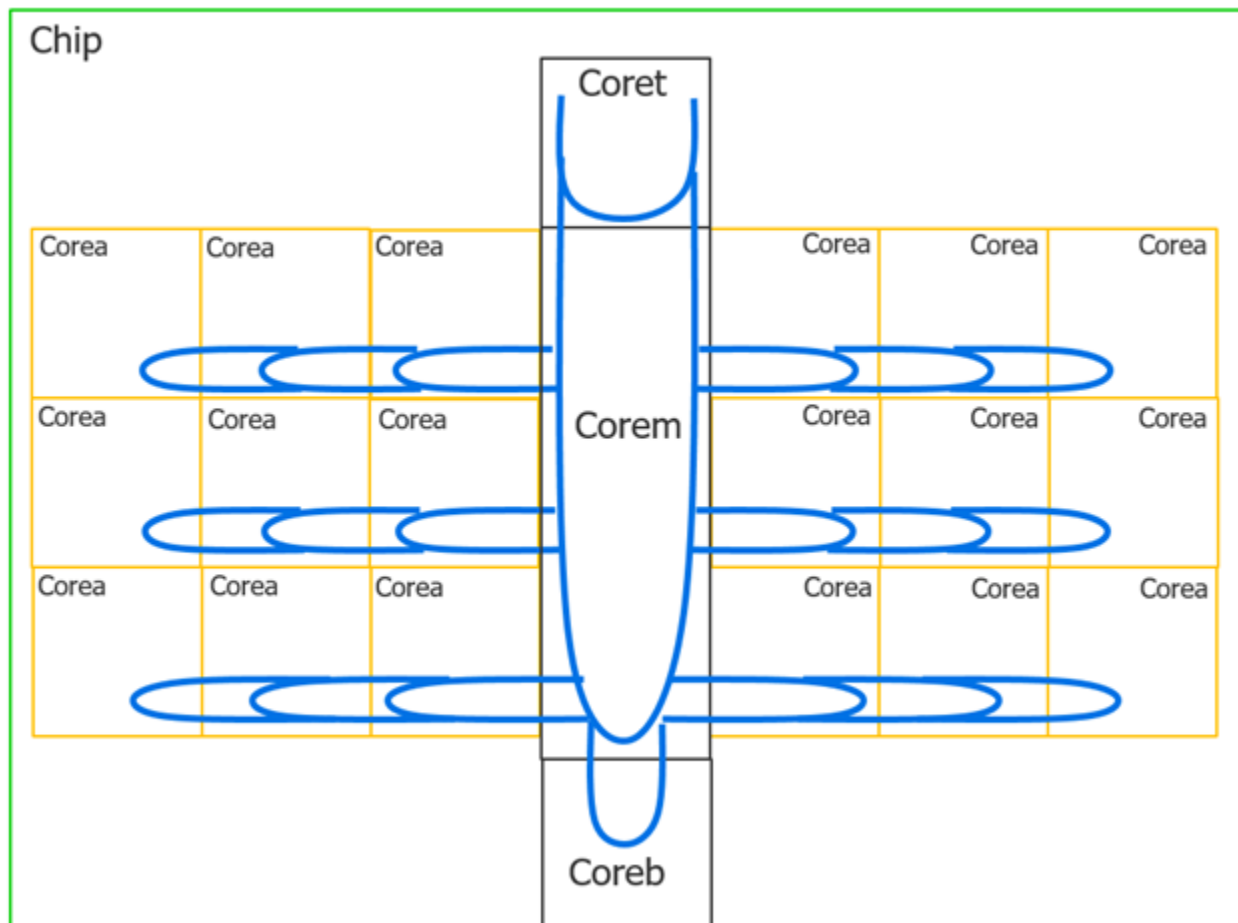
## Combining Forward and Loop Timing for Tiling Layout Flows

When you are implementing a pure tiling layout flow, you do not have any logic between the blocks. Everything is self-contained within the child physical blocks, which simply abut each other. In such a case, it is not possible to use a hierarchical clock tree to synchronize a clock to all child physical blocks. Instead, each block provides the clock to the next. As the clock goes through several blocks, it accumulates a large delay because of the clock delay in each block. By the time the clock reaches the *N*th block, the delay is so large that it is no longer possible to reliably strobe the data on the tester. To avoid accumulating delay in a tiling configuration, you must implement the datapath through your physical block so that it goes and comes back through the block as shown in [Example 3 of the SSN](#) topic. Use a forward timing interface to go from one physical block to the next and a loop timing interface to get the returned data back

from the next block. Both the input and the output data are localized in the first block, so it is easy to synchronize with the tester. Refer to the next section to see how to minimize the loop timing between the chip and the tester to increase the SSN shift rate.

Figure 9-8 shows a block diagram of a chip where the SSN datapath starts from the center top of the device and makes a loop going down the middle of the chip. The datapath also branches out to both sides in mirrored groups of physical blocks called “Corea.” The GPIO are all localized in the top center block, including the clock input. The timing with the tester is completely localized into that physical block and is predictable before you complete the full chip layout. As Example 3 of the SSN topic illustrates, the timing of the datapath is localized between each pair of blocks and not affected by distant elements. In the example, there are three instances of corea on top of each other. These correspond to three Corea instances in following figure, where the bottom corea instance from the example is the Corea instance closest to the center in the figure.

**Figure 9-8. SSN Datapath in Tiling Architecture**

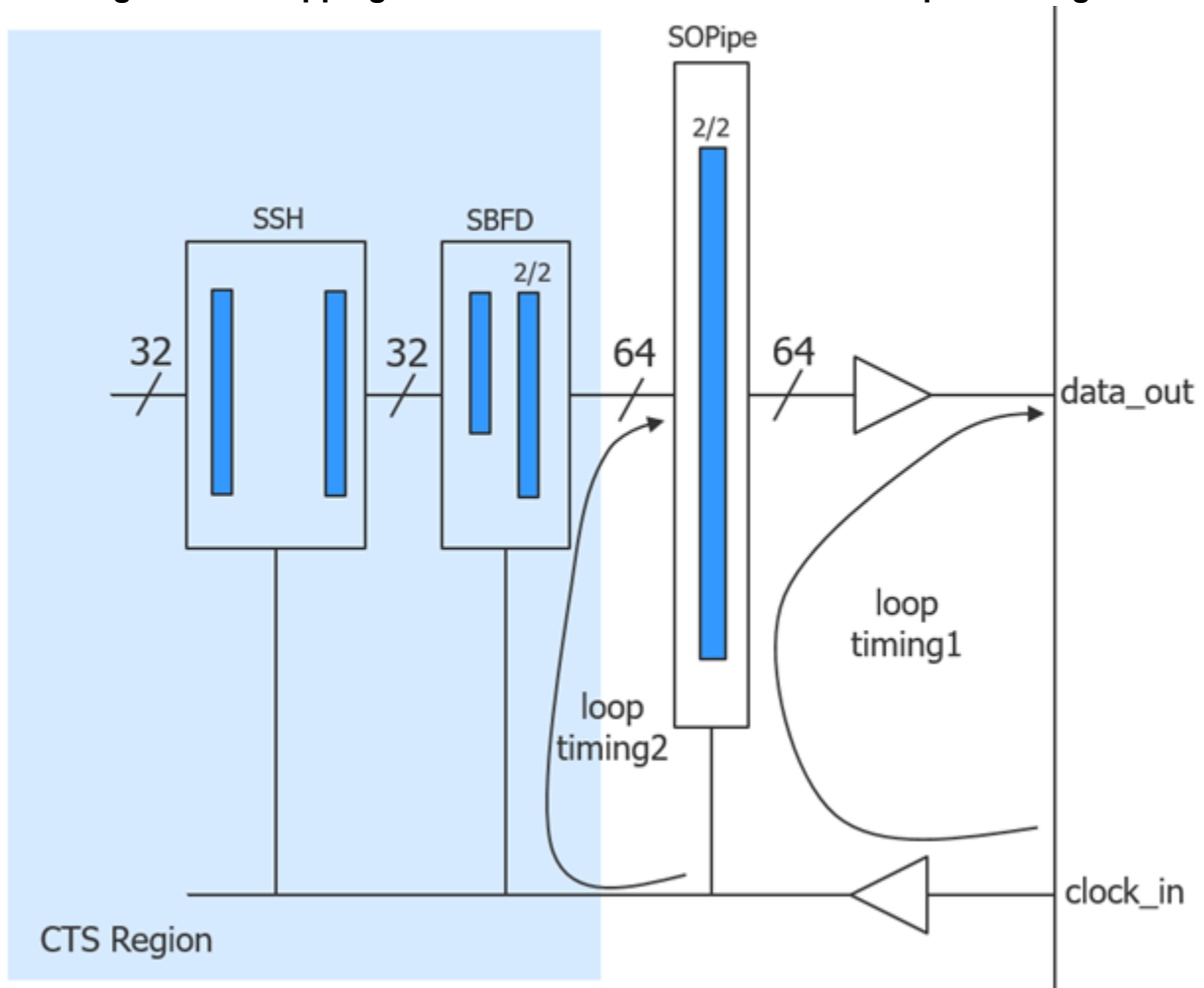


### Minimizing Loop Delay With the Tester

When a tester interacts with the SSN datapath of a chip, the critical timing path is always the loop timing path that starts from the clock supplied by the tester and continues to the data coming back from the chip. To be able to shift the SSN bus at 200 MHz, the worst case delay of

the clock coming into the chip and reaching the last stage of the pipelining, with the data coming out of the chip, must be less than 4 ns. As described in the [Using the BusFrequencyDivider to Enable Faster Internal Bus Clock Frequency](#) section of the BusFrequencyDivider topic in the *Tessent Shell Reference Manual*, when the delay is 4 ns in the worst case, it is about 1.5 ns in the best case. This leaves very little opening in which to place a strobe point on the ATE that is reliable in both best- and worst-case conditions. To minimize the internal bus width, it is easy to operate the SSN bus at 400 MHz, but you need to use a BusFrequencyDivider node to bring the external data rate to 200 MHz so that the loop timing delay is less than a clock period. If you used a hierarchical clock tree, a large portion of the loop timing would be consumed in the clock tree delay itself. [Figure 9-9](#) shows how to use an [OutputPipeline](#) node placed on a miniature local clock tree to retime the data before it goes out the chip. This eliminates the large CTS delay in the loop timing.

**Figure 9-9. Stepping Down the Clock Tree on the Last Pipeline Stage**





## Broadcast to Identical SSN Datapaths

When multiple identical SSN datapaths exist, it is beneficial to reuse the same copy of scan data going to each datapath. When you broadcast scan data to identical datapaths, you must set up observation for the outputs of each identical datapath independently.

You can use input broadcast for test when the physical implementation of the datapaths would benefit from broadcasting to their inputs from the same source. When you use a copy of the same data across multiple identical datapaths, this reduces scan data volume, because the same copy of input data tests the datapaths. Furthermore, testing identical datapaths concurrently can reduce test time.

However, not all designs with multiple datapaths benefit from input broadcast. For example, multi-die designs with nonidentical datapaths do not benefit from broadcasting their inputs from the same source, because nonidentical die do not use the same scan payload. Instead, each datapath requires its own payload. Similarly, designs with mux access to their datapaths would not benefit from input broadcast, because the mux access to the different die requires serial testing.

“Identical” refers to both the physical specifications of the datapaths and the configuration of the datapaths. For example, if one instance has on-chip compare enabled and a second otherwise identical instance has on-chip compare disabled, these instances are not eligible for input broadcast. The datapaths must be physically identical and their SSHs must be configured identically. The only exception to this is that you can include extra pipelining or a *disabled* SSH (which acts like pipeline stages) after the last active SSH.

Even when broadcasting to identical datapaths, top-level ATPG can still be run. This means that broadcast to SSN datapaths has a broader use than retargeting. In this case, the lower-level instances must load the same scan data, whether performing retargeting or top-level ATPG.

Independent observation of datapaths is required both during loading and unloading of the datapath.

## Determine Failing Cores on ATE With SSN

Test engineers can make binning decisions on ATE, based on which core instances pass or fail. Core instance binning is useful in salvaging die with a failing core or two to sell as a reduced-functional version of the design. Core instance binning is also valuable for yield analytics, which enables product engineers to determine which cores are more sensitive to defects.

Hierarchical scan methodologies typically connect the EDT outputs of core instances directly to pins for particular retargeting modes. This methodology simplifies binning on an ATE because of the one-to-one relationship between a failing pin and a failing core instance. SSN adds a layer of complexity because packetized outputs of the cores-under-test can occur on any output of the SSN bus. Binning on an ATE requires mapping a core instance to a pin and cycle combination.

## SSN and On-Chip Compare

It is straightforward to determine the failing core instance when the cores use the on-chip compare function of SSN. For these cores, the sticky\_status bit in the SSH captures the pass/fail status of the core instance. Read this bit through the IJTAG network to observe the TDO pin.

### Find the Failing SSH Instance

During a retargeting mode using retargeted patterns, the test reads the sticky status bit for each on-chip compare instance during the SSN\_end portion of the test. The tool annotates the vectors in the retargeted pattern file corresponding to each sticky\_status bit. The annotation indicates the SSH instance associated with the sticky status bit.

The following figure shows an example sticky\_status bit annotation. Identify the vector annotation by searching for “sticky\_status” at the end of the register name. The first portion of the register name ending with “\_inst” is the SSH instance name.

**Figure 9-10. Example sticky\_status Bit Annotation**

```
Ann {* TESSANT_MARKER:Previous Compare : pin TDO , ICL register =  
GPS_2.gps_baseband_rtl1_tessant_ssn_scan_host_1_inst.inner.sticky_status *}
```

### Find the Core Instance

To determine the core instance, use another set of annotations in the pattern file. Each core instance in the retargeting mode has an associated annotation section indicating which SSH instances are in each core instance. The following figure shows an example:

**Figure 9-11. Example Core Instance Annotations**

```
Ann {* Begin_Core_Instance_Section *}  
Ann {* instance = {core_g2_xtea_piccpu_m8051_1} *}  
Ann {* ssh_instance =  
{core_g2_xtea_piccpu_m8051_1/core_g2_xtea_piccpu_m8051_gate2_tessant_ssn_scan_host_piccpu_inst} *}  
Ann {* ssh_instance =  
{core_g2_xtea_piccpu_m8051_1/core_g2_xtea_piccpu_m8051_gate2_tessant_ssn_scan_host_xtea_inst} *}  
Ann {* ssh_instance =  
{core_g2_xtea_piccpu_m8051_1/core_g2_xtea_piccpu_m8051_gate2_tessant_ssn_scan_host_m8051_inst} *}  
Ann {* ssh_instance =  
{core_g2_xtea_piccpu_m8051_1/core_g2_xtea_piccpu_m8051_gate2_tessant_ssn_scan_host_main_inst} *}  
Ann {* End_Core_Instance_Section *}
```

Normally there is one SSH instance per core instance, but there can be more than one SSH per core as shown in the figure. Every SSH instance has its own packet location on the SSN bus.

## SSN Without On-Chip Compare

For core instances that do not use on-chip compare, there is no sticky\_status bit to indicate which one is failing. Instead, their failures can be observable on any pin of the SSN bus because they are interwoven with the outputs of all cores in the retargeting mode. Further complicating matters, bandwidth tuning allocates more data to the cores with larger patterns so that all cores

finish simultaneously to minimize wasted time. This packet rotation forms a repetitive cycle enabling a simple modulus calculation based on the cycle number and the number of cycles in each repetition.

The following figure illustrates this rotation for an example SSN bus with five pins for core instances a and b. Core “a” has two bits per scan word: a1 and a2. Core “b” has five bits per scan word: b1 through b5. These bit locations repeat every seven cycles.

**Figure 9-12. SSN Packet Rotation Cycles**

|       | Repetition 1 |      |      |      |      |      |      | Repetition 2 |      |      |      |      |      |      | Repetition 3 |      |      |      |      |      |      |
|-------|--------------|------|------|------|------|------|------|--------------|------|------|------|------|------|------|--------------|------|------|------|------|------|------|
| cycle | 0            | 1    | 2    | 3    | 4    | 5    | 6    | 7            | 8    | 9    | 10   | 11   | 12   | 13   | 14           | 15   | 16   | 17   | 18   | 19   | 20   |
| Pin1  | a[1]         | b[4] | b[2] | a[2] | b[5] | b[3] | b[1] | a[1]         | b[4] | b[2] | a[2] | b[5] | b[3] | b[1] | a[1]         | b[4] | b[2] | a[2] | b[5] | b[3] | b[1] |
| Pin2  | a[2]         | b[5] | b[3] | b[1] | a[1] | b[4] | b[2] | a[2]         | b[5] | b[3] | b[1] | a[1] | b[4] | b[2] | a[2]         | b[5] | b[3] | b[1] | a[1] | b[4] | b[2] |
| Pin3  | b[1]         | a[1] | b[4] | b[2] | a[2] | b[5] | b[3] | b[1]         | a[1] | b[4] | b[2] | a[2] | b[5] | b[3] | b[1]         | a[1] | b[4] | b[2] | a[2] | b[5] | b[3] |
| Pin4  | b[2]         | a[2] | b[5] | b[3] | b[1] | a[1] | b[4] | b[2]         | a[2] | b[5] | b[3] | b[1] | a[1] | b[4] | b[2]         | a[2] | b[5] | b[3] | b[1] | a[1] | b[4] |
| Pin5  | b[3]         | b[1] | a[1] | b[4] | b[2] | a[2] | b[5] | b[3]         | b[1] | a[1] | b[4] | b[2] | a[2] | b[5] | b[3]         | b[1] | a[1] | b[4] | b[2] | a[2] | b[5] |

The following figure summarizes the information using a table for each pin on the SSN bus. Each table also indicates the modulus and core instance relationships:

**Figure 9-13. Pin Tables With Modulus and Core Instance Relationships**

| Pin1 look-up table |               | Pin2 look-up table |               | ... | Pin5 look-up table |               |
|--------------------|---------------|--------------------|---------------|-----|--------------------|---------------|
| Modulus            | Core instance | Modulus            | Core instance |     | Modulus            | Core instance |
| 0                  | a             | 0                  | a             |     | 0                  | b             |
| 1                  | b             | 1                  | b             | ... | 1                  | b             |
| 2                  | b             | 2                  | b             |     | 2                  | a             |
| 3                  | a             | 3                  | b             |     | 3                  | b             |
| 4                  | b             | 4                  | a             |     | 4                  | b             |
| 5                  | b             | 5                  | b             |     | 5                  | a             |
| 6                  | b             | 6                  | b             |     | 6                  | b             |

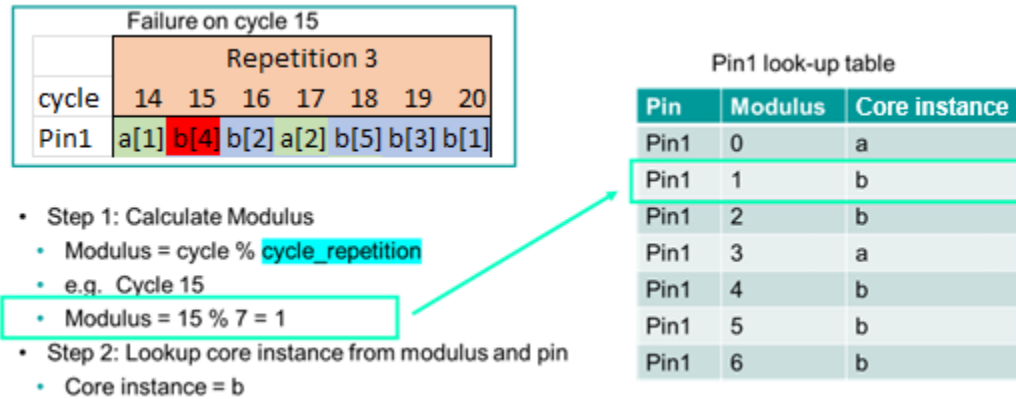
These pin tables associate each failing pin and cycle combination with a particular core instance based on the following modulus calculation:

$$\text{Modulus} = \text{Cycle} \% \text{Cycle Repetition}$$

The cycle repetition is seven, which was first established in [Figure 9-12](#). If there is a failure in cycle 15 on Pin1, the equation  $15 \% 7$  calculates that the modulus is 1. Using the Pin 1 look-up

table in Figure 9-13, modulus 1 reveals that the failure came from core instance “b”. The following figure has a summary:

**Figure 9-14. Determine Failing Core Instance With Modulus and Pin Tables**



There are annotations in the pattern file to provide the necessary information to calculate modulus and setup pin tables. The following figure shows an example of these annotations, which are located in the “ssn\_mapping” section:

**Figure 9-15. Example Modulus and Pin Table Annotations**

```
Ann { * TESSSENT_PRAGMA ssn_mapping -begin *}

Ann { * TESSSENT_PRAGMA ssn_mapping -datapath_id 1 -cycle_repetition 7
      -cycle_mod 0 -design_port "Pin1" -ssh_icl_instance
      a.a_rtl1_tessent_ssn_scan_host_1_inst -core_instance "a" *}

Ann { * TESSSENT_PRAGMA ssn_mapping -datapath_id 1 -cycle_repetition 7
      -cycle_mod 1 -design_port "Pin1" -ssh_icl_instance
      b.b_rtl1_tessent_ssn_scan_host_1_inst -core_instance "b" *}

...

Ann { * TESSSENT_PRAGMA ssn_mapping -datapath_id 1 -cycle_repetition 7
      -cycle_mod 6 -design_port "Pin5" -ssh_icl_instance
      b.b_rtl1_tessent_ssn_scan_host_1_inst -core_instance "b" *}

Ann { * TESSSENT_PRAGMA ssn_mapping -end *}

```

The final piece of information that you need is the cycle where the rotation begins. The vector in the retargeted pattern file that has the following annotation is the cycle where the rotation begins:

**Figure 9-16. Annotation Where the Cycle Rotation Begins**

```
Ann {* TESSent_PRAGMA ssn_mapping -cycle_repetition_beginning *}
```

## Yield Statistics on ATE With SSN

You may use per-pin fail limits to prevent a single core from filling up the fail buffer on the tester during manufacturing. You may also perform partial good die testing and need to identify failing cores live on the tester without performing full failure diagnosis. In these cases, you can create SSN patterns with packet constraints to enable both partial good die testing and yield statistics tabulation for failing cores while using per-pin fail limits. This feature is useful in cases where the pass-fail sticky bit is unavailable for indicating the presence of a failure.

---

### Note

 By default, the pattern includes annotations that specify the mapping from the top-level ports to the SSH, regardless of whether packet rotation is on or off. You can control these annotations with the “[set\\_ssn\\_options -ssh\\_mapping\\_annotations](#)” command.

---

---

### Note

 Creating SSN patterns with packet constraints is useful for testing *non*-identical cores for which the SSH is not implemented with the on-chip compare feature.

---

Use the “[set\\_ssn\\_options -packet\\_constraints no\\_rotation](#)” command to stop rotation of the packet around the SSN bus, as shown in [Figure 9-17](#). In this figure, the packet is nine bits wide and contains five bits of Core A and four bits of Core B. The SSN bus is eight bits wide [7:0]. The packet rotates around the bus by one bit in bus\_clock cycle 0 in the top half of the figure. As the bus\_clock cycles increase, the packet continues to rotate around the bus. When you stop rotation of the packet with the [set\\_ssn\\_options](#) command, the tool pads the packet to make the packet size a multiple of the bus width, as shown in the bottom half of the figure. In this example, the packet increases to two bus\_clock cycles wide (0 and 1), with seven bits of padding added to time slots one through seven in Cycle 1.

**Figure 9-17. Packet Rotation on the SSN Bus**

|            |   | Normal operation (no padding, possible packet rotation) |       |       |       |       |       |   |  |
|------------|---|---------------------------------------------------------|-------|-------|-------|-------|-------|---|--|
|            |   | Packet = 9 bits                                         |       |       |       |       |       |   |  |
|            |   | Cycle                                                   |       |       |       |       |       |   |  |
|            |   |                                                         | 5     | 4     | 3     | 2     | 1     | 0 |  |
| Bus<br>bit | 7 | A-5-2                                                   | A-4-3 | A-3-4 | B-2-0 | B-1-1 | B-0-2 |   |  |
|            | 6 | A-5-1                                                   | A-4-2 | A-3-3 | A-2-4 | B-1-0 | B-0-1 |   |  |
|            | 5 | A-5-0                                                   | A-4-1 | A-3-2 | A-2-3 | A-1-4 | B-0-0 |   |  |
|            | 4 | B-4-3                                                   | A-4-0 | A-3-1 | A-2-2 | A-1-3 | A-0-4 |   |  |
|            | 3 | B-4-2                                                   | B-3-3 | A-3-0 | A-2-1 | A-1-2 | A-0-3 |   |  |
|            | 2 | B-4-1                                                   | B-3-2 | B-2-3 | A-2-0 | A-1-1 | A-0-2 |   |  |
|            | 1 | B-4-0                                                   | B-3-1 | B-2-2 | B-1-3 | A-1-0 | A-0-1 |   |  |
|            | 0 | A-4-4                                                   | B-3-0 | B-2-1 | B-1-2 | B-0-3 | A-0-0 |   |  |

↓

|            |   | Packet padding to disable packet rotation |       |       |       |       |       |   |  |
|------------|---|-------------------------------------------|-------|-------|-------|-------|-------|---|--|
|            |   | Packet = 16 bits (2 tester cycles)        |       |       |       |       |       |   |  |
|            |   | Cycle                                     |       |       |       |       |       |   |  |
|            |   |                                           | 5     | 4     | 3     | 2     | 1     | 0 |  |
| Bus<br>bit | 7 | ----                                      | B-2-2 | ----  | B-1-2 | ----  | B-0-2 |   |  |
|            | 6 | ----                                      | B-2-1 | ----  | B-1-1 | ----  | B-0-1 |   |  |
|            | 5 | ----                                      | B-2-0 | ----  | B-1-0 | ----  | B-0-0 |   |  |
|            | 4 | ----                                      | A-2-4 | ----  | A-1-4 | ----  | A-0-4 |   |  |
|            | 3 | ----                                      | A-2-3 | ----  | A-1-3 | ----  | A-0-3 |   |  |
|            | 2 | ----                                      | A-2-2 | ----  | A-1-2 | ----  | A-0-2 |   |  |
|            | 1 | ----                                      | A-2-1 | ----  | A-1-1 | ----  | A-0-1 |   |  |
|            | 0 | B-2-3                                     | A-2-0 | B-1-3 | A-1-0 | B-0-3 | A-0-0 |   |  |

Regardless of the number of bus words the packet occupies, when you use the “set\_ssn\_options -packet\_constraints no\_rotation” command, the tool adds padding to the packet to make the packet a multiple of the bus width. This enables a predictable mapping from top-level data\_out ports to the active SSHs. Because the packet can be multiple bus words, the mapping is different for each bus\_clock cycle, as shown in the following example.

This example illustrates the annotations that the write\_patterns command adds to the SSN STIL or WGL pattern files to show the SSH to SSN data\_out port mapping required when you use the “set\_ssn\_options -packet\_constraints no\_rotation” command.

**Example 9-1. Annotations for SSN Mapping With Packet Rotation Disabled**

```

Ann { * TESSENT_PRAGMA ssn_mapping -begin * }
Ann { * TESSENT_PRAGMA ssn_mapping -datapath_id 1 -cycle_repetition 2
-cycle_mod 0 -design_port "ssn_bus_data_out[3]" -ssh_icl_instance
xtea_3.xtea_rtl_tessent_ssn_scan_host_1_inst -core_instance "xtea_3" * }
Ann { * TESSENT_PRAGMA ssn_mapping -datapath_id 1 -cycle_repetition 2
-cycle_mod 0 -design_port "ssn_bus_data_out[4]" -ssh_icl_instance
xtea_3.xtea_rtl_tessent_ssn_scan_host_1_inst -core_instance "xtea_3" * }
Ann { * TESSENT_PRAGMA ssn_mapping -datapath_id 1 -cycle_repetition 2
-cycle_mod 0 -design_port "ssn_bus_data_out[5]" -ssh_icl_instance
xtea_2.xtea_rtl_tessent_ssn_scan_host_1_inst -core_instance "xtea_2" * }
Ann { * TESSENT_PRAGMA ssn_mapping -datapath_id 1 -cycle_repetition 2
-cycle_mod 0 -design_port "ssn_bus_data_out[0]" -ssh_icl_instance
xtea_3.xtea_rtl_tessent_ssn_scan_host_1_inst -core_instance "xtea_3" * }
Ann { * TESSENT_PRAGMA ssn_mapping -datapath_id 1 -cycle_repetition 2
-cycle_mod 0 -design_port "ssn_bus_data_out[1]" -ssh_icl_instance
xtea_3.xtea_rtl_tessent_ssn_scan_host_1_inst -core_instance "xtea_3" * }
Ann { * TESSENT_PRAGMA ssn_mapping -datapath_id 1 -cycle_repetition 2
-cycle_mod 0 -design_port "ssn_bus_data_out[2]" -ssh_icl_instance
xtea_3.xtea_rtl_tessent_ssn_scan_host_1_inst -core_instance "xtea_3" * }
Ann { * TESSENT_PRAGMA ssn_mapping -datapath_id 1 -cycle_repetition 2
-cycle_mod 1 -design_port "ssn_bus_data_out[3]" -ssh_icl_instance
xtea_2.xtea_rtl_tessent_ssn_scan_host_1_inst -core_instance "xtea_2" * }
Ann { * TESSENT_PRAGMA ssn_mapping -datapath_id 1 -cycle_repetition 2
-cycle_mod 1 -design_port "ssn_bus_data_out[4]" -ssh_icl_instance
xtea_2.xtea_rtl_tessent_ssn_scan_host_1_inst -core_instance "xtea_2" * }
Ann { * TESSENT_PRAGMA ssn_mapping -datapath_id 1 -cycle_repetition 2
-cycle_mod 1 -design_port "ssn_bus_data_out[5]" -ssh_icl_instance
xtea_2.xtea_rtl_tessent_ssn_scan_host_1_inst -core_instance "xtea_2" * }
Ann { * TESSENT_PRAGMA ssn_mapping -datapath_id 1 -cycle_repetition 2
-cycle_mod 1 -design_port "ssn_bus_data_out[0]" -ssh_icl_instance
xtea_2.xtea_rtl_tessent_ssn_scan_host_1_inst -core_instance "xtea_2" * }
Ann { * TESSENT_PRAGMA ssn_mapping -datapath_id 1 -cycle_repetition 2
-cycle_mod 1 -design_port "ssn_bus_data_out[1]" -ssh_icl_instance
xtea_2.xtea_rtl_tessent_ssn_scan_host_1_inst -core_instance "xtea_2" * }
Ann { * TESSENT_PRAGMA ssn_mapping -datapath_id 1 -cycle_repetition 2
-cycle_mod 1 -design_port "ssn_bus_data_out[2]" -ssh_icl_instance
xtea_2.xtea_rtl_tessent_ssn_scan_host_1_inst -core_instance "xtea_2" * }
Ann { * TESSENT_PRAGMA ssn_mapping -end * }

```

This mapping enables the ATE to set up a fail limit per  $pin \times (shift\_cycle \% cycle\_repetition)$ , where *cycle\_repetition* is the number of bus words a packet occupies. In this way, you can ensure that all cores can be observed and the ATE can enforce a per-pin limit for failure messages.

If you need to map back to the specific from\_scan\_out bus of the SSH without dealing with the complexities of bandwidth tuning, you can disable data throttling, combined with disabling the packet rotation. This typically costs test efficiency, so it may not suit your overall run time needs. Use the “set\_ssn\_options -packet\_constraints no\_rotation -throttling off” command to disable data throttling and stop the rotation of the packet around the bus.

The following example shows annotations that the `write_patterns` command adds to the SSN STIL or WGL pattern files to show the SSH to SSN `data_out` port mapping required when you use the “`set_ssn_options -packet_constraints no_rotation`” command.

### Example 9-2. Annotations for SSN Mapping With Packet Rotation and Throttling Disabled

```
Ann { * TESSENT_PRAGMA ssn_mapping -begin *}
Ann { * TESSENT_PRAGMA ssn_mapping -datapath_id 1 -cycle_repetition 2
-cycle_mod 0 -design_port "ssn_bus_data_out[3]" -ssh_icl_instance
xtea_3.xtea_rtl_tessent_ssn_scan_host_1_inst -core_instance "xtea_3"
-ssh_chain_group 3 -ssh_chain_group_chain_index 2 *}
Ann { * TESSENT_PRAGMA ssn_mapping -datapath_id 1 -cycle_repetition 2
-cycle_mod 0 -design_port "ssn_bus_data_out[4]" -ssh_icl_instance
xtea_3.xtea_rtl_tessent_ssn_scan_host_1_inst -core_instance "xtea_3"
-ssh_chain_group 3 -ssh_chain_group_chain_index 3 *}
Ann { * TESSENT_PRAGMA ssn_mapping -datapath_id 1 -cycle_repetition 2
-cycle_mod 0 -design_port "ssn_bus_data_out[5]" -ssh_icl_instance
xtea_2.xtea_rtl_tessent_ssn_scan_host_1_inst -core_instance "xtea_2"
-ssh_chain_group 1 -ssh_chain_group_chain_index 0 *}
Ann { * TESSENT_PRAGMA ssn_mapping -datapath_id 1 -cycle_repetition 2
-cycle_mod 0 -design_port "ssn_bus_data_out[0]" -ssh_icl_instance
xtea_3.xtea_rtl_tessent_ssn_scan_host_1_inst -core_instance "xtea_3"
-ssh_chain_group 2 -ssh_chain_group_chain_index 4 *}
Ann { * TESSENT_PRAGMA ssn_mapping -datapath_id 1 -cycle_repetition 2
-cycle_mod 0 -design_port "ssn_bus_data_out[1]" -ssh_icl_instance
xtea_3.xtea_rtl_tessent_ssn_scan_host_1_inst -core_instance "xtea_3"
-ssh_chain_group 3 -ssh_chain_group_chain_index 0 *}
Ann { * TESSENT_PRAGMA ssn_mapping -datapath_id 1 -cycle_repetition 2
-cycle_mod 0 -design_port "ssn_bus_data_out[2]" -ssh_icl_instance
xtea_3.xtea_rtl_tessent_ssn_scan_host_1_inst -core_instance "xtea_3"
-ssh_chain_group 3 -ssh_chain_group_chain_index 1 *}
```



```

Ann {* TESSENT_PRAGMA ssn_mapping -datapath_id 1 -cycle_repetition 2
-cycle_mod 1 -design_port "ssn_bus_data_out[3]" -ssh_icl_instance
xtea_2.xtea_rtl_tessent_ssn_scan_host_1_inst -core_instance "xtea_2"
-ssh_chain_group 1 -ssh_chain_group_chain_index 4 *}
Ann {* TESSENT_PRAGMA ssn_mapping -datapath_id 1 -cycle_repetition 2
-cycle_mod 1 -design_port "ssn_bus_data_out[4]" -ssh_icl_instance
xtea_2.xtea_rtl_tessent_ssn_scan_host_1_inst -core_instance "xtea_2"
-ssh_chain_group 2 -ssh_chain_group_chain_index 0 *}
Ann {* TESSENT_PRAGMA ssn_mapping -datapath_id 1 -cycle_repetition 2
-cycle_mod 1 -design_port "ssn_bus_data_out[5]" -ssh_icl_instance
xtea_2.xtea_rtl_tessent_ssn_scan_host_1_inst -core_instance "xtea_2"
-ssh_chain_group 2 -ssh_chain_group_chain_index 1 *}
Ann {* TESSENT_PRAGMA ssn_mapping -datapath_id 1 -cycle_repetition 2
-cycle_mod 1 -design_port "ssn_bus_data_out[0]" -ssh_icl_instance
xtea_2.xtea_rtl_tessent_ssn_scan_host_1_inst -core_instance "xtea_2"
-ssh_chain_group 1 -ssh_chain_group_chain_index 1 *}
Ann {* TESSENT_PRAGMA ssn_mapping -datapath_id 1 -cycle_repetition 2
-cycle_mod 1 -design_port "ssn_bus_data_out[1]" -ssh_icl_instance
xtea_2.xtea_rtl_tessent_ssn_scan_host_1_inst -core_instance "xtea_2"
-ssh_chain_group 1 -ssh_chain_group_chain_index 2 *}
Ann {* TESSENT_PRAGMA ssn_mapping -datapath_id 1 -cycle_repetition 2
-cycle_mod 1 -design_port "ssn_bus_data_out[2]" -ssh_icl_instance
xtea_2.xtea_rtl_tessent_ssn_scan_host_1_inst -core_instance "xtea_2"
-ssh_chain_group 1 -ssh_chain_group_chain_index 3 *}
Ann {* TESSENT_PRAGMA ssn_mapping -end *}

```

## Manufacturing Patterns With SSN

When you use the streaming scan network (SSN), you create manufacturing patterns from a netlist that has been through layout and timing closure. You should create these patterns without any ATPG pattern limits to achieve maximum test coverage. Simulate these patterns using a cycle-based simulator, backannotating SDF delays on the design. For larger designs, it may not be practical to simulate all scan patterns.

### Note

 The process of creating manufacturing patterns is different from creating signoff patterns. For a description of signoff patterns with SSN, refer to the section “[Signoff Patterns With SSN](#)” on page 577.

Use the `set_load_unload_timing_options` command to set the timing of the SSN when creating manufacturing patterns. If you share the SDC procedure `set_load_unload_timing_options` between the Tessent tools and static timing analysis, it is only required for you to source the shared file. Sharing the SDC procedure `set_load_unload_timing` ensures the timing of the manufacturing patterns written from the Tessent tools precisely matches the timing of the design.

[Table 9-1](#) shows the patterns you should use to test your silicon with SSN. These patterns are recommended for both first silicon test and production test. It is also recommended for both that you start with the least complex patterns and incrementally verify the SSN with each pattern up through the most complex patterns. The ICLNetwork Verify patterns are the least complex, gradually leading to Retargeted SSN patterns and (where applicable) Top-Level ATPG SSN patterns. For more detailed descriptions of the patterns in this table, refer to the section “[Signoff Patterns With SSN](#)” on page 577.

Only designs that you have implemented with on-chip compare require the OComp Self-Test Pattern shown in this table. This pattern verifies that the SSN on-chip compare logic is free of any stuck-at faults. For more information about SSN on-chip compare patterns, refer to “How to Write SSN On-Chip Compare Patterns” in this section.

**Table 9-1. SSN Manufacturing Pattern Summary**

|                       | Order of Priority<br>ICLNetwork<br>Verify Patterns <sup>1</sup> | →<br>Continuity<br>Pattern <sup>2</sup> | →<br>OComp Self-<br>Test Pattern <sup>3</sup> | →<br>Loopback Pattern | Order of Priority<br>Top-Level ATPG<br>and Retargeted<br>SSN Patterns |
|-----------------------|-----------------------------------------------------------------|-----------------------------------------|-----------------------------------------------|-----------------------|-----------------------------------------------------------------------|
| First silicon<br>test | X                                                               | X                                       | X                                             | X                     | X (chain test)                                                        |
|                       |                                                                 |                                         |                                               |                       | X (scan test)                                                         |
| Production<br>test    | 4                                                               | 4                                       | X                                             | 4                     | X (chain test)                                                        |
|                       |                                                                 |                                         |                                               |                       | X (scan test)                                                         |

1. Includes verification of streaming-through-IJTAG.

2. Based on multiplexers in the active datapath, you may need multiple continuity patterns.
3. Required only when the SSN is equipped with on-chip compare. If your datapath includes multiplexers, required only when the multiplexers are configured such that the datapath contains ScanHost nodes with OCComp.
4. Optional pattern that can be used to identify cause of failure.

### Note

 Manufacturing patterns achieve maximum test coverage of the device under test and do not have any pattern limits when created.

The following topics describe the SSN patterns and, because you can write the procedures that make up SSN patterns to separate files, they also describe the strict order you must follow when applying the manufacturing patterns to the tester when you have written the different procedures of the SSN pattern to separate files:

|                                                                  |            |
|------------------------------------------------------------------|------------|
| <b>Manufacturing Pattern Quick Reference</b> .....               | <b>559</b> |
| <b>SSN Pattern Structure</b> .....                               | <b>560</b> |
| <b>How to Write Complete SSN Patterns</b> .....                  | <b>561</b> |
| <b>How to Write JTAG and Payload Procedures Separately</b> ..... | <b>562</b> |
| <b>How to Write SSN On-Chip Compare Patterns</b> .....           | <b>566</b> |
| <b>How To Write Streaming-Through-IJTAG Patterns</b> .....       | <b>568</b> |
| <b>SSN Debug on the Tester</b> .....                             | <b>570</b> |

## Manufacturing Pattern Quick Reference

The table in this topic summarizes the manufacturing patterns for use at first silicon test and production test. You should simulate the Verilog pattern equivalent of these patterns without error before releasing them to be used on the tester.

**Table 9-2. Manufacturing Pattern Quick Reference**

| Pattern Type      | Limits | Description                                                                                                                                                                                                     |
|-------------------|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ICLNetwork verify | None   | Verifies ICL instruments (tool and user) are readable and writable along the IJTAG serial path.<br>Verifies the full scope of the IJTAG network, including streaming-through-IJTAG path.<br>Runs at TCK period. |
| Continuity        | None   | Verifies SSN bus integrity between bus data_in and data_out.<br>Runs at bus_clock period.                                                                                                                       |

**Table 9-2. Manufacturing Pattern Quick Reference (cont.)**

| Pattern Type        | Limits | Description                                                                                                                                                                                                           |
|---------------------|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Parallel scan       | None   | Verifies each scan register can reliably capture.<br>Capture clock and scan signals are cut points on SSH that testbench pulses.<br>For retargeted scan patterns, can verify top-level clocking to lower-level cores. |
| OCCompare self-test | None   | Verifies no stuck-at faults exist in SSH on-chip compare logic.                                                                                                                                                       |
| SSH loopback        | None   | Verifies SSN network can reliably deliver packetized data (excluding path to EDT).<br>All SSN nodes are programmed with ssn_setup procedure as though full pattern is to be run.<br>Runs at bus_clock period.         |
| Serial chain        | None   | Delivers packetized data to all SSH and verifies all chains shift without error. Saves only nonmasking patterns.<br>The bus_clock in the SSH generates scan signals.<br>Runs off bus_clock.                           |
| Serial scan         | None   | Delivers packetized data to all SSH and verifies all chains shift and capture without error. SSN end-to-end test.<br>The bus_clock in the SSH generates scan signals.<br>Runs off bus_clock.                          |

## SSN Pattern Structure

This section describes the basic structure of the SSN pattern as the write\_patterns command writes it.

When you use the streaming scan network (SSN), there are additional procedures that are part of every SSN pattern that do not appear in non-SSN patterns: ssn\_setup and ssn\_end. Furthermore, SSN delivers the scan test data as a stream of data over the SSN bus. The ssn\_setup, test\_payload, and ssn\_end procedures combine with the existing test\_setup and test\_end procedures to create an SSN pattern as shown in [Figure 9-18](#).

The ssn\_setup and ssn\_end procedures configure the SSN nodes before and after you apply the test\_payload to the network. The test\_payload is packetized scan data that streams over the SSN bus, which delivers it to the streaming scan hosts (SSH) and scan chains. The ssn\_setup procedure prepares each active SSH to receive streaming data by programming the SSH ijttag\_registers. The ssn\_end procedure prepares the SSN for the next test\_payload by resetting the SSN nodes without resetting the IJTAG programming. When you write SSN patterns using the -max\_loads option, inclusion of an ssn\_end procedure after each test\_payload enables the

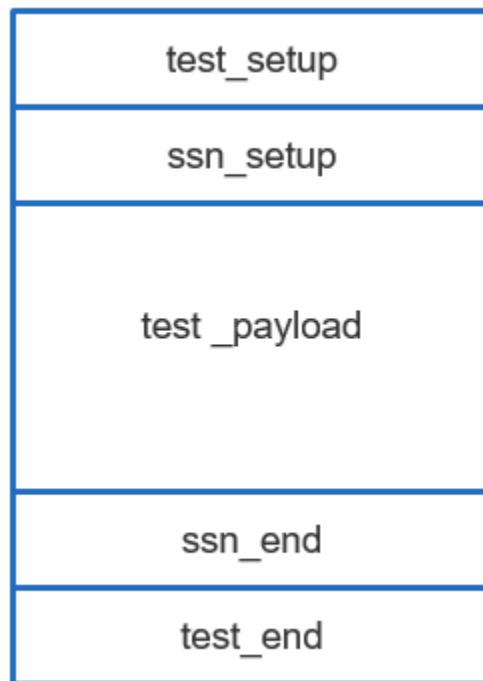
individual test\_payload patterns to be applied in any order. The ssn\_end procedures also unload the SSH sticky bits when on-chip compare is active.

To produce the simplest SSN scan pattern, run the write\_patterns command using only a format and the -replace switch. The following example and figure show an SSN pattern with the SSN-specific procedures along with the test\_setup and test\_end procedures. The ssn\_setup and ssn\_end SSN procedures occur directly before and after the test\_payload, respectively.

### Example 9-3. Writing the Simplest Complete SSN Scan Pattern

```
write_patterns chip.stil -stil -replace
```

Figure 9-18. SSN Pattern Structure



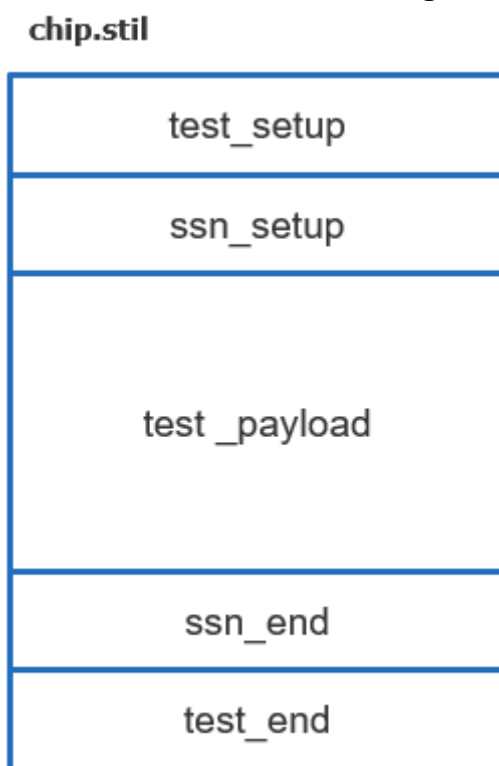
## How to Write Complete SSN Patterns

You can write complete SSN patterns into a single pattern file.

When you write SSN patterns, as in the following example, the [write\\_patterns](#) command stores the test\_setup, ssn\_setup, test\_payload, ssn\_end, and test\_end procedures all in a single file:

```
write_patterns chip.stil -stil -replace
```

**Figure 9-19. SSN Patterns in a Single STIL File**



Because of tool default values, the preceding command is a shortened form of this one:

```
write_patterns chip.stil -test_setup one_per_file -ssn_setup one_per_file -test_end one -stil \
-replace
```

---

**Note**

 If on-chip compare (OCComp) is turned off, the tool automatically removes the ssn\_end procedure at the end of the payload.

---

## How to Write JTAG and Payload Procedures Separately

Duplicating setup procedures for large manufacturing counts may result in the setup procedures dominating test time that should be allocated to applying the test\_payload. To avoid this, you can write procedures for the SSN patterns to separate files in a similar fashion to how you can write test\_setup and test\_end procedures to separate files for non-SSN patterns. This enables more tester time to be spent applying test\_payload patterns.

---

**Caution**

 When you write out the test\_setup, ssn\_setup, ssn\_end, test\_end, and test\_payload separately, you are responsible for ensuring that the patterns are applied to the tester in the correct order. If they are not, the tester produces mismatches on the SSN bus output.

---

Follow these rules when you write the procedures of SSN patterns to separate files:

- Each `write_patterns` command must use the same pattern parameters (such as `-begin/-end` and `-pattern_sets`).
- The individual procedures must be applied in the correct order on the tester.
- No additional clock cycles (including TCK) or pin constraints should be applied to the DUT before or after applying the different pattern files.

---

**Tip**

-  SSN patterns are natively written to a single pattern file. Use `write_patterns` command options to control whether it excludes a procedure when it writes the pattern files
- 

---

**Tip**

-  When you write the SSN payload out separately and use the `-max_loads` option, ensure the number of loads is greater than the number of cycles of `ssn_setup` and `ssn_end`.
- 

---

**Tip**

-  The constraint “`timeplate_constraints same_period`” applies to SSN patterns when you write the JTAG procedures and payload to the same pattern file. When you write them into separate pattern files, the tool uses “`timeplate_constraints none`”. Writing JTAG and payload procedures into separate files provides the most efficient use of tester memory. Refer to the “[set\\_tester\\_options](#)” command for more information about `-timeplate_constraints`.
- 

[Example 9-4](#) shows how to optionally write some of the procedures of the SSN pattern to separate pattern files. It writes the `test_setup`, `ssn_setup`, and `test_end` procedures separately from the SSH loopback and payload patterns. By writing out `test_setup`, `ssn_setup`, and `test_end` separately, you can apply numerous payload patterns without the penalty of applying slow TCK procedures more than once.

Separate `ssn_setup` patterns are required for SSH loopback, chain payload, and scan payload data to ensure the SSH programming matches the payload.

### Example 9-4. SSN Patterns With Separate test\_setup and ssn\_setup

```
write_patterns test_setup.stil -test_setup only -stil -replace

write_patterns ssn_setup_loopback.stil -ssn_setup only \
  -pattern_set ssh_loopback -stil -replace
write_patterns ssh_loopback_payload.stil -test_payload \
  -pattern_set ssh_loopback -stil -replace
write_patterns ssn_end.stil -ssn_end only -pattern_set ssh_loopback \
  -stil -replace

write_patterns ssn_setup_chain.stil -ssn_setup only -pattern_set chain \
  -stil -replace
write_patterns chain_payload.stil -test_payload -maxloads <n> \
  -pattern_set chain -stil -replace
write_patterns ssn_end.stil -ssn_end only -pattern_set chain -stil \
  -replace

write_patterns ssn_setup_scan.stil -ssn_setup only -pattern_set scan \
  -stil -replace
write_patterns scan_payload.stil -test_payload -maxloads <n> \
  -pattern_set scan -stil -replace
write_patterns ssn_end.stil -ssn_end only -pattern_set scan -stil -replace

write_patterns test_end.stil -test_end only -stil -replace
```

The write\_patterns commands in this example demonstrate how to write the payload and JTAG procedures separately. When you write these procedures separately, the tck\_ratio in the pattern is one and the tool can maximize the compression for ATE memory.

Figure 9-20 illustrates the correct order in which to apply the pattern files. The test\_setup procedure is your custom chip setup and is identical to what you would use without SSN. The ssn\_setup procedure prepares the SSN by configuring the ScanHost nodes through the programming in the embedded IJTAG registers. Each pattern type requires its own ssn\_setup procedure, because the JTAG programming may differ between them. For example, the throttling (that is, bits per packet) for SSH loopback and chain test patterns may be different. The payload pattern files are applied after the ssn\_setup procedure. The ssn\_end procedure is applied after the payload and prepares the network for the next pattern file when you use the -max\_loads argument. When on-chip compare is active, the ssn\_end procedures spit out the on-chip compare sticky status bits, which are visible on the TDO. The test\_end procedure is the last procedure of the pattern set and reconfigures the IJTAG network to a state that is equivalent to the post-iReset state.

---

#### Note

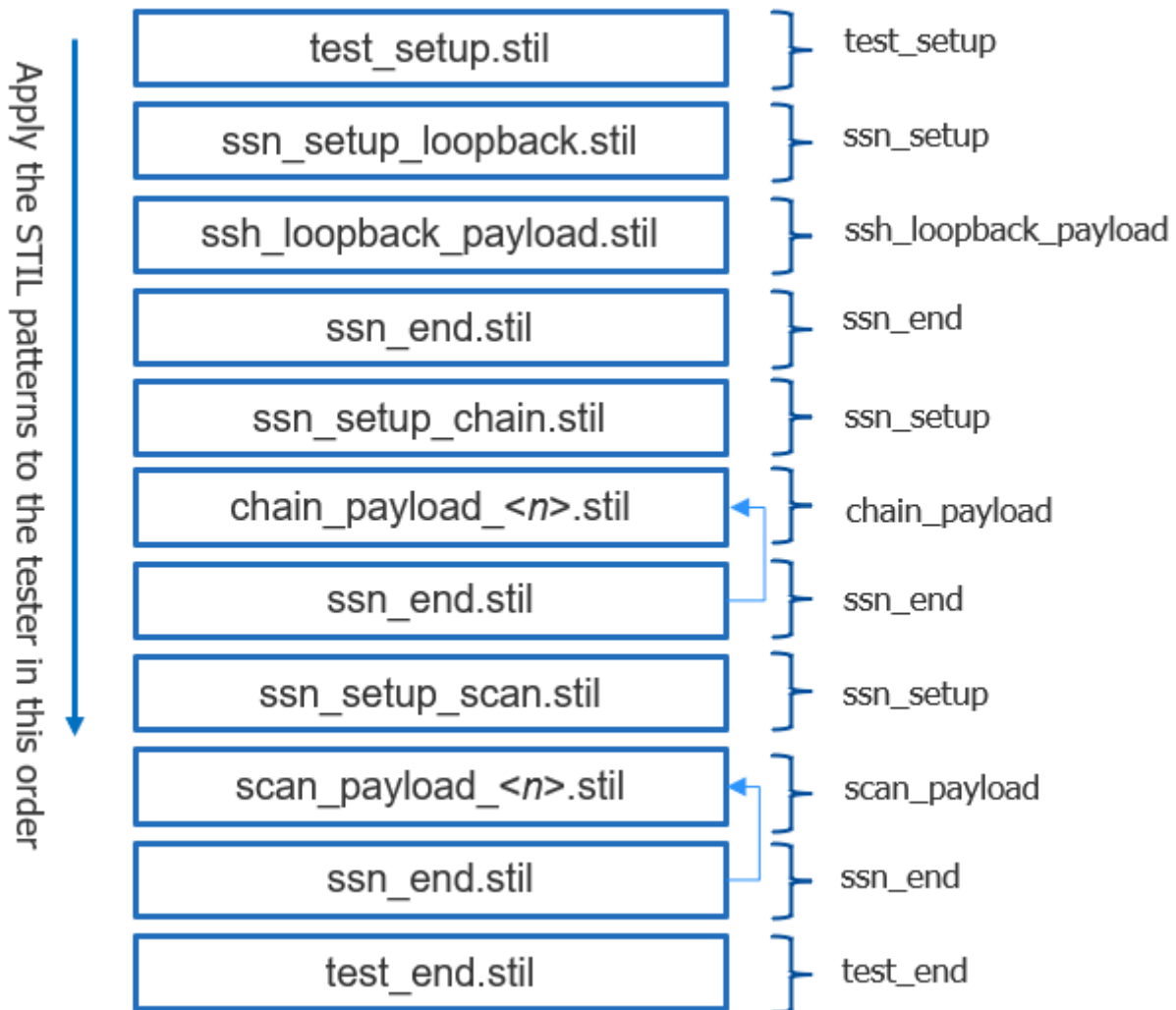


The payload patterns can be applied in any order as long as the ssn\_end procedure is applied after each payload pattern file.

---



**Figure 9-20. SSN Pattern Files With Order for Application on Tester**



### Using -all\_\* Options To Minimize File Count

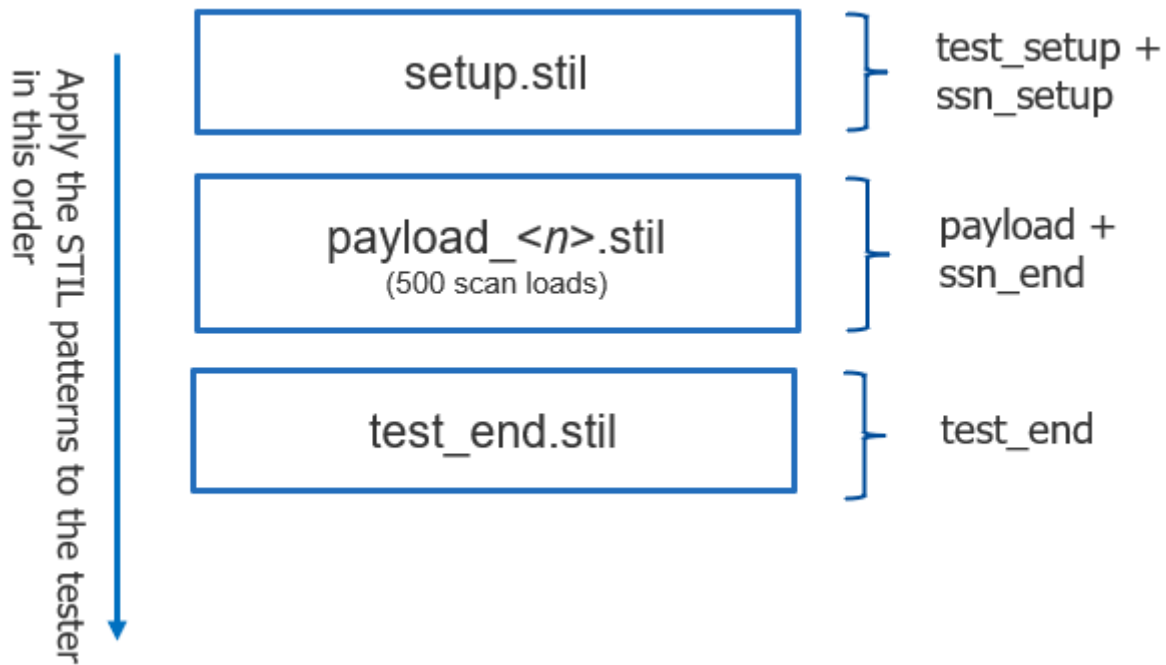
You can group the test\_setup and ssn\_setup procedures into a single file by specifying the -all\_setup\_only switch, as in the following example.

#### Example 9-5. SSN Pattern Files With Combined Single test\_setup and ssn\_setup

```
write_patterns setup.stil -all_setup_only -stil -replace
write_patterns payload.stil -test_payload -max_loads 500 \
  -pattern_set scan -stil -replace
write_patterns test_end only -stil -replace
```

When you write SSN patterns with the -all\_setup\_only switch, it produces a combined setup STIL file that is separate from the test\_payload and SSN and test end procedures, as in the following figure.

**Figure 9-21. SSN Pattern Files With Combined Setup Showing Tester Order**



---

**Note**

 You cannot combine the -all\_end\_only switch with the -max\_loads option, because the ssn\_end procedure must be applied after each payload pattern file, as shown in [Figure 9-20](#) on page 565 and [Figure 9-21](#).

---

## How to Write SSN On-Chip Compare Patterns

The on-chip compare feature of SSN is one of the primary features to make your test process more efficient. It is common to use it when you have identical core instances or you are performing partial good die testing. When present, the on-chip compare hardware in the SSH compares expected and measured scan responses. You must test and verify that the on-chip compare hardware and the sticky bit status registers in each SSH are free of any stuck-at faults.

The following example shows how to write the on-chip compare self-test pattern alongside the other SSN patterns. It also uses the option to write the test\_end procedure to a separate file. Use these commands as a guide for using on-chip compare with your design.

**Example 9-6. Writing Each Procedure of the SSN Pattern to Individual Files**

```
write_patterns test_setup.stil -test_setup only -stil -replace

write_patterns ssn_setup_loopback.stil -ssn_setup only \
  -pattern_set ssh_loopback -stil -replace
write_patterns ssh_loopback_payload.stil -test_payload \
  -pattern_set \ssh_loopback -stil -replace
write_patterns ssn_end.stil -ssn_end only -pattern_set ssh_loopback
  -stil -replace

write_patterns ssn_setup_occomp.stil -ssn_setup only \
  -pattern_set ssh_on_chip_compare -stil -replace
write_patterns ocomp_payload.stil -test_payload \
  -pattern_set ssh_on_chip_compare -stil -replace
write_patterns ssn_end.stil -ssn_end only \
  -pattern_set ssh_on_chip_compare -stil -replace

write_patterns ssn_setup_chain.stil -ssn_setup only \
  -pattern_set chain -stil -replace
write_patterns chain_payload.stil -test_payload -maxloads <n> \
  -pattern_set chain -stil -replace
write_patterns ssn_end.stil -ssn_end only -pattern_set chain -stil \
  -replace

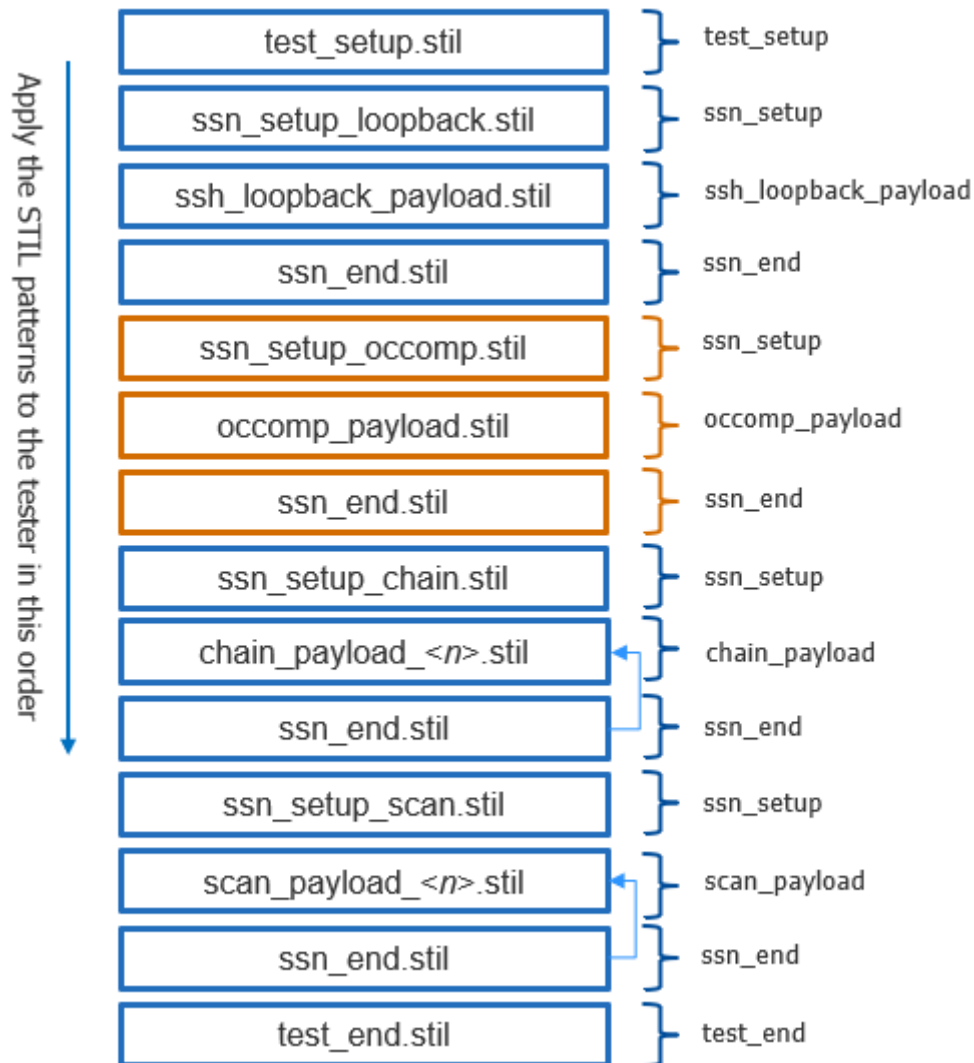
write_patterns ssn_setup_scan.stil -ssn_setup only -pattern_set scan \
  -stil -replace
write_patterns scan_payload.stil -test_payload -maxloads <n> \
  -pattern_set scan -stil -replace
write_patterns ssn_end.stil -ssn_end only -pattern_set scan -stil -replace

write_patterns test_end.stil -test_end only -stil -replace
```

The JTAG and payload patterns in this example must be applied on the tester in the correct order to avoid mismatches on the SSN bus output. The SSN data stream depends on the order of how the patterns are applied on the tester.

This example extends the idea of the example in the section [“How to Write JTAG and Payload Procedures Separately”](#) on page 562 by writing the on-chip compare self-test. As described previously, the on-chip compare self-test verifies the on-chip compare logic in the SSH. The on-chip compare self-test should be applied before any payload patterns when you have enabled on-chip compare.

**Figure 9-22. SSN Pattern Files With On-Chip Compare Self-Test**



The on-chip compare self test uses the SSN bus for the following:

- To deliver the stimulus used to test the on-chip compare hardware for stuck-at faults.
- To unload the status of the hardware test.

Unloaded data is compared on the bus output.

## How To Write Streaming-Through-IJTAG Patterns

Streaming-Through-IJTAG patterns are SSN patterns delivered to the active SSHs through TDI and measured on TDO with TCK as the synchronous clock source.

---

**Note**

 Streaming-Through-IJTAG patterns can only be applied when you have selected the streaming interface using the [set\\_ssn\\_options](#) command.

---

While Streaming-Through-IJTAG patterns are being applied, the TAP finite-state machine is put into the Shift-DR state while data is being delivered to the network through TDI. The SSH functional behavior during Streaming-Through-IJTAG is the same as when you use the parallel bus. The only difference is the delivery of streaming scan data to the network as a single bit through TDI.

---

**Note**

 Any SSN pattern created using the parallel bus can be retargeted through the top-level TAP controller and converted to a Streaming-Through-IJTAG pattern.

---

---

**Note**

 Writing the on-chip compare hardware self-test pattern is not compatible with Streaming-Through-IJTAG mode.

---

### Example 9-7. Writing Streaming-Through-IJTAG Patterns

```
write_patterns test_setup.stil -test_setup only -stil -replace

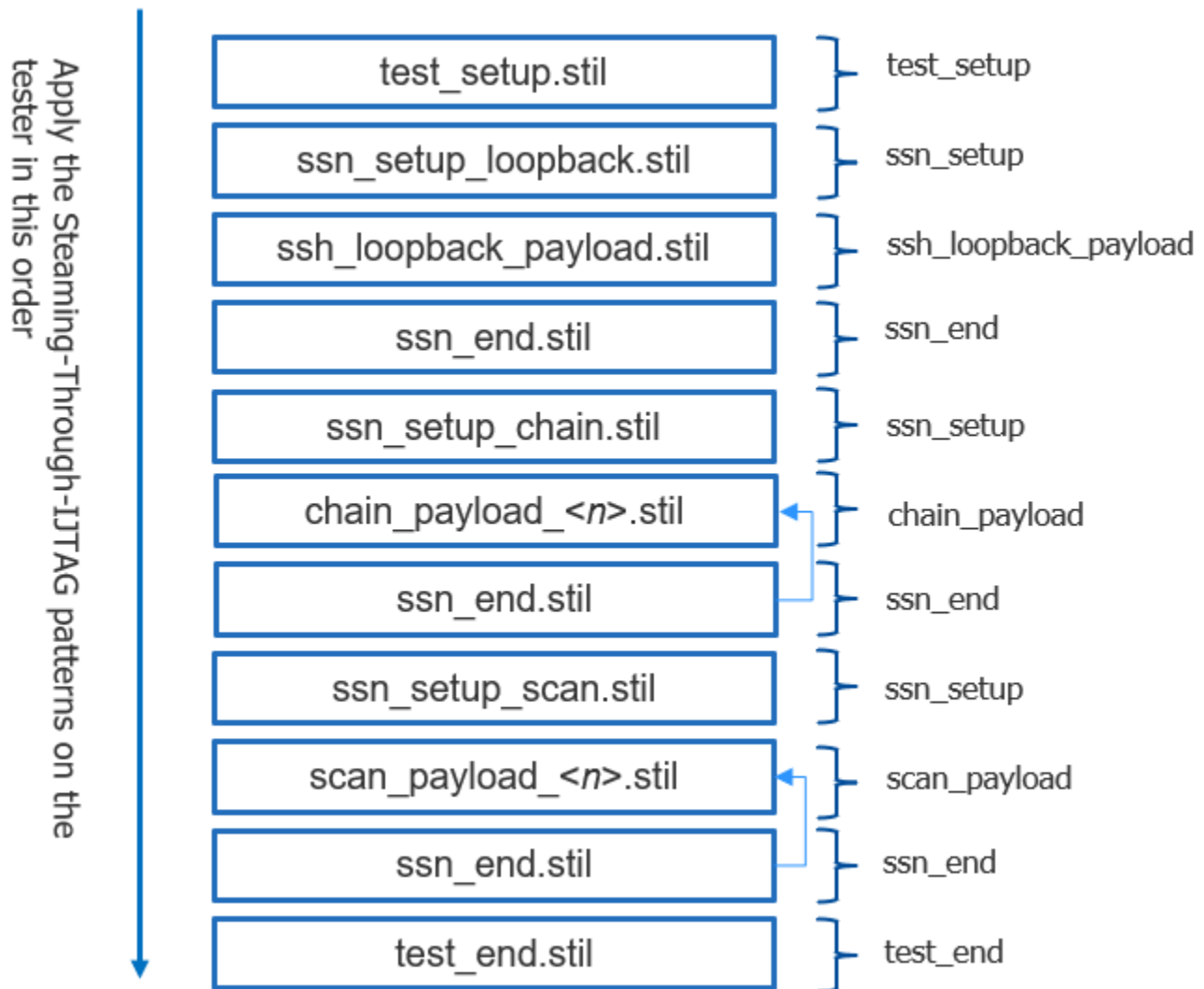
write_patterns ssn_setup_loopback.stil -ssn_setup only \
  -pattern_set ssh_loopback -stil -replace
write_patterns ssh_loopback_payload.stil -test_payload \
  -pattern_set ssh_loopback -stil -replace
write_patterns ssn_end.stil -ssn_setup off -ssn_end only -stil -replace

write_patterns ssn_setup_chain.stil -ssn_setup only -pattern_set chain \
  -stil -replace
write_patterns chain_payload.stil -test_payload -pattern_set chain -stil \
  -replace
write_patterns ssn_end.stil -ssn_setup off -ssn_end only -stil -replace

write_patterns ssn_setup_scan.stil -ssn_setup only -pattern_set scan \
  -stil -replace
write_patterns scan_payload.stil -test_payload -max_loads 500 \
  -pattern_set scan -stil -replace
write_patterns ssn_end.stil -ssn_setup off -ssn_end only -stil -replace

write_patterns test_end.stil -test_end only -stil -replace
```

Figure 9-23. Tester Application Order for Streaming-Through-IJTAG



## SSN Debug on the Tester

Use a combination of patterns to help narrow down the source of the failure when SSN patterns fail on the tester. It is important to apply SSN patterns to the tester in the correct order.

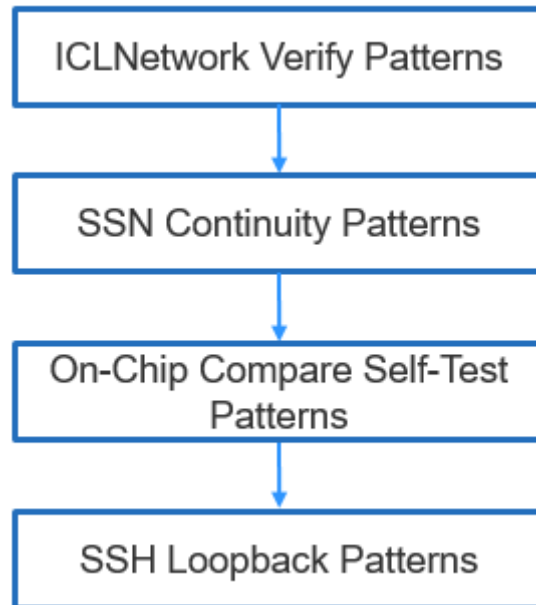
During silicon debug, you can reuse the verification patterns used during insertion and integration of the SSN. Use the following patterns to narrow the scope of the potential source of the failure when applied in the correct order, incrementally verifying the functional behavior of the SSN:

- **ICLNetwork Verify Patterns** — Confirms the IJTAG registers of the SSN can be accessed.
- **Continuity Patterns** — Confirms the parallel bus is connected to each SSN node and each branch of SSN multiplexer is accessible from bus\_in to bus\_out.

- **SSH Loopback Pattern** — Confirms the network can reliably deliver packetized data to the active SSHs without involving the EDT/scan chains.
- **On-Chip Compare Self-Test Pattern** — Confirms there are no stuck-at faults in the on-chip compare logic, including the sticky bit status registers.

The examples in the following sections show how to create and apply these patterns on the tester. This flow diagram indicates the order in which to apply them and incrementally verify the functional behavior of the SSN:

**Figure 9-24. Order To Apply Patterns for SSN Pattern Failure Debugging**



## ICLNetwork Verify Patterns

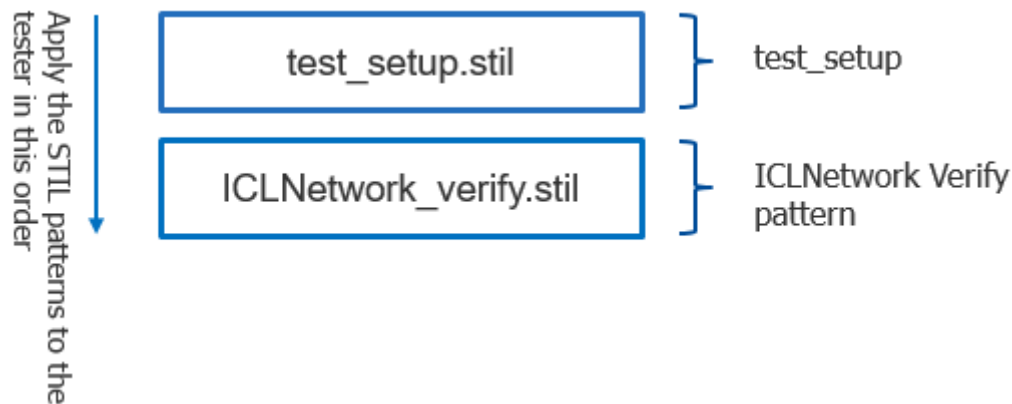
The following example shows the commands to write ICLNetwork verify patterns that you can apply on the tester. These patterns verify that the IJTAG registers in the SSN are accessible and that they can be read and written without any errors. Passing ICLNetwork verify patterns indicates the SSN network is properly configured, including the SSH nodes that are programmed as part of the `ssn_setup` procedure. To see the order of application for ICLNetwork verify patterns, refer to [Figure 9-25](#).

### Example 9-8. Writing ICLNetwork Verify Patterns in STIL Format

```

set_context patterns -ijtag
...
open_pattern_set icl_network
  create_icl_verification_patterns
close_pattern_set
write_patterns ICLNetwork_verify.stil -stil -replace
  
```

**Figure 9-25. Order To Apply ICLNetwork Verify Patterns to the Tester**



## Continuity Patterns

The following example shows the commands to write SSN continuity patterns for application on the tester. These patterns verify that the parallel bus is connected to each SSN node without any bit swizzles or opens and that each branch of an SSN multiplexer is accessible from bus\_data\_in to bus\_data\_out. A passing SSN continuity pattern indicates the parallel bus in the design has no opens or gaps. To see the order of application for SSN continuity patterns, refer to [Figure 9-26](#).

---

### Note

 Each of the SSN multiplexers along the parallel bus is configured individually with an IJTAG iProc followed by an iCall to that procedure. Refer to the script in the section [“Second DFT Insertion Pass: Inserting Top-Level EDT, OCC, and SSN”](#) on page 511 for a working example of how the iCall reconfigures the SSN multiplexers.

---

---

### Note

 The tool does not support SVF output for SSN continuity patterns.

---

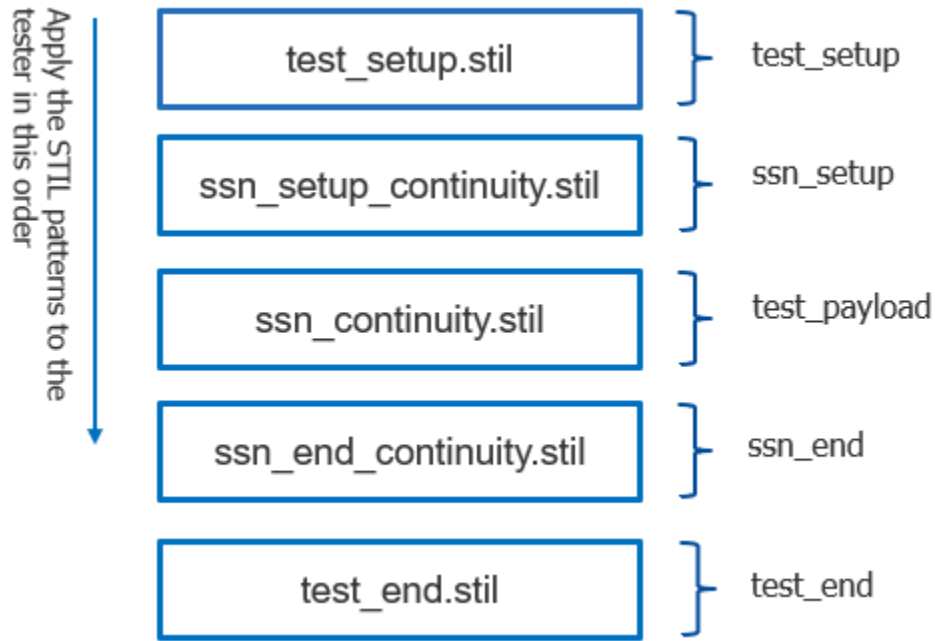
## Example 9-9. Writing SSN Continuity Patterns in STIL Format

```
set_context patterns -ijtag
...
open_pattern_set ssn_continuity -usage ssn
create_ssn_continuity_patterns
close_pattern_set
write_patterns test_setup.stil -test_setup only -stil -replace
write_patterns ssn_setup_continuity.stil -ssn_setup only \
    -pattern_set ssn_continuity -stil -replace
write_patterns ssn_continuity.stil -test_payload -pattern_set continuity \
    -stil -replace
write_patterns ssn_end_continuity.stil -ssn_end only \
    -pattern_set continuity -stil -replace
write_patterns test_end.stil -test_end only -stil -replace
```



**Note**

 The `ssn_end` procedure is an empty procedure when it is written and the pattern set is `ssn_continuity`.

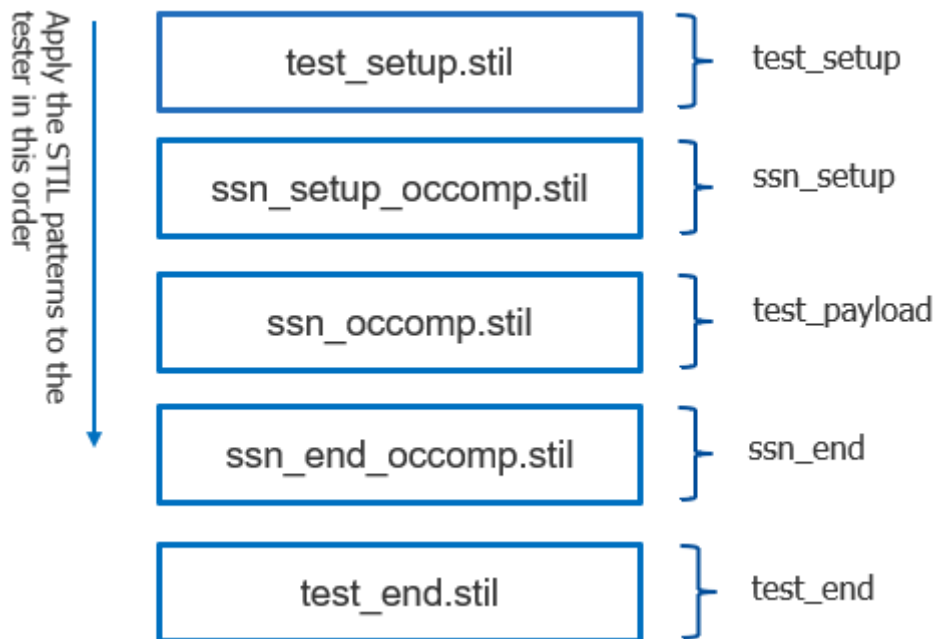
**Figure 9-26. Order To Apply SSN Continuity Patterns to the Tester****On-Chip Compare Self-Test Patterns**

The SSH on-chip compare self-test is only necessary when one or more SSHs are equipped with on-chip compare hardware. The following example shows the commands to write the SSH on-chip compare self-test patterns for application on the tester. A passing on-chip compare self-test indicates the on-chip compare hardware and sticky bit status registers are free of any stuck-at faults. To see the order of application for on-chip compare self-test patterns, refer to [Figure 9-27](#).

**Example 9-10. Writing SSH On-Chip Compare Self-Test Patterns**

```
write_patterns test_setup.stil -test_setup only -stil -replace
write_patterns ssn_setup_occomp.stil -ssn_setup only \
  -pattern_set ssh_on_chip_compare -stil -replace
write_patterns ssn_occomp.stil -test_payload \
  -pattern_set ssh_on_chip_compare -stil -replace
write_patterns ssn_end_occomp.stil -ssn_end only \
  -pattern_set ssh_on_chip_compare -stil -replace
write_patterns test_end.stil -test_end only -stil -replace
```

**Figure 9-27. Order To Apply SSN On-Chip Compare Self-Test Patterns**



## SSH Loopback Patterns

The SSH loopback pattern is the culmination of the first two test patterns: ICLNetwork verify and SSN continuity patterns. Use this pattern with any configuration of active cores during ATPG and scan pattern retargeting. The following example shows commands to write the SSH loopback patterns for application on the tester.

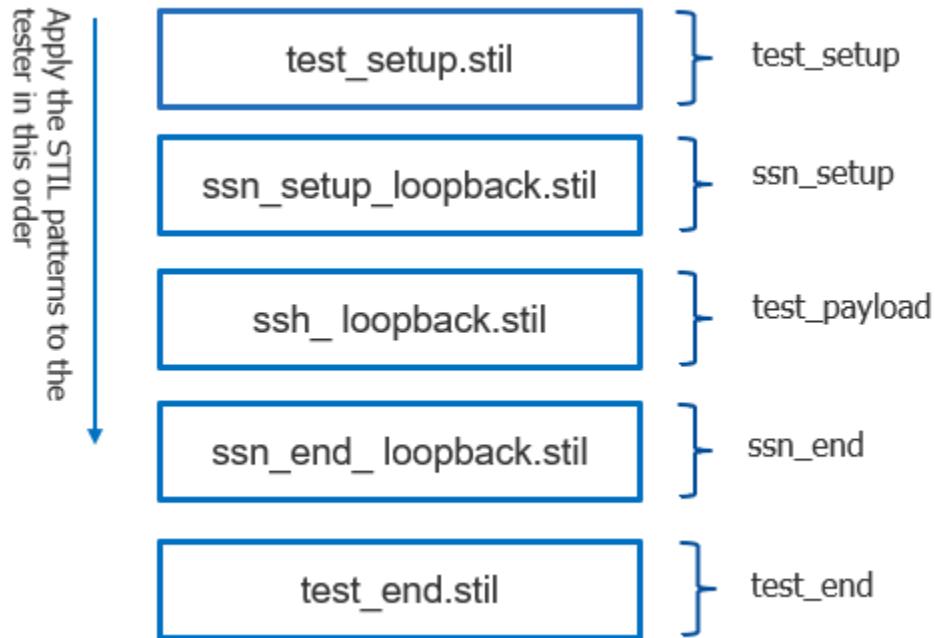
### Example 9-11. Writing SSH Loopback Patterns in STIL Format

```
write_patterns test_setup.stil -test_setup only -stil -replace
write_patterns ssn_setup_loopback.stil -ssn_setup only \
  -pattern_set ssh_loopback -stil -replace
write_patterns ssh_loopback.stil -test_payload -pattern_set ssh_loopback \
  -stil -replace
write_patterns ssn_end_loopback.stil -ssn_end only \
  -pattern_set ssh_loopback -stil -replace
write_patterns test_end.stil -test_end only -stil -replace
```

In this example, the SSH loopback patterns test the SSH without involving the EDT and scan chains. The SSH loopback pattern uses the ssn\_setup procedure to configure the SSH based on the payload of the active cores. During the SSH loopback pattern, the streaming interface (bus or IJTAG) delivers packetized data to each active SSH that is programmed to match the payload. The ssn\_end procedure is applied to the network after the SSH loopback pattern. When you combine SSH loopback patterns and payload patterns, you must place the SSH loopback patterns before the payload patterns, because the SSH loopback patterns confirm that the network can reliably deliver packetized data, as described previously. A passing SSH loopback pattern indicates the SSN can reliably deliver packetized data to the current configuration of

active cores. To see the order of application for SSH loopback patterns, refer to the following figure.

**Figure 9-28. Order To Apply SSH Loopback Patterns**

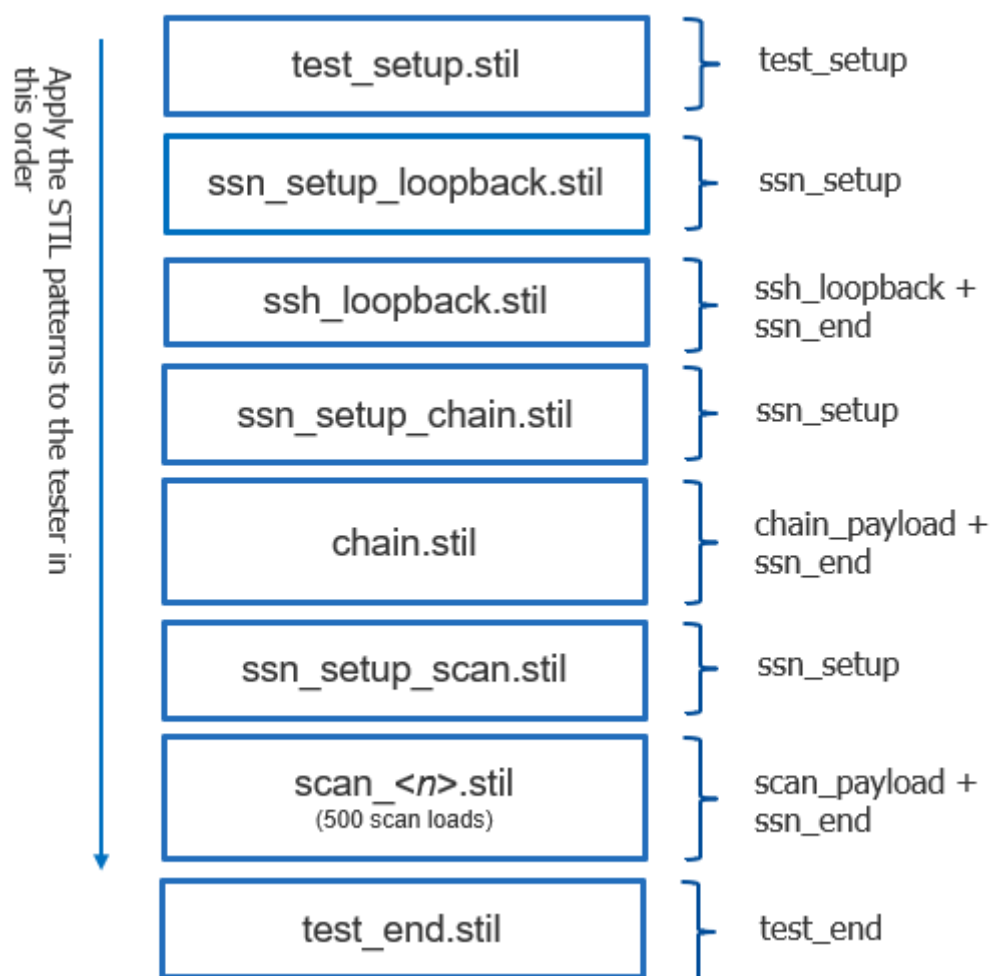


The following example shows the commands to write the chain and scan patterns with the loopback pattern for application on the tester. By extending the previous example to include the chain and scan patterns, you can further narrow the scope of failing patterns on silicon. To see the order of application for SSH loopback, chain, and scan patterns on the tester, refer to [Figure 9-29](#).

### Example 9-12. Writing SSH Loopback, Chain, and Scan Patterns to Individual Files

```
write_patterns test_setup.stil -test_setup only -stil -replace
write_patterns ssn_setup_loopback.stil -ssn_setup only \
  -pattern_set ssh_loopback -stil -replace
write_patterns ssh_loopback.stil -test_setup off -ssn_setup off \
  -ssn_end on -pattern_set ssh_loopback -stil -replace
write_patterns ssn_setup_chain.stil -ssn_setup only -pattern_set chain \
  -stil -replace
write_patterns chain.stil -test_setup off -ssn_setup off -ssn_end on \
  -pattern_set chain -stil -replace
write_patterns ssn_setup_scan.stil -ssn_setup only -pattern_set scan \
  -stil -replace
write_patterns scan.stil -test_setup off -ssn_setup off -ssn_end on \
  -max_loads 500 -pattern_set scan -stil -replace
write_patterns test_end.stil -test_end only -stil -replace
```

**Figure 9-29. Order To Apply SSH Loopback, Chain, and Scan Patterns**



## Signoff Patterns With SSN

This section describes the list of signoff patterns you should simulate at the block level, during top-level ATPG, and during scan retargeting. Signoff patterns are the minimum set of patterns you should simulate to verify your design with the DFT-inserted structures before releasing the design to layout. Use this signoff process to find potential problems in your design with a minimum set of patterns that provides a maximum amount of simulation coverage.

---

### Note



The process of creating signoff patterns is different from creating manufacturing patterns. For a description of manufacturing patterns with SSN, refer to the section “[Manufacturing Patterns With SSN](#)” on page 558.

---

The following sections describe the unique behavior of each type of signoff pattern when written with SSN present. The signoff pattern guidelines in these sections minimizes the risk of problems occurring in your post-layout netlists. For each test, the section does the following:

- Identifies the synchronous source of the SSN.
- Explains the purpose of the test.
- Describes the behavior of the SSH when writing patterns at the block level, during top-level ATPG, and during pattern retargeting.

You should create signoff patterns without any ATPG pattern limits (for example, by specifying “[set\\_atpg\\_limits](#) -pattern\_count off”). The methods for writing the patterns are described in the following sections. For more examples of writing patterns, refer to “[Tessent SSN Workflows](#)” on page 478. Create patterns using both the stuck-at and transition fault models. Simulate the patterns in the following order, which incrementally verifies the SSN network:

1. Parallel Scan
2. SSH Loopback
3. Serial Chain
4. Serial Scan

Each pattern uses the previously validated step. You should create and simulate these patterns until they are error-free before signoff, regardless of whether you choose parallel bus or JTAG streaming for SSN.

---

### Note



You should create signoff patterns only after you have successfully simulated the ICL Network Verify and SSN Continuity patterns without any errors.

---

**Note**

 Verification of the SSN at intermediate levels of hierarchy in your design is necessary prior to top-level ATPG or pattern retargeting when you have assembled the datapath by hand or using a script.

|                                              |            |
|----------------------------------------------|------------|
| <b>Block-Level Signoff Patterns .....</b>    | <b>578</b> |
| <b>Top-Level Signoff Patterns .....</b>      | <b>582</b> |
| <b>Signoff Pattern Quick Reference .....</b> | <b>585</b> |

## Block-Level Signoff Patterns

You should simulate the block-level patterns without error before signoff. Block-level patterns include parallel scan patterns, SSH loopback patterns, serial chain patterns, and serial scan patterns. Each of these pattern types has its own set of prerequisites.

### Parallel Scan Patterns

**Prerequisites:**

- Passing ICL verification patterns
- Passing SSN continuity patterns

During parallel pattern simulation with SSN, only scan pins of synchronous elements are active during scan shift and capture procedures, as in testing without SSN. These patterns' purpose is to validate the capture procedure for a large volume of scan patterns by simulating them in a reasonable amount of time.

While you simulate these patterns, the synchronous clock source to the SSN is not active, and the data delivered to the scan cells is not packetized data. The SSH is not actively transferring data between the SSN bus and the EDT/scan chains, and the scan signals generated by the SSH in the serial patterns are replaced by cut points that the tool automatically creates. The transitioning of the scan signals precisely models the protocol of the scan pattern. The `shift_capture_clock` and `capture clock` run at their default clock periods unless you use the `set_load_unload_timing_options` and `set_capture_timing_options` commands to specify otherwise.

The following is an example of writing SSN parallel patterns:

```
write_patterns parallel.v -verilog -pattern_set scan -replace
```

This should be the first pattern you simulate before signoff.

## SSH Loopback Patterns

### Prerequisites:

- Passing ICL verification patterns
- Passing SSN continuity patterns

During SSH loopback pattern simulation, the tool delivers packetized data to the SSH. These patterns' purpose is to verify that SSN can correctly process packetized data without error. The SSH loopback pattern tests exclude the EDT/scan chains from the test and include the SSH and other SSN nodes between bus\_in and bus\_out.

During simulation of this pattern, the SSN bus\_clock is the clock source, and packetized data synchronously streams over the SSN. The SSH extracts the correct bits from the packet based on the ssn\_setup procedure. The data loops back into the SSH, skipping the EDT/scan chains. After the data loops back into the SSH, it returns to the packet in the same time slot as the input data. As the packet synchronously streams through the network, the expected values are strobed on bus\_out. During simulation of this pattern, only the bus\_clock runs, and the scan signals from the SSH do not toggle and are constrained to zero.

The following is an example of writing SSH loopback patterns:

```
write_patterns ssh_loopback.v -verilog -serial -pattern_set ssh_loopback -replace
```

## Serial Chain Patterns

### Prerequisites:

- Passing SSH loopback patterns

During serial chain pattern simulation, the tool delivers packetized scan shift data to the EDT/scan chains through SSH. The purpose of these patterns is to verify that the scan chains can reliably shift scan data without error.

During simulation of these patterns, the SSN bus\_clock is the clock source, and packetized data synchronously streams over the SSN. The SSH extracts the correct number of bits from the packet and transfers them to the EDT/scan chains. The scan signals are synchronously created within the SSH and clock the scan circuit. When SSN is present, the tool suppresses the capture clock, regardless of whether you use the Tessent OCC or a third-party OCC. Without the capture cycle, the SSH holds the scan\_en high during chain tests. During unloading of the scan chains, the SSH puts the scan unload data back into the packet. When on-chip compare is off, the scan unload data is added back into the packet in the same time slots as the input data. When on-chip compare is enabled, the data is added back into the packet but does not overwrite the input data. It is added to the packet in a location determined by the number of status groups.

The shift\_capture\_clock runs at the default clock period unless you use the set\_load\_unload\_timing\_options command to specify otherwise.

Typically, the SSN bus\_clock runs at a slower frequency than specified with the set\_load\_unload\_timing\_options command at the block level. The bus clock period is governed by the packet size relative to the bus width. When the packet size is less than two bus\_clock cycles, the SSN bus\_clock is reduced to the shift\_capture\_clock frequency. To increase the bus\_clock frequency to the default or to the maximum frequency you specified using the set\_load\_unload\_timing\_options command, use the [SIM\\_SSN\\_MAXIMUM\\_BUS\\_SPEED](#) testbench parameter. When you write the testbench with this parameter set to one, the tool increases the size of the packet so that the bus\_clock runs at its maximum frequency in a Verilog testbench.

It can be difficult to debug mismatches in a serial SSN simulation, because the SSH obfuscates the EDT channels/scan chains. Furthermore, there may be any number of pipeline stages along the SSN bus, further complicating the analysis of the root cause to the failure. To debug serial SSN simulations, use the [SIM\\_PARALLEL\\_MONITOR](#) testbench parameter. When you set this parameter to one, the testbench monitors the load and unload values of each memory element and echoes the instance name to the log file for easier analysis of any failure.

For large designs, simulating all chain test patterns can be time-consuming and impractical. Use the “set\_chain\_test -type nomask” command to enable you to efficiently simulate the shift path of all your scan chains without testing the magic logic of the EDT decoder. Combine the “set\_chain\_test -type nomask” command with the “write\_patterns -end 0” command to write a single chain test pattern that tests the shift paths of all the scan chains. A single chain test pattern that simulates all scan chains is all you require.

The following example commands write a single serial chain test pattern that excludes testing any of the EDT decode logic:

```
set_chain_test -type nomask  
write_patterns chain_test.v -verilog -serial -end 0 -pattern_set chain -replace
```

## Serial Scan Patterns

### Prerequisites:

- Passing SSN serial chain patterns

During serial scan pattern simulation, the tool delivers packetized scan shift data to the EDT/scan chains through SSH. These patterns’ purpose is to verify that the scan chains can reliably shift and capture without error.

When simulating this pattern, the SSN network operates exactly as described in the preceding subsection, Serial Chain Patterns, with the addition that capture cycles are included in this pattern. The correct number of capture pulses comes through the OCC clock control bits. During stuck-at fault testing, the SSH synchronously creates the capture clock from the SSN bus\_clock. The SSH capture clock frequency comes from the default clock period unless you use the set\_capture\_timing\_options command to specify otherwise. During transition fault testing, the functional clock is the capture clock. This pattern is an SSN end-to-end test.



For large designs, simulating all serial scan patterns can be time-consuming and impractical. Use the “write\_patterns -end 0” command to write a single end-to-end scan test pattern that is sufficient to test the SSN along with the previously specified patterns.

You can use the SIM\_SSN\_MAXIMUM\_BUS\_SPEED testbench parameter, as described in the [Serial Chain Patterns](#) section, to simulate the serial scan patterns at the maximum SSN bus frequency.

You can use the SIM\_PARALLEL\_MONITOR testbench parameter, as described in the [Serial Chain Patterns](#) section, to debug the serial scan patterns.

The following example command writes a single serial scan test pattern:

```
write_patterns scan_test.v -verilog -serial -end 0 -pattern_set scan -replace
```

## Block-Level Signoff Pattern Summary

The following table summarizes the block-level patterns used for verification from integration to pattern creation of the SSN. You should run the pattern verification from least complex to most complex. When you follow this order, it verifies the SSN incrementally, making it easier to identify problems along the way.

Depending on your flow, you can perform verification of the SSN during integration either at the RTL or the gate level. For both cases, the ICLNetwork verification patterns also verify the [Streaming-Through-IJTAG Scan Data](#) path of the SSN. The entry point and path within the SSH is the same, regardless of whether you are accessing IJTAG registers in the SSH or streaming scan data through the IJTAG interface of the SSH.

**Table 9-3. Block-Level Verification Pattern Summary**

|                                            | Least complex<br>ICLNetwork<br>Verify<br>Patterns <sup>1</sup> | →<br>Continuity<br>Pattern | →<br>Scan Pattern<br>Parallel | →<br>Loopback<br>Pattern | Most complex<br>Scan Pattern<br>Serial |
|--------------------------------------------|----------------------------------------------------------------|----------------------------|-------------------------------|--------------------------|----------------------------------------|
| SSN integration <sup>2</sup>               | X                                                              | X                          |                               |                          |                                        |
| Post-synthesis<br>validation<br>(non-scan) | X                                                              | X                          |                               |                          |                                        |
| Core-level<br>ATPG pattern<br>validation   |                                                                |                            |                               | X                        | X (chain test)                         |
|                                            |                                                                |                            | X (scan test)                 |                          | X (scan test)                          |

1. Includes verification of the streaming-through-IJTAG SSH path

2. SSN integration at RTL, gate level, or both

## Top-Level Signoff Patterns

You should simulate the top-level patterns without error before signoff. Block-level patterns include parallel scan patterns, SSH loopback patterns, serial chain patterns, and serial scan patterns. Each of these pattern types has its own set of prerequisites.

As part of the signoff process, you should retarget cores on a per-module basis. Do not group different modules for retargeting during signoff. Grouping different modules for retargeting is a process performed for manufacturing.

### Parallel Scan Patterns

#### Prerequisites:

- Passing ICL verification patterns
- Passing SSN continuity patterns
- Passing child core scan patterns, as described in “[Block-Level Signoff Patterns](#)” on page 578
- Scan graybox has been created for all lower-level blocks

Top-level parallel scan patterns operate in two different situations:

- **Parallel pattern during top-level ATPG with child cores** — During ATPG, the top-level scan chains and the wrapper scan chains of each child core are active. This pattern tests the intra-domain capture between the top level and the child cores. The active SSHs in the parallel pattern are capture-aligned. You should write the parallel pattern without a pattern count limit.
- **Parallel pattern during pattern retargeting** — Normally, during scan pattern retargeting with SSN, each core transitions between shift and capture independently. However, a Verilog testbench cannot accurately model this behavior for the cores. Therefore, in this testbench the active cores appear to be capture-aligned, although this is not the way that the implemented design behaves. Use this pattern to verify the clocking during pattern retargeting.

For a description of parallel scan pattern behavior and an example of how to write the pattern, refer to the section “[Parallel Scan Patterns](#)” in the topic “[Block-Level Signoff Patterns](#)” on page 578.

### SSH Loopback Patterns

#### Prerequisites:

- Passing ICL verification patterns
- Passing SSN continuity patterns

During top-level ATPG when child cores are present, the top-level SSH loopback pattern delivers packetized data to the top-level SSH and the SSH of each child core. The cores are in external mode. During simulation of this pattern, the tool forces the scan signals at the edges of the SSHs low. The packet contains data for the top-level SSH and the SSH for each child core. This pattern is short and simulates quickly. It verifies that each SSH can precisely extract and replace data in the packet.

You cannot retarget SSH loopback patterns. During pattern retargeting, the packet used in the SSH loopback test is created from the scan data for each core whose patterns is being retargeted. In this case, the purpose of the SSH loopback pattern is to test the SSN with a representative packet to be used during pattern retargeting.

For a description of SSH loopback pattern behavior and an example of how to write the pattern, refer to the section “[SSH Loopback Patterns](#)” in the topic “[Block-Level Signoff Patterns](#)” on page 578.

## Serial Chain Patterns

### Prerequisites:

- Passing SSH loopback patterns

Top-level serial chain patterns operate in two different situations:

- **Serial chain pattern during top-level ATPG with child cores** — During ATPG, this pattern delivers packetized scan shift data to the top-level SSH and to the SSH of each child core. The child cores are in external mode, and the local SSH to those child cores shifts their wrapper chains. During simulation of this pattern, the capture clock is suppressed and the top-level SSH and child core SSHs are capture-aligned.
- **Serial chain pattern during pattern retargeting** — During simulation of this pattern, the tool creates the packet from each core whose patterns are being retargeted. Each core operates independently of the others, receiving a different number of bits per packet based on data throttling.

For a description of serial chain pattern behavior and an example of how to write the pattern, refer to the section “[Serial Chain Patterns](#)” in the topic “[Block-Level Signoff Patterns](#)” on page 578.

## Serial Scan Patterns

### Prerequisites:

- Passing serial chain patterns

Top-level serial chain patterns operate in two different situations:

- **Serial scan pattern during top-level ATPG with child cores** — During ATPG, this pattern delivers packetized scan data to the top-level SSH and to the SSH of each child core. The child cores are in external mode, and the local SSH to those child cores shifts their wrapper chains. During simulation of this pattern, the scan chains shift and capture, and the top-level SSH and child core SSHs are capture-aligned.
- **Serial scan pattern during pattern retargeting** — During simulation of this pattern, the tool creates the packet from each core whose patterns are being retargeted. Each core operates independently of the others, receiving a different number of bits per packet based on data throttling.

For a description of serial chain pattern behavior and an example of how to write the pattern, refer to the section “[Serial Scan Patterns](#)” in the topic “[Block-Level Signoff Patterns](#)” on page 578.

## Top-Level Signoff Pattern Summary

The following table summarizes the top-level patterns used for verification from integration to pattern creation of the SSN. You should run the pattern verification from least complex to most complex. If your design has more than two levels of hierarchy (for example, more than block and top), you should verify the SSN at all intermediate levels. This ensures that the datapath is properly connected. When you follow this order, it verifies the SSN incrementally, making it easier to identify problems along the datapath.

Depending on your flow, you can perform verification of the SSN during integration either at the RTL or the gate level. For both cases, the ICLNetwork verification patterns also verify the [Streaming-Through-IJTAG Scan Data](#) path of the SSN. The entry point and path within the SSH is the same, regardless of whether you are accessing IJTAG registers in the SSH or streaming scan data through the IJTAG interface of the SSH.

**Table 9-4. Top-Level Verification Pattern Summary**

|                                            | Least complex<br>ICLNetwork<br>Verify<br>Patterns <sup>1</sup> | →<br>Continuity<br>Pattern | →<br>Scan Pattern<br>Parallel | →<br>Loopback<br>Pattern | Most complex<br>Scan Pattern<br>Serial |
|--------------------------------------------|----------------------------------------------------------------|----------------------------|-------------------------------|--------------------------|----------------------------------------|
| SSN integration <sup>2</sup>               | X                                                              | X                          |                               |                          |                                        |
| Post-synthesis<br>validation<br>(non-scan) | X                                                              | X                          |                               |                          |                                        |
| Top-level ATPG<br>pattern<br>validation    |                                                                |                            |                               | X                        | X (chain test)                         |
|                                            |                                                                |                            | X (scan test)                 |                          | X (scan test)                          |

**Table 9-4. Top-Level Verification Pattern Summary (cont.)**

|                                      | Least complex<br><b>ICLNetwork<br/>Verify<br/>Patterns<sup>1</sup></b> | →<br><b>Continuity<br/>Pattern</b> | →<br><b>Scan Pattern<br/>Parallel</b> | →<br><b>Loopback<br/>Pattern</b> | Most complex<br><b>Scan Pattern<br/>Serial</b> |
|--------------------------------------|------------------------------------------------------------------------|------------------------------------|---------------------------------------|----------------------------------|------------------------------------------------|
| Pattern<br>retargeting<br>validation |                                                                        |                                    |                                       | X                                | X (chain test)                                 |
|                                      |                                                                        |                                    | X (scan test) <sup>3</sup>            |                                  | X (scan test)                                  |

1. Includes verification of streaming-through-IJTAG

2. SSN integration at RTL, gate level, or both

3. SSN retargeting parallel testbench only verifies clocking during scan pattern retargeting

## Signoff Pattern Quick Reference

The table in this topic summarizes the signoff patterns at the block and top levels. You should simulate these patterns without error before releasing the design for layout.

**Table 9-5. Signoff Pattern Quick Reference**

| Pattern Type      | Pattern Limits | Description                                                                                                                                                                                                           |
|-------------------|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ICLNetwork verify | None           | Verifies ICL instruments (tool and user) are readable and writable along the IJTAG serial path.<br>Verifies the full scope of the IJTAG network, including streaming-through-IJTAG path.<br>Runs at TCK period.       |
| Continuity        | None           | Verifies SSN bus integrity between bus data_in and data_out.<br>Runs at bus_clock period.                                                                                                                             |
| Parallel scan     | None           | Verifies each scan register can reliably capture.<br>Capture clock and scan signals are cut points on SSH that testbench pulses.<br>For retargeted scan patterns, can verify top-level clocking to lower-level cores. |
| SSH loopback      | None           | Verifies SSN network can reliably deliver packetized data (excluding path to EDT).<br>All SSN nodes are programmed with ssn_setup procedure as though full pattern is to be run.<br>Runs at bus_clock period.         |

**Table 9-5. Signoff Pattern Quick Reference (cont.)**

| <b>Pattern Type</b> | <b>Pattern Limits</b> | <b>Description</b>                                                                                                                                                                       |
|---------------------|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Serial chain        | 1                     | Delivers packetized data to all SSH and verifies all chains shift without error. Saves only nonmasking patterns.<br>bus_clock in the SSH generates scan signals.<br>Runs off bus_clock.  |
| Serial scan         | 1                     | Delivers packetized data to all SSH and verifies all chains shift and capture without error. SSN end-to-end test.<br>bus_clock in the SSH generates scan signals.<br>Runs off bus_clock. |

# SSN SDC Constraints in the Design Flow

---

The following topics describe how to work with SDC constraints when SSN is present in the design.

- Overview of SSN-Related SDC Procs and Constraints . . . . . 588**
  - Changes to SDC Procs for SSN Timing . . . . . 589
  - SSN/SDC Proc Usage . . . . . 591
- SSN/SSH SDC Constraint Descriptions. . . . . 597**
  - SSN Bus Clock and SSN Bus Port Timing Constraints . . . . . 597
  - SSN Bus Data Port Delays . . . . . 598
  - SSN Datapath Timing Constraints . . . . . 600
  - SSN FIFO Timing Constraints . . . . . 603
  - SSN ScanHost Timing Constraints and Clock Tree Synthesis. . . . . 606

Note - Viewing PDF files within a web browser causes some links not to function. Use HTML for full navigation.

## Overview of SSN-Related SDC Procs and Constraints

SSN requires updates to the logictest-related constraint procs in the SDC output file that the `extract_sdc` command produces.

The following topics describe these updates to the SDC file procs and how to use them in synthesis, layout, hierarchical signoff static timing analysis (STA), and full-chip STA in the SSN context.

SSN SDC constraints work in conjunction with many of the Tcl procs dedicated to Tessent-logictest SDC constraints, such as those that define the `shift`, `slow_capture`, and `fast_capture` STA modes. This is possible because SSN is simply a faster and more flexible scan implementation than traditional implementations.

The generated constraints added for SSN perform the following functions:

- Declare the SSN bus clocks and define the timing of the SSN bus ports.
- Relax paths across node multipliers and node dividers with MCP constraints.
- Accurately time the SSH scan interface circuits, which includes creating the scan clocks and adding timing exceptions for the scan control signals and serial paths.
- Add timing exceptions to and from subphysical block boundaries.
- Provide a way to time the SSN/SSH logic of the lower-level physical blocks.
- Properly time SSN nodes that can only stream through IJTAG, with no SSN bus clock.
- Provide ways to disable the SSN logic if needed, and prevent other mode clocks, such as `ijtag_tck`, from propagating to the scan flops.

For more information about constraints, refer to the section “[SSN/SSH SDC Constraint Descriptions](#)” on page 597.

The chapter “[Timing Constraints \(SDC\)](#)” on page 969 describes the flow usage of the standard logictest Tcl procs. The addition of SSN constraints adds only minor changes to the usage of these procs. You still use the following design flow:

1. In synthesis, invoke the `non_modal` SDC proc constraints.
2. In layouts, sequentially invoke `non_modal` SDC constraints followed by some modal logictest SDC procs.
3. In post-layout STA, time your different logictest modes one at a time, calling the provided “`*ltest_modal*`” procs for each mode.

The following sections provide more information about SDC procs with SSN:

|                                                  |            |
|--------------------------------------------------|------------|
| <b>Changes to SDC Procs for SSN Timing</b> ..... | <b>589</b> |
| <b>SSN/SDC Proc Usage</b> .....                  | <b>591</b> |



## Changes to SDC Procs for SSN Timing

The presence of SSN adds new constraint procs to the SDC file and modifies some other preexisting logictest-related SDC procs.

For a detailed description of the procs that are modified for SSN, refer to their original descriptions in the section “[LOGICTEST Instruments](#)” on page 993.

---

### Note



When you specify the SSH [scan\\_signals\\_bypass](#), all modified logictest proc constraints cover both the SSN mode and the bypass mode timing paths at the same time, so there is no need to separate your bypass-mode STA from your SSN-mode STA runs.

---

## Summary of New and Modified Tcl Procs for SSN Timing Support

```
<prefix> ::= tesseract_set
```

### New SSN-Specific Procs

#### **<prefix>\_ltest\_ssn**

- Applies set\_input/output delay constraints on SSN bus ports
- Creates clock groups between SSN clocks, functional clocks, and other DFT clocks
- Adds SSH constraints
- Adds MCP constraints for frequency multiplier and divider nodes
- Accepts “mode” argument with values “shift” or “non\_modal”

#### **set\_load\_unload\_timing\_options**

- The SDC file equivalent of the command of the same name in Tessent Shell  
For usage information, refer to the [set\\_load\\_unload\\_timing\\_options](#) command reference page.
- Sets global Tcl timing variable values, which are used in SSN constraints

### Modified Preexisting Procs

#### **<prefix>\_ltest\_create\_clocks**

- Creates SSN bus clocks on top-level ports and generated clocks within the SSH logic

#### **<prefix>\_ltest\_non\_modal**

- Calls *<prefix>\_ltest\_ssn* with the argument “non\_modal”

#### **<prefix>\_ltest\_modal\_shift**

- Calls *<prefix>\_ltest\_ssn* with the argument “shift”

### **<prefix>\_ltest\_modal\_edt\_slow\_capture**

- Calls <prefix>\_ltest\_ssn with the argument “non\_modal”
- Conditionally calls the <prefix>\_ltest\_relax\_xdomain\_paths proc, which does the following:
  - Creates one slow generated clock at the slow\_clock input pin of each OCC.
  - Uses these per-OCC clocks in set\_multicycle\_path commands to relax the hold of cross-domain capture paths in slow capture mode. This enables checking the timing of the scan\_enable signal, along with some other paths within the OCCs, while avoiding false hold timing violations on interdomain paths.
  - Reads a global Tcl dictionary variable that contains per-OCC generated clock target pins and identifies the SSH instance in the fan-in for these pins. You can create this dictionary yourself or extend an existing one if you use custom OCC module instances.

### **<prefix>\_ltest\_modal\_edt\_fast\_capture**

- Forces the SSH scan\_en output signal inactive

### **<prefix>\_ltest\_disable**

- Blocks ts\_tck\_tck from propagating to SSH and scannable logic

---

#### **Note**

 This proc still permits ts\_tck\_tck to propagate to the SSH logic so it can properly time the serial scan path to and from the IJTAG network. It uses the assumption that timing paths between the SSH IJTAG registers and the SSH logic always meet a single-cycle path of ts\_tck\_tck, with no excessive stress to the quality of results (QoR) for the layout. Clock tree synthesis can balance both branches of ts\_tck\_tck within the SSH logic.

---

### **New Procs for Chip-Level SSN STA**

These are global chip-level versions of the procs in the previous subsection, which apply the same constraints over all SSN node instances across sub-physical block hierarchies.

- <prefix>\_ltest\_ssn\_with\_sub\_PBs
- <prefix>\_ltest\_create\_clocks\_with\_sub\_PBs
- <prefix>\_ltest\_modal\_shift\_with\_sub\_PBs
- <prefix>\_ltest\_modal\_edt\_slow\_capture\_with\_sub\_PBs
- <prefix>\_ltest\_modal\_edt\_fast\_capture\_with\_sub\_PBs

## SSN/SDC Proc Usage

Use SSN-related SDC procs during preparation for synthesis, layout, and STA (both pre- and post-layout). SSN/SSH implementation has a few important points in which it differs from implementation without SSN.

Much of this material is covered in greater detail in some portions of the chapter “[Timing Constraints \(SDC\)](#)” on page 969. However, the following sections describe some details that are specific to SSN/SSH implementations.

### SSN/SDC Constraints for Synthesis

Preparing a block with SSN for RTL-to-gate synthesis does not differ significantly from preparing a standard EDT-inserted block, as described in “[Timing Constraints \(SDC\)](#)” on page 969. That section contains example synthesis scripts for both Design Compiler and Cadence Encounter that can help you understand the SDC constraints for synthesis. The primary difference from the non-SSN flows for SSN is that you must specify your SSH scan interface global Tcl variables by calling the `set_load_unload_timing_options` proc instead of overriding them directly with a “set” command in your mainSDC script. Refer to the [set\\_load\\_unload\\_timing\\_options](#) command reference page for usage information for the command and Tcl proc.

The following summarizes the usage flow for preparing a block with SSN for synthesis:

1. Load your design.
2. Set your functional constraints.
3. Source the Siemens EDA *<design>.sdc* file.
4. Call the `tessent_default_variables` proc.
5. Redefine the global Tessent variables to meet your needs.
6. Call the “`set_load_unload_timing_options -usage ssn`” proc with the proper option values.

This redefines the default SSH global Tcl timing variables.

7. Call the `<prefix>_non_modal` proc.

This adds SDC for all of your Tessent DFT, including SSN.

8. Add synthesis control commands and run synthesis according to the process in the “[Timing Constraints \(SDC\)](#)” chapter.

#### Note

 The `<prefix>_non_modal` proc declares all SSN bus clocks and SSH-generated scan clocks and propagates them alongside your functional clocks to all your scannable domains. Such clocks may expose some invalid `shift_capture_clock` (SCC)-speed capture paths between otherwise asynchronous functional domains. Because synthesis usually runs with ideal clocks, these bogus timing paths are unlikely to fail hold timing; however, it is possible that they could still fail setup timing if your specified scan frequency is too high. Such a proc is also unusable as-is in layout without major modification, as described in the subsection “[Single-Mode vs Dual-Mode Constraining in Synthesis/Layout](#)” in the section “[LOGICTEST Instruments](#)” on page 993.

---

## SSN/SDC Constraints for Layout

As described in the section “[Running Layout with Tessent Shell DFT](#)” on page 977, the recommendation for layout with Tessent is to load multiple timing mode constraints. This section also shows the preparation steps required prior to loading the constraints themselves.

If your design includes at least one SSH controller, your clock tree synthesis (CTS) step must include the CTS exception declarations listed by the `tessent_get_cts_skew_groups_dict` proc inside your generated SDC file. These points are located at the base of the SSH `edt_clock` and `shift_capture_clock` generated clock source pins. These prevent the potentially problematic balancing of the SSN datapath clock network with your potentially very large scan clock network.

The following sections describe how the presence of SSN affects these SDC constraints.

### Mode 1: All Tessent DFT Logic Except logictest

**proc:** `<prefix>_non_modal off`

Loads non-modal constraints for all Tessent DFT controllers but turns off the logictest DFT logic, including the ScanHost-generated scan clocks. The maximum-frequency SSN bus clock is still defined and propagated to all SSN datapath nodes. In this mode, some constraints are needed to prevent `ts_tck_tck` from propagating to the scannable domains through the SSH Streaming-through-IJTAG and “capture with tck” logic. For more details, refer to the subsection “[<ltest\\_prefix>\\_non\\_modal](#)” on page 1003.

### Mode 2: SSN Bus and Modal Shift

**proc:** `<prefix>_modal_shift`

Forces the SSH and SSH bypass `scan_enable` signals to be tied active and covers all SSN bus and SSH scan interface timing paths. If the SSH supports bypass mode, both bypass and SSH modes are covered at once.

### Mode 3: SSN Bus and Capture With Slow Clock

**proc:** `<prefix>_modal_edt_slow_capture`

Adds the same SSN constraints as the `<prefix>_modal_shift` proc but leaves the SSH `scan_en` signals toggling in order to time them using specifications from the [set\\_load\\_unload\\_timing\\_options](#) command. Optionally relaxes SCC intra- and cross-domain paths to avoid false violations. Its main function is to cover the `scan_en` signal timing path.

You can merge the preceding Modes 2 and 3 into a single stage to save on total layout runtime and complexity. You can do this only if your design meets the following criteria:

- Clock tree synthesis (CTS) has balanced the entire SSH `shift_capture_clock` source fanout, along with the same SSH `edt_clock` source fanout, so that there are no hold issues on same-edge cross-domain paths.
- All cross-domain paths can still meet setup timing in one SCC cycle. Capture clock waveforms are assumed to be the same as those of the shift clock.

In this case, you only need to run the `<prefix>_modal_edt_slow_capture` proc, which then covers both the shift paths and capture paths in addition to properly timing the `scan_enable` signal with the specified number of setup and hold cycles.

For designs with SSN, the `<prefix>_ltest_modal_edt_slow_capture` proc includes the possibility of defining one set of generated clocks per OCC in the fanout of each SSH. This results in relaxed hold for all capture between different OCCs. This also relaxes paths between SIBs or Bscan to or from other OCC domains. Turn on this feature by defining the following global Tcl variable in your top-level calling script:

```
set tessent_relax_xdomain_capture_paths 1
```

Setting this variable means the tool supports hardened OCCs and all-design-level STA. Each generated clock is defined at the OCC `slow_clock` input pin, to permit a possible `shift_capture_clock` from the parent level to go through the `fast_clock` pin in multi-level designs. If the OCC persistent `slow_clock` buffer is visible in the netlist, the multicycle path constraint is defined on its input pin.

You can modify each generated clock OCC pin by editing a global OCC dictionary, but support for full custom OCC is not available.

If you do not define this variable, you can also choose from the following options:

- Not relaxing any path—this means closing stuck-at slow capture timing, for setup and hold, across those generated clocks coming from the OCC
- (default) Applying a global same-edge multicycle path -hold to the SSH `shift_capture_clocks`

**Limitation for bypass mode constraints:** When the SSH bypass is present, the bypass mode `scan_en` and `edt_update` input ports have no MCP declaration, unlike the SSH local `scan_en` signal sources, which MCP constraints relax based on timing specifications from the

set\_load\_unload\_timing\_options command. For more details, refer to the section “[SSN/SSH SDC Constraint Descriptions](#)” on page 597.

Furthermore, the bypass mode test clock frequency is assumed to be the same as the SSH test clock frequency, which is unlikely to be the case. If you intend to run the SSH-bypass scan at a slower frequency than the SSH-based scan, you must override the SSH-bypass test\_clock definition in your main script after calling the procs listed previously. You may also need to add your own MCP declarations for both scan\_en and edt\_update top-level ports, as in the following example:

```
<prefix>_modal_edt_slow_capture
create_clock -period <bypass test_clock period> \
  [get_ports test_clock port]
Set_multicycle_path 2 -setup -from [get_ports <scan_enable port>]
Set_multicycle_path 2 -hold -from [get_ports <scan_enable port>]
Set_multicycle_path 2 -setup -from [get_ports <edt_update port>]
Set_multicycle_path 2 -hold -from [get_ports <edt_update port>]
```

## SSN/SDC Constraints for STA Signoff on the Current Design

The Tessent logictest DFT STA Signoff flow is described in the sections “[Checking Your Functional Logic Alone](#)” on page 983 and “[LOGICTEST Instruments](#)” on page 993. The modal SDC procs these sections describe also cover both the fast SSN bus logic and the SSH scan interface logic. For more details on the SSN constraints, refer to the section “[SSN/SSH SDC Constraint Descriptions](#)” on page 597. Therefore, to perform STA signoff on your SSN DFT logic, you must run the same modes and the same procs as for the EDT/logictest logic; that is, the following:

- **Mode shift STA** — Call proc `<prefix>_modal_shift`
- **Mode edt\_slow\_capture STA** — Call proc `<prefix>_modal_edt_slow_capture`
- **Mode edt\_fast\_capture STA** — Call proc `<prefix>_modal_edt_fast_capture`

---

### Note

 As with [SSN/SDC Constraints for Layout](#), if your design meets the configuration conditions described in the note in that section, you can run only the `<prefix>_modal_edt_slow_capture` proc to cover both the shift and edt\_slow\_capture modes simultaneously.

---

## SSN/SDC Constraints for Hierarchical or Whole-Chip STA

The hierarchical STA flow with SSN stays the same as that described in the section “[Hierarchical STA in Tessent](#)” on page 1011 for as long as you use extracted timing models (ETM or .lib) to represent the timing of child physical blocks (PBs). If your flow requires full or

partial child PB instance netlists, you must replace your call to the usual logictest modal procs with their “\*\_with\_sub\_PBs” equivalent; that is, the following:

- `<prefix>_ltest_modal_shift_with_sub_PBs`
- `<prefix>_ltest_modal_edt_slow_capture_with_sub_PBs`
- `<prefix>_ltest_modal_edt_fast_capture_with_sub_PBs`

These procs include retargeted SDC constraints for all SSN logic inside all of your individual child PBs. Tessent-generated graybox models include all SSN logic. Without them, the parent SSN bus clock would enter through the block’s SSN bus pins and propagate directly to the block’s scannable logic. This, in turn, would overconstrain the child SSH-generated DFT signals. The following is an example of such a retargeted constraint:

```
array set tessent_ssh_mapping_with_sub_PBs {
  ssh2 top_rtl2_tessent_ssn_scan_host_1_inst
  ssh0 corea_i1/corea_rtl2_tessent_ssn_scan_host_1_inst
  ssh1 corea_i2/corea_rtl2_tessent_ssn_scan_host_1_inst
}
...
if {$mode eq "shift"} {
  set_case_analysis 1 [tessent_get_pins \
    $tessent_ssh_mapping_with_sub_PBs(ssh0)/tessent_persistent_cell_scan_en_buf/Y]
  set_case_analysis 1 [tessent_get_pins \
    $tessent_ssh_mapping_with_sub_PBs(ssh1)/tessent_persistent_cell_scan_en_buf/Y]
  set_case_analysis 1 [tessent_get_pins \
    $tessent_ssh_mapping_with_sub_PBs(ssh2)/tessent_persistent_cell_scan_en_buf/Y]
}
```

When you run top-level logictest STA with isolated lower-level PBs, your calling script must also call the proc `<prefix>_ltest_lower_pbs_external_mode`. This takes care of constraining the DFT signal and the OCC logic of the lower PBs. For example, to set the shift STA mode, you must run the following command sequence:

```
tessent_set_default_variables
set_load_unload_timing_options <list of timing options>
# (or source the file containing this command)
<prefix>_ltest_modal_shift_with_sub_PBs
<prefix>_lower_pbs_external_mode
```

### Note



Using \*\_with\_sub\_PBs procs is subject to the following limitations:

- These procs create retargeted SSN/SSH SDC constraints for all instances of SSN/SSH modules in the child PBs; however, all PBs end up using the same set of timing parameter variables (such as the bus clock, the shift clock frequency, and the number of scan\_en/edt\_update extra setup/hold cycles) as the level of the current design. Child PBs may have been designed and laid out with different parameter sets, with potentially

faster internal shift clocks than their parents. If this applies to any of your PBs, you must manually fix their associated constraints in your `extract_sdc` file.

- SSN/SSH constraints are declared for all PB instances, so you must load all of those instances to avoid errors in reading constraints.
  - You cannot set child PBs to internal ltest mode only using the provided procs in the SDC file. If you need to do this, you can copy your `<prefix>_lower_pbs_external_mode` proc, change its name, and manually change the settings of the `int_ltest_en` and `ext_ltest_en` signals to put all child PBs into internal mode.
-



## SSN/SSH SDC Constraint Descriptions

The following sections provide more detailed descriptions of SDC constraints for SSN and SSHs.

|                                                                       |            |
|-----------------------------------------------------------------------|------------|
| <b>SSN Bus Clock and SSN Bus Port Timing Constraints .....</b>        | <b>597</b> |
| <b>SSN Bus Data Port Delays.....</b>                                  | <b>598</b> |
| <b>SSN Datapath Timing Constraints.....</b>                           | <b>600</b> |
| <b>SSN FIFO Timing Constraints.....</b>                               | <b>603</b> |
| <b>SSN ScanHost Timing Constraints and Clock Tree Synthesis .....</b> | <b>606</b> |

## SSN Bus Clock and SSN Bus Port Timing Constraints

The SSN bus clock is declared using variables that contain a period in nanoseconds and a multiplier that aligns the units with your timing tool.

The following example shows the variables you use to declare the SSN bus clock:

```
global tessent_ssn_bus_clock_network_period
global time_unit_multiplier
set local_ssn_bus_clock_network_period \
    [expr $tessent_ssn_bus_clock_network_period * $time_unit_multiplier]
create_clock <port> -name tessent_ssn_bus_clock_network \
    -period $local_ssn_bus_clock_network_period -add
```

In this example, the `tessent_ssn_bus_clock_network_period` variable is always given in ns. The `time_unit_multiplier` variable converts the ns unit to the one used by the timing tool. For example, set a value of 1000 if the timing tool uses ps instead of ns. In the Tessent recommended flow, you set the bus clock network period variable indirectly with the following proc in your primary timing script:

```
set_load_unload_timing_options -usage ssn -ssn_bus_clock_period
```

The option values corresponds to the maximum possible SSN network speed across your entire design, even if your current level test patterns do not require that speed. Refer to the [set\\_load\\_unload\\_timing\\_options](#) command reference page for more information on using it in the SSN reference flow.

### Note

 If your SSN scan host implementation uses only Streaming-Through-IJTAG, your generated SDC file does not contain this declaration. For more information about Streaming-Through-IJTAG, refer to “[Streaming-Through-IJTAG Scan Data](#)” on page 532.

## SSN Bus Data Port Delays

The SSN data bus port external delays reflect the hard-coded timeplates in Tessent test patterns, relative to the `tessent_ssn_bus_clock_network` clock edges.

The SSN test patterns do the following:

- Force SSN bus data primary inputs at 0% of the tester period or 50% of the `bus_clock` period. Refer to the following discussion for further explanation.
- Cause the bus clock to rise at 0% of the tester period.
- Measure SSN bus data primary outputs at 0 ns into the next tester period.

This translates to an external delay of zero for both directions.

The first node of an SSN bus might capture its input data at every rising edge of the bus clock or on the first phase of a multi-phase node. This is the case for nodes of type Pipeline, ScanHost, or Multiplexer, or one of BusFrequencyMultiplier with its `capture_phase` property set to “transmitter”. For such nodes, the following is true:

- If the current design level is chip, the SSN patterns force the bus inputs at 50% of the bus clock period inside the tester period, thus giving a half `bus_clock_period` margin for both setup and hold.
- Otherwise, the patterns assume a synchronous data path across the block boundaries and force the bus inputs at 0% of the tester period.

For all other node types, SSN patterns force the bus inputs at 0% of the tester period.

The 0% output delay permits a full bus clock period to the setup margin of the bus data out loop timing path, as shown in the figure “[Schematic Showing Loop Timing Path for Bus Data Out](#)” in the section “[BusFrequencyDivider](#)” in the *Tessent Shell Reference Manual*.

The input/output delay constraints include an “`-add_delay`” option that enables them to share their primary pin with functional signals when you apply the non-modal constraints.

The following example illustrates the input/output delay constraints:

```
set_input_delay 0.0 -add_delay \  
-clock tessent_ssn_bus_clock_network \  
[tessent_get_ports {gpio[0]}]  
  
set_output_delay 0.0 -add_delay \  
-clock tessent_ssn_bus_clock_network \  
[tessent_get_ports {gpio[16]}]
```

At the chip level, the tester directly feeds input/outputs. Therefore, input/output pin delays are based on the actual pattern waveforms, and no adjustments or virtual clocks are necessary. For a chip, constraints look like the following:

```
set_input_delay 0.0 -add_delay \  
-clock tessent_ssn_bus_clock_network \  
[tessent_get_ports {gpio[1]}]  
  
set_output_delay 0.0 -add_delay \  
-clock tessent_ssn_bus_clock_network \  
[tessent_get_ports {gpio[6]}]
```

For a block, in order to account for pre- and post-layout designs with varying `ssn_bus_clock` latency and in order to facilitate timing path budgeting, the proc declares a virtual clock and `set_input/output_delay` constraints use global user-modifiable Tcl variables, such as in the following example:

#### **proc xxx\_ltest\_set\_timing\_variables\_default:**

```
set tessent_ssn_bus_input_delay_percentage 25.  
set tessent_ssn_bus_output_delay_percentage 25.
```

#### **proc xxx\_ltest\_ssn:**

```
set local_ssn_bus_clock_network_period \  
[expr $tessent_ssn_bus_clock_network_period * $time_unit_multiplier]  
set local_ssn_bus_input_delay  
[expr $tessent_ssn_bus_input_delay_percentage/100. * \  
$local_ssn_bus_clock_network_period]  
set local_ssn_bus_output_delay \  
[expr $tessent_ssn_bus_output_delay_percentage/100. * \  
$local_ssn_bus_clock_network_period]  
  
set_input_delay $local_ssn_bus_input_delay -add_delay \  
-clock tessent_ssn_virtual_bus_clock_network \  
[tessent_get_ports {ssn_bus_data_in[0]}]  
set_output_delay $local_ssn_bus_output_delay -add_delay \  
-clock tessent_ssn_virtual_bus_clock_network \  
[tessent_get_ports {ssn_bus_data_out[0]}]
```

---

#### **Note**



To accurately reflect the generated test patterns, the following occurs at the chip level:

- If the first SSN datapath node is a receiver\_1x\_pipeline (which captures at the falling edge of the `ssn_bus_clock`), the preceding input delay is set to 0.0 ns.
  - If the first node is anything else (that is, captures at the rising edge of the `ssn_bus_clock`), the input delay is set to  $0.5 \times \text{ssn\_bus\_clock\_period}$ .
-

For a core, the input delay is always set to  $0.0 + \langle \text{an external delay timing budget} \rangle$ , assuming the following:

The driving node exists inside the chip, either in the parent design or in another core.

SSN source nodes always toggle their output data on the `ssn_bus_clock` posedge, and full-speed data paths likely need balanced source/destination clocks.

If the datapath crosses a non-balanced clock, it does so through an SSN divider/multiplier combination. This combination is less sensitive to these input/output delay constraints, because it relaxes with additional multicycle path (MCP) constraints.

## SSN Datapath Timing Constraints

Some sections of the SSN datapath may run with a bus data rate that is a fraction of the bus clock speed, such as at half- or quarter-speed, thus behaving as multicycle paths (MCPs) across SSN nodes. Such paths occur when inserting SSN nodes of the types `BusFrequencyDivider`, `BusFrequencyMultiplier`, or `OutputPipeline` in your datapath specifications.

For more details about these nodes and their usage, refer to the section “[BusFrequencyDivider](#)” in the *Tessent Shell Reference Manual*.

The three previously mentioned nodes (`BusFrequencyDivider`, `BusFrequencyMultiplier`, and `OutputPipeline`) intercept the data bus bits with rows of flops that capture or update all bus data bits at a specified phase of the bus clock. In the SDC file, the phase of each node is represented by two numbers “(n m)” where *m* is the number of bus clock cycles for a single data cycle. For example, consider a quarter-speed SSN data bus section. The section may require MCPs between the following types of nodes:

- A `BusFrequencyDivider` node with a (1 4) phase

This means that the divider latches its output data at the beginning of the first (1) cycle for a duration of four (4) cycles.

- A `BusFrequencyMultiplier` node with a (3 4) phase

The specified phase number (3) represents the clock cycle number that the input data is latched at the beginning of. The output data rate of a multiplier always equals the clock rate.

- An `OutputPipeline` node with a (1 4) phase

The node features a single pipeline flop row that captures at the beginning of the first (1) cycle of four (4).

As another example, a bus datapath between a (1 4) divider and a (3 4) multiplier ends up with timing margins of two cycles of setup and two cycles of hold, making that combination suitable for data handoff between two skewed CTS regions:

```
# -----
# From bus_frequency_divider (phase 1 4) to bus_frequency_multiplier
# (phase 3 4)
set_multicycle_path -setup 2 -start \
    -from [tessent_get_cells <divider node instance>/datapath/r1*] \
    -to    [tessent_get_cells <multiplier node instance>/datapath/r0*]
set_multicycle_path -hold 3 -start \
    -from [tessent_get_cells <divider node instance>/datapath/r1*] \
    -to    [tessent_get_cells <multiplier node instance>/datapath/r0*]
```

A bus datapath between the previously referenced (1 4) divider node and a (1 4) output pipeline node features timing margins of four cycles of setup and zero cycles of hold, thus requiring balanced source and destination clocks:

```
# -----
# From bus_frequency_divider (phase 1 4) to output_pipeline (phase 1 4)
set_multicycle_path -setup 4 -start \
    -from [tessent_get_cells <divider node instance>/datapath/r1*] \
    -to    [tessent_get_cells <multiplier node instance>/datapath/r0*]
set_multicycle_path -hold 3 -start \
    -from [tessent_get_cells <divider node instance>/datapath/r1*] \
    -to    [tessent_get_cells <multiplier node instance>/datapath/r0*]
```

Datapaths from SSN bus output ports of a (1 4) OutputPipeline node also feature four cycles of setup margin, as shown in the figure “[Output Data Slow Down Using BusFrequencyDivider Node](#)” in the section “[BusFrequencyDivider](#)” in the *Tessent Shell Reference Manual*, because the test patterns capture the bus data at the rising edge of Phase 1, leaving all four bus cycles for data to close its output timing loop:

```
# -----
# From output_pipeline (phase 1 4) to output bus ports
set_multicycle_path -setup 4 -start \
    -from [tessent_get_cells <divider node instance>/datapath/r1*] \
    -to    [tessent_get_ports <list of output bus ports>]
set_multicycle_path -hold 3 -start \
    -from [tessent_get_cells <divider node instance>/datapath/r1*] \
    -to    [tessent_get_ports <list of output bus ports>]
```

More subtle MCP constraints are required when handling datapath sections that run across bus clocks that operate at frequencies that are ratios of each other. This could result in, for example, a (1 2) divider fanning out to a (3 4) multiplier, which implies that the destination multiplier is clocked at double the frequency of the source divider. In such cases, the MCP constraint always

sets its setup and hold number of cycles relative to the faster of the two clocks, using either the -start or the -end option:

```
# -----  
# From bus_frequency_divider (phase 1 2) to bus_frequency_multiplier  
# (phase 3 4)  
set_multicycle_path -setup 2 -end \  
    -from [tessent_get_cells <divider node instance>/datapath/r1*] \  
    -to   [tessent_get_ports <multiplier node instance>/datapath/r0*]  
set_multicycle_path -hold 2 -end \  
    -from [tessent_get_cells <divider node instance>/datapath/r1*] \  
    -to   [tessent_get_ports <multiplier node instance>/datapath/r0*]
```

Input and output pins of child physical blocks may also include phases if they internally connect to either SSN multiplier or divider nodes. These phases are reported in the module description of their .icl file. Because the internal cells of a physical block are not normally visible to the parent SDC, you must set MCP constraints using the -through switch with the PB boundary pins, as in the following example:

```
# -----  
# From PB output pins (phase 1 2) to PB input pins (phase 0.5 1)  
set_multicycle_path -setup 1 -start \  
    -through [tessent_get_pins <PB1 list of bus data output pins> \  
    -through [tessent_get_pins <PB2 list of bus data input pins>]  
set_multicycle_path -setup 1 -start \  
    -through [tessent_get_pins <PB1 list of bus data output pins> \  
    -through [tessent_get_pins <PB2 list of bus data input pins>]
```

The extract\_sdc command itself actively traces the .icl files of the design to find which actual SSN node is connected to which other SSN nodes and whether phase specifications are involved. Such tracing enables handling of bus networks with multiplexer nodes and connections between partial lists of SSN bus ports or subphysical block pins, which can further complicate the list of MCPs. For the sake of clarity and self-documentation, the SDC file reports the traced list of connections under a comment banner labeled “SSN list of node connections.”

This banner shows all connections, including those not involving clock phases, as in the following example:

```
#-----
# SSN list of node connections
#  output_pipeline 'top_rtl2_tessent_ssn_out_pipe_out_inst'
#    --> SSN Bus Data Outputs
#  bus_frequency_divider 'top_rtl2_tessent_ssn_bus_freq_div_1_inst'
#    --> output_pipeline 'top_rtl2_tessent_ssn_out_pipe_out_inst'
#  scan_host 'top_rtl2_tessent_ssn_scan_host_1_inst'
#    --> bus_frequency_divider 'top_rtl2_tessent_ssn_bus_freq_div_1_inst'
#  pipeline 'top_rtl2_tessent_ssn_pipe_2_inst'
#    --> scan_host 'top_rtl2_tessent_ssn_scan_host_1_inst'
#  bus_frequency_multiplier 'top_rtl2_tessent_ssn_bus_freq_mult_1_inst'
#    --> pipeline 'top_rtl2_tessent_ssn_pipe_2_inst'
#  physical_block 'corea_i1' (launching phase = 1/2)
#    --> bus_frequency_multiplier 'top_rtl2_tessent_ssn_bus_freq_mult_1_inst'
#  physical_block 'corea_i2' (launching phase = 1/2)
#    --> physical_block 'corea_i1' (strobing phase = 0.5/1)
#  pipeline 'top_rtl2_tessent_ssn_pipe_1_inst'
#    --> physical_block 'corea_i2' (strobing phase = 0.5/1)
#  SSN Bus Data Inputs
#    --> pipeline 'top_rtl2_tessent_ssn_pipe_1_inst'
#-----
```

Datapath instances retimed using a trailing edge (TE) register at their inputs have a multicyle path timing exception from the reset register that controls the datapath registers to the TE retiming registers. This includes the Receiver1XPipeline, and it also includes the FIFO and BusFrequencyDivider instances if their datapaths are retimed with TE registers. The input data for these instances is held at zero during reset.

```
# Reset of the TE datapath retiming register can have a setup MCP of 2 because
# the initial data in this register is flushed and not observed during test.
# The data input of the registers is also zero during reset.
set_multicycle_path -setup 2 \
  -from [tessent_get_cells \
    top_tessent_ssn_receiver_1x_pipe_in_inst/fsm/bus_sync_reset_ff*] \
  -to [tessent_get_cells \
    top_tessent_ssn_receiver_1x_pipe_in_inst/datapath/r0_n*]
set_multicycle_path -hold 1 \
  -from [tessent_get_cells \
    top_tessent_ssn_receiver_1x_pipe_in_inst/fsm/bus_sync_reset_ff*] \
  -to [tessent_get_cells \
    top_tessent_ssn_receiver_1x_pipe_in_inst/datapath/r0_n*]
```

## SSN FIFO Timing Constraints

An SSN FIFO advantageously replaces a combination of BusFrequencyDivider and BusFrequencyMultiplier nodes in cases where the clock domain handoff can occur within a single physical partition.

For a full description of the FIFO circuit and node definition usage, refer to the section “[Fifo](#)” in the *Tessent Shell Reference Manual*. This section also includes information about how Tessent

Shell identifies which clocks are associated with the first and last sequential elements on the SSN datapath for SDC extraction when a Fifo node is present.

A Fifo node includes an input register for storing input packets, similar to that of a BusFrequencyDivider, and it selects the output packet based on the phase counter value, similar to the operation of a BusFrequencyMultiplier. Similar to datapaths between BusFrequencyDivider and BusFrequencyMultiplier explained in the section “[SSN Datapath Timing Constraints](#)” on page 600, a set\_multicycle\_path declaration can relax FIFO internal paths from the input register. The setup and hold parameters for this declaration depend the following property values inside the DftSpecification/SSN/Datapath/Fifo wrapper:

- frequency\_ratio
- in\_clock\_to\_out\_clock
- in\_clock\_to\_out\_clock\_skew\_programmable

You can think of the last two properties as defining a “deskewing” mode.

Data output from the FIFO input register remains stable for a number of clock cycles equal to the value of the FIFO frequency\_ratio property. The FIFO output register captures this data either in the middle of these cycles (when in\_clock\_to\_out\_clock\_skew is early\_or\_delayed) or at the end of these cycles (when in\_clock\_to\_out\_clock\_skew is delayed\_only). Refer to the figures “[Waveform for FIFO early\\_or\\_delayed Configuration](#)” and “[Waveform for FIFO delayed\\_only Configuration](#)” in the “Fifo” reference page in the *Tessent Shell Reference Manual* to see examples of these capture cycles. Because of this, both of your SDC FIFO multicycle path (MCP) constraint “-setup” and “-hold” parameters depend on the FIFO frequency\_ratio and in\_clock\_to\_out\_clock\_skew parameters. You can adjust them at run time if the in\_clock\_to\_out\_clock\_skew\_programmable parameter is on.

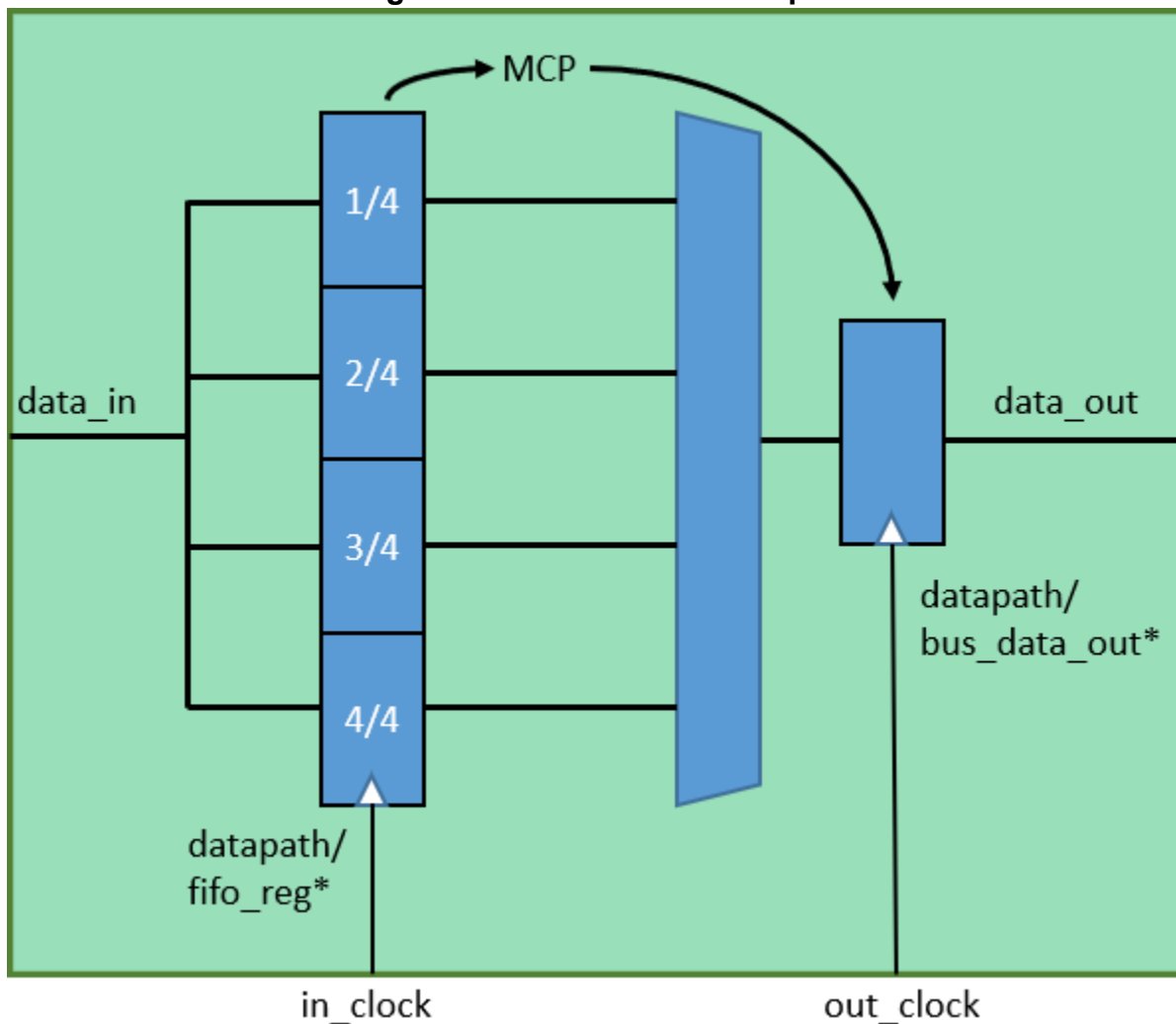
In the SDC, the setup and hold calculations are done explicitly using intermediate Tcl variables.

If you set in\_clock\_to\_out\_clock\_skew\_programmable to “on”, the SDC enables you to dynamically select the deskewing mode just before applying the SDC in the timing tool. For each such FIFO instance in your design, select the deskewing mode by defining the Tcl variable tessent\_ssn\_fifo\_deskew\_setting(sfifo<n>) in your primary script. This variable name is mapped; refer to the primary calling script in the “[Examples](#)” section in this topic to see how to use these variables. The in\_clock\_to\_out\_clock\_skew property determines the default deskew mode when you do not set these variables.

The following figure illustrates how the MCP is applied in a FIFO.



Figure 9-30. FIFO MCP Example



## Examples

### Example of FIFO Settings in Your Primary Calling Script

The following example shows how to use Tcl variables in the primary calling script to set up the FIFO and control programmable skew settings:

```
tessent_set_default_variables
tessent_ssn_fifo_deskew_setting(sfifo0)    delayed_only
tessent_ssn_fifo_deskew_setting(sfifo1)    early_or_delayed
tessent_ssn_fifo_deskew_setting(sfifo2)    delayed_only
tessent_set_non_modal
# this proc calls the tessent_set_ltest_ssn proc, which appears in the
# next example section
```

### Examples of FIFO SDC File Contents

The following is an example of content appearing in the SDC proc `tessent_set_default_variables`:

```
array set tessent_sfifo_mapping {
    sfifo0 top_rtl2_tessent_ssn_fifo_1_inst
    sfifo1 core1/core_rtl2_tessent_ssn_fifo_1_inst
    sfifo2 core2/core_rtl2_tessent_ssn_fifo_1_inst
}
```

The following is an example of content appearing in the SDC proc `tessent_set_ltest_ssn`:

```
global tessent_ssn_fifo_deskew_setting
array set sfifo_setup_multiplier {delayed_only 1 early_or_delayed 0.5}
# sfifo0:
#   frequency_ratio : 4
#   in_clock_to_out_clock_skew: delayed_only
#   in_clock_to_out_clock_skew_programmable: on
set deskew_setting delayed_only

if {[info exists tessent_ssn_fifo_deskew_setting(sfifo0)]} {
    set deskew_setting $tessent_ssn_fifo_deskew_setting(sfifo0)
}

set frequency_ratio 4
set_multicycle_path \
    -setup [expr int($frequency_ratio* $sfifo_setup_multiplier($deskew_setting))] \
    -from [tessent_get_cells $tessent_sfifo_mapping(sfifo0)/datapath/fifo_reg*] \
    -to [tessent_get_cells $tessent_sfifo_mapping(sfifo0)/datapath/bus_data_out*]
set_multicycle_path \
    -hold [expr $frequency_ratio - 1] \
    -from [tessent_get_cells $tessent_sfifo_mapping(sfifo0)/datapath/fifo_reg*] \
    -to [tessent_get_cells $tessent_sfifo_mapping(sfifo0)/datapath/bus_data_out*]
```

---

#### Note



The preceding constraints repeat for `sfifo1` and `sfifo2`, but with variations for the actual MCP constraints that depend on the `DftSpecification` property settings for these FIFOs.

---

---

#### Note



The section in bold in the previous example is present only when programmable skew is enabled; that is, `in_clock_to_out_clock_skew_programmable` is set to “on”.

---

## SSN ScanHost Timing Constraints and Clock Tree Synthesis

The ScanHost node generates the scan control and data signals when it drives the logictest modules of a block. It generates all of these signals from the SSN `bus_clock`.

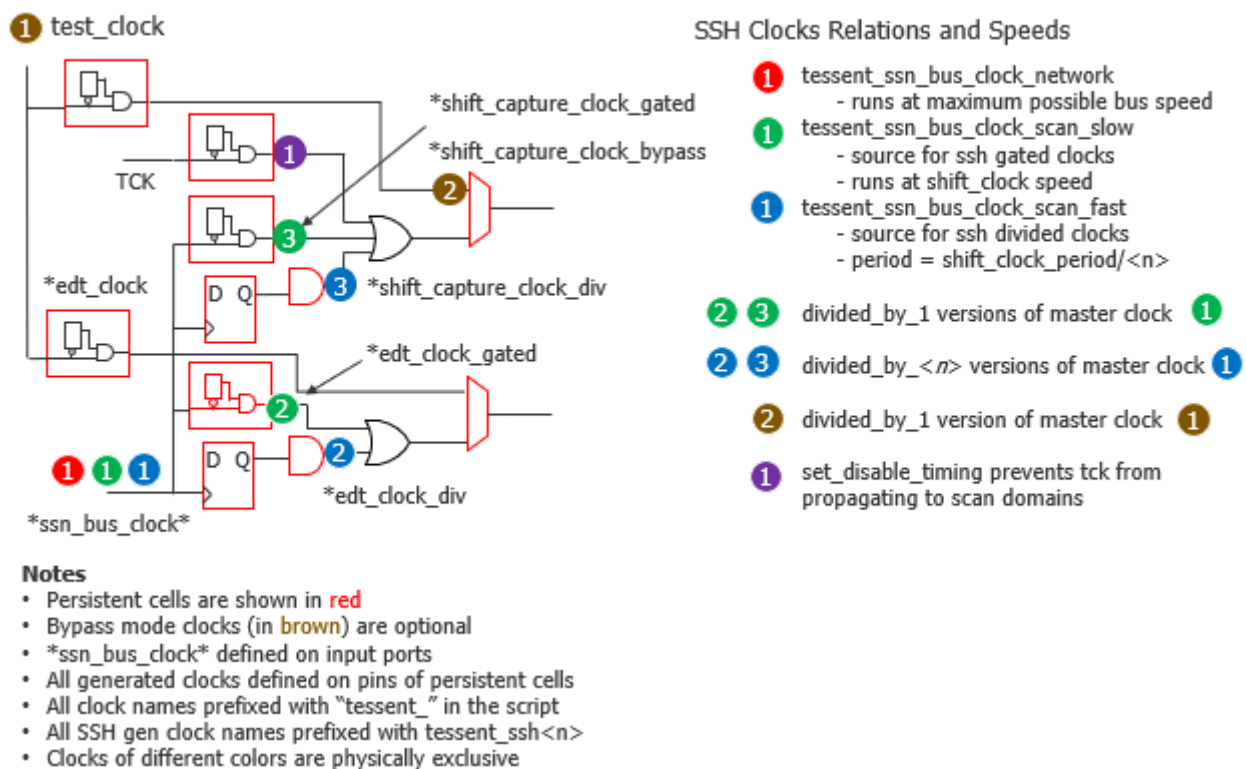
As a prerequisite for the SSH SDC descriptions in this section, it is recommended that you clearly understand the SSH scan interface operation and features described in the section

“ScanHost Scan Interface Operation and Timing Requirements” of the [ScanHost](#) reference page in the *Tessent Shell Reference Manual*.

## SSH Scan Clocks

A given Tessent test SSN scan pattern may drive the SSN bus clock port with one of three different frequencies. The frequency depends on whether the ScanHost is bypassed or used for scan with gated or with divided shift clocks. To precisely determine the timing that the hardware implements, the SDC needs to create three clocks on that port. The following figure shows all root clocks and generated clocks that may exist in the SDC.

**Figure 9-31. SSH Clock Signal Generation**  
SSH Scan Interface Clocks



Call the [set\\_load\\_unload\\_timing\\_options](#) command to configure the SDC clock and control signal timing parameters, and to configure pattern generation so that the parameters of the generated patterns are consistent with those used during timing closure and analysis. The following table lists [set\\_load\\_unload\\_timing\\_options](#) arguments that define Tcl variables used in SDC. If your primary timing script does not explicitly set these options, then the tool uses the default values from the table.

**Table 9-6. set\_load\_unload\_timing\_options Arguments and SDC Variables**

| Option                | SDC Timing Variable                  | Default |
|-----------------------|--------------------------------------|---------|
| -ssn_bus_clock_period | tessent_ssn_bus_clock_network_period | 2.5 ns  |

**Table 9-6. set\_load\_unload\_timing\_options Arguments and SDC Variables**

| Option                         | SDC Timing Variable                    | Default |
|--------------------------------|----------------------------------------|---------|
| -shift_clock_period            | tessent_ssn_bus_clock_scan_slow_period | 10.0 ns |
| -scan_en_setup_extra_cycles    | tessent_scan_en_setup_extra_cycles     | 1       |
| -scan_en_hold_extra_cycles     | tessent_scan_en_hold_extra_cycles      | 1       |
| -edt_update_setup_extra_cycles | tessent_edt_update_setup_extra_cycles  | 1       |
| -edt_update_hold_extra_cycles  | tessent_edt_update_hold_extra_cycles   | 1       |

The first SSN bus clock is called `ssn_bus_clock_network` (red dot in [Figure 9-31](#)). It times the SSN network operation at its maximum possible datapath speed, whether or not the ScanHost node is active.

When the ScanHost node is active, the SSN bus clock pin generates the `shift_capture_clock` and the `edt_clock` used to operate the scan circuitry. One scan bit per chain gets shifted in per SSN packet. If the SSN packet is small enough to occupy less than two bus clock cycles on the network, the scan clocks can be generated from the SSN bus clock using a clock gater. The test patterns then adjust the SSN bus clock frequency to match the required `shift_clock_period`. However, when the SSN packet is large enough to occupy a minimum of  $N$  bus clock cycles on the network, a faster frequency clock is applied to the `ssn_bus_clock` pin and is divided to generate the shift clocks using a clock divider flop, maintaining as much as possible a fifty-percent duty cycle. Because these two clock paths differ slightly, both must be covered in SDC and STA, requiring the SDC to create one generated clock for the gated clock and another for the divided clock.

When the test patterns generate the scan clocks using a clock gater, they must increase the SSN bus clock period to be as large as the shift clock period, which defaults to 10 ns. The SDC version of the SSN bus clock created at the `shift_clock_period` is called `ssn_bus_clock_scan_slow` (green dot labeled 1 in [Figure 9-31](#)) and is referenced as the source of the generated clocks `ssn_shift_capture_clock_gated` and `ssn_edt_clock_gated`, which are defined on the outputs of the `edt_clock_cg` and `shift_capture_clock_cg` persistent cells (green dots 2 and 3 in [Figure 9-31](#)).

When the test patterns generate the scan clocks using a clock divider, another SDC clock called `ssn_bus_clock_scan_fast` is required (blue dot 1 in [Figure 9-31](#)) and becomes the -master option of the divided\_by  $N$  generated clocks called `ssn_shift_capture_clock_div` and `ssn_edt_clock_div`, which are defined on the output of the `edt_clock_div` and `shift_capture_clock_div` cells (blue dots 2 and 3 in [Figure 9-31](#)).

The SDC aims to set the period of the `ssn_bus_clock_scan_fast` clock to exactly half that of the `ssn_bus_clock_scan_slow` clock. Then it applies a “divided\_by 2” generated clock at the output of the clock divider flop. This enables all scan timing paths depending on that divided clock to be constrained to their tightest possible limits. If the target `ssn_bus_clock_scan_fast` period would actually make the SSN bus exceed its maximum speed, then it is instead made equal to the `ssn_bus_clock_network` period. Consider an example where the `ssn_bus_clock` period is

2.5 ns and the shift\_clock period is 10 ns. In this case, the SDC would set the ssn\_bus\_clock\_scan\_fast period to  $10 \text{ ns} / 2 = 5 \text{ ns}$ . If the ssn\_bus\_clock period were instead set to 6 ns, then the SDC would set the ssn\_bus\_clock\_scan\_fast period to the same 6 ns, making the divided generated scan clock slower than the target shift\_clock; this is acceptable because it correctly reflects the reality of the test patterns.

#### Note

 In practice, the divided clock frequency ratio might often exceed 2, because that value depends on the size of the SSN bus packet. The packet size can be quite large if many SSHs are driven in parallel or if the number of EDT channels greatly exceeds the SSN bus width.

Even more clock declarations are required if the SSH supports the optional scan bypass mode. These clocks are represented by the brown dots in [Figure 9-31](#). Both the \*edt\_clock and \*ssh\_shift\_capture\_clock are divided\_by 1 versions of the test\_clock. For simplification of the constraints, the generated SDC file assumes a test\_clock period that always matches the value specified by the “set\_load\_unload\_timing\_options -shift\_clock\_period” command, even though the latter value ideally applies only to the SSN scan mode and not the bypass mode.

The preceding clocks belong to clock groups that correspond to their dot colors in [Figure 9-31](#). Red, blue, green, and brown clocks never run at the same time. Therefore, the SDC declares them as “-physically\_exclusive.” All such clocks are also asynchronous to all functional domain clocks, because both clock types may interact at the same time on the pre-OCC branches during scan. The following is an excerpt from a sample design:

**Figure 9-32. SSH Clock Group Constraints**

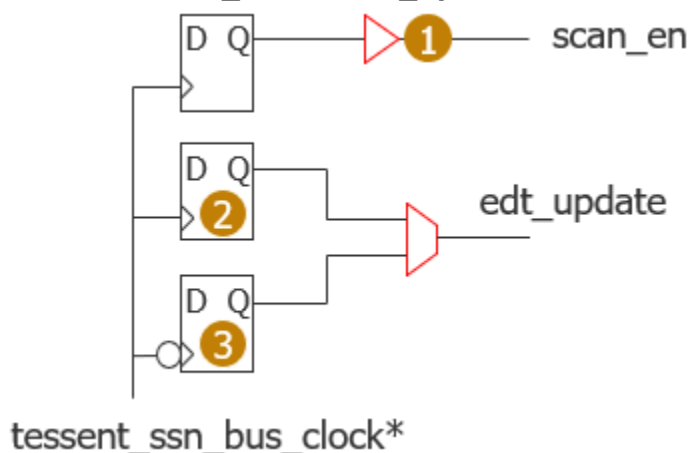
```
# Maximum SSN network speed
set_clock_groups -physically_exclusive \
    -group {tessent_ssn*_bus_clock_network*}

# Mutually exclusive logictest scan mode clocks
set_clock_groups -physically_exclusive \
    -group {tessent_ssn_bus_clock_scan_fast* tessent_ssh*_div*} \
    -group {tessent_ssn_bus_clock_scan_slow* tessent_ssh*_gated*} \
    -group {tessent_ssh*_bypass tessent_virtual* tessent_test_clock}
```

## scan\_en and edt\_update Signal Timing

The following figure shows the SSH circuit that generates the scan\_en and edt\_update signals.

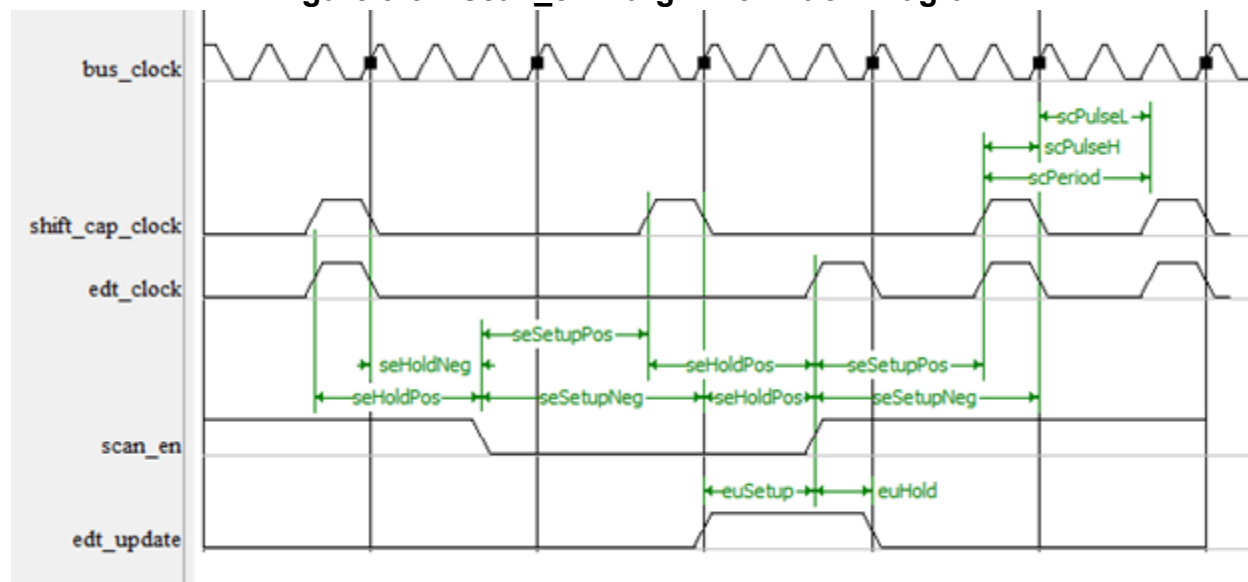
**Figure 9-33. SSH scan\_en and edt\_update Generation Circuit**



The SSH generates the `edt_update` signal with a negedge flop (brown dot 3) when `bus_clock_period = scan_clock_period` to provide negedge retiming to its destination EDT controller's posedge flops. The posedge flops run off the late `edt_clock` relative to the `bus_clock`. When `edt_clock` is divided, on the other hand, a posedge source flop (brown dot 2) can properly toggle the `edt_update` signal 0.5 `edt_clock` cycles before and after the posedge of the destination flop.

Although the `scan_en` signal may propagate to either posedge or negedge scannable flops on a delayed SCC clock tree, the fact that SCC is gated while `scan_en` toggles makes both its setup and hold timing safer without needing to rely on a negedge flop source like for `edt_update`. Furthermore, the `scan_en` setup margin to negedge flops gets a half-cycle bonus, because the first SCC falling edge after a `scan_en` transition always occurs after the first rising edge. The following figure shows the margin definitions.

**Figure 9-34. scan\_en Margin Definition Diagram**



By default, although these signals show some very safe setup and timing margins, you can still extend them by calling the [set\\_load\\_unload\\_timing\\_options](#) command. As discussed previously, this proc sets several global Tcl variables used in timing exceptions discussed in the following information.

In the Tessent SDC file, the hold and setup margins of all MCP constraints are derived dynamically from the variable values in [Table 9-6](#). The following excerpt from an SDC file shows how all setup and hold margins are calculated. In this excerpt, the Tcl variable names intentionally match the setup and hold margin names shown in [Figure 9-34](#). As previously mentioned, it is helpful when reading this excerpt to remember that the divided scan clock is a “divided\_by 2” version of the `ssn_bus_clock_scan_fast` clock.

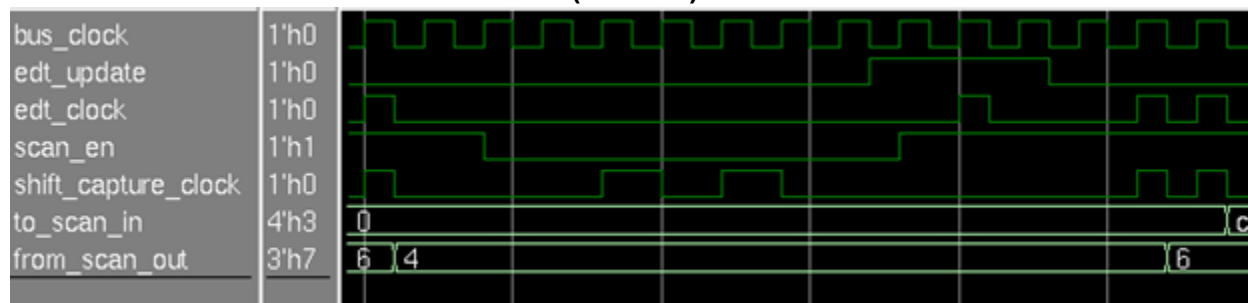
```
# Scan_en to gated scan clocks:
set seSetupPos [expr 1 + $tessent_scan_en_setup_extra_cycles]
set seHoldPos [expr 1 + $tessent_scan_en_hold_extra_cycles]
set seSetupNeg [expr 1 + $seSetupPos]
set seHoldNeg $tessent_scan_en_hold_extra_cycles
# Scan_en to divided by 2 scan clocks
set seSetupPos_div [expr $seSetupPos * 2]
set seHoldPos_div [expr $seHoldPos * 2]
set seSetupNeg_div [expr $seSetupPos_div + 1]
set seHoldNeg_div [expr $seHoldPos_div - 1]
# edt_update to gated scan clocks:
set setup [expr 1 + $tessent_edt_update_setup_extra_cycles]
set hold $tessent_edt_update_hold_extra_cycles
# edt_update to divided scan clocks:
set setup [expr 1 + $tessent_edt_update_setup_extra_cycles * 2]
set hold [expr 1 + $tessent_edt_update_hold_extra_cycles * 2]
```

The following is an example of how the preceding \$setup and \$hold Tcl variables are typically used in the SDC file MCP constraints:

```
set_multicycle_path -setup $setup -start \
  -from [tessent_get_cells $tessent_ssh_mapping(ssh0)/clock_gen/edt_update_delayed*] \
  -to [tessent_get_clocks tessent_ssh*_div]
set_multicycle_path -hold [expr $setup-1 + $hold] -start \
  -from [tessent_get_cells $tessent_ssh_mapping(ssh0)/clock_gen/edt_update_delayed*] \
  -to [tessent_get_clocks tessent_ssh*_div]
```

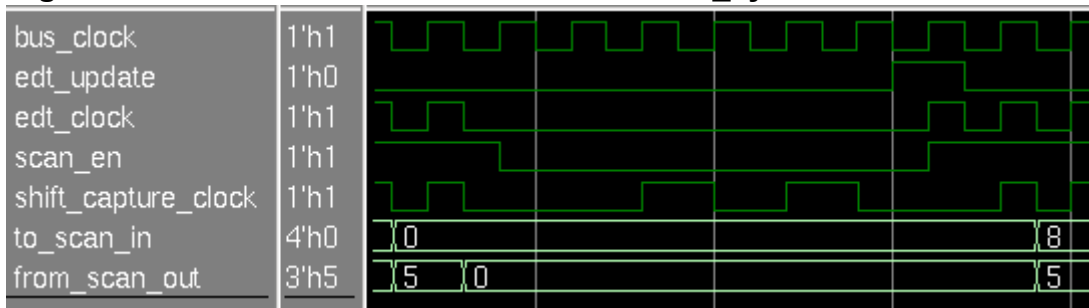
To help you visualize these constraints, the following figures show some actual simulation results of the timing of the scan\_en and edt\_update signals relative to the SSN bus and SSH scan clocks:

**Figure 9-35. Gated Scan Clocks With All \*extra\_cycle\* Variables Set to 1 (Default)**



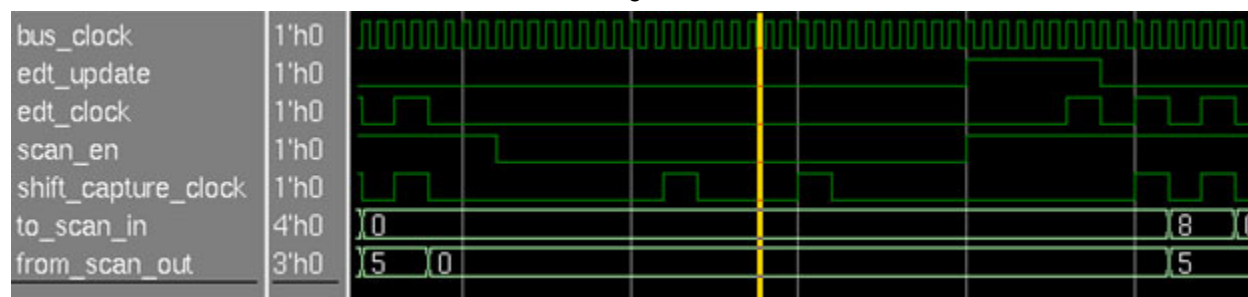
The following figure is the same as the previous, but the \*extra\_cycle\* variables are set to zero. This results in shorter setup and hold margins.

**Figure 9-36. Gated Scan Clocks With All \*extra\_cycle\* Variables Set to 0**



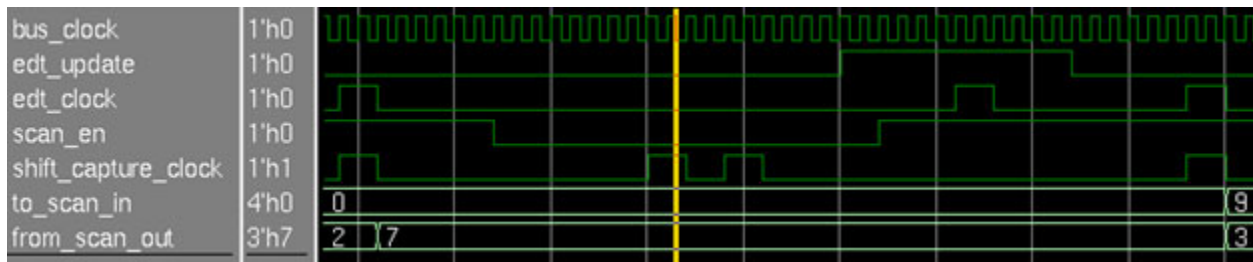
In this figure, due to some inner SSH logic requirements, the scan\_en setup and hold paths get some bonus margins over the gated clocks case.

**Figure 9-37. Divided Shift Clocks With Ratio of 4 and \*extra\_cycle\* Variables at 0**





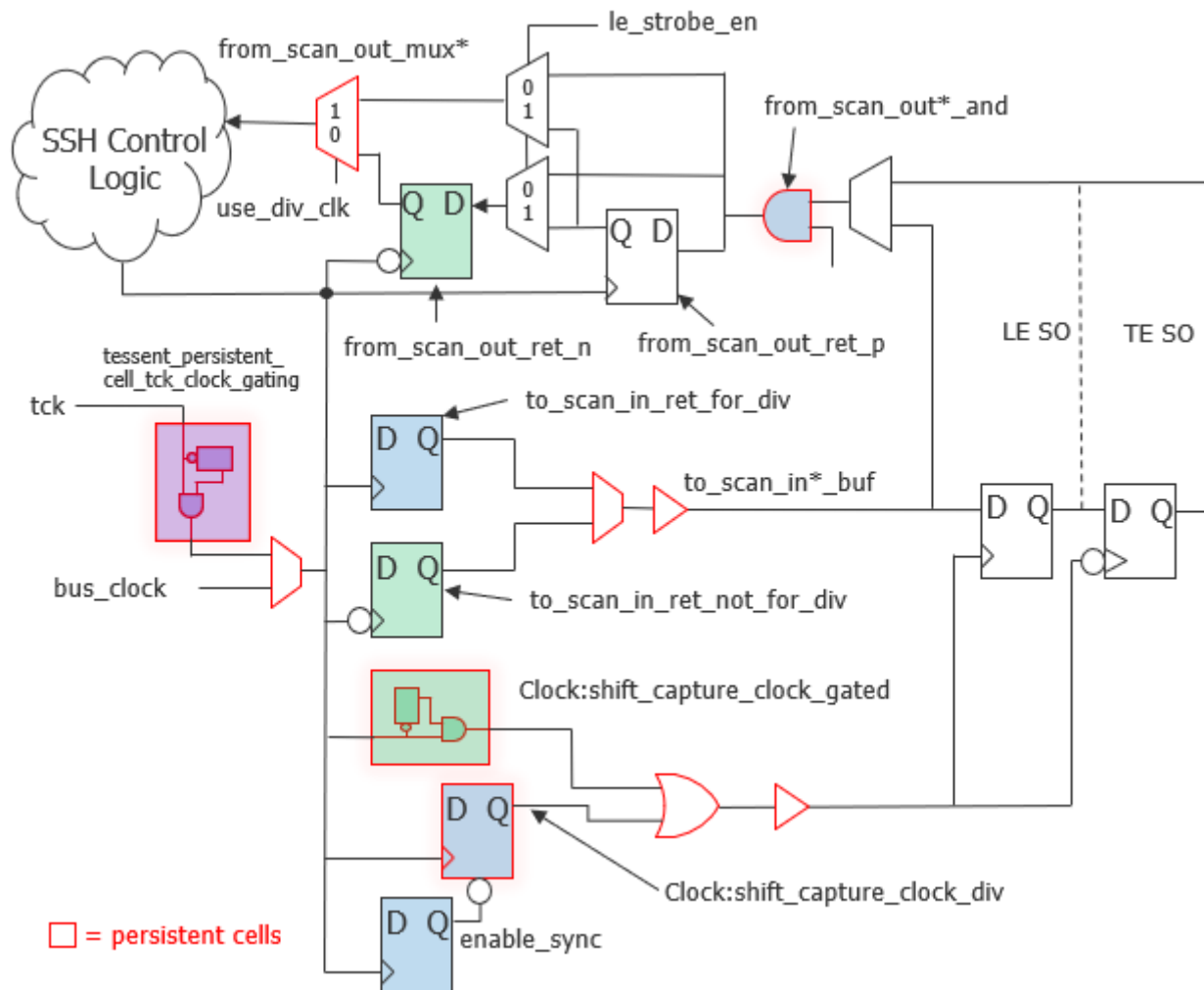
**Figure 9-38. Divided Shift Clocks With Ratio of 4 and \*extra\_cycle\* Variables at 1**



## Scan Chain Interface Timing

The SSH scan chain interface circuit appears in the following figure. Given the many different clocks and the two operation modes involved, this circuit requires many timing exceptions of different natures, such as set\_disable\_timing, set\_false\_path, and set\_multicycle\_paths.

**Figure 9-39. SSH Scan Chain Interface Circuit**



The following describes how this circuit operates:

- Persistent cells appear with red borders.
- All clock names appear without the “tessent\_” prefix.
- The two flops on the right-hand side represent the scan chains. Their SI input comes from the SSH to\_scan\_in\* pins, and their SO outputs go back to the SSH through the from\_scan\_out\* pins.
- Direct loopback paths exist in some modes between the SSH to\_scan\_in\* and from\_scan\_out\* pins.
- Logic elements that are active only during scan with gated shift clocks are colored green.
- Logic elements that are active only during scan with divided shift clocks are colored blue.
- The from\_scan\_out\*\_and (blue) cell strobes the SO data of the returning chain when running with divided shift clocks, at the last of the N cycles of the divided clock. It is transparent when running with the gated clock.
- Chains may feature either a trailing edge (TE) or leading edge (LE) chain terminating SO flop.
  - When running with gated shift clocks, the from\_scan\_out\_ret\_n (green) flop captures TE SO data.
  - With both gated and divided clocks, the from\_scan\_out\_ret\_p flop captures the LE SO data.
  - With divided clocks, the SSH control logic directly captures TE SO data through the from\_scan\_out\_mux.
- The tck\_clock\_gater (purple) cell feeds a gated version of ts\_tck\_tck to the SSH logic when it runs in Streaming-Through-IJTAG mode.

### Scan Chain Interface Timing Exceptions

- The clock tessent\_ssn\_bus\_clock\_scan\_fast\* serves as a master clock for creating the SSH divided clock and for timing the interactions of the tessent\_ssh\*\_div (divided) clocks with the posedge flops of the SSH scan interface circuit. However, because all of the SSH controller’s intradomain paths can be timed to their maximum speeds with the clock tessent\_ssn\_bus\_clock\_network\*, the [extract\\_sdc](#) command can declare the following:

```
set_false_path -from [tessent_get_clocks tessent_ssn_bus_clock_scan_fast*] \  
               -to   [tessent_get_clocks tessent_ssn_bus_clock_scan_fast*]
```

- All of the preceding SSH scan interface negedge (green) flops are used only in combination with the slower `tessent_ssn_bus_clock_scan_slow` and `tessent_ssh*_gated` clocks:

```
set_false_path -fall_from [tessent_get_clocks tessent_ssn_bus_clock_scan_fast*]
set_false_path -fall_to   [tessent_get_clocks tessent_ssn_bus_clock_scan_fast*]
```

- Similarly, all green flops are active only when a slow shift-speed clock is injected on the `ssn_bus_clock` port. Therefore, the resulting half-cycle paths are all false with regard to the maximum speed of the `tessent_ssn_bus_clock_network`. The simpler `-fall_from/-fall_to tessent_ssn_bus_clock_network` constraint cannot be used, because it might also kill some valid half-cycle timing paths inside the SSN 1x receiver pipelining nodes.

```
set negedge_flops_list [list \
    <SSH instance>/clock_gen/edt_update_ret* \
    <SSH instance>/datapath/to_scan_in*_ret_not_for_div* \
    <SSH instance>/datapath/from_scan_out*_ret_n* \
    <SSH instance>/datapath/last_in_bits_in_current_bus_word_ret* \
]
set_false_path -from [tessent_get_clocks "tessent_ssn_bus_clock_network*"] \
    -to [tessent_get_cells $negedge_flops_list]
set_false_path -to [tessent_get_clocks "tessent_ssn_bus_clock_network*"] \
    -from [tessent_get_cells $negedge_flops_list]
```

- The SSH with the posedge flop that sources the `to_scan_in` signal is only active with the divided clock and toggles in the middle of the `tessent_ssh*_div` clock cycle in order to act as a retiming scan element.

```
set_false_path -from [tessent_get_cells \
    <SSH instance>/datapath/to_scan_in*_ret_for_div*] \
    -to [tessent_get_clocks tessent_ssh*_gated]
set_multicycle_path -hold 1 -start \
    -from [tessent_get_cells \
    <SSH instance>/datapath/to_scan_in*_ret_for_div*] \
    -to [tessent_get_clocks tessent_ssh*_div]
```

- The `from_scan_out` logic constraints are more complex than other constraints because they support scan SO paths coming from either leading-edge (LE) or trailing-edge (TE) flops.

```
# With a gated clock, all chain SO are captured by a single strobing flop. All
# other paths are false.
set_false_path -from [tessent_get_clocks tessent_ssh*_gated] \
               -through [tessent_get_pins \
                        <SSH instance>/datapath/tessent_persistent_cell_from_scan_out*_mux*/Y]

# With a gated clock, the chain SO TE flop is captured by the TE strobe register
set_false_path -fall_from [tessent_get_clocks tessent_ssh*_gated] \
               -to [tessent_get_cells \
                   <SSH instance>/datapath/from_scan_out*_ret_p*]
# With a gated clock, the chain SO LE flop is captured by the LE strobe register
set_false_path -rise_from [tessent_get_clocks tessent_ssh*_gated] \
               -to [tessent_get_cells \
                   <SSH instance>/datapath/from_scan_out*_ret_n*]

# Divided clock to fast clock logic inside the SSH
set_multicycle_path -setup 2 -end \
                   -from [tessent_get_clocks tessent_ssh*_div] \
                   -through [tessent_get_pins \
                             <SSH instance>/tessent_persistent_cell_from_scan_out*_and/A0] \
                   -to [tessent_get_clocks tessent_ssn_bus_clock_scan_fast_0]
set_multicycle_path -hold 1 -end \
                   -from [tessent_get_clocks tessent_ssh*_div] \
                   -through [tessent_get_pins \
                             <SSH instance>/tessent_persistent_cell_from_scan_out*_and/A0] \
                   -to [tessent_get_clocks tessent_ssn_bus_clock_scan_fast_0]
```

- For SSH loopback logic, only valid paths exist inside the SSH controller, and they always meet one cycle of the slow shift clock with zero hold, so they can be accurately timed using only the SSN slow scan clock. However, because the SSH constraints are multimode, the absence of case\_analysis settings enable unexpected invalid paths to pass through the SSH\*\_bypass ports, coming from different irrelevant clocks, including those from other SSHs. Such clocks must be disabled.

```
set_non_ssn_slow_clocks [remove_from_collection [all_clocks] \
               [tessent_get_clocks tessent_ssn_bus_clock_scan_slow*]]
set_false_path \
               -from $non_ssn_slow_clocks \
               -through [tessent_get_pins \
                       <SSH instance>/tessent_persistent_cell_to_scan_in*_buf/Y] \
               -through [tessent_get_pins \
                       <SSH instance>/tessent_persistent_cell_from_scan_out*_and/A0]
set_false_path \
               -through [tessent_get_pins \
                       <SSH instance>/tessent_persistent_cell_to_scan_in*_buf/Y] \
               -through [tessent_get_pins \
                       <SSH instance>/tessent_persistent_cell_from_scan_out*_and/A0] \
               -to $non_ssn_slow_clocks
```

- Disable false paths going through SSH scan control signals to external EDT pipelining flops, which are only used when the SSH is inactive.

```
set_false_path \
  -from [tessent_get_clocks tessent_ssn_bus_clock*] \
  -through [tessent_get_pins \
    <SSH instance>/tessent_persistent_cell_from_scan_out*_and/*] \
  -to [tessent_get_clocks tessent_ssh*]
```

- Some multiplexers switch clocks statically. In Synopsys® PrimeTime® products, the following command suppresses noisy clock gating check warnings, such as the PrimeTime PTE-060 warning:

```
set_disable_clock_gating_check [tessent_get_cells <SSH instance>/*clock_mux]
```

- The setup margin of full- and half-cycle ijtag\_tck timing paths is relaxed with a two-cycle multicycle path (MCP), and the hold margin is set to false.

```
# Static SSH TDR registers to SSH mission logic are all stable during test.
# Both source and destination are in the fanout of the ijtag clock.
set_ssh_flops [tessent_get_flops \
  [list $tessent_ssh_mapping(ssh0)/* \
    $tessent_ssh_mapping(ssh0)/*/* \
    $tessent_ssh_mapping(ssh0)/*/*/*] \
  -filter {is_hierarchical==false}]
set_ssh_ijtag_flops [tessent_get_flops \
  $tessent_ssh_mapping(ssh0)/ijtag_registers/* \
  -filter {is_hierarchical==false}]
set_ssh_mission_flops [remove_from_collection $ssh_flops $ssh_ijtag_flops]
set_multicycle_path 2 -from $ssh_ijtag_flops -to $ssh_mission_flops
set_false_path -hold -from $ssh_ijtag_flops -to $ssh_mission_flops
```

- The following provides various additional necessary constraints that may be present depending on the SSH options:

```
# Block ts_tck_tck used as capture clock
set_disable_timing [tessent_get_pins \
  <SSH instance>/clock_gen/clock_signals/tessent_persistent_cell_*_or2/<A1>

# Prevent false paths to scannable tessent memory_bist logic, which uses
# ts_tck_tck in setup mode.
set_false_path -from [tessent_get_clocks $tessent_clock_mapping(ts_tck_tck)] \
  -through [tessent_get_pins \
    <SSH instance>/tessent_persistent_cell_scan_en_buf/<out>]

# ssh0 enable_sync is static during test and fans out to ltest logic through the
# scan_en and edt_update signals.
set_false_path -from [tessent_get_cells <SSH instance>/fsm/enable_sync*] \
  -to [tessent_get_clocks "tessent_ssh*_gated tessent_ssh*_div"]

# bscan_capture_shift_clock is in the same clock group as ts_tck_tck, and timing
# paths from static SSH IJTAG TDRs may reach the bscan cells through the
# SSH/scan_en signal.
if {[sizeof_collection [tessent_get_clocks bscan_capture_shift_clock -quiet]]} {
  set_false_path -from [tessent_get_cells <SSH instance>/ijtag_registers/*_reg] \
    -to [tessent_get_clocks bscan_capture_shift_clock]
}
```

## SDC Constraints and the Optional Clock Shaper Cell

The optional `use_clock_shaper_cell` property in the SSH configuration wrapper specifies to use library clock shaper cells instead of flip-flops and clock gaters to generate SSH output scan clocks such as `edt_clock`, `shift_capture_clock`, and `test_clock`. You can program a clock shaper cell either to simply gate its incoming clock input or to divide it by a factor of  $N$  with a 50% duty cycle, while maintaining a constant insertion delay across all selected ratios. For more details about the clock shaper cell and its impact on the SSH SDC, refer to “[ScanHost](#)” in the *Tessent Shell Reference Manual*.

When you specify the clock shaper, the Tessent SDC provides two generated clocks for each clock shaper in the SSH: a gated clock and a divided clock, which enable accurate timing of the design.

You can simplify the clock definitions by specifying that the SSH generates only the `test_clock` source, which in turn feeds the external `edt_clock` and `shift_capture_clock` clock gaters. Refer to the figure “[ScanHost Node Sourcing test\\_clock With Following DFT Signal Clock Gater](#)” in the *ScanHost* reference page in the *Tessent Shell Reference Manual*. The following shows how the SDC creates the gated clock and divided clock from their master clock:

```
# Master clock
create_clock [tessent_get_ports ssn_bus_clock] \
  -name tessent_ssn_bus_clock_scan_slow_0 \
  -period $local_ssn_bus_clock_scan_slow_period -add
create_clock [tessent_get_ports ssn_bus_clock] \
  -name tessent_ssn_bus_clock_scan_fast_0 \
  -period $local_ssn_bus_clock_scan_fast_period -add
# SSH generated clocks
create_generated_clock [tessent_get_pins \
  <ssh_instance>/clock_gen/clock_signals/*_test_clock_shaper/clk_out] \
  -source [tessent_get_ports ssn_bus_clock] \
  -name tessent_ssh0_test_clock_gated \
  -master tessent_ssn_bus_clock_scan_slow_0 -add \
  -combinational \
  -divide_by 1
create_generated_clock [tessent_get_pins \
  ssh_instance/clock_gen/clock_signals/*_test_clock_shaper/clk_out] \
  -source [tessent_get_ports ssn_bus_clock] \
  -name tessent_ssh0_test_clock_div \
  -master tessent_ssn_bus_clock_scan_fast_0 -add \
  -combinational \
  -divide_by 2
```

The fact that a single clock source pin fans out to both scannable logic and EDT controller circuit branches makes clock tree synthesis (CTS) balance these two branches together by default. This helps maximize the shift speed of the scan mode. Although `edt_clock` and `shift_capture_clock` have separate clock gaters due to their functionality, they are considered to be on the same clock tree from an STA point of view. For that reason, when an SSH is configured to generate `test_clock`, STA needs only one pair of gated and divided clocks defined for that SSH.

It is also possible to simplify the SDC when SSH bypass mode is present and the test\_clock DFT signal is shared with the ssn\_bus\_clock port, by setting the SSH use\_ssn\_bus\_clock\_as\_test\_clock\_bypass property to “on”. In this case, the SSH hardware does not add a bypass mode multiplexer for test\_clock injection, and the SDC does not need to create a separate test\_clock, because both SSN and bypass clock trees are identical. The figure “[ScanHost Sourcing test\\_clock Reused for Legacy Bypass Mode](#)” in the ScanHost reference page in the *Tessent Shell Reference Manual* provides a detailed example.

To summarize, when you combine the following SSH property settings:

```
ScanHost (id) {  
    use_clock_shaper_cell : on;  
    scan_signal_bypass : on;  
    use_ssn_bus_clock_as_test_clock_bypass: on;  
    Interface {  
        ChainGroup {  
            test_clock_present : on;  
            shift_capture_clock_present : off;  
            edt_clock_present : off;  
        }  
    }  
}
```

it reduces the resulting SDC from defining six different generated clocks:

```
tessent_ssh*_{edt_clock|shift_capture_clock}_{gated | div | bypass}
```

to two clocks:

```
tessent_ssh*_test_clock_gated  
tessent_ssh*_test_clock_div
```

This eliminates a large number of set\_clock\_groups, set\_false\_path, and set\_multicycle\_path commands that refer to these defined clocks.

At the design implementation stage, you should use a library cell for the clock shaper. Do this by loading the cell library model of the clock shaper cell into Tessent Shell before running the process\_dft\_specification command.

The clock shaper RTL cell is useful for simulation, but you should not use it for design implementation. Using the clock shaper RTL cell at the design implementation stage could result in under constraint of the design and could simultaneously create false timing violations. If you want to use the clock shaper RTL cell during design implementation, you must characterize additional timing arcs for the clock multiplexer cell being used, and you must apply additional clock gating constraints to the design, which are not supplied in the Tessent SDC.

## Clock Tree Synthesis Strategy for SSN

The SSN bus and the SSN ScanHost (SSH) nodes are intended to have all their logic balanced together in CTS, including all registers and the buffer or multiplexer persistent cell that drives

each scan clock out of the SSH. Each buffer or multiplexer that drives a scan clock out of the SSH needs a CTS stop point. Siemens EDA defines a CTS stop point as a leaf cell to which the clock tree is balanced; refer to [Figure 9-40](#) on page 621.

All scan clocks driven out of a single SSH should start new clock trees and be balanced together in CTS, including `edt_clock`. Although `edt_clock` usually drives less logic than `shift_capture_clock`, it must be balanced with `shift_capture_clock`. If you do not balance the clocks together, it could cause large hold violations from the scan chains to the EDT logic that would need to be fixed in the physical design phase. SSH configurations that have both `shift_clock` and `capture_clock` must have those clocks balanced with `edt_clock`. SSH configurations that drive a single `test_clock` require only a new balanced CTS clock tree starting at the SSH persistent cell that drives `test_clock`.

A single SSH can control multiple functional clock domains. Each functional clock that is asynchronous to other functional clocks has its own on-chip clock controller (OCC). In most cases, you perform CTS on the functional clocks first at the output of each OCC. The resulting clock trees driven by each OCC usually have different insertion delays from each other. To address this, add any necessary CTS buffers upstream from the OCCs to balance all clock endpoints on `shift_capture_clock` driven from a single SSH.

Large designs may need pipeline stages from the ScanHost node to the EDT logic and from the EDT logic back to the ScanHost node. Such pipeline stages must have their clock insertion delays managed to deskew the clock insertion delay between the source and destination clocks in steps. For more information about using pipelines with the SSH and EDT, refer to the `DftSpecification/EDT/Controller/Connections` wrapper in the *Tessent Shell Reference Manual*.

If you use an SSN FIFO to minimize the loop timing delay of the SSN input clock to the data driven out to the tester, that FIFO requires a CTS exclude point exception on the clock sub-tree that communicates with the SSN ports. The location of the exclude point depends on whether the FIFO is on the input or output of the datapath:

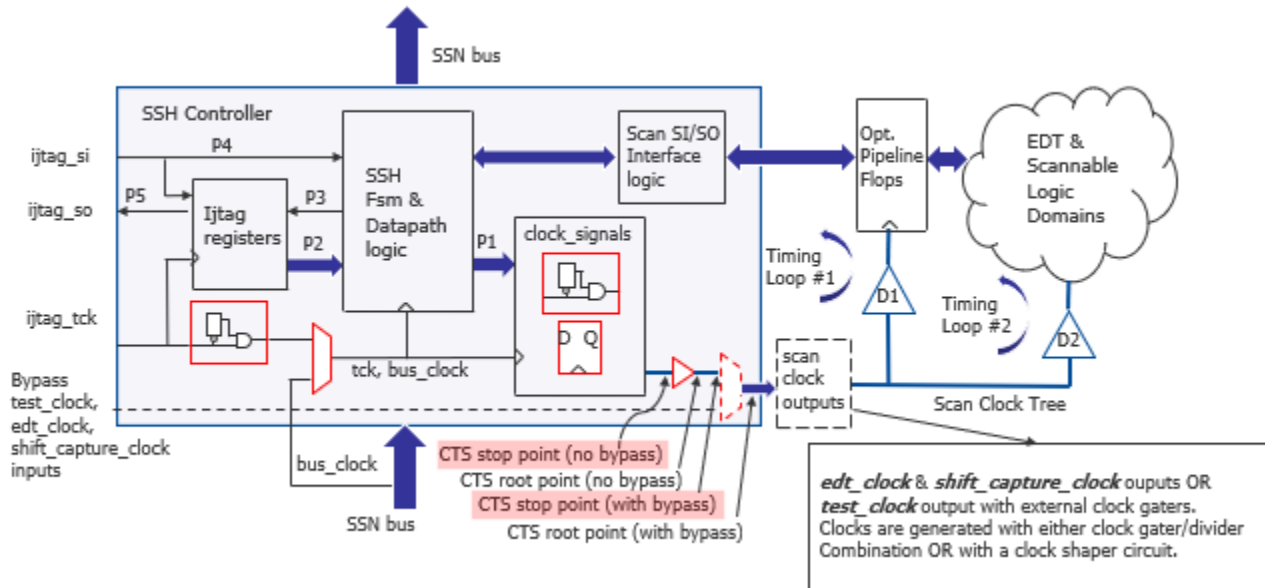
- An SSN FIFO on the input of the datapath would have a CTS exclude point on its write-side clock pin.
- An SSN FIFO on the output of the datapath would have a CTS exclude point on its read-side clock pin.

Siemens EDA defines a CTS exclude point to be a pin where CTS clock balancing does not happen at that point or at any point downstream in the clock tree from that exclude point. The exclude point should still be treated as a clock pin for design rule fixing. Tessent tools also write out CTS exclude points in their SDC files for logic internal to the OCC that generates clock gating signals. Refer to your place and route tool documentation to determine which CTS exception command matches the description of a CTS exclude point provided in this section, and use that command as appropriate with your tool.



## CTS Exceptions for the SSH

Figure 9-40. Recommended SSH CTS Stop Points



The preceding figure shows where Siemens EDA recommends specifying CTS instructions for balancing stop points. If you omit these stop points from your CTS script, the tool interprets this script as indicating that it must balance all of your scannable functional domains along with your SSN network bus clock tree. This can lead to the following notable timing closure issues during layout, among others:

- A large increase in the total SSN bus clock tree latency. This creates excessive SSN clock skew between neighboring tile blocks, your top-level SSN data bus ports, or both. This, in turn, demands either large BusFrequencyMultiplier/BusFrequencyDivider nodes or deep SSN FIFO nodes and could also potentially impact your maximum SSN bus clock frequency.
- Placement of your SSH clock generation subcircuit in the SSH clock\_signal block in Figure 9-40 at the very root of the clock tree generated by CTS, instead of at the leaf cells in the fsm and datapath block. This, in turn, results in a large reduction in your P1 timing path setup margin, by the amount of the clock tree propagation D2. This can make it impossible to meet timing, even for large functional trees or slow clock frequencies. You cannot add a pipelining flop to the path, because this breaks the SSN scan protocol.
- Increased IJTAG clock network latency, because the ijtag\_clock tree would come through the scannable logic to the clock\_signals logic.

If you specify the recommended CTS stop points, it makes the scan clock tree delay (D2) part of the chain SI/SO timing paths in a predictable manner. This might force you to insert one or more rows of Tessent pipelining flops along your EDT channels. The design of the SSH Scan SI/SO Interface already handles a minimal level of skew, where all to\_scan\_in paths are retimed

and all from\_scan\_out timing paths are full-cycle paths of the slow clock. The from\_scan\_out circuit requires no retiming, because it includes a single-direction clock timing loop, as shown in the figure. Timing Loop #2 includes the potentially very large D2 delay, which the pipelining flops can reduce to the D1 value. The number of required pipelining flops depends on both the size of your scannable domain and your target scan frequency. A large number requires more effort during physical design to select the correct clock tree tapping point for each flop layer. Automation has the potential to reduce these efforts, as does reducing the scan domain size by multiplying the number of SSH instances at the DFT planning stage.

No other CTS constraints are required for the SSH bus clock and the ijtag\_tck clocks. Tessent SDC declares these clocks as physically exclusive, so the CTS process does not need to balance them together. However, the ijtag\_tck feeds both the ijtag\_registers block, which contains standard TDR setup logic, and the fsm and datapath block, which contains the SSH scan mode control logic. This enables timing tck paths between the two blocks. The clock path to the fsm and datapath block is enabled only during streaming\_through\_ijtag or bus\_register access modes. When these modes are active, timing paths P3, P4, and P5 must meet timing in 0.5 tck cycles, while P2 paths are static control signals that Tessent SDC declares as false. All other tck timing paths within the fsm/datapath logic are more tightly constrained by the faster SSN bus clock. CTS balances these two tck branches by default in the absence of specific guidance, which improves the maximum IJTAG network speed without placing too much of a penalty on the total tck insertion delay.

In the output SDC file, the extract\_sdc command writes a Tcl proc named tessent\_get\_cts\_skew\_groups\_dict, which returns a dictionary of the CTS exceptions for clock skew groups across all SSH and OCC instances in your design. You can use that dictionary with a custom script that issues CTS commands specific to your layout tool. A CTS root point in [Figure 9-40](#) on page 621 shows where a new CTS clock tree should begin. Clocks starting at CTS root points grouped under the same SSH in the tessent\_get\_cts\_skew\_groups\_dict proc

should be balanced together. The header comment for the proc contains instructions on how to use it during CTS. The following is an example:

```
proc tessent_get_cts_skew_groups_dict {} {

    # This proc returns a dictionary of clock source pins from where clock
    # tree synthesis balancing should stop.
    # Use it in your CTS script, along with your proper tool command.
    # In Synopsys ICC, invoke the following:
    #     set_clock_tree_exceptions -exclude_pins <exclude_pin>
    # In Cadence Innovus, invoke the following:
    #     create_ccopt_skew_group -sources <exclude_pin> -auto_sinks
    #     -skew_group <group name>
    # The effect of those commands is:
    # * to prevent adding delay buffers to the small OCC internal clock tree,
    #   due to balancing with all flops in the OCC fanout, therefore
    #   helping the OCC clock_enable signals meet setup timing.
    # * to prevent adding long delays to the SSN clock tree due to balancing
    #   with the functional clock tree in the fanout of each SSN scan host
    #   (SSH).
    #
    #
    # You can use the dictionary the following way:
    #     set_cts_skew_groups_dict [tessent_get_cts_skew_groups_dict]
    #     dict for {skew_group sub_dict} $cts_skew_groups_dict {
    #         dict with sub_dict {
    #             foreach pin $cts_exclude_pins {
    #                 puts "$skew_group : $dc_instance/$pin"
    #                 <insert your CTS command here>
    #             }
    #         }
    #     }
    #
    set return_dict {
        cts_skew_group(occ0) {
            dc_instance      top_rtl2_tessent_occ_parent_clk_a_inst
            cts_exclude_pins {occ_control/
                tessent_persistent_cell_ltest_ntc_sync_cell/CK occ_control/
                tessent_persistent_cell_cgc_SHIFT_REG_CLK/CK occ_control/
                tessent_persistent_cell_SHIFT_REG_CLK_mux/A1}
        }
        cts_skew_group(occ1) {
            dc_instance      top_rtl2_tessent_occ_parent_clk_b_inst
            cts_exclude_pins {occ_control/
                tessent_persistent_cell_ltest_ntc_sync_cell/CK occ_control/
                tessent_persistent_cell_cgc_SHIFT_REG_CLK/CK occ_control/
                tessent_persistent_cell_SHIFT_REG_CLK_mux/A1}
        }
        ...
        cts_skew_group(ssh0) {
            dc_instance top_rtl2_tessent_ssn_scan_host_1_inst
            cts_stop_pins {
                clock_gen/clock_signals/
                tessent_persistent_cell_edt_clock_buf/A
                clock_gen/clock_signals/
                tessent_persistent_cell_shift_capture_clock_buf/A } }
            cts_root_pins {
                clock_gen/clock_signals/
```

```
        tessent_persistent_cell_edt_clock_buf/Y
clock_gen/clock_signals/
        tessent_persistent_cell_shift_capture_clock_buf/Y} }
    }
}
return $return_dict
}
```

# Fast IO

SSN Fast IO is an SSN configuration in which the top-level external input and output ports of the SSN run faster than the internal SSN bus.

These input and output ports are single-ended or DDR I/O ports, not SerDes or differential I/O ports. These I/O ports can reliably run at 200 MHz and higher speeds. When you implement an SSN Fast IO datapath, you place a [BusFrequencyDivider](#) (BFD) and [BusFrequencyMultiplier](#) (BFM) along with Fifo nodes at either end of the SSN. These nodes collectively retime the data between the fast external interface and the slower internal SSN bus.

Note

 The I/O ports that make up the fast external interface to the SSN must be part of the design before you implement SSN Fast IO. The Tessent implementation does not supply these ports.

The following topics describe how to implement and use SSN Fast IO functionality:

|                                              |            |
|----------------------------------------------|------------|
| <b>Design Requirements</b>                   | <b>625</b> |
| <b>SSN Fast IO Functionality</b>             | <b>627</b> |
| Single Data Rate Clocking                    | 627        |
| Double Data Rate Clocking                    | 628        |
| <b>Modeling a Double Data Rate Interface</b> | <b>630</b> |
| ICL Modeling                                 | 630        |
| Verilog Modeling                             | 634        |
| <b>Adaptive Tuning on the ATE</b>            | <b>636</b> |
| ATE Calibration                              | 637        |
| ATE Input Delay Tuning                       | 638        |
| Output Strobe Tuning                         | 638        |
| <b>SSN Fast IO Bandwidth Improvement</b>     | <b>639</b> |
| <b>Fast IO Pattern Generation</b>            | <b>640</b> |
| <b>SSN Fast IO Limitations</b>               | <b>640</b> |

## Design Requirements

To implement SSN Fast IO, you must have a design with a single data rate or double data rate digital interface that can be reused as the external interface to the SSN.

The interface should be capable of running faster than standard GPIO, which typically is limited to a maximum operating frequency of 200 MHz. You can use a single clock as the source for both input and output of the SSN. You can also use an optional second clock as one of the SSN clock sources, as shown in [Figure 9-41](#) on page 628.

Furthermore, you can also use a slower external interface to share access to the SSN. This requires an equal number of data inputs and outputs to be used as the SSN `bus_data_in` and `bus_data_out`. It also requires a single slower clock as the clock source to the SSN when you use the slower external interface. A multiplexer, `slow_fast_mux`, enables the slower external interface to share access to the SSN, as shown in [Figure 9-42](#) on page 628.

## SSN Fast IO Functionality

SSN Fast IO functionality uses enhanced BFD and BFM node definitions that create internal clocks for the SSN, using the fast external clock source.

Creating a divided clock from the BFD on the SSN datapath input provides a clock that matches the slower internal data rate. Likewise, the clock created from the BFM retimes the data from the slower internal bus back to the faster external interface on the SSN datapath output. Refer to [Figure 9-41](#) on page 628 to see the placement of the BFD and BFM nodes and the clock distribution of each. The figure also shows how the FIFO instances on either end of the SSN use two separate copies of `ssn_bus_clock`. One copy of the `ssn_bus_clock` is created from the BFD and a second copy of the `ssn_bus_clock` is created from the BFM. The clock output from the BFD is the source clock to the `FIFOin` and the internal SSN. The clock output from the BFM is the source clock to only the read side of the `FIFOout`. This creates a loop timing path between the `FIFOout` and the BFM. Both FIFO instances ensure that data reliably transfers between the two versions of `ssn_bus_clock`.

**Single Data Rate Clocking** ..... 627

**Double Data Rate Clocking**..... 628

### Single Data Rate Clocking

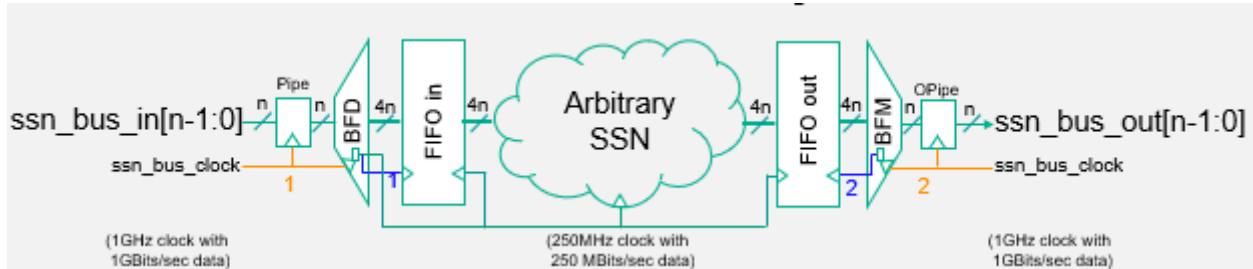
When you use single data rate (SDR) clocking, data is driven into the SSN from the I/O pads using a single edge of the input clock.

On the input side of the SSN, a fast external interface drives the BFD through input pipeline stages, as shown in [Figure 9-41](#). The BFD slows the input data rate by the frequency ratio  $N$  and widens it by the same ratio. The tool creates two separate clocks from the BFD node, illustrated by the blue 1. Both clocks are created from the fast external input clock. The two clocks are copies of each other and come from the same source in the BFD. One clock output, `bus_clock_out_local`, is the write clock to `FIFOin`, and the other clock, `bus_clock_out`, is the read clock for `FIFOin`. You can also use `bus_clock_out` as the clock source for the slower, wider internal SSN and the write clock to `FIFOout`, which the figure also illustrates. For more information about the clocks created by the BFD and how to use them with SSN Fast IO, refer to “[BusFrequencyDivider](#)” in the *Tessent Shell Reference Manual*.

The BFM, on the output side of the SSN, is the interface from the slower, wider SSN to the faster external interface, as in the figure. The BFM speeds up the internal data rate of the SSN by the frequency ratio  $N$  and narrows it by the same ratio. The `FIFOout` retimes the data coming from the slower internal SSN to the BFM that runs at the faster external interface frequency. The output clock of the BFM is created from the fast external clock input and creates a loop timing path between the `FIFOout` and the BFM, allowing a full clock cycle of the fast clock for the data to propagate to the BFM. The data returns to the tester through pipeline stages that are also running at the faster external frequency. For more information about the clocks created by

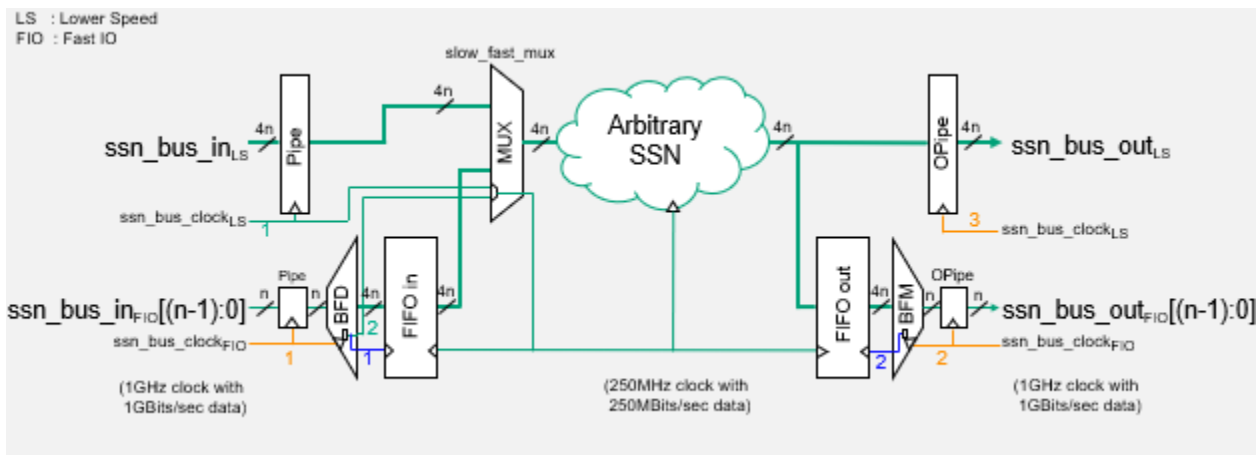
the BFM and how to use them with SSN Fast IO, refer to “[BusFrequencyMultiplier](#)” in the *Tessent Shell Reference Manual*.

**Figure 9-41. Example SSN Fast IO Bus With SDR Clcking**



As an alternative to the previous configuration (where the fast external interface is the only access to the SSN), you can add a `slow_fast_mux`, as shown in the following figure, to share access to the SSN through a slower external interface. The multiplexer enables you to share access to the SSN between the two interfaces. The `slow_fast_mux` enables you to develop the SSN and create patterns using the slower external interface while you work on the access through the faster external interface. You can remove the `slow_fast_mux` once you are satisfied with the faster external interface as the only access mechanism to the SSN.

**Figure 9-42. Example SSN Fast IO Bus With SDR Clcking and Alternate External Interface Access to the SSN**



## Double Data Rate Clcking

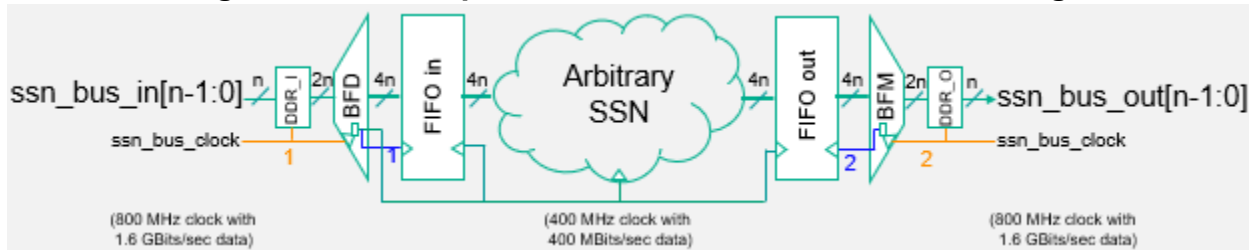
SSN Fast IO with double data rate (DDR) clcking functions similarly to SSN Fast IO with SDR clcking. The primary difference is that the DDR external interface provides 2× the data rate of the SDR external interface.

When you use SSN Fast IO with DDR clcking, input data is sampled using both the leading and falling edges of the clock. Similarly, the SSN output data bus is strobed using both the leading and falling edges of the clock.



The following figure shows an example of SSN Fast IO with DDR clocking. When you use the DDR external interface, the input and output pipelines on the SSN are optimal due to the nature of the DDR clocking scheme.

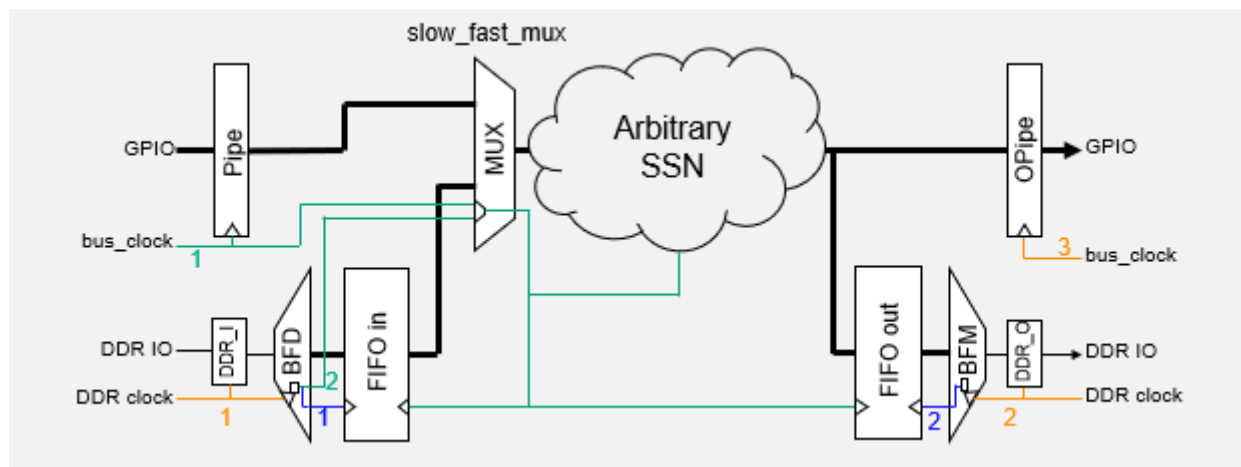
**Figure 9-43. Example SSN Fast IO Bus With DDR Clocking**



When you use the DDR external interface, the clocks generated from the BFD and BFM are still required, as described in “[Single Data Rate Clocking](#)” on page 627.

As with SDR clocking, you can add an alternate external interface to the SSN Fast IO with DDR clocking configuration. The alternate external interface can access the SSN through a `slow_fast_mux`, shown in the following figure. This mux enables you to develop the SSN and create patterns while you work on the access through the faster external interface. You can remove the `slow_fast_mux` once you are satisfied with the DDR interface as the only access mechanism to the SSN.

**Figure 9-44. Example SSN Fast IO Bus With DDR Clocking and Alternate External Interface Access to the SSN**



## Modeling a Double Data Rate Interface

An ICL representation of the DDR interface is required to model the DDR clock and the delay of the data through the DDR macro when you use SSN Fast IO with DDR clocking. Additionally, you can model DDR inputs and outputs using Verilog RTL.

A cycle-accurate simulation model is required for the DDR interface. This model is used during validation of the SSN pattern.

When you use the DDR external interface, the clock is defined relative to the data. The data rate is one bit per edge of the clock. On the rising edge of the clock, one data bit is strobed. On the falling edge of the clock, a different data bit is strobed. Define the clock so that one bit of data is strobed for each edge of the clock.

Defining the clock with a frequency divider of two (half of the data rate) means that each edge of the clock strobes data:

**add\_clocks ssn\_ddr\_clock\_name -freq\_divider 2**

For example, if the DDR clock period is defined as 10 ns, the data is strobed every 5 ns: once on each clock edge, rising and falling.

When you use the PatternsSpecification to create DDR verification patterns, define the DDR clock with the same frequency division, as in the following example:

```
PatternsSpecification(...) {  
  Patterns(...) {  
    ClockPeriods {  
      <ssn_ddr_clock_name> : tester, 0.5 ;  
    }  
  }  
}
```

Refer to the PatternsSpecification reference topic and its subtopic Patterns in the “Configuration-Based Specification” chapter of the *Tessent Shell Reference Manual* for more information about using a PatternsSpecification.

The following topics provide specific information about modeling the interface in various languages:

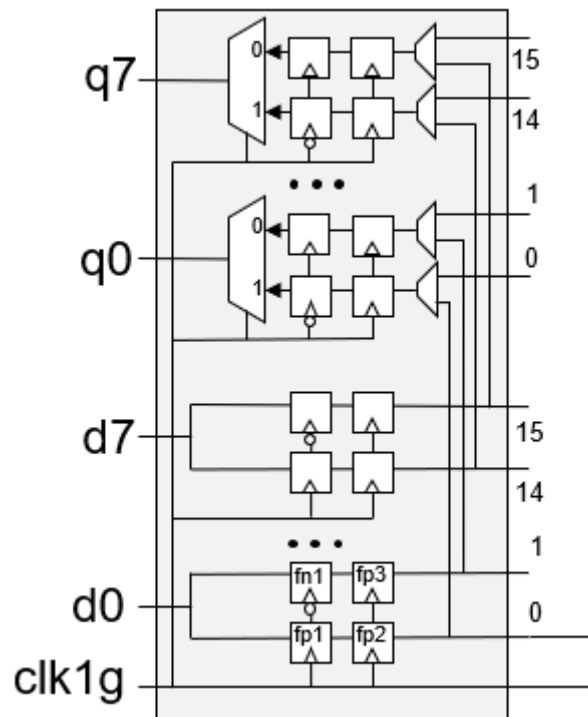
|                               |            |
|-------------------------------|------------|
| <b>ICL Modeling</b> .....     | <b>630</b> |
| <b>Verilog Modeling</b> ..... | <b>634</b> |

## ICL Modeling

The following sections show examples of how to model the DDR input and DDR output in ICL. New ICL attributes are required for the DDR ICL models. These attributes are highlighted in these examples.

These ICL models describe the following schematic :

### Figure 9-45. Sample DDR Interface Schematic



## DDR Input Model

The DDR input ICL model has the following properties:

- The DataInPort has “Attribute tessent\_ssn\_function = "ddr\_bus\_data\_input" ;”.
- This attribute distinguishes a DDR input cell from other SSN logic in which one input has fanout to several DataRegisters.
- The di\_pos\_ret DataRegister has “Attribute tessent\_active\_clock\_edge = "rising" ;”, and di\_neg\_ret has “Attribute tessent\_active\_clock\_edge = "falling" ;”.

These two registers determine the destination bus bit for the stimulus values provided to the primary input during both of the following periods:

- Between the rising and falling edges of the SSN DDR clock.
- Between the falling and rising edges of the SSN DDR clock.
- The `do_ret` DataRegister spans all outputs and aligns the incoming data to the same clock edge. This DataRegister does not necessarily have to be part of the DDR input ICL. For example, it can be the register of the first pipeline. It is only necessary that it must use the same clock.

The following is an example of the input ICL model:

```
Module ddr_in {
  ClockPort clk {
    Attribute function_modifier = "tessent_ssn_clock";
    Attribute forced_high_dft_signal_list = "ssn_en";
  }
  ToClockPort clk_out {
    Attribute function_modifier = "tessent_ssn_clock";
    Source clk;
  }
  DataInPort di {
    Attribute function_modifier = "tessent_ssn_bus_data";
    Attribute tessent_ssn_function = "ddr_bus_data_input";
    Attribute forced_high_dft_signal_list = "ssn_en";
  }
  DataOutPort do[1:0] {
    Attribute function_modifier = "tessent_ssn_bus_data";
    Attribute forced_high_dft_signal_list = "ssn_en";
    Source do_ret[1:0];
  }
  DataRegister do_ret[1:0] {
    WriteDataSource di_neg_ret, di_pos_ret;
    WriteEnSource 1'b1;
  }
  DataRegister di_neg_ret {
    Attribute tessent_active_clock_edge = "falling";
    WriteDataSource di;
    WriteEnSource 1'b1;
  }
  DataRegister di_pos_ret {
    Attribute tessent_active_clock_edge = "rising";
    WriteDataSource di;
    WriteEnSource 1'b1;
  }
}
```

## DDR Output Model

The DDR output ICL model has the following properties:

- The DataOutPort has “Attribute tessent\_ssn\_function = "ddr\_bus\_data\_output" ;”.

This attribute is required to identify the DDR output without ambiguity and to apply checks verifying that the output is correctly set up.

- A DataMux selects between two inputs and is controlled by an instance output of a special module, as described in the following example.

This multiplexer selects between the source of the value that can be observed at the primary output between the rising and falling edges of the SSN DDR clock and the source of the value that can be observed at the same primary output between the falling and rising edges of the SSN DDR clock.

- A special module has “Attribute tessent\_ssn\_function = "ddr\_out\_selector" ;”, along with a ClockPort and a DataOutPort that converts the SSN clock into a data signal that can be used as a mux select input.

This module is required to specify to the tool which SSN clock is associated with the multiplexer in the DDR output cell. ICL syntax does not allow the SSN clock to control the multiplexer directly, so this special module converts the clock signal to a data signal.

The following is an example of the output ICL model:

```
Module ddr_out {
  ClockPort clk {
    Attribute function_modifier = "tessent_ssn_clock";
    Attribute forced_high_dft_signal_list = "ssn_en";
  }
  ToClockPort clk_out {
    Attribute function_modifier = "tessent_ssn_clock";
    Source clk;
  }
  DataInPort di[1:0] {
    Attribute function_modifier = "tessent_ssn_bus_data";
    Attribute forced_high_dft_signal_list = "ssn_en";
  }
  DataOutPort do {
    Attribute function_modifier = "tessent_ssn_bus_data";
    Attribute tessent_ssn_function = "ddr_bus_data_output";
    Attribute forced_high_dft_signal_list = "ssn_en";
    Source ddr_mux;
  }
  DataRegister di_ret[1:0] {
    WriteDataSource di[1:0];
    WriteEnSource 1'b1;
  }
  DataRegister di_ret1 {
    WriteDataSource di_ret[1];
    WriteEnSource 1'b1;
  }
  DataRegister di_ret0 {
    Attribute tessent_active_clock_edge = "falling";
    WriteDataSource di_ret[0];
    WriteEnSource 1'b1;
  }
  Instance do_select Of ddr_out_do_select {
    InputPort in = clk;
  }
  DataMux ddr_mux SelectedBy do_select.out {
    1'b0 : di_ret1;
    1'b1 : di_ret0;
  }
}
Module ddr_out_do_select {
  Attribute tessent_ssn_function = "ddr_out_selector";
  ClockPort in {
    Attribute function_modifier = "tessent_ssn_clock";
  }
  DataOutPort out;
}
```

## Verilog Modeling

The following sections show examples of how to model the DDR input and DDR output using Verilog HDL. Use these examples to help create your simulation models if needed.

## DDR Inputs

```
module ddr_in (clk, di, do, clk_out);
    input clk;
    input di;
    output reg [1:0] do;
    output clk_out;

    reg di_neg_ret, di_pos_ret;
    always @ (negedge clk) begin
        di_neg_ret <= di;
    end
    always @ (posedge clk) begin
        di_pos_ret <= di;
        do <= {di_neg_ret, di_pos_ret};
    end
    assign clk_out = clk;

endmodule
```

## DDR Outputs

```
module ddr_out (clk, di, do, clk_out);
    input clk;
    input [1:0] di;
    output do;
    output clk_out;

    reg [1:0] di_ret;
    reg      di_ret1, di_ret0;
    wire      select;

    always @ (posedge clk) begin
        di_ret <= di;
        di_ret1 <= di_ret[1];
    end
    always @ (negedge clk) begin
        di_ret0 <= di_ret[0];
    end
    assign #0 select = clk;
    assign do = (select) ? di_ret0 : di_ret1;
    assign clk_out = clk;

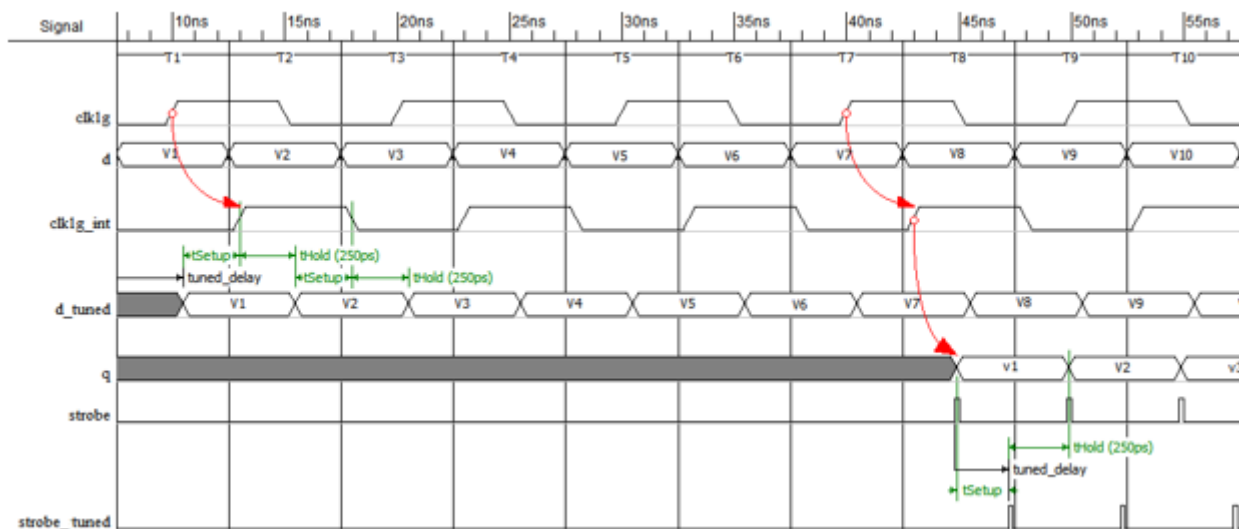
endmodule
```

## Adaptive Tuning on the ATE

When you use SSN Fast IO to test your chip, you may need to calibrate the ATE to reliably test your chip.

You may need to delay the output strobe on the ATE to match the delay of the data coming from the chip. Similarly, you may need to delay the input stimulus from the ATE to account for clock latency within the chip. As clock frequencies increase, the delays within each chip may vary across process corners, taking up more of the clock period and requiring each chip to be individually calibrated to test reliably. The following figure shows an example of properly tuning the input delay, `d_tuned`, and the output strobe, `strobe_tuned`, to account for delays within a chip.

**Figure 9-46. Proper Tuning of Input Data and the Output Strobe**



The waveform of `d-V1` at time zero shows the data perfectly centered around the rising edge of the clock `clk1g`, which provides a sufficient setup and hold margin. Both are shown to be applied at time zero without considering the delays inside the chip. However, the waveform for `clk1g_int` in cycle T2 shows a roughly 400 ps delay before the first rising edge appears at the internal clock pin. To reliably strobe the input stimulus, the input data from the ATE must be delayed to account for the delay on the clock net. When it is not delayed and forced at time zero, as is shown in the preceding figure, there is not enough setup and hold margin to be strobed by `clk1g_int`. Therefore, the input `d-V2` is mistakenly strobed.

As a result, to sample the `d-V1` input data with sufficient setup and hold margin, it must be delayed to account for the delay on the clock net inside the chip. The waveform `d_tuned-V1` shows how the ATE can provide a delayed version of the input data that is properly centered around the rising edge of `clk1g_int`, providing sufficient setup and hold margin.

The output strobe is more commonly required to be tuned even at lower frequencies, to account for the delays internal to the chip, because those delays translate into the data appearing on the output later in the cycle compared to ideal timing. In [Figure 9-46](#), the strobe is placed at the



falling edge of the clock clk1g, as shown in cycle T8. This strobe placement is based on timing of the clock being applied at time zero without considering the delays inside the chip.

The waveform of q-V1 shows the actual timing of the output based on the delay of the data before it appears on the output. The “strobe” waveform shows the output strobe can be moved to later in the cycle T8 while still providing sufficient setup and hold margin on q-V1. The tuned placement of the output strobe is shown in the waveform strobe\_tuned.

The previous figure and accompanying description demonstrate the need to perform adaptive tuning to test your chip reliably when you use SSN Fast IO. Refer to the section “[ATE Calibration](#)” on page 637 for the process of calibrating the tester to adapt to the internal delays of your chip using the SSN continuity pattern.

The following topics provide more information about adaptive tuning:

|                               |            |
|-------------------------------|------------|
| <b>ATE Calibration</b>        | <b>637</b> |
| <b>ATE Input Delay Tuning</b> | <b>638</b> |
| <b>Output Strobe Tuning</b>   | <b>638</b> |

## ATE Calibration

Use the SSN continuity pattern to calibrate the tester to the delays in your chip.

To calibrate the ATE, start by slowing the pattern by a factor of four and setting the output strobe point just before the end of the tester cycle. Slowing the pattern to this degree enables you to have a reliable output strobe and a slow enough clock to strobe the input data reliably without any delay from the tester.

When you test your device at a frequency of 200 MHz or higher, you must find a reliable output strobe. When your external clock frequency is 400 MHz or higher, you may also need to tune the input data from the tester to account for internal clock delays across all PVT conditions. Testing your part without properly calibrating the tester may result in yield loss.

Use the following equations to calculate the clock period and the location of the output strobe. In these equations, T is the clock period (ps) of the SDR or DDR clock defined for the SSN continuity pattern:

**Equation 1: Clock Period for the SSN Continuity Pattern When Calibrating the ATE**

Clock period (calibration) =  $T \times 4$

**Equation 2: Location of the Output Strobe, in Picoseconds, When Calibrating the ATE**

Strobe point (calibration) =  $(T \times 4) - 1 \text{ ps}$

## ATE Input Delay Tuning

Tune the tester input delay by increasing the delay until the output shows failures.

### Note



This section uses the DDR clock waveform in [Figure 9-46](#) on page 636 for reference.

---

With the SSN continuity pattern running at a 4× slower speed, increase the delay on d-V1 from the tester until the output shows failures. The delay added to d-V1 moves the data change edge of d-V1 toward the rising edge of the clock `clk1g_int`, causing a setup violation, which results in failures on the SSN bus output.

This is the amount of delay that causes the pattern to stop working and is the same for any clock frequency. Use this delay to calculate the amount of delay needed by the tester before applying the input data d-V1. Use the following equation to calculate this delay:

### Equation 3: Input Delay To Be Applied by the Tester

Input Delay (tuned) = delay - 750 ps

Subtracting 750 ps from the delay puts the data eye of d-V1 into the timing window T2, around the rising edge of `clk1g_int`, and it provides 250 ps of hold margin, as shown in `d_tuned` in the waveform. Similarly, the delay puts the data eye of d-V2 into the timing window T3 and around the falling edge of `clk1g_int`, with the same hold margin. This is also shown in `d_tuned` in the waveform.

## Output Strobe Tuning

Tune the output strobe by delaying it to later in the cycle until the strobe no longer works.

### Note



This section uses the DDR clock waveform in [Figure 9-46](#) on page 636 for reference.

---

Start with the clock period of the SSN continuity pattern at full rate. You must restore the clock to its maximum frequency before slowing it down to determine the input delay. If you have moved the output strobe, you must also return it to its original position. It should be in the middle of the data eye of q-V1 in the timing window T8. With the clock running at full frequency and with the input tuned to the delay of the clock, delay the output strobe by moving it toward the end of the T8 timing window and the data change edge of q-V1. Increase the delay until the strobe point no longer works, which occurs when you have moved it sufficiently close to the data change edge between q-V1 and q-V2. The point of failure occurs when the strobe violates the hold time of q-d1. This amount of delay causes the pattern to stop working. Use this delay to calculate the output strobe point, as shown in the following equation:

### Equation 4: Delay To Apply to Output Strobe

Output Strobe Delay (tuned) = delay - 250 ps

Subtracting 250 ps from the measured delay puts `strobe_tuned` toward the end of the data eye of q-V1. This provides a sufficient setup margin and 250 ps of hold margin, as shown in the figure.

## SSN Fast IO Bandwidth Improvement

Implementing SSN Fast IO can improve the bandwidth of your SSN as compared to an SSN with a slower external interface.

Table 9-7 shows how an SSN with a number  $N$  of I/O ports running at 800 MHz has  $4\times$  the bandwidth of the same SSN running at 200 MHz. For example, an SSN Fast IO with SDR clocking and 8 I/O ports running at 800 MHz yields 6.4 Gbps of bandwidth. The same SSN with an external interface running at 200 MHz yields only 1.6 Gbps. To achieve the equivalent bandwidth of the SSN running with the faster external interface, it requires  $4\times$  the number of I/O ports on the slower external interface. Thus, the slower external interface must increase to 32 I/O ports to achieve the same results.

### Tip

 Calculate SSN bandwidth by multiplying the number of I/O ports by their maximum operating frequency.

In some cases when a faster external interface with SDR clocking has fewer I/O ports than the number of slower external I/O ports, the Fast IO interface does not yield a higher bandwidth for the SSN. The following table shows that an SSN using SDR clocking with a slower external interface of 32 I/O ports yields 6.4 Gbps when running at 200 MHz. To achieve the same bandwidth with a faster external interface requires 16 I/O ports running at 400 MHz or eight I/O ports running at 800 MHz.

However, an SSN Fast IO can still provide value even if there is less throughput while testing a single part at a time. If the bandwidth of a Fast IO interface is less than that of the slower external interface, multi-site testing can improve the test time. For example, the following table shows you can test one SSN device with 32 I/O ports running at 200 MHz in one-fourth of the test time of the same SSN device with a faster external interface running at 800 MHz with two I/O ports.

**Table 9-7. SSN Bandwidth Comparison for I/O Port/Frequency Combinations**

| Number of I/O Ports | Data Rate (Gbps)<br>@ 800 MHz | Data Rate (Gbps)<br>@ 400 MHz | Data Rate (Gbps)<br>@ 200 MHz |
|---------------------|-------------------------------|-------------------------------|-------------------------------|
| 2                   | 1.6                           | 0.8                           | 0.4                           |
| 4                   | 3.2                           | 1.6                           | 0.8                           |
| 8                   | 6.4                           | 3.2                           | 1.6                           |
| 12                  | 9.6                           | 4.8                           | 2.4                           |

**Table 9-7. SSN Bandwidth Comparison for I/O Port/Frequency Combinations**

| Number of I/O Ports | Data Rate (Gbps)<br>@ 800 MHz | Data Rate (Gbps)<br>@ 400 MHz | Data Rate (Gbps)<br>@ 200 MHz |
|---------------------|-------------------------------|-------------------------------|-------------------------------|
| 16                  | 12.8                          | 6.4                           | 3.2                           |
| 32                  | 25.6                          | 12.8                          | 6.4                           |

Testing four devices concurrently running at 800 MHz yields a combined throughput of 6.4 Gbps. If you use fewer ports for Fast IO than slower I/O ports, you may not see the benefits of using Fast IO functionality when testing individual devices. However, you may still see benefits when testing devices in parallel using multi-site testing.

## Fast IO Pattern Generation

The SSN pattern generation flow for SSN Fast IO is identical to the normal SSN pattern generation flow.

When you have more than one external interface, as in the figure in Example 4 in “[BusFrequencyDivider](#)” in the *Tessent Shell Reference Manual*, the default access mechanism is the I/O port group with the ID matching the datapath ID in the extracted ICL. Use the `-alternate_input_id <id>` and `-alternate_output_id <id>` arguments of the `set_ssn_datapath_options` to change the active external interface of the SSN.

## SSN Fast IO Limitations

Creation of SSN Fast IO patterns is subject to certain process limitations.

- An equal number of data inputs and outputs must be used as the SSN `bus_data_in` and `bus_data_out`.
- When you create SSN Fast IO patterns, you must use frequency-similar input and output ports of the same datapath.

For example, when you retarget SSN patterns, both the `bus_data_in` and `bus_data_out` must operate at the same frequency. You cannot mix a fast external interface with a slower external interface.

- The clock input of a BFD cannot be driven by the clock output of another BFD (for example, `bus_clock_out` or `bus_clock_out_local`).

Similarly, the clock input of a BFM cannot be driven by the clock input of another BFM (for example, `bus_clock_out`).

- Because of limitations in WGL for representing DDR patterns, DDR clocking does not support the WGL pattern format.

- The following Fast IO cases are not permitted:
  - DDR clocking with frequency-multiplied clocks.
  - DDR clocking with  $1\times$  clocks.
  - SDR input with DDR outputs.

# Burn-In Pattern Support

The Tessent Streaming Scan Network (SSN) supports creation of burn-in patterns. Burn-in patterns are a silicon aging technique that enable you to make predictions about how the silicon you are designing ages with use. They are commonly used during high-temperature operating life (HTOL) testing.

**Burn-In Pattern Overview** ..... 642

**Recommended Flows for Creating Burn-In Patterns** ..... 644

    Creating Burn-In Patterns With the Retargeting Flow ..... 644

    Creating Burn-In Patterns With the Flat ATPG Flow ..... 645

**Burn-In Functionality Limitations**..... 646

## Burn-In Pattern Overview

Burn-in pattern support for SSN uses a specially designed burn-in mode that you can configure to generate patterns that meet your needs.

Historically, the burn-in process has used EDT to drive pseudorandom patterns into chains. You would toggle the `edt_update` signal low for the duration, and drive EDT channels and pulse the clocks like `edt_clock` and `shift_clock`. Custom observability for burn-in used “proof of life” (activity) with a counter or a path to the edge of the design through the GPIO.

During generation of burn-in patterns with SSN, the tool configures the SSH hardware into “infinite shift” mode. In this mode, capture is disabled, and the burn-in payload loops without an `ssn_end` procedure.

The burn-in payload can target all SSHs (the default) or one or more SSHs. The packet is serially distributed along the SSN bus. The packet size calculation is based on the number of input channels of each active EDT. Each EDT has its own time slots in the packet. There is no data throttling during the application of the burn-in patterns. Unlike normal SSN operation, the output response is not added back into the packet for observation at the SSN outputs.

The default payload sequence for burn-in is “1100”: the first two packets drive all bus inputs to 1, and the next two drive all bus inputs to 0. This triggers pseudorandom data out of the EDT and is compatible with uncompressed chains.

To configure the burn-in pattern, use the [set\\_burnin\\_options](#) command, which controls the target SSH instance or instances, the payload sequence, and the number of repetitions of the sequence in the pattern.

**Note**

 Because low-power shift hardware interferes with the operation of burn-in patterns, you must disable any low-power shift hardware in your design to use the burn-in pattern functionality.

## Handling EDT Channel Pipeline Stages

To ensure proper handling of EDT pipeline stages during burn-in, `edt_clock` is the default clock source for pipelines when added to both the input and output EDT channels. When the EDT is equipped with `edt_bypass`, `edt_clock` and `edt_update` are combined with additional logic to ensure the EDT channel pipeline stages are properly controlled during ATPG and initialized for simulation. During EDT bypass, EDT channel pipeline stages are treated as scan cells. When present on either input and output EDT channels, the pipeline stages are equipped with hold logic to ensure each pipeline stages holds its current value during the `load_unload` procedure. Reset logic is added to each input pipeline stage to ensure known values during simulation.

When you bypass both the EDT and SSH and use tool automation of `add_dft_signals` to create `shift_capture_clock` and `edt_clock` from `test_clock`, the tool ensures `edt_clock` is correctly controlled. The pulsing of `test_clock` results in pulsing of the `edt_clock` during the `load_unload` and shift procedures. If you do not use automation of the `add_dft_signals` in this case, you must manually define the `edt_clock` on an independent top-level port and manually constrain it off (to 0) to avoid scan chain blockages and unknowns during simulation:

```
SETUP> add_clocks my_edt_clock  
SETUP> add_input_constraints my_edt_clock -C0
```

Changing the clock source for the EDT channel pipeline stages to `shift_capture_clock` and using `edt_bypass` mode results in DRC violations during ATPG. In addition, the pipeline stages are uninitialized for simulation, resulting in unknowns being driven into the EDT decompressor and scan chains.

## Recommended Flows for Creating Burn-In Patterns

There are two recommended flows for creating burn-in patterns: a retargeting flow for large designs, and a flat ATPG flow for designs running flat ATPG.

The retargeting flow follows the scan pattern retargeting approach. Because it requires a lower amount of machine memory than the flat ATPG flow, it is suited for large designs.

The flat ATPG flow uses the same setup that ATPG uses, simplifying the process to configure this flow.

|                                                                  |            |
|------------------------------------------------------------------|------------|
| <b>Creating Burn-In Patterns With the Retargeting Flow .....</b> | <b>644</b> |
| <b>Creating Burn-In Patterns With the Flat ATPG Flow .....</b>   | <b>645</b> |

## Creating Burn-In Patterns With the Retargeting Flow

Use the retargeting flow to create burn-in patterns for large designs. If you use a retargeting flow for scan patterns, you should also use this flow to create burn-in patterns. This flow is designed to minimize machine memory usage to accommodate such designs.

In addition to reducing memory usage, this flow also improves run time by using TCD files. The tool makes certain assumptions about the state of each core when loading TCD files and applies a simplified (reduced) set of DRCs. The full DRC process can take significantly longer for a full hierarchical design.

In normal (non-burn-in) SSN operation, when a core has at least one lower-level child core, the parent is in internal test mode, and the child cores are in external test mode, where only the wrapper chains of these child cores are active. However, when you create burn-in patterns for a parent-child core configuration with SSN, only the parent-level scan chains are active and the tool resolves the module instances for child cores using IJTAG grayboxes. The child cores are not configured into external mode in this case.

Only the IJTAG graybox design view is required for all design levels (including the top level) to create burn-in patterns following the retargeting flow. The IJTAG graybox design view is only a small fraction of the design size, and it is already a recommended part of the SSN flow.

---

**Tip**

 When you set up your burn-in patterns, use a mode name that includes a phrase like “burnin” so you can easily identify your burn-in patterns. Refer to Step 2.a for an example of this. You can use this mode name with the [set\\_current\\_mode](#) command.

---

## Prerequisites

- The IJTAG graybox view is highly recommended for the retargeting flow. Although you can still create burn-in patterns without them, doing so causes the flow to use significantly more memory and take much longer.



## Procedure

1. Generate a Tessent core description (TCD) file for each level of hierarchy, starting at the lowest level.
  - a. Create a burn-in TCD at the wrapped core level.

The setup for this TCD is identical to what you use for the “patterns -scan” context. It represents a core configured for burn-in. Treat any lower levels of hierarchy as IJTAG grayboxes (for example, “read\_design -view ijtag\_graybox”).
  - b. Configure the tool into internal mode. Add the core instances for the EDTs and OCCs using a command such as [read\\_core\\_descriptions](#) or [add\\_core\\_instances](#).
  - c. Check the design rules with the [check\\_design\\_rules](#) command.

The tool automatically adds the SSH core instance information.
  - d. Write the TCD out with the [write\\_tsdb\\_data](#) command.

This command uses the name (if any) you supplied with the [set\\_current\\_mode](#) command.
  - e. (Optional) Write out the testbench, using the [set\\_burnin\\_options](#) command, to verify the burn-in setup and sequence.
  - f. Repeat this process for each hierarchical level, including the top level.
2. Create a top-level burn-in pattern. The setup is identical to what you would use for pattern retargeting, including the context (“set\_context patterns -scan\_retargeting”).
  - a. Target the top level and any lower-level cores added by the TCD file. For example:  
**add\_core\_instances -instances {top corea\_i1 corea\_i2} -mode edt\_mode\_burnin**
  - b. Check the design rules again with the [check\\_design\\_rules](#) command.
  - c. Write out the testbench, using the [set\\_burnin\\_options](#) command, to verify the burn-in setup and sequence
  - d. (Optional) Configure the burn-in pattern setup with the [set\\_burnin\\_options](#) command.
  - e. Write the burn-in STIL pattern and testbench for verification.

## Creating Burn-In Patterns With the Flat ATPG Flow

Use the flat ATPG flow to create burn-in patterns for designs already running flat ATPG. The flat ATPG flow requires a full flat netlist and uses unwrapped scan mode.

### Prerequisites

- The flat ATPG burn-in flow uses the same setup that flat ATPG uses.

## Procedure

1. Configure the tool into internal mode.
2. Add the core instance for the EDTs and OCCs using a command such as [read\\_core\\_descriptions](#) or [add\\_core\\_instances](#).
3. Check the design rules with the [check\\_design\\_rules](#) command.  
The tool automatically adds the SSH core instance information.
4. (Optional) Configure the burn-in pattern setup with the [set\\_burnin\\_options](#) command.
5. Write the burn-in STIL pattern and testbench for verification.

## Burn-In Functionality Limitations

The following limitations apply to the Tessent burn-in functionality with SSN:

- Tessent burn-in functionality does not support observation of design activity during application of the pattern using SSN. You must supply your own device proof of life or activity.
- The verification testbench has limited functionality. A passing simulation does not confirm setup or payload presence; you must inspect for these manually.

## LVX Support

Tessent software includes the capability for laser voltage imaging (LVI) and laser voltage probing (LVP) functionality. Tessent LVI and LVP support is referred to collectively as “LVX” support.

The LVX capabilities of laser scanning microscopy platforms measure timing and functional characteristics of semiconductor devices. Their architecture usually supports docking with automated test equipment (ATE) and back-side analysis for localization of cell- and transistor-level faults and design marginalities.

Tessent LVX functionality includes the following:

- Packet-based delivery of any repeating sequence, such as “1100”, from the tester through the SSN network to all chains of an EDT (using EDT LVX hardware) or uncompressed chains.
- Broadcasting the sequence to all chains of an EDT or to a specific chain that you select on the tester.
- Looping a payload pattern set (after setup) that applies the repeating sequence on internal chains without interruption. You do not need to apply the `ssn_end` procedure between iterations, as ATPG patterns require.
- Diagnosis reporting of the failing chains with the full ICL instance pathname of the driving EDT instances and the chain indices.

|                                                            |            |
|------------------------------------------------------------|------------|
| <b>LVX Hardware Requirements</b>                           | <b>647</b> |
| <b>Creating EDT With LVX Hardware</b>                      | <b>648</b> |
| <b>LVX Operation Modes With LVX Hardware Configuration</b> | <b>653</b> |
| <b>Configuring LVX Patterns</b>                            | <b>654</b> |
| <b>Writing LVX Loop Patterns</b>                           | <b>666</b> |
| <b>Writing LVX Verification Patterns</b>                   | <b>667</b> |
| <b>LVI_PATCHING_INFO</b>                                   | <b>668</b> |

## LVX Hardware Requirements

Support for Tessent LVX requires the following hardware setup.

- SSH hardware generated with the 2021.2 release or a later release.
- EDT hardware generated with LVX enabled in the `DftSpecification`. The EDT hardware can have one of the following two configurations:
  - One-hot chain selection support (more hardware but reduced LVX noise).
  - Broadcast only to all chains driven by EDT (less hardware).

Tessent LVX minimizes EDT hardware overhead by reusing the existing hardware when this is possible.

## Creating EDT With LVX Hardware

Specify LVX settings within the EDT DftSpecification to generate EDT with LVX hardware.

### Procedure

Use the following wrappers and properties to generate the EDT with LVX hardware:

```
DftSpecification(module_name, id) {  
  EDT {  
    Controller {  
      LVxMode {  
        present: on | off | auto ;  
        enable_one_chain: on | off ;  
      }  
    }  
  }  
}
```

---

#### Note



If you specify enable\_one\_chain as “off” instead, the EDT can broadcast the LVX sequence only to the scan chains.

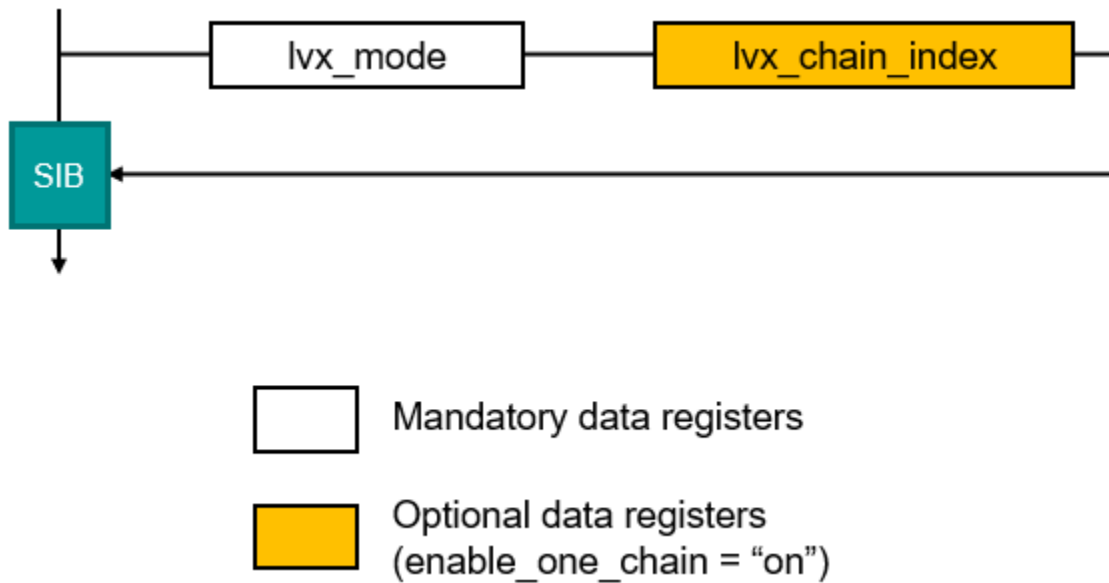
---

### Results

The DftSpecification results in an EDT with two new TDRs for controlling the LVX hardware:

- lvx\_mode
- lvx\_chain\_index

These TDRs are controlled behind a SIB, as shown in the following figure, and accessible through the IJTAG interface.

**Figure 9-47. Tessent LVX TDRs**

The PDL represents both of these TDRs with iWriteVars, so they are annotated in the pattern set and you can patch them on the tester.

With the LVX setup, the EDT controller has an IJTAG scan interface. The tool resets the EDT hold registers using the `ijtag_reset` signal.

#### Tip

**i** Maintain no more than one EDT or one uncompressed chain per SSH chain group.

If you do not put each EDT and uncompressed chain into its own chain group, this results in the tool adding padding to the packet. This, in turn, increases the number of shifts between each EDT/chain in the `lvx_loop` sequence. Adding each EDT and uncompressed chain into its own SSH chain group results in a more efficient packet for applying the `lvx_loop` to all the EDT/chains.

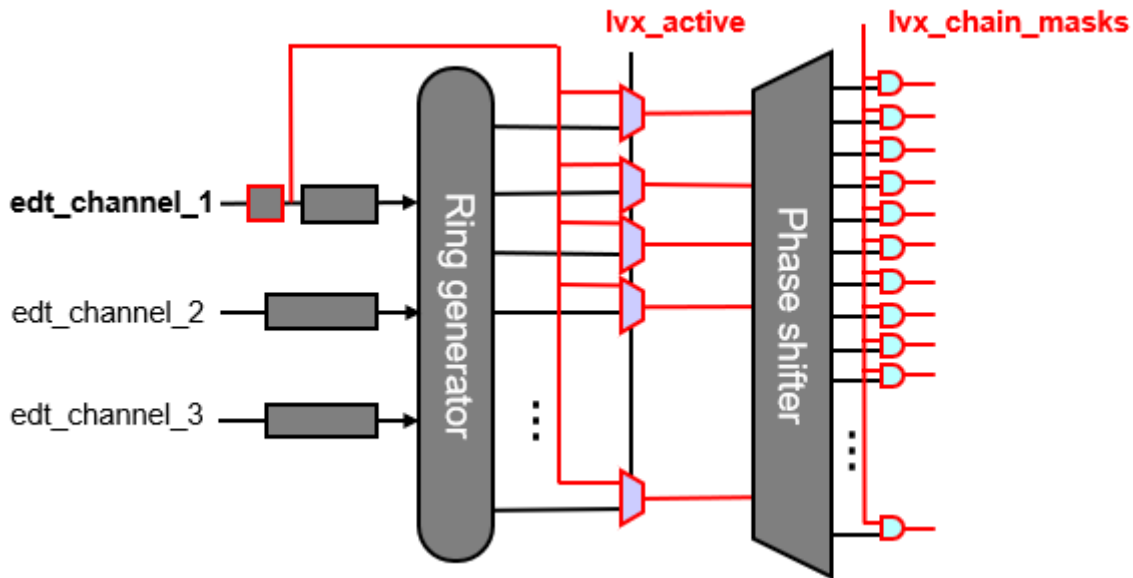
The tool delivers the LVX payload as packetized data serially along the SSN bus, as with normal scan operation. Unlike normal scan operation, the LVX payload consists of a single bit per active EDT. For example, using the preceding recommendation, the packet size for a single SSH with two active internal mode EDTs, each in their own SSH chain\_group, is two bits: a single bit for each EDT. The least significant bit (LSB) of each SSH chain group drives a specific scan chain or broadcasts to all chains, based on the LVX hardware for each EDT.

#### LVX Hardware for Channel Input

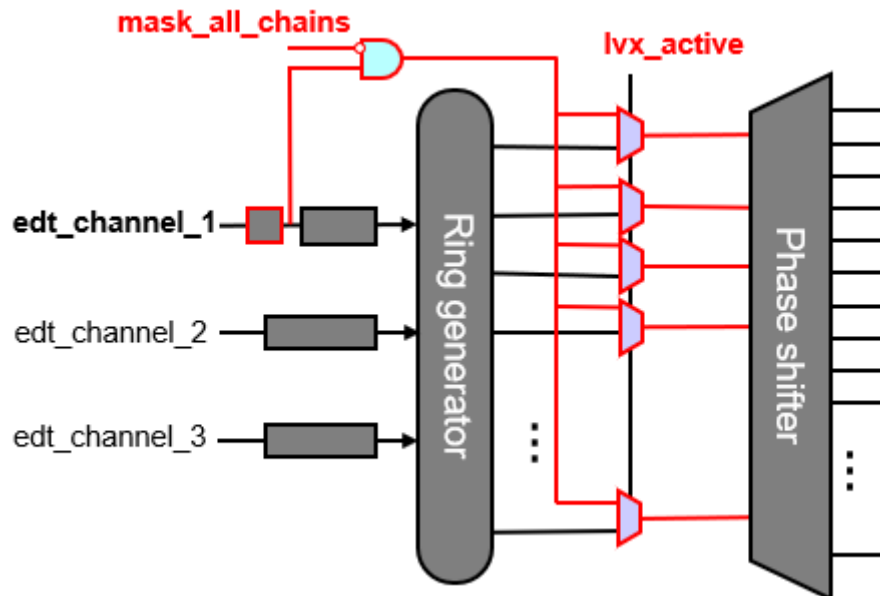
The following figures show the LVX hardware for the EDT channel input both with and without `enable_one_chain` specified as part of the `DftSpecification`. The `enable_one_chain` specification adds chain masks after the phase shifter. This is the hardware that the tool adds to the EDT to support LVX. Specifying `enable_one_chain` enables you to observe individual chains during the application of the LVX pattern. Without `enable_one_chain`, the LVX hardware does not have

the granularity to block individual chains. Instead, the hardware includes an additional `mask_all_chains` signal when you do not specify `enable_one_chain`.

**Figure 9-48. LVX Hardware With `enable_one_chain` for Channel Input**



**Figure 9-49. LVX Hardware Without `enable_one_chain` for Channel Input**

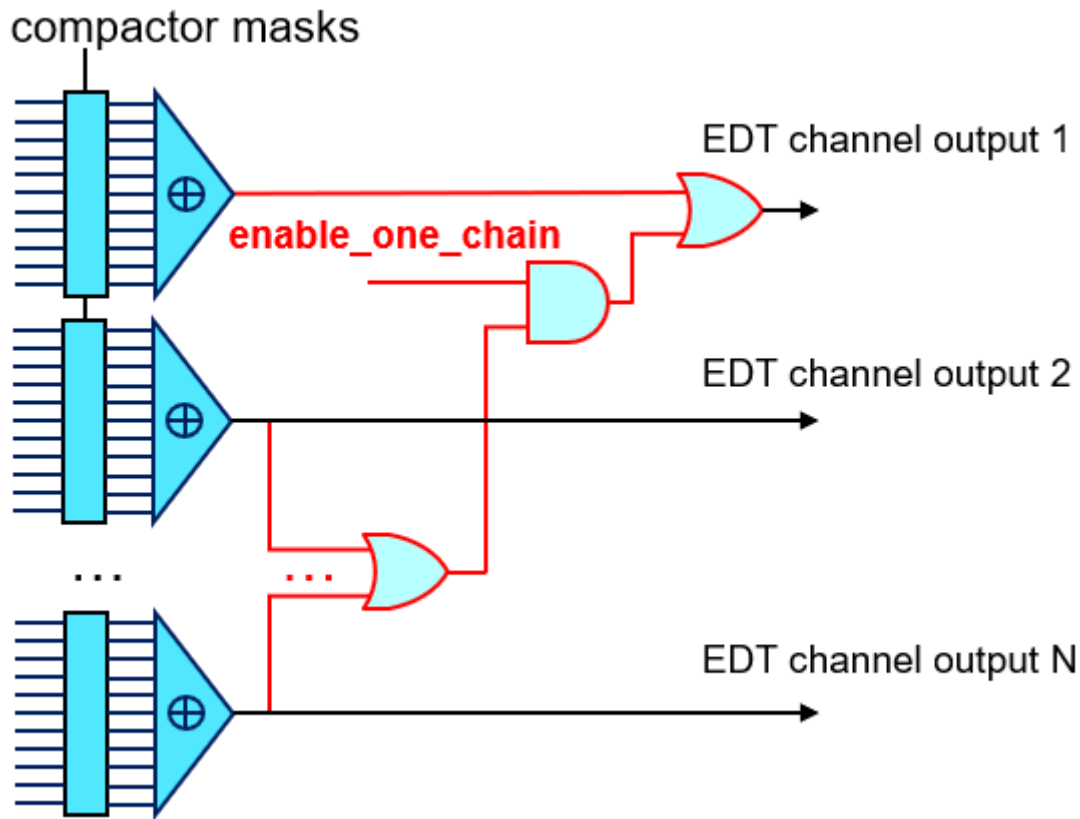


When multiple cores are active in an `lvx_loop` pattern, each core can be independently controlled through the EDT LVX IJTAG registers for each individual core. This enables some cores to broadcast the `lvx_loop` sequence to all scan chains, while other cores target unique scan chains through the `lvx_chain_index` TDR. Control this by patching the `iWriteVars` in the pattern.

### LVX Hardware for Channel Output

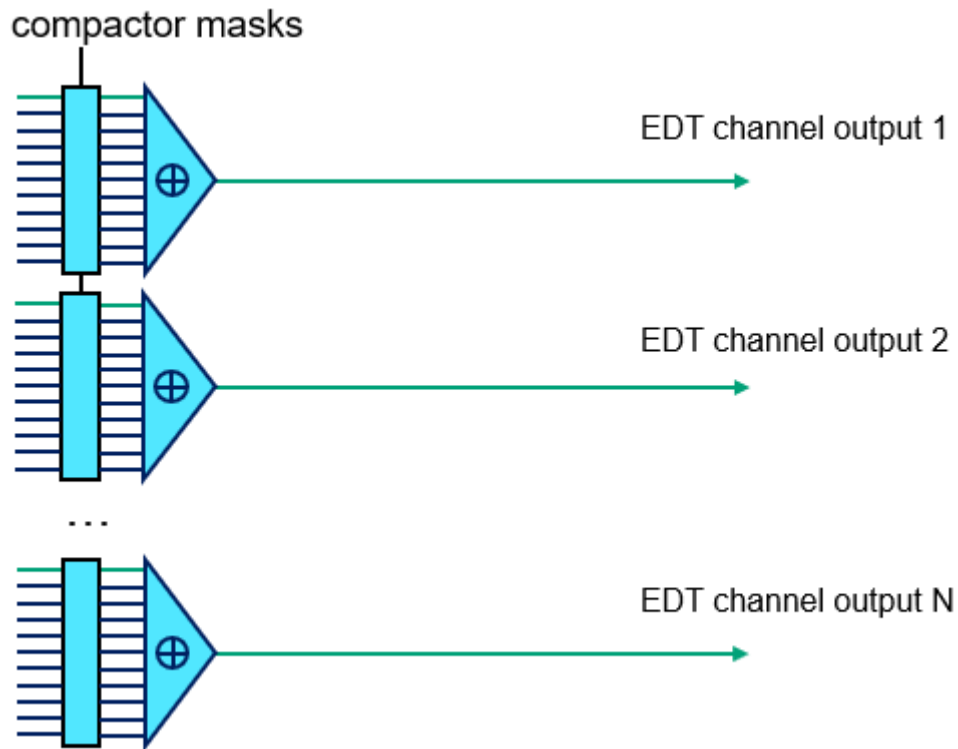
The following figures show the propagation of LVX scan data for the EDT channel output both with and without enable\_one\_chain hardware. The enable\_one\_chain specification adds some logic to accommodate the enable\_one\_chain signal on EDT channel output 1. When a single SSH and EDT are active, the LVX mode output appears on channel output 1. The SSH loads the LVX output into the packet, and it is visible on the SSN bus output. When observing a single chain, the EDT masking logic is controlled, and the one-hot masking logic that is already part of the EDT propagates the selected chain to the first EDT channel output and the other chains are masked off. When broadcasting the LVX payload to all chains, the chain selected by lvx\_chain\_index is visible on the first channel output.

**Figure 9-50. Propagation of LVX Scan Outputs With enable\_one\_chain Hardware for Channel Output**



If the enable\_one\_chain hardware is unavailable (broadcast-only hardware) during enable\_all\_chains mode, the first scan chain output of each compactor propagates through the EDT channel output.

**Figure 9-51. Propagation of the LVX Scan Output Without enable\_one\_chain Hardware for Channel Output**



When using the `lvx_loop` pattern with `enable_one_chain`, the SSH unloads channel 1 of each active EDT back into the packet, and that random data is observable at the SSN bus output. The `lvx_loop` pattern lacks the ability to confirm the current LVX configuration, because the expected output seen at the SSN output does not provide a pass/fail status. The `lvx_verification` pattern, however, enables you to verify and confirm the LVX payload. The `lvx_verification` pattern is functionally the same as the `lvx_loop` pattern except that the output observed on the SSN bus output is compared against an expected value and provides a pass/fail status. Changing an `iWriteVar` such as `lvx_chain_index` in the `lvx_loop` pattern causes the two patterns, `lvx_loop` and `lvx_verification`, to differ.

When generating the `lvx_verification` pattern set, the tool generates sufficient data on the input side for one full shift through the scan chains. This means that the pattern set consists of two patterns. The first is applied and shifts in the LVX sequence into the scan chains, and the second unloads the LVX sequence from the scan chains and compares it with the calculated expected sequence.



The following example pattern is from a STIL2005 file:

```
Pattern scan_test {
  Ann {* Pattern:0 Vector:0 TesterCycle:0 *}
  Call "test_setup" ;
  Call "ssn_setup" ;
  Ann {* Pattern:0 Vector:3436 TesterCycle:5109 *}
  "pattern 0":
    Macro "load_unload_edt_grp1"{"_ssn_datapath1_ssn_bus_data_in[0]_" =
      01001111100000011111000000111110000001111100000011111000000111110;
      "_ssn_datapath1_ssn_bus_data_out[0]_" = \r51 X HHHHHLLLLL;
    }
  Ann {* Last unload *}
  "last unload":
    Macro "load_unload_edt_grp1" {
      "_ssn_datapath1_ssn_bus_data_in[0]_" =
        00001111100000011111000000111110000001111100000011111000000111110;
      "_ssn_datapath1_ssn_bus_data_out[0]_" =
        LHHHHHHLLLLLHHHHHHLLLLLHHHHHHLLLLL \r29 X ;
    }
  Call "test_end" ;
}
```

This pattern illustrates how every pattern has an expected sequence on the SSN bus data output ports.

## LVX Operation Modes With LVX Hardware Configuration

Tessent LVX hardware can operate in several different modes that change the LVX behavior. The hardware configuration for the `enable_one_chain` functionality affects how these modes behave.

You must specify the `enable_one_chain` hardware configuration to make the `enable_one_chain` mode available. Whether or not your hardware configuration includes the `enable_one_chain` specification affects how the LVX modes behave. Control the LVX hardware using core instance parameters. The parameters in [Table 9-8](#) apply to the EDT with LVX hardware, based on whether you created the EDT with or without specifying `enable_one_chain`.

The following examples show how to specify the LVX mode with the “`lvx_mode`” parameter value in the `set_core_instance_parameters` command:

**# Select chain 5 with the `lvx_chain_index` parameter**

```
set_core_instance_parameters -instrument_type edt -parameter_values \
  {lvx_mode enable_one_chain lvx_chain_index 5}
```

or

**# Broadcast to all scan chains with the `lvx_mode` parameter**

```
set_core_instance_parameters -instrument_type edt -parameter_values \
  {lvx_mode enable_all_chains}
```

**Table 9-8. LVX Modes With enable\_one\_chain Hardware Configuration**

| Hardware Configuration          | LVX Mode          | Behavior                                                                                                                                                                                                                                                                               |
|---------------------------------|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| With enable_one_chain (default) | off               | Normal non-LVX operation.                                                                                                                                                                                                                                                              |
|                                 | enable_one_chain  | Specified chain gets the value from EDT channel input 1 and propagates the value through EDT output channel 1. All other chains load 0s.<br><br>Default operation mode in pattern generation.<br><br>Default chain index is 0. Change this with the “lvx_chain_index <int>” parameter. |
|                                 | enable_all_chains | All chains get values from EDT channel input 1 (broadcast).<br><br>The specified chain propagates through EDT output channel 1.                                                                                                                                                        |
|                                 | mask_all_chains   | All chains are masked on both input and output.                                                                                                                                                                                                                                        |
| Without enable_one_chain        | off               | Normal non-LVX operation.                                                                                                                                                                                                                                                              |
|                                 | enable_one_chain  | Not permitted.                                                                                                                                                                                                                                                                         |
|                                 | enable_all_chains | All chains get values from EDT channel input 1 (broadcast).<br><br>The first chain propagates through EDT output channel 1.                                                                                                                                                            |
|                                 | mask_all_chains   | All chains are masked on the input side by forcing 0 on the first channel input.                                                                                                                                                                                                       |

## Configuring LVX Patterns

Tessent LVX functionality contains a number of options that enable you to tailor the generated patterns to meet your design needs. Unless otherwise specified, these configurations are optional.

### Procedure

1. Configure LVX patterns and LVP settings with the [set\\_lvx\\_options](#) command.

General LVX configuration enables you to specify a subset of the active SSHs to target, a sequence for the pattern set, and the minimum number of repetitions.

Refer to the Examples section, following, for examples of how various options may affect the LVX pattern files.

2. Configure EDT for LVX by setting the LVX mode with the [set\\_core\\_instance\\_parameters](#) command.

This is typically for one of the following use cases:

- You have generated the EDT with enable\_one\_chain hardware, so that enable\_one\_chain is the default LVX mode, and you want to switch to broadcast mode (enable\_all\_chains) instead.
- You want to change the default chain selected from chain 0. For example:

```
set_core_instance_parameters -instrument_type edt \
    -parameter_values {lvx_mode enable_one_chain 487}
```

## Examples

The following examples show variations in STIL files resulting from different configurations with the set\_lvx\_options command. The gold text highlights important differences in the output.

### Change Pattern Sequence (-sequence)

Running “set\_lvx\_options -sequence 101101” alters the pattern sequence to 101101 from the default sequence of 1100:

#### Default Sequence (1100)

```
Pattern scan_test {
  //Pattern:0 Vector:0 TesterCycle:0
  Ann {* Pattern:0 Vector:0 TesterCycle:0 *}
  Ann {* Test_setup procedure omitted from this pattern set *}
  //Pattern:0 Vector:0 TesterCycle:0
  Ann {* Pattern:0 Vector:0 TesterCycle:0 *}
  Macro "scan_edt_grp1" {
    "_ssn_datapath1_ssn_bus_data_in[0]_" = 1;
  }
  //Pattern:1 Vector:1 TesterCycle:1
  Ann {* Pattern:1 Vector:1 TesterCycle:1 *}
  "pattern 1":
  Macro "scan_edt_grp1" {
    "_ssn_datapath1_ssn_bus_data_in[0]_" = 1;
  }
  //Pattern:2 Vector:2 TesterCycle:2
  Ann {* Pattern:2 Vector:2 TesterCycle:2 *}
  "pattern 2":
  Macro "scan_edt_grp1" {
    "_ssn_datapath1_ssn_bus_data_in[0]_" = 0;
  }
  Macro "scan_edt_grp1" {
    "_ssn_datapath1_ssn_bus_data_in[0]_" = 0;
  }
  Ann {* Total count Patterns:3 Vectors:4 TesterCycles:4 *}
}
```

#### set\_lvx\_options -sequence 101101

```
Pattern scan_test {
  //Pattern:0 Vector:0 TesterCycle:0
  Ann { * Pattern:0 Vector:0 TesterCycle:0 *}
  Ann { * Test_setup procedure omitted from this pattern set *}
  //Pattern:0 Vector:0 TesterCycle:0
  Ann { * Pattern:0 Vector:0 TesterCycle:0 *}
  Macro "scan_edt_grp1" {
    "_ssn_datapath1_ssn_bus_data_in[0]" = 1;
  }
  //Pattern:1 Vector:1 TesterCycle:1
  Ann { * Pattern:1 Vector:1 TesterCycle:1 *}
  "pattern 1":
  Macro "scan_edt_grp1" {
    "_ssn_datapath1_ssn_bus_data_in[0]" = 0;
  }
  //Pattern:2 Vector:2 TesterCycle:2
  Ann { * Pattern:2 Vector:2 TesterCycle:2 *}
  "pattern 2":
  Macro "scan_edt_grp1" {
    "_ssn_datapath1_ssn_bus_data_in[0]" = 1;
  }
  //Pattern:3 Vector:3 TesterCycle:3
  Ann { * Pattern:3 Vector:3 TesterCycle:3 *}
  "pattern 3":
  Macro "scan_edt_grp1" {
    "_ssn_datapath1_ssn_bus_data_in[0]" = 1;
  }
  //Pattern:4 Vector:4 TesterCycle:4
  Ann { * Pattern:4 Vector:4 TesterCycle:4 *}
  "pattern 4":
  Macro "scan_edt_grp1" {
    "_ssn_datapath1_ssn_bus_data_in[0]" = 0;
  }
  Macro "scan_edt_grp1" {
    "_ssn_datapath1_ssn_bus_data_in[0]" = 1;
  }
  Ann { * Total count Patterns:5 Vectors:6 TesterCycles:6 *}
}
```

#### Repeating Sequence to Generate Activity on All EDTs (-repetitions load\_chains)

Running “set\_lvx\_options -repetitions load\_chains” specifies for the tool to repeat the sequence in the payload until there is sufficient activity across all EDTs. In this scenario, the tool repeated the sequence (1100) approximately 25 times for a single payload. This results in using 99 tester cycles, because each bit in the sequence repeats 25 times. By default, the sequence in the payload does not repeat.

**Default Behavior**

```
Pattern scan_test {
  //Pattern:0 Vector:0 TesterCycle:0
  Ann { * Pattern:0 Vector:0 TesterCycle:0 *}
  Ann { * Test_setup procedure omitted from this pattern set *}
  //Pattern:0 Vector:0 TesterCycle:0
  Ann { * Pattern:0 Vector:0 TesterCycle:0 *}
  Macro "scan_edt_grp1" {
    "_ssn_datapath1_ssn_bus_data_in[0]" = 1;
  }
  //Pattern:1 Vector:1 TesterCycle:1
  Ann { * Pattern:1 Vector:1 TesterCycle:1 *}
  "pattern 1":
  Macro "scan_edt_grp1" {
    "_ssn_datapath1_ssn_bus_data_in[0]" = 1;
  }
  //Pattern:2 Vector:2 TesterCycle:2
  Ann { * Pattern:2 Vector:2 TesterCycle:2 *}
  "pattern 2":
  Macro "scan_edt_grp1" {
    "_ssn_datapath1_ssn_bus_data_in[0]" = 0;
  }
  Macro "scan_edt_grp1" {
    "_ssn_datapath1_ssn_bus_data_in[0]" = 0;
  }
  Ann { * Total count Patterns:3 Vectors:4 TesterCycles:4 *}
}
```

#### set\_lvx\_options -repetitions load\_chains

```
Pattern scan_test {
  //Pattern:0 Vector:0 TesterCycle:0
  Ann { * Pattern:0 Vector:0 TesterCycle:0 *}
  Ann { * Test_setup procedure omitted from this pattern set *}
  //Pattern:0 Vector:0 TesterCycle:0
  Ann { * Pattern:0 Vector:0 TesterCycle:0 *}
  Macro "scan_edt_grp1" {
    "_ssn_datapath1_ssn_bus_data_in[0]" = 1;
  }
  //Pattern:1 Vector:1 TesterCycle:1
  Ann { * Pattern:1 Vector:1 TesterCycle:1 *}
  "pattern 1":
  Macro "scan_edt_grp1" {
    "_ssn_datapath1_ssn_bus_data_in[0]" = 1;
  }
  //Pattern:2 Vector:2 TesterCycle:2
  Ann { * Pattern:2 Vector:2 TesterCycle:2 *}
  "pattern 2":
  Macro "scan_edt_grp1" {
    "_ssn_datapath1_ssn_bus_data_in[0]" = 0;
  }
  Macro "scan_edt_grp1" {
    "_ssn_datapath1_ssn_bus_data_in[0]" = 0;
  }
  ...
  //Pattern:96 Vector:96 TesterCycle:96
  Ann { * Pattern:96 Vector:96 TesterCycle:96 *}
  "pattern 96":
  Macro "scan_edt_grp1" {
    "_ssn_datapath1_ssn_bus_data_in[0]" = 1;
  }
  //Pattern:97 Vector:97 TesterCycle:97
  Ann { * Pattern:97 Vector:97 TesterCycle:97 *}
  "pattern 97":
  Macro "scan_edt_grp1" {
    "_ssn_datapath1_ssn_bus_data_in[0]" = 1;
  }
  Macro "scan_edt_grp1" {
    "_ssn_datapath1_ssn_bus_data_in[0]" = 0;
  }
  Ann { * Total count Patterns:98 Vectors:99 TesterCycles:99 *}
}
```

#### Repeating Sequence a Fixed Number of Times (-repetitions <int>)

Running “set\_lvx\_options -repetitions 10” specifies that the sequence (1100) repeats 10 times in the payload. This results in using 40 tester cycles to run one payload.

**Default Behavior**

```
Pattern scan_test {
  //Pattern:0 Vector:0 TesterCycle:0
  Ann { * Pattern:0 Vector:0 TesterCycle:0 *}
  Ann { * Test_setup procedure omitted from this pattern set *}
  //Pattern:0 Vector:0 TesterCycle:0
  Ann { * Pattern:0 Vector:0 TesterCycle:0 *}
  Macro "scan_edt_grp1" {
    "_ssn_datapath1_ssn_bus_data_in[0]_" = 1;
  }
  //Pattern:1 Vector:1 TesterCycle:1
  Ann { * Pattern:1 Vector:1 TesterCycle:1 *}
  "pattern 1":
  Macro "scan_edt_grp1" {
    "_ssn_datapath1_ssn_bus_data_in[0]_" = 1;
  }
  //Pattern:2 Vector:2 TesterCycle:2
  Ann { * Pattern:2 Vector:2 TesterCycle:2 *}
  "pattern 2":
  Macro "scan_edt_grp1" {
    "_ssn_datapath1_ssn_bus_data_in[0]_" = 0;
  }
  Macro "scan_edt_grp1" {
    "_ssn_datapath1_ssn_bus_data_in[0]_" = 0;
  }
  Ann { * Total count Patterns:3 Vectors:4 TesterCycles:4 *}
}
```

#### set\_lvx\_options -repetitions 10

```
Pattern scan_test {
  //Pattern:0 Vector:0 TesterCycle:0
  Ann {* Pattern:0 Vector:0 TesterCycle:0 *}
  Ann {* Test_setup procedure omitted from this pattern set *}
  //Pattern:0 Vector:0 TesterCycle:0
  Ann {* Pattern:0 Vector:0 TesterCycle:0 *}
  Macro "scan_edt_grp1" {
    "_ssn_datapath1_ssn_bus_data_in[0]_" = 1;
  }
  //Pattern:1 Vector:1 TesterCycle:1
  Ann {* Pattern:1 Vector:1 TesterCycle:1 *}
  "pattern 1":
  Macro "scan_edt_grp1" {
    "_ssn_datapath1_ssn_bus_data_in[0]_" = 1;
  }
  //Pattern:2 Vector:2 TesterCycle:2
  Ann {* Pattern:2 Vector:2 TesterCycle:2 *}
  "pattern 2":
  Macro "scan_edt_grp1" {
    "_ssn_datapath1_ssn_bus_data_in[0]_" = 0;
  }
  ...
  //Pattern:37 Vector:37 TesterCycle:37
  Ann {* Pattern:37 Vector:37 TesterCycle:37 *}
  "pattern 37":
  Macro "scan_edt_grp1" {
    "_ssn_datapath1_ssn_bus_data_in[0]_" = 1;
  }
  //Pattern:38 Vector:38 TesterCycle:38
  Ann {* Pattern:38 Vector:38 TesterCycle:38 *}
  "pattern 38":
  Macro "scan_edt_grp1" {
    "_ssn_datapath1_ssn_bus_data_in[0]_" = 0;
  }
  Macro "scan_edt_grp1" {
    "_ssn_datapath1_ssn_bus_data_in[0]_" = 0;
  }
  Ann {* Total count Patterns:39 Vectors:40 TesterCycles:40 *}
}
```

#### Repeating the Payload (-payload\_loop\_count <int>)

Running “set\_lvx\_options -payload\_loop\_count 5” specifies that the payload repeats five times. In the test payload, a Loop 5 wrapper surrounds the sequence (1100) of the payload. This instructs the ATE to reapply the payload five times. By default, the payload is applied only once.



**Default Behavior**

```
Pattern scan_test {
  //Pattern:0 Vector:0 TesterCycle:0
  Ann { * Pattern:0 Vector:0 TesterCycle:0 *}
  Ann { * Test_setup procedure omitted from this pattern set *}
  //Pattern:0 Vector:0 TesterCycle:0
  Ann { * Pattern:0 Vector:0 TesterCycle:0 *}
  Macro "scan_edt_grp1" {
    "_ssn_datapath1_ssn_bus_data_in[0]" = 1;
  }
  //Pattern:1 Vector:1 TesterCycle:1
  Ann { * Pattern:1 Vector:1 TesterCycle:1 *}
  "pattern 1":
  Macro "scan_edt_grp1" {
    "_ssn_datapath1_ssn_bus_data_in[0]" = 1;
  }
  //Pattern:2 Vector:2 TesterCycle:2
  Ann { * Pattern:2 Vector:2 TesterCycle:2 *}
  "pattern 2":
  Macro "scan_edt_grp1" {
    "_ssn_datapath1_ssn_bus_data_in[0]" = 0;
  }
  Macro "scan_edt_grp1" {
    "_ssn_datapath1_ssn_bus_data_in[0]" = 0;
  }
  Ann { * Total count Patterns:3 Vectors:4 TesterCycles:4 *}
}
```

#### set\_lvx\_options -repetitions 10

```
Pattern scan_test {
  //Pattern:0 Vector:0 TesterCycle:0
  Ann {* Pattern:0 Vector:0 TesterCycle:0 *}
  Ann {* Test_setup procedure omitted from this pattern set *}
  Loop 5 {
    //Pattern:0 Vector:0 TesterCycle:0
    Ann {* Pattern:0 Vector:0 TesterCycle:0 *}
    Macro "scan_edt_grp1" {
      "_ssn_datapath1_ssn_bus_data_in[0]" = 1;
    }
    //Pattern:1 Vector:1 TesterCycle:1
    Ann {* Pattern:1 Vector:1 TesterCycle:1 *}
    "pattern 1":
    Macro "scan_edt_grp1" {
      "_ssn_datapath1_ssn_bus_data_in[0]" = 1;
    }
    //Pattern:2 Vector:2 TesterCycle:2
    Ann {* Pattern:2 Vector:2 TesterCycle:2 *}
    "pattern 2":
    Macro "scan_edt_grp1" {
      "_ssn_datapath1_ssn_bus_data_in[0]" = 0;
    }
    Macro "scan_edt_grp1" {
      "_ssn_datapath1_ssn_bus_data_in[0]" = 0;
    }
  } // end loop
  Ann {* Total count Patterns:3 Vectors:4 TesterCycles:20 *}
}
```

#### Pulsing a Port at the Start of Each Pattern Set (-sync\_pulse\_port and -sync\_pulse\_cycles)

Running “set\_lvx\_options -sync\_pulse\_port GPIO3\_3 -sync\_pulse\_cycles 3” specifies that the tool pulses the supplied port, GPIO3\_3, for three cycles at the start of the pattern set, after which it forces it low for the remainder of the payload. When the payload is reapplied, the tool again pulses the port for the first three cycles.

**Default Behavior**

```
Pattern scan_test {
  //Pattern:0 Vector:0 TesterCycle:0
  Ann { * Pattern:0 Vector:0 TesterCycle:0 *}
  Ann { * Test_setup procedure omitted from this pattern set *}
  //Pattern:0 Vector:0 TesterCycle:0
  Ann { * Pattern:0 Vector:0 TesterCycle:0 *}
  Macro "scan_edt_grp1" {
    "_ssn_datapath1_GPIO3_0_" = 1;
    "_ssn_datapath2_GPIO3_1_" = 1;
  }
  //Pattern:1 Vector:1 TesterCycle:1
  Ann { * Pattern:1 Vector:1 TesterCycle:1 *}
  "pattern 1":
  Macro "scan_edt_grp1" {
    "_ssn_datapath1_GPIO3_0_" = 1;
    "_ssn_datapath2_GPIO3_1_" = 1;
  }
  //Pattern:2 Vector:2 TesterCycle:2
  Ann { * Pattern:2 Vector:2 TesterCycle:2 *}
  "pattern 2":
  Macro "scan_edt_grp1" {
    "_ssn_datapath1_GPIO3_0_" = 1;
    "_ssn_datapath2_GPIO3_1_" = 1;
  }
  //Pattern:3 Vector:3 TesterCycle:3
  Ann { * Pattern:3 Vector:3 TesterCycle:3 *}
  "pattern 3":
  Macro "scan_edt_grp1" {
    "_ssn_datapath1_GPIO3_0_" = 1;
    "_ssn_datapath2_GPIO3_1_" = 1;
  }
}
```

```
//Pattern:4 Vector:4 TesterCycle:4
Ann {* Pattern:4 Vector:4 TesterCycle:4 *}
"pattern 4":
Macro "scan_edt_grp1" {
    "_ssn_datapath1_GPIO3_0_" = 0;
    "_ssn_datapath2_GPIO3_1_" = 0;
}
//Pattern:5 Vector:5 TesterCycle:5
Ann {* Pattern:5 Vector:5 TesterCycle:5 *}
"pattern 5":
Macro "scan_edt_grp1" {
    "_ssn_datapath1_GPIO3_0_" = 0;
    "_ssn_datapath2_GPIO3_1_" = 0;
}
//Pattern:6 Vector:6 TesterCycle:6
Ann {* Pattern:6 Vector:6 TesterCycle:6 *}
"pattern 6":
Macro "scan_edt_grp1" {
    "_ssn_datapath1_GPIO3_0_" = 0;
    "_ssn_datapath2_GPIO3_1_" = 0;
}
Macro "scan_edt_grp1" {
    "_ssn_datapath1_GPIO3_0_" = 0;
    "_ssn_datapath2_GPIO3_1_" = 0;
}
Ann {* Total count Patterns:7 Vectors:8 TesterCycles:8 *}
}
```

**set\_lvx\_options -sync\_pulse\_port GPIO3\_3 -sync\_pulse\_cycles 3**

```

Pattern scan_test {
  //Pattern:0 Vector:0 TesterCycle:0
  Ann { * Pattern:0 Vector:0 TesterCycle:0 *}
  Ann { * Test_setup procedure omitted from this pattern set *}
  //Pattern:0 Vector:0 TesterCycle:0
  Ann { * Pattern:0 Vector:0 TesterCycle:0 *}
  Macro "scan_edt_grp1" {
    "_ssn_datapath1_GPIO3_0_" = 1;
    "_ssn_datapath2_GPIO3_1_" = 1;
    "_datapath1_GPIO3_3_" = 1;
  }
  //Pattern:1 Vector:1 TesterCycle:1
  Ann { * Pattern:1 Vector:1 TesterCycle:1 *}
  "pattern 1":
  Macro "scan_edt_grp1" {
    "_ssn_datapath1_GPIO3_0_" = 1;
    "_ssn_datapath2_GPIO3_1_" = 1;
    "_datapath1_GPIO3_3_" = 1;
  }
  //Pattern:2 Vector:2 TesterCycle:2
  Ann { * Pattern:2 Vector:2 TesterCycle:2 *}
  "pattern 2":
  Macro "scan_edt_grp1" {
    "_ssn_datapath1_GPIO3_0_" = 1;
    "_ssn_datapath2_GPIO3_1_" = 1;
    "_datapath1_GPIO3_3_" = 1;
  }
  //Pattern:3 Vector:3 TesterCycle:3
  Ann { * Pattern:3 Vector:3 TesterCycle:3 *}
  "pattern 3":
  Macro "scan_edt_grp1" {
    "_ssn_datapath1_GPIO3_0_" = 1;
    "_ssn_datapath2_GPIO3_1_" = 1;
    "_datapath1_GPIO3_3_" = 0;
  }
  //Pattern:4 Vector:4 TesterCycle:4
  Ann { * Pattern:4 Vector:4 TesterCycle:4 *}
  "pattern 4":
  Macro "scan_edt_grp1" {
    "_ssn_datapath1_GPIO3_0_" = 0;
    "_ssn_datapath2_GPIO3_1_" = 0;
    "_datapath1_GPIO3_3_" = 0;
  }
  ...

```

```
//Pattern:13 Vector:13 TesterCycle:13
Ann {* Pattern:13 Vector:13 TesterCycle:13 *}
"pattern 13":
Macro "scan_edt_grp1" {
  "_ssn_datapath1_GPIO3_0_" = 0;
  "_ssn_datapath2_GPIO3_1_" = 0;
  "_datapath1_GPIO3_3_" = 0;
}
//Pattern:14 Vector:14 TesterCycle:14
Ann {* Pattern:14 Vector:14 TesterCycle:14 *}
"pattern 14":
Macro "scan_edt_grp1" {
  "_ssn_datapath1_GPIO3_0_" = 0;
  "_ssn_datapath2_GPIO3_1_" = 0;
  "_datapath1_GPIO3_3_" = 0;
}
Macro "scan_edt_grp1" {
  "_ssn_datapath1_GPIO3_0_" = 0;
  "_ssn_datapath2_GPIO3_1_" = 0;
  "_datapath1_GPIO3_3_" = 0;
}
Ann {* Total count Patterns:15 Vectors:16 TesterCycles:16 *}
}
```

## Writing LVX Loop Patterns

Generate the LVX loop pattern set by writing separate patterns for setup and payload.

### Note

 If you generate LVX loop and LVX verification patterns, you can reuse the `ssn_setup` procedures. Refer to “[Writing LVX Verification Patterns](#)” on page 667 for more information.

## Procedure

1. Generate the LVX loop setup files:

```
write_patterns lvx_test_setup.stil -test_setup only -stil
write_patterns lvx_ssn_setup.stil -ssn_setup only -pattern_sets lvx_loop -stil
```

The following alternative command generates combined setup patterns, which include both the `test_setup` and `ssn_setup` procedures:

```
write_patterns lvx_all_setup.stil -all_setup_only -pattern_sets lvx_loop -stil
```

### Note

 The `test_setup` procedure for LVX mode contains significant differences from the procedure without LVX mode specified. In LVX mode, the procedure programs the IJTAG registers inside the EDT. This does not occur outside LVX mode.

2. Generate the LVX loop payload:

```
write_patterns lvx_loop_payload.stil -test_payload pattern_sets lvx_loop -stil
```

This writes the pattern set on which to loop onto the ATE.

## Results

The `lvx_loop` pattern set includes the following functionality:

- It contains the shift vectors you loop through when performing LVI/LVP.
- There is no measurement of the datapath output for this payload.
- During LVX, you loop through the `lvx_loop` payload without applying an `ssn_end` procedure.

---

### Note



The chain index in the `test_setup.stil` file can be modified on the ATE, which eliminates needing to rewrite the pattern for a different change index.

---

## Examples

Refer to the example in “[Writing LVX Verification Patterns](#)” on page 667, which contains both LVX loop and LVX verification patterns.

## Writing LVX Verification Patterns

Writing LVX verification patterns is optional when writing LVX loop patterns. The `lvx_verification` patterns measure the SSN bus output, so you can use them to verify any arbitrary LVX sequence.

The `lvx_verification` pattern is functionally equivalent to the `lvx_loop` pattern, except that the output observed on the SSN bus is compared against an expected value to provide a pass/fail status. The pass/fail status confirms the response of the targeted shift registers. Do not loop on an LVX verification pattern.

---

### Note



Changing the `chain_index` from the value used when writing the `lvx_verification` pattern can lead to pattern failures. The expected value is calculated based on the original `chain_index`. This change can occur when the `chain_index` is switched to a chain with a different length or inversion.

---

Both the `lvx_loop` and `lvx_verification` pattern sets share the same `ssn_setup` programming. You can optionally apply the `lvx_verification` pattern to the DUT to confirm the targeted shift register response. Then you must apply the `test_setup` and `ssn_setup` for the `lvx_loop` before applying the `lvx_loop` payload pattern.

### Note



You can generate the LVX verification setup and payload files before the LVX loop files.

---

## Procedure

1. Generate the LVX verification setup files:

```
write_patterns lvx_test_setup.stil -test_setup only -stil  
write_patterns lvx_ssn_setup.stil -ssn_setup only -pattern_sets lvx_verification -stil
```

The following alternative command generates combined setup patterns, which include both the test\_setup and ssn\_setup procedures:

```
write_patterns lvx_all_setup.stil -all_setup_only -pattern_sets lvx_verification -stil
```

2. Generate the LVX verification payload:

```
write_patterns lvx_verification_payload.stil -test_payload \  
-pattern_sets lvx_verification -stil
```

## Examples

The following example commands write the test\_setup, ssn\_setup, lvx\_verification, and lvx\_loop patterns.

```
write_patterns lvx_test_setup.stil -test_setup only -stil  
write_patterns lvx_ssn_setup.stil -ssn_setup only -pattern_sets lvx_verification -stil  
write_patterns lvx_verification.stil -test_payload -pattern_sets lvx_verification -stil  
write_patterns lvx_loop.stil -test_payload -pattern_sets lvx_loop -stil
```

### Note



The command to write the ssn\_setup pattern uses the “-pattern\_sets lvx\_verification” argument. Because the SSN programming for lvx\_loop and lvx\_verification is identical for both patterns, you can share the ssn\_setup information for both the lvx\_loop and lvx\_verification patterns.

---

## LVI\_PATCHING\_INFO

The LVI\_PATCHING\_INFO table enables the use of fault isolation techniques for SSN by providing two pieces of information: the ICL instance name for EDT instances driving failing chains and the chain index of the failing chain (as reflected in the STIL pattern).

LVI is an electrical fault isolation (EFI) technique for localizing defects in scan chains. LVI requires a repeating periodic pattern on the chain of interest. This periodic pattern causes transistors in the scan chain to turn on and off within the field of view of the LVI equipment. LVI collects the signals and analyzes them in the frequency domain. The analysis detects transistors that switch with the expected pattern (and those that do not) and produces an image.

During hardware generation in hierarchical DFT flows, the tool can only generate the LVX verification pattern for the specified default chain. The tool can generate the loop pattern for any



chain driven by the LVX hardware. However, the tool's default pattern set contains the loop pattern for the default chain, which is the same default chain specified when creating the LVX hardware. You can specify scan chains with `lvx_chain_index`, or you can enable broadcast mode for all chains. See “[LVX Operation Modes With LVX Hardware Configuration](#)” on page 653 for more information.

Note

 For failure analysis (FA) purposes, you must patch the patterns to apply the looping pattern to the (failing) chain of interest. When using SSN, usually the FA engineer that runs LVI patches the patterns.

The goal of patching is to ensure that a repeating sequence drives the failing chain of interest. The FA engineer requires the full ICL instance pathname of the EDT instance that drives the failing chain, and the chain index of the failing chain to patch the original LVI patterns delivered by design engineering teams.

When the tool generates a diagnosis report, it automatically adds the `LVI_PATCHING_INFO` table within the XMAP table if the design has LVI or LVP hardware present.

### Example XMAP Table

The following example shows an `LVI_PATCHING_INFO` table within an XMAP table. The `LVI_PATCHING_INFO` table contains the chain name and index, and the ICL and EDT instance names for symptom 1.

```
diagnosis_mode=chain
#symptoms=1 #suspects=1 CPU_time=0.70sec fail_log=chain.flog

symptom=1    #suspects=1    faulty_chain=edt._chain_8    fault_type=STUCK
suspect score type                value pin_pathname    cell_name net_pathname \
-----\
1          96    STUCK (CELL+OUT) 1          /XTEA..._reg[10]/Q SDFFR_X1  /XTEA...hi[10] \
-----\

cell_number chain_name    memory_type shift_clock
-----
34          edt..._chain_8 MASTER          /xtea..._clock_out_mux_Z
-----

XMAP_TABLE_BEGIN
...
LVI_PATCHING_INFO_BEGIN
    symptom chain_name    chain_index ICL_instance    EDT_instance
    1      edt..._chain_8  7          xtea..._edt_c1_inst /xtea..._edt_c1_inst
LVI_PATCHING_INFO_END
...
XMAP_TABLE_END
```

## ICL Instance Name of an EDT Controller

Patching starts by locating in your STIL pattern the ICL instance name of the EDT controller provided by the LVI\_PATCHING\_INFO table. The following example is an excerpt of a STIL pattern. The “Targets” annotation has LVX chain index variables that hold the chain index values for LVI inspection. Highlighted in red is the ICL instance name of an EDT controller that has three variable bits for the lvx\_chain\_index variable.

```
Ann { * + Targets: * }
Ann { * + Apply 0 * }
Ann { * + writes: * }
Ann { * + xtea_2.edt_three.edt_bypass = 0 * }
Ann { * + xtea_2.edt_three.edt_low_power_shift_en = 0 * }
Ann { * + xtea_2.edt_three.lvx_chain_index[1:0] = 00 * }
Ann { * + xtea_2.edt_three.lvx_mode[1:0] = 00 * }
Ann { * + xtea_2.xtea_rtl_tessent_edt_one_inst.edt_bypass = 0 * }
Ann { * + xtea_2.xtea_rtl_tessent_edt_one_inst.lvx_chain_index[2:0] = 000 * }
Ann { * + xtea_2.xtea_rtl_tessent_edt_one_inst.lvx_mode[1:0] = 00 * }
```

## Relative Cycles by Pin

Tessent pragma annotations identify the relative cycles by pin to reach an ICL instance. In the following pragma, the ICL instance name of the EDT controller is highlighted in red. The variable to patch is lvx\_chain\_index. The range of variable bits of the chain index are in the value {2:0} of the -var\_bits parameter. The -pin parameter identifies the tdi pin. The relative cycles to patch on the tdi pin are in the value {30:28} of the -relative\_cycles parameter.

```
Ann { * + Shift-DR * }
...
Ann { * TESSENT_PRAGMA variable
    xtea_2.xtea_rtl_tessent_edt_one_inst.lvx_chain_index \
    -type write \
    -var_bits {2:0} -pin tdi -relative_cycles {30:28} * }
...
```

## Signal Groups

The pin of the Tessent pragma is in a signal group of your pattern. In the following SignalGroups, the tdi pin is in the \_pi\_ signal group at the third position from the end.

```
SignalGroups {
    ...
    _pi_ = "ssn_bus_data_in"[11] + "ssn_bus_data_in"[10] +
    "ssn_bus_data_in"[9] + "ssn_bus_data_in"[8] + "ssn_bus_data_in"[7] +
    "ssn_bus_data_in"[6] + "ssn_bus_data_in"[5] + "ssn_bus_data_in"[4] +
    "ssn_bus_data_in"[3] + "ssn_bus_data_in"[2] + "ssn_bus_data_in"[1] +
    "ssn_bus_data_in"[0] + "ssn_bus_clock" + "reset_pad" + "xtea_1_rst" +
    "xtea_2_rst" + "tck" + tdi + "tms" + "trst";
    ...
}
```

## Vector Locations

Tessent pragma annotations at each bit variable mark vectors of each index location. The following annotations identify the three vectors to patch.

```

Ann {* TESSSENT_PRAGMA bit_variable \
    xtea_2.xtea_rtl_tessent_edt_one_inst.lvchain_index \
    -type write -var_bits {0} -pin tdi -inversion 0b0 *}
V {
    _pi_=\r13 0 1001001;
    _po_=\r13 X;
}
Ann {* TESSSENT_PRAGMA bit_variable \
    xtea_2.xtea_rtl_tessent_edt_one_inst.lvchain_index \
    -type write -var_bits {1} -pin tdi -inversion 0b0 *}
V {
    _pi_=\r13 0 1001001;
}
Ann {* TESSSENT_PRAGMA bit_variable \
    xtea_2.xtea_rtl_tessent_edt_one_inst.lvchain_index \
    -type write -var_bits {2} -pin tdi -inversion 0b0 *}
V {
    _pi_=\r13 0 1001001;
}

```

## Retention Testing With SSN

Tessent SSN supports retention testing. If you use retention tests with SSN, you can enable the SSN to lose its state and be part of the powered-off logic. The tool uses multiple pattern files to achieve this functionality.

For a full description of Tessent Shell retention testing capabilities, refer to the section “[Retention Test Pattern Generation](#)” in the *Tessent Scan and ATPG User’s Manual*.

To enable loss of state functionality, use the “[set\\_retention\\_test\\_options](#) -allow\_ssn\_to\_lose\_state\_during\_test” command. For more information about this functionality, refer to that command description.

For SSN, the test\_setup and ssn\_setup procedures for retention are different from those of non-retention pattern sets. Use the “[write\\_patterns](#)” command to write those two IJTAG procedures separately:

```

write_patterns retention_test_setup.stil -stil -test_setup only -pattern_set retention -replace
write_patterns retention_ssn_setup.stil -stil -ssn_setup only -pattern_set retention -replace

```

For more information about writing SSN patterns, with examples, refer to the section “[Manufacturing Patterns With SSN](#)” on page 558.

**Note**

 We recommend that you write the IJTAG and payload procedures for all SSN patterns to separate files.

---

Like logic test SSN patterns, SSN retention test patterns require failure mapping when a miscompare is identified on the SSN bus output. The failure mapping step identifies the SSH the failing register is associated with. You can identify the precise failing register using Tessent Diagnosis. The following is a recipe for both failure mapping and diagnosis with a retention test pattern:

1. Failure mapping recipe:
  - a. `set_context -failure_mapping`
  - b. `read_design <my_top.v>`
  - c. `read_core_description`
  - d. `set_current_design`  
`check_design_rules`
  - e. `read_patterns <.stil...>`
  - f. `read_failures ...<tessent failure file — cycle based>`
  - g. `write_failures <reverse mapped failure file>`
2. Tessent Diagnosis recipe:
  - a. `set_context -scan_diagnosis`
  - b. `read_flat_model <core_flat_model>`
  - c. `read_patterns <core_patterns>`
  - d. `read_failures <reverse mapped failure file> -pattern`

For more information on the SSN failure mapping step and Tessent Diagnosis, refer to [“Performing Reverse Failure Mapping for SSN Pattern Diagnosis”](#) on page 526 and the *Tessent Diagnosis User’s Manual*.

**Note**

 Retention testing with SSN requires a scan host generated with a 2022.1 or newer release.

Streaming-Through-IJTAG mode does not support retention testing.

---

## Size Resolution Technology With SSN

Using size resolution technology with SSN enables you to reduce the size of the ScanHost node, providing improvements in area and timing at the cost of flexibility and data efficiency.

The area of an SSN ScanHost node depends on the width of the SSN parallel bus and the size of the EDT that connects to the node. The area of the typical SSH is comparable to an EDT, and adding on-chip compare logic to an SSH generally makes it 1.5× to 2.0× larger than an EDT. However, for some use models, an SSH with on-chip compare can be even larger if you are using logic redundancy or die harvesting, or if you require a very large EDT when using on-chip compare. Furthermore, if you set the sticky status resolution type for on-chip compare to `output_chain`, the area of the SSH can further increase. In rare cases, it may be difficult to close timing when using a very wide parallel bus and a fast bus clock.

The flexibility of the SSH hardware can also contribute to the size of the SSH. The default ScanHost hardware is flexible enough to support delivery of any number of bits based on the EDT and scan chains it connects to. The hardware can operate on bits of any number (that is, multiples of one). For example, the SSH can receive as little as one bit of data and as much as a full scan word of data during ATPG and retargeting. (A full scan word is the amount of data shifted with a single clock pulse into all EDT input channels, uncompressed scan chains, or the sum of both.)

Creating a ScanHost with size resolution technology can offset some of the factors contributing to an increased SSH area. When you use a ScanHost equipped with size resolution, you give up some of the efficiency of the SSH to gain better area and timing. An SSH with size resolution changes the minimum number of bits that the SSH can deliver, instead operating on numbers of bits that are multiples of two, four, or eight. As an example, an SSH with size resolution of four can deliver only a minimum of four bits, and any number of bits that is a multiple of four. If ATPG or retargeting calls for a smaller number of bits, the tool adds padding to increase the number of bits to satisfy the size resolution bit requirement.

Turn on size resolution by specifying the `DftSpecification/SSN/Datapath/ScanHost/size_resolution` property. Legal values are 2, 4, and 8.

The following designs may benefit from size resolution technology:

- Designs with a ScanHost using on-chip compare that requires an EDT with a large output channel count.
- Designs in which the area of the ScanHost is too large and must be reduced.
- Designs that fail to meet timing in the ScanHost.

---

### Caution



Size resolution can have a noticeable impact on scan data volume. Therefore, we recommend using a ScanHost equipped with size resolution technology cautiously, and only when necessary.

---

The following circumstances are the most likely scenarios for which padding is necessary to meet size resolution requirements:

- During ATPG when using an EDT with a smaller channel count for wrapper chains.
- When using on-chip compare where the output bits are not a multiple of the size resolution requirements.

You can use a ScanHost equipped with size resolution technology during ATPG and scan pattern retargeting. There are no limitations or restrictions associated with using ScanHost nodes with and without size resolution technology together. You can also use ScanHosts with identical or different size resolution values concurrently.

**Size Resolution Limitations. . . . . 674**

## Size Resolution Limitations

A ScanHost node equipped with size resolution technology is subject to limitations on the values for the SSN bus\_width and input and output chain group bit counts. If these values are not multiples of the size resolution value for the SSH, the tool may respond with an error or by padding packets.

The following table summarizes the tool response to values that are not multiples of the size resolution setting. For more details, refer to the sections following the table.

**Table 9-9. Size Resolution Value Limitations Summary**

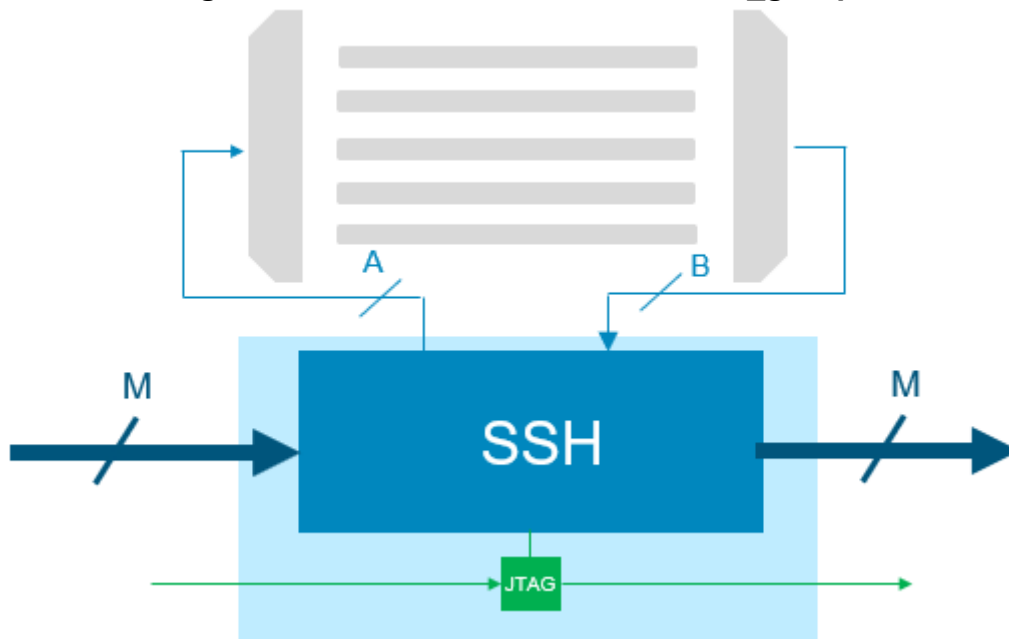
| Value Not a Multiple of the SSH Size Resolution Setting     | Tool Response |
|-------------------------------------------------------------|---------------|
| SSN bus_width                                               | Error         |
| Single chain group input or output bit count                | Pads packets  |
| Sum of input or output bit counts for multiple chain groups | Pads packets  |

### Error Condition — Bus Width

The SSN bus\_width must be a multiple of the size resolution value. If there are multiple ScanHost nodes with different size resolution values, the bus width must satisfy all of them. If the bus\_width is not a multiple of the size resolution value, the tool reports an error.

For example, if the ScanHost has a size resolution value of 8 and the bus width (M in the following figure) is 50 bits, the tool reports an error. You must change the bus width or the size resolution value to resolve this error.

**Figure 9-52. ScanHost With One chain\_group**



### Padding Condition — Inputs and Outputs for a Single Chain Group

If a single chain\_group is present, the SSH chain\_group inputs and outputs must individually be a multiple of the size resolution value of the ScanHost. If an input or output bit count is not a multiple of the size resolution value, the tool adds padding to the packets to ensure that this condition is met.

#### Examples of Padding for a Single Chain Group

Refer to [Figure 9-52](#) for these examples.

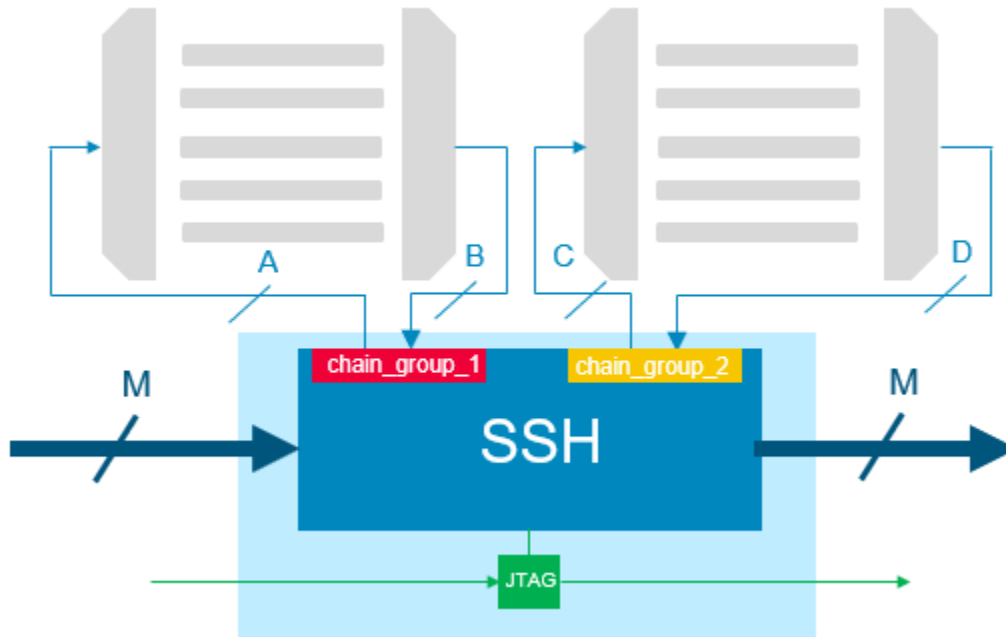
- If the SSH has a size resolution value of 8 and the EDT has one channel, the tool adds seven bits of padding to satisfy the size resolution requirements.
- If the SSH uses on-chip compare, has a size resolution value of 8, and uses an EDT with one output channel and a single on-chip compare status group, the tool calculates the number of output bits as three ( $3 \times$  the EDT output channel count). Because three is not a multiple of the size resolution value, the tool adds padding to satisfy the size resolution requirements: it adds 21 bits to the 3 to make the total number of output bits a multiple of 8, because the tool adds padding to each packet.

### Padding Condition — Inputs and Outputs for Multiple Chain Groups

If multiple chain\_groups are present, the sum of the input bit counts for the SSH chain\_group must be a multiple of the size resolution value of the ScanHost. Likewise, the sum of the output bit counts for the SSH chain\_group must also be a multiple of the size resolution value. If these sums are not multiples of the size resolution value, the tool adds padding to the packets to ensure that this condition is met.

For example, in the following figure, the sums of the input bit counts for chain\_group\_1 and chain\_group\_2 ( $A+C$ ) must be a multiple of the SSH size resolution value, and the sums of the output bit counts ( $B+D$ ) must also be a multiple of the size resolution value.

**Figure 9-53. ScanHost With Multiple chain\_groups**



#### Examples of Padding for Multiple Chain Groups

Refer to [Figure 9-53](#) for these examples.

- If the SSH has a size resolution value of 4 and the input counts for the chain groups are 3 and 5, then the sum (8) is a multiple of the size resolution value and the tool does not add padding.
- If the SSH has on-chip compare and a size resolution value of 4, and the output bit counts for the chains are  $B=3$  and  $D=5$ , the tool calculates the output bits for each chain group as  $9 (3 \times B)$  and  $15 (3 \times D)$ . The sum of these output counts is  $9+15=24$ , which is a multiple of the size resolution value, so the tool does not add padding.



# Tessent SSN Examples and Solutions

This section contains Tessent solutions for DFT scenarios and problems specific to SSN. This includes applications and flows that solve issues resulting from the presence of specific design objects or the use of specific implementation flows.

The solutions are organized in the following sections:

**Third-Party OCCs With SSN** ..... 677

## Third-Party OCCs With SSN

Third-party OCC connections require specific handling when you are using SSN.

### Problem

Third-party OCCs need connections to `scan_en` and `shift_capture_clock`, which do not exist until the SSN [ScanHost](#) node is inserted. This section explains how to automate these connections.

### Solution

The third-party OCCs are assumed to be pre-inserted into the design when SSN is inserted as described in the section “[Second DFT Insertion Pass: Inserting Block-Level EDT, OCC, and SSN](#)” on page 483. The `scan_en` and `slow_clock` pins on the OCC instances are tied low.

Follow these steps to properly connect the OCC to the ScanHost node:

1. Before calling the [check\\_design\\_rules](#) command, run the “[add\\_dft\\_control\\_points -type dynamic\\_dft\\_signal -dft\\_signal\\_name scan\\_en](#)” command while pointing to the `scan_en` pin of each OCC instance.

```
set occs [get_instances my_occ*]
add_dft_control_points [get_pins scan_en -of_inst $occ] \
    -type dynamic_dft_control \
    -dft_signal_source_name scan_en
```

2. Run the “[add\\_dft\\_control\\_points -type dynamic\\_dft\\_signal -dft\\_signal\\_name shift\\_capture\\_clock](#)” command while pointing to the `slow_clock` pin of each OCC instance:

```
add_dft_control_points [get_pins slow_clock -of_inst $occ] \
    -type dynamic_dft_control \
    -dft_signal_source_name shift_capture_clock
```

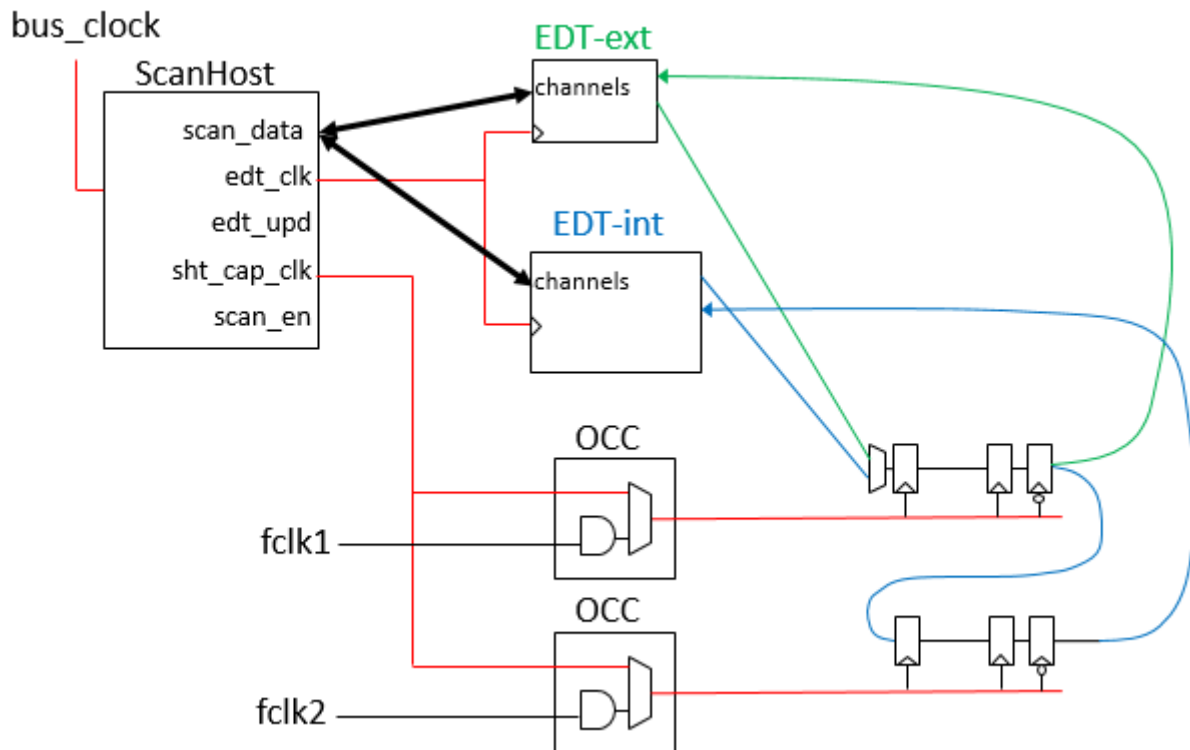
### Discussion

In the solution described in the previous subsection, you use the `add_dft_control_points` command to make the `scan_en` and `shift_capture_clock` connections to the OCC. This is the

general command you use to connect any DFT signal to any destination. When you insert the [ScanHost](#) node, the insertion process remaps dynamic DFT signals `scan_en`, `edt_update`, `edt_clock`, and `shift_capture_clock` to the output pins of the ScanHost node.

Because the ScanHost node delivers the scan data in both the internal and external scan modes, your third-party OCC must still be able to inject the `slow_clock` into the clock\_domains when `scan_en` is high, even when the OCCs are not active. In a wrapped core, you have an OCC on each clock input. Those OCCs are active during the internal scan modes but are typically inactive during the external test modes, such that the sourcing for the capture clock is from a single OCC localized at the functional clock source of the domain. Those inactive OCCs must, however, still inject the `shift_capture_clock` generated by the local ScanHost node when `scan_en` is high during the external modes. This is the [shift\\_only\\_mode](#) feature described in the Tessent OCC reference page. The presence of this feature enables easier timing closure because the entire shift timing is internal to the physical block and completely derived from the SSN bus clock, as illustrated in the following figure:

**Figure 9-54. Localized Scan Timing With Shift-Only Mode OCC**



If you want your legacy scan access mechanism to coexist with SSN in your first few designs that use SSN, add the `scan_en` and other dynamic DFT signals on their original sources as you do currently. Then, continue to preconnect the `scan_en` and `slow_clock` inputs of your third-party OCC to those sources. When the ScanHost node is inserted, the complete fanout of the specified sources moves so that the output pins of the ScanHost node drive them, and your original sources are connected to the `bypass_in` pins of the ScanHost. See [Example 2 in the ScanHost](#) reference page for more information about this flow.

Unlike with the Tessent OCC, you must describe the third-party OCC to the Tessent Tool. To do this, use the “[add\\_clocks -capture\\_only](#)” command manually on each OCC output clock pin, and provide a clock control definition as described in the “[Support for Internal Clock Control](#)” section of the *Tessent Scan and ATPG User’s Manual*.

## SSN Frequently Asked Questions

This section contains frequently asked questions and possible solutions about SSN.

1. **Is it possible for an SSN datapath to have a branch that is narrower than the main datapath?**

Tessent SSN does support this, but it is not recommended. IPs with narrower bus widths than the parent design are supported, but the IP should be rebuilt with the proper width.

Use the SSN multiplexer node to isolate the narrower branch of the bus from the parent design. When an SSH on the narrower bus is active, the effective bus width is scaled down to the narrower bus width.

An IP with a wider bus than the parent design is not supported.

2. **Can I have multiple independent active datapaths?**

SSN supports multiple SSN datapaths operating concurrently at different frequencies for retargeting. During top-level ATPG, the effective bus width is scaled down to the narrower datapath width. The two datapaths are capture-aligned, with padding added to the packet to ensure capture alignment.

3. **Can I change the SSN bus\_clock and shift\_capture\_clock frequencies during retargeting?**

You can change the bus\_clock and shift\_capture\_clock frequencies during retargeting after the patterns have already been created at the block level.

During retargeting, set the set\_current\_physical\_block command to the scope of the physical block that you want to change either frequency. Use the set\_load\_unload\_timing\_options command to change the frequencies.

---

**Note**



The SSN bus\_clock frequency operates at the slowest specified frequency set by the set\_load\_unload\_timing\_options command at any physical block.

---

4. **How do I exclude a physical block from top-level ATPG with SSN?**

Do not add the core instance for the EDT inside the physical block that you want to exclude. Without the core instance of the EDT added, the SSH is disabled and behaves as a two-stage pipeline.

**5. What design view of my physical blocks should I use during pattern retargeting?**

Use the graybox design view of the physical block whose patterns you want to retarget.

**6. How do I program my SSN multiplexer nodes?**

Use the generated PDL to write the select of the SSN multiplexer using the `set_test_setup_icall` command.

```
set_test_setup_icall \  
    "chip_top_ssh_insertion_tessent_ssn_mux_returnPath_inst.setup  
    select_secondary_bus 1" -append
```

**7. How is the negative edge of scan\_en synchronized across all SSH during top-level ATPG?**

During top-level ATPG, one SSH may enter the capture state before another SSH in the same active datapath. This may happen with certain bus configurations and is normal behavior. During top-level ATPG, all active SSH nodes are capture-aligned. Each SSH receives the same number of packets before entering or leaving the capture state. This guarantees that each physical block does not miss any capture clock pulses.

**8. Can I use a custom test\_setup sequence with “pulse TCK” and “force TMS” to initialize an analog IP and still use SSN?**

Yes, add your custom test\_setup sequence using the `set_procfile_name` command, and the SSN initialization is appended at the end of your custom sequence.

**9. How do I enable Streaming-Through-IJTAG?**

Use the `set_ssn_options` command to enable the Streaming-Through-IJTAG mode.

```
set_ssn_options -streaming_interface ijtag
```

**10. How do I decide the width of my SSN bus?**

Reuse the same number of GPIO used for scan in/out without SSN. The SSN data\_in and data\_out ports should be symmetrical.

**11. How fast can I run my SSN bus?**

The default SSN bus\_clock frequency is 400 MHz. A wide high-speed bus requires physical resources. Consult with your physical implementation team to work within the physical design budget.

**12. Do all my physical blocks have to shift at the same frequency?**

No, each physical block can shift at an independent frequency.

**13. Can I change the shift frequency of one of my partitions during retargeting?**

You can lower the shift frequency of a physical block after the patterns have been created. Set the `set_current_physical_block` command to the physical block to change

the frequency and use the “set\_load\_unload\_timing\_options -shift\_clock\_period” command to change the shift clock frequency. You can see the changed frequency using the report\_load\_unload\_timing\_options or get\_load\_unload\_timing\_options commands.

**14. Do I have to insert SSH, EDT, and OCC in the same DFT insertion step?**

The SSH, EDT, and OCC do not have to be created or inserted at the same time. However, if you do not insert them at the same time in one DFT insertion step, you are responsible for making the physical connections between them and for the multiplexing logic that shares the path back to the SSH between external and internal test EDT channels.

## SSN Limitations

Use of SSN is subject to limitations in pattern writing, ScanHost identification, failure mapping, Streaming-Through-IJTAG mode, automatic configuration, and certain commands that are incompatible with SSN.

### Pattern Writing Limitations

Pattern writing using the write\_patterns command for designs including SSN has the following limitations:

- The following write\_patterns command switches are not supported for SSN:
  - -MEMory\_size
  - -SCAN\_Memory\_size
- For hierarchical designs, writing core-level patterns in the PatDB format using the write\_patterns command is the only source for retargeting scan patterns to the top level. ASCII format patterns are not supported for retargeting.
- SSN patterns do not support SVF output format.

### ScanHost Identification Limitations

Other instruments, such as EDT and OCC, can use the add\_core\_instances command to associate a core description currently in memory with a specified core instance or instances in the design.

- Do not use the add\_core\_instances command to add SSN ScanHost core instances.

The ScanHost instances are inferred from the ICL, and the active ScanHost instances are automatically identified.

## Streaming-Through-IJTAG Limitations

The status of the on-chip compare self-test is available only through the SSN parallel output bus. If you try to write the on-chip compare self-test using Streaming-Through-IJTAG, the tool reports an error.

You cannot use Streaming-Through-IJTAG if all of the following are true:

- The IJTAG control signals include additional pipelines to enable an increased TCK clock shift frequency.
- One or more of the active scan hosts on that path are capture-aligned with other active scan hosts in the streaming scan network.
- You are writing a pattern that requires capture alignment, such as “write\_patterns -pattern\_set scan” or “write\_patterns -pattern\_set retention”.

For more information about Streaming-Through-IJTAG, refer to the section “[Streaming-Through-IJTAG Scan Data](#)” on page 532.

## Automated SSN Datapath Configuration

SSN does not support automated programming of SSN multiplexers. You must manually configure these multiplexers using IJTAG to ensure that all of the SSH you intend to use are along the active datapath between the SSN bus\_in and bus\_out.

## Manual Integration of SSN Datapath

The SSN datapath is subject to the following limitations for manual integration of the datapath:

- You must ensure there are no incidental inversions or permutations of bits on the SSN bus.
- The bit sequence of the datapath must be maintained throughout the SSN bus.

## STARTPAT and ENDPAT in SSN Serial Testbench

SSN serial testbenches do not contain STARTPAT and ENDPAT statements. The STARTPAT/ENDPAT mechanism does not work with serial testbenches in SSN, because the SSN stream is a single shift pattern that cannot be broken up. If you need to break up the pattern manually, save it again using -begin and -end values instead. Refer to the section “[Verilog Plusargs](#)” in the *Tessent Scan and ATPG User’s Manual* for more information about using Verilog plusargs like STARTPAT and ENDPAT.

## Reuse of SSN Datapath Wires in SSN Bypass Mode

If the tool is in SSN bypass mode (“[set\\_ssn\\_options](#) off”) and you reuse the SSN datapath wires for EDT channels, the tool requires an ATPG model to trace through any time multiplexing and

Fifo nodes along the wires between the top-level pins and EDT. This includes the BusFrequencyDivider, BusFrequencyMultiplier, and Fifo nodes.





# Chapter 10

## Tessent Examples and Solutions

---

This chapter contains Tessent solutions for specific DFT scenarios and problems. This includes applications and flows that solve common, difficult issues encountered in DFT.

|                                                                                         |            |
|-----------------------------------------------------------------------------------------|------------|
| <b>How to Handle Parameterized Blocks During DFT Insertion .....</b>                    | <b>686</b> |
| Problem .....                                                                           | 687        |
| Solution .....                                                                          | 689        |
| Discussion .....                                                                        | 701        |
| <b>How to Avoid Simulation Issues When Using the Two-Pin Serial Port Controller ...</b> | <b>710</b> |
| <b>How to Handle Clocks Sourced by Embedded PLLs During Logic Test.....</b>             | <b>713</b> |
| <b>How to Design Capture Windows for Hybrid TK/LBIST.....</b>                           | <b>719</b> |
| <b>How to Use Boundary Scan in a Wrapped Core.....</b>                                  | <b>721</b> |
| <b>How to Use an Older Core TSDB With Newly-Inserted DFT Cores .....</b>                | <b>723</b> |
| <b>TAP Configuration .....</b>                                                          | <b>725</b> |
| Insert a Stand-Alone TAP in a Design .....                                              | 725        |
| Insert a TAP with an IJTAG Host Scan Interface .....                                    | 727        |
| Insert a Compliance Enable TAP With an IJTAG Interface .....                            | 729        |
| Insert a Daisy-Chained TAP .....                                                        | 730        |
| Insert a Primary TAP .....                                                              | 731        |
| Insert a Secondary TAP .....                                                            | 733        |
| Connecting to a Third-Party TAP .....                                                   | 736        |
| <b>How to Create a WSP Controller in Tessent Shell.....</b>                             | <b>736</b> |
| <b>How to Set Up Third-Party Synthesis .....</b>                                        | <b>744</b> |
| <b>How to Set Up Support for Third-Party OCCs .....</b>                                 | <b>747</b> |
| How to Configure Files for Third-Party OCCs .....                                       | 747        |
| Test Logic Insertion .....                                                              | 748        |
| Configuration for Scan Insertion .....                                                  | 750        |
| Pattern Generation and Simulation .....                                                 | 750        |
| <b>Post-Synthesis Update .....</b>                                                      | <b>754</b> |
| Limitations Related to SystemVerilog Interface Arrays .....                             | 754        |
| Matching Requirements for Port Names in Post-Synthesis Update .....                     | 755        |
| Design Name Mapping Commands .....                                                      | 756        |

# How to Handle Parameterized Blocks During DFT Insertion

---

Physical blocks and sub-blocks that are parameterized in the top module of the block present special challenges for DFT insertion. The Tessent solution handles all types of parameterization, including simple parameters, parameterized types, SystemVerilog interface ports with modports, and SystemVerilog generic interface ports.

The steps you must take depend on the degree of impact on the DFT insertion. This section covers three different scenarios. In all three scenarios, the parameterized block is instantiated with parameter overrides.

## Multiple Unique Sets of Parameter Overrides with No Impact on RTL DFT Insertion

The simplest of the three scenarios has multiple unique sets of parameter overrides. However, the effect of those parameter overrides on RTL DFT insertion is no different than the default parameter values. Synthesis creates multiple netlists that must be mapped to a single RTL view.

## Two or More Unique Sets of Parameter Overrides That Impact RTL DFT Insertion Differently

In the typical scenario, the parameter overrides affect RTL DFT insertion; however, the effect on insertion for each set of overrides is the same. During insertion, the block must be instantiated in the same way as it is instantiated in the parent block. Any of the override sets can be selected, because they all have the same effect on insertion. Synthesis creates multiple netlists when there are multiple unique sets of parameter overrides. Those netlists must be mapped to a single RTL view, as in the simplest scenario.

## Two or More Unique Sets of Parameter Overrides That Impact RTL DFT Insertion Differently

In the most complex scenario, each set of parameter overrides can affect RTL DFT insertion differently. The parameterized block must be uniquified such that all sets of parameter overrides associated with a given uniquified block affect RTL DFT insertion in the same way. The material in this section regarding insertion and the mapping of multiple netlists applies separately for each uniquified block.

The following topics provide full problem descriptions, solutions, and examples for all three of these scenarios:

|                         |            |
|-------------------------|------------|
| <b>Problem</b> .....    | <b>687</b> |
| <b>Solution</b> .....   | <b>689</b> |
| <b>Discussion</b> ..... | <b>701</b> |

## Problem

Different design scenarios with parameterization wrappers have different impacts on the DFT insertion process.

### **Multiple Unique Sets of Parameter Overrides with No Impact on RTL DFT Insertion**

In the simplest scenario, the parameters have no impact on the DFT logic that is being inserted. In the insertion steps the design is elaborated with its default parameter values. It makes no difference if the parameter values are different when the block's root module is instantiated into its parent module because the parameters do not affect the DFT logic that is inserted. The distinctive characteristic of this scenario is multiple unique sets of parameter overrides; therefore, synthesis creates multiple netlists. Those netlists must be mapped to a single RTL view with DFT inserted. A small modification to the standard flow enables this mapping.

In addition to having normal Verilog parameters, your design may have SystemVerilog interface ports with modports or it may have parameterized types. In the first case, as long as the modports that are specified when the block's root module is instantiated do not affect RTL DFT insertion, the above scenario and its solution are applicable. In the second case, as long as the actual type specified for the parameterized type when the block's root module is instantiated does not affect RTL DFT insertion, the above scenario and its solution are applicable.

### **One or More Unique Sets of Parameter Overrides That Impact RTL DFT Insertion in the Same Way**

In the typical scenario, the parameters do have an impact on the DFT logic inserted during RTL DFT insertion. There may be multiple unique sets of parameter overrides; however, in this scenario, all of them have the same effect on RTL DFT insertion. For example, your design may have a parameter that controls the size of a generate loop which instantiates memories. Given that memory BIST must be configured to test the right amount of memories, it is mandatory that, during DFT insertion, the parameter be set to the same value it has when the block's root module is instantiated into its parent module.

Tessent Shell provides parameterization wrappers to ensure that the block's root module is instantiated in exactly the same way as in its parent module. A parameterization wrapper is a module that instantiates the block's root module with the same parameter overrides used in the parent module's instantiation. If the design has multiple unique sets of parameter overrides, the choice of which set to target in the parameterization wrapper is arbitrary since they all have the same effect on RTL DFT insertion.

In addition, your design may have one or more of the following:

- System Verilog interface ports with modports specified in the instantiation of the block's root module. If the modports that are specified in the instantiation affect RTL DFT insertion, a parameterization wrapper must ensure that the modports that are specified during insertion are the same as in the parent module instantiation.

- System Verilog generic interface ports that are bound in the instantiation of the block's root module to System Verilog interface nets. In this case, a parameterization wrapper is always required. You cannot perform standalone elaboration of the top module of a design that has generic interface ports. The generic interface ports must be bound to actual interface nets.
- System Verilog parameterized types and the default types are overridden in the instantiation of the block's root module. If the actual types that are specified in the instantiation affect RTL DFT insertion, a parameterization wrapper ensures that the types that are specified during insertion are the same as in the parent module instantiation.

Parameterization wrappers properly handle those constructs; that is, they specify that the block's root module be instantiated with the same actuals (modports, interface nets, or types) as in the instantiation of the block in the parent module.

### **Two or More Unique Sets of Parameter Overrides That Impact RTL DFT Insertion Differently**

In the most complex scenario, your parameterized block must be uniquified in such a way that the sets of parameter overrides associated with a given uniquified block have the same effect on RTL DFT insertion. Tessent Shell provides a uniquification feature that enables you to associate multiple instantiations with a single uniquified block. The Tessent flow facilitates the solution steps for each uniquified block. Once you decide which instantiations of the block to associate with a given uniquified block, the tool performs the required uniquification, creates the needed parameterization wrappers, and provides the structures to facilitate RTL DFT insertion, synthesis, and gate-level insertion on each uniquified block and its associated netlists.

## Solution

---

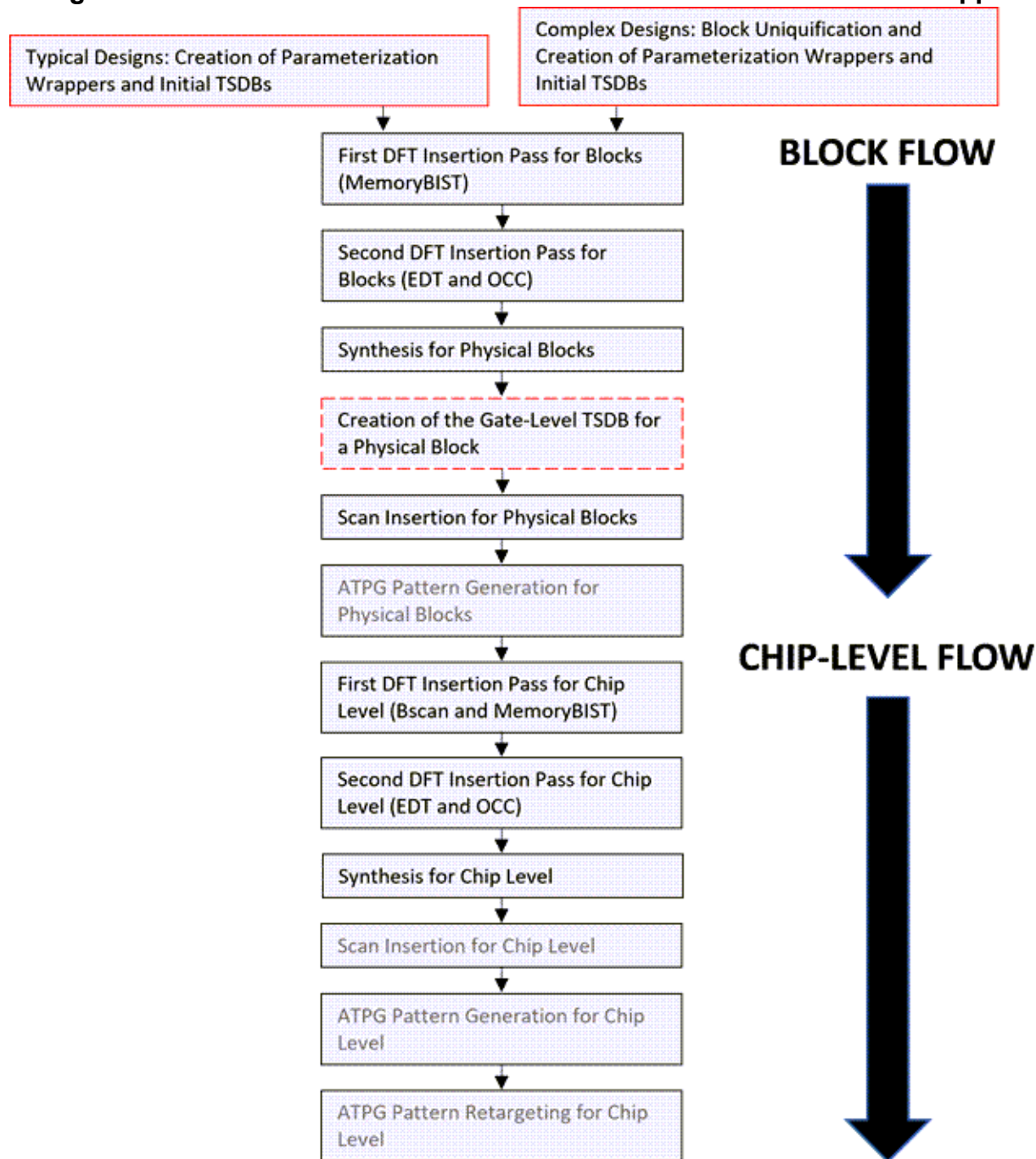
This section presents the Tessent parameterization wrappers solution flow.

“[Tessent Shell Flow for Hierarchical Designs](#)” on page 205 describes the basic flow. However, except for the simplest scenario, you must add a step that performs **uniquification** of the top module of a block (when required) and creates the parameterization wrappers. You must also replace a few existing command sequences with new ones.

[Figure 10-1](#) shows the flow for using parameterization wrappers. It uses the following conventions:

- Dashed red border—this is an additional step and is optional.
- Solid red border—this is an additional step and is mandatory.
- Solid black border—this is an existing step in the standard flow.
  - Normal black text—this step has differences with the corresponding step in the standard flow.
  - Gray text—this step is identical to the corresponding step in the standard flow.

**Figure 10-1. Hierarchical DFT Insertion Flow with Parameterization Wrappers**



The red step on the top left in [Figure 10-1](#), “Creation of parameterization wrappers and initial TSDBs”, creates parameterization wrappers for each parameterized child block that requires RTL DFT insertion. For most parameterized designs, parameterization wrappers are required for insertion as well as for synthesis. If your design has one or more unique sets of parameter overrides that impact RTL DFT insertion in the same way, this step is mandatory. We also recommend this step in the simpler case of multiple unique sets of parameter overrides with no

impact on RTL DFT insertion. Refer to “[Creation of Parameterization Wrappers and Initial TSDBs for Typical Designs](#)” on page 691 for details.

The red step on the top right in [Figure 10-1](#), “Block Uniquification and Creation of Parameterization Wrappers and Initial TSDBs”, uniquifies the design such that a given uniquified parameterized block is only instantiated with parameter overrides that impact RTL DFT insertion in the same way. The tool must create parameterization wrappers for each (possibly uniquified) parameterized block before it performs RTL DFT insertion on the block. If your design has two or more unique sets of parameter overrides that impact RTL DFT insertion differently, this step is mandatory. Refer to “[Block Uniquification and Creation of Parameterization Wrappers and Initial TSDBs for Complex Designs](#)” on page 693 for details.

The remaining steps in [Figure 10-1](#) apply to all three of the scenarios described in the [Problem](#) section.

The dashed red optional step, “Creation of the gate-level TSDB for a Physical Block”, is required only if you do not use Tessent Scan for scan insertion.

The following subsections describe the command changes in detail:

|                                                                                                                       |            |
|-----------------------------------------------------------------------------------------------------------------------|------------|
| <b>Creation of Parameterization Wrappers and Initial TSDBs for Typical Designs . . . . .</b>                          | <b>691</b> |
| <b>Block Uniquification and Creation of Parameterization Wrappers and Initial TSDBs for Complex Designs . . . . .</b> | <b>693</b> |
| <b>Modifications to the First DFT Insertion Pass for Blocks . . . . .</b>                                             | <b>695</b> |
| <b>Modifications to the Second DFT Insertion Pass for Blocks . . . . .</b>                                            | <b>696</b> |
| <b>Synthesis for Physical Blocks . . . . .</b>                                                                        | <b>697</b> |
| <b>Creation of the Gate-Level TSDB for a Physical Block . . . . .</b>                                                 | <b>697</b> |
| <b>Scan Insertion for Physical Blocks . . . . .</b>                                                                   | <b>698</b> |
| <b>Modifications to the First DFT Insertion Pass for Chips . . . . .</b>                                              | <b>699</b> |
| <b>Modifications to the Second DFT Insertion Pass for Chips . . . . .</b>                                             | <b>699</b> |
| <b>Synthesis for the Chip Level . . . . .</b>                                                                         | <b>700</b> |

## Creation of Parameterization Wrappers and Initial TSDBs for Typical Designs

This step of the flow generates the parameterization wrappers for RTL DFT insertion, and generates additional files that help automate many of the details of the synthesis flow and provide for predictable naming of the netlists generated during synthesis. You must perform this step if your parameterized design has one or more unique sets of parameter overrides that impact RTL insertion in the same way. We also recommend this step if your parameterized design has multiple unique sets of parameter overrides with no impact on RTL DFT insertion.

This step generates a parameterization wrapper for each view of a parameterized block. The tool arbitrarily chooses one of those wrappers during RTL DFT insertion to elaborate the block with the proper parameter overrides. The synthesis tool uses all of the wrappers to generate a netlist for each parameterized view of the block.

In this step of the flow, you tag the design files to indicate the block to which they belong, load the design files, and elaborate the design. You then create a configuration specification for the `process_parameterized_design_specification` command and invoke that command. That command creates the parameterization wrappers for each parameterized block and the side files for synthesis. It also creates a TSDB with a sub-directory for each block and writes the blocks to the TSDB. Finally, it verifies the correctness of the design source dictionaries for the blocks.

## Prerequisites

None.

## Procedure

1. Specify the Tessent Shell context with a unique `design_id` and set the TSDB output directory:

```
set_context dft -rtl -design_id rtl0
set_tsdb_output_directory golden_rtl_contexts.tsdb
```

2. Tag the design files to indicate the block to which they belong and load the design files:

```
set_read_design_tag <root_module_name_of_block>
read_verilog <design_files_for_the_block>
```

Repeat these two commands for each block you list in the configuration specification, including the top block of the design. You must invoke the [set\\_read\\_design\\_tag](#) command before loading the design files for each block.

3. Elaborate the design:

```
set_current_design <root_module_name_of_top_block>
```

4. Create the configuration specification:

```
read_config_data -replace -from_string {
  ParameterizedDesignSpecification(<root_module_name_of_top_block>,
    <design_level_of_block>) {
    Block(<root_module_name_of_block>,<design_level>) {
    // The design_level must be physical_block, sub_block, or chip.
    }
    ...
  }
}
```

Refer to the description of the [ParameterizedDesignSpecification](#) in the *Tessent Shell Reference Manual* for the full syntax of the configuration specification. Include a Block wrapper for each parameterized block and each ancestor block of a parameterized block. You must include the immediate parent blocks to generate the side files that promote



error-free linking of child netlists to root module names in child block instantiations in the parent block during synthesis. Include the remaining ancestor blocks to facilitate hierarchical insertion up to the top block.

5. Invoke the `process_parameterized_design_specification` command:

```
process_parameterized_design_specification  
ParameterizedDesignSpecification(<root_module_name_of_top_block>,  
<design_level_of_block>)
```

The tool creates an initial set of TSDBs for the blocks. The next step is “[Modifications to the First DFT Insertion Pass for Blocks](#)” on page 695. We also recommend that you use the synthesis approach exemplified in the scripts in “[Synthesis Guidelines for Parameterization Wrapper Flows](#)” on page 1046.

For a discussion of these steps applied to an example design, refer to “[Creation of the Parameterization Wrappers and Initial TSDBs](#)” on page 702.

## Block Uniquification and Creation of Parameterization Wrappers and Initial TSDBs for Complex Designs

This step of the flow uniquifies the design as specified in the configuration specification described below. Each parameterized block must be uniquified such that any two sets of parameter overrides used to instantiate the block have the same effect on RTL DFT insertion. You must perform this step if your parameterized design has two or more unique sets of parameter overrides that impact RTL DFT insertion differently.

This step of the flow also generates the parameterization wrappers for RTL DFT insertion, and generates additional files that help automate many of the details of the synthesis flow and provide for predictable naming of the netlists generated during synthesis.

This step generates a parameterization wrapper for each view of a uniquified block. The tool arbitrarily chooses one of those wrappers during RTL DFT insertion to elaborate the block with the proper parameter overrides. The synthesis tool uses all of the wrappers to generate a netlist for each parameterized view of the block.

In this step of the flow, you tag the design files to indicate the block to which they belong, load the design files, and elaborate the design. You then create a configuration specification for the `process_parameterized_design_specification` command and invoke that command. That command uniquifies the blocks and creates the parameterization wrappers and side files for synthesis. It creates a TSDB with a sub-directory for each block and writes the blocks to the TSDB. Finally, it verifies the correctness of the design source dictionaries for the blocks.

### Prerequisites

None.

## Procedure

1. Specify the Tessent Shell context with a unique design\_id and set the TSDB output directory:

```
set_context dft -rtl -design_id rtl0
set_tsdb_output_directory golden_rtl_contexts.tsdb
```

2. Tag the design files to indicate the block to which they belong and load the design files:

```
set_read_design_tag <root_module_name_of_block>
read_verilog <design_files_for_the_block>
```

Repeat these two commands for each block listed in the configuration specification described below, including the top block of the design. You must invoke the [set\\_read\\_design\\_tag](#) command before loading the design files for each block.

3. Elaborate the design:

```
set_current_design <root_module_name_of_top_block>
```

4. Create the configuration specification:

```
read_config_data -replace -from_string {
  ParameterizedDesignSpecification(<root_module_name_of_top_block>,\
    <design_level_of_block>) {
    Block(<root_module_name_of_block>, <design_level>) {
      // The design_level must be one of
      // physical_block, sub_block, or chip.
      UniquificationGroup {
        Instances {
          <path_to_instance_of_root_module_of_block>
          ...
          // List all instances whose parameter overrides have the
          // same effect on RTL DFT insertion.
        }
      }
      UniquificationGroup {
        Instances {
          <path_to_instance_of_root_module_of_block>
          ...
          // List all instances whose parameter overrides have the
          // same effect on RTL DFT insertion.
        }
      }
      // The uniquification groups must cover all block instances.
    }
    ...
    // List all parameterized blocks and all ancestors of such
    // blocks.
  }
}
```

Refer to the description of the [ParameterizedDesignSpecification](#) in the *Tessent Shell Reference Manual* for the full syntax of the configuration specification.

You must create UniquificationGroup wrappers if your parameterized design has two or more unique sets of parameter overrides that impact RTL DFT insertion differently. List the ancestor blocks for two reasons:

- You must list the immediate parent blocks to generate the side files that promote error-free linking of child netlists to root module names in child block instantiations in the parent block during synthesis.
- You must list the remaining ancestor blocks in case uniquification of such blocks is implied by uniquification of a child block and to facilitate hierarchical insertion up to the top block.

5. Invoke the `process_parameterized_design_specification` command as follows:

```
process_parameterized_design_specification \  
  ParameterizedDesignSpecification(<root_module_name_of_top_block>,\  
  <design_level_of_block>)
```

The tool creates an initial set of TSDBs for the blocks. The next step is “[Modifications to the First DFT Insertion Pass for Blocks](#)” on page 695. We also recommend that you use the synthesis approach exemplified in the scripts in “[Synthesis Guidelines for Parameterization Wrapper Flows](#)” on page 1046.

For a discussion of these steps applied to an example design, refer to “[Block Uniquification and Creation of Parameterization Wrappers and Initial TSDBs](#)” on page 708.

## Modifications to the First DFT Insertion Pass for Blocks

You must make the changes in this step of the flow to use parameterization wrappers.

### Prerequisites

- Refer to “[First DFT Insertion Pass: Performing Block-Level MemoryBIST](#)” on page 211 and modify that flow as follows:

### Procedure

1. Immediately after setting the TSDB output directory, open the TSDB created in the first step of the flow:

```
open_tsdb golden_rtl_contents.tsdb
```

2. Open the TSDBs for any immediate child blocks:

```
open_tsdb <root_module_name_of_block>.tsdb
```

3. Load the interface modules for any immediate child blocks in case those modules have external dependencies:

```
read_design <root_module_name_of_block> -design_id \  
  <id_of_last_RTL_insertion_pass> -view interface
```

4. Replace the invocations of the [read\\_verilog](#) command for your block's design files with the following [read\\_design](#) command:

```
read_design <root_module_name_of_block> -design_id rtl0
```

## Results

These modified steps of the flow accomplish the following actions:

- The tool automatically loads the parameterization wrapper for the block and uses it to elaborate the design.
- The tool loads any files containing external dependencies that are present in the parameterization wrapper but not in the block's RTL design files.

## Modifications to the Second DFT Insertion Pass for Blocks

You must make the changes in this step of the flow to use parameterization wrappers.

### Prerequisites

- Refer to “[Second DFT Insertion Pass: Inserting Block-Level EDT and OCC](#)” on page 213 and modify that flow as follows:

### Procedure

1. Open the TSDBs for any immediate child blocks:

```
open_tsdb <root_module_name_of_block>.tsdb
```

2. Load the interface modules for any immediate child blocks in case those modules have external dependencies:

```
read_design <root_module_name_of_block> -design_id \  
  <id_of_last_RTL_insertion_pass> -view interface
```

3. Replace the invocations of the [read\\_verilog](#) command for your block's design files with the following [read\\_design](#) command:

```
read_design <root_module_name_of_block> -design_id rtl1
```

4. Replace the invocation of [write\\_design\\_import\\_script](#) with the following:

```
set td synth/[get_master_module_name]  
file delete -force $td  
file mkdir $td  
write_design_import_script $td/analyze.tcl -use_relative_path $td \  
  -include_child_physical_block_interface_modules -replace
```

## Results

These modified steps of the flow accomplish the following actions:

- The tool automatically loads the parameterization wrapper for the block and uses it to elaborate the design.
- The tool loads any files containing external dependencies that are present in the parameterization wrapper but not in the block's RTL design files.
- The `-include_child_physical_block_interface_modules` option of the `write_design_import_script` command causes the tool to load the interface modules for the child blocks and any files needed to resolve external dependencies in the interface modules.
- The `write_design_import_script` command creates a separate *analyze.tcl* file for each parameterized view of the block to distinguish any differences in external dependencies across views. Those files are distinguished by the suffix "`_<i>`", where "i" is an integer starting at 1.

## Synthesis for Physical Blocks

The parameterization wrapper flows present special challenges for synthesis. We supply Tcl procs for interoperability purposes to help you manage the files created by the flow when using third-party synthesis tools.

Refer to "[Synthesis Guidelines for Parameterization Wrapper Flows](#)" on page 1046 for more information.

Refer to "[Synthesis for Physical Blocks Example](#)" on page 704 for a discussion of an example.

## Creation of the Gate-Level TSDB for a Physical Block

This step of the flow is required if you do not use Tessent Scan for scan insertion. You must create a gate-level TSDB to contain the synthesized physical blocks and associated side files. Repeat this step for each physical block netlist.

### Prerequisites

None.

### Procedure

1. Specify the Tessent Shell usage context with a unique `design_id`:

```
set_context dft -no_rtl -design_id gate
```

2. Specify the same TSDB output directory you used in the first and second RTL DFT insertion passes:

```
set_tsdb_output_directory <RTL DFT  
insertion output directory>
```

3. Load the synthesized netlist:

```
read_verilog <path to netlist>
```

4. Load the design data for the last RTL insertion pass:

```
read_design <root_module_name_of_RTL_block> -design_id \  
<id_of_last_RTL_insertion_pass> -no_hdl
```

5. Load the gate level interface views of any child physical blocks:

```
read_design <root_module_name_of_synthesized_netlist> \  
-design_id gate -view interface
```

6. Elaborate the design. Use the “icl\_module” option to map the netlist back to the correct RTL block:

```
set_current_design <root_module_name_of_synthesized_netlist> \  
-icl_module <root_module_name_of_RTL_block>
```

7. Set the design level:

```
set_design_level physical_block
```

8. Write the design to the TSDB:

```
write_design -tsdb -softlink_netlist
```

## Scan Insertion for Physical Blocks

The standard Tessent flow to perform scan insertion for a physical block applies with potential modifications for designs with parameterized blocks. The changes in this step are required only if your parameterized block is instantiated using more than one unique set of parameter overrides. In that case, you must add the `-icl_module` option to the `set_current_design` command to map the netlist to the correct RTL view.

## Prerequisites

- Refer to “[Performing Scan Chain Insertion: Wrapped Core](#)” on page 218 and modify that flow as follows:

## Procedure

When the design is elaborated using the `set_current_design` command, use the `-icl_module` option to map the netlist back to the correct RTL block:

```
set_current_design <root_module_name_of_synthesized_netlist> \  
-icl_module <root_module_name_of_RTL_block>
```

## Modifications to the First DFT Insertion Pass for Chips

You must make the changes in this step of the flow to use parameterization wrappers.

## Prerequisites

- Refer to “[First DFT Insertion Pass: Performing Top-Level MemoryBIST and Boundary Scan](#)” on page 231 and modify that flow as follows:

## Procedure

1. Immediately after setting the TSDB output directory, open the TSDB created in the first step of the flow:

```
open_tsdb golden_rtl_contents.tsdb
```

2. Open the TSDBs for any immediate child blocks:

```
open_tsdb <root_module_name_of_block>.tsdb
```

3. Load the interface modules for any immediate child blocks in case those modules have external dependencies:

```
read_design <root_module_name_of_block> -design_id \  
<id_of_last_RTL_insertion_pass> -view interface
```

4. Replace the invocations of the `read_verilog` command for your block’s design files with the following `read_design` command:

```
read_design <root_module_name_of_block> -design_id rtl0
```

## Modifications to the Second DFT Insertion Pass for Chips

You must make the changes in this step of the flow to use parameterization wrappers.

## Prerequisites

- Refer to “[Second DFT Insertion Pass: Inserting Top-Level EDT and OCC](#)” on page 234 and modify that flow as follows:

## Procedure

1. Open the TSDBs for any immediate child blocks:

```
open_tsdb <root_module_name_of_block>.tsdb
```

2. Load the interface modules for any immediate child blocks in case those modules have external dependencies:

```
read_design <root_module_name_of_block> -design_id \  
  <id_of_last_RTL_insertion_pass> -view interface
```

3. Replace the invocations of `read_verilog` for your block’s design files with the following:

```
read_design <root_module_name_of_block> -design_id rtl1
```

4. Replace the invocation of [write\\_design\\_import\\_script](#) with the following:

```
set td synth/[get_master_module_name]  
file delete -force $td  
file mkdir $td  
write_design_import_script $td/analyze.tcl -use_relative_path $td \  
  -include_child_physical_block_interface_modules -replace
```

The `-include_child_physical_block_interface_modules` option causes the tool to load the interface modules for the child blocks and any files needed to resolve external dependencies in the interface modules.

## Synthesis for the Chip Level

The parameterization wrapper flows present special challenges for synthesis. We supply Tcl procs for interoperability purposes to help you manage the files created by the flow when using third-party synthesis tools.

Refer to “[Synthesis Guidelines for Parameterization Wrapper Flows](#)” on page 1046 for more information.

Refer to “[Synthesis for the Chip Level Example](#)” on page 705 for a discussion of an example.



# Discussion

Parameterization wrappers enable you to succinctly describe design variations at a high level. The Tessent solution handles all types of parameterization, including simple parameters, parameterized types, SystemVerilog interface ports with modports, and SystemVerilog generic interface ports.

The following examples show how to apply this capability for different design types:

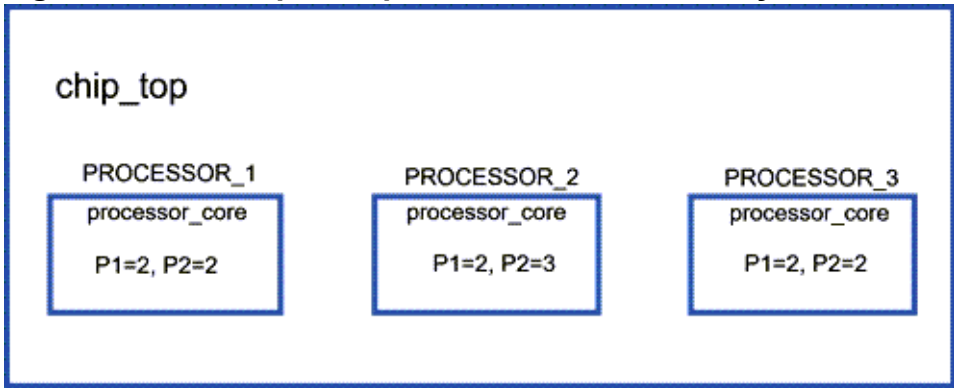
|                                                                                      |            |
|--------------------------------------------------------------------------------------|------------|
| <b>Typical Scenario Without Uniquification</b> .....                                 | <b>702</b> |
| Creation of the Parameterization Wrappers and Initial TSDBs .....                    | 702        |
| Synthesis for Physical Blocks Example .....                                          | 704        |
| Synthesis for the Chip Level Example .....                                           | 705        |
| Creation of the Gate-Level TSDB for a Physical Block (Optional) .....                | 706        |
| Scan Insertion for a Physical Block .....                                            | 706        |
| <b>Complex Scenario With Uniquification</b> .....                                    | <b>708</b> |
| Block Uniquification and Creation of Parameterization Wrappers and Initial TSDBs ... | 708        |

## Typical Scenario Without Uniquification

The example chip shown in the following figure shows how the Tessent tools handle typical design scenarios. The typical scenario has one or more unique sets of parameter overrides that impact RTL insertion in the same way. This example also applies to the simpler scenario of multiple unique sets of parameter overrides with no impact on RTL DFT insertion. The example shows how parameterization wrappers can be used to perform RTL DFT insertion in parameterized blocks in designs with one or more unique sets of parameter overrides that impact RTL insertion in the same way, which is the typical case.

In [Figure 10-2](#), processor\_core is a parameterized physical block that is instantiated three times with three sets of parameter overrides, only two of which are unique. The parameters are indicated using the symbols P1 and P2. The values in each box are the parameter overrides. In the simplest case, the overrides have no effect on RTL DFT insertion. In the typical case, the overrides do affect RTL DFT insertion and the effect is the same for all three sets.

**Figure 10-2. Example Chip With Parameterized Physical Blocks**



The following topics apply to this example of a typical scenario. They also discuss common modifications to the flow for parameterization wrappers in all three scenarios detailed in [“Problem”](#) on page 687.

|                                                                              |            |
|------------------------------------------------------------------------------|------------|
| <b>Creation of the Parameterization Wrappers and Initial TSDBs.....</b>      | <b>702</b> |
| <b>Synthesis for Physical Blocks Example .....</b>                           | <b>704</b> |
| <b>Synthesis for the Chip Level Example.....</b>                             | <b>705</b> |
| <b>Creation of the Gate-Level TSDB for a Physical Block (Optional) .....</b> | <b>706</b> |
| <b>Scan Insertion for a Physical Block .....</b>                             | <b>706</b> |

## Creation of the Parameterization Wrappers and Initial TSDBs

The script in the example below creates the parameterization wrappers for the two views of processor\_core as well as the initial TSDBs.

Refer to [“Creation of Parameterization Wrappers and Initial TSDBs for Typical Designs”](#) on page 691 for more information about this step.

In lines 39-40, the Block wrapper for processor\_core is empty because no uniquification is required.

In lines 46-47, the `process_parameterized_design_specification` command creates the parameterization wrappers, the TCD, the design source dictionaries, and the info dictionaries needed for synthesis, and then writes the initial TSDBs.

### Example 10-1. Script for Creation of Parameterization Wrappers and Initial TSDBs

```
1 set_context dft -rtl -design_id rtl0
2 # Create a separate TSDB for the parameterization wrapper files and other
3 # files needed for synthesis.
4 set_tsdb_output_directory golden_rtl_contexts.tsdb
5 read_cell_library ./library/standard_cells/tessent/adk.tcelllib
6 # Point to Verilog libraries for pad I/O macros.
7 set_design_sources -format verilog \
8   -v ./top/rtl/noncore_blocks/pad8_io_macro.v \
9   -v ./top/rtl/noncore_blocks/iopad_sel.v \
10  -v ./top/rtl/noncore_blocks/iopad.v
11 # Create a list for the top block files.
12 set_file_list(chip_top) {
13   {./top/rtl/noncore_blocks/p11.v -interface_only \
14     -exclude_from_file_dictionary}
15   {./top/rtl/chip_top.v}
16   {./top/rtl/rds_process.v}
17 }
18 # Tag the top block with the name of its top module and read the
19 # top block files.
20 set_read_design_tag chip_top
21 foreach file $file_list(chip_top) {
22   read_verilog $file
23 }
24 # Tag each child block with the name of its top module and read the
25 # block's files.
26 set_read_design_tag processor_core
27 read_verilog -f ./wrapped_cores/processor_core/rtl_file_list
28 # Point to the memory library model.
29 set_design_sources -format tcd_memory -y ./library/memories \
30   -ext tcd_memory
31 # Read the memory Verilog model.
32 read_verilog ./library/memories/*.v -exclude_from_file_dictionary
33 # Elaborate the complete design (top block and its child blocks).
34 set_current_design chip_top
35 # Create the configuration specification for this multi-level
36 # parameterized design.
37 read_config_data -replace -from_string {
38   ParameterizedDesignSpecification(chip_top, chip) {
39     Block(processor_core, physical_block) {
40     }
41   }
42 }
43 # Apply the configuration specification to the elaborated design to
44 # create the parameterization wrappers and other files mentioned above.
45 # Write out the initial TSDBs.
```

```

46 process_parameterized_design_specification \
47   ParameterizedDesignSpecification(chip_top, chip)

```

In this example, the `processor_core` physical block is instantiated with parameter overrides that affect RTL DFT insertion and requires modifications to the first pass and second pass DFT insertion steps.

Refer to the following sections for more information on modifications to the RTL DFT insertion passes:

- [“Modifications to the First DFT Insertion Pass for Blocks”](#) on page 695
- [“Modifications to the Second DFT Insertion Pass for Blocks”](#) on page 696
- [“Modifications to the First DFT Insertion Pass for Chips”](#) on page 699
- [“Modifications to the Second DFT Insertion Pass for Chips”](#) on page 699

In the step in the second pass of DFT insertion that invokes the `write_design_import_script` command, the `“-include_child_physical_block_interface_modules”` option is not applicable at the block level in this example because `processor_core` has no child blocks. However, it is applicable in the general case and in the second RTL DFT insertion pass for `chip_top` in this example.

For further discussion of this example, proceed to [“Synthesis for Physical Blocks Example”](#) on page 704.

## Synthesis for Physical Blocks Example

The script in the following example shows how the synthesis step for a physical block works with parameterization wrappers.

Refer to [“Synthesis Guidelines for Parameterization Wrapper Flows”](#) on page 1046 for more information including the `synthesize_block` Tcl proc.

This example script creates a netlist for each of the two views of `processor_core`: `processor_core_1` and `processor_core_2`.

### Example 10-2. Script for Synthesis of a Physical Block

```

1  set design_name processor_core
2  set design_id rtl2
3  set tsdb ../../wrapped_cores/processor_core/tsdb_outdir_pv1
4  set preserve_type icl extract
5  set clock_dictionary { dco_clk {3.0 dco_clk} }
6  # Directives to the synthesis tool. Add others for your flow.
7  set_app_var timing_enable_multiple_clocks_per_reg true
8  set_app_var compile_enable_constant_propagation_with_no_boundary_opt
9  set_app_var compile_seqmap_propagate_high_effort false
10 set_app_var compile_delete_unloaded_sequential_cells false
11 set_app_var sh_continue_on_error false

```

```
12 set_app_var sh_script_stop_severity E
13 # Source the Tcl script
14 source <path_to_synthesis_flow_procs>/flow_procs.tcl
15 # Invoke top-level proc
16 synthesize_block $design_name $design_id $tsdb \
17   -preserve_type $preserve_type \
18   -clock_dictionary_name clock_dictionary
```

## Synthesis for the Chip Level Example

The script in the following example shows how the synthesis step for the top level of a chip works with parameterization wrappers.

Refer to “[Synthesis Guidelines for Parameterization Wrapper Flows](#)” on page 1046 for more information including the `synthesize_block` Tcl proc.

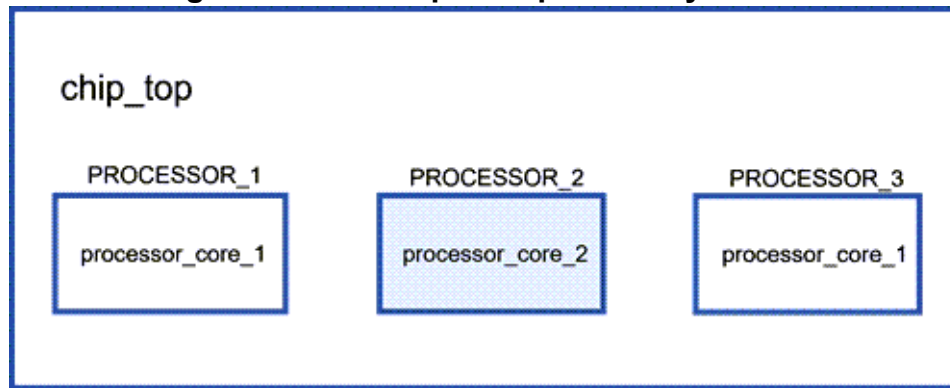
This script creates the netlist for the `chip_top`. The netlist instantiates `processor_core_1` and `processor_core_2`. This synthesis script differs from the script in “[Synthesis for Physical Blocks Example](#)” on page 704 only in the values of the `design_name` and `clock_dictionary` variables.

### Example 10-3. Script for Synthesis of the Top Block

```
1 set design_name chip_top
2 set design_id rtl2
3 set tsdb ../../top/tsdb_outdir
4 set preserve_type icl_extract
5 set clock_dictionary { REF_CLK {48.0 REF_CLK} }
6 # Directives to the synthesis tool. Add others for your flow.
7 set_app_var timing_enable_multiple_clocks_per_reg true
8 set_app_var compile_enable_constant_propagation_with_no_boundary_opt \
9   false
10 set_app_var compile_seqmap_propagate_high_effort false
11 set_app_var compile_delete_unloaded_sequential_cells false
12 set_app_var sh_continue_on_error false
13 set_app_var sh_script_stop_severity E
14 # Source the Tcl script
15 source <path>/flow_procs.tcl
16 # Invoke top-level proc
17 synthesize_block $design_name $design_id $tsdb \
18   -preserve_type $preserve_type \
19   -clock_dictionary_name clock_dictionary
```

Figure 10-3 shows the post-synthesis diagram for the example design shown in the figure in “[Typical Scenario Without Uniquification](#)” on page 702. Two separate netlists are created for `processor_core`: `processor_core_1` and `processor_core_2`.

**Figure 10-3. Example Chip - Post Synthesis**



## Creation of the Gate-Level TSDB for a Physical Block (Optional)

The script in the following example creates the gate-level TSDB. You must perform this step if you do not use Tessent scan insertion.

Line 11 maps the netlist `processor_core_1` to the top ICL module in the RTL view `processor_core`. The tool creates a new TSDB for this gate-level view in line 15. In this TSDB, the ICL is uniquified. This step must be repeated for the `processor_core_2` netlist.

Refer to the “[Scan Insertion for a Physical Block](#)” topic for a detailed discussion of the ICL uniquification process.

### Example 10-4. Script for Creation of the Gate-Level TSDB for a Physical Block

```
1 # Set the design_id in the set_context command to "gate".
2 set_context dft -no_rtl -design_id gate
3 # Output directory for the first and second RTL DFT insertion passes is
4 # assumed to be tsdb_outdir. Specify that as the output directory here:
5 set_tsdb_output_directory tsdb_outdir
6 # Load the synthesized netlist for processor_core_1 as follows:
7 read_verilog <path to netlist file for processor_core_1>
8 # Load the design data for the second RTL DFT insertion pass as follows:
9 read_design processor_core_1 -design_id rtl2 -no_hdl
10 # Elaborate the design as follows:
11 set_current_design processor_core_1 -icl_module processor_core
12 # Set the design level as follows:
13 set_design_level physical_block
14 # Write the design to the TSDB as follows:
15 write_design -tsdb -softlink_netlist
```

## Scan Insertion for a Physical Block

For `processor_core`, there are two sets of parameter overrides. Although those overrides have the same impact on DFT insertion, the synthesis tool generates two netlists—one for each unique set of parameter overrides. When the ICL extraction was performed on `processor_core`, only one ICL module was created for the top module of the block. When one of the netlists is

elaborated in preparation for scan insertion, map that netlist to the proper ICL module in the RTL view.

Use the [set\\_current\\_design](#) command as follows:

```
set_current_design processor_core_<i>-</i> -icl_module processor_core
```

where *<i>* indicates which netlist is being elaborated.

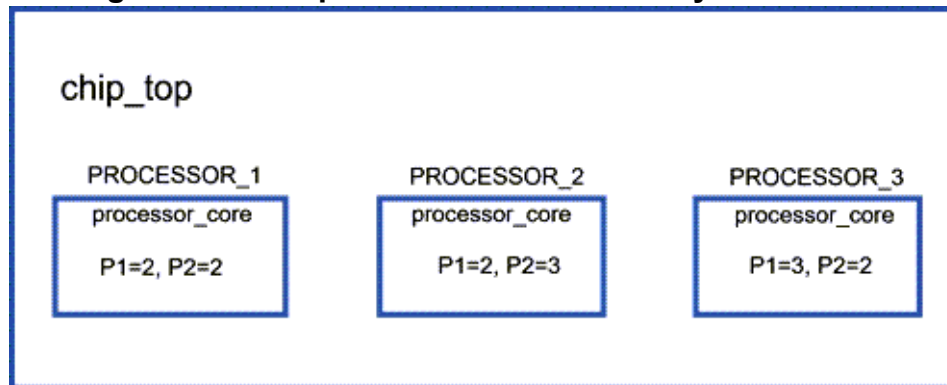
The `-icl_module` option specifies the ICL module that maps to the netlist being elaborated. After scan insertion is completed, a new TSDB is created for this gate-level view. Because there are two gate views for `processor_core`, the corresponding ICL needs to be uniquified. All references to `processor_core` are updated to either `processor_core_1` or `processor_core_2` in the `.icl`, `.tcd`, and `.tsdb_info` files.

When scan insertion is done at the parent level (`chip_top`) and the new TSDB is written, the ICL hierarchy which is referenced in the `chip_top.icl` file is updated to reflect both netlists for the `processor_core` block. The `ChildBlockModules` wrappers in the `.tcd` and `.tsdb_info` files are updated as well.

## Complex Scenario With Uniquification

The example chip in the figure below demonstrates how you can use parameterization wrappers to perform RTL DFT insertion in parameterized blocks that need uniquification.

**Figure 10-4. Chip With Parameterized Physical Blocks**



The block `processor_core` is a parameterized physical block that is instantiated three times with three sets of parameter overrides. These parameters are shown in the figure as P1 and P2. The values shown in each block are the parameter overrides. The overrides for `PROCESSOR_1` and `PROCESSOR_2` have the same effect on DFT insertion. The overrides for `PROCESSOR_3` affect DFT insertion differently than those for the other two instances.

**Block Uniquification and Creation of Parameterization Wrappers and Initial TSDBs 708**

## Block Uniquification and Creation of Parameterization Wrappers and Initial TSDBs

This example focuses on the uniquification of the `processor_core` block for the complex scenario with two or more unique sets of parameter overrides that impact RTL DFT insertion differently.

This example shows only the changes to the steps in “[Creation of the Parameterization Wrappers and Initial TSDBs](#)” on page 702 for a typical design.

The following example shows the additional uniquification steps required for a design with two or more unique sets of parameter overrides that impact RTL DFT insertion differently.

### Example 10-5. Specifying the Uniquification Requirements in the Configuration Specification

```
1 # The steps from the typical scenario up to creation of the configuration
  # specification are the same for this scenario.
2 # Create the configuration specification for this multi-level
  # parameterized design.
3 read_config_data -replace -from_string {
```



```

ParameterizedDesignSpecification(chip_top, chip) {
  Block(processor_core, physical_block) {
    UniquificationGroup {
      Instances {
        PROCESSOR_1;
        PROCESSOR_2;
      }
    }
    UniquificationGroup {
      Instances {
        PROCESSOR_3;
      }
    }
  }
}

```

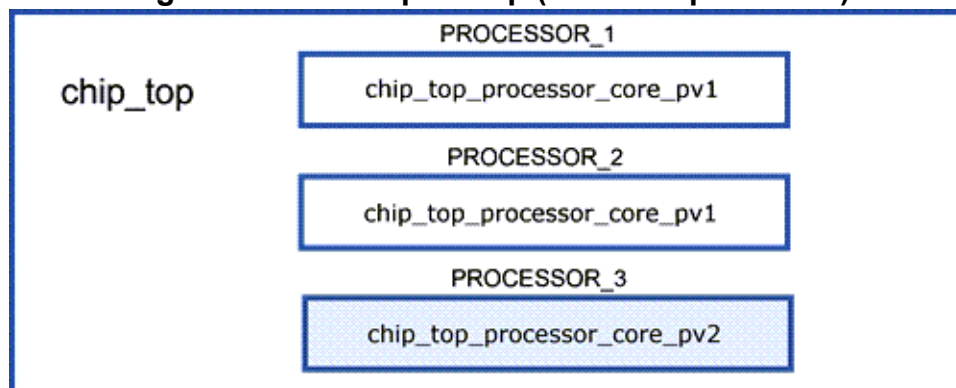
- 4 # Apply the configuration specification to the elaborated design to  
# unify the design, create the parameterization wrappers, and create  
# the info dictionary files needed for synthesis. Write out the initial  
# TSDBs.
- 5 process\_parameterized\_design\_specification \  
ParameterizedDesignSpecification(chip\_top, chip)

By default, the tool prefixes each uniquified block name with the top block name, `chip_top` in this example. This enables the uniquified block to be used in multiple contexts.

The tool uniquifies the `processor_core` block such that the parameter overrides that are associated with the instances specified in the uniquification group wrapper have the same effect on RTL DFT insertion.

Figure 10-5 shows the same design after uniquification for the example design shown in “Complex Scenario With Uniquification” on page 708. The tool uniquifies the block `processor_core` into `chip_top_processor_core_pv1` and `chip_top_processor_core_pv2`.

**Figure 10-5. Example Chip (Post-Uniquification)**



In this example, the `processor_core` physical block is instantiated with parameter overrides that affect RTL DFT insertion and requires modifications to the first pass and second pass DFT insertion steps.

Refer to the following sections for more information on modifications to the RTL DFT insertion passes:

- [“Modifications to the First DFT Insertion Pass for Blocks”](#) on page 695
- [“Modifications to the Second DFT Insertion Pass for Blocks”](#) on page 696
- [“Modifications to the First DFT Insertion Pass for Chips”](#) on page 699
- [“Modifications to the Second DFT Insertion Pass for Chips”](#) on page 699

For further discussion of this example, proceed to [“Synthesis for Physical Blocks Example”](#) on page 704.

## How to Avoid Simulation Issues When Using the Two-Pin Serial Port Controller

You may observe some compare failures during signoff simulations using the Two-Pin Serial Port (TPSP) interface. This topic describes a common cause and its solution.

### Problem

Some pad models with internal pull resistors also model delays for the pull value, which allows a “Z” or “X” value to propagate to the clock pin of the TRST generator. This causes a simulation artifact that propagates an “X” on the network’s TRST signal and effectively fails the simulation.

### Solution

The failure is purely a simulation artifact. The solution uses a Verilog side file that prevents an undetermined value from propagating to the TRST generator’s clock pins.

```
`timescale 1ns/1ns
module tpsp_artifact_solution;
  `ifndef TPSP_INST
    `define TPSP_INST top_rtl_tessent_twopinserialport_tpsp_inst
  `endif
  always@(TB.DUT_inst.`TPSP_INST.tessent_persistent_cell_spio_in_trst_clk_buf.A) begin
    if (TB.DUT_inst.`TPSP_INST.tessent_persistent_cell_spio_in_trst_clk_buf.A == 1'bx)
      force TB.DUT_inst.`TPSP_INST.tessent_persistent_cell_spio_in_trst_clk_buf.Y = 1'b0;
    else
      #0.1 release TB.DUT_inst.`TPSP_INST.tessent_persistent_cell_spio_in_trst_clk_buf.Y;
    end
  end
endmodule
```

### Using the “run\_testbench\_simulation” command inside Tessent Shell

1. Create a Verilog side file using the code above, such as “tpsp\_artifact\_solution.v”.

2. Determine the path to your TPSP instance. If you do not know the path, then you can use the commands below to find it.

```
set tsp_module [get_icl_modules -filter
tessent_instrument_subtype=="tsp_controller"]

set tsp_icl_instance [get_icl_instances -of_modules $tsp_module]

set tsp_instance [get_single_name [get_instances -of_icl_instances
$tsp_icl_instance]]

set tsp_sim_path [convert_design_path_format $tsp_instance -to_dot_separator]
```

3. Define the library sources for simulation in the Tessent shell.

```
set_simulation_library_sources -v ../data/sim_models.v ../data/pad_models.v
```

4. Run the simulation using the extra options to provide the TPSP controller's path, the extra Verilog side file, and the module inside.

```
run_testbench_simulations \
-simulation_macro_definitions "TPSP_INST=$tsp_sim_path" \
-extra_verilog_files ../data/tsp_artifact_solution.v \
-extra_top_modules tsp_artifact_solution
```

### Using the Questa Simulator outside Tessent Shell

1. Create a Verilog side file using the code above, such as "tsp\_artifact\_solution.v".
2. Load your design files with Questa's "vlog" command.
3. Load the side file and provide the path to the TPSP controller instance using the "+define+" command unless it is hard coded in the side file.

```
vlog -work dut_work "../data/tsp_artifact_solution.v" \
+define+TPSP_INST=path.to.instance.top_rtl_tessent_twopinserialport_tsp_inst
```

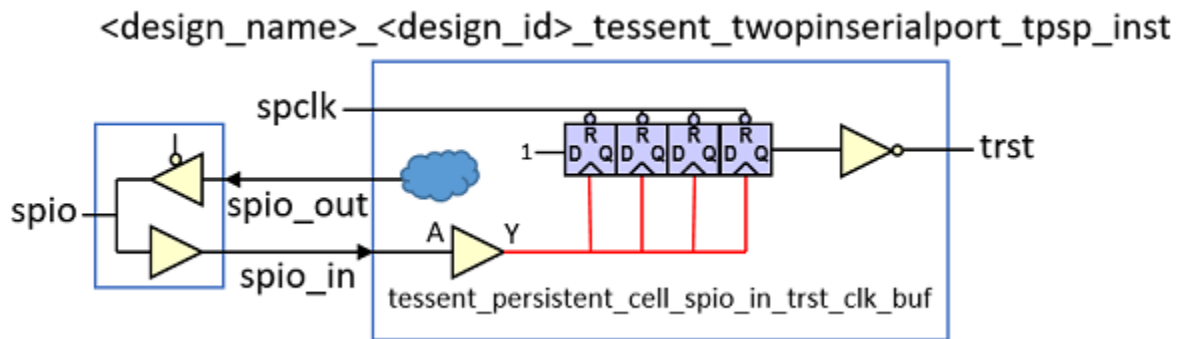
4. Run the simulation using Questa's "vsim" command, adding the module's name from the Verilog side file. (Use "acc" optimization if simulating with SDF.)

```
vsim -lib "dut_work" -L dut_work -do 'run -all; exit' \
-voptargs="+acc" \
p1_configuration \
p1_sdf \
tsp_artifact_solution
```

### Discussion

The failure is purely a simulation artifact. The proposed solution is to use a Verilog side file to prevent an undetermined value from propagating to the TRST generator's clock pins.

The [Two-Pin Serial Port](#) (TPSP) controller is an interface for the 1149.1 TAP controller and can drive a TAP with two pins on the chip boundary. A 4-bit shift register inside the TPSP generates the TRST signal pulse for the TAP controllers it drives.

**Figure 10-6. TRST Pulse Generator inside TPSP**

In Figure 1, an undetermined value (“X”) may occur on the internal “spio\_in” net and reach the clock pin of the TRST generator (shown in red), causing corruption of the simulation results. This register is sensitive to these states, and the TPSP protocol is such that the inout port is put in a high impedance state every third cycle. For these reasons, Tessent Shell requires a pull resistor modeled for the SPIO port, whether an [Inout Pad](#) or [Input Pad](#) model.

The pull value drives the input immediately if the pad has no delay modeled. The registers do not get corrupted, and the simulation ends with the correct results. However, if your pad models a delay for the pull value, you will observe the problem. This problem is purely a simulation artifact and is not present during manufacturing tests, and you can use a Verilog side file to avoid it.

To solve the simulation artifact when your pad model has a delay on the pull value, you can use a Verilog side file to prevent a corrupting value from reaching the TRST generator registers’ clock pins. The side file changes the behavior of the persistent buffer inside the TPSP controller, “tessent\_persistent\_cell\_spio\_in\_trst\_clk\_buf” (see Figure 1.), to disable the propagation of an undetermined value through the buffer. The output of the buffer cell (Y) is forced to “0” when the undetermined value is present on its input (A). The forced value is released with minor delay when the value on the buffer’s input becomes determined.

An example Verilog side file was provided in the “[Solution](#)” on page 710. The code uses the “define” statement to provide the TPSP controller instance’s design path. However, the path could be hard coded as well. Copy the code and create a new Verilog side file (Step 1). Before you can compile your new Verilog side file and run it with the testbench generated by Tessent, you must define values for several Tcl variables (Step 2). Define the library sources for simulation using the [set\\_simulation\\_library\\_sources](#) command (Step 3). Finally, run the [run\\_testbench\\_simulations](#) command with extra options to use the Verilog side file (Step 4).

You can also compile and run the new Verilog side file directly in a simulation tool such as Questa. Copy the code and create a new Verilog side file (Step 1). Load your design files into the Questa simulator (Step 2). Load the Verilog side file and define the path to the TPSP controller instance using a “+define+” statement unless it is already hard coded in the side file (Step 3). Finally, run Questa simulation using the additional options to utilize the Verilog side file (Step 4).

In summary, you can use a Verilog side file to resolve a simulation artifact. A pad modeled with a delay on the pull value exhibits the problem. You can use a Verilog side file to prevent “X” value propagation by changing the behavior of a buffer inside the TPSP. The side file solution works with a Tessent testbench inside the Tessent shell or the Questa simulator outside the Tessent shell.

## How to Handle Clocks Sourced by Embedded PLLs During Logic Test

Clocks sourced by an embedded PLL have local OCCs that are reused when testing parent physical regions. The flow is different when the parent physical regions are wrapped cores.

### Problem

For embedded PLLs, such as the one shown inside “corec” in [Figure 10-7](#), the OCC inserted on the VCO output of the PLL is used during the internal logic test modes of the core. The OCC is also reused during the internal test modes of its parent physical regions.

When the PLL is inside a wrapped core that is the child of another wrapped core, you must ensure that the OCC is still controllable by the scan chains when running logic test modes of the top level.

### Solution

#### Wrapped Core Only Used in the Top Level

If the PLL is embedded inside a wrapped core that is only used inside the top level physical region, then no further action is needed. The standard flow handles this scenario automatically, as described in “[Tessent Shell Flow for Hierarchical Designs](#)” on page 205.

#### Wrapped Cores Used Inside Other Wrapped Cores

You must use the following procedure when the wrapped core in which the embedded PLL is located has another wrapped core above it, as shown in [Figure 10-7](#)

1. Add an extra DFT signal when you process the wrapped core that embeds the PLL in “[Second DFT Insertion Pass: Inserting Top-Level EDT and OCC](#)” on page 234 (for example, when processing corec) as follows:

```
add_dft_signals promoted_cells_mode
```

2. Create a new scan mode when you process the wrapped core that embeds the PLL in the “Scan Chain Insertion” step (for example, when processing corec) as follows:

```
set promoted_instances \
  [get_attributed_objects \
    -attribute_name wrapper_type_from_clock_source \
    -object_type instance]
if {[sizeof_collection $promoted_instances] > 0} {
  add_scan_mode promoted_cells_mode \
    -include_elements [get_scan_elements \
      -of_instances $promoted_instances]
}
```

The new scan mode contains the control flip-flops of the OCCs that need to be accessible from the top-level.

3. Modify the definition of the external scan mode when you process the parent wrapped core in the scan chain insertion step of the flow (for example, when processing corea).

```
set_attribute_value corea_il -name active_child_scan_mode \
  -value promoted_cells_mode

set ext_scan_elements [add_to_collection \
  [get_scan_elements -class wrapper] \
  [get_scan_elements -of_child_scan_modes promoted_cells_mode ]]

add_scan_mode ext_mode -chain_length 32 \
  -include_elements $ext_scan_elements
```

Modifying the external scan mode definition enables you to include the promoted scan chains from the child wrapped cores in the external chain of the current wrapped core. This step ensures that the control flip-flops of the embedded OCCs are accessible by the test modes of the top level.

4. Depending on the hierarchy of your design, do one of the following:
  - If your design only uses the wrapped core at the top level of the chip (such as corea), then no further modifications are required for the standard flow.
  - If your design embeds the wrapped core inside another wrapped core (for example, there was a layer of wrapped core between corea and top), you need to recreate the promoted scan mode at the current level.

This step provides access to the OCC control bits in its external scan mode. The method shown in Step 3 is reapplied to that parent wrapped core as follows:

```
set_attribute_value corea_il -name active_child_scan_mode \
  -value promoted_cells_mode
add_scan_mode promoted_cells_mode \
  -include_elements [get_scan_elements \
    -of_child_scan_modes promoted_cells_mode]
```

## Discussion

The example chip shown in the following figure illustrates functional clocking when a PLL is embedded inside a child physical region.

**Figure 10-7. Example Chip with PLL Embedded Inside Lower Core**

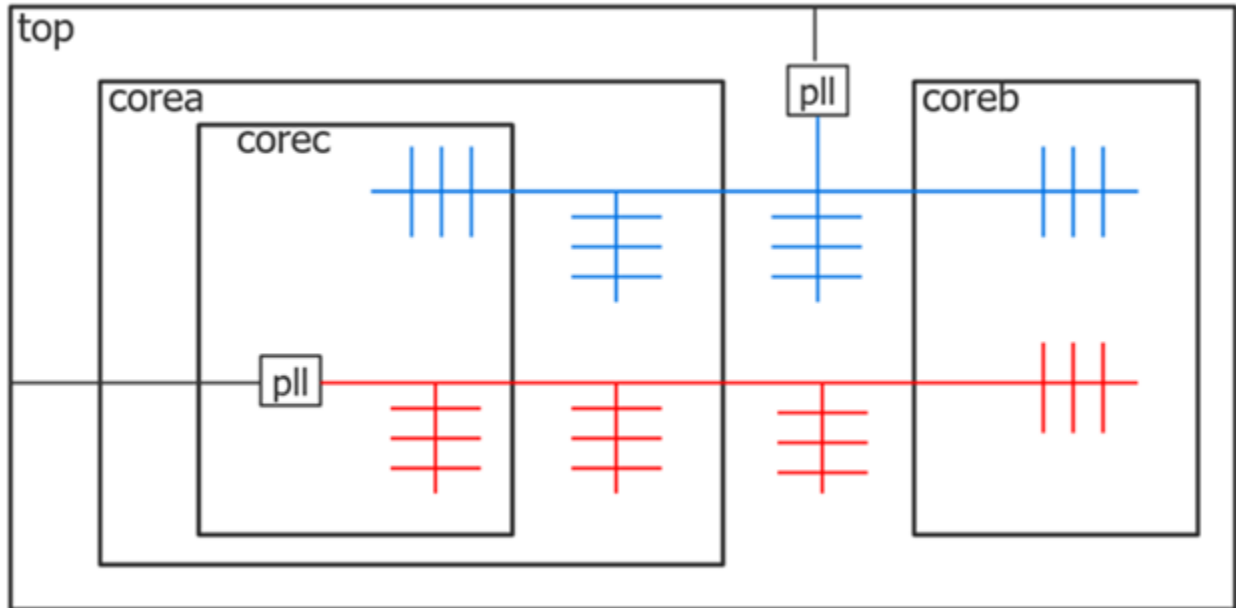
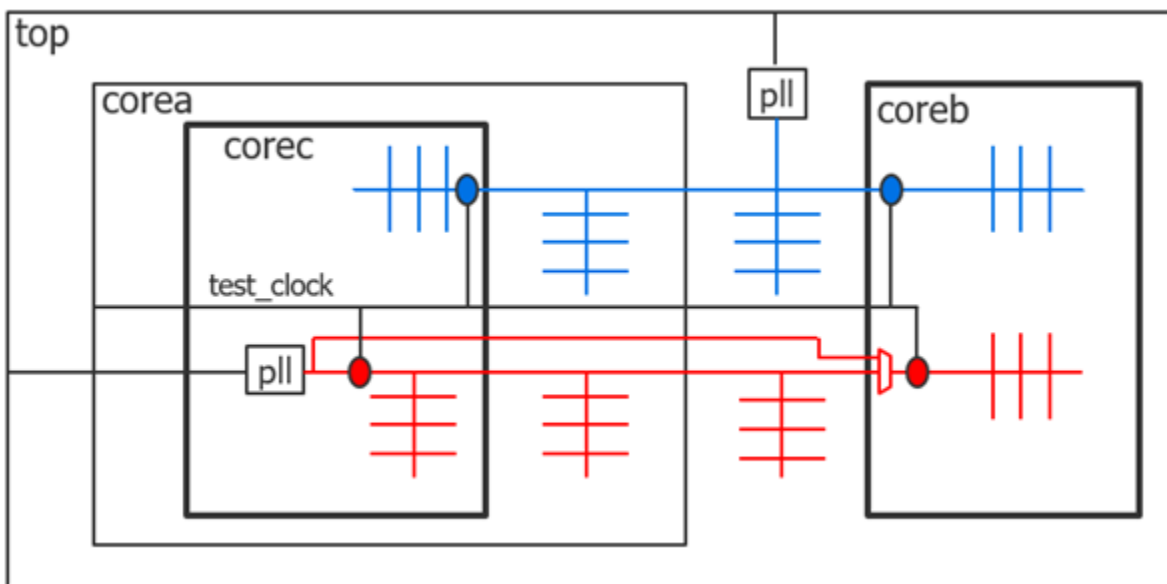


Figure 10-8 shows the location of the OCCs inserted inside corec and coreb. The two OCCs inside corec are active during its internal test modes to inject the shift clock during the shift cycles and to chop the functional clocks during capture cycles. The two OCCs inside coreb are also active during its internal test modes.

**Figure 10-8. Active OCCs During Internal Test Modes of corec and coreb**



If you want to run the internal test modes of corec in parallel with those in coreb, you need to provide a clock bypass path, such that the free running output of the PLL can continue to source the red clock of coreb when the OCC inside corec is active.

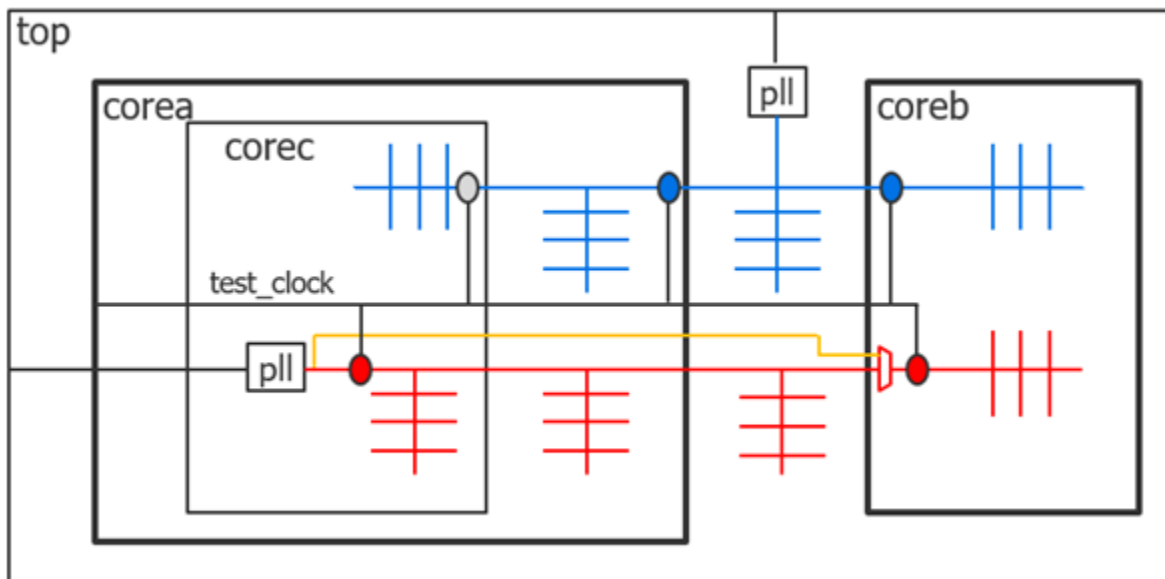
The bypass clock path is not required. The tool issues an error if you try to re-target an internal test mode of coreb in parallel with an internal test mode of corec or corea, and the bypass path is not present. The reason for this check is that the OCC at the input of the red domain of coreb expects and requires a free running clock when active. The red clock output of corea is not a free running clock when the red OCC inside corec is active. Instead, it is alternating between the shift clock and bursts of at-speed clock pulses.

If you provide the bypass clock path, you reduce the overall chip test time as you can concurrently test corec or corea with coreb. If you want to insert the clock bypass path within the DFT insertion process, use a [process\\_dft\\_specification.post\\_insertion](#) callback to create the ports and make the connections. Use the [intercept\\_connection](#) command to insert the multiplexer inside coreb. The best option to control the select input of the multiplexer is to register and add a new DFT signal. See the [register\\_static\\_dft\\_signal\\_names](#) command for more information.

The [get\\_dft\\_signal](#) command in the `process_dft_specification.post_insertion` callback gets the connection point for the added DFT signal. If the bypass path is manually added in the golden RTL, leave the select input of the multiplexer tied low and connect it to the DFT signal during the DFT process with the [add\\_dft\\_control\\_points](#) command. See of the [register\\_static\\_dft\\_signal\\_names](#) command description section for an example.

When you move up to corea, another OCC is inserted at the base of the blue clock domain to be used during its internal logic test modes, as shown in [Figure 10-9](#).

**Figure 10-9. Active OCCs During Internal Test Modes of corea and coreb**





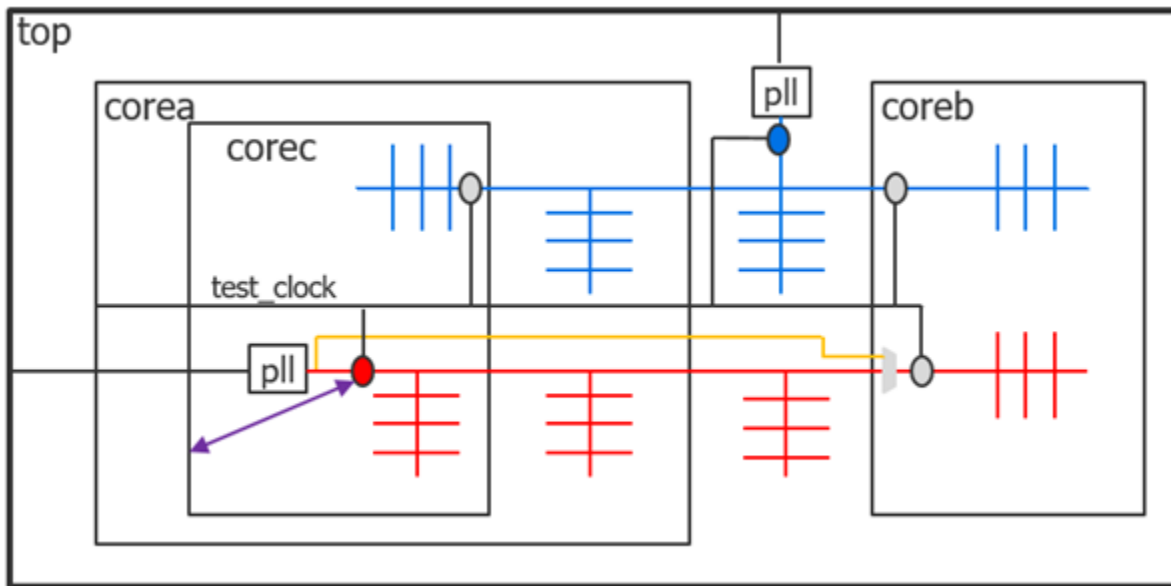
The OCC on the blue domain inside corec is kept inactive, as it is during the functional mode. The clocking of the entire blue clock domain is controlled by the OCC located inside of corea at the base of the clock tree. The red OCC inside corec is active during the internal test mode of corea to control the scannable flip-flops on the red domain inside corea, as well as the wrapper flops on the red domain inside corec. Because the “fast\_clock” input of that red OCC is not sourced by an input of corec, its scan chain is automatically promoted to its wrapper chains. This promotion enables the scan chain to be under ATPG control when running the internal test mode of corea.

If corec was not embedded within another wrapped core (such as coreb that is directly instantiated in the top level), the handling is completely automated and no deviation from the standard flow, described under “[Tessent Shell Flow for Hierarchical Designs](#)” on page 205, is necessary. However, when the wrapped core containing the embedded PLL is inside another wrapped core, you must follow the steps described under “[Solution](#)” on page 713 to enable the embedded OCC inside corec to be under ATPG control at the top level.

### Extra DFT Signals

[Figure 10-10](#) shows the active OCCs when running the logic test mode of the top level. The red OCC inside corec is active, which requires that the scan segment that contains the control flip-flops of the red OCC must be accessible by the scan chains of the top level. This requirement is why Step 1 shows how to add a new DFT signal called “promoted\_cells\_mode” to use as the enable for that scan chain configuration.

The OCCs within the cores are implemented with the [shift only mode](#) parameter. This has the advantage of keeping the internal core shift timing identical between internal and external modes. The relative shift timing between the many functional clock domains and the EDT clock domain is derived from the common test\_clock entering each core. You can close timing during layout on each core and not have to wait until you run final STA from the top level to know how fast each core shifts in external mode.

**Figure 10-10. Active OCCs During Test Modes of Top Level**

If the OCCs do not have the shift only mode, the shift clock is injected through the red and the blue OCCs. The relative timing of the shift clock as it arrives at the input of the two clocks inside coreb is not known until the clock trees of the two functional clocks are completed, which does not happen until you complete the layout of the top level.

When you use the [add\\_dft\\_signals](#) command to add the `ext_ltest_en` DFT signal to a given core, the OCCs of that core are automatically equipped with the shift only mode.

### New Scan Mode

Step 2 shows how to create the scan mode that contains the control flip-flops of the promoted OCCs. The OCC instances that have their “fast\_clock” input controlled by an internal source have an attribute named “wrapper\_type\_from\_clock\_source” set on them automatically. The [get\\_attributed\\_objects](#) command is used to find them and make them part of the special scan mode.

### Promoted Scan Chains

When you get to the corea level, you need to promote the special scan chain on corec into the wrapper chain of corea, such that the control flip-flops of the red OCC are under ATPG control from the top level. This task is described in step 3.

Step 4 provides instructions for when corea is not directly instantiated into the top level, but instead is embedded within another wrapper core. In that case, you need a special scan mode to collect the embedded OCC chains such that they can be included in the wrapper chains of the parent wrapped core. You follow step 3 on the parent wrapped core to include the promoted scan mode of corea within its wrapper chain.

The purple line in [Figure 10-10](#) represents the promoted scan chain that contains the control flip-flops of the red OCC. This scan segment is inserted in the “ext\_mode” of corea in step 3 such that the OCC is part of the scan chains of the top level.

## Related Topics

[Tessent OCC Types and Usage](#)

[OCC](#)

# How to Design Capture Windows for Hybrid TK/LBIST

A capture window is a group of capture cycles defined by one or more clocks. Test logic can be configured to run a selected number of patterns for each capture window. This approach gives full control over the patterns to optimize test coverage and test power consumption.

## Problem

The fundamental objectives for LBIST are increased test coverage, decreased test time, and lower power consumption for the test run. In a typical flow, the full set of data needed to perform optimal insertion to meet these objectives is not available until the gate-level netlist is ready. In practice, test circuitry is closely integrated into the design and the design suffers when test is treated as an ad-hoc component or inserted later in the gate-level netlist.

To address these issues and achieve a high level of test coverage and performance, the Hybrid TK/LBIST flow enables you to insert DFT logic at the RTL level, before the gate-level netlist is ready. Capture windows enable the flexibility to add test logic in the RTL and test accurately.

## Solution

The solution is provided in two parts:

- Define capture windows (which clocks and how many pulses from each clock).
- Determine how many patterns to run per capture domain to achieve the targeted coverage.

### Define Capture Windows

If the clocking structure is known and determined during the insertion of the IP, define capture windows for this flow by following the examples under “[NCP Index Decoder](#)” in the *Hybrid TK/LBIST Flow User's Manual*.

If the clocking structure is not yet defined or finalized, use the following procedure:

1. Create a gate-level netlist using quick synthesis.

2. Run ATPG with the following constraints:

- a. Use the same setup and constraints used for LBIST.
- b. Set the number of random patterns:

```
set_random_pattern integer
```

where the integer argument is the number of patterns that are planned for use in LBIST.

- c. Run pattern simulation:

```
simulate_patterns -source random -store all
```

- d. Report the test coverage per clock domain:

```
report_statistics -clock all
```

- e. Report the generated patterns:

```
report_patterns
```

- f. Design your capture windows using information from the ATPG run and the `report_statistics` command.
  - The ATPG run helps you identify the test clock domains in the design.
  - The `report_statistics` command helps you identify the number of clock pulses per clock domain
- g. Determine how many patterns to run for each capture domain to achieve the targeted coverage.

In the following example, the design has three clock domains, with possible interaction between CLK2 and CLK3. The highest faults per domain is at CLK2 at 78%, followed by CLK3 at 43%.

| Clock Domain Summary | % faults<br>(total) | Test Coverage<br>(total relevant) |
|----------------------|---------------------|-----------------------------------|
| CLK3_OCC             | 43.07%              | 99.13%                            |
| CLK2_OCC             | 78.80%              | 99.08%                            |
| CLK1_OCC             | 22.98%              | 98.17%                            |

### Patterns to Run per Capture Domain

For stuck-at-fault, try to use as many clock domains in the capture window as possible. This saves test time. Complete the steps under Define Capture Windows in the preceding to select the number of patterns per capture window.

In the following example, to achieve the best coverage with the lowest number of patterns, use a two-cycle capture window for 20% of the patterns and use a single CLK2\_OCC/CLK1\_OCC pulse capture window for 80% of the patterns.

| patt. # | pre-filtering<br>pattern # | type             | cycles | loads | capture_clock_sequence      |
|---------|----------------------------|------------------|--------|-------|-----------------------------|
| 0       | 0                          | basic            | 1      | 1     | [CLK1_OCC, *]               |
| 1       | 1                          | basic            | 1      | 1     | [CLK2_OCC, *]               |
| ...     | ...                        | ...              | ...    | ...   | ...                         |
| 1       | 400                        | basic            | 1      | 1     | [CLK2_OCC, *]               |
| 2       | 401                        | clock_sequential | 2      | 1     | [CLK2_OCC, *] [CLK3_OCC, *] |
| 3       | 405                        | clock_sequential | 2      | 1     | [CLK2_OCC, *] [CLK3_OCC, *] |
| 3       | 500                        | clock_sequential | 2      | 1     | [CLK2_OCC, *] [CLK3_OCC, *] |

Note: [\*] = "SHIFT\_CLOCK", which is a pulse-in-capture clock.

## How to Use Boundary Scan in a Wrapped Core

You must perform a specific procedure if a wrapped core has embedded pads that require boundary scan. Care must be taken during the insertion of the boundary scan and during ATPG.

### Problem

Pads embedded inside a wrapped core must be identified such that boundary scan cells can be added. The resulting boundary scan cells must also be made visible to the tool in order for them to be included in scan chains and to apply scan patterns.

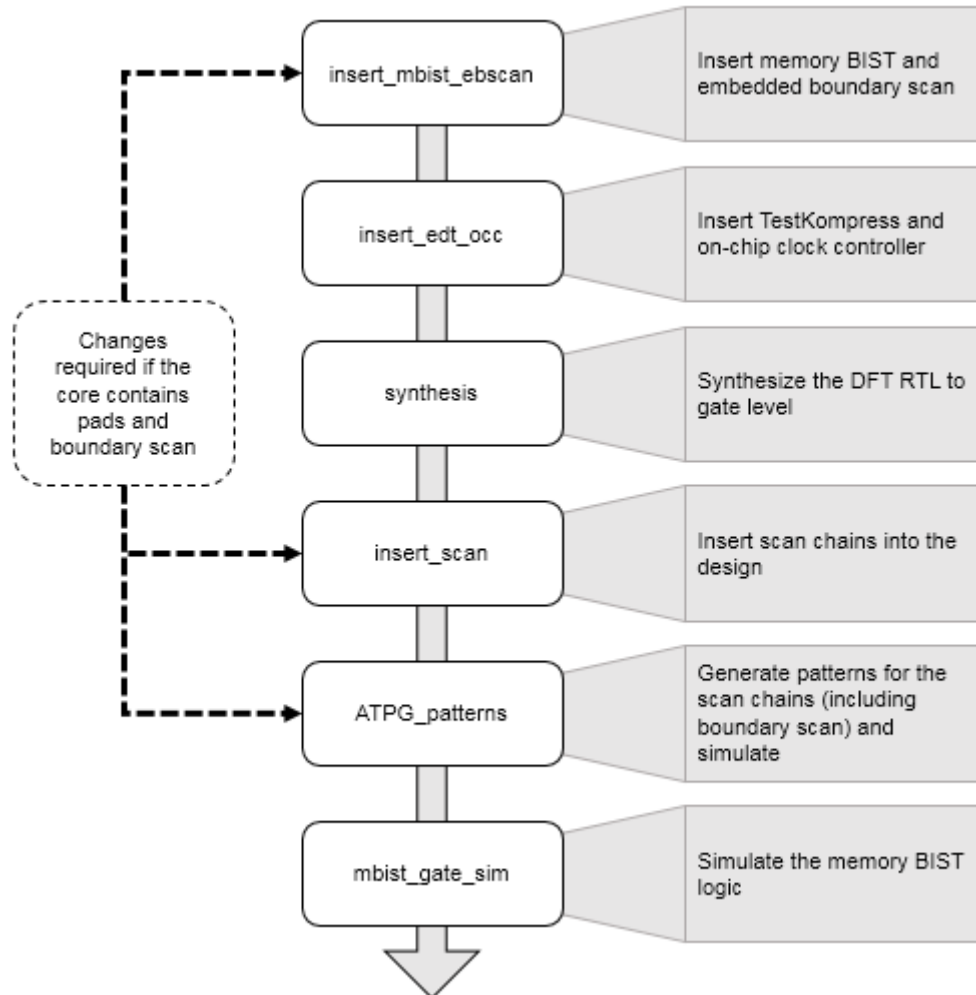
### Solution

The solutions are given for wrapped core and chip-level flows.

### Wrapped Core Flow

The following figure shows the standard command flow for creating a wrapped core that includes embedded boundary scan:

**Figure 10-11. Wrapped Core Boundary Scan Flow**



If the core contains pads and boundary scan, you must include the following commands in the wrapped core flow shown in [Figure 10-11](#) on page 722:

- Insert memory BIST and embedded boundary scan using during the `insert_mbist_ebscan` step as follows:
  - a. Specify DFT requirements to insert memory test and embedded boundary scan:

```
set_dft_specification_requirements -memory_test on \  
-boundary_scan on
```

- b. Insert embedded boundary scan cells and identify the pads:

```
set_boundary_scan_port_options \
-pad_io_ports [list taclk ta_cci0a ta_cci0b ta_cci1a ta_cci1b
ta_cci2a ta_cci2b ta_out0 ta_out1 ta_out2]
```

#### **Note**

 The order specified is the order in which the boundary scan cells are inserted.

- Specify not to add any dedicated wrapper cells on the embedded pad I/O during the insert\_scan step:

```
set_dedicated_wrapper_cell_options off \
-ports {ta_out0 ta_out1 ta_out2}

set_dedicated_wrapper_cell_options off \
-ports {ta_cci0a taclk ta_cci0b ta_cci1a ta_cci1b ta_cci2a
ta_cci2b}
```

- Preserve the boundary scan instances in the graybox during the ATPG\_patterns (graybox generation) step.

```
set_preserve_bscan {}

set_preserve_bscan \
[get_instances -hier *_tessent_bscan_logical_group_*]
analyze_graybox -preserve_instances $preserve_bscan
write_design -tsdb -graybox -verbose
```

### Chip-Level Flow

For boundary scan at the chip-level, follow the standard chip-level flow by inserting boundary scan as the first step in the flow. There are no specific additions for embedded boundary scan at this phase of the flow. The boundary scan segments from the wrapped core are picked up automatically during chip-level boundary scan insertion.

## How to Use an Older Core TSDB With Newly-Inserted DFT Cores

When you have existing cores with Tessent DFT inserted, you can use them with automation through the Tessent Shell database (TSDB).

### Prerequisites

- Legacy core TSDB file.

### Procedure

- Open the TSDB of the legacy core:

```
open_tsdb <tsdb_directory>/legacy_core.tsdb
```

Tessent Shell provides automation through the TSDB directory and TCD files. From the TSDB, the tool finds all available TCD files that apply to the design.

2. Load the current design:

```
read_design <core_name> -design_id <final_design_id>
```

3. Prepare your legacy cores by configuring them into the correct mode.

- a. If scan insertion was performed using Tessent Scan, use the [import\\_scan\\_mode](#) command during pattern generation to import the EDT and OCC settings.
- b. If the scan insertion was not performed using Tessent Scan, but there are TCD files in the TSDB, use the [add\\_core\\_instances](#) command to configure the EDT and OCC.

```
add_core_instances -module <module_name>
```

- c. If scan insertion was not performed using Tessent Scan and there are no TCD files for OCC, manually add clock information using the [add\\_clocks](#) command and define the clock definition as a third-party OCC.

```
# For OCC clock definition
add_clocks <occ_output_mux/y_pin> -capture_only
# For procedure setup of OCC
set_test_setup_icall "<OCC_name.setup>"
# For OCC clock definition file
read_procfile <occ_control.proc>
```

4. Perform pattern generation with a regular ATPG session.

## Results

You have successfully added legacy cores to newly-inserted cores using the TSDB.



# TAP Configuration

Tessent Shell typically relies on an IEEE 1149.1-based Test Access Port (TAP) as the primary access mechanism to the DFT it inserts. When boundary scan is implemented, the Tessent TAP fully complies with IEEE 1149.1 standard requirements, however many other possible configurations are possible.

**Note**  
For IEEE 1149.1-based TAP controllers using the optional TRST signal, TRST must be high for functional mode and to force a synchronous reset with the TMS and TCK signals. Driving TMS high for five TCK clock cycles forces the controller into the test/reset state. You can use the “[PatternsSpecification/AdvancedOptions/ConstantPortSettings](#)” wrapper to force TRST high, as in the following example:

```
PatternsSpecification(MyDesign, rtl, signoff) {  
  AdvancedOptions {  
    ConstantPortSettings {  
      TRST : 1 ;  
    }  
  }  
}
```

This section provides a quick reference to the various TAP insertion styles that are supported, how to specify them, and what to expect from such implementations.

|                                                                     |            |
|---------------------------------------------------------------------|------------|
| <b>Insert a Stand-Alone TAP in a Design</b> .....                   | <b>725</b> |
| <b>Insert a TAP with an IJTAG Host Scan Interface</b> .....         | <b>727</b> |
| <b>Insert a Compliance Enable TAP With an IJTAG Interface</b> ..... | <b>729</b> |
| <b>Insert a Daisy-Chained TAP</b> .....                             | <b>730</b> |
| <b>Insert a Primary TAP</b> .....                                   | <b>731</b> |
| <b>Insert a Secondary TAP</b> .....                                 | <b>733</b> |
| <b>Connecting to a Third-Party TAP</b> .....                        | <b>736</b> |

## Insert a Stand-Alone TAP in a Design

Insert a stand-alone TAP in a design. The TAP generated in this example can be further enhanced with additional IR opcodes, IJTAG host scan interfaces, and decoded outputs to enable downstream logic, such as another TAP.

### Solution

1. Set the design level to “chip.”

2. Use the following dofile example to insert the TAP:

```
set DFT [create_dft_specification]
read_config_data -in $DFT -from_string {
  IjtagNetwork {
    HostScanInterface(ijtag) {
      Interface {
        tck : tck;
      }
      Tap(single) {
      }
    }
  }
}
process_dft_specification
```

The generated TAP connects to the trst, tck, tms, tdi, and tdo pads that were present in the pre-DFT design.

## Discussion

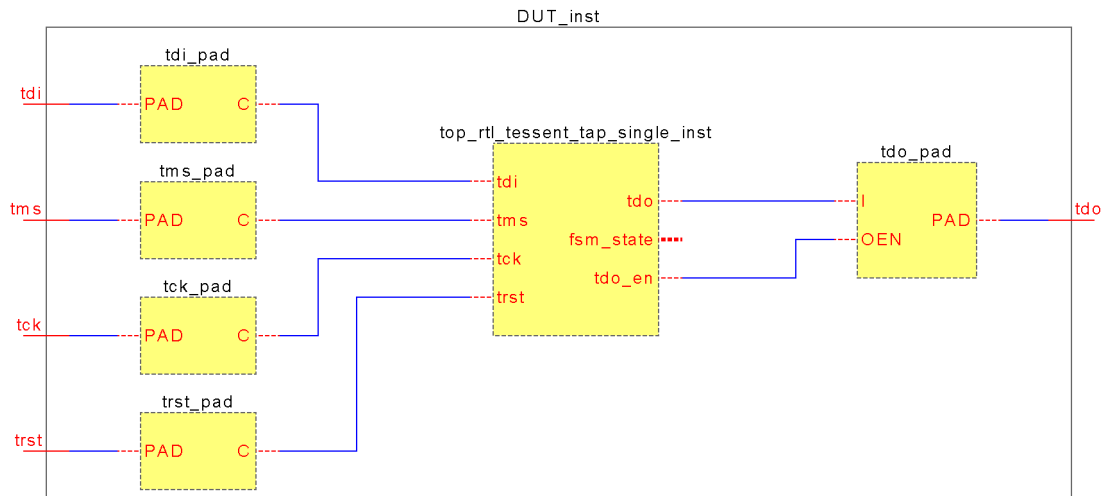
If the TAP pins in the current design do not use the default names (trst, tck, tms, tdi, or tdo), then they can be mapped using one of the following:

- The `DftSpecification::IjtagNetwork::HostScanInterface::Interface` wrapper.
- The following dofile command:

```
set_attribute_value portname -name function \
  -value [trst | tck | tms | tdi | tdo]
```

The tool issues errors if the TAP pads are not yet present in the current design. If the design is only temporary, you can specify the “`set_design_level sub_block`” command instead. Many TAP-specific DRCs are not run in such a case and other side effects may result. You should read an updated design that includes TAP pads as soon as possible.

The following is a schematic representation of this example:



## Related Topics

- [Insert a TAP with an IJTAG Host Scan Interface](#)
- [Insert a Compliance Enable TAP With an IJTAG Interface](#)
- [Insert a Daisy-Chained TAP](#)
- [Insert a Primary TAP](#)
- [Insert a Secondary TAP](#)
- [Connecting to a Third-Party TAP](#)

# Insert a TAP with an IJTAG Host Scan Interface

Insert a TAP equipped with an IJTAG host scan interface. An IJTAG network can then be connected to the TAP in a subsequent test insertion phase.

## Solution

1. Set the design level to “chip.”

2. Use the following dofile example to insert the TAP:

```
set DFT [create_dft_specification]
read_config_data -in $DFT -from_string {
  IjtagNetwork {
    HostScanInterface(ijtag) {
      Interface {
        tck : tck;
      }
      Tap(single) {
        HostIjtag(1) {
        }
      }
    }
  }
}
process_dft_specification
```

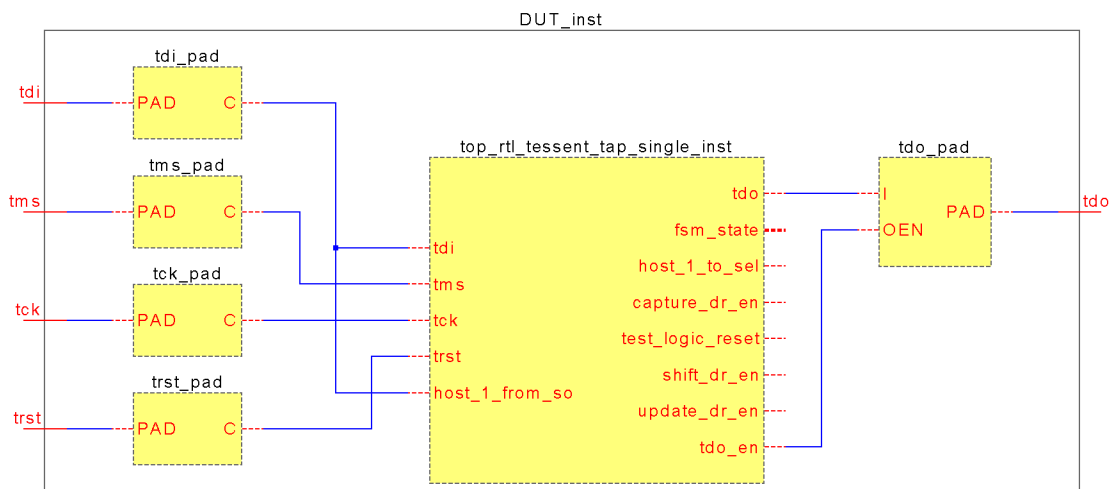
## Discussion

The host scan interface on the generated TAP can be used to control any type of IJTAG-compliant network.

The host scan interface provides a `test_logic_reset` output to reset downstream logic; it asserts the `host_<hostname>_to_sel` output to 1 when accessing the client IJTAG network.

The `capture_dr_en`, `shift_dr_en`, and `update_dr_en` outputs are consumed by the client IJTAG network or instruments after qualification with the `host_<hostname>_to_sel` port.

The following is a schematic representation of this example:



## Related Topics

[Insert a Stand-Alone TAP in a Design](#)

## Insert a Compliance Enable TAP With an IJTAG Interface

Insert a TAP in a design with compliance enable (CE) decoding logic. The CE decoding logic ensures that the TAP can only be accessed if all of the specified values are present on the primary inputs. The same scheme is also used when the TAP pins are shared with functional pins. The CE pins ensure that the TAP is only activated during intended test modes and not accidentally, for example, while the functional pins toggle during normal operation.

### Solution

1. Set the design level to “chip.”
2. Ensure that at least one primary input pad is set to 0 or 1 to enable the TAP logic.
3. Use the following dofile example to insert the TAP:

```
set DFT [ create_dft_specification \  
    -active_low_compliance_enables {tap_en0} \  
    -active_high_compliance_enables {tap_en1} ]  
read_config_data -in $DFT/IjtagNetwork/HostScanInterface(tap) \  
    -from_string {  
        Tap(CE_decoded) {  
            HostIjtag(1) {  
            }  
        }  
    }  
process_dft_specification
```

### Discussion

Note how the CE pins are specified as options to the `create_dft_specification` command.

The CE decoding module relies on the CE pin values to select the active TDO and TDO\_EN signals and route both to the primary TDO output pad.

The same CE module also gates the TMS signal such that when the proper values are not present, the TMS input on the TAP is kept to 0. This normally has the effect of “parking” the TAP in one of the stable FSM states (refer to the IEEE 1149.1 FSM state diagram for details).

---

#### Caution

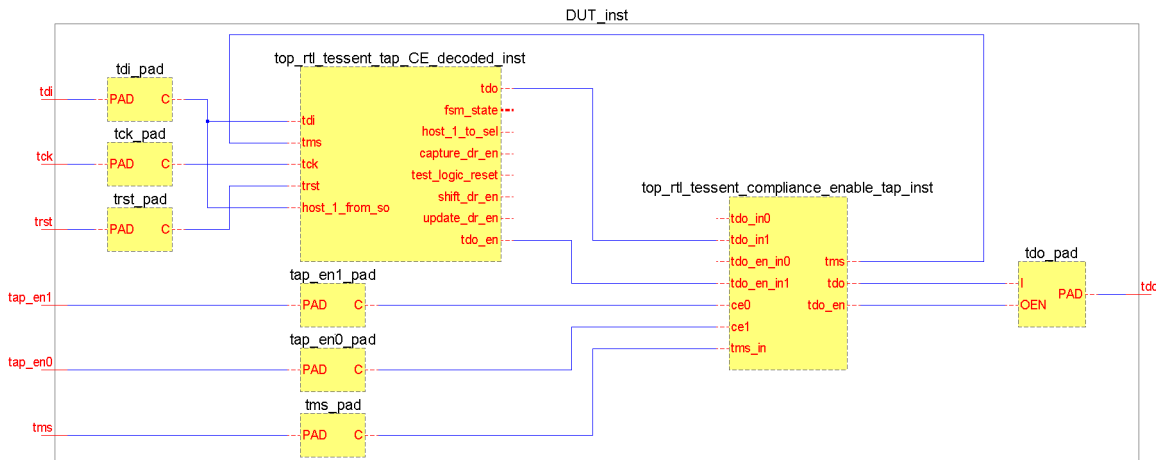
 Do not confuse the CE decoding module input port names with expected CE values! For instance, in the preceding diagram the “ce0” input is actually sourced by the CE pin driven at 1; the “ce1” input is connected to the CE pin driven at 0.

---

The general rule is that if  $n$  CE inputs are specified, the tool creates CE decoding logic with input ports ranging from `ce< $n-1$ >` down to `ce0`.

If internal CE nodes have to be used (that is, when the pre-DFT design already contains decoding logic and hookup points to connect the new TAP to), declare those hierarchical nodes in a new `DftSpecification::IjtagNetwork::HostScanInterface::Interface` wrapper. The `Tap<name>` wrapper then has to be put within the very same interface wrapper.

The following is a schematic representation of this example:



## Related Topics

[Insert a Stand-Alone TAP in a Design](#)

## Insert a Daisy-Chained TAP

This example inserts a second TAP in series with an existing one. The TRST, TCK, and TMS signals are shared between both TAPs, which implies that the complete JTAG chain (for example, top-level TDI to TDO) always shifts through both TAPs.

## Solution

### Caution

 You should not daisy-chain TAP controllers for BoundaryScan. The result is not IEEE 1149.1 compliant, and as a result you cannot create BoundaryScan patterns for the design with Tessent Shell. During BoundaryScan testing, the scan paths contain both TAP controllers. This works for the instruction and the BoundaryScan registers, but the bypass and the `device_id` path are longer than the mandated size.

1. Set the design level to “chip.”

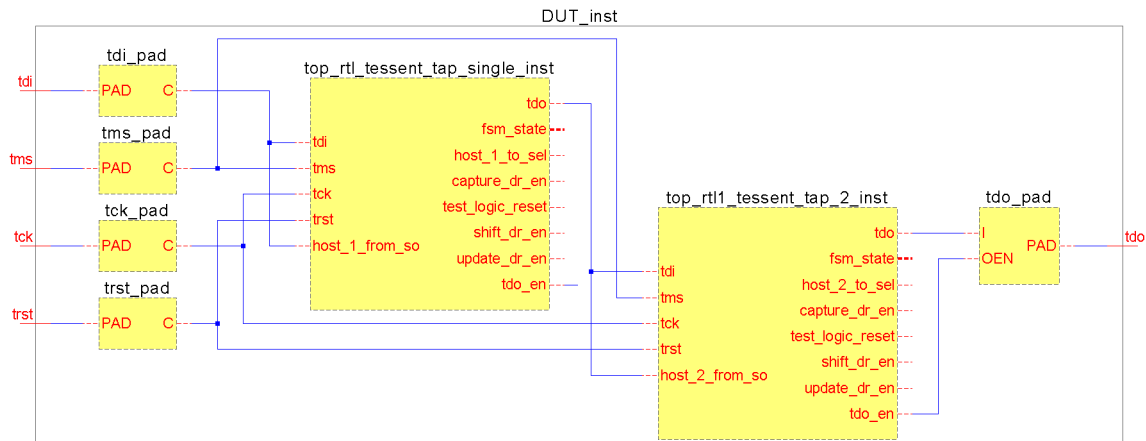
2. Use the following dofile example to insert the TAP:

```
set DFT [create_dft_specification]
read_config_data -in $DFT -from_string {
  IjtagNetwork {
    HostScanInterface(tap) {
      Interface {
        tck : tck;
        daisy_chain_with_existing_client : on;
      }
      Tap(2) {
        HostIjtag(2) {
        }
      }
    }
  }
}
process_dft_specification
```

## Discussion

The first TAP in this example has a name that starts with “top\_rtl\_”, while the second TAP begins with “top\_rtl1\_”. These names are used because this step is typically done as a subsequent insertion pass, such that the current design already contains at least one valid TAP.

The following is a schematic representation of this example:



## Related Topics

[Insert a Stand-Alone TAP in a Design](#)

## Insert a Primary TAP

This example inserts a TAP into a design that does not contain one. The generated TAP is used as a primary TAP. Primary TAPs provide outputs to enable or disable secondary TAPs, which are inserted in subsequent insertion passes.

## Solution

1. Set the design level to “chip.”
2. In the IjtagNetwork::HostScanInterface::Tap wrapper, create one or several DataOut output ports on the primary TAP. These enable a secondary TAP when the corresponding DataOut port is asserted:

```
set DFT [create_dft_specification]
read_config_data -in $DFT -from_string {
  IjtagNetwork {
    HostScanInterface(tap) {
      Interface {
        tck : tck;
      }
      Tap(primary) {
        HostIjtag(1) {
        }
        DataOutPorts {
          count : 1;
        }
      }
    }
  }
}
process_dft_specification
```

## Discussion

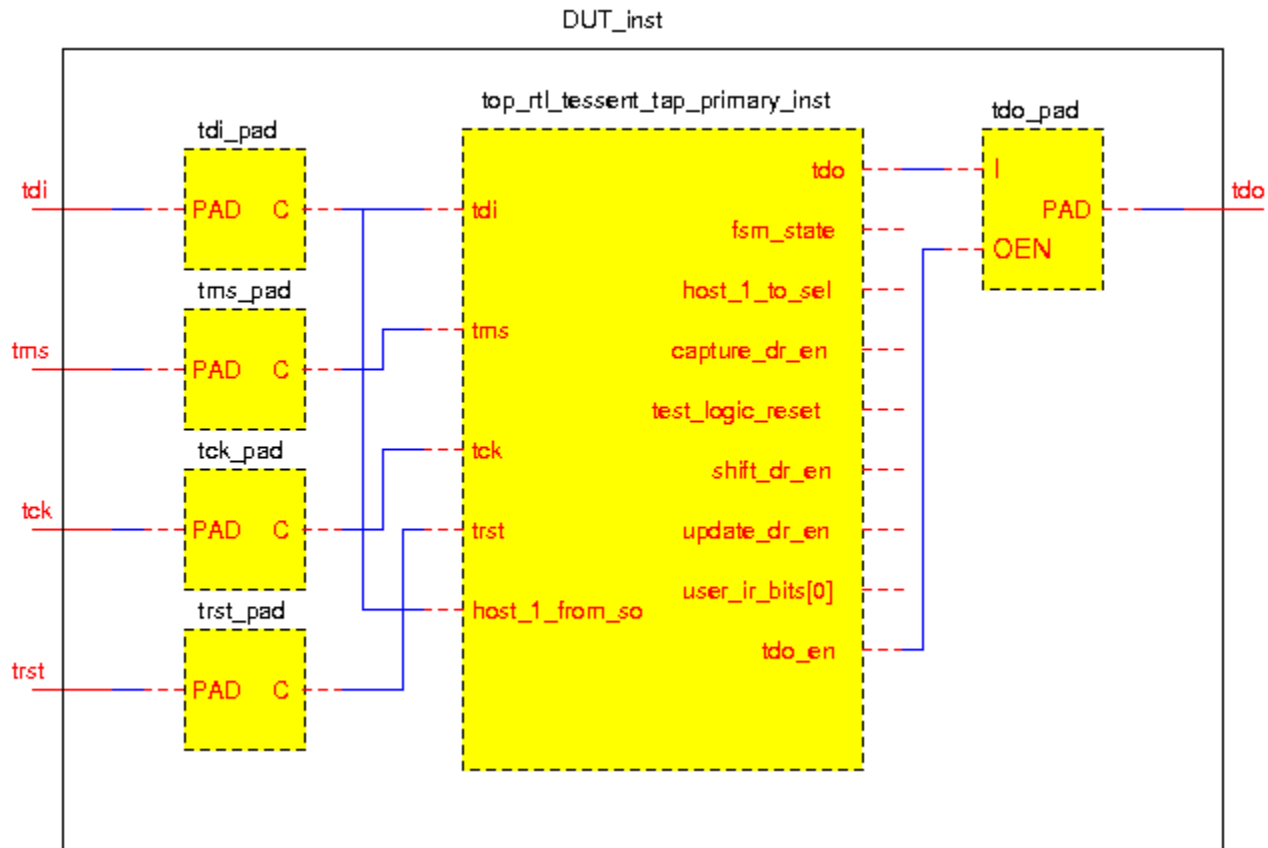
Except for the new user\_ir\_bits[0] output port created using the DataOutPorts wrapper, this TAP is functionally identical to a stand-alone TAP with a host scan interface.

The added DataOutPort ports are TAP IR bits. If you need to create these bits as DR bits, insert a TDR on the existing host scan interface using the following command:

```
create_dft_specification -existing_ijtag_host_scan_in hierarchical_pin
```



The following is a schematic representation of this example:



## Related Topics

[Insert a Stand-Alone TAP in a Design](#)

[Insert a Secondary TAP](#)

## Insert a Secondary TAP

This example generates a primary TAP, a 2-to-1 scan mux, and a secondary TAP. The primary TAP is enabled by default.

## Solution

1. Set the design level to “chip.”

2. Use the following dofile example to insert the TAP:

```
set DFT [create_dft_specification]
read_config_data -in $DFT -from_string {
  IjtagNetwork {
    HostScanInterface(tap) {
      Interface {
        tck : tck;
      }
      Tap(primary) {
        HostIjtag(0) {
        }
        DataOutPorts {
          count : 1 ;
        }
      }
      ScanMux(1) {
        select : Tap(primary)/DataOut(0) ;
        Input(0) {
          Tap(secondary) {
            HostIjtag(1) {
            }
          }
        }
      }
    }
  }
}
process_dft_specification
```

## Discussion

To trace through the secondary TAP implementations, begin with the TDO output and trace back toward the primary TDI input.

The inserted ScanMux selects the TDI input by default and routes it to its own mux\_out output.

The reset value of user\_ir\_bits[0] is 1. When this output transitions to 0, the secondary TAP is enabled and the complete JTAG scan path goes through both TAPs.

---

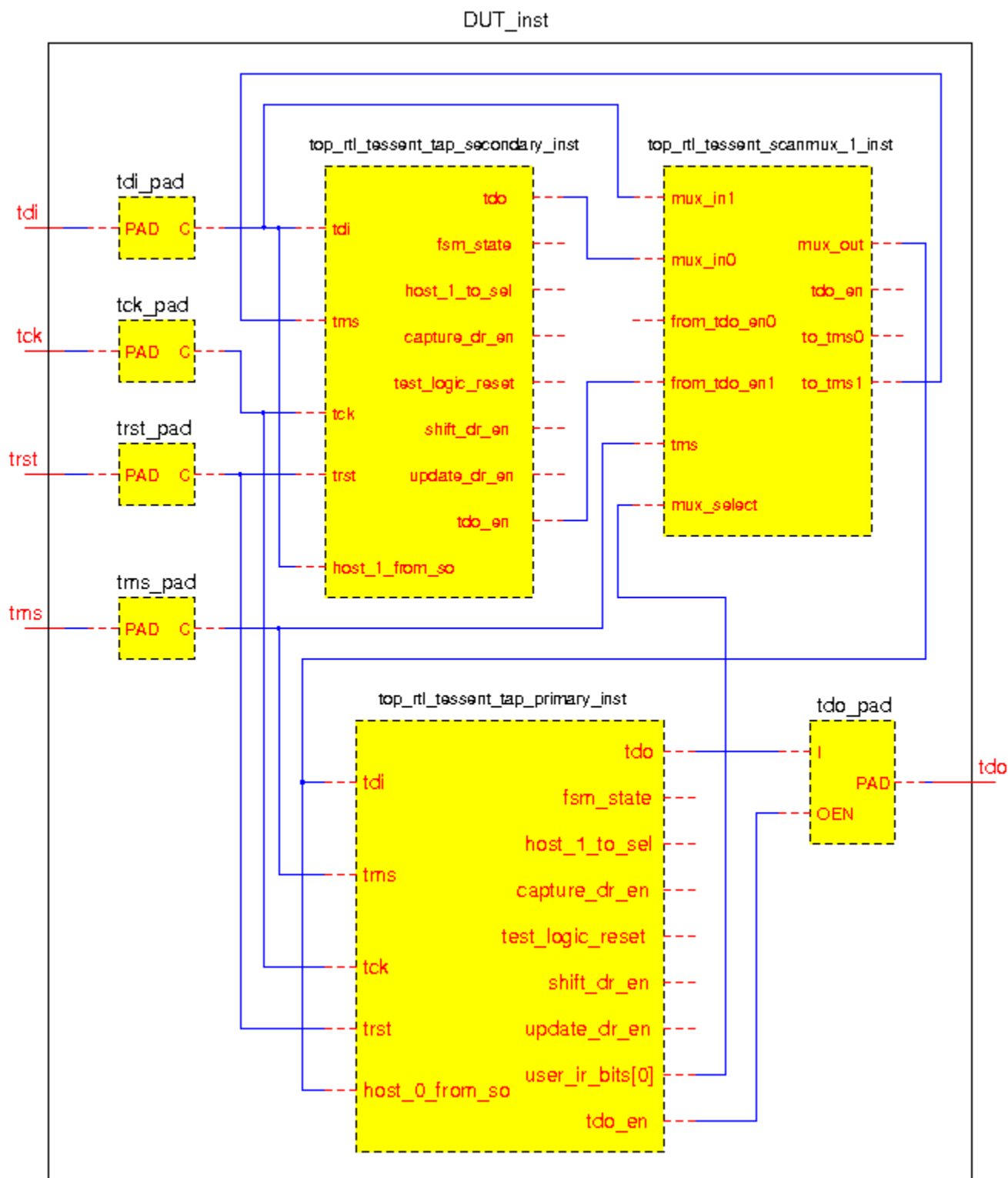
### Note



When the primary TAP is used to host the BoundaryScan chain, it should be the only one between TDI and TDO to ensure that the design is IEEE 1149.1 compliant.

---

The following is a schematic representation of this example:



## Related Topics

[Insert a Stand-Alone TAP in a Design](#)

[Insert a Primary TAP](#)

# Connecting to a Third-Party TAP

Connecting an IJTAG network or instruments to a third-party TAP requires reading the Verilog module of that TAP along with its ICL before setting the current design in Tessent Shell.

## Solution

When creating the DFT specification, specify the `-existing_ijtag_host_scan_in` port option and point to the ScanInPort on the third-party TAP that receives the scanned-out data from the inserted IJTAG network or instruments. The DFT specification is then created accordingly.

## Related Topics

[Insert a Stand-Alone TAP in a Design](#)

# How to Create a WSP Controller in Tessent Shell

Create a fully compliant IEEE 1500 Wrapper Serial Port (WSP) using Tessent Shell.

## Problem

Tessent Shell does not natively generate and insert WSP Controllers for the IEEE 1500 standard. However, you can build one using Tessent Shell's IJTAG features for the IEEE 1687 standard.

## Solution

To create a WSP controller, use an empty Verilog module and an IJTAG network that you can specify using DftSpecification wrappers for the Tessent Shell. See the “[Discussion](#)” for details of the IEEE 1687 IJTAG network model of the IEEE 1500 WSP interface.

---

### Note



See “[IJTAG Network Insertion](#)” in the *Tessent IJTAG User's Manual* for more information about the insertion flow and how to edit or modify a DftSpecification.

---

### Create a Verilog Module

Begin the implementation by preparing a WSP interface module in Verilog. Use two buffers to provide logic0 and logic1 constraints. These constraints create the default capture values that define the device ID (dev\_id).

```
module WSP ();
  buf02 logic0 (.A(1'b0), .Y());
  buf02 logic1 (.A(1'b1), .Y());
endmodule
```

### Create an IJTAG Network Model

1. Create a DftSpecification using an IjtagNetwork wrapper.

```
DftSpecification(wsp, l_ijtag) {
  IjtagNetwork {
  }
}
```

2. Create a DataInPort inside the DftSpecification/IjtagNetwork wrapper. Specify the port name as “wir\_select” to control the select input of a ScanMux Instruction Register (scanmux\_ir).

```
DataInPorts {
  port_naming : wir_select
}
```

3. Create a HostScanInterface wrapper with one ScanMux inside the DftSpecification/IjtagNetwork wrapper. This is the ScanMux to select the Instruction Register. It is driven by DataIn(0), which corresponds to the wir\_select signal in the DataInPorts wrapper.

```
HostScanInterface(wsp) {
  ScanMux(scanmux_ir) {
    select : DataIn(0);
    Input(1) {}
    Input(0) {}
  }
}
```

4. Create a TDR for the WSP Instruction Register (wsp\_ir) that connects to Input(1) of the scanmux\_ir. The length of the TDR depends on the number of registers in the hierarchy. Use a TDR length greater than or equal to the base 2 logarithm of the number of registers.

$$tdr\_length \geq \log_2(\text{number of registers})$$

```
Tdr(wsp_ir) {
  length : tdr_length;
  DataInPorts {
    connection(tdr_length-1) : logic0/Y;
    ...
    connection(1) : logic1/Y;
    connection(0) : logic0/Y;
  }
  DecodedSignal(level1_mux_select) {
    decode_values : <tdr_length>'bXX...X1;
  }
  DecodedSignal(level2_mux_select) {
    decode_values : <tdr_length>'bXX...1X;
  }
  DecodedSignal(level<tdr_length>_mux_select) {
    decode_values : <tdr_length>'b1X...XX;
  }
}
```

5. Create the remaining IJTAG network hierarchy under Input(0) of the scanmux\_ir. The bypass register must be accessible only through code consisting of zeros or ones. The logic constraints specify the device ID (dev\_id) default capture value.

```
Tdr(dev_id) {
  length : 32;
  DataInPorts {
    connection(31:18) : logic0/Y;
    connection(17) : logic1/Y;
    connection(16:2) : logic0/Y;
    connection(2:0) : logic1/Y;
  }
}
```

6. The following code is the complete DftSpecification that models the IEEE 1500 WSP.

```

DftSpecification(wsp,1_ijtag) {
  IjtagNetwork {
    DataInPorts {
      port_naming : wir_select;
    }
    HostScanInterface(wsp) {
      ScanMux (ir_dr) {
        select : DataIn(0);
        Input(1) {
          Tdr(wsp_ir) {
            length : <tdr_length>;
            DataInPorts {
              connection(tdr_length-1) : logic0/Y;
              ...
              connection(1) : logic1/Y;
              connection(0) : logic0/Y;
            }
            DecodedSignal(level1_mux_select) {
              decode_values : <tdr_length>'bXX...X1;
            }
            DecodedSignal(level2_mux_select) {
              decode_values : <tdr_length>'bXX...1X;
            }
            DecodedSignal(level<tdr_length>_mux_select) {
              decode_values : <tdr_length>'b1X...XX;
            }
          }
        }
      }
      Input(0) {
        // part of the network connected to input 0 of the scanmux
        ...
      }
    }
  }
}

```

## Discussion

You can specify IJTAG networks in Tessent Shell using DftSpecification wrappers. The DftSpecification interface can mimic an IEEE 1500 Wrapper Serial Port (WSP). The implementation is fully compliant with the IEEE 1500 standard.

The WSP interface is almost the same as an IJTAG interface fully supported by Tessent Shell with only one small modification required for the SelectWIR port. A primary input port must be created for the SelectWIR. The ijtag\_sel port is not connected in the model for the WSP.

[Table 10-1](#) summarizes the signals.

**Table 10-1. Comparison of IEEE 1687 and IEEE 1500 Ports**

| Function       | IEEE 1687 IJTAG | IEEE 1500 WSP |
|----------------|-----------------|---------------|
| Scan Input     | ijtag_si        | WSI           |
| Capture Enable | ijtag_ce        | CaptureWR     |

**Table 10-1. Comparison of IEEE 1687 and IEEE 1500 Ports (cont.)**

| Function                    | IEEE 1687 IJTAG | IEEE 1500 WSP |
|-----------------------------|-----------------|---------------|
| Shift Enable                | ijtag_se        | ShiftWR       |
| Update Enable               | ijtag_ue        | UpdateWR      |
| Test Clock                  | ijtag_tck       | WRCK          |
| Reset                       | ijtag_reset     | WRSTN         |
| Scan Output                 | ijtag_so        | WSO           |
| Select                      | ijtag_sel       | -             |
| Select Instruction Register | -               | SelectWIR     |

If the WSP module is not hosted by IJTAG, you must assert the `ijtag_sel` port to logic 1.

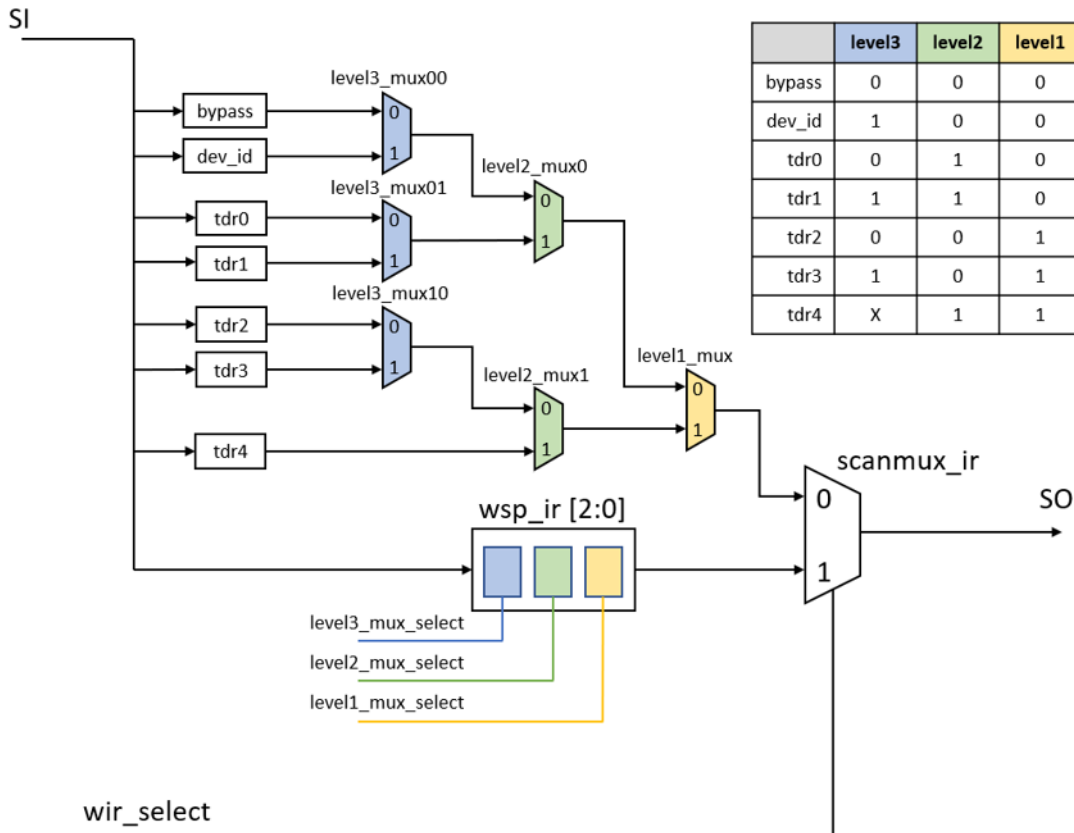
IJTAG does not have the `SelectWIR` port. Therefore, the primary input port created with the `DataInPort` wrapper has no relation to the `ToIRSelectPort` of the TAP.

The `DataIn` port is used to control the `ScanMux Instruction Register` (`scanmux_ir`). The WSP `Instruction Register` (`wsp_ir`) operates as a normal `DataRegister` in the IJTAG model.

[Figure 10-12](#) shows an IJTAG network model of the IEEE 1500 WSP interface. It consists of a bypass register, a `dev_id` register, and five other registers.



**Figure 10-12. IEEE 1687 IJTAG Network Model of the IEEE 1500 WSP Interface**



The following method is recommended to create a DftSpecification for the IJTAG network model of Figure 10-12.

Connect the scanmux\_ir to the scan output port. Use wir\_select primary input signal to drive the scanmux\_ir select signal to model the IEEE 1500 functionality.

Sort the scan muxes by hierarchy level. The yellow level1\_mux is the first level in the mux hierarchy. The green level2\_mux0 and level2\_mux1 are the second level in the mux hierarchy. The blue level3\_mux00, level3\_mux01, and level3\_mux10 are the third level in the mux hierarchy.

The wsp\_ir generates signals to control the active scan path. Each bit of the wsp\_ir controls one level of mux hierarchy. Bit 0, the yellow bit, drives the select signal of the level1\_mux. Bit 1, the green bit, drives both select signals of the two level2\_muxes. Bit 2, the blue bit, drives all three select signals of the level3\_muxes.

Table 10-2 lists the wsp\_ir bit values that drive each select signal in the mux hierarchy. A logic 0 value selects the input labeled 0 of the mux. A logic 1 value selects the input labeled 1 of the mux.

**Table 10-2. Mux Select Signals and wsp\_ir Value to Select Input 1**

| Signal Name       | wsp_ir Value |
|-------------------|--------------|
| level3_mux_select | 1XX          |
| level2_mux_select | X1X          |
| level1_mux_select | XX1          |

You can create the DftSpecification after understanding the model structure and the signal values driving the select signals of the muxes. Recreate the structure of this IJTAG network in the DftSpecification. The additional wir\_select signal drives the select signal of the final scanmux\_ir.

```
DftSpecification(test,rtl) {  
  IjtagNetwork {  
    DataInPorts {  
      port_naming : wir_select;  
    }  
    HostScanInterface(wsp) {  
      ScanMux (ir_dr) {  
        select : DataIn(0);  
        Input(1) {  
          Tdr(wsp_ir) {  
            length : 3;  
            DataInPorts {  
              connection(1) : logic0/Y;  
              connection(0) : logic1/Y;  
            }  
            DecodedSignal(level1_mux_select) {  
              decode_values : 3'bXX1;  
            }  
            DecodedSignal(level2_mux_select) {  
              decode_values : 3'bX1X;  
            }  
            DecodedSignal(level3_mux_select) {  
              decode_values : 3'b1XX;  
            }  
          }  
        }  
      }  
    }  
  }  
}
```

```

Input(0) {
  ScanMux (level1_mux) {
    select : Tdr(wsp_ir)/DecodedSignal(level1_mux_select);
    Input(1) {
      ScanMux (level2_mux1) {
        select : Tdr(wsp_ir)/DecodedSignal(level2_mux_select);
        Input(1) {
          Tdr(tdr4) {
            length : 10;
          }
        }
      }
      Input(0) {
        ScanMux (level3_mux10) {
          select : Tdr(wsp_ir)/DecodedSignal(level3_mux_select);
          Input(1) {
            Tdr(tdr3) {
              length : 8;
            }
          }
          Input(0) {
            Tdr(tdr2) {
              length : 6;
            }
          }
        }
      }
    }
  }
}

Input(0) {
  ScanMux (level2_mux0) {
    select : Tdr(wsp_ir)/DecodedSignal(level2_mux_select);
    Input(1) {
      ScanMux (level3_mux01) {
        select : Tdr(wsp_ir)/DecodedSignal(level3_mux_select);
        Input(1) {
          Tdr(tdr1) {
            length : 4;
          }
        }
      }
      Input(0) {
        Tdr(tdr0) {
          length : 2;
        }
      }
    }
  }
}

```

```
Input(0) {
  ScanMux (level3_mux00) {
    select : Tdr(wsp_ir)/DecodedSignal(level3_mux_select);
    Input(1) {
      Tdr(dev_id) {
        length : 32;
      }
    }
    Input(0) {
      Tdr(bypass) {
      }
    }
  }
}
}
```

In summary, an IJTAG network can model an IEEE 1500 WSP interface and be fully compliant with the IEEE 1500 standard. The `DftSpecification/IjtagNetwork` wrappers can describe the model for Tessent Shell to generate and insert it into your design.

## How to Set Up Third-Party Synthesis

You must synthesize DFT logic generated in the Tessent Shell flow in order to perform scan insertion, test pattern generation, and other processes.

### Solution

You must define constraints in synthesis tools to enable accurate gate level representations of the RTL. These constraints are primarily made up of clock definitions and timing constraints. Use the following procedures to set up constraints for Synopsys and Cadence synthesis tools within the Tessent Shell flow.

### Synopsys

The following procedure provides a high-level overview of the synthesis flow for Synopsys. For complete details, refer to “[Example Design Compiler Synthesis Script](#)” on page 1019 for an example.

1. Source the SDC file generated by the tool.

The generated SDC file is located in the TSDB of the current design under the design ID to which the DFT was inserted and the ICL was extracted.

2. Set and redefine the Tessent Tcl variables.

The SDC file includes variables that need to be set in order for the script to operate correctly. For example, the period of the TCK clock needs to be set. Depending on your DFT flow, you may need to define additional variables. The variables that you must set are identified in the SDC script file.

3. Verify the declaration of the functional clocks.

You must define all functional clocks. The tool automatically generates clock definitions based on the information from your DFT insertion script. You must review these definitions for accuracy.

4. Redefine other Tessent Tcl variables.

You must review and customize any additional variables. For example, a variable must be set that defines the input delay for the primary pins. This setting is based on the pre-defined TCK period and a custom multiplier value. The following is an example:

```
set tessent_input_delay [expr 0.3 * $tessent_tck_period]
```

---

**Note**



The value of `tessent_tck_period` might depend on the maximum tck clock frequency that can be applied to the circuit. See “[IJTAG Network Performance Optimization](#)” in the *Tessent IJTAG User’s Manual* showing how to maximize the frequency of the IJTAG network test clock.

---

5. Load the design into your synthesis tool.

The Tessent Shell tool automatically creates a script to import your design. The tool then sources the generated script to elaborate the design.

6. Apply the SDC constraints.

At this stage of the flow, the SDC constraints for the DFT logic are applied by sourcing the appropriate procedure. The procedures are described under the “[extract\\_sdc](#)” command of the *Tessent Shell Reference Manual*.

7. Prepare the DFT logic for synthesis.

You must set up the DFT logic by configuring synthesis tool settings to ensure proper synthesis. For example, some instances need to be preserved during synthesis to ensure

that the gate-level analysis functions correctly. The following statements are examples of configuration settings:

```
set_app_var  
compile_enable_constant_propagation_with_no_boundary_opt false  
set_preserve_instances [tessent_get_preserve_instances icl_extract]  
set_boundary_optimization $preserve_instances false  
set_ungroup $preserve_instances false  
set_app_var compile_seqmap_propagate_high_effort false  
set_app_var compile_delete_unloaded_sequential_cells false  
set_boundary_optimization [tessent_get_optimize_instances] true  
set_size_only -all_instances [tessent_get_size_only_instances]
```

8. Synthesize your design.

The synthesis script compiles the design to create a gate-level representation of the RTL.

9. Write out the SDC and the final gate-level netlist.

The final SDC is a combination of the functional and the DFT constraints.

### Genus

For synthesis with the Genus tool, follow the procedure for the Synopsys tool, but ensure that you adjust steps 3 through 9 to match the script shown in “[Example Genus Synthesis Script](#)” on page 1022. These steps are different because the tools use different commands to run synthesis. Otherwise, the flow and results are the same.

---

#### Tip



See “[Solutions for Genus Synthesis Issues](#)” on page 1059 if you experience issues synthesizing mixed Verilog/VHDL designs.

---

# How to Set Up Support for Third-Party OCCs

Tessent tools provide automation that enables you to insert Tessent OCCs as well as define and configure them for ATPG. If the design contains third-party OCCs, Tessent tools can set up and operate the OCC throughout the flow by reading and processing files that describe the operation of the OCC.

|                                                          |            |
|----------------------------------------------------------|------------|
| <b>How to Configure Files for Third-Party OCCs .....</b> | <b>747</b> |
| <b>Test Logic Insertion .....</b>                        | <b>748</b> |
| <b>Configuration for Scan Insertion .....</b>            | <b>750</b> |
| <b>Pattern Generation and Simulation .....</b>           | <b>750</b> |

## How to Configure Files for Third-Party OCCs

In order to automate the flow as much as possible, the following files set up and control how the tool interacts with the OCC.

### Solution

You must place the ICL and PDL files in the same directory as the Verilog of the OCC using the same file base name. For this example, the OCC Verilog module is in a file named *third\_party\_occ.v*.

Moving the ICL and PDL files into the same directory as the Verilog enables the tool to automatically load them and carry the information throughout the flow.

You must prepare the following files and definitions as described:

- **ICL** — Provide an ICL file based on the IEEE 1687 IJTAG standard to describe the ports on the OCC that need to be controlled by IJTAG during test. Name the file *third\_party\_occ.icl* and place it in the same directory as the Verilog file.
- **PDL** — Provide a PDL file based on the IEEE 1687 IJTAG standard to describe the procedures that configure the third-party OCC. Name the file *third\_party\_occ.pdl* and place it in the same directory as the Verilog file.
- **TCD Scan** — Provide a TCD Scan file based on the Tessent Core description language to describe the OCC's programming shift register (chain segment) that needs to be connected to the design's scan chains during scan insertion.

The following is an example TCD Scan file for a third-party OCC. The sub-chain port information must be included in the ScanChain wrapper and in the Clock and ScanEn wrappers to define the polarity of each port.

```
Core(third_party_occ) {  
  Scan {  
    module_type           : occ;  
    allow_scan_out_retiming : 0;  
    is_hard_module        : 1;  
    traceable             : 0;  
    pre_scan_drc_on_boundary_only : 1;  
    Clock(slow_clock) {  
      off_state : 1'b0;  
    }  
    ClockOut(clock_out_mux/y) {  
      slow_clock_input : slow_clock;  
      fast_clock_input : fast_clock;  
    }  
    ScanEn(scan_en) {  
      active_polarity : 1'b1;  
    }  
    ScanChain {  
      length      : 4;  
      scan_in_port : scan_in;  
      scan_out_port : scan_out;  
      scan_in_clock : slow_clock;  
      scan_out_clock : ~slow_clock;  
    }  
  }  
}
```

- **Clock Control Definitions** — Provide a test procedure file that contains Clock Control Definitions (CCDs) for the OCC instances in the design. For details on the format of CCDs, refer to “[Clock Control \(Optional\)](#)” on page 811.

## Test Logic Insertion

During test logic insertion (for example, EDT), the tool reads and processes the ICL and PDL files and includes this information in design files stored in the TSDB. Additionally, any OCC ports described in the ICL file that need to be controlled by IJTAG are connected to the generated IJTAG network such that the OCCs can be easily configured in subsequent steps.

See “[How to Configure Files for Third-Party OCCs](#)” on page 747 for instructions on setting up the ICL and PDL files.

## Solution

### EDT Example

The following EDT example uses a post-insertion procedure to connect the `slow_clock` port of the OCC to the newly-generated `shift_capture_clock` DFT signal. This enables the OCC and EDT logic to use the same clock source. The post-insertion script is run after the `process_dft_specification` command creates all other logic defined in the `DftSpecification`. For detailed information on how to create and use a post-insertion procedure, refer to “[process\\_dft\\_specification.post\\_insertion](#)” in the *Tessent Shell Reference Manual*.



```
set_context dft -rtl
set_tsdb_output_directory ../tsdb_outdir

# Read the design and OCC module. The ICL and PDL files with the
# same base name are automatically read from the same directory
read_verilog ../rtl/RDS_process_with_occ.v
read_verilog ../rtl/third_party_occ.v

set_current_design RDS_process
set_design_level physical_block

# Define DFT Signals for EDT and scan test.
add_dft_signals scan_en edt_update test_clock -source_node \
    {scan_en_r edt_update test_clock_r}
add_dft_signals edt_clock shift_capture_clock -create_from_other_signals
set_dft_specification_requirements -logic_test on
add_clocks clock1 -period 3ns
add_clocks clock2 -period 4ns

check_design_rules

# Create the DFT Spec for adding EDT
set_spec [create_dft_specification -sri_sib_list {edt} ]
read_config_data -in $spec -from_string {
    EDT {
        ijtag_host_interface : Sib(edt);
        Controller (c1) {
            longest_chain_range : 50, 65;
            scan_chain_count : 60;
            input_channel_count : 2;
            output_channel_count : 2;
        }
    }
}

# Post-insertion procedure to change the existing connection to
# third-party OCC's slow_clock port and connect to the shift_capture_clock
# DFT Signal
proc process_dft_specification.post_insertion {RDS_process args} {
    set occ_insts [get_instances -of_module third_party_occ -silent]
    if {[sizeof_collection $occ_insts] > 0} {
        set slow_clock_pins [get_pins slow_clock -of_instances $occ_insts]
        foreach_in_collection occ_pin $slow_clock_pins {
            delete_connection $occ_pin
            create_connection [get_dft_signal shift_capture_clock] $occ_pin
        }
    }
}
process_dft_specification
extract_icl

# Create a synthesis script of all RTL (original and newly created)
write_design_import_script use_in_synthesis.tcl -use_relative_path_to . \
-replace
exit
```

## Configuration for Scan Insertion

You must configure the tool to read the OCC files before running scan insertion. The Tessent OCC provides independent control of each clock domain in your design in the context of DFT.

See “[How to Configure Files for Third-Party OCCs](#)” on page 747 for instructions on setting up the OCC files.

### Solution

When inserting scan into the post-synthesis netlist, you must specify the location of the TCD Scan file using the `set_design_sources` command. The tool uses the description of the OCCs’ scan segment to stitch them into the design’s scan chains.

The following is an example:

```
set_context dft -scan
set_tsdb_output_directory ../tsdb_outdir

# Specify the location of the TCD Scan file that describes the OCC's scan
# segment
set_design_sources -format tcd_scan -Y ../rtl -extension tcd_scan

# Read the cell library and synthesized netlist from DC Shell
read_cell_library ../library/tessent/adk.tcelllib
read_verilog ../2.synthesis/RDS_process_synthesized.vg

# Use read_design to read in information (DFT signals, etc.) performed in
# previous pass
read_design RDS_process -design_identifier rtl -no_hdl
set_current_design RDS_process
set_system_mode analysis

# Add a scan mode and specify EDT instances to which scan chains should be
# connected
set edt_instance [get_instances -of_icl_instances [get_icl_instances \
                                                    -filter tessent_instrument_type==mentor::edt]]
add_scan_mode edt -edt_instance $edt_instance
analyze_scan_chains
insert_test_logic
exit
```

## Pattern Generation and Simulation

You must configure the tool to read the OCC files before generating patterns. The Tessent OCC provides independent control of each clock domain in your design in the context of DFT.

See “[How to Configure Files for Third-Party OCCs](#)” on page 747 for instructions on setting up the OCC files.

## Solution

### Pattern Configuration

For stuck-at ATPG, much of the setup is automatically imported using the `import_scan_mode` command. You should provide additional clock definitions and settings for the OCC for the OCC's asynchronous source clocks and the clocks on the output of the OCC instances.

```
set_context patterns -scan
set_tsdb_output_directory ../tsdb_outdir

# Read the cell library and design
read_cell_library ../library/tessent/adk.tcelllib
read_design RDS_process -design_id gate
set_current_design RDS_process

# Specify a test mode for stuck-at ATPG
set_current_mode edt_stuck

# Import scan, EDT, and clock configuration for ATPG
import_scan_mode edt

# Define external and OCC clocks
set_clock_options test_clock_r -pulse_in_capture on
set_occ_insts [get_instances -of_module third_party_occ* -silent]
if {[sizeof_collection $occ_insts] > 0} {
    foreach_in_collection occ_inst $occ_insts {
        set inst_name [get_single_name $occ_inst]
        add_clocks [get_pins ${inst_name}/clock_out_mux/y] -capture_only
    }
}
# Run iProc for third_party OCC to set default values (test_mode = 1)
set_test_setup_icall "${inst_name}.setup" -append
}
set_system_mode analysis

# Specify a procedure file containing clock control definition for the OCC
# instances
read_procfile third_party_occ_clock_controls.proc
create_patterns
write_tsdb_data -replace
write_patterns patterns/RDS_process_stuck_parallel.v -verilog -parallel \
    -replace
set_pattern_filtering -sample_per_type 2
write_patterns patterns/RDS_process_stuck_serial.v -verilog -serial \
    -replace
```

In analysis mode, specify a procedure file that contains the clock control definitions for the OCC instances.

For transition fault ATPG, a number of changes to the dofile are highlighted in the following example. Define any additional internal clocks that are needed for the third-party OCC, similar to those defined in the example (for example, if the OCC internally gates the fast clock during transition test).

Additionally, specify any parameters to the OCC's iCalls that must be set to configure the OCC for transition/fast-capture test.

You must constrain the design's I/Os that are not used for transition test.

```
set_context patterns -scan
set_tsdb_output_directory ../tsdb_outdir

# Read the cell library and design
read_cell_library ../library/tessent/adk.tcelllib
read_design RDS_process -design_id gate
set_current_design RDS_process

# Specify a test mode for transition ATPG
set_current_mode edt_transition

# Import scan, EDT, and clock configuration for ATPG
import_scan_mode edt
import_clocks

# Define external and OCC clocks
set_clock_options test_clock_r -pulse_in_capture on
set_occ_insts [get_instances -of_module third_party_occ* -silent]
if {[sizeof_collection $occ_insts] > 0} {
    foreach_in_collection occ_inst $occ_insts {
        set inst_name [get_single_name $occ_inst]
        add_clocks [get_pins ${inst_name}/clock_out_mux/y] -capture_only
        add_clocks \
            [get_pins ${inst_name}/occ_control/cgc_SHIFT_REG_CLK/clkg] \
            -pulse_in_capture
    }
}
# Execute iProc for third_party OCC and enable fast capture mode for OCCs
set_test_setup_icall "${inst_name}.setup fast_capture_mode 1" \
    -append
}

add_input_constraints -hold
add_output_masks -all

set_system_mode analysis

# Specify a procedure file containing clock control definition for the OCC
# instances
read_procfile third_party_occ_clock_controls.proc

set_fault_type transition
set_external_capture_options -pll_cycles 5 [lindex [get_timeplate_list] 0]

create_patterns
write_tsdb_data -replace
write_patterns patterns/RDS_process_transition_parallel.v -verilog \
    -parallel -replace
set_pattern_filtering -sample_per_type 2
write_patterns patterns/RDS_process_transition_serial.v -verilog \
    -serial -replace
```

## Pattern Simulation

You must run a Verilog simulation of the generated patterns to ensure no mismatches are reported and the patterns function as expected during the tester application.

For parallel load patterns specified by the “write\_patterns -parallel” command, you simulate all the patterns.

For serial load patterns, a handful of patterns are sufficient because the run time for simulating serial load patterns can be significant.

## Post-Synthesis Update

Mapping complex ports during synthesis may cause name and footprint changes in the design.

After logic synthesis with Design Compiler, Genus, or Oasys, ports may be flattened or bit-blasted, causing design footprint changes. Post-synthesis names differ from those in the initial RTL. Refer to “[ICL and TCD Post-Synthesis Update](#)” on page 98 for more information.

The [update\\_icl\\_attributes\\_from\\_design](#) command updates ICL port, instance, and module attributes for the gate views of a design. Refer to “[Updating ICL Attributes From the Design](#)” on page 99 for more information.

|                                                                            |            |
|----------------------------------------------------------------------------|------------|
| <b>Limitations Related to SystemVerilog Interface Arrays .....</b>         | <b>754</b> |
| <b>Matching Requirements for Port Names in Post-Synthesis Update .....</b> | <b>755</b> |
| <b>Design Name Mapping Commands .....</b>                                  | <b>756</b> |

## Limitations Related to SystemVerilog Interface Arrays

Designs using SystemVerilog interface arrays synthesized with Design Compiler require special handling.

If your design contains ports declared using SystemVerilog interface arrays, and your netlist is created with Synopsys Design Compiler, the syntax for these declarations should be restricted as follows:

Use the following syntax to declare the SystemVerilog interface ports array when creating your RTL design. The packed range must be from 0 to a positive right index. Without this syntax, Tessent Shell cannot match and automatically update ICL objects and TCD models post-synthesis with Design Compiler.

```
<SV interface type> <port name> [0:<positive right index>]
```

For example, if your design has an interface type named `intf1` and a port named `p1`, the port declaration in the design file should be written so that the range is restricted:

```
intf1 p1 [0:<positive_index>];
```

The left index must be 0 and the right index must be positive. If you do not follow this convention, the tool reports a warning similar to the following:

```
// Warning: SystemVerilog Interface Array: 'p1' is defined with  
// a descending range, and will cause Tessent Shell errors  
// post-synthesis(Design Compiler only).
```

## Matching Requirements for Port Names in Post-Synthesis Update

The following matching rules are supported when looking up a port name in a netlist:

1. Support the transformation performed by the Synopsys Design Compiler® tool (dc\_shell) with “change\_names -rules verilog” to remove the escaping and replace special characters with underscores (“\_”). A single underscore matches multiple underscores.
2. Support the transformation performed by Genus to get the gate name from the RTL name. All strings and numerical indices in the RTL name are preserved. Only the delimiters are changed.
3. Support the Genus transformation described in rule #2 but with escaping removed and special characters replaced with an underscore (“\_”) as in rule #1.
4. Support bit-blasted escaped names generated by a layout tool. The bit select is incorporated into the escaped name.
5. Support user specified matching rules.

## Design Name Mapping Commands

The `add_rtl_to_gates_mapping`, `report_rtl_to_gates_mapping`, and `delete_rtl_to_gates_mapping` commands have several features to help you control design name mapping.

|                                                                                                                                                   |            |
|---------------------------------------------------------------------------------------------------------------------------------------------------|------------|
| <b>Design Name Mapping .....</b>                                                                                                                  | <b>756</b> |
| <b>Default Matching Rules for the <code>get_pins</code>, <code>get_ports</code>, and <code>get_instances -match_rtl_reg</code> Commands .....</b> | <b>756</b> |

## Design Name Mapping

Use the `add_rtl_to_gates_mapping`, `report_rtl_to_gates_mapping`, and `delete_rtl_to_gates_mapping` commands to control design name mapping.

You define custom name mapping rules for RTL to post-synthesis or post-layout name mapping with the [add\\_rtl\\_to\\_gates\\_mapping](#) command. You can specify substrings, prefixes, and suffixes in addition to basic mapping rules. Use this command if the matching rules described in “[Default Matching Rules for the `get\_pins`, `get\_ports`, and `get\_instances -match\_rtl\_reg` Commands](#)” on page 756 are not sufficient for your application.

### Related Topics

[report\\_rtl\\_to\\_gates\\_mapping](#)

[delete\\_rtl\\_to\\_gates\\_mapping](#)

## Default Matching Rules for the `get_pins`, `get_ports`, and `get_instances -match_rtl_reg` Commands

The tool provides several default rules.

- **Component Names** — All strings between non-escaped delimiters (component names) in the name to be looked up (lookup name) must match the corresponding strings in a netlist name unless they contain special characters. The recognized delimiters are periods (“.”), forward slashes (“/”), brackets (“[]”), and underscores (“\_”).

There are two types of component names: subnames and indices. Any of the above characters can delimit subnames. Indices can be delimited only by brackets or underscores. A right bracket must follow a component name that is preceded by a left bracket.

---

### Note



Negative indices are not supported.

---



An example lookup name:

```
abc.def
```

The “abc” string in the name to be looked up must exactly match the string before the first delimiter in a netlist name. The “def” string in the name to be looked up must exactly match the string after the last delimiter in the same netlist name.

- **Escaping** — Escape characters are not matched. They are only used to direct the matching procedure.
- **Delimiters** — Delimiters in the lookup name are used only to extract the component names.

All component names after the first are assumed to have a leading delimiter and optionally a trailing delimiter in a netlist name. There are two cases to consider when matching the delimiters for a component name in the netlist:

- The port name in the netlist is escaped.

Possible matches for component name delimiters are as follows:

- Leading and trailing delimiters of brackets (“[]”).
- Leading delimiter of underscore (“\_”) and trailing delimiter of underscore or null.
- Leading delimiter of period (“.”) or slash (“/”) and trailing delimiter of null.

- The port name in the netlist is not escaped.

Possible matches for component name delimiters are as follows:

- Leading delimiter of underscore and trailing delimiter of underscore or null.

- **Special Characters in the Lookup Name** — When matching to a non-escaped name in the netlist, any characters not allowed by the language without escaping in the lookup name are replaced with the underscore character (“\_”). Matching allows truncation in the netlist name to two trailing underscores.

---

**Note**

---



This truncation rule applies only to underscore characters derived from special characters, not delimiters.

---

- **Bit Select Lookup Name** — A bit select lookup name (last component name is an index) can match a one-dimensional vector with the final bit select applied to the match or match directly to a bit select.



# Chapter 11

## Test Procedure File

---

Test procedure files specify how the scan circuitry within a design operates. Previously-defined scan clocks and other control signals specify the scan circuitry operation. To operate the scan circuitry in your design, the tool must have a specification of the scan circuitry and a test procedure file to describe its operation. If you used Tessent Shell to insert the test logic or used the TCD IP mapping flow, the tool automatically creates the test procedure files for most standard configurations. You can also create and modify test procedure files manually. The design rule checking (DRC) process, which occurs when you switch from setup to analysis mode, performs extensive checking to ensure the scan circuitry operates correctly.

|                                                                 |            |
|-----------------------------------------------------------------|------------|
| <b>Test Procedure File Creation</b> .....                       | <b>760</b> |
| <b>Test Procedure File Syntax</b> .....                         | <b>760</b> |
| <b>Test Procedure File Structure</b> .....                      | <b>765</b> |
| #include Statement .....                                        | 765        |
| Set Statement .....                                             | 766        |
| Alias Definition .....                                          | 769        |
| Timing Variables .....                                          | 771        |
| Timeplate Definition .....                                      | 774        |
| Multiple-Pulse Clocks .....                                     | 780        |
| Always Block .....                                              | 782        |
| Procedure Definition .....                                      | 783        |
| <b>Test Procedures</b> .....                                    | <b>795</b> |
| Test_Setup (Optional) .....                                     | 796        |
| Shift (Required) .....                                          | 799        |
| Alternate Shift Procedure (Optional) .....                      | 801        |
| Load_Unload (Required) .....                                    | 803        |
| Shadow_Control (Optional) .....                                 | 806        |
| Master_Observe (Sometimes Required) .....                       | 808        |
| Shadow_Observe (Optional) .....                                 | 808        |
| Skew_Load (Optional) .....                                      | 809        |
| Clock_Control (Optional) .....                                  | 811        |
| Clock_run (Optional) .....                                      | 823        |
| Capture Procedures (Optional) .....                             | 824        |
| Clock_sequential (Optional) .....                               | 830        |
| Init_force (Optional) .....                                     | 831        |
| Test_end (Optional, all ATPG tools) .....                       | 832        |
| Sub_procedure .....                                             | 833        |
| <b>Additional Support for Test Procedure Files</b> .....        | <b>835</b> |
| <b>Creating Test Procedure Files for End Measure Mode</b> ..... | <b>836</b> |

|                                                                     |            |
|---------------------------------------------------------------------|------------|
| <b>Serial Register Load and Unload for LogicBIST and ATPG .....</b> | <b>840</b> |
| Register Load and Unload Use Models .....                           | 840        |
| Static Versus Dynamic Register Variables .....                      | 840        |
| Test Procedure File Modifications .....                             | 841        |
| Dofile Modifications .....                                          | 844        |
| Serial Load and Unload DRC Rules .....                              | 847        |
| <b>Notes About Using the stil2tessent Tool .....</b>                | <b>853</b> |
| Extraction of Strobe Timing Information From STIL (SPF) .....       | 853        |
| The STIL ClockStructures Block .....                                | 853        |
| <b>Test Procedure File Commands and Output Formats .....</b>        | <b>854</b> |

## Test Procedure File Creation

You insert scan circuitry and create test procedure files for ATPG operations using the “patterns-scan” context of Tessent Shell. If your design already contains scan circuitry, you need to create a test procedure file to describe its operation either by hand or with Tessent Shell.

You can specify a test procedure file in setup mode using the `add_scan_groups` command. The tools can also read in procedure files by using the `read_procfile` command or the `write_patterns` command when not in setup mode. When you load more than one test procedure file, the tool merges the timing and procedure data.

You can also translate a STIL Procedure File (SPF) into a dofile and test procedure file using the `stil2tessent` tool. This tool produces a dofile that defines clocks, scan chains, scan groups, and pin constraints. This tool also creates test procedure files with a timeplate and the following standard scan procedures: `test_setup`, `load_unload`, and `shift`. For more information about `stil2tessent`, refer to “[Notes About Using the stil2tessent Tool](#)” on page 853.

You can run the `report_procedures` command to list the procedures you created and the procedures the tool created automatically.

The following subsections describe the syntax and rules of test procedure files, give examples for the various types of scan architectures, and outline the checking that determines whether the circuitry is operating correctly.

## Test Procedure File Syntax

The test procedure file uses common syntactical conventions such as bold and italic fonts, and reserved characters. The file supports Tcl conditional statements.

## Syntax Conventions

The following syntax conventions are used in this chapter:

- **Bold** — Indicates a keyword. Enter the keyword exactly as shown.
- *Italic* — Indicates lexical elements such as identifiers, strings, or numbers. Replace the italicized word with the appropriate name or integer.
- | — A vertical bar or pipe character indicates a logical “OR” as in “select ham OR ham\_not”.
- [ ] — Square brackets indicate optional elements. Do not include the brackets.
- ... — An ellipsis indicates a repeatable item or set.

## Reserved Characters

If you have a pin or pathname that uses a reserved punctuation character, you must enclose that name in quotation marks (“”). See [Table 11-1](#) for a list of reserved punctuation characters.

For example, the following statement is illegal because it uses the exclamation point outside of quotation marks.

```
force /inst_my_adder_1/xclk_header!x1!x1/op1[9] 1
```

The signal name contains a reserved punctuation character, the exclamation point (!), so it must be enclosed inside quotation marks. The correct syntax would be:

```
force "/inst_my_adder_1/xclk_header!x1!x1/op1[9]" 1
```

**Table 11-1. Reserved Punctuation Characters**

| Name                      | Character |
|---------------------------|-----------|
| Ampersand/AND             | &         |
| Caret/Circumflex/XOR      | ^         |
| Comma                     | ,         |
| Equals                    | =         |
| Exclamation mark          | !         |
| Left/Opening brace        | {         |
| Left/Opening parenthesis  | (         |
| Right/Closing brace       | }         |
| Right/Closing parenthesis | )         |
| Semicolon                 | ;         |

**Table 11-1. Reserved Punctuation Characters (cont.)**

| Name            | Character |
|-----------------|-----------|
| Vertical bar/OR |           |

Throughout this chapter, value = 0, 1, X, or Z.

## Using Tcl in the Test Procedure File

Procedure files support the Tcl conditional statements “if”, “else”, and “elseif” using the following syntax:

```
if { tcl_expr } {  
    procedure file statements  
}  
elseif { tcl_expr } {  
    procedure file statements  
}  
else {  
    procedure file statements  
}
```

Where a “tcl\_expr” is any Boolean Tcl expression that uses Tcl variables, dofile variables, or environment variables. Just as when doing variable substitution in the procedure file, other Tcl statements and defining Tcl variables are not supported. All variables must be defined in the dofile or from the shell as environment variables.

The body of these Tcl conditional statements should contain only legal procedure file syntax, not any other Tcl statements. The Tcl conditional statements are treated as preprocessor statements in the procedure file parser. They are not preserved in the tool after parsing is finished; only the procedure file code selected by the evaluation of the Tcl “if” expression is stored in the tool. Therefore, when using write\_procfile to write out the procedure file, none of the Tcl conditional statements are present, and the procedure file code not used is also not present. For more information, refer to “[The Tessent Tcl Interface](#)” on page 1033.

## Introductory Test Procedure File Example

The following is an example of a test procedure file.

```
// Comments use "//" characters
//
// Set the base time increment for use in all timeplates
set time scale 1.0 ns;

// Define the strobe time for the measure statements
set strobe_window time 1;

// This design uses a single timeplate, named "tp1", for all
// vectors.

timeplate tp1 =
    force_pi 0;
    measure_po 1;
    pulse CLK0_7 2 1;
    pulse CLK8_15 2 1;
    period 4;
end;

// The shift and load_unload procedures define how the design
// must be configured to allow shifting data through the scan
// chains. The procedures define the timeplate to use
// and the scan group that it references.

procedure shift =
    scan_group grp1;
    timeplate tp1;
    cycle =
        force_sci;
        measure_sco;
        pulse CLK8_15;
        pulse CLK0_7;
    end;
end;

procedure load_unload =
    scan_group grp1;
    timeplate tp1;
    cycle=
        force CLEAR 0;
        force CLK0_7 0;
        force CLK8_15 0;
        force scen1 1;
    end;
    apply shift 8;
end;

// The capture procedure is a "non-scan" procedure. This
// procedure describes the timeplate to use for the
// capture cycle. It also defines the number of cycles
// to use in the capture cycle. In this example there is
// just one cycle.

procedure capture =
    timeplate tp1;
    cycle =
        force_pi;
        measure_po;
```

```
        pulse_capture_clock;  
    end;  
end;
```



# Test Procedure File Structure

The test procedure file consists of many structural elements that display in a specific order, starting with `#include` statements. Some of these elements are required and others are optional.

```
#include "<file_name>";
[set_statement ...]
[alias_definition ...]
[timing_variables ...]
timeplate_definition           // includes pulse clock statements
always_block
procedure_definition
clock control definition
```

|                                    |            |
|------------------------------------|------------|
| <b>#include Statement</b> .....    | <b>765</b> |
| <b>Set Statement</b> .....         | <b>766</b> |
| <b>Alias Definition</b> .....      | <b>769</b> |
| <b>Timing Variables</b> .....      | <b>771</b> |
| <b>Timeplate Definition</b> .....  | <b>774</b> |
| <b>Multiple-Pulse Clocks</b> ..... | <b>780</b> |
| <b>Always Block</b> .....          | <b>782</b> |
| <b>Procedure Definition</b> .....  | <b>783</b> |

## #include Statement

The “`#include`” statement specifies that the tool read test procedure data from a specified file.

The following rules apply to `#include` statements and files:

- The “`#include`” statement can occur anywhere in the file, and multiple “`#include`” statements can occur in one file. For example:  

```
#include "ham.proc";
```
- The filename to be included must be enclosed in quotation marks (“”), and the statement must be followed by a semicolon.
- All timeplates and procedure rules apply to the statements placed in `#include` files.
- Included files can use the “`#include`” statement to include other files, up to a maximum include depth of 512. If you later use the [write\\_procfile](#) command to write out procedure data, the “`#include`” statements are not preserved, and the tool writes all procedure data to a single file.

## Set Statement

The Set statements define specific parameters used throughout the test procedure file.

The following statements are available:

```
set time scale tscale;  
set strobe_window time window_width;  
set default_timeplate timeplate_name;  
set autoforce off;
```

### set time scale Statement

Defines the time scale and unit. The “set time scale” statement must be at the beginning of the procedure file, before any timeplate or procedure definition. If you do not specify the time scale, the default value is 1 ns.

The tool applies the time scale and unit to the test procedure file and timeplates. The *tscale* you specify can be any real number. Time values in the timeplate, however, must be integers, representing whole time scale units. If you find you are specifying fractional times in the timeplate, you must reduce the time scale unit so you can specify integer time values in the timeplate. For example, the following would result in a syntax error:

```
set time scale 1 ns ;  
set strobe_window time 1 ;  
  
timeplate fast_clk_tp =  
    force_pi 0 ;  
    measure_po 0.500 ;  
    pulse CLKA 0.750 1.50 ;  
    pulse CLKB 0.750 1.50 ;  
    period 3.000 ;  
end ;
```

To correct the syntax, you could change the time scale to picoseconds, and adjust the time value to meet the scale as follows:

```
set time scale 10 ps ;  
set strobe_window time 1 ;  
  
timeplate fast_clk_tp =  
    force_pi 0 ;  
    measure_po 50 ;  
    pulse CLKA 75 150 ;  
    pulse CLKB 75 150 ;  
    period 300 ;  
end ;
```

The units supported are ms, us, ns, ps, and fs.

The tool translates the time scale in the procedure file into a Verilog ‘timescale directive in the Verilog testbench when writing patterns in Verilog format.

If the time scale number you specify in the test procedure file is 1 or larger, the resulting Verilog ‘timescale directive has the same time unit (resolution) and time precision. For example, “set time scale 1 ns ;” would result in this Verilog directive:

```
`timescale 1ns / 1ns
```

If you want the testbench to have smaller precision than resolution, there are several ways to designate this:

- Specify a time scale number of less than 1 in the procedure file. For example, “set time scale 0.5 ns ;” produces this Verilog directive:

```
`timescale 1ns / 100ps
```

- Add non-zero significant bits to the time scale in the procedure file. For example, “set time scale 10.05 ns ;” produces this Verilog directive:

```
`timescale 1ns / 10ps
```

- Add trailing zeros as significant bits for an asynchronous clock period or pattern\_set period (when creating IJTAG patterns). For example, an “add\_clocks -period 10.00ns” command produces this Verilog directive:

```
`timescale 1ns / 10ps
```

- Use the SIM\_PRECISION parameter file keyword. For example, “SIM\_PRECISION 0.5ns;” produces this Verilog directive:

```
`timescale 1ns / 100ps
```

The precision in the Verilog testbench can originate from any of the previous sources, and the tool uses the smallest specified precision when writing out the Verilog testbench.

The resolution in the Verilog testbench originates from the procedure file. When you use multiple procedure files, the various “set time scale” statements can specify different values, and the tool uses the smallest specified resolution when writing the Verilog testbench.

## set\_strobe\_window time Statement

Defines the strobe-window width. The “set\_strobe\_window time” statement must be at the beginning of the procedure file, before any timeplate or procedure definition. If you do not specify the strobe\_window time, it defaults to the maximum allowable size. For example, if you look at a timeplate, and if there is a 10 ns window between the measure\_po event and the next event (or end of timeplate), then that is the size of the strobe window. If there are multiple timeplates, then the smallest strobe window from the timeplates is the maximum allowable strobe window.

Some tester formats measure primary outputs (POs) at the exact time that you specify with the measure\_po statement in the timeplate. However, other tester formats, such as STIL, require

that output measurements occur during a specified window of time (*window\_width*). For WGL, this statement changes the strobe window in the output file.

A strobe window can only stretch from the *measure\_po* time to the end of the cycle or the next force or pulse event. For example, if you issue a *measure\_po* at time 10 and the rising edge of a pulse at time 30, the strobe window can only be a maximum of 20. *Strobe\_window* lets you know that, starting at the *measure\_po* time, the primary output should be stable for the time specified by the strobe window.

---

**Note**



Strobe\_window only affects the following formats: STIL, TSTL2, and WGL.

---

### set default\_timeplate Statement

Specifies a timeplate that can be used for any procedure definition that does not explicitly specify a timeplate. The referenced *timeplate\_name* must be defined prior to the Set Default\_timeplate statement in the procedure file.

### set autoforce Statement

An optional statement that controls the behavior for automatically adding force events. If included, this statement must appear at the beginning of the procedure file, prior to any procedures.

By default (without this statement), if a constrained pin is not forced to the constrained value in the *test\_setup* procedure, a force event is automatically added to the first cycle of the *test\_setup* procedure. If the *test\_setup* procedure starts with a call to a subprocedure, then the force event is added to the first cycle following the subprocedure. If a force event already exists in the *test\_setup* procedure for a constrained pin, then no additional force event is added for that pin.

When you include the “set autoforce off” statement, the tool does not add any force events on the constrained pins to the test setup procedure.

Including the “set autoforce off” statement also prevents the tool from forcing clock pins to the inactive state at the beginning of the *load\_unload* procedure. The statement also disables copying the last value forced on an input port in *test\_setup* and prevents it from becoming a force at the start of the *load\_unload* procedure.

The “set autoforce off;” statement also turns off auto forcing Z values on bidis in the *load\_unload* procedure—see [Load\\_Unload \(Required\)](#).

## Alias Definition

The Alias definition groups multiple signal names or cell paths into a single alias name. Signal Alias statements are useful in procedures or timeplates where multiple signals need to be assigned to the same value at the same time.

Cell Alias statements are used to group cell paths into a single alias name. You must define aliases before using them. The definition can occur at any place in the procedure file outside of a timeplate or procedure definition.

---

### Note



When saving STIL2005, CTL, or Structural\_STIL patterns, all aliases defined in the procedure file are defined as SignalGroups in the resulting STIL file.

---

There is a predefined alias available for specifying all bidirectional pins. The “\_ALL\_BIDI” keyword may be useful for forcing all bidirectional pins to a specified value without having to identify each individual pin. For example:

```
force _ALL_BIDI Z;
```

In using a cell Alias statement to group cell paths from condition statements into a single alias name, it is possible to override a condition statement in a named capture procedure with a subsequent condition statement that occurs in the same place (global condition, or local to a specific cycle). A condition statement can only override a previous condition if the first condition is specified using an alias name, and if the second condition is specified without using an Alias statement.

---

### Tip



When using multiple named capture procedures where each procedure requires many condition statements, it is helpful to group cells into a common name and apply the condition statement once to the entire group of cells, and then override specific cells that need a different value than what was applied to the group. This frees you from having to enter numerous condition statements for each named capture procedure, while only a handful of the cells require different values for each procedure.

---

The Alias definition has the following format:

```
alias alias_name = pin_name [, pin_name ...];
```

or

```
alias alias_name = cell_name [, cell_name ...];
```

- ***alias\_name***

A string that specifies the name of the alias.

### Note

 The **alias\_name** you specify should begin with an alphabetical character. If you want the name to begin with a numerical character, you must put the name in quotation marks.

---

- **pin\_name**

A repeatable string that specifies the pin name to associate with the alias name.

- **cell\_name**

A repeatable string that specifies the cell name to associate with the alias name.

## Alias Examples

This example groups two signal names into a single alias name.

```
alias my_group = T, U;
```

This next example shows how a named capture procedure should look when using an Alias statement for condition cells. The example sets each of four cells to a value of 0, and then the fourth cell (/inst\_3/blockb/reg\_2/Q) is overridden with a value of 1.

```
alias cond_cells = "/inst_0/blocka/reg_1/Q", "/inst_1/blocka/reg_1/Q",  
                  "/inst_2/blocka/reg_1/Q", "/inst_3/blockb/reg_2/Q";  
timeplate tp1 =  
    force_pi 0;  
    measure_po 10;  
    pulse ref_clk 50 50;  
    period 100;  
end;  
procedure capture capture1 =  
    timeplate tp1;  
    condition cond_cells 0;  
    condition /inst_3/blockb/reg_2/Q 1;  
                                     // overrides condition in previous statement  
    cycle =  
        force scan_en 0;  
        force ctrl_a 1;  
        force_pi;  
        pulse ref_clk;  
    end;  
    cycle =  
        force_pi;  
        measure_po;  
        pulse ref_clk;  
    end;  
end;
```

This example shows how to define a user-defined bus in a procedure file:

```
alias wdata = "D[0]", "D[1]", "D[2]", "D[3]";
```

And this shows how to assign a value to that user-defined bus:

```
procedure sub_procedure subpro1 =  
    scan_group grp1 ;  
    timeplate shift_tp ;  
    cycle =  
        force wdata 0010 ;  
        pulse ref_clk ;  
    end;  
end;
```

## Timing Variables

Two timing variable block definitions enable a procedure file to express timing using variables and equations, and to have this equation-based timing preserved in the tool and reproduced in the correct syntax in pattern output files.

Test data languages such as WGL and STIL have the ability to express time values in the timing blocks as numerical values or as equations based on variables. Using equation-based timing enables one value to be specified for a global attribute, such as the test cycle period, while other values are derived from this using equations.

The two timing block definitions are called “timing\_variables” and “variables”. In the “timing\_variables” block, variables can be defined and assigned timing values. These values are expressed in the time scale that is already specified by the Set Time Scale statement. The “timing\_variables” block must be defined before the timeplate definitions.

The “variables” block is used to define variables that are not time values and have no units associated with them. These variables can only be assigned integer numbers, and can be used as scaling multipliers in the timing equations.

The variables in the “timing\_variables” block can also be assigned timing equations instead of time values. These equations can use either timing values or previously defined variables or timing variables as operands. When using timing equations, you must ensure that the final value has a valid time unit. When the tool parses the procedure file, timing variables that are computed to have an invalid time unit cause a [W42](#) DRC error.

---

### Note



The event statements in the timeplate definition block accept timing values and timing variables.

---

When saving patterns in the Verilog, WGL, and STIL supported formats, the waveform tables in these formats are written using the equations and variables, and the variables are defined in the appropriate definition blocks that exist in each format. When saving patterns in formats that don't support equation-based timing, the equations are computed and the timing information is specified as the resulting numeric values in the pattern file. Setting the ALL\_FLATTEN\_TIMING parameter file keyword to “1” causes Verilog, WGL, and STIL

outputs to compute the timing equations and use only the resulting numeric values in the output files. Any equation that does not compute to an integer value is rounded to the nearest integer value.

The “timing\_variables” block has the following syntax:

```
variables =  
    variable_name = integer;  
    [variable_name = integer; ...]  
end;  
  
timing_variables =  
    variable_name = time_or_equation;  
    [variable_name = time_or_equation; ...]  
end;
```

Note that *time\_or\_equation* can either be an integer time value or a time equation. A time equation is expressed using operators and operands. An operator is one of +, -, \*, or /. An operand can be a time value or a variable name (time or scaling variable). The multiplication and division operators (\* and /) take precedence over the addition and subtraction operators (+ and -). You can use parenthesis to group operations for precedence.

---

#### **Note**



In the timeplate definitions, any place where a time value can be used, a timing variable is also valid. A scaling variable from the “variables” block cannot be used in a timeplate definition. These can only be used in time equations.

---

Variable names can be any identifier except for reserved keywords used in the procedure file syntax (such as “period” and “force\_pi”). The variable names must conform to the rules that apply to all identifiers used in the procedure file (alpha numeric string, starting with an alpha character, and no reserved punctuation marks). If reserved characters or reserved words are used in a variable name, the name must be enclosed in quotation marks.

## **Equation-Based Timing Example**

The following is a partial example of using equation-based timing.



```
set time scale 1.0 ns;
variables =
    v_scale = 1;
end;

timing_variables =
    t_period = 100;
    t_force = 0;
    t_meas = ((t_period * 0.1) * v_scale );
    t_rise = ((t_period / 2) * v_scale );
    t_width = ((t_period * 0.2) * v_scale);
end;

timeplate tp1 =
    force_pi t_force;
    measure_po t_meas;
    pulse ref_clk t_rise t_width;
    period t_period;
end;
```

This is how the preceding timing example would be represented in the STIL output:

```
Spec STUCK_spec {
    Category STUCK_cat {
        v_scale = '1';
        t_period = '100ns';
        t_force = '0ns';
        t_meas = '(t_period*0.1)*v_scale';
        t_rise = '(t_period/2)*v_scale';
        t_width = '(t_period*0.2)*v_scale';
    }
}

Timing STUCK_timing {
    WaveformTable tset_tp1 {
        Period 't_period';
        Waveforms {
            input_time_gen_0 { 01 { 't_force' D/U; }}
            input_time_gen_1 { 01 { '0ns' D; 't_rise' D/U;
                                   't_rise+t_width' D;}}
            _po_ { LHX { '0ns' X; 't_meas' 1/h/x; 't_rise' X;}}
        }
    }
}
```

This is how the timing example would be represented in the WGL output:

```
equationsheet STUCK_sheet
  exprset STUCK_set
    v_scale := 1.0;
    t_period := 100nS;
    t_force := 0nS;
    t_meas := (t_period * 0.1) * v_scale;
    t_rise := (t_period / 2) * v_scale;
    t_width := (t_period * 0.2) * v_scale;
    _tp1_fall_1 := t_rise + t_width;
  end
end

timeplate tp1 period t_period
  "input_a" := input [t_force:S];
  ...
  "output_z" := output [0nS:X, t_meas:Q, t_rise:X];
  ...
  "refclk" := input [0nS:D, t_rise:S, _tp1_fall_1:D];
end
```

## Timeplate Definition

The timeplate definition describes a single tester cycle and specifies where in that cycle all event edges are placed.

You must define all timeplates before they are referenced. A procedure file must have at least one timeplate definition. All clocks must be defined in the timeplate definition. The period statement must be the last statement in the timeplate definition.

The timeplate definition has the following format:

```
timeplate timeplate_name =
  timeplate_statement
  [timeplate_statement ...]
  period time;
end;
```

The following list contains available timeplate\_statement statements. The timeplate definition should contain at least the force\_pi and measure\_po statements. You are not required to include pulse statements for the clocks. But if you do not “pulse” any of the clocks, the tool uses two cycles to pulse a clock, resulting in larger patterns.

The tool uses the pulse\_clock statement rather than individual pulse statements when generating default procedures.

```
timeplate_statement:  
  offstate pin_name off_state;  
  force_pi time;  
  bidi_force_pi time;  
  measure_po time;  
  bidi_measure_po time;  
  force pin_name time;  
  measure pin_name time;  
  pulse pin_name time width [, time width];  
  pulse_clock time width [, time width];
```

---

**Note**

 In “timeplate\_statement” definitions, you can use timing variables instead of time values. For more information, refer to “[Timing Variables](#)” on page 771.

---

The following is a list of statements in the timeplate definition:

- ***timeplate\_name***

A string that specifies the name of the timeplate.

---

**Note**

 The ***timeplate\_name*** you specify should begin with an alphabetical character. If you want the name to begin with a numerical character, you must put the name in quotation marks.

---

- ***offstate pin\_name off\_state***

A literal and double string that specifies the inactive, off-state value (0 or 1) for a specific named primary input pin that is pulsed within this timeplate but is not defined as a clock pin by the add\_clocks command. The complex timeplates are most useful in the shift procedure where a non-clock pin must be pulsed while still maintaining a single cycle in the shift procedure.

This statement must occur before all other timeplate\_statement statements. This statement is required for any pin that is not defined as a clock pin by the add\_clocks command but is pulsed within this timeplate.

---

**Note**

 An “offstate” statement does not automatically force *pin\_name* to its off state at time 0. For that to occur, you must force or pulse *pin\_name* appropriately in a procedure.

---

- ***force\_pi time***

A literal and integer pair that specifies the force time for all primary inputs.

- ***bidi\_force\_pi time***

A literal and integer pair that specifies the force time for all bidirectional pins. This statement enables the bidirectional pins to be forced after applying the tri-state control

signal, so the system avoids bus contention. This statement overrides “force\_pi” and “measure\_po”.

- **measure\_po time**

A literal and integer pair that specifies the time at which the tool measures (or strobes) the primary outputs.

**Note**



Capture clocks pulsing on the same cycle must have an overlapped clock waveform. If they do not, the tool splits the capture cycle into two cycles. This results in an error reporting that a measure\_po event is absent.

---

- **bidi\_measure\_po time**

A literal and integer pair that specifies the time at which the tool measures (or strobes) the bidirectional pins. This statement overrides “force\_pi” and “measure\_po”.

- **force pin\_name time**

A literal, string, and integer that specifies the force time for a specific named pin.

**Note**



This force time overrides the force time specified in force\_pi for this specific pin.

---

- **measure pin\_name time**

A literal, string, and integer that specifies the measure time for a specific named pin. You can use a “measure” statement in timeplates only to specify a measure time for a pin.

**Note**



This measure time overrides the measure time specified in measure\_po for this specific pin.

---

- **pulse pin\_name time width [, time width]...**

A literal, string, and repeatable integer set that specifies the pulse timing for a specific named pin.

**pin\_name** — String that refers to a pin in the design. Valid pins must meet one of the following conditions:

Defined as a clock pin using the add\_clocks command.

Not defined as a clock pin, but has a pulse signal and an off-state specified by the “offstate” statement.

**time** — Integer that defines the offset from time 0 to the leading edge of the pulse.

**width** — Integer that defines the width of the pulse.

To define a multiple-pulse waveform, include multiple time and width pairs separated by a comma.

All pulses must occur within the tester cycle period. You define this period using the “period” keyword.

---

**Note**

---

 Multiple pulses are only supported for the following output formats: Verilog, WGL, STIL, STIL2005, CTL, FJTDL, MITDL, and TSTL2. Additionally, the TSTL2 output format does not support more than two pulses.

---

For MITDL format there is restriction that multiple pulse timing must be a cyclical repetition of the first pulse. Consequently, multi-pulse and double-pulse timing in the procedure file only works in the MITDL output without an error if the timing fits the restrictions of the MITDL syntax.

- **pulse\_clock *time width* [, *time width* ]...**

A literal and integer set that specifies the pulse timing for all signals defined as clocks, unless another statement such as a “force” or “pulse” exists for a particular clock signal. This is similar to the force\_pi statement, which specifies the timing for ports that are not explicitly overridden by a force statement for those specific ports.

**time** — Integer that defines the offset from time 0 to the leading edge of the pulse for the first pulse. For subsequent edges in a multi pulse clock, time equals the time of the previous leading edge plus the period of the clock.

**width** — Integer that defines the width of the pulse.

You can use the pulse\_clock statement to ensure that any added clocks (especially internal clocks) automatically have defined timing.

You may have multiple pulse\_clock statements within a timeplate definition, as long as each statement has a different number of offset and width pairs. If two pulse\_clock statements in the timeplate definition have the same number of offset and width pairs, the tool issues an error.

For more information on multiple-pulse clocks, see “[Multiple-Pulse Clocks](#).”

All pulses must occur within the tester cycle period. You define this period using the “period” keyword.

For example (1x pulse\_clock):

```
timing_variables =  
    tester_period = 10;  
    strobe_1 = (0.96 * tester_period);  
    t_time = (0.25 * tester_period);  
    t_width = (0.5 * tester_period);  
end;  
  
timeplate tesseract_ijtag =  
    force_pi 0 ;  
    measure_po strobe_1;  
    force tck 0;  
    pulse_clock t_time t_width;  
    period tester_period;  
end;
```

- **period *time***

A literal and integer pair that defines the period of a tester cycle. This statement ensures that the cycle contains sufficient time, after the last force event, for the circuit to stabilize. The time you specify should be greater than or equal to the final event time.

## Example 1

```
timeplate tp1 =  
    force_pi 0;  
    pulse T 30 30;  
    pulse R 30 30;  
    measure_po 90;  
    period 100;  
end;
```

## Example 2

The following example shows a shift procedure that pulses b\_clk with an off-state value of 0. The timeplate tp\_shift defines the off-state for pin b\_clk. The b\_clk pin is not declared as a clock in the ATPG tool.

```
timeplate tp_shift =  
  offstate b_clk 0;  
  force_pi 0;  
  measure_po 10;  
  pulse clk 50 30;  
  pulse b_clk 140 50;  
  period 200;  
end;  
  
procedure shift =  
  timeplate tp_shift;  
  cycle =  
    force_sci;  
    measure_sco;  
    pulse clk;  
    pulse b_clk;  
  end;  
end;
```

### Example 3

In the following example, the pin `b_clk` is not declared as a clock in the ATPG tool. However, the shift procedure specifies that the pin is to be pulsed twice with an off-state of 0.

```
timeplate tp_shift =  
  offstate b_clk 0;  
  force_pi 0;  
  measure_po 10;  
  pulse clk 50 30;  
  pulse b_clk 40 50, 140 50;  
  period 200;  
end;  
  
procedure shift =  
  timeplate tp_shift;  
  cycle =  
    force_sci;  
    measure_sco;  
    pulse clk;  
    pulse b_clk;  
  end;  
end;
```

## Multiple-Pulse Clocks

You can use the `pulse_clock` statement to handle multiple-pulse clock timing and still use a single generic timeplate template definition in various flows. The `pulse_clock` statement handles multiple-pulse timing definitions in the same manner as the `pulse` statement.

|                                                                                        |            |
|----------------------------------------------------------------------------------------|------------|
| <b>The <code>pulse_clock</code> Statement .....</b>                                    | <b>780</b> |
| <b>Inferred Timing .....</b>                                                           | <b>781</b> |
| <b>Differences Between Default <code>add_clock</code> and 1x Multiplier Clock.....</b> | <b>782</b> |

### The `pulse_clock` Statement

In a timeplate statement, each `pulse_clock` statement has a different number of integer pairs for the offset and width of the clocks. The different statements represent 1x, 2x, 3x, and so on, pulse timing for clocks. The frequency multiplier that you specify for the clocks identifies the appropriate pulse timing used for them.

When you specify a multiplier for a clock and no `pulse_clock` statement exists for that clock, the tool creates inferred timing for the clock using the timeplate period. The tool issues an error when a port-specific pulse or force statement exists for the clock and the timing in the statement does not specify the correct number of pulse statements to match the frequency multiplier of the clock.

The following example has multiple `pulse_clock` statements. It is a timeplate that includes both a port specific pulse statement for a clock and the generic `pulse_clock` statements for all other clocks. The first `pulse_clock` statement sets the default clock timing for this timeplate and contains a single pair of numbers (offset and width). Clocks other than “SlowClockA” that do not have frequency multipliers use this timing. Clocks with 2x multipliers use the second `pulse_clock` statement, and clocks with 4x multipliers use the third `pulse_clock` statement. The tool uses inferred timing for clocks specified with any other frequency multiplier and issues an error if "SlowClockA" has a frequency multiplier of 2x or more.

```
timeplate tp1 =  
    force_pi 0;  
    measure_po 95;  
    pulse SlowClockA 20 50;  
    pulse_clock 30 30;  
    pulse_clock 13 25, 63 25;  
    pulse_clock 6 12, 31 12, 56 12, 81,12;  
    period 100;  
end;
```

The following example shows a timeplate with one port-specific pulse and one `pulse_clock` statement for 4x multiplier timing. Clocks other than “ClockA” that do not have a frequency multiplier use `force_pi` timing. Inferred timing only occurs when clocks have frequency multipliers, which means other clocks still use NRZ timing when you use multiple cycles to create the clock pulse.



```
timeplate tp1 =
  force_pi 0;
  measure_po 95;
  pulse ClockA 20 50;
  pulse_clock 6 12, 31 12, 56 12, 81, 12;
  period 100;
end;
```

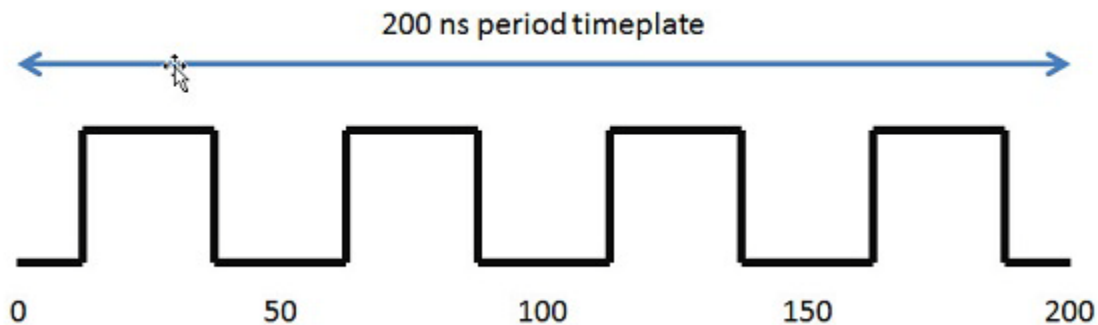
## Inferred Timing

Based on the period of the timeplate, the tool creates inferred timing for frequency multiplied clocks when there are no `pulse_clock` statements in the timeplate with the correct number of clock edges for the clock.

The total period of a clock pulse is the period of the timeplate divided by the frequency multiplier of the clock. The offset for the leading edge of the first clock pulse is one-fourth of the period of the clock. For example, the inferred timing for a 4x clock in a timeplate with a period of 200 is equivalent to the following `pulse_clock` definition:

```
pulse_clock 13 25, 63 25, 113 25, 163 25
```

**Figure 11-1. 200ns Timing Waveform**



The first edge is one-fourth the period of the clock rounded to the nearest integer, and the width of the pulse is one-half the period of the clock. Each subsequent edge is the previous leading edge plus the period of the clock.

The tool bases the inferred timing for a 1x frequency-multiplied clock on any other clock timing found for a single pulse clock; otherwise, it uses the inferred timing formula.

When you specify a timeplate period and a clock frequency multiplier that combine to create inferred timing that is too small to be integer timing, the tool automatically reduces the procedure file timescale and scales all timing numbers to larger values.

For example, suppose the timescale for the procedure file is 1ns, the period of the timeplate is 5, and the clock frequency multiplier is 10. The tool automatically adjusts the timescale to 100ps and the period of the timeplate becomes 50. It adjusts all of the other numbers accordingly. In this case, the tool multiplies them by 10.

## Differences Between Default add\_clock and 1x Multiplier Clock

Default add\_clock clocks use the time specified by pulse\_clock statements with one pulse or by pin-specific pulse statements. However, frequency-multiplied clocks use the timing specified for them only if they match the number of pulses. If this is not the case, the tool uses inferred timing and issues an error if there are mismatches.

Clocks specified with a frequency multiplier of one behave differently than default clocks. For this reason, you must specify the timing for a 1x frequency-multiplied clock with a single clock pulse. The tool does not use NRZ timing with multiple cycles or forced timing edges, and it issues an error if the timeplate specifies a double-pulse or multiple-pulse timing for the 1x clock.

## Always Block

This optional block definition specifies events that happen in all cycles of all procedures. Because the always block specifies events for all cycles, it is used with all timeplates and does not require a timeplate to be referenced in the block. Also, any signal that is pulsed in the always block must have a pulse waveform in all timeplate definitions.

If you defined any pulse-always clocks using the add\_clocks command, an always block is automatically created in the procedure file, if one does not already exist, and a pulse statement added for each clock. Similarly, if you pulse a clock signal in the always block, the signal is automatically defined as a pulse-always clock. For more information, refer to the [add\\_clocks](#) description in the *Tessent Shell Reference Manual*.

---

### Note



Pulse-always clocks are not automatically pulsed in a named capture procedure. The clocks must be pulsed explicitly.

---

All events specified in the always block is subject to rules checks that apply to each procedure. In other words, the events in the always block is added to each cycle of each procedure, and all DRC rules still apply to these events.

When saving patterns that preserve the structure of the procedures as macros (such as the CTL pattern file, or structural STIL pattern file), the events in the always block is placed in the cycles of each procedure. The always block is not present in the structural pattern file as a macro or procedure.

## Always Block Syntax

The always block has the following syntax.

```
always =  
    always_statement ;  
    [always_statement ; ... ]  
end ;
```

The `always_statement` is defined as one of the following.

```
pulse pin_name ;  
force pin_name value ;
```

### Always Block Example

The following is a partial example of an always block.

```
set time scale 1,0 ns ;  
  
timeplate tp1 =  
    force_pi 0 ;  
    measure_po 10 ;  
    pulse ref_clk 20 20, 60 20 ;  
    pulse shift_clk 50 20 ;  
    period 100 ;  
end ;  
always =  
    pulse ref_clk ;  
end ;  
  
procedure shift =  
    timeplate tp1 ;  
    cycle =  
        force_sci ;  
        measure_sco ;  
        pulse shift_clk ;  
    end ;  
end ;
```

## Procedure Definition

The procedure definition is the heart of the procedure file. The procedure defines precisely how the scan circuitry operates.

All procedure definitions contain one or more cycle definitions. Each cycle definition in the procedure specifies a vector; each statement in the cycle specifies which events occur in that vector. The timeplate being used specifies any timing associated with that vector. The following is a list of rules for writing procedure definitions:

- If more than one timeplate is defined, you can assign a specific timeplate for each procedure definition or for each cycle within the procedure definitions. You must assign a timeplate at some point within a procedure definition.
- You must group all procedure statements, except `scan_group`, `timeplate`, `apply`, and `loop`, into cycle statements.

- You cannot specify time values in cycle statements.
- You cannot specify loop statements within cycle statements.
- The order of events within a cycle definition does not matter. The assigned timeplate specifies the order.
- Within a procedure definition, you can specify a scan group.
- Each scan group needs a unique test procedure file. You associate the test procedure file with the scan group when you specify the [add\\_scan\\_groups](#) command.
- Text following “//” is a comment and is ignored.
- You can include blank lines.
- You define a procedure type for a particular scan group (with the exception of the seq\_transparent and clock procedures) only once in a test procedure file.
- You can only have a single test\_setup procedure, even if you define multiple scan groups for your design.

The procedure definition has the following general format, but certain statements are restricted to certain procedures.

```

procedure procedure_type [proc_name] =
    [scan_group scan_group_name;]
    proc_statement [proc_statement ...]
end;

proc_statement:
    [timeplate timeplate_name;]
    cycle =
        cycle_statement [cycle_statement ...]
    end;
    annotate "quoted string";
    apply proc_name #times;
    loop loop_count =
        cycle =
            cycle_statement [ cycle_statement ...]
        end;
    end;

    cycle_statement:
        force_pi;
        bidi_force_pi;
        force_sci;
        force_sci_equiv;
        measure_po;
        bidi_measure_po;
        measure_sco;
        restore_pi;
        restore_bidi;
        bidi_force_off;
        pulse_capture_clock;
        force_capture_clock_on ;
        force_capture_clock_off ;
        pulse_read_clock;
        pulse_write_clock;
        force pin_name value;
        expect pin_name value;
        condition cell_name value;
        measure pin_name;
        initialize instance_name [value];
        pulse pin_name;
        timeplate timeplate_name;
        annotate "quoted string";

```

- ***procedure\_type***

A string that specifies the type of procedure that follows. The following list contains valid procedures types:

- |                  |                    |                   |
|------------------|--------------------|-------------------|
| • test_setup     | • capture          | • seq_transparent |
| • shift          | • ram_passthru     | • test_end        |
| • master_observe | • clock_sequential | • skew_load       |
| • load_unload    | • sequential       | • clock           |
| • shadow_observe | • init_force       | • sub_procedure   |

- **shadow\_control**

For more information, refer to “[Test Procedures](#)” on page 795.

- **proc\_name**

An optional string that specifies the user-defined name of the procedure. Because you can specify multiple seq\_transparent and clock procedures in a test procedure file, these procedure types require explicit procedure names, *proc\_name*, for each procedure that you define.

---

**Note**



The *proc\_name* you specify should begin with an alphabetical character. If you want the name to begin with a numerical character, you must put the name in quotation marks.

---

- **scan\_group scan\_group\_name**

A literal and string pair that specifies a scan group within a scan procedure. Because some of the scan procedures are scan group specific, you can specify scan groups within scan procedures. This makes it possible to define the scan procedures (shift, load\_unload) for multiple scan groups within the same procedure file. You can then specify this file on the [add\\_scan\\_groups](#) command for each scan group in this file. If you use the [read\\_procfile](#) command to read a procedure file, you must include this statement. However, if you use the [add\\_scan\\_groups](#) command, this statement is optional because the group is specified on the command line. When the tool writes out a procedure file, it produces the scan\_group statement.

---

**Note**



The scan\_group\_name argument is case-sensitive if the netlist used is case-sensitive.

---

- **timeplate timeplate\_name**

A literal and string pair that specifies the name of the timeplate the procedure uses.

A timeplate statement at the beginning of the procedure, outside of the cycle definitions, is the timeplate used by the entire procedure, if no other timeplates are referenced.

A timeplate statement within a cycle is the timeplate used for that cycle and all other subsequent cycles until another timeplate statement is encountered. For more information about timeplates, refer to “[Timeplate Definition](#)” on page 774.

---

**Note**



The *timeplate\_name* you specify should begin with an alphabetical character. If you want the name to begin with a numerical character, you must put the name in quotation marks.

---

- **annotate "quoted string";**

A literal and string pair that reports the Verilog testbench annotations during simulation.

The annotate statement is optional and must always include a quoted string. All procedures can be annotated, including sub-procedures. The annotate statement can occur inside or outside of cycle blocks, including before the first cycle or after the last cycle.

A special use of the annotate statement is to include TESSANT\_PRAGMA directives within a procedure. TESSANT\_PRAGMA directives can affect the behavior of the tool when it writes patterns.

This example shows the annotate statement in the beginning of a cycle along with the cycle timeplate statement, before any event statements:

```
CYCLE =  
  [ TIMEPLATE tp_name ; ]  
  [ ANNOTATE "quoted string" ; ]  
  event_statement;  
  ...  
END ;
```

The following is an example of using the annotate statement outside of a cycle block:

```
procedure test_setup =  
  timeplate tp1 ;  
  annotate "Before first cycle" ;  
  cycle =  
    ...  
  end;  
  annotate "start sub procedure" ;  
  apply mySub 1 ;  
  ...  
end;
```

The following is an example of an annotate statement used in a test\_setup procedure, and how this appear sin a STIL pattern file.

```
Procedure test_setup =  
  timeplate tp1;  
  cycle =  
    annotate "first cycle in test_setup" ;  
    force reset 1;  
    force clock 0;  
  end;  
  cycle =  
    annotate "next annotation" ;  
    force reset 0;  
  end;  
  ...
```

This is a segment of the resulting STIL pattern file:

```
W tset_tpl;  
V { _pi_ = 0X11XXX;  
  _po_ = XXXX;  
}  
Ann { * Begin chain test *}  
Ann { * first cycle in test_setup *}  
V { ...  
}  
Ann { * next annotation *}  
V { ...  
}
```

The following is an example of an annotate statement used to include TESSENT\_PRAGMA directives. This pair of simulation\_only directives instructs the tool to include the cycles they surround only in a simulation format. The cycles are eliminated, along with the annotations, for any other formats.

```
procedure test_setup =  
  timeplate tpl ;  
  annotate "TESSENT_PRAGMA simulation_only begin";  
  cycle =  
    ...  
  end;  
  iCall ...;  
  ...  
  annotate "TESSENT_PRAGMA simulation_only end";  
  ...  
end;
```

---

#### Note

 The [ALL\\_IGNORE\\_SIMULATION\\_ONLY](#) parameter keyword can affect which output files contain simulation\_only directives. For more information about the simulation\_only directive, refer to the description of the *<format\_switch>* parameter in the [write\\_patterns](#) command in the *Tessent Shell Reference Manual*.

---

- **label "quoted\_string";**

A literal and string pair for including pattern labels in saved patterns. As with the annotation statement, you can have one label statement per cycle in a procedure definition. The quoted\_string becomes a pattern label for the vector that corresponds to that procedure cycle.

Only the STIL and WGL pattern formats support a pattern label statement. For pattern file formats that don't support a pattern label, the label is present as an annotation statement that has the string "label:" added at the beginning of the label string.

For the simulation testbench, the label is also present as an annotation that has the string "label:" added at the beginning, and the annotation is echoed when the patterns are simulated. You can use the existing parameter file keyword SIM\_ANNOTATE\_QUIET to turn off echoing the annotations and labels while simulating.



Each pattern label is a unique identifier, with its vector count appended to the end of the label string.

This statement can be used at the start of any cycle, just like an annotation statement. A cycle cannot contain both a label and an annotation statement.

The following example shows how to use the label statement within the test\_setup procedure:

```
procedure test_setup =
  timeplate my_tp;
  cycle =
    force my_sig 0;
  end;
  cycle =
    label "end of test_setup" ;
    force my_sig 1;
  end;
end;
The previous example produces the following STIL vectors:
v { _pi_ = ...;
}
"end of test_setup_1": v { _pi_ = ...;
}
```

- **apply *proc\_name* #*times***

A literal and double-argument string that tells the tool to apply the specified procedure the specified number of times. You must use the apply shift statement at least once in the load\_unload procedure. For the apply shift statement, you must enter a proper *#times* parameter.

If required, you must enter the apply shadow\_control statement immediately after the apply shift procedure statement, and you must set the *#times* argument to 1. The apply statement is only valid outside of the cycle blocks because it specifies another group of cycles within another procedure to be added at that point.

- **loop *loop\_count***

A literal and integer pair that specifies the loop count and is followed by a block of statements. The loop procedure statement takes the loop count and causes all cycles within the loop block to be repeated by the number of times specified by the count. For

example, the following test procedure file excerpt specifies 3 cycles within the loop that are each repeated 20 times:

```
procedure test_setup =  
    timeplate tp1 ;  
    cycle =  
        force_pi;  
        measure_po;  
    end;  
    loop 20 =  
        cycle =  
        end;  
        cycle =  
            pulse tck;  
        end;  
        cycle =  
        end;  
    end; // end loop  
end;
```

Nesting loops within other loops is permitted. For example, the following test procedure file excerpt causes tck to be pulsed 20 times and clk\_a to be pulsed 100 times:

```
procedure test_setup =  
    timeplate tp1 ;  
    cycle =  
        force_pi;  
        measure_po;  
    end;  
    loop 20 =  
        cycle =  
        end;  
        cycle =  
            pulse tck;  
        end;  
        loop 5 =  
            cycle =  
                pulse clk_a;  
            end;  
        end; // end inside loop  
        cycle =  
        end;  
    end; // end outside loop  
end;
```

This statement can be used in procedures but must be specified outside of the cycle statement.

The loop statement is preserved in the flat model when the tool writes the model and is also present in the TCD files.

When writing out the patterns in tester pattern formats, the loops are preserved where possible, and unrolled if the syntax of the pattern file does not support loops. Specifying the [ALL\\_NO\\_LOOP](#) parameter keyword unrolls loops in the pattern files in similar fashion to sub procedures that are applied more than once. Using the loop statement to

repeat a certain number of cycles  $N$  times is exactly equivalent to putting those cycles within a sub procedure, then applying that procedure  $N$  times.

- **start\_pulse *pin\_name***

A literal and string pair that specifies for the tool to start pulsing the named clock pin in every cycle, starting with the next cycle encountered. This enables you to specify that a clock that is not a pulse\_always clock should be pulsed in every cycle of an iCall statement. It also enables you to restart a pulse\_always clock that has been temporarily suppressed with the stop\_pulse keyword.

This keyword can also be used inside a cycle\_statement.

- **stop\_pulse *pin\_name***

A literal and string pair that specifies for the tool to stop pulsing the named clock pin, starting with the next cycle encountered. This enables you to suppress a pulse\_always clock for a known number of tester cycles until you restart it with the start\_pulse keyword. It also enables you to stop a clock started with the start\_pulse keyword when you no longer need it to be pulsed.

This keyword can also be used inside a cycle\_statement.

- **cycle\_statement**

The following list describes valid cycle\_statement keywords. Cycle\_statements cannot contain time values.

- force\_pi

A literal that specifies for the tool to force all primary inputs.

- bidi\_force\_pi

A literal that specifies for the tool to force all bidirectional pins.

- force\_sci

A literal that specifies for the tool, in the shift procedure, to place values on the scan chain inputs, thus implementing scan cell controllability.

- force\_sci\_equiv

A literal that acts the same as the force\_sci statement, except that it also forces all pins equivalent to the scan input pins. Using this statement places the complement value on the associated differential pin of a scan input during scan loading. This statement is necessary because the test procedures do not consider pin equivalence relationships (those specified with add\_input\_constraints -equivalent).

- measure\_po

A literal that specifies for the tool to measure or strobe the primary outputs.

- `bidi_measure_po`  
A literal that specifies for the tool to measure or strobe the bidirectional pins.
- `measure_sco`  
A literal that specifies for the tool, in the shift procedure, to measure scan output values, thus implementing scan cell observability. In End Measure Mode (refer to [“Creating Test Procedure Files for End Measure Mode”](#) on page 836), `measure_sco` is also used in the `load_unload` procedure.
- `restore_pi`  
A literal that returns primary inputs to their original states (before running this procedure). Use the `restore_pi` statement at the end of a `seq_transparent` procedure.
- `restore_bidi`  
A literal that returns bidirectional pins to their original states (before running this procedure). Use the `restore_bidi` statement at the end of a “clock” procedure.
- `bidi_force_off`  
A literal that specifies for the tool to force all unconstrained bidirectional pins off.
- `pulse_capture_clock`  
A literal that specifies for the tool to pulse the capture clock.
- `force_capture_clock_on`  
A literal that specifies the cycle when the capture clock goes active. This statement and the `force_capture_clock_off` statement can be used in place of the `pulse_capture_clock` statement.  
  
The “\_on” refers to the active state of the clock, which is not necessarily the high binary value. This statement is used only with the non-scan procedures and cannot be mixed with the following statements in the same procedure:
  - `pulse_capture_clock`
  - `pulse_write_clock`
  - `pulse_write_clock`
- `force_capture_clock_off`  
A literal that specifies the cycle when the capture clock goes inactive. This statement and the `force_capture_clock_on` statement can be used in place of the `pulse_capture_clock` statement.

The “\_off” refers to the inactive state of the clock, which is not necessarily the low binary value. This statement is used only with the non-scan procedures and cannot be mixed with the following statements in the same procedure:

- pulse\_capture\_clock
- pulse\_write\_clock
- pulse\_read\_clock
- pulse\_read\_clock

A literal that specifies for the tool to pulse the RAM read clock.
- pulse\_write\_clock

A literal that specifies for the tool to pulse the RAM write clock.
- force *pin\_name value*

A literal and pair of strings that forces the specified value of 0, 1, X, or Z on the specified pin. The pin names you specify must be valid pin pathnames for primary inputs.
- expect *name value*

A literal and pair of strings that causes the tool to expect the specified value of 0, 1, X, or Z on the specified internal pin or port. The default value is X. You can use “expect” statements only in the test\_setup and test\_end procedure. Internal pins are checked by DRC and are compared in the testbench. For ports, the tool validates the values in the testbench, the DRCs, and in the tester pattern formats.
- condition *cell\_name value*

A literal and pair of strings that you use at the beginning of a seq\_transparent procedure to identify the necessary scan cell states (conditions) to establish transparency in non-scan cells. You identify the scan cell by the pin pathname associated with the output of its state element. The path from the defined pin to the scan cell must only contain buffers and inverters. The value argument sets the value at the specified pin pathname, which may be inverted relative to the associated scan cell value.
- measure *pin\_name*

A literal and string pair that specifies for the tool to measure the value of the named pin. You can use a “measure” statement in the capture procedure only to specify a measure on a pin in a different cycle than the measure\_po event.
- initialize *instance\_name value*

A literal and pair of strings that initializes the named memory element to the value given. This statement is particularly useful for initializing the finite state machine in

the TAP controller of boundary scan circuitry, when the TAP does not contain the TRST signal. Once set to a binary state, the TCK and TMS pins can place the finite state machine in a known state. If not set, these pins remain at X.

If you do not specify a value, the tool chooses a random value to assign to all latches and flip-flops with the specified instance name.

- pulse *pin\_name*

A literal and string pair that specifies for the tool to pulse the named clock pin.

- start\_pulse *pin\_name*

A literal and string pair that specifies for the tool to start pulsing the named clock pin in every cycle. This enables you to specify that a clock that is not a pulse\_always clock should be pulsed in every cycle of an iCall statement. It also enables you to restart a pulse\_always clock that has been temporarily suppressed with the stop\_pulse keyword.

This keyword can also be used outside of a cycle\_statement. In this case, it takes effect starting with the next cycle encountered.

- stop\_pulse *pin\_name*

A literal and string pair that specifies for the tool to stop pulsing the named clock pin. This enables you to suppress a pulse\_always clock for a known number of tester cycles until you restart it with the start\_pulse keyword. It also enables you to stop a clock started with the start\_pulse keyword when you no longer need it to be pulsed.

This keyword can also be used outside of a cycle\_statement. In this case, it takes effect starting with the next cycle encountered.

- observe\_method *value*

A literal and string pair set to a value of master, slave, or shadow, to specify for a specific observe method to be defined for each named capture procedure.

The following example shows how to use a cycle\_statement to force scan inputs and measure scan outputs:

```
procedure shift =  
    scan_group grp1;  
    timeplate tpl;  
    cycle =  
        force_sci;  
        measure_sco;  
        pulse T;  
    end;  
end;
```

# Test Procedures

The test procedure file contains scan and clock procedures, and non-scan procedures. The scan and clock-related procedures inform the tool how to operate the scan chain and pulse clocks. The non-scan procedures can represent any type of pattern that the tool produces.

You can use the non-scan procedures to specify in which cycles of the procedure “potential events” happen. A potential event is an event that the ATPG engine may or may not have created to cover a certain fault.

To avoid DRC violations, each non-scan procedure must contain the proper statements in the correct order with the timing from the timeplate. The statements in a non-scan procedure can be spread over any number of cycles using a different timeplate for each cycle if needed.

A basic pattern consists of loading the scan chains, a default capture procedure, followed by unloading the scan chains; however, you do not specify the loading and unloading of scan chains in non-scan procedures. The following shows the basic pattern for non-scan procedures.

## Basic Pattern

Force primary inputs  
Measure primary outputs  
Pulse capture clock

All example procedures shown in this section use one of the following two timeplates, unless otherwise stated:

```
timeplate tp1 =  
    force_pi 0;  
    measure_po 10;  
    pulse scan_clk 30 10;  
    pulse sys_clk 30 10;  
    period 50;  
end;  
  
timeplate tp2 =  
    force_pi 0;  
    measure_po 10;  
    pulse scan_mclk 15 10;  
    pulse scan_sclk 30 10;  
    period 50;  
end;
```

|                                                   |            |
|---------------------------------------------------|------------|
| <b>Test_Setup (Optional)</b> .....                | <b>796</b> |
| <b>Shift (Required)</b> .....                     | <b>799</b> |
| <b>Alternate Shift Procedure (Optional)</b> ..... | <b>801</b> |
| <b>Load_Unload (Required)</b> .....               | <b>803</b> |
| <b>Shadow_Control (Optional)</b> .....            | <b>806</b> |

|                                                  |            |
|--------------------------------------------------|------------|
| <b>Master_Observe (Sometimes Required)</b> ..... | <b>808</b> |
| <b>Shadow_Observe (Optional)</b> .....           | <b>808</b> |
| <b>Skew_Load (Optional)</b> .....                | <b>809</b> |
| <b>Clock_Control (Optional)</b> .....            | <b>811</b> |
| <b>Clock_run (Optional)</b> .....                | <b>823</b> |
| <b>Capture Procedures (Optional)</b> .....       | <b>824</b> |
| <b>Clock_sequential (Optional)</b> .....         | <b>830</b> |
| <b>Init_force (Optional)</b> .....               | <b>831</b> |
| <b>Test_end (Optional, all ATPG tools)</b> ..... | <b>832</b> |
| <b>Sub_procedure</b> .....                       | <b>833</b> |

## Test\_Setup (Optional)

This optional procedure, which can only contain force, pulse, init, and expect event statements, sets non-scan elements to the required states for the load\_unload procedure. You may use this procedure only once for all scan groups, and it appears only once at the beginning of the test pattern set.

This procedure is particularly useful for initializing boundary scan circuitry. For an example using this procedure to set up boundary scan circuitry, refer to “[Pattern Generation for a Boundary Scan Circuit](#)” in the *Tessent Scan and ATPG User’s Manual*.

---

### Note

 Use the [read\\_modelfile](#) command instead of this procedure to specify the initial values for RAMs.

---

## IJTAG Embedded Instruments

In the ATPG context patterns -scan, IJTAG is valid in the test\_setup and test\_end procedures only. It is invalid in any other procedure, as well as in ATPG’s analysis mode. Also, only [iReset](#) and [iCall](#) commands are valid in the test procedures.

For detailed information, see “[How to Set Up Embedded Instruments Through Test Procedures](#)” in the *Tessent IJTAG User’s Manual*.

## Bidirectional Scan Out Pins

The value of all bidirectional scan out pins must be forced to the Z state (indicating it is operating in “output” mode) to properly sensitize the scan chain. When reading in a test procedure file, the tool automatically adds force events to the beginning of the load\_unload procedure to force all bidi pins to Z.



Bidi pins that are clocks or constrained pins are not forced to a Z, as they were already forced to the off-state or the constrained values. Bidi scan-in pins are also not forced to a Z. Any bidi pin already forced later in the load\_unload procedure is not be forced to a Z. Any bidi forced to a specific value in the test\_setup procedure is instead be forced to this value instead of a Z.

Like previous automatic force values, these can be disabled by putting the “set autoforce off;” statement at the beginning of the procedure file.

## Pin Constraints

If you use the add\_input\_constraints command to set pin constraints, be aware this command only forces pins during capture. To constrain these pins during test\_setup, you should include the same pin constraints in the test\_setup procedure. This ensures the pins are in the same state for loading the first pattern as for loading all subsequent patterns.

If you do not properly constrain the pins prior to the end of the test\_setup procedure, the tool automatically constrains them by inserting a cycle statement in the test\_setup procedure. However, this automatic handling may not insert the events with the timing you want. Also, the automatic handling is not included in DRC.

If you have defined input constraints but have not provided a test\_setup procedure, the tool automatically generates a test\_setup procedure to force those pins to their constrained values.

You can use both the [write\\_procfile](#) and the [report\\_procedures](#) commands to see the contents of the test\_setup procedure the tool has generated. The write\_procfile command writes existing procedure and timing data to a specified file. The report\_procedures command writes the information to the screen.

## Example 1

The following is an example using a sub\_procedure. In this example, the signal named C retains its value of 1 during the test unless it is forced to a different value in a later cycle, by another procedure, or it is overwritten by WGL patterns.

```
procedure sub_procedure initialize =  
    template soc_timeplate ;  
    cycle =  
        force C 1;  
    end;  
end;
```

The following example shows how to apply the previous sub\_procedure. For more information, see “[Sub\\_procedure](#)” on page 833.

```
procedure test_setup =  
    timeplate soc_timeplate;  
    cycle =  
        force test_en 1; // force test_en 1  
        force chip_en 0; // force chip_en to 0  
    end;  
    apply initialize 10; // force C to 1 for 10 cycles  
end;
```

## Example 2

The following example shows a way to reset a memory. The RST signal is active for the first 128 cycles, then it is deactivated in the next cycle (cycle 129).

```
procedure sub_procedure reset_mem =  
    timeplate soc_timeplate;  
    cycle =  
        force RST 1;  
    end;  
end;  
procedure test_setup =  
    timeplate soc_timeplate;  
    apply reset_mem 128;  
    cycle =  
        force RST 0; // deactivate RST  
    end;  
end;
```

## Example 3

The following example shows a way to use an expect statement in a test\_setup procedure. The output signal (DFT) is expected to 1 in the first cycle and X in the remaining cycle. An “expect” statement does not work the same as a force or pulse statement. When none is present, it is assumed to mean do not measure.

```
procedure test_setup =  
    timeplate soc_timeplate;  
    cycle =  
        expect DFT 1;  
    end;  
end;
```

## Example 4

This example shows a way to start pulsing a clock in a test\_setup procedure. The SYSCLK starts pulsing at cycle number 2 until the end of test.

```

timeplate soc_timeplate =
    force pi;
    measure_po 90;
    pulse SYSCLK 50 50;
    period 100;
end;

procedure test_setup =
    timeplate soc_timeplate;
    cycle =
        force RST_L 0;
    end;
    cycle =
        pulse SYSCLK;
    end;
end;

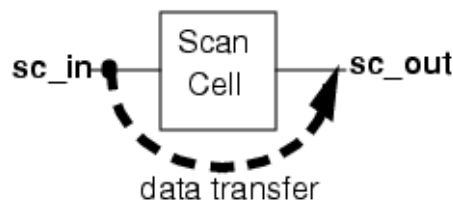
```

## Shift (Required)

This required procedure describes how to shift data one position down the scan chain by forcing the scan input, toggling the clock(s), and strobing the scan output.

Figure 11-2 shows the data flow process for the shift procedure.

**Figure 11-2. Shift Procedure**



Within this procedure, you must use the `force_sci`, or `force_sci_equiv`, and the `measure_sco` event statements. You can also use the `force` and `pulse` event statements. A shift procedure can contain more than one cycle, although not all pattern formats can support multiple cycles and parallel load. Pattern formats that do not support multiple cycles are any parallel format other than STIL and Verilog. If you use `write_patterns` to write out one of these other parallel formats with a multicycle shift procedure, the command generates an AG11 error.

The times at which the timeplate used by the shift procedure applies the `force_sci` and `measure_sco` commands must be consistent with proper operation of the `load_unload` procedure. The `measure_sco` occurs at the `measure_po` time specified in the timeplate. The `force_sci` occurs at the `force_pi` time specified in the timeplate.

The following are examples of the shift procedure for both mux-DFF and LSSD architectures.

## Mux-DFF Example

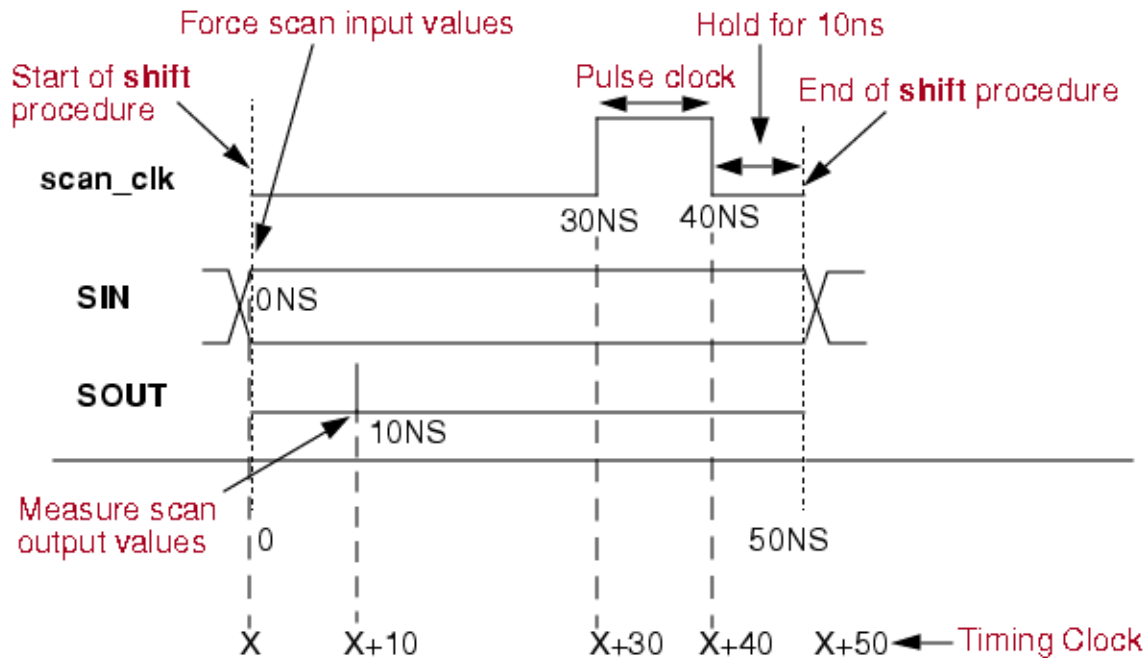
```
procedure shift =  
  timeplate tp1;  
  cycle =  
    // force scan chain input  
    force_sci;  
    // measure scan chain output  
    measure_sco;  
    // pulse the scan clock  
    pulse scan_clk;  
  end;  
end;
```

## LSSD Example

```
procedure shift =  
  timeplate tp2;  
  cycle =  
    // force scan chain input  
    force_sci;  
    // measure scan chain output  
    measure_sco;  
    // pulse master clock  
    pulse scan_pclk;  
    // pulse slave clock  
    pulse scan_sclk;  
  end;  
end;
```

[Figure 11-3](#) graphically displays the waveforms for the clock pin, the scan-in pin, and the scan-out pin derived from the Mux-DFF shift procedure example. This timing diagram shows one scan chain shift cycle, assuming the time unit is 1ns.

Figure 11-3. Timing Diagram for Shift Procedure



The procedure contains four scan events: forces scan input values at 0ns, strobes (or measures) scan output values at 10ns, pulses the scan clock scan\_clk (turning it on at 30ns and off at 40ns), and holds the state of the last event until the procedure finishes at 50ns.

A timing clock monitors when each significant event occurs. If the timing clock is at X when the shift procedure begins, the timing clock assigns those four events with time values X, X+10, X+30, and X+40. When the shift procedure finishes, the timing clock advances to X+50. The shift cycle ending time becomes the starting time for the next shift cycle.

## Alternate Shift Procedure (Optional)

When using on-chip clock generators, such as programmable PLLs, it is sometimes necessary to change values on input (control) signals to the clock generator a cycle or two before the change in generated clocking schemes is realized. When the shift clocks for a scan chain are also provided by the on-chip clock generator, it is sometimes not possible to reprogram the clock generator near the end of the scan chain shifting in order to stop the shift clock and prepare for the capture clocks. To accomplish this you might want to use an alternative shift procedure.

Alternate shift procedures have names, as described in the following paragraph. Alternate shift procedures can only be used for single shifts (a pre shift or a post shift), and there must be one un-named normal shift as the main shift in the required load\_unload procedure. See [Load\\_Unload \(Required\)](#).

The shift procedure enables an optional name following the shift procedure type. For each scan group, one shift procedure must be defined that has the default name of shift. For each scan group, additional alternate shift procedures can be defined as long as each has a unique name.

Each shift procedure is required to contain a `force_sci` or `force_sci_equiv` statement and a `measure_sco` statement.

## Syntax

```
procedure shift [ procedure_name ] =  
...  
end ;
```

## Example

The following is a partial example of how the alternate shift procedure might be used in a procedure file for a scan chain with a length of 100.

```
timeplate tp1 =
  force_pi 0;
  measure_po 10;
  pulse ref_clk 50 50;
  period 100;
end;
procedure shift =
  timeplate tp1;
  scan_group grp1;
  cycle =
    force ctrl_a 1;
    force_sci;
    measure_sco;
    pulse ref_clk;
  end;
end;
procedure shift shift_last =
  timeplate tp1;
  scan_group grp1;
  cycle =
    force ctrl_a 0;
    force_sci;
    measure_sco;
    pulse ref_clk;
  end;
end;
procedure load_unload =
  timeplate tp1;
  scan_group grp1;
  cycle =
    force ref_clk 0;
    force scan_en 1;
    force ctrl_a 1;
  end;
  apply shift 98;
  apply shift_last 1;
  apply shift_last 1;
end;
```

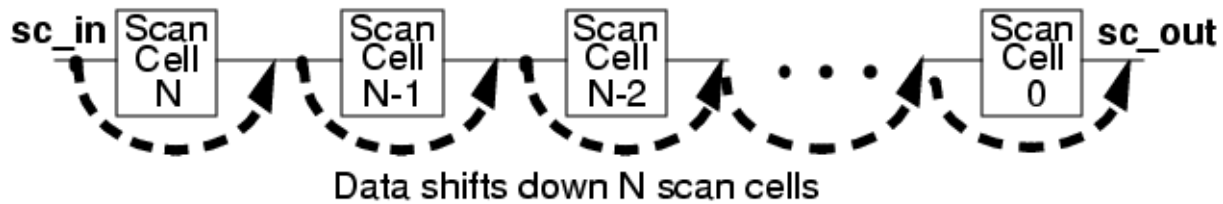
## Load\_Unload (Required)

This required procedure describes how to load and unload the scan chains in the scan group. To load the scan chain, you must force the circuit into the appropriate state for the start of the shift sequence. This includes forcing clocks, resets, RAM write control signals, and any other signals that need to be at their off-states for scan chain loading. Also, if a reset signal is defined as a clock, and pin constrained to its off-state in the dofile, it needs to again be forced to its off-state in the load\_unload and named capture procedures in order to avoid a P34 DRC.

The tool automatically adds the off-state for clock pins, constrained pin values, and other pins that have values forced in the test\_setup procedure as force statements to the beginning of the load\_unload procedure (if not present). This helps reduce DRC failures.

Figure 11-4 shows the data flow for the load\_unload procedure.

**Figure 11-4. Load\_Unload Procedure**



If the scan out pin is bidirectional, you must force its value to the Z state (indicating it is operating in “output” mode) to properly sensitize the scan chain. If there is a scan enable signal, you must force it on to enable the scan chain prior to the shift. You then use the apply shift statement to specify the number of shift cycles (which equals the number of scan elements in the chain). If you have optionally included the shadow\_control procedure (which, if used, immediately follows the shift procedure), you must also include the apply command.

The following list includes the basic statements in the load\_unload procedure:

### Mux-DFF Example

```
procedure load_unload =
  timeplate tp1;
  cycle =
    // force clocks off
    force RST 0;
    force CLK 0;
    // activate scanning mode
    force scan_en 1;
  end;
  // shift data thru each of 7 cells
  apply shift 7;
end;
```

### LSSD Example

```
procedure load_unload =
  timeplate tp2;
  cycle =
    // force all clocks off
    force RST 0;
    force CLK 0;
    force scan_sclk 0;
    force scan_mclk 0;
  end;
  // apply shift procedure 7 times
  apply shift 7;
end;
```

The timing for the load\_unload procedure is generally straightforward. The load\_unload procedure contains the apply statement. Therefore, the total time for a load\_unload procedure



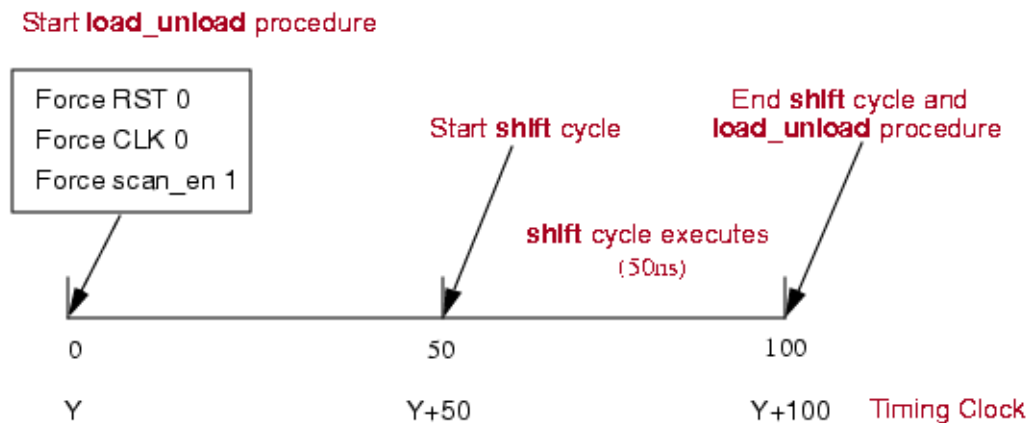
includes the time specified by the timeplate being used plus the time required to run the apply cycles.

For example, examine the following load\_unload procedure, using the example shift procedure in the previous section.

```
procedure load_unload =
  timeplate tp1;
  cycle =
    force RST 0;
    force CLK 0;
    force scan_en 1;
  end;
  apply shift 1;
end;
```

The timeplate of the load\_unload procedure specifies the period is 50 ns. However, the load\_unload procedure includes an apply statement that runs one shift procedure. The shift procedure requires an additional 50 ns. Thus, the load\_unload procedure actually requires a total time of 100 ns, as shown in [Figure 11-5](#).

**Figure 11-5. Timing Diagram for Load\_Unload Procedure**



Within the load\_unload procedure, after the completion of the cycle block, the shift procedure starts at 50 ns, runs for 50 ns, and ends at 100 ns. Thus, the load\_unload procedure also ends at 100 ns.

As with the shift procedure, the timing clock determines the event times for the load\_unload procedure. If the timing clock is at Y when the load\_unload procedure begins, the first three events happen at time Y. When the apply cycle runs, the timing clock advances to Y+50, which is when the shift procedure begins. As mentioned previously, the shift procedure requires 50 time units. Therefore, when the apply cycle finishes, the timing clock reads Y+100.

Because it is the last event in the load\_unload procedure, the end of the apply cycle determines the end of the load\_unload procedure.

## Shadow\_Control (Optional)

The optional shadow\_control procedure, which may only contain force and pulse event statements, describes how to load the contents of a scan cell into the associated shadow.

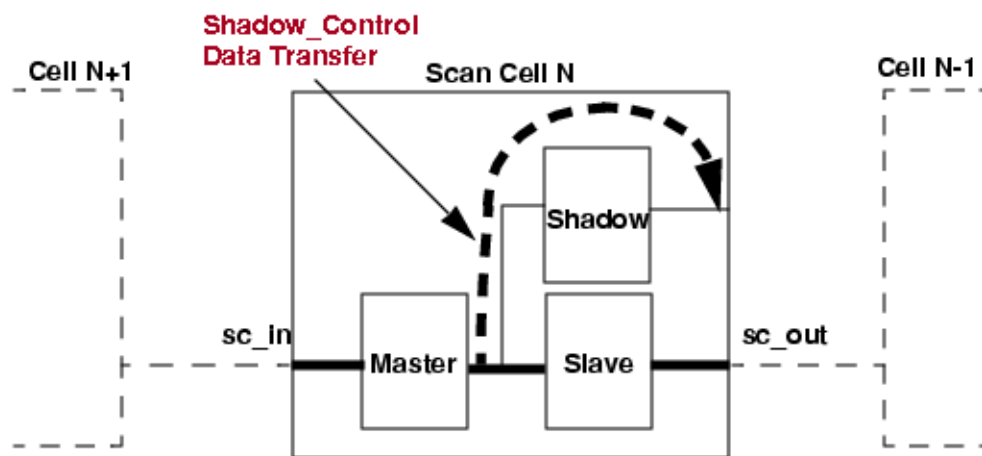
### Note

 This procedure is not supported when using SSN.

---

If you use this procedure, you must also apply the shadow\_control command in the load\_unload procedure. This procedure must not disturb the contents of any of the scan cells. [Figure 11-6](#) shows the data flow for the shadow\_control procedure.

**Figure 11-6. Shadow\_Control Procedure**



The following procedure file example demonstrates the syntax for applying a shadow\_control procedure within a load\_unload procedure:

```

proc shift =
    force_sci      0;
    measure_sco    0;
    force SK2      1 1;
    force SK2      0 2;
    force SK1      1 3;
    force SK1      0 4;
end;

proc load_unload =
    force WE      0 0;
    force ABC     1 0;
    force COMB_B  1 0;
    force SK2     0 0;
    force CLK     0 0;
    force SK1     0 0;
    force sh_clk  0 0;
    apply shift   5 1;
    apply shadow_control 1 2;
end;

proc shadow_control =
    force sh_clk      1 1;
    force sh_clk      0 2;
end;

proc master_observe =
    force WE      0 0;
    force SK2     0 0;
    force CLK     0 0;
    force SK1     0 0;
    force sh_clk  0 0;
    force SK1     1 1;
    force SK1     0 2;
end;

proc shadow_observe =
    force WE      0 0;
    force EN      1 0;
    force SK2     0 0;
    force CLK     0 0;
    force SK1     0 0;
    force sh_clk  0 0;
    force CLK     1 1;
    force CLK     0 2;
    force SK1     1 3;
    force SK1     0 4;
end;

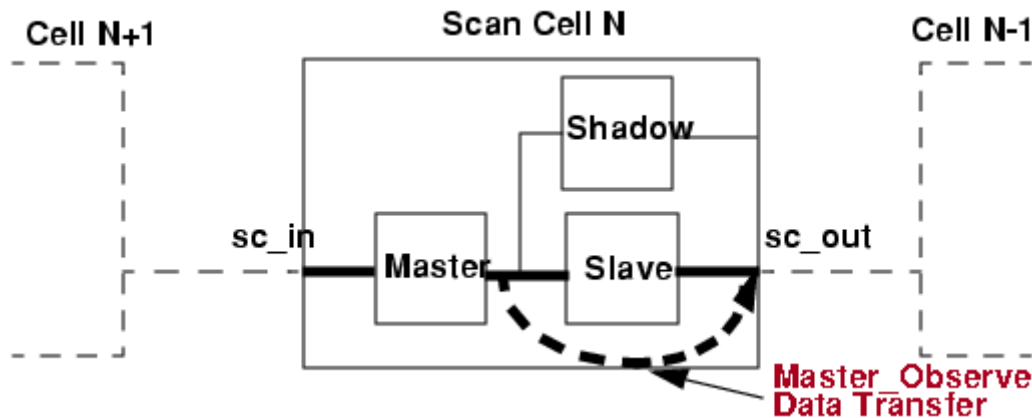
```

## Master\_Observe (Sometimes Required)

The master\_observe procedure, which may only contain force and pulse event statements, describes how to place the contents of a master element into the output of its scan cell, where you can observe it by using the unload operation.

Figure 11-7 shows the data flow for the master\_observe procedure.

**Figure 11-7. Master\_Observe Procedure**



You do not need to use this procedure if the master element's output is the output of the scan cell. The D1 rule ensures that this procedure does not disturb the master memory element's contents. You can override this requirement by changing the D1 rule handling. The following example shows a master\_observe procedure for the LSSD architecture:

```
// LSSD architecture example
procedure master_observe =
  timeplate tpl;
  cycle =
    // Force all clocks off
    force scan_sclk 0;
    force scan_mclk 0;
    force rst      0;
    force clk      0;
    // Pulse the slave clock
    pulse scan_sclk;
  end;
end;
```

## Shadow\_Observe (Optional)

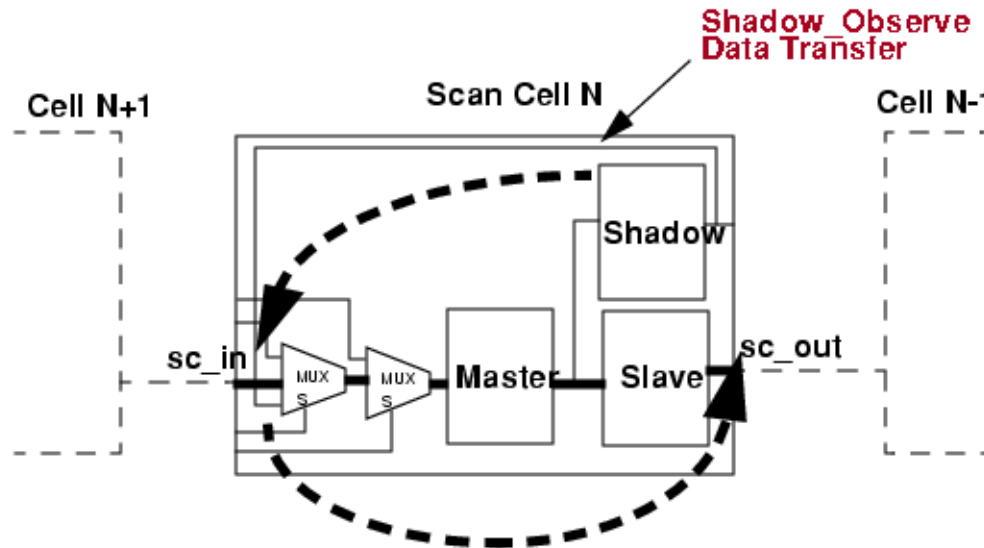
The optional shadow\_observe procedure, which may only contain force and pulse event statements, describes how to place the contents of a shadow into the output of its scan cell, assuming that data can be transferred in this way in the scan cell. Once the data is at the scan cell output, you can observe it by applying the unload command. This procedure enables the shadow to be used as an observation point in the design.

### Note

This procedure is not supported when using SSN.

Figure 11-8 shows the data flow of the shadow\_observe procedure.

**Figure 11-8. Shadow\_Observe Procedure**



## Skew\_Load (Optional)

The optional skew\_load procedure propagates the output value of the preceding scan cell into the master memory element of the current cell without changing the slave, for all scan cells. Using only force and pulse event statements, this procedure defines how to apply an additional pulse of the master shift clock after the scan chains are loaded.

Figure 11-9 shows the data flow of the skew\_load procedure.

Figure 11-9. Skew\_Load Procedure

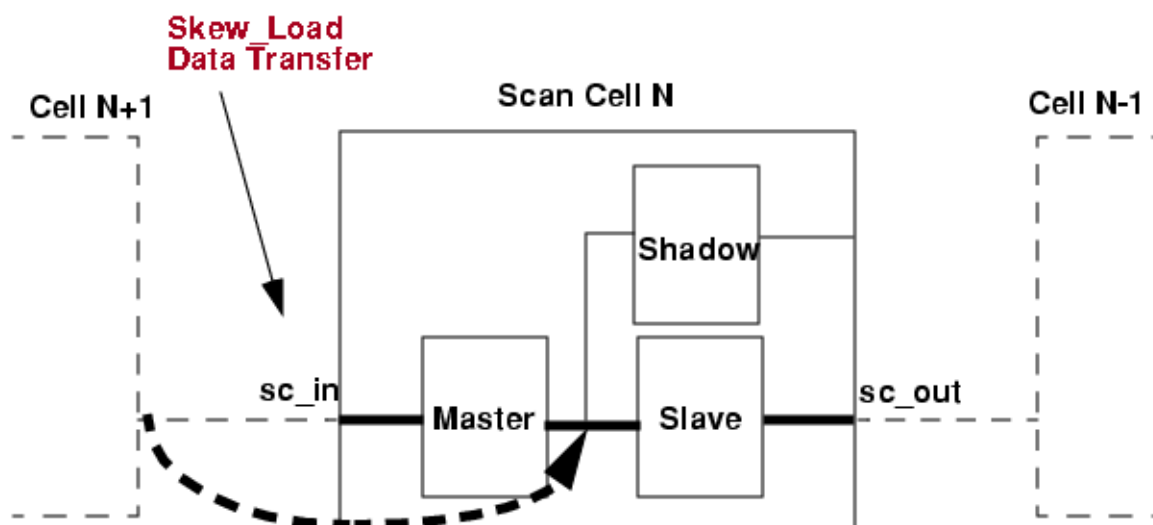
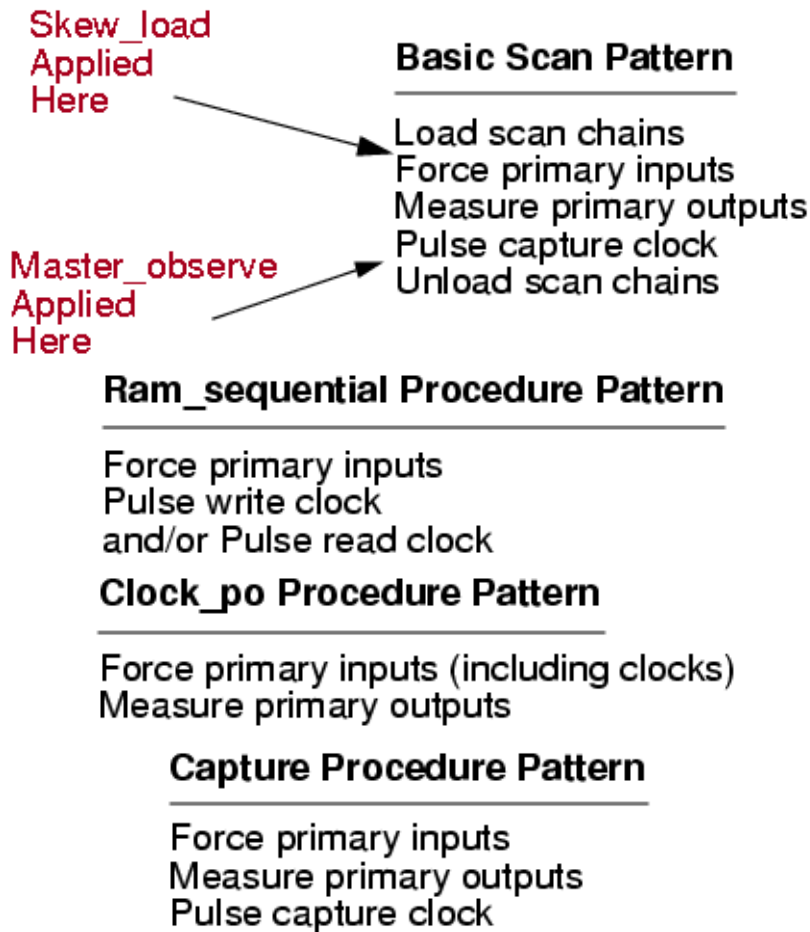


Figure 11-10 shows where you apply the skew\_load procedure and the master\_observe procedure within the basic scan pattern events.

Figure 11-10. Skew\_load applied within Pattern



## Clock\_Control (Optional)

You can manually create clock control definitions in the test procedure file.

For complete information about when you use this definition, refer to “[Support for Internal Clock Control](#)” in the *Tessent Scan and ATPG User’s Manual*.

### ATPG Restrictions

The following restrictions apply to ATPG when clock control definitions are enabled:

- In undefined cycles, the internal clock is assumed to be off, even if the source clock pulses.
- Source clocks are pulsed regardless of clock restrictions. You should define explicitly any false paths using DC or the [add\\_false\\_paths](#) command.
- External clocks without clock control definitions are controlled through top-level pins.

- Clock control definitions applied to a clock defined as equivalent also applies to all associated equivalent clocks.
- Timeplate definitions apply only to external clocks.
- If you use the “[set\\_clock\\_restriction](#) -same\_clocks\_between\_loads” command, you must use one of the following definitions to pulse the controlled clock:
  - {ATPG\_SEQUENCE, END}
  - {ATPG\_SEQUENCE <N> <M>, END} with N starting from 0. The generated test pattern includes M+1 capture cycles between the scan loading operation and the scan unloading operation.
  - {ATPG\_CYCLE <i>, END} with i=0 defined

If none of these statements are present, the tool displays a warning during ATPG about clock controls that cannot be applied because of clock restrictions.

## Rules for Clock Control Definitions

The following rules apply to the clock control definitions in the test procedure file:

- Global control conditions and source clocks defined for equivalent clocks must be the same.
- When a clock is forced off, it cannot be used as a source in the same definition.
- A FORCE statement cannot force a clock pin to an on state.
- Clock control cannot be applied to an asynchronous free-running clock.
- Condition statements cannot be applied to non-scan state elements.
- You must specify a condition to turn on the internal clock; otherwise, it is assumed to be off.
- When multiple sequence clock control definitions are defined for the same clock, they must use mutually exclusive pulse conditions as follows:
  - The clock control condition to pulse a clock in sequence mode must be mutually exclusive with the clock control condition for the same clock in per-cycle mode.
  - The condition to pulse a clock in sequence mode must be mutually exclusive with the condition to pulse the same clock by using another sequence mode.

## Keywords

The following is a list of keywords used in clock control statement:

- **ATPG\_CYCLE *cycle\_number***



A literal and integer pair that specifies a test pattern capture cycle to map the clock control to. The specified capture cycle values start from 0, which corresponds to the first capture cycle after scan loading.

Multiple ATPG\_CYCLE definitions can be declared to pulse the internal clock at the same capture cycle with different sets of conditions.

Use a FORCE statement to turn the clock off, or the clock continues to pulse when the conditions are satisfied.

The specified and actual capture cycles may differ—see “[Capture Cycle Determination](#)” in the *Tessent Scan and ATPG User’s Manual*.

- **ATPG\_SEQUENCE N M**

A literal and an integer pair that specify a range of capture cycles for clock pulsing from N to M consecutively.

Define the condition to pulse the clock continuously from capture cycle N ( $N \geq 0$ ) to capture cycle M ( $M \geq N$ ) right after scan loading. If N is greater than 0, the clock is automatically set to off state from the first capture cycle right after scan loading to the capture cycle N - 1. When the generated test pattern includes more than M capture cycles after scan loading, the clock is set to off state from the M + 1 capture cycle to the last capture cycle.

You can declare multiple ATPG\_SEQUENCE definitions, as necessary.

Use a FORCE statement to turn the clock off, or the clock continues to pulse when the conditions are satisfied.

The specified and actual capture cycles may differ—see “[Capture Cycle Determination](#)” in the *Tessent Scan and ATPG User’s Manual*.

- **ATPG\_SEQUENCE**

A literal that specifies clock pulsing. When a condition list is provided, the controlled clock pulses in all capture cycles in the pattern when the conditions are met. When checking the off conditions for cycles outside the capture window, the conditions listed in this special atpg\_sequence are ignored.

When there is no condition list, the controlled clock pulses unconditionally in every capture cycle between scan loading.

You can declare multiple ATPG\_SEQUENCE definitions, as necessary.

Use a FORCE statement to turn the clock off, or the clock continues to pulse when the conditions are satisfied.

The specified and actual capture cycles may differ—see “[Capture Cycle Determination](#)” in the *Tessent Scan and ATPG User’s Manual*.

- **CLOCK\_CONTROL pin\_pathname**

A literal and string value that specifies the pin pathname of the PI for the internal clock. The specified pin must be an existing clock. You must define internal clocks using the [add\\_clocks](#) command.

- **CONDITION *cell\_name value***

An optional literal and double string that specifies necessary scan cell states (conditions). The scan cell is specified by the pin pathname associated with the output of its state element. The value argument specifies the value loaded into the scan cell at the end of shift.

The specified and actual capture cycles may differ—see “[Capture Cycle Determination](#)” in the *Tessent Scan and ATPG User’s Manual*.

- **FORCE *pin\_pathname value***

A literal and double string that forces a value of 0, 1, or Z on a specified pin. The specified pin names must be valid pin pathnames for primary inputs. This keyword is used to force necessary pins off during capture cycles when the controlled clock is pulsed.

- **SOURCE\_CLOCK *pin\_pathname...***

A literal and repeatable string that specifies one or more source clocks to drive the internal clock logic to pulse in the specified capture cycle(s). If no source clock is specified, the source clock is assumed to be an always-capture clock that pulses in every capture cycle.

- **FAST\_CAPTURE\_STAGGERED\_GROUP *group\_number***

A literal and integer pair that specifies the encoding of the fast capture staggered group register. Group numbering starts at 0. CONDITION statements in each FAST\_CAPTURE\_STAGGERED\_GROUP wrapper define the fast capture staggered group register value that assigns a source clock to that group. The ATPG tool determines this value automatically based on which clocks to pulse together and which clocks to stagger in capture. This value then loads into the fast capture staggered group register as a part of the scan chain loading process, similar to other CONDITION registers. Refer to “[Fast Capture Staggered Group Example](#)” on page 821 to see how these CONDITION statements might be written.

The tool performs the following checks for the definition:

| If the following is true:                                               | The tool performs this check...                                                                            |
|-------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| A clock control with a fast capture staggered group register is defined | No source clock definition exists.<br>OR<br>The source clock is a pulse_always/<br>pulse_in_capture clock. |

| If the following is true:                                             | The tool performs this check...                                                                                                                                                                                |
|-----------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| A fast capture staggered group register is defined                    | The tool is in “fast capture” mode.<br>For more information about clock control operation modes, refer to the section “ <a href="#">Operating Modes</a> ” in the <i>Tessent™ Scan and ATPG User’s Manual</i> . |
| The “ <a href="#">add_core_instances</a> ” command has been performed | It is consistent with the OCC setting.                                                                                                                                                                         |

#### Note

 The tool treats any clock controls for which no fast capture staggered group register is defined as though it is always in the first staggering group (0). You cannot pulse these clocks in any other staggering group.

Group numbers are global across the design.

- **END**

Required literal the specifies the end of an ATPG\_CYCLE or ATPG\_SEQUENCE block, or at the end of the clock control definition.

## Global Condition Statements

Global conditions are **CONDITION** statements that are accessible in every scope of a clock control definition. You can use global conditions to define default conditions within a clock control definition.

For example, you can specify a **CONDITION** statement for all ATPG\_CYCLE/ATPG\_SEQUENCE blocks within a definition. Then you can define a **CONDITION** statement within individual ATPG\_CYCLE/ATPG\_SEQUENCE blocks to override the global **CONDITION** variable when necessary.

The following example uses local conditions (*italicized*) to define some of the control bits necessary for each scan cell to pulse the clock. The global conditions define other conditions that must be satisfied for all clock cycles.

```
CLOCK_CONTROL /clk_ctrl/int_clk1 =  
    SOURCE_CLOCK ref_clk;  
    CONDITION /clk_ctrl/enable_1/q 0;  
    CONDITION /clk_ctrl/enable_2/q 0;  
    ATPG_CYCLE 0 =  
        CONDITION /clk_ctrl/F0/q 1;  
    END;  
    ATPG_CYCLE 1 =  
        CONDITION /clk_ctrl/F1/q 1;  
    END;  
    ATPG_CYCLE 2 =  
        CONDITION /clk_ctrl/F2/q 1;  
    END;  
    ATPG_CYCLE 3 =  
        CONDITION /clk_ctrl/F3/q 1;  
    END  
END;
```

The previous example is equivalent to the following:

```
CLOCK_CONTROL /clk_ctrl/int_clk1 =  
    SOURCE_CLOCK ref_clk;  
    ATPG_CYCLE 0 =  
        CONDITION /clk_ctrl/F0/q 1;  
        CONDITION /clk_ctrl/enable_1/q 0;  
        CONDITION /clk_ctrl/enable_2/q 0;  
    END;  
    ATPG_CYCLE 1 =  
        CONDITION /clk_ctrl/F1/q 1;  
        CONDITION /clk_ctrl/enable_1/q 0;  
        CONDITION /clk_ctrl/enable_2/q 0;  
    END;  
    ATPG_CYCLE 2 =  
        CONDITION /clk_ctrl/F2/q 1;  
        CONDITION /clk_ctrl/enable_1/q 0;  
        CONDITION /clk_ctrl/enable_2/q 0;  
    END;  
    ATPG_CYCLE 3 =  
        CONDITION /clk_ctrl/F3/q 1;  
        CONDITION /clk_ctrl/enable_1/q 0;  
        CONDITION /clk_ctrl/enable_2/q 0;  
    END  
END;
```

The previous example demonstrates the importance of ensuring that global conditions do not conflict with local conditions. To further illustrate this point, consider the following incorrect definition of global conditions:

```
// Example of incorrect definition of global conditions
CLOCK_CONTROL /clk_ctrl/int_clk1 =
SOURCE_CLOCK ref_clk;
    CONDITION /clk_ctrl/F0/q 0;
    ATPG_CYCLE 0 =
        CONDITION /clk_ctrl/F0/q 1;
    END;
    ATPG_CYCLE 1 =
        CONDITION /clk_ctrl/F1/q 1;
    END;
    ATPG_CYCLE 2 =
        CONDITION /clk_ctrl/F2/q 1;
    END;
    ATPG_CYCLE 3 =
        CONDITION /clk_ctrl/F3/q 1;
    END
END;
```

On the surface, it may appear correct that the condition in ATPG\_CYCLE 0 overrides the global condition while the other cycles can still be satisfied. However, after the global condition is expanded to all cycles, the clock control definition looks like this:

```
// Example of incorrect definition of global conditions
CLOCK_CONTROL /clk_ctrl/int_clk1 =
SOURCE_CLOCK ref_clk;
    ATPG_CYCLE 0 =
        CONDITION /clk_ctrl/F0/q 1;
    END;
    ATPG_CYCLE 1 =
        CONDITION /clk_ctrl/F1/q 1;
        CONDITION /clk_ctrl/F0/q 0;
    END;
    ATPG_CYCLE 2 =
        CONDITION /clk_ctrl/F2/q 1;
        CONDITION /clk_ctrl/F0/q 0;
    END;
    ATPG_CYCLE 3 =
        CONDITION /clk_ctrl/F3/q 1;
        CONDITION /clk_ctrl/F0/q 0;
    END
END;
```

You can now see that it would not be possible to pulse the clock in ATPG\_CYCLE 0 while also pulsing it in any other cycle. The tool can load only one value into /clk\_ctrl/F0, so it can either pulse the clock in cycle 0 by loading a 1 or pulse it in another cycle by loading a 0.

## Per-Cycle Clock Control Definition Example

The following example defines per-cycle clock control for two internal clocks (/top/core1/clk1 and /top/core1/clk2):

```
CLOCK_CONTROL /top/core1/clk1 =
  ATPG_CYCLE 0 =
    CONDITION /pll_ctl/cell_0/Q 1;
  END;
  ATPG_CYCLE 1 =
    CONDITION /pll_ctl/cell_1/Q 1;
    CONDITION /pll_ctl/cell_4/Q 0;
  //both conditions must be satisfied for clock to pulse in
  //capture cycle 1
  END;
END;
CLOCK_CONTROL /top/core1/clk2 =
  ATPG_CYCLE 0 =
    CONDITION /pll_ctl/cell_2/Q 1;
  END;
  ATPG_CYCLE 1 =
    CONDITION /pll_ctl/cell_3/Q 1;
    CONDITION /pll_ctl/cell_5/Q 1;
  END;
END;
```

## Sequence Clock Control Definition Example

The following example defines sequence clock control for two internal clocks (/top/core/clk1 and /top/core1/clk2) derived from the source clock clk\_src:

```
CLOCK_CONTROL /top/core1/clk1 =
  SOURCE_CLOCK clk_src;
  ATPG_SEQUENCE 0 1 =
  // Pulses 2 consecutive cycles if the scan cell
  // is loaded with 1, and the source clock is pulsed.
    CONDITION /pll_ctl/cell_1/Q 1;
  END;
END;
CLOCK_CONTROL /top/core1/clk2 =
  SOURCE_CLOCK clk_src;
  ATPG_SEQUENCE 0 1=
    CONDITION /pll_ctl/cell_2/Q 1;
  END;
END;
```

The following example defines sequence clock control for two internal clocks (/top/core/clk1 and /top/core1/clk2) derived from the source clock clk\_src. The clock pulses in all capture cycles when the conditions are met.

```
CLOCK_CONTROL /top/core1/clk1 =
    SOURCE_CLOCK clk_src;
    ATPG_SEQUENCE =
// Pulse clock in all capture cycles if the scan cell
// is loaded with 1, and the source clock is pulsed.
    CONDITION /pll_ctl/cell_1/Q 1;
    END;
END;
CLOCK_CONTROL /top/core1/clk2 =
    SOURCE_CLOCK clk_src;
    ATPG_SEQUENCE =
    CONDITION /pll_ctl/cell_2/Q 1;
    END;
END;
```

The following example pulses clock /top/core1/clk1 unconditionally in every capture cycle between scan loading:

```
CLOCK_CONTROL /top/core1/clk1 =
    ATPG_SEQUENCE =
// empty body
    END;
END;
```

The following example defines a multi-sequence clock control definition:

```
CLOCK_CONTROL /top/core/clk1_int =
    SOURCE_CLOCK /clk1;
    ATPG_SEQUENCE 0 2 =
        CONDITION /pll/ctl_1/Q 1;
        FORCE ENABLE_1 1;
    END;
    ATPG_SEQUENCE 3 4 =
        CONDITION /pll/ctl_1/Q 0;
        FORCE ENABLE_1 1;
    END;
END;
```

Exclusive conditions ensure that only one sequence block is applied per capture cycle (otherwise, no sequence is applied). If no cycle numbers are specified for sequence clock control, the clock pulses in every capture cycle when conditions are loaded.

## Multiple Sets of Conditions for the Same Cycle Example

The following example shows that if a clock can be pulsed in a particular cycle or sequence of cycles when there are multiple sets of conditions where any one set can activate the clock for that cycle, the same cycle can be defined multiple times:

```
CLOCK_CONTROL /top/core1/clk1 =  
  ATPG_CYCLE 0 =  
    CONDITION /pll_ctl/cell_1/Q 1;  
  END;  
  ATPG_CYCLE 0 =  
    CONDITION /pll_ctl/cell_2/Q 1;  
  END;  
END;
```

The previous example shows that /top/core1/clk1 can be pulsed in ATPG\_CYCLE 0 when any set of the specified conditions are met. This demonstrates the case where loading a '1' into either /pll\_ctl/cell\_1 or /pll\_ctl/cell\_2 pulses the clock in cycle 0.

Similarly, the following example defines multiple sets of conditions for the same sequence of cycles, which can overlap. The sequence of cycles must have mutually exclusive conditions to ensure conditions for each ATPG\_SEQUENCE can be satisfied without conflicting with other sequences.

```
CLOCK_CONTROL /top/core1/clk1 =  
  ATPG_SEQUENCE 0 2 =  
    CONDITION /pll_ctl/cell_1/Q 1;  
    CONDITION /pll_ctl/cell_2/Q 0;  
    CONDITION /pll_ctl/cell_3/Q 0;  
  END;  
  ATPG_SEQUENCE 0 3 =  
    CONDITION /pll_ctl/cell_1/Q 0;  
    CONDITION /pll_ctl/cell_2/Q 1;  
    CONDITION /pll_ctl/cell_3/Q 0;  
  END;  
  ATPG_SEQUENCE 1 4 =  
    CONDITION /pll_ctl/cell_1/Q 0;  
    CONDITION /pll_ctl/cell_2/Q 0;  
    CONDITION /pll_ctl/cell_3/Q 1;  
  END;  
END;
```



## Source Clocks with Different Frequencies Example

The following example defines source clocks that have different frequencies when using clock control definitions:

```
timeplate _default_WFT_ =
    force_pi 0 ;
    measure_po 40 ;
    pulse clk1 45 10;
    pulse ref_clock 15 5, 40 5, 65 5, 90 5;
    pulse clocks_02/my_controller/U2/Z 45 10;
    pulse clocks_03/my_controller/U2/Z 45 10;
    pulse clocks_04/my_controller/U2/Z 45 10;
    period 100 ;
end;

procedure capture =
    timeplate _default_WFT_;
    cycle =
        force_pi ;
        measure_po ;
        pulse_capture_clock ;
    end;
end;
```

In this example, for one pulse of clk1, there are 4 pulses of ref\_clock, specifically the ref\_clock frequency is 4 times the frequency of clk1.

## Fast Capture Staggered Group Example

The following example uses the FAST\_CAPTURE\_STAGGERED\_GROUP keyword to define four groups (highlighted in green) each for two clocks, “fast\_clk1” and “fast\_clk2”. This corresponds to the example waveform shown in section “[Fast Capture Clock Staggering](#)” in the *Tessent Scan and ATPG User’s Manual*.

```
CLOCK_CONTROL /clk_ctrl/fast_clk1_gated =  
  SOURCE_CLOCK fast_clk1;  
  FAST_CAPTURE_STAGGERED_GROUP 0 =  
    CONDITION /clk1_occ_inst/occ_control/fast_capture_staggered_group_reg[0] 0;  
    CONDITION /clk1_occ_inst/occ_control/fast_capture_staggered_group_reg[1] 0;  
  END;  
  FAST_CAPTURE_STAGGERED_GROUP 1 =  
    CONDITION /clk1_occ_inst/occ_control/fast_capture_staggered_group_reg[0] 1;  
    CONDITION /clk1_occ_inst/occ_control/fast_capture_staggered_group_reg[1] 0;  
  END;  
  FAST_CAPTURE_STAGGERED_GROUP 2 =  
    CONDITION /clk1_occ_inst/occ_control/fast_capture_staggered_group_reg[0] 0;  
    CONDITION /clk1_occ_inst/occ_control/fast_capture_staggered_group_reg[1] 1;  
  END;  
  FAST_CAPTURE_STAGGERED_GROUP 3 =  
    CONDITION /clk1_occ_inst/occ_control/fast_capture_staggered_group_reg[0] 1;  
    CONDITION /clk1_occ_inst/occ_control/fast_capture_staggered_group_reg[1] 1;  
  END;  
  ATPG_CYCLE 0 =  
    CONDITION /clk1_occ_inst/occ_control/ShiftReg/FF_reg[0]/q 1;  
  END;  
  ATPG_CYCLE 1 =  
    CONDITION /clk1_occ_inst/occ_control/ShiftReg/FF_reg[1]/q 1;  
  END;  
END;
```

```
CLOCK_CONTROL /clk_ctrl/fast_clk2_gated =  
  SOURCE_CLOCK fast_clk2;  
  FAST_CAPTURE_STAGGERED_GROUP 0 =  
    CONDITION /clk2_occ_inst/occ_control/fast_capture_staggered_group_reg[0] 0;  
    CONDITION /clk2_occ_inst/occ_control/fast_capture_staggered_group_reg[1] 0;  
  END;  
  FAST_CAPTURE_STAGGERED_GROUP 1 =  
    CONDITION /clk2_occ_inst/occ_control/fast_capture_staggered_group_reg[0] 1;  
    CONDITION /clk2_occ_inst/occ_control/fast_capture_staggered_group_reg[1] 0;  
  END;  
  FAST_CAPTURE_STAGGERED_GROUP 2 =  
    CONDITION /clk2_occ_inst/occ_control/fast_capture_staggered_group_reg[0] 0;  
    CONDITION /clk2_occ_inst/occ_control/fast_capture_staggered_group_reg[1] 1;  
  END;  
  FAST_CAPTURE_STAGGERED_GROUP 3 =  
    CONDITION /clk2_occ_inst/occ_control/fast_capture_staggered_group_reg[0] 1;  
    CONDITION /clk2_occ_inst/occ_control/fast_capture_staggered_group_reg[1] 1;  
  END;  
  ATPG_CYCLE 0 =  
    CONDITION /clk2_occ_inst/occ_control/ShiftReg/FF_reg[0]/q 1;  
  END;  
  ATPG_CYCLE 1 =  
    CONDITION /clk2_occ_inst/occ_control/ShiftReg/FF_reg[1]/q 1;  
  END;  
END;
```

In the preceding example, to assign the clock “fast\_clk1” to group 0, ATPG must set both fast\_capture\_staggered\_group\_reg[0] and fast\_capture\_staggered\_group\_reg[1] to 0. To assign

it to group 1, ATPG must set `fast_capture_staggered_group_reg[0]` to 1 and `fast_capture_staggered_group_reg[1]` to 0, and so on.

## Clock\_run (Optional)

For every controller, or concurrent controller group, you can write a `clock_run` procedure, if needed. The `clock_run` procedure has both an internal mode as well as an external mode.

You can specify only one `clock_run` procedure per controller or concurrent group; however, you do not need to specify a separate procedure for each controller instance. The same procedure can be used for multiple controllers. You need to specify a separate procedure for a controller instance only if it maps to a different set of internal clocks.

In case of controllers running concurrently, and some of these controllers clocks are driven by PLL internal clocks, the `clock_run` procedure is required per concurrent group. It is not required for every BIST controller participating in the group to have its clock driven by a PLL internal clock. For some controllers, their clocks can be driven by a PLL reference clock or even by a system clock.

The tool relies on you to control the PLL control signal. This can be achieved by forcing the PLL control signal to a proper value in a `test_setup` procedure and in external mode of `clock_run` procedure as well (it depends on the PLL model behavior).

A `clock_run` procedure has to have a N-to-1 or 1-to-N ratio between internal and external cycles; that is, either the internal mode has to have only one cycle, or the external mode has to have only one cycle. You cannot have, for example, two external cycles and three internal cycles.

## Capture Procedures (Optional)

There are three types of capture procedures. These procedures are optional in the “patterns -scan” context.

- The default capture procedure is an optional capture procedure, without a name, that provides information on how the series of capture events are broken into cycles and which timeplates these cycles use. The default capture procedure is defined in the procfile as part of the scan group definition or internally derived by the tool when you do not define one.
- The named capture procedure is an optional capture procedure with a name that defines explicit clock cycles. You can create multiple named capture procedures, each with a unique name, using the [create\\_capture\\_procedures](#) command. If you need to manually create or edit named capture procedures, see “Rules for Creating and Editing Named Capture Procedures” in this chapter. For information on using named capture procedures to create at-speed test patterns, see “[At-Speed Test With Named Capture Procedures](#)” in the *Tessent Scan and ATPG User’s Manual*.
- The external\_capture procedure is an optional capture procedure used for all capture cycles between each scan load, even when the pattern is a multiple load pattern. External\_capture procedures are used by the “[set\\_external\\_capture\\_options](#) -capture\_procedure” command. External\_capture procedures that are used with this command have several restrictions:
  - The procedure can only have one force\_pi statement and no measure\_po statements. This is because to use the “set\_external\_capture\_options -capture\_procedure” switch, the patterns to be saved must be hold\_pi and mask\_po patterns. The statements in the capture procedure must match up to this.
  - Unlike named capture procedures, the external\_capture procedure cannot have any load\_cycles as it is meant to be used between each scan load of a pattern. The external\_capture cannot contain any events on internal signals.
  - When using the -capture\_procedure switch with set\_external\_capture\_options, all clocks in the design that are internally connected (don’t trace to just a cut point), must be controlled. In addition, the clocks must either be constrained, always-capture, controlled by a clock\_control definition, or the clock must have an event in the external\_capture procedure.

|                                                                         |                            |
|-------------------------------------------------------------------------|----------------------------|
| <b>Rules for Creating and Editing a Default Capture Procedure .....</b> | <b><a href="#">825</a></b> |
| <b>Rules for Creating and Editing Named Capture Procedures .....</b>    | <b><a href="#">825</a></b> |
| <b>Slow and Load Types in the Cycle Statement .....</b>                 | <b><a href="#">828</a></b> |
| <b>launch_capture_pair Statement .....</b>                              | <b><a href="#">829</a></b> |

## Rules for Creating and Editing a Default Capture Procedure

There are several issues to consider when working with default capture procedures.

- The default procedure may only contain `force_pi`, `measure_po`, `pulse_capture_clock`, `bidir_force`, `bidir_force_pi`, `bidir_force_off`, and `bidir_measure_po` event statements that represent the non-scan activity for a normal pattern. There is no overlap between the capture procedure and the existing clock procedure.
- Use the `pulse_capture_clock` statement in the default capture procedure to indicate in which cycle one or more capture clocks should be pulsed.
- Do not specify any complex clocking that needs to be described for capture clocks or other clocks in the default capture procedure; specify it in the clock procedure or by using a named capture procedure.
- Do not specify any type of pin or ATPG constraint in the default capture procedure. For example, specifying that a certain pin is to be held at a certain state in the default capture procedure does not restrict the ATPG engine from applying different values to that pin. However, you can use the `bidir_force` and `bidir_force_pi` statements in the default capture procedure to force all bidirectional pins off in one cycle and force the ATPG values on the bidirectional pins in the next cycle.

## Rules for Creating and Editing Named Capture Procedures

There are several issues to consider when working with named capture procedures.

- A named capture procedure may only contain `force_pi`, `measure_po`, `observe_method`, `pulse` (named clock), and `condition` statements.
- If you use mode definitions, all cycles in a procedure must be defined within mode definitions. Use the keyword “mode” with two mode blocks: “internal” and “external”. Use the `mode_internal` definition to describe what happens on the internal side of the on-chip PLL. Use the `mode_external` definition to describe what happens on the external side of the on-chip PLL.
- All events in a named capture procedure that use modes must be duplicated in both modes. The only difference is that the internal mode uses only internal clocks and the external mode uses only external clocks. The number of cycles and timeplates used can be different as long as the total period of both modes is the same.
- Signal events used in both internal and external modes must happen at the same time. Examples of these events are `force_pi`, `measure_po`, and other signal forces, but also include clocks that can be used in both modes.

- If a `measure_po` statement is used, it can only appear in the last cycle of the internal mode and must occur before the last clock pulse. If no `measure_po` statement is used, the tool issues a warning that the primary outputs cannot be observed.
- The cumulative time from the start of the first cycle to the `measure_po` must be the same in both modes.
- The external mode cannot pulse any internal clocks or force any internal control signals.
- A `force_pi` statement needs to appear in the first cycle of both modes and occur before the first pulse of a clock.
- If an external clock goes to the PLL and to other internal circuitry, a C2 DRC violation is issued.
- At-speed cycles need to be continuous; that is, a named capture procedure cannot have more than one at-speed clocking subsequence.
- All defined real clocks (excluding internal clocks) must be forced to off state first in the `mode_internal` definition.

For more information, see “[Internal and External Modes Definition](#)” in the *Tessent Scan and ATPG User’s Manual*.

- Do not use the `pulse_capture_clock` statement in a named capture procedure. The clocks used are explicitly pulsed.

- If you want to specify the internal conditions that need to be met at certain scan cells in order to enable a clock sequence, use the condition statement before the cycle statement in the named capture procedure, as shown in the following example:

```

procedure capture capture1 =
    scan_group groupcnt ;
    timeplate gen_tpl ;
    condition /inst_0/clkenhold/reg_sco/Q 1;
    condition /inst_1/clkenhold/reg_sco/Q 1;
    condition /inst_2/clkenhold/reg_sco/Q 1;
    condition /inst_3/clkenhold/reg_sco/Q 1;
    condition /inst_4/clkenhold/reg_sco/Q 1;
    condition /inst_5/clkenhold/reg_sco/Q 0;
    condition /inst_6/clkenhold/reg_sco/Q 0;
    condition /inst_7/clkenhold/reg_sco/Q 0;
    condition /inst_8/clkenhold/reg_sco/Q 0;
    condition /inst_9/clkenhold/reg_sco/Q 0;
    cycle =
        force clear 0 ;
        force clock[0] 0 ;
        force clock[1] 0 ;
        force clock[2] 0 ;
        force clock[3] 0 ;
        force clock[4] 0 ;
        force clock[5] 0 ;
        force clock[6] 0 ;
        force clock[7] 0 ;
        force clock[8] 0 ;
        force clock[9] 0 ;
        force test_clk 0 ;
        force_pi ;
        measure_po ;
        pulse clock[0] ;
        pulse clock[1] ;
        pulse clock[2] ;
        pulse clock[3] ;
        pulse clock[4] ;
    end;
end; // capture1

```

- If you want to define a specific observe method for each named capture procedure, use the observe\_method statement in the named capture procedure; otherwise, the ATPG engine automatically selects master, slave, or shadow observation.

#### Note

 The [write\\_patterns](#) command enables you to save internal or external clock patterns. Internal clock patterns can be used to simulate the DUT without having the PLL modeled, while the external patterns only exercise the PLL external clocks and control signals. Internal patterns are the default for ASCII and binary formats, and external patterns are the default for tester formats.

- If you generate patterns using a named capture procedure that has both internal and external modes and you save them in STIL or WGL format, you must use the

`write_patterns` command's "internal" option to read them back into the tool (for example, to use in diagnosis). For more information and for information about special considerations that apply to LBIST mode in the TK/LBIST Hybrid flow, refer to the `-Mode_internal` and `-Mode_external` switches for the [write\\_patterns](#) command in the *Tessent Shell Reference Manual*.

DRC rules W20 through W36 check named capture procedures. If a DRC error prevents use of a capture procedure, the run aborts.

## Slow and Load Types in the Cycle Statement

Optionally, you can add a "slow" or a "load" type to the cycle definition.

For example:

```
cycle slow =  
...  
end;
```

- The slow cycle indicates that at-speed faults cannot be launched or captured. The tool must know which at-speed cycles are slow to get accurate at-speed fault coverage simulation numbers; therefore, be sure to include "slow" when defining cycles that are not at-speed cycles in an at-speed capture procedure.

---

### Note



At-speed cycles need to be continuous; that is, a named capture procedure cannot have more than one at-speed clocking subsequence.

---

- The load cycle indicates that the cycle is always preceded by an extra scan load. The first cycle in a named capture procedure is always a load (with or without the load type designation), so you typically apply "load" to subsequent cycles. An at-speed launch cycle can be a load cycle; however, none of the cycles that follow in the at-speed sequence, up to and including the capture cycle, can be load cycles.

---

### Note



To get extra loads, you must enable the tool's multiple load and clock sequential capabilities by issuing the [set\\_pattern\\_type](#) command with "-multiple load on" and "-sequential <2 or greater>". For more information, see "[Multiple Load Patterns](#)" in the *Tessent Scan and ATPG User's Manual*.

---



The following example illustrates the “slow” and “load” attributes:

```
procedure capture multiple_load_example =  
  timeplate tp1;  
  // first cycle is always a load, with or without load type designation  
  cycle slow =  
    force_pi;  
    force wr_enable 1;  
    pulse int_clk1;  
  end;  
  cycle slow load =  
    pulse int_clk1;  
  end;  
  cycle =  
    force re_enable 1;  
    pulse int_clk1; // launch clock  
  end;  
  cycle =  
    pulse int_clk1; // capture clock  
  end;  
end; // end of capture procedure
```

## launch\_capture\_pair Statement

Optionally, you can add one or more “launch\_capture\_pair” statements to the beginning of a named capture procedure. This statement defines legal at-speed launch and capture points in non-adjacent cycles. If you do not use the launch\_capture\_pair statement, the tool launches and capture only in adjacent cycles. If at least one launch and capture clock pair is defined, the launch and capture points are derived from the defined launch and capture clock pairs.

---

### Note



This statement is only supported when using a named capture procedure to perform test generation.

---

The syntax of the launch\_capture\_pair statement is as follows:

```
launch_capture_pair <launch_clock_pin_name> <capture_clock_pin_name>;
```

Where:

- launch\_clock\_pin\_name is the clock used to launch the transition.
- capture\_clock\_pin\_name is the clock used to capture the transition.

The launch clock cycle is used to check the transition condition. The capture clock cycle is used to capture the transition fault effect. The cycles between the launch clock and capture clock must be at-speed cycles. They cannot include any slow cycles between them. The faults to be tested by the named capture procedure with the defined launch and capture clock pair are the faults that can be launched by the launch clock and captured by the capture clock defined in the launch\_capture\_pair statement.

The following is an example of the `launch_capture_pair` statement:

```
procedure capture c1_c1 =  
  launch_capture_pair c1 c1;  
  
  cycle = // cycle 1  
    force_pi;  
    force c1 0;  
    force c2 0;  
    force c3 0;  
    pulse c1;  
  end;  
  
  cycle = // cycle 2  
    pulse c2;  
  end;  
  
  cycle = // cycle 3  
    pulse c1;  
    pulse c3;  
  end;  
end; // end of capture procedure
```

In this example, a valid launch can happen in cycle 1. A valid capture can happen in cycle 3 only with `c1` as the capture clock. A launch in cycle 1 and a capture in cycle 2 is not used for fault detection. The faults to be tested by this named capture procedure are the faults that can be launched and captured by clock `c1`.

## Clock\_sequential (Optional)

The `clock_sequential` procedure (optional for “patterns -scan” context), which may only contain `force_pi`, `pulse_write_clock`, `pulse_read_clock`, `pulse_capture_clock`, `bidi_force_pi`, and `bidi_force_off` event statements, represents the clock sequential events in a clock sequential pattern. Use this procedure with the capture procedure.

The following shows the `clock_sequential` procedure pattern.

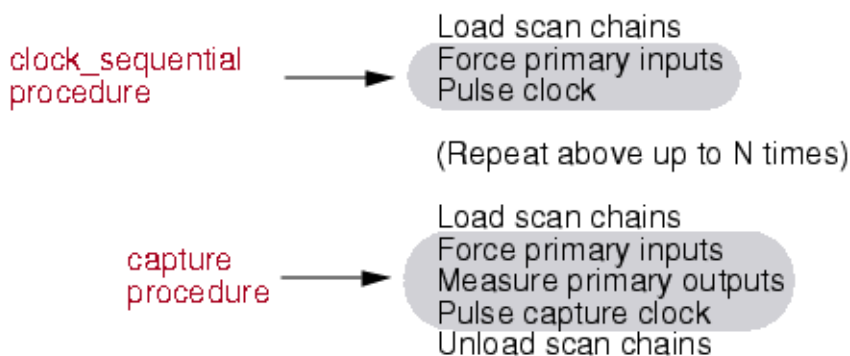
### **Clock\_sequential Procedure Pattern**

---

**Force primary inputs**  
**Pulse write clock**  
**and/or Pulse read clock**  
**and/or Pulse capture clock**

Figure 11-11 shows an entire clock sequential pattern, which illustrates where the `clock_sequential` and capture procedures are used.

**Figure 11-11. Full Clock Sequential Pattern**  
**Clock Sequential Pattern**



## Init\_force (Optional)

The `init_force` procedure (optional for “patterns -scan” context), which may only contain `force_pi` event statements, represents the force cycle that is used in an ATPG pattern that targets a transition fault. The transition must be launched off of the last scan chain shift. This procedure is used when the fault type is set to transition fault and either the depth is set to 2 or less or the ATPG engine fails to find a sequential pattern that can cover this transition fault. Use this procedure with the capture procedure.

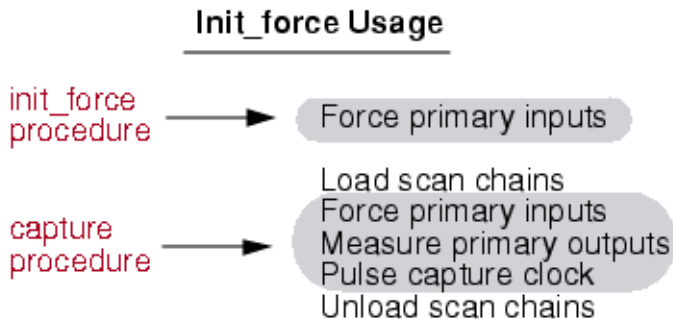
The following illustrates the format of the `init_force` procedure pattern.

### **Init\_force Procedure Pattern**

**Force primary inputs**

Figure 11-12 shows the pattern that uses the `init_force` procedure.

**Figure 11-12. Init\_force Procedure Usage**



## Test\_end (Optional, all ATPG tools)

The optional test\_end procedure is used to add a sequence of events to the end of a test pattern set.

The test\_end procedure may only contain force and pulse event statements (see the following exception), and can only be defined once for all scan groups. When saving patterns, the test\_end procedure is applied to the end of each pattern set saved.

The following shows the general pattern for the test\_end procedure pattern.

### Test\_end Procedure Pattern

---

Force primary inputs  
Pulse clocks

## IJTAG Embedded Instruments

In the ATPG context patterns -scan, IJTAG is valid in the test\_setup and test\_end procedures only. It is invalid in any other procedure, as well as in ATPG's analysis mode. Also, only [iReset](#) and [iCall](#) commands are valid in the test procedures.

For detailed information, see “[How to Set Up Embedded Instruments Through Test Procedures](#)” in the *Tessent IJTAG User's Manual*.

## Example 1: Using test\_end in a Procedure File

The following is a partial example of how the test\_end procedure might be used in a procedure file:

```
timeplate tp1 =  
  force_pi 0;  
  measure_po 10;  
  pulse ref_clk 50 50;  
  period 100;  
end;  
  
procedure test_end =  
  timeplate tp1;  
  cycle =  
    force ctrl_a 1;  
    force tms 0;  
    pulse ref_clk;  
  end;  
end;
```

## Example 2: Test\_end Procedure with Timeplate

The following is an example test\_end procedure with its corresponding timeplate:

```
timeplate tp4 =
    force_pi 0;
    pulse TCK 10 10;
    measure_po 30;
    period 40;
end;

procedure test_end =
    timeplate tp4;
    cycle =
        // TMS = 1, change to select-DR state
        force TDI 1;
        force TMS 1;
        pulse TCK;
    end;

    cycle =
        // TMS = 0, change to capture-DR state

...
    cycle =
        // Scan out signature (MISR has length of 4)
        force TDI 1;
        force TMS 0;
        pulse TCK;
    end;
    cycle =
        force TDI 1;
        force TMS 0;
        pulse TCK ;
    end;
...
end;
```

## Sub\_procedure

The sub\_procedure procedure eliminates the need to insert duplicate actions within a procedure. Once you have defined a sub\_procedure, you can specify this procedure within other procedures using the apply statement.

You can also set the tool to reissue the sub\_procedure as many times as needed by specifying the repeat\_count. Because the repeat\_count is required when using apply sub\_procedure, you must enter a minimum of 1 for this parameter.

## Sub\_procedure Looping

Sub\_procedure looping is used to reduce the size of pattern files. The default behavior of the sub\_procedure is to use “loops” or “repeats” in all applicable pattern formats to repeat the contents of the sub\_procedure N times, where N is greater than 1.

## Disabling Sub\_procedure Looping

If you want the event data in a sub\_procedure to be expanded and represented as N sets of vectors in the pattern file, where N is the number of times the sub\_procedure is applied, use the “[ALL\\_NO\\_LOOP 1](#)” parameter file keyword to disable the use of “loop” or “repeat” statements.

For example, if the test\_setup procedure has the following statement:

```
apply pulse_bclock 1000;
```

The vector data for the sub\_procedure “pulse\_bclock” would be expanded to be 1000 vectors. The default for the ALL\_NO\_LOOP keyword is off (0).

## Sub\_procedure Definition Format

The sub\_procedure definition has the following format.

```
procedure sub_procedure my_subprocedure =  
    timeplate tpl;  
    cycle =  
        force_pi;  
        measure_po;  
    end;  
end;
```

## Using the Sub\_procedure in a Procedure

The following is an example of how to use the sub\_procedure in a procedure.

```
procedure shift =  
    scan_group grp1;  
    timeplate tpl;  
    apply my_subprocedure 4;  
    cycle =  
        force_sci;  
        measure_sco;  
        pulse T;  
    end;  
end;
```

---

### Note



You must first define a sub\_procedure before using it in a procedure. Next, you can apply a sub\_procedure within any procedure type. Also, you cannot use a sub\_procedure within the “cycle =” and “end;” statements.

---

# Additional Support for Test Procedure Files

Tessent Shell provides additional support so that you can use environmental variable, merge, default template capabilities with your test procedure files.

## Tcl Variables Used for Substitution and Conditional Selection

The procfile can include Tcl variables to provide parameterized values within the procfile statements. The Tcl variables must be set in the global Tcl namespace. If you are setting the variables within a proc, make sure to use `::` as a prefix to the variable name such that it is set in the global namespace. For example, use `"set ::my_period 4"` such that `$my_period` exists when processing the proc files.

The Tcl variable can also be used to evaluate the condition of a Tcl if command located inside the procfile. The element of a list of ports in the procfile must be separated by commas. If the port names are escaped identifiers, they must be enclosed in quotation marks. Use the method shown in the following example to convert a list of port names into a quoted comma-separated list needed in the proc file:

Command invoked in the dofile:

```
set ::add_clocks_timing 1
set ::edges "25 75"
set ::port_list [get_name_list [get_clocks -type sync_source]]
set ::all_clocks [string cat {"} [join $port_list {"", "}] {""}]
set _procfile_name ./myproc
```

Given the content of the `./myproc` is:

```
alias all_clocks = $all_clocks;
timeplate mytimeplate =
    if {$add_clocks_timing} {
        pulse all_clocks $edges;
    }
end;
```

The `report_procedures` command displays the following:

```
alias all_clocks = "clk1", "clk2", clk3";
timeplate mytimeplate =
    pulse all_clocks 25 75;
end;
```

## Merging Procedure Files

It is possible to specify more than one procedure file for a design. You can specify a procedure file with the `add_scan_groups` command or with the `read_procfile` command. You need to supply (to the ATPG tool) a minimum set of information in the procedure file with the `add_scan_groups` command. You must supply all event information for the scan procedures.

However, after leaving setup mode, it is possible to specify non-scan procedures, timeplates, and new timing for the scan procedures by reading in an additional procedure file with the `read_procfile` command. Specifying new information for the same design, from more than one procedure file, is known as “merging the procedure files.” To properly merge the information from multiple procedure files, the Vector Interfaces code follows these rules:

- All scan procedures that you use must be specified in the procedure file that you load with the `add_scan_groups` command.
- If you load a procedure that contains nothing but the procedure name, a timeplate name, and an optional scan group, it is a template procedure. If a procedure already exists by that name for that scan group (if it is a group-specific procedure), then the timeplate is mapped onto the existing procedure. If no procedure already exists with that name, the tool stores the template procedure for future use.
- If you load a new complete procedure (not a template) and a procedure already exists by that name for the specified scan group (if applicable), the new procedure overwrites the existing one.
- In both cases, when a procedure overwrites an existing one, or if a new timeplate is mapped to an old procedure, the tool checks the procedures to make sure that the sequence of events in the new procedure does not differ from the old procedure.

### Default Information Provided by the Tool

When you issue the `write_patterns` command, the tool checks to make sure that all procedures and timeplates needed to save the patterns in the specified format are present.

If there are any missing non-scan procedures, the tool creates default procedures and issues a warning.

For any procedures that are created or that do not have a timeplate specified, the default timeplate is mapped to these procedures, if it is set. You can set the default timeplate by using the `set default_timeplate` statement previously described in the “[Set Statement](#)” section. If you use this statement, the timeplate specified when creating default procedures is used. If the default procedure needs to be created and no default timeplate has been set, then the first timeplate specified is used. If no timeplates are specified, a default timeplate is created as well.

## Creating Test Procedure Files for End Measure Mode

You can create test procedure files that enable end measure mode. End measure mode refers to the special handling that the Vector Interfaces code needs to move the measure to the end of the shift and capture cycle.



## Prerequisites

- A test procedure file.

## Procedure

1. Create a new timeplate that measures the outputs after the clock pulse.
2. Change the timeplate for the shift and load\_unload to point to the new timeplate.
3. Add the measure\_sco statement to the load\_unload procedure.
4. Make sure all shift procedures have the measure\_sco statement after the shift clock. When end measure mode is enabled, the measure\_sco statement measures the next value from the output of the scan chain. The very first value for the output of the scan chain is measured by a measure\_sco statement in the load\_unload procedure.
5. Change the timeplate for the capture cycle by breaking it into two cycles. Move the capture clock to the second cycle of the capture procedure to enable the measure at the end. In the first cycle, the force\_pi and measure\_po are performed. In the second cycle, the capture clock is pulsed. When using end measure mode, a measure cannot be performed after the capture clock.

## Examples

```
set time scale 1.000000 ns ;
set strobe_window time 10 ;
timeplate gen_tp1 =
    force_pi 0 ;
    measure_po 10 ;
    pulse clk 20 10;
    pulse edt_clock 20 10;
    pulse ramclk 20 10;
    period 40 ;
end;
// CREATE A NEW TIMEPLATE THAT MEASURES AFTER THE CLOCK PULSE
timeplate gen_tp2 =
    force_pi 0 ;
    //      measure_po 10 ;
    pulse clk 20 10;
    pulse edt_clock 20 10;
    pulse ramclk 20 10;
    measure_po 35 ;          // <== NEW MEASURE STATEMENT
    period 40 ;
end;
// FOR CAPTURE SPLIT INTO TWO CYCLES
procedure capture =
    timeplate gen_tp1 ;
    cycle =
        force_pi ;
        measure_po ;
    end ;
    cycle =
        pulse_capture_clock ;
    end;
end;
// FOR THE SHIFT AND LOAD_UNLOAD, USE THE NEW TIMEPLATE
procedure shift =
    scan_group grp1 ;
    timeplate gen_tp2 ; //<=== NEW TIMEPLATE
    cycle =
        force_sci ;
        force edt_update 0 ;
        measure_sco ;
        pulse clk ;
        pulse edt_clock ;
    end;
end;
// ADD A MEASURE_SCO TO THE LOAD_UNLOAD PROCEDURE
procedure load_unload =
    scan_group grp1 ;
    timeplate gen_tp2 ; //<=== NEW TIMEPLATE
    cycle =
        force clk 0 ;
        force edt_bypass 0 ;
        force edt_clock 0 ;
        force edt_update 1 ;
        force ramclk 0 ;
        force scan_en 1 ;
        pulse edt_clock ;
    measure_sco;          //<=== NEW MEASURE STATEMENT
```

```
    end ;  
    apply shift 39;  
end;  
procedure test_setup =  
    timeplate gen_tp1 ;  
    cycle =  
        force edt_clock 0 ;  
    end;  
end;
```

# Serial Register Load and Unload for LogicBIST and ATPG

---

You can use the `test_setup` or `test_end` test procedure file procedures to serially load or unload control registers in the circuit under test. Additionally, this section discusses the commands you must add to your dofile to use this functionality.

Serial register load and unload targets the following flows:

- LogicBIST — Used to serially unload certain registers (for example, MISRs) using the `test_end` procedure.
- Low pin count test — Used to serially load registers with test information through a serial access port such as a JTAG TAP controller.

|                                                       |            |
|-------------------------------------------------------|------------|
| <b>Register Load and Unload Use Models .....</b>      | <b>840</b> |
| <b>Static Versus Dynamic Register Variables .....</b> | <b>840</b> |
| <b>Test Procedure File Modifications .....</b>        | <b>841</b> |
| <b>Dofile Modifications.....</b>                      | <b>844</b> |
| <b>Serial Load and Unload DRC Rules .....</b>         | <b>847</b> |

## Register Load and Unload Use Models

Using serial register load and unload enables you to identify a load/unload register variable value in the test procedure file and define this variable, either static or dynamic, using commands in your Tessent dofile. By using this functionality, you can load control registers by using a type of load procedure where the data value for the register can be passed as a string of values, and the load register uses a shift mechanism to show how this string of data values is shifted into the register.

To serially load/unload register value variables, you must make modifications to your test procedure file and your Tessent dofile.

The register value variable is used in the test procedure file to load the value into a register through the data input pin, or unload a value from a register through the data output pin. The load/unload procedures support pin inversions, but you must explicitly specify this.

The registers to be loaded/unloaded can be cascaded such that one load or unload operation loads or unloads multiple values, for example PRPG/MISR values.

## Static Versus Dynamic Register Variables

You can define both static and dynamic register value variables.

- Static variable — A specific register value variable string you specify during setup mode. Once set, you cannot change this variable.
- Dynamic variable — A register value variable string that you can define or change later, and can use tool-specific switches.

## Static Variable

A static register variable is one where the value is assigned to the variable when the variable is defined (using the [add\\_register\\_value](#)), and this value then cannot be changed. This static value is used by DRC when simulating the procedures. A static variable cannot be set using tool specific switches to link the variable to tool computed values, as these may change.

## Dynamic Variable

A dynamic register value variable uses tool-specific switches and arguments to link the variable to a value computed by the tool. If you use a dynamic variable, then you must specify the register's width unless the tool knows this value at the time DRC is run (for example, PRPG size).

This method is used for defining a variable that has a tool-specific computed value that is available when patterns are saved and needs to be loaded or unloaded by the test\_setup or test\_end procedures. The value defaults to all X bits, and you can specify the actual value in non-Setup mode by using the set\_register\_value command.

If no value is specified the first time DRC is invoked after adding a register value variable, the tool considers the variable to be dynamic for DRC and any future invocation of DRC.

# Test Procedure File Modifications

In the test procedure file, the load\_unload\_registers Procedure and shift Keyword are used.

## load\_unload\_registers Procedure

The load\_unload\_registers procedure is a named procedure; consequently, you can have multiple occurrences of this procedure, corresponding to multiple groups of registers that need to be loaded or unloaded. The load\_unload\_registers procedure can only be called from the following procedures:

- test\_setup
- test\_end

The load\_unload\_registers procedure can load, unload, or both. Additionally, one application of the procedure can load/unload multiple cascaded registers.

When the test procedure file explicitly calls the `load_unload_registers` procedure from either the `test_setup` or `test_end` procedures, the values to load or unload are passed to the procedure using the string of binary values (value hard-coded in the procedure application) or the register value variables.

## shift Keyword

The `load_unload_registers` procedure uses the `shift` keyword to define a block of events that result in one shift operation. This is similar to the IEEE STIL syntax where the `shift` keyword defines a shift block within a load or unload procedure. It is analogous to embedding the shift procedure into the `load_unload` procedure.

## Apply Statement Extension

The `apply` statement in the procedure file syntax is extended to accept a statement that associates a set of string values or register value variables with a particular input pin, output pin, or alias. This pin or alias must then be used within the shift statement and have a “#” character associated with it. This character is a placeholder for the values that is loaded or unloaded using the string of values passed to the procedure, or the internally generated values associated with the value name.

## Test Procedure File Syntax

The following illustrates using the `load_unload_registers` procedure and `shift` keyword:

```
procedure load_unload_registers procedure_name =  
    ...  
    [ cycle blocks ]  
    shift =  
        cycle blocks  
    end ;  
    [ cycle blocks ]  
end ;
```

The `load_unload_registers` procedure must have a shift block defined, which has one or more cycles used to shift the data into the data input pin or the data out of the data output pin. The procedure can also optionally have cycles that precede or follow the shift block, to be used to put the circuit into shift mode or finish the shift mode when done, similar to how a `load_unload` procedure is used.

## Event Statements

Within a shift block, at least one event statements in a `load_unload_registers` procedure must use the “#” character to denote where the shift data passed into the procedure is used.

The event statement must be either a “force” event or an “expect” event. An event statement with the “#” character can also occur in the cycles preceding or following the “shift” block to

express pre-shifts and post-shifts for loading the register. This event statement has the following syntax:

```
<force | expect > pin_or_alias_name # ;
```

The apply statement used to call the sub\_procedure also calls the shift and load\_unload\_registers procedures. However, when calling the shift and load\_unload\_registers procedures, the #times argument in the apply statement is replaced with one or more value assignments for the data in or data out pins.

```
apply procedure_name [ #times | shift_data_assignment  
[ , shift_data_assignment ...] ] ;
```

A shift\_data\_assignment has the following syntax:

```
identifier = < value_string | register_value_variable >  
[<value_string | register_value_variable>...]
```

where identifier is either a pin name or an alias name.

The identifier used in the shift\_data\_assignment must match an identifier used within the procedure in one of the event statements with the “#” character.

## Register Value Strings

A register value string (value\_string) is one of the following:

- A binary string of 0s, 1s, or Xs, where the length of the string determines the number of shifts to load the register.
- A string of a different radix as long as the Verilog syntax of identifying the radix and width are used, such as “32’h” for a 32 bit hexadecimal value.

If a register\_value\_variable is used in the shift\_data\_assignment, then a value computed by the tool for that register\_value\_variable (as bound within the dofile) or hard-coded in the dofile is loaded or unloaded at that time and the shift length is also provided by the tool. It is possible for more than one value\_string or register\_value\_variable to be assigned to an identifier in one shift\_data\_assignment. In this case, the extra values are separated by spaces. This enables multiple shorter values to be shifted into one register group.

The typical usage is that each register\_value\_variable corresponds to one register being loaded, and the specification of multiple variables is used when those registers are cascaded and loaded/unload on after another.

## Alias Names

It is possible to use an alias name in the procedure type for loading shift data, even if that alias name refers to multiple pins. If this is the case, the number of bits assigned by the “#” character

for each shift is equal to the width of the alias being used. The length of the value string being passed to the procedure must be a multiple of the width of the alias.

Loading or unloading an alias can be used if performing parallel load/unload and no shift is required. For example, if each MISR bit is connected to a separate primary output. Even if no shifting is required, the functionality is still useful to bind the expected values to the signature computed by the tool.

If multiple `shift_data_assignments` are passed to a procedure, then all of them must have the same shift length such that each pin being loaded/unloaded requires the same number of cycles to load/unload all the data. No padding is performed by the tool.

If a “measure” event is used in one of these procedures to unload a register, then these measure values can be compared in the final patterns and the Verilog testbench.

## Dofile Modifications

You define register value variables using commands you issue to the tool interactively or in a dofile. The value variables are subsequently referenced in the procedure file.

You use the following commands to perform these operations:

- `add_register_value`
- `set_register_value`
- `delete_register_value`
- `report_register_value`

### Addition of a Register Value Variable

The `add_register_value` command defines the register value variables. You can define the variables as either static value variables or dynamic value variables.

You can only use this command in setup mode. You must define the register value variables in the dofile before the variables are used in a test procedure file to load values into the registers.

### Static Value Variables

You specify a static value variable by stating a specific value string. This value variable then has this value string as a constant value.

You must specify this value in setup mode; the value is considered to be static by default, and the value is present when simulating procedures in DRC.

```
add_register_value value_name {value_string [-Radix {Binary | Decimal |  
Octal | hexadecimal} -Width integer]} optional_arguments
```



The *value\_name* is a user-specified identifier, and the *value\_string* is a state string in a particular radix. The default is binary radix.

If the `-Radix` switch is used to change this to a different radix, then a register width must also be specified using the `-Width` switch.

## Dynamic Value Variables

You specify a dynamic value variable using the following variation of the command:

```
add_register_value value_name value_string -Dynamic -Width integer  
optional_arguments
```

The *value\_string* is optional and defaults to all X bits if not specified.

You can subsequently enter the actual value once you have exited setup mode using the [set\\_register\\_value](#) command. If no value is specified the first time DRC is invoked after adding a register value variable, it is considered as dynamic in this and any future invocation of DRC.

## Data Pin Inversions

When using either a static or dynamic variable, you can optionally specify inversions on the input and output data pins by using the `-INput_pin_inversion` and `-OUtput_pin_inversion` switches, respectively.

These are optional switches you use to specify that the data value in this variable should be inverted before it is loaded into the register through the `load_unload_register` procedure.

## Definition of a Dynamic Register Value Variable

You can define a dynamic register value variable by using tool-specific switches and arguments to link the variable to a value computed by the tool. These variables are dynamic, and the width must be specified unless the width is known to the tool at the time DRC is run.

This method is used for defining a variable that has a tool-specific computed value that is available when patterns are saved and needs to be loaded or unloaded by the `test_setup` or `test_end` procedures.

In this case, you use the `add_register_value` command with the `-Width` switch and omit the value while in setup mode. Once you have exited setup mode, you can use the [set\\_register\\_value](#) command to set the variable:

```
set_register_value value_name value_string [-RADix {BInary | HEx | OCtal |  
Decimal}]
```

The name of the register value and its width are known prior to parsing the procedure file and running DRCs. The value is all X bits for DRC.

This command can only be used for a register value that was defined without a value string, and the width of the value string must match the width specified in the [add\\_register\\_value](#) command.

If the value overflows the width specified, an error is issued.

By default, the bits extracted by force/measure “#” in the procedure are from MSB to LSB. If the value specified should be shifted in/out in the opposite order, with the LSB bits applied/measured first, use the “-LSB\_shifted\_first” optional switch.

## Deletion of a Register Value Variable

The [delete\\_register\\_value](#) command deletes all or a specified register value variable. You can only use this command in setup mode.

## Register Value Variable Reports

The [report\\_register\\_value](#) command provides a detailed report of the register value variables to stdout or, optionally, a file.

## Serial Load and Unload DRC Rules

The P13 and P54, P66, and W5 DRC rules are applied to the procedures and syntax in the section “Procedure Examples.”

The P1 “syntax error” message is used to catch many potential issues, such as specifying a “#times” value for applying a `load_unload_registers` procedure instead of the `shift_data_assignment`.

|                                 |            |
|---------------------------------|------------|
| <b>P13 and P54</b> .....        | <b>847</b> |
| <b>P66</b> .....                | <b>847</b> |
| <b>W5</b> .....                 | <b>848</b> |
| <b>Procedure Examples</b> ..... | <b>849</b> |

### P13 and P54

The P13 and P54 rules are used to check the shift assignment statements when calling a `load_unload_registers` procedure.

If the shift assignment uses a value string that is not properly formatted or contains illegal characters for that radix, then a P13 is issued.

```
Error: Invalid state value state string. (P13)
```

If the shift assignment references an undefined register value variable name, a P54 is issued.

```
Error: Undefined identifier identifier_string referenced by signal_name.  
(P54)
```

### P66

The P66 rule is used to check for missing statements within a `load_unload_registers` procedure.

For example, if no Shift block is specified, or if an event statement using the “#” character is not present for each `shift_assignment` passed to the `load_unload_registers` procedure, then a P66 is issued.

```
Error: Procedure procedure_name is missing required statement_string  
statement. (P66)
```

The following examples illustrate the types of P66 DRC errors you could encounter.

## Example 1

In the following example, the tool issues this error if there is no shift block within the load\_unload\_registers procedure:

```
Error: Procedure procedure_name is missing required shift statement.  
(P66)
```

## Example 2

In the following example, the tool issues this error if there are no events in the load\_unload\_register procedure that use the “#” substitute character.

```
Error: Procedure procedure_name is missing required substitute event  
statement. (P66)
```

## Example 3

In the following example, the tool issues this error if an apply statement that uses a load\_unload\_registers procedure has a shift data assignment that uses a signal that does not appear in the load\_unload\_registers procedure with the substitute character “#”.

```
Error: Procedure procedure_name is missing event using shift assign  
signal_name statement. (P66)
```

## W5

The W5 DRC error is used to flag any extra events or statements in a load\_unload\_registers procedure, or any events that are not legal.

For example, if an event type other than Force or Expect is used with the “#” substitute character, then a W05 rule is issued. If the load\_unload\_registers procedure contains more than one shift block, this rule is issued. If there is a Force or Expect statement using a “#” character for a signal name that is not being passed to the procedure as a shift assignment, this rule is issued. The W05 rule is used when shift assignments for a particular Apply statement do not all have the same length.

```
Error: Procedure procedure_name has an illegal event statement or event  
order (event_statement_string) (W5)
```

## Example 1

The following error is issued if an event in the load\_unload\_register procedure uses the “#” substitute character, but no shift data for this signal is passed into the procedure when it is called “extra shift block”:

```
<force | measure> signal_name # without matching shift assign data
```

## Example 2

The following error is issued if there is more than one shift block in the load\_unload\_registers procedure.

```
"extra shift block"
```

## Example 3

The following error is issued if more than one event of the same type in the shift block uses the same signal name with the “#” substitute character.

```
"too many events using shift assign signal_name"
```

## Example 4

The following error is issued for an apply statement that uses a load\_unload\_registers procedure but has no shift data assignments in the apply statement.

```
"apply with no shift data assignment"
```

## Example 5

The following error is issued for an apply statement that uses a load\_unload\_registers procedure and has more than one shift assignment, however, the target of the shift assignments are aliases and they are not the same width.

```
"unmatched shift assign signal width"
```

## Example 6

This is issued for an apply statement that uses a load\_unload\_registers procedure and has more than one shift assignment and the assignments have different shift lengths.

```
"unmatched shift lengths"
```

## Procedure Examples

The definition of the load\_unload\_registers procedure would look as follows. Notice how this procedure uses the “shift=” statement and the “force tdi #” statement to denote the shifting and where the string of data is applied, one bit at a time.

```
procedure load_unload_registers load_prpg1 =  
    timeplate tp1 ;  
    cycle =  
        force prpg_select 1 ;  
        ... // setup tap controller to load prpg  
    end;  
    shift =  
        cycle =  
            force tdi # ;  
            pulse tck ;  
        end;  
    end;  
end;
```

This procedure could then be applied in the test\_setup procedure to initialize the PRPG, specifying the string of bits to apply to “tdi” during shifting.

```
procedure test_setup =  
    timeplate tp1 ;  
    cycle =  
        force clk1 0 ;  
        force clk2 1 ;  
        force tck 0 ;  
        ...  
    end;  
    apply load_prpg1 tdi = 000000000000000000000001 ;  
    cycle =  
        ...  
    end;  
end;
```

For loading a register with the shift length during test\_setup, using the values computed by the tool, the procedure definition would look the same, but how the procedure is called is slightly different. The following example both loads and unloads the register, as this same procedure could be used in a test\_end procedure to unload the shift length value. The dofile for this example contains the following command:

**add\_register\_value length\_val -shift\_length -width 16**

The procedure file would contain the following:

```

procedure load_unload_registers load_unload_length =
    timeplate tp1 ;
    cycle =
        force clk1 0 ;
        ...
    end;
    shift =
        cycle =
            force tdi # ;
            expect tdo # ;
            pulse tck ;
        end;
    end;
end;
procedure test_setup =
    timeplate tp1 ;
    cycle =
        ...
    end;
    apply load_unload_length tdi = length_val, tdo = XXXXXXXXXXXXXXXXXX;
    cycle =
        ...
    end;
end;

```

This next example uses an alias to group three tdi signals into one alias, and also adds a post shift cycle to the load\_unload\_registers procedure. When this procedure is called, the data passed to it is consumed three bits at a time, with each bit being shifted into tdi1, tdi2, and tdi3 in order. The total number of shifts applied in the “shift” block is the total length of the value string divided by three, and then minus one for the post shift. Each shift consumes three bits, and the final cycle adds a post shift that assigns the last three bits. The length of the value string being passed to this procedure must be a multiple of three.

```

alias TDI_GRP = tdi1, tdi2, tdi3 ;
procedure load_unload_registers load_reg1 =
    timeplate tp1 ;
    cycle =
        force clk1 0 ;
        ...
    end;
    shift =
        cycle =
            force TDI_GRP # ;
            pulse tck ;
        end;
    end;
    cycle =
        force TDI_GRP # ;
        pulse tck;
    end;
end;

```

This final example shows how to use a dofile commands to set up a user-defined register value that is used to store total number of scan patterns when the final patterns are saved.

```
add_register_value scan_pat_count -pattern_count scan_test -width 24
```



# Notes About Using the stil2tessent Tool

You can use the stil2tessent tool to create a dofile and procedure file. The stil2tessent tool reads a STIL Procedure File (SPF) and then creates a dofile and test procedure file.

The dofile defines clocks, scan chains, scan groups, and pin constraints. The test procedure file contains a timeplate and the following standard scan procedures: test\_setup, load\_unload, and shift. For more information about this tool, refer to the [stil2tessent](#) command description in the *Tessent Shell Reference Manual*.

|                                                                      |            |
|----------------------------------------------------------------------|------------|
| <b>Extraction of Strobe Timing Information From STIL (SPF) .....</b> | <b>853</b> |
| <b>The STIL ClockStructures Block.....</b>                           | <b>853</b> |

## Extraction of Strobe Timing Information From STIL (SPF)

In the STIL WaveformTable, strobe window and edge strobe are indicated by which event characters are used for the measure timing. Using the ‘H’ and ‘L’ characters indicates that the measure is an edge strobe, while using the ‘h’ and ‘l’ character indicates a window strobe with the length of the strobe window being determined by the following X event in the event list for that WaveformCharacter.

If the window strobe events (‘h’ or ‘l’) are used for a measure event, then the strobe window is calculated taking into account the strobe window indicated by these events. The shortest strobe window calculated is the one used for the procedure file.

If “-edge\_strobe\_processing on” is specified to the tool, and the edge strobe events (‘H’ or ‘L’) are used for a measure event, then the strobe window is set to 0 in the procedure file to indicate edge strobe timing.

If the “-edge\_strobe\_processing” option is not used or is set to off, and edge strobe events (‘H’ and ‘L’) are used for a measure event, then the existing behavior is maintained where the strobe window is calculated based on the next force event or the period of the Waveform table. The strobe window is not set to 0.

## The STIL ClockStructures Block

When parsing STIL Procedure Files (SPF), stil2tessent recognizes the Synopsys-defined ClockStructures block and uses this information to create clock\_control definitions. The Latency statement within the ClockStructures block controls the “set\_external\_capture\_options” statement in the generated dofile.

# Test Procedure File Commands and Output Formats

The test procedure file is supported by a set of commands and a set of output formats.

## Test Procedure File Tool Commands

The following table provides a summary of the tool commands that support the use of test procedure files. For a detailed description of each command, refer to the corresponding command reference page in the *Tessent Shell Reference Manual*.

**Table 11-2. Procedure File Tool Command Summary**

| Command                           | Description                                                                                                                                 |
|-----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">add_scan_groups</a>   | Adds a scan group using the scan procedures in the named procedure file.                                                                    |
| <a href="#">read_procfile</a>     | Reads a new procedure file in non-setup mode. Merges new procedure and timing data with existing data loaded from previous procedure files. |
| <a href="#">write_procfile</a>    | Writes out existing procedure and timing data as the named procedure file.                                                                  |
| <a href="#">write_patterns</a>    | Loads a cycle before saving patterns and merges the new data with the existing data.                                                        |
| <a href="#">report_procedures</a> | Reports (displays) a named procedure to the screen. The -All switch displays all procedures to the screen.                                  |
| <a href="#">report_timeplates</a> | Reports (displays) a named timeplate to the screen. The -All switch displays all timeplates to the screen.                                  |

## Test Procedure File Output Formats

The test procedure file format supports the following output formats:

- Fujitsu TDL
- Mitsubishi TDL
- STIL
- TI TDL
- WGL
- TSTL2
- VERILOG

The -PROcfile switch causes the write\_patterns command to get its timing information from the procedure file. For more information, refer to the [write\\_patterns](#) command in the *Tessent Shell Reference Manual*.



# Chapter 12

## Tessent Visualizer

---

Tessent Visualizer is a graphical user interface for Tessent Shell. This chapter describes the various features of Tessent Visualizer and how to work with those features. With Tessent Visualizer, you can use schematics, browsers, tables, and other reports to analyze and understand DFT issues. By cross-referencing between these different viewpoints of your data and performing various analyses on them, you can more quickly arrive at solutions to those issues.

|                                                            |            |
|------------------------------------------------------------|------------|
| <b>Invoking Tessent Visualizer</b> .....                   | <b>859</b> |
| Invoke Explicitly on the Same Machine. ....                | 859        |
| Invoke Explicitly on a Different Machine .....             | 860        |
| Invoke Implicitly. ....                                    | 861        |
| License-Protected Tabs. ....                               | 862        |
| <b>Framework Overview</b> .....                            | <b>863</b> |
| <b>Tessent Visualizer Components and Preferences</b> ..... | <b>866</b> |
| Tables .....                                               | 867        |
| Schematics .....                                           | 876        |
| Tessent Visualizer Preferences .....                       | 909        |
| Macros .....                                               | 910        |
| Tooltips .....                                             | 911        |
| Saving and Restoring the Session State .....               | 912        |
| Window Title Prefixes .....                                | 913        |
| <b>Tessent Visualizer GUI Reference</b> .....              | <b>915</b> |
| Hierarchical Schematic .....                               | 915        |
| Flat Schematic. ....                                       | 921        |
| ICL Schematic .....                                        | 923        |
| Instance Browser. ....                                     | 929        |
| ICL Instance Browser .....                                 | 933        |
| Cell Library Browser .....                                 | 934        |
| DRC Browser .....                                          | 935        |
| Config Data Browser .....                                  | 938        |
| Transcript .....                                           | 942        |
| Wave Viewer .....                                          | 944        |
| Text/HDL Viewer .....                                      | 947        |
| IJTAG Network Viewer .....                                 | 949        |
| iProc Viewer .....                                         | 952        |
| Diagnosis Report Viewer .....                              | 953        |

---

|                                                              |                            |
|--------------------------------------------------------------|----------------------------|
| <b>Search Features .....</b>                                 | <b><a href="#">955</a></b> |
| <b>Using Tessent Visualizer to Debug Design Issues .....</b> | <b><a href="#">959</a></b> |
| <b>Accessing Tutorials for Tessent Visualizer .....</b>      | <b><a href="#">963</a></b> |
| <b>Accessing Videos for Tessent Visualizer .....</b>         | <b><a href="#">964</a></b> |
| <b>Keyboard Shortcuts.....</b>                               | <b><a href="#">965</a></b> |

# Invoking Tessent Visualizer

Tessent Visualizer is a client-server application, that is, Tessent Shell is the server and Tessent Visualizer is a client of that server.

Therefore, you can invoke Tessent Visualizer in different ways, depending on the task you need to perform. Tessent Visualizer can be launched directly from the Tessent Shell application or remotely from another machine or a different terminal on the same machine.

Irrespective of how you invoke the Tessent Visualizer GUI, two separate processes run and communicate with each other using the TCP/IP protocol. If you use Tessent Shell commands to invoke the tool, the details of the TCP/IP protocol such as hostname, port, and the one-time authorization code are automatically passed to the client. If you choose to launch the GUI directly in the console, you must pass the TCP/IP protocol details using the optional command-line switches or provide them in the Tessent Visualizer GUI dialog box.

Use any of the following methods to invoke the Tessent Visualizer GUI:

|                                                       |            |
|-------------------------------------------------------|------------|
| <b>Invoke Explicitly on the Same Machine</b> .....    | <b>859</b> |
| <b>Invoke Explicitly on a Different Machine</b> ..... | <b>860</b> |
| <b>Invoke Implicitly</b> .....                        | <b>861</b> |
| <b>License-Protected Tabs</b> .....                   | <b>862</b> |

## Invoke Explicitly on the Same Machine

Use the `open_visualizer` command to run the Tessent Visualizer GUI on the same machine as the Tessent Shell. Typically, this is how you run the tool unless you have restrictions on running GUI applications on the same machine as the Tessent Shell.

### Prerequisites

- Comply with the hardware and operating systems requirements listed under “[Supported Hardware and Operating Systems](#)” in the *Managing Tessent Software* manual.

The requirements also apply to the machine providing the X Window desktop (as defined by the `DISPLAY` environment variable).

- Set the `DISPLAY` environment variable correctly.

### Procedure

1. Set a context using the `set_context` command.
2. Load a design and library using the `read_verilog` and `read_cell_library` commands. You can also load an ICL model using the `read_icl` command.
3. Set the current design using the `set_current_design` command.

### Note

 You can invoke Tessent Visualizer without running these commands, but most of the user interface is disabled until you load a design and library, or an ICL model, of your design. To debug certain JTAG issues, loading and extracting an ICL file is sufficient to enable you to examine the ICL network.

You can also use these commands in the Transcript tab after you invoke Tessent Visualizer.

---

4. Invoke Tessent Visualizer explicitly on the same machine Tessent Shell is running:

```
ANALYSIS> open_visualizer
// Note: Tessent Visualizer client successfully started and
connected to the server.
```

## Invoke Explicitly on a Different Machine

In certain cases, it is preferable to run the Tessent Visualizer GUI client and the Tessent Shell server on separate machines. For example, if the Tessent Visualizer GUI is not backward-compatible with an older variant SLES machine, use different machines to run the Tessent Visualizer client and the Tessent Shell server after ensuring the TCP/IP protocol is not blocked between these two machines.

### Prerequisites

- Comply with the hardware and operating systems requirements listed under “[Supported Hardware and Operating Systems](#)” in the *Managing Tessent Software* manual.

The requirements also apply to the machine providing the X Window desktop (as defined by the DISPLAY environment variable).

- Set the DISPLAY environment variable correctly.

### Procedure

1. Set a context using the [set\\_context](#) command.
2. Load a design and library using the [read\\_verilog](#) and [read\\_cell\\_library](#) commands. You can also load an ICL model using the [read\\_icl](#) command.
3. Set the current design using the [set\\_current\\_design](#) command.



---

**Note**

---

 You can invoke Tessent Visualizer without running these commands, but most of the user interface is disabled until you load a design and library, or an ICL model, of your design. To debug certain IJTAG issues, loading and extracting an ICL file is sufficient to enable you to examine the ICL network.

You can also use these commands in the Transcript tab after you invoke Tessent Visualizer.

---

#### 4. Invoke Tessent Visualizer explicitly on a different machine.

To start the server in Tessent Shell, invoke the `open_visualizer` command with the `-server_only` switch. Optionally, specify the TCP/IP port using the `-tcp_port` switch. Valid port numbers are from 1024 to 65535.

```
ANALYSIS> open_visualizer -server_only
// Server: started
//   - Hostname:          server01.example.com
//   - TCP/IP port:      43527
//   - Authorization code: 821703
//   Client: not connected
// Note: To connect Tessent Visualizer client to the server, issue
// the following command in a Linux console:
// -----
// <path>/tessent -visualizer -server server01.example.com \
//   -tcp_port 43527 -authorization_code 821703
// -----
```

To start the client, use the “`tessent -visualizer`” command:

```
$TESSENT_HOME/bin/tessent -visualizer
```

This opens a remote connection dialog box that prompts for the hostname, port, and authorization code.

The six-digit authorization code is a one-time password that is cleared upon successful connection to a server or after five invalid attempts. To reconnect to the same server, generate a new authorization code by invoking `open_visualizer` on that server with the `-authorization_code` switch.

For more information, see the “`tessent`” command description in the *Tessent Shell Reference Manual*.

---

**Note**

---

 The IP address and port of the server must be accessible from the client machine.

---

## Invoke Implicitly

Some Tessent Shell commands run Tessent Visualizer to perform their functions. Therefore, when you run any of these commands they launch the Tessent Visualizer GUI.

## Prerequisites

- Comply with the hardware and operating systems requirements listed under “[Supported Hardware and Operating Systems](#)” in the *Managing Tessent Software* manual.

The requirements also apply to the machine providing the X Window desktop (as defined by the DISPLAY environment variable).

- Set the DISPLAY environment variable correctly.

## Procedure

1. Set a context using the [set\\_context](#) command.
2. Load a design and library using the [read\\_verilog](#) and [read\\_cell\\_library](#) commands. You can also load an ICL model using the [read\\_icl](#) command.
3. Set the current design using the [set\\_current\\_design](#) command.

---

### Note



You can invoke Tessent Visualizer without running these commands, but most of the user interface is disabled until you load a design and library, or an ICL model, of your design. To debug certain IJTAG issues, loading and extracting an ICL file is sufficient to enable you to examine the ICL network.

You can also use these commands in the Transcript tab after you invoke Tessent Visualizer.

---

4. Invoke Tessent Visualizer implicitly using one of the commands listed in “[open\\_visualizer](#)” in the *Tessent Shell Reference Manual*.

The following example opens the Flat Schematic tab in a Tessent Visualizer window with the location of the instance with the C7-6 DRC violation highlighted:

```
ANALYSIS> analyze_drc_violation C7-6
```

## License-Protected Tabs

Some tabs in the Tessent Visualizer window require a Tessent IJTAG (mtijtagf) license. These tabs are referred to as license-protected tabs.

The following tabs are license-protected:

- [ICL Instance Browser](#)
- [ICL Schematic](#)
- [IJTAG Network Viewer](#)
- [iProc Viewer](#)
- Search - ICL Instances

- Search - ICL Pins
- Search - ICL Aliases
- Search - iProcs

---

**Note**

---

 When starting the Tessent Visualizer GUI, the tool tries to check out a Tessent IJTAG license to restore any licensed tabs from the previous session. If the tool displays a message stating that a required Tessent IJTAG license is unavailable, it could mean that you selected “Do not show this dialog again” and clicked **OK** in a previous session when a dialog box informed you that some tabs to be restored require a Tessent IJTAG license. Use the “[open\\_visualizer -display](#)” command and specify any non-license protected tab to start the GUI. For example, you can use the “`open_visualizer -display instance_browser`” command to start the GUI.

---

## Framework Overview

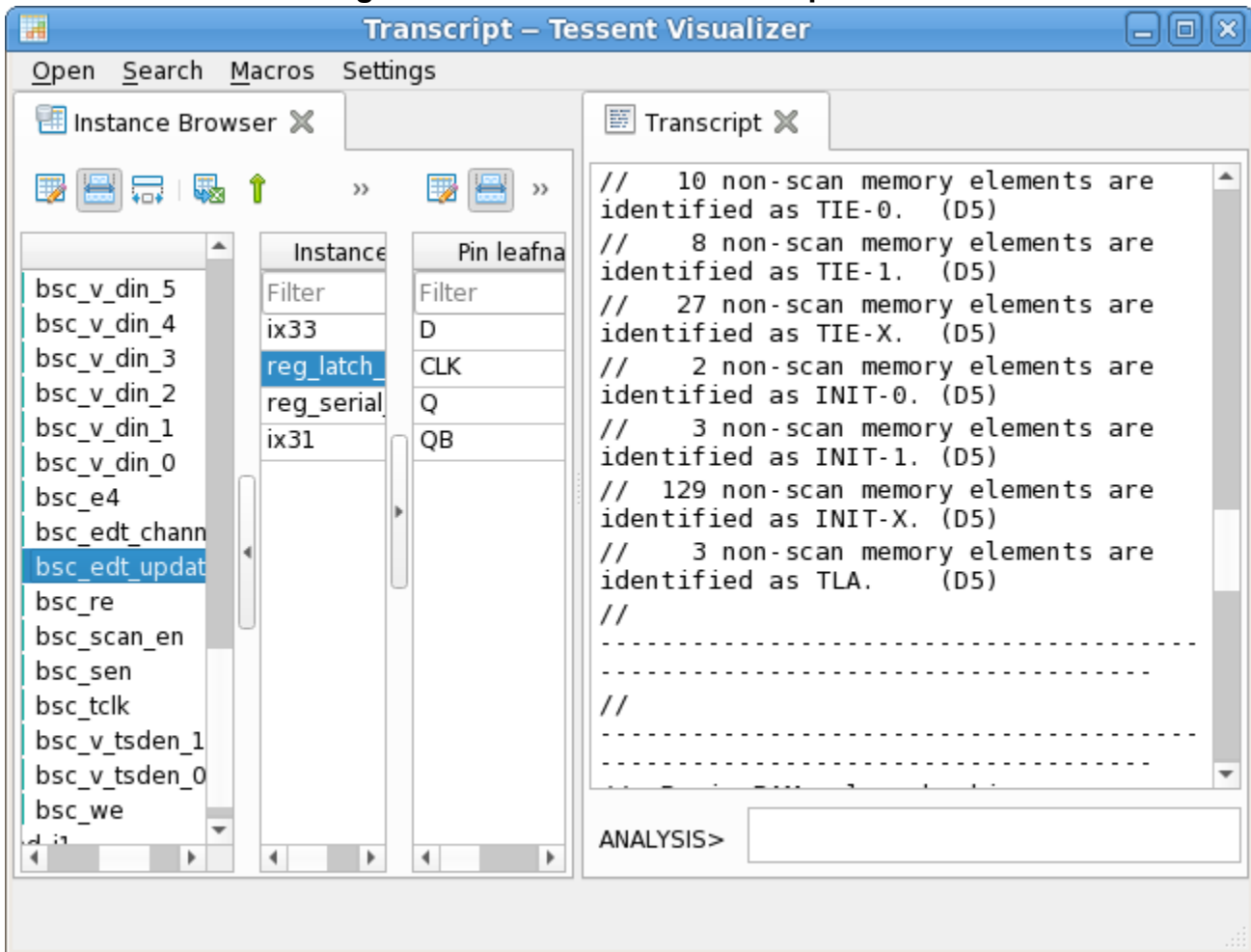
Tessent Visualizer provides multiple tabs for viewing and debugging design and simulation data in specialized windows.

The Tessent Visualizer framework consists of the following:

- **Windows** — Windows contain one or two panes, a global top menu, and a status bar. The menu and status bar are identical across multiple windows; only the pane content differs.
- **Panes** — There are at most two panes per window. By default, one pane provides multiple tabs. Windows can be split into two panes by dragging and dropping. Each pane has its own tabs.
- **Tabs** — You can view and select multiple tabs within a pane. Each functional task in Tessent Visualizer is placed as a tab. You can reorder the tabs by dragging and dropping, or cycle through them with Ctrl+Tab (next tab) or Ctrl+Shift+Tab (previous tab). See “[Keyboard Shortcuts](#)” on page 965 for more ways to interact with Tessent Visualizer using the keyboard. See “[License-Protected Tabs](#)” on page 862 for information about how the tool handles license-protected tabs.

Each window can contain two panes (separate sets of tabs). This is called a split view. To create a split, drag one of the tabs to either side. Drag and drop a tab to move it from one side of the split to the other. There are at most two work areas (panes) per window, as shown in [Figure 12-1](#).

**Figure 12-1. Tessent Visualizer Split View**



You can undock any tab from a window as a separate (floating) window and redock as necessary. Floating windows have the original window's full functionality, with access to features such as global settings and the capability to offer a split view. To undock a tab, double-click the tab or drag and drop it away from the parent window. To redock, drag it back to the parent window (or any other Tessent Visualizer window) and drop it as a tab in that window.

You can move a tab to the left or right pane with the context menu, as shown in the following figure.

**Figure 12-2. Moving a Tab to the Left or Right Pane**



# Tessent Visualizer Components and Preferences

---

Tables and schematics represent different views of pins, instances, nets, and other design objects, as well as actions associated with those design objects. For example, a hierarchical pin shown on a hierarchical schematic, the instance browser, or a search window can also be shown on a flat schematic.

|                                                     |            |
|-----------------------------------------------------|------------|
| <b>Tables .....</b>                                 | <b>867</b> |
| <b>Schematics .....</b>                             | <b>876</b> |
| <b>Tessent Visualizer Preferences .....</b>         | <b>909</b> |
| <b>Macros.....</b>                                  | <b>910</b> |
| <b>Tooltips .....</b>                               | <b>911</b> |
| <b>Saving and Restoring the Session State .....</b> | <b>912</b> |
| <b>Window Title Prefixes .....</b>                  | <b>913</b> |

---

## Tables

---

Tessent Visualizer presents data such as tracing results, pin listings, instances, and nets in tabular form in all windows, including schematics. These tables are configurable, and data can be filtered dynamically.

|                                         |            |
|-----------------------------------------|------------|
| <b>Table Toolbar Features</b> .....     | <b>868</b> |
| <b>Columns and Filters Editor</b> ..... | <b>870</b> |
| <b>Table Filters</b> .....              | <b>871</b> |
| <b>Data Sorting</b> .....               | <b>874</b> |
| <b>Row Highlighting</b> .....           | <b>875</b> |

## Table Toolbar Features

All tables used in Tessent Visualizer have several common elements, including toolbars, dynamic headers, filter fields, and data rows and cells.

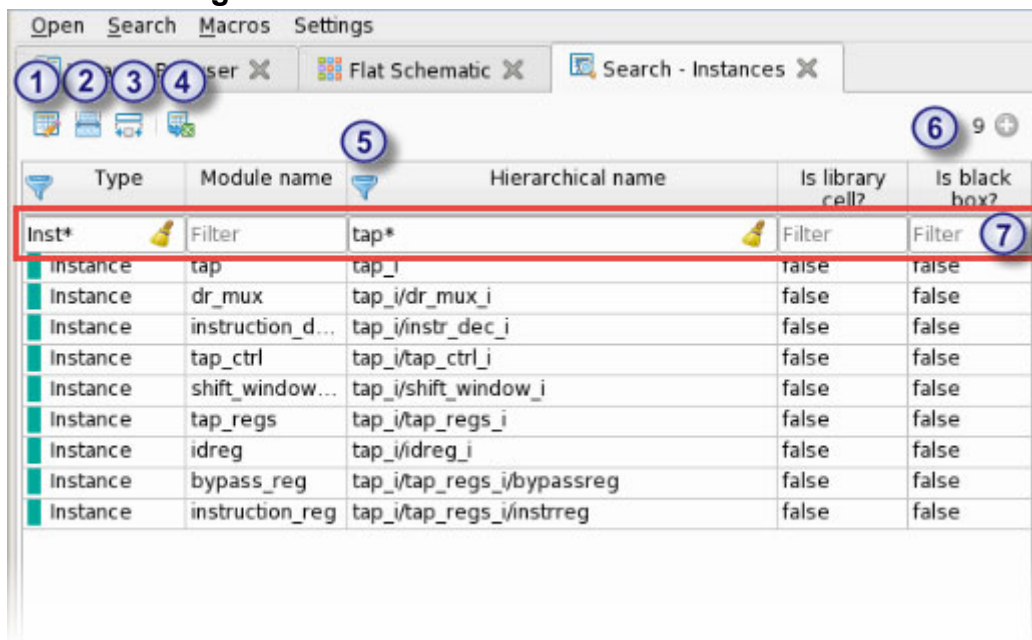
### Description

Most report tables in Tessent Visualizer use a spreadsheet-like format with customizable columns, a toolbar, and filter fields for each column. The following are additional features common to tables:

- Customize the data types included in the table with the Columns and Filters Editor.
- Select one or more table rows in the Tessent Visualizer window and perform various compatible actions using the popup menus available by right-clicking.
- Export table data in CSV format with the Export Table icon.
- Add line numbers to tables from the Preferences dialog box.

Figure 12-3 illustrates the locations of these items.

**Figure 12-3. Common Table Toolbar Features**



### Objects

| Object | Description                                                                    |
|--------|--------------------------------------------------------------------------------|
| 1      | <b>Columns and Filters Editor.</b> Controls the form and content of the table. |



| Object | Description                                                                                                                                                                                                                                                                                                                                                                                                    |
|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ②      | <b>Automatic/Manual Column Width.</b> Toggles between automatic and manual mode for controlling column widths and column header wrap. In automatic mode, column widths stretch to fit the table and column headers wrap if necessary to accommodate narrow data.                                                                                                                                               |
| ③      | <b>Resize columns to contents.</b> This fits the column widths to accommodate the size of the displayed data.                                                                                                                                                                                                                                                                                                  |
| ④      | <b>Export table</b> in CSV format. Alternately, use Ctrl+S.                                                                                                                                                                                                                                                                                                                                                    |
| ⑤      | <b>Funnel.</b> If present, this indicates that filtering is active. This symbol is not displayed unless a filter is applied.                                                                                                                                                                                                                                                                                   |
| ⑥      | <b>Loaded row counter.</b> By default, the table loads up to 250 items. Change the default threshold from the Preferences dialog box.<br><br>The “+” next to this number indicates that all data for the table has been loaded if it is displayed in a gray circle. A green circle indicates that additional data exists in Tessent Shell. Load this additional data by scrolling down or clicking the circle. |
| ⑦      | <b>Filter fields.</b> These control which data are displayed in the table.                                                                                                                                                                                                                                                                                                                                     |

## Related Topics

[Columns and Filters Editor](#)

[Tessent Visualizer Preferences](#)

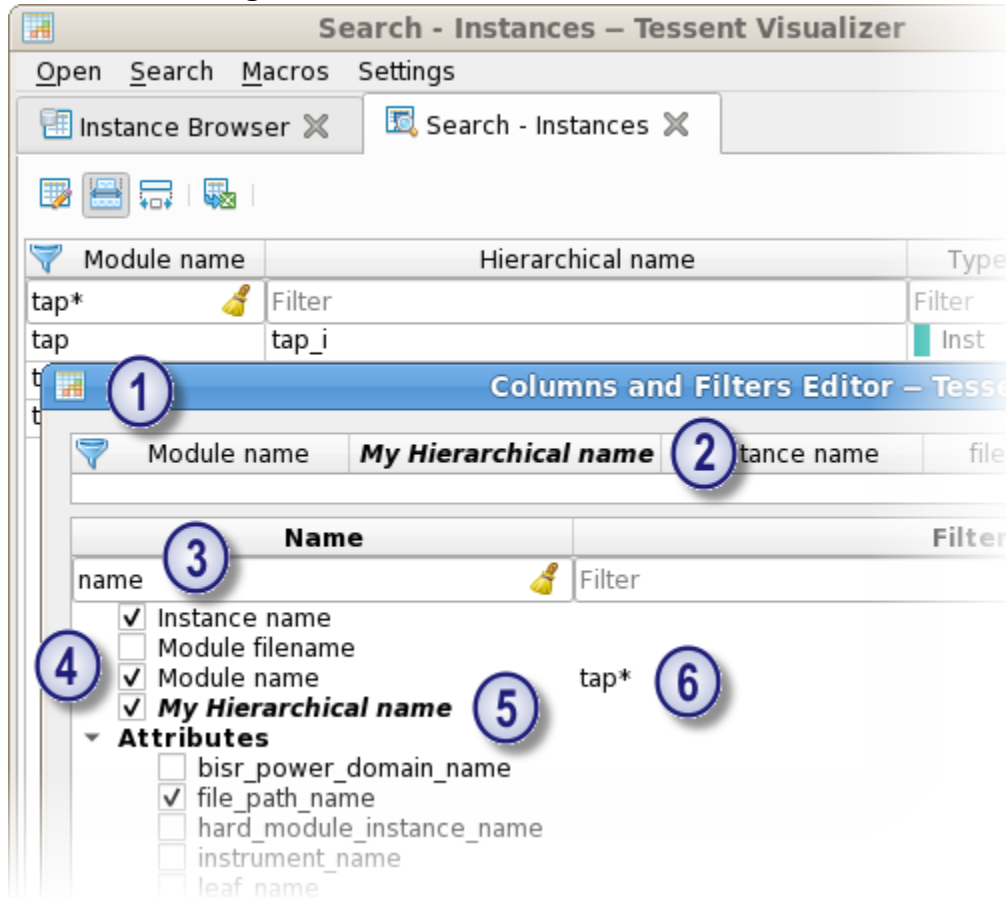
## Columns and Filters Editor

To access: click the “Columns and Filters Editor” icon in the Tessent Visualizer toolbar.  
Use the Columns and Filters Editor to control the content and format of tables.

### Description

Figure 12-4 shows features of the Columns and Filters Editor:

**Figure 12-4. Columns and Filters Editor**



### Objects

| Object | Description                                                                                                                                                                                                                                                                  |
|--------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ①      | Apply filters by typing terms in the filter fields, shown in Figure 12-4. When a column is being filtered, the funnel icon is displayed next to the column name in both the Column and Filters Editor and the tabbed window being modified by the Column and Filters Editor. |
| ②      | Reorder columns by dragging and dropping the header items at the top of the Columns and Filters Editor.                                                                                                                                                                      |

| Object | Description                                                                                                                                                                                         |
|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ③      | Search for specific data types by using string matching in the column for the property name. Search results are displayed as you type.                                                              |
| ④      | Add or remove table columns by checking or unchecking the boxes associated with column names.                                                                                                       |
| ⑤      | Customize column label names by double-clicking the name representing the column. The name change is indicated by a bold italic font in the column and filters editor as well as the results table. |
| ⑥      | Add or edit column filters by double-clicking the filter cell.                                                                                                                                      |

## Usage Notes

The report table content is automatically refreshed when you accept the modifications by clicking **OK**.

## Table Filters

Filters limit the data displayed in tables using the search criteria you specify.

The filter field is located directly beneath the column headers, as shown in [Figure 12-3](#) on page 868. Tessent Visualizer uses the following filters:

- **Wildcard match** — The default filtering mechanism used in table filters is wildcard matching. If you do not provide any other keywords or comparators, the tool uses wildcard matching for the table filters.

Wildcard matching uses the following special characters:

- The asterisk (\*) symbol matches any number of characters
- The question mark (?) symbol matches a single character

The tool uses adaptive formatting if the filter text does not contain any special keyword such as, NOT, GLOB, LIKE, or REGEXP, or any comparison or logical operator. The tool wraps the text within single quotation marks (") when the focus is out of the filter field.

If the tool detects Tcl-escaped text, that is, text within braces ({} ) or double braces ({{{}}), it replaces the braces with single quotation marks. For example, `{{{core_i}}}` results in `"{{core_i}}"`. Consequently, if you want to input text that is enclosed within braces, wrap it with single quotation marks. Wildcard matching does not work within an escaped Verilog identifier.

By default, wildcard matching is case-insensitive. However, you can also specify the “Case sensitive filtering” option for wildcard matching. You can set this option in the

global [Tessent Visualizer Preferences](#) dialog box. All the string literals specified with operators or keywords such as NOT, GLOB, LIKE, or REGEXP must be enclosed within single quotation marks.

- **Exact match** — A string literal or value, prepended with an equal sign (=). This restricts the display of data to those objects matching the string or value.
- **Exact inverse match** — A string literal or value prepended with the not equals sign (!=). This restricts the display of data to those objects not matching the string or value.
- **Numerical comparison operators** — Data columns that include numerical information can be filtered using standard numerical comparison operators (<, <=, >, >=).
- **Logical operators** — AND, OR, and NOT. For example:
  - != abc AND != xyz restricts the display of data to all objects that are not named abc and are not named xyz.
  - abc OR xyz restricts the display of data to those objects named abc or xyz.
  - abc\* OR x?z restricts the display of data to objects matching both wildcard expressions.
  - NOT operator can be used to negate the GLOB, LIKE, or REGEXP expression.
- **GLOB and LIKE** — Table filters support SQL's GLOB and LIKE operators.

GLOB syntax:

- GLOB expressions are always case-sensitive.
- The asterisk (\*) character matches any number of characters.
- The question mark (?) character matches exactly one character.
- The set operator ([]) matches one character from the list of characters enclosed within square brackets. To match wildcard characters or an opening square bracket '[', specify them with an extra set operator. For example, GLOB 'pinname[[]\*]' matches 'pinname[1]', 'pinname[10]', and 'pinname[]'.

LIKE syntax:

- LIKE expressions are always case-insensitive.
  - The percent sign (%) matches any number of characters.
  - The underscore (\_) character matches any single character.
  - To escape special characters, use the backslash (\). For example, LIKE 'core\\_i' matches 'core\_i', 'CORE\_i', or any other case-insensitive combination.
- **Standard regular expressions** — Data can be filtered using regular expressions by using the construct REGEXP 'RE'. For example, REGEXP 'clk\\_d+\$' restricts the

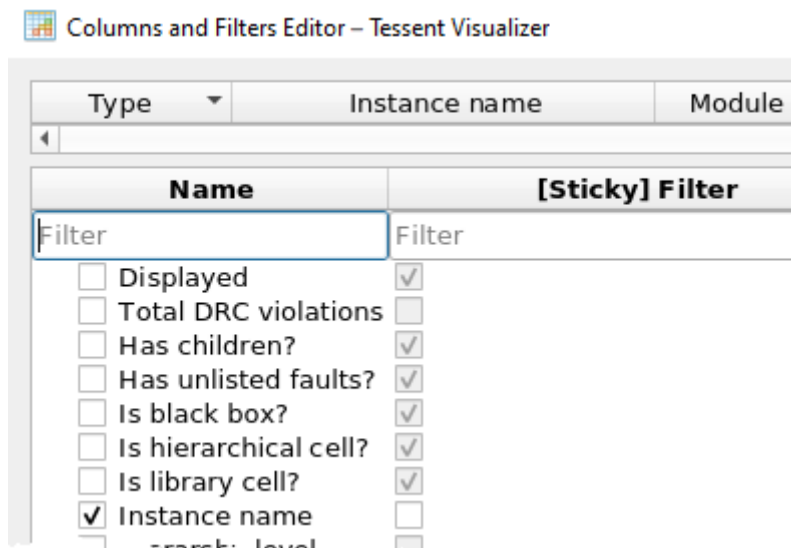
display of data to all objects with names ending in the string `clk_` followed by one or more decimal digits.

**Note**

 In this filter, the keyword “REGEXP” and the single quotation marks are required.

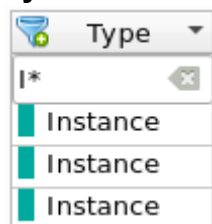
You can make a filter for a hierarchical data structure “sticky” with the [Columns and Filters Editor](#) as shown in [Figure 12-5](#). Select the checkbox to enable the sticky behavior for the corresponding column. If the sticky behavior is enabled, a filter defined for one hierarchy level is applied for all other hierarchy levels. If it is not enabled, the filter applies only to the hierarchical level where it is defined.

**Figure 12-5. Sticky Column**



The sticky filter is also marked in the Hierarchical and ICL Instance Browser with a plus sign (+) on the standard filter icon as shown in [Figure 12-6](#).

**Figure 12-6. Sticky Indicator in Column Header**



This feature is only available for the Instance Browser and ICL Instance Browser’s child instance tables displaying hierarchical data.

**Table 12-1. Filtering Summary**

| Filter Type           | Filter Format                                                                   | Examples                                                                                             |
|-----------------------|---------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| Exact match           | <i>value</i><br><i>=value</i>                                                   | <b>abc</b><br><b>=xyz</b><br><b>=12</b>                                                              |
| Exact match (inverse) | <b>!=value</b>                                                                  | <b>!=abc</b>                                                                                         |
| Numerical operators   | <<br><=<br>><br>>=                                                              | <b>&gt;10</b><br><b>&gt;20 AND &lt;= 50</b>                                                          |
| Wildcards             | GLOB ' <i>expression</i> '                                                      | GLOB ' <b>abc*</b> '<br>GLOB ' <b>*xyz*</b> '<br>GLOB ' <b>abc?xyz</b> '<br>GLOB ' <b>abcdef??</b> ' |
| Regular expression    | REGEXP ' <i>expression</i> '                                                    | REGEXP ' <b>^abc.*</b> '<br>REGEXP ' <b>.*_\d0\$</b> '                                               |
| Logical operators     | AND<br>OR<br>NOT GLOB ' <i>expression</i> '<br>NOT REGEXP ' <i>expression</i> ' | <b>abc OR xyz</b><br><b>!=abc AND != xyz</b><br><b>NOT GLOB 'abc*'</b><br><b>NOT REGEXP '^abc.*'</b> |

**Note**

 Filters applied to table columns are processed in Tessent Shell on the complete data model, no matter how many resulting rows appear in the Tessent Visualizer tabular reports.

## Data Sorting

The default order of rows in tabular data is determined by the underlying data structure in Tessent Shell. In some tables, you can sort this data by clicking the column title cell. The ordering cycles through each of ascending, descending, and the default ordering.

Data sorting is enabled when the number of rows loaded is less than the limit (250 by default, but configurable in the **Preferences** menu). Exceptions: sorting is always enabled in the following cases:

- A table with child instances in the **Instance Browser**.
- A table with modules in the **Cell Library Browser**.

---

**Tip**

 For better performance, limit the set of visible data with filtering before using the sorting function.

---

## Related Topics

[Tessent Visualizer Preferences](#)

## Row Highlighting

Tables in Tessent Visualizer use color coding to indicate the status of rows.

**Figure 12-7. Selection Colors**

| Type ^ | Instance name | Module name        | Displayed |
|--------|---------------|--------------------|-----------|
| Filter | Filter        | Filter             | Filter    |
| Inst   | tap_i         | tap                | false     |
| Inst   | core_i        | test_design_edt... | false     |
| Inst   | bsr_i1        | bsr_instance_1     | true      |
| Inst   | pad_i1        | pad_instance_1     | false     |

- **Dark blue** — Selected objects
- **Light blue** — Active cell
- **Gray** — Objects currently displayed in the schematic

Most popup menus operate on the selected object (dark blue). Copy and paste actions (Ctrl+C and Ctrl+V) work only on the active cell (light blue). The gray highlighting shows which objects are currently displayed in a schematic.

Use Shift+click to select multiple adjacent objects in a table and Ctrl+click to select multiple nonadjacent objects.

You can also use a Boolean value in the “Displayed” column for easy filtering of objects displayed in a schematic.

## Schematics

Tessent Visualizer schematics contain features that enable you to inspect and navigate your design.

The figure in the “Toolbar” section shows a Flat Schematic with the following GUI objects highlighted:

- **Toolbar** — Controls how objects are displayed in the schematic and export schematic objects for use by other tools.
- **Address Bar** — Displays the name of the currently selected object. You can also use the address bar to add new objects to the schematic by name.
- **Selection Buttons for Context Table** — Select which context table you want to display.

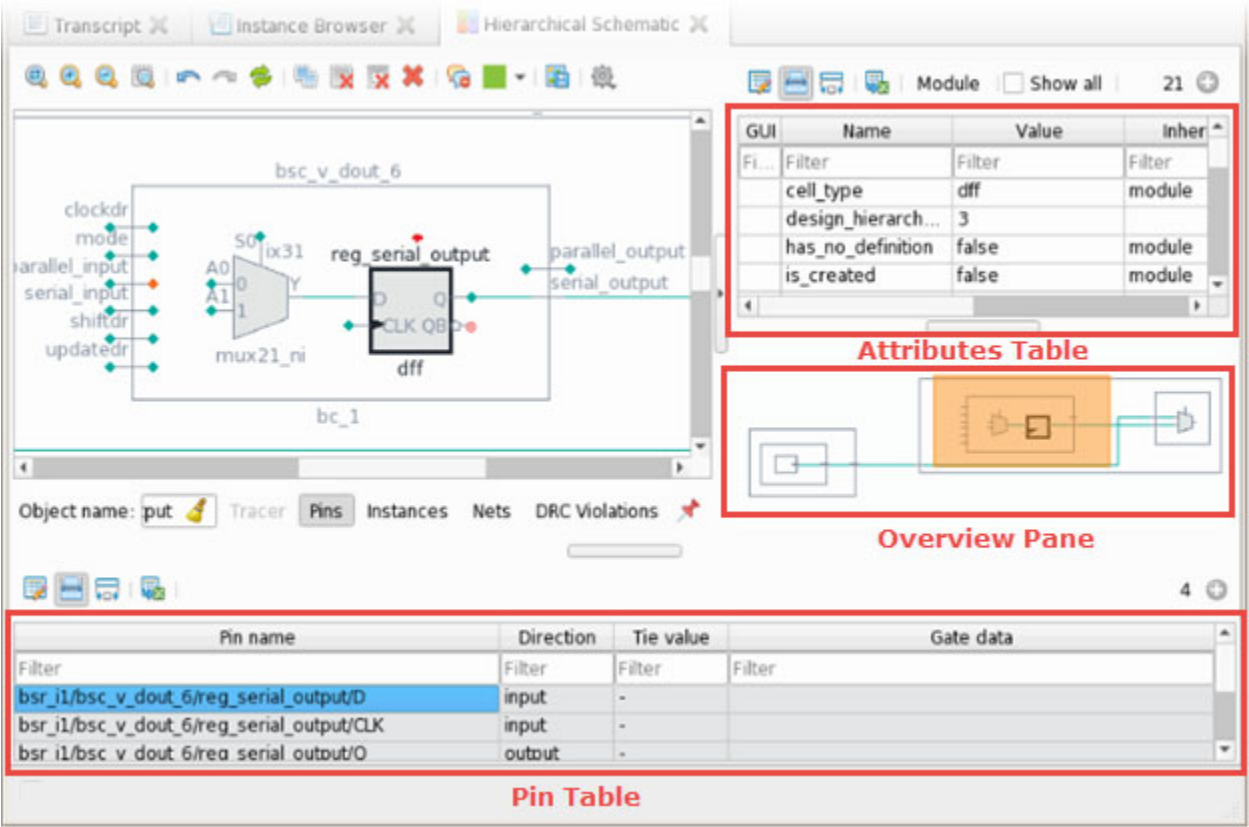
Figure 12-8 illustrates some additional schematic common features:

- **Attributes Table for the Selected Object** — Displays information about the currently selected object. The specific types of data in this table depend on the object selected.
- **Overview Pane** — Shows all objects that have been added to the schematic. The currently zoomed view is highlighted. Drag and drop the highlight rectangle to pan the view.
- **Pin Table** — Displays information about the pins on the currently selected object.

Many objects can be transferred from one schematic to another by dragging and dropping or by right-clicking and using the context menu. Any highlighting on these objects is preserved when they appear in the target schematic. If you transfer an object into a schematic where it already exists, any highlighting on the new copy of the object overrides any highlighting on the previous one. If you transfer multiple objects that include connectivity obtained with signal tracing from the Flat Schematic to the Hierarchical Schematic, or from the IJTAG Network viewer to the ICL Schematic, that connectivity is preserved if you choose “Transfer connectivity” from the schematic settings menu. This option is enabled by default.



Figure 12-8. Schematic With Overview Pane and Attributes Table



|                                     |     |
|-------------------------------------|-----|
| Toolbar .....                       | 877 |
| Schematic Symbols .....             | 880 |
| Context Tables .....                | 892 |
| Signal Net Tracing Strategies ..... | 902 |
| Context Menu Tracing .....          | 905 |
| Displayed Property .....            | 905 |
| Tessent Shell Attributes.....       | 906 |
| Mouse Gestures .....                | 907 |

Toolbar

There is a toolbar at the top of each Tessent Visualizer schematic.

Figure 12-9 shows the location of the schematic toolbar.

**Figure 12-9. Schematic Elements: Toolbar, Address Bar, and Selection Buttons**

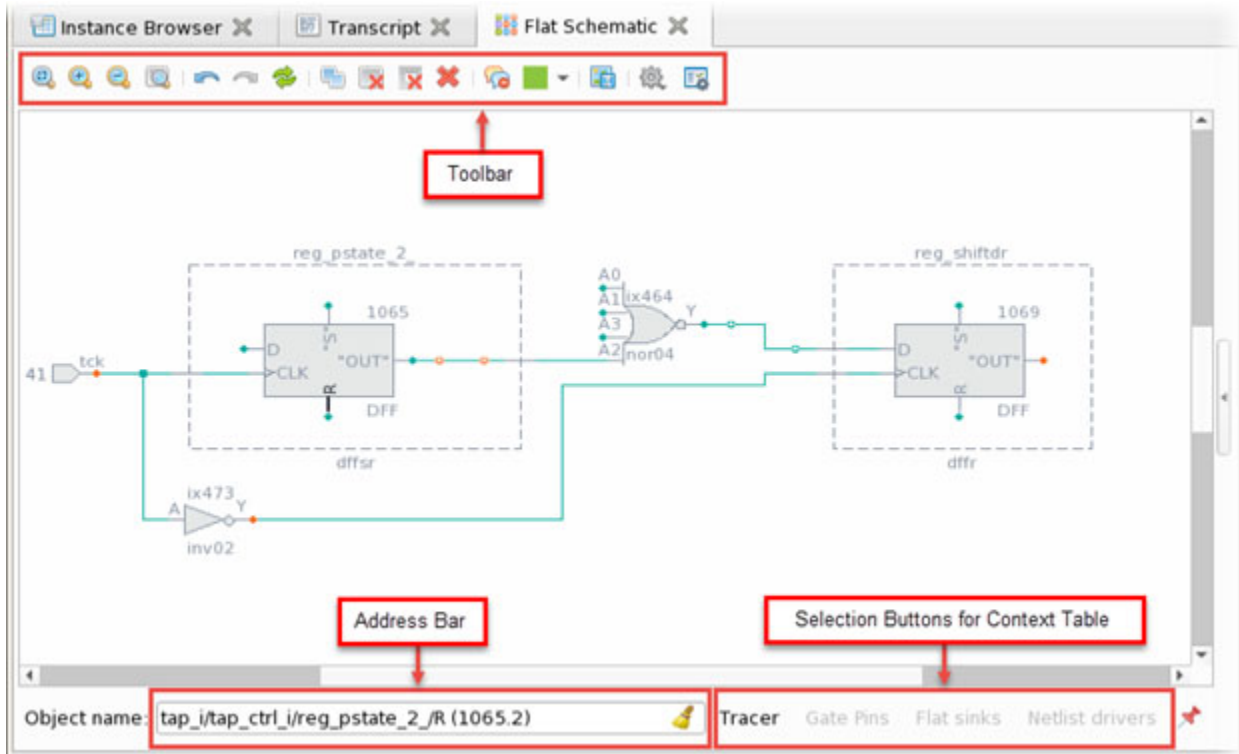


Table 12-2 summarizes the contents of the toolbar.

**Table 12-2. Schematic Toolbar Actions**

| Tool                                                                                | Description                                                                 |
|-------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
|  | <b>Zoom all:</b> fit the current view of the schematic in the window.       |
|  | <b>Zoom in.</b>                                                             |
|  | <b>Zoom out.</b>                                                            |
|  | <b>Zoom selected:</b> zoom in on and center the selected object or objects. |
|  | <b>Undo:</b> undo the previous action.                                      |
|  | <b>Redo:</b> redo the previous action.                                      |
|  | <b>Redraw:</b> refresh the schematic view.                                  |
|  | <b>Select all:</b> select all objects in the schematic view.                |

Table 12-2. Schematic Toolbar Actions (cont.)

| Tool                                                                                | Description                                                                                                                                                                                                                                                                                                                                                                                   |
|-------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|    | <b>Delete selected:</b> delete all selected objects from the schematic view.                                                                                                                                                                                                                                                                                                                  |
|    | <b>Delete unselected:</b> delete all objects from the schematic view except those currently selected. There are two exceptions: <ul style="list-style-type: none"><li>• If a selected object is an instance that has some pins displayed, those pins are not deleted.</li><li>• If the selected objects are pins that are connected with a net, that net connection is not deleted.</li></ul> |
|    | <b>Delete all:</b> delete all objects from the schematic view.                                                                                                                                                                                                                                                                                                                                |
|    | <b>Collapse callouts:</b> collapse all callout messages to their icon representations, or expand if currently collapsed.                                                                                                                                                                                                                                                                      |
|    | <b>Mark/Unmark:</b> highlight the currently selected object or objects with the selected color, or clear the highlighting on the selected object. If the color you want to use is already active, you can use Ctrl+M to highlight the selected object(s).                                                                                                                                     |
|   | <b>Export schematic:</b> save the current view of the schematic in PDF or PostScript format, or as a schematic dofile. You can run the generated schematic dofile (or any other dofile) using the <b>Execute dofile</b> option under the <b>Open</b> menu, or by using the <b>dofile</b> command in the Transcript or shell window.                                                           |
|  | <b>Schematic options:</b> set schematic-specific options (different schematic types have different options in this submenu).                                                                                                                                                                                                                                                                  |
|  | <b>Open Gate Report Settings dialog:</b> provides access to a subset of the <a href="#">set_gate_report</a> command options. Different options are active depending on the data models available in Tessent Shell.                                                                                                                                                                            |

**Note**

 When a dofile generated with the **Export schematic** action is run in a future session, the displayed schematic is re-created as closely as possible. In rare circumstances, some objects may not be displayed or connected exactly as in the original schematic.

## Schematic Symbols

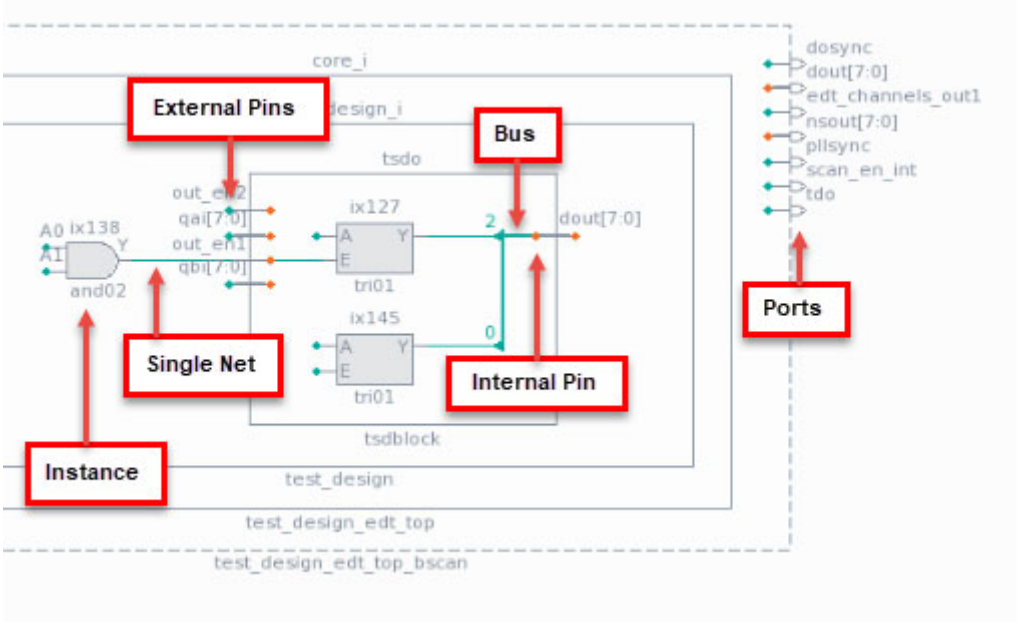
Tessent Visualizer schematics use a variety of conventional symbols to present the structure, objects, and connections of a design.

|                                              |            |
|----------------------------------------------|------------|
| <b>Object Types in Schematics</b> .....      | <b>880</b> |
| <b>Markers</b> .....                         | <b>886</b> |
| <b>Buffer and Inverter Collapsing</b> .....  | <b>889</b> |
| <b>Folding and Unfolding Instances</b> ..... | <b>891</b> |

## Object Types in Schematics

The object types displayed in schematics are instances, pins, ports, and nets. You interact with these objects to explore your design, obtain information about the objects’ properties, trace nets forward and backward, and so on.

**Figure 12-10. Hierarchical Schematic Showing Pins, Ports, Instances, and Nets**



## Instances

Instances are individual copies of library cells, primitives, or groups of cells connected by nets to form hierarchical instances. Both the **Hierarchical Schematic** and **Flat Schematic** tabs can contain instances of library cells and primitives, which display as conventional logic symbols (such as NAND, NOR, and MUX). Library cells display as rectangles in the **Hierarchical Schematic** tab, but you can see their functional representation in the **Flat Schematic** tab.

**Note**

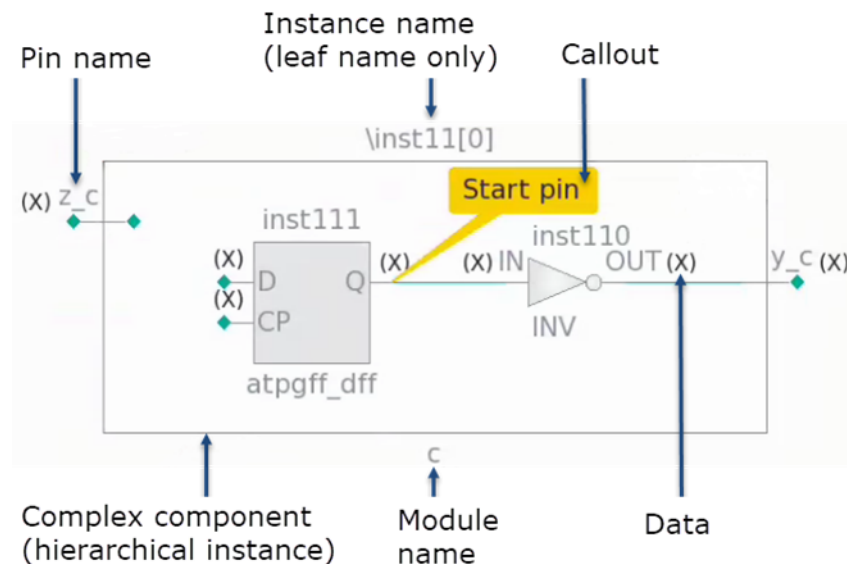
The ordering of pins on displayed instances matches the ordering as provided by report\_gates.

**Hierarchical Instances**

Tessent Visualizer schematics show groupings of symbols by enclosing them in rectangles, objects called hierarchical instances. Hierarchical instances include their pins. All instances except those at the leaf level display with both the internal and external connections to those pins.

Figure 12-11 depicts an instance in the **Hierarchical Schematic** tab. Callouts convey information about specific design objects in certain contexts, and the schematic shows simulation data on design pins where appropriate.

**Figure 12-11. Hierarchical Instance With Callout and Pin Data**



Use the right mouse button as shown in Figure 12-12 to show the internal connectivity of a selected hierarchical instance. If the number of objects at the selected instance's interface is large, this option is not available. In this case, use context tables to selectively add instances and connections to the schematic.

**Figure 12-12. Showing the Internal Connectivity of a Hierarchical Instance**

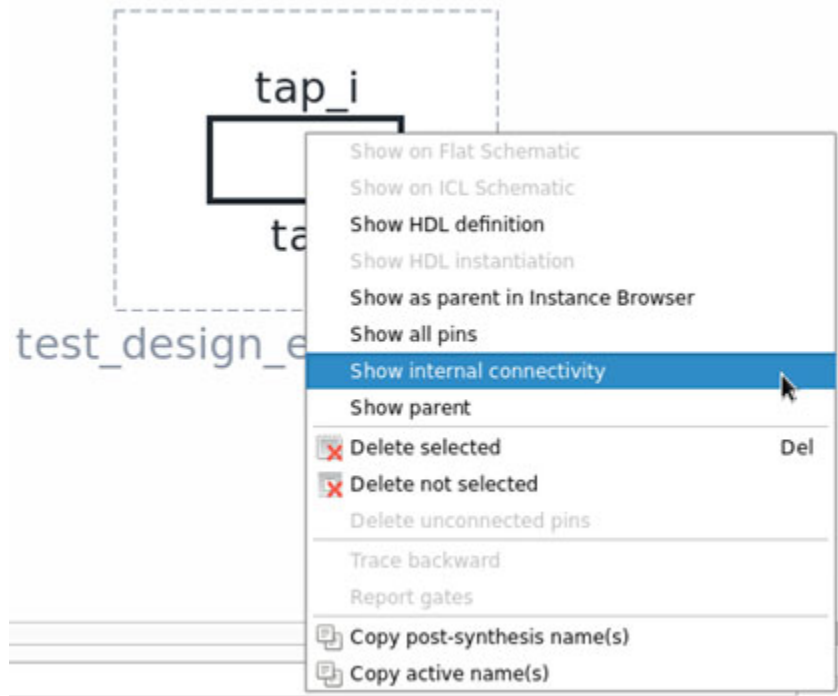
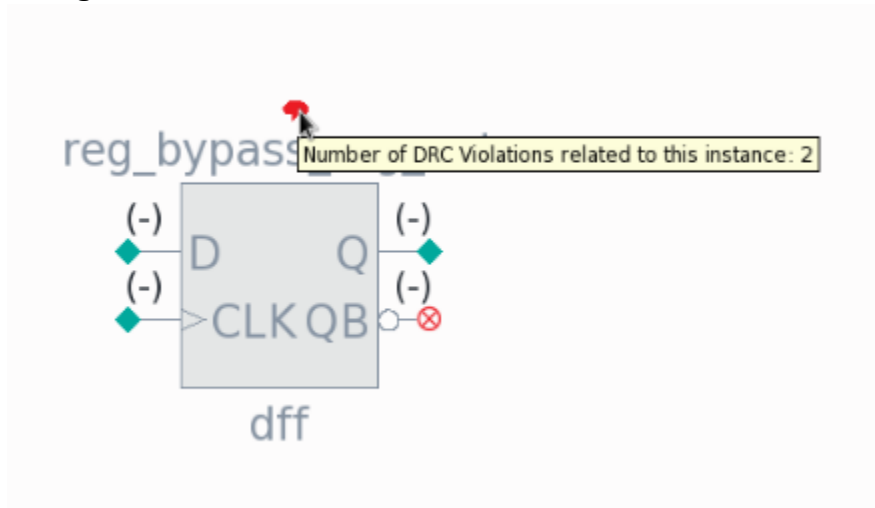


Figure 12-13 shows an instance in the **Hierarchical Schematic** tab with a red callout symbol. Hover the mouse pointer over the symbol to show the number of DRC violations, and click the symbol to display a table of those violations.

**Figure 12-13. Hierarchical Instance With DRC Callout**



Hierarchical cells (HCells, defined by ``celldefine` and ``endcelldefine` in Verilog) are depicted with a slightly lighter gray color than the library cells. You can view objects inside hierarchical cells, but these are not annotated with gate data from the flat model.

Figure 12-14. Hierarchical Cell

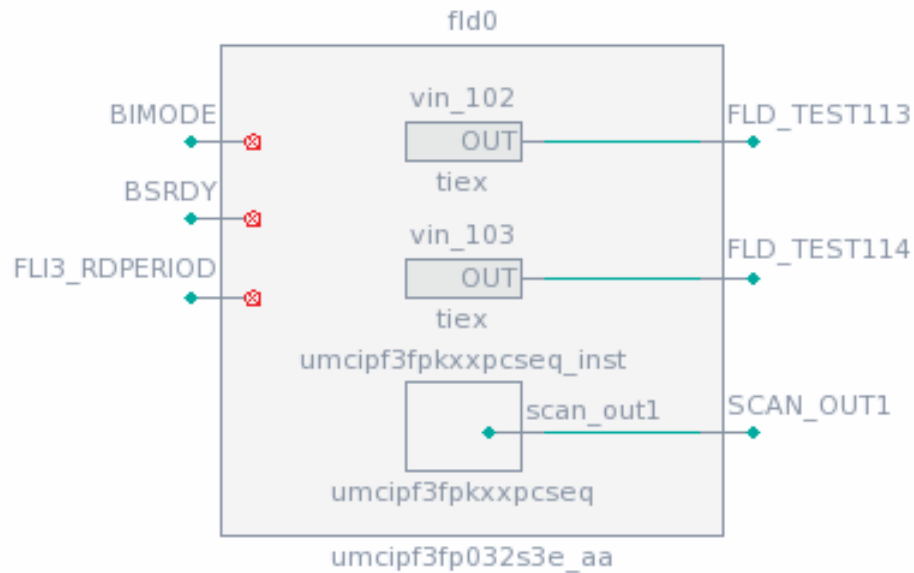


Figure 12-15. Hierarchical Instance Representing an Assign Statement



The code for the assign statement symbol is similar to this:

```
module RDS_process_rtl_tessent_buf_49(a, y);
  input  a;
  output y;

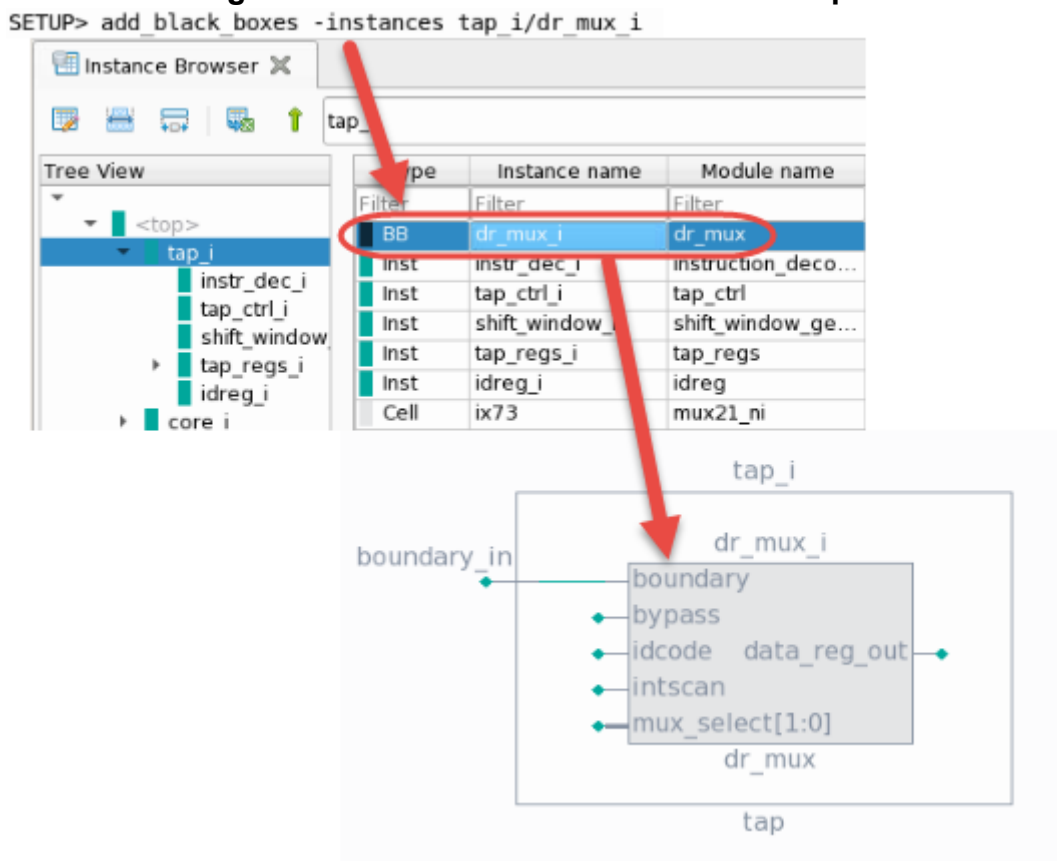
  assign y = a;
endmodule
```

### Black-Boxed Instances

A black-boxed instance is an instance with no defined model that you create with the `add_black_boxes` command.

Figure 12-16 shows how the `add_black_boxes` command creates a blackbox from the `dr_mux_i` instance within the `tap_i` module. The instance then becomes a BB type and displays as a solid gray rectangle in the **Hierarchical Schematic** tab.

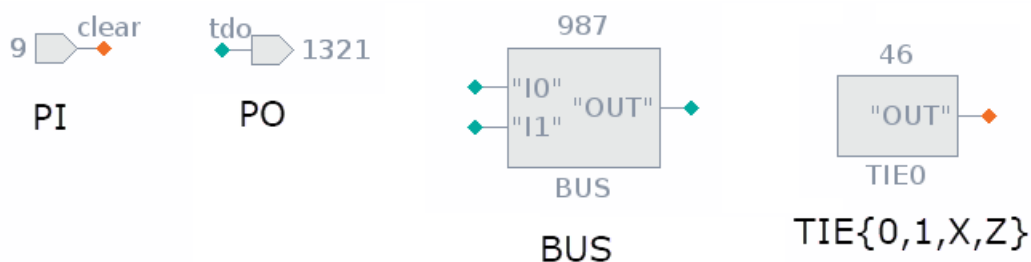
**Figure 12-16. Black-Boxed Instance Example**



### Instances in the Flat Schematic Tab

Flattening the design creates certain artificial gates, as shown in [Figure 12-17](#).

**Figure 12-17. Objects in the Flat Schematic Tab**



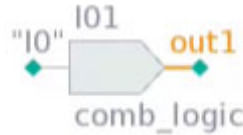
The flattening process creates primary inputs and outputs, as well as bus instances necessary for modeling bidirectional pins and ports. In addition, there are instances that tie a net to a fixed value.

User-defined cells display as rectangles with a solid fill. User-added ports display with an orange fill.



Figure 12-18 shows a persistent buffer symbol in the **Flat Schematic** tab. The same buffer would look like Figure 12-15 in the **Hierarchical Schematic** tab because you typically implement persistent buffers using an “assign” statement.

**Figure 12-18. Persistent Buffer Symbol**



## Pins and Ports

Pins and ports are electrical connections between an instance and a net. Both of these, along with buses (a collection of related pins) indicate signal sources or destinations in the schematic, and are characterized by their size and direction (input, output, or bidirectional). For bus naming conventions, see “[Buses](#)” on page 917.

## Callout Messages

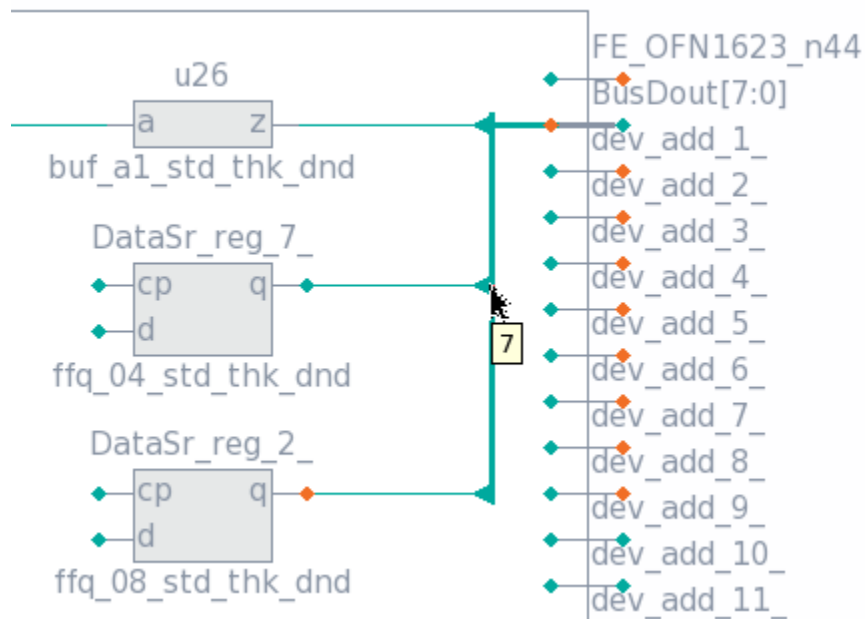
Callouts are informational messages that display on a schematic, as shown in [Figure 12-11](#) on page 881. You can add callouts to any instance, pin, or net object that exists on the schematic using the [add\\_schematic\\_callout](#) command.

Callouts can also display in a collapsed format, as shown as “collapsed callouts” in [Table 12-3](#) on page 886. The table describes how to view the text of a collapsed callout. Use the “Collapse callouts” button as described in [Table 12-2](#) on page 878 to collapse all callouts. Use Ctrl+left mouse button to drag and drop individual callouts.

## Nets

Single nets display as thin green lines, and bus nets display as thicker green lines. When a bus net branches to a single net, the bus index displays when the mouse pointer is hovered over the triangular rip point symbol, as shown in [Figure 12-19](#).

**Figure 12-19. Bus Index Displayed at Rip Point**



## Markers

Tessent Visualizer uses marker symbols in schematics to indicate additional properties of the elements shown. Some, such as trace markers and buffer-collapsing markers, have actions associated with them.

Table 12-3 lists a summary of marker symbols used in Tessent Visualizer schematics, and whether they are displayed in flat schematics (F), hierarchical schematics (H), ICL schematics (I), or the IJTAG Network Viewer (N).

**Table 12-3. Marker Symbols**

| Symbol | Type | Description                                                                                                                                                                                                                            |
|--------|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|        | H,F  | Green diamond: pin with single connection available to display. Click to display unambiguous connection from this pin.                                                                                                                 |
|        | H,F  | Orange diamond: pin with multiple connections available to display. Click to open the Tracer table and select the tracing destination.                                                                                                 |
|        | H    | Blue diamond: ambiguous bidirectional pin. When all drivers and drains of the port are visible, the color changes to green or orange, as appropriate. Click to open the Tracer table and select the tracing direction and destination. |
|        | H    | Half-filled diamond (green): single connection left on bus. Remainder of displayed pins tied or unconnected. Click to open the Pins context table to show the tie values on all bus pins.                                              |

**Table 12-3. Marker Symbols (cont.)**

| Symbol                                                                              | Type    | Description                                                                                                                                                                                                                                                                                                           |
|-------------------------------------------------------------------------------------|---------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|    | H       | Half-filled diamond (blue): one or more ambiguous bidirectional pins on bus. Remainder of displayed pins tied or unconnected. When all drivers and drains of the port are visible, the color changes to green or orange, as appropriate. Click to open the Pins context table to show the tie values on all bus pins. |
|    | H       | Half-filled diamond (orange): multiple connections on bus, some pins tied or unconnected. Click to open the Pins context table to show the tie values on all bus pins.                                                                                                                                                |
|    | H       | Red triangle: pin with coercion (connection against a declared direction). Click to open the Tracer context table and select the tracing destination.                                                                                                                                                                 |
|    | H       | Blue square: RTL connection (no connection in hierarchical model). Click to open the Text/HDL Viewer with a textual representation of this connection.                                                                                                                                                                |
|    | I       | Tan square: implicit connection on the ICL Schematic. The pin is not constrained by the parent module, and is assumed to be driven or active as needed.                                                                                                                                                               |
|   | H,F,I,N | Crossed circle (red): no connection.                                                                                                                                                                                                                                                                                  |
|  | H,I     | Circle with 0/1/X/Z (green): tied to 0/1/X/Z. When shown on a bus port, click to open the Pins context table to show the tie values on all bus pins.                                                                                                                                                                  |
|  | H,I     | Circle with “a”: ambiguous tie values (mix of some or all of 0/1/X/Z) on bus. Click to open the Pins context table to show the tie values on all bus pins.                                                                                                                                                            |
|  | H,F,I,N | Diamond with “+”: collapsed buffers and inverters. Click this symbol to expand. <ul style="list-style-type: none"> <li>• Green: no hidden fanout.</li> <li>• Orange: fanout within collapsed region.</li> </ul>                                                                                                       |
|  | H,F,I,N | Diamond with “-”: expanded buffers and inverters. Click this symbol to collapse. <ul style="list-style-type: none"> <li>• Green: no fanout in collapsible region.</li> <li>• Orange: collapsing hides potential fanout.</li> </ul>                                                                                    |

**Table 12-3. Marker Symbols (cont.)**

| Symbol                                                                              | Type    | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|-------------------------------------------------------------------------------------|---------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|    | H,F,I,N | Collapsed callout. The green symbol indicates that the callout was added with the “add_schematic_callout -collapsed” command. View the callout text as a tooltip by mouse hover. The yellow symbol is a native callout, which you can expand with the “Collapse/Uncollapse callout” toolbar button.                                                                                                                                                                                           |
|    | H       | DRC violation callout. This symbol indicates that there are DRC violations related to the marked object. View the number of DRC violations by mouse hover. Click the symbol to open the list of violations in the DRC Violations table.                                                                                                                                                                                                                                                       |
|    | I,N     | Simulation data callout. Hover over or click this symbol to show simulation data for scan registers.                                                                                                                                                                                                                                                                                                                                                                                          |
|    | H,I     | Light blue callout. It is displayed on an instance on the hierarchical schematic to indicate that a corresponding ICL instance exists, and on the ICL schematic to indicate that a corresponding instance exists on the hierarchical schematic.                                                                                                                                                                                                                                               |
|    | H       | Red scissors: cut point (input connections cut or disconnected by the user or the tool).                                                                                                                                                                                                                                                                                                                                                                                                      |
|   | F       | Green scissors: cut point driver (single sink). Click the green scissors symbol to show the driver or sink port.                                                                                                                                                                                                                                                                                                                                                                              |
|  | F       | Orange scissors: cut point driver (multiple sinks). Click the orange scissors symbol to display a context table showing the drivers or sinks.                                                                                                                                                                                                                                                                                                                                                 |
|  | H,I     | Orange polygon: pseudo-port (primary output added by the user or tool) on the hierarchical schematic. Also used to indicate an ICL alias defined on an ICL instance or port on the ICL Schematic. Click this symbol on the ICL Schematic to show alias information in the context table.<br> <b>Note:</b> Orange fill is also used to indicate primary input or output pseudo-ports on the flat schematic. |
|  | I       | Blue polygon: indicates a pin that is driven by an ICL alias on the ICL Schematic. Click this symbol to show alias information in the context table.                                                                                                                                                                                                                                                                                                                                          |

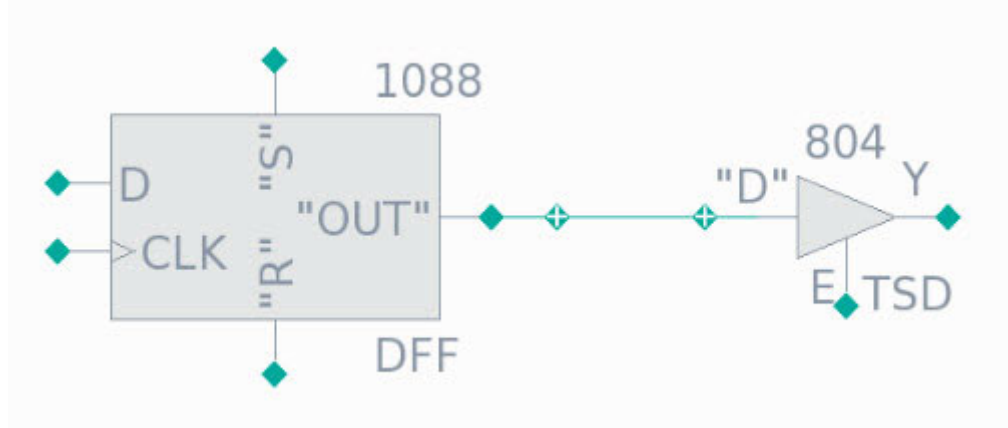
**Table 12-3. Marker Symbols (cont.)**

| Symbol                                                                            | Type    | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|-----------------------------------------------------------------------------------|---------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | H,F,I,N | <p>Tessent Shell attributes markers with gui_marking_index set and “set_attribute_option -display_in_gui” enabled.</p> <ul style="list-style-type: none"> <li>• Filled circle: all displayed objects with the marker have a value set for the associated attribute.</li> <li>• Half circle: not all displayed objects with the marker have a value set for the associated attribute.</li> <li>• Empty circle: displayed in the attributes table; indicates that the attribute is set to the default value and hence is not marked on the schematic.</li> </ul> |

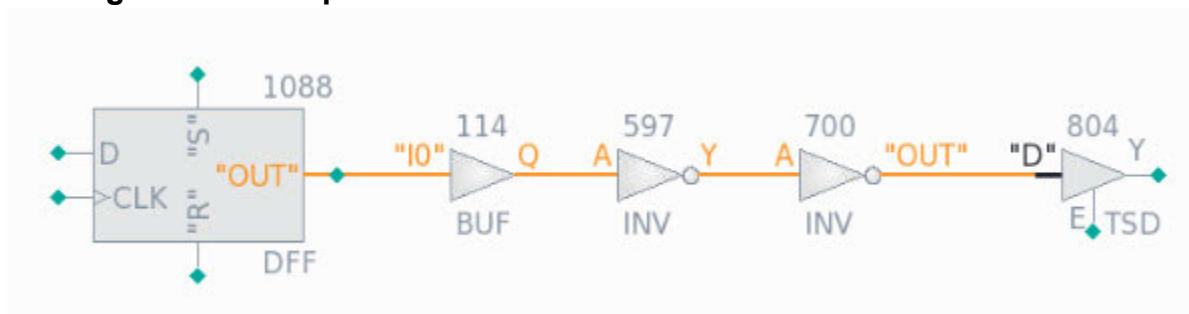
## Buffer and Inverter Collapsing

Buffers and inverters can be collapsed and expanded in schematics to improve readability. Set the preference for this feature in the **Options** (gear) menu of the schematic toolbar.

The collapsed and expanded symbols are shown in either green or orange. Green indicates that there are no hidden fanouts associated with the collapsed objects, as shown in [Figure 12-20](#):

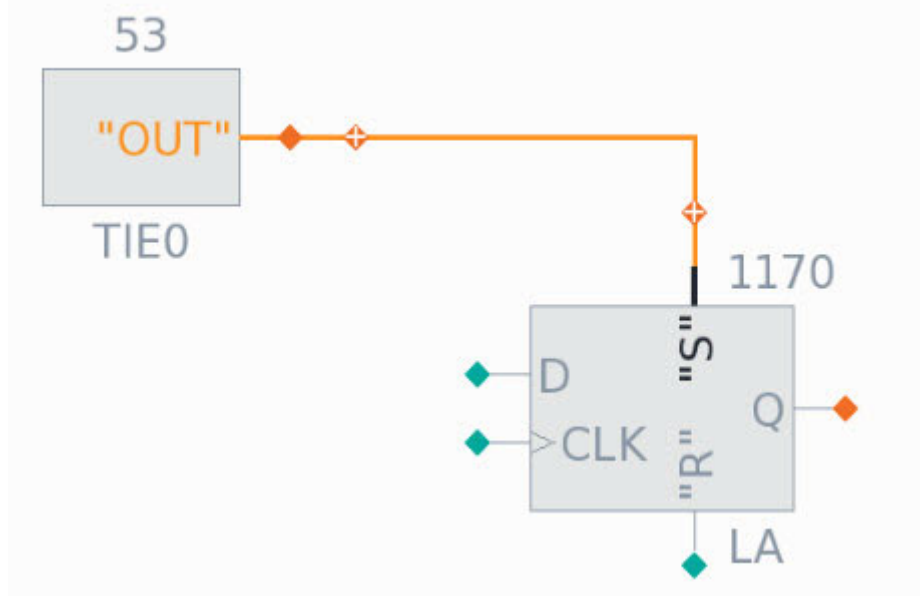
**Figure 12-20. Collapsed Buffers and Inverters With No Hidden Fanout**

After expanding, there is no additional fanout, as shown in [Figure 12-21](#):

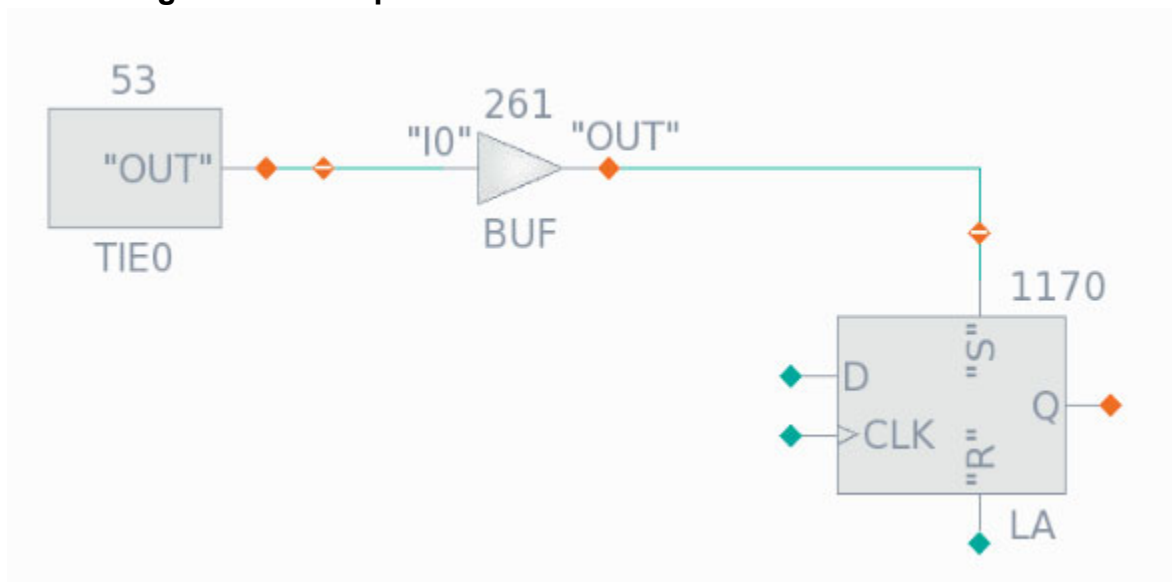
**Figure 12-21. Expanded Buffers and Inverters With No Hidden Fanout**

Orange indicates that there is additional fanout hidden by the buffer/inverter collapsing, as shown in [Figure 12-22](#) and [Figure 12-23](#):

**Figure 12-22. Collapsed Buffers/Inverters With Hidden Fanout**

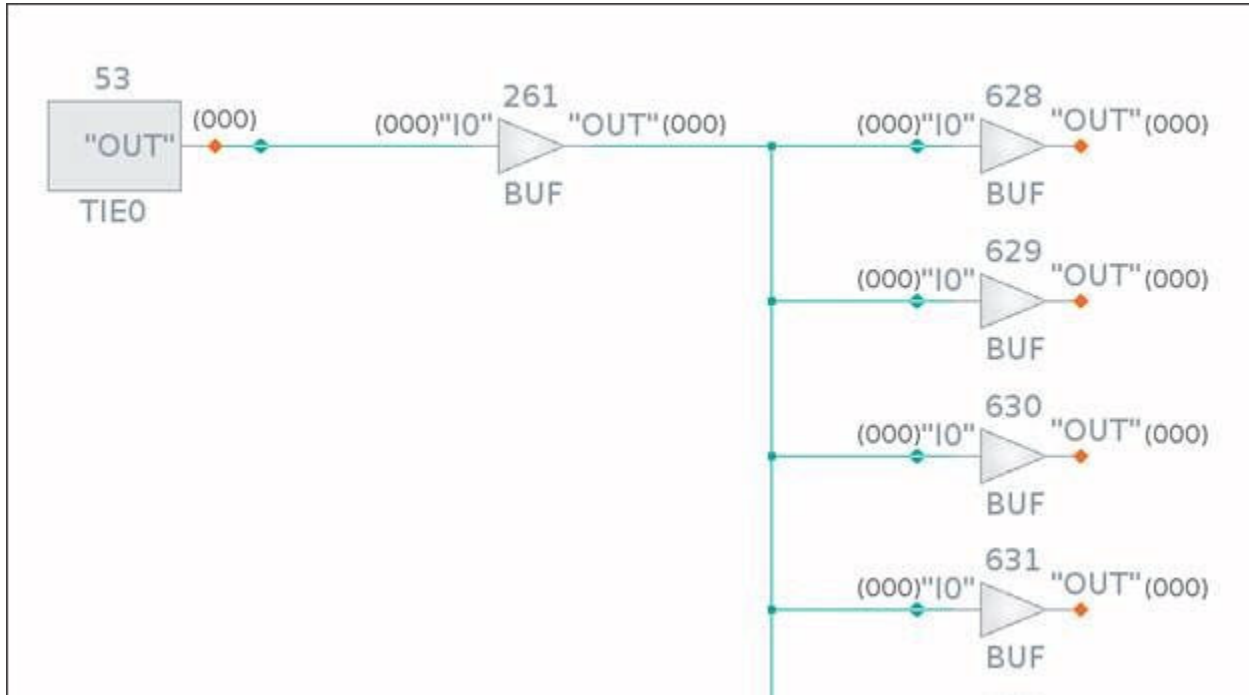


**Figure 12-23. Expanded Buffers/Inverters With Hidden Fanout**



Additional objects appear after adding the fanouts to the schematic, and the color of the markers changes to green as shown in [Figure 12-24](#):

**Figure 12-24. Expanded Buffers/Inverters With Fanout Revealed**



#### Note

Only buffers and inverters added implicitly to the schematic as a result of tracing are collapsible. These are identified with a gradient fill. Buffers and inverters added explicitly by name or gate ID are not collapsible and are identified with a solid fill.

## Folding and Unfolding Instances

Cell grouping boxes in the Flat Schematic and instances in the Hierarchical Schematic can be folded and unfolded.

Folding and unfolding grouping boxes or instances is analogous to collapsing and expanding buffers and inverters, as described in “[Buffer and Inverter Collapsing](#)” on page 889. Double-click the grouping box or instance to do this. [Figure 12-25](#) shows how you can also unfold hierarchical instances by clicking the plus sign (+) symbol at the top-left corner.

Figure 12-25. Unfolding an Instance

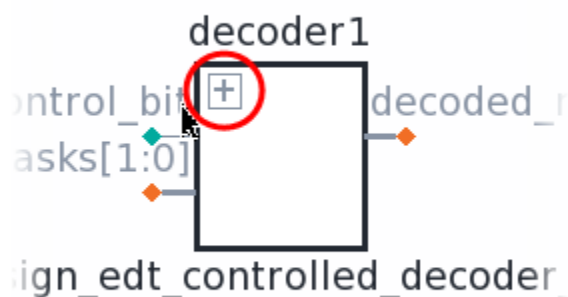


Figure 12-26. Folded Cell Group

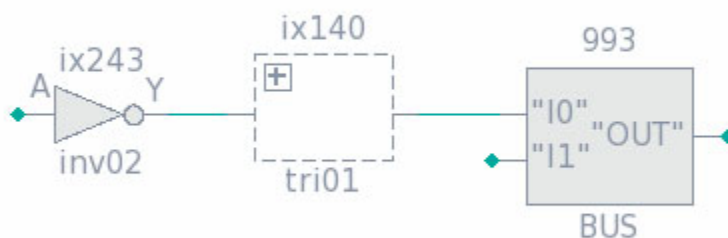
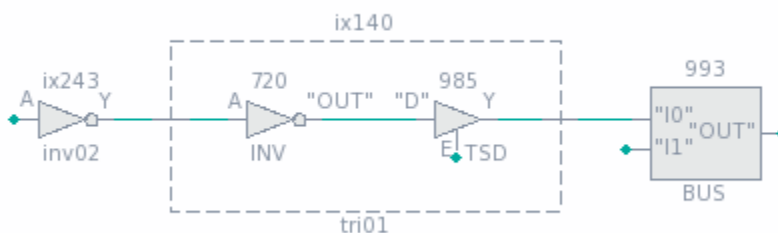


Figure 12-27. Unfolded Cell Group



## Context Tables

Schematics include tables to display information about the objects in the schematic. The tables take different forms depending on the current selection in the schematic.



Context tables are interactive. They display information, but you also use them to select and manipulate objects in the schematic. The following example shows a context table listing the data for a selected instance.

**Figure 12-28. Schematic Elements: Context Table**

The screenshot shows the Tessent Visualizer Schematics window. The main area displays a schematic diagram with various components and connections. A red box highlights a specific component labeled "reg\_shiftdr" (1069), which is identified as the "Selected object". Below the schematic, the "Object name" field shows "tap\_i/tap\_ctrl\_i/reg\_shiftdr (1069)". To the right of the object name are buttons for "Tracer", "Gate Pins", "Flat sinks", and "Netlist drivers". Below these buttons is a context table for the selected object. A red arrow points from the "Context table for selected object" label to the table.

| Gate pin ID | Pin leafname | Direction |
|-------------|--------------|-----------|
| Filter      | Filter       | Filter    |
| 1069.0      | "OUT"        | output    |
| 1069.1      | "S"          | input     |
| 1069.2      | R            | input     |

## Tracer

The context table for the tracing function provides information about the objects available to the tracing process.

When you trace from an ambiguous point (orange trace marker) or click the **Tracer** button next to the address bar when a pin is selected, the tracer context table opens. Use this table to add the next object in the tracing path to the schematic by double-clicking the appropriate row in the table. You can select the tracing direction and strategy, as described in [“Signal Net Tracing Strategies”](#) on page 902.

Figure 12-29. Tracer Context Table

The screenshot shows the Tessent Visualizer interface with the 'Hierarchical Schematic' tab selected. The main window displays a circuit diagram with a block labeled 'data1' containing a 'register8\_scan1' component. The component has pins for 'clear', 'clock', 'd[7:0]', 'scan\_in1', and 'sen'. The output of the register is labeled 'q[7:0]' and 'scan\_out1'. Below the diagram, the 'Object name' field is set to 'core\_i/test\_design\_i/data1/q'. The 'Tracer' button is active, and the 'Direction' is set to 'Find drivers' and the 'Strategy' is 'Decision point'. The 'Tracer Context Table' is displayed below the controls.

| Pin name (start)                | Pin name (end)                        | Distance |
|---------------------------------|---------------------------------------|----------|
| Filter                          | Filter                                | Filter   |
| core_i/test_design_i/data1/q[5] | core_i/test_design_i/data1/reg_q_5_/Q | 1        |
| core_i/test_design_i/data1/q[4] | core_i/test_design_i/data1/reg_q_4_/Q | 1        |
| core_i/test_design_i/data1/q[3] | core_i/test_design_i/data1/reg_q_3_/Q | 1        |

## Pins and Gate Pins

Context tables are available for instance pins in the Hierarchical Schematic and gate pins in the Flat Schematic.

You can select an instance in the **Hierarchical Schematic** tab or **Flat Schematic** tab and click the **Pins** button or the **Gate Pins** button near the address bar, respectively, to view information about the pins belonging to that instance.

**Figure 12-30. Context Table for Instance Pins (Hierarchical Schematic)**

The screenshot shows the Tessent Visualizer interface with the Hierarchical Schematic tab selected. The main window displays a schematic diagram of a register component labeled 'register8\_scan1'. The component has several pins: 'clear', 'clock', 'd[7:0]', 'scan\_in1', 'sen', 'q[7:0]', and 'scan\_out1'. A black box highlights the 'sen' and 'scan\_in1' pins. Below the schematic, the 'Object name' field contains 'core\_i/test\_design\_i/data1'. The 'Tracer' section has buttons for 'Pins', 'Instances', and 'Nets', with 'Pins' currently selected. The context table below shows the following data:

| Direction | Pin name                             | Is part of bus? | Is pseudo-port? |
|-----------|--------------------------------------|-----------------|-----------------|
| Filter    | Filter                               | Filter          | Filter          |
| input     | core_i/test_design_i/data1/sen       | false           | false           |
| output    | core_i/test_design_i/data1/scan_out1 | false           | false           |
| input     | core_i/test_design_i/data1/scan_in1  | false           | false           |

### Instances (Hierarchical Schematic)

A context table is available for instances in the **Hierarchical Schematic** tab.

Select an instance in the **Hierarchical Schematic** tab and click the **Instances** button near the address bar to view information about the child instances.

**Figure 12-31. Context Table for Instances (Hierarchical Schematic)**

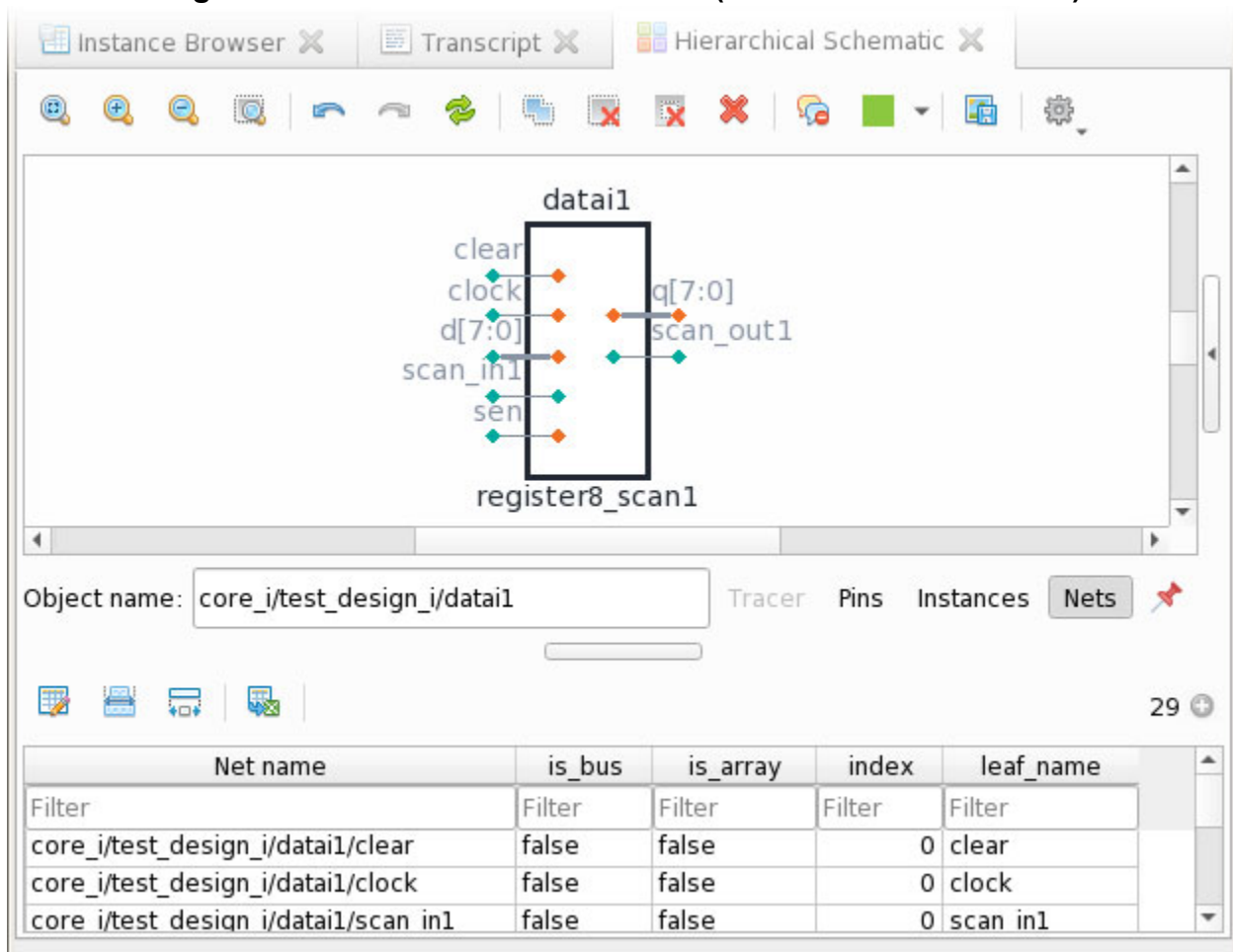
The screenshot shows the Tessent Visualizer interface with the 'Hierarchical Schematic' tab selected. The main window displays a schematic diagram of a circuit component labeled 'data1' within a larger block 'register8\_scan1'. The schematic includes inputs like 'clear', 'clock', 'd[7:0]', 'scan\_in1', and 'sen', and outputs like 'q[7:0]' and 'scan\_out1'. Below the schematic, the 'Object name' field is set to 'core\_i/test\_design\_i/data1'. The 'Instances' tab is active in the bottom panel, displaying a context table with 9 items.

| Type   | Hierarchical name                   | Module name | Total Faults |
|--------|-------------------------------------|-------------|--------------|
| Filter | Filter                              | Filter      | Filter       |
| Cell   | core_i/test_design_i/data1/uu1      | buf02       | 0            |
| Cell   | core_i/test_design_i/data1/reg_q_0_ | sffr_ni     | 0            |
| Cell   | core_i/test_design_i/data1/reg_q_1  | sffr_ni     | 0            |

## Nets (Hierarchical Schematic)

A context table is available for nets in the **Hierarchical Schematic** tab.

Select an instance in the **Hierarchical Schematic** tab and click the **Nets** button near the address bar to list the nets one level below that instance.

**Figure 12-32. Context Table for Nets (Hierarchical Schematic)**

### Flat Sinks (Flat Schematic)

Flat sinks are the set of primary inputs that you add to control internal pins. For example, you can add a primary input at the output of a PLL to model the clock, while keeping the PLL black-boxed.

The green dashed line in the following figure representing the original fanout net shows that this replacement is complete, and that there is only one user-added primary input associated with this fanout. The red scissors symbol indicates that a pseudo-port now drives that fanout.

**Figure 12-33. Fanout Replaced by User-Added Primary Input**

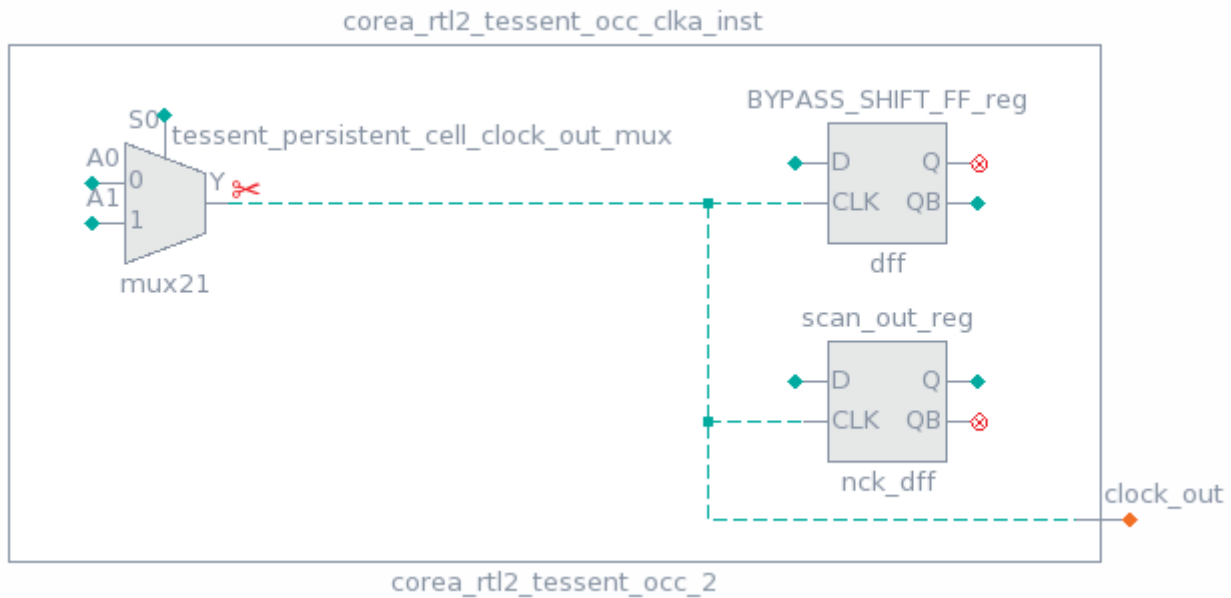


Figure 12-34 shows the same MUX in the Flat Schematic, as well as the user-added primary input. The primary input is added to the schematic when you click the green scissors symbol on the MUX output.

**Figure 12-34. Flat Sink**

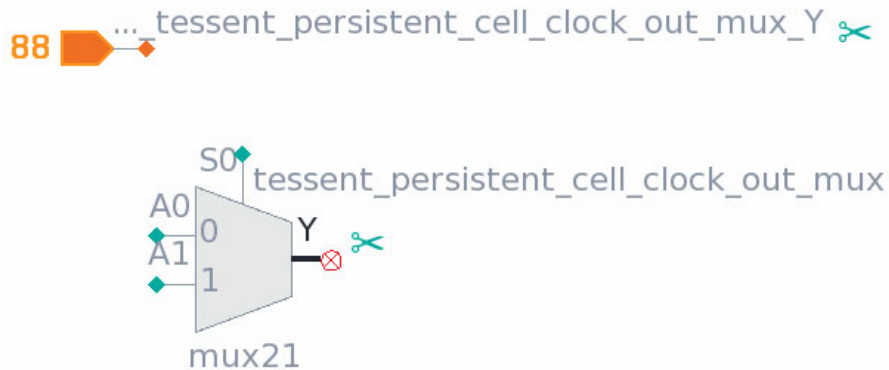


Figure 12-35 shows a Hierarchical Schematic view of the same clock MUX, but in this case it has had all of its fanout replaced by multiple user-added primary inputs. The orange dashed line representing the original fanout net shows that this replacement is complete, and that there is more than one user-added primary input associated with this fanout. The red scissors symbols indicate that pseudo-ports now drive that fanout.

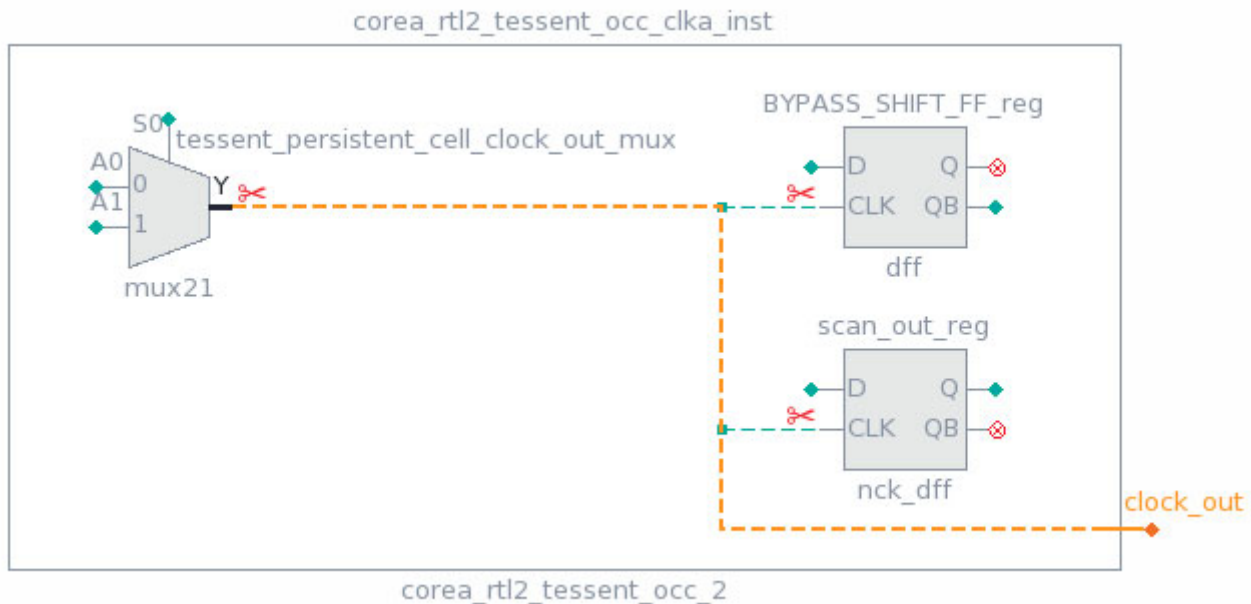
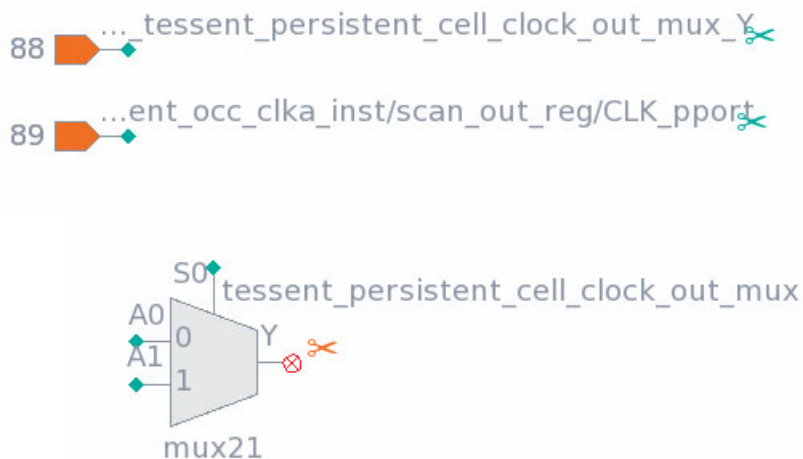
**Figure 12-35. Fanout Replaced by Multiple User-Added Primary Inputs**

Figure 12-36 shows the same MUX in the Flat Schematic, as well as the user-added primary inputs. The primary inputs are added to the schematic when you click the orange scissors symbol.

**Figure 12-36. Multiple Flat Sinks**

Clicking the orange scissors symbol displays the context table for the user-added primary inputs, and double-clicking a row displays the flat sink.

**Figure 12-37. Context Table for Multiple Flat Sinks**

Object name: corea\_rt12\_tessent\_occ\_clka\_inst/tessent\_persistent\_cell\_cl



| Gate pin ID | Pin leafname                                                        | Direction |
|-------------|---------------------------------------------------------------------|-----------|
| Filter      | Filter                                                              | Filter    |
| 88.0        | corea_rt12_tessent_occ_clka...                                      | output    |
| 89.0        | corea_rt12_tessent_occ_clka...<br>scan_out_reg/CLK_pport            | output    |
| 90.0        | corea_rt12_tessent_occ_clka...<br>BYPASS_SHIFT_FF_reg/<br>CLK_pport | output    |

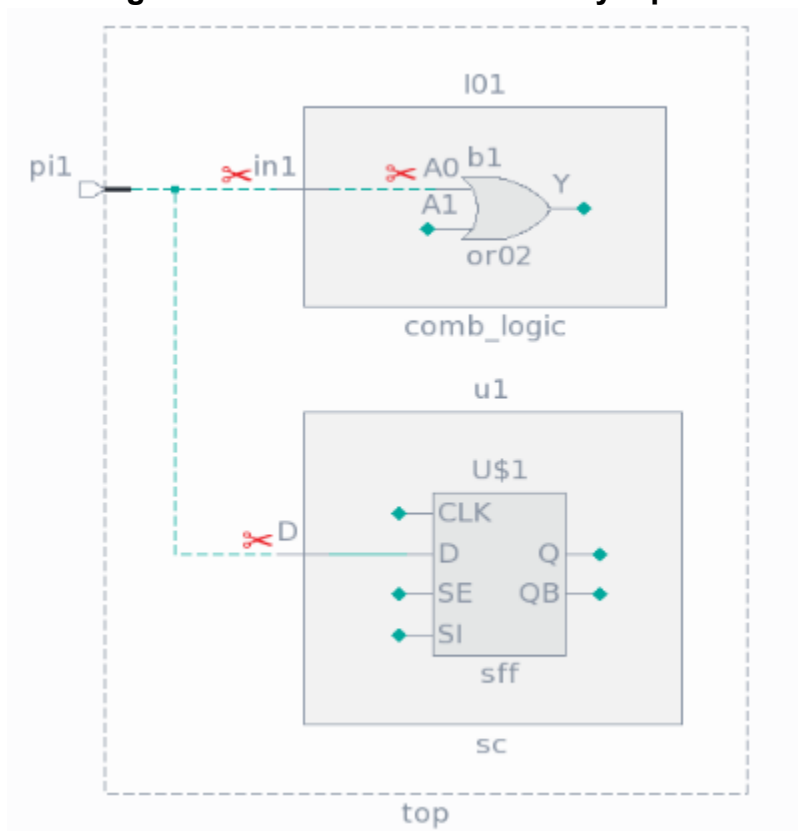
## Netlist Drivers (Flat Schematic)

Netlist drivers are the original drivers of the fanout of user-added primary inputs.

Use the following commands to add primary inputs. These are indicated by the red scissors symbol in the **Hierarchical Schematic** tab.

```
add_primary_inputs I01/in1 -internal -pin INTERNAL_1
add_primary_inputs I01/b1/A0 -internal -pin INTERNAL_2
add_primary_inputs u1/D -internal -pin INTERNAL_3
```



**Figure 12-38. User-Added Primary Inputs**

As a result, the original fanout of the pi1 pin was disconnected, as indicated by the dashed line.

Figure 12-39 shows the pi1 pin in the Flat Schematic. The circle with the red “X” shows that it is disconnected, and the orange scissors symbol indicates that its original fanout is now driven by multiple user-added primary inputs. Click the orange scissors symbol to display the context table listing the flat sinks, as shown in the following figure.

Figure 12-39. Netlist Driver and Context Table for User-Added Primary Inputs

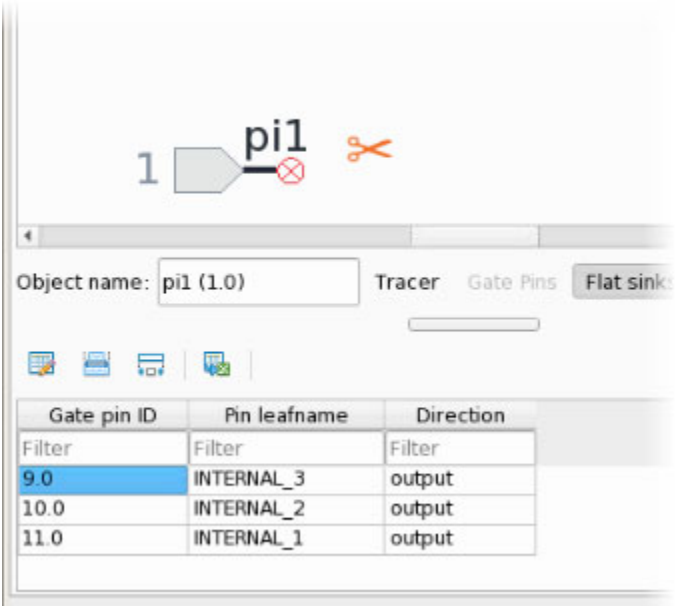
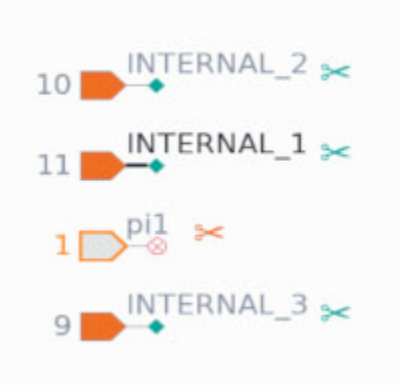


Figure 12-40 shows the user-added primary inputs in the Flat Schematic. The orange fill indicates that these are user-added, and the green scissors symbol indicates that they have a single original driver. Clicking the scissors symbol shows the driver.

Figure 12-40. User-Added Primary Inputs (Flat Schematic)



## Related Topics

[Markers](#)

## Signal Net Tracing Strategies

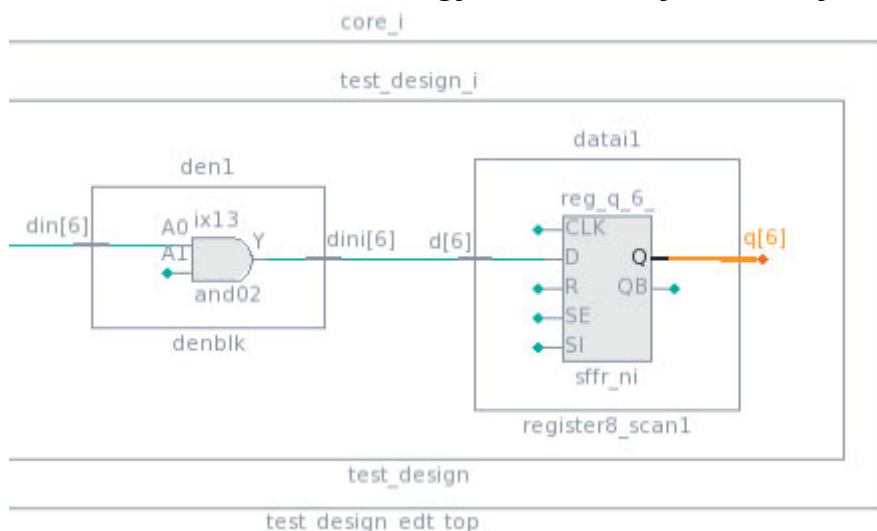
The Tessent Visualizer schematic tabs feature multiple tracing strategies. They are available when the Tracer context table is activated for a pin, either by clicking an orange trace marker in the schematic or the **Tracer** button after a pin is selected. These tracing strategies are common for all schematics.

For the green trace markers, the following tracing strategies are available in the schematics:

- Trace to decision point (default)
- Trace by one

Use the **Options** button (⚙️) in the schematic toolbar to select an option. A decision point is any point where a logic decision is made, such as a gate, state element, or a hierarchical pin with multiple fanouts, as shown in [Figure 12-41](#).

**Figure 12-41. Decision Point Strategy on Hierarchy Boundary at q[6] Port**



**Figure 12-42. Choosing a Path From the Tracing Table**

| Pin name (start)                | Pin name (end)                                  | Distance |
|---------------------------------|-------------------------------------------------|----------|
| Filter                          | Filter                                          | Filter   |
| core_i/test_design_i/data1/q[3] | core_i/test_design_i/data1/reg_q_3/D            | 2        |
| core_i/test_design_i/data1/q[2] | core_i/test_design_i/nonscanblock1/req_dout_2/D | 2        |

You can continue tracing from the decision point by double-clicking one of the paths in the table.

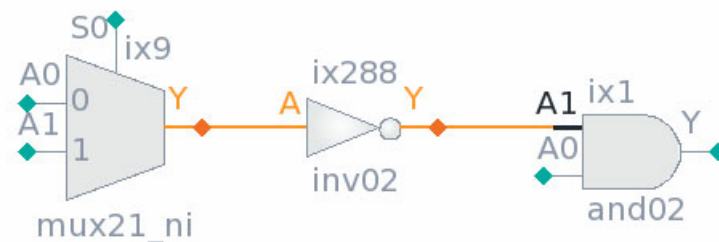
For additional information about tracing buses, see [“Buses”](#) on page 917.

## Decision Point Tracing Strategy

Decision point tracing is the default strategy. In the decision point tracing strategy, a path is traced to the point where a logic decision is to be made. Tracing continues through buffers, inverters, wire primitives, and hierarchical boundaries as long as there is only one fanout (forward tracing) or fan-in (backward tracing), and stops on other gates, cells, and logic primitives.

Figure 12-43 shows the result of a trace using the Decision Point strategy. Pin A1 on the AND gate has been traced back through the inverter to the MUX, which is the first logic decision point.

**Figure 12-43. Decision Point Strategy**



## Endpoint Tracing Strategy

When you use the endpoint tracing strategy, the tracing stops on any endpoint element. An endpoint is defined as any primary input or output, or any sequential element, memory, or black box. Tracing under the endpoint strategy traverses all buffers, inverters, logic gates, and hierarchical boundaries between the tracing start point (the pin selected on the schematic) and the endpoint, and the resulting paths includes those elements.

## Nearest State Element Tracing Strategy

When you use the nearest state element tracing strategy, tracing stops on the first state element encountered. State elements are sequential elements or memories. Tracing under the nearest state element strategy includes all buffers, inverters, logic gates, and hierarchical boundaries between the tracing start point and the state-element endpoint, and the resulting paths include those elements.

## Nearest Scan Element Tracing Strategy

The nearest scan element tracing strategy is similar to the nearest state element tracing strategy. The difference is that while the nearest state element tracing strategy includes all state elements in the fanout of the starting trace point, the nearest scan element tracing strategy only considers those elements that are included in the scan chain path.

## Complete Fan-in and Fanout Tracing Strategy

Complete fan-in or fanout tracing performs full structural tracing. When tracing with the complete fan-in and fanout strategy, the entire input or output logic cone is included in the results.

## Stop on First Match Tracing Strategy

For the first match tracing strategy, you provide additional data in the form of a filter. The complete fan-in / fanout strategy is used, but tracing stops when the first element matching the filter is found on the path. A single result is reported.

## Trace by One Tracing Strategy

When you use the trace by one strategy, tracing stops at the next hierarchical level found in the Hierarchical Schematic, or the next logic element found in the Flat Schematic.

## Context Menu Tracing

The Tessent Visualizer schematic tabs support multiple actions for context menu tracing. These actions are available when the context table is activated after you select pins on the schematic.

The following tracing actions are available in the schematic tabs:

- **Trace backward** — The tool activates this action if at least one input or bidirectional pin is selected. This option enables you to trace backward to the immediately preceding decision point from the selected pin, bus, or instance objects. If you trace from an instance object in the schematic, the tool performs the tracing from all the input pins of the schematic.
- **Trace pins** — The tool activates this action if you select exactly two pins, ports, or bus pins. The tool tries to connect the selected pins on the schematic. If the tool cannot find a connection between the specified pins, it displays a corresponding message.

## Displayed Property

All objects in Tessent Visualizer have a property called “displayed.”

The “displayed” property of an object refers to the corresponding schematic for the object, and indicates whether that object is added to the schematic. The context of being displayed is important for pin buses because all tracer actions are run for parts of the bus being added to the schematic. By default, the “displayed” property is indicated in tables by a distinct background color for a row, and can also be added as a column in those tables.

## Tessent Shell Attributes

Tessent Visualizer includes functionality to view Tessent Shell attributes and annotate the objects displayed in schematics with them.

The Tessent Shell attributes for the currently selected object in the schematic are shown in a table.

**Figure 12-44. Tessent Shell Attributes Table**

The screenshot shows the Tessent Visualizer interface with the 'Flat Schematic' tab selected. On the left, a schematic diagram shows a component 'bypass\_logic\_i' with a pin 'edt\_scan\_in[3:0]'. A tooltip 'direction = output' is displayed over a marker on this pin. On the right, the 'Tessent Shell Attributes Table' is visible, showing a list of attributes and their values. The 'GUI' column contains colored circles corresponding to the markers in the schematic. Below the table, there is a section for 'Pin leafname', 'Direction', and 'Gate data'.

| GUI | Name                | Value             |
|-----|---------------------|-------------------|
|     | Filter              | Filter            |
|     | design_hierarch...  | 3                 |
| ●   | direction           | output            |
|     | has_functional_s... | true              |
|     | index               | <multiple values> |
| ●   | is_MSB              | <multiple values> |
|     | is_bus              | true              |
|     | is_created          | false             |
|     | is_hard_module      | false             |
| ○   | is_non_editable     | false             |
|     | is_non_editable_... |                   |

| Pin leafname   | Direction | Gate data |
|----------------|-----------|-----------|
| Filter         | Filter    | Filter    |
| edt_scan_in[3] | output    |           |
| edt_scan_in[2] | output    |           |
| edt_scan_in[1] | output    |           |

By default, only those attributes with non-default values, or that were registered with the “-show\_default” option, are shown. You can use the **Show all** checkbox to show all attributes, including those with default values.

You can double-click a cell or use the right mouse button context menu in the GUI column to select a color in the GUI marking index and show a corresponding marker near schematic objects that have that attribute set to a non-default value. Hover the mouse pointer over that marker in the schematic as shown in [Figure 12-44](#) to show a tooltip with the attribute name and value.

---

**Note**

---

 The circles in the GUI column correspond to the integer GUI marking indices from the “[set\\_attribute\\_options -gui\\_marking\\_index](#)” command. You can use “> 0” or “!=0” in the filter field of this column to search for attributes with this property set.

---

---

**Note**

---

 For Boolean attributes, the marker is only shown if the value is set to “true.”

---

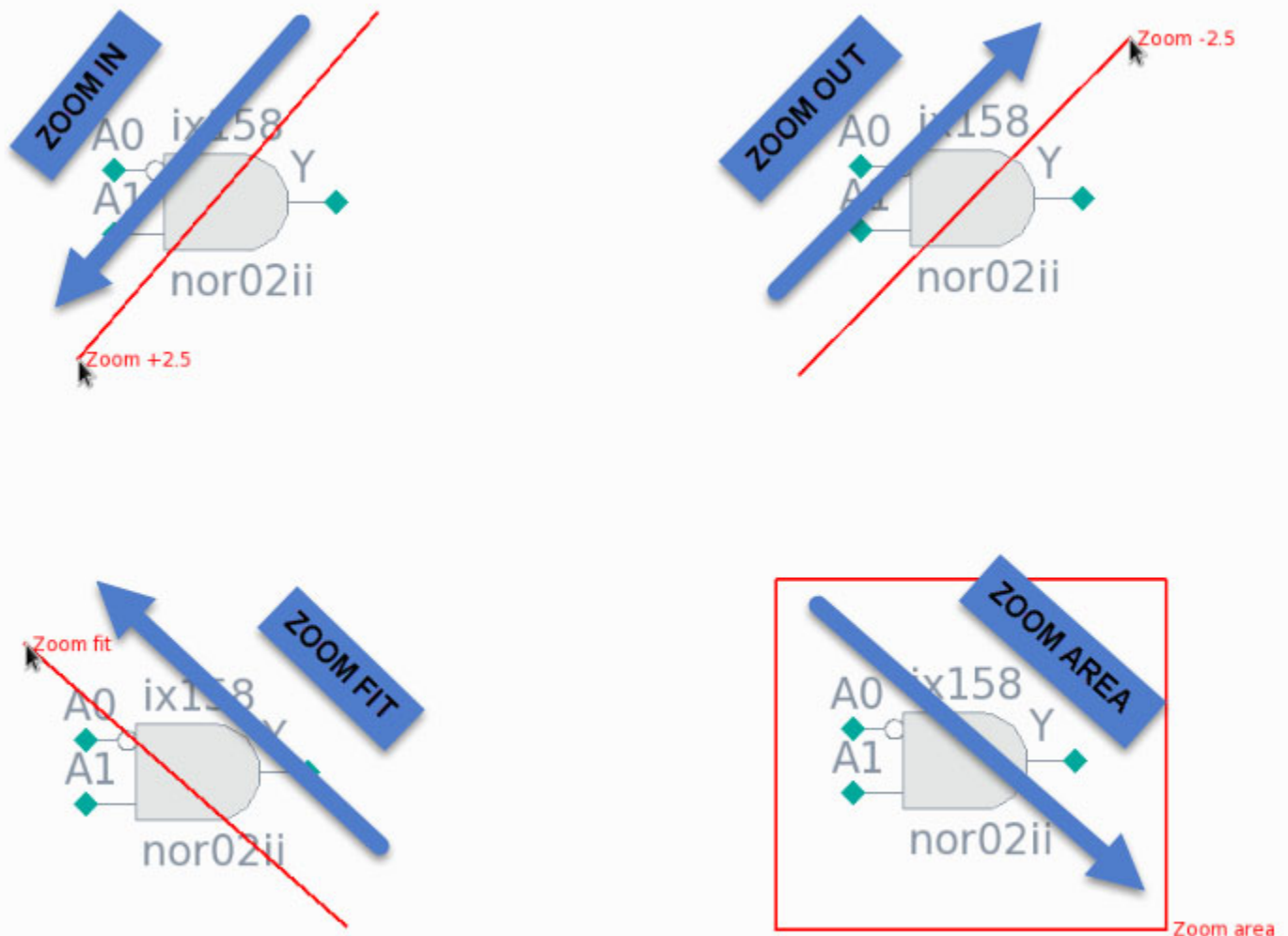
An empty circle in the GUI column indicates that the attribute is set to the default value. A corresponding marker is not displayed in the schematic window in this case. A half-filled circle applies to buses only and indicates that not all of the pins composing the bus have the relevant attribute set to nondefault values. Additionally, the “Value” column indicates “<multiple values>” for buses where the values for this attribute of separate pins are not consistent.

## Mouse Gestures

Zoom in or out within schematics using the left mouse button.

[Figure 12-45](#) summarizes the four mouse gestures (stroke commands) available to perform a zoom.

**Figure 12-45. Mouse Gestures in Tessent Visualizer Schematics**



You can also zoom in and out using the scroll wheel of your mouse.

Additional mouse gestures are available:

- Shift-click and drag with the left mouse button: area select.
- Click and drag with the scroll wheel: pan the view.

Customize the mapping of mouse gestures to specific mouse buttons and keyboard modifiers in the Preferences dialog box. See [“Tessent Visualizer Preferences”](#) on page 909.



# Tessent Visualizer Preferences

Customize Tessent Visualizer with the Preferences dialog box, which is available from the **Settings** pulldown menu.

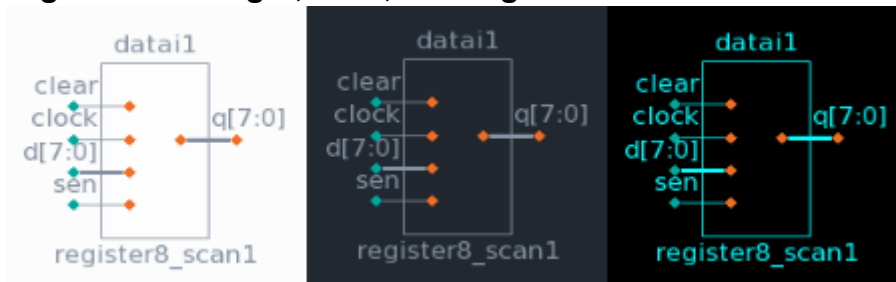
The Preferences dialog box contains several tabs:

- **Schematics**
- **Tables**
- **Text viewers**
- **Other**

Use the **Schematics** tab to define how schematics look and operate. Set maximum lengths for names, set the width of net representations, redefine how the mouse works when interacting with schematics, and choose the color theme.

There are three predefined color themes available in Tessent Visualizer schematics: light, dark, and high-contrast. Set the color scheme with the Preferences dialog box.

**Figure 12-46. Light, Dark, and High-Contrast Color Themes**

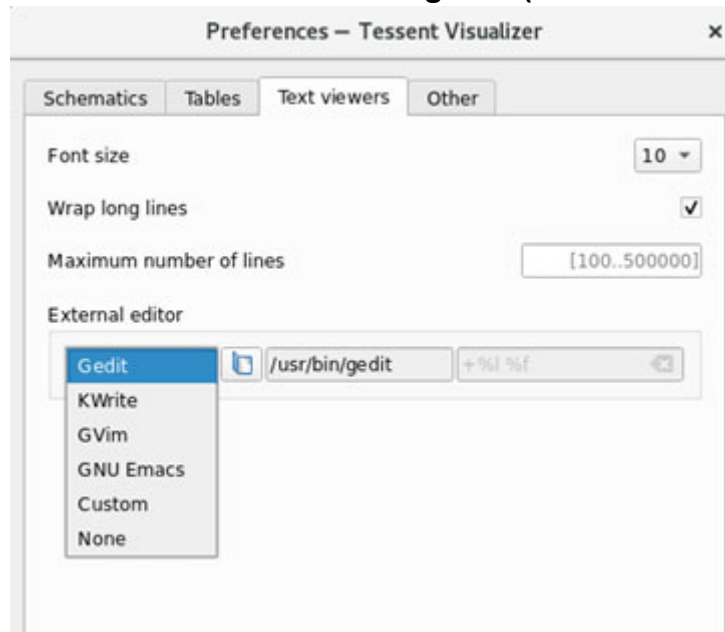


Use the **Tables** tab to set the maximum number of rows loaded in tables and to show line numbers in tables.

Use the **Text viewers** tab to set the font size and line-wrapping policy in the Text Viewer. You can also configure the maximum number of lines displayed in the text viewer, the default being 20,000. You can use the **Text viewers** tab to define an external text editor that can be launched from the Text/HDL Viewer tab. Choose from several common text editors, or define your own preferred one. Use the variables “%l” (line number) and “%f” (filename) as arguments to the text editor executable. For example, this text editor definition opens the file currently in the Text/HDL Viewer tab in the Gedit editor with the cursor positioned at the current line:

```
/usr/bin/gedit +%l %f
```

**Figure 12-47. Preferences Dialog Box (Text viewers Tab)**



---

**Note**



Tessent Visualizer invokes whichever executable you specify as the external text editor. Ensure that valid and working options are set here.

---

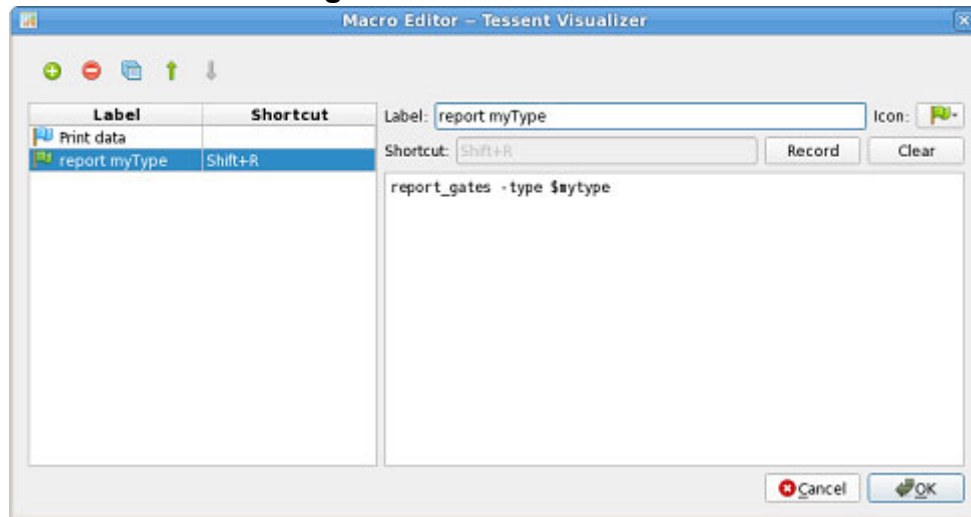
Use the **Other** tab to set the global font size.

## Macros

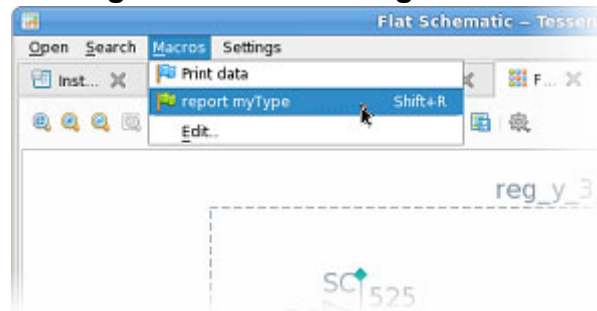
You can define a macro for any script or command that you can run in the Tessent Shell environment.

Open the **Macros** menu to define or run custom macros. You can use the Macro Editor dialog box to create new macros or modify existing macros. [Figure 12-48](#) shows the Macro Editor with two simple macros defined. With the Macro Editor, you can do the following:

- Add, remove, or duplicate a macro.
- Reorder the macros in the **Macros** menu.
- Record a keyboard shortcut for a macro.
- Undefine a macro keyboard shortcut.
- Select an icon for a macro.

**Figure 12-48. Macro Editor**

The following example shows the macros menu after creating the “report myType” macro in the editor. In this example, the macro runs the `report_gates` command with the `-type` option and the predefined variable shown in [Figure 12-48](#) above.

**Figure 12-49. Running a Macro**

You can define a keyboard shortcut, for example, Shift+R, for a new macro. You can run the macro directly from the macros menu, or from the keyboard using that shortcut. If you attempt to record a keyboard shortcut for a macro using an existing shortcut, an error message is reported.

## Tooltips

Many Tessent Visualizer features have associated tooltips, which are informational text popups that appear when you hover the mouse pointer over a graphical element such as an object, filter box, or button.

[Figure 12-50](#), [Figure 12-51](#), and [Figure 12-52](#) show examples of tooltips for several Tessent Visualizer features.

Figure 12-50. Tooltip for a Collapsed Buffer Indicator

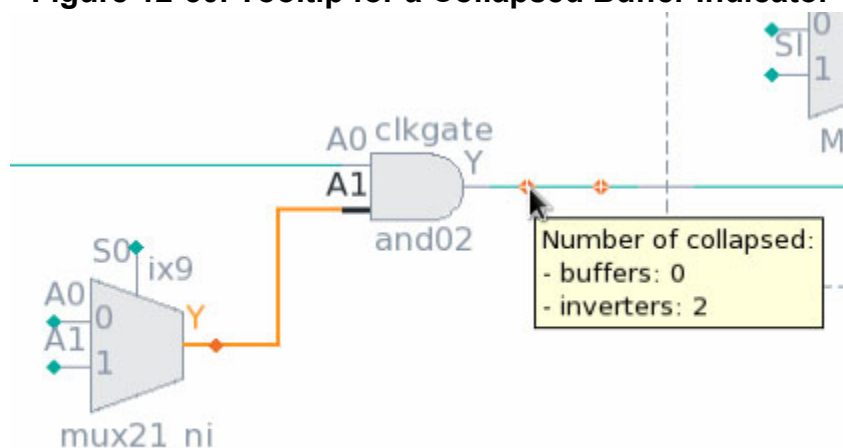
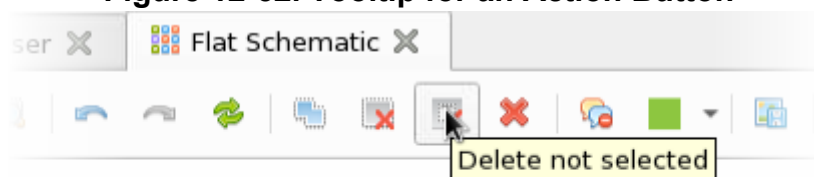


Figure 12-51. Tooltip for a Filter Box

| Type     | Instance name | Module name          | Total Faults  | Test Co |
|----------|---------------|----------------------|---------------|---------|
| Filter   | Filter        | Filter               | Filter        | Filter  |
| Instance | ta            | Wildcards:           | value*, va?ue | 0       |
| Instance | cc            | Exact match:         | = 'value'     | 0       |
| Instance | bs            | Exact inverse match: | != 'value'    | 0       |
| Instance | pa            | Operators:           | <, <=, >, >=  | 0       |
|          |               | Logical operators:   | AND, OR, NOT  |         |
|          |               | GLOB expression:     | GLOB 'expr'   |         |
|          |               | LIKE expression:     | LIKE 'expr'   |         |
|          |               | Regular expression:  | REGEXP 'expr' |         |

Figure 12-52. Tooltip for an Action Button



## Saving and Restoring the Session State

The session state, such as the tabs displayed in a window and specific user preferences, is stored in a hidden file in your home directory. Some other specific settings such as filtering and

instance highlighting persist in memory until the Tessent Visualizer server is closed. For example, if you close a Tessent Visualizer client and later start a new client connected to the same server, the session is displayed as you left it.

You can save and restore specific configurations with **Settings > Export configuration** and **Settings > Import configuration**. You can also start Tessent Visualizer with a specific configuration using `-import_configuration` option of the `open_visualizer` with either the `open_visualizer` command or **tessent -visualizer** invocation.

## Window Title Prefixes

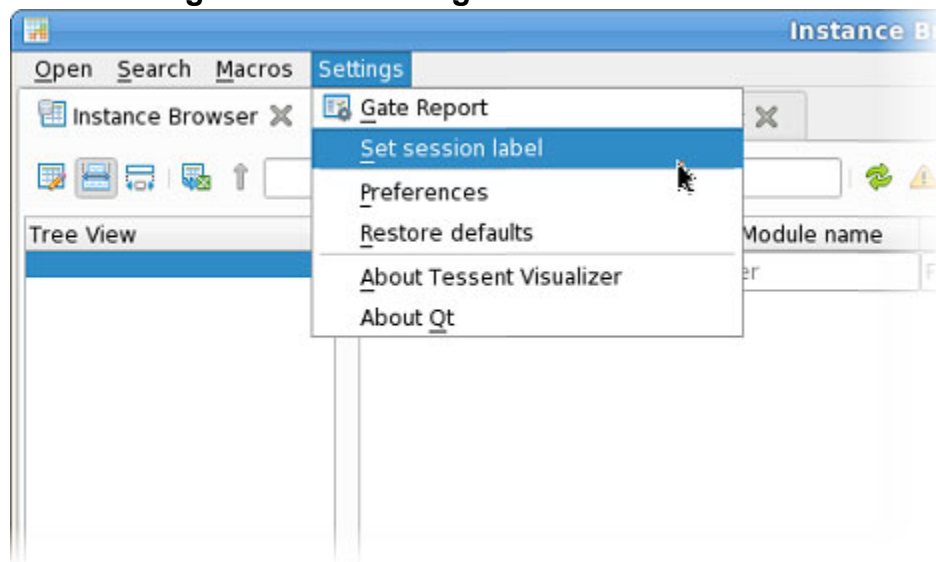
You can define a prefix to be added to the title bars of the Visualizer windows, to distinguish between multiple sessions running on the same machine.

To define a window title prefix, invoke Tessent or open the Visualizer session with the `-label` option:

```
% $TESSENT_HOME/bin/tessent -visualizer -label name  
  
SETUP> open_visualizer -label name
```

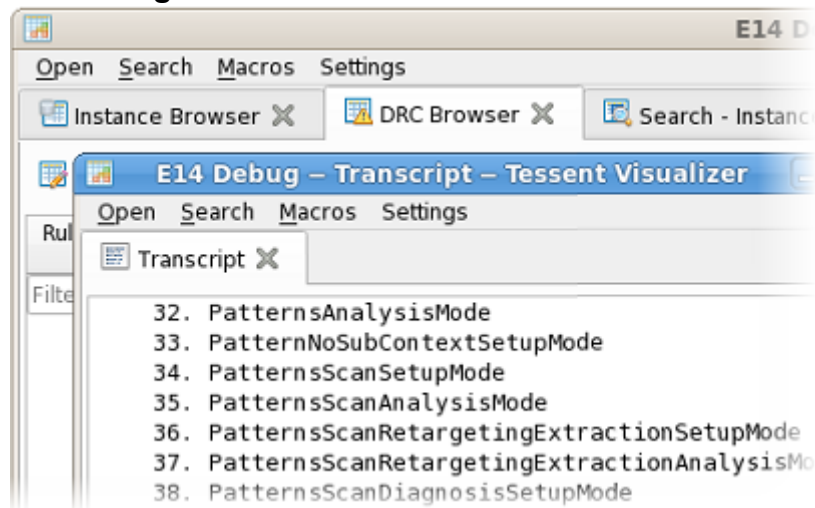
Alternately, set the prefix directly in Visualizer as shown in [Figure 12-53](#):

**Figure 12-53. Setting a Window Title Prefix**



The standard window table is prepended with the string you set, as shown in [Figure 12-54](#). In addition, “Tessent Visualizer” is added to the window title as a suffix:

**Figure 12-54. Visualizer Window Label**



# Tessent Visualizer GUI Reference

The Tessent Visualizer GUI consists of several tabbed windows that function as different views on design data. You can look at objects as part of schematics, browse tables, view a report, or look at different types of syntactically-highlighted text.

|                                      |            |
|--------------------------------------|------------|
| <b>Hierarchical Schematic</b> .....  | <b>915</b> |
| <b>Flat Schematic</b> .....          | <b>921</b> |
| <b>ICL Schematic</b> .....           | <b>923</b> |
| <b>Instance Browser</b> .....        | <b>929</b> |
| <b>ICL Instance Browser</b> .....    | <b>933</b> |
| <b>Cell Library Browser</b> .....    | <b>934</b> |
| <b>DRC Browser</b> .....             | <b>935</b> |
| <b>Config Data Browser</b> .....     | <b>938</b> |
| <b>Transcript</b> .....              | <b>942</b> |
| <b>Wave Viewer</b> .....             | <b>944</b> |
| <b>Text/HDL Viewer</b> .....         | <b>947</b> |
| <b>IJTAG Network Viewer</b> .....    | <b>949</b> |
| <b>iProc Viewer</b> .....            | <b>952</b> |
| <b>Diagnosis Report Viewer</b> ..... | <b>953</b> |

## Hierarchical Schematic

This section describes features available only in the **Hierarchical Schematic** tab. The Hierarchical Schematic shows a representation of the design before design flattening. Use this feature to explore the structure of your design and the relationships between the elements making up the design.

---

### Note



“[Schematics](#)” on page 876 describes schematic features common to both the Hierarchical Schematic and the Flat Schematic.

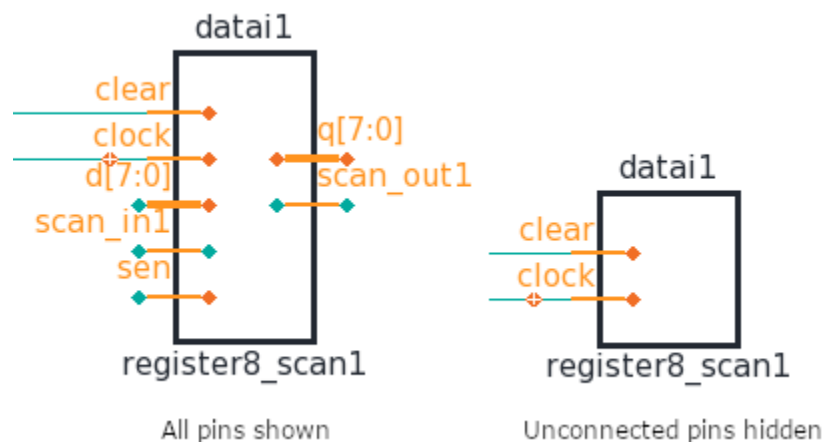
---

You can use the tools available from the [Toolbar](#), or you can use the context menu to do the following:

- Show the selected objects in the Flat Schematic.
- Show the HDL definition for the selected object in the [Text/HDL Viewer](#).
- Show the HDL instantiation for the selected object in the Text/HDL Viewer.

- Show all pins, or hide unconnected pins, on the selected instance (as shown in [Figure 12-55](#)).
- Show the internal connectivity of a hierarchical instance.
- Show an instance as parent in the Instance Browser.
- Delete all selected objects from the schematic.
- Delete all unselected objects from the schematic.
- Trace backward, or trace between two selected pins or bus pins.
- Report gates on selected pins.
- Copy the post-synthesis names of selected objects to the system clipboard.
- Copy the active names of selected objects to the system clipboard. Active names are compatible with Tessent introspection commands.

**Figure 12-55. Hierarchical Instances With Pins Shown and Hidden**




---

**Note**

 The menu items **Show HDL definition** and **Show HDL instantiation** are available only when the Tessent Shell context is set to “dft -rtl.” The **Show HDL definition** menu item is available for library cells in all contexts when the library cell file is available.

---



---

**Note**

 By default, all pins are initially shown on an instance when it is added to the Hierarchical Schematic if the total number of single pins and bus ports is 16 or fewer.

---

The following tables relate specifically to the Hierarchical Schematic:

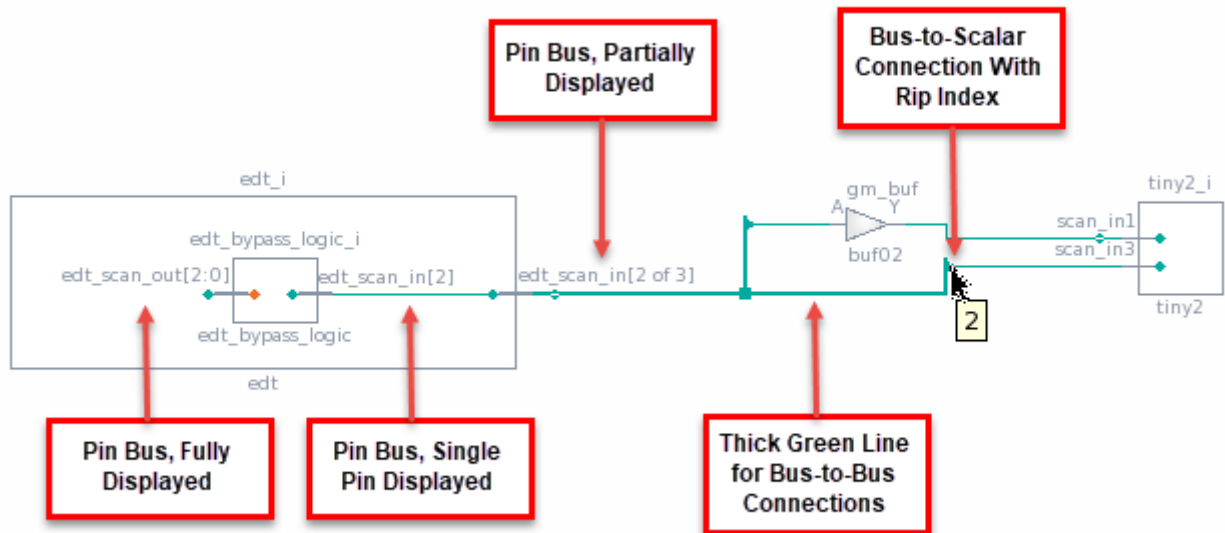
- **Instances Table** — Displays information about the instances contained within a selected instance. For more information, see “[Instances \(Hierarchical Schematic\)](#)” on page 895.



- **Nets Table** — Displays information about the nets of a selected instance. For more information, see “[Nets \(Hierarchical Schematic\)](#)” on page 896.

Figure 12-56 shows several symbols specific to the Hierarchical Schematic:

**Figure 12-56. Hierarchical Schematic Symbols**



Design hierarchy is shown with rectangles surrounding the objects in a design instance, and library cells are shown without rectangles and filled with gray. The leaf-level instance names are displayed above each instance, and the module or cell names are displayed below them.

## Buses

The Hierarchical Schematic can display either complete or partial buses, using the following naming conventions:

- **Fully displayed** — Shows the full range. For example, “scan[7:0]”.
- **Partially displayed** — Indicates the count of displayed pins versus the total pin count. For example, “scan[2 of 8]”.
- **Single pin of a bus** — The index of the displayed pin is shown. For example, “scan[3]”.

You can add or remove bus pins in the display from any table that lists hierarchical pins, such as the Pins table in the Instance Browser or the Pins context table at the bottom of the Hierarchical Schematic.

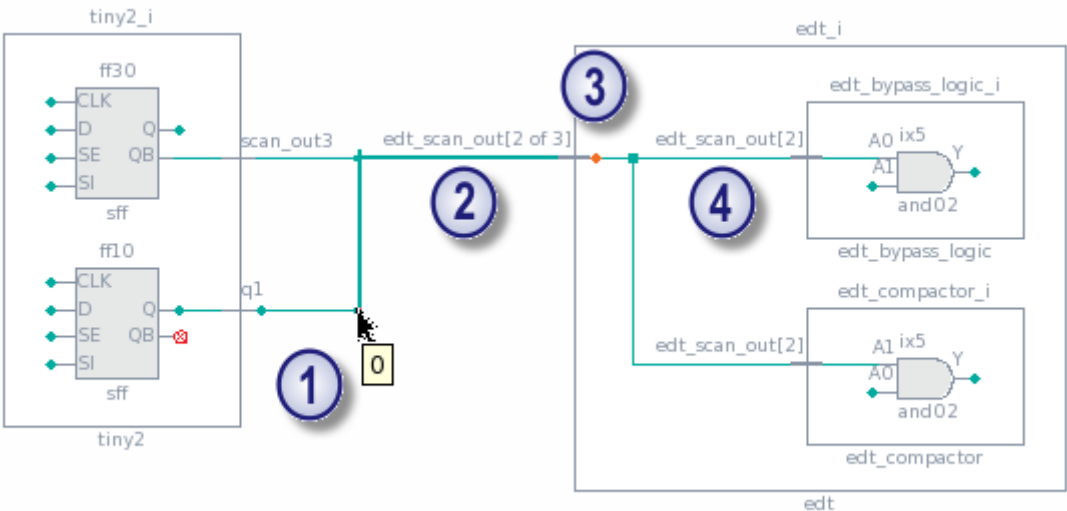
You can also add bus pins with the following procedure:

1. Select the instance in the Hierarchical Schematic. The instance name is displayed in the address bar (see [Figure 12-9](#) on page 878).
2. Append text to the instance name to add the bus pins. For a single pin, include the pin index (for example, "data[3]" in "mytop/parent\_module/this\_instance/data[3]"). To designate the complete bus, use the "\*" wildcard character (for example, "data\*").
3. Press Enter. The address bar shows both the pins and the nets. Click the pins line. The pins are added to the instance in the schematic.

The display of bus pins affects the operation of the tracing function, because the tracer starts from displayed bus pins. Add individual pins for tracing to the Hierarchical Schematic, or apply filtering in the tracer table for the pins of interest.

Tessent Visualizer uses several naming and symbol conventions for buses in schematic windows. See [Figure 12-57](#) for examples.

**Figure 12-57. Bus Tracing in the Hierarchical Schematic**



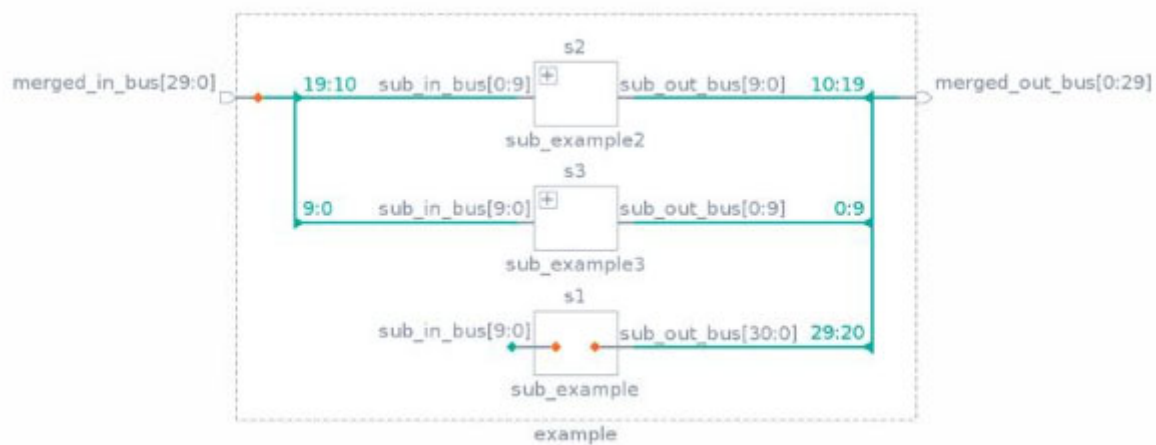
| Item | Description                                                                                                |
|------|------------------------------------------------------------------------------------------------------------|
| ①    | Bus-to-scalar connection with rip index ("0" in this case) shown with mouse pointer hover over rip symbol. |
| ②    | Generated name for a partial bus (two bits of a three-bit bus in this case).                               |
| ③    | Orange diamond representing multiple paths not shown from this bus pin.                                    |

| Item | Description                                                                              |
|------|------------------------------------------------------------------------------------------|
| ④    | Generated name of a partial bus with a single bit displayed (bit index 2, in this case). |

Bus ranges can be split into sub-ranges as shown in [Figure 12-58](#). In this case, ten bits of the 31-bit output bus from instance s1 are merged with all ten bits of the output buses from instances s2 and s3 to form the 30-bit merged\_out\_bus.

Bus rip indices can also be shown directly on the hierarchical schematic by choosing this feature from the options menu.

**Figure 12-58. Bus Sub-Ranges**



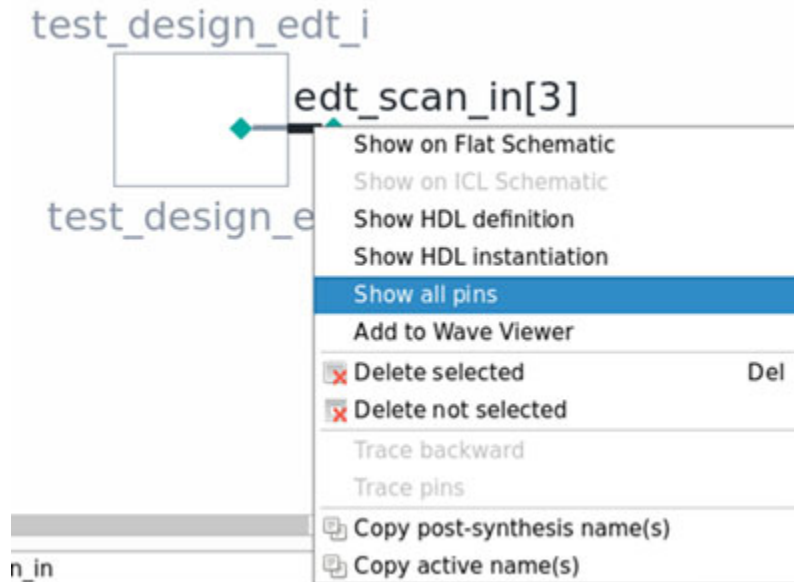
#### Note



This schematic does not indicate which ten bits of the output bus from s1 are connected. Use the Tracer table to determine connectivity.

To show all bus pins when only a subset is displayed, select one or more bus pins and right-click to access the **Show all pins** item from the context menu:

**Figure 12-59. Showing All Bus Pins in the Hierarchical Schematic**

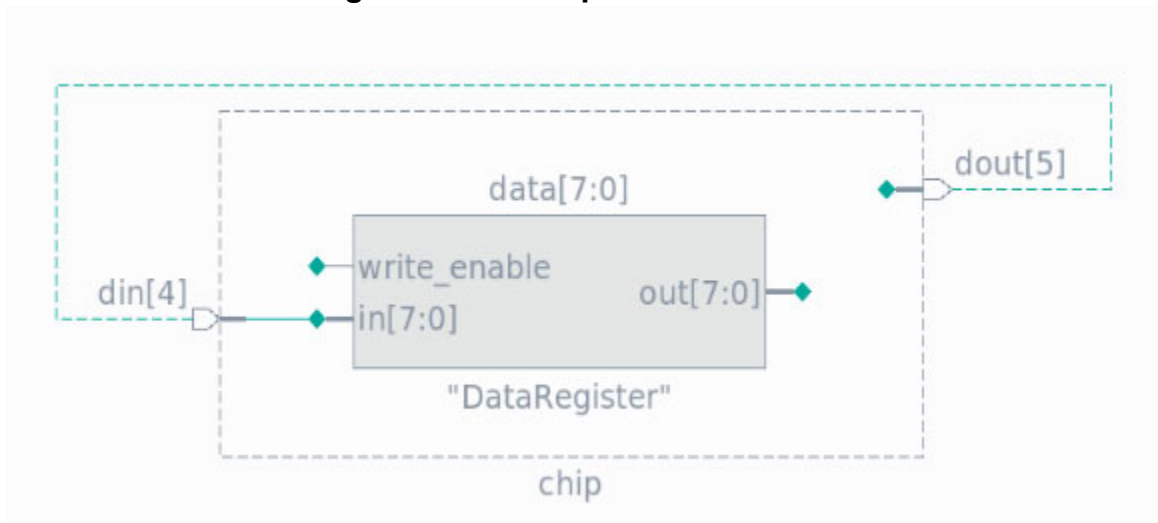


If the width of the bus is greater than 256 bits, this feature is disabled. To show all pins in this case, use the Pins Table.

## Loopbacks

Loopbacks defined with [add\\_loadboard\\_loopback\\_pairs](#) are indicated on the Hierarchical Schematic as shown in [Figure 12-60](#). All indicators such as trace markers, the tracer table, and net visualization are available on primary inputs and outputs with these connections.

**Figure 12-60. Loopback Connection**

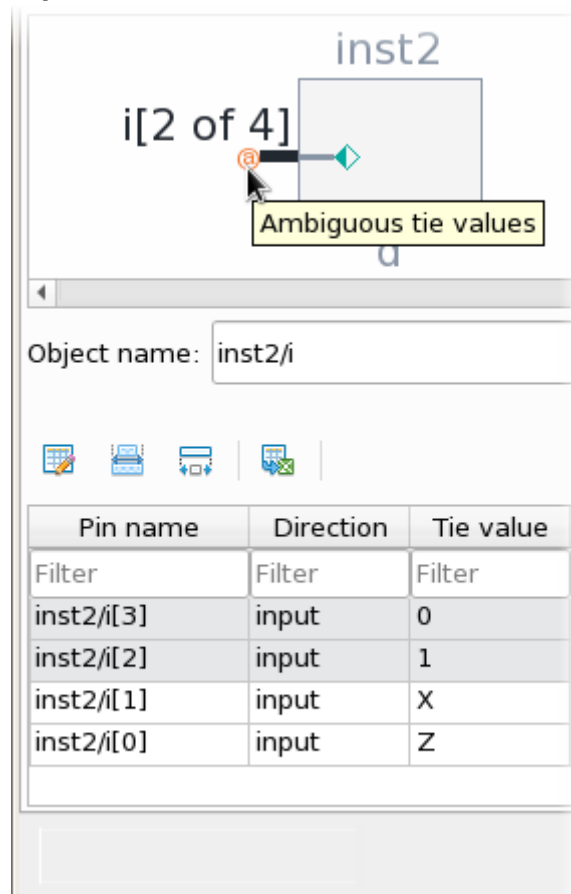


The tool also displays loopbacks on the [ICL Schematic](#).

## Other Symbols

Additional marker symbols indicate other hierarchical schematic features, such as tied-value indicators. A tied-value indicator is displayed when a net is tied to a logic value or an unknown. See [Figure 12-61](#) for an example, and [Table 12-3](#) on page 886.

**Figure 12-61. Example of a Tied-Value Marker With Associated Pin Table**



## Flat Schematic

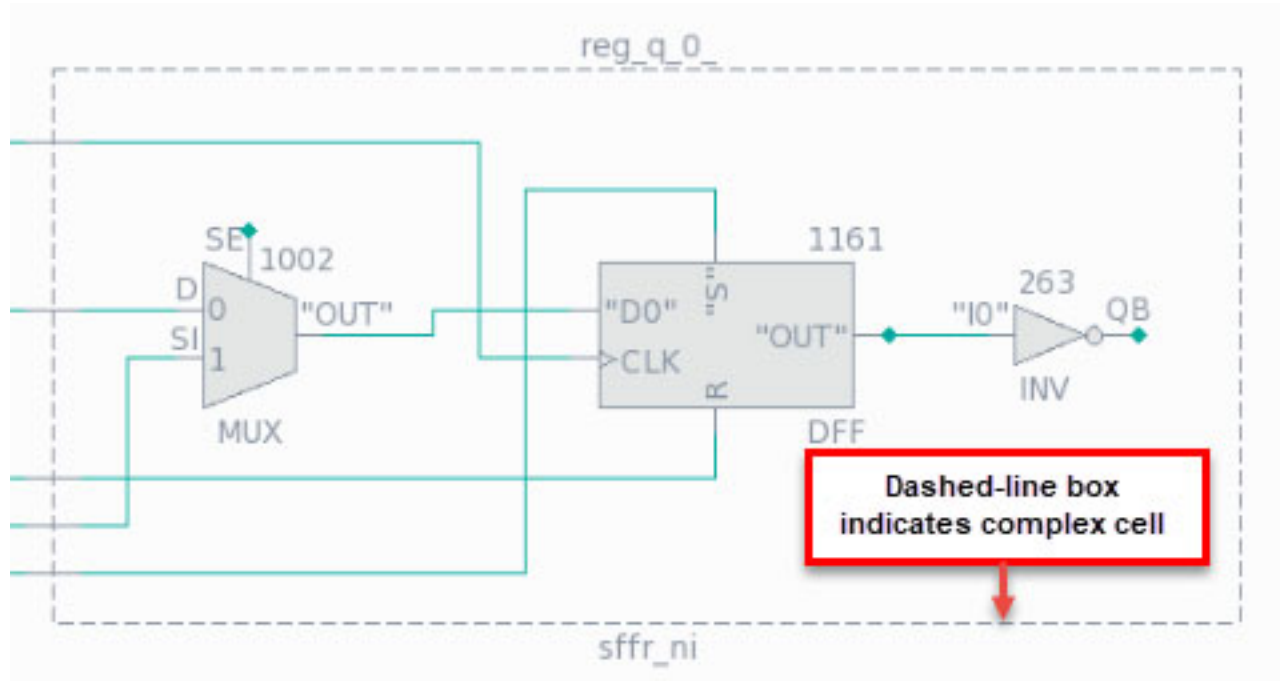
The Flat Schematic shows a representation of the design after design flattening. Use the Flat Schematic to analyze and debug issues when the details required for analysis are not available in the Hierarchical Schematic. For example, simulation data and functional representations of hierarchical and library cells are available in the Flat Schematic.

Library and hierarchical cells in the design appear in the Flat Schematic as gates. Cells that consist of a single gate are displayed with the conventional symbols for those gates.

[Figure 12-62](#) shows cells that consist of multiple gates, displayed either with or without a

bounding box showing the grouping. You can control this display option using the “Cell grouping” checkbox in the **Options** menu available from the toolbar.

**Figure 12-62. Flat Schematic (Cell Grouping On)**



You can right-click the dashed-line bounding box and use the context menu to delete the instance from the Flat Schematic, show it in the Hierarchical Schematic, or show all gates within the grouping.

#### Note

In a library cell bounding box, a NAND function that has at least one (but not all) inputs inverted is displayed as an OR symbol. A NOR function that has at least one (but not all) inputs inverted is displayed as an AND symbol.

#### Note

When cell grouping is turned on, instances shown on the schematic are identified by their module or primitive names (below the symbol) and leaf-level cell library names (above the symbol). When it is turned off, they are identified by their module/primitive names and their gate IDs from the flat model.

Many features of the Flat Schematic are similar to those in the [Hierarchical Schematic](#). However, one difference is that you cannot control the display of pins on most instances, with the exception of RAM and ROM instances. You can add or remove pins for these instances by using the popup menu or by dragging and dropping from the pins table.

Pin names on instances appear in quotation marks when the pin is not part of the design hierarchy.

## ICL Schematic

The Tessent Visualizer ICL Schematic is similar to the standard Tessent Visualizer Hierarchical Schematic, including a main schematic window, a small overview window, and an attributes table for selected instances. There are, however, several differences and additional features.

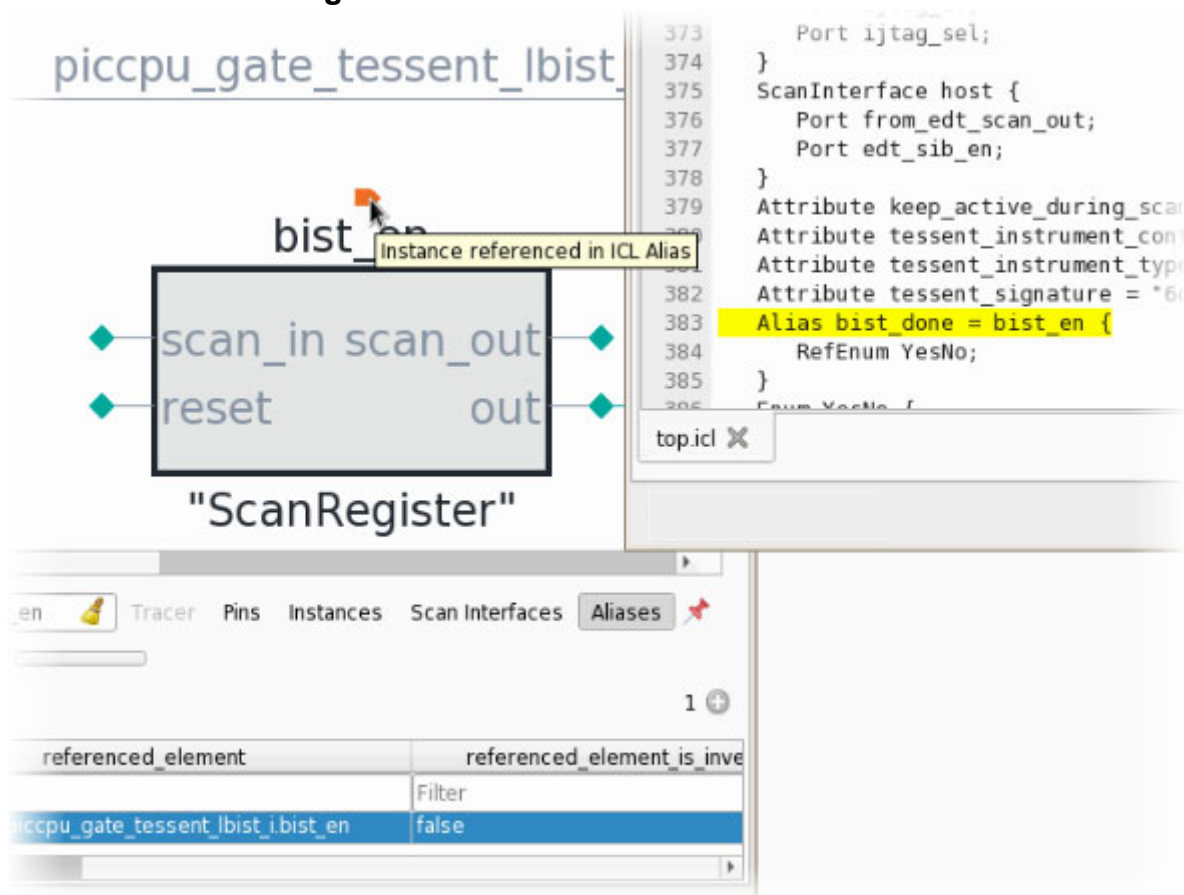
### Context Tables

In addition to the Tracer, Pins, and Instances context tables, the ICL Schematic includes two additional context tables:

- **Scan Interfaces** — Shows the same information for selected instances as the Scan Interfaces table in the ICL Instance Browser does.
- **Aliases** — Shows aliases that reference the selected pin or instance.

Instances and pins referenced by aliases are marked on the ICL Schematic with an orange or blue port symbol, as shown in [Figure 12-63](#). As with the Hierarchical and Flat Schematics, objects can be cross-referenced to their text definitions from the ICL Schematic, as shown in the figure.

Figure 12-63. Alias in the ICL Schematic



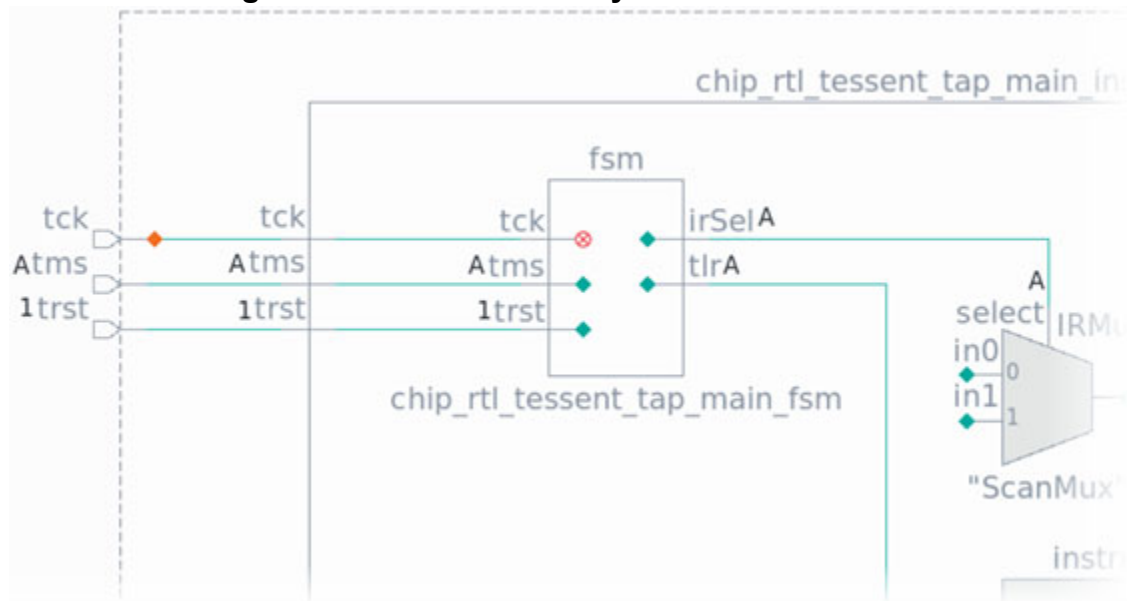
In addition to the options also available in the Hierarchical Schematic options menu, the ICL Schematic options menu includes options specific to ICL.

The “Show simulation values” option annotates the ICL Schematic object’s pins with their simulation data. The tool displays a 0 or 1 to indicate the current ICL simulator value. The tool encloses the 0 and 1 bits within parentheses (“()”) to indicate that these bits are part of a symbolic variable and their values are patchable at run time. See [Retargeted Symbolic Variables](#) in the “[Tessent IJTAG User’s Manual](#)” for more information.

The tool displays a question mark (“?”) to indicate that the ICL simulator value is not relevant and that the tool has deferred determining the value until it is needed. The value may be determined by the IJTAG retargeter after an iApply command when it must be determined to fulfill the current iApply targets. The remaining unknown values are resolved when the close\_pattern\_set command finalizes the pattern.

The tool displays the value as an “A” for automatically determined signals. The tool considers these signals to be always correct. See [Figure 12-64](#) for an example of automatically determined signal values on the TAP’s FSM module.

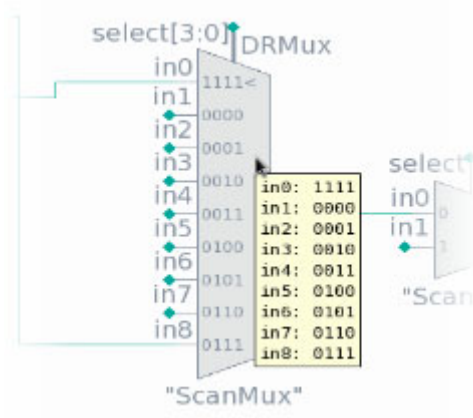


**Figure 12-64. Automatically Determined Values**

The “Highlight scan path” option displays the Highlight Scan Path toolbar. Refer to [“JTAG Network Viewer”](#) on page 949 for complete information on this toolbar. Select the “Show scan pins by default” option to display scan in and scan out ports on instances, or the “Show SSN pins by default” option to display the SSN pins on the instances. This enables you to easily trace the scan path or SSN datapath.

## Mux Select Values

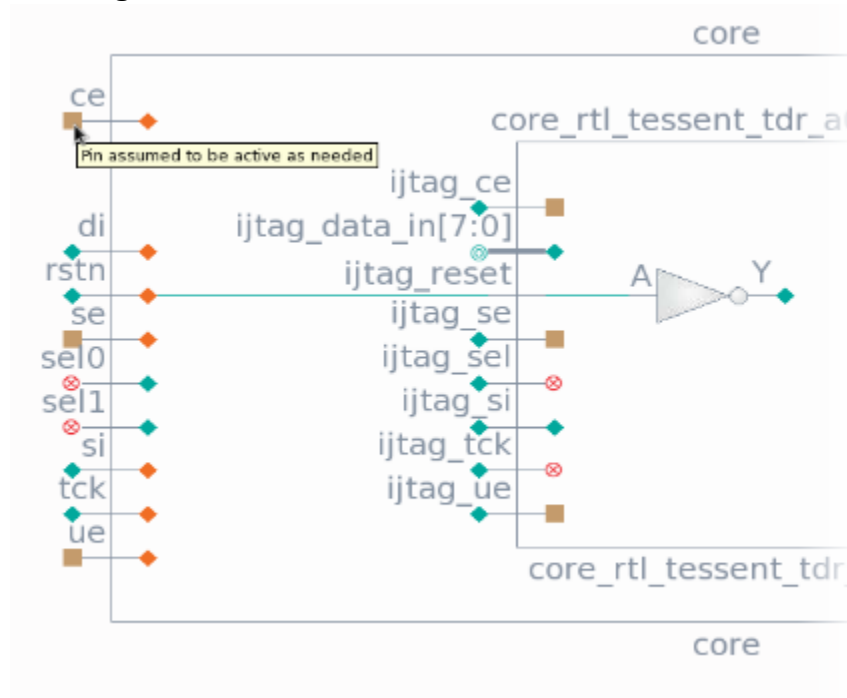
Hover the mouse pointer over a mux in the ICL Schematic to view its select values in a tooltip. The tool also displays the active mux input with “<” on the symbol.

**Figure 12-65. Mux With Select Values Displayed**

## Implicit Connections and Driven-As-Needed Pins

The tool displays a tan square marker on a hierarchical pin on the outside of a block to indicate that the pin is not constrained by the parent module and is assumed to be driven or active as needed.

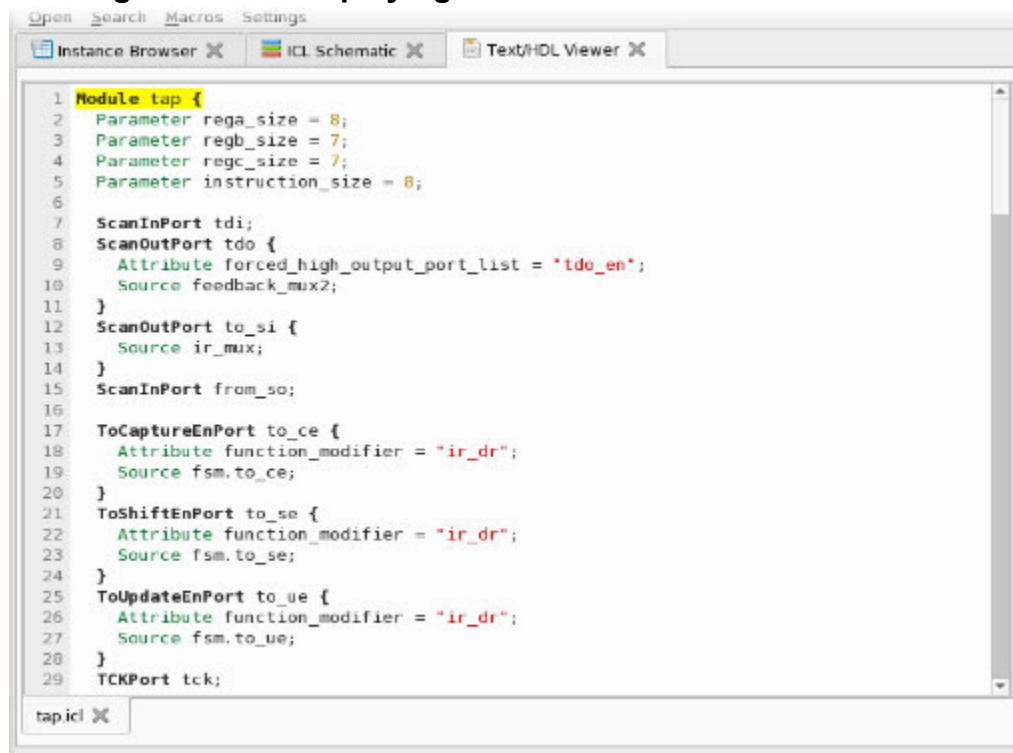
**Figure 12-66. Pins Driven or Active As Needed**



The tool displays a tan square marker on a hierarchical pin on the inside of a block to indicate that every element in the block normally required to be connected to the indicated pin is implicitly connected to the signal associated with that pin.



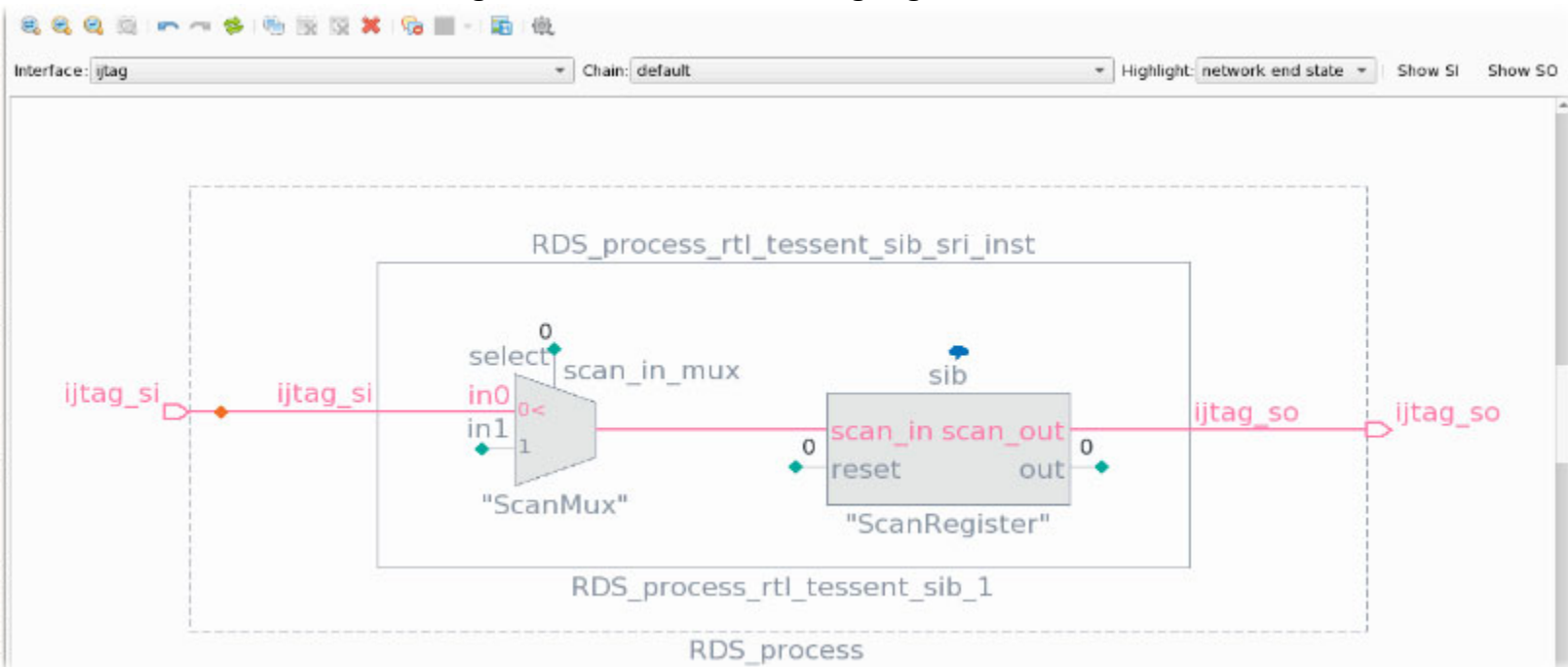
**Figure 12-68. Displaying the Definition of an ICL Instance**



## Tracing the Current Scan Path

To trace the scan path, select both the scan input and scan output pins of interest in the ICL Schematic or IJTAG Network window, click one with the right mouse button, and select “Trace pins along scan path.”.

Figure 12-69. Traced and Highlighted Scan Path



Choose between “network end state” and “last used” in the “Highlight” menu.

## Loopbacks

The tool displays loopbacks created with [add\\_loadboard\\_loopback\\_pairs](#) on the ICL Schematic. See “[Hierarchical Schematic](#)” on page 915 for complete information.

## Displaying Bus Pins

When a subset of pins is displayed for a bus, you can display all the pins by selecting the bus (or buses), right-clicking, and choosing the “Show all pins” feature. See “[Hierarchical Schematic](#)” on page 915 for complete information.

# Instance Browser

Use the Instance Browser to navigate your design and obtain reports about its elements. It is analogous to a file browser.

The Instance Browser consists of four major elements:

- **Tree View** — A pane that shows the instances in your design hierarchy directly. Click the pane header repeatedly to sort the tree in ascending alphabetical, descending alphabetical, and default ordering.
- **Child Instances table** — A pane that displays information about the children of the instance currently selected in the tree view or the Address Bar.

- **Address Bar** — A text field that shows the instance currently selected in the tree view and the parent instance for instances displayed in the Child Instances table. Type an instance name in this field to select an instance without using the Tree View or Child Instances table.
- **Pins and DRC Violations tables** — A table that lists the pins or DRC violations for the instance(s) currently selected in the Child Instances table.

---

**Note**



For huge designs, it may be more efficient to select an instance and then use the filtering functions in the Child Instances table to find objects and explore the hierarchy from the search results.

---

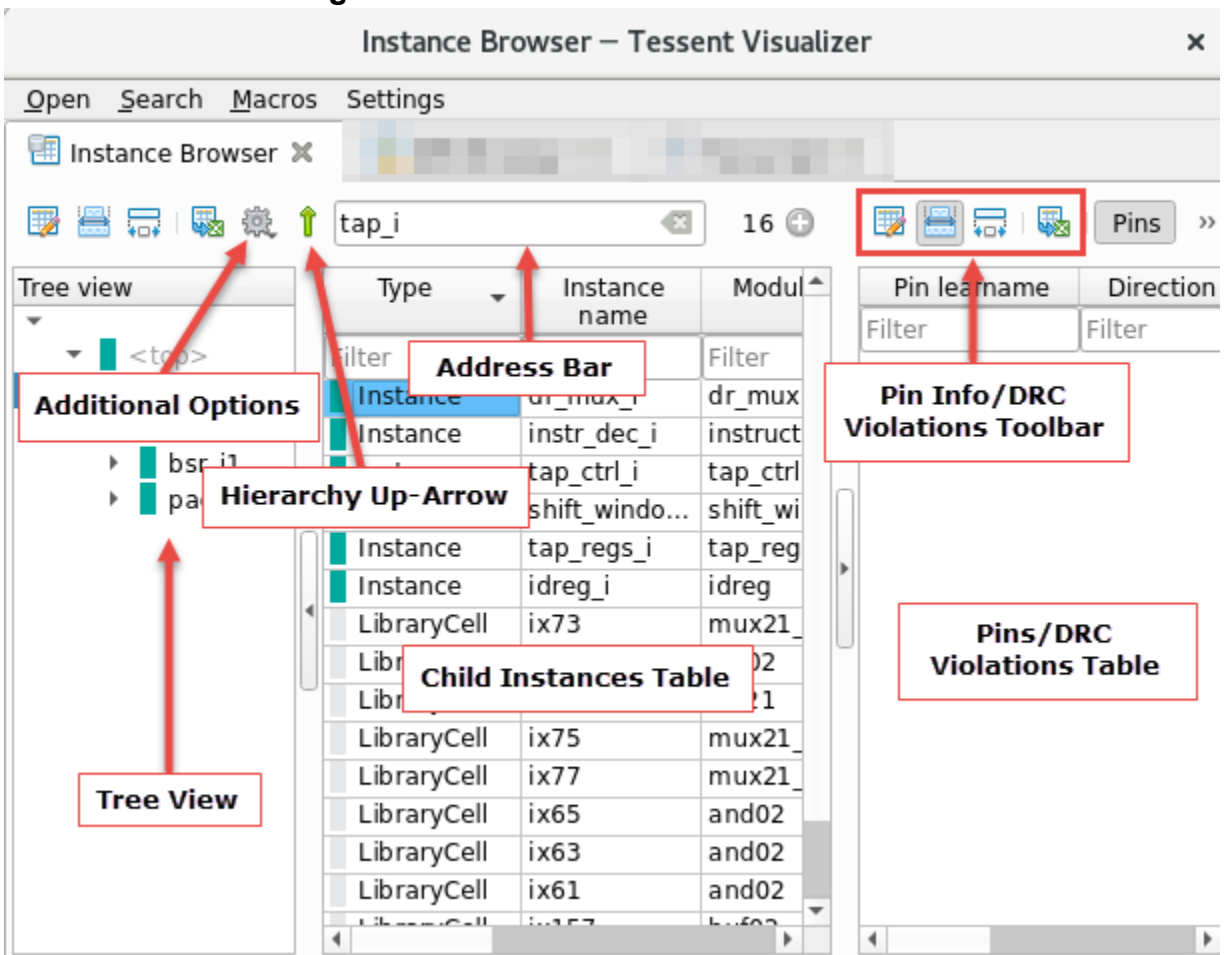
Click an instance in the tree view to display that instance's contents in the Child Instances table. Double-clicking an instance in the tree view expands the node in the tree. First, the node is selected, and it becomes the parent instance. If the double-clicked node is different than the one previously selected, the Child Instances table is reloaded with the children of the new parent.

Select an instance of any type in the Child Instances table to display its pins or DRC violations in the appropriate context table. Double-click, or use the Enter key, to open the next level of hierarchy (if any) in the Child Instances table. Press the Backspace key to navigate up one level of hierarchy.

Show instances of cells or modules in the Hierarchical Schematic, the Flat Schematic, or the Text/HDL Viewer by right-clicking the cell or module in the tree view or Child Instances table and choosing the appropriate option from the popup menu.

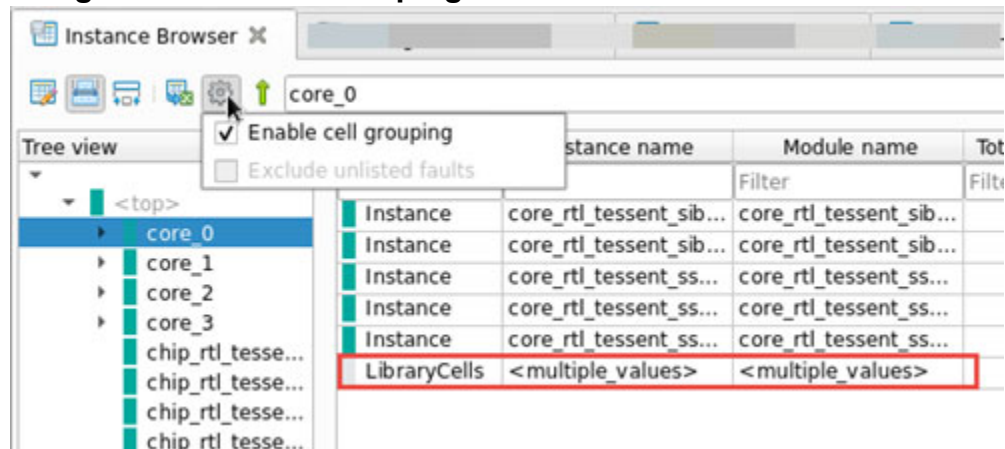
Copy the active or post-synthesis name of an instance to the system clipboard by right-clicking the name in the tree view and choosing the appropriate option from the popup menu. Active names are compatible with Tessent introspection commands; post-synthesis names are not.

Figure 12-70. Instance Browser Elements



The Child Instances table and Pin/DRC Violations table include toolbars that provide the common controls described in “[Tables](#)” on page 867, as well as buttons you can use to switch between pin information and DRC violations in the table. The Child Instances table has a **Hierarchy-Up** button to navigate to the parent of the currently selected instance. In addition, the Child Instances table has a gear icon for additional **Options**. If you click the gear icon, a dropdown list with options displays (See [Figure 12-71](#) on page 932). If you select the “Enable cell grouping” checkbox, the tool enables cell grouping and groups all the library cells at the current hierarchy level into a single new node. The tool creates this new node with the instance type “LibraryCells”. The fault statistics for all the cells in the current library are collected under this node. If you select the “Exclude unlisted faults” checkbox, the tool runs the [report\\_statistics](#) command with the `-hierarchy` and `-exclude_unlisted_faults` switches to calculate coverage statistics.

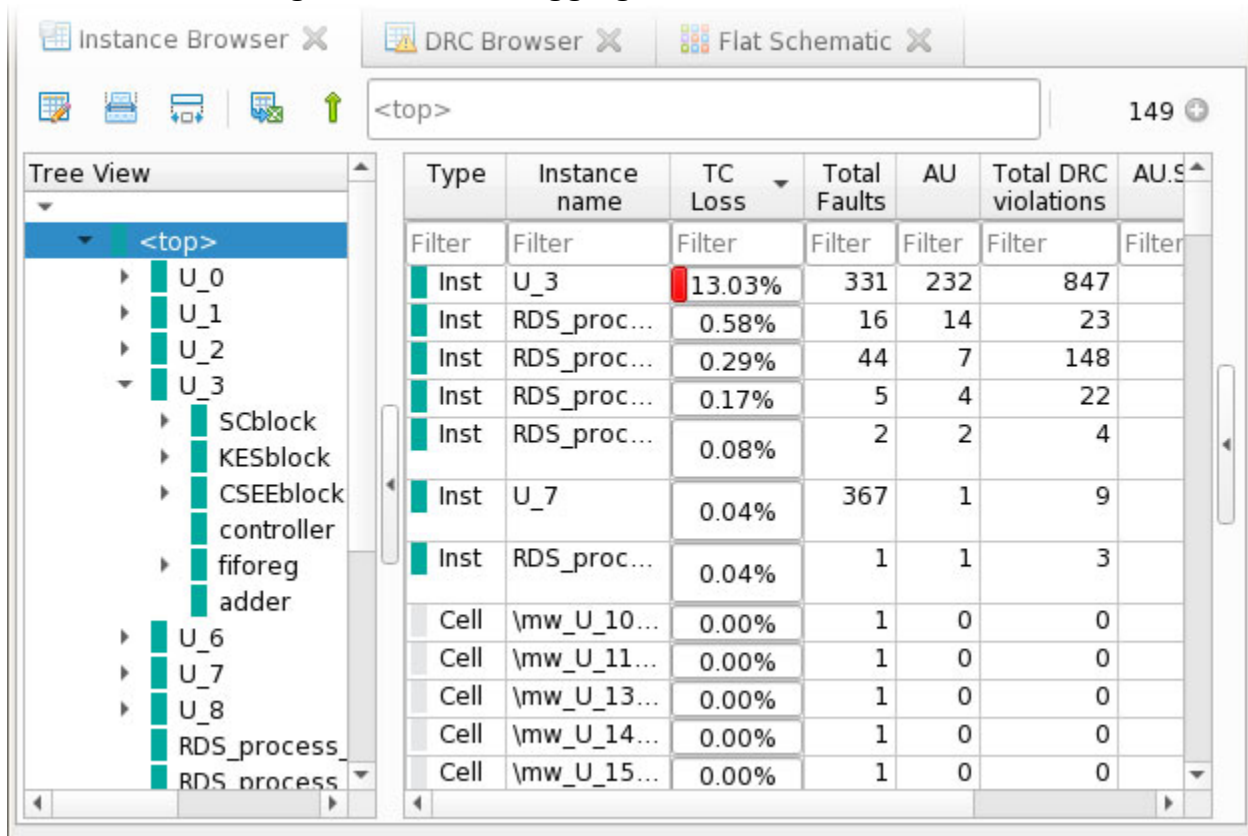
Figure 12-71. Cell Grouping in Hierarchical Instance Browser



**Note**  
The tool provides cell grouping only for the Hierarchical Instance Browser.

To debug directly in the instance browser, use the sorting and filtering functions as shown in [Figure 12-72](#) to investigate faults and design rule violations, and to discover instances with problematic coverage statistics. In this example, the “test coverage loss” and “total DRC violations” columns have been added and sorted, and a significant problem can be seen in the U\_3 instance.



**Figure 12-72. Debugging in the Instance Browser**

## ICL Instance Browser

The Tessent Visualizer ICL Instance Browser is similar in form and function to the standard Tessent Visualizer Instance Browser for hierarchical designs. There are, however, several additional features.

### ICL Instances

You can examine several types of ICL objects in the ICL Instance Browser:

- Instance
- ScanMux
- DataMux
- ClockMux
- ScanRegister
- DataRegister
- LogicSignal

- OneHotDataGroup
- OneHotScanGroup
- Primitive

See “[Instrument Connectivity Language](#)” in the *Tessent Shell Reference Manual* for information on these and other ICL objects.

## ICL Instance Browser Actions

You can perform several operations on objects in the ICL Instance Browser:

- **Show ICL definition** — Similar to the **Show HDL definition** action in the hierarchical design Instance Browser. Displays the ICL object’s definition in the Text/HDL Viewer.
- **Show on ICL Schematic** — Similar to the **Show on Hierarchical Schematic** action in the hierarchical design Instance Browser. Displays the ICL object on the ICL Schematic.
- **Show on Hierarchical Schematic** — Displays the hierarchical design object associated with the ICL object in the Hierarchical Schematic, if a mapping from the ICL object to a hierarchical design object exists.
- **Copy name(s)** — Copies the name of the selected ICL object to the system clipboard.

## Context Tables

The ICL Instance Browser includes two context tables: one for the ICL instance pins, and another for the ICL scan interfaces. These interactive tables display information about the ICL objects. You can use these tables to select and manipulate these objects. Use the Columns and Filters editor to control the information that is displayed in these tables.

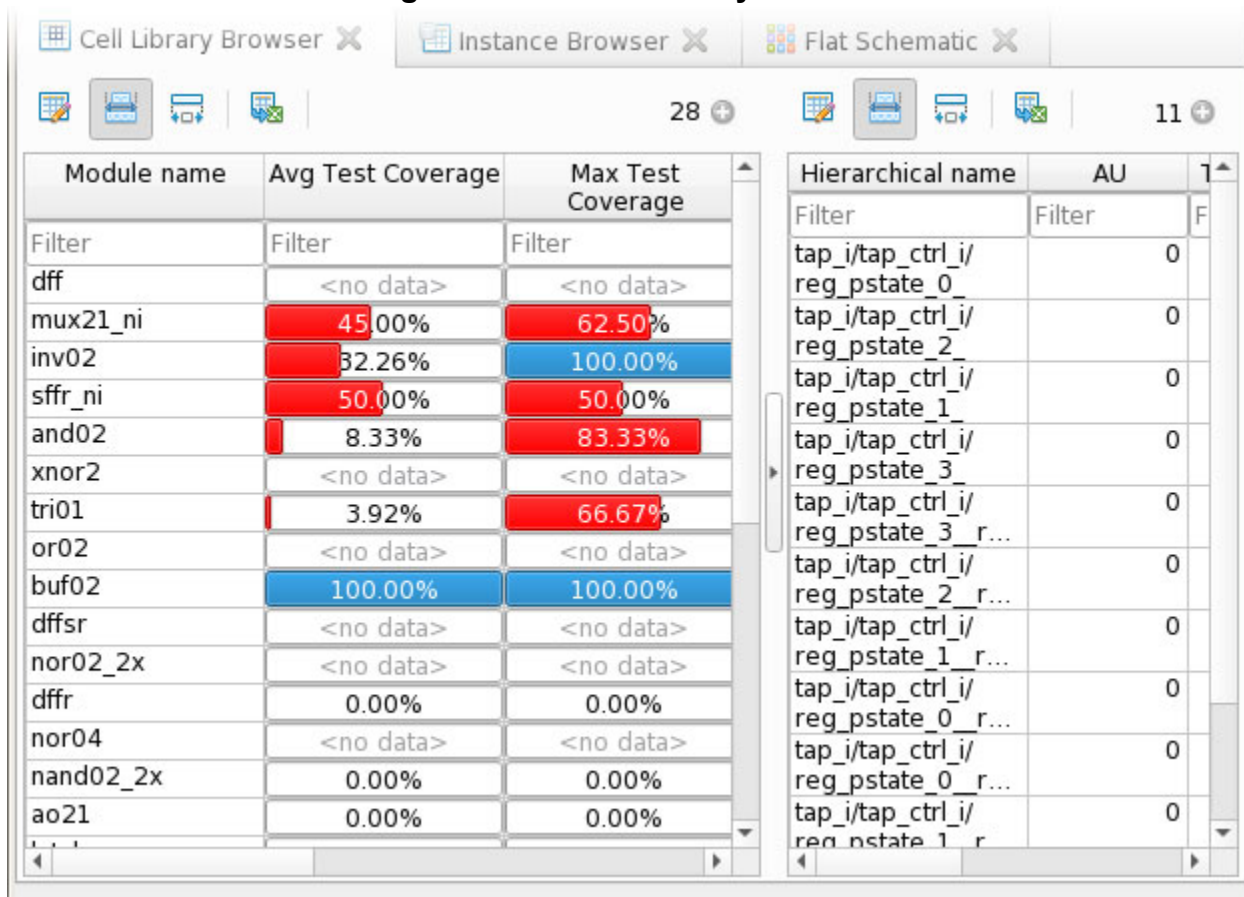
# Cell Library Browser

Use the Cell Library Browser to debug test coverage and fault coverage loss from the perspective of library cells.

The Cell Library Browser consists of two panes:

- **Left Pane** — Lists the available cell libraries. This pane can include data such as the module name, statistics including the number of instances in the design, and fault coverage.
- **Right Pane** — Shows the instantiations of those library cells in the design. You can add columns to display data such as the attributes of those instantiations, the hierarchical name, the parent module, and fault information.

Figure 12-73. Cell Library Browser



| Module name | Avg Test Coverage | Max Test Coverage |
|-------------|-------------------|-------------------|
| Filter      | Filter            | Filter            |
| dff         | <no data>         | <no data>         |
| mux21_ni    | 45.00%            | 62.50%            |
| inv02       | 32.26%            | 100.00%           |
| sffr_ni     | 50.00%            | 50.00%            |
| and02       | 8.33%             | 83.33%            |
| xnor2       | <no data>         | <no data>         |
| tri01       | 3.92%             | 66.67%            |
| or02        | <no data>         | <no data>         |
| buf02       | 100.00%           | 100.00%           |
| dffsr       | <no data>         | <no data>         |
| nor02_2x    | <no data>         | <no data>         |
| dffr        | 0.00%             | 0.00%             |
| nor04       | <no data>         | <no data>         |
| nand02_2x   | 0.00%             | 0.00%             |
| ao21        | 0.00%             | 0.00%             |

| Hierarchical name                      | AU     | 1 |
|----------------------------------------|--------|---|
| Filter                                 | Filter | F |
| tap_i/tap_ctrl_i/<br>reg_pstate_0      | 0      |   |
| tap_i/tap_ctrl_i/<br>reg_pstate_2      | 0      |   |
| tap_i/tap_ctrl_i/<br>reg_pstate_1      | 0      |   |
| tap_i/tap_ctrl_i/<br>reg_pstate_3      | 0      |   |
| tap_i/tap_ctrl_i/<br>reg_pstate_3_r... | 0      |   |
| tap_i/tap_ctrl_i/<br>reg_pstate_2_r... | 0      |   |
| tap_i/tap_ctrl_i/<br>reg_pstate_1_r... | 0      |   |
| tap_i/tap_ctrl_i/<br>reg_pstate_0_r... | 0      |   |
| tap_i/tap_ctrl_i/<br>reg_pstate_0_r... | 0      |   |
| tap_i/tap_ctrl_i/<br>reg_pstate_0_r... | 0      |   |
| tap_i/tap_ctrl_i/<br>reg_pstate_1_r    | 0      |   |

Click in a row in the left pane to show the instantiation information for that cell type in the right pane. Right-click an instance name in the table in the right pane to get access to all actions available for hierarchical instances.

## DRC Browser

The DRC Browser is a tabular reporting tool for exploring the rule violations in your design and ICL data models. Customize the browser to group and filter DRC violations by various criteria.

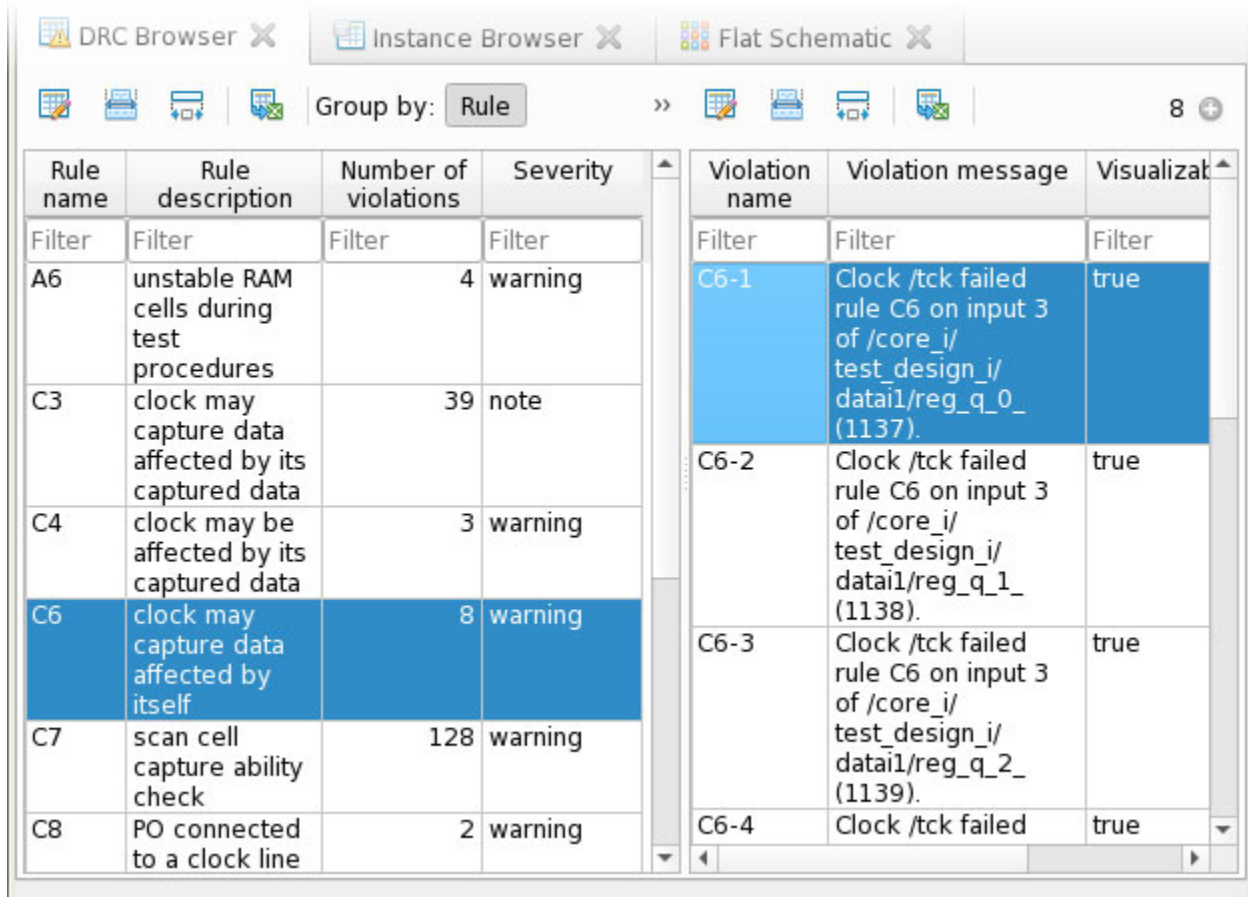
The DRC Browser consists of two panes. The left pane enables you to group the violation list by rule name (the default), instance, or module.

The right pane shows specific information about the violations associated with the rule, instance, or module selected in the left pane, such as:

- Violation name
- Description of the violation
- Indication of whether the violation is visualizable

If nothing is selected in the left grouping table, the right table displays all violations currently registered in Tessent Shell.

**Figure 12-74. DRC Browser With Data Grouped By Rule**



| Rule name | Rule description                                     | Number of violations | Severity       |
|-----------|------------------------------------------------------|----------------------|----------------|
| Filter    | Filter                                               | Filter               | Filter         |
| A6        | unstable RAM cells during test procedures            | 4                    | warning        |
| C3        | clock may capture data affected by its captured data | 39                   | note           |
| C4        | clock may be affected by its captured data           | 3                    | warning        |
| <b>C6</b> | <b>clock may capture data affected by itself</b>     | <b>8</b>             | <b>warning</b> |
| C7        | scan cell capture ability check                      | 128                  | warning        |
| C8        | PO connected to a clock line                         | 2                    | warning        |

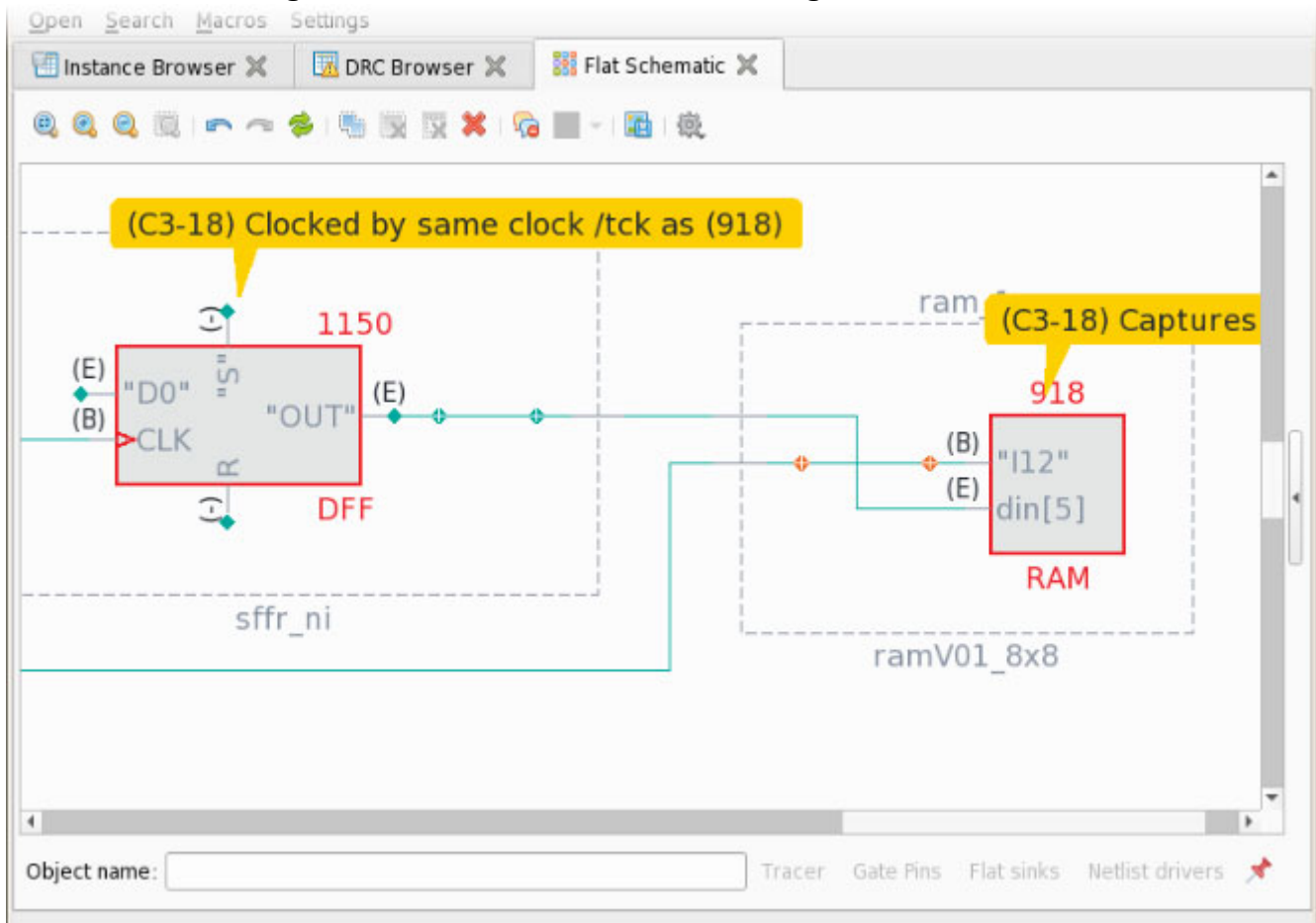
| Violation name | Violation message                                                                             | Visualizati |
|----------------|-----------------------------------------------------------------------------------------------|-------------|
| Filter         | Filter                                                                                        | Filter      |
| <b>C6-1</b>    | <b>Clock /tck failed rule C6 on input 3 of /core_i/ test_design_i/ data1/reg_q_0_ (1137).</b> | <b>true</b> |
| C6-2           | Clock /tck failed rule C6 on input 3 of /core_i/ test_design_i/ data1/reg_q_1_ (1138).        | true        |
| C6-3           | Clock /tck failed rule C6 on input 3 of /core_i/ test_design_i/ data1/reg_q_2_ (1139).        | true        |
| C6-4           | Clock /tck failed                                                                             | true        |

If you group by instance or module, data for the violations in the selected instance or module is displayed.

You can display the specifics for a visualizable rule violation in a schematic or the Text/HDL Viewer by double-clicking a row in the right pane of the DRC Browser, or by right-clicking and choosing the appropriate item from the popup menu. You cannot visualize all DRC violations.

#### Note

 This method is equivalent to issuing the `analyze_drc_violation` command on the Tessent Shell or Transcript command line.

**Figure 12-75. Flat Schematic Showing a DRC Violation**

You can view the results of analyzing many ICL DRC violations in Tessent Visualizer by double-clicking them in the DRC Browser window or using `analyze_drc_violation` on the command line.

The following DRC violations are visualized in the Text/HDL Viewer:

- ICL1-ICL28 (except ICL8 and ICL24)
- ICL92-ICL96
- ICL100
- ICL113
- ICL136

The following DRC violations are visualized in the ICL Schematic:

- ICL68
- ICL76

- ICL77
- ICL80
- ICL111
- ICL117-ICL119
- ICL134
- ICL140

## Related Topics

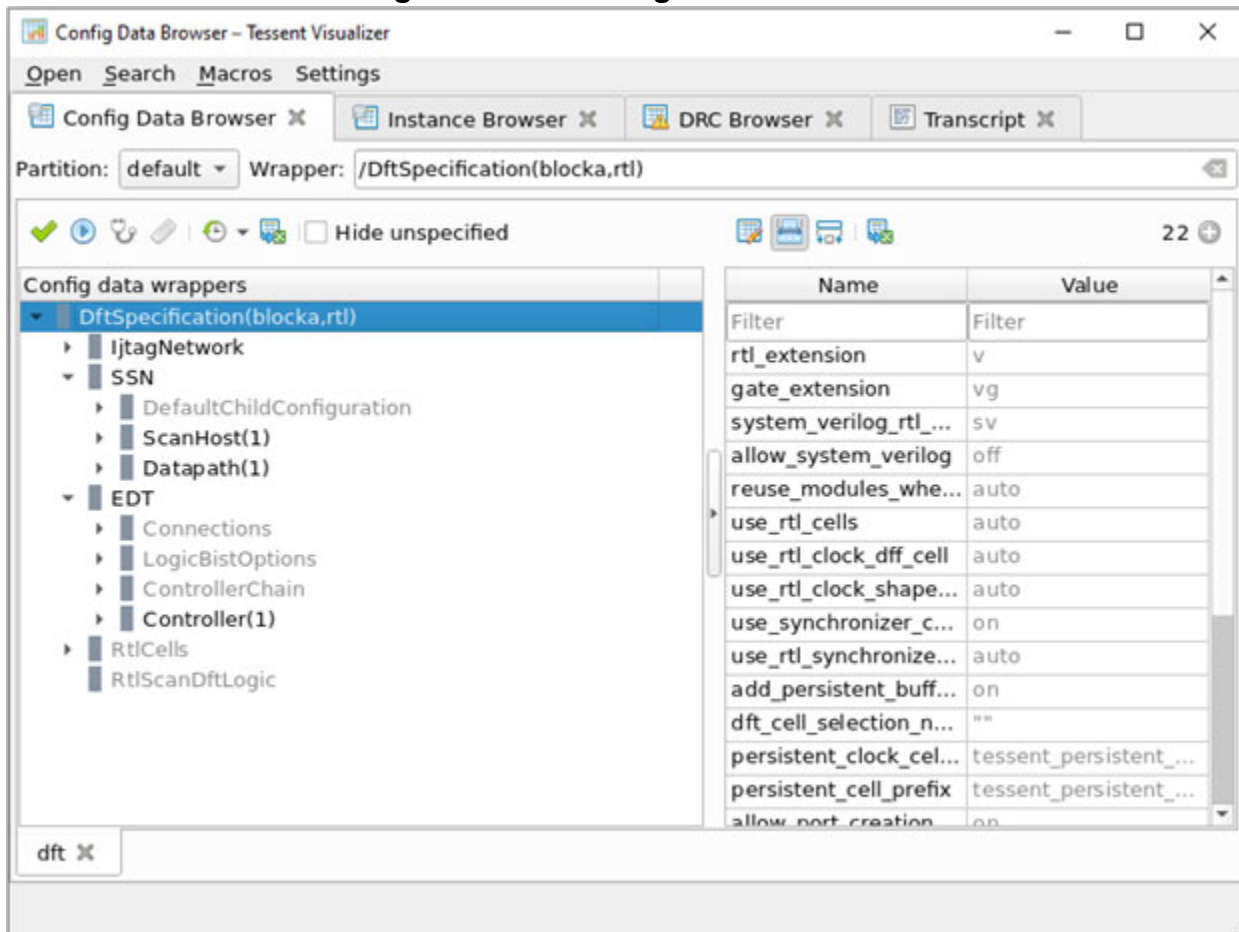
[analyze\\_drc\\_violation](#)

# Config Data Browser

The Config Data Browser enables you to view and edit configuration specification trees that are loaded in the current Tessent Shell session.

The Config Data Browser includes a top toolbar and a dedicated tab for each top-level wrapper in the configuration data that has been added for visualization in the GUI. Each tab has two panes, as shown in [Figure 12-76](#).

Figure 12-76. Config Data Browser



When you open the Config Data Browser with the **Open** menu action, no wrappers or tabs are displayed unless a previous session was restored. Use the navigation bar in the top toolbar to add new tabs with configuration trees. First, choose a partition from the Partition dropdown list in the toolbar. Then, choose a wrapper by typing or pasting a hierarchical name in the navigation bar. You can use tab completion while typing a wrapper name. If the wrapper has already been loaded into the GUI, the existing tab is raised and the specified wrapper is selected. If the wrapper has not yet been loaded into the GUI, a new tab is opened and the specified wrapper is selected in the tree view.

Alternately, you can use the `add_config_tab` command to open a configuration tree in the Config Data Browser.

The tree view pane enables you to browse wrappers in the configuration hierarchy, and the table pane displays the properties for the currently selected wrappers. The property table includes features common to other tabular views in Tessent Visualizer such as filtering and exporting.

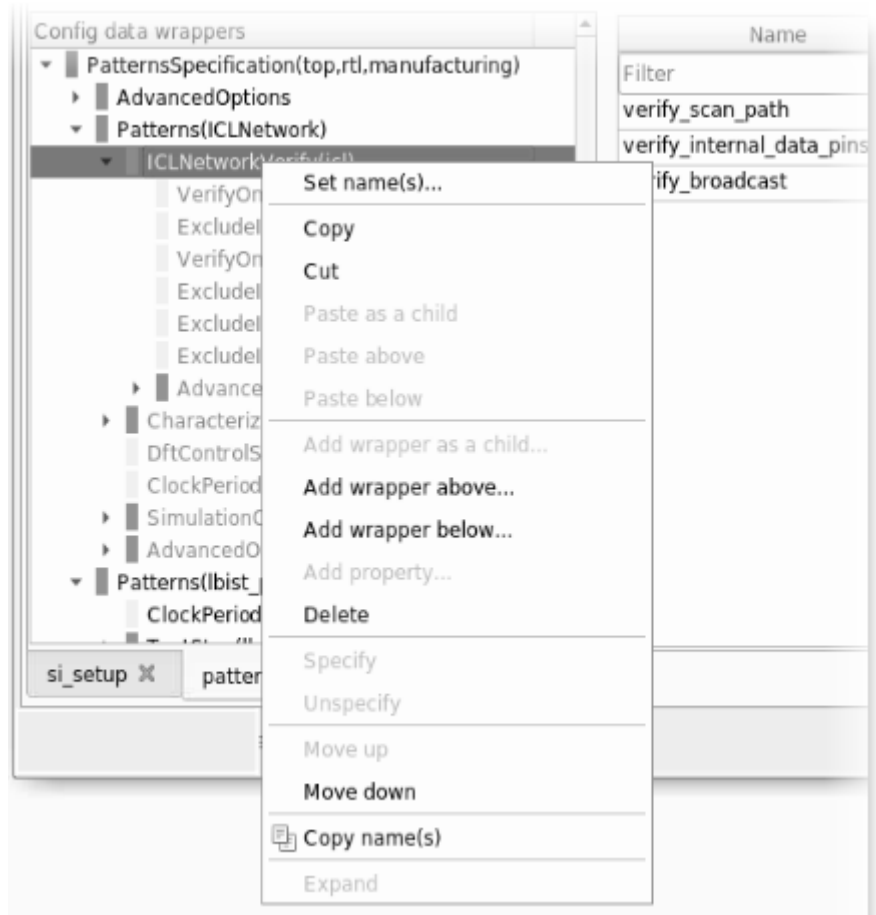
Unspecified wrappers and properties with unspecified default values appear in gray text. User-specified wrappers and properties appear in black text, unless there is an error. If an error is



attached to a wrapper or property, the text appears in red, and the parent wrappers of the node with the error appear in orange.

The “default” partition enables you to edit the configuration in the GUI. Right-click a property and choose one of the options to edit it or copy its contents. You can also double-click a property name or a value to open a popup to change the value. You can also use the context menu on a wrapper to add data properties or repeatable properties to the wrapper, or to add new wrappers above as parents or below as children of the wrapper, as shown in the following figure.

Figure 12-77. Wrapper Editing in the Context Menu



Choose **Set name(s)** to change the names of named wrappers (ICLNetworkVerify is shown in the figure). Other actions enable you to cut or copy and paste wrappers, delete wrappers, or move wrappers to new positions. Use **Specify** and **Unspecify** with some wrapper types to mark them as user-specified or to reset their properties to their default values, respectively.

The toolbar in the view pane for the wrapper tree includes several action buttons, as described in the following table.



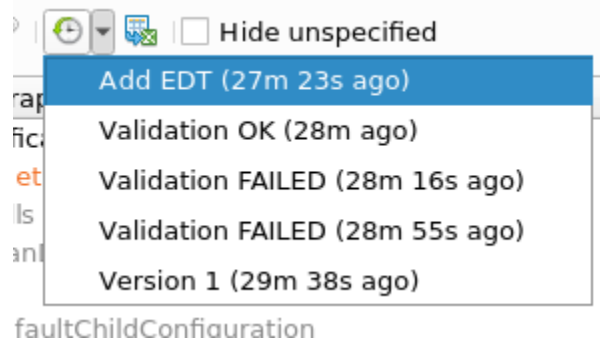
Table 12-4. Config Data Browser Buttons and Actions

| Button                                                                              | Action                                                                                                                                                                                                                                                                                                                                              |
|-------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|    | <b>Validate</b> — validates the correctness of the specified configuration. This action is available for the DftSpecification and PatternsSpecification wrappers. It is equivalent to running the “ <a href="#">process_dft_specification -validate</a> ” or “ <a href="#">process_patterns_specification -validate</a> ” command in Tessent Shell. |
|    | <b>Process/Execute</b> — runs the same commands as the “Validate” action, except that it excludes the “-validate” option. In SiliconInsight, also simulates the test patterns. See the <i>Tessent SiliconInsight User’s Manual</i> for details.                                                                                                     |
|    | <b>Diagnose</b> — this is a SiliconInsight-specific action for the PatternsSpecification() wrapper only. See the <i>Tessent SiliconInsight User’s Manual</i> for details.                                                                                                                                                                           |
|    | <b>Clear status</b> — this is a SiliconInsight-specific action for the PatternsSpecification() wrapper only. See the <i>Tessent SiliconInsight User’s Manual</i> for details.                                                                                                                                                                       |
|    | <b>Configuration snapshot</b> — saves a snapshot of the GUI configuration data for the specified root wrapper. The current list of saved snapshots is available in the dropdown list.                                                                                                                                                               |
|  | <b>Export config data</b> — saves the displayed configuration data to a file using the write_config_data command.                                                                                                                                                                                                                                   |
| <input type="checkbox"/> Hide unspecified                                           | <b>Hide unspecified</b> — removes all unspecified wrappers from the current view.                                                                                                                                                                                                                                                                   |

The tool stores a snapshot of the GUI configuration data for the root wrapper displayed in the Config Data Browser. To save the current state of configuration, click  and specify the snapshot name in the dialog box that opens. The tool also automatically saves a snapshot of the current configuration whenever you click **Validate** or **Process/Execute**. In this case, the tool creates a concatenated name using the action performed and the outcome of the action, and saves the configuration snapshot with this name.

The following figure demonstrates how the tool displays the current list of saved snapshots:

**Figure 12-78. List of Configuration Snapshots**



If you want to restore the GUI configuration to one saved previously, select the required configuration from the dropdown list. The tool restores the Config Data Browser to the saved configuration. The configuration you select becomes the latest saved configuration and moves to the top of the dropdown list with its updated timestamp.

The tool can distinguish between the snapshots you save and the ones that are automatically saved. If you try to save a snapshot of the current configuration and if the previously saved configuration is the same, the tool updates the name and timestamp of the previously saved snapshot and moves it to the top of the dropdown list.

The number of snapshots of each type per configuration tree is limited to 10. The tool removes the oldest snapshot from the list once the limit is reached. The tool also deletes all saved snapshots when the Tessent Visualizer session is closed ([close\\_visualizer](#) -server) or if the specified root wrapper is removed.

## Related Topics

[add\\_config\\_tab](#)

[delete\\_config\\_tabs](#)

[process\\_dft\\_specification](#)

[process\\_patterns\\_specification](#)

[report\\_config\\_data](#)

[Tessent SiliconInsight User's Manual for Tessent Shell](#)

[write\\_config\\_data](#)

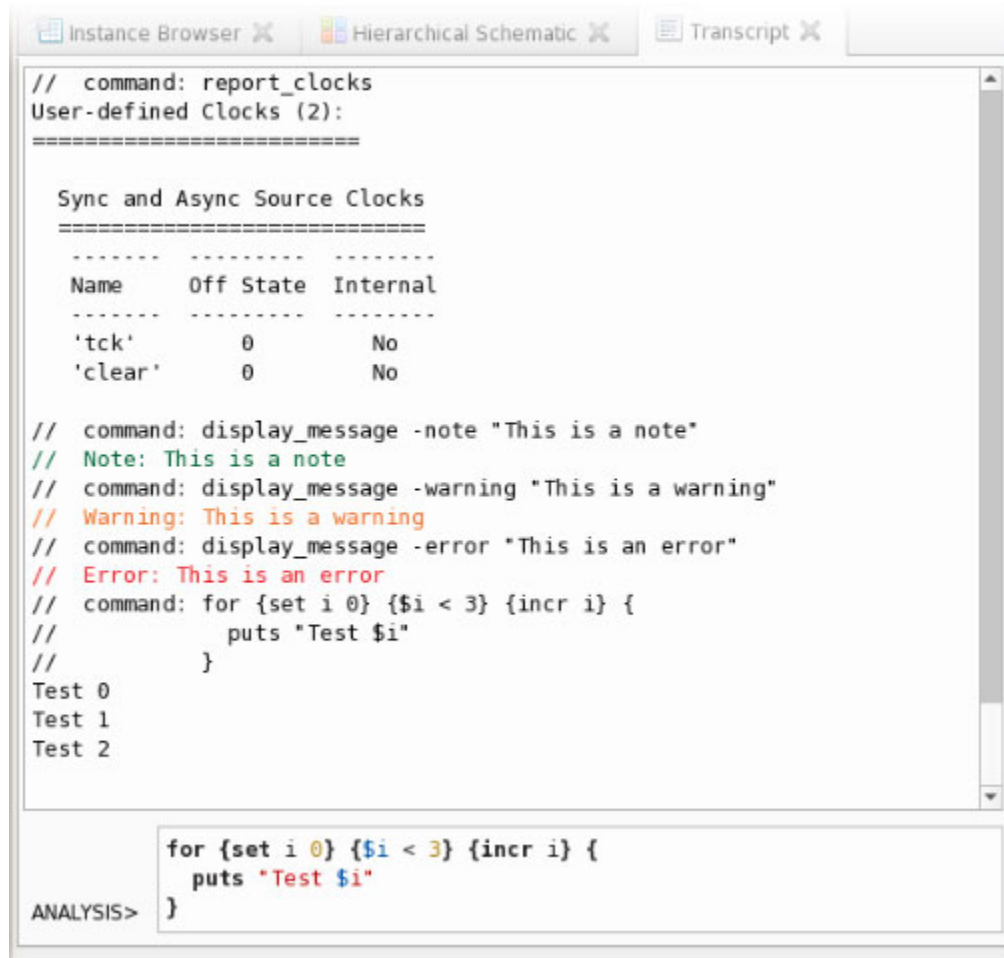
## Transcript

The Tessent Visualizer **Transcript** tab provides access to the Tessent Shell command line and a record of commands used and responses to those commands.

Text in the Transcript is highlighted as shown in [Figure 12-79](#):

- **Green** — informational notes
- **Orange** — warning messages
- **Red** — error messages

**Figure 12-79. Transcript Tab With an Interactive Command Line**



You can issue Tessent Shell commands directly from the Transcript, with the same features as the standard Tessent Shell command line. (For example, command history using the up and down arrow keys, command completion using the Tab key, and multi-line commands.) Commands are syntactically highlighted as you type them.

Press Ctrl+Enter to enable multi-line command mode. The background color of the command entry box changes to light blue to indicate that multi-line command mode is active. Use Shift+Enter to insert a new line and the arrow keys to navigate over the multi-line command. Multi-line commands can be recalled with the up-arrow and edited as shown in [Figure 12-79](#)

## Wave Viewer

---

Use the Tessent Visualizer Wave Viewer tab to debug and analyze Test Setup and Test End data. This tab displays the Test Setup and Test End data as waveforms with their corresponding signal values over time. The Wave Viewer tab contains a toolbar, a signal pane, and a generic plotting widget. It also provides an option to export the waveform data to Value Change Dump (VCD) files. You can use these VCD files with other waveform viewers to further analyze and validate the behavior of your design.

---

### Note



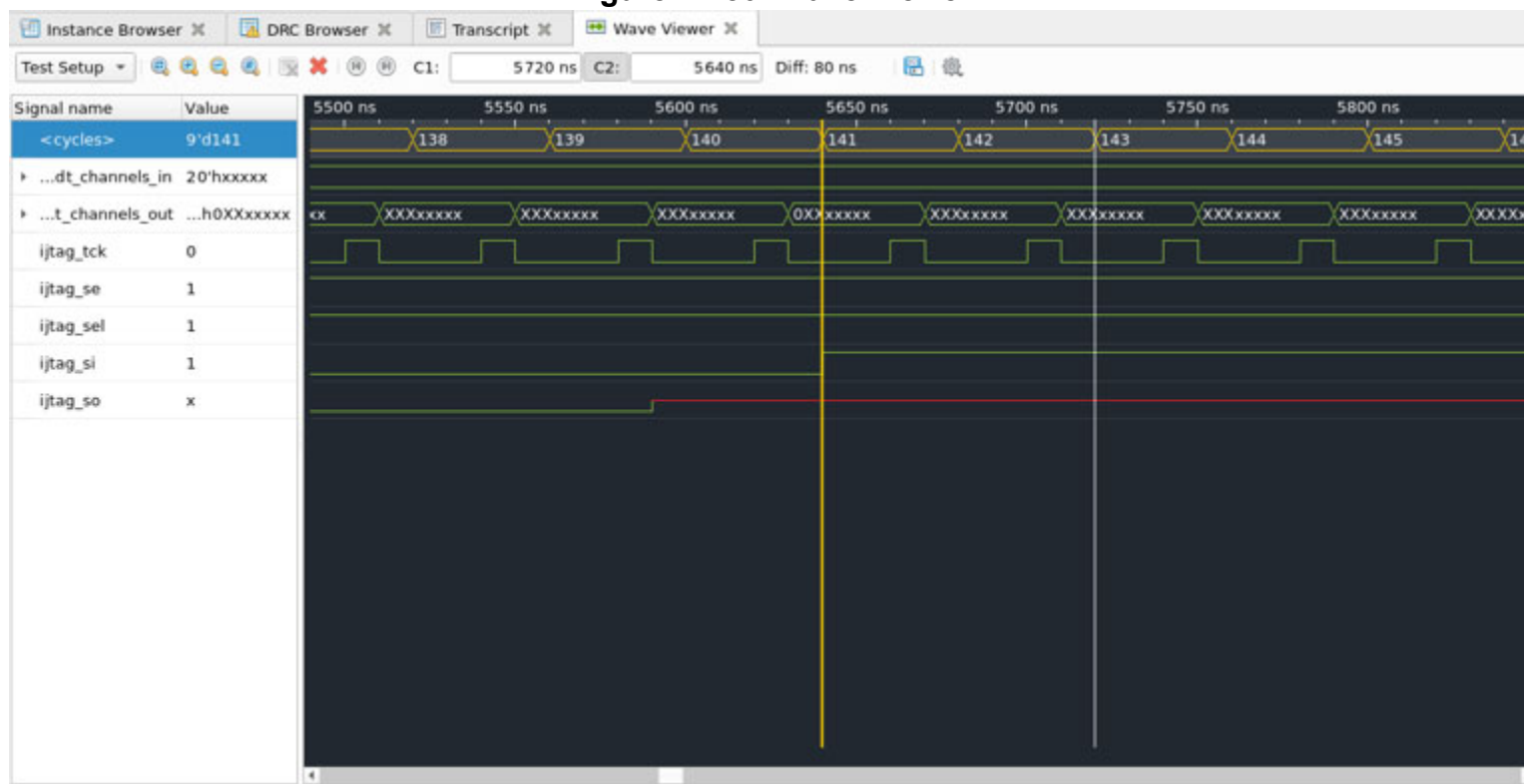
To use the Wave Viewer tab, it is necessary to have a flat model of the design. For any hierarchical pins, the flat model must include their equivalent pins.

---

To open the Wave Viewer tab, right-click the pins of a hierarchical or flat schematic. From the context menu that appears, select **Add to Wave Viewer**. Additionally, you have the option to add pins to the Wave Viewer from various sources such as search tabs, the Instance Browser, or from context tables that display pins or gate pins. When adding pins individually to Wave Viewer, use the **Add to Wave Viewer** option. If any of the selected pins are either a bus or a bi-directional pin, the **Add to Wave Viewer as** option is enabled. The **Add to Wave Viewer** option is also available to add all the signals for an element of type “Instance”, such as a library cell or a flat gate, from schematics and tables.

In addition, there are two more ways to open the Wave Viewer. First, you can use the [add\\_wave\\_viewer\\_signals](#) command to open the Wave Viewer and add signals directly. This command enables you to specify the specific signals you want to view in Wave Viewer. Second, you can access the Wave Viewer directly from the dropdown menu in the Tessent Visualizer window. Click the **Open** menu and select the option to open the Wave Viewer.

Figure 12-80. Wave Viewer



In the Wave Viewer toolbar (see “[Wave Viewer Toolbar](#)” on page 946), you can select either the Test Setup or Test End procedure. If the Test Setup or Test End procedure is not loaded into Tessent Shell, but there is a flat model, you can still add signals to the list, but there is no data on the plots. The tool automatically switches to the “None” option and displays a warning message on the toolbar.

After you have chosen the desired procedure, you can export the waveform data to a VCD file or a dofile. Use the “Export Wave Viewer data” option in the toolbar to create a dofile that you can use to restore the pins that are currently displayed in the Wave Viewer tab.

Right-click a set of signal names in the left pane to access a context menu. This menu displays a list of supported actions that you can choose from to perform various operations on the selected signals. See [Table 12-6](#) on page 947 for a detailed list of the available actions. You can also use the keyboard shortcuts and mouse actions described in “[Keyboard Shortcuts](#)” on page 965.

To remove specific signals from the Wave Viewer, select the signals you want to delete and then utilize the delete actions found in the Wave Viewer toolbar. You could also right-click a set of signals and delete them from the signal pane using the context menu. Another option is to use the [delete\\_wave\\_viewer\\_signals](#) command.

The following topics provide more information about the specific actions that the tool can perform in the Wave Viewer tab:

|                                  |            |
|----------------------------------|------------|
| <b>Wave Viewer Toolbar .....</b> | <b>946</b> |
| <b>Context Menu Actions.....</b> | <b>947</b> |

## Wave Viewer Toolbar

You can instruct the tool to perform specific actions in the Wave Viewer tab using the Wave Viewer toolbar.

**Table 12-5. Wave Viewer Toolbar**

| Button                                                                              | Description                                                                                                                                                                                                         |
|-------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| None/Test Setup/Test End                                                            | Choose whether to use the test setup or the test end procedure. If there is no simulation data present, the tool sets this option to “None”.                                                                        |
|    | Fits all waveforms to view their full range.                                                                                                                                                                        |
|    | Zoom in on the active cursor by a factor of 2x.                                                                                                                                                                     |
|   | Zoom out on the active cursor by a factor of 2x.                                                                                                                                                                    |
|  | Zoom in the region between the C1 and C2 cursors.                                                                                                                                                                   |
|  | Deletes the selected signals.                                                                                                                                                                                       |
|  | Deletes all signals.                                                                                                                                                                                                |
|  | Moves the active cursor to the previous transition for the selected signals.                                                                                                                                        |
|  | Moves the active cursor to the next transition for the selected signals.                                                                                                                                            |
| C1:                                                                                 | Displays the current position of Cursor 1. Use the range field to move C1 to a different location. The range displayed varies based on the data loaded into Wave Viewer. Click <b>C1</b> to highlight or select it. |
| C2:                                                                                 | Displays the current position of Cursor 2. Use the range field to move C2 to a different location. By default, C2 is set at 0 ns. Click <b>C2</b> to highlight or select it.                                        |
| Diff:                                                                               | Calculates and displays the absolute distance between the C1 and C2 cursors.                                                                                                                                        |
|  | Export Wave Viewer data to a VCD file or dofile.                                                                                                                                                                    |
|  | Additional options: <ul style="list-style-type: none"> <li>Show hierarchical signal names — Displays the hierarchical path name for each signal.</li> </ul>                                                         |

## Context Menu Actions

The left signal pane in the Wave Viewer tab supports a context menu with the following actions.

**Table 12-6. Wave Viewer Context Menu**

| Option                         | Description                                                                                                                                                                                                                          |
|--------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Show on Hierarchical Schematic | Displays the selected signal on the Hierarchical Schematic. This option is available only if you select a hierarchical pin.                                                                                                          |
| Show on Flat Schematic         | Displays the selected signal on the Flat Schematic.                                                                                                                                                                                  |
| Insert separator               | Adds a separator below the currently selected signals. The separator acts as a visual marker for grouping between signals. You can drag and place the separator at any location within the signal list.                              |
| Radix                          | Set the default radix for a selected signal. The radix for each signal can be binary, octal, decimal, or hexadecimal. This option is available only if you select a bus signal or the cycles signal.                                 |
| Color                          | Set a color for the selected signal. This option is available even if you select multiple signals. The tool colors an x-value signal red, and a z-value signal blue by default. Test procedure cycles are displayed in yellow color. |
| Delete Selected                | Delete selected signal. You cannot delete a subset of signals of a bus pin or the test procedure cycles signal.                                                                                                                      |
| Copy name(s)                   | Copy the names of the selected signals. This action creates a Tcl list in the system clipboard that can be pasted elsewhere.                                                                                                         |

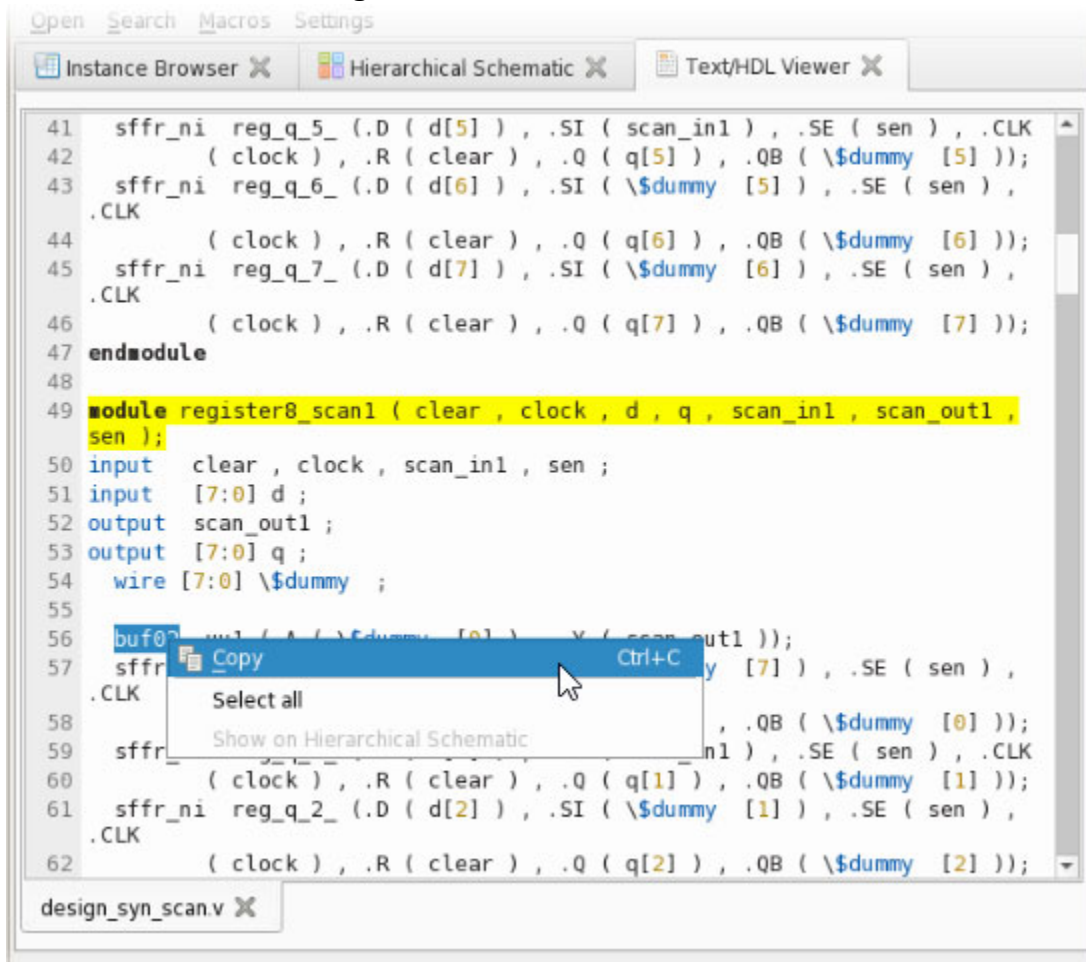
## Text/HDL Viewer

The **Text/HDL Viewer** enables you to examine the HDL description of your design or cell library. The text displayed in this window includes syntax highlighting, and can be copied to the system clipboard for use elsewhere.

Open the Text/HDL Viewer by right-clicking objects in the Instance Browser, Cell Library Browser, or a schematic and choosing **Show HDL definition** or **Show HDL instantiation** from the popup menu. [Figure 12-81](#) shows the Text/HDL Viewer for an object with the appropriate line in the HDL highlighted where the definition or instantiation was found. These actions are available on any hierarchical instance, including library cells, and any table that lists these instances.



Figure 12-81. Text/HDL Viewer



The files opened by the Text/HDL Viewer display on tabs at the bottom of the viewer. To open the current file in an external text editor, right-click the tab and choose **Open in external editor** from the popup menu. Use the [Tessent Visualizer Preferences](#) dialog box to specify the external text editor.

To copy the file path of the current file to the system clipboard, right-click the tab and choose **Copy file path** from the popup menu.

#### Note

When Tessent Shell is in the “dft -rtl” context, you can select a text object in the viewer and display it in the Hierarchical Schematic. To do so, select a text fragment, right-click, and choose **Show on Hierarchical Schematic**.

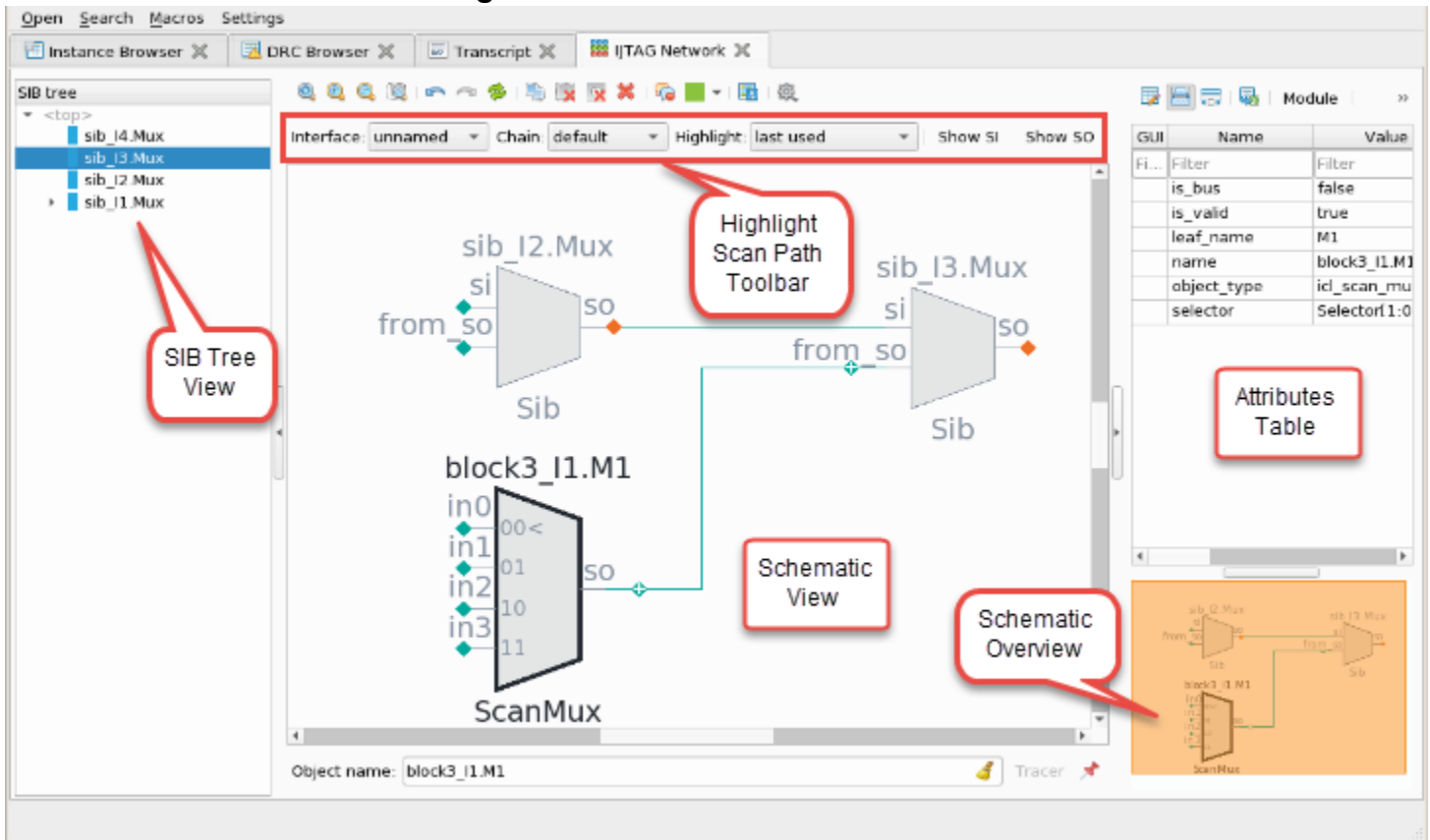


## IJTAG Network Viewer

The IJTAG Network viewer enables you to navigate and introspect the ICL network in a schematic window. The schematic shows a simplified topological view.

The IJTAG Network viewer includes a SIB tree view on the left side of the window and a schematic view in the center of the window. Navigate the schematic with the toolbar, address bar, and the tracer context table, which are similar to those in other types of schematic windows.

**Figure 12-82. IJTAG Network Viewer**



The SIB tree in the left pane displays a hierarchical view of the SIB instances in the network. Select an instance in the SIB tree and choose **Show on IJTAG Network** from the right-click popup menu to display it in the schematic view. The IJTAG Network viewer also includes a schematic overview (similar to the overview in the Flat and Hierarchical Schematics) and a table of attributes for the selected instance in the right pane. The right pane is collapsed by default. Expand it to see the schematic overview and the attributes table.

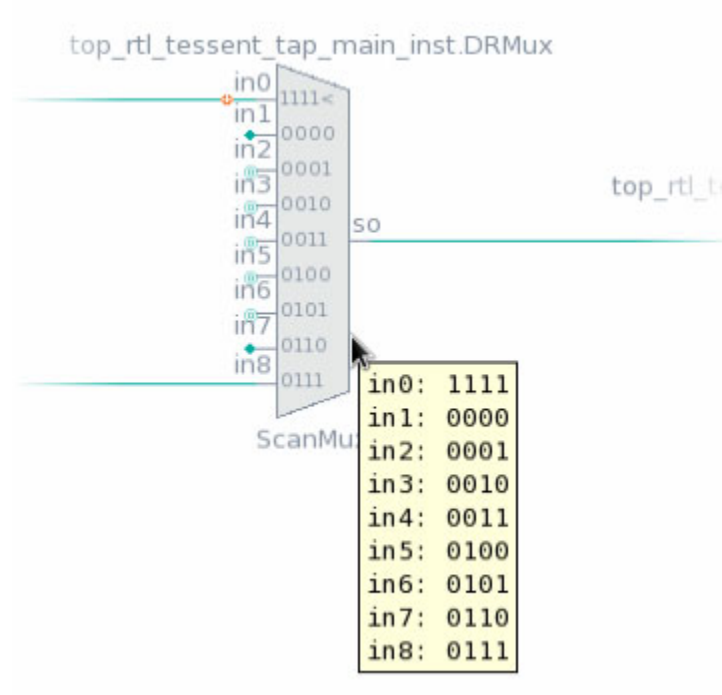
Select an instance in the IJTAG Network viewer and choose **Show on ICL schematic** from the right-click context menu to display it in the ICL Schematic. You can also choose **Trace to scan input**, **Trace to scan output**, and **Trace backward** from this menu. When you select an instance, the IJTAG Network viewer also highlights the parent SIB of the selected instance in the SIB tree.

Select a “from\_so” pin on a SIB instance in the IJTAG Network viewer. Right-click and choose **Expand SIB ring** from the context menu to show the associated IJTAG nodes hosted by the SIB. The tool traces scan paths from the “from\_so” and “si” pins on the SIB to find the closest common point and displays these scan paths with the IJTAG nodes that belong to those paths.

You can also trace to scan inputs and outputs by selecting an instance in the SIB tree and using the right-click context menu.

Hover the mouse pointer over a mux symbol in the IJTAG Network viewer to display the select values, as shown in [Figure 12-83](#). The tool also displays the active mux input with “<” on the symbol.

**Figure 12-83. Mux Select Values and Active Input**



Use the **Options** button (  ) in the toolbar to set options unique to the IJTAG Network:

- **Collapse scan registers** — Select this option to collapse the scan registers. This is similar to “Collapse buffers/inverters” in the Hierarchical or Flat Schematics. See [“Buffer and Inverter Collapsing”](#) in the *Tessent Shell User’s Manual* for further information.
- **Show simulation values** — Select this option to display simulation values in the IJTAG Network viewer. Hover the mouse pointer over the callout associated with a shift register to view the simulation values, as shown in [Figure 12-84](#).

**Figure 12-84. Simulation Values in IJTAG Network Viewer**

- **Highlight scan path** — Select this option to display the Highlight Scan Path toolbar as shown in [Figure 12-82](#) on page 949. Use this toolbar to select the current top interface with its scan chain. This toolbar provides GUI access to the “set\_icl\_network -top\_client\_interface” and “set\_icl\_network -current\_scan\_chain\_number” commands, which are available in the “patterns -silicon\_insight” sub-context.

Choose a scan path highlighting type:

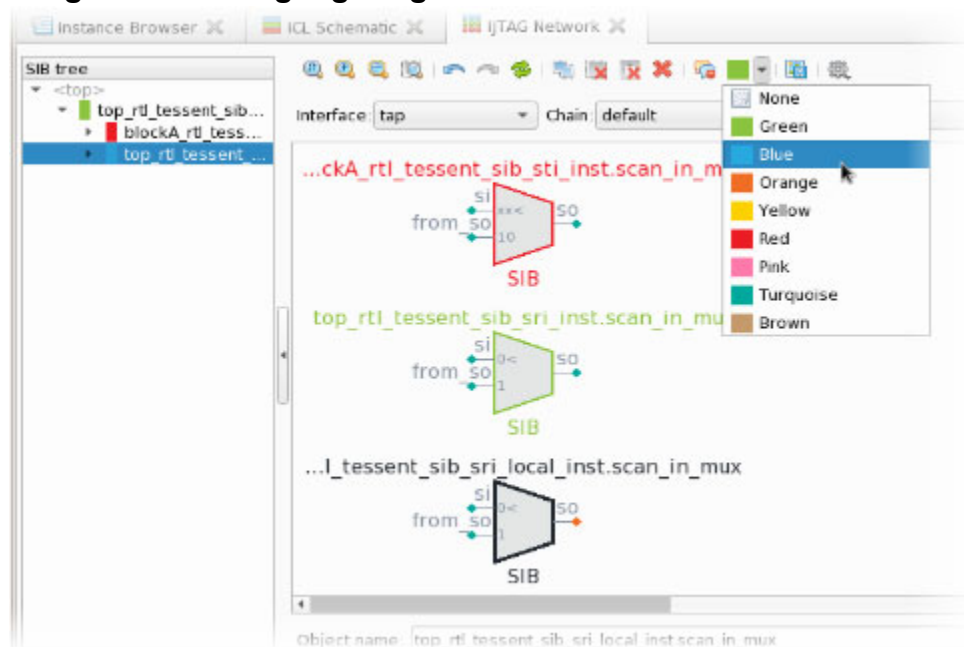
- **last used** — Highlights the scan path determined when the [iApply](#) command was most recently run.
- **network end state** — Highlights the scan path determined when the [close\\_pattern\\_set](#) command was run. If you have not run `close_pattern_set`, the scan path is not highlighted. The highlighted path is an IJTAG data path. The value of the “-network\_end\_state” switch for the `close_pattern_set` command affects the highlighting of the scan path. See the *Tessent Shell Reference Manual* for details.

If the `close_pattern_set` command is run immediately after the `iApply` command and without the `-network_end_state` switch (or with the default value of that switch), the “network end state” choice behaves like the “last used” choice. An exception to this is when the `iApply` affects only the instruction path.

Use the **Show SI** and **Show SO** buttons to show respectively the scan-in or scan-out port of the currently selected scan interface.

When you highlight SIBs in the IJTAG Network Viewer directly or with the [add\\_schematic\\_objects -highlight](#) options, the corresponding entry in the SIB tree is also highlighted as shown in [Figure 12-85](#).

Figure 12-85. Highlighting SIBs in the IJTAG Network Viewer



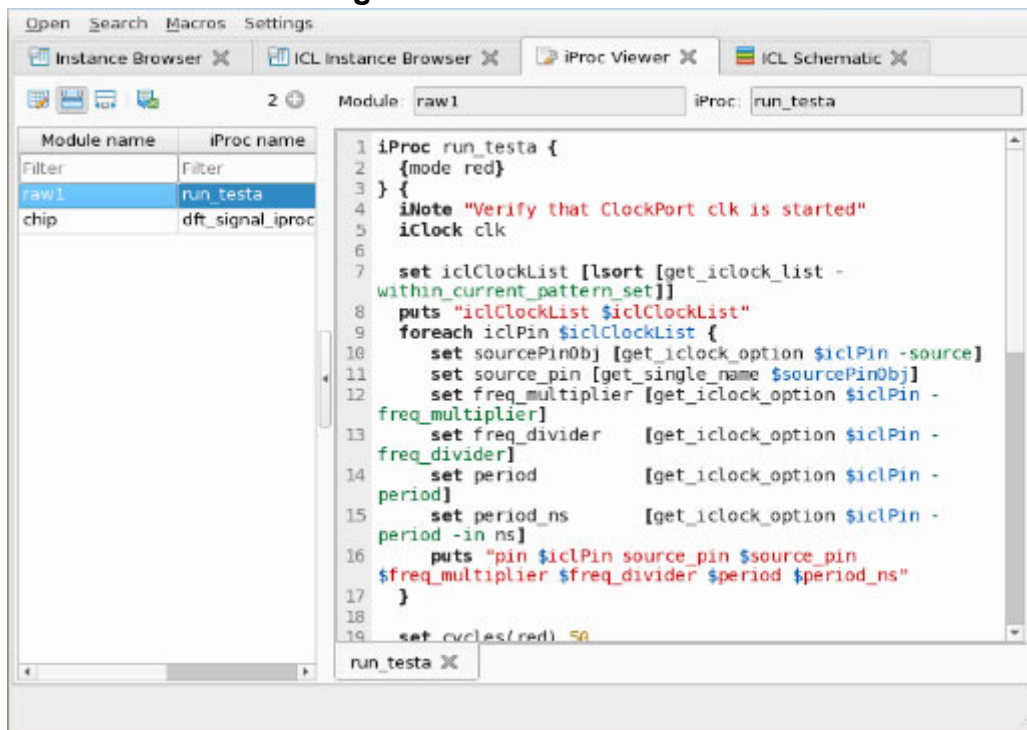
## iProc Viewer

The Tessent Visualizer iProc Viewer is similar in form and function to the Tessent Visualizer Text/HDL Viewer, with additional features.

Choose the ICL module and iProc from the table to the left of the viewer pane as shown in [Figure 12-86](#). To show iProcs in the viewer pane, select one or more rows in the table, right-click, and choose **Show in iProc Viewer**. Alternately, drag the selected row or rows from the table into the viewer pane.

The iProc Viewer features color syntax highlighting and a find function similar to the Text/HDL Viewer.

Figure 12-86. iProc Viewer



To display data about the iProc, hover the mouse pointer over its tab in the viewer as shown in Figure 12-87.

Figure 12-87. iProc Viewer Tooltip



If the filepath of the iProc is available, and the file is accessible on disk, you can right-click the tab and open the iProc in an external editor.

## Diagnosis Report Viewer

Use the Diagnosis Report Viewer to open reports created with the `write_diagnosis` command.

Open the Diagnosis Report Viewer by choosing the **Open > Diagnosis Report Viewer** menu item. Then you can open a diagnosis report using the toolbar at the top of the Diagnosis Report Viewer or by issuing the `display_diagnosis_report` command. You can open multiple reports using either of these methods. Select either text view or table view mode from the same toolbar.

In text view mode (Figure 12-88), the diagnosis report opens in the window, annotated with hypertext links for each design object affected. These links refer to the following:

- Symptoms
- Suspects
- Sub-suspects
- Pin and net objects associated with suspects

You can click a link to show the indicated object in the Hierarchical Schematic. Right-click to display a popup menu for other actions available to examine these objects.

**Figure 12-88. Diagnosis Report Viewer (Text View)**

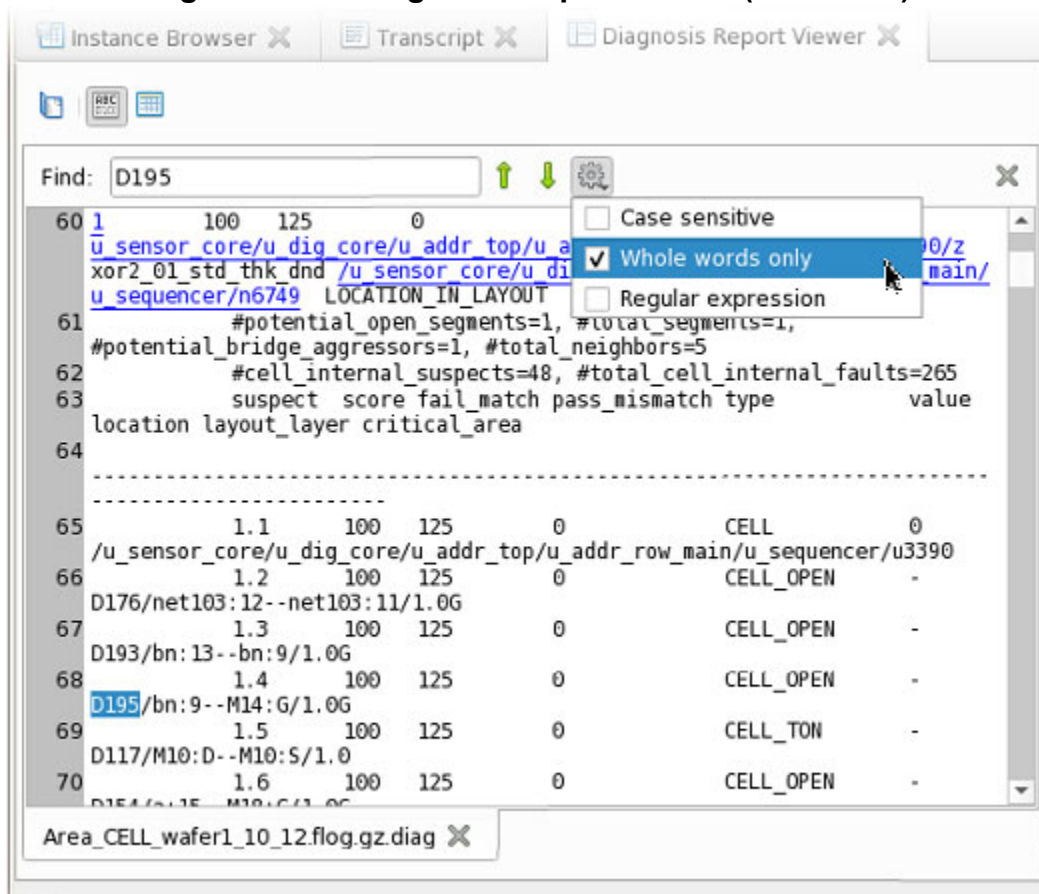


Figure 12-88 also illustrates text searching in the Diagnosis Report Viewer.

Figure 12-89 shows the three panes of the tabular layout for the diagnosis report. The left pane is a list of the symptoms denoted with integer IDs. Select a symptom in this pane to display a list of suspects in the right top table. Double-click an object in the table, or right-click and choose **Show on Hierarchical Schematic**, to show it in the Hierarchical Schematic. If a suspect has sub-suspects, the bottom table displays a list of sub-suspects.

Figure 12-89. Diagnosis Report Viewer (Table View)

3 +

| Symptom id | Suspect id | Score  | Type          | Fail match |
|------------|------------|--------|---------------|------------|
| Filter     | Filter     | Filter | Filter        | Filter     |
| 1          | 1          | 100    | INDETERMINATE | 12!        |
| 1          | 2          | 100    | INDETERMINATE | 12!        |
| 1          | 3          | 81     | INDETERMINATE | 12!        |

51 +

| Symptom id | Suspect id | Physical id | Score  | Type      |
|------------|------------|-------------|--------|-----------|
| Filter     | Filter     | Filter      | Filter | Filter    |
| 1          | 1          | 1           | 100    | CELL      |
| 1          | 1          | 2           | 100    | CELL_OPEN |
| 1          | 1          | 3           | 100    | CELL_OPEN |
| 1          | 1          | 4           | 100    | CELL_OPEN |

Area\_CELL\_wafer1\_10\_12.flog.gz.diag

## Search Features

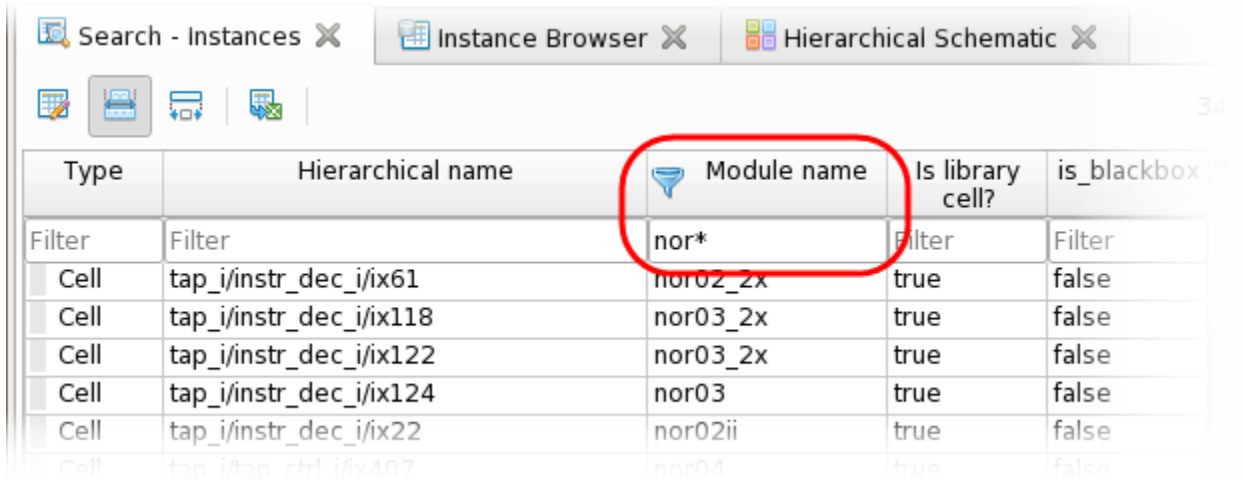
From the main menu bar, open the **Search** menu to search for instances, pins, gate pins, or specific ICL object types in your design. You can also search for iProcs or configuration elements currently defined in the Tessent Shell session.

### Instances

Figure 12-90 shows a search for all instances in the design with module names that begin with the string “nor”.



**Figure 12-90. Search Tab: Instances**



| Type   | Hierarchical name       | Module name | Is library cell? | is_blackbox |
|--------|-------------------------|-------------|------------------|-------------|
| Filter | Filter                  | nor*        | Filter           | Filter      |
| Cell   | tap_i/instr_dec_i/ix61  | nor02_2x    | true             | false       |
| Cell   | tap_i/instr_dec_i/ix118 | nor03_2x    | true             | false       |
| Cell   | tap_i/instr_dec_i/ix122 | nor03_2x    | true             | false       |
| Cell   | tap_i/instr_dec_i/ix124 | nor03       | true             | false       |
| Cell   | tap_i/instr_dec_i/ix22  | nor02ii     | true             | false       |
| Cell   | tap_i/tap_ctrl_i/ix407  | nor04       | true             | false       |

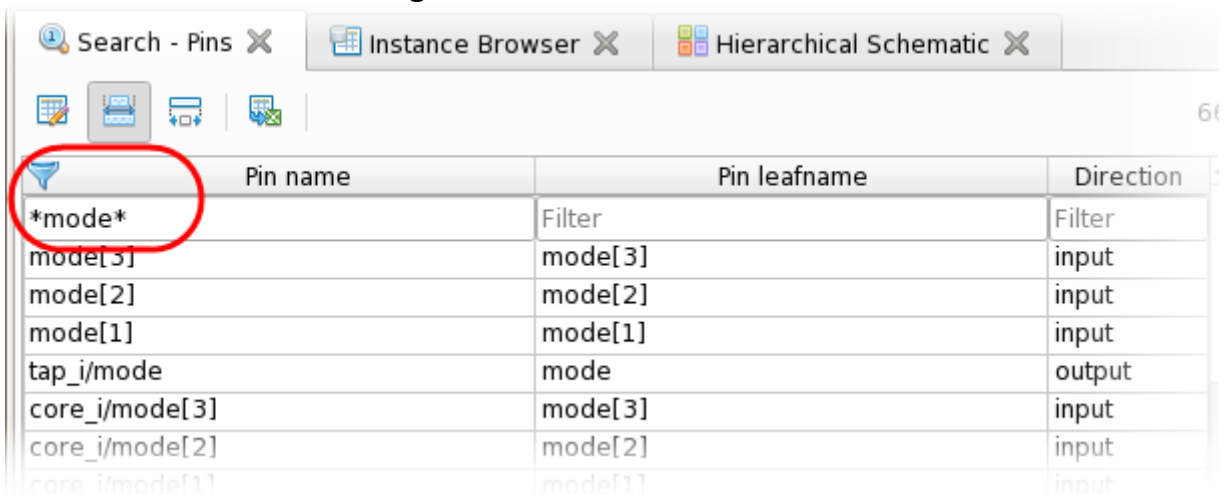
## Pins

Figure 12-91 shows a search for all hierarchical pins with the string “mode” in the pin name.

### Note

The search for pins in a large design can be time consuming because the tool searches across all hierarchical instances and their pins.

**Figure 12-91. Search Tab: Pins**



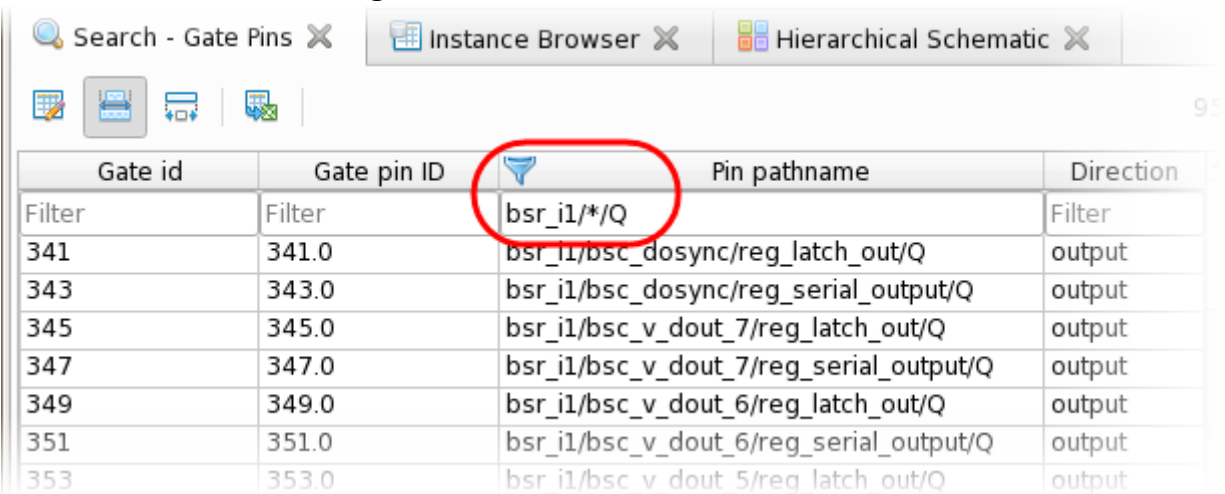
| Pin name       | Pin leafname | Direction |
|----------------|--------------|-----------|
| *mode*         | Filter       | Filter    |
| mode[3]        | mode[3]      | input     |
| mode[2]        | mode[2]      | input     |
| mode[1]        | mode[1]      | input     |
| tap_i/mode     | mode         | output    |
| core_i/mode[3] | mode[3]      | input     |
| core_i/mode[2] | mode[2]      | input     |
| core_i/mode[1] | mode[1]      | input     |

## Gate Pins

Figure 12-92 shows a search for flat pins (gate pins) that have a hierarchical pin with a leaf name of “Q” below the bsr\_i1 instance.



Figure 12-92. Search Tab: Gate Pins

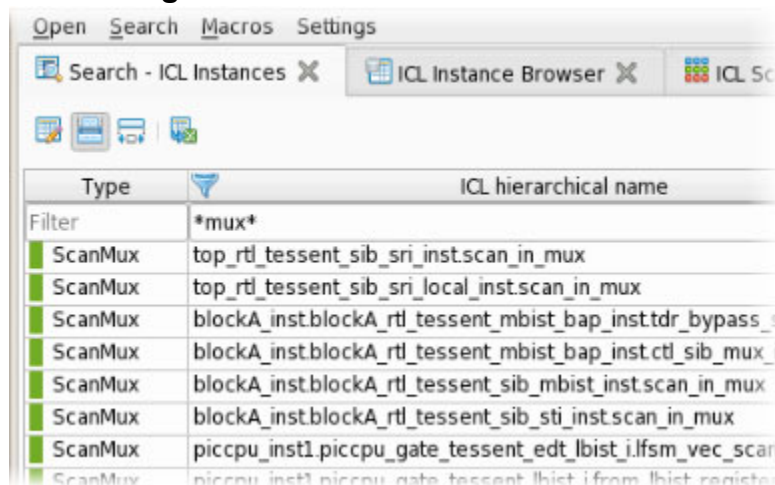


| Gate id | Gate pin ID | Pin pathname                            | Direction |
|---------|-------------|-----------------------------------------|-----------|
| Filter  | Filter      | bsr_i1/*Q                               | Filter    |
| 341     | 341.0       | bsr_i1/bsc_dosync/reg_latch_out/Q       | output    |
| 343     | 343.0       | bsr_i1/bsc_dosync/reg_serial_output/Q   | output    |
| 345     | 345.0       | bsr_i1/bsc_v_dout_7/reg_latch_out/Q     | output    |
| 347     | 347.0       | bsr_i1/bsc_v_dout_7/reg_serial_output/Q | output    |
| 349     | 349.0       | bsr_i1/bsc_v_dout_6/reg_latch_out/Q     | output    |
| 351     | 351.0       | bsr_i1/bsc_v_dout_6/reg_serial_output/Q | output    |
| 353     | 353.0       | bsr_i1/bsc_v_dout_5/reg_latch_out/Q     | output    |

## ICL Search Functions

You use **Search > Search - ICL Instances** in the same way you use **Search > Search - Instances** for the hierarchical design model, and **Search > Search - ICL Pins** the same way as you use **Search > Search - Pins** for the hierarchical design model.

Figure 12-93. ICL Instances Search



| Type    | ICL hierarchical name                                     |
|---------|-----------------------------------------------------------|
| Filter  | *mux*                                                     |
| ScanMux | top_rtl_tessent_sib_sri_instscan_in_mux                   |
| ScanMux | top_rtl_tessent_sib_sri_local_instscan_in_mux             |
| ScanMux | blockA_instblockA_rtl_tessent_mbist_bap_insttdr_bypass_s  |
| ScanMux | blockA_instblockA_rtl_tessent_mbist_bap_instctl_sib_mux_i |
| ScanMux | blockA_instblockA_rtl_tessent_sib_mbist_instscan_in_mux   |
| ScanMux | blockA_instblockA_rtl_tessent_sib_sti_instscan_in_mux     |
| ScanMux | piccpu_inst1_piccpu_gate_tessent_edt_lbist_1lfsm_vec_scan |
| ScanMux | piccpu_inst1_piccpu_gate_tessent_lbist_1lfsm_vec_scan     |

You can select one or more rows in a filtered search table. Then, use the right-click popup menu to show those objects in a schematic, add them to the pin data (pin searches), or show the HDL definition (instance searches).

### Note

The display properties for Search Instances and Search Pins are based on the Hierarchical Schematic. The display properties for Search Gate Pins are based on the Flat Schematic.

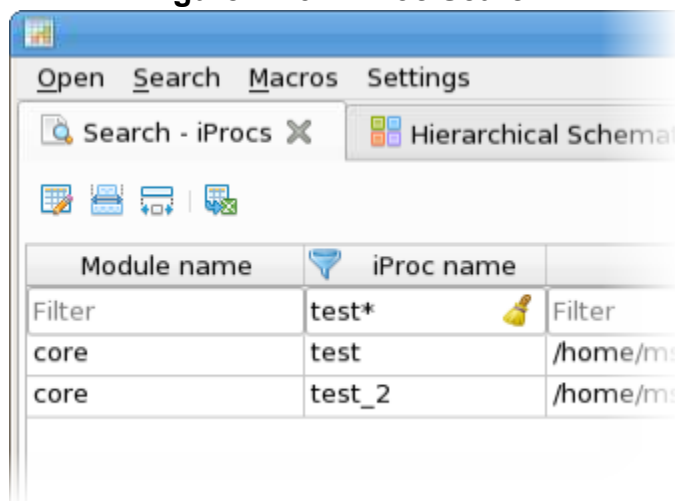
You can search for ICL aliases in a similar fashion with **Search > Search - ICL Aliases**.

See “[Tessent Visualizer Components and Preferences](#)” on page 866 for information about interacting with these tables in Tessent Visualizer.

## iProc Search Functions

Use **Search > Search - iProcs** to search for available iProcs. You can filter the column data just as you do with the other search functions.

**Figure 12-94. iProc Search**



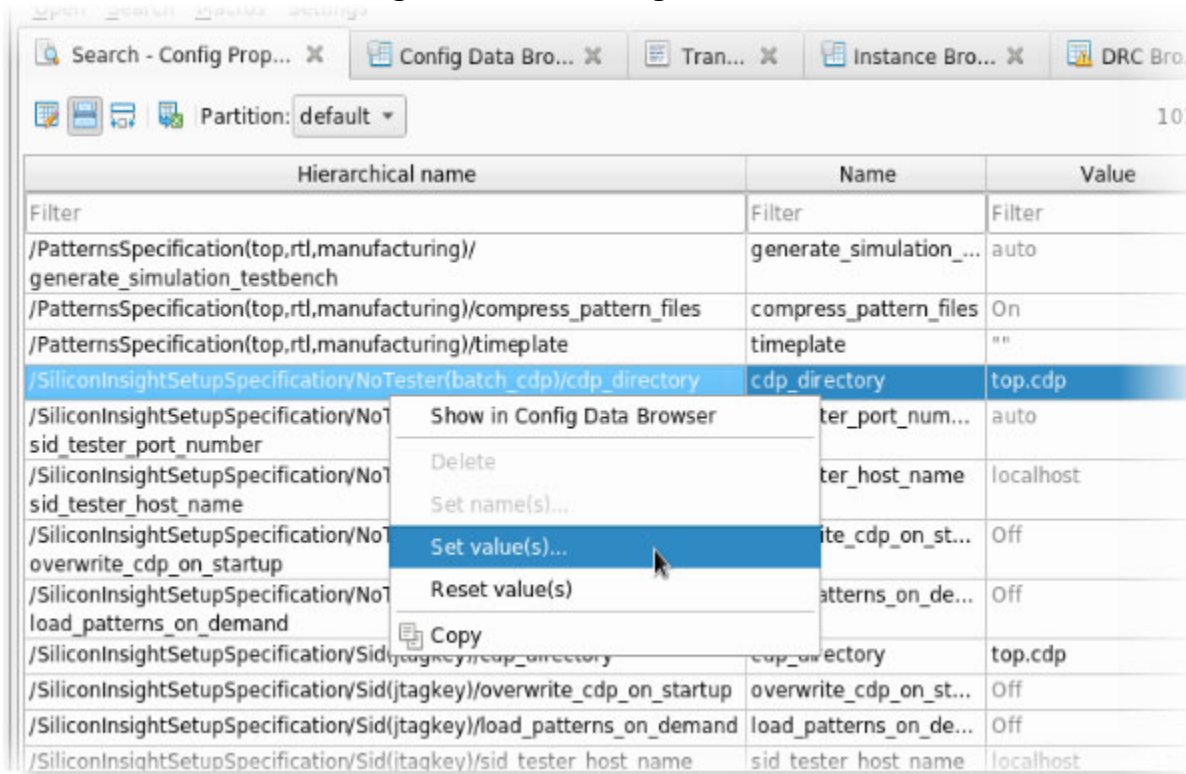
To view iProcs from the iProc Search window, select one or more rows, right-click them, and choose “Show in iProc Viewer.” Alternately, drag and drop the rows into the iProc Viewer.

## Config Properties

Use **Search > Search - Config Properties** to find a particular configuration property or wrapper path in the currently loaded configuration. Scroll through the list of wrapper paths and properties or use filtering to find the path and property of interest. Use the right-mouse-button context menu as shown in the following figure to set the property value or to reset it to its default value, to change the name of a named wrapper, or to show the wrapper path in the Config Data Browser.

See “[Config Data Browser](#)” on page 938 for more information.

Figure 12-95. Config Data Search



## Related Topics

[iProc Viewer](#)

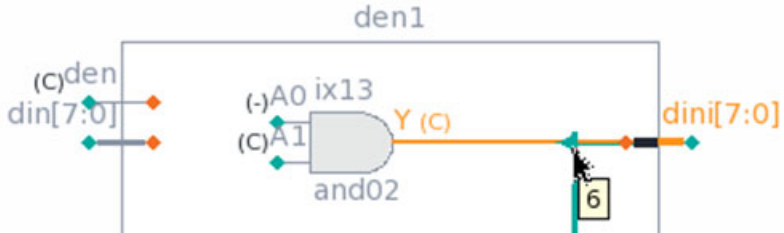
# Using Tessent Visualizer to Debug Design Issues

You can use Tessent Visualizer to solve various design problems. You can view state elements or observation points from a pin fanout, add or hide pins on an instance in the hierarchical schematic, search for pins by name, trace the bits of a bus, and debug issues by viewing simulation waveforms on pins.

## Procedure

Perform any of the following actions:

| If you want to...                                            | Do the following:                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|--------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| List all state elements in the fanout of a pin.              | <ol style="list-style-type: none"> <li>1. Click a pin in a schematic.<br/>The Tracer table displays below the schematic.</li> <li>2. Click the “Strategy:” dropdown list and select “Nearest state element.”</li> </ol> <p>The Tracer table lists all the state elements and the distance from the source in terms of number of pins in the traced path. The pin count for the same traced path is higher in the Hierarchical Schematic than in the Flat Schematic because of additional pins on hierarchical boundaries.</p> |
| Add pins to an instance on a hierarchical schematic.         | <ol style="list-style-type: none"> <li>1. Select an instance on a Hierarchical Schematic, then click the Pins button.<br/>A table listing the pins on the selected instance displays.</li> <li>2. Click the <b>Pins</b> button below the schematic to display a table listing the pins on the selected instance.</li> <li>3. Use the filter feature to search for the pin you want.</li> <li>4. Add the pin to the schematic by double-clicking or by dragging it onto the schematic.</li> </ol>                              |
| Search for pins by name.                                     | <ol style="list-style-type: none"> <li>1. Choose the <b>Search &gt; Search - Pins</b> menu item.</li> <li>2. Filter or sort the list of pins as needed.</li> <li>3. Right-click the pin name of interest in the table and choose one of these menu items to display the pin in a schematic: <ul style="list-style-type: none"> <li>• <b>Show on Hierarchical Schematic</b></li> <li>• <b>Show on Flat Schematic</b></li> </ul> </li> </ol>                                                                                    |
| Debug test setup or test end issues using a waveform viewer. | <ol style="list-style-type: none"> <li>1. Select one or more pins on a flat/hierarchical schematic.</li> <li>2. Right-click the selected signals and choose <b>Add to Wave Viewer</b> from the popup menu.<br/>This displays the selected signals in the Wave Viewer tab.</li> </ol>                                                                                                                                                                                                                                          |

| If you want to...            | Do the following:                                                                                                                                                                                                                                                                                                                                  |
|------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Trace a single bit of a bus. | <p>When tracing a single bit of a bus, the bit remains split from the bus. When tracing a net that recombines into a bus, the bit indices display in the schematic when you hover the mouse pointer, over the split point as shown in the following figure:</p>  |

## Results

Listing all state elements in the fanout of a pin: the Tracer table lists all the state elements and the distance from the source in terms of number of pins in the traced path. The pin count for the same traced path is higher in the Hierarchical Schematic than in the Flat Schematic because of additional pins on hierarchical boundaries.

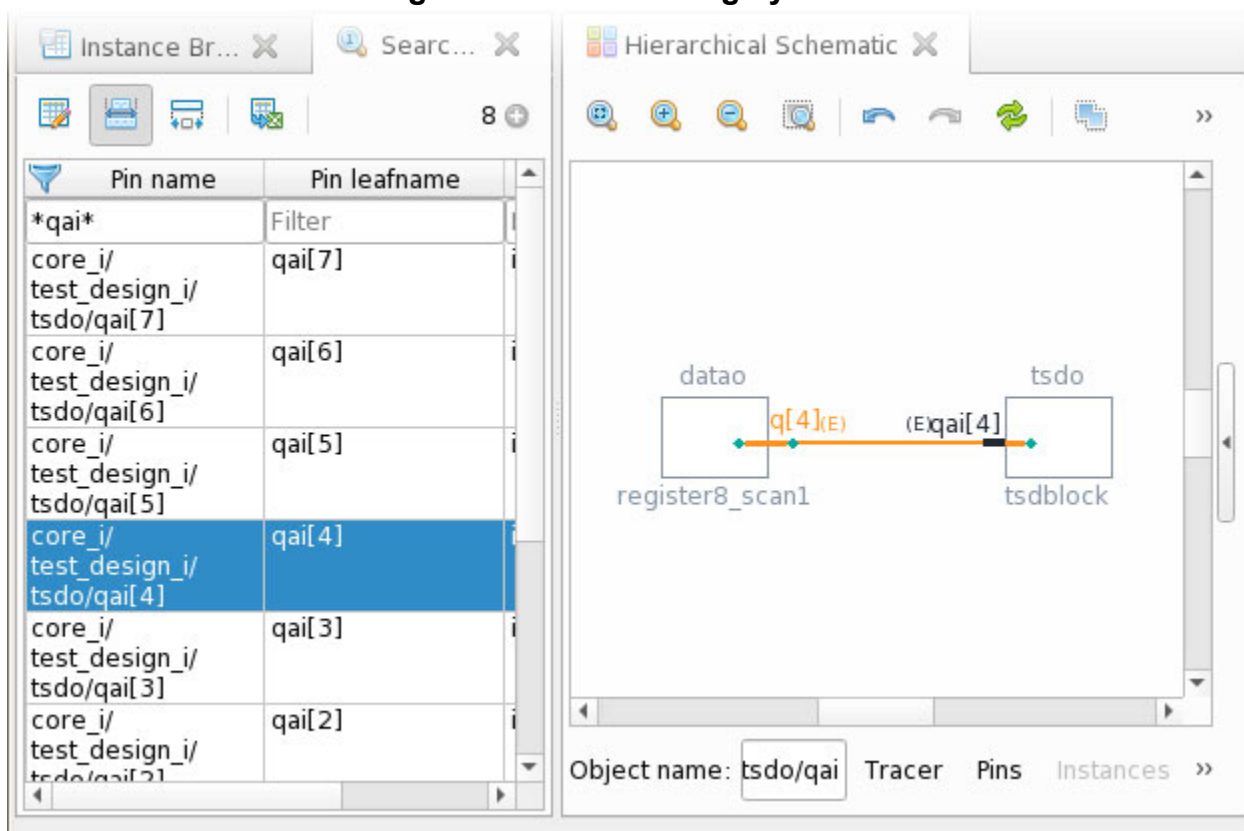
Adding pins to an instance in a hierarchical schematic: when you add an instance to a schematic directly or by tracing to it, the schematic displays only the pins that have connections (except for cases where there are very few pins in total). This feature simplifies the schematic to focus on pins of interest, improving run time.

## Examples

### Searching for Pins by Name

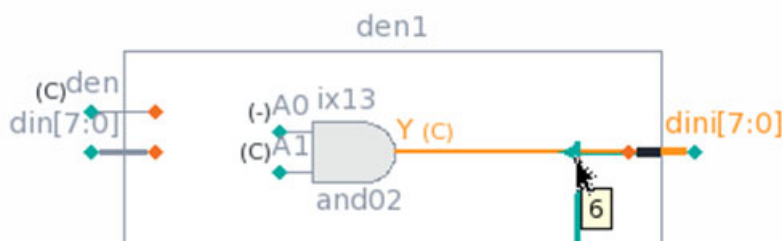
The following figure shows a search for the “qai” bus, and subsequent selection of an element of that bus, which is displayed in the hierarchical schematic. You can then trace the signal.

**Figure 12-96. Searching by Name**



### Tracing a Single Bit of a Bus

When tracing a single bit of a bus, the bit remains split from the bus. When tracing a net that recombines into a bus, the bit indices display in the schematic when you hover the mouse pointer, over the split point as shown in the following figure:



### Related Topics

[Nearest State Element Tracing Strategy](#)

[Wave Viewer](#)

# Accessing Tutorials for Tessent Visualizer

Tutorial instructions and design data are available for download in an Example Kit (eKit) file. The tutorials demonstrate some basic operations you can perform with Tessent Visualizer. The eKit is a ZIP file containing the instructions and design data that you need to perform the Tessent Visualizer tutorials.

## Procedure

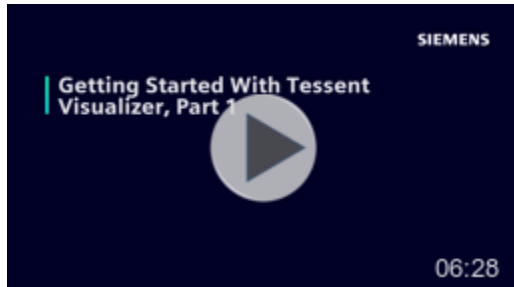
1. Log on to Support Center:  
<https://support.sw.siemens.com>
2. Find the Tessent product page.
3. Click the Documentation link in the page banner.
4. Use the “Restrict content to version” dropdown list to specify the current Tessent version.
5. Click the Getting Started Guide in Document Types on the left side of the page.
6. Click the link for “Try It: Tessent Visualizer Quick Start and Example Kit.”  
This downloads the eKit ZIP file to your default location.
7. Move the downloaded file to a working directory that you can access with a Linux shell.
8. From a Linux shell, unpack the download file:  
**\$ unzip <path>/tesvis\_qs\_ekit.zip**
9. Navigate to the *tesvis\_qs\_ekit/Docs* directory and open the *tesvis\_quickstart.pdf* file.  
This file contains instructions for performing the tutorials.

## Results

You are now ready to perform the Tessent Visualizer tutorials.

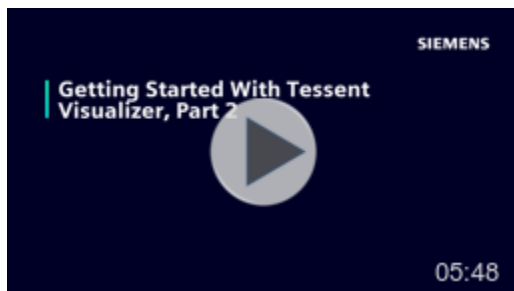
## Accessing Videos for Tessent Visualizer

*Getting Started with Tessent Visualizer* is a set of two videos that show some basic operations you can perform with Tessent Visualizer.



*Part 1: Analyzing and Improving Test Coverage* demonstrates how to analyze test coverage and explore areas of the design that need coverage improvement:

- Invoking Tessent Visualizer
- Using the Instance Browser to navigate a design
- Adding data columns to a table
- Sorting and filtering data
- Examining DRC violations and locating a violation on a schematic
- Using the Cell Library Browser to examine a library module



*Part 2: Troubleshooting Problems With Verilog Design Files* shows how to troubleshoot a problem with a Verilog design file, which includes the following operations:

- Examining the source of a DRC violation on a schematic
- Troubleshooting an error with scan chain insertion
- Tracing a signal path on a schematic
- Recognizing problems on a schematic and locating the problem source in HDL code
- Determining the number of times a library module is instantiated in a design



## Keyboard Shortcuts

A number of keyboard shortcuts are available for Tessent Visualizer.

**Table 12-7. Global Shortcuts**

| Shortcut                       | Action                       |
|--------------------------------|------------------------------|
| Ctrl+Shift+H                   | Open Hierarchical Schematic  |
| Ctrl+Shift+F                   | Open Flat Schematic          |
| Ctrl+Shift+B                   | Open Instance Browser        |
| Ctrl+Shift+L                   | Open Cell Library Browser    |
| Ctrl+Shift+D                   | Open DRC Browser             |
| Ctrl+Shift+V                   | Open Text/HDL Viewer         |
| Ctrl+Shift+R                   | Open Diagnosis Report Viewer |
| Ctrl+Shift+T                   | Open Transcript              |
| Ctrl+Shift+Q                   | Quit Tessent Visualizer      |
| Ctrl+Shift+W                   | Close Current Tab            |
| Ctrl+Shift+Tab<br>or Ctrl+PgUp | Go To Previous Tab           |
| Ctrl+Tab or<br>Ctrl+PgDown     | Go To Next Tab               |

**Table 12-8. Schematic Shortcuts**

| Shortcut     | Action                                |
|--------------|---------------------------------------|
| Ctrl+0       | Zoom all                              |
| Ctrl++       | Zoom in                               |
| Ctrl+-       | Zoom out                              |
| Ctrl+Z       | Undo                                  |
| Ctrl+Shift+Z | Redo                                  |
| Ctrl+A       | Select all                            |
| Del          | Delete selected                       |
| Ctrl+M       | Mark with chosen color                |
| Ctrl+S       | Export schematic (also: export table) |

**Table 12-8. Schematic Shortcuts (cont.)**

| Shortcut   | Action          |
|------------|-----------------|
| Arrow Keys | Move schematic  |
| Esc        | Clear selection |

**Table 12-9. Instance Browser Shortcuts**

| Shortcut   | Action                  |
|------------|-------------------------|
| Enter      | Go down in hierarchy    |
| Backspace  | Go up in hierarchy      |
| Arrow Keys | Move to the chosen cell |

**Table 12-10. Filter Editing Shortcuts**

| Shortcut  | Action                                |
|-----------|---------------------------------------|
| Tab       | Next filter                           |
| Shift+Tab | Previous filter                       |
| Enter     | Accept filters                        |
| Esc       | Discard changes to the current filter |

**Table 12-11. Text Search Shortcuts**

| Shortcut     | Action                   |
|--------------|--------------------------|
| Ctrl+F       | Open text find function  |
| Esc          | Close text find function |
| Ctrl+G       | Find next                |
| Ctrl+Shift+G | Find previous            |

**Table 12-12. Wave Viewer Mouse Shortcuts**

| Shortcut          | Action                                                         |
|-------------------|----------------------------------------------------------------|
| Ctrl+Scroll up    | Zoom in on the active cursor by a factor of 2x                 |
| Ctrl+Scroll down  | Zoom out on the active cursor by a factor of 2x                |
| Shift+Scroll up   | Scroll the signals listed vertically in the upward direction   |
| Shift+Scroll down | Scroll the signals listed vertically in the downward direction |

---

**Note**



The Ctrl+G and Ctrl+Shift+G keyboard shortcuts are not supported in all desktop environments.

---



# Chapter 13

## Timing Constraints (SDC)

---

This chapter describes the Synopsys Design Constraints (SDC) generated by Tessent Shell, and how you use it to constrain the test mode of the Embedded Test hardware generated by the tool.

|                                                                          |             |
|--------------------------------------------------------------------------|-------------|
| <b>Generating and Using SDC for Tessent Shell Embedded Test IP.....</b>  | <b>970</b>  |
| <b>SDC File Generation With Tessent Shell.....</b>                       | <b>970</b>  |
| <b>SDC Design Synthesis With Tessent Shell.....</b>                      | <b>972</b>  |
| Preparation Step 1: Sourcing SDC File .....                              | 972         |
| Preparation Step 2: Setting and Redefining Tessent Tcl Variables .....   | 972         |
| Preparation Step 3: Verifying the Declaration of Functional Clocks ..... | 974         |
| Preparation Step 4: Redefining Other Tessent Tcl Variables .....         | 975         |
| Synthesis Step 1: Applying the SDC Constraints .....                     | 976         |
| Synthesis Step 2: Preparing the DFT Logic for Synthesis .....            | 976         |
| Synthesis Step 3: Synthesizing Your Design .....                         | 977         |
| Synthesis Step 4: Writing Out Your Final SDC .....                       | 977         |
| Synthesis Step 5: Writing Out Your Final Netlist .....                   | 977         |
| <b>Running Layout with Tessent Shell DFT .....</b>                       | <b>977</b>  |
| <b>SDC for Modal Static Timing Analysis .....</b>                        | <b>983</b>  |
| Checking Your Functional Logic Alone .....                               | 983         |
| Checking Your Embedded Test Logic .....                                  | 984         |
| <b>VHDL and Mixed Language Designs.....</b>                              | <b>986</b>  |
| VHDL Generate Blocks .....                                               | 986         |
| <b>SDC File Contents .....</b>                                           | <b>988</b>  |
| tessent_set_default_variables .....                                      | 988         |
| tessent_create_functional_clocks .....                                   | 989         |
| tessent_<design_name>_set_dft_signals .....                              | 989         |
| <ltest_prefix>_disable .....                                             | 990         |
| tessent_set_non_modal .....                                              | 991         |
| tessent_<design_name>_kill_functional_paths .....                        | 991         |
| IJTAG Instrument .....                                                   | 992         |
| LOGICTEST Instruments .....                                              | 993         |
| MemoryBIST Instrument .....                                              | 1007        |
| BoundaryScan Instrument .....                                            | 1010        |
| Hierarchical STA in Tessent .....                                        | 1011        |
| Mapping Procs .....                                                      | 1016        |
| Synthesis Helper Procs .....                                             | 1017        |
| <b>Example Scripts Using Tessent Tool-Generated SDC .....</b>            | <b>1019</b> |
| Example Design Compiler Synthesis Script .....                           | 1019        |

|                                                |      |
|------------------------------------------------|------|
| Example Fusion Compiler Synthesis Script ..... | 1020 |
| Example Genus Synthesis Script .....           | 1022 |
| Example PrimeTime STA Script .....             | 1025 |

## Generating and Using SDC for Tessent Shell Embedded Test IP

We guide you through an example SDC flow, from generation of the Tessent SDC file, through adjusting the variables used within the file, to using it in your design synthesis.

First, you learn how to generate an SDC file for your design, which may be at the chip or the physical block level. SDCs for sub blocks are automatically merged by Tessent with the constraints of their parent physical block.

Then we show you how to adjust variables in the SDC file to match your particular synthesis setup or requirements. There is no need however, to ever directly edit the generated SDC files.

Next, you see how to use the generated SDC file in a step-by-step example synthesis flow, followed by additional notes on Static Timing Analysis (STA), mixed-language and VHDL specific issues.

Key SDC procs are then explained. These procedures are used by Tessent instruments (like MBIST or OCC) or maybe useful to use in your own synthesis scripts.

“[Example Scripts Using Tessent Tool-Generated SDC](#)” on page 1019 shows examples of using and adjusting Tessent SDC files in third-party synthesis and STA environments.

Note: The SDC described in this manual is to be used in conjunction with your functional SDC for the synthesis of your design after Tessent Shell RTL insertion and the static timing analysis of your design after synthesis. The Tessent Shell gate insertion flow uses a different set of SDC constraints during the [run\\_synthesis](#) command, but the generated SDC should still be used for layout.

## SDC File Generation With Tessent Shell

You generate your design SDC file using Tessent Shell.

In the standard Tessent Shell flow, the tool creates the SDC for the current design when you run the [extract\\_icl](#) command, which does the following:

- Stores the SDC in Tcl procs
- Writes the SDC to a single file named *design\_name.sdc*, where *design\_name* is the name of the current design loaded in Tessent Shell

The SDC file is located next to the extracted ICL, which is typically in the [dft\\_inserted\\_designs](#) directory as follows:

```
${tsdb}/dft_inserted_designs/${design_name}_${design_id}.dft_inserted_design
```

Every instrument type (for example, MBIST, IJTAG) of the current design and all sub-blocks that provide SDC constraints are represented by a separate proc in the SDC file. Constraints for logictest-related instruments such as OCC, EDT, or LBIST, including logictest-related DFT signals, are all grouped under similar placeholder “ltest” instrument procs.

There is no need to call each instrument’s individual SDC proc. Tessent Shell provides user procs, which are customized to call all relevant SDC procs for your design.

If you encounter a problem with SDC generation preventing ICL extraction, you can temporarily skip SDC generation by using the following command options:

```
extract_icl -skip_sdc_extraction
```

To regenerate your SDC file for a given design and to take advantage of changes in the SDC constraints in a newer version that does not reflect a hardware change, you can extract the SDC of your design without running ICL extraction by loading the design’s full view and running the Tessent Shell [extract\\_sdc](#) command. You should load the interface view of all physical blocks of your design. For example:

```
read_design design_name -design_identifier design_id -view full  
extract_sdc
```

When it is time to use the SDC, locating your SDC file requires the following:

- Your latest TSDB location for the design you want to synthesize/analyze.
- Your design’s name.
- The *design\_id* of the latest insertion pass of your design.

For example, if your design “myChip” used only a single TSDB directory, the default “tsdb\_outdir”, then your SDC files would be located in the following places:

- For the processed DftSpecification(myChip,rtl1):

```
tsdb_outdir/dft_inserted_designs/myChip_rtl1.dft_inserted_design/myChip.sdc
```

- For the processed DftSpecification(myChip,rtl2):

```
tsdb_outdir/dft_inserted_designs/myChip_rtl2.dft_inserted_design/myChip.sdc
```

---

**Note**

The latest SDC file is always a superset of the previous one. If you use a two-pass flow, you only need to load the latest file.

---

## SDC Design Synthesis With Tessent Shell

What follows is a list of steps you must follow during your synthesis flow to properly handle Tessent Embedded Test IP. We use a non-modal approach to constrain the inserted test logic. This means that we let both functional and test clocks through muxes and we do not set any constant anywhere. This lets the synthesis tools optimize both the functional and test timing paths simultaneously.

The following steps are run after you have loaded the functional design and applied functional SDC. These steps do not describe the entire flow, but, rather, only highlight typical steps affected by the presence of Tessent IP. Design Compiler commands are used as an example: see “[Example Design Compiler Synthesis Script](#)” on page 1019 and “[Example Genus Synthesis Script](#)” on page 1022 for complete example scripts.

|                                                                                 |            |
|---------------------------------------------------------------------------------|------------|
| <b>Preparation Step 1: Sourcing SDC File</b> .....                              | <b>972</b> |
| <b>Preparation Step 2: Setting and Redefining Tessent Tcl Variables</b> .....   | <b>972</b> |
| <b>Preparation Step 3: Verifying the Declaration of Functional Clocks</b> ..... | <b>974</b> |
| <b>Preparation Step 4: Redefining Other Tessent Tcl Variables</b> .....         | <b>975</b> |
| <b>Synthesis Step 1: Applying the SDC Constraints</b> .....                     | <b>976</b> |
| <b>Synthesis Step 2: Preparing the DFT Logic for Synthesis</b> .....            | <b>976</b> |
| <b>Synthesis Step 3: Synthesizing Your Design</b> .....                         | <b>977</b> |
| <b>Synthesis Step 4: Writing Out Your Final SDC</b> .....                       | <b>977</b> |
| <b>Synthesis Step 5: Writing Out Your Final Netlist</b> .....                   | <b>977</b> |

### Preparation Step 1: Sourcing SDC File

The first step is to source the Tessent Shell tool-generated SDC file, associated to the last insertion pass, from within your synthesis script. We suggest creating central variables for the TSDB location and design name:

```
set design_name myChip
set tsdb ../../../../golden_repo/${design_name}_tsdb
set design_id rtl2
source \
${tsdb}/dft_inserted_designs/${design_name}_${design_id}.dft_inserted_design \
/${design_name}.sdc
```

### Preparation Step 2: Setting and Redefining Tessent Tcl Variables

The Tessent Shell tool-generated SDC file makes extensive use of Tcl variables. This section discusses the redefinition of those variables



Their definitions are contained in the proc `tessent_set_default_variables`. You must call this procedure to set all Tessent variables to their design dependent default value. All other Tessent SDC-related procs make use of these variables. This procedure call is mandatory:

```
tessent_set_default_variables
```

Tessent SDC variables can and should be redefined to better suit your setup and design. For example, you can change the TCK period from the default of 100.0ns to any value by redefining the variable `tessent_tck_period`.

```
set tessent_tck_period 80.0
```

---

**Note**

 The value of `tessent_tck_period` might depend on the maximum tck clock frequency that can be applied to the circuit. See the “[IJTAG Network Performance Optimization](#)” section in the *Tessent IJTAG User’s Manual* showing how to maximize the frequency of the IJTAG network test clock.

---

You do not need to edit the Tessent Shell-generated SDC file, but rather use the “set” command in your synthesis script to overwrite the value of the Tessent Shell tool-specific SDC variable `tessent_tck_period`. Refer to “[Example Design Compiler Synthesis Script](#)” on page 1019 and “[Example Genus Synthesis Script](#)” on page 1022 for synthesis script examples.

For designs with Siemens EDA Logictest IP, you possibly need to update the global Tcl variables that reproduce your fastscan test\_proc timeplate specifications, so that your SDC constraints closely match your simulation waveforms. For more information on these Tcl variables, see “[LOGICTEST Instruments](#)” on page 993.

When the design contains LogicBIST instruments, create the `shift_clock_src` of the LogicBIST controller and indicate its name to the SDC procs by using the `tessent_lbist_shift_clock_src` Tcl array variable. Creating this clock and setting its name with the `tessent_lbist_shift_clock_src` variable is mandatory.

Specify this variable as follows:

```
set tessent_lbist_shift_clock_src(lbist_inst<index>) clock_name
```

For example:

```
create_clock -name lbist_clock pll/clk_out_2 -period 20  
set tessent_lbist_shift_clock_src(lbist_inst0) lbist_clock
```

If multiple controllers use the same clock, create the clock once only. However, you must still specify the `tessent_lbist_shift_clock_src` variable for each controller with the same clock name. Starting at 0, specify as many `lbist_inst` indices as the number of LogicBIST controllers in the current design level minus one. For example, for a design with two LogicBIST controllers, set the `tessent_lbist_shift_clock_src(lbist_inst0)` and `tessent_lbist_shift_clock_src(lbist_inst1)` variables.

You can find the mapping of the `lbist_inst<index>` identifier to the instance path name of the LogicBIST controllers in the `tessent_lbist_mapping` Tcl array variable inside the `tessent_set_default_variables` SDC proc. When the clock is created with a different name than its source, specify the name of the clock as specified with `-name` option of the `create_clock` SDC command. Existence of this Tcl variable and the clock it refers to is rule checked by the generated SDC.

Similarly, when the design contains InSystemTest instruments, indicate the clocks to SDC by using the `tessent_ist_clock` Tcl array variable. You can find the mapping from the `ist_inst<index>` identifier to the InSystemTest controller instance path name in the `tessent_ist_mapping` variable.

Another key variable to possibly overwrite is the `tessent_clock_mapping` array explained in the next section. Other, much less frequently used variables are discussed in “[Preparation Step 4: Redefining Other Tessent Tcl Variables](#)” on page 975.

## Preparation Step 3: Verifying the Declaration of Functional Clocks

In Tessent Shell’s system mode ‘setup’, you may have specified asynchronous clocks, reference clocks, and branch clocks. Some might be functional clock domain bases of some instruments, namely MemoryBist, and they could be used in some timing constraints.

The tool stores this information in the `tessent_clock_mapping` array. It is defined in the generated SDC file, within the proc `tessent_set_default_variables`, and has the following form:

```
array set tessent_clock_mapping {
    <TessentShell ClockLabel> <FunctionalSDC ClockLabel>
    [...]
}
```

This array is used by the Tessent Shell tool-generated SDC constraints to refer to your functional clocks. They do so only through a remapped “`tessent_clock_mapping(<TessentShell ClockLabel>)`” array element. By default, in `tessent_set_default_variables`, `<TessentShell ClockLabel>` and `<FunctionalSDC ClockLabel>` are identical.

---

### Note



It is imperative that you examine this array and look for cases where this default mapping must be updated. Overriding names are not necessary if you have used the “`-label`” option of the [add\\_clocks](#) command in Tessent Shell to match the “`-name`” value in the SDC.

---

If you need to update one of the `tessent_clock_mapping()` array values, please do so in your main synthesis script, immediately after the call to the `tessent_set_default_variables` proc. For

example, if you had a clock defined with a label "CK25" in Tessent Shell, but in your SDC constraints the same clock is defined with a name "PIN\_CK25", you need to specify:

```
set tessent_clock_mapping(CK25) PIN_CK25
```

The new definition of `tessent_clock_mapping(CK25)` overrides the default one contained in `tessent_set_default_variables`.

It is important to understand that you do not need to edit the Tessent-generated SDC file, but rather use the 'set' command in your synthesis script to overwrite the value of the Tessent SDC variable array element you want to change. Alternatively, if many of your Tessent defined clocks and respective SDC clocks have different names, you might want to redefine a block of the array:

```
set tessent_clock_mapping(CLK25) pCLK25
set tessent_clock_mapping(CLK_PLL_REF) pCLK100
```

OR

```
array set tessent_clock_mapping {
  CLK25 pCLK25
  CLK_PLL_REF pCLK100
}
```

**IMPORTANT:** You do not need to define `tessent_clock_mapping()` entries for clocks that you have not specified in Tessent Shell. Those are not used in the timing constraints.

## Preparation Step 4: Redefining Other Tessent Tcl Variables

The `tessent_tck_period` and `tessent_clock_mapping` array variables are the most important Tessent SDC variables you should double check and overwrite as needed. Tessent SDC uses a number of additional variables that are all defined and set to a default value in the `tessent_set_default_variables` proc.

For example, you can independently change the input and output delay (from the default, 25% of the `tessent_tck_period`) or change the hierarchy separator character. Please see the header of the generated SDC file and the embedded comments in the `tessent_set_default_variables` proc, explaining each such variable. Within the synthesis tool, you can use the Tcl command "info body tessent\_set\_default\_variables" to see what variables are being defined.

### Note

 The value of `tessent_tck_period` might depend on the maximum tck clock frequency that can be applied to the circuit. See the "[IJTAG Network Performance Optimization](#)" section in the *Tessent IJTAG User's Manual* showing how to maximize the frequency of the IJTAG network test clock.

## Synthesis Step 1: Applying the SDC Constraints

If you use Design Compiler, you need to set the variable “timing\_enable\_multiple\_clocks\_per\_reg” to true. This is needed to properly constrain MemoryBIST in the presence of MemoryBIST clock muxing.

```
set_app_var timing_enable_multiple_clocks_per_reg true
```

To apply the Tessent Shell-generated SDC constraints, run the merged non\_modal proc:

```
tessent_set_non_modal
```

This Tessent SDC proc in turn calls all instrument non-modal procs as needed by your design and its sub-blocks. There is nothing else for you to do than call this one proc to apply all non-modal SDC constraints.

## Synthesis Step 2: Preparing the DFT Logic for Synthesis

Once your design is loaded, you need to make sure to preserve instances of Tessent Shell-inserted “persistent” cells and possibly instrument instances, depending on your downstream needs.

Refer to “[Synthesis Helper Procs](#)” on page 1017 for more information on the different options.

Persistent cells should not be optimized as Tessent Shell SDC uses them as anchor points for layout and STA, and different instances need to be preserved depending on if you do scan insertion with Tessent Shell or more DFT insertion. Procs are provided in the .sdc file to generate a list of all instances that need to be preserved in different ways: tessent\_get\_preserve\_instances, tessent\_get\_optimize\_instances, and tessent\_get\_size\_only\_instances.

First you need to prevent boundary optimization of DFT objects using the proc tessent\_get\_preserve\_instances:

```
set_app_var compile_enable_constant_propagation_with_no_boundary_opt false
set_preserve_instances [tessent_get_preserve_instances scan_insertion]
set_boundary_optimization $preserve_instances false
set_ungroup $preserve_instances false
set_app_var compile_seqmap_propagate_high_effort false
set_app_var compile_delete_unloaded_sequential_cells false
```

Then, because boundary optimization applies hierarchically, you re-enable boundary optimization of the child instances of DFT objects using tessent\_get\_optimize\_instances:

```
set_boundary_optimization [tessent_get_optimize_instances] true
```

Avoid inverting the logic signals of any Tessent IP instantiated in Tessent Shell. Signal inversion can cause design rule checks of the gate-level netlist to fail or disrupt the ATPG flow. Find all the Tessent sequential cells and turn off any output inversion by the synthesis tool.

```
set allETRegs [all_registers -edge_triggered]
set TessentETRegs [filter_collection $allETRegs "full_name =~ *tessent*"]
set_register_output_inversion $TessentETRegs false
```

Cells from your provided Tessent Cell Library instantiated in Tessent Shell must also be preserved but can be resized to enable critical data path timing optimization during synthesis. You can get them using `tessent_get_size_only_instances`:

```
set_size_only -all_instances [tessent_get_size_only_instances]
```

If your design contains shared bus assemblies, you can save significant area by ungrouping their content, with the exception of the MemoryBIST controller. For examples of the suggested procedures, refer to the “Synthesis Step 2” portions in the [“Example Scripts Using Tessent Tool-Generated SDC”](#) on page 1019. For background information, refer to [“Implementing MemoryBIST With Memory Shared Bus Interface”](#) in the *Tessent MemoryBIST User’s Manual*.

## Synthesis Step 3: Synthesizing Your Design

You can now proceed with the compilation of your design:

```
link
check_design
compile -boundary_optimization [...]
```

## Synthesis Step 4: Writing Out Your Final SDC

Use the `write_sdc` command after this step to export the merged SDC (Functional and Tessent Shell) for use with your layout tool:

```
write_sdc ${design_name}.merged_sdc
```

## Synthesis Step 5: Writing Out Your Final Netlist

You can now proceed with writing out your synthesized netlist.

```
write -format verilog -output ${design_name}.vg ${design_name} -hier
```

## Running Layout with Tessent Shell DFT

There exist a few possible ways to apply your Tessent SDC constraints in layout.

### Note



This section uses the following convention to shorten proc names:

`<ltest_prefix> := tessent_set_ltest`

---

You can do either of the following:

- Use the SDC that you wrote out in previous synthesis Step 5 and feed it directly to your layout tool, or
- Define all your functional SDC constraints and add Tessent DFT constraints, like you previously did in synthesis, by repeating the steps:
  - Define your functional constraints.
  - Set your global Tcl Tessent variables.
  - Call `tessent_set_non_modal`.

You also must preserve your Tessent DFT persistent cells from further layout logic optimization. Get the list of these cells by calling your SDC file proc `tessent_get_size_only_instances`.

That is all you must do if your design does *not* feature Tessent logictest DFT. If you used 3rd-party scan insertion tools, please refer to their instructions as to how to constrain that mode. The rest of this section details what you must do in layout with Tessent logictest DFT.

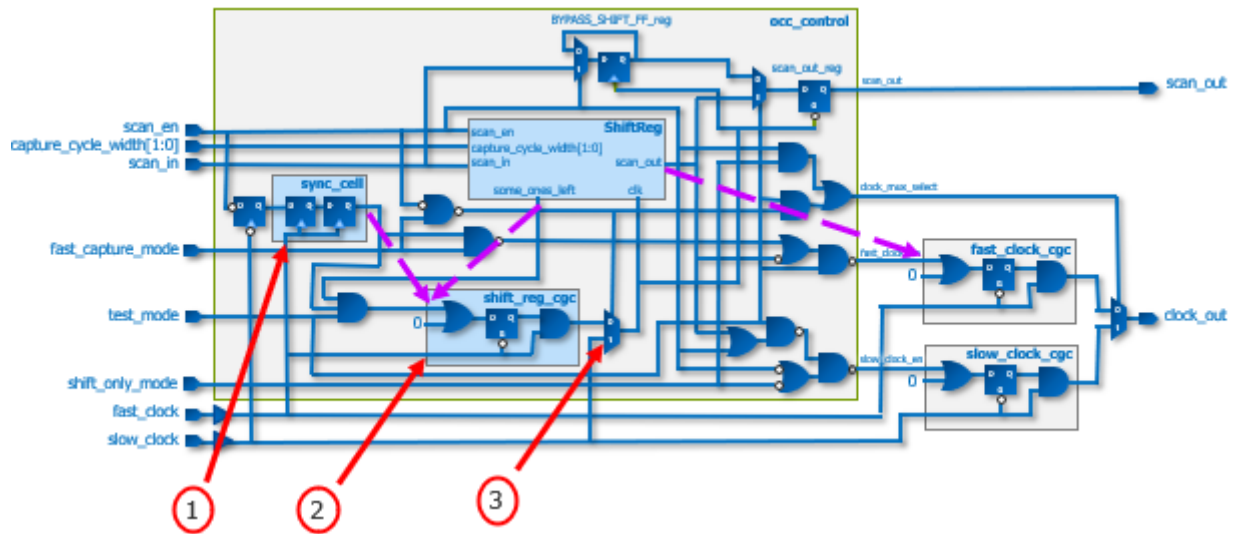
## Clock Tree Synthesis With Tessent On-Chip Clock Controllers (OCCs)

If your design includes Tessent OCC IP, you may need to instruct your clock tree synthesis command to stop balancing the inner flop clock pins of your OCC with the leaves of the functional domain clock tree that the OCC controls. This balancing adds a significant amount of latency to the OCC flop clocks, which causes a drastic reduction to the setup margin of the internal `fast_clock_en` and `slow_clock_en` timing paths of the OCC to the OCC clock gaters at the root of the controlled functional domain tree.

Locate the `tessent_get_cts_skew_groups_dict` inside the output `.sdc` file from the `extract_sdc` command, and follow the instructions provided in the banner of that proc to disable this balancing.

## Clock Tree Synthesis With On-Chip Controllers

**Figure 13-1. Locations for Exclude Points for OCC CTS**



The preceding figure illustrates a schematic of a Tessent standard OCC module, with red indicators showing where CTS exclude points must be applied to prevent balancing internal OCC logic with its driven clock tree. That balancing would considerably increase the OCC logic clock insertion delay; this would, in turn, shorten the setup margin of the OCC critical at-speed paths, shown by the dashed purple arrows, when the OCC runs in fast capture mode. This could sometimes make it impossible to close timing.

In the Tessent OCC, balancing must be blocked in the following locations, corresponding to the numbers in the figure:

1. The synchronizer cell that captures the transitions of the scan\_enable
2. The fast\_clock clock gater cell
3. The shift register block clock multiplexer

Adding a CTS exclude point directly at the output of the OCC would forbid any eventual balancing of the shift clock tree across multiple OCCs. This permits maximizing the shift clock frequency of the scan chain between scan domains and the EDT controller. It would also have blocked balancing of functionally synchronous clocks across different OCCs. You can still add any additional required CTS control commands after the OCC output.

The `extract_sdc` command writes a Tcl proc named `tessent_get_cts_skew_groups_dict` to the output SDC file. This proc returns a dictionary of the CTS exclude points or skew groups across all OCC instances in your design. You can use this dictionary with a custom script that issues

CTS commands specific to your layout tool. The proc includes instructions in the header comment to help you use the proc during CTS. Refer to the following example:

```
proc tessent_get_cts_skew_groups_dict {} {  
  
    # This proc returns a dictionary of clock source pins from where clock tree synthesis  
    # balancing should stop.  
    # Use it in your CTS script, along with your proper tool command.  
    # In Synopsys ICC, invoke:  
    #     set_clock_tree_exceptions -exclude_pins <exclude_pin>  
    # In Cadence Innovus, invoke:  
    #     create_ccopt_skew_group -sources <exclude_pin> -auto_sinks -skew_group <group name>  
    # The effect of that command is:  
    # * to prevent adding delay buffers to the small OCC internal clock tree, due to balancing  
    # with all flops in the OCC fanout, therefore helping the OCC clock_enable signals meet  
    # setup timing.  
    # * to prevents adding long delays to the SSN clock tree due to balancing with the  
    # functional clock tree in the fanout of each SSN scan host (SSH).  
    #  
    # You can use the dictionary the following way:  
    #     set cts_skew_groups_dict [tessent_get_cts_skew_groups_dict]  
    #     dict for {skew_group sub_dict} $cts_skew_groups_dict {  
    #         dict with sub_dict {  
    #             foreach pin $cts_exclude_pins {  
    #                 puts "$skew_group : $dc_instance/$pin"  
    #                 <insert your CTS command here>  
    #             }  
    #         }  
    #     }  
    #  
    set return_dict {  
        cts_skew_group(occ0) {  
            dc_instance      top_rtl2_tessent_occ_parent_clka_inst  
            cts_exclude_pins {occ_control/tessent_persistent_cell_ltest_ntc_sync_cell/CK  
            occ_control/tessent_persistent_cell_cgc_SHIFT_REG_CLK/CK  
            occ_control/tessent_persistent_cell_SHIFT_REG_CLK_mux/A1}  
        }  
        cts_skew_group(occ1) {  
            dc_instance      top_rtl2_tessent_occ_parent_clkb_inst  
            cts_exclude_pins {occ_control/tessent_persistent_cell_ltest_ntc_sync_cell/CK  
            occ_control/tessent_persistent_cell_cgc_SHIFT_REG_CLK/CK  
            occ_control/tessent_persistent_cell_SHIFT_REG_CLK_mux/A1}  
        }  
        ...  
    }  
    return $return_dict  
}
```

## Loading SDC Constraints in Layout With Tessent Logictest

In layout, some issues may limit one's ability to constrain the logictest DFT simultaneously with all other logic. For instance:

1. Letting scan\_en=X, unblocks a large number of false at-speed path between neighboring flops on the same scan chains, unless you have your own custom way to globally disable these paths in your own design environment and flow.



2. As opposed to synthesis, where all clocks are ideal, propagating test\_clock with scan\_en=X in layout may enable false cross-domain paths between unbalanced functional domains. Those paths become single-cycle paths of test\_clock which, if test\_clock is unbalanced, may cause both setup and hold false timing violation.
3. Some false at-speed capture paths might get unblocked between test-only dedicated wrapper cells and functional logic.

If your layout flow provides custom methods to work around these issues, you can run layout using only one global set of constraints that covers both your functional and DFT modes. In such a case, you would only need to call the tessent\_set\_non\_modal proc with no argument, or use the extracted .sdc file from synthesis. If not, then you need to use the multi-mode feature of your layout tool and sequentially load the following modal SDC constraints:

#### Mode 1:

- Set your functional SDC constraints.
- Call tessent\_set\_non\_modal with the argument “off”, like in:

```
tessent_set_non_modal off
```

This adds all Tessent DFT constraints, but disables all logictest DFT paths.

#### Mode 2:

- Call proc <ltest\_prefix>\_modal\_shift
- This forces the “shift” mode over your design, asserting scan\_en to 1 creating scan mode clocks and taking care of DFT signals, OCC, and EDT logic setup. This covers all of your scan chain and EDT channels paths, as well as any pipelining flop timing along the way.
- Run layout incrementally, so as to fix any leftover timing paths that were blocked in the first pass.

#### Mode 3:

- Call proc <ltest\_prefix>\_modal\_edt\_slow\_capture. This proc does the following:
  - Creates test\_clock and propagates it across the design.
  - Lets scan=X, which enables covering the timing of that signal as well as some leftover timing paths inside Tessent’s OCC clock gating circuit.
  - Disables same-edge paths from test\_clock to test\_clock, leaving only retimed cross-domain paths enabled, so as to prevent false timing violations across functional domains. Intra domain paths are assumed covered more tightly by the functional mode constraints.

- Enables capture paths across your design's top ports as single-cycle paths of test\_clock.
- Run layout incrementally again.

To save time and resources, you can choose to skip mode #3 if you know you are running scan mode at a very slow clock speed and that all your capture paths are guarded with extra cycles of test\_clock.

# SDC for Modal Static Timing Analysis

The following are guidelines to help you perform static timing analysis of the embedded test hardware that was inserted by Tessent Shell.

|                                                   |            |
|---------------------------------------------------|------------|
| <b>Checking Your Functional Logic Alone .....</b> | <b>983</b> |
| <b>Checking Your Embedded Test Logic.....</b>     | <b>984</b> |

## Checking Your Functional Logic Alone

Tessent automatically turns off logictest circuitry when you add the “tessent\_set\_non\_modal off” proc to your SDC file. For most designs, this is the only step you need. You can provide constraints for specific instruments to address any specific concerns.

If you want to only focus on functional logic and completely disable all Tessent Shell embedded test IP, include the following steps in your primary functional STA script.

If flops in your cell library do not propagate constant settings through their set or reset pin to their data output pin, you need to set a PrimeTime variable:

```
set case_analysis_sequential_propagation always
```

This turns off all embedded test modes with just a few asserts of the TAP’s reset port.

For example:

```
-----  
... loading your functional constraints ....  
set case_analysis_sequential_propagation always  
# Hold the DFT in reset and kill TCK to make sure only functional logic is  
# active.  
set_case_analysis TRST 0  
set_case_analysis TCK 0  
-----
```

You also need to disable your scan circuit timing by forcing your scan\_enable pin to its inactive value, and by issuing a set\_false\_path to/from all your lockup cells and your pipeline flops. Those can normally be found with regular expressions such as:

```
[get_pins -hier <standard naming>*/d]  
[get_pins -hier <standard naming>*/q]
```

You might also have to turn your BISR clock or reset pin off, if present, like in:

```
set_case_analysis bisr_clk      0  
set_case_analysis bisr_reset   0
```

When you are working with a physical block or sub-block, you can turn off all DFT signals accessible at its interface. For example:

```
set_case_analysis scan_en      0
set_case_analysis ijtag_tck    0
set_case_analysis ijtag_reset  1
set_case_analysis ijtag_sel    0
set_case_analysis ijtag_se     0
set_case_analysis ijtag_ce     0
set_case_analysis ijtag_ue     0
set_case_analysis ijtag_si     0
set_case_analysis edt_update   0
set_case_analysis edt_clock    0
```

Resetting the IJTAG logic should also reset any dedicated wrapper cells (DWC). If you want to explicitly identify and turn off DWCs or other test logic, you can use the [report\\_insert\\_test\\_logic\\_options](#) command to find the name infix used by the tool when it inserted the logic. Then use the introspection commands to find the instances you want. For example, the following returns pins on DWCs:

```
set_false_path -through [get_pins -hier *dihw_*/DI]
set_false_path -through [get_pins -hier *dihw_*/DO]
set_false_path -through [get_pins -hier *diw_*/DI]
set_false_path -through [get_pins -hier *diw_*/DO]
set_false_path -through [get_pins -hier *dohw_*/DI]
set_false_path -through [get_pins -hier *dohw_*/DO]
set_false_path -through [get_pins -hier *dow_*/DI]
set_false_path -through [get_pins -hier *dow_*/DO]
```

---

#### Tip

 To set your own naming convention, use the “[set\\_insert\\_test\\_logic\\_options -logic\\_infix](#)” command before test logic insertion.

---

## Related Topics

[tessent\\_set\\_non\\_modal](#)

## Checking Your Embedded Test Logic

You can find a self-documenting example script of how to run STA using Tessent Shell generated hardware.

Refer to “[Example PrimeTime STA Script](#)” on page 1025. Carefully read the instructions in this script before proceeding.

The script runs the following:

- Loads the generated SDC file and initializes some Tcl variables used by Tessent Shell SDC constraints.

- Sets some PrimeTime control variables to their required value.
- Sources user input files if present in the same directory as follows:
  - *user\_timing\_data.tcl*
  - *user\_remapping\_procs.tcl*
  - *load\_design.pt*
- Loads your final netlist.
- Sequentially runs all EmbeddedTest modes present in your design, each separated by a “reset\_design” command.
- Runs an example “report\_constraints” command for each mode and redirects its output to a file.

## VHDL and Mixed Language Designs

Synthesis tools have similar but sometimes different internal representations of elaborated designs. Here are some instructions on how to deal with those differences to be able to use the SDC constraints generated by Tessent Shell.

**VHDL Generate Blocks..... 986**

### VHDL Generate Blocks

Required tool settings and other mixed-language issues.

#### Synopsys Design Compiler

Constraints applying to cells and pins in VHDL generate loops are problematic to apply in `dc_shell`. The default naming scheme of the internally synthesized design is different for Verilog and VHDL. To illustrate, here is a Verilog path of an instance within two nested generate for loops: `blk[0].jblk[0].i0`. The equivalent configuration in VHDL would be changed during `dc_shell`'s GTECH pre-synthesis and would be named `i0_0_0`. Because Tessent Shell's SDC does not "flatten" generate loops, if you have VHDL generate loops in your design, you must use the following setting in `dc_shell` to restore the loop naming:

```
set_app_var hdlin_enable_upf_compatible_naming true
```

This variable ill restores VHDL generate loop naming to follow Verilog style: `blk(0)/jblk(0)/i0`

#### Cadence Encounter and Genus

There are no special steps for these two synthesis tools to be able to apply Tessent Shell generated SDC constraints to a mixed language design.

#### Oasys

Use the following commands in Oasys to generate a naming scheme for generate loops (Verilog and VHDL) that is compatible with the Tessent SDC constraints:

```
set_parameter generate_naming_style %s[%d]  
set_parameter if_generate_naming_style %s
```

#### Dealing with Unrolled VHDL Generate for Loops

Tessent Shell tries to minimize unrolling of generate loops. If VHDL generate loops are unrolled then one "if-generate" block is generated for each loop iteration. The trailing closing square brace, that would normally be changed to an underscore, has to be truncated because of VHDL language restrictions. This means that functional SDC constraints pointing to cells or pins within those generate blocks do not match even if special characters are mapped to

wildcards. Ex: the instance path blk[0].i0 which would need to be unrolled would be written out as blk\_0.i0.

For this reason, you should use the provided procs tessent\_get\_pins and tessent\_get\_cells. They are able to match your functional instance path “blk[0].i0” to blk\_0.i0 using a caching algorithm that searches for generate blocks in the path and tries to match with our unrolling strategy if the path does not match outright. Refer to the “[Mapping Procs](#)” on page 1016 to see the procs’ description.

## SDC File Contents

---

This section describes the procs contained in the generated SDC file. Note that many of these procs are called automatically by other procs in the file. You do not need to call all of these procs.

---

### Note



This section uses the following convention to shorten proc names:

`<ltest_prefix> := tessent_set_ltest`

---

|                                                                |             |
|----------------------------------------------------------------|-------------|
| <b>tessent_set_default_variables</b> .....                     | <b>988</b>  |
| <b>tessent_create_functional_clocks</b> .....                  | <b>989</b>  |
| <b>tessent_&lt;design_name&gt;_set_dft_signals</b> .....       | <b>989</b>  |
| <b>&lt;ltest_prefix&gt;_disable</b> .....                      | <b>990</b>  |
| <b>tessent_set_non_modal</b> .....                             | <b>991</b>  |
| <b>tessent_&lt;design_name&gt;_kill_functional_paths</b> ..... | <b>991</b>  |
| <b>IJTAG Instrument</b> .....                                  | <b>992</b>  |
| <b>LOGICTEST Instruments</b> .....                             | <b>993</b>  |
| <b>MemoryBIST Instrument</b> .....                             | <b>1007</b> |
| <b>BoundaryScan Instrument</b> .....                           | <b>1010</b> |
| <b>Hierarchical STA in Tessent</b> .....                       | <b>1011</b> |
| <b>Mapping Procs</b> .....                                     | <b>1016</b> |
| <b>Synthesis Helper Procs</b> .....                            | <b>1017</b> |

## tessent\_set\_default\_variables

This proc defines the initial/default value of all global variables used in all procs of the sdc file. The variables contained in this proc are there for you to customize the constraints without having to edit the constraints themselves. A good example of this would be the variable “tessent\_tck\_period”. It currently defaults to 100.0 (nanoseconds). If your TCK clock has a different period than 100ns, you can overwrite the value of this variable after having called the proc tessent\_set\_default\_variables but before calling the constraint procs.

---

### Note



The value of tessent\_tck\_period might depend on the maximum tck clock frequency that can be applied to the circuit. See the “[IJTAG Network Performance Optimization](#)” section in the *Tessent IJTAG User’s Manual* showing how to maximize the frequency of the IJTAG network test clock.

---



The mapping between LogicBIST and InSystemTest controller IDs and their instance path names is available in this proc. Use the mapping information to indicate the clocks of these instruments to the other ltest SDC procs as described in “[Preparation Step 2: Setting and Redefining Tessent Tel Variables](#)” on page 972.

This proc must always be the first proc to be run.

## tessent\_create\_functional\_clocks

This proc is generated for your convenience. It contains the SDC create\_clock and create\_generated\_clock commands to define the various functional clocks needed by the instruments constrained by the SDC procs. Typically, your functional clocks would already be defined by your functional SDC. If that is the case, you should override the default values of the tessent\_clock\_mapping array to match the clock names you used in the definition of your clocks.

This proc would typically not be called in your synthesis SDC.

Required clocks are determined with ICL tracing. create\_clock constraints are added for all clock sources like top level ClockPort ports and source ToClockPort ports (embedded crystals). create\_generated\_clock constraints are added for all generated clocks (add\_clocks -reference), branch clocks and unidentified ICL ToClockPort ports (typically PLLs). If your design contains an ICL instrument with a ToClockPort that should not require a new generated clock, please set the [exclude\\_from\\_sdc](#) attribute on the port.

## tessent\_<design\_name>\_set\_dft\_signals

This procedure forces all your specified static dft\_signal sources to either reset or get their default value during all\_test, depending on the proc's argument.

argument mode ::= reset | logic\_off

- **logic\_off** — Recommended argument for running non-logictest STA modes such as memoryBIST. But if you added Siemens EDA TestKompress, your sdc file also gets the proc <ltest\_prefix>\_disable. You then only need to call that proc for running the other non-logictest modes such as modal memoryBIST.
- **reset** — Disables all DFT-inserted logic.

If you are using functional-only mode, you can optionally call the proc with the “reset” argument under one of the following circumstances:

1. Many static dft signals come from top-level ports.
2. Although all static dft signals sourced by the Siemens EDA IJTAG Test Data Registers (TDR) reset when the IJTAG reset port is asserted, the timing tool would not propagate

that assert through the TDR flops. That may happen in Synopsys PrimeTime when the control variable “case\_analysis\_sequential\_propagation” is set to “never”.

## <ltest\_prefix>\_disable

This proc disables all Logictest logic in your design. You call this proc when setting up exclusive STA mode for your IJTAG, BoundaryScan or MemoryBIST, and you do not want to bother about ltest logic and scan chains timing at the same time.

This proc is invoked by proc tessent\_set\_non\_modal when called with the logictest argument set to “off”. Specifying “on” would invoke proc tessent\_set\_ltest\_non\_modal proc instead. You would typically use “on” during pre-layout synthesis and “off” during layout. In layout, you would then need to apply two different mode scripts in your layout tool. The second mode would time logictest-only shift logic, for which you would invoke the proc <ltest\_prefix>\_shift. For more information about this proc, see “<ltest\_prefix>\_modal\_shift” on page 1000.

You can call the proc with or without an argument:

- <ltest\_prefix>\_disable — Disables logictest controller and forces the all\_test dft signal active. Use this when setting up the membist STA mode. For an example, see “[Example PrimeTime STA Script](#)” on page 1025.
- <ltest\_prefix>\_disable all\_test\_x — Disables the logictest controller but does not force the all\_test dft signal. Use this when constraining your design for synthesis or for layout. For more information, see “[tessent\\_set\\_non\\_modal](#)” on page 991

When SSH is present in the design, this proc permits ts\_tck\_tck to propagate to the SSH logic so it can properly time the serial scan path to and from the IJTAG network. It assumes that timing paths between the SSH IJTAG registers and the SSH logic always meet a single-cycle path of ts\_tck\_tck, with no excessive stress to the quality of results (QoR) for the layout. Clock tree synthesis can balance both branches of ts\_tck\_tck within the SSH logic.

In this case, constraints added at the SSH scan clock sources and scan\_en pin prevent ts\_tck\_tck from leaking into the scan domains and prevent potential false paths to memory BIST logic clocked by tck. The following example illustrates these constraints:

```
# Stop ts_tck_tck from propagating to core logic
set_disable_timing [tessent_get_pins $tessent_ssh_mapping(ssh0)/ \
  clock_gen/clock_signals/tessent_persistent_cell_edt_clock_mux/Y]
set_disable_timing [tessent_get_pins $tessent_ssh_mapping(ssh0)/ \
  clock_gen/clock_signals/ \
  tessent_persistent_cell_shift_capture_clock_mux/Y]
# Prevent false paths to scannable tessent memory_bist logic, which uses
# ts_tck_tck in setup mode.
set_false_path -from [tessent_get_clocks \
  $tessent_clock_mapping(ts_tck_tck)] \
  -through [tessent_get_pins $tessent_ssh_mapping(ssh0)/ \
  tessent_persistent_cell_scan_en_buf/Y]
```

## tessent\_set\_non\_modal

This proc contains calls to the non\_modal procs of all constrained instruments, which are documented in the sections that follow. For synthesis, this is the only constraint proc you need to call.

You can call this proc with no argument, or with the argument “off”:

```
tessent_set_non_modal off
```

The argument “off” disables logictest circuitry by asserting dft\_signal source pins and prevents adding any non-modal logictest clocks or constraints. The default is to call the proc tessent\_set\_ltest\_non\_modal.

This is an example of the tessent\_set\_non\_modal proc using the design “blka”:

```
proc tessent_set_non_modal {{logictest "on"}} {  
    tessent_set_ijtag_non_modal  
    tessent_set_memory_bisr_non_modal  
    tessent_set_memory_bist_non_modal  
    if {$logictest eq "on"} {  
        tessent_set_ltest_non_modal  
    } else {  
        tessent_set_ltest_disable all_test_x  
    }  
}
```

For an example of how to use the proc, see “Single-Mode vs Dual-Mode Constraining in Synthesis/Layout” in “<ltest\_prefix>\_non\_modal” on page 1003.

## tessent\_<design\_name>\_kill\_functional\_paths

This procedure disables all timing paths involving non-Tessent flops. It enables running test-only timing analysis without having to fix functional path violations with your own functional SDC constraints. It helps making functional mode STA and Siemens EDA DFT mode STA fully orthogonal, and reduces the Siemens EDA DFT mode STA run-time.

### Caution



Do not call this procedure for any logic test STA modes. It is only intended for use in the Siemens EDA DFT mode, as shown in “[Example PrimeTime STA Script](#)” on page 1025.

You can optionally run this procedure when you know that your functional logic has many intra-domain timing paths that cannot meet default single-cycle path timing with the clocks being set by tessent\_create\_functional\_clocks and tessent\_set\_\* procs.

Disabling some sequential cells in your design might accidentally block the test clocks feeding your inserted instruments. Examples of such cells are Integrated Clock Gating (ICG) cells or

flops that belong to clock dividers. You can fix this by specifying those cells either by module name or by instance names.

By module name:

```
set ClockSeqCellModuleRegExp "<regularExpression>"
```

---

**Note**

---

 This only finds library cells that match this module name pattern. If your module is hierarchical and the cells you are trying to preserve are below it, use the instance name option described in the following.

---

By instance name:

```
set ClockSeqCellInstanceRegExp "<regularExpression>"
```

---

**Note**

---

 This expression is tested against the full hierarchical instance paths. A pattern that would match a hierarchical instance preserves all cells within it.

---

The procedure optionally creates a report file containing a sorted list of all disabled flops instance names. To enable this option, you need to define the following variable prior to calling `tessent_kill_functional_paths`:

```
set CreateDisabledFlopsReport 1
```

## IJTAG Instrument

Tessent IJTAG provides the following procs:

- `set_ijtag_retargeting_options`
  - This proc would typically be called directly in your synthesis scripts to set the supported options and related variables.

This proc is a replica of the Tessent Shell command [set\\_ijtag\\_retargeting\\_options](#) supporting options that are pertinent to SDC. Any unsupported option specified to the SDC version of the command is ignored.

**Table 13-1. `set_ijtag_retargeting_options` Arguments and SDC Variables**

| Option                   | SDC Timing Variable             | Default  |
|--------------------------|---------------------------------|----------|
| <code>-tck_period</code> | <code>tessent_tck_period</code> | 100.0 ns |

- `tessent_set_ijtag_non_modal`

- This proc would typically not be called directly in your synthesis script, it is called as part of `tessent_set_non_modal`.

This proc contains the clock creation for your TCK clock and configurable input and output delays for created ports at the `sub_block` and `physical_block` design levels.

It also creates an asynchronous clock group with all TCK clocks. It is used to prevent synthesis from balancing paths between TCK and functional clocks. This clock group could be recreated with additional clocks if your design contains Tessent BoundaryScan or Tessent OCC hardware. This is managed via the “`tessent_tck_clock_list`” global variable. This variable is built from different procs (`ijtag` and `jtag_bscan`) and used to create an asynchronous clock group. If you have your own TCK generated clocks that are not used in the Tessent Shell SDC constraints, we suggest adding their name to this list right after calling `tessent_set_default_variables`.

```
tessent_set_default_variables
lappend tessent_tck_clocks_list my_tck_generate_clock
```

At the sub-block and physical block levels, constraints are added to reflect the timing of the IJTAG protocol. The IJTAG reset port is declared a false path and the IJTAG select port's hold paths are declared false and its setup paths are given two cycles with a `multicycle_path` constraint.

The select signal source of all non-scan SIBs (part of the SRI network) is relaxed to MCP setup 2 hold 2. This is to reflect the protocol and enable deep combinational paths to close timing. This constraint did not include the SIBs part of the STI network as those should be local to each physical region, and it should be easier for timing tools to close timing. If you experience problems with scan-testable SIB select signals, you can add those to the list of SIBs that are relaxed. However, this has an adverse impact on scan patterns.

## LOGICTEST Instruments

The section lists the logic test-based procs. LogicBIST and InSystemTest-related procs listed in the following are included only when LogicBIST and InSystemTest controllers are present in the design.

### Note



This section uses the following convention to shorten proc names:

```
<ltest_prefix> := tessent_set_ltest
```

1. Non-modal constraints, to be merged with other DFT instrument constraints, for synthesis or layout. See this proc description for a discussion on single-mode synthesis/layout vs multi-mode synthesis/layout flow:
  - [`<ltest\_prefix>\_non\_modal`](#)

- `tessent_set_in_system_test_non_modal`
- 2. Modal procs, used for signoff STA and optionally for synthesis or layout constraining:
  - `<ltest_prefix>_modal_shift`
  - `<ltest_prefix>_disable`
  - `<ltest_prefix>_modal_lbist_shift`
- 3. Procs only for per-mode signoff STA constraints:
  - `<ltest_prefix>_edt_shift`
  - `<ltest_prefix>_bypass_shift`
  - `<ltest_prefix>_single_bypass_chain_shift`
  - `tessent_set_edt_slow_capture`
  - `<ltest_prefix>_edt_fast_capture`
  - `<ltest_prefix>_modal_lbist_shift`
  - `<ltest_prefix>_modal_lbist_capture`
  - `<ltest_prefix>_modal_lbist_setup`
  - `<ltest_prefix>_modal_lbist_single_chain`
  - `<ltest_prefix>_modal_lbist_controller_chain`
- 4. Procs sub-invoked by the preceding procs.
  - `<ltest_prefix>_create_clocks`
  - `<ltest_prefix>_set_pin_delays`

### **<ltest\_prefix>\_create\_clocks**

This proc creates the slow-speed test clocks for use during scan mode. There are a few possible clock configurations:

- **tessent\_test\_clock** — This is a DFT signal in Tessent Shell. It should be your tester's synchronous clock and is used to create the `shift_capture_clock` and the `edt_clock`.
- **tessent\_edt\_clock** — This clock is created when the `edt_clock` source is a primary input port, separate from the `shift_capture_clock`. It is not necessary when `edt_clock` is generated from the `test_clock`.
- **tessent\_shift\_capture\_clock** — This clock can either be a primary input port or it can be generated from the `test_clock` and is the main scan mode clock propagating to your functional logic through the OCCs.

- **tessent\_virtual\_force\_pi** and **tessent\_virtual\_measure\_po** — Two virtual clocks of the same frequency as the preceding **tessent\_test\_clock**, used to time all top-level ports that directly interface with your EDT controllers and your scan chains, through the “set\_input\_delay” and “set\_output\_delay” constraints.
- **tessent\_lbist\_shift\_clock\_src** — This clock is typically an internally generated free-running clock used as the source for LogicBIST tests.

When using LogicBIST, the clock gates (CGC) shown in the following figure for **tessent\_edt\_clock** and **tessent\_shift\_capture\_clock** are located inside the LogicBIST controller, and the clock source automatically updates to this new location. This clock is created by the user outside of the Tessent-generated SDC.

This proc is also called by every SDC proc that propagates the **edt\_clock** or **shift** clocks, specifically:

- `<ltest_prefix>_non_modal`
- `<ltest_prefix>_shift`
- `<ltest_prefix>_modal_slow_capture`
- `<ltest_prefix>_modal_lbist_shift`
- `<ltest_prefix>_modal_lbist_capture`

The following is an example of the created clocks you get if you run the following commands:

```
add_dft_signals test_clock -source_nodes [get_ports test_clock]
add_dft_signals {edt_clock shift_capture_clock} -create_from_other_signals
```

```

set slow_clock_period \
    [expr $tessent_slow_clock_period * $time_unit_multiplier]
set sc_rise_time \
    [expr $tessent_shift_clock_edge1_percentage/100. * $slow_clock_period]
set sc_fall_time \
    [expr $tessent_shift_clock_edge2_percentage/100. * $slow_clock_period]
set sc_waveform "$sc_rise_time $sc_fall_time"
set force_pi_rise \
    [expr $tessent_force_pi_percentage/100. * $slow_clock_period]
set measure_po_rise \
    [expr $tessent_measure_po_percentage/100. * $slow_clock_period]
set min_width [expr 0.25 * $slow_clock_period]
set force_pi_waveform \
    "$force_pi_rise [expr $force_pi_rise + $min_width]"
set measure_po_waveform \
    "$measure_po_rise [expr $measure_po_rise + $min_width]"

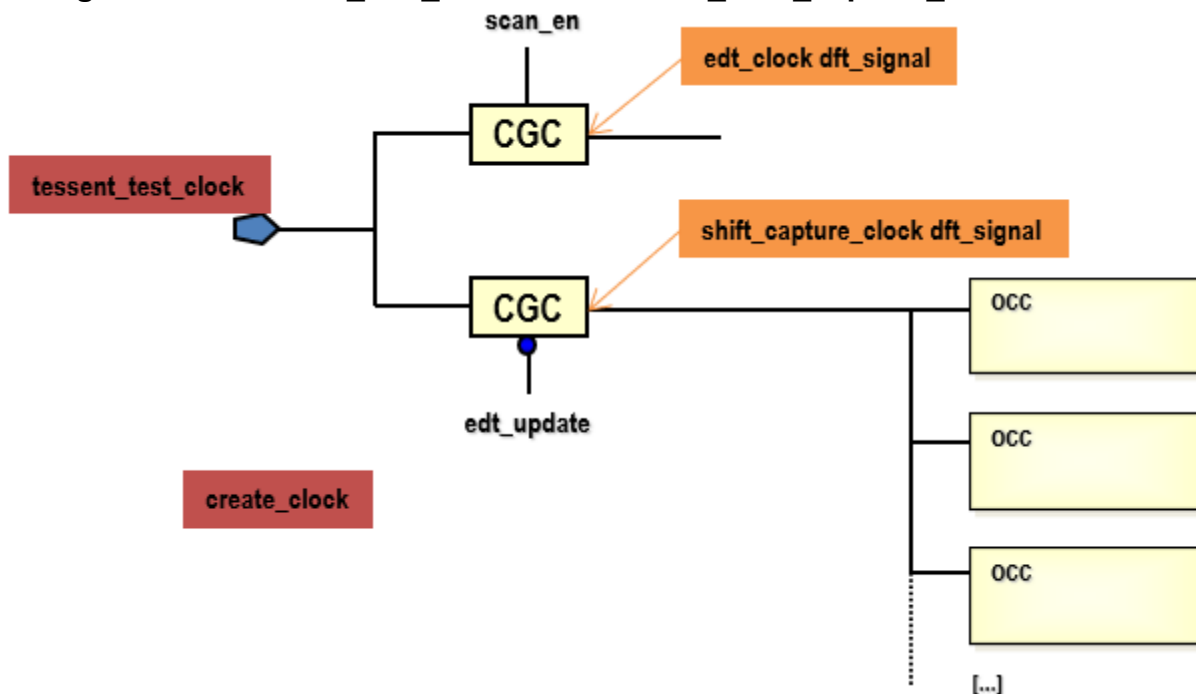
# test_clock:
create_clock [tessent_get_ports test_clock] \
    -add -period $slow_clock_period -waveform $sc_waveform \
    -name tessent_test_clock

# Virtual force_pi clock, to comply with your timeplate "force_pi"
# specifications:
create_clock -period $slow_clock_period -waveform $force_pi_waveform \
    -name tessent_virtual_force_pi

# Virtual measure_po clock, to comply with your timeplate "measure_po"
# specifications:
create_clock -period $slow_clock_period -waveform $measure_po_waveform \
    -name tessent_virtual_measure_po

```

**Figure 13-2. tessent\_test\_clock and tessent\_shift\_capture\_clock Creation**





## Tessent Slow Clocks Waveform Definitions

To maintain consistency of the scan clocks and scan ports timing between your SDC and your backannotated scan simulations, you need to define the following variables based on your Fastscan timeplate specifications. These variables shape the waveforms of the `tessent_test_clock`, `tessent_virtual_force_pi`, and `tessent_virtual_measure_po` clock definitions in your SDC. The variables are as follows:

- **tessent\_slow\_clock\_period**

Tessent slow clock period (the same for all clocks).

Default: 40ns.

- **tessent\_shift\_clock\_edge1\_percentage**

Timeplate position of the shift clock rising edge as a percentage of its period.

Default: 50.

- **tessent\_shift\_clock\_edge2\_percentage**

Timeplate position of the shift clock falling edge as a percentage of its period.

Default: 75.

- **tessent\_force\_pi\_percentage**

Timeplate position of the `force_pi` point as a percentage of the shift clock period.

Default: 0.

- **tessent\_measure\_po\_percentage**

Timeplate position of the `measure_po` point as a percentage of the shift clock period.

Default: 25.

Using percentages instead of absolute time values enables you to quickly modify the `tessent_slow_clock_period` specification without also having to update the other Tcl variables.

The defaults described in the preceding reflect the Tessent FastScan timeplate default specifications, that is:

```
timeplate gen_tpl =  
    force_pi 0 ;  
    measure_po 10 ;  
    pulse_clock 20 10 ;  
    period 40 ;  
end;
```

resulting into the following default clock definitions:

```
create_clock <port> -add -period 40. -waveform {20 30} -name tesseract_test_clock
create_clock -period 40. -waveform {0 10} -name tesseract_virtual_force_pi
create_clock -period 40. -waveform {10 20} -name tesseract_virtual_measure_po
```

Finally, two more global Tcl variables might be necessary to specify the timing budget of the scan data paths outside of the current module:

- **tesseract\_scan\_input\_delay**

Default value: 0.

- **tesseract\_scan\_output\_delay**

Default value: 0.

Update these variables if you plan to retarget your test vectors from a higher level module and you budgeted some propagation delay for your scan signals in that module. Here is how the variables are used:

```
set_input_delay $tesseract_scan_input_delay -clock tesseract_virtual_force_pi <port>
set_output_delay $tesseract_scan_output_delay -clock tesseract_virtual_measure_po <port>
```

Currently, the [extract\\_sdc](#) command assumes that timeplate specified values are identical for both load\_unload and capture phases, which is the Tesseract FastScan default. Although, the “non\_modal” ltest proc has to use only one set of specifications, modal shift and capture STA may have to run with different timeplate specifications. If it is your case, you can do it by updating the preceding Tcl variables prior to calling the selected shift or capture modal proc. The fast\_capture modal proc ignores those values, because it uses your functional waveform specifications.

### **<ltest\_prefix>\_edt\_fast\_capture**

This proc must be called in your STA to constrain the EDT in fast capture mode. Logictest fast capture mode is the one that detects your at-speed transition faults. It counts on user SDC to provide the capture clocks and timing exceptions that would actually time the capture paths. Typically this would be your original functional SDC constraints, but those might still require some adjustments if your test conditions (clock frequencies, false paths) slightly differ from what they are in functional mode. Obviously, you also need to remove any constraint that resets the JTAG network or ties the scan\_enable or test\_enable control pins to zero.

If present, the following dft signals are turned off:

- scan\_enable (putting the circuit in capture mode)
- control\_test\_point\_en (these hold during capture)

The proc also enables you to check that some dft signals such as `x_bounding_en`, `memory_bypass_en`, `observe_testpoint_en` are doing their isolation job correctly. You can optionally force them to a known value by setting global Tcl variables prior to calling the proc.

The following demonstrates how this is done (annotations are in red):

```
global memory_bypass_en_value <<< set this variable if needed in
                                   your primary script
if {[info exists memory_bypass_en_value]} {
    set_case_analysis $memory_bypass_en_value \
        [tessent_get_pins eltl1_mbist_tessent_tdr_sri_ctrl_inst/
tessent_persistent_cell_memory_bypass_en/Y]
}
global x_bounding_en_value <<< set this variable if needed in
                              your primary script
if {[info exists x_bounding_en_value]} {
    set_case_analysis $x_bounding_en_value \
        [tessent_get_pins eltl1_dft_tessent_tdr_sri_ctrl_inst/
tessent_persistent_cell_x_bounding_en/Y]
}
global observe_test_point_en_value <<< set this variable if needed
                                      in your primary script
if {[info exists observe_test_point_en_value]} {
    set_case_analysis $observe_test_point_en_value \
        [tessent_get_pins eltl1_dft_tessent_tdr_sri_ctrl_inst/
tessent_persistent_cell_observe_test_point_en/Y]
}
```

Finally, the proc also disables any other timing paths that are ignored during capture, such as those going through the EDT channel pins.

## tessent\_set\_edt\_slow\_capture

This proc must be called in your STA to constrain the EDT in slow capture mode.

The proc does the following:

- Forces the test\_enable dft signals on.
- Declares the test\_clock and lets your scan\_en dft signal toggling, so that it can be timed and relaxed with your number of dead cycles specification.
- Disables same-edge paths from test\_clock to test\_clock, leaving only retimed cross-domain paths enabled, so as to prevent false timing violations across functional domains. Intra domain paths are assumed covered more tightly by the functional mode constraints; enables capture paths across your design's top ports as single-cycle paths of test\_clock.
- Disables ignored timing paths.
- Sets an MCP for your scan\_en signal using your global Tcl variables `tessent_scan_en_setup_extra_cycles` and `tessent_scan_en_hold_extra_cycles`.

- Set an MCP for your `edt_update` signal, if present, using your global variables `tessent_edt_update_setup_extra_cycles` and `tessent_edt_update_hold_extra_cycles`.
- Enables capture paths across your design's top ports as single-cycle paths of `test_clock`.

---

**Note**

 Just as with the preceding `<prefix>shift_ltest_non_modal` proc, propagating `test_clock` may create false capture timing paths across asynchronous domains as single-cycle paths of `test_clock`. This may or may not be a problem, given that `test_clock` fanout is balanced and running at slow speed. By default, this proc assumes that all valid intra-domain hold timing paths were already covered by your functional modes STA, and therefore sets a `multicycle_path` of 1 hold to all paths from `test_clock` to itself. Remember that all scan mode shift paths are properly covered for both setup and hold by the “`xxx_ltest_shift`” mode proc defined in the preceding. If you need to, you can still force the proc to revert to zero-hold constraint by setting the Tcl variable `tessent_time_hold_in_slow_capture` to value 1 in your calling script.

---

### **`<ltest_prefix>_modal_shift`**

This proc sets the circuit in scan shift mode. It covers both EDT shift modes and all available bypass modes, assuming you run them all with the same shift clock frequency. If you need more specific timing analysis, you can choose to apply the following procs in separate STA runs. These procs sub-invoke the `<ltest_prefix>_shift` proc:

- `<ltest_prefix>_edt_shift`
- `<ltest_prefix>_bypass_shift`
- `<ltest_prefix>_single_bypass_chain_shift`

The `<ltest_prefix>_shift` proc applies a set of common constraints required for shifting. Then they only need to complete their mode setting by forcing the `edt_bypass` and `single_chain_bypass` signals to the required constant value.

The proc `<ltest_prefix>_shift` does the following:

- Creates the shift clocks.
- Applies `set_input/output_delays` to scan control pins.
- Forces dft signals `ltest_en` and `scan_en` to 1.
- Applies any LogicBIST mode constraints as applicable.

---

**Note**

 You can save on total STA time and flow complexity if you already plan to apply the same `test_clock` (with the same period) for all shift and bypass modes. In this case you only need to call the `<ltest_prefix>_shift` proc directly in place of all of the three procs defined in the preceding.

---

---

**Note**

 If you plan to feed your synthesis or layout tool with two separate mode scripts (functional/dft and logictest), the `<ltest_prefix>_shift` proc is the one you need to apply for the logictest mode. That way, all possible shift paths are timed, and because scan\_enable is forced active, it prevents the existence of a potentially large number of false capture timing paths across your functional logic. It also covers all of your possible scan chain configurations at once, including the internal and external mode for cores. See later discussion on logictest single-mode or dual mode usage—see `<ltest_prefix>_non_modal`.

---

### `<ltest_prefix>_modal_lbist_shift`

You can configure timing analysis for this mode to run with `shift_clock_src`, `test_clock`, or `ijtag_tck`. The default is `shift_clock_src`. Add the optional `test_clock` or `ijtag_tck` argument when calling this SDC Tcl proc to analyze timing with `test_clock` or `ijtag_tck`, respectively.

The proc does the following:

- Adds multicycle path exceptions on dynamic signals from the LogicBIST controller to design scan cells and hybrid EDT blocks. These include LogicBIST mode scan enable, `prpg_en`, `misr_en`, and LogicBIST synchronous reset for the hardware default mode of operation.
- Adds case analysis on the mux, which chooses between the top-level scan enable for ATPG and the LogicBIST controller-generated scan enable to enable only the LogicBIST mode paths.
- Adds case analysis to set `lbist_en` to active.
- Disables single chain mode scan chain concatenation paths.
- Disables paths from `edt_chain_mask` masking registers to the scan cells. This constraint is required because even though this register is configured using `tck` as the source, the clock net connected to the `edt_lbist_clock` signal carries both `edt_clock` and `shift_clock_src` clocks.

### `<ltest_prefix>_modal_slow_capture` and `<ltest_prefix>_modal_fast_capture`

You get these procs in your SDC file only under the following conditions:

- You added the following dft signals in your current design level: `scan_en`, `test_clock`, or (`edt_clock` and `shift_capture_clock`).
- And you did not insert a Tessent logictest-related controller, such as EDT or logicbist in that same level.

These two procs properly assert all static dft signals, such as scan\_en and ltest\_en. They are the “no-EDT” version of the following two procs:

- `<ltest_prefix>_modal_edt_slow_capture`
- `<ltest_prefix>_modal_edt_fast_capture`

Because no EDT is present in the current design, `extract_sdc` does not generate these procs:

- `<ltest_prefix>_edt_bypass_shift`
- `<ltest_prefix>_edt_single_chain_shift`

However, if your design meets all of the following conditions:

- Your core instances actually do support `edt_bypass` chain or `single_chain` modes.
- You intend to run these modes at a `test_clock` frequency that differs from their EDT mode.
- You intend to use core-extracted timing models in a hierarchical STA flow.

Then you should load a per-mode core timing model, and then run the `<ltest_prefix>_modal_shift` proc as a general setup for the top-level. For each such STA run, you must properly set the global Tcl variable `tessent_slow_clock_period` value.

For `<ltest_prefix>_modal_edt_fast_capture`, all `edt_bypass` and `edt_chain_bypass` asserts are skipped by default. If you do not want to skip these asserts, define the “`tessent_block_edt_bypass_in_fast_capture`” global variable.

### **`<ltest_prefix>_modal_lbist_capture`**

This mode is similar to `<ltest_prefix>_modal_lbist_shift`, except that the LogicBIST mode scan enable is constrained to off.

To use this proc in your SDC file, you must do the following:

1. Define all functional clocks.
2. Based on the clocking scheme, declare false paths between clock domains that do not pulse together.

### **`<ltest_prefix>_modal_lbist_setup`**

This mode propagates `tck` through the IJTAG network within the LogicBIST controller and hybrid EDT blocks to time the LogicBIST `test_setup` paths that initialize registers such as `PRPG` and `edt_chain_mask`, as well as `test_end` paths that read the MISR signature.

This mode does not propagate `tck` to the design scan cells because it is only intended to check the IJTAG network paths.

## <ltest\_prefix>\_modal\_lbist\_single\_chain

This mode is available when single chain mode logic that enables LogicBIST diagnosis is enabled during IP generation. This mode propagates tck through the JTAG network paths and design scan cells, including the concatenation of the internal scan chains for single-chain shifting. The LogicBIST scan enable is constrained to 1 to time only the shift paths in the design with the tck signal.

## <ltest\_prefix>\_modal\_lbist\_controller\_chain

This mode is available when controller chain mode (CCM) logic is enabled during IP generation. When CCM is present, all of the previously described LogicBIST modes disable the controller chain logic by constraining the ccm\_en signal to off. This mode tests the CCM paths by constraining the ccm\_en signal to on. The clock for this mode is either tck or test\_clock, as specified during IP creation.

This mode only tests the LogicBIST and hybrid EDT controller blocks; clocks are not propagated to design scan cells.

Paths that are normally disabled during the LogicBIST operation modes described in the preceding become valid single-cycle paths in CCM and are timed accordingly.

- Signals such as lbist\_en are not constrained to 1 because paths from the TDR register in the LogicBIST controller to the hybrid EDT blocks that use this signal are tested with ATPG in CCM. Paths from static signals—such as x\_bounding\_en, test\_point\_en, and capture\_per\_cycle—to the design scan cells are correctly excluded because no clocks are propagated to the scan cells.
- Paths from dynamic control signals that are declared as multicycle for other modes—such as scan\_en, prpg\_en, and misr\_en—that go from the LogicBIST controller to the EDT blocks are tested as valid single-cycle paths.
- Paths from JTAG network nodes such as SIBs are timed as valid single-cycle paths of the CCM clock.
- Input and output pin delays from top-level ports to EDT blocks and CCM scan I/O ports are included for this mode. This differs from the LogicBIST shift and capture modes, which do not have any active paths from or to design ports.

## <ltest\_prefix>\_non\_modal

This procedure provides SDC timing constraints to add to your combined functional-dft non-modal timing scripts for one-pass synthesis or layout. Its contents is merged with both your functional constraints and all other Tessent controller's non-modal constraints. It is called by the umbrella proc “tessent\_set\_non\_modal” proc.

The procedure calls the previously described procs:

- `<ltest_prefix>_create_clocks`
- `<ltest_prefix>_set_pin_delays`

It also:

- Defines a specific clock group (using 'set\_clock\_groups') for all logictest slow clocks, isolating them from your declared functional clocks.
- Adds “set\_false\_path” constraints from each of your primary ports assigned to a static dft signals such as “all\_test”, “ltest\_en”, and “edt\_mode”. If the same signals come instead from an internal IJTAG Test Data Register (TDR), the “set\_clock\_groups” command between ts\_tck\_tck and the scan clocks replaces the individual set\_false\_path commands.
- Provides constraints to prevent “tck” and your functional clocks to propagate to unwanted paths inside your Siemens EDA OCCs, as well as constraints that prevent false timing violations on the select pin of clock muxes inside that OCC.
- Provides constraints that disable automatic clock gating checks across input pins of the Siemens EDA OCC clock multiplexers, because all OCC clock selection is either performed statically during test setup or at slow speed in a glitchless way during scan.
- Adds an optional set\_multicycle\_path constraint from your primary 'scan\_en' port and “edt\_update” signal, based on your setting of the global variables:
  - `tessent_scan_en_hold_extra_cycles`
  - `tessent_scan_en_setup_extra_cycles`
  - `tessent_edt_update_hold_extra_cycles`
  - `tessent_edt_update_setup_extra_cycles`

which all default to zero, meaning single-cycle paths of slow clock.

- When using LogicBIST, includes a 3-to-1 shift\_clock\_select mux at the clock structure root of the LogicBIST controller. This mux enables any one of three clocks—shift\_clock\_src, test\_clock or tck—to be used as the source clock for LogicBIST test.
  - Blocks tck propagation through the shift\_clock\_select mux into the design scan cells. This removes analysis of the slowest LogicBIST mode of operation using tck to reduce timing analysis run time.
  - Propagates both shift\_clock\_src and test\_clock to the design scan cells, but all interactions between them are blocked.



- Adds the following LogicBIST-related exceptions:
  - Declares multicycle paths from LogicBIST scan enable generation logic to the design scan cells. The number of cycles are based on the `pre_post_shift_dead_cycles` IP generation parameter and defaults to eight.
  - Declares multicycle paths from logic that generates the `prpg_en`, `misr_en`, and LogicBIST synchronous reset for hardware default mode operation signals to the hybrid EDT blocks. The number of cycles are based on the `pre_post_shift_dead_cycles` IP generation parameter.
  - Disables paths from static control registers in the LogicBIST controller—such as `lbist_en` and `x_bounding_en`—to design scan cells and hybrid EDT blocks. This is implicitly performed by declaring `tck` as asynchronous to other clocks in the IJTAG non-modal proc. When CCM is implemented with the EDT clock as the CCM clock, explicit false paths are added to disable such paths.
- Excludes single chain mode logic scan chain concatenation paths through case analysis on the single bypass chain control TDR output.
- When CCM is implemented with `test_clock` as the CCM clock, the `test_clock` propagates to all IJTAG scan elements on the `tck` domain. Constraints are added to disable such paths to the `tessent_lbist_shift_clock_src` clock.

### Single-Mode vs Dual-Mode Constraining in Synthesis/Layout

**Important:** As it stands in the current release, leaving the `scan_enable` signal free to toggle in this proc is known to create false `shift_clock`-frequency timing paths across your functional design, which may or may not affect timing closure or quality of results in layout. Siemens EDA customers' experience greatly varies on this front. Here are some examples of such false paths:

1. Scan chains intra-domain shift-only paths are constrained at the speed of the functional clock.
2. As a result of creating only one `shift_capture_clock` source and letting it propagate to all functional clock domains, all cross-domain capture paths become single-cycle of the shift clock, regardless of whether they are asynchronous in functional mode or not.

This said, these new timing paths do not instantly condemn your layout or physical synthesis tool to fail timing closure or perform a bad P&R job, knowing that clock tree synthesis also balances the `test_clock` fanout by default. Non-physical synthesis is generally not a problem.

On the other hand, re-constraining of all those bogus timing paths would typically require a very large number of design-dependent timing exceptions, which [extract\\_sdc](#) is not able to provide at this point. Applying them could also slow down some layout tools considerably.

The alternative to single-mode constraining is to apply individual mode scripts at different times in the synthesis or layout tool. It is a well-known solution that has been applied by most Siemens EDA customers historically.

1. Functional/Dft mode:
  - Apply your functional constraints
  - Invoke proc constrain\_<design\_name>\_non\_modal off
2. Logictest-only mode, covering all EDT scan configurations, such as EDT shift and EDT bypass shift:
  - Invoke proc <ltest\_prefix>\_modal\_shift
3. If your design contains hybrid EDT/LBIST, another logictest-only mode to cover the LBIST shift configuration:
  - Invoke proc <ltest\_prefix>\_modal\_lbist\_shift

This script cannot be combined with EDT scan configuration script because these modes are not compatible; they propagate different shift clocks, have different case analysis constraints on the lbist\_en signal, and have different timing requirements for scan enable with respect to the shift clock.

### <ltest\_prefix>\_set\_pin\_delays

This procedure assigns the clock “tessent\_virtual\_slow\_clock” to your top-level ports that directly interface with your EDT controllers or your scan chains during scan or EDT mode, using the “set\_input\_delay” and “set\_output\_delay” timing constraints.

This proc is called by every SDC proc which propagate the EDT or shift clocks, that is:

- <ltest\_prefix>\_non\_modal
- <ltest\_prefix>\_shift
- <ltest\_prefix>\_modal\_slow\_capture

### Input/Output Pin Delay Constraints

Input and output delay constraints are declared for the EDT control and channel pins. For example:

```
# channel_input:
set_input_delay $tessent_scan_input_delay \
                -clock tessent_virtual_slow_clock \
                [get_ports {my_edt_channels_in[0]}]

# channel_output:
set_output_delay $tessent_scan_output_delay \
                -clock tessent_virtual_slow_clock \
                [get_ports {my_edt_channels_out[0]}]

# edt_update:
set_input_delay $tessent_scan_input_delay \
                -clock tessent_virtual_slow_clock \
                [get_ports edt_update]
```

## DFT Signals Handling in ltest STA Procs

All EDT-related modal STA procs in the following force these dft signals, when present, to their active value:

- `all_test` (active in all tests)
- `ltest_en` (active in logictest only)

For example:

```
set_case_analysis 1 [get_ports all_test]
set_case_analysis 1 [get_ports ltest_en]
```

The same procs leave all other logictest-related dft signals toggling, and add a `false_path` constraint if they come from a primary port. For example:

```
set_false_path -from [get_ports async_set_reset_static_disable]
set_false_path -from [get_ports control_test_point_en]
set_false_path -from [get_ports ext_ltest_en]
set_false_path -from [get_ports ext_mode]
```

If the same signals come instead from an internal IJTAG Test Data Register (TDR), the “`set_clock_groups`” command between `ts_tck_tck` and the scan clocks replaces the individual `set_false_path` commands.

Procedure “`tessent_set_ltest_edt_fast_capture`”, on the other hand, optionally asserts some of these dft\_signals under control of a user-declared variable.

## MemoryBIST Instrument

Tessent MemoryBIST provides the following procs:

- `tessent_set_memory_bist_non_modal` for chip synthesis
  - This proc would typically not be called in your synthesis script, it is called as part of `tessent_set_non_modal`.
- `tessent_set_memory_bist_modal` for STA
  - This proc must be called in your STA script if you want to constrain the MemoryBist mode.

- `tessent_<design_name>_top_set_dft_signals logic_off`

Specify this call if your design also feature logictest-based dft signals that need to be turned off in membist STA mode.

- `tessent_mbist_set_ai_timing_mode`

This procedure assures all functional clocks are defined and properly propagated, but leaves the asynchronous interface free of constraints so they can be formally analyzed by procedure `tessent_mbist_report_controller_ai_timing`. This proc is not present or needed if none of the controllers being constrained utilize an asynchronous interface.

- `tessent_mbist_report_ai_timing [-verbose 1|0] [-tck_period time_in_ns]`

This procedure runs `tessent_mbist_report_controller_ai_timing` once for every MemoryBIST controller in the current design. This proc is not present or needed if none of the controllers being constrained utilize an asynchronous interface.

- `tessent_mbist_report_controller_ai_timing [-verbose 1|0] [-controller_id id] [-tck_period time_in_ns]`

This procedure accurately measures the BAP AI timing paths to the specified MemoryBIST controller. The covered AI signals are: BIST\_SHIFT, BIST\_SI, BIST\_SO, and BIST\_HOLD.

Because of the asynchronous nature of this interface, the signals listed in the preceding cannot be accurately timed with SDC constraints. Although the circuit is designed to always work with a low TCK frequency, this procedure validates that it works at the frequency you specified.

Each AI signal timing margin depends on a combination of the TCK period, the controller's BIST\_CLK period, and the skew related to the TCK branch reaching that AI TCK transition detector. We recommend running this procedure only once with your worst case operating conditions. In most cases, AI signal timing is expected to largely exceed their setup timing requirements. Hold timing is never a problem because of the design of the interface.

This proc is not present or needed if none of the controllers being constrained utilize an asynchronous interface.

They contain many multicycle path constraints to and from controller registers aiming to minimize the impact of the MemoryBIST DFT on your timing closure.

## Multi-Clock Memories

Additional timing exceptions were added for synthesis if you have multi-port memories functionally driven by multiple clocks. A mux is inserted in the memory interface to enable both clock ports of the memory to be driven using a single clock during the Memory BIST mode. The insertion of this clock multiplexer adds different timing modes between functional and test modes that must be described correctly in the SDC file.

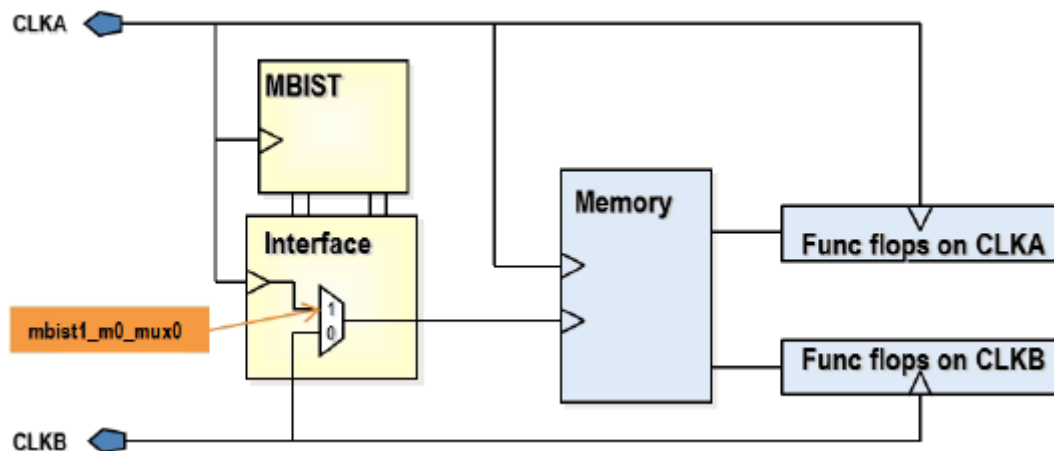
**Note**

If you want to skip the memoryBIST constraints of clock muxing (creating the generated clock and false paths) for multi-clock memories, set the `tessent_apply_mbist_mux_constraints` variable to “0” in the `tessent_set_default_variable` proc. This variable should be set to “0” in two cases:

- For synthesis: When the BIST clock used by the MemoryBIST tests is declared as false path with the memory’s functional clock in the user’s SDC script.
- For STA: When verifying the timing of the scan modes.

In the following, we explain key points of the MemoryBist constraints to handle these modes harmoniously. Suppose the situation where CLKA and CLKB are two of your functional clocks that drive two clock ports of a single memory. The MemoryBIST logic is inserted and runs on CLKA. CLKA is multiplexed onto the CLKB branch of the memory during tests.

**Figure 13-3. Generated Clock Muxed Multi-Clock Memories**



First, we need to create a generated clock on the input of the clock mux. In our example, this clock is called `mbist1_m0_mux0`.

```
create_generated_clock [tessent_get_pins
$tessent_memory_bist_mapping(mbist1_m3)/tessent_persistent_cell_MUX1/b] \
-name mbist1_m0_mux0 \
-source [get_ports CLKA] \
-add -master_clock $tessent_clock_mapping(CLKA) \
-divide_by 1
```

Creating this clock at the input of the clock mux (instead of the output) enables all input clocks to propagate through the mux and times the following interactions precisely:

- Timing paths between the memory BIST logic and the memory
- Timing paths between the functional flops and the memory

With the mbist1\_m0\_mux0 defined, we can now define clock-based exceptions to declare as false any interaction between CLKB and the memory BIST flops, and between mbist1\_m0\_mux0 and the functional flops on CLKB. Those false paths are described using the following SDC commands:

```
set nonMBISTclocks [remove_from_collection [all_clocks] [get_clocks
"$tessent_clock_mapping(CLKA) mbist1_m0_mux0 "]]
set_false_path -from [get_clocks mbist1_m0_mux0] \
-to $nonMBISTclocks
set_false_path -from $nonMBISTclocks \
-to [get_clocks mbist1_m0_mux0]
set_false_path -from [get_clocks $tessent_clock_mapping(CLKB)] \
-to [tessent_get_cells [concat \
$tessent_memory_bist_mapping(mbist1_m3)/SCAN_OBS_FLOPS* \
$tessent_memory_bist_mapping(mbist1_m3)/MBISTPG_STATUS/GO_ID_REG* \
$tessent_memory_bist_mapping(mbist1_m3)/FREEZE_STOP_ERROR*]]
set_false_path -from [tessent_get_cells [concat \
$tessent_memory_bist_mapping(mbist1_m3)/Q1_SCAN_IN* \
$tessent_memory_bist_mapping(mbist1_m3)/Q2_SCAN_IN* \
$tessent_memory_bist_mapping(mbist1_m3)/BIST_INPUT_SELECT*]] \
-to [get_clocks $tessent_clock_mapping(CLKB)]
set_false_path -from [tessent_get_cells [concat \
$tessent_memory_bist_mapping(mbist1)/MBISTPG_PORT_COUNTER/PORT_COUNTER* \
$tessent_memory_bist_mapping(mbist1)/MBISTPG_FSM/STATE* \
$tessent_memory_bist_mapping(mbist1)/MBISTPG_POINTER_CNTRL/
EXECUTE_OP_SELECT_CMD** \
$tessent_memory_bist_mapping(mbist1)/MBISTPG_SIGNAL_GEN/JCNT* \
$tessent_memory_bist_mapping(mbist1)/MBISTPG_SIGNAL_GEN/OPSET_SELECT_REG* \
$tessent_memory_bist_mapping(mbist1)/MBISTPG_DATA_GEN/WDATA_REG* \
$tessent_memory_bist_mapping(mbist1)/MBISTPG_DATA_GEN/X_ADDR_BIT_SEL_REG* \
$tessent_memory_bist_mapping(mbist1)/MBISTPG_DATA_GEN/Y_ADDR_BIT_SEL_REG* \
$tessent_memory_bist_mapping(mbist1)/MBISTPG_ADD_GEN/AX_ADD_REG* \
$tessent_memory_bist_mapping(mbist1)/MBISTPG_ADD_GEN/AY_ADD_REG* \
$tessent_memory_bist_mapping(mbist1)/MBISTPG_OPTION/BIST_EN_RETIME2* \
$tessent_memory_bist_mapping(mbist1)/MEM_SELECT_REG*]] \
-to [get_clocks $tessent_clock_mapping(CLKB)]
```

By making these timing exceptions, we are precisely timing the memory BIST interaction with the memory using CLKA reaching the two-clock ports of the memory and the functional logic to memory interaction using the CLKB reaching the memory clock port. The added logic is precisely timed to simplify timing closure. These constraints make physical design tools aware of the mux on the clock port of the memory and they place the mux in a location where it does not affect timing.

## BoundaryScan Instrument

Tessent BoundaryScan provides the following proc:

- tessent\_set\_jtag\_bscan\_non\_modal
  - This proc would typically not be called directly in your synthesis script, it is called as part of tessent\_set\_non\_modal.

It creates generated clocks from each BoundaryScan interface and relaxes the timing between them.

## Hierarchical STA in Tessent

This section explains how to use Tessent-generated SDC constraint procs to run the hierarchical STA flow with Siemens EDA DFT. This flow runs STA separately on each individually laid-out physical block. If a block instantiates other lower-level physical blocks, partial or full back-annotated netlists or extracted models of those physical blocks are loaded. The procs enable you to cover all Siemens EDA DFT timing paths crossing the lower physical block boundaries, including those related to the IJTAG interface, BISR register interface, bscan interface, and logictest scan/capture paths, while preventing interference and false violations from irrelevant functional paths inside these physical blocks.

When the following conditions occur, the [extract\\_sdc](#) command creates a few callable SDC timing procs to help with the hierarchical STA flow:

- The current DFT-inserted design instantiates lower-level physical blocks.
- Physical blocks have been DFT-inserted with Siemens EDA IP, using [add\\_dft\\_signals](#) commands, along with the DftSpecification flow.
- For “xxx\_ltest\_xxx” procs, lower-level physical blocks are assumed to have the following:
  - One or more EDT controllers.
  - Optional Tessent OCCs.
  - An external logictest mode, controlled with DFT signals ext\_ltest\_en and optionally int\_ltest\_en.

---

### Note

 This hierarchical STA flow is not the same as running flat logictest STA on a full design netlist with multiple physical levels. Running flat STA would require an extra layer of complexity for the needed constraints, especially if you intend to run each level with a different test\_clock frequency. Tessent Shell currently generates no SDC procs to support this type of procedure.

---

## Extracting a Timing Model for Combined Tessent/Functional Mode

The `tessent_set_non_modal` proc in the `extract_sdc` output file provides SDC constraints that time your inserted Tessent DFT logic. You can merge those constraints with your functional SDC constraints to form a unique STA mode (called *non\_modal* in this description). This mode fills all of your non-logictest timing requirements. You can apply these `non_modal` constraints

to your core before extracting the timing model. Doing this means your core timing model includes timing arcs that cover the following core pins:

- IJTAG data and control signals
- Embedded boundary-scan interface signals
- BISR register interface pins
- SSN bus clock and data pins
- All functional timing paths

The Tessent logictest inserted DFT is likely to create clock configuration and timing path incompatibilities with your functional mode. To address this, you must run the `tessent_set_non_modal` proc with the argument “logictest=off” and cover the remainder of your logictest-dependent core interface paths with other procs and methods, described later in this section.

However, if your core logictest DFT logic is controlled entirely by the SSN ScanHost (SSH) node, with no SSH bypass mode, and your core has no dedicated wrapper cells that change the functional data paths in logictest mode, you can time your core boundaries fully using only the extracted model described previously, in all parent-level timing modes including the logictest ones. This is possible because of the following:

- Scan chain shifting inside the core is directly under the control of the internal SSH of the core even when running in external test mode. As a result, all scan data transits first through the core SSN bus pins, which are already timed by this external test mode.
- If your functional data paths across the core are the same as when they are tested in the parent TDF mode of the core, there are no further logictest-only cross-core timing paths that are not covered.

If your SSH supports the SSH bypass mode (SSH is turned off and all scan chains are accessed through other standard EDT channel pins on your core), then you must extract specialized timing models dedicated to your shift and slow\_capture modes. This process is described in the following sections.

## Extracting a Timing Model for Logictest

The `extract_sdc` command provides the following hierarchical STA procs:

- **tessent\_set\_modal\_lower\_pbs** — Enables all Siemens EDA DFT non-logictest timing paths through physical block pins and disables all paths through the rest of the same physical block pins.
- **tessent\_set\_ltest\_pb\_external\_mode** — Enables the external logictest mode on an isolated physical block, forcing its dedicated wrapper cells to select the boundary logic.



Call this proc before extracting your current physical block design timing model, for use at the next level up.

- **<ltest\_prefix>\_lower\_pbs\_external\_mode** — Configures the instantiated lower physical blocks of the parent design in their external mode, enabling running parent logictest modes with loaded lower physical block full or partial netlists.

The `extract_sdc` command generates the `<xxx>_mentor_modal_lower_pbs` proc even when the Logictest IP comes from non-Siemens EDA sources. The command only generates the other two if the IP comes from Tessent TestKompress.

## Hierarchical STA Procs Descriptions

The following describe the hierarchical STA procs in detail.

### **tessent\_set\_modal\_lower\_pbs**

The tool creates this proc only for parent designs instantiating lower-level physical blocks.

The proc provides STA coverage of the Siemens EDA non-logictest DFT timing path going across the physical block pins while blocking all other paths. Call it along with either the non-modal constraints of the current design or the modal DFT STA constraints (membist, BISR, bscan, IJTAG, with disabled logictest). You need to load some timing models for your lower physical block instances.

The proc does the following:

- Disables timing through your physical block interface pins, except those active used Siemens EDA non-logictest DFT, such as the IJTAG pins, the BISR register pins, or the embedded boundary scan control pins. All Siemens EDA logictest IP related pins are also disabled.
- Relaxes some paths in physical block's IJTAG network.
- Blocks TCK from propagating to physical block logic through the eventual Siemens EDA OCC muxes.

Assuming that your current hierarchical STA flow methodology may require loading a mix of black boxes from lower physical blocks, extracted timing models, or backannotated netlists, the proc always checks whether a physical block's internal pin is actually loaded in memory before attempting to apply a constraint to it. As a result, the call to the proc never errors out because of unloaded lower physical block logic.

Disabling timing on all your physical block input's clock pins purposely kills their non-DFT internal functional timing paths. Likewise, physical block scan logic does not require any additional SDC constraints, because of the following:

- The global `scan_enable` control signal is assumed already off in their parent design, leaving all scan flops in functional mode.

- All physical block relevant logic (such as controllers, functional scan flops, LogicBist) receives no clock.

The proc is called along with current level <xxx>\_non\_modal or Siemens EDA DFT modal constraints procs to complete timing coverage of both non-modal synthesis and pre/post-layout modal STA steps.

The following is a typical example:

```
<load your current design files>
<load your lower physical blocks timing models or annotated netlist>
<link your design>
source ${tsdb}/dft_inserted_designs/.../${design}.sdc
tessent_set_default_variables
<override global variable values if needed>
```

The following example includes non-modal constraints (\*):

```
<define your functional clocks and your timing exceptions>
tessent_set_non_modal <arg>
tessent_set_modal_lower_pbs
update_timing
```

---

#### Note



The <arg> value is required only if your design contains Siemens EDA logictest IP.

---

The following example includes Tessent MemoryBIST modal constraints (\*):

```
tessent_create_functional_clocks
tessent_set_ijtag_non_modal
tessent_set_jtag_bscan_non_modal
tessent_set_memory_bisr_non_modal
tessent_set_memory_bist_modal
tessent_kill_functional_paths
tessent_set_modal_lower_pbs
update_timing
```

(\*) IJTAG constraints are always present in any design. The presence of all other procs are design-dependent.

#### tessent\_set\_ltest\_pb\_external\_mode

The tool creates this proc only for isolated physical blocks with the “ext\_ltest\_en” DFT signal present. It forces ext\_ltest\_en active and int\_ltest\_en (if present) inactive. Use this call after a previous call to one of the xxx\_ltest\_modal\_xxx procs, to select the external mode version of your shift, bypass, or capture modes. The proc merely forces the lower physical block’s ext\_ltest\_en and int\_ltest\_en DFT signals to their external mode requirements. You should always call this proc when extracting your physical block’s timing model for later use in your parent ltest STA modes, because it prevents ambiguous timing paths in your extracted model timing arcs.

Current-level <xxx>\_ltest\_modal\* STA procs aim to cover both internal and external mode test paths in the same STA run, despite that letting ext\_ltest\_en toggling may introduce a few false paths that could affect your design timing closure. For most cases, it is a risk worth taking, because merging multiple STA modes together is highly desirable for minimizing your total number of STA runs.

The following shows an example of this proc:

```
<load your current design files>
<link your design>
source ${tsdb}/dft_inserted_designs/.../${design}.sdc
tessent_set_default_variables
<override global variable values if needed>

# Extract one timing model per ltest mode
# Under some conditions, you can merge edt_shift and bypass_shift into
# "shift".
foreach mode {edt_shift bypass_shift edt_slow_capture} {
  reset_design
  tessent_set_ltest_modal_${mode}
  tessent_set_ltest_pb_external_mode
  set_propagated_clock [all_clocks]
  update_timing
  # SYNOPSIS PrimeTime-specific command
  extract_model -output $design_name}_ext_${mode}_etm -format lib -library
}
```

#### <ltest\_prefix>\_lower\_pbs\_external\_mode

The extract\_sdc command writes this proc only for designs instantiating lower-level physical blocks featuring the ext\_ltest\_en DFT signal. The objective of this proc is to set all lower physical blocks into external mode when running one of the logictest STA modes at the parent level, which covers logictest mode timing paths through the boundary pins of these physical blocks. Path coverage includes all logic between the physical block pins and their wrapper cells, in addition to the wrapper chains shift mode paths.

Assuming that your current hierarchical STA flow methodology may require loading a mix of black boxes from lower physical blocks, extracted timing models, or back-annotated netlists, the proc always checks whether a physical block's internal pin is actually loaded in memory before attempting to apply a constraint on it. As a result, the call to the proc never errors out because of unloaded lower physical block logic.

Such constraints include:

- Preventing logictest slow clock reconvergent timing paths inside the physical block OCC logic.
- Asserting the ltest\_en dft\_signal.
- Asserting and deasserting the ext\_ltest\_en and int\_ltest\_en dft\_signals.
- Deasserting the lbist\_en dft\_signal, if present.

- Conditionally asserting or deasserting the following dft\_signals:
  - edt\_mode, int\_edt\_mode, ext\_edt\_mode, edt\_bypass
  - memory\_bypass\_en
  - async\_set\_reset\_static\_disable
- Preventing false timing checks within the physical block OCC clock multiplexing logic.

The following shows an example of this proc:

```
<load your current design files>
<load your lower physical blocks timing models or netlist>
<link your design>
source ${tsdb}/dft_inserted_designs/.../${design}.sdc
tessent_set_default_variables
<override some global tessent Tcl variables value if needed>
foreach mode {shift edt_shift bypass_shift edt_slow_capture} {
    reset_design
    tessent_set_ltest_modal_${mode}
    tessent_set_ltest_lower_pbs_external_mode
    set_propagated_clock [all_clocks]
    update_timing
    <report_timing and other checks>
}
```

## Mapping Procs

Because you can use the Tessent SDC constraints both pre- and post-synthesis, the tool must perform some mapping to find all pins and cells with the pre-synthesis instance path.

The mapping is accomplished with the following procs:

- tessent\_get\_ports
- tessent\_get\_pins
- tessent\_get\_cells
- tessent\_get\_flops
- tessent\_map\_to\_verilog
- tessent\_remap\_vhdl\_path\_list

The procs tessent\_get\_ports, tessent\_get\_pins, and tessent\_get\_cells are wrapper procs of the original SDC commands get\_ports, get\_pins, and get\_cells, and they are used throughout the Tessent SDC. These wrapper procs use the tessent\_map\_to\_verilog and tessent\_remap\_vhdl\_path\_list procs as required. Use these mapping procs (as opposed to get\_ports, get\_pins, and get\_cells) in your functional SDC if you have a design with unrolled VHDL generate loops, and you have problems applying your functional SDC to the RTL. See [“Dealing with Unrolled VHDL Generate for Loops”](#) on page 986 for more information.

The `tessent_get_flops` proc runs the `tessent_get_cells` proc and then filters the result collection for sequential elements. The proc uses a filtering method appropriate for the current tool.

The `tessent_map_to_verilog` proc adds wildcarding to pre-synthesis instance paths to match post-synthesis instance paths for layout and STA. For example, the instance path `core_inst/my_reg[0]` would return the pattern `core_inst/my_reg?0?`. This is to be able to match both the RTL instance path and the post-synthesis path `core_inst/my_reg_0` even if “change\_names -rules Verilog” was used. It also maps the slash (/) coming from Tessent Shell instance paths to any user-defined “tessent\_hierarchy\_separator”. This proc is called by `tessent_get_cells` and `tessent_get_pins`. You should not need to call this proc directly.

The `tessent_remap_vhdl_path_list` proc is used when the instance path cannot be found with the simple mapping. It should only be needed when Tessent Shell has unrolled some VHDL generate loops to do uniquified insertion in the instances within them. See “[Dealing with Unrolled VHDL Generate for Loops](#)” on page 986 for more information. The proc supports finding cells/instances that have a trimmed last character, for example a closing brace or a question mark. This proc is called by `tessent_get_cells` and `tessent_get_pins` if needed. You should not need to call this proc directly. You can also use the proc in the unlikely event that your synthesis tool changed the names of cells in an unexpected way such as trimming any trailing closing brace, for example “`my_reg[0]`” changed to “`my_reg_0`” instead of the expected “`my_reg_0_`”

If you require additional mapping capabilities, the `tessent_map_to_verilog` proc provides a method for custom mapping of regular expressions/substitutions that run prior to the tool’s mapping process. To use this, you must add your own mapping by defining the global array variable.

For example:

```
global tessent_custom_mapping_regsub
array set tessent_custom_mapping_regsub {
  {\} (/| |$) } {\1}
}
```

This example maps all RTL instance paths from the SDC file to remove any closing bracket preceding a hierarchy separator (/) or at the end of the path (space for end of list element, \$ for end of list). You would need this mapping expression during your STA run if the synthesis tool was trimming the closing brace of registers after a `change_name`. For example, `my_reg[0]` -> `my_reg_0` instead of `my_reg_0_` as expected.

## Synthesis Helper Procs

To be able to apply Tessent Shell SDC constraints on your post-synthesis netlist, some steps are required to preserve the boundary of certain instruments as well as to preserve certain “persistent” cells, which must not be optimized away.

Three procs are provided to return the design instances that need to be boundary preserved, optimized, or preserved:

- `tessent_get_preserve_instances preservation_intent`
- `tessent_get_optimize_instances`
- `tessent_get_size_only_instances`

`tessent_get_preserve_instances preservation_intent` returns a collection of instances whose boundaries must be preserved. It must be provided an argument to establish the future use of the netlist and determine which instances need to be preserved. The “select” value you need to choose depends on whether you intend to use your post-synthesis netlist with our tools. Here are the valid values for the “select” argument, in increasing order of inclusiveness:

- [add\\_core\\_instances](#)

This usage selection returns instances that need to be preserved for applying SDC constraints for STA to your post-synthesis design, and instances required for the TCD automation flow with ATPG.

- `scan_insertion`

This usage selection returns all instances of the previous selection, plus any instance of modules having existing scan segments described in a [TCD scan file](#) and non-scan instances (ICL attribute `keep_active_during_scan_test=true`). This selection is required if you intend to do scan insertion on your design with Tessent Shell or a third-party tool.

- `icl_extract`

This usage selection returns all instances of the two previous selections, plus any instance of modules providing an ICL definition. This context is needed if you intend to go through the DftSpecification flow again, this time with your gate level netlist. ICL extraction requires that all ICL instances be present and traceable in the design.

If persistent cells added by Tessent Shell are RTL constructs (RTL cells or wrappers from the cell library), they are returned by `tessent_get_preserve_instances`. If they are library leaf cells, they are not returned by `tessent_get_preserve_instances`. Use the `tessent_get_size_only_instances` proc to obtain these (see following).

The `tessent_get_optimize_instances` proc returns a collection of child instances within Tessent instruments whose boundaries can be optimized. This should be used when your synthesis tools propagates boundary optimization attributes downward hierarchically.

The `tessent_get_size_only_instances` proc returns a collection of instances of all persistent cells, although the collection may be empty under certain conditions. These cells are required to be intact to apply SDC constraints for layout and STA. However, the synthesis tool can change the size of the driving stage as needed.

# Example Scripts Using Tessent Tool-Generated SDC

This section provides example scripts for use when using Tessent Shell tool-generated SDC.

|                                                       |             |
|-------------------------------------------------------|-------------|
| <b>Example Design Compiler Synthesis Script .....</b> | <b>1019</b> |
| <b>Example Fusion Compiler Synthesis Script .....</b> | <b>1020</b> |
| <b>Example Genus Synthesis Script .....</b>           | <b>1022</b> |
| <b>Example PrimeTime STA Script .....</b>             | <b>1025</b> |

## Example Design Compiler Synthesis Script

The following is an example synthesis script to use with Design Compiler.

```
# Prerequisite: Functional Synthesis Script
<load your functional design and constraints here, including all clock
definitions and functional timing exceptions>
#source ${design_name}.dc_shell_import_script
#elaborate $design_name
#create_clock [get_ports pCLK25] -name pCLK25 -period 25.0
#create_clock [get_ports pCLK100] -name pCLK100 -period 100.0

# Preparation Step 1: Sourcing SDC File
set design_name top
set tsdb ../tsdb_outdir

source \
${tsdb}/dft_inserted_designs/${design_name}_rtl.dft_inserted_design/ \
${design_name}.sdc

# Preparation Step 2: Setting and Redefining Tessent Tcl Variables
tessent_set_default_variables
set_ijtag_retargeting_options -tck_period 80.0

# Uncomment the following line if you wish to skip the memoryBIST
# constraints of clock muxing for multi-clock memories:
#set tessent_apply_mbist_mux_constraints 0

# Preparation Step 3: Verifying the Declaration of Functional Clocks
# Set clock names in mapping array
array set tessent_clock_mapping {
    CLK25 pCLK25
    CLK_PLL_REF pCLK100
}
# Preparation Step 4: Redefining Other Tessent Tcl Variables
# set tessent_input_delay [expr 0.3 * $tessent_tck_period]

# Synthesis Step 1: Applying the SDC Constraints
set_app_var timing_enable_multiple_clocks_per_reg true
tessent_set_non_modal
```

```

# Synthesis Step 2: Preparing the DFT Logic for Synthesis
set_app_var compile_enable_constant_propagation_with_no_boundary_opt \
    false
set preserve_instances [tessent_get_preserve_instances icl_extract]
set_boundary_optimization $preserve_instances false
set_ungroup $preserve_instances false
set_app_var compile_seqmap_propagate_high_effort false
set_app_var compile_delete_unloaded_sequential_cells false

set_boundary_optimization [tessent_get_optimize_instances] true

set allETRegs [all_registers -edge_triggered]
set TessentETRegs [filter_collection $allETRegs "full_name =~ *tessent*"]
set_register_output_inversion $TessentETRegs false

set_size_only -all_instances [tessent_get_size_only_instances]

# shared bus assembly ungrouping
foreach_in_collection assembly \
    [get_designs -hierarchical *_tessent_mbist*_shared_bus_assembly] {
        current_design $assembly
        set_ungroup \
            [get_cells -filter "ref_name!~*_tessent_mbist*_controller*"] true
    }
current_design $design_name

# Synthesis Step 3: Synthesizing Your Design
link
check_design
compile -boundary_optimization

# Synthesis Step 4: Writing Out Your Final SDC
write_sdc ${design_name}.merged_sdc

# Synthesis Step 5: Writing Out Your Final Netlist
write -format verilog -output ${design_name}.vg ${design_name} -hier

```

## Example Fusion Compiler Synthesis Script

The following is an example synthesis script to use with Fusion Compiler.



```

set tsdb_dir tsdb_outdir
set design_name top
set design_id dft_insertion
set gate_file top.vg

#Prerequisite: Functional Synthesis Script
<load your functional design and constraints here, including all clock
definitions and functional timing exceptions>
#source ${design_name}.fc_shell_import_script

# Preparation Step 1: Sourcing SDC File
source \
${tsdb_dir}/dft_inserted_designs/ \
${design_name}_${design_id}.dft_inserted_design/${design_name}.sdc

# Preparation Step 2: Setting and Redefining Tessent Tcl Variables
tessent_set_default_variables
set_ijtag_retargeting_options -tck_period 80.0

#   Uncomment the following line if you wish to skip the MemoryBIST
#   constraints of clock muxing for multi-clock memories:
#set tessent_apply_mbist_mux_constraints 0

# Preparation Step 3: Verifying the Declaration of Functional Clocks
#   Overwrite clock names in mapping array in case the clocks in your own
#   SDC have different names than the ones given in Tessent Shell.
array set tessent_clock_mapping {
    CLK25 pCLK25
    CLK_PLL_REF pCLK100
}

# Preparation Step 4: Redefining Other Tessent Tcl Variables
#set tessent_input_delay_percentage 0.3

# Synthesis Step 1: Applying the SDC Constraints
tessent_set_non_modal

```

```
# Synthesis Step 2: Preparing the DFT Logic for Synthesis
#   Some Tessent instruments require their boundary to be preserved
#   depending on your next steps. See Tessent Shell User's Manual
#   for more details.
set_preserve_instances [tessent_get_preserve_instances icl_extract]
set_boundary_optimization $preserve_instances none
set_ungroup $preserve_instances false
set_boundary_optimization [tessent_get_optimize_instances] true

# shared bus assembly ungrouping
foreach_in_collection assembly \
  [get_designs -hierarchical *_tessent_mbist_*_shared_bus_assembly] {
    current_design $assembly
    set_ungroup \
      [get_cells -filter "ref_name!~*_tessent_mbist_*_controller*"] true
  }

#   Preserve Tessent logic signals - do not invert them.
#   See Tessent Shell User's Manual for more details.
set_allETRegs [all_registers -edge_triggered]
set_TessentETRegs [filter_collection $allETRegs "full_name =~ *tessent*"]
set_register_output_inversion $TessentETRegs false

set_size_only -all_instances [tessent_get_size_only_instances]
#   Exclude DFT instruments from automatic clock gating
set_clock_gating_objects -exclude [get_modules *_tessent_*]

set_app_options -list {compile.seqmap.scan {false}}
set_app_options -list {compile.seqmap.remove_unloaded_registers {false}}
set_app_options -list {compile.seqmap.enable_register_merging {false}}
set_app_options -list \
  {compile.flow.enable_high_effort_constant_propagation {false}}
set_app_options -list \
  {compile.flow.constant_and_unloaded_propagation_with_no_boundary_opt \
    {false}}

# Synthesis Step 3: Synthesizing Your Design
#   "initial_map" will give a simple netlist to continue
#   with scan insertion in Tessent Shell
compile_fusion -to initial_map

# Synthesis Step 4: Write your final netlist
write_verilog -hierarchy all $gate_file
```

## Example Genus Synthesis Script

This section provides an example synthesis script to use with Genus.

---

### Tip

 See [“Solutions for Genus Synthesis Issues”](#) on page 1059 if you experience issues synthesizing mixed Verilog/VHDL designs.

---

```
# Proposed global settings
foreach {attrName attrValue} {
    hdl_bidirectional_assign false
    bus_naming_style {%s(%d)}
    uniquify_naming_style {%s_%d}
    hdl_auto_async_set_reset true
    init_blackbox_for_undefined false
    write_vlog_top_module_first true
    remove_assigns false
    minimize_uniquify true
    continue_on_error false
    log_command_error true
    report_tcl_command_error true
} {
    dict set global_attributes_dict $attrName $attrValue
    set_db $attrName $attrValue
}

# Prerequisite: Functional Synthesis Script
<load your functional design and constraints here, including all
clock definitions and functional timing exceptions>

# Preparation Step 1: Sourcing SDC File
set design_name top
set tsdb ../tsdb_outdir
source ${tsdb}/dft_inserted_designs/ \
${design_name}_rtl.dft_inserted_design/${design_name}.sdc

# Preparation Step 2: Setting and Redefining Tessent Tcl Variables
tessent_set_default_variables
set_ijtag_retargeting_options -tck_period 80.0
# Uncomment the following line to skip the memoryBIST constraints
# of clock muxing for multi-clock memories:
#set tessent_apply_mbist_mux_constraints 0

# Preparation Step 3: Verifying the Declaration of Functional Clocks
# Set clock names in mapping array.
# The array keys are the names Tessent Shell knew of the clocks.
# The array values are the names of those clocks in your current SDC.
array set tessent_clock_mapping {
    CLK25 pCLK25
    CLK_PLL_REF pCLK100
}

# Preparation Step 4: Redefining Other Tessent Tcl Variables
# set tessent_input_delay_factor 0.3
# set tessent_hierarchy_separator "_"

# Synthesis Step 1: Applying the SDC Constraints
tessent_set_non_modal
```

```
# Synthesis Step 2: Preparing the DFT Logic for Synthesis
proc do_set_boundary_optimization { instances_collection state } {
    if { [sizeof_collection $instances_collection] == 0 } { return }
    puts "Modules of the instances"
    set modules_collection [get_db $instances_collection .module]
    puts "    [join [get_db $instances_collection .module] "\n "]"
    # Preserve all instances, even those on which boundary optimization
    # is requested.
    set_db $instances_collection .ungroup_ok false
    # Boundary optimization is avoided by preserving the footprint (pins).
    if { !$state } {
        # Boundary optimization is avoided for all modules of the instances.
        # - Preserve the footprint of all the pins
        # - Do not propagate constant inputs while optimizing internal logic
        # - Do not consider any input of opposite polarity
        set_db $modules_collection .boundary_opto strict_no
    }
}

proc do_set_size_only { instances_collection } {

    if { [sizeof_collection $instances_collection] == 0 } { return }
    foreach {preserveValue isSequential} [list map_size_ok \
        true size_ok false] {
        set instancesFiltered [filter_collection \
            $instances_collection "is_sequential==$isSequential"]
        if { [llength $instancesFiltered] == 0 } { continue }
        puts "\nSetting preserve attribute '$preserveValue' to \
            '$isSequential' on:"
        set_db $instancesFiltered .preserve $preserveValue
    }
}

set preserveInstances [tessent_get_preserve_instances icl_extract]
do_set_boundary_optimization $preserveInstances false
set optimizeInstances [tessent_get_optimize_instances]
do_set_boundary_optimization $optimizeInstances true
set set_size_only_collection [tessent_get_size_only_instances]
do_set_size_only $set_size_only_collection

# Ungroup shared bus assemblies if any are present
set sb_mods [vfind . -subdesign *_tessent_mbist_*_shared_bus_assembly]
set sb_insts [get_db $sb_mods .instance]
foreach sb_inst $sb_insts {
    # ungroup everything in the assembly but the controller
    ungroup [get_db $sb_inst -depth {1 1} -invert -if \
        { .module =~ *_tessent_mbist_*_controller}]
}

#Avoid inverting the logic signals of any Tessent IP
set tessent_hierarchical_instances \
    [get_db [get_db modules *_tessent*] .hinsts]
set cells_in_tessent_hierarchical_instances \
    [get_db $tessent_hierarchical_instances .insts]
foreach cell $cells_in_tessent_hierarchical_instances {
    if {[get_db $cell .is_sequential]} {
        set_db $cell .dont_use_qbar_pin true
    }
}
}
```

```
# Exclude DFT instruments from automatic clock gating
set_db $tessent_hierarchical_instances .lp_clock_gating_exclude true
set_db $cells_in_tessent_hierarchical_instances \
    .lp_clock_gating_exclude true

# Synthesis Step 3: Synthesizing Your Design
syn_generic
syn_map
syn_opt
# Synthesis Step 4: Writing Out Your Final SDC
write_sdc $design_name > ${design_name}.merged_sdc
# Synthesis Step 5: Writing Out Your Final Netlist
write_hdl $design_name > $design_name.vg
```

## Example PrimeTime STA Script

The following section provides an example STA script to use with PrimeTime. This example shows an option to merge DFT mode with functional mode. Merging requires that the functional clocks have the same frequency and clock paths as those used during MemoryBIST test.

```
# RunAndReport
#
proc RunAndReport { mode } {
    update_timing
    redirect violations_${mode}.report {report_constraint -all_violators}
}
set tsdb ./tsdb_outdir
set design_name top
set design_id rtl
set netlist ${design_name}.vg
# Allow asserts on flop reset pins to propagate to their Q output.
set case_analysis_sequential_propagation always

# Disable timing through the enable side of clock gating cells,
# so as to prevent clock reconverging paths.
set timing_clock_gating_propagate_enable false

# Source Tessent Shell generated sdc file and set default variables
source \
    ${tsdb}/dft_inserted_designs/${design_name}_${design_id}.dft_inserted_design \
    /${design_name}.sdc
tessent_set_default_variables
```

## Timing Constraints (SDC)

### Example PrimeTime STA Script

---

```
# Optionally override the above Tcl variables values using "set" commands.  For
# example:
# set tessent_slow_clock_period 20
# set_ijtag_retargeting_options -tck_period 50
# array set tessent_clock_mapping {
#     ts_tck_tck my_tck
#     CLK3 CLK_F300
# }

# Change your hierarchy separator variable if not appropriate
set tessent_hierarchy_separator "/"

# Override data in above "tessent_set_default_variables" by creating your
# own timing data file and placing it at the same level as this script.
if {[file exists user_timing_data.tcl]} {
    source user_timing_data.tcl
}

# You can provide your own path remapping procedures file
# (tessent_get_xxx). You only need to copy and edit the procedures you
# want to change and put them in file ./user_remapping_procs.tcl.
if {[file exists user_remapping_procs.tcl]} {
    source user_remapping_procs.tcl
}

# If you have fake cells or cells that are not part of your library
# you can provide this file to load them.
if {[file exists load_lib_cells.tcl]} {
    source load_lib_cells.tcl
}

# Create the script "load_design.pt" only if you need to load your design
# with a different command than the "read_verilog" below.
if {[file exists load_design.pt]} {
    source load_design.pt
} else {
    read_verilog $netlist
}
current_design $design_name
link_design
```

```
#####
#
# Tessent DFT
#
# Run the DFT mode with or without merging it with the functional mode.
#
# The first option below is the merged functional and DFT mode. You first
# source all functional SDC constraints for your design, and then merge them
# with the Tessent DFT constraints, the same way you presumably did when
# synthesizing the design before layout.
#
# To isolate all Tessent DFT timing paths from your functional timing paths
# for debugging purposes or if your DFT clock network or frequencies differ
# from your functional mode, refer to the second option in the example.
#
# Both options cover all Tessent-inserted DFT IP, such as IJTAG network,
# boundary scan, MemoryBIST, and BISR. Neither option covers scan or EDT mode,
# which require their own separate STA mode.
#####

# First option: Merging your functional mode SDC with Tessent DFT SDC
echo "\n\nAnalysis in merged functional/DFT mode.***** \n"
reset_design
<apply your functional mode constraints here>
# Merge with DFT mode
tessent_set_non_modal off
RunAndReport TESSENT_MERGED_DFT_MODE

# Second option: Run with Tessent DFT SDC alone
echo "\n\nAnalysis in Tessent DFT mode.***** \n"
reset_design
# Apply constraints for STA
tessent_create_functional_clocks ; # Only clocks needed for membist operation
tessent_set_ijtag_non_modal ; # For your IJTAG network
tessent_set_bscan_non_modal ; # Only if Tessent boundary scan present
tessent_set_bisr_non_modal ; # Only if BISR logic present
tessent_set_memory_bist_modal ; # Modal constraints for MemoryBist
tessent_set_ltest_non_modal off ; # Disables logictest DFT logic
tessent_kill_functional_paths ; # Kill any path unrelated to Tessent
DFTRunAndReport TESSENT_DFT_MODE
```

The following block is only needed for v2020.3 and earlier designs that incorporate the MemoryBIST Asynchronous Interface. Designs from v2020.4 and later incorporate the Enhanced MemoryBIST Access hardware.

```
#####
# For v2020.3 and earlier designs only, formally check your BAP Asynchronous
# interface timing paths
#####
if {[info procs tessent_mbist_report_ai_timing] ne ""} {
  echo "\n\nChecking your BAP asynchronous interface paths timing***** \n" ;
  echo "See the results in output report file AITiming.report"
  reset_design
  tessent_set_default_variables
  tessent_mbist_set_ai_timing_mode
  update_timing
  # set "-verbose" to 1 for a more detailed timing report
  # change the "-tck_period" value for analysis of different shift speeds
  redirect AITiming.report { tessent_mbist_report_ai_timing /
    -tck_period $tessent_tck_period -verbose 1 }
}
```

The following steps assume you have EDT IP in your design:

```
#####
#                               Combined Edt Shift Modes
#####
reset_design
LoadBackAnnotationData
tessent_set_ltest_modal_shift
set_propagated_clock [all_clocks]
RunAndReport SHIFT

#####
#                               EDT Slow CaptureMode
#####
reset_design
LoadBackAnnotationData
set tessent_time_hold_in_slow_capture 1
set tessent_scan_en_setup_extra_cycles 1
set tessent_scan_en_hold_extra_cycles 1
set tessent_edt_update_setup_extra_cycles 1
set tessent_edt_update_hold_extra_cycles 1
tessent_set_ltest_modal_edt_slow_capture
set_propagated_clock [all_clocks]
RunAndReport SLOW_CAPTURE
```



```
#####
#                               EDT Fast Capture Mode
#####
# Important: The following variable setting is important in order to allow
# at-speed coverage of your Tessent OCC's internal shift register and clock gaters
# paths with scan_enable=0:
set case_analysis_sequential_propagation never
reset_design
LoadBackAnnotationData
<load your functional constraints here, including all clock definitions and
functional timing exceptions>
set memory_bypass_en_value 1
set x_bounding_en_value 1
set observe_test_point_en_value 0
tessent_set_ltest_modal_edt_fast_capture
set_propagated_clock [all_clocks]
RunAndReport FAST_CAPTURE
```

The LogicBIST modes described in the following are required when using the hybrid TK/LBIST flow.

You can run LogicBIST tests using one of three test clock sources chosen at pattern generation time: `shift_clock_src`, `test_clock`, or `TCK`. Because `TCK` is a slow clock, STA verification is not performed explicitly when using this clock source for LogicBIST; the tool verifies all paths with one of the other two clocks, except for the clock network differences prior to the LogicBIST controller.

The default clock is `shift_clock_src`, which is typically connected to an internally generated, free-running clock because this is the most suitable source for running LogicBIST in-system test.

If you generate LogicBIST patterns to run with `test_clock` as the LogicBIST clock source, specify “`test_clock`” as an argument in the following procs:

```
tessent_set_ltest_modal_lbist_shift test_clock
tessent_set_ltest_modal_lbist_capture test_clock
```

The tool supports the values `ltest_clock` and `edt_clock` as aliases for “`test_clock`”.

If you generate two sets of LogicBIST patterns to run LogicBIST tests, run the LogicBIST shift mode proc described in the following twice: first with the `shift_clock_src` argument and second with `test_clock` argument.

For LogicBIST shift mode and LogicBIST capture mode using the `shift_clock_src` argument, create the `shift_clock_src` clock in the timing tool and indicate its name using the Tessent SDC Tcl variable `tessent_lbist_shift_clock_src`, as mentioned in [“Preparation Step 2: Setting and Redefining Tessent Tcl Variables”](#) on page 972.

```
#####  
#                               LogicBIST Shift Mode  
#####  
reset_design  
LoadBackAnnotationData  
tessent_set_ltest_modal_lbist_shift  
set_propagated_clock [all_clocks]  
RunAndReport LBIST_SHIFT
```

Unlike EDT, which uses separate fast and slow capture modes, LogicBIST uses a combined capture mode. The NCPs and fault type used during fault simulation determine whether LogicBIST test targets stuck-at or transition faults. These NCPs can use single or multiple clock pulses sourced from either the LogicBIST clock or functional clock, as determined by the NcpIndexDecoder and OCC settings. Similar to the shift mode, use either one or both of shift\_clock\_src and test\_clock arguments for analysis. The value for observe\_test\_point\_en should reflect the setting used during fault simulation. Control test points are typically enabled for both stuck-at and transition tests.

```
#####  
#                               LogicBIST Capture Mode  
#####  
reset_design  
LoadBackAnnotationData  
tessent_set_ltest_modal_lbist_capture  
<when LogicBIST test targets transition faults, load your functional constraints  
here, including all clock definitions and functional timing exceptions>  
set memory_bypass_en_value 1  
set x_bounding_en_value 1  
# When observe_test_points are disabled for transition test:  
set observe_test_point_en_value 0 ;  
set_propagated_clock [all_clocks]  
RunAndReport LBIST_CAPTURE
```

The following proc analyzes timing for the IJTAG network paths used to initialize the BIST controller and read the MISR values after test is complete. This mode uses TCK.

```
#####  
#                               LogicBIST Setup (IJTAG Network Paths)  
#####  
reset_design  
LoadBackAnnotationData  
set_propagated_clock [all_clocks]  
tessent_set_ltest_modal_lbist_setup  
RunAndReport LBIST_SETUP
```

The following proc analyzes timing for the single chain-based diagnosis mode. This mode uses TCK for reading all the scan cells through the IJTAG network.

```
#####  
#                               LogicBIST Single Chain Diagnosis  
#####  
reset_design  
LoadBackAnnotationData  
set_propagated_clock [all_clocks]  
tessent_set_ltest_modal_lbist_single_chain  
RunAndReport LBIST_SINGLE_CHAIN
```

When you implement the optional controller chain mode (CCM), the following proc analyzes timing for CCM. In this mode, the tool removes timing exceptions between the LogicBIST controller and the hybrid TK/LBIST blocks (as seen in the shift and capture modes) to reflect CCM pattern generation setup.

```
#####  
#                               LogicBIST Controller Chain Mode  
#####  
reset_design  
LoadBackAnnotationData  
set_propagated_clock [all_clocks]  
tessent_set_ltest_modal_lbist_controller_chain  
RunAndReport LBIST_CONTROLLER_CHAIN
```



# Appendix A

## The Tessent Tcl Interface

---

Tessent Shell provides a Tcl-based command interface.

|                                                                        |             |
|------------------------------------------------------------------------|-------------|
| <b>General Tcl Guidelines in Tessent Shell .....</b>                   | <b>1033</b> |
| <b>Encoding Tcl Scripts in Tessent Shell .....</b>                     | <b>1035</b> |
| <b>Guidelines for Modifying Existing Dofiles for Use With Tcl.....</b> | <b>1036</b> |
| <b>Special Tcl Characters.....</b>                                     | <b>1037</b> |
| <b>Custom Tcl Packages in Tessent Shell.....</b>                       | <b>1039</b> |
| <b>Tcl Resources .....</b>                                             | <b>1040</b> |

## General Tcl Guidelines in Tessent Shell

If Tcl procedures are available in separate files, you can source these files from within the tool or dofiles. You can also place Tcl procedures in a *.tool\_startup* file so that they are available for use. Tcl file input/output is also supported.

You should be aware of the following guidelines and behaviors when using Tcl in Tessent Shell:

- You can use Tcl variables interchangeably with legacy Tessent tool variables and environment variables.
- You should use Tcl syntax for setting and referencing variables, including using `$env(ENVARNAME)` for accessing environment variables.

For example, if the value of environment variable “ham” equals “mode1” (`$ham = mode1`), you can compare this value to a Tcl variable value using the following syntax:

```
set eggs mode2
if {$env(ham) == $eggs} {puts Match} else {puts "No match"}
```

---

### Note



Use Tcl namespaces to avoid creating procedures that conflict with existing tool or Tcl commands.

---

- When processing comments within a dofile, the tool does not write comments preceded with “//” characters to the transcript. However, the tool does write comments that are preceded with a pound sign (#) (the Tcl comment delimiter) to the transcript.
- A variable can contain a command string or be empty; attempting to run by referencing `$variable` results in an error if `$variable` is empty.

- When a tool command error occurs, the command interpreter prints the error message and does not return anything.
- You can use the [catch\\_output](#) command to issue a specified tool command line and prevent command errors from aborting an enclosing dofile or Tcl proc. The `catch_output` command can optionally capture the output of a command or the returned result.
- When a command error occurs nested inside a Tcl construct or Tcl proc, you can obtain additional information about the error by issuing the following command:

**> set errorInfo**

This command prints the value of the `$errorInfo` Tcl variable, which may contain a traceback of the nested Tcl calls so that you can determine the root cause of the error.

## Difference Between the Dofile Command and the Tcl Source Command

You normally use the tool's [dofile](#) command to run a file of tool commands. The Tcl "source" command also runs a file of commands, but you should only use it to load Tcl procs, set Tcl variables, or do other strictly Tcl commands.

You should use the `dofile` command to run a command file containing tool commands for the following reasons:

- The `dofile` command is affected by the "[set\\_tcl\\_shell\\_options -abort\\_dofile\\_on\\_error](#)" command, which provides you with the ability to specify whether the tool aborts when an error condition is detected. The Tcl "source" command is not affected by the `-abort_dofile_on_error` option.
- The `dofile` command transcripts the commands to the shell and logfile. The Tcl "source" command always stops running if any command in the file encounters an error.

## Example 1

In this example, the `add_scan_chains` command defines every scan chain in the design. This command is sometimes used hundreds of times and makes the dofile very long. Using Tcl, you can shorten the dofile substantially by using a loop construct as shown here:

```
for {set xx 1} {$xx < 257} {incr xx} {  
    add_scan_chains int chain$xx group1 /top/edt_si$xx /top/edt_so$xx  
}
```

For the values of `xx` from 1 up to 256, the tool runs a separate `add_scan_chains` command for each value. The transcript and logfile contain all 256 `add_scan_chains` commands (preceded with "`// subcommand:` ").

## Example 2

The following example controls the flow of creating test patterns and uses a variable to run different commands based on the variable's value:

```
if {$mode == stuck} {  
    set_fault_type stuck  
    . . .  
} elseif {$mode == transition} {  
    set_fault_type transition  
    set_pattern_type -sequential 2  
    . . .  
}
```

## Example 3

Module hierarchies may become ungrouped through either synthesis or layout and ATPG may operate on a flattened view of the design. Automated DRCs find and trace the EDT IP and subsequently find and trace the flattened scan chains automatically. In the following example, we are capturing the output of the [report\\_scan\\_chains](#) command and placing it into the variable “rsc” by using the [catch\\_output](#) command. The foreach loop iterates on the report\_scan\_chains output, capturing the chain name, group name and input and output connections in the variables “chain”, “group”, “input”, and “output,” respectively. This information is written into the file *add\_chains.txt*.

```
set out [open "add_chains.txt" w]  
catch_output {report_scan_chains} -output rsc  
set rsc1 [split $rsc "\n"]  
  
foreach a $rsc1 {  
    puts "NEWLINE =\t$a"  
    regexp {.*chain = (.*)\s+group = (.*)\s+input = \'(.*)\'\s+output = \  
\'(.*)\'\s+.*} $a match chain group input output  
    puts $out "add_scan_chains -internal $chain $group \{$input\} \{$output\  
}"  
}  
  
close $out
```

# Encoding Tcl Scripts in Tessent Shell

Tessent Shell enables the distribution of custom Tcl scripts without exposing the source code.

Tessent Shell reads files encoded by the [encode\\_tcl\\_scripts](#) command with Tcl's existing “source” command and decodes them automatically. Tessent Shell blocks introspection of the procedures in the file by Tcl's “info body” command, but the procedure names are visible with the “info procs” command.

For details on how to use the `encode_tcl_scripts` command, refer to the *Tessent Shell Reference Manual*.

## Guidelines for Modifying Existing Dofiles for Use With Tcl

When using an existing dofile with the Tcl interface, you should evaluate the dofile for issues that could cause incorrect evaluation by the Tessent Tcl interpreter.

[Table A-1](#) provides guidance for correcting common dofile issues.

The most common issues you can run across with an existing dofile is accounting for Tcl special characters. For more information, refer to [Table A-2](#) on page 1038, which provides a list of typically-used Tcl special characters.

**Table A-1. Common Dofile Issues and Solutions**

| Dofile Issue                             | Solution                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Stopping dofile runs at a specific point | Use the native Tcl “error” command to stop the run of a dofile at any point. The “error” command requires a message string, which the tool outputs. But to avoid any message you can use an empty string:<br><b>error ""</b>                                                                                                                                                                                                                                                                                         |
| Escaping Tcl special characters          | Use the following techniques to escape Tcl special characters: <ul style="list-style-type: none"><li>• Quotation marks (“”) group tokens but enable \$variable and [command] evaluations.</li><li>• Braces ({} ) also group tokens and disable \$variable and [command] evaluations.</li><li>• Brackets ([]) implement command substitution and are used to nest or embed commands.</li><li>• A backslash (\) escapes the next character. Use this to tame a Tcl special character such as \$, [, {, or ".</li></ul> |
| Using dollar signs in pathnames          | A dollar sign (\$) specifies variable substitution. In some netlists, pathnames (for example, <i>ham/pin\$p7</i> ) can contain the dollar sign. When using pathnames with dollar signs, enclose the pathname with braces ({} ) to prevent the tool from substituting the value as shown in the following example:<br><code>report_gates {ham/pin\$p7}</code>                                                                                                                                                         |



Table A-1. Common Dofile Issues and Solutions (cont.)

| Dofile Issue                    | Solution                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|---------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Escaping quotation marks        | <p>In Tcl, quotation marks (“”) instruct the Tcl interpreter to treat the enclosed words as a single argument. For example:</p> <pre>puts " Hello World "</pre> <p>If embedded quotation marks are required, you must use backslashes ( \ ) to escape the embedded quotation marks. For example:</p> <pre>puts " Hello \"World\" "</pre> <p>Otherwise, the tool issues an error message.</p>                                                     |
| Using brackets                  | <p>Brackets ([ ]) implement command substitution and are used to nest or embed commands. A command and its arguments enclosed in square brackets is evaluated and its result inserted in place in the enclosing command.</p>                                                                                                                                                                                                                     |
| Optional single quotation marks | <p>Optional single quotation marks ( ' ) are no longer valid for Tessent commands. For example, the following produces an error:</p> <pre>set_design_sources '-v MODB.v -v MODC.v'</pre> <p>Correct this by using no quotation marks or braces:</p> <pre>set_design_sources -v MODB.v -v MODC.v</pre> <pre>set_design_sources "-v MODB.v -v MODC.v"</pre> <pre>set_design_sources {-v MODB.v -v MODC.v}</pre>                                    |
| Environment variables           | <p>You should use Tcl syntax for setting and referencing variables, including using <code>\$env(ENVARNAME)</code> for accessing environment variables. For example, if the value of environment variable “ham” equals “mode1” (<code>\$ham = mode1</code>), you can compare this value to a Tcl variable value using the following syntax:</p> <pre>set eggs mode2 if {\$env(ham) == \$eggs}     {puts "Match"} else     {puts "No match"}</pre> |

## Special Tcl Characters

In Tcl scripts, you often see characters used for special purposes. For a complete list of special characters, you should consult a Tcl resource.

[Table A-2](#) lists and describes the more common special characters you can encounter when reading the examples in this manual. See the additional resources described in [“Tcl Resources”](#) on page 1040.

**Table A-2. Common Tcl Characters**

| Character | Description                                                                                                                                                                                                                                                                                                                                                                                                                      |
|-----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ;         | The semicolon terminates the previous command, enabling you to place more than one command on the same line.                                                                                                                                                                                                                                                                                                                     |
| \         | Used at the end of a line, the backslash continues a command on the following line.                                                                                                                                                                                                                                                                                                                                              |
| \\$       | The backslash with other special characters, like a dollar sign, instructs the Tcl interpreter to treat the character literally.                                                                                                                                                                                                                                                                                                 |
| \n        | The backslash with the letter “n” instructs the Tcl interpreter to create a new line.                                                                                                                                                                                                                                                                                                                                            |
| \$        | The dollar sign in front of a variable name instructs the Tcl interpreter to access the value stored in the variable.                                                                                                                                                                                                                                                                                                            |
| [ ]       | Square brackets group a command and its arguments, instructing the Tcl interpreter to treat everything within the brackets as a single syntactical object. You use square brackets to write nested commands. For example:<br><b>set chain_report [report_scan_chains -subchains -verbose]</b>                                                                                                                                    |
| { }       | Braces instruct the Tcl interpreter to treat the enclosed words as a single string. The Tcl interpreter accepts the string as is, without performing any variable substitution or evaluation. For example, to create a string that contains special characters such as \$ or \:<br><b>set my_string {This book costs \$25.98.}</b>                                                                                               |
| " "       | Quotation marks instruct the Tcl interpreter to treat the enclosed words as a single string. However, when the Tcl interpreter encounters variables or commands within string in quotation marks, it evaluates the variables and commands to generate a string. For example, to create a string that displays a final cost calculated by adding two numbers:<br><b>set my_string “This book costs \\${expr \$price + \$tax}”</b> |

**Table A-2. Common Tcl Characters (cont.)**

| Character | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| #         | <p>The pound sign (#) indicates a comment and directs the Tcl compiler to not evaluate the rest of the line. When using the pound sign, you must use it where a command starts, and at the beginning of a command, not within a command. Be aware of the following:</p> <ul style="list-style-type: none"> <li>• Evaluate does not equal parse. Despite the pound sign, the following comment gives an error because Tcl detects an open lexical clause.</li> </ul> <pre># if (some condition) { if { new text condition } { ... }</pre> <ul style="list-style-type: none"> <li>• The apparent beginning of a line is not always the beginning of a command:</li> </ul> <pre># This is a comment</pre> <p>In the following code snippet, the line beginning with “# -type” is not a comment, because the line immediately above it has a line continuation character (\). In fact, the “#” confuses the Tcl interpreter, resulting in an error when it attempts to create the tk_messageBox.</p> <pre>tk_messageBox -message "The diagnosis report was successfully written." \ # -type ok</pre> <p># and this is also a comment. This one spans \ multiple lines, even without the # at the beginning \ of the second and third lines.</p> <p>In general, it is good practice to begin all comment lines with a #. Be aware that the beginning of a command is not always at the beginning of a line. Usually, you begin new commands at the beginning of a line. That is, the first non-space character is the first character of the command name. However, you can combine multiple commands into one line using the semicolon “;” to designate the end of the previous command:</p> <pre>set myname "John Doe" ; set this_string "next command" set yourname "Ted Smith" ; # this is a comment</pre> |

## Custom Tcl Packages in Tessent Shell

Tessent Shell supports several standard techniques for adding your own Tcl packages with the Tcl “package require” command, which you can issue from Tessent Shell.

You can use any of the following ways to specify directory locations of Tcl packages. Any directory that contains a Tcl package must also contain a *pkgIndex.tcl* file within its hierarchy.

The following methods are listed in order of the precedence in effect if there is more than one package with the same name:

- Default location for Tessent Shell Tcl packages. You can place your package underneath a directory named *tessent\_plugin/packages* that is located at the top of your Tessent install tree. When Tessent Shell finds a package in this directory upon invocation, it appends the directory path to *tessent\_plugin/packages* to the *auto\_path* Tcl global variable, if it exists.
- *TESSENT\_PLUGIN\_PATH* environment variable. You can set this variable to a colon-separated list of directory paths that contain Tcl packages in a *packages* subdirectory. When Tessent Shell finds a package, it appends the directory path to *packages* to the *auto\_path* Tcl global variable, if it exists.
- *auto\_path* Tcl global variable. You can issue the following command in Tessent Shell to specify a package location:

```
> lappend auto_path pathToTclPackageDir
```

- *TCLLIBPATH* environment variable. You can set this variable to a space-separated list of directory paths that contain Tcl packages. A *packages* subdirectory is not required under any of the paths.

---

**Note**



Tessent Shell ignores the *TCL\_LIBRARY* environment variable.

---

## Tcl Resources

The following website is a place to start in your search for the reference material that works best for you. It is not an endorsement of any book or website.

<http://www.tcl.tk/>

# Appendix B

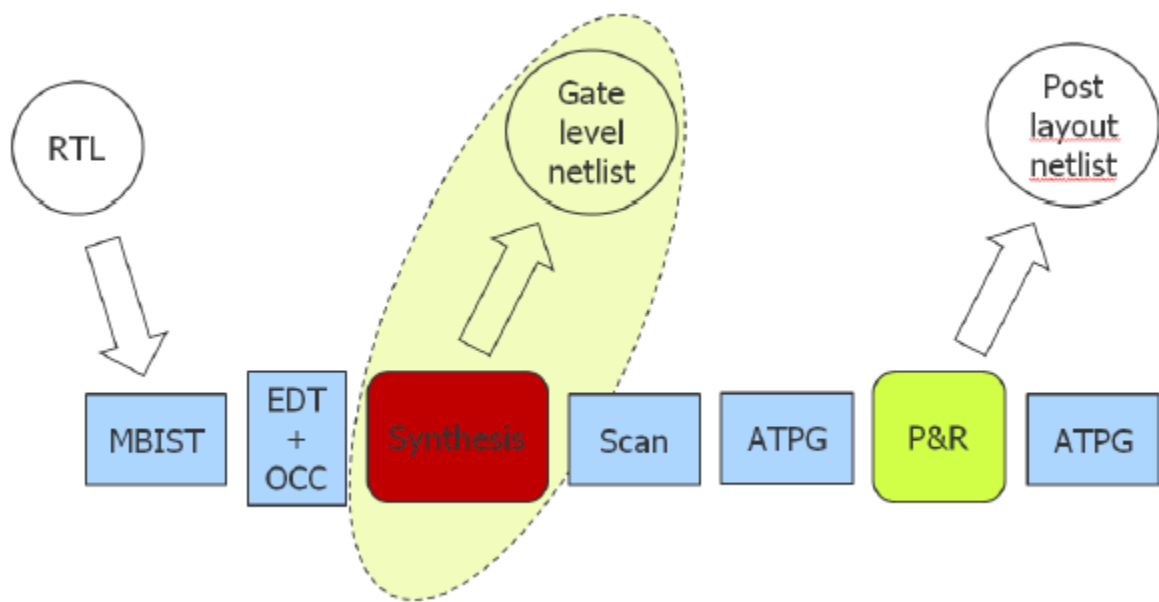
## Synthesis Guidelines for RTL Designs with Tessent Inserted DFT

---

This appendix provides guidelines for you to use when synthesizing your design with Tessent created IP RTL with a third-party synthesis tool, specifically Synopsys DC Ultra™.

You may experience problems with synthesis optimization using DC Ultra caused by propagating constants even if the boundary optimization is false. This synthesis optimization results in some of the DFT logic that is not connected until scan insertion being optimized away, and the ports being kept. This creates a situation where some of the necessary Tessent logic is eliminated during optimization, which breaks the design.

**Figure B-1. Problem Occurrence Creating the Gate Level Netlist**



This problem does not appear to be present with the basic DC compile command; it seems to only be an issue when using the compile\_ultra command in certain tool versions.

|                                                                         |             |
|-------------------------------------------------------------------------|-------------|
| <b>DFT Insertion With Tessent .....</b>                                 | <b>1042</b> |
| <b>Synthesis Guidelines .....</b>                                       | <b>1043</b> |
| <b>SystemVerilog and Port Name Matching .....</b>                       | <b>1045</b> |
| <b>Synthesis Guidelines for Parameterization Wrapper Flows .....</b>    | <b>1046</b> |
| Tcl Procedures for Synthesis in the Parameterization Wrapper Flow ..... | 1047        |

## DFT Insertion With Tessent

When the Tessent tools create the IP logic to add to the design, the logic includes some persistent instances that are used as anchor points for items such as clocks, control signals, constraints, and so on. For automation, the tool uses modules and port matching, and, consequently, requires that some specific modules be preserved after synthesis and place and route.

There are two types of persistent instances for synthesis:

- Cells
  - Requires the use of a `set_size_only` command
- Design Modules
  - Requires `ungroup` to be set to off
  - Requires boundary optimization to be set to off

Boundaries of all instances of modules with Tessent Core Descriptions (TCDs) need to be preserved, which means the following:

- No boundary optimization.
- No ungrouping.
- No new ports.
- No logic optimization.

The logic test module types that require this include EDT, LBIST, OCC, STI SIB, Bscan interface, and any module with `tcd_scan` (pre-existing scan chain). ICL extraction uses modules and ports of ICL modules so they also need to be preserved. You can avoid preserving the IJTAG instance boundaries, which enables the synthesis tool to obtain a better optimization result. You can use the ICL file from RTL in post synthesis and place and route.

The boundary optimization option during synthesis is global in the sense that it applies to every design hierarchy on and below the specified module or instance. Hence, if you want to globally optimize your design, you specify it on the root (top) module and then boundary optimization runs through the entire design doing its work (assuming that there are no “don’t touch” cores, which were already synthesized earlier). The problem is that valid constrained or not (yet) connected ports or nets get optimized away. To prevent this, tell the DC tool not to apply this boundary optimization to selected modules or instances.

DC has two compilation modes:

- Basic compile (using the *compile* command):
  - Problems are avoided because the tool does not ungroup and optimize boundaries.

- Constants are not optimized away.
- Optimization On (using *compile\_ultra* command):
  - The synthesis tool does its best to remove any dead logic (either because of constraints propagation or missing loads or unused).
  - DC propagates constraints even if boundary optimization is set false for design instances.

As this issue is becoming more prevalent, there is an informational message from DC Ultra when using the *compile\_ultra* command that reports the following:

```
Information: Starting from 2013.12 release, constant propagation is
enabled even when boundary optimization is disabled. (OPT-1318)
```

If you receive this message, you may not fully understand the ramifications or how to correctly synthesize the Tessent IP in this environment. If not properly handled, you may see additional information messages from the tool similar to the following:

```
Information: Removing unused design 'corea_rtl_tessent_occ_shift_reg_1'.
(OPT-1055)
Information: The register
'corea_rtl_tessent_occ_clk_inst/occ_control/scan_out_reg' is a constant
and will be removed. (OPT-1206)
Information: Ungrouping hierarchy tessent_persistent_cell_edt_clock
before Pass 1 (OPT-776)
```

## Synthesis Guidelines

When synthesizing a design for Tessent DFT, certain considerations must be taken into account to avoid issues later in the flow.

When using Cadence Genus, set the attribute *hdl\_flatten\_complex\_port* to “true” for any designs that include complex ports declared using unions to facilitate post-synthesis port name matching.

When using any of the Synopsys Design Compiler family of synthesis tools, declare the ranges of any ports declared as SystemVerilog interface arrays be declared as follows to facilitate post-synthesis port-name matching:

```
<interface_type> <port_name> [0:<positive right index>]
```

When using DC Ultra, there are two key commands to be aware of to synthesize correctly with the Tessent IP.

- Turn off constant propagation with no boundary optimization by setting an application variable. This variable works globally and is used with the following syntax:

```
set_app_var compile_enable_constant_propagation_with_no_boundary_opt false
```

- For instance level control, use the following compile directive before using the `compile_ultra` command to turn off the constant propagation for cells:

```
set_compile_directives -constant_propagation false [get_cells <hier-cell-name>]
```

Or for pin-level granularity, use the following command:

```
set_compile_directives -constant_propagation false [get_pins <hier-pin-name>]
```

Other means of restricting the synthesis tool from changing and optimizing instances and levels of hierarchy include the `set_size_only` command and ungrouping. When `set_size_only` is used for a specified list of leaf cells, the tool can only change their drive strength, or sizing. The `ungroup`, `group`, `set_ungroup`, and `uniquify` commands can be used to control how the tool removes (or not) levels of hierarchy and how reused blocks are uniquified during synthesis. The `dont_touch` attribute is also effective for adding to designs, sub designs, and cells to prevent them from being ungrouped during optimization.

---

#### Note



During the synthesis of MemoryBIST logic, there are two types of warnings that may be issued by synthesis tools that are related to the optimization of registers. These warnings may be ignored as synthesis tools are very reliable. Additionally, formal verification can be used to confirm the functionality is not affected. The warning types are:

- Registers with no fanout are removed, along with the combinational logic driving the inputs.
  - Registers with inputs and outputs that are always identical are merged.
- 

Different guidelines apply to the type of flow you are using, whether a bottom-up synthesis flow, or a top-down approach.

### Bottom-Up Flow Using DC Ultra

1. Read full design in synthesis tool.
2. Set current design to each DFT IP and compile.
3. Set don't touch or set size only to all the DFT IP.
4. Set current design TOP.
5. Read functional SDC.
6. Read DFT SDC.
7. Set ungroup false to all DFT IP and persistent modules.
8. Set boundary optimization false to all DFT IP and persistent modules.



9. Set size only to all persistent cells.
10. Disable constant propagation during boundary optimization.
11. Compile ultra.

### Top-Down Flow Using DC Ultra

1. Read full design in synthesis tool.
2. Read functional SDC.
3. Read DFT SDC.
4. Set ungroup false to all DFT IP and persistent modules.
5. Set boundary optimization false to all DFT IP and persistent modules.
6. Set size only to all persistent cells.
7. Disable constant propagation during boundary optimization.
8. Compile ultra.

## SystemVerilog and Port Name Matching

Any ports declared as interface arrays in a SystemVerilog RTL design should be declared as follows:

```
<interface_type><port_name> [0:N]
```

“N” is a positive integer that represents the size of the array minus 1. You can use this method to properly map the port names in a netlist generated by any of the Design Compiler family of tools.

## Synthesis Guidelines for Parameterization Wrapper Flows

---

We recommend using or adapting the script and Tcl procedures in this section for the parameterization wrapper flows. Tessent Shell supplies the Tcl procedures for interoperability with third-party synthesis tools to help you manage the multiple files required for parameterized designs.

---

### Note

 To use the Tcl procedures, you must run the [process\\_parameterized\\_design\\_specification](#) command before the RTL DFT insertion steps for the blocks in your design. After the RTL DFT insertion steps for a block, run the “[write\\_design\\_import\\_script -include\\_child\\_physical\\_block\\_interface\\_modules](#)” command. This command generates the prerequisite files for the Tcl procedures that help you manage synthesis. Refer to “[How to Handle Parameterized Blocks During DFT Insertion](#)” on page 686 for more information about creating and using parameterization wrappers.

---

The set of Tcl procedures in “[Tcl Procedures for Synthesis in the Parameterization Wrapper Flow](#)” on page 1047 and the example script in this section are designed for interoperability with the Synopsys Design Compiler® tool (dc\_shell). If you are using another synthesis tool, we recommend adapting these procedures for use with that tool.

You can use these procedures for synthesis of all parameterized blocks, including the following situations:

- The block is parameterized and is instantiated with parameter overrides. The block *does not* instantiate parameterized child blocks with parameter overrides.
- The block is parameterized and is instantiated with parameter overrides. The block *does* instantiate parameterized child blocks with parameter overrides.
- The block is not parameterized or is not instantiated with parameter overrides; however, the block instantiates one or more parameterized child blocks with parameter overrides.

Use or adapt the following example script. Invoke the script from the same directory where the [write\\_design\\_import\\_script](#) command wrote the loading scripts for the RTL design files for the block.

```
# Parameters for the rest of the synthesis script.
set design_name <design_name>
set design_id <design_id>
set tsdb <tsdb_path>
set preserve_type icl_extract
# Clock(s) to create on the design
set clock_dictionary { <port_name> { <period> <clock_name> } }
# Directives to the synthesis tool. Add others for your flow.
set_app_var hdlin_enable_upf_compatible_naming true
set_app_var timing_enable_multiple_clocks_per_reg true
set_app_var compile_enable_constant_propagation_with_no_boundary_opt \
    false
set_app_var compile_seqmap_propagate_high_effort false
set_app_var compile_delete_unloaded_sequential_cells false
# The following directives may be useful - your decision.
set_app_var sh_continue_on_error false
set_app_var sh_script_stop_severity E
source <path_to_synthesis_flow_procs>/flow_procs.tcl
# Invoke top-level proc
synthesize_block $design_name $design_id $tsdb \
    -preserve_type $preserve_type \
    -clock_dictionary_name clock_dictionary
```

**Tcl Procedures for Synthesis in the Parameterization Wrapper Flow..... 1047**

## Tcl Procedures for Synthesis in the Parameterization Wrapper Flow

Tessent supplies Tcl procedures (procs) for synthesis to aid interoperability with third-party synthesis tools for blocks in parameterized block hierarchies.

The `synthesize_block` and other procedures in the following Tcl files target `dc_shell`, which is the Synopsys Design Compiler® tool. If you are using another synthesis tool, we recommend adapting the procedures they contain:

- ***flows\_procs.tcl*** — contains the `synthesize_block` and `synthesize_view` procedures.
- ***support\_procs.tcl*** — contains support procedures used in `flow_procs.tcl`.

---

### Note



The `synthesize_block` procedure is the top-level proc to call from your synthesis script. The `flow_procs.tcl` file has usage instructions at the top.

---

Find these Tcl files in your installed release tree under the share directory:

- `<installed release tree>/share/ParamWrappers/SynthScripts/flows_procs.tcl`
- `<installed release tree>/share/ParamWrappers/SynthScripts/support_procs.tcl`

Refer to “[How to Handle Parameterized Blocks During DFT Insertion](#)” on page 686 for more information about different design scenarios and their impact on the DFT insertion process when using parameterization wrappers.

The Tcl procedures source and parse one or two files generated by the [process\\_parameterized\\_design\\_specification](#) command for parameterized blocks or blocks that instantiate parameterized blocks.

- The `<design_name>.parameterization_wrapper_info_dictionary` file is generated for all parameterized blocks. The following is a template for this file:

```
set parameterization_wrapper_info_dictionary {
  <design_name> {
    parameterized_view_<i> {
      // <i> is an integer starting at 1. Omitted if only one view.
      post_synthesis_uniquified_block_name <post_synthesis_name>
      parameterization_wrapper_file_name \
      <post_synthesis_name>.<lang>_parameterization_wrapper
      block_instance_list_in_parent <list of instances which \
      instantiate above view>
    }
    ... // Additional views
  }
}
```

- The `<design_name>.consolidated_child_blocks_info_dictionary` file is generated for blocks that instantiate parameterized child blocks. The following is a template for that file:

```
set consolidated_child_blocks_info_dictionary {
  <child_block_design_name> {
    parameterized_view_<i> {
      // <i> is an integer starting at 1. Omitted if only one view.
      post_synthesis_uniquified_block_name <post_synthesis_name>
      parameterization_wrapper_file_name \
      <post_synthesis_name>.<lang>_parameterization_wrapper
      block_instance_list_in_parent <list of instances which \
      instantiate above view>
    }
    ... // Additional views
  }
  ... // Additional child blocks
}
```

# Appendix C

## Clocking Architecture Examples

The clocking architecture in designs may vary, and they play a role during test planning, especially for hierarchical test.

This appendix provides examples of common clocking architectures and how to insert the on-chip clock controllers (OCCs) given those architectures. The examples assume that you are performing hierarchical test as described in “[DFT Architecture Guidelines for Hierarchical Designs](#)” on page 105”.

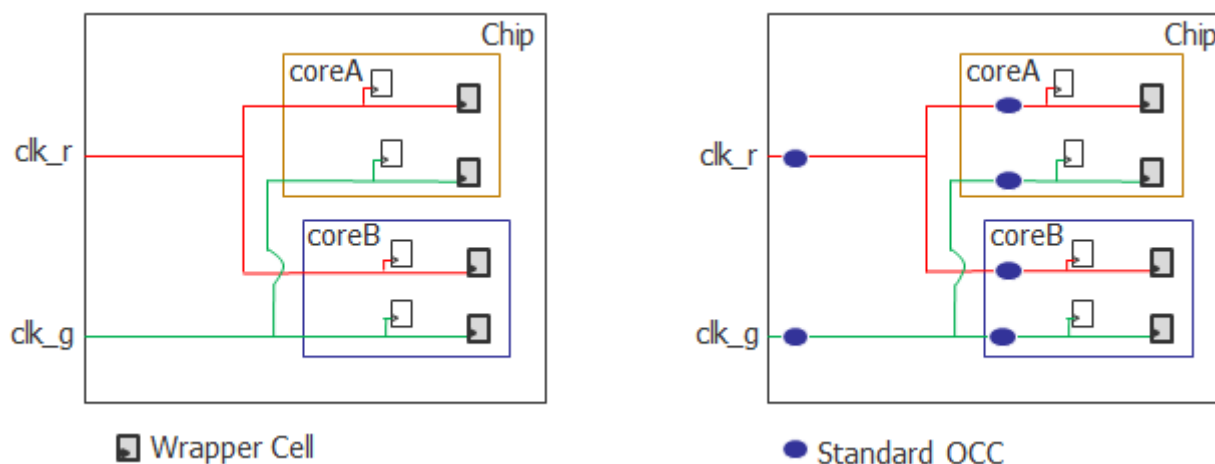
For information about the OCCs supported by Tessent, refer to “[Tessent On-Chip Clock Controller](#)” in the *Tessent Scan and ATPG User’s Manual*.

|                                                          |             |
|----------------------------------------------------------|-------------|
| <b>Clocks Driven by Primary Inputs</b> .....             | <b>1049</b> |
| <b>Clock Generators Outside the Cores</b> .....          | <b>1050</b> |
| <b>Clock Generators Inside the Cores</b> .....           | <b>1050</b> |
| <b>Clock Sourced From a Core With Embedded PLL</b> ..... | <b>1051</b> |
| <b>Clock Mesh Synthesis</b> .....                        | <b>1052</b> |

## Clocks Driven by Primary Inputs

In the following figure, coreA and coreB are wrapped cores, and the clk\_r and clk\_g clocks are asynchronous to each other. Insert standard OCCs inside coreA and coreB for both clk\_r and clk\_g. At the chip-level, insert standard OCCs on the clk\_r and clk\_g ports.

**Figure C-1. OCCs for Clocks Driven by Primary Inputs**

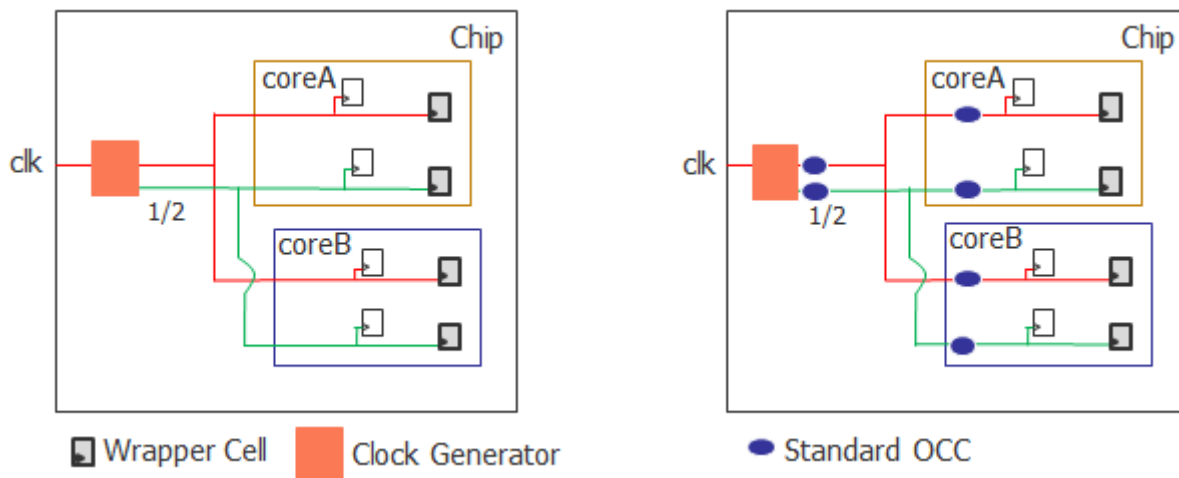


During intest mode for the wrapped cores, the OCCs inside the cores are active and the OCCs outside the cores are inactive. During extest, the OCCs outside the cores are active and the OCCs inside the cores are inactive.

## Clock Generators Outside the Cores

In the following figure, coreA and coreB are wrapped cores, and a clock generator at the chip level divides the clock frequency by half to generate the green clock. The red and green clocks are asynchronous to each other. Insert standard OCCs inside coreA and coreB for both the red and green clocks. At the chip-level, insert standard OCCs on both the red and green clocks generated at the output of the clock generator.

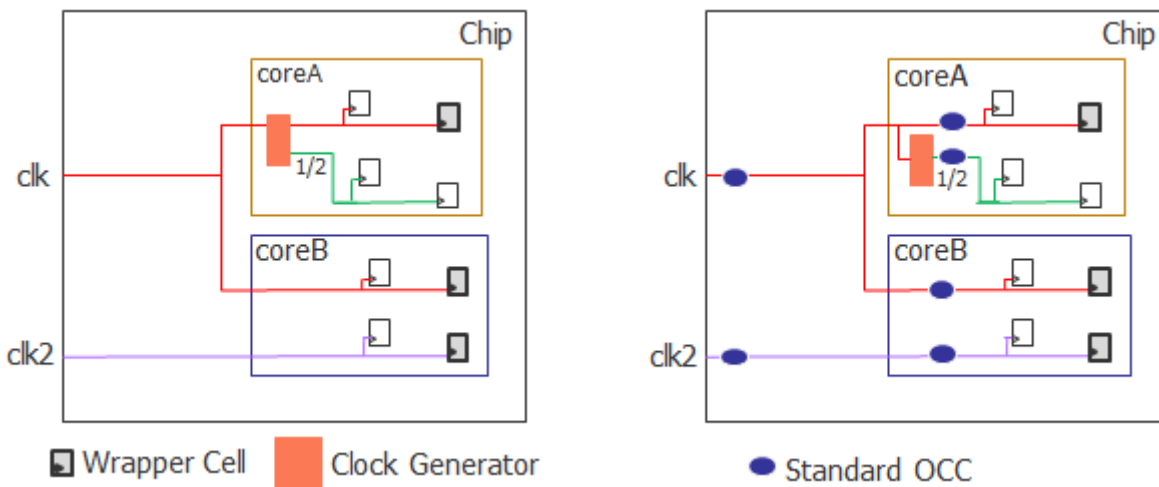
**Figure C-2. OCCs With Clock Generators at Chip Level, Asynchronous Clocks**



## Clock Generators Inside the Cores

In the following example, coreA and coreB are wrapped cores and a clock generator is located inside coreA. The red and green clocks are asynchronous to each other inside coreA, so you would insert standard OCCs inside coreA. In addition, the red and purple clocks are asynchronous to each other, so you insert standard OCCs for these clocks inside coreB as well as at the chip-level.

**Figure C-3. OCCs With Clock Generators Inside the Cores, Asynchronous Clocks**

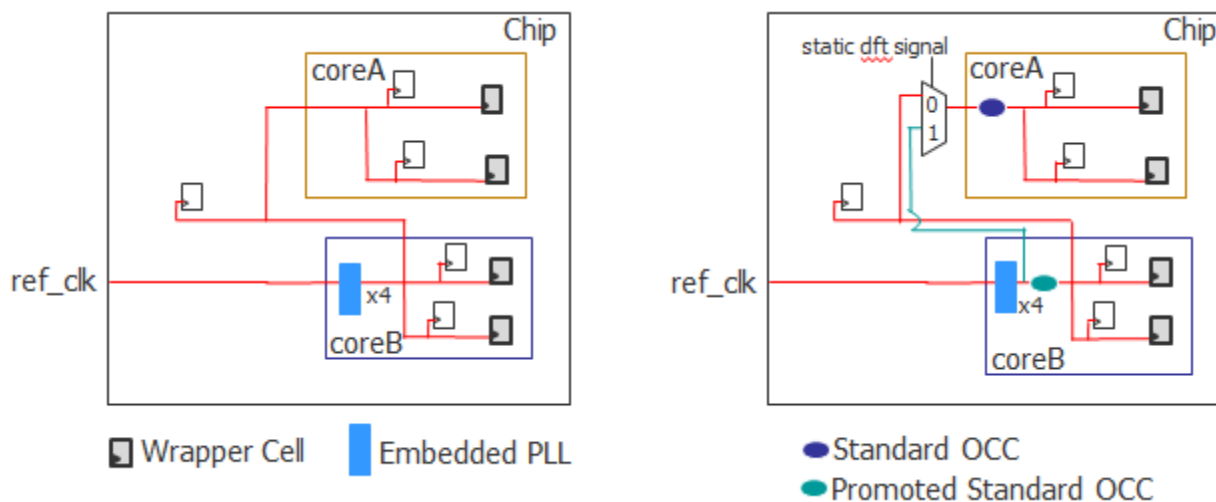


During intest mode for the wrapped cores, the OCCs inside the cores are active, and the OCCs outside the cores are inactive. During extest, the OCCs outside the cores are active, and the OCCs inside the cores are inactive.

## Clock Sourced From a Core With Embedded PLL

In the following figure, coreA and coreB are wrapped cores, and coreB contains an embedded PLL module. coreB sources the clock that feeds both coreA and the chip. In this case, you would retarget coreA and coreB at the same time.

**Figure C-4. Clock Sourced From a Core With Embedded PLL, With MUX**



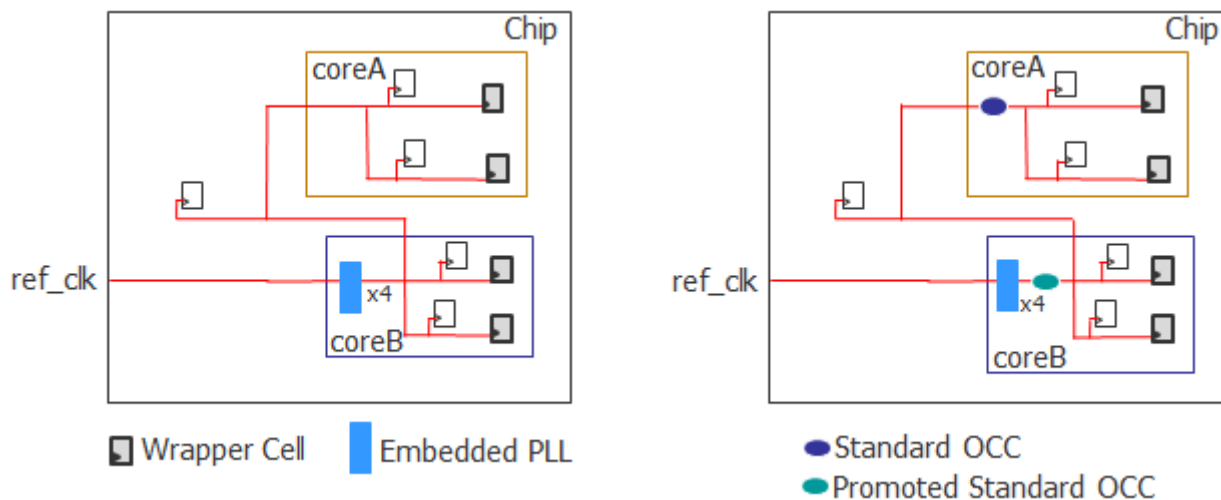
When you are retargeting coreA and coreB at the same time, you must add a mux. Use a clock net before reaching the OCC inside coreB as a clock to feed the OCC inside coreA to avoid having back-to-back OCCs. To retarget both coreA and coreB at the same time, the OCCs inside coreA and coreB must be active at the same time.

Insert standard OCCs inside the cores. The standard OCC inside the core with the embedded PLL (coreB) gets promoted so that it is also included in the external chains. When in external mode, the OCC inside coreB can be reused. For details, refer to “[How to Handle Clocks Sourced by Embedded PLLs During Logic Test](#)” on page 713.

During intest, the standard OCCs inside coreA and coreB are active. The mux is set to 1 to avoid cascading OCCs. During extest, only the promoted standard OCC within coreB is active, and the mux is held at 0.

In the following example, you are retargeting coreA and coreB in separate runs, so there is no need to insert a mux. During intest of coreA, only coreA’s OCC is active, and then during intest of coreB, only coreB’s OCC is active. During extest of the cores, only the promoted standard OCC within coreB is active.

**Figure C-5. Clock Sourced From a Core With Embedded PLL, Without MUX**

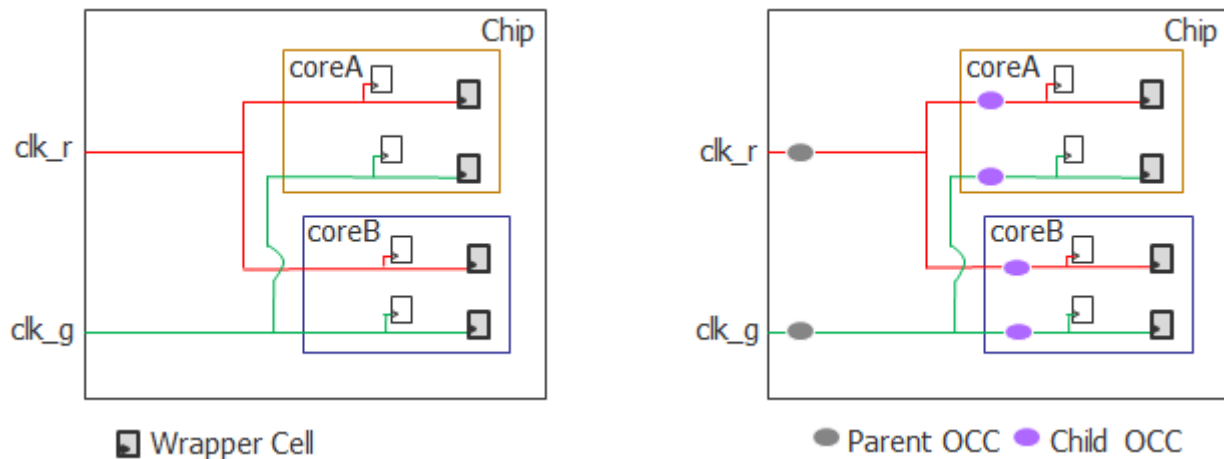


## Clock Mesh Synthesis

The most common method for clock balancing is clock tree synthesis. However, you can also use clock mesh synthesis to balance the clocks. For clock mesh, rather than standard OCCs, use parent OCCs at the chip level and child OCCs inside the wrapped cores. In the following figure, coreA and coreB are wrapped cores, and the clk\_r and clk\_g clocks are asynchronous to each other.



**Figure C-6. Clock Mesh Synthesis**



Core-level child OCCs have additional timing requirements because a single clock on the boundary of the wrapped core is used for shift as well as for slow and fast capture. For details, refer to “[Recommendations](#)” in the *Tessent Scan and ATPG User’s Manual*. Using SDC for timing closure is described in “[Timing Constraints \(SDC\)](#)” on page 969.

During intest of coreA and coreB, the child OCCs are active, and the parent OCCs are in parent mode. During extest of coreA and coreB, the parent OCCs are in standard mode and the child OCCs are inactive.



# Appendix D

## Formal Verification

---

Tessent™ Shell-based products do not generate scripts for use with formal verification tools such as Synopsys® Formality® or Cadence® Conformal®. You can, however, set constraints in your design that are used with these tools.

For EDT, constraining scan\_en to 0 is sufficient to proceed with equivalence checking. The formal verification tool reports some unmatched nodes (such as the newly added EDT pins, clock/update/bypass/low\_power/channel) and registers inside the EDT logic.

For OCC, if it has an IjtagInterface, or if the test\_mode pin of the OCC is connected to an IjtagNetwork, then keeping the IJTAG network in the reset state is sufficient. If the test\_mode pin is connected to a top-level port, this port needs to be constrained to 0.

The following sections provide guidance on how to set up formal verification. Use the correct syntax for your chosen tool.

|                                                                |             |
|----------------------------------------------------------------|-------------|
| <b>Golden RTL Versus DFT-Inserted RTL .....</b>                | <b>1055</b> |
| <b>Pre-Layout Versus Post-Layout .....</b>                     | <b>1056</b> |
| <b>Embedded Boundary Scan at the Physical Block Level.....</b> | <b>1057</b> |

## Golden RTL Versus DFT-Inserted RTL

Use the following guidance to create a script for use with your formal verification of golden RTL versus DFT-inserted RTL.

Set the following:

- Set the circuit to functional mode to hold the IJTAG network or TAP in reset.
- If you are at the chip level, set a constant 0 on TRST. If you are at the sub\_block or physical\_block level, set a constant on the ijtag\_reset ports to their active value. This is typically 0.
- If your design has DftSignals coming from ports in both the chip and design levels, these ports must be held at their reset values. The reset automatically handles DftSignals from the TDR.
- If you are at the sub\_block or physical\_block level, suppress the verification of ports created by Tessent Shell to prevent reporting violations.

- Assert the `ijtag_reset` pins to their active values on all instruments and SIBs inserted by Tessent tools.
- Because the ICL network may already be present, the SIBs could result in mismatches on the ICL network scan path. Set the SIB scan in/outs as “don’t verify” points.
- Use the “Consistency” mode for the verification passing mode rather than the “Equality” mode by setting the following:

```
set verification_passing_mode Consistency
```

The default verification passing mode of Formality is the “Consistency” mode.

- If you want to use the “Equality” mode for verification, add the following setting to deal with clock gating logic inserted by Tessent:

```
set_app_var verification_clock_gate_hold_mode any
```

## Pre-Layout Versus Post-Layout

Use the following guidance to create a script for use with your formal verification of pre-layout versus post-layout.

Ensure that the layout step did not affect DFT functionality. Ignore violations caused by scan chain reordering. Also, constrain the `scan_en` signal low, but do not constrain anything else on the inserted DFT.

Tessent Shell may insert lockup elements at the end of the scan chains during scan insertion. These are marked as stop points in the scanDEF file and are not reordered. After scan chain reordering, you may see mismatches on these lockup latches during layout.

For example:

```
GPIO_2/\p2out_reg[1]  ( IN SI ) ( OUT Q )  
+ STOP GPIO_2/ts_1_lockup_latchn_clkc6_intno266_i D\
```

When this occurs, and you perform formal verification on the pre- versus post-layout, the tool returns warnings that these lockup latches are no longer connected to the same functional flop as previously.

This is not an issue with the other scan flops because you can disable the scan path by setting the scan enable to 0. However, the lockup latch is a flop/latch without scan-in.

If you encounter this situation, use the following workaround: Specify `set_dont_verify_point` for the lockup latches in both the reference and implemented designs to see if formal verification passes. To find the lockup latches, introspect the design in Tessent Shell using the following:

```
get_instances ts_1_lockup*
```

# Embedded Boundary Scan at the Physical Block Level

Use the following guidance to create a script for use with your formal verification checking tool.

Most signals are blocked by the `bscan_select` signal, but you should set the following:

- `add_pin_constraints 0 bscan_select -golden`
- `add_pin_constraints 0 bscan_select -revised`
- `add_ignored_outputs bscan_scan_out -golden`
- `add_ignored_outputs bscan_scan_out -revised`
- `add_pin_constraints 0 bscan_force_disable -both`
- `add_pin_constraints 0 bscan_select_jtag_output -both`
- `add_pin_constraints 0 bscan_clock -both`
- `add_pin_constraints 0 bscan_select_jtag_input -both`



# Appendix E

## Solutions for Genus Synthesis Issues

---

You may encounter some issues when using Cadence Genus to synthesize mixed Verilog/VHDL designs. This section describes some common issues and proposed solutions.

- **Problem: There is a mismatch between the port list of the VHDL component definition for a memory and the port list of an auto-created blackbox for the memory.** — Solution:
  - Use the Synopsys `translate_off/translate_on` pragmas to effectively comment out everything in the memory module definition after the port list. This avoids performance issues during synthesis.
  - In the Genus synthesis script for synthesizing a design that instantiates one or more memories, do the following:
    - Load the file containing the memory module definition with the internals surrounded by pragmas, as mentioned in the previous solution. This avoids creating a blackbox with a mismatch between the port list in the VHDL component definition of the memory and the port list of the blackbox.
    - Immediately after elaborating the design, add the following lines to prevent the memory definition from being written to the netlist:

```
set mem_mod [vfind . -module "<name of the memory module>"]
set_db $mem_mod .write_vlog_skip_subdesign true
```

- **Problem: A blackbox for a memory is written to the netlist.** — In the Genus synthesis script for synthesizing a design that instantiates one or more memories, do the following:
  - Immediately after elaborating the design, add the following lines to prevent the memory definition from being written to the netlist:

```
set mem_mod [vfind . -module "<name of the memory module>"]
set_db $mem_mod .write_vlog_skip_subdesign true
```

- **Problem: Uniquification of logic abstracts for a memory.** — These are simply unresolved references. Only one such logic abstract should be created. Set the following root attribute before loading/elaborating the design to prevent uniquification of logic abstracts:

```
set_db / .write_vlog_empty_module_for_logic_abstract false
```





There are several ways to get help when setting up and using Tessent software tools. Depending on your need, help is available from documentation, online command help, and Siemens EDA Support.

|                                                  |             |
|--------------------------------------------------|-------------|
| <b>The Tessent Documentation System .....</b>    | <b>1061</b> |
| <b>Global Customer Support and Success .....</b> | <b>1061</b> |

## The Tessent Documentation System

From the Tessent 2024.1 release, Siemens provides an improved method to access your Siemens Digital Industries Software product documentation. The default option serves your product documentation from Support Center, giving you immediate access to the latest release-specific documentation, and an enhanced search of HTML and PDF files. This new method also eliminates the requirement to install product documentation as part of the local software installation.

For easy access, you can now view Support Center documentation using a documentation proxy on your network. This documentation proxy removes the need for your users to have a Support Center account or to log into Support Center to view documentation.

We understand that some customers use our products on restricted networks without internet access. You have the option to download the documentation and set up a Siemens Documentation Server to view the documentation package on your local network.

See “[Configuring Documentation Access](#)” in the *Managing Tessent Software* manual for instructions to set up the documentation proxy or the documentation server.

## Global Customer Support and Success

A support contract with Siemens Digital Industries Software is a valuable investment in your organization’s success. With a support contract, you have 24/7 access to the comprehensive and personalized Support Center portal.

Support Center features an extensive knowledge base to quickly troubleshoot issues by product and version. You can also download the latest releases, access the most up-to-date documentation, and submit a support case through a streamlined process.

<https://support.sw.siemens.com>

If your site is under a current support contract, but you do not have a Support Center login, register here:

<https://support.sw.siemens.com/register>

## — Symbols —

#include statement, 765

## — A —

Alias statements, 769

Always block, 782

Always block example, 783

Always block statements

    always\_statement, 783

    force, 783

    pulse, 783

Always block syntax, 783

## — C —

Clock\_run procedure, 823

Cycle statements

    force\_pi, 791

Cycle statements

    bidi\_force\_off, 792

    bidi\_force\_pi, 791

    bidi\_measure\_po, 792

    condition, 793

    expect, 793

    force, 793

    force\_sci, 791

    force\_sci\_equiv, 791

    initialize, 793

    measure, 793

    measure\_po, 791

    measure\_sco, 792

    observe\_method, 794

    pulse, 794

    pulse\_capture\_clock, 792

    pulse\_read\_clock, 793

    pulse\_write\_clock, 793

    restore\_bidi, 792

    restore\_pi, 792

## — E —

Environment Variables

    TESSSENT\_LICENSE\_ORDER, 38

Environment variables

    application-specific, 38

    MGCDFT\_STARTUP, 38

external capture procedure, 824

## — I —

Include statement, 765

## — N —

Named capture procedure, 824

## — P —

Procedure statements

    apply, 789

    cycle, 783

    label, 788

    scan\_group, 786

    timeplate, 786

## — S —

Set statement, 766

Set statements

    default\_timeplate, 768

    strobe\_window time, 767

    time scale, 766

Shift procedure, 799

    alternate, 801

Sub\_procedure, 833

## — T —

Tessent Visualizer tutorials, 963

Test procedure file

    defined, 760

    DRC checking, 760

    statements, 766

    timing variables, 771

Test procedures

    capture, 824

    clock\_sequential, 830

    init\_force, 831

---

- load\_unload, [803](#)
- master\_observe, [808](#)
- shadow\_control, [806](#)
- shadow\_observe, [809](#)
- shift, [799](#)
- skew\_load, [809](#)
- sub\_procedure, [833](#)
- test\_end, [832](#)
- test\_setup, [796](#)
- Test\_end procedure, [832](#)
- Time scale
  - setting, [766](#)
- Timeplate statements
  - bidi\_force\_pi, [775](#)
  - bidi\_measure\_po, [776](#)
  - force, [776](#)
  - force\_pi, [775](#)
  - measure, [776](#)
  - measure\_po, [776](#)
  - offstate, [775](#)
  - period, [778](#)
  - pulse, [776](#)
  - pulse\_clock, [777](#)
- tscale, [766](#)
- Tutorials for Tessent Visualizer, [963](#)

## Third-Party Information

Details on open source and third-party software that may be included with this product are available in the `<your_software_installation_location>/legal` directory.

